```
student@ubuntu-lab:~$ cat README
```

Linux+ Hands-On Lab Book

Step-by-step labs for every CompTIA Linux+ exam objective, explained in plain English.

EXAM XK0-006 · OBJECTIVES V5.0

ubuntu-lab	**rocky-lab**
Ubuntu Server 24.04 LTS	Rocky Linux 9
main lab system	RHEL-family topics

```
$ ls ~/labs
Part 1  System Management                          23%   7 labs
Part 2  Services and User Management               20%   6 labs
Part 3  Security                                   18%   6 labs
Part 4  Automation, Orchestration, and Scripting   17%   5 labs
Part 5  Troubleshooting                            22%   5 labs
$ ./start --lab 0
```

Unofficial study guide. CompTIA and Linux+ are trademarks of CompTIA, Inc.

Contents

How to Use This Book 4

Lab 0 Build Your Lab 7

Part 1: System Management 19
- Lab 1.1 Explore Basic Linux Concepts 20
- Lab 1.2 Manage Devices, Kernel Modules, and the initrd 42
- Lab 1.3 Manage Storage 60
- Lab 1.4 Manage Network Services and Configurations 91
- Lab 1.5 Use Common Shell Operations 115
- Lab 1.6 Back Up and Restore a Linux Server 143
- Lab 1.7 Run Virtual Machines with QEMU, KVM, and libvirt 167

Part 2: Services and User Management 192
- Lab 2.1 Manage Files and Directories 193
- Lab 2.2 Manage Local Accounts 215
- Lab 2.3 Manage Processes and Jobs 240
- Lab 2.4 Configure and Manage Software 268
- Lab 2.5 Manage Linux Using systemd 297
- Lab 2.6 Manage Applications in Containers 319

Part 3: Security 342
- Lab 3.1 Authentication, Authorization, and Accounting 343
- Lab 3.2 Configure and Implement Firewalls 367
- Lab 3.3 Operating System Hardening 389
- Lab 3.4 Harden User Accounts 413
- Lab 3.5 Use Cryptography on Linux 435
- Lab 3.6 Compliance, Auditing, and Detection 459

Part 4: Automation, Orchestration, and Scripting 486
- Lab 4.1 Automation and Orchestration 487
- Lab 4.2 Automate Tasks with Shell Scripts 525
- Lab 4.3 Python Basics for System Administration 551
- Lab 4.4 Version Control with Git 573
- Lab 4.5 Use AI Responsibly as a Linux Admin 594

Part 5: Troubleshooting 613
- Lab 5.1 Monitoring Concepts and Configurations 614
- Lab 5.2 Troubleshoot Hardware, Storage, and OS Issues 632
- Lab 5.3 Troubleshoot Networking Issues 678
- Lab 5.4 Troubleshoot Security Issues 710
- Lab 5.5 Troubleshoot Performance Issues 754

Appendices 802
- Appendix A: Answer Key 803
- Appendix B: Exam Corner Review Sheet 813
- Appendix C: Exam Objective Coverage Map 830
- Appendix D: Lab VM Troubleshooting 847

Appendix E: Acronym List 851

How to Use This Book

This book teaches the CompTIA Linux+ XK0-006 exam objectives by having you **do** them. Every exam objective has its own lab. In each lab you type real commands on real Linux systems, watch what happens, and read an explanation of why it happened. By the end you will have done, at least once, nearly everything the exam asks about.

Who this book is for

You do not need any Linux experience. You need:

- A computer that can run two virtual machines (VMs) at the same time. In practice that means 8 GB of RAM (16 GB is more comfortable) and about 80 GB of free disk space.
- Virtualization software. Any of these work: VirtualBox (free, all platforms), VMware Workstation Pro or Fusion (free for personal use), Hyper-V (built into Windows Pro), or UTM (macOS on Apple silicon).
- Patience to type commands carefully and read what comes back.

How the book is organized

- **Lab 0** builds your lab: an Ubuntu Server 24.04 LTS VM named `ubuntu-lab` and a Rocky Linux 9 VM named `rocky-lab`. Do this first.
- **Parts 1 to 5** match the five exam domains. Each part has one lab per exam objective, numbered the same way as the objective. Lab 2.4, for example, covers objective 2.4.
- **The appendices** contain the answers to every review question, a coverage map that shows where each exam topic is practiced, and the official acronym list.

You can do the labs in order, which is the easiest path, or jump to the objective you want to study. Every lab cleans up after itself, so labs do not depend on each other.

What is inside every lab

Section	What it gives you
Lab header	The official objective, exam weight, time needed, which VM to use, and whether to take a snapshot first.
What you'll practice	The exam topics this lab covers, in the exam's own words.
How it works	A plain-English explanation of the concepts before you touch the keyboard.
Tasks	Numbered, step-by-step instructions. Each command is followed by sample output and a **What just happened** box.
Checkpoints	Quick checks that prove each task worked before you move on.
Challenge	Optional extra tasks with no step-by-step help, to test yourself.
Clean up	Steps that undo the lab's changes so the next lab starts fresh.
Exam corner	The facts to memorize and review questions. Answers are in Appendix A.

Conventions

Commands you type are shown in code blocks like this. Type them exactly, or copy and paste them:

```
lsblk
```

Commands run on **ubuntu-lab** unless the first line of the block says otherwise:

```
# on rocky-lab
sudo dnf install -y httpd
```

Output that the computer prints back is shown in a shaded block. Your output will differ in small details such as dates, sizes, process IDs, and IP addresses. That is normal.

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   25G  0 disk
```

Inside commands, words in CAPITALS such as `ROCKY_IP` are placeholders. Replace them with your own values.

The boxes in the text mean:

> **What just happened:** explains the command you just ran and how to read its output.

> **TIP**
> A shortcut, a good habit, or a difference between Ubuntu and Rocky Linux.

> **CAUTION**
> Something that can break your VM or lose data. Read it before you continue.

> **Checkpoint:** a quick test that proves the task worked.

> **ON THE EXAM**
> A fact that is likely to appear on the exam.

Safety rules

1. **Never do these labs on a computer you care about.** Use only the lab VMs.
2. **Take snapshots.** A snapshot saves the whole state of a VM so you can roll back in seconds. Take one called `clean` at the end of Lab 0, and take another whenever a lab header says "Snapshot first: Yes". If something goes wrong, restore the snapshot and start the lab again.
3. **Keep a second session open** when you change SSH, firewall, sudo, or login settings. If your change locks you out, the open session lets you undo it. The VM console in your virtualization software always works as a last resort.
4. **Read before you press Enter**, especially for commands that start with `sudo`, `rm`, `dd`, `mkfs`, `wipefs`, or `fdisk`.

A note on the exam

Linux+ XK0-006 has up to 90 questions, both multiple-choice and performance-based, and you get 90 minutes. Performance-based questions put you in front of a simulated terminal or configuration screen and ask you to complete a task. The hands-on practice in this book is designed for exactly that kind of question.

This is an unofficial study guide. CompTIA and Linux+ are trademarks of CompTIA, Inc. The objective titles are quoted from the official CompTIA Linux+ XK0-006 Exam Objectives, version 5.0.

Lab 0 Build Your Lab

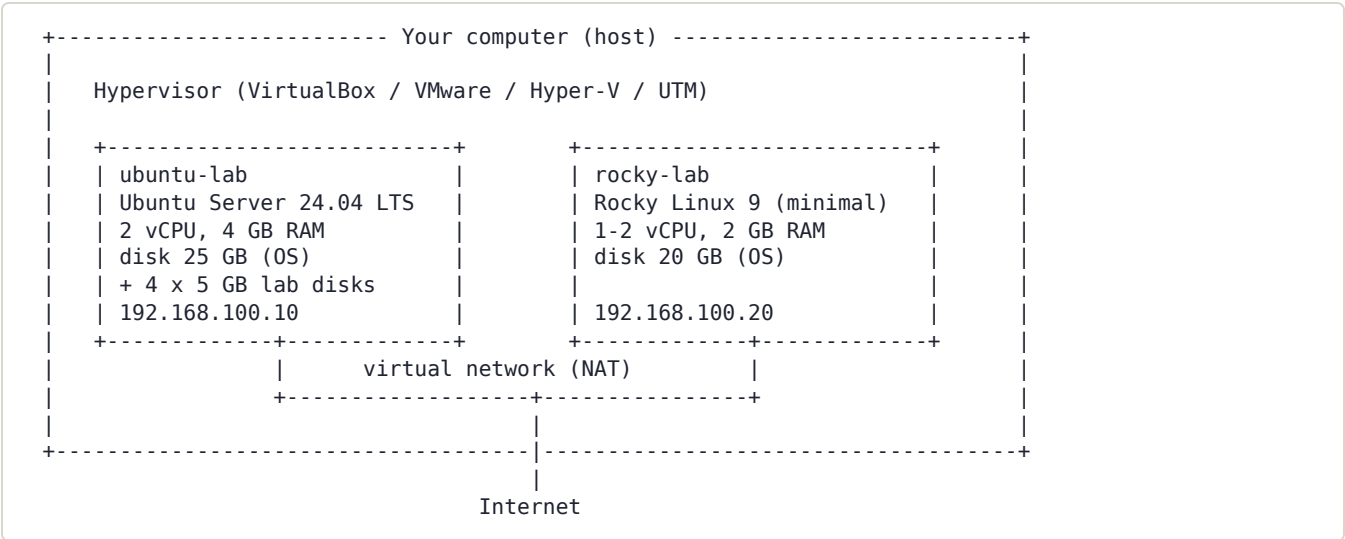
Goal: Build the two virtual machines used in every lab.
Time: about 90 minutes (most of it is waiting for installers)
You will end with: `ubuntu-lab` (Ubuntu Server 24.04 LTS) and `rocky-lab` (Rocky Linux 9) on the same network, each with a snapshot named `clean`

The labs in this book need two Linux servers. Most labs use Ubuntu Server, the most popular Linux server distribution. A few labs need a Red Hat-style system, because the exam covers tools that only exist there: SELinux, firewalld, dnf, rpm, dracut, and Kickstart. For those labs you will use Rocky Linux, a free rebuild of Red Hat Enterprise Linux (RHEL). Anything you learn on Rocky works the same way on RHEL.

How it works

A **virtual machine (VM)** is a computer simulated in software. Your real computer, the **host**, runs a program called a **hypervisor** (VirtualBox, VMware, Hyper-V, or UTM). The hypervisor carves out some of the host's CPU, memory, and disk and presents them to the VM as if they were real hardware. The operating system inside the VM, the **guest**, does not know the difference.

VMs are perfect for practice because you can break them without consequences. A **snapshot** freezes the complete state of a VM's disks (and optionally its memory). If a lab goes wrong, you restore the snapshot and you are back where you started.



Both VMs sit on the same virtual network. They can reach each other and the internet, and you can reach both of them from your host with SSH.

The four extra 5 GB disks on `ubuntu-lab` are for the storage labs (partitions, LVM, RAID, encryption, and disk troubleshooting). They start empty, and every lab that uses them wipes them clean again at the end.

What you need

Item	Where to get it
Hypervisor	VirtualBox (virtualbox.org), VMware Workstation Pro or Fusion (free for personal use), Hyper-V (Windows Pro and up), or UTM (Apple silicon Macs)
Ubuntu Server 24.04 LTS ISO	ubuntu.com/download/server. Choose the latest 24.04.x point release.
Rocky Linux 9 minimal ISO	rockylinux.org/download. Choose the latest 9.x **Minimal** ISO for your CPU type.
Free resources on the host	About 6 GB of RAM and 80 GB of disk for both VMs

> TIP
> **Apple silicon Macs (M1 and later):** download the **ARM64** (aarch64) ISOs of both distributions. Everything in the book works the same way. Where a lab downloads an `x86_64` file, such as a cloud image in Lab 1.7, choose the `aarch64` version instead.

Tasks

Task 1: Prepare a virtual network

Both VMs need to be on a network where they can see each other and reach the internet. The simplest choice depends on your hypervisor.

1. Set up the network for your hypervisor:
   - **VirtualBox:** open **File > Tools > Network Manager**, select the **NAT Networks** tab, and click **Create**. Name it `LabNet`, set the IPv4 prefix to `192.168.100.0/24`, and keep **Enable DHCP** checked. You will attach both VMs to this NAT network. (The plain "NAT" mode does **not** let VMs talk to each other.)
   - **VMware Workstation or Fusion:** use the default **NAT** network (VMnet8, or "Share with my Mac"). Its subnet is chosen at random when VMware is installed, for example `192.168.47.0/24`. You will find yours in Task 4.
   - **Hyper-V:** use the **Default Switch**. It provides NAT and DHCP. Its subnet can change when the host reboots, so in Task 4 you will keep DHCP and update `/etc/hosts` if the addresses change.
   - **UTM:** use the default **Shared Network** mode for both VMs.

> **What just happened:** You created (or chose) a private virtual network. **NAT** (Network Address Translation) lets the VMs reach the internet through your host's connection, while hiding them from the outside world. **DHCP** hands each VM an IP address automatically when it boots.

Task 2: Create and install ubuntu-lab

1. Create a new VM with these settings:

Setting	Value
Name	`ubuntu-lab`
Type	Linux, Ubuntu (64-bit), or ARM64 on Apple silicon
CPUs	2
Memory	4096 MB
Disk	25 GB, dynamically allocated (thin provisioned)
Network	The network from Task 1
Firmware	UEFI if offered (BIOS also works)
Install media	The Ubuntu Server 24.04 ISO

> **TIP**
> **VirtualBox:** turn **off** "Unattended Installation" (check "Skip Unattended Installation"), so that you see and learn the installer screens.

Do **not** add the four extra lab disks yet. Adding them after the installation guarantees the installer cannot touch them, and that your system disk is the first disk (`sda` or `vda`).

2. Start the VM and walk through the installer. Use the arrow keys, Tab, Space, and Enter to move around.

Installer screen	What to choose
Language	English
Installer update	“Continue without updating” is fine
Keyboard	Your layout
Type of installation	**Ubuntu Server** (not minimized)
Network	Accept the DHCP address it found. Write it down.
Proxy	Leave empty
Ubuntu archive mirror	Accept the default
Guided storage	**Use an entire disk**, keep **Set up this disk as an LVM group** checked
Storage configuration	See the tip below, then **Done** and **Continue**
Profile	Your name: `Student`, server name: `ubuntu-lab`, username: `student`, and a password you will remember
Ubuntu Pro	Skip for now
SSH configuration	**Install OpenSSH server**: checked. Do not import keys.
Featured server snaps	Select none

> **TIP**
> On the storage summary, the installer usually gives the root logical volume (`ubuntu-lv`) only about half of the disk and leaves the rest free in the volume group. That is fine. You will grow it to full size in Task 3 with one command, which is also a sneak preview of Lab 1.3.

3. Wait for the installation to finish, choose **Reboot Now**, and remove the ISO if your hypervisor asks you to. Log in at the console as `student`.

Checkpoint: you see a prompt like `student@ubuntu-lab:~$`, and `hostnamectl` shows `Static hostname: ubuntu-lab` and `Operating System: Ubuntu 24.04.x LTS`.

Task 3: First steps on ubuntu-lab

1. Update the system. Updating takes a few minutes the first time.

```
sudo apt update
sudo apt full-upgrade -y
```

> **What just happened:** `sudo` runs a command as the administrator (root). It asks for **your** password the first time. `apt update` downloads the latest list of available packages from Ubuntu's servers. `apt full-upgrade` installs newer versions of everything already installed, and `-y` answers "yes" to the confirmation prompt.

2. Grow the root filesystem to use the whole disk:

```
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
df -h /
```

```
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   23G  5.1G   17G  24% /
```

> **What just happened:** The installer used **LVM** (Logical Volume Manager), which lets you resize storage on the fly. `lvextend` grew the logical volume into all the free space (`-l +100%FREE`), and `-r` also resized the filesystem inside it, so the extra space is usable immediately. `df -h /` shows the size of the root filesystem in human-readable units. You will learn exactly how this works in Lab 1.3.

3. Install a few tools that many labs use:

```
sudo apt install -y vim nano curl wget git tree bash-completion \
  net-tools dnsutils traceroute tcpdump unzip
```

4. Set your time zone so log timestamps match your clock. List zones with `timedatectl list-timezones`, then set yours (this example uses US Central time):

```
sudo timedatectl set-timezone America/Chicago
timedatectl
```

5. Reboot to load the updated kernel:

```
sudo reboot
```

Task 4: Give ubuntu-lab a fixed IP address (recommended)

Servers normally keep the same IP address forever, so other machines can find them. A fixed (static) address also means the `/etc/hosts` entries you create in Task 7 never go stale.

> TIP
> **Hyper-V Default Switch users:** skip this task and keep DHCP. If your VMs' addresses change after a host reboot, update `/etc/hosts` on both VMs (Task 7).

1. Find your current address, network, and gateway:

```
ip -br address
ip route
```

```
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.4/24 fe80::a00:27ff:fe4e:66a1/64
default via 192.168.100.1 dev enp0s3 proto dhcp src 192.168.100.4 metric 100
192.168.100.0/24 dev enp0s3 proto kernel scope link src 192.168.100.4 metric 100
```

What just happened: `ip -br address` lists your network interfaces briefly (`-br`). Here the interface is named `enp0s3` and has the address `192.168.100.4` on a `/24` network (the first three numbers identify the network). `ip route` shows the routing table. The `default via 192.168.100.1` line is your **default gateway**, the router used to reach everything outside your own network.

Write down three things: your **interface name** (for example `enp0s3` , `ens33` , or `eth0`), your **network** (for example `192.168.100`), and your **gateway** (for example `192.168.100.1` ; on VMware NAT it usually ends in `.2`).

2. Look at the current Netplan configuration. Netplan is Ubuntu's network configuration tool, and it reads YAML files in `/etc/netplan/` :

```
ls /etc/netplan/
sudo cat /etc/netplan/50-cloud-init.yaml
```

The file name may differ slightly. Whatever its name, that file currently tells the interface to use DHCP.

3. Create a new configuration file that sets a static address. Replace `enp0s3` , the address, and the gateway with **your** values. Keep the address in your own network and end it in `.10` :

```
cat <<'EOF' | sudo tee /etc/netplan/60-static.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.100.10/24
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses: [1.1.1.1, 8.8.8.8]
EOF
sudo chmod 600 /etc/netplan/60-static.yaml
```

What just happened: `cat <<'EOF' ... EOF` is a **here document**. Everything between the two `EOF` markers is fed to `sudo tee` , which writes it into the file with root privileges. You will use this trick often, because it avoids typos in an editor. The YAML says: for this interface, turn DHCP off, use this address, send everything else to this gateway, and use these DNS servers. Netplan files are combined in alphabetical order, so `60-static.yaml` overrides the DHCP setting from `50-cloud-init.yaml` . `chmod 600` makes the file readable only by root, because Netplan warns about files that other users can read.

4. Apply the change safely with `netplan try`. It applies the new settings and **automatically rolls back after 120 seconds** unless you press Enter to confirm:

```
sudo netplan try
```

   If the network still works (for example `ping -c 2 8.8.8.8` succeeds in another session or after you confirm), press **Enter** to keep the configuration.

> **CAUTION**
> Do this step from the VM's own console, not over SSH. If you mistype the gateway or interface name, you can lose network access. The console always works.

5. Prevent cloud-init from rewriting the network configuration at a later boot:

```
echo 'network: {config: disabled}' | \
  sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
```

Checkpoint: `ip -br address` shows your `.10` address, and both `ping -c 2 8.8.8.8` and `ping -c 2 ubuntu.com` succeed.

Task 5: Create and install rocky-lab

1. Create a second VM:

Setting	Value
Name	`rocky-lab`
Type	Linux, Red Hat (64-bit), or ARM64 on Apple silicon
CPUs	1 (2 if you can spare it)
Memory	2048 MB
Disk	20 GB, dynamically allocated
Network	The same network as ubuntu-lab
Install media	The Rocky Linux 9 minimal ISO

2. Boot the VM and choose **Install Rocky Linux 9**. The graphical installer, Anaconda, shows an **Installation Summary** hub. Configure each item:

Item	What to choose
Language	English
Installation Destination	Select the 20 GB disk, keep **Automatic** storage, click **Done**
Software Selection	**Minimal Install**
Network & Host Name	Switch the Ethernet adapter **ON**. Set Host Name to `rocky-lab` and click **Apply**.
Root Password	Leave root **locked** ("Lock root account" checked)
User Creation	Full name `Student`, user name `student`, check **Make this user administrator**, set a password
Time & Date	Your time zone

3. Click **Begin Installation**. When it finishes, click **Reboot System** and log in at the console as `student`.

> **What just happened:** "Make this user administrator" adds `student` to the `wheel` group. On Red Hat-style systems, members of `wheel` may use `sudo`. Ubuntu uses a group named `sudo` for the same purpose. Both distributions lock the root account by default, so you always work as a normal user and use `sudo` for admin tasks. That is a security best practice you will study in Lab 3.4.

4. Update the system and install a few tools:

```
# on rocky-lab
sudo dnf -y upgrade
sudo dnf -y install vim nano curl wget git tree bash-completion \
  bind-utils traceroute tcpdump tar unzip policycoreutils-python-utils
sudo reboot
```

> **What just happened:** `dnf` is the package manager on Red Hat-style systems. It does the same job as `apt` on Ubuntu. `upgrade` updates all installed packages, and `-y` answers yes to prompts. `bind-utils` provides `dig` and `nslookup`, and `policycoreutils-python-utils` provides `semanage`, which the SELinux labs use.

Task 6: Give rocky-lab a fixed IP address (recommended)

Rocky Linux manages networking with **NetworkManager**, controlled from the command line with `nmcli`.

1. Find the connection name and current settings:

```
# on rocky-lab
nmcli connection show
ip -br address
ip route
```

```
NAME    UUID                                  TYPE      DEVICE
enp0s3  5fb06bd0-0bb0-7ffb-45f1-d6edd65f3e03  ethernet  enp0s3
lo      2f2b8e4a-5c3a-4b2f-9a1f-0c3b2a1d0e9f  loopback  lo
```

2. Set a static address ending in `.20` in the same network. Replace the connection name, address, and gateway with your values:

```
# on rocky-lab
sudo nmcli connection modify enp0s3 \
  ipv4.method manual \
  ipv4.addresses 192.168.100.20/24 \
  ipv4.gateway 192.168.100.1 \
  ipv4.dns "1.1.1.1 8.8.8.8"
sudo nmcli connection up enp0s3
```

> **What just happened:** `nmcli connection modify` changed the saved connection profile: `ipv4.method manual` turns off DHCP, and the other settings set the address, gateway, and DNS servers. `nmcli connection up` re-activates the profile so the change takes effect. NetworkManager stores the profile in `/etc/NetworkManager/system-connections/`, so it survives reboots.

Checkpoint: `ip -br address` on rocky-lab shows the `.20` address, and `ping -c 2 rockylinux.org` works.

Task 7: Let the VMs find each other by name

DNS turns names into IP addresses, but your lab network has no DNS server for your own VMs. Instead you will use `/etc/hosts`, a simple local lookup file that Linux checks before DNS.

1. On **both** VMs, add these lines to `/etc/hosts`. Use your real addresses:

```
echo "192.168.100.10  ubuntu-lab" | sudo tee -a /etc/hosts
echo "192.168.100.20  rocky-lab"  | sudo tee -a /etc/hosts
```

```
# on rocky-lab
echo "192.168.100.10  ubuntu-lab" | sudo tee -a /etc/hosts
echo "192.168.100.20  rocky-lab"  | sudo tee -a /etc/hosts
```

> **What just happened:** `tee -a` **appends** (`-a`) to the file instead of overwriting it. Running `tee` through `sudo` is the standard way to write to a root-owned file from a pipe. `sudo echo ... >> /etc/hosts` would fail, because the `>>` redirection is done by your own shell, which is not running as root.

2. Test name resolution and connectivity in both directions:

```
ping -c 2 rocky-lab
ssh student@rocky-lab hostname
```

The first time you connect with SSH, you are asked to confirm the server's **host key fingerprint**. Type `yes`, then enter student's password on rocky-lab. The command prints `rocky-lab`.

Checkpoint: `ping rocky-lab` works from ubuntu-lab, and `ping ubuntu-lab` works from rocky-lab.

Task 8: Connect from your host with SSH

Typing in the VM console works, but a terminal on your host is far more comfortable: you get copy and paste, scrollback, and several windows at once.

1. From your host, open a terminal (macOS or Linux Terminal, or Windows Terminal/PowerShell, which include an SSH client) and connect:

```
ssh student@192.168.100.10
```

   - **VirtualBox NAT Network:** the host cannot reach VMs on a NAT Network directly. Add port forwarding: in **Network Manager > NAT Networks > LabNet > Port Forwarding**, add rule `ubuntu-ssh`, host port `2210`, guest IP `192.168.100.10`, guest port `22`, and rule `rocky-ssh`, host port `2220`, guest IP `192.168.100.20`, guest port `22`. Then connect with `ssh -p 2210 student@127.0.0.1` and `ssh -p 2220 student@127.0.0.1`.
   - **VMware, Hyper-V, and UTM:** the host can reach the VM addresses directly.

2. Open a second terminal and connect to rocky-lab the same way. From now on you can work in host terminals.

> TIP
> Many labs ask you to keep a second session open while you change login or network settings. Two host terminal windows connected to the same VM give you exactly that.

Task 9: Add the four lab disks to ubuntu-lab

1. Shut down ubuntu-lab:

```
sudo poweroff
```

2. In your hypervisor's settings for ubuntu-lab, add **four new virtual hard disks**, each **5 GB** and dynamically allocated. Put them on the same controller as the system disk (SATA, SCSI, or VirtIO).

3. Start ubuntu-lab, log in, and list the disks:

```
lsblk -d -o NAME,SIZE,TYPE,MODEL
```

```
NAME  SIZE TYPE MODEL
sda    25G disk VBOX HARDDISK
sdb     5G disk VBOX HARDDISK
sdc     5G disk VBOX HARDDISK
sdd     5G disk VBOX HARDDISK
sde     5G disk VBOX HARDDISK
```

What just happened: `lsblk` lists **block devices** (storage devices). `-d` shows only whole disks, not partitions, and `-o` chooses the columns. Linux names SATA and SCSI disks `sda`, `sdb`, `sdc`, and so on, in the order they are detected. VirtIO disks are named `vda`, `vdb`, and so on, and NVMe disks `nvme0n1`, `nvme0n2`, and so on.

CAUTION

The labs call the extra disks `/dev/sdb`, `/dev/sdc`, `/dev/sdd`, and `/dev/sde`. If your names differ, for example `vdb` or `nvme0n2`, **substitute your names every time**. Always run `lsblk` before any command that writes to a disk, and make sure you never touch the 25 GB system disk.

Task 10: Enable nested virtualization (for Lab 1.7)

Lab 1.7 runs a small VM **inside** ubuntu-lab. That is called **nested virtualization**, and it is much faster if the hypervisor passes the CPU's virtualization features through to the VM. Lab 1.7 also works without nesting, just more slowly, so do not worry if your setup does not support it.

1. With ubuntu-lab powered off, enable nesting:
   - **VirtualBox:** Settings > System > Processor > check **Enable Nested VT-x/AMD-V**. If it is greyed out, run this on the host: `VBoxManage modifyvm ubuntu-lab --nested-hw-virt on`.
   - **VMware:** VM Settings > Processors > check **Virtualize Intel VT-x/EPT or AMD-V/RVI**.
   - **Hyper-V:** in an administrator PowerShell on the host: `Set-VMProcessor -VMName ubuntu-lab -ExposeVirtualizationExtensions $true`
   - **UTM on Apple silicon:** not available on most macOS versions. Lab 1.7 uses emulation instead.
2. Start ubuntu-lab and check:

```
grep -cE 'vmx|svm' /proc/cpuinfo
```

What just happened: `vmx` (Intel) and `svm` (AMD) are CPU flags that mean "hardware virtualization is available". `grep -c` counts the matching lines in `/proc/cpuinfo`, one per CPU core. A number greater than 0 means nesting works. `0` means Lab 1.7 will use software emulation.

Task 11: Take the clean snapshots

1. Shut down both VMs (`sudo poweroff`).
2. In your hypervisor, take a snapshot of each VM and name it `clean`.
   - **VirtualBox:** select the VM, open the **Snapshots** view, and click **Take**.
   - **VMware:** VM > Snapshot > Take Snapshot.
   - **Hyper-V:** right-click the VM > Checkpoint.
   - **UTM:** UTM has no snapshot button. Instead, shut down the VM, right-click it, and choose **Clone** to keep a backup copy.
3. Start both VMs again.

Checkpoint: each VM has a snapshot named `clean`, and you can log in to both from your host.

TIP

Restoring a snapshot: shut down the VM (or power it off), select the snapshot, and choose **Restore** (VirtualBox), **Revert to Snapshot** (VMware), or **Apply** (Hyper-V). The VM returns to exactly the state it was in when you took the snapshot.

Your lab at a glance

	ubuntu-lab	rocky-lab
Operating system	Ubuntu Server 24.04 LTS	Rocky Linux 9 (minimal)
Address (example)	192.168.100.10	192.168.100.20
Login	`student` (in the `sudo` group)	`student` (in the `wheel` group)
Package manager	`apt`, `dpkg`	`dnf`, `rpm`
Network configuration	Netplan (`/etc/netplan/`)	NetworkManager (`nmcli`)
Firewall	`ufw` (off by default)	`firewalld` (on by default)
Mandatory access control	AppArmor	SELinux (enforcing)
Extra disks	`sdb`, `sdc`, `sdd`, `sde` (5 GB each)	none

You are ready. Start with Lab 1.1, or go to any objective you want to practice.

PART 1 · DOMAIN 1.0

Part 1: System Management

23% of the exam · 7 labs

1.1	Explain basic Linux concepts.
1.2	Summarize Linux device management concepts and tools.
1.3	Given a scenario, manage storage in a Linux system.
1.4	Given a scenario, manage network services and configurations on a Linux server.
1.5	Given a scenario, manage a Linux system using common shell operations.
1.6	Given a scenario, perform backup and restore operations for a Linux server.
1.7	Summarize virtualization on Linux systems.

Lab 1.1 Explore Basic Linux Concepts

Objective: 1.1 Explain basic Linux concepts.
Domain: 1.0 System Management (23% of the exam)
Time: about 75 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes. This lab edits the bootloader configuration and reboots ubuntu-lab.

In this lab you follow a Linux system from power-on to login prompt, change a bootloader setting and a kernel parameter and watch the change take effect, and then tour the standard directory layout. You also find out which CPU architecture, distribution family, desktop components, and software licenses your two VMs use. These are the "vocabulary" topics of Linux+: every later lab assumes you know them, and the exam asks about them directly.

What you'll practice

- Basic boot process: bootloader (configuration files), kernel (parameters), initrd, PXE
- Filesystem Hierarchy Standard (FHS): `/`, `/bin`, `/boot`, `/dev`, `/etc`, `/home`, `/lib`, `/proc`, `/sbin`, `/tmp`, `/usr`, `/var`
- Server architectures: AArch64, RISC-V, x86, x86_64/AMD64
- Distributions: RPM-based, dpkg-based
- Graphical user interface (GUI): display managers, window managers, X Server, Wayland
- Software licensing: open source, free software, proprietary, copyleft

How it works

The boot process. Starting a Linux computer is a relay race. Each runner does one job and hands the baton to the next:

```
Power on
   |
Firmware (BIOS or UEFI)      tests hardware, finds a boot device
   |                         (local disk, USB, or the network = PXE)
Bootloader (GRUB 2)          shows a menu, reads its configuration file,
   |                         loads the kernel + initrd into memory
Kernel (vmlinuz)             starts with its parameters (the "command line"),
   |                         detects hardware, mounts the initrd
initrd / initramfs           tiny temporary root filesystem with the drivers
   |                         needed to find and mount the real root disk
Real root filesystem  /
   |
systemd (PID 1)              starts services, reaches a target
   |
Login prompt (text or graphical display manager)
```

1. **Firmware.** The BIOS (older) or UEFI (modern) firmware is built into the motherboard or the VM. It checks the hardware and picks a boot device.
2. **Bootloader.** A bootloader is a small program whose only job is to load the operating system. Linux almost always uses **GRUB 2** (GRand Unified Bootloader). GRUB reads a generated **configuration file**, `/boot/grub/grub.cfg` on Ubuntu or `/boot/grub2/grub.cfg` on Rocky. You never edit that file by hand. Instead you edit `/etc/default/grub` (the settings you choose) and run a tool that rebuilds `grub.cfg`: `update-grub` on Ubuntu, `grub2-mkconfig` on Rocky.
3. **Kernel.** The **kernel** is the core of the operating system. It talks to the hardware and shares the CPU and memory between programs. Its file is `/boot/vmlinuz-<version>`. GRUB passes it **kernel parameters**, such as `root=` (where the real root filesystem is), `ro` (mount it read-only at first), or `quiet` (show fewer messages). You can read the parameters of the running kernel in `/proc/cmdline`.
4. **initrd.** The kernel needs a driver to read the root disk, but the drivers live on that disk. The **initrd** (initial RAM disk), today called an **initramfs**, solves this chicken-and-egg problem. It is a compressed archive that GRUB loads into memory next to the kernel. It holds just enough drivers and scripts to find and mount the real root filesystem, and then it hands over control. On Ubuntu it is `/boot/initrd.img-<version>` (built by `update-initramfs`); on Rocky it is `/boot/initramfs-<version>.img` (built by `dracut`).
5. **systemd.** The first real program, process ID 1, is `systemd`. It starts all services and brings the system to a **target** (a group of services), such as `multi-user.target` (text login) or `graphical.target` (desktop login).

PXE (Preboot eXecution Environment, said "pixie") is booting from the network instead of a disk. The network card's firmware asks a **DHCP** server for an IP address and is told "your boot file is on this **TFTP** server." It downloads a network bootloader (for example `pxelinux.0` or `grubnetx64.efi`), which downloads its configuration, a kernel, and an initrd, and boots them. Data centers use PXE to install hundreds of servers without touching a USB stick.

The Filesystem Hierarchy Standard (FHS) is the agreed map of where things live on a Linux system. Think of it as the floor plan every distribution follows, so an administrator can find the logs or the configuration on any system:

Directory	What lives there
`/`	The root of the single directory tree. Everything hangs below it.
`/bin`	Essential user commands (`ls`, `cp`). On modern systems a link to `/usr/bin`.
`/sbin`	Essential system administration commands (`fdisk`, `ip`). A link to `/usr/sbin`.
`/boot`	The kernel, the initrd, and GRUB files.
`/dev`	Device files, such as `/dev/sda` (a disk) or `/dev/null`.
`/etc`	System-wide configuration files (text).
`/home`	Personal directories for users, such as `/home/student`.
`/lib`	Shared libraries and kernel modules. A link to `/usr/lib`.
`/proc`	A virtual filesystem: live information from the kernel, not stored on disk.
`/tmp`	Temporary files that anyone can write. Often cleared at boot.
`/usr`	Installed programs, libraries, and documentation (read-mostly).
`/var`	Variable data that grows: logs, caches, mail, databases, print queues.

Architectures. A CPU architecture is the instruction language a processor understands. A program compiled for one architecture does not run on another. **x86** is the original 32-bit Intel family. **x86_64**, also called **AMD64** (AMD invented the 64-bit extension) or `amd64` in Debian/Ubuntu package names, is the 64-bit version used by most PCs and servers. **AArch64** (also called ARM64, `arm64` in Ubuntu) is 64-bit ARM, used in phones, Apple silicon Macs, Raspberry Pi, and cloud servers such as AWS Graviton. **RISC-V** is a newer, open (royalty-free) instruction set used in embedded boards and growing in servers.

Distribution families. A distribution is the Linux kernel plus tools, a package format, and an installer. Two big families matter for the exam. **dpkg-based** distributions (Debian, Ubuntu, Mint) use `.deb` packages, the low-level tool `dpkg`, and the high-level tool `apt`. **RPM-based** distributions (Red Hat Enterprise Linux, Rocky, AlmaLinux, Fedora, openSUSE) use `.rpm` packages, the low-level tool `rpm`, and high-level tools such as `dnf` or `zypper`.

The GUI stack. A Linux desktop is built from separate layers. The **display server** draws pixels and handles the keyboard and mouse. The classic one is the **X Server** (X11, Xorg); the modern replacement is **Wayland**, a protocol where the compositor itself is the display server. A **window manager** decides where windows go and draws their borders and title bars (examples: Mutter in GNOME, KWin in KDE, Openbox, i3). A **display manager** is the graphical login screen that starts your session (GDM, SDDM, LightDM). A desktop environment (GNOME, KDE Plasma, Xfce) bundles all of this with apps. Servers usually have no GUI at all.

Software licensing. Open source means the source code is available and you may study, change, and share it under the license. **Free software** (as defined by the Free Software Foundation) means "free as in freedom": the four freedoms to run, study, change, and redistribute. In practice most open source software is also free software. **Copyleft**

licenses such as the GNU GPL say “if you distribute a changed version, you must share your source under the same license.” Permissive licenses (MIT, BSD, Apache) let others make closed versions. **Proprietary** software is owned and controlled by a company; you get a license to use it, usually without source code, and you may not redistribute it (examples: some NVIDIA drivers, many commercial databases).

Tasks

Task 1: Look at the boot files on ubuntu-lab

1. Log in to ubuntu-lab as `student` and create the lab working directory. `mkdir -p` creates the folder (and any missing parent folders) without complaining if it already exists, and `cd` moves you into it.

```
mkdir -p ~/labs/1.1 && cd ~/labs/1.1
```

2. Find out whether the VM booted with UEFI or legacy BIOS firmware. UEFI systems expose the folder `/sys/firmware/efi`; BIOS systems do not.

```
[ -d /sys/firmware/efi ] && echo "UEFI boot" || echo "Legacy BIOS boot"
```

```
UEFI boot
```

> **What just happened:** `[ -d PATH ]` is a test that is true if `PATH` exists and is a directory. `&&` runs the next command only if the test succeeded, and `||` runs its command only if the test failed. Either answer is fine for this lab. On UEFI systems, the firmware loads GRUB from a small FAT partition mounted at `/boot/efi`.

3. List the contents of `/boot`. This is where GRUB looks for the kernel and the initrd.

```
ls -lh /boot
```

```
-rw-r--r-- 1 root root 287K Aug  9 14:02 config-6.8.0-45-generic
drwx------ 3 root root 4.0K Jan  1  1970 efi
drwxr-xr-x 5 root root 4.0K Oct 10 09:12 grub
lrwxrwxrwx 1 root root   28 Oct 10 09:10 initrd.img -> initrd.img-6.8.0-45-generic
-rw-r--r-- 1 root root  67M Oct 10 09:11 initrd.img-6.8.0-45-generic
-rw------- 1 root root 8.9M Aug  9 14:02 System.map-6.8.0-45-generic
lrwxrwxrwx 1 root root   25 Oct 10 09:10 vmlinuz -> vmlinuz-6.8.0-45-generic
-rw------- 1 root root  15M Aug  9 14:02 vmlinuz-6.8.0-45-generic
```

What just happened: `ls -lh` shows a long listing (`-l`) with human-readable sizes (`-h`). Read it like this:

- `vmlinuz-6.8.0-45-generic` is the compressed **kernel**. The "z" means compressed.
- `initrd.img-6.8.0-45-generic` is the **initrd** (initramfs) for that exact kernel version. It is much larger than the kernel because it contains drivers and tools.
- `config-...` is the list of options the kernel was compiled with, and `System.map-...` is a table of kernel symbol addresses used for debugging.
- `vmlinuz` and `initrd.img` without a version are **symbolic links** (shortcuts, shown by `->`) that always point at the newest kernel.
- `grub/` holds GRUB's modules and its generated configuration file. `efi/` is the EFI system partition (only on UEFI systems).

Your version numbers will differ. You may see two kernels if updates were installed; Ubuntu keeps the previous one as a fallback.

4. Compare the file names with the kernel that is running now. `uname -r` prints the running kernel release.

```
uname -r
```

```
6.8.0-45-generic
```

Checkpoint: You can name the kernel file and the initrd file in `/boot` that match the output of `uname -r` . Run `ls /boot/*$(uname -r)*` and you should see the `config` , `initrd.img` , `System.map` , and `vmlinuz` files for your running kernel.

Task 2: Read the bootloader configuration files and the kernel command line

1. Display GRUB's settings file. This is the file **you** edit.

```
cat /etc/default/grub
```

```
GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release; echo ${NAME:-Ubuntu} ) 2>/dev/null || echo Ubuntu`
GRUB_CMDLINE_LINUX_DEFAULT=""
GRUB_CMDLINE_LINUX=""
```

What just happened: `cat` prints a file. The comment lines (starting with `#`) were left out of the sample. The important settings are:

- `GRUB_DEFAULT=0` boots the first menu entry by default.
- `GRUB_TIMEOUT_STYLE=hidden` and `GRUB_TIMEOUT=0` hide the menu and boot immediately. (Holding **Shift** on BIOS, or pressing **Esc** on UEFI, during boot still shows the menu.)
- `GRUB_CMDLINE_LINUX_DEFAULT` holds kernel parameters for normal boots only. Ubuntu Desktop puts `quiet splash` here.
- `GRUB_CMDLINE_LINUX` holds kernel parameters added to **every** boot entry, including recovery mode.

TIP

Ubuntu also reads files in `/etc/default/grub.d/` after `/etc/default/grub`, and they can override it. Cloud images often have one there. Check with `ls /etc/default/grub.d/`. If a file there sets `GRUB_TIMEOUT`, that value wins.

2. Look at the generated configuration file that GRUB actually reads at boot. It is long, so search it for the lines that load the kernel and initrd. `grep -E` searches using an extended regular expression, and `-m 4` stops after four matches. The pattern means: at the start of a line (`^`), after any spaces or tabs (`\s*`), the word `menuentry` followed by a space, or `linux` or `initrd` followed by whitespace (`\s`). The extra space and `\s` stop it from matching helper lines such as `menuentry_id_option=` near the top of the file.

```
sudo grep -E -m 4 '^\s*(menuentry |linux\s|initrd\s)' /boot/grub/grub.cfg
```

```
menuentry 'Ubuntu' --class ubuntu --class gnu-linux --class gnu --class os ... {
        linux   /vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
        initrd  /initrd.img-6.8.0-45-generic
menuentry 'Ubuntu, with Linux 6.8.0-45-generic' --class ubuntu ...
```

What just happened: Each `menuentry` is one choice in the GRUB menu. Inside it, the `linux` line names the kernel file and its **parameters** (`root=` and `ro` here), and the `initrd` line names the initrd file. Notice the top of the real file says `DO NOT EDIT THIS FILE`: it is rebuilt from `/etc/default/grub` and the scripts in `/etc/grub.d/` every time `update-grub` runs, so manual edits would be lost.

3. Read the parameters the running kernel actually received.

```
cat /proc/cmdline
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
```

What just happened: `/proc/cmdline` is not a real file on disk. `/proc` is a virtual filesystem, and reading this "file" asks the kernel to print the command line it was started with. `BOOT_IMAGE=` is added by GRUB to say which kernel file it loaded. This matches the `linux` line in `grub.cfg`, which proves which configuration was used.

Checkpoint: You can explain the difference between `/etc/default/grub` (you edit it), `/boot/grub/grub.cfg` (generated, GRUB reads it), and `/proc/cmdline` (what the kernel received). `cat /proc/cmdline` should show a `root=` parameter.

Task 3: Read the story of the last boot

1. Ask systemd how long the last boot took.

```
systemd-analyze
```

```
Startup finished in 3.512s (firmware) + 2.104s (loader) + 2.311s (kernel)
+ 9.874s (userspace) = 17.802s
graphical.target reached after 9.801s in userspace.
```

What just happened: `systemd-analyze` splits the boot into the stages from the diagram: **firmware** (UEFI only), **loader** (GRUB), **kernel** (kernel plus initrd), and **userspace** (systemd starting services). On a BIOS VM the firmware and loader times are not shown. Do not worry about "graphical.target" on a server; on Ubuntu Server it is an alias that adds nothing visible.

2. See which services took the longest, and the chain of units that the boot waited for. Press `q` to leave the pager if one opens.

```
systemd-analyze blame | head -5
systemd-analyze critical-chain
```

What just happened: `blame` lists units sorted by how long each took to start; `head -5` keeps the first five lines. `critical-chain` shows the path of units that the final target had to wait for, with times after `@` (when it started) and `+` (how long it took). Slow boots are usually explained by the top entries.

3. Read the kernel's own messages from the current boot. `journalctl` reads the systemd journal; `-b` limits it to the current boot and `-k` to kernel messages. `--no-pager` prints straight to the screen.

```
journalctl -b -k --no-pager | head -8
```

```
Oct 10 09:20:01 ubuntu-lab kernel: Linux version 6.8.0-45-generic ...
Oct 10 09:20:01 ubuntu-lab kernel: Command line: BOOT_IMAGE=/vmlinuz-6.8.0-45-generic
root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
Oct 10 09:20:01 ubuntu-lab kernel: KERNEL supported cpus:
Oct 10 09:20:01 ubuntu-lab kernel:   Intel GenuineIntel
Oct 10 09:20:01 ubuntu-lab kernel:   AMD AuthenticAMD
...
```

What just happened: The very first kernel messages show the kernel version and, again, the command line. If you run `journalctl --list-boots`, you see a numbered list of earlier boots; `journalctl -b -1` shows the previous boot. Ubuntu keeps the journal on disk in `/var/log/journal`, so these survive reboots.

4. Find the moment the initrd handed over to the real root filesystem.

```
journalctl -b --no-pager | grep -i -m 3 -E 'initrd|switch root'
```

What just happened: You should see lines such as `Reached target initrd-switch-root.target` or `Switching root.`. That is systemd inside the initramfs finishing its job and switching to your real `/`. On some Ubuntu builds the initramfs uses shell scripts instead of systemd and the wording differs; any line mentioning initrd or switching root is fine.

Checkpoint: You can name the four stages `systemd-analyze` reports and find the kernel command line in `journalctl -b -k`.

Task 4: Change GRUB and add a kernel parameter (then revert)

CAUTION

Take a snapshot of ubuntu-lab now if you have not already. A typo in `/etc/default/grub` can stop the VM from booting. You will make a backup copy first, and you should work at the VM console (not only over SSH) so you can see the GRUB menu.

1. Make a backup copy of the settings file. `cp` copies a file; `sudo` is needed because only root can write in `/etc`.

```
sudo cp /etc/default/grub /etc/default/grub.lab-backup
```

2. Show the GRUB menu for 10 seconds and add a harmless made-up kernel parameter, `labtest=1`. You can edit the file with `sudo nano /etc/default/grub`, or use the three `sed` commands below, which do the same edits without typos.

```
sudo sed -i 's/^GRUB_TIMEOUT_STYLE=.*/GRUB_TIMEOUT_STYLE=menu/' /etc/default/grub
sudo sed -i 's/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/' /etc/default/grub
sudo sed -i \
  's/^GRUB_CMDLINE_LINUX_DEFAULT="\(.*\)"/GRUB_CMDLINE_LINUX_DEFAULT="\1 labtest=1"/' \
  /etc/default/grub
```

What just happened: `sed` is a stream editor. `-i` edits the file in place. The expression `s/OLD/NEW/` substitutes text. `^` means "start of the line" and `.*` means "anything", so `s/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/` replaces the whole `GRUB_TIMEOUT` line. In the third command, `\(.*\)` captures whatever was already between the quotes, and `\1` puts it back, followed by `labtest=1`. That way any existing parameters (such as `quiet splash`) are kept.

3. Check your edits before going further.

```
grep -E '^GRUB_(TIMEOUT|CMDLINE)' /etc/default/grub
```

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
GRUB_CMDLINE_LINUX_DEFAULT=" labtest=1"
GRUB_CMDLINE_LINUX=""
```

What just happened: The leading space inside the quotes is harmless; the kernel ignores extra spaces. If any line looks wrong, fix it with `sudo nano /etc/default/grub` before continuing.

4. Rebuild `grub.cfg` from your settings.

```
sudo update-grub
```

```
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-45-generic
Found initrd image: /boot/initrd.img-6.8.0-45-generic
Adding boot menu entry for UEFI Firmware Settings ...
done
```

What just happened: `update-grub` is an Ubuntu/Debian shortcut for `grub-mkconfig -o /boot/grub/grub.cfg`. It reads `/etc/default/grub`, runs each script in `/etc/grub.d/` (for example `10_linux` finds kernels in `/boot`), and writes a fresh `grub.cfg`. The last line appears only on UEFI systems. If you also see a warning that `os-prober will not be executed`, it only means GRUB will not look for other operating systems such as Windows. That is expected on a server.

5. Confirm the new parameter is now in the generated file, then reboot.

```
sudo grep -m 1 'labtest' /boot/grub/grub.cfg
sudo reboot
```

6. Watch the VM console. You now see the GRUB menu with a 10-second countdown. Use the arrow keys to look at the entries (including **Advanced options for Ubuntu**), but do not press `e` yet. Let it boot the default entry.

TIP

At the GRUB menu, pressing `e` opens the selected entry for a **one-time** edit. You can add or remove kernel parameters on the `linux` line and press **Ctrl+X** to boot. Nothing is saved. Administrators use this to rescue a system, for example by adding `systemd.unit=rescue.target` to boot into single-user rescue mode.

7. Log back in and check what the kernel received.

```
cat /proc/cmdline
sudo dmesg | grep -i labtest
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro labtest=1
[    0.071234] Unknown kernel command line parameters "BOOT_IMAGE=/vmlinuz-6.8.0-45-generic
labtest=1", will be passed to user space.
```

What just happened: The parameter traveled the whole chain: `/etc/default/grub` to `grub.cfg` (via `update-grub`) to GRUB to the kernel. The kernel did not recognize `labtest=1`, so it printed a notice and passed it on to user space (programs can still read it in `/proc/cmdline`). That is why it is harmless. Real parameters work the same way, for example `nomodeset` (do not load graphics drivers), `systemd.unit=multi-user.target`, or `mitigations=off`. `dmesg` prints the kernel's message buffer; `sudo` is needed because Ubuntu restricts it to root.

8. Revert to the original settings and rebuild `grub.cfg` again.

```
sudo cp /etc/default/grub.lab-backup /etc/default/grub
sudo update-grub
sudo grep -c labtest /boot/grub/grub.cfg
```

```
0
```

What just happened: You restored the backup and regenerated the file. `grep -c` counts matching lines; `0` proves the parameter is gone from the configuration. It disappears from `/proc/cmdline` at the next reboot. Leave the backup file for now; you delete it in Clean up.

Checkpoint: Before the revert, `cat /proc/cmdline` showed `labtest=1`. After the revert, `sudo grep -c labtest /boot/grub/grub.cfg` prints `0` and `grep ^GRUB_TIMEOUT= /etc/default/grub` shows the original value.

Task 5: Compare GRUB tools and look inside the initrd on both systems

1. On ubuntu-lab, list what is inside the initrd. `lsinitramfs` (from the `initramfs-tools-core` package, installed by default) lists the files in an initramfs image.

```
sudo lsinitramfs /boot/initrd.img-$(uname -r) | wc -l
sudo lsinitramfs /boot/initrd.img-$(uname -r) | grep -E 'bin/(busybox|lvm|udevadm)$'
```

```
3417
usr/bin/busybox
usr/bin/udevadm
usr/sbin/lvm
```

What just happened: `$(uname -r)` is **command substitution**: the shell runs `uname -r` and puts its output into the file name, so the command always targets the running kernel. `wc -l` counts lines, so you see the initrd holds thousands of files. The `grep` finds three key tools: `busybox` (a small all-in-one shell and command set), `udevadm` (detects devices), and `lvm` (needed here because Ubuntu's default install puts `/` on an LVM volume). Without `lvm` in the initrd, the kernel could not find `root=/dev/mapper/ubuntu--vg-ubuntu--lv`.

2. Switch to rocky-lab (log in at its console or run `ssh student@rocky-lab`). Rocky uses the same GRUB 2, but the commands have a `2` in their names and the files are in different places.

```
# on rocky-lab
cat /etc/default/grub
ls /boot
ls /boot/loader/entries/
```

```
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap"
GRUB_DISABLE_RECOVERY="true"
GRUB_ENABLE_BLSCFG=true
...
config-5.14.0-427.13.1.el9_4.x86_64
efi
grub2
initramfs-5.14.0-427.13.1.el9_4.x86_64.img
loader
vmlinuz-5.14.0-427.13.1.el9_4.x86_64
...
a1b2c3d4e5f6-5.14.0-427.13.1.el9_4.x86_64.conf
```

What just happened: Rocky's initrd is named `initramfs-<version>.img`. `GRUB_ENABLE_BLSCFG=true` means Rocky uses the **Boot Loader Specification** (BLS): each kernel gets its own small entry file in `/boot/loader/entries/` instead of a long `menuentry` block in `grub.cfg`. The `rd.lvm.lv=` parameters tell the initramfs (`rd` means "RAM disk") which LVM volumes to activate.

3. On Rocky, the recommended tool for changing kernel parameters is `grubby`, which edits the BLS entries directly. Look at the default entry, add the test parameter, check it, and remove it again.

```
# on rocky-lab
sudo grubby --default-kernel
sudo grubby --update-kernel=ALL --args="labtest=1"
sudo grubby --info=DEFAULT | grep ^args
sudo grubby --update-kernel=ALL --remove-args="labtest=1"
sudo grubby --info=DEFAULT | grep ^args
```

```
/boot/vmlinuz-5.14.0-427.13.1.el9_4.x86_64
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap labtest=1"
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=/dev/mapper/rl-swap
rd.lvm.lv=rl/root rd.lvm.lv=rl/swap"
```

What just happened: `--default-kernel` shows the kernel GRUB boots by default. `--update-kernel=ALL` applies a change to every installed kernel, and `--args` / `--remove-args` add or remove parameters. `--info=DEFAULT` prints the default entry, and its `args=` line is what goes into `/proc/cmdline` at the next boot. You did not reboot, so nothing changed in the running system.

TIP

If you change `/etc/default/grub` on Rocky (for example `GRUB_TIMEOUT`), rebuild with `sudo grub2-mkconfig -o /boot/grub2/grub.cfg`. That path is correct on both BIOS and UEFI systems in Rocky 9. On Rocky 9.3 and later, `grub2-mkconfig` only copies `GRUB_CMDLINE_LINUX` into existing BLS entries if you add `--update-bls-cmdline`, which is one more reason to prefer `grubby` for kernel parameters.

4. Look inside Rocky's initrd. `lsinitrd` comes with `dracut`; with no file name it inspects the initramfs of the running kernel.

```
# on rocky-lab
sudo lsinitrd | head -15
sudo lsinitrd -m
```

What just happened: The first lines of `lsinitrd` show the image size and the dracut version, followed by the **dracut modules** included (such as `lvm`, `systemd`, `kernel-modules`) and then the file list. `-m` lists only the dracut modules. Lab 1.2 shows how to rebuild this image with `dracut -f`.

Checkpoint: You can name the bootloader configuration tool on each system (`update-grub` on Ubuntu, `grub2-mkconfig` or `grubby` on Rocky), the initrd file name on each, and the command that lists its contents (`lsinitramfs` vs `lsinitrd`). On rocky-lab, `sudo grubby --info=DEFAULT | grep -c labtest` prints `0`.

Task 6: Explore PXE network booting

TIP

Explore/simulate: A real PXE boot needs a DHCP server, a TFTP server, and a client VM with network boot enabled, and a second DHCP server on your lab network could break your other VMs. Instead, you will write the configuration files a PXE server would use, so you can recognize each piece on the exam. Nothing is installed or started.

1. Here is the conversation a PXE client has when it powers on:

```
PXE client (no OS)                  Servers on the network
------------------                  ----------------------
DHCPDISCOVER  "I need an IP + boot info"  ---> DHCP server
              <--- DHCPOFFER  IP 192.168.100.50
                   next-server 192.168.100.10  (TFTP server)
                   filename "grubnetx64.efi"   (or pxelinux.0 for BIOS)
TFTP GET grubnetx64.efi                 ---> TFTP server
TFTP GET grub/grub.cfg                  ---> TFTP server
TFTP/HTTP GET vmlinuz + initrd          ---> TFTP or HTTP server
boot kernel with parameters from the config
```

> **What just happened:** DHCP (Dynamic Host Configuration Protocol) gives out addresses and can also say where the boot file is. TFTP (Trivial File Transfer Protocol, UDP port 69) is a tiny file transfer protocol simple enough to fit in network card firmware. After the bootloader is running, larger files are often fetched over HTTP, which is faster.

2. Write an example `dnsmasq` configuration. `dnsmasq` is a small program that can be both the DHCP and TFTP server. This lab uses a **heredoc**: everything between `<<'EOF'` and the line `EOF` is fed to `cat` as input, and `cat` writes it into the file named after `>`. The quotes around `'EOF'` stop the shell from changing anything inside.

```
mkdir -p ~/labs/1.1/pxe/pxelinux.cfg
cat <<'EOF' > ~/labs/1.1/pxe/dnsmasq-pxe.conf
# Example only: DHCP + TFTP for PXE clients on the lab network
interface=enp0s3
dhcp-range=192.168.100.50,192.168.100.99,12h
enable-tftp
tftp-root=/srv/tftp
# BIOS clients get pxelinux.0, UEFI x86_64 clients get GRUB
dhcp-match=set:efi64,option:client-arch,7
dhcp-boot=tag:efi64,grubnetx64.efi
dhcp-boot=tag:!efi64,pxelinux.0
EOF
```

3. Write the menu file a BIOS client would download after `pxelinux.0`. PXELINUX looks for `pxelinux.cfg/default` when no host-specific file exists.

```
cat <<'EOF' > ~/labs/1.1/pxe/pxelinux.cfg/default
DEFAULT install
PROMPT 0
TIMEOUT 50
LABEL install
  MENU LABEL Install Ubuntu Server 24.04 over the network
  KERNEL ubuntu/vmlinuz
  APPEND initrd=ubuntu/initrd ip=dhcp url=http://192.168.100.10/ubuntu-24.04.iso
EOF
cat ~/labs/1.1/pxe/pxelinux.cfg/default
```

> **What just happened:** This file has the same parts as a local GRUB entry: a **kernel** (`KERNEL`), an **initrd** (`initrd=`), and **kernel parameters** (`ip=dhcp` tells the initrd to get an address, `url=` tells the Ubuntu installer where to download the ISO). The only difference from a disk boot is that every file arrives over the network. A UEFI client would read a `grub.cfg` with `linux` and `initrd` lines instead.

> **Checkpoint:** `ls -R ~/labs/1.1/pxe` shows `dnsmasq-pxe.conf` and `pxelinux.cfg/default` , and you can explain the order: DHCP, then TFTP download of the bootloader, then its configuration, then kernel and initrd.

Task 7: FHS scavenger hunt

Answer each question by running the command. Try to guess the answer before you run it.

Start with a look at the top of the tree. `ls /` lists the directories directly under the root directory `/` .

```
ls /
```

```
bin                boot  etc   lib    lib64       media  opt   root  sbin  srv  tmp  var
bin.usr-is-merged  dev   home  lib.usr-is-merged  lost+found  mnt   proc  run
sbin.usr-is-merged  snap  sys  usr
```

What just happened: Every name the FHS table lists is here, plus a few more: `/opt` (add-on software from vendors), `/srv` (data served by this system, such as websites), `/mnt` and `/media` (mount points for temporary and removable storage), `/run` (runtime data such as process ID files, kept in RAM), `/root` (the root user's home directory, not to be confused with `/`), and `/sys` (another virtual filesystem, for devices and drivers). The `*.usr-is-merged` entries are markers Ubuntu leaves to record that the merged `/usr` layout is in place. Now answer the hunt questions.

1. Which top-level directories are really links? Ubuntu 24.04 and Rocky 9 both use the "merged /usr" layout.

```
ls -l / | grep -- '->'
```

```
lrwxrwxrwx   1 root root     7 Apr 22 13:08 bin -> usr/bin
lrwxrwxrwx   1 root root     7 Apr 22 13:08 lib -> usr/lib
lrwxrwxrwx   1 root root     9 Apr 22 13:08 lib64 -> usr/lib64
lrwxrwxrwx   1 root root     8 Apr 22 13:08 sbin -> usr/sbin
```

What just happened: `/bin`, `/sbin`, and `/lib` still exist for compatibility, but they point into `/usr`. The `--` tells `grep` that `->` is a pattern, not an option. On the exam, still remember the classic meaning: `/bin` holds essential user commands, `/sbin` essential system administration commands, and `/lib` essential libraries.

2. Where do kernel modules (drivers) live, and where does the `ip` command live?

```
ls /lib/modules/
command -v ip ls fdisk
```

What just happened: `/lib/modules/` has one folder per installed kernel version. `command -v` prints the full path of each command. You will see `/usr/bin/ls` and `/usr/sbin/ip` and `/usr/sbin/fdisk`, which shows the `/sbin` = system administration convention.

3. What is in `/dev`? Find your disks and the "black hole" device.

```
ls -l /dev/sd* /dev/null
echo "this disappears" > /dev/null
```

```
brw-rw---- 1 root disk 8,  0 Oct 10 09:20 /dev/sda
brw-rw---- 1 root disk 8, 16 Oct 10 09:20 /dev/sdb
crw-rw-rw- 1 root root 1,  3 Oct 10 09:20 /dev/null
```

> **What just happened:** The first letter of the permissions shows the device type: `b` is a **block** device (stores data in blocks, like a disk) and `c` is a **character** device (a stream of bytes, like a terminal or `/dev/null`). Anything written to `/dev/null` is thrown away. `/dev` is created in memory by the kernel and `udev` at every boot. If your disks are named `vda` or `nvme0n1`, use `/dev/vd*` or `/dev/nvme*` instead.

4. What does `/proc` know about your CPU and memory, and which process is number 1?

```
grep -m 1 'model name' /proc/cpuinfo
grep MemTotal /proc/meminfo
cat /proc/1/comm
```

> **What just happened:** `/proc` files are generated by the kernel on demand. `cpuinfo` and `meminfo` describe hardware; each numbered folder is a running process, and `/proc/1/comm` prints the name of process 1, which is `systemd`. Run `df -hT /proc` (`-T` adds a Type column) and you will see it is type `proc` with size 0: it uses no disk.

5. Where are configuration files, home directories, and logs?

```
cat /etc/hostname
ls /home
ls /var/log | head
sudo du -sh /var/log /var/cache
```

> **What just happened:** `/etc/hostname` is a plain text configuration file. `/home` contains `student`. `/var/log` holds logs and `/var/cache` holds downloaded package files: both grow over time, which is why some administrators put `/var` on its own partition. `du -sh` shows the total size (`-s` summary, `-h` human-readable).

6. Why can everyone write to `/tmp`, but not delete each other's files?

```
ls -ld /tmp
findmnt /tmp
```

```
drwxrwxrwt 14 root root 4096 Oct 10 09:30 /tmp
```

> **What just happened:** `-d` lists the directory itself instead of its contents. `rwxrwxrwx` means everyone can write. The final `t` is the **sticky bit**: in a sticky directory, users can delete only their own files. On Ubuntu 24.04, `findmnt /tmp` usually prints nothing because `/tmp` is a normal folder on the root disk that is cleaned at boot; on some systems (such as Fedora) `/tmp` is a `tmpfs` in RAM.

7. What is under `/usr`?

```
ls /usr
```

```
bin  games  include  lib  lib64  libexec  local  sbin  share  src
```

What just happened: `/usr/bin` and `/usr/sbin` hold programs installed by the package manager, `/usr/share` holds architecture-independent data such as documentation and licenses, and `/usr/local` is for software you install by hand, so the package manager never overwrites it.

Checkpoint: Without looking, say which directory you would check for: a log file (`/var/log`), an SSH server setting (`/etc/ssh`), a kernel image (`/boot`), a disk device (`/dev`), live process information (`/proc`), and a user's documents (`/home`).

Task 8: Identify the architecture and the distribution family

1. Ask for the CPU architecture three different ways.

```
uname -m
lscpu | head -4
dpkg --print-architecture
```

```
x86_64
Architecture:            x86_64
  CPU op-mode(s):        32-bit, 64-bit
  Address sizes:         39 bits physical, 48 bits virtual
  Byte Order:            Little Endian
amd64
```

What just happened: `uname -m` prints the machine hardware name the kernel uses: `x86_64` . `lscpu` shows CPU details; "op-mode(s) 32-bit, 64-bit" means this x86_64 CPU can also run old 32-bit **x86** code. `dpkg --print-architecture` shows the Debian name for the same thing: `amd64` . So **x86_64** and **AMD64** are two names for one architecture. If your VM runs on an Apple silicon Mac, you will see `aarch64` and `arm64` instead: two names for **AArch64**. A **RISC-V** system shows `riscv64` .

2. Check which architecture a program was compiled for.

```
sudo apt install -y file
file -L /bin/ls
```

```
/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, ...
```

What just happened: `file` identifies a file's type; `-L` follows the symbolic link. ELF is the Linux program format, and `x86-64` is the target CPU. A program built for `ARM aarch64` would not run on this VM. This is why distributions publish separate downloads per architecture.

3. Identify the distribution on ubuntu-lab.

```
grep -E '^(NAME|VERSION_ID|ID|ID_LIKE)=' /etc/os-release
dpkg -l | head -7
dpkg -l | grep -c '^ii'
dpkg -S /usr/bin/ls
```

```
NAME="Ubuntu"
VERSION_ID="24.04"
ID=ubuntu
ID_LIKE=debian
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name           Version        Architecture Description
+++-==============-==============-============-=================================
ii  adduser        3.137ubuntu1   all          add and remove users and groups
ii  apparmor       4.0.1-0ubuntu0 amd64        user-space parser utility for AppArmor
612
coreutils: /usr/bin/ls
```

What just happened: `/etc/os-release` exists on every modern distribution. `ID_LIKE=debian` tells you Ubuntu belongs to the Debian (**dpkg-based**) family. `dpkg -l` lists installed packages; `ii` means "desired: install, status: installed". The Architecture column shows `amd64` or `all` (no CPU-specific code). `grep -c '^ii'` counts installed packages. `dpkg -S` searches for the package that owns a file.

4. Run the RPM equivalents on rocky-lab.

```
# on rocky-lab
grep -E '^(NAME|VERSION_ID|ID|ID_LIKE)=' /etc/os-release
rpm -qa | head -3
rpm -qa | wc -l
rpm -qf /usr/bin/ls
uname -m
```

```
NAME="Rocky Linux"
VERSION_ID="9.4"
ID="rocky"
ID_LIKE="rhel centos fedora"
libgcc-11.4.1-3.el9.x86_64
crypto-policies-20240202-1.git283706d.el9.noarch
tzdata-2024a-1.el9.noarch
402
coreutils-8.32-35.el9.x86_64
x86_64
```

What just happened: `ID_LIKE="rhel centos fedora"` marks Rocky as Red Hat family (**RPM-based**). `rpm -qa` queries (`-q`) all (`-a`) installed packages. RPM package names end with the architecture: `x86_64` , or `noarch` for packages with no CPU-specific code (Debian calls this `all`). `rpm -qf` finds the package that owns a file, like `dpkg -S` .

Checkpoint: You can fill in this table from your own output: architecture name from `uname -m` (`x86_64`), Debian name (`amd64`), family of ubuntu-lab (dpkg-based), family of rocky-lab (RPM-based), and the "who owns this file" command on each (`dpkg -S` / `rpm -qf`).

Task 9: Explore the GUI stack and software licenses

TIP

Explore/simulate: Your VMs are servers without a desktop, and installing one takes gigabytes. You will instead ask the system what it would use and read the package descriptions of the GUI pieces.

1. Check which systemd target the system boots into and what kind of session you are using.

```
systemctl get-default
echo "$XDG_SESSION_TYPE"
systemctl status display-manager --no-pager
```

```
graphical.target
tty
Unit display-manager.service could not be found.
```

What just happened: `systemctl get-default` shows the boot target. Ubuntu Server may print `graphical.target` or `multi-user.target` ; with no desktop installed, both end at a text login. Rocky minimal prints `multi-user.target` . `XDG_SESSION_TYPE` is `tty` for a text console or SSH; on a desktop it would be `x11` or `wayland` . `display-manager.service` is an alias that points to whichever **display manager** (GDM, SDDM, LightDM) is installed. Here none is, so the unit is not found.

2. Read what each GUI layer does, using the package descriptions. `apt show` describes a package without installing it; `2>/dev/null` hides a harmless warning about the command-line interface.

```
apt show gdm3 2>/dev/null | grep -A1 '^Description'
apt show xserver-xorg-core 2>/dev/null | grep -A1 '^Description'
apt show openbox 2>/dev/null | grep -A1 '^Description'
apt show libwayland-server0 2>/dev/null | grep -A1 '^Description'
```

What just happened: `grep -A1` prints each match plus one line after it. You should see: `gdm3` is the GNOME Display Manager (the graphical login screen, a **display manager**); `xserver-xorg-core` is the Xorg **X Server** (the classic display server); `openbox` is a standalone **window manager**; and `libwayland-server0` is the library for **Wayland** compositors. On Ubuntu Desktop 24.04, GNOME uses GDM and runs on Wayland by default, with "Ubuntu on Xorg" as a login option. On a desktop you would switch with `sudo systemctl set-default graphical.target` .

3. Look at the license texts Ubuntu ships, and at the copyright file of one package.

```
ls /usr/share/common-licenses/
grep -m 3 -i 'license' /usr/share/doc/bash/copyright
```

```
Apache-2.0  CC0-1.0  GFDL-1.2  GPL    GPL-2  LGPL    LGPL-2.1  MPL-1.1
Artistic    GFDL     GFDL-1.3  GPL-1  GPL-3  LGPL-2  LGPL-3    MPL-2.0
BSD
License: GPL-3+
```

What just happened: `/usr/share/common-licenses/` stores one copy of each common license so packages do not each carry it. Every Debian/Ubuntu package documents its license in `/usr/share/doc/<package>/copyright`. Bash is under the GPL version 3 or later (`GPL-3+`), a **copyleft** license: anyone who distributes a modified bash must release the source under the GPL. Apache-2.0, BSD, and MIT are permissive **open source** licenses.

4. See where Ubuntu keeps non-free software.

```
grep -h '^Components' /etc/apt/sources.list.d/ubuntu.sources
```

```
Components: main restricted universe multiverse
Components: main restricted universe multiverse
```

What just happened: Ubuntu splits its archive into components. `main` and `universe` contain **free software**. `restricted` contains **proprietary** device drivers (such as some GPU firmware), and `multiverse` contains software with license or legal restrictions. You see two lines because the file has one section for the main archive and one for security updates. Your file may list fewer components.

5. On rocky-lab, the license is a field in each RPM package's header.

```
# on rocky-lab
rpm -qi bash | grep -E '^(Name|License)'
rpm -q --qf '%{NAME}: %{LICENSE}\n' openssh zlib
rpm -qa --qf '%{LICENSE}\n' | sort | uniq -c | sort -rn | head -5
```

```
Name        : bash
License     : GPLv3+
openssh: BSD
zlib: zlib and Boost
...
```

What just happened: `rpm -qi` prints a package's information block, including **License**. `--qf` (query format) prints only chosen fields. The last command lists the license of every installed package, then `sort | uniq -c | sort -rn` counts each license and sorts by frequency. You will see GPL variants (copyleft) near the top, along with permissive licenses such as MIT and BSD.

Checkpoint: You can name the display manager, window manager, and display server layers with an example of each, and find a package's license on both systems (`/usr/share/doc/<pkg>/copyright` on Ubuntu, `rpm -qi <pkg>` on Rocky).

Challenge

CHALLENGE

1. On ubuntu-lab, use a one-time edit at the GRUB menu (press `e`) to boot once with `systemd.unit=multi-user.target` added to the `linux` line. After logging in, prove it with `cat /proc/cmdline`, then reboot and prove it is gone.
2. Write a UEFI version of the PXE menu in `~/labs/1.1/pxe/grub.cfg` using `menuentry`, `linux`, and `initrd` lines that boot the same kernel and initrd as your `pxelinux.cfg/default`.
3. Find three packages on rocky-lab whose license is not GPL and describe whether each license is permissive or copyleft.

Clean up

1. On ubuntu-lab, confirm the original GRUB settings are back, delete the backup, and rebuild the configuration one last time to be safe.

```
sudo diff /etc/default/grub /etc/default/grub.lab-backup && echo "identical"
sudo rm /etc/default/grub.lab-backup
sudo update-grub
```

2. Reboot ubuntu-lab and confirm `labtest` is no longer on the kernel command line.

```
sudo reboot
```

```
grep -c labtest /proc/cmdline
```

   The second command, run after you log in again, should print `0`.

3. On rocky-lab, confirm no entry still has the test parameter.

```
# on rocky-lab
sudo grubby --info=ALL | grep -c labtest
```

   It should print `0`. If not, run `sudo grubby --update-kernel=ALL --remove-args="labtest=1"`.

4. Remove the lab working directory on ubuntu-lab. The `file` package you installed is small and harmless; leave it.

```
rm -rf ~/labs/1.1
```

Exam corner

- Boot order: firmware (BIOS/UEFI) then bootloader (GRUB 2) then kernel then initrd/initramfs then systemd (PID 1) then the default target.
- Edit `/etc/default/grub`, never `grub.cfg`. Rebuild with `update-grub` or `grub-mkconfig -o /boot/grub/grub.cfg` (Debian/Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (RHEL/Rocky). `grubby` changes kernel parameters on RHEL-family systems.
- `GRUB_CMDLINE_LINUX` applies to all entries; `GRUB_CMDLINE_LINUX_DEFAULT` to normal boots only. `GRUB_TIMEOUT` and `GRUB_DEFAULT` control the menu.
- The running kernel's parameters are in `/proc/cmdline`. Press `e` at the GRUB menu for a one-time change.
- The initrd/initramfs holds drivers needed to mount `/`. Ubuntu: `update-initramfs`, `lsinitramfs`. Rocky: `dracut`, `lsinitrd`.
- PXE = network boot: DHCP gives an address and the boot file name, TFTP delivers the bootloader, then the kernel and initrd.
- FHS: `/etc` config, `/var` variable data and logs, `/proc` virtual kernel and process info, `/dev` device files, `/boot` kernel and initrd, `/usr` installed software, `/tmp` temporary (sticky bit), `/home` user files, `/bin` and `/sbin` and `/lib` essential commands and libraries.
- x86_64 = AMD64 = `amd64` (Debian). AArch64 = ARM64 = `arm64` (Debian). RISC-V is an open instruction set. `uname -m` shows the architecture.
- dpkg-based: `.deb`, `dpkg`, `apt` (Debian, Ubuntu). RPM-based: `.rpm`, `rpm`, `dnf` / `yum` / `zypper` (RHEL, Rocky, Fedora, SUSE).
- Display manager = graphical login (GDM, SDDM, LightDM). Window manager = places and decorates windows. X Server (Xorg) and Wayland are display servers; Wayland is the modern default.
- Copyleft (GPL) requires derivative works to keep the same license. Permissive (MIT, BSD, Apache) does not. Proprietary = closed source, restricted redistribution. Free software = the four freedoms.

Review questions

1. An administrator added `nomodeset` to `GRUB_CMDLINE_LINUX_DEFAULT` in `/etc/default/grub` on an Ubuntu server and rebooted, but `/proc/cmdline` does not show it. What step was missed?
   A. Running `update-initramfs -u`
   B. Running `update-grub`
   C. Editing `/proc/cmdline`
   D. Running `systemctl daemon-reload`

2. Which part of the boot process contains the drivers needed to mount the real root filesystem?
   A. The display manager
   B. `/etc/fstab`
   C. The initrd (initramfs)
   D. The UEFI system partition

3. A server must be installed automatically with no local media. The network card asks a DHCP server for a boot file name and then downloads it. Which technology is this, and which file transfer protocol does it typically use first?

4. Which directory contains files that are generated by the kernel and use no disk space, such as `cpuinfo`?
   A. `/dev`
   B. `/var`
   C. `/proc`
   D. `/sys/firmware`

5. `uname -m` prints `aarch64` on a cloud server. Which Debian architecture name do its packages use?
   A. `amd64`
   B. `arm64`
   C. `i386`
   D. `riscv64`

6. A company modifies a GPL-licensed program and sells it to customers. What does the copyleft clause require?
   A. Nothing, because the GPL is a permissive license
   B. They must release the modified source code under the GPL to those customers
   C. They must pay a royalty to the original author
   D. They must convert the program to a proprietary license

Lab 1.2 Manage Devices, Kernel Modules, and the initrd

Objective: 1.2 Summarize Linux device management concepts and tools.
Domain: 1.0 System Management (23% of the exam)
Time: about 70 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes. This lab loads kernel modules and rebuilds the initrd on both VMs.

In this lab you load, configure, block, and unload a kernel module, take a full hardware inventory with every device tool on the exam list, and rebuild the initrd on Ubuntu and Rocky. You also try the server-hardware tools (IPMI, sensors, GPU monitoring) and see what they report inside a VM. On the job, these are the commands you reach for when a new disk, network card, or GPU "does not show up."

What you'll practice

- Kernel modules: `depmod`, `insmod`, `lsmod`, `modinfo`, `modprobe`, `rmmod`
- Device management: `dmesg`, `dmidecode`, `ipmitool`, `lm_sensors`, `lscpu`, `lshw`, `lsmem`, `lspci`, `lsusb`
- initrd management: `dracut`, `mkinitrd`
- Custom hardware: embedded systems; GPU use cases (`nvtop`)

How it works

Kernel modules. The Linux kernel needs a driver for every piece of hardware and many features (filesystems, firewalls, virtual network devices). Building all of them into the kernel would make it huge. Instead, most drivers are **kernel modules**: separate files that the kernel can load into memory when needed and unload when not. Think of the kernel as a game console and modules as cartridges you plug in.

Modules for each kernel version live in `/lib/modules/<version>/`. On Ubuntu 24.04 they end in `.ko.zst` (a kernel object compressed with zstd); on Rocky 9 they end in `.ko.xz`. Many modules need other modules first. For example, a filesystem module may need a checksum module. The file `modules.dep` in the same folder lists these dependencies.

```
/lib/modules/6.8.0-45-generic/
  kernel/drivers/net/dummy.ko.zst     <- module files
  modules.dep                         <- "who needs whom", built by depmod
  modules.alias                       <- hardware IDs -> module names

modprobe dummy  --> reads modules.dep + /etc/modprobe.d/*.conf
                --> loads dependencies first, then dummy (with options)
insmod FILE     --> loads exactly one file, no dependencies, no config
rmmod dummy     --> unloads one module
modprobe -r     --> unloads a module and its unused dependencies
```

The commands:

Command	Job
`lsmod`	List loaded modules, their size, and which modules use them.
`modinfo`	Show a module's file, description, license, dependencies, and options (parameters).
`modprobe`	The smart way to load (`modprobe NAME`) or unload (`modprobe -r NAME`) a module. Handles dependencies and reads configuration from `/etc/modprobe.d/`.
`insmod`	The low-level way to load one module **file** by its full path. No dependency handling.
`rmmod`	The low-level way to unload one module.
`depmod`	Rebuilds `modules.dep` and the other index files. Run it after adding a module file by hand. Package installs run it for you.

Most of the time you never load modules yourself: when the kernel or `udev` (the device manager) sees new hardware, it looks up the hardware ID in `modules.alias` and runs `modprobe` automatically. Files in `/etc/modprobe.d/` can set options (`options NAME key=value`), stop automatic loading (`blacklist NAME`), or stop all loading (`install NAME /bin/false`). Files in `/etc/modules-load.d/` list modules to load at every boot.

Device inspection tools. Linux gives you a tool for each kind of hardware question:

Tool	Answers
`lscpu`	What CPU, how many cores, which architecture?
`lsmem`	How is memory laid out, how much is online?
`lspci`	What is on the PCI bus (disk controllers, network and graphics cards)? `-k` shows the driver in use.
`lsusb`	What USB devices are attached?
`lshw`	Everything, as a tree (package `lshw`).
`dmidecode`	What does the firmware (DMI/SMBIOS table) say: vendor, model, serial number, memory slots?
`dmesg`	What did the kernel say while detecting hardware?
`lm_sensors` (`sensors`)	What are the temperatures, voltages, and fan speeds?
`ipmitool`	Talk to the server's BMC (Baseboard Management Controller) over IPMI: power on/off remotely, read hardware sensors and logs.
`nvtop`	How busy are the GPUs (like `top` for graphics cards)?

The initrd. As you saw in Lab 1.1, the initrd (initramfs) is a small archive loaded with the kernel. It carries the modules needed to reach the root filesystem (disk controller, LVM, RAID, encryption drivers). If you change hardware or need a new driver early in boot, you must rebuild it. Ubuntu uses **initramfs-tools** and the command `update-initramfs`. Red Hat-family systems use **dracut**. **mkinitrd** is the old Red Hat tool that dracut replaced; where it still exists it is a thin wrapper that calls dracut.

Custom hardware. An **embedded system** is a computer built into a device to do one job: a router, a smart TV, a car dashboard, an industrial controller. They often use ARM or RISC-V CPUs, little memory, flash storage, and a stripped-down Linux built with tools like Yocto or Buildroot, often with BusyBox instead of the full GNU tools. Hardware on ARM boards is described by a **device tree** file instead of being discovered over PCI. A **GPU** (graphics processing unit) has thousands of small cores and is used on servers for AI and machine learning training and inference, video transcoding, scientific computing (HPC), and virtual desktops. GPUs need a driver (often a proprietary kernel module such as `nvidia`) and monitoring tools such as `nvtop` or `nvidia-smi`.

Tasks

Task 1: List and inspect kernel modules

1. Create the lab directory on ubuntu-lab.

```
mkdir -p ~/labs/1.2 && cd ~/labs/1.2
```

2. List the loaded modules. There are many, so look at the first few lines and count them.

```
lsmod | head -6
lsmod | wc -l
```

```
Module                  Size  Used by
tls                   155648  0
isofs                  61440  0
binfmt_misc            28672  1
intel_rapl_msr         20480  0
psmouse               217088  0
84
```

> **What just happened:** `lsmod` reads `/proc/modules` and prints it as a table. **Module** is the module name, **Size** is the memory it uses in bytes, and **Used by** is a count followed by the names of modules that depend on it. A module with a count above 0 cannot be unloaded until its users are gone. Your list will differ depending on your virtualization software.

3. Inspect the `dummy` module. It creates fake network interfaces, so it is safe to experiment with.

```
modinfo dummy
```

```
filename:       /lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
alias:          rtnl-link-dummy
description:    Dummy netdevice driver which discards all packets sent to it
license:        GPL
srcversion:     6D4E8F1A0B2C3D4E5F6A7B8
depends:
retpoline:      Y
intree:         Y
name:           dummy
vermagic:       6.8.0-45-generic SMP preempt mod_unload modversions
sig_id:         PKCS#7
...
parm:           numdummies:Number of dummy pseudo devices (int)
```

What just happened: `modinfo` reads information stored inside the module file. The key fields are: **filename** (where the file is), **alias** (other names that trigger it; `rtnl-link-dummy` means "load me when someone asks for a dummy link"), **license**, **depends** (empty: no other modules needed), **vermagic** (the kernel version it was built for; it must match the running kernel), and **parm** (options you can set, here `numdummies`, an integer). The `sig_...` and `signature` lines (shortened to `...` above) are the module's digital signature, which Ubuntu uses so that Secure Boot systems load only trusted modules.

TIP

If `modinfo dummy` says `Module dummy not found`, your kernel package does not include the extra drivers. Install them with `sudo apt install -y linux-modules-extra-$(uname -r)` and try again.

4. Ask for single fields. `-F` prints one field, and `-n` prints only the file name.

```
modinfo -F license dummy
modinfo -n dummy
modinfo -F depends btrfs
modinfo ext4 | head -2
```

```
GPL
/lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
raid6_pq,xor,libcrc32c
name:           ext4
filename:       (builtin)
```

What just happened: The `btrfs` filesystem module depends on three other modules: `raid6_pq` and `xor` (RAID math) and `libcrc32c` (checksums). That is why `modprobe` exists: it loads those three first. The last command shows that `ext4` is different on Ubuntu: its file name is `(builtin)`, which means it is compiled into the kernel itself. A built-in driver is always present, never appears in `lsmod`, and cannot be unloaded. (On Rocky 9, `ext4` is a loadable module.)

Checkpoint: `lsmod | grep -c dummy` prints `0` (not loaded yet), and `modinfo -F parm dummy` prints the `numdummies` option.

Task 2: Load and unload modules with modprobe, insmod, and rmmod

1. Load `dummy` with `modprobe` and check that it worked.

```
sudo modprobe dummy
lsmod | grep dummy
ip -br link show type dummy
```

```
dummy                  12288  0
dummy0           DOWN           3a:9f:21:c4:07:5e <BROADCAST,NOARP>
```

What just happened: `modprobe dummy` found the module through `modules.dep` and loaded it. `modprobe` prints nothing when it succeeds. The module created one fake interface, `dummy0`. `ip -br link show type dummy` lists network links (`-br` means brief output) of type dummy.

2. Unload it with `modprobe -r`, then load it again with an option.

```
sudo modprobe -r dummy
sudo modprobe dummy numdummies=3
ip -br link show type dummy
```

```
dummy0           DOWN           3a:9f:21:c4:07:5e <BROADCAST,NOARP>
dummy1           DOWN           d2:41:8c:5b:e0:13 <BROADCAST,NOARP>
dummy2           DOWN           7e:02:9a:61:3f:c8 <BROADCAST,NOARP>
```

What just happened: Options (module parameters) are written as `name=value` after the module name. `numdummies=3` told the driver to create three interfaces. Options only take effect when a module is loaded, which is why you had to unload it first.

3. Unload it with the low-level `rmmod`, then load it with the low-level `insmod`. `insmod` needs the full path to the file, which `modinfo -n` gives you.

```
sudo rmmod dummy
sudo insmod $(modinfo -n dummy)
lsmod | grep dummy
sudo rmmod dummy
```

What just happened: `rmmod dummy` unloaded the module by name. `$(modinfo -n dummy)` expanded to the full path, for example `/lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst`, and `insmod` loaded that exact file. `insmod` does not look up dependencies and does not read `/etc/modprobe.d/`. For a module with dependencies, such as `btrfs`, `insmod` would fail with `Unknown symbol in module` unless you loaded every dependency first by hand. That is why administrators use `modprobe` and only use `insmod` for testing a module file that is not installed in `/lib/modules`.

4. See what `modprobe` would do for a module with dependencies, without loading anything.

```
modprobe --show-depends vxlan
```

```
insmod /lib/modules/6.8.0-45-generic/kernel/net/ipv4/udp_tunnel.ko.zst
insmod /lib/modules/6.8.0-45-generic/kernel/net/ipv6/ip6_udp_tunnel.ko.zst
insmod /lib/modules/6.8.0-45-generic/kernel/drivers/net/vxlan/vxlan.ko.zst
```

What just happened: `--show-depends` prints the `insmod` commands `modprobe` would run, in order: dependencies first, then the module. This is literally what `modprobe` does for you. Lines starting with `builtin` mean that piece is compiled into the kernel and needs no loading.

Checkpoint: `lsmod | grep -c dummy` prints `0` again, and you can explain why `modprobe` is preferred over `insmod` (dependencies and configuration).

Task 3: Configure modules with /etc/modprobe.d and depmod

1. Make an option permanent. Files in `/etc/modprobe.d/` must end in `.conf`. This lab uses a **heredoc** with `sudo tee`: everything between `<<'EOF'` and the line `EOF` is sent to `tee`, which writes it to the file. You need `tee` because a plain `>` redirect would run without root rights.

```
cat <<'EOF' | sudo tee /etc/modprobe.d/lab-dummy.conf
# Lab 1.2: always create two dummy interfaces
options dummy numdummies=2
EOF
sudo modprobe dummy
ip -br link show type dummy
sudo modprobe -r dummy
```

```
# Lab 1.2: always create two dummy interfaces
options dummy numdummies=2
dummy0           DOWN           3a:9f:21:c4:07:5e <BROADCAST,NOARP>
dummy1           DOWN           d2:41:8c:5b:e0:13 <BROADCAST,NOARP>
```

What just happened: `tee` echoed the file contents to the screen as it wrote them. When `modprobe` loaded `dummy`, it read the `options` line and passed `numdummies=2` automatically. This is how you tune drivers permanently, for example `options kvm_intel nested=1`.

2. Watch automatic loading happen. Asking for a dummy-type interface makes the kernel load the module by its alias.

```
sudo ip link add lab0 type dummy
lsmod | grep dummy
sudo ip link delete lab0
sudo modprobe -r dummy
```

What just happened: You never ran `modprobe`, yet `dummy` appears in `lsmod`. The kernel saw a request for link type `dummy`, asked `modprobe` for `rtnl-link-dummy` (the alias you saw in `modinfo`), and the module was loaded. Hardware drivers load the same way when `udev` sees a new device's ID.

3. Blacklist the module and try the same thing again.

```
echo "blacklist dummy" | sudo tee /etc/modprobe.d/lab-blacklist.conf
sudo ip link add lab0 type dummy
```

```
blacklist dummy
Error: Unknown device type.
```

What just happened: `blacklist dummy` tells `modprobe` to ignore the module's aliases, so automatic loading no longer works and `ip` cannot create the interface. Administrators blacklist drivers that misbehave or conflict, for example blacklisting `nouveau` before installing the proprietary NVIDIA driver.

4. A blacklist does **not** stop someone from loading the module by its real name. Prove it, then block it completely.

```
sudo modprobe dummy && lsmod | grep dummy
sudo modprobe -r dummy
echo "install dummy /bin/false" | sudo tee -a /etc/modprobe.d/lab-blacklist.conf
sudo modprobe dummy
```

```
dummy                  12288  0
install dummy /bin/false
modprobe: ERROR: ../libkmod/libkmod-module.c:1047 command_do() Error running install
command '/bin/false' for module dummy: retcode 1
modprobe: ERROR: could not insert 'dummy': Invalid argument
```

What just happened: The explicit `modprobe dummy` still worked with only a blacklist. `tee -a` appends (`-a`) a second line. An `install` line replaces the loading step with a command of your choice; `/bin/false` always fails, so the module can never be loaded through `modprobe`. Security guides use this to disable unneeded filesystems and protocols (for example `install usb-storage /bin/false` to block USB disks).

5. Remove the blacklist so the module works again.

```
sudo rm /etc/modprobe.d/lab-blacklist.conf
sudo modprobe dummy && sudo modprobe -r dummy && echo "dummy loads again"
```

6. Look at the dependency database and rebuild it with `depmod`.

```
ls -l /lib/modules/$(uname -r)/modules.dep
grep -E '/btrfs\.ko' /lib/modules/$(uname -r)/modules.dep
sudo depmod -a
ls -l /lib/modules/$(uname -r)/modules.dep
```

```
-rw-r--r-- 1 root root 412345 Oct 10 09:11 /lib/modules/6.8.0-45-generic/modules.dep
kernel/fs/btrfs/btrfs.ko.zst: kernel/crypto/xor.ko.zst kernel/lib/raid6/raid6_pq.ko.zst
kernel/lib/libcrc32c.ko.zst
-rw-r--r-- 1 root root 412345 Oct 10 10:02 /lib/modules/6.8.0-45-generic/modules.dep
```

What just happened: Each line of `modules.dep` is `module: dependencies`. `depmod -a` scanned every module under `/lib/modules/$(uname -r)` and rewrote this file and the related index files (`modules.alias`, `modules.symbols`, and binary `.bin` versions). The timestamp changed, the content did not, because nothing was added. You run `depmod` after copying a module into `/lib/modules/<version>/` yourself (for example a vendor driver in the `updates/` or `extra/` folder); otherwise `modprobe` cannot find it. On Rocky, all these commands work exactly the same way.

Checkpoint: `ls /etc/modprobe.d/lab-*` shows only `lab-dummy.conf`, and `sudo modprobe dummy && ip -br link show type dummy` shows two interfaces. Unload it again with `sudo modprobe -r dummy`.

Task 4: Take a hardware inventory

Start by checking which inventory tools are already installed. `command -v` prints the path of each command it finds and stays silent for the ones it does not.

```
command -v lscpu lsmem lspci lsusb lshw dmidecode
```

```
/usr/bin/lscpu
/usr/bin/lsmem
/usr/bin/lspci
/usr/bin/lsusb
/usr/sbin/dmidecode
```

What just happened: `lscpu` and `lsmem` come from `util-linux`, which every Linux system has. `lspci`, `lsusb`, `lshw`, and `dmidecode` are separate packages, and a minimal install may lack some of them (in this sample, `lshw` is missing). The first step below installs whatever is missing; `apt` simply reports "already the newest version" for the rest.

1. Install the tools that are not always present. `pciutils` provides `lspci`, `usbutils` provides `lsusb`, and `util-linux` (always installed) provides `lscpu` and `lsmem`.

```
sudo apt update
sudo apt install -y pciutils usbutils lshw dmidecode
```

2. CPU and memory.

```
lscpu | grep -E '^(Architecture|CPU\(s\)|Model name|Thread|Core|Socket|Hypervisor)'
lsmem
```

```
Architecture:                         x86_64
CPU(s):                               2
Model name:                           Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz
Thread(s) per core:                   1
Core(s) per socket:                   2
Socket(s):                            1
Hypervisor vendor:                    KVM
RANGE                                  SIZE  STATE REMOVABLE  BLOCK
0x0000000000000000-0x00000000dfffffff  3.5G online       yes   0-27
0x0000000100000000-0x000000011fffffff  512M online       yes  32-35

Memory block size:               128M
Total online memory:               4G
Total offline memory:              0B
```

What just happened: `lscpu` shows the CPU model and topology: CPUs = sockets x cores x threads. "Hypervisor vendor" proves you are in a VM. `lsmem` shows memory as ranges of physical addresses split into blocks (128 MB each here). **STATE** `online` means the kernel is using it; on large servers, memory can be taken offline or hot-added, which is what **REMOVABLE** refers to.

3. PCI and USB devices.

```
lspci
lspci -k | grep -A3 -i ethernet
lsusb
```

```
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:02.0 VGA compatible controller: VMware SVGA II Adapter
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
        Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
        Kernel driver in use: e1000
        Kernel modules: e1000
Bus 001 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
```

What just happened: `lspci` lists every device on the PCI bus. The first column is the bus address (`bus:device.function`). `-k` adds the **kernel driver in use** and the **kernel modules** that could drive it: this is the link between hardware and the modules from Task 2. If a network card shows no "Kernel driver in use" line, its driver is missing. `lsusb` lists USB devices with their vendor:product ID (`80ee` is VirtualBox's vendor ID). If your VM has no USB controller, `lsusb` prints nothing. Your device names depend on your hypervisor (VirtualBox, VMware, KVM, or Hyper-V).

4. The full tree with `lshw`, and the firmware's view with `dmidecode`. Both need root to see everything.

```
sudo lshw -short | head -15
sudo lshw -class disk -class network -short
sudo dmidecode -s system-manufacturer
sudo dmidecode -s system-product-name
sudo dmidecode -t memory | grep -E '^\s*(Size|Type|Speed):' | head -6
```

```
H/W path        Device     Class          Description
=====================================================
                           system         VirtualBox
/0                         bus            VirtualBox
/0/0                       memory         128KiB BIOS
/0/1                       memory         4GiB System memory
/0/2                       processor      Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz
/0/100/3        enp0s3     network        82540EM Gigabit Ethernet Controller
/0/100/d/0      /dev/sda   disk           26GB VBOX HARDDISK
/0/100/d/1      /dev/sdb   disk           5368MB VBOX HARDDISK
...
innotek GmbH
VirtualBox
        Size: 4 GB
        Type: Other
```

What just happened: `lshw -short` prints one line per device in a hardware tree; `-class` filters by type (disk, network, memory, processor, display). It is the quickest way to match a device name like `enp0s3` or `/dev/sdb` to the physical hardware. `dmidecode` decodes the DMI (also called SMBIOS) table that the firmware provides. `-s` asks for one string (use it to get a serial number with `-s system-serial-number` for an asset inventory), and `-t` asks for one type, such as `memory`, `bios`, or `system`. In a VM the "hardware" is the hypervisor's virtual hardware, so you see VirtualBox, VMware, or QEMU.

5. Read the kernel's hardware messages with `dmesg`.

```
sudo dmesg -T | grep -E 'sd[a-e]\]|e1000|virtio' | head -8
sudo dmesg --level=err,warn | tail -5
```

```
[Fri Oct 10 09:20:03 2026] sd 2:0:0:0: [sda] 52428800 512-byte logical blocks: (26.8 GB/25.0
GiB)
[Fri Oct 10 09:20:03 2026] sd 3:0:0:0: [sdb] 10485760 512-byte logical blocks: (5.37 GB/5.00
GiB)
[Fri Oct 10 09:20:03 2026] e1000 0000:00:03.0 eth0: Intel(R) PRO/1000 Network Connection
[Fri Oct 10 09:20:03 2026] e1000 0000:00:03.0 enp0s3: renamed from eth0
```

What just happened: `dmesg` prints the kernel ring buffer, a memory log of kernel messages since boot. `-T` converts the "seconds since boot" stamps into readable times. `--level=err,warn` shows only errors and warnings, a fast first step when a device misbehaves. You can see each disk being detected with its size, and the network driver naming the card `eth0` before `udev` renamed it to a predictable name (`enp0s3`). Run `sudo dmesg -w` to watch new messages live (for example, while plugging in a USB device), and press **Ctrl+C** to stop.

Checkpoint: Using only the tools from this task, you can answer: How many CPUs? (`lscpu`) How much memory is online? (`lsmem`) Which kernel module drives your network card? (`lspci -k`) What model is the "server"? (`dmidecode -s system-product-name`) What size is `/dev/sdb` ? (`lshw -class disk -short` or `dmesg`)

Task 5: Try the server and GPU hardware tools

TIP

Explore/simulate: Virtual machines have no temperature sensors, no BMC for IPMI, and no GPU. You will install each tool, run it, and read the error, which is exactly what you would see on a real server with a missing driver or device. Then you will see the commands you would use on real hardware.

1. Install the three tools. On Ubuntu the sensor package is called `lm-sensors` ; on Rocky it is `lm_sensors` .

```
sudo apt install -y lm-sensors ipmitool nvtop
```

2. Detect sensors and read them.

```
sudo sensors-detect --auto | tail -5
sensors
```

```
Sorry, no sensors were detected.
Either your system has no sensors, or they are not supported, or
they are connected to an I2C or SMBus adapter that is not
supported. If you find out what chips are on your board, check
https://hwmon.wiki.kernel.org/device_support_status for driver status.
No sensors found!
Make sure you loaded all the kernel drivers you need.
Try sensors-detect to find out which these are.
```

What just happened: `lm_sensors` (the project behind the `sensors` command) reads hardware monitoring chips. `sensors-detect --auto` probes for those chips, accepts all the default answers (`tail -5` keeps only its last five lines), and tells you which kernel modules to load (for example `coretemp` for Intel CPU temperatures). A VM has no such chips, so nothing is found. If you do see an entry such as `acpitz-acpi-0`, your hypervisor exposes a virtual sensor; that is fine. On a physical server, `sensors` prints lines such as `Core 0: +45.0°C (high = +80.0°C, crit = +100.0°C)` and fan speeds in RPM.

3. Try to talk to a BMC with `ipmitool`.

```
sudo ipmitool sdr list
```

```
Could not open device at /dev/ipmi0 or /dev/ipmi/0 or /dev/ipmidev/0: No such file or
directory
```

What just happened: IPMI (Intelligent Platform Management Interface) is a standard for the BMC, a tiny separate computer on server motherboards (sold as Dell iDRAC, HPE iLO, Lenovo XClarity). It works even when the server is powered off or the OS has crashed. Locally, `ipmitool` talks to it through the device file `/dev/ipmi0`, which the `ipmi_si` and `ipmi_devintf` modules create when a BMC exists. No BMC means no device file.

4. Write down the commands you would use on a real server. Save them as a reference file. This does not run them.

```
cat <<'EOF' > ~/labs/1.2/ipmi-cheatsheet.txt
# Local BMC (needs: sudo modprobe ipmi_si ipmi_devintf)
sudo ipmitool sdr list                # all sensors: temps, fans, voltages
sudo ipmitool sel list                # System Event Log (hardware errors)
sudo ipmitool lan print 1             # BMC network settings
# Remote BMC over the network (IPMI v2.0 = lanplus interface)
ipmitool -I lanplus -H 10.0.0.50 -U admin -P 'secret' chassis power status
ipmitool -I lanplus -H 10.0.0.50 -U admin -P 'secret' chassis power cycle
ipmitool -I lanplus -H 10.0.0.50 -U admin -P 'secret' sol activate   # serial console
EOF
cat ~/labs/1.2/ipmi-cheatsheet.txt
```

What just happened: `-I lanplus` selects the IPMI 2.0 network interface, `-H` is the BMC's IP address, and `-U` / `-P` are its user name and password. `chassis power` turns a server on, off, or power-cycles it remotely. `sol activate` (Serial over LAN) gives you the server's text console over the network, so you can watch it boot from home.

5. Look for a GPU and try `nvtop`.

```
lspci | grep -i -E 'vga|3d|display'
nvtop
```

```
00:02.0 VGA compatible controller: VMware SVGA II Adapter
No GPU to monitor.
```

What just happened: The VM has only a basic virtual display adapter, not a compute GPU. `nvtop` ("Neat Videocard TOP") monitors NVIDIA, AMD, and Intel GPUs: on a real GPU server it shows a live graph of GPU utilization, GPU memory use, temperature, power, and which processes use the GPU. A data center GPU shows up in `lspci` as `3D controller: NVIDIA Corporation ...`, and `lspci -k` would show the driver module (`nvidia` or the open-source `nouveau` or `amdgpu`).

Checkpoint: `command -v sensors ipmitool nvtop` prints three paths, and you can explain why each one reports "nothing found" in a VM.

Task 6: Rebuild the initrd on Ubuntu with update-initramfs

1. Check that `dummy` is not currently in the initrd. `lsinitramfs` lists the files inside.

```
sudo lsinitramfs /boot/initrd.img-$(uname -r) | grep -c 'dummy.ko'
```

```
0
```

2. Back up the current initrd, ask initramfs-tools to include the module, and rebuild. `/etc/initramfs-tools/modules` lists extra modules, one per line.

```
sudo cp /boot/initrd.img-$(uname -r) /boot/initrd.img-$(uname -r).lab-backup
echo "dummy" | sudo tee -a /etc/initramfs-tools/modules
sudo update-initramfs -u -k $(uname -r)
```

```
dummy
update-initramfs: Generating /boot/initrd.img-6.8.0-45-generic
```

What just happened: `update-initramfs -u` updates (`-u`) an existing initrd, and `-k $(uname -r)` picks the running kernel's one (without `-k`, it updates the newest installed kernel, which may not be the one you booted). It works by re-running all the scripts in `/etc/initramfs-tools/` and `/usr/share/initramfs-tools/`. Other options: `-k all` rebuilds for every installed kernel, and `-c` creates a new one (used when a kernel is first installed). The package manager runs this automatically after installing a kernel or a driver package, which is why you rarely run it by hand.

3. Verify the module is now inside.

```
sudo lsinitramfs /boot/initrd.img-$(uname -r) | grep 'dummy.ko'
```

```
usr/lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
```

4. Remove the line you added and rebuild so the initrd is back to normal. `sed -i '/^dummy$/d'` deletes lines that contain exactly `dummy`.

```
sudo sed -i '/^dummy$/d' /etc/initramfs-tools/modules
sudo update-initramfs -u -k $(uname -r)
sudo lsinitramfs /boot/initrd.img-$(uname -r) | grep -c 'dummy.ko'
```

What just happened: After the rebuild the count is `0` again. The backup copy you made is your safety net: if a rebuilt initrd ever stops a system from booting, you can pick an older kernel in the GRUB menu or boot rescue media and copy the backup back.

Checkpoint: `grep -c '^dummy$' /etc/initramfs-tools/modules` prints `0`, and `ls /boot/*.lab-backup` shows your backup.

Task 7: Rebuild the initrd on Rocky with dracut (and meet mkinitrd)

1. On rocky-lab, back up the current initramfs and list the dracut modules it contains.

```
# on rocky-lab
sudo cp /boot/initramfs-$(uname -r).img /boot/initramfs-$(uname -r).img.lab-backup
sudo lsinitrd -m | head -12
sudo lsinitrd | grep -c 'dummy.ko'
```

What just happened: dracut builds an initramfs from **dracut modules** (not the same as kernel modules): small script bundles such as `lvm`, `crypt`, `nfs`, `systemd`, and `kernel-modules`. By default dracut runs in "host-only" mode and includes only what this machine needs to boot, so the image is small. The count of `dummy.ko` is `0`.

2. Tell dracut to add the `dummy` kernel driver, then rebuild. Configuration goes in `/etc/dracut.conf.d/` in files ending with `.conf`. The spaces inside the quotes matter: dracut joins lists, and the spaces stop names from running together.

```
# on rocky-lab
echo 'add_drivers+=" dummy "' | sudo tee /etc/dracut.conf.d/lab.conf
sudo dracut -f
sudo lsinitrd | grep 'dummy.ko'
```

```
add_drivers+=" dummy "
-rw-r--r--   1 root  root  2412 Apr 30 12:00 usr/lib/modules/5.14.0-427.13.1.el9_4.x86_64/
kernel/drivers/net/dummy.ko.xz
```

What just happened: `dracut` with no file name rebuilds `/boot/initramfs-$(uname -r).img`. `-f` (force) allows it to overwrite the existing file; without `-f` it refuses. You could also build an image for another kernel with `sudo dracut -f /boot/initramfs-VERSION.img VERSION`, add `-v` for verbose output, or add a driver just once with `--add-drivers dummy`. `lsinitrd` confirms the driver is inside.

3. Check for `mkinitrd`, the legacy tool.

```
# on rocky-lab
command -v mkinitrd && rpm -qf "$(command -v mkinitrd)" || echo "mkinitrd is not installed"
```

What just happened: `mkinitrd` was the Red Hat initrd builder before RHEL 6. Today, where it exists, it is a compatibility script shipped with dracut that translates the old syntax `mkinitrd /boot/initramfs-VERSION.img VERSION` into a `dracut` command. Depending on your Rocky release you will see either a path owned by the `dracut` package or the message that it is not installed. Either way, on the exam remember: **dracut** is the modern RHEL tool, **mkinitrd** is the legacy one, and Debian/Ubuntu use `update-initramfs` (mkinitramfs underneath).

4. Remove the lab configuration and rebuild to return to normal.

```
# on rocky-lab
sudo rm /etc/dracut.conf.d/lab.conf
sudo dracut -f
sudo lsinitrd | grep -c 'dummy.ko'
```

Checkpoint: On rocky-lab, `ls /etc/dracut.conf.d/` does not show `lab.conf`, `sudo lsinitrd | grep -c dummy.ko` prints `0`, and the backup file exists in `/boot`.

Task 8: Explore embedded-system traits

TIP

Explore/simulate: You do not need a Raspberry Pi for this task. You will look for the things that make embedded Linux different and see that your x86 VM lacks them, and you will use BusyBox, the toolbox most embedded systems are built on.

1. Look for a device tree. ARM and RISC-V boards describe their hardware in a device tree that the bootloader passes to the kernel; PCs discover hardware with ACPI and PCI instead.

```
ls /proc/device-tree 2>/dev/null || echo "No device tree: ACPI/PCI (PC-style) system"
ls /sys/class/gpio 2>/dev/null || echo "No GPIO pins exposed"
```

What just happened: On a Raspberry Pi, `/proc/device-tree/model` would print something like `Raspberry Pi 4 Model B`, and `/sys/class/gpio` (or the newer `gpiod` tools) would let you control the GPIO (general-purpose input/output) pins that switch LEDs, relays, and sensors. Your VM has neither.

2. Use BusyBox. Ubuntu ships it for the initrd, and embedded systems use it as their whole command set.

```
command -v busybox || sudo apt install -y busybox-static
busybox | head -2
busybox --list | wc -l
busybox uptime
```

```
BusyBox v1.36.1 (Ubuntu 1:1.36.1-6ubuntu3.1) multi-call binary.
BusyBox is copyrighted by many authors between 1998-2015.
401
10:15:22 up  1:02,  1 users,  load average: 0.00, 0.01, 0.00
```

What just happened: The first line installs BusyBox only if no `busybox` command exists yet (`||` runs the install only when `command -v` fails). Ubuntu Server normally already has it. BusyBox is one small program that contains simplified versions of hundreds of commands (`ls` , `cp` , `sh` , `ip` , `vi`). On an embedded device, `/bin/ls` and friends are just links to `/bin/busybox` , which saves a lot of flash storage. Embedded devices are also usually built for ARM or RISC-V, run from read-only flash with a small writable area, and are updated by replacing the whole image rather than with `apt` .

Checkpoint: You can list three ways an embedded Linux system differs from your server VM (CPU architecture, device tree instead of PCI discovery, BusyBox and flash storage).

Challenge

CHALLENGE

1. Find which kernel module drives your VM's disk controller using `lspci -k` , then use `modinfo` to find its license and file path, and `lsmod` to see what depends on it. Explain why `sudo modprobe -r` on it would fail. (If `lspci -k` names a driver that `lsmod` does not list, that driver is built into the kernel, as `ata_piix` and `virtio_blk` are on Ubuntu; pick the network card's driver instead.)
2. Create `/etc/modules-load.d/lab.conf` so that `dummy` loads at every boot, reboot, prove it with `lsmod` , then remove the file and reboot again.
3. On rocky-lab, build a second initramfs file in your home directory for the running kernel with dracut without replacing the one in `/boot` , then list its contents with `lsinitrd` and delete it.

Clean up

1. On ubuntu-lab, unload the module and remove the lab configuration files.

```
sudo modprobe -r dummy
sudo rm -f /etc/modprobe.d/lab-dummy.conf /etc/modprobe.d/lab-blacklist.conf
sudo rm -f /etc/modules-load.d/lab.conf
```

2. Make sure the initramfs-tools module list no longer contains `dummy` , rebuild once more, and delete the backup.

```
sudo sed -i '/^dummy$/d' /etc/initramfs-tools/modules
sudo update-initramfs -u -k $(uname -r)
sudo rm -f /boot/initrd.img-$(uname -r).lab-backup
```

3. On rocky-lab, remove the dracut configuration, rebuild, and delete the backup.

```
# on rocky-lab
sudo modprobe -r dummy
sudo rm -f /etc/dracut.conf.d/lab.conf /etc/modules-load.d/lab.conf
sudo dracut -f
sudo rm -f /boot/initramfs-$(uname -r).img.lab-backup
```

4. Remove the lab directory on ubuntu-lab. The installed tools (`lshw`, `lm-sensors`, `ipmitool`, `nvtop`) are harmless and may be used by later labs, so you can leave them.

```
rm -rf ~/labs/1.2
```

Exam corner

- `lsmod` lists loaded modules (from `/proc/modules`). `modinfo` shows a module's file, license, dependencies, and parameters.
- `modprobe NAME` loads a module plus its dependencies and reads `/etc/modprobe.d/*.conf`; `modprobe -r NAME` unloads it.
- `insmod /full/path/file.ko` loads one file without dependencies; `rmmod NAME` unloads one module.
- `depmod -a` rebuilds `modules.dep` in `/lib/modules/$(uname -r)/`; run it after adding a module file by hand.
- In `/etc/modprobe.d/`: `options NAME key=value` sets parameters, `blacklist NAME` stops automatic (alias) loading, `install NAME /bin/false` stops all loading. `/etc/modules-load.d/` loads modules at boot.
- `lspci -k` shows which kernel driver each PCI device uses; `lsusb` lists USB devices; `lscpu` CPU; `lsmem` memory blocks; `lshw` everything (`-short`, `-class`).
- `dmidecode` reads the firmware DMI/SMBIOS table: manufacturer, model, serial number, memory slots. `dmesg` shows kernel (hardware detection) messages.
- `lm_sensors`: run `sensors-detect`, then `sensors` for temperatures, voltages, and fans. `ipmitool` talks to the BMC locally (`/dev/ipmi0`) or remotely (`-I lanplus -H host`).
- Ubuntu rebuilds the initrd with `update-initramfs -u` (`-k all` for all kernels). RHEL/Rocky uses `dracut -f`; `mkinitrd` is the legacy tool (now a dracut wrapper). `lsinitrd` and `lsinitramfs` list contents.
- Embedded systems: single-purpose devices, often ARM/RISC-V, BusyBox, device tree, flash storage. GPUs are used for AI/ML, HPC, video transcoding, and VDI; monitor them with `nvtop`.

Review questions

1. A new network card is not working. Which command shows whether a kernel driver is bound to it?
   A. `lsusb -v`
   B. `lspci -k`
   C. `lsmem`
   D. `dmidecode -t bios`

2. An administrator copied a vendor driver `acme.ko` into `/lib/modules/$(uname -r)/extra/`, but `modprobe acme` says the module is not found. What should be run first?
   A. `insmod acme`
   B. `depmod -a`
   C. `rmmod acme`
   D. `dracut --list-modules`

3. Which line in a file under `/etc/modprobe.d/` prevents a module from being loaded even when someone runs `modprobe NAME` directly?
   A. `blacklist NAME`
   B. `options NAME disable=1`
   C. `install NAME /bin/false`
   D. `alias NAME off`

4. Which command rebuilds the initramfs for the running kernel on Rocky Linux 9, overwriting the existing file?

5. A technician needs to power-cycle a hung physical server remotely through its BMC. Which tool is designed for this?
   A. `lm_sensors`
   B. `nvtop`
   C. `ipmitool`
   D. `lshw`

6. Which command shows the system's manufacturer, model, and serial number as reported by the firmware?
   A. `dmidecode`
   B. `lscpu`
   C. `dmesg`
   D. `modinfo`

Lab 1.3 Manage Storage

Objective: 1.3 Given a scenario, manage storage in a Linux system.
Domain: 1.0 System Management (23% of the exam)
Time: about 2.5 hours (you can stop after any task and continue later, as long as you do not reboot between Tasks 4 and 9)
VM: ubuntu-lab (Rocky optional in Task 9)
Snapshot first: Yes. This lab partitions, formats, and erases the four extra disks.

Storage is where most real administration happens: a database runs out of space, a new disk is added, a disk in a mirror dies, or a team needs a shared folder. In this lab you use the four blank 5 GB disks to practice every storage skill on the exam: partitioning, LVM, filesystems and their repair tools, mount options, RAID, network shares, automounting, and inodes.

What you'll practice

- Logical Volume Manager (LVM) logical volumes: `lvchange`, `lvcreate`, `lvdisplay`, `lvremove`, `lvresize` / `lvextend`, `lvs`
- LVM volume groups: `vgchange`, `vgcreate`, `vgdisplay`, `vgexport`, `vgextend`, `vgimport`, `vgremove`, `vgs`, `vgscan`
- LVM physical volumes: `pvcreate`, `pvdisplay`, `pvmove`, `pvremove`, `pvresize`, `pvs`, `pvscan`
- Partitions: `blkid`, `fdisk` / `gdisk`, `growpart`, `lsblk`, `parted`
- Filesystems: `xfs`, `ext4`, `btrfs`, `tmpfs`; utilities `df`, `du`, `fio`, `fsck`, `mkfs`, `resize2fs`, `xfs_growfs`, `xfs_repair`
- Redundant Array of Independent Disks (RAID): `/proc/mdstat`, `mdadm`
- Mounted storage: `/etc/fstab`, `/etc/mtab`, `/proc/mounts`, `autofs`, `mount`, `umount`; options `noatime`, `nodev`, `nodiratime`, `noexec`, `nofail`, `nosuid`, `remount`, `ro`, `rw`; network mounts: Network File System (NFS), Server Message Block (SMB) with Samba
- Inodes

How it works

Disks, partitions, and filesystems. A new disk is like an empty warehouse. **Partitioning** puts up walls to divide it into rooms (partitions). **Formatting** (`mkfs`, "make filesystem") puts shelves and a catalog into a room so files can be stored and found. **Mounting** attaches a formatted room to a folder in the directory tree, called the **mount point**, so you can reach it with a path like `/mnt/data`.

There are two partition table formats. **MBR** (Master Boot Record, also called DOS) is the old one: at most 4 primary partitions and disks up to 2 TiB. **GPT** (GUID Partition Table) is the modern one: 128 partitions, huge disks, and a backup copy of the table at the end of the disk. `fdisk` handles both (it started with MBR), `gdisk` is a GPT-focused

clone of fdisk, and `parted` handles both and can be scripted. `lsblk` shows disks and partitions as a tree, and `blkid` shows each one's filesystem type and **UUID** (a unique ID that never changes, unlike `/dev/sdb`, which can change if disks are added).

Filesystems you need to know:

Filesystem	Key facts
`ext4`	Debian/Ubuntu default. Journaling. Can grow online, shrink only offline. Tools: `mkfs.ext4`, `resize2fs`, `e2fsck` (via `fsck`).
`xfs`	RHEL/Rocky default. Fast with large files. Can grow online, **cannot shrink**. Tools: `mkfs.xfs`, `xfs_growfs`, `xfs_repair`.
`btrfs`	Copy-on-write with built-in snapshots, subvolumes, checksums, and multi-disk support. Default on openSUSE and Fedora desktops.
`tmpfs`	Lives in RAM (and swap). Very fast, gone at reboot. Used for `/run` and often `/tmp`.

Inodes. Every file has an **inode** (index node): a small record that stores the file's owner, permissions, size, timestamps, and where its data blocks are. The file **name** is not in the inode; it lives in the directory, which maps names to inode numbers. ext4 creates a fixed number of inodes when you format, so a disk can run out of inodes (lots of tiny files) while still having free space. `df -i` shows inode use; `ls -i` shows inode numbers.

LVM. The Logical Volume Manager adds a flexible layer between disks and filesystems. Picture pouring several jugs of water (disks) into one tank (a volume group), then filling bottles of any size from the tank (logical volumes). Later you can pour in another jug and make a bottle bigger without touching its contents.



- A **physical volume (PV)** is a disk or partition prepared for LVM with `pvcreate`.
- A **volume group (VG)** is a pool made of one or more PVs, created with `vgcreate`. Space in it is divided into **extents** (4 MiB chunks by default).
- A **logical volume (LV)** is a slice of the VG that acts like a partition. It appears as `/dev/VGNAME/LVNAME` (and `/dev/mapper/VGNAME-LVNAME`).

RAID combines disks for speed or safety. **RAID 0** stripes data across disks (fast, no protection). **RAID 1** mirrors two disks (either can die). **RAID 5** stripes with parity (needs 3+ disks, survives one failure). **RAID 6** has double parity (4+ disks, survives two). **RAID 10** is mirrors that are striped. Linux software RAID is managed with `mdadm` and creates devices like `/dev/md0`; its live status is in `/proc/mdstat`.

Mounting. `mount DEVICE DIR` attaches a filesystem and `umount DIR` detaches it. Mounts listed in **`/etc/fstab`** are mounted at boot. Each fstab line has six fields: device, mount point, type, options, dump flag, and fsck order. The kernel's live list of mounts is **`/proc/mounts`**; **`/etc/mtab`** used to be a separate file but is now a link to it. **Mount options** change behavior: `ro` / `rw` (read-only/read-write), `noexec` (programs cannot run from here), `nosuid` (setuid bits are ignored), `nodev` (device files do not work), `noatime` / `nodiratime` (do not update access times), `nofail` (do not stop the boot if the device is missing), and `remount` (change options on a mounted filesystem).

Network mounts. NFS (Network File System) is the Unix/Linux way to share a directory over the network: the server lists shares in `/etc/exports`. **SMB** (Server Message Block, also called CIFS) is the Windows way; **Samba** is the Linux server for it, and `mount -t cifs` is the client. **autofs** mounts shares automatically when someone opens the folder and unmounts them after they have been idle.

Tasks

Task 1: Find your extra disks and install the tools

1. Create the lab directory and list all block devices.

```
mkdir -p ~/labs/1.3 && cd ~/labs/1.3
lsblk
```

```
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   25G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0    2G  0 part /boot
└─sda3                      8:3    0   23G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0 11.5G  0 lvm  /
sdb                         8:16   0    5G  0 disk
sdc                         8:32   0    5G  0 disk
sdd                         8:48   0    5G  0 disk
sde                         8:64   0    5G  0 disk
sr0                        11:0    1 1024M  0 rom
```

What just happened: `lsblk` lists block devices as a tree. **TYPE** is `disk`, `part` (partition), `lvm` (logical volume), `raid1`, or `rom` (CD drive). **MOUNTPOINTS** shows where each is mounted. `sda` is the OS disk: notice Ubuntu itself already uses LVM (`ubuntu--vg-ubuntu--lv`). The four 5 GB disks with no children and no mount points are your lab disks.

> **CAUTION**
> This lab erases `/dev/sdb`, `/dev/sdc`, `/dev/sdd`, and `/dev/sde`. If your extra disks have other names (`vdb` .. `vde` or `nvme0n2` .. `nvme0n5`), replace the names in **every** command. Never run these commands against the disk that holds `/` (usually `sda`). Take a snapshot now.

2. Make sure the lab disks are really empty. `wipefs` without options only **lists** filesystem and partition signatures; it changes nothing.

```
sudo wipefs /dev/sdb /dev/sdc /dev/sdd /dev/sde
```

> **What just happened:** No output means no signatures: the disks are blank. If you see lines (for example from an earlier attempt at this lab), clear them with `sudo wipefs -a /dev/sdX` for each lab disk.

3. Install every package this lab uses. `mdadm` is installed with `--no-install-recommends` so that apt does not also pull in a mail server, which would stop to ask setup questions.

```
sudo apt update
sudo apt install -y lvm2 xfsprogs btrfs-progs gdisk parted cloud-guest-utils fio
sudo apt install -y --no-install-recommends mdadm
sudo apt install -y nfs-kernel-server nfs-common samba smbclient cifs-utils autofs
```

> **What just happened:** `lvm2` gives the LVM commands, `xfsprogs` and `btrfs-progs` give the XFS and btrfs tools, `gdisk` and `parted` are partitioning tools, `cloud-guest-utils` provides `growpart`, and `fio` is a disk benchmark. The last line installs the NFS server and client, the Samba server and client, the CIFS mount helper, and the automounter. Some of these are already installed on Ubuntu Server; apt simply skips them.

4. Create the mount points you will use. `-p` creates parents and ignores folders that already exist; the braces expand to one folder per name.

```
sudo mkdir -p /mnt/{part,data,logs,btrfs,ram,tiny,raid,nfs,smb}
```

> **Checkpoint:** `lsblk -d -o NAME,SIZE,TYPE` shows four 5G disks besides the OS disk, `command -v mdadm growpart fio gdisk` prints four paths, and `ls /mnt` shows the nine folders.

Task 2: Partition a disk with fdisk (MBR) and parted

1. Open `/dev/sdd` in `fdisk`. It is interactive: you type one-letter commands and nothing is written until you press `w`.

```
sudo fdisk /dev/sdd
```

2. Type these answers, one per line, pressing **Enter** after each. Lines in parentheses describe what fdisk asks.

```
o          (create a new empty DOS/MBR partition table)
n          (new partition)
p          (primary)
1          (partition number 1)
<Enter>    (first sector: accept the default 2048)
+1G        (last sector: make it 1 GiB)
n          (new partition)
p          (primary)
2          (partition number 2)
<Enter>    (first sector: default)
+512M      (size 512 MiB)
t          (change a partition type)
2          (partition 2)
82         (Linux swap)
p          (print the table)
w          (write and exit)
```

```
Disk /dev/sdd: 5 GiB, 5368709120 bytes, 10485760 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Disklabel type: dos
Disk identifier: 0x5b1c2d3e

Device     Boot   Start     End Sectors  Size Id Type
/dev/sdd1          2048 2099199 2097152    1G 83 Linux
/dev/sdd2       2099200 3147775 1048576  512M 82 Linux swap / Solaris

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

What just happened: `o` wrote a new, empty MBR table (in memory only). `n` created partitions; the first one starts at sector 2048 (1 MiB) so it lines up with the physical blocks of modern disks. `t` changed partition 2's **type ID** from `83` (Linux) to `82` (Linux swap). Type IDs are only labels; they tell tools what is meant to be there. `w` wrote the table to disk and told the kernel to re-read it. If you make a mistake before `w`, press `q` to quit without saving.

3. Look at the result with `lsblk` and with `parted`.

```
lsblk /dev/sdd
sudo parted /dev/sdd print
```

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sdd      8:48   0    5G  0 disk
├─sdd1   8:49   0    1G  0 part
└─sdd2   8:50   0  512M  0 part
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdd: 5369MB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type     File system  Flags
 1      1049kB  1075MB  1074MB  primary
 2      1075MB  1612MB  537MB   primary
```

What just happened: `parted` calls an MBR table `msdos`. By default it shows sizes in decimal megabytes (MB = 1,000,000 bytes), while fdisk's `1G` meant 1 GiB (1,073,741,824 bytes), which is why the numbers look different. The "File system" column is empty because you have not formatted anything yet.

4. `parted` can also do everything in one non-interactive command, which is handy in scripts. Replace the MBR table with a GPT table and create one partition. `-s` (script mode) means "do not ask questions."

```
sudo parted -s /dev/sdd mklabel gpt mkpart labpart ext4 1MiB 1GiB
sudo parted /dev/sdd unit MiB print free
```

```
Partition Table: gpt
Number  Start    End      Size     File system  Name     Flags
        0.02MiB  1.00MiB  0.98MiB  Free Space
 1      1.00MiB  1024MiB  1023MiB               labpart
        1024MiB  5120MiB  4096MiB  Free Space
```

What just happened: `mklabel gpt` replaced the whole partition table (the old partitions are gone), and `mkpart NAME FSTYPE START END` created a partition. The `ext4` word is only a hint; parted does not format. `unit MiB` displays sizes in MiB and `print free` also shows unused space. Unlike fdisk, parted writes each change **immediately**, with no `w` step, so be careful with it.

5. Erase the disk again so it is blank for later tasks.

```
sudo wipefs -a /dev/sdd
```

```
/dev/sdd: 8 bytes were erased at offset 0x00000200 (gpt): 45 46 49 20 50 41 52 54
/dev/sdd: 8 bytes were erased at offset 0x13ffffe00 (gpt): 45 46 49 20 50 41 52 54
/dev/sdd: 2 bytes were erased at offset 0x000001fe (PMBR): 55 aa
```

What just happened: `wipefs -a` erases all signatures it finds: here the primary GPT header, the backup GPT header at the end of the disk, and the "protective MBR" that GPT keeps at the start so old tools do not think the disk is empty.

Checkpoint: `lsblk /dev/sdd` shows a disk with no partitions. You can say which tool needs `w` to save (`fdisk` , `gdisk`) and which writes immediately (`parted`).

Task 3: Create a GPT partition with gdisk, then grow it with growpart

1. Open `/dev/sde` in `gdisk` and create one 1 GiB Linux partition.

```
sudo gdisk /dev/sde
```

Type these answers:

```
o          (create a new empty GPT; answer y to "Proceed?")
n          (new partition)
1          (partition number)
<Enter>    (first sector: default)
+1G        (last sector: 1 GiB in)
8300       (type code: Linux filesystem; Enter also picks it)
p          (print)
w          (write; answer y to "Do you want to proceed?")
```

```
Number  Start (sector)    End (sector)  Size       Code  Name
   1            2048         2099199   1024.0 MiB  8300  Linux filesystem

Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): y
OK; writing new GUID partition table (GPT) to /dev/sde.
The operation has completed successfully.
```

What just happened: `gdisk` works like `fdisk` but always creates GPT. Its type codes are four hex digits: `8300` Linux filesystem, `8200` Linux swap, `8e00` Linux LVM, `fd00` Linux RAID, `ef00` EFI system partition. Nothing was written until `w` .

2. Format the partition as ext4, mount it, and check its size. `mkfs.ext4` creates an ext4 filesystem; `df -h` shows mounted filesystem sizes in human units.

```
sudo mkfs.ext4 -L labpart /dev/sde1
sudo mount /dev/sde1 /mnt/part
df -h /mnt/part
```

```
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 9f1d6c1e-2b7a-4c3d-8e5f-0a1b2c3d4e5f
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376
Writing superblocks and filesystem accounting information: done

Filesystem      Size  Used Avail Use% Mounted on
/dev/sde1       974M   24K  907M   1% /mnt/part
```

What just happened: `-L labpart` gave the filesystem a label (a human-friendly name). `mke2fs` reports the block size (4k), the number of **inodes** created (65,536: the most files this filesystem can ever hold), and its UUID. Size shows a little under 1 GiB because the filesystem's own structures (inode tables, journal) use some space.

3. Show the identifiers with `blkid`.

```
sudo blkid /dev/sde1
```

```
/dev/sde1: LABEL="labpart" UUID="9f1d6c1e-2b7a-4c3d-8e5f-0a1b2c3d4e5f" BLOCK_SIZE="4096"
TYPE="ext4" PARTLABEL="Linux filesystem" PARTUUID="3c6a9e2b-7f41-4d8a-9b0c-1d2e3f4a5b6c"
```

What just happened: `blkid` prints the filesystem **UUID** and **LABEL** (stored inside the filesystem) and the GPT **PARTUUID** and **PARTLABEL** (stored in the partition table). You use the UUID in `/etc/fstab` later because it stays the same even if the disk becomes `/dev/sdf` one day.

4. Now imagine this is a cloud server whose disk was just enlarged in the provider's console: the partition must grow to fill the disk. `growpart DISK PARTNUMBER` grows a partition into the free space after it, even while it is mounted. Note the space between the disk and the number.

```
sudo growpart /dev/sde 1
lsblk /dev/sde
```

```
CHANGED: partition=1 start=2048 old: size=2097152 end=2099199 new: size=10483679
end=10485726
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sde      8:64   0   5G  0 disk
└─sde1   8:65   0   5G  0 part /mnt/part
```

5. The partition is bigger, but the filesystem inside it is not. Grow the ext4 filesystem with `resize2fs`, online.

```
df -h /mnt/part
sudo resize2fs /dev/sde1
df -h /mnt/part
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sde1       974M   24K  907M   1% /mnt/part
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/sde1 is mounted on /mnt/part; on-line resizing required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/sde1 is now 1310459 (4k) blocks long.

Filesystem      Size  Used Avail Use% Mounted on
/dev/sde1       4.9G   24K  4.6G   1% /mnt/part
```

What just happened: Growing storage is always two steps: grow the container (partition or LV), then grow the filesystem. `growpart` did the first, `resize2fs` the second. With no size given, `resize2fs` fills the whole partition. This exact sequence (`growpart /dev/sda 1` then `resize2fs /dev/sda1` or `xfs_growfs /`) is how you expand a cloud VM's root disk.

6. Unmount and erase the disk. `-a` on a whole disk also removes the partition table.

```
sudo umount /mnt/part
sudo wipefs -a /dev/sde1
sudo wipefs -a /dev/sde
```

Checkpoint: `lsblk /dev/sdd /dev/sde` shows two disks with no partitions, and you can describe the two steps to grow a filesystem on a partition (`growpart` , then `resize2fs` or `xfs_growfs`).

Task 4: Build LVM storage

1. Turn `/dev/sdb` and `/dev/sdc` into physical volumes, then list them.

```
sudo pvcreate /dev/sdb /dev/sdc
sudo pvs
sudo pvdisplay /dev/sdb
```

```
  Physical volume "/dev/sdb" successfully created.
  Physical volume "/dev/sdc" successfully created.
  PV         VG        Fmt  Attr PSize   PFree
  /dev/sda3  ubuntu-vg lvm2 a--  <23.00g 11.50g
  /dev/sdb             lvm2 ---    5.00g  5.00g
  /dev/sdc             lvm2 ---    5.00g  5.00g
  "/dev/sdb" is a new physical volume of "5.00 GiB"
  --- NEW Physical volume ---
  PV Name               /dev/sdb
  VG Name
  PV Size               5.00 GiB
  Allocatable           NO
  PE Size               0
  Total PE              0
  Free PE               0
  Allocated PE          0
  PV UUID               Xb3k2L-....
```

What just happened: `pvcreate` wrote an LVM label on each disk. You can use whole disks (as here) or partitions of type `8e00` / `8e`. `pvs` is the short, one-line-per-PV summary; `pvdisplay` is the long, detailed view. The new PVs have no **VG** yet, and **PE** (physical extent) counts are 0 until they join a group. `/dev/sda3` is Ubuntu's own PV.

2. Create a volume group named `labvg` from `/dev/sdb` only. You add `/dev/sdc` later to practice growing.

```
sudo vgcreate labvg /dev/sdb
sudo vgs
sudo vgdisplay labvg | grep -E 'VG Name|VG Size|PE Size|Total PE|Free  PE'
```

```
  Volume group "labvg" successfully created
  VG        #PV #LV #SN Attr   VSize   VFree
  labvg       1   0   0 wz--n-  <5.00g <5.00g
  ubuntu-vg   1   1   0 wz--n- <23.00g 11.50g
  VG Name               labvg
  VG Size               <5.00 GiB
  PE Size               4.00 MiB
  Total PE              1279
  Free  PE / Size       1279 / <5.00 GiB
```

What just happened: `vgs` shows the number of PVs, LVs, and snapshots (`#SN`) in each group, its size (**VSize**), and free space (**VFree**). In **Attr**, `w` means writable, `z` resizable, and `n` the normal allocation policy. `vgdisplay` shows the extent size (4 MiB) and how many extents exist. `<5.00g` means "slightly less than 5 GiB"; LVM metadata uses a little space.

3. Create two logical volumes. `-n` names the LV, `-L` gives its size, and the last argument is the VG.

```
sudo lvcreate -n data -L 1G labvg
sudo lvcreate -n logs -L 1G labvg
sudo lvs labvg
sudo lvdisplay labvg/data | grep -E 'LV Path|LV Size|LV Status'
```

```
  Logical volume "data" created.
  Logical volume "logs" created.
  LV   VG    Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  data labvg -wi-a----- 1.00g
  logs labvg -wi-a----- 1.00g
  LV Path                /dev/labvg/data
  LV Status              available
  LV Size                1.00 GiB
```

What just happened: Each LV is now a block device at `/dev/labvg/data` and `/dev/labvg/logs`. In the `lvs` **Attr** column, `w` means writable, `i` inherited allocation, and `a` **active** (usable). When an LV is mounted, a sixth letter `o` (open) appears. You could also size by extents (`-l 100`) or by percent (`-l 50%FREE`).

4. Put an ext4 filesystem on `data` and an XFS filesystem on `logs`, mount them, and save a test file on each.

```
sudo mkfs.ext4 -q /dev/labvg/data
sudo mkfs.xfs -q /dev/labvg/logs
sudo mount /dev/labvg/data /mnt/data
sudo mount /dev/labvg/logs /mnt/logs
echo "LVM test data" | sudo tee /mnt/data/hello.txt /mnt/logs/hello.txt
df -hT /mnt/data /mnt/logs
```

```
LVM test data
Filesystem             Type  Size  Used Avail Use% Mounted on
/dev/mapper/labvg-data ext4  974M   28K  907M   1% /mnt/data
/dev/mapper/labvg-logs xfs   960M   51M  909M   6% /mnt/logs
```

What just happened: `-q` (quiet) hides the long `mkfs` output. `tee` can write to several files at once. `df -hT` adds the filesystem **Type** column. Notice the device name: `/dev/labvg/data` is a link to `/dev/mapper/labvg-data`, the device-mapper name the kernel uses. Both names work everywhere.

Checkpoint: `sudo lvs labvg` shows `data` and `logs`, both with `o` (open) in their attributes, and `cat /mnt/data/hello.txt /mnt/logs/hello.txt` prints the test line twice.

Task 5: Grow, move, export, and import LVM storage

1. Add `/dev/sdc` to the volume group to make the pool bigger.

```
sudo vgextend labvg /dev/sdc
sudo vgs labvg
```

```
  Volume group "labvg" successfully extended
  VG    #PV #LV #SN Attr   VSize VFree
  labvg   2   2   0 wz--n- 9.99g 7.99g
```

2. Grow the ext4 LV by 512 MiB with `lvextend`, then grow its filesystem with `resize2fs`, without unmounting.

```
sudo lvextend -L +512M /dev/labvg/data
sudo resize2fs /dev/labvg/data
df -h /mnt/data
```

```
  Size of logical volume labvg/data changed from 1.00 GiB (256 extents) to 1.50 GiB (384
  extents).
  Logical volume labvg/data successfully resized.
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/labvg/data is mounted on /mnt/data; on-line resizing required
The filesystem on /dev/labvg/data is now 393216 (4k) blocks long.
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/labvg-data  1.5G   28K  1.4G   1% /mnt/data
```

What just happened: `-L +512M` means "add 512 MiB" (without the `+`, `-L 2G` means "make it exactly 2 GiB"). The LV grew first, then `resize2fs` stretched ext4 to fill it. Same two-step rule as with partitions.

3. Grow the XFS LV by 1 GiB. For XFS the filesystem tool is `xfs_growfs`, and it takes the **mount point**, not the device.

```
sudo lvextend -L +1G /dev/labvg/logs
sudo xfs_growfs /mnt/logs
df -h /mnt/logs
```

```
meta-data=/dev/mapper/labvg-logs isize=512    agcount=4, agsize=65536 blks
...
data blocks changed from 262144 to 524288
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/labvg-logs  2.0G   67M  1.9G   4% /mnt/logs
```

4. Now do both steps at once. `lvresize` can grow or shrink, and `-r` (`--resizefs`) makes it resize the filesystem too, whatever type it is.

```
sudo lvresize -r -L +512M /dev/labvg/data
sudo lvs labvg
```

```
  Size of logical volume labvg/data changed from 1.50 GiB (384 extents) to 2.00 GiB (512
  extents).
  Logical volume labvg/data successfully resized.
resize2fs 1.47.0 (5-Feb-2023)
The filesystem on /dev/mapper/labvg-data is now 524288 (4k) blocks long.
  LV   VG    Attr       LSize
  data labvg -wi-ao---- 2.00g
  logs labvg -wi-ao---- 2.00g
```

What just happened: `lvextend -r` and `lvresize -r` call the right filesystem tool for you (`resize2fs` for ext4, `xfs_growfs` for XFS). This is the safest habit, because you cannot forget the second step. Shrinking is different: XFS can never shrink, and ext4 can only shrink while unmounted, so plan sizes carefully and grow later.

ON THE EXAM

On the exam, if a filesystem "did not get bigger" after `lvextend`, the answer is the missing filesystem step: `resize2fs` (ext4) or `xfs_growfs` (XFS), or using `lvextend -r` in the first place.

5. See which disk each LV lives on. `-o +devices` adds a column; `pvs` shows the used space per PV.

```
sudo lvs -o +devices labvg
sudo pvs /dev/sdb /dev/sdc
```

```
  LV   VG    Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Devices
  data labvg -wi-ao---- 2.00g                                             /dev/sdb(0)
  data labvg -wi-ao---- 2.00g                                             /dev/sdb(512)
  data labvg -wi-ao---- 2.00g                                             /dev/sdb(896)
  logs labvg -wi-ao---- 2.00g                                             /dev/sdb(256)
  logs labvg -wi-ao---- 2.00g                                             /dev/sdb(640)
  PV         VG    Fmt  Attr PSize  PFree
  /dev/sdb   labvg lvm2 a--  <5.00g 1020.00m
  /dev/sdc   labvg lvm2 a--  <5.00g  <5.00g
```

What just happened: Both LVs are made of several pieces (segments), all on `/dev/sdb` because it still had room. The number in brackets is the starting extent. `/dev/sdc` is still empty.

6. Pretend `/dev/sdb` is failing and must be replaced. `pvmove` moves all data off a PV to other PVs in the same VG, **while the filesystems stay mounted and in use**.

```
sudo pvmove /dev/sdb
sudo pvs /dev/sdb /dev/sdc
cat /mnt/data/hello.txt
```

```
  /dev/sdb: Moved: 12.50%
  /dev/sdb: Moved: 50.00%
  /dev/sdb: Moved: 100.00%
  PV         VG    Fmt  Attr PSize  PFree
  /dev/sdb   labvg lvm2 a--  <5.00g <5.00g
  /dev/sdc   labvg lvm2 a--  <5.00g 1020.00m
LVM test data
```

What just happened: `pvmove` copied every extent from `/dev/sdb` to free extents on `/dev/sdc` and switched the LVs over, without downtime. It only works if the other PVs have enough free space. `/dev/sdb` is now completely free (PFree equals PSize).

7. Take `/dev/sdb` out of the VG and remove its LVM label, as you would before pulling the disk.

```
sudo vgreduce labvg /dev/sdb
sudo pvremove /dev/sdb
sudo pvs
```

```
  Removed "/dev/sdb" from volume group "labvg"
  Labels on physical volume "/dev/sdb" successfully wiped.
  PV         VG        Fmt  Attr PSize   PFree
  /dev/sda3  ubuntu-vg lvm2 a--  <23.00g 11.50g
  /dev/sdc   labvg     lvm2 a--   <5.00g 1020.00m
```

8. Run the scan and resize commands. `pvscan` and `vgscan` search all disks for LVM labels (useful after attaching disks from another server). `pvresize` updates a PV after its disk or partition was enlarged; here nothing changed, so it just reports success.

```
sudo pvscan
sudo vgscan
sudo pvresize /dev/sdc
```

```
  PV /dev/sdc    VG labvg       lvm2 [<5.00 GiB / 1020.00 MiB free]
  PV /dev/sda3   VG ubuntu-vg   lvm2 [<23.00 GiB / 11.50 GiB free]
  Total: 2 [<27.99 GiB] / in use: 2 [<27.99 GiB] / in no VG: 0 [0   ]
  Found volume group "labvg" using metadata type lvm2
  Found volume group "ubuntu-vg" using metadata type lvm2
  Physical volume "/dev/sdc" changed
  1 physical volume(s) resized or updated / 0 physical volume(s) not resized
```

What just happened: On a real server, after a cloud or SAN disk grows from 5 GB to 10 GB, you would run `pvresize /dev/sdc` and VFree would grow by 5 GB, ready for `lvextend -r`. If the PV is a partition, you run `growpart` first, then `pvresize`, then `lvextend -r`: three layers, three commands.

9. Unmount both LVs, then deactivate one LV with `lvchange -an` (`-a` activate, `n` no).

```
sudo umount /mnt/data /mnt/logs
sudo lvchange -an labvg/logs
sudo lvs labvg
ls /dev/labvg/
```

```
  LV   VG    Attr       LSize
  data labvg -wi-a----- 2.00g
  logs labvg -wi------- 2.00g
data
```

What just happened: An inactive LV keeps its data, but its device file disappears (only `data` is left in `/dev/labvg/`) and it cannot be mounted. The `a` in the Attr column is gone. Reactivate with `lvchange -ay`. `lvchange` can also set an LV read-only (`-p r`) or stop it from activating at boot.

10. Deactivate the whole VG with `vgchange -an`, then export it. Exporting marks the VG as "ready to move to another computer," so this server will not use it.

```
sudo vgchange -an labvg
sudo vgexport labvg
sudo pvs /dev/sdc
sudo vgs
```

```
  0 logical volume(s) in volume group "labvg" now active
  Volume group "labvg" successfully exported
  PV         VG    Fmt  Attr PSize  PFree
  /dev/sdc   labvg lvm2 ax-  <5.00g 1020.00m
  VG        #PV #LV #SN Attr   VSize   VFree
  labvg       1   2   0 wzx-n-  <5.00g 1020.00m
  ubuntu-vg   1   1   0 wz--n- <23.00g 11.50g
```

> **What just happened:** `vgexport` refuses to run while any LV is active, which is why you ran `vgchange -an` first. The PV attribute and the VG attribute (`wzx-n-`) now include `x` (exported). `vgs` still lists the group, but LVM refuses to activate or change it until it is imported. At this point you would shut down, move the disk to the new server, and run `vgimport` there. This prevents two servers from using the same VG name or the same disks by accident.

11. Import it back (as the "new server") and activate it.

```
sudo vgimport labvg
sudo vgchange -ay labvg
sudo lvs labvg
```

```
  Volume group "labvg" successfully imported
  2 logical volume(s) in volume group "labvg" now active
  LV   VG    Attr       LSize
  data labvg -wi-a----- 2.00g
  logs labvg -wi-a----- 2.00g
```

12. Mount them again and prove the data survived the trip.

```
sudo mount /dev/labvg/data /mnt/data
sudo mount /dev/labvg/logs /mnt/logs
cat /mnt/data/hello.txt /mnt/logs/hello.txt
```

> **Checkpoint:** `sudo vgs labvg` shows `#PV` = 1 and Attr `wz--n-` (no `x`), `sudo lvs labvg` shows both LVs at 2.00g with `ao` in their attributes, `/dev/sdb` no longer appears in `sudo pvs` , and both `hello.txt` files print `LVM test data` .

Task 6: Work with btrfs, tmpfs, inodes, and the filesystem utilities

1. Make a btrfs filesystem on the now-free `/dev/sdb` , mount it, and create a subvolume (an independent directory tree that can be snapshotted on its own).

```
sudo mkfs.btrfs -L labbtrfs /dev/sdb
sudo mount /dev/sdb /mnt/btrfs
sudo btrfs subvolume create /mnt/btrfs/projects
sudo btrfs subvolume list /mnt/btrfs
sudo btrfs filesystem usage -h /mnt/btrfs | head -6
```

```
btrfs-progs v6.6.3
Label:              labbtrfs
Filesystem size:    5.00GiB
...
Create subvolume '/mnt/btrfs/projects'
ID 256 gen 8 top level 5 path projects
Overall:
    Device size:                   5.00GiB
    Device allocated:            536.00MiB
    Device unallocated:            4.48GiB
    Used:                        144.00KiB
```

What just happened: `mkfs.btrfs` formatted the whole disk (no partition needed, just as with LVM). If it ever refuses because it finds an old signature, add `-f` to force it. A **subvolume** looks like a folder but can be mounted separately and snapshotted (`btrfs subvolume snapshot`). `btrfs filesystem usage` is the btrfs-aware way to check space, because btrfs allocates space in chunks and plain `df` can be misleading.

2. Mount a `tmpfs` (RAM disk) with a size limit.

```
sudo mount -t tmpfs -o size=100M tmpfs /mnt/ram
df -hT /mnt/ram
```

```
Filesystem     Type   Size  Used Avail Use% Mounted on
tmpfs          tmpfs  100M     0  100M   0% /mnt/ram
```

What just happened: `-t tmpfs` sets the filesystem type, and since there is no device, the word `tmpfs` is used as a placeholder source. `size=100M` is a limit, not a reservation: memory is only used as files are written. Everything in it vanishes at unmount or reboot. Run `df -hT | grep tmpfs` to see that `/run` and `/dev/shm` are tmpfs too.

3. Look at inodes. `ls -i` shows a file's inode number, `stat` shows the inode's contents, and a hard link (`ln` without `-s`) is just a second name for the same inode.

```
cd /mnt/data
sudo ln hello.txt hello-link.txt
ls -li
stat -c 'inode=%i links=%h size=%s owner=%U' hello.txt
df -i /mnt/data
cd ~/labs/1.3
```

```
total 24
12 -rw-r--r-- 2 root root    14 Oct 10 11:02 hello-link.txt
12 -rw-r--r-- 2 root root    14 Oct 10 11:02 hello.txt
11 drwx------ 2 root root 16384 Oct 10 11:01 lost+found
inode=12 links=2 size=14 owner=root
Filesystem              Inodes IUsed  IFree IUse% Mounted on
/dev/mapper/labvg-data  131072    13 131059    1% /mnt/data
```

What just happened: Both names show inode `12` and a link count of `2`: one file, two names. Deleting one name just lowers the count; the data is freed when the count reaches 0. `df -i` shows inode totals: **IUsed**, **IFree**, and **IUse%**. Note the total grew to 131,072 when you enlarged the LV, because ext4 adds inodes as it grows.

4. Run a filesystem out of inodes. Create a tiny tmpfs that allows only 100 inodes (`nr_inodes=100`) and is writable by everyone (`mode=1777`), then try to create 200 empty files.

```
sudo mount -t tmpfs -o size=50M,nr_inodes=100,mode=1777 tmpfs /mnt/tiny
for i in $(seq 1 200); do
  touch /mnt/tiny/f$i || { echo "Failed at file $i"; break; }
done
df -h /mnt/tiny
df -i /mnt/tiny
```

```
touch: cannot touch '/mnt/tiny/f100': No space left on device
Failed at file 100
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            50M     0   50M   0% /mnt/tiny
Filesystem     Inodes IUsed IFree IUse% Mounted on
tmpfs             100   100     0  100% /mnt/tiny
```

What just happened: The `for` loop runs `touch` for `f1` through `f200`. `||` runs the block in braces only if `touch` fails, and `break` stops the loop. The error says **No space left on device**, yet `df -h` shows 0% used. The real cause is in `df -i`: 100% of inodes are used (the root folder took one, so file 100 failed). This is a classic troubleshooting scenario: a mail spool or cache with millions of tiny files. The fix is to delete files or recreate the filesystem with more inodes (`mkfs.ext4 -N`). XFS and btrfs allocate inodes dynamically, so this mostly happens on ext4.

5. Check disk usage with `df` and `du`. `df` reports per filesystem; `du` adds up the sizes of files under a folder.

```
df -hT -x tmpfs -x squashfs
sudo du -sh /var/log
sudo du -h --max-depth=1 /var | sort -h | tail -5
```

What just happened: `-x TYPE` hides filesystem types you do not care about. `du -s` gives one summary total, and `--max-depth=1` gives one total per subfolder; `sort -h` sorts human-readable sizes so the biggest folders come last. Use `df` to find **which** filesystem is full and `du` to find **what** is filling it.

6. Check and repair filesystems. Both tools must run on an **unmounted** filesystem. `fsck` is a front end that calls the right checker (`e2fsck` for ext4). `-f` forces a full check even if the filesystem looks clean.

```
sudo umount /mnt/data
sudo fsck -f /dev/labvg/data
```

```
fsck from util-linux 2.39.3
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
/dev/mapper/labvg-data: 13/131072 files (0.0% non-contiguous), 26798/524288 blocks
```

What just happened: `e2fsck` checked inodes, directories, link counts (like the `2` from your hard link), and free-space totals. `13/131072 files` is inode usage. Add `-y` to answer "yes" to every repair question automatically. Never run `fsck` on a mounted read-write filesystem: it can cause the corruption it is trying to fix.

7. Check the XFS filesystem with `xfs_repair`. `-n` means "no modify": only report problems.

```
sudo umount /mnt/logs
sudo xfs_repair -n /dev/labvg/logs
sudo xfs_repair /dev/labvg/logs
```

```
Phase 1 - find and verify superblock...
Phase 2 - using internal log
...
Phase 7 - verify and correct link counts...
done
```

What just happened: `xfs_repair` runs seven phases. Running `fsck` on XFS does nothing useful (`fsck.xfs` just exits), so remember: **XFS uses `xfs_repair`**. If it complains about a dirty log, mount and unmount the filesystem once to replay the log; `-L` (zero the log) is a last resort that can lose recent changes.

8. Mount both again and measure performance with `fio` (flexible I/O tester).

```
sudo mount /dev/labvg/data /mnt/data
sudo mount /dev/labvg/logs /mnt/logs
sudo fio --name=labtest --directory=/mnt/data --rw=randrw --bs=4k --size=256M \
  --ioengine=libaio --direct=1 --runtime=15 --time_based --group_reporting
sudo rm -f /mnt/data/labtest.*
```

```
labtest: (groupid=0, jobs=1): err= 0: pid=4242: Fri Oct 10 11:20:01 2026
  read: IOPS=2154, BW=8617KiB/s (8824kB/s)(126MiB/15001msec)
  write: IOPS=2149, BW=8597KiB/s (8803kB/s)(126MiB/15001msec)
...
Run status group 0 (all jobs):
   READ: bw=8617KiB/s (8824kB/s), io=126MiB, run=15001-15001msec
  WRITE: bw=8597KiB/s (8803kB/s), io=126MiB, run=15001-15001msec
```

What just happened: `fio` created a 256 MB test file and did random reads and writes (`--rw=randrw`) in 4 KiB blocks (`--bs=4k`) for 15 seconds. `--direct=1` bypasses the page cache so you measure the disk, not RAM, and `--ioengine=libaio` uses Linux asynchronous I/O. **IOPS** (I/O operations per second) matters for databases; **BW** (bandwidth) matters for big files. Try `--rw=read --bs=1M` for sequential throughput. Your numbers depend entirely on your host's disk.

Checkpoint: `findmnt -t btrfs,tmpfs,ext4,xfs -o TARGET,FSTYPE | grep /mnt` lists `/mnt/data` (ext4), `/mnt/logs` (xfs), `/mnt/btrfs` (btrfs), and `/mnt/ram` and `/mnt/tiny` (tmpfs). `df -i /mnt/tiny` shows 100% IUse.

Task 7: Use mount options, /etc/fstab, and the mount tables

Start by looking at the options `/mnt/data` received when you mounted it without asking for any. `findmnt` shows mounted filesystems; `-o` picks the columns.

```
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS /mnt/data
```

```
TARGET    SOURCE                 FSTYPE OPTIONS
/mnt/data /dev/mapper/labvg-data ext4   rw,relatime
```

What just happened: A plain `mount` uses the defaults: `rw` (read-write), `relatime` (update access times only occasionally), and, not shown because they are the default, `suid` , `dev` , and `exec` . In the steps below you switch each protection on with `remount` , watch what it blocks, and then return to the defaults.

1. Test `noexec` . Create a tiny script on `/mnt/data` , run it, then remount with `noexec` and run it again. `-o remount,OPTION` changes options without unmounting.

```
printf '#!/bin/bash\necho "script ran"\n' | sudo tee /mnt/data/hi.sh
sudo chmod +x /mnt/data/hi.sh
/mnt/data/hi.sh
sudo mount -o remount,noexec /mnt/data
/mnt/data/hi.sh
sudo mount -o remount,exec /mnt/data
```

```
script ran
-bash: /mnt/data/hi.sh: Permission denied
```

What just happened: `printf` writes the text with `\n` turned into new lines. With `noexec` , the kernel refuses to run any program stored on that filesystem, even though the file has execute permission. Administrators mount `/tmp` , `/var/tmp` , and `/dev/shm` with `noexec` so that malware downloaded there cannot be run directly. (`bash /mnt/data/hi.sh` still works, because then `bash` is the program and it lives elsewhere.)

2. Test `ro` and `rw` .

```
sudo mount -o remount,ro /mnt/data
sudo touch /mnt/data/newfile
sudo mount -o remount,rw /mnt/data
sudo touch /mnt/data/newfile && echo "write OK"
```

```
touch: cannot touch '/mnt/data/newfile': Read-only file system
write OK
```

What just happened: Even root cannot write to a filesystem mounted `ro`. The kernel also switches a filesystem to read-only by itself when it detects errors, which is why "Read-only file system" in a log often means "run fsck." `mount -o remount,rw /` is a common step in rescue mode.

3. Test `nosuid`. A **setuid** program runs as its owner instead of as the person who started it. Copy `whoami` (which prints the effective user), make the copy setuid root, and compare.

```
sudo cp /usr/bin/whoami /mnt/data/whoami-suid
sudo chmod u+s /mnt/data/whoami-suid
/mnt/data/whoami-suid
sudo mount -o remount,nosuid /mnt/data
/mnt/data/whoami-suid
sudo rm /mnt/data/whoami-suid
```

```
root
student
```

What just happened: `chmod u+s` set the setuid bit, so the copy ran as its owner, `root`, even though you are `student`. That is dangerous on a removable disk or shared folder: anyone could bring their own setuid-root program. With `nosuid`, the kernel ignores the bit and the program runs as `student`. You deleted the copy right away because a setuid-root binary is a security risk.

4. Test `nodev`. Create a device file that points to the same kernel device as `/dev/null` (character device, major 1, minor 3) and write to it.

```
sudo mknod /mnt/data/mynull c 1 3
sudo chmod 666 /mnt/data/mynull
echo test > /mnt/data/mynull && echo "device works"
sudo mount -o remount,nodev /mnt/data
echo test > /mnt/data/mynull
sudo rm /mnt/data/mynull
```

```
device works
-bash: /mnt/data/mynull: Permission denied
```

> **What just happened:** `mknod NAME c MAJOR MINOR` creates a character device file. Without `nodev`, a device file anywhere gives access to that device, so someone could put a copy of `/dev/sda` on a USB stick and read your disk. `nodev` blocks device files on that filesystem. Use `nodev`, `nosuid`, and `noexec` together on removable media, `/tmp`, and home directories.

5. Test `noatime`. Every read normally updates a file's **access time** (atime). Ubuntu mounts with `relatime` by default, which updates it only if it is older than the last modification or more than a day old. Set an old access time, read the file, and compare.

```
sudo touch -a -d '2 days ago' /mnt/data/hello.txt
stat -c 'atime: %x' /mnt/data/hello.txt
cat /mnt/data/hello.txt > /dev/null
stat -c 'atime: %x' /mnt/data/hello.txt
sudo mount -o remount,noatime /mnt/data
sudo touch -a -d '2 days ago' /mnt/data/hello.txt
cat /mnt/data/hello.txt > /dev/null
stat -c 'atime: %x' /mnt/data/hello.txt
```

```
atime: 2026-10-08 11:30:12.000000000 +0000
atime: 2026-10-10 11:30:15.123456789 +0000
atime: 2026-10-08 11:30:20.000000000 +0000
```

> **What just happened:** `touch -a -d` set only the access time to two days ago. With `relatime`, reading the file updated the atime to now. With `noatime`, the read did not change it. Skipping these updates saves disk writes on busy servers. `nodiratime` does the same for directories only; `noatime` already includes it.

6. Check the current options, then unmount and remount to return to defaults.

```
findmnt -o TARGET,OPTIONS /mnt/data
sudo umount /mnt/data
sudo mount /dev/labvg/data /mnt/data
findmnt -o TARGET,OPTIONS /mnt/data
```

```
TARGET    OPTIONS
/mnt/data rw,nosuid,nodev,noatime
TARGET    OPTIONS
/mnt/data rw,relatime
```

7. Compare `/etc/mtab` and `/proc/mounts`.

```
ls -l /etc/mtab
grep labvg /proc/mounts
```

```
lrwxrwxrwx 1 root root 19 Apr 22 13:08 /etc/mtab -> ../proc/self/mounts
/dev/mapper/labvg-logs /mnt/logs xfs rw,relatime,attr2,inode64,logbufs=8,... 0 0
/dev/mapper/labvg-data /mnt/data ext4 rw,relatime 0 0
```

What just happened: `/proc/mounts` is the kernel's live list of mounted filesystems, in the same six-field format as fstab. `/etc/mtab` was once a file that `mount` updated by hand (and could get out of sync); today it is a symbolic link to the kernel's list, so both always agree. `/etc/fstab` is different: it is what **should** be mounted at boot.

CAUTION

The next steps edit `/etc/fstab`. A mistake in this file can stop the VM from booting normally. Make sure you took the snapshot at the start of the lab, keep the backup copy you make below, and use `tee -a` (append), never `tee` alone, which would replace the whole file.

8. Make the LV mounts permanent in `/etc/fstab`. Back up the file first. The commands use `blkid -s UUID -o value` to print only the UUID, so you do not have to copy it by hand.

```
sudo cp /etc/fstab /etc/fstab.lab-backup
DATA_UUID=$(sudo blkid -s UUID -o value /dev/labvg/data)
LOGS_UUID=$(sudo blkid -s UUID -o value /dev/labvg/logs)
echo "UUID=$DATA_UUID /mnt/data ext4 defaults,noatime,nofail 0 2" | sudo tee -a /etc/fstab
echo "UUID=$LOGS_UUID /mnt/logs xfs  defaults,nofail 0 0" | sudo tee -a /etc/fstab
```

```
UUID=4c1e6a2f-... /mnt/data ext4 defaults,noatime,nofail 0 2
UUID=8d2b7f3a-... /mnt/logs xfs  defaults,nofail 0 0
```

What just happened: The six fields are: **device** (`UUID=...`), **mount point**, **type**, **options** (`defaults` means `rw,suid,dev,exec,auto,nouser,async`, then `noatime` and `nofail` change some of those), **dump** (`0`, legacy backup flag), and **fsck pass** (`1` for `/`, `2` for other ext4 filesystems checked at boot, `0` to skip; XFS always uses `0`). Shell variables like `DATA_UUID` hold the UUIDs; `$DATA_UUID` inserts the value. `tee -a` appends instead of overwriting, which matters a lot here.

9. Test the new lines without rebooting. A broken fstab line can drop the system into emergency mode at boot, so always test.

```
sudo umount /mnt/data /mnt/logs
sudo systemctl daemon-reload
sudo findmnt --verify
sudo mount -a
findmnt /mnt/data /mnt/logs
```

```
Success, no errors or warnings detected
TARGET    SOURCE                 FSTYPE OPTIONS
/mnt/data /dev/mapper/labvg-data ext4   rw,noatime
/mnt/logs /dev/mapper/labvg-logs xfs    rw,relatime,attr2,inode64,logbufs=8,...
```

> **What just happened:** `systemctl daemon-reload` makes systemd re-read fstab (systemd turns each line into a `.mount` unit). `findmnt --verify` checks the file for errors. `mount -a` mounts everything in fstab that is not mounted yet; if it prints nothing, every line worked. `findmnt` with mount points shows they came back with the fstab options (`noatime` on `/mnt/data`).

10. See what `nofail` does. Add a line for a disk that does not exist, with `nofail` .

```
sudo mkdir -p /mnt/missing
echo "UUID=00000000-1111-2222-3333-444444444444 /mnt/missing ext4 defaults,nofail 0 0" \
  | sudo tee -a /etc/fstab
sudo systemctl daemon-reload
sudo mount -a && echo "mount -a finished without errors"
sudo sed -i '/\/mnt\/missing/d' /etc/fstab
sudo systemctl daemon-reload
```

> **What just happened:** `mount -a` quietly skipped the missing device because of `nofail` . Without `nofail` , `mount -a` would print an error, and at boot systemd would wait about 90 seconds for the device and then drop to emergency mode, which is a disaster on a remote server. Use `nofail` for removable, network, and non-essential disks. The `sed` command deleted every line containing `/mnt/missing` (the slashes are escaped with `\` because `/` is sed's separator).

> **Checkpoint:** `grep -c labvg /proc/mounts` prints `2` , `grep -c missing /etc/fstab` prints `0` , and `sudo findmnt --verify` reports no errors.

Task 8: Build and break a RAID 1 mirror with mdadm

1. Create a RAID 1 array named `/dev/md0` from `/dev/sdd` and `/dev/sde` . When mdadm warns that the array may not be suitable as a boot device and asks `Continue creating array?` , type `y` .

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdd /dev/sde
```

```
mdadm: Note: this array has metadata at the start and
    may not be suitable as a boot device.  If you plan to
    store '/boot' on this device please ensure that
    your boot-loader understands md/v1.x metadata, or use
    --metadata=0.90
mdadm: size set to 5237760K
Continue creating array? y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

2. Watch the two disks synchronize. `/proc/mdstat` is the kernel's live RAID status. `watch -n 1` reruns a command every second; press **Ctrl+C** when the progress bar is gone.

```
watch -n 1 cat /proc/mdstat
```

```
Personalities : [raid1] [linear] [raid0] [raid6] [raid5] [raid4] [raid10]
md0 : active raid1 sde[1] sdd[0]
      5237760 blocks super 1.2 [2/2] [UU]
      [=======>.............]  resync = 38.4% (2011648/5237760) finish=0.3min
speed=168000K/sec
```

What just happened: `md0 : active raid1` names the array and level, followed by its members with their slot numbers. `[2/2]` means 2 devices wanted, 2 working, and `[UU]` shows each member's state (`U` = up, `_` = missing or failed). The resync copies one disk onto the other so they start identical. You can use the array during the resync.

3. Format the array, mount it, and save a file.

```
sudo mkfs.ext4 -q /dev/md0
sudo mount /dev/md0 /mnt/raid
echo "mirrored data" | sudo tee /mnt/raid/important.txt
sudo mdadm --detail /dev/md0 | grep -E 'Raid Level|State :|Active Devices|/dev/sd'
```

```
mirrored data
        Raid Level : raid1
             State : clean
    Active Devices : 2
       0       8       48        0      active sync   /dev/sdd
       1       8       64        1      active sync   /dev/sde
```

4. Simulate a disk failure: mark `/dev/sde` as failed, check the status, and prove the data is still readable.

```
sudo mdadm /dev/md0 --fail /dev/sde
cat /proc/mdstat
cat /mnt/raid/important.txt
```

```
mdadm: set /dev/sde faulty in /dev/md0
md0 : active raid1 sde[1](F) sdd[0]
      5237760 blocks super 1.2 [2/1] [U_]
mirrored data
```

What just happened: `(F)` marks the failed member, `[2/1]` means only one of two devices works, and `[U_]` shows the second slot is down. The array is **degraded** but your file is still there, served from `/dev/sdd`. On a real server, `mdadm --monitor` (the `mdmonitor` service) would email you now.

5. Replace the disk: remove the failed member and add it back (on real hardware you would add the new disk). Watch it rebuild.

```
sudo mdadm /dev/md0 --remove /dev/sde
sudo mdadm /dev/md0 --add /dev/sde
watch -n 1 cat /proc/mdstat
```

```
mdadm: hot removed /dev/sde from md0
mdadm: added /dev/sde
md0 : active raid1 sde[2] sdd[0]
      5237760 blocks super 1.2 [2/1] [U_]
      [====>.............]  recovery = 22.0% (1152000/5237760) finish=0.4min
```

What just happened: `--remove` takes the failed disk out of the array, and `--add` adds a disk as a spare, which immediately starts **recovery** (rebuilding the mirror). When it finishes, `/proc/mdstat` shows `[2/2] [UU]` again. To make an array start automatically at boot you would save its definition with `sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf` and run `sudo update-initramfs -u`; this lab skips that because you remove the array in Clean up.

Checkpoint: After recovery finishes, `cat /proc/mdstat` shows `[2/2] [UU]`, and `cat /mnt/raid/important.txt` prints `mirrored data`.

Task 9: Share storage over the network with NFS, Samba, and autofs

1. Create an NFS share. You will export it to ubuntu-lab itself (`127.0.0.1`) and to the lab network. Back up `/etc/exports`, create the folder, and give it to `student`.

```
sudo cp /etc/exports /etc/exports.lab-backup
sudo mkdir -p /srv/nfs/share
sudo chown student:student /srv/nfs/share
OPTS="rw,sync,no_subtree_check"
echo "/srv/nfs/share 127.0.0.1($OPTS) 192.168.100.0/24($OPTS)" | sudo tee -a /etc/exports
sudo exportfs -ra
sudo exportfs -v
```

```
/srv/nfs/share
127.0.0.1(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,root_squash,...)
/srv/nfs/share  192.168.100.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,...)
```

What just happened: `OPTS` is a shell variable that holds the option list so the line stays short; `$OPTS` inserts it. Each `/etc/exports` line is `DIRECTORY CLIENT(OPTIONS) CLIENT(OPTIONS)...`. **There must be no space between a client and its parentheses**; a space would export to everyone with default options. `rw` allows writing, `sync` makes the server save data to disk before replying, and `no_subtree_check` avoids a slow and unreliable check. `exportfs -ra` re-reads the file and applies it, and `exportfs -v` shows the active exports with all default options, including `root_squash`. Change `192.168.100.0/24` to your lab network if it differs.

2. Mount the NFS share as a client and test it.

```
showmount -e 127.0.0.1
sudo mount -t nfs 127.0.0.1:/srv/nfs/share /mnt/nfs
findmnt /mnt/nfs
touch /mnt/nfs/from-student.txt && ls -l /mnt/nfs
sudo touch /mnt/nfs/from-root.txt
```

```
Export list for 127.0.0.1:
/srv/nfs/share 192.168.100.0/24,127.0.0.1
TARGET   SOURCE                  FSTYPE OPTIONS
/mnt/nfs 127.0.0.1:/srv/nfs/share nfs4   rw,relatime,vers=4.2,rsize=524288,...
-rw-rw-r-- 1 student student 0 Oct 10 11:45 from-student.txt
touch: cannot touch '/mnt/nfs/from-root.txt': Permission denied
```

What just happened: `showmount -e` asks the server what it exports. The NFS source is written `SERVER:/path`. The client negotiated NFS version 4.2 (type `nfs4`). `student` could write because the folder belongs to `student` (NFS checks user ID numbers, so UIDs should match on client and server). **root** could not write: with `root_squash` (the default), root on the client is mapped to the powerless user `nobody` on the server, so a client's root cannot take over the server's files. An fstab line for this share would be `ubuntu-lab:/srv/nfs/share /mnt/nfs nfs defaults,_netdev,nofail 0 0`.

TIP

Optional, from rocky-lab: `sudo dnf install -y nfs-utils`, then `sudo mount -t nfs ubuntu-lab:/srv/nfs/share /mnt` and `ls /mnt`. Unmount with `sudo umount /mnt` when done. Ubuntu's firewall (`ufw`) is inactive by default, so no port needs opening; on a server with a firewall you would allow NFS (TCP port 2049).

3. Create an SMB share with Samba. Back up the configuration, create the folder, and add a share definition at the end of `/etc/samba/smb.conf`.

```
sudo cp /etc/samba/smb.conf /etc/samba/smb.conf.lab-backup
sudo mkdir -p /srv/samba/lab
sudo chown student:student /srv/samba/lab
cat <<'EOF' | sudo tee -a /etc/samba/smb.conf
[labshare]
   path = /srv/samba/lab
   read only = no
   valid users = student
EOF
testparm -s 2>/dev/null | grep -A4 '\[labshare\]'
```

What just happened: This uses a **heredoc**: the lines between `<<'EOF'` and `EOF` are sent to `sudo tee -a`, which appends them to the file with root rights. A Samba share is a section in brackets (the share name) followed by settings. `testparm` checks `smb.conf` for errors and prints the result; `-s` skips the "press Enter" prompt.

4. Samba keeps its own password list, separate from Linux passwords. Give `student` a Samba password (use something simple, such as `Lab12345`, and type it twice), then restart Samba.

```
sudo smbpasswd -a student
sudo systemctl restart smbd
smbclient -L //127.0.0.1 -U student
```

```
New SMB password:
Retype new SMB password:
Added user student.
Password for [WORKGROUP\student]:

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        labshare        Disk
        IPC$            IPC       IPC Service (ubuntu-lab server (Samba, Ubuntu))
```

5. Mount the share with the CIFS client. `uid` and `gid` make the mounted files appear to belong to you. Enter the Samba password when asked.

```
sudo mount -t cifs //127.0.0.1/labshare /mnt/smb \
  -o username=student,uid=$(id -u),gid=$(id -g)
touch /mnt/smb/hello-from-smb.txt
ls -l /srv/samba/lab
findmnt -o TARGET,SOURCE,FSTYPE /mnt/smb
```

```
Password for student@//127.0.0.1/labshare:
-rw-r--r-- 1 student student 0 Oct 10 11:52 hello-from-smb.txt
TARGET   SOURCE                FSTYPE
/mnt/smb //127.0.0.1/labshare  cifs
```

What just happened: SMB paths are written `//SERVER/SHARE`. The `cifs` type is provided by the `cifs-utils` package. The file you created through the share appeared in the real folder on the server. Windows computers would reach the same share as `\\ubuntu-lab\labshare`. For fstab, put the user name and password in a root-only credentials file and use `credentials=/root/.smbcred` instead of typing the password.

6. Set up **autofs** to mount the NFS share on demand. First unmount the manual NFS mount. Then create an **indirect map**: a master entry that says "manage folder `/auto` using map file `/etc/auto.lab`," and a map that says "the key `nfs` under it is this NFS share."

```
sudo umount /mnt/nfs
grep auto.master.d /etc/auto.master
echo "/auto /etc/auto.lab --timeout=30" | sudo tee /etc/auto.master.d/lab.autofs
echo "nfs -fstype=nfs,rw 127.0.0.1:/srv/nfs/share" | sudo tee /etc/auto.lab
sudo systemctl restart autofs
```

```
+dir:/etc/auto.master.d
/auto /etc/auto.lab --timeout=30
nfs -fstype=nfs,rw 127.0.0.1:/srv/nfs/share
```

What just happened: The `grep` confirmed that the main file `/etc/auto.master` includes every `*.autofs` file in `/etc/auto.master.d/`. (If it prints nothing, add the line `/auto /etc/auto.lab --timeout=30` to `/etc/auto.master` instead.) In the map file, each line is `KEY OPTIONS LOCATION`. `--timeout=30` unmounts the share after 30 idle seconds. A **direct map** (master line `/- /etc/auto.direct`) would instead list full paths such as `/mnt/projects`.

7. Trigger the automount.

```
ls /auto
ls /auto/nfs
findmnt -t nfs4
```

```
from-student.txt
TARGET    SOURCE                    FSTYPE OPTIONS
/auto/nfs 127.0.0.1:/srv/nfs/share nfs4   rw,relatime,vers=4.2,...
```

What just happened: `ls /auto` showed nothing, because nothing was mounted yet. The moment you accessed `/auto/nfs`, autofs mounted the share. Wait about a minute without touching `/auto/nfs` and run `findmnt -t nfs4` again: it prints nothing because autofs unmounted the idle share. This saves resources and avoids hung mounts when a server is down.

Checkpoint: `ls /auto/nfs` shows `from-student.txt`, `ls /mnt/smb` shows `hello-from-smb.txt`, and `sudo exportfs -v` lists `/srv/nfs/share`.

Challenge

CHALLENGE

1. Create a new VG `chvg` on `/dev/sdb` (after unmounting btrfs and wiping it), create an LV that uses `100%FREE`, format it as XFS, add it to `/etc/fstab` with `nosuid,nodev,noexec,nofail`, and prove the options with `findmnt`. Remove it all afterwards.
2. Stop `/dev/md0`, wipe its members, and build a RAID 0 array from the same two disks instead. Compare its size with the RAID 1 array and explain the difference.
3. Add a second key to `/etc/auto.lab` that automounts the Samba share using `-fstype=cifs` and a credentials file readable only by root.

Clean up

Run these steps in order. Each one undoes part of the lab.

1. Stop autofs and remove its configuration.

```
sudo systemctl stop autofs
sudo rm -f /etc/auto.master.d/lab.autofs /etc/auto.lab
sudo systemctl disable autofs
```

2. Unmount the SMB share, restore the Samba configuration, delete the Samba user, and stop Samba.

```
sudo umount /mnt/smb
sudo cp /etc/samba/smb.conf.lab-backup /etc/samba/smb.conf
sudo rm /etc/samba/smb.conf.lab-backup
sudo smbpasswd -x student
sudo systemctl disable --now smbd nmbd
sudo rm -rf /srv/samba
```

3. Remove the NFS export and stop the NFS server.

```
sudo umount /mnt/nfs 2>/dev/null
sudo cp /etc/exports.lab-backup /etc/exports
sudo rm /etc/exports.lab-backup
sudo exportfs -ra
sudo systemctl disable --now nfs-server
sudo rm -rf /srv/nfs
```

4. Stop the RAID array and erase its metadata from both disks.

```
sudo umount /mnt/raid
sudo mdadm --stop /dev/md0
sudo mdadm --zero-superblock /dev/sdd /dev/sde
```

5. Restore `/etc/fstab` and unmount everything else.

```
sudo cp /etc/fstab.lab-backup /etc/fstab
sudo rm /etc/fstab.lab-backup
sudo systemctl daemon-reload
sudo umount /mnt/data /mnt/logs /mnt/btrfs /mnt/ram /mnt/tiny
```

6. Remove the LVM objects: LVs first, then the VG, then the PV.

```
sudo lvremove -y labvg/data labvg/logs
sudo vgremove labvg
sudo pvremove /dev/sdc
```

```
  Logical volume "data" successfully removed.
  Logical volume "logs" successfully removed.
  Volume group "labvg" successfully removed
  Labels on physical volume "/dev/sdc" successfully wiped.
```

7. Wipe all four lab disks and remove the mount points and lab folder.

```
sudo wipefs -a /dev/sdb /dev/sdc /dev/sdd /dev/sde
sudo rmdir /mnt/{part,data,logs,btrfs,ram,tiny,raid,nfs,smb,missing}
rm -rf ~/labs/1.3
lsblk
```

`lsblk` should show `sdb` to `sde` as plain 5G disks with no children. The installed packages can stay; their services are stopped and disabled.

Exam corner

- LVM order: `pvcreate` then `vgcreate` then `lvcreate` then `mkfs` then `mount`. Removal is the reverse: `umount`, `lvremove`, `vgremove`, `pvremove`.
- Summaries: `pvs`, `vgs`, `lvs`. Details: `pvdisplay`, `vgdisplay`, `lvdisplay`. Discovery: `pvscan`, `vgscan`.
- Grow: `vgextend` adds a PV; `lvextend` / `lvresize -L +SIZE`; then `resize2fs` (ext4) or `xfs_growfs MOUNTPOINT` (XFS), or use `-r` to do both. XFS cannot shrink.
- `pvmove` migrates extents off a PV online; then `vgreduce` and `pvremove`. `pvresize` after the disk grows. Move a VG between hosts: `vgchange -an`, `vgexport`, then `vgimport`, `vgchange -ay`. `lvchange -an/-ay` deactivates/activates one LV.
- MBR: 4 primary partitions, 2 TiB limit. GPT: 128 partitions, backup table. `fdisk` and `gdisk` save with `w`; `parted` writes immediately. `growpart DISK N` grows a partition (cloud disks).
- `blkid` shows UUID and TYPE; use `UUID=` in `/etc/fstab`. Fields: device, mount point, type, options, dump, fsck pass. Test with `mount -a` and `findmnt --verify`.
- Options: `noexec`, `nosuid`, `nodev` (harden `/tmp` and removable media), `ro` / `rw`, `remount`, `noatime` / `nodiratime`, `nofail` (do not block boot), `_netdev` (network mounts).
- `/proc/mounts` is the kernel's live list; `/etc/mtab` links to it.
- `fsck` (ext family) and `xfs_repair` (XFS) only on unmounted filesystems. `df -h` space, `df -i` inodes, `du -sh` folder size, `fio` benchmarks I/O.
- "No space left on device" with free space = inode exhaustion; check `df -i`.
- RAID 0 stripe, 1 mirror, 5 parity (3+ disks), 6 double parity (4+), 10 striped mirrors. `mdadm --create`, `--detail`, `--fail`, `--remove`, `--add`; status in `/proc/mdstat` (`[UU]` healthy, `[U_]` degraded).
- NFS: `/etc/exports`, `exportfs -ra`, `showmount -e`, `mount -t nfs host:/path`; `root_squash` is the default. SMB: Samba `smb.conf`, `smbpasswd -a`, `mount -t cifs //host/share`. autofs: `/etc/auto.master` plus map files, mounts on access.

Review questions

1. After running `lvextend -L +5G /dev/vg01/web`, `df -h` still shows the old size for the XFS filesystem mounted at `/var/www`. Which command fixes this?
   A. `resize2fs /dev/vg01/web`
   B. `xfs_growfs /var/www`
   C. `pvresize /dev/vg01/web`
   D. `xfs_repair /dev/vg01/web`

2. A server reports “No space left on device” when creating files, but `df -h` shows the filesystem is only 40% full. What should the administrator check next?

3. Which `/etc/fstab` option prevents the system from failing to boot if a USB backup disk is not connected?
   A. `noauto`
   B. `nofail`
   C. `nodev`
   D. `ro`

4. An administrator must replace a disk in a volume group without downtime. After adding the new disk with `vgextend`, which command moves the data off the old disk?
   A. `vgexport`
   B. `lvchange -an`
   C. `pvmove`
   D. `vgreduce`

5. `/proc/mdstat` shows `md0 : active raid1 sdc[1](F) sdb[0]` and `[2/1] [U_]`. What does this mean?

6. Which mount option stops programs from being executed from `/tmp`, and which one makes setuid bits ineffective there?

Lab 1.4 Manage Network Services and Configurations

Objective: 1.4 Given a scenario, manage network services and configurations on a Linux server.
Domain: 1.0 System Management (23% of the exam)
Time: about 100 minutes
VM: ubuntu-lab and rocky-lab (both running)
Snapshot first: Yes. This lab changes network settings on both VMs; a mistake can cut off your network connection.

Almost every Linux problem ticket eventually becomes a network question: "Is the address right? Can it reach the gateway? Does the name resolve? Is the port open?" In this lab you configure addresses with Netplan on Ubuntu and NetworkManager on Rocky, trace how a host name becomes an IP address, and then use every network tool on the exam against your two VMs and the internet.

What you'll practice

- Name resolution files: `/etc/hosts`, `/etc/resolv.conf`, `/etc/nsswitch.conf`
- NetworkManager: `nmcli`, `nmconnect` (connection profiles) and `nmtui`
- Netplan: `netplan apply`, `netplan status`, `netplan try`; `/etc/netplan`
- Common tools: `arp`, `curl`, `dig`, `ethtool`, `hostname`, `ip` (`ip address`, `ip link`, `ip route`), `iperf3`, `mtr`, `nc`, `nmap`, `nslookup`, `ping` / `ping6`, `ss`, `tcpdump`, `tracepath`, `traceroute`

How it works

The basics of a network connection. For a Linux server to talk on a network it needs four things:

1. A working **link**: the network interface (for example `enp0s3`) must exist and be "up." Interface names describe where the card sits: `en` = Ethernet, `p0s3` = PCI bus 0, slot 3. VMware VMs often have `ens33`, KVM VMs `enp1s0`.
2. An **IP address** with a prefix length, such as `192.168.100.10/24`. The `/24` means the first 24 bits are the network part, so every `192.168.100.x` address is on the same local network.
3. A **default route** (default gateway): the router that receives traffic for every network that is not local.
4. **DNS servers** that translate names such as `rockylinux.org` into IP addresses.

On the local network, computers actually deliver frames by **MAC address** (the hardware address of a network card). **ARP** (Address Resolution Protocol) asks "who has 192.168.100.20?" and caches the answer in the **neighbor table**.

Who configures the network? The `ip` command changes the running kernel settings immediately, but those changes are lost at reboot. Permanent settings come from a network configuration service:

```
Ubuntu Server 24.04                    Rocky Linux 9
-------------------                    -------------
/etc/netplan/*.yaml   (you edit)       nmcli / nmtui   (you run)
       |                                      |
  netplan generate / apply            /etc/NetworkManager/system-connections/
       |                                 *.nmconnection  (connection profiles)
systemd-networkd  (renderer)                  |
       |                              NetworkManager  (service)
       +--------------> Linux kernel <---------+
                   (addresses, links, routes)
```

Netplan is Ubuntu's configuration layer. You describe the network in YAML files in `/etc/netplan/`, and Netplan hands the settings to a "renderer": `systemd-networkd` on servers or NetworkManager on desktops. `netplan try` applies a change and automatically rolls it back after a timeout unless you confirm it; `netplan apply` applies it right away; `netplan status` shows the current state.

NetworkManager is the standard on Red Hat-family systems (and Ubuntu Desktop). It works with **connection profiles**: named sets of settings (addresses, DNS, gateway) that are applied to a **device** (an interface). On Rocky 9 each profile is stored as a keyfile ending in `.nmconnection`. The exam's wording "nmconnect" refers to this NetworkManager connection management. You manage profiles with `nmcli` (command line) or `nmtui` (a text menu).

Name resolution. When a program looks up a name, the C library reads `/etc/nsswitch.conf` to decide where to look and in what order:

```
ping rocky-lab
     |
/etc/nsswitch.conf    "hosts: files dns"
     |                      |
  files  -------->  /etc/hosts        found? use it, stop
     |
  dns    -------->  /etc/resolv.conf  "nameserver 127.0.0.53"
                          |
             Ubuntu: systemd-resolved stub (127.0.0.53)
                          |  forwards to the real DNS servers
             Rocky:  the real DNS server listed directly
```

- `/etc/hosts` is a simple local table of IP address and names. It is checked first, so it can override DNS.
- `/etc/resolv.conf` lists the DNS servers (`nameserver`) and the search domains (`search`). On Ubuntu it points to `127.0.0.53`, a local stub run by **systemd-resolved**, which forwards to the real servers (see them with `resolvectl status`). On Rocky, NetworkManager writes the real servers into it.
- Important: `dig` and `nslookup` are DNS tools. They talk to DNS servers directly and never read `/etc/hosts` or `nsswitch.conf`. `getent hosts NAME` follows the same path as normal programs.

The toolbox:

Tool	Question it answers
`ip address`, `ip link`, `ip route`	What addresses, interfaces, and routes do I have?
`hostname`	What is this machine called?
`ping`, `ping6`	Can I reach that host (IPv4/IPv6)?
`arp`, `ip neigh`	Which MAC address belongs to that local IP?
`traceroute`, `tracepath`, `mtr`	Which routers are on the path, and where does it break?
`dig`, `nslookup`	What does DNS say about this name?
`curl`	Does the web service answer, and what does it return?
`ss`	Which ports are listening, and which connections are open?
`nc`	Can I open a TCP/UDP connection to that port? Send raw data.
`nmap`	Which ports are open on that host?
`iperf3`	How fast is the network between two hosts?
`ethtool`	What speed, duplex, and driver does the network card use?
`tcpdump`	What packets are actually on the wire?

Tasks

Task 1: Inspect the network with hostname and ip

1. On ubuntu-lab, create the lab directory and check the host name.

```
mkdir -p ~/labs/1.4 && cd ~/labs/1.4
hostname
hostname -I
hostnamectl
```

```
ubuntu-lab
192.168.100.10
 Static hostname: ubuntu-lab
       Icon name: computer-vm
         Chassis: vm
Operating System: Ubuntu 24.04.1 LTS
          Kernel: Linux 6.8.0-45-generic
    Architecture: x86-64
```

What just happened: `hostname` prints the system's name, and `hostname -I` prints all its IP addresses (IPv4 and IPv6) on one line. `hostnamectl` shows the name plus system details; the permanent name is stored in `/etc/hostname` and you change it with `sudo hostnamectl set-hostname NEWNAME`.

2. List the network interfaces (links).

```
ip link
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group ...
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode ...
    link/ether 08:00:27:4e:66:a1 brd ff:ff:ff:ff:ff:ff
```

What just happened: `ip link` (short for `ip link show`) lists every interface. `lo` is the **loopback** interface, which the computer uses to talk to itself (`127.0.0.1`). `enp0s3` is the network card. `UP` means it is enabled, `LOWER_UP` means a cable (virtual, here) is connected, `mtu 1500` is the largest packet size in bytes, and `link/ether` is the MAC address. Write down your interface name: this lab uses `enp0s3`, so replace it with yours everywhere.

3. Show the IP addresses, in long and brief form.

```
ip address show enp0s3
ip -br address
```

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group ...
    link/ether 08:00:27:4e:66:a1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.10/24 metric 100 brd 192.168.100.255 scope global dynamic enp0s3
       valid_lft 85912sec preferred_lft 85912sec
    inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
       valid_lft forever preferred_lft forever
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.10/24 fe80::a00:27ff:fe4e:66a1/64
```

What just happened: `ip address` (you can abbreviate it to `ip a` or `ip addr`) shows addresses. `inet` is IPv4, `inet6` is IPv6. `dynamic` means it came from DHCP and `valid_lft` is how long the lease has left; a static address shows `forever`. The `fe80::` address is an IPv6 **link-local** address that every interface gives itself automatically. `-br` (brief) prints one line per interface, which is the quickest overview.

4. Show the routing table and ask which route a packet would take.

```
ip route
ip route get 1.1.1.1
```

```
default via 192.168.100.1 dev enp0s3 proto dhcp src 192.168.100.10 metric 100
192.168.100.0/24 dev enp0s3 proto kernel scope link src 192.168.100.10 metric 100
192.168.100.1 dev enp0s3 proto dhcp scope link src 192.168.100.10 metric 100
1.1.1.1 via 192.168.100.1 dev enp0s3 src 192.168.100.10 uid 1000
    cache
```

What just happened: `ip route` lists routes. `default via 192.168.100.1` is the default gateway: anything not matched by a more specific route goes there. The `192.168.100.0/24 ... scope link` line says the local network is reached directly, without a router. `ip route get` shows the decision for one destination, which is the fastest way to answer "which interface and gateway would this use?"

5. Practice runtime changes safely on a dummy interface (the fake interface type from Lab 1.2). These changes affect only the running system and vanish at reboot.

```
sudo ip link add lab0 type dummy
sudo ip link set lab0 up
sudo ip address add 10.50.0.1/24 dev lab0
sudo ip route add 10.60.0.0/24 via 10.50.0.2 dev lab0
ip -br address show lab0
ip route get 10.60.0.9
sudo ip link delete lab0
```

```
lab0             UNKNOWN        10.50.0.1/24 fe80::d4b1:2cff:fe9a:13e7/64
10.60.0.9 via 10.50.0.2 dev lab0 src 10.50.0.1 uid 1000
    cache
```

What just happened: `ip link add ... type dummy` created an interface, `ip link set ... up` enabled it (`down` would disable it), `ip address add` gave it an address, and `ip route add NETWORK via GATEWAY` added a static route. `ip route get` confirms traffic for `10.60.0.9` would go out through `lab0` to `10.50.0.2`. Deleting the link removed its address and route too. This is how you test settings quickly; then you make them permanent with Netplan or NetworkManager. On the exam, remember that old `ifconfig` and `route` (package `net-tools`) are replaced by `ip address` and `ip route`.

Checkpoint: You know your interface name, IP address, prefix, and default gateway, and `ip link show lab0` reports that the device does not exist (it was deleted).

Task 2: Follow name resolution through hosts, nsswitch, and resolv.conf

1. Look at the three name resolution files on ubuntu-lab.

```
cat /etc/hosts
grep '^hosts' /etc/nsswitch.conf
ls -l /etc/resolv.conf
grep -v '^#' /etc/resolv.conf
```

```
127.0.0.1 localhost
127.0.1.1 ubuntu-lab
192.168.100.10 ubuntu-lab
192.168.100.20 rocky-lab
::1     ip6-localhost ip6-loopback
hosts:          files dns
lrwxrwxrwx 1 root root 39 Apr 22 13:10 /etc/resolv.conf -> ../run/systemd/resolve/stub-
resolv.conf
nameserver 127.0.0.53
options edns0 trust-ad
search .
```

What just happened: `/etc/hosts` lines are `IP NAME [ALIASES...]`; your Lab 0 entries for both VMs are there. The `hosts:` line in `/etc/nsswitch.conf` says "check `files` (`/etc/hosts`) first, then `dns`." `/etc/resolv.conf` is a symbolic link managed by systemd-resolved and points to the local stub at `127.0.0.53`. Do not edit it by hand on Ubuntu; your changes would be overwritten. `grep -v '^#'` hides the comment lines.

2. Find the real DNS servers behind the stub.

```
resolvectl status enp0s3
```

```
Link 2 (enp0s3)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.100.1
       DNS Servers: 192.168.100.1
```

What just happened: `resolvectl` talks to systemd-resolved. The DNS server for `enp0s3` came from DHCP (often your virtual router or host). systemd-resolved receives every query on `127.0.0.53`, caches answers, and forwards misses to this server.

3. Add a test name to `/etc/hosts` and resolve it with three different tools. You can use `sudo nano /etc/hosts`, or append the line with `tee -a`. The first command makes sure `dig` is installed (it comes in the `bind9-dnsutils` package; apt just says "already the newest version" if you have it).

```
sudo apt install -y bind9-dnsutils
echo "192.168.100.20 fileserver.lab fileserver" | sudo tee -a /etc/hosts
getent hosts fileserver
ping -c 2 fileserver
dig +short @1.1.1.1 fileserver.lab
```

```
192.168.100.20 fileserver.lab fileserver
192.168.100.20  fileserver.lab fileserver
PING fileserver.lab (192.168.100.20) 56(84) bytes of data.
64 bytes from rocky-lab (192.168.100.20): icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from rocky-lab (192.168.100.20): icmp_seq=2 ttl=64 time=0.398 ms
```

What just happened: `getent hosts` and `ping` follow `nsswitch.conf`, find the name in `/etc/hosts`, and get `192.168.100.20`. `dig @1.1.1.1` asked a public DNS server directly, which has never heard of `fileserver.lab`, so `+short` printed nothing. This is the key troubleshooting lesson: **if `ping NAME` works but `dig NAME` fails (or the reverse), the name is coming from `/etc/hosts`**, not DNS. (On Ubuntu, plain `dig fileserver.lab` would succeed because the systemd-resolved stub also reads `/etc/hosts`. On Rocky, which has no stub, it would fail.)

4. Look at the same files on rocky-lab, where NetworkManager writes `resolv.conf`.

```
# on rocky-lab
grep '^hosts' /etc/nsswitch.conf
ls -l /etc/resolv.conf
cat /etc/resolv.conf
```

```
hosts:      files dns myhostname
-rw-r--r--. 1 root root 72 Oct 10 09:21 /etc/resolv.conf
# Generated by NetworkManager
search localdomain
nameserver 192.168.100.1
```

What just happened: On Rocky, `/etc/resolv.conf` is a regular file written by NetworkManager, and it lists the real DNS server. `myhostname` in `nsswitch.conf` is a systemd module that always resolves the machine's own name. To change DNS on Rocky you change the NetworkManager connection (Task 4), not this file directly, because NetworkManager rewrites it.

Checkpoint: On ubuntu-lab, `getent hosts fileserver` returns `192.168.100.20`, and you can explain why `dig @1.1.1.1 fileserver.lab` returns nothing.

Task 3: Configure a second IP address with Netplan

CAUTION

Do this task at the ubuntu-lab **console**, not over SSH. A wrong Netplan file can disconnect the network. `netplan try` protects you by rolling back automatically, but only if you can still see the prompt. Take a snapshot now if you have not.

1. Look at the existing Netplan configuration. The files are readable only by root.

```
ls -l /etc/netplan/
sudo cat /etc/netplan/*.yaml
```

```
-rw------- 1 root root 104 Oct 10 09:10 50-cloud-init.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
```

> **What just happened:** The installer wrote `50-cloud-init.yaml`. YAML uses **indentation with spaces** (never tabs) to show structure: `network` contains `ethernets`, which contains the interface `enp0s3`, which has `dhcp4: true` (get an IPv4 address from DHCP). Your file name may differ (for example `00-installer-config.yaml`). Netplan reads every `.yaml` file in `/etc/netplan/` in alphabetical order and merges them.

2. Check the current state with `netplan status`.

```
sudo netplan status
```

```
     Online state: online
    DNS Addresses: 127.0.0.53 (stub)
       DNS Search: .

●  2: enp0s3 ethernet UP (networkd: enp0s3)
      MAC Address: 08:00:27:4e:66:a1 (Intel Corporation)
        Addresses: 192.168.100.10/24 (dhcp)
                   fe80::a00:27ff:fe4e:66a1/64 (link)
    DNS Addresses: 192.168.100.1
           Routes: default via 192.168.100.1 from 192.168.100.10 metric 100 (dhcp)
                   192.168.100.0/24 from 192.168.100.10 metric 100 (link)
```

> **What just happened:** `netplan status` summarizes each interface: which backend manages it (`networkd: enp0s3`), its addresses and where they came from (`dhcp`, `link`), DNS servers, and routes. Interfaces Netplan does not manage are listed at the end as "inactive or unmanaged."

3. Create a new file that adds a second, static address to the same interface. Choose an unused address on your lab network; this lab uses `192.168.100.110`. The heredoc below writes the file: everything between `<<'EOF'` and the closing `EOF` goes into `tee`, which saves it with root rights. Replace `enp0s3` if your interface has another name.

```
cat <<'EOF' | sudo tee /etc/netplan/60-lab.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 192.168.100.110/24
EOF
sudo chmod 600 /etc/netplan/60-lab.yaml
```

> **What just happened:** Because Netplan merges files, this file adds an `addresses` list to `enp0s3` while `dhcp4: true` from the first file stays in effect. Using a separate file makes it easy to undo: delete it and apply. `chmod 600` limits the file to root; Netplan warns about files that others can read, because they may contain Wi-Fi passwords.

4. Apply it with `netplan try`. Press **Enter** when asked to keep the settings.

```
sudo netplan try
```

```
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 118 seconds
Configuration accepted.
```

What just happened: `netplan try` checked the YAML, applied it, and started a countdown (120 seconds by default; change it with `--timeout`). If the change had broken your connection, you would not be able to press Enter (or would not be able to reach the prompt over SSH), and Netplan would restore the previous configuration by itself. Pressing Enter made it permanent.

5. Verify the new address, and test it from rocky-lab.

```
ip -br address show enp0s3
```

```
enp0s3           UP             192.168.100.110/24 192.168.100.10/24
fe80::a00:27ff:fe4e:66a1/64
```

```
# on rocky-lab
ping -c 2 192.168.100.110
```

6. Watch the automatic rollback. Remove the extra address from the configuration, run `netplan try` with a short timeout, and this time do **not** press Enter.

```
sudo mv /etc/netplan/60-lab.yaml ~/labs/1.4/60-lab.yaml.disabled
sudo netplan try --timeout 20
ip -br address show enp0s3
```

```
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in  1 seconds
Reverting.
enp0s3           UP             192.168.100.110/24 192.168.100.10/24
fe80::a00:27ff:fe4e:66a1/64
```

What just happened: `netplan try` applied the configuration without the extra address, waited 20 seconds, received no confirmation, and reverted to the configuration that was active before, so `192.168.100.110` is still there. Because it saved a backup of `/etc/netplan` before trying, it even put `60-lab.yaml` back. Check with `ls /etc/netplan/`. This is your safety net for remote changes.

7. Now remove the change for real with `netplan apply`, which applies immediately with no rollback.

```
ls /etc/netplan/
sudo rm -f /etc/netplan/60-lab.yaml
sudo netplan apply
ip -br address show enp0s3
```

```
50-cloud-init.yaml  60-lab.yaml
enp0s3           UP             192.168.100.10/24 fe80::a00:27ff:fe4e:66a1/64
```

What just happened: `netplan apply` generated the backend configuration (for systemd-networkd, files under `/run/systemd/network/`) and reloaded the network. Only the DHCP address remains. If the extra address is still listed, run `sudo ip address del 192.168.100.110/24 dev enp0s3` once to remove the leftover runtime address.

8. For reference, this is what a fully static configuration looks like. Save it as a file in your lab folder; do **not** copy it into `/etc/netplan/`.

```
cat <<'EOF' > ~/labs/1.4/static-example.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.100.10/24
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
        search: [lab.local]
EOF
```

What just happened: A static setup lists the address, a default route (`to: default` with `via:` the gateway; the older `gateway4:` key is deprecated), and DNS servers under `nameservers`. `renderer: networkd` is the server default; Ubuntu Desktop uses `renderer: NetworkManager`. `sudo netplan generate` would turn YAML into backend files without applying them, which is a handy syntax check.

Checkpoint: `ls /etc/netplan/` shows only the original file, `ip -br address show enp0s3` shows only the DHCP address, and `sudo netplan status` reports `Online state: online`.

Task 4: Manage connections with NetworkManager (nmcli and nmtui) on rocky-lab

> **CAUTION**
> Work at the rocky-lab console for this task. The commands use `nmcli device reapply`, which applies changes without taking the link down, but a typo in an address can still interrupt SSH.

1. Look at NetworkManager's view of the system: overall status, devices, and connection profiles.

```
# on rocky-lab
nmcli general status
nmcli device status
nmcli connection show
```

```
STATE      CONNECTIVITY  WIFI-HW  WIFI     WWAN-HW  WWAN
connected  full          missing  enabled  missing  enabled
DEVICE  TYPE      STATE                   CONNECTION
enp0s3  ethernet  connected               enp0s3
lo      loopback  connected (externally)  lo
NAME    UUID                                  TYPE      DEVICE
enp0s3  a3f9c1d2-5b6e-4f70-8a91-b2c3d4e5f607  ethernet  enp0s3
lo      0c1d2e3f-4a5b-4c6d-8e7f-a0b1c2d3e4f5  loopback  lo
```

> **What just happened:** A **device** is a network interface; a **connection** is a profile of settings applied to a device. Here the profile is named `enp0s3`, like its device, but it could be called `Wired connection 1` or `System eth0`. In the commands below, replace `enp0s3` with your **connection NAME** where the command says `connection`, and with your **DEVICE** where it says `device`.

2. Find the profile file. This is NetworkManager's "nmconnect" storage: one keyfile per connection.

```
# on rocky-lab
sudo ls /etc/NetworkManager/system-connections/
sudo cat /etc/NetworkManager/system-connections/enp0s3.nmconnection
```

```
enp0s3.nmconnection
[connection]
id=enp0s3
uuid=a3f9c1d2-5b6e-4f70-8a91-b2c3d4e5f607
type=ethernet
interface-name=enp0s3

[ipv4]
method=auto

[ipv6]
addr-gen-mode=eui64
method=auto
```

What just happened: Rocky 9 stores connection profiles as INI-style keyfiles ending in `.nmconnection`. `method=auto` means DHCP. You can edit these files, but then you must run `sudo nmcli connection reload`; using `nmcli` is safer because it validates every value. (Older systems, and Rocky 9 machines upgraded from 8, may still use `/etc/sysconfig/network-scripts/ifcfg-*` files instead.)

3. Show the IPv4 settings of the profile, then add a secondary address. The `+` in `+ipv4.addresses` means "add to the list" instead of replacing it.

```
# on rocky-lab
nmcli -f ipv4.method,ipv4.addresses,ipv4.dns connection show enp0s3
sudo nmcli connection modify enp0s3 +ipv4.addresses 192.168.100.120/24
sudo nmcli device reapply enp0s3
ip -br address show enp0s3
```

```
ipv4.method:                            auto
ipv4.addresses:                         --
ipv4.dns:                               --
Connection successfully reapplied to device 'enp0s3'.
enp0s3           UP             192.168.100.20/24 192.168.100.120/24
fe80::a00:27ff:fe9a:13e7/64
```

What just happened: `-f` (fields) limits the output to the settings you asked for. `nmcli connection modify` changed the saved profile only; nothing happened on the network until `nmcli device reapply` pushed the updated profile to the device without bringing it down. (`sudo nmcli connection up enp0s3` would also apply it, but it restarts the connection, which briefly drops traffic.) With `method=auto` plus a static address, the interface keeps its DHCP address and adds the static one, just like your Netplan example.

4. Override the DNS servers that DHCP provides.

```
# on rocky-lab
sudo nmcli connection modify enp0s3 ipv4.dns "1.1.1.1 9.9.9.9" ipv4.ignore-auto-dns yes
sudo nmcli device reapply enp0s3
cat /etc/resolv.conf
```

```
Connection successfully reapplied to device 'enp0s3'.
# Generated by NetworkManager
search localdomain
nameserver 1.1.1.1
nameserver 9.9.9.9
```

What just happened: `ipv4.dns` sets the DNS server list, and `ipv4.ignore-auto-dns yes` tells NetworkManager to ignore the DNS servers offered by DHCP. NetworkManager rewrote `/etc/resolv.conf` immediately. This is the correct way to change DNS on a NetworkManager system.

5. Undo both changes and confirm the profile is back to DHCP only. The `-` removes one value from a list, and an empty string clears a setting.

```
# on rocky-lab
sudo nmcli connection modify enp0s3 -ipv4.addresses 192.168.100.120/24
sudo nmcli connection modify enp0s3 ipv4.dns "" ipv4.ignore-auto-dns no
sudo nmcli device reapply enp0s3
nmcli -f ipv4.addresses,ipv4.dns,ipv4.ignore-auto-dns connection show enp0s3
ip -br address show enp0s3
```

```
ipv4.addresses:                         --
ipv4.dns:                               --
ipv4.ignore-auto-dns:                   no
enp0s3           UP             192.168.100.20/24 fe80::a00:27ff:fe9a:13e7/64
```

6. Open `nmtui`, the text menu for the same settings.

```
# on rocky-lab
sudo dnf install -y NetworkManager-tui
sudo nmtui
```

> **What just happened:** `nmtui` shows three choices: **Edit a connection**, **Activate a connection**, and **Set system hostname**. Use the arrow keys, **Tab**, and **Enter** to open your connection and look at its IPv4 settings, then choose **Cancel** or **Back** and **Quit** without saving. `nmtui` and `nmcli` edit the same `.nmconnection` profiles, so you can use whichever you prefer.

Checkpoint: On rocky-lab, `nmcli -f ipv4.addresses,ipv4.dns connection show enp0s3` shows `--` for both, and `cat /etc/resolv.conf` shows your original DHCP DNS server again.

Task 5: Test reachability and the path with ping, arp, and traceroute tools

1. Install the tools that may be missing on ubuntu-lab. `net-tools` provides the classic `arp` command; `iputils-tracepath` provides `tracepath`.

```
sudo apt update
sudo apt install -y iputils-ping iputils-tracepath traceroute mtr-tiny net-tools
```

2. Ping rocky-lab over IPv4, then IPv6.

```
ping -c 4 rocky-lab
ping6 -c 2 ::1
```

```
PING rocky-lab (192.168.100.20) 56(84) bytes of data.
64 bytes from rocky-lab (192.168.100.20): icmp_seq=1 ttl=64 time=0.521 ms
64 bytes from rocky-lab (192.168.100.20): icmp_seq=2 ttl=64 time=0.402 ms
64 bytes from rocky-lab (192.168.100.20): icmp_seq=3 ttl=64 time=0.389 ms
64 bytes from rocky-lab (192.168.100.20): icmp_seq=4 ttl=64 time=0.411 ms

--- rocky-lab ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3052ms
rtt min/avg/max/mdev = 0.389/0.430/0.521/0.052 ms
PING ::1 (::1) 56 data bytes
64 bytes from ::1: icmp_seq=1 ttl=64 time=0.031 ms
64 bytes from ::1: icmp_seq=2 ttl=64 time=0.045 ms
```

What just happened: `ping` sends ICMP echo requests and waits for replies. `-c 4` stops after four (without it, ping runs until **Ctrl+C**). Read the summary: packet loss shows reliability and `rtt` (round-trip time) shows latency. `ttl` (time to live) is the hop counter; a reply from the same network keeps the starting value (64 for Linux). `ping6` is the IPv6 version (modern systems also accept `ping -6`); `::1` is the IPv6 loopback address. If `ping6` is not found, use `ping -6 -c 2 ::1`.

3. Ping rocky-lab's IPv6 link-local address. Link-local addresses exist on every interface, so you must say which interface to use with `%INTERFACE`. Find the address on rocky-lab first.

```
# on rocky-lab
ip -6 -br address show enp0s3
```

```
ping6 -c 2 fe80::a00:27ff:fe9a:13e7%enp0s3
```

What just happened: Replace the address with rocky-lab's `fe80::` address and `%enp0s3` with ubuntu-lab's interface name. IPv6 works on your lab network even if no one configured it, which is why the exam expects you to know `ping6`.

4. Look at the neighbor (ARP) table. Ping first so the entry is fresh.

```
ip neigh
arp -n
```

```
192.168.100.20 dev enp0s3 lladdr 08:00:27:9a:13:e7 REACHABLE
192.168.100.1 dev enp0s3 lladdr 52:54:00:12:35:02 STALE
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.100.20           ether   08:00:27:9a:13:e7   C                     enp0s3
192.168.100.1            ether   52:54:00:12:35:02   C                     enp0s3
```

What just happened: Both commands show the same cache: which MAC address (`lladdr` / `HWaddress`) belongs to each local IP. `REACHABLE` means recently confirmed and `STALE` means it will be rechecked before use. `-n` shows numbers instead of names. Compare the MAC address with `ip link` on rocky-lab: they match. Two IPs with the same MAC, or one IP whose MAC keeps changing, points to an IP conflict.

5. Trace the path to an internet host with three tools. The first hops are your virtual router and home router; inside a VM some hops may only show `*` or `no reply`, which is normal.

```
tracepath -n 1.1.1.1
traceroute -n 1.1.1.1
mtr -rwc 5 1.1.1.1
```

```
 1?: [LOCALHOST]                      pmtu 1500
 1:  192.168.100.1                                         0.512ms
 2:  192.168.1.1                                           1.870ms
 3:  100.64.0.1                                            9.334ms
 ...
traceroute to 1.1.1.1 (1.1.1.1), 30 hops max, 60 byte packets
 1  192.168.100.1  0.402 ms  0.371 ms  0.355 ms
 2  192.168.1.1  1.912 ms  1.880 ms  1.851 ms
 3  * * *
...
Start: 2026-10-10T12:10:01+0000
HOST: ubuntu-lab                  Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 192.168.100.1               0.0%     5    0.5   0.5   0.4   0.6   0.1
  2.|-- 192.168.1.1                 0.0%     5    1.9   2.0   1.8   2.3   0.2
  3.|-- 100.64.0.1                  0.0%     5    9.4   9.8   9.1  11.0   0.7
  ...
  8.|-- 1.1.1.1                     0.0%     5   12.1  12.4  11.9  13.2   0.5
```

What just happened: All three send packets with increasing TTL values; each router that drops a packet because its TTL reached 0 reports back, revealing the path one hop at a time. `traceroute` sends UDP probes by default (`-I` uses ICMP, `-T` uses TCP) and shows three timings per hop; `* * *` means that router did not answer. `tracepath` needs no root and also discovers the path MTU (`pmtu`). `mtr` combines ping and traceroute and keeps measuring: `-r` report mode, `-w` wide output, `-c 5` five cycles. High **Loss%** that starts at one hop and continues to the end shows where a problem begins.

Checkpoint: `ping -c 2 rocky-lab` shows 0% packet loss, `ip neigh | grep 192.168.100.20` shows rocky-lab's MAC address, and `mtr -rwc 3 1.1.1.1` lists at least one hop.

Task 6: Query DNS with dig and nslookup, and test web services with curl

1. Look up an address with `dig`.

```
dig rockylinux.org
```

```
; <<>> DiG 9.18.28-0ubuntu0.24.04.1-Ubuntu <<>> rockylinux.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41022
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
;rockylinux.org.                        IN      A

;; ANSWER SECTION:
rockylinux.org.         300     IN      A       104.21.32.1
rockylinux.org.         300     IN      A       172.67.155.10

;; Query time: 24 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
```

What just happened: `dig` (from the `bind9-dnsutils` package) shows the full DNS conversation. **status: NOERROR** means success (**NXDOMAIN** means the name does not exist, **SERVFAIL** means the server failed). The **ANSWER SECTION** lists records: name, **TTL** in seconds (how long it may be cached), class `IN` (internet), type `A` (IPv4 address), and the value. **SERVER** shows who answered: the systemd-resolved stub. Your addresses will differ.

2. Ask for specific record types and use a specific server.

```
dig +short rockylinux.org AAAA
dig +short @1.1.1.1 ubuntu.com MX
dig +short -x 1.1.1.1
```

```
2606:4700:3030::6815:2001
2606:4700:3036::ac43:9b0a
1 mx.ubuntu.com.
one.one.one.one.
```

What just happened: `+short` prints only the answers. `AAAA` records hold IPv6 addresses and `MX` records name mail servers (with a priority number). `@1.1.1.1` sends the question to that server instead of the one in `resolv.conf`, which is how you test whether a problem is your DNS server or the record itself. `-x` does a **reverse lookup** (IP address to name, a `PTR` record).

3. Do the same with `nslookup`, the older tool that also exists on Windows.

```
nslookup rockylinux.org
nslookup -type=mx ubuntu.com 1.1.1.1
```

```
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   rockylinux.org
Address: 104.21.32.1
...
Server:         1.1.1.1
Address:        1.1.1.1#53

Non-authoritative answer:
ubuntu.com      mail exchanger = 1 mx.ubuntu.com.
```

What just happened: `nslookup NAME [SERVER]` gives a shorter, friendlier answer. "Non-authoritative" means the answer came from a cache, not from the domain's own DNS server. `-type=` chooses the record type.

4. Test a web server with `curl`.

```
curl -I https://rockylinux.org
curl -s -o /dev/null -w 'code=%{http_code} time=%{time_total}s\n' https://ubuntu.com
```

```
HTTP/2 200
date: Fri, 10 Oct 2026 12:15:02 GMT
content-type: text/html; charset=utf-8
server: cloudflare
...
code=200 time=0.412s
```

What just happened: `curl` transfers data with URLs. `-I` fetches only the **headers**: the status line (`200` OK, `301` / `302` redirect, `404` not found, `500` server error) and metadata such as the server software. The second command is a quick health check: `-s` silent, `-o /dev/null` throws the page away, and `-w` prints only the status code and total time. Add `-L` to follow redirects and `-v` to see the whole TLS and HTTP conversation.

Checkpoint: `dig +short rockylinux.org` prints at least one IP address, and the `curl -w` command prints `code=200`.

Task 7: Find listening ports and test them with ss, nc, and nmap

1. List listening sockets on ubuntu-lab.

```
sudo ss -tulpn
```

```
Netid State  Recv-Q Send-Q  Local Address:Port  Peer Address:Port Process
udp   UNCONN 0      0       127.0.0.53%lo:53         0.0.0.0:*  users:(("systemd-
resolve",...))
udp   UNCONN 0      0 192.168.100.10%enp0s3:68       0.0.0.0:*  users:(("systemd-
network",...))
tcp   LISTEN 0      4096    127.0.0.53%lo:53         0.0.0.0:*  users:(("systemd-
resolve",...))
tcp   LISTEN 0      4096          0.0.0.0:22         0.0.0.0:*  users:(("sshd",pid=801,...),
("systemd",pid=1,...))
tcp   LISTEN 0      4096             [::]:22            [::]:*  users:(("sshd",pid=801,...),
("systemd",pid=1,...))
```

What just happened: `ss` (socket statistics) replaces the old `netstat`. The options are easy to remember as "tulpn": `-t` TCP, `-u` UDP, `-l` listening only, `-p` show the process (needs `sudo`), `-n` numeric ports instead of names. **Local Address:Port** tells you where a service listens: `0.0.0.0:22` and `[::]:22` mean SSH accepts connections on every IPv4 and IPv6 address, while `127.0.0.53%lo:53` means the DNS stub is only reachable from this machine. Port 22 shows two processes because Ubuntu 24.04 uses **socket activation** for SSH: `systemd` (PID 1) opens the port through the `ssh.socket` unit and starts `sshd` when the first connection arrives. If nobody has connected yet, you may see only `systemd`. `ss -tn` without `-l` shows established connections, such as your SSH session.

2. Open a temporary port on rocky-lab's firewall and start a listener there. On Rocky, `nc` comes from the `nmap-ncat` package. Leave this command running.

```
# on rocky-lab
sudo dnf install -y nmap-ncat
sudo firewall-cmd --add-port=5000/tcp
nc -l 5000
```

What just happened: firewalld is active on Rocky and blocks most ports. `firewall-cmd --add-port=5000/tcp` without `--permanent` opens the port only until the next reload or reboot, which is ideal for a test. `nc -l 5000` (netcat) listens on TCP port 5000 and prints whatever arrives.

3. On ubuntu-lab, connect to it and type a few lines. Each line appears on rocky-lab. Type on rocky-lab and it appears on ubuntu-lab too. Press **Ctrl+C** to end.

```
nc rocky-lab 5000
```

What just happened: You built a simple chat over a raw TCP connection. This proves the whole path works: name resolution, routing, the firewall rule, and the listening process. When an application "cannot connect," running `nc -l PORT` in place of the real service tells you whether the problem is the network or the application.

4. Use `nc` as a quick port tester. `-z` scans without sending data and `-v` prints the result.

```
nc -zv rocky-lab 22
nc -zv -w 3 rocky-lab 80
```

```
Connection to rocky-lab (192.168.100.20) 22 port [tcp/ssh] succeeded!
nc: connect to rocky-lab (192.168.100.20) port 80 (tcp) failed: No route to host
```

What just happened: Port 22 is open. For port 80, firewalld rejected the connection with an ICMP "host prohibited" message, which `nc` reports as "No route to host." Learn the three outcomes: **succeeded** (open), **Connection refused** (the host is reachable but nothing listens on that port), and **timed out** or **No route to host** (a firewall is in the way). `-w 3` sets a 3-second timeout.

5. Scan rocky-lab with `nmap`. Only scan systems you own or have permission to scan.

```
sudo apt install -y nmap
nmap rocky-lab
sudo nmap -sV -p 22 rocky-lab
nmap -sn 192.168.100.0/24
```

```
Nmap scan report for rocky-lab (192.168.100.20)
Host is up (0.00045s latency).
Not shown: 997 filtered tcp ports (host-prohibited)
PORT     STATE  SERVICE
22/tcp   open   ssh
5000/tcp closed upnp
9090/tcp closed zeus-admin
...
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.7 (protocol 2.0)
...
Nmap scan report for 192.168.100.10
Host is up.
Nmap scan report for rocky-lab (192.168.100.20)
Host is up (0.00051s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.41 seconds
```

What just happened: `nmap HOST` checks the 1,000 most common TCP ports. **open** = a service answered, **closed** = the port is reachable but nothing listens (5000 is closed now because your `nc` listener ended, and 9090 is allowed by firewalld for the Cockpit web console, which is not running), **filtered** = a firewall dropped the probes. `-sV` asks the service for its software version, `-p` picks ports, and `-sn` is a "ping scan" that only finds which hosts are up on a network.

Checkpoint: `nc -zv rocky-lab 22` succeeds, and `nmap rocky-lab` lists `22/tcp open ssh`.

Task 8: Measure, inspect, and capture with iperf3, ethtool, and tcpdump

1. Install `iperf3` on both VMs. On Ubuntu, setting `DEBIAN_FRONTEND=noninteractive` skips a question about running iperf3 as a permanent service (the default answer, no, is what you want).

```
sudo DEBIAN_FRONTEND=noninteractive apt install -y iperf3 ethtool tcpdump
```

```
# on rocky-lab
sudo dnf install -y iperf3
sudo firewall-cmd --add-port=5201/tcp
iperf3 -s -1
```

What just happened: On rocky-lab you opened TCP port 5201 (iperf3's default) until the next firewall reload and started an iperf3 **server**. `-s` means server mode and `-1` means "handle one test and then exit," so you do not leave a service running.

2. Run the client on ubuntu-lab for 5 seconds.

```
iperf3 -c rocky-lab -t 5
```

```
Connecting to host rocky-lab, port 5201
[  5] local 192.168.100.10 port 41822 connected to 192.168.100.20 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   212 MBytes  1.78 Gbits/sec    0    1.02 MBytes
...
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-5.00   sec  1.04 GBytes  1.79 Gbits/sec    0             sender
[  5]   0.00-5.00   sec  1.04 GBytes  1.78 Gbits/sec                  receiver
```

What just happened: `-c HOST` runs the client and `-t 5` tests for 5 seconds. The **Bitrate** is the measured throughput between the VMs; **Retr** counts TCP retransmissions, and high values indicate packet loss. Between VMs on one host the speed depends on your CPU, not a real cable. To test the other direction, start the server again and run `iperf3 -c rocky-lab -R` (reverse). `-u` tests UDP instead.

3. Inspect the network card with `ethtool`.

```
sudo ethtool enp0s3 | grep -E 'Speed|Duplex|Auto-negotiation|Link detected'
sudo ethtool -i enp0s3
sudo ethtool -S enp0s3 | head -6
```

```
        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: on
        Link detected: yes
driver: e1000
version: 6.8.0-45-generic
firmware-version:
bus-info: 0000:00:03.0
NIC statistics:
     rx_packets: 48231
     tx_packets: 21877
     rx_bytes: 61250312
     tx_bytes: 2840211
     rx_broadcast: 0
```

What just happened: `ethtool IFACE` shows link settings: **Speed**, **Duplex** (full = send and receive at once), **Auto-negotiation**, and **Link detected** (is a cable connected?). `-i` shows the driver (the kernel module from Lab 1.2) and its PCI address, and `-S` shows the card's own counters, where rising error or drop counters point to a bad cable or port. A `virtio_net` card (KVM) may show `Speed: Unknown!`, which is normal for a virtual card.

TIP

Explore/simulate: A virtual card has no physical link to negotiate, so do not change these settings in the lab. On a physical server with a duplex mismatch, you would force the settings with `sudo ethtool -s enp0s3 speed 1000 duplex full autoneg off` (or turn negotiation back on with `autoneg on`), and make it permanent with NetworkManager (`nmcli connection modify NAME 802-3-ethernet.speed 1000 802-3-ethernet.duplex full 802-3-ethernet.auto-negotiate no`) or, on Ubuntu with systemd-networkd, with a systemd `.link` file in `/etc/systemd/network/` (settings `BitsPerSecond=`, `Duplex=`, and `AutoNegotiation=`). `ethtool -p enp0s3 10` blinks the port LED for 10 seconds so you can find the right cable in a rack.

4. Capture packets with `tcpdump` while you ping rocky-lab. `sudo -v` asks for your password first, so the background `sudo` command does not stop to ask for it. `&` runs tcpdump in the background, and `wait` waits for it to finish.

```
sudo -v
sudo tcpdump -i enp0s3 -n -c 4 icmp &
sleep 1; ping -c 2 rocky-lab; wait
```

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:30:01.101234 IP 192.168.100.10 > 192.168.100.20: ICMP echo request, id 7, seq 1, length
64
12:30:01.101601 IP 192.168.100.20 > 192.168.100.10: ICMP echo reply, id 7, seq 1, length 64
12:30:02.102877 IP 192.168.100.10 > 192.168.100.20: ICMP echo request, id 7, seq 2, length
64
12:30:02.103210 IP 192.168.100.20 > 192.168.100.10: ICMP echo reply, id 7, seq 2, length 64
4 packets captured
```

What just happened: `tcpdump` printed every packet that matched the **filter** `icmp`. `-i` picks the interface (`-i any` captures on all), `-n` shows numbers instead of names, and `-c 4` stops after four packets. Each line shows the time, source `>` destination, and the protocol details: two requests and two replies. Other useful filters: `host 192.168.100.20`, `port 22`, `tcp port 5000 and host rocky-lab`. If you only saw requests and no replies, the problem would be on the other side or on the way back.

5. Save a capture to a file and read it back. A `.pcap` file can also be opened in Wireshark on your desktop.

```
sudo tcpdump -i enp0s3 -n -c 2 -w /tmp/dns.pcap port 53 &
sleep 1; dig +short @1.1.1.1 rockylinux.org; wait
tcpdump -n -r /tmp/dns.pcap
```

```
104.21.32.1
172.67.155.10
reading from file /tmp/dns.pcap, link-type EN10MB (Ethernet)
12:31:10.201 IP 192.168.100.10.50211 > 1.1.1.1.53: 41022+ [1au] A? rockylinux.org. (55)
12:31:10.225 IP 1.1.1.1.53 > 192.168.100.10.50211: 41022 2/0/1 A 104.21.32.1, A
172.67.155.10
```

What just happened: `-w FILE` writes raw packets to a file instead of printing them, and `-r FILE` reads them back (reading a file needs no root). The file goes in `/tmp` because Ubuntu's tcpdump drops root rights to a user named `tcpdump` once capture starts, and that user cannot write into your home directory. The capture shows the DNS question (`A? rockylinux.org.`) sent to port 53 and the answer with two `A` records. Because you asked `1.1.1.1` directly, the packets left the VM; a normal lookup would go to the local stub first. tcpdump stops after 2 packets (one question, one answer). If it is still waiting because a packet was lost, press **Ctrl+C**.

Checkpoint: `iperf3` reported a bitrate, `sudo ethtool -i enp0s3` shows your driver, and `ls -l /tmp/dns.pcap` shows a non-empty capture file.

Challenge

CHALLENGE

1. On rocky-lab, create a **new** NetworkManager connection profile named `lab-static` for the same device with a static address, gateway, and DNS using `nmcli connection add`, activate it, test it, then switch back to the original profile and delete `lab-static`.
2. Using only `nc` on both VMs, transfer a file from rocky-lab to ubuntu-lab, and capture the transfer with `tcpdump` on ubuntu-lab using a port filter.
3. Write a Netplan file in your lab folder (not in `/etc/netplan`) that gives ubuntu-lab a static address with two DNS servers and a static route to `10.20.0.0/16` via `192.168.100.20`. Check its syntax by copying it to `/etc/netplan/`, running `sudo netplan generate`, and then removing it without applying.

Clean up

1. On ubuntu-lab, remove the test line from `/etc/hosts` and confirm the Netplan directory contains only the original file.

```
sudo sed -i '/fileserver/d' /etc/hosts
ls /etc/netplan/
```

2. If `60-lab.yaml` is still listed, remove it and apply.

```
sudo rm -f /etc/netplan/60-lab.yaml
sudo netplan apply
```

3. Make sure no lab interface or address is left, and remove the lab folder.

```
sudo ip link delete lab0 2>/dev/null
ip -br address
sudo rm -f /tmp/dns.pcap
rm -rf ~/labs/1.4
```

4. On rocky-lab, remove the temporary firewall ports and confirm the connection profile is back to DHCP.

```
# on rocky-lab
sudo firewall-cmd --remove-port=5000/tcp
sudo firewall-cmd --remove-port=5201/tcp
sudo firewall-cmd --list-ports
nmcli -f ipv4.method,ipv4.addresses,ipv4.dns connection show enp0s3
```

`--list-ports` should print an empty line, and the profile should show `auto`, `--`, `--`. If not, repeat Task 4, step 5.

5. Make sure no `iperf3` or `nc` process is still running on either VM. The installed tools can stay.

```
pgrep -a -x 'iperf3|nc|ncat|tcpdump' || echo "nothing running"
```

`pgrep -a` prints the process ID and full command line of matching processes, and `-x` requires the whole process name to match, so names that merely contain `nc` (such as `systemd-timesyncd`) are not listed. If something is listed, stop it with `kill PID`.

Exam corner

- `ip address` (addresses), `ip link` (interfaces, `set dev X up/down`), `ip route` (routes, `ip route add NET via GW`). `ip` changes are not persistent.
- Netplan: YAML in `/etc/netplan/`, spaces not tabs. `netplan try` applies with automatic rollback (default 120 s), `netplan apply` applies immediately, `netplan status` shows state, `netplan generate` builds backend files. Renderer is `networkd` (server) or `NetworkManager` (desktop).
- NetworkManager: `nmcli general status`, `nmcli device status`, `nmcli connection show|add|modify|up|down|delete`, `nmcli device reapply`. Profiles are `.nmconnection` keyfiles in `/etc/NetworkManager/system-connections/` ("nmconnect"). `nmtui` is the text menu.
- Name lookup order comes from the `hosts:` line in `/etc/nsswitch.conf` (`files dns`); `/etc/hosts` is checked first. `/etc/resolv.conf` lists `nameserver` and `search`; on Ubuntu it points to the systemd-resolved stub `127.0.0.53` (`resolvectl status` shows real servers).
- `dig` and `nslookup` query DNS directly and ignore `/etc/hosts`; `getent hosts` follows nsswitch. `dig @SERVER NAME TYPE`, `dig +short`, `dig -x IP` for reverse.
- `ping` / `ping6` test reachability; `arp -n` / `ip neigh` show IP-to-MAC mappings; `traceroute`, `tracepath` (no root, shows PMTU), and `mtr` (continuous) show the path.
- `ss -tulpn` lists listening TCP/UDP ports with processes (replaces `netstat`). `nc -l PORT` listens, `nc -zv HOST PORT` tests a port. `nmap HOST` scans ports: open, closed, filtered.

- `curl -I URL` shows HTTP headers and status code; `-L` follows redirects, `-o` saves output.
- `iperf3 -s` (server) and `iperf3 -c HOST` (client) measure throughput, default TCP port 5201.
- `ethtool IFACE` shows speed, duplex, link; `-i` driver; `-S` statistics; `-s` changes settings.
- `tcpdump -i IFACE -n FILTER`, `-c N` count, `-w file.pcap` write, `-r file.pcap` read.
- `hostname` shows the name, `hostname -I` the addresses; `hostnamectl set-hostname` changes it permanently.

Review questions

1. An administrator must change the IP address of a remote Ubuntu server over SSH and wants the change undone automatically if the connection is lost. Which command should be used?
   A. `netplan apply`
   B. `netplan try`
   C. `netplan generate`
   D. `ip address add`

2. `ping app01` works, but `dig app01` returns NXDOMAIN on a Rocky server. What is the most likely explanation?

3. Which command adds the DNS server `10.0.0.53` to the NetworkManager connection `ens192` so it survives reboots?
   A. `echo "nameserver 10.0.0.53" >> /etc/resolv.conf`
   B. `nmcli connection modify ens192 ipv4.dns 10.0.0.53`
   C. `ip route add dns 10.0.0.53`
   D. `resolvectl dns 10.0.0.53`

4. A web application cannot reach a database on port 5432. Which command, run on the database server, confirms that something is listening on that port and shows which process it is?
   A. `ss -tulpn`
   B. `arp -n`
   C. `tracepath db01`
   D. `ethtool -S eth0`

5. `nc -zv db01 5432` returns “Connection refused.” What does this tell you?

6. Which tool would you use to measure the maximum TCP throughput between two Linux servers?
   A. `mtr`
   B. `iperf3`
   C. `nmap`
   D. `curl`

Lab 1.5 Use Common Shell Operations

Objective: 1.5 Given a scenario, manage a Linux system using common shell operations.
Domain: 1.0 System Management (23% of the exam)
Time: about 150 minutes (you can split it at any task boundary)
VM: ubuntu-lab
Snapshot first: Optional. This lab only changes files in your home directory, and the clean-up restores them.

The shell is where a Linux administrator spends most of the day. In this lab you build a small practice dataset (a fake web server log and a server inventory in CSV format) and then use it to practise the everyday shell skills: moving around the file system, working with environment variables, customizing your prompt and `PATH`, redirecting input and output, slicing text with classic tools like `grep`, `awk`, and `sed`, reusing your command history, and editing files in `vim` and `nano`. The exam tests these skills with "which command would…" scenarios, so the goal is that each tool feels familiar.

What you'll practice

- Environment variables: `DISPLAY`, `HOME`, `PATH`, `PS1`, `SHELL`, `USER`
- Paths: absolute (`~`, `/`) and relative (`.`, `..`, `-`)
- Shell environment configs: `.bashrc`, `.bash_profile`, `.profile` (login and non-login shells)
- Redirection: `<`, `>`, `<<`, `>>`, `|`; standard output, standard error, standard input; here docs; here strings `<<<`
- Utilities: `!`, `!!`, `alias`, `awk`, `bc`, `cat`, `cut`, `echo`, `grep`, `head`, `history`, `less`, `more`, `printf`, `sed`, `sort`, `source`, `tail`, `tee`, `tr`, `uname`, `uniq`, `wc`, `xargs`
- Editors: `vi` / `vim` and `nano`

How it works

The shell. When you log in, Linux starts a program called a *shell*. The shell reads the line you type, works out what you mean (expanding `~`, `*`, and `$VARIABLES`), finds the program, and runs it. On Ubuntu and Rocky the default shell is **Bash** (`/bin/bash`). Everything in this lab is Bash.

Paths. Every file lives somewhere in one big tree that starts at `/` (the *root* directory). A path tells the shell where a file is:

- An **absolute** path starts at the root, so it always works no matter where you are: `/etc/hostname`, `/home/student`. The shortcut `~` means "my home directory", so `~/labs` is also absolute (the shell turns it into `/home/student/labs`).
- A **relative** path starts from your *current working directory* (where you are now). `.` means "this directory", `..` means "the parent directory", and `cd -` means "go back to the directory I was just in".

Think of an absolute path as a full postal address and a relative path as "two doors down from here".

Variables and the environment. The shell keeps named values called *variables*. Some of them are *environment variables*: they are copied into every program the shell starts, so programs can read them. Important ones:

Variable	Holds	Example
`HOME`	Your home directory	`/home/student`
`USER`	Your user name	`student`
`SHELL`	Your login shell	`/bin/bash`
`PATH`	Directories searched for commands, separated by `:`	`/usr/local/bin:/usr/bin:/bin...`
`PS1`	The text of your prompt	`\u@\h:\w\$`
`DISPLAY`	Which X11 graphical display GUI programs should draw on	`:0` or `localhost:10.0` (empty on a server)

When you type `ls`, the shell does not search the whole disk. It looks in each directory in `PATH`, left to right, and runs the first `ls` it finds. That is why a script in a directory that is not in `PATH` gives "command not found" until you type its path or add the directory to `PATH`.

Startup files. Variables you set at the prompt disappear when you log out. To make a setting permanent you put it in a *startup file* that Bash reads when it starts. Which file depends on the type of shell:

```
LOGIN shell (SSH login, console login, "bash -l")
  /etc/profile
  then the FIRST one that exists of:
    ~/.bash_profile  ->  ~/.bash_login  ->  ~/.profile
                                              |
                       (Ubuntu's ~/.profile sources ~/.bashrc)

NON-LOGIN interactive shell (typing "bash", a new terminal tab in a GUI)
  /etc/bash.bashrc
  ~/.bashrc
```

Ubuntu ships `~/.profile` and `~/.bashrc` for every new user but **no** `~/.bash_profile`. Its `~/.profile` runs `~/.bashrc`, so both kinds of shell end up reading `~/.bashrc`. The usual rule is: put **environment variables** such as `PATH` in `~/.profile` (read once at login, then inherited by every child process), and put **aliases, prompt settings, and functions** in `~/.bashrc` (read by every interactive shell, because aliases are *not* inherited). If you create `~/.bash_profile`, Bash reads it *instead of* `~/.profile`, which surprises many people. You will prove that in Task 4. The `source` command (or its short form `.`) reads a file into the *current* shell, so you can apply changes without logging out.

Streams and redirection. Every program starts with three open "pipes" called *streams*:

```
                    +-------------+  stdout (1) --> screen
keyboard --> stdin  |   program   |
             (0)    +-------------+  stderr (2) --> screen
```

- **standard input** (stdin, number 0): where the program reads input. Normally the keyboard.
- **standard output** (stdout, number 1): normal results. Normally the screen.

- **standard error** (stderr, number 2): error messages. Also the screen, but a separate stream.

Redirection reconnects these streams to files: `>` sends stdout to a file (replacing it), `>>` appends, `2>` sends stderr, `<` feeds a file into stdin, `<<` (a *here doc*) feeds lines you type, and `<<<` (a *here string*) feeds one string. A *pipe* `|` connects the stdout of one program to the stdin of the next, which lets you build a "pipeline" of small tools that each do one job. That idea, small tools joined by pipes, is the heart of Unix text processing.

Editors. Sooner or later you will edit a config file on a server with no GUI. `nano` is simple and shows its shortcuts on screen. `vi` / `vim` is on almost every Unix system ever made, is very powerful, and is *modal*: keys mean different things in different modes. You will practise both.

Tasks

Task 1: Prepare the lab and build a practice dataset

1. Log in to ubuntu-lab as `student` (over SSH or on the console). Install the tools this lab uses. Most are already present; `apt` will say so.

```
sudo apt update
sudo apt install -y bc vim nano less
```

> **What just happened:** `apt update` refreshes the list of available packages, and `apt install -y` installs the named packages, answering "yes" to the confirmation. `bc` is a command-line calculator, `vim` is the improved `vi` editor, `nano` is a simple editor, and `less` is a pager (a program that shows long text one screen at a time). Tools like `grep`, `sed`, `awk`, `sort`, and `cut` are part of every Ubuntu install.

2. Create the lab working directory and move into it.

```
mkdir -p ~/labs/1.5 && cd ~/labs/1.5
```

`mkdir -p` creates the directory and any missing parent directories without complaining if they already exist. `&&` means "run the second command only if the first one succeeded".

3. Create a fake web server access log using a **here doc**. Copy the whole block, including the final `EOF` line.

```
cat > access.log <<'EOF'
192.168.100.20 - - [10/Oct/2026:08:01:12 +0000] "GET /index.html HTTP/1.1" 200 5120
192.168.100.20 - - [10/Oct/2026:08:01:13 +0000] "GET /style.css HTTP/1.1" 200 1830
10.0.0.15 - - [10/Oct/2026:08:02:40 +0000] "GET /login HTTP/1.1" 200 2310
10.0.0.15 - - [10/Oct/2026:08:02:55 +0000] "POST /login HTTP/1.1" 302 0
203.0.113.7 - - [10/Oct/2026:08:03:01 +0000] "GET /admin HTTP/1.1" 403 512
203.0.113.7 - - [10/Oct/2026:08:03:02 +0000] "GET /wp-login.php HTTP/1.1" 404 320
203.0.113.7 - - [10/Oct/2026:08:03:03 +0000] "GET /.env HTTP/1.1" 404 320
10.0.0.22 - - [10/Oct/2026:08:05:10 +0000] "GET /index.html HTTP/1.1" 200 5120
10.0.0.22 - - [10/Oct/2026:08:05:11 +0000] "GET /images/logo.png HTTP/1.1" 200 20480
192.168.100.20 - - [10/Oct/2026:08:07:45 +0000] "GET /reports HTTP/1.1" 500 128
10.0.0.15 - - [10/Oct/2026:08:09:30 +0000] "GET /dashboard HTTP/1.1" 200 8800
203.0.113.7 - - [10/Oct/2026:08:10:00 +0000] "GET /phpmyadmin HTTP/1.1" 404 320
198.51.100.4 - - [10/Oct/2026:08:12:19 +0000] "GET /index.html HTTP/1.1" 200 5120
198.51.100.4 - - [10/Oct/2026:08:12:20 +0000] "GET /style.css HTTP/1.1" 304 0
10.0.0.22 - - [10/Oct/2026:08:15:42 +0000] "GET /reports HTTP/1.1" 500 128
EOF
```

What just happened: `cat > access.log` means "run `cat` and send its standard output into the file `access.log`". Normally `cat` would read from the keyboard, but `<<'EOF'` starts a **here doc**: the shell collects every following line until it sees a line containing only `EOF`, and feeds those lines to `cat` as standard input. `cat` copies them to its output, which lands in the file. `EOF` is just a conventional marker word; any word works. The quotes around `'EOF'` tell the shell *not* to expand `$variables` or backticks inside the text, so it is copied exactly. Each log line has: client IP, two unused fields (`-`), a timestamp, the HTTP request, the status code (200 = OK, 404 = not found, 500 = server error), and the number of bytes sent.

4. Create a server inventory in CSV (comma-separated values) format the same way.

```
cat > servers.csv <<'EOF'
hostname,os,ram_gb,role,site
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
db01,rocky,16,database,paris
db02,rocky,16,database,berlin
mon01,ubuntu,2,monitoring,paris
app01,debian,8,app,berlin
app02,ubuntu,8,app,london
EOF
```

5. See the difference between a quoted and an unquoted here doc marker.

```
cat <<EOF
Unquoted: my home is $HOME
EOF
cat <<'EOF'
Quoted: my home is $HOME
EOF
```

```
Unquoted: my home is /home/student
Quoted: my home is $HOME
```

What just happened: With an unquoted marker the shell expands variables inside the here doc, which is handy for generating config files that contain the current user or hostname. With a quoted marker the text is taken literally. When you write scripts or config files that contain `$`, quote the marker.

6. Check which system you are on with `uname`.

```
uname -a
uname -r
uname -m
```

```
Linux ubuntu-lab 6.8.0-45-generic #45-Ubuntu SMP PREEMPT_DYNAMIC ... x86_64 x86_64 x86_64
GNU/Linux
6.8.0-45-generic
x86_64
```

What just happened: `uname` prints system information. `-a` prints all of it: kernel name (`Linux`), host name, kernel release, kernel build version, machine hardware type, and operating system. `-r` prints only the kernel release, which you check before installing drivers or after a kernel update. `-m` prints the CPU architecture (`x86_64` or `aarch64`). Other options: `-s` kernel name, `-n` host name, `-o` operating system. Your kernel number will differ.

What just happened: You now have two small but realistic files to practise on. `access.log` imitates the log an Apache or Nginx web server writes to `/var/log/apache2/access.log` or `/var/log/nginx/access.log`, and `servers.csv` imitates the kind of inventory export you get from a spreadsheet or a monitoring system. Every command in the rest of the lab works the same way on real files, only with more lines.

Checkpoint: `ls -l ~/labs/1.5` shows `access.log` and `servers.csv`. `wc -l access.log servers.csv` reports 15 and 8 lines.

Task 2: Navigate with absolute and relative paths

1. Find out where you are, then jump around using absolute paths.

```
pwd
cd /
pwd
cd /var/log
pwd
cd ~
pwd
```

```
/home/student/labs/1.5
/
/var/log
/home/student
```

What just happened: `pwd` (print working directory) shows your current directory. `cd /` moves to the root of the tree. `cd /var/log` uses an absolute path: it starts with `/`, so it works from anywhere. `cd ~` goes home; plain `cd` with no argument does the same thing.

2. Now use relative paths.

```
cd labs/1.5
pwd
cd ..
pwd
cd ./1.5
pwd
cd ../..
pwd
```

```
/home/student/labs/1.5
/home/student/labs
/home/student/labs/1.5
/home/student
```

What just happened: `labs/1.5` has no leading `/`, so the shell looks for `labs` inside the current directory (`/home/student`). `..` is the parent directory, so `cd ..` goes up one level, and `../..` goes up two. `./1.5` means "`1.5` in this directory"; the `./` is optional for `cd` but required when you run a program from the current directory (for example `./myscript.sh`), because `.` is not in `PATH`.

3. Use `cd -` to bounce between two directories.

```
cd /etc
cd ~/labs/1.5
cd -
cd -
```

```
/etc
/home/student/labs/1.5
```

What just happened: `cd -` switches to the previous directory and prints its name. Bash stores the previous directory in the variable `OLDPWD`. This is very useful when you are editing a config in `/etc` and checking results in another directory.

4. See `.` and `..` as real directory entries, and use `~` with another user name.

```
ls -a ~/labs/1.5
ls -ld ~root
```

```
.  ..  access.log  servers.csv
drwx------ 5 root root 4096 Oct 10 07:55 /root
```

What just happened: `ls -a` shows all entries, including hidden ones whose names start with a dot. Every directory contains `.` (itself) and `..` (its parent). `~root` expands to the home directory of the user `root`, which is `/root`. `ls -ld` shows the directory itself instead of its contents.

Checkpoint: Starting from `/tmp`, you can reach `~/labs/1.5` with one absolute `cd` and get back to `/tmp` with `cd -`. Try it, and run `pwd` after each step.

Task 3: Explore environment variables

1. Print the key variables.

```
echo "$HOME"
echo "$USER"
echo "$SHELL"
echo "$PATH"
echo "$PS1"
```

```
/home/student
student
/bin/bash
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/sn
ap/bin
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:...
```

What just happened: `echo` prints its arguments. A `$` in front of a name tells the shell to replace it with the variable's value *before* `echo` runs. The double quotes keep the value as one argument even if it contains spaces. `PATH` is a list of directories separated by colons. Ubuntu's `PS1` looks scary because it contains color codes and a window-title code; you will replace it with something readable in Task 4. Your `PATH` may also contain `/home/student/.local/bin` or `/home/student/bin` if those directories existed when you logged in.

2. List all environment variables and pick out a few.

```
printenv | sort | head -n 15
printenv HOME
env | grep -E '^(USER|SHELL|LANG)='
```

What just happened: `printenv` (and `env` with no arguments) prints every *environment* variable as `NAME=value`. You piped the output into `sort` to alphabetize it and `head -n 15` to see only the first 15 lines. `printenv HOME` prints one variable without the `$`. `grep -E` searches using an extended regular expression; `^` means "start of line", so this matches lines that begin with exactly `USER=`, `SHELL=`, or `LANG=`.

3. Learn the difference between a shell variable and an environment variable.

```
COLOR=blue
echo "$COLOR"
bash -c 'echo "child shell sees: [$COLOR]"'
export COLOR
bash -c 'echo "child shell sees: [$COLOR]"'
unset COLOR
```

```
blue
child shell sees: []
child shell sees: [blue]
```

What just happened: `COLOR=blue` (no spaces around `=`) creates a *shell variable* that only the current shell knows. `bash -c '...'` starts a child shell that runs one command; it saw nothing because shell variables are not passed to children. `export COLOR` marks the variable for export, turning it into an *environment variable*, so the child now sees it. You can also do both in one step: `export COLOR=blue`. `unset` deletes the variable. This is why `PATH` must be exported (it already is): programs started by your shell need it.

4. Look at `DISPLAY`. ubuntu-lab is a server with no graphical desktop.

```
echo "DISPLAY is: [$DISPLAY]"
```

```
DISPLAY is: []
```

What just happened: `DISPLAY` tells graphical (X11) programs where to draw their windows. The format is `host:display.screen`; `:0` means "the first display on this machine". On a server with no GUI it is empty, so a graphical program such as `xeyes` or `virt-manager` would fail with "cannot open display". The fix is *X11 forwarding*: if you connect with `ssh -X student@ubuntu-lab` from a computer that runs an X server (a Linux desktop, macOS with XQuartz, or Windows with an X server such as VcXsrv or WSLg), SSH sets `DISPLAY` to something like `localhost:10.0` and tunnels the windows back to your screen. The server needs `X11Forwarding yes` in `/etc/ssh/sshd_config` (Ubuntu's default) and the `xauth` package.

TIP

Explore/simulate: If your workstation has an X server, try `ssh -X student@ubuntu-lab`, then `echo $DISPLAY` (you should see `localhost:10.0`), then `sudo apt install -y x11-apps` and `xeyes`. Without an X server, just check the server side: `grep -i x11forwarding /etc/ssh/sshd_config` shows `X11Forwarding yes`. Remove `x11-apps` afterwards if you installed it.

Checkpoint: You can explain why `bash -c 'echo $COLOR'` printed nothing until you ran `export`, and `echo $DISPLAY` prints an empty line on ubuntu-lab.

Task 4: Customize your shell with PS1, PATH, aliases, and startup files

In this task you change your prompt and `PATH`, first temporarily, then permanently. Start by backing up the two startup files so the clean-up can restore them exactly.

1. Back up your startup files.

```
cp ~/.bashrc ~/.bashrc.lab15.bak
cp ~/.profile ~/.profile.lab15.bak
ls ~/.bash_profile
```

```
ls: cannot access '/home/student/.bash_profile': No such file or directory
```

   The error is expected: Ubuntu does not create `~/.bash_profile`. (If yours exists, back it up too.)

2. Change the prompt for the current shell only.

```
PS1='[\u@\h \W]\$ '
```

   Your prompt becomes `[student@ubuntu-lab 1.5]$`. Now try a two-line prompt with the time:

```
PS1='\t \u@\h:\w\n\$ '
```

   What just happened: `PS1` is the *primary prompt string*. Bash replaces special backslash codes each time it draws the prompt: `\u` user name, `\h` short host name, `\w` full current directory (with `~` for home), `\W` only the last part of the directory, `\t` time (24-hour HH:MM:SS), `\n` a new line, and `\$` shows `$` for normal users and `#` for root. Single quotes stop the shell from interpreting the backslashes when you set the variable. Because you did not save it anywhere, the change disappears when this shell exits.

3. Make a personal tools directory with a small script and try to run it.

```
mkdir -p ~/labtools
cat > ~/labtools/hello-lab <<'EOF'
#!/bin/bash
echo "Hello from $(hostname), I am running as $USER"
EOF
chmod +x ~/labtools/hello-lab
hello-lab
```

```
hello-lab: command not found
```

   What just happened: You wrote a two-line script. `#!/bin/bash` (the “shebang”) tells the kernel which program runs the file, and `chmod +x` makes it executable. Typing `hello-lab` fails because the shell only searches the directories in `PATH`, and `~/labtools` is not one of them. You could still run it with its full path: `~/labtools/hello-lab`.

4. Add the directory to `PATH` for this shell and test it.

```
export PATH="$PATH:$HOME/labtools"
hello-lab
which hello-lab
```

```
Hello from ubuntu-lab, I am running as student
/home/student/labtools/hello-lab
```

What just happened: `"$PATH:$HOME/labtools"` builds a new value from the old `PATH` plus a colon and your directory, so nothing existing is lost. Putting your directory at the *end* means system commands win if names clash; putting it at the *front* (`"$HOME/labtools:$PATH"`) would let your version override a system command. Never replace `PATH` with only your directory, or commands like `ls` will stop working in that shell. `which` shows which file the shell would run.

5. Make the changes permanent. `PATH` belongs in `~/.profile`; the prompt and aliases belong in `~/.bashrc`. The `>>` operator *appends* to a file instead of overwriting it, which is critical here: a single `>` would wipe out your existing `.bashrc`.

```
cat >> ~/.profile <<'EOF'

# >>> lab 1.5 >>>
export PATH="$PATH:$HOME/labtools"
# <<< lab 1.5 <<<
EOF
```

```
cat >> ~/.bashrc <<'EOF'

# >>> lab 1.5 >>>
PS1='[\t] \u@\h:\w\$ '
alias ll='ls -alF'
alias lab='cd ~/labs/1.5'
alias grep='grep --color=auto'
# <<< lab 1.5 <<<
EOF
tail -n 8 ~/.bashrc
```

What just happened: You appended a marked block to each file. The marker comments make the block easy to find and remove later. An `alias` is a short name that the shell replaces with longer text: after this, typing `ll` runs `ls -alF`. Ubuntu's `~/.bashrc` already defines `ll`, `la`, `l`, and a colorized `grep`; defining them again is harmless, and the later definition wins. You added your lines at the *end* of `~/.bashrc` on purpose: near the top, Ubuntu's file stops early ("return") for non-interactive shells, so settings at the end only affect interactive shells, which is what you want for prompts and aliases. You could make the same edits with `nano ~/.bashrc`; the here doc just avoids typos.

6. Apply the changes without logging out by sourcing the files.

```
source ~/.bashrc
alias
lab
pwd
```

```
...
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias lab='cd ~/labs/1.5'
alias ll='ls -alF'
alias ls='ls --color=auto'
...
/home/student/labs/1.5
```

What just happened: `source ~/.bashrc` runs the file's commands *inside the current shell*, so the new prompt and aliases take effect right now. (Running `bash ~/.bashrc` would not work: it would apply them in a child shell that exits immediately.) `. ~/.bashrc` is the same as `source ~/.bashrc`. `alias` with no arguments lists all aliases; `alias ll` shows one; `unalias name` removes one; and `\ls` or `command ls` runs the real command, skipping any alias. Your prompt now looks like `[14:05:31] student@ubuntu-lab:~/labs/1.5$`.

7. Tell login shells from non-login shells.

```
echo $0
shopt login_shell
bash
echo $0
shopt login_shell
exit
```

```
-bash
login_shell     on
bash
login_shell     off
```

What just happened: Your SSH or console session is a *login shell*. Bash shows this by putting a dash in front of its name (`-bash`) and setting the `login_shell` option. Typing `bash` started a *non-login* interactive shell inside it, which reads only `/etc/bash.bashrc` and `~/.bashrc`. `exit` returned you to the login shell. `bash -l` (or `su - username`) starts a login shell, which reads `/etc/profile` and then `~/.bash_profile`, `~/.bash_login`, or `~/.profile`.

8. Prove that `~/.bash_profile` hides `~/.profile`. Create a one-line `~/.bash_profile`, then start a login shell and look for your `lab` alias (which only reaches login shells because `~/.profile` sources `~/.bashrc`).

```
echo 'echo "~/.bash_profile was read"' > ~/.bash_profile
bash -l
type lab
exit
```

```
~/.bash_profile was read
bash: type: lab: not found
```

Now fix it the standard way, by making `~/.bash_profile` read `~/.profile`:

```
echo '[ -f ~/.profile ] && . ~/.profile' >> ~/.bash_profile
bash -l
type lab
exit
rm ~/.bash_profile
```

```
~/.bash_profile was read
lab is aliased to `cd ~/labs/1.5'
```

What just happened: When `~/.bash_profile` exists, a login Bash reads it and **skips** `~/.profile` entirely, so Ubuntu's chain (`.profile` sources `.bashrc`) never ran and your alias was missing. Adding `[ -f ~/.profile ] && . ~/.profile` ("if the file exists, source it") restored the chain. Rocky Linux uses `~/.bash_profile` (which sources `~/.bashrc`) instead of `~/.profile`. You deleted the test file so your account is back to Ubuntu's normal layout. `type` tells you what a name is: an alias, a shell builtin, a function, or a file on disk.

9. Confirm the `PATH` change works in a fresh login. Log out (`exit`) and log back in over SSH, then:

```
echo "$PATH" | tr ':' '\n' | tail -n 3
hello-lab
```

```
/usr/local/games
/snap/bin
/home/student/labtools
Hello from ubuntu-lab, I am running as student
```

TIP

Ubuntu's `~/.profile` already adds `~/bin` and `~/.local/bin` to `PATH` at login **if those directories exist**. So the simplest way to get a personal script directory on Ubuntu is `mkdir ~/bin`, then log out and back in. This lab used `~/labtools` so you could see the whole process.

ON THE EXAM

On the exam, remember: `~/.bashrc` is for non-login interactive shells (aliases, prompt), `~/.bash_profile` or `~/.profile` is for login shells (environment variables), and `source file` (or `. file`) applies a file to the current shell. System-wide versions live in `/etc/profile`, `/etc/profile.d/*.sh`, and `/etc/bash.bashrc` (Rocky: `/etc/bashrc`).

Checkpoint: After a fresh login, your prompt starts with the time, `ll` and `lab` work, and `hello-lab` runs from any directory. `ls ~/.bash_profile` reports "No such file or directory".

Task 5: Redirect standard input, output, and error

1. Run a command that produces both normal output and an error.

```
cd ~/labs/1.5
ls /etc/hostname /nope
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
```

Both lines appear on the screen, but they come from different streams: the error from **standard error** and the file name from **standard output**.

2. Send each stream to its own file.

```
ls /etc/hostname /nope > out.txt
cat out.txt
ls /etc/hostname /nope 2> err.txt
cat err.txt
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
/etc/hostname
ls: cannot access '/nope': No such file or directory
```

What just happened: `>` is short for `1>`: it redirects stream 1 (stdout) to `out.txt`, so only the error still reached your screen. `2>` redirects stream 2 (stderr) to `err.txt`, so only the normal output reached the screen. A single `>` creates the file or **empties it first** if it already exists.

3. Append instead of overwrite.

```
echo "first line" > notes.txt
echo "second line" >> notes.txt
echo "oops" > notes.txt
cat notes.txt
```

```
oops
```

The third command used `>` and wiped out the first two lines. Use `>>` whenever you want to keep what is already in a file (log files, `~/.bashrc`).

> **TIP**
> `set -o noclobber` makes Bash refuse to overwrite an existing file with `>` (you then need `>|` to force it). Some admins add it to `~/.bashrc` as a safety net. Turn it off with `set +o noclobber`.

4. Send both streams to one file, and learn why order matters.

```
ls /etc/hostname /nope > all.txt 2>&1
cat all.txt
ls /etc/hostname /nope &> all2.txt
cat all2.txt
ls /etc/hostname /nope 2>&1 > wrong.txt
```

```
ls: cannot access '/nope': No such file or directory
/etc/hostname
ls: cannot access '/nope': No such file or directory
/etc/hostname
ls: cannot access '/nope': No such file or directory
```

> **What just happened:** `2>&1` means "send stream 2 to wherever stream 1 is going *right now*". The shell processes redirections left to right. In the first command, stdout was already pointing to `all.txt`, so stderr followed it there. In the last command, `2>&1` came first, while stdout still pointed at the screen, so stderr went to the screen and only stdout went to `wrong.txt`. `&>` is a Bash shortcut for "both streams to this file" (`&>>` appends both).

5. Throw away output you do not want with `/dev/null`.

```
find /etc -name "*.conf" 2> /dev/null | head -n 5
grep -r "PermitRootLogin" /etc/ssh 2>/dev/null
ls /nope &> /dev/null
echo "exit status: $?"
```

> **What just happened:** `/dev/null` is a special file that discards everything written to it (the "bit bucket"). Running `find` as a normal user produces many "Permission denied" errors; `2> /dev/null` hides them so you see only real results. The last `ls` hid all output, but you can still check whether it worked: `$?` holds the *exit status* of the previous command, `0` for success and non-zero (here `2`) for failure. Scripts use this pattern constantly: "run quietly and just tell me if it worked".

6. Feed a file to standard input with `<`, and a single string with a **here string** `<<<`.

```
wc -l < access.log
tr 'a-z' 'A-Z' < servers.csv | head -n 2
wc -w <<< "one two three"
bc <<< "2^10"
```

```
15
HOSTNAME,OS,RAM_GB,ROLE,SITE
WEB01,UBUNTU,4,WEB,LONDON
3
1024
```

What just happened: `< access.log` connects the file to the program's standard input. `wc -l access.log` would print `15 access.log`, but `wc -l < access.log` prints just `15` because `wc` never learns the file name, which is handy in scripts. `tr` only reads standard input (it cannot take a file name), so `<` is the normal way to feed it. `<<<` passes one string, followed by a newline, as standard input: a one-line version of a here doc.

7. Pipes carry only standard output.

```
ls /nope | wc -l
ls /nope 2>&1 | wc -l
```

```
ls: cannot access '/nope': No such file or directory
0
1
```

What just happened: The pipe `|` connects stdout of `ls` to stdin of `wc`. The error went to stderr, straight to your screen, so `wc` counted 0 lines. With `2>&1` before the pipe, stderr joins stdout and flows into `wc`. Bash also accepts `|&` as a shortcut for `2>&1 |`.

8. Use `tee` to save output *and* keep it flowing down the pipeline.

```
grep ' 404 ' access.log | tee 404s.txt | wc -l
cat 404s.txt
date | tee -a run.log
date | tee -a run.log
cat run.log
```

```
3
203.0.113.7 - - [10/Oct/2026:08:03:02 +0000] "GET /wp-login.php HTTP/1.1" 404 320
...
```

What just happened: `tee` is named after a T-shaped pipe fitting: it copies its standard input to a file *and* to its standard output. Here the 404 lines were saved in `404s.txt` while `wc -l` counted them. `tee -a` appends instead of overwriting.

9. Use `tee` to write a file that needs root permission.

```
sudo echo "lab 1.5 was here" > /opt/lab15.txt
echo "lab 1.5 was here" | sudo tee /opt/lab15.txt
cat /opt/lab15.txt
```

```
-bash: /opt/lab15.txt: Permission denied
lab 1.5 was here
lab 1.5 was here
```

What just happened: The first command failed even with `sudo`. The redirection `>` is performed by *your* shell, running as `student`, before `sudo` even starts; only `echo` ran as root. In the second command, `tee` itself runs as root through `sudo`, so it can write the file. This `| sudo tee file` pattern (or `| sudo tee -a file` to append) is how you write to protected files from a pipeline, and it is the pattern this book uses with here docs: `cat <<'EOF' | sudo tee /etc/...`.

Checkpoint: `cat all.txt` shows both the error and the file name; `cat wrong.txt` shows only `/etc/hostname`; `cat /opt/lab15.txt` shows `lab 1.5 was here`.

Task 6: View and filter text with cat, less, more, head, tail, grep, cut, sort, uniq, and wc

1. View files in different ways.

```
cat servers.csv
cat -n servers.csv
head -n 3 access.log
tail -n 2 access.log
tail -n +2 servers.csv
```

What just happened: `cat` (concatenate) prints whole files; with several files it joins them (`cat a b > c`). `-n` numbers the lines. `head -n 3` shows the first 3 lines (default 10), and `tail -n 2` the last 2. `tail -n +2` is special: it means "start from line 2", which is the standard way to skip a CSV header row.

2. Page through long output with `less` and `more`.

```
less /etc/services
```

Inside `less`: press Space or Page Down to go forward, `b` to go back, `g` for the top, `G` for the bottom, type `/ssh` and press Enter to search, `n` for the next match, `N` for the previous, and `q` to quit. Now try `more`:

```
more /etc/services
```

Press Space for the next page, Enter for the next line, and `q` to quit.

What just happened: `more` is the original pager; it mainly moves forward. `less` is newer and can scroll both ways and search ("less is more"). Commands like `man` and `systemctl` use `less` to show long output. Pipe any long output into it: `ps aux | less`.

3. Watch a growing file with `tail -f`. Start a background job that appends a line every 2 seconds, then follow the file:

```
(for i in 1 2 3 4 5; do echo "event $i"; sleep 2; done) >> live.log &
tail -f live.log
```

Watch the events appear, then press Ctrl+C to stop `tail`.

> **What just happened:** The parentheses run the loop in a subshell, `>> live.log` appends its output to the file, and `&` runs it in the background so you get your prompt back. `tail -f` (follow) prints the end of the file and then waits, printing new lines as they arrive. This is how admins watch logs live, for example `sudo tail -f /var/log/syslog`.

4. Search with `grep`.

```
grep '404' access.log
grep -c ' 404 ' access.log
grep -v ' 200 ' access.log | wc -l
grep -i 'post' access.log
grep -n 'reports' access.log
grep -E ' (403|404) ' access.log | wc -l
```

```
...
3
8
10.0.0.15 - - [10/Oct/2026:08:02:55 +0000] "POST /login HTTP/1.1" 302 0
10:192.168.100.20 - - [10/Oct/2026:08:07:45 +0000] "GET /reports HTTP/1.1" 500 128
15:10.0.0.22 - - [10/Oct/2026:08:15:42 +0000] "GET /reports HTTP/1.1" 500 128
4
```

> **What just happened:** `grep pattern file` prints lines that contain the pattern. `-c` counts matching lines instead of printing them. The spaces in `' 404 '` stop `404` from matching inside a byte count or timestamp. `-v` inverts the match (lines that do *not* contain `200`). `-i` ignores case. `-n` adds line numbers. `-E` enables extended regular expressions, where `(403|404)` means "403 or 404". Other useful options: `-r` searches directories recursively, `-w` matches whole words only, and `-l` lists only file names.

5. Cut out columns with `cut`.

```
cut -d, -f1,3 servers.csv | head -n 4
cut -d' ' -f1 access.log | head -n 3
```

```
hostname,ram_gb
web01,4
web02,4
db01,16
192.168.100.20
192.168.100.20
10.0.0.15
```

What just happened: `cut -d, -f1,3` splits each line at commas (`-d` sets the delimiter) and keeps fields 1 and 3. In the log, fields are separated by spaces, so `-d' '` with `-f1` extracts the client IP. `cut` treats every single delimiter as a boundary, so it is perfect for CSV but awkward when columns are separated by a varying number of spaces; that is a job for `awk` (next task). `cut -c1-10` cuts by character position instead.

6. Sort, de-duplicate, and count: the most common log analysis pipeline. Which IP address made the most requests?

```
cut -d' ' -f1 access.log | sort | uniq -c | sort -rn
```

```
      4 203.0.113.7
      3 192.168.100.20
      3 10.0.0.22
      3 10.0.0.15
      2 198.51.100.4
```

What just happened: Read the pipeline left to right. `cut` extracts the IPs. `sort` puts identical IPs next to each other. `uniq -c` collapses adjacent identical lines into one and prefixes a count; `uniq` only compares *neighboring* lines, which is why you must `sort` first. The final `sort -rn` sorts numerically (`-n`) in reverse (`-r`), so the busiest client is on top. Lines with equal counts may appear in a different order on your system. `203.0.113.7` is the client probing for `/admin`, `/.env`, and `/phpmyadmin`: an attacker scanning for weaknesses.

7. More sorting and counting.

```
tail -n +2 servers.csv | sort -t, -k3,3n | head -n 3
tail -n +2 servers.csv | cut -d, -f5 | sort -u
tail -n +2 servers.csv | cut -d, -f2 | sort | uniq -c
wc access.log
wc -l servers.csv
```

```
mon01,ubuntu,2,monitoring,paris
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
berlin
london
paris
      1 debian
      2 rocky
      4 ubuntu
  15  150 1181 access.log
8 servers.csv
```

What just happened: `sort -t, -k3,3n` sorts by field 3 only (`-k3,3`), using a comma as the field separator (`-t,`), numerically (`n`). Without `n`, `16` would sort before `2` because text is compared character by character. `sort -u` sorts and removes duplicates in one step, the same as `sort | uniq`. `wc` (word count) prints lines, words, and bytes; `-l`, `-w`, and `-c` pick one.

Checkpoint: Using one pipeline, list each HTTP status code with how many times it appears, most common first. (Hint: the status is field 9 when you split on spaces: `cut -d' ' -f9 access.log | sort | uniq -c | sort -rn`.) You should see `7 200` on the first line.

Task 7: Transform text with awk, sed, tr, printf, bc, and xargs

1. Use `awk` to work with columns and do arithmetic.

```
awk '{print $1, $9}' access.log | head -n 3
awk '$9 >= 500 {print $4, $7}' access.log
awk '{sum += $10} END {print "Total bytes:", sum}' access.log
awk -F, 'NR > 1 {ram += $3} END {print "Total RAM:", ram, "GB"}' servers.csv
```

```
192.168.100.20 200
192.168.100.20 200
10.0.0.15 200
[10/Oct/2026:08:07:45 /reports
[10/Oct/2026:08:15:42 /reports
Total bytes: 50508
Total RAM: 58 GB
```

What just happened: `awk` is a small programming language for column-based text. By default it splits each line on runs of spaces or tabs into fields `$1`, `$2`, … (`$0` is the whole line, `NF` is the number of fields, `NR` is the current line number). A program has the form `pattern {action}`: the action runs for every line that matches the pattern. `$9 >= 500` selects server errors. `sum += $10` adds up the bytes column, and the `END` block runs once after the last line. `-F,` sets the field separator to a comma, and `NR > 1` skips the header line. Ubuntu's `awk` is `mawk`; Rocky's is `gawk`. Both run these examples.

2. Use `sed` (stream editor) to edit text on the fly.

```
sed 's/ubuntu/Ubuntu/' servers.csv | head -n 3
sed -n '2,4p' servers.csv
sed '1d' servers.csv | head -n 2
sed '/rocky/d' servers.csv
```

```
hostname,os,ram_gb,role,site
web01,Ubuntu,4,web,london
web02,Ubuntu,4,web,london
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
db01,rocky,16,database,paris
web01,ubuntu,4,web,london
web02,ubuntu,4,web,london
hostname,os,ram_gb,role,site
web01,ubuntu,4,web,london
...
```

What just happened: `sed 's/old/new/'` substitutes the first match of `old` on each line; add `g` (`s/old/new/g`) to replace every match on the line. `sed -n '2,4p'` turns off automatic printing (`-n`) and prints (`p`) only lines 2 to 4. `1d` deletes line 1. `/rocky/d` deletes every line containing `rocky`. None of these changed the file: `sed` writes to standard output unless you ask it to edit in place.

3. Edit a file in place with `sed -i`, keeping a backup.

```
cp servers.csv servers-copy.csv
sed -i.bak 's/debian/ubuntu/g' servers-copy.csv
grep app01 servers-copy.csv servers-copy.csv.bak
```

```
servers-copy.csv:app01,ubuntu,8,app,berlin
servers-copy.csv.bak:app01,debian,8,app,berlin
```

What just happened: `-i` edits the file in place. `-i.bak` first saves the original with a `.bak` suffix, a good habit when changing config files with `sed` (for example `sudo sed -i.bak 's/^#Port 22/Port 2222/' /etc/ssh/sshd_config`). When `grep` searches more than one file, it prefixes each match with the file name.

4. Translate or delete characters with `tr`.

```
head -n 2 servers.csv | tr ',' '\t'
echo "Hello World" | tr 'a-z' 'A-Z'
echo "too    many     spaces" | tr -s ' '
echo "phone: 555-0142" | tr -d -c '0-9\n'
```

```
hostname    os  ram_gb  role    site
web01   ubuntu  4   web london
HELLO WORLD
too many spaces
5550142
```

What just happened: `tr set1 set2` replaces each character in set 1 with the matching character in set 2: commas become tabs (`\t`), lowercase becomes uppercase. `-s` squeezes runs of a repeated character into one. `-d` deletes characters, and `-c` complements the set, so `-d -c '0-9\n'` deletes everything that is *not* a digit or newline. `tr` works on characters, not words: `tr 'cat' 'dog'` changes every `c` to `d`, every `a` to `o`, and every `t` to `g`.

5. Format output with `printf` and compare it with `echo`.

```
echo "Server: web01"
echo -n "no newline here"; echo " <- same line"
echo -e "col1\tcol2"
printf "%-8s %-10s %3d GB\n" web01 web 4
awk -F, 'NR > 1 {printf "%-8s %-10s %3d GB\n", $1, $4, $3}' servers.csv
```

```
Server: web01
no newline here <- same line
col1    col2
web01    web          4 GB
web01    web          4 GB
web02    web          4 GB
db01     database    16 GB
db02     database    16 GB
mon01    monitoring   2 GB
app01    app          8 GB
app02    app          8 GB
```

What just happened: `echo` prints its arguments and a newline. `-n` drops the newline; `-e` makes it understand escapes like `\t` (tab) and `\n`. `printf` takes a *format string* and never adds a newline unless you write `\n`. In the format, `%s` is a string, `%d` an integer, `%-8s` a string left-aligned in 8 characters, and `%3d` a number right-aligned in 3 characters. That is how you produce neat columns. `awk` has its own `printf` with the same rules.

6. Calculate with `bc`.

```
echo "10 / 3" | bc
echo "scale=2; 10 / 3" | bc
bc <<< "scale=2; 50508 / 15"
bc -l <<< "sqrt(2)"
```

```
3
3.33
3367.20
1.41421356237309504880
```

What just happened: `bc` is an arbitrary-precision calculator that reads expressions from standard input. By default it does integer division, so `10 / 3` is `3`. `scale=2` sets two decimal places. The third line computes the average bytes per request from the total you got with `awk`. `-l` loads the math library (which also sets `scale=20`). Run `bc` alone for an interactive calculator, and type `quit` to leave. Bash itself only does integer math: `echo $((10 / 3))` prints `3`.

7. Turn lines into arguments with `xargs`.

```
tail -n +2 servers.csv | cut -d, -f1 | xargs echo
tail -n +2 servers.csv | cut -d, -f1 | head -n 3 | xargs -I {} echo "Checking {}..."
touch temp{1..5}.tmp
ls *.tmp
find . -name "*.tmp" | xargs rm -v
```

```
web01 web02 db01 db02 mon01 app01 app02
Checking web01...
Checking web02...
Checking db01...
temp1.tmp  temp2.tmp  temp3.tmp  temp4.tmp  temp5.tmp
removed './temp1.tmp'
...
```

What just happened: Many commands (`rm` , `echo` , `ping`) take *arguments*, not standard input. `xargs` bridges the gap: it reads lines from stdin and adds them as arguments to a command. The first example turned seven lines into one `echo` with seven arguments. `-I {}` runs the command once per input line, replacing `{}` with the line. `touch temp{1..5}.tmp` uses Bash *brace expansion* to create five files. `find ... | xargs rm -v` deletes every file `find` reported; `-v` makes `rm` say what it removed. If file names may contain spaces, use `find . -name "*.tmp" -print0 | xargs -0 rm` . `xargs -n 1` passes one argument per command run.

Checkpoint: Produce a report of every request that got a 404, printed as `IP -> path` using `awk` . (Answer: `awk '$9 == 404 {print $1, "->", $7}' access.log` prints three lines for `203.0.113.7` .)

Task 8: Reuse commands with history, !, and !!

1. Show your command history.

```
history | tail -n 5
history 5
echo "$HISTSIZE $HISTFILESIZE $HISTCONTROL"
```

```
  201  ...
  205  history 5
1000 2000 ignoreboth
```

What just happened: `history` lists previous commands with numbers. `history 5` shows the last five. Bash keeps the list in memory and writes it to `~/.bash_history` when the shell exits. Ubuntu's `~/.bashrc` sets `HISTSIZE=1000` (commands kept in memory), `HISTFILESIZE=2000` (lines kept in the file), and `HISTCONTROL=ignoreboth` , which skips duplicate commands and any command that starts with a space. That last trick is handy: start a command with a space and it is not saved in history. Your numbers will differ.

2. Re-run the last command with `!!` , and fix a forgotten `sudo` .

```
uname -r
!!
cat /etc/shadow | head -n 2
sudo !!
```

```
6.8.0-45-generic
uname -r
6.8.0-45-generic
cat: /etc/shadow: Permission denied
sudo cat /etc/shadow | head -n 2
root:*:19800:0:99999:7:::
daemon:*:19800:0:99999:7:::
```

What just happened: `!!` expands to the previous command. Bash prints the expanded command line first so you can see what actually ran. `sudo !!` is the classic "I forgot sudo" fix: it re-runs the last command with `sudo` in front. `/etc/shadow` holds password hashes and is readable only by root.

3. Re-run by number, by prefix, and reuse the last argument.

```
history | grep uname
```

Pick a line number from your output (for example `198`) and run it:

```
!198
```

Then try these:

```
!grep
ls -l /etc/hostname
cat !$
```

```
...
cat /etc/hostname
ubuntu-lab
```

What just happened: `!n` runs history entry number `n`. `!grep` runs the most recent command that *started with* `grep`. `!$` is the last argument of the previous command, so `cat !$` became `cat /etc/hostname`. Be careful with `!` shortcuts as root, because they run immediately; `!grep:p` prints the command without running it. You can also press Ctrl+R and start typing to search history interactively (press Ctrl+R again for older matches, Enter to run, or Ctrl+C to cancel), and the Up arrow steps back through history.

TIP

Because `!` triggers history expansion, `echo "Hello!"` can behave strangely in an interactive shell. Use single quotes (`echo 'Hello!'`) when you want a literal `!`.

Checkpoint: `history | tail -n 3` shows the commands you just ran, and you can explain what `!!`, `!n`, `!grep`, and `!$` each expand to.

Task 9: Edit files with vim and nano

1. Create a practice file for `vim`.

```
cd ~/labs/1.5
cat > vim-practice.txt <<'EOF'
The colour of the server rack is grey.
This line has a typo: teh.
Delete this whole line.
The colour of the cables is blue.
EOF
```

2. Open it in `vim`. On Ubuntu, `vi` also starts `vim`.

```
vim vim-practice.txt
```

> **What just happened:** `vim` opens in **Normal mode**, where keys are *commands*, not text. This is the part that confuses beginners. There are three modes you need:
>
> - **Normal mode** (the default): move around, delete, copy, paste. Press Esc to get back here from anywhere.
> - **Insert mode**: typing inserts text. Enter it with `i` (insert before the cursor), `a` (append after it), `o` (open a new line below), or `A` (append at end of line). The bottom line shows `-- INSERT --`.
> - **Command-line mode**: press `:` in Normal mode to type commands such as `:w` (write/save) and `:q` (quit).

3. Follow these steps exactly inside `vim`. Press Esc whenever you are unsure which mode you are in.
    1. Type `:set number` and press Enter. Line numbers appear.
    2. Move with the arrow keys or `h` (left), `j` (down), `k` (up), `l` (right). Press `G` to jump to the last line, `gg` for the first line, `0` for the start of the line, `$` for the end.
    3. Fix the typo: type `/teh` and press Enter to search. The cursor jumps to `teh`. Type `cw` (change word), type `the`, then press Esc.
    4. Delete line 3: type `3G` to go to line 3, then `dd` to delete the line.
    5. Undo and redo: press `u` (the line comes back), then Ctrl+R (it is deleted again).
    6. Add a line: press `G`, then `o`, type `Added in vim.`, and press Esc.
    7. Copy and paste: press `gg`, then `yy` (yank, which means copy, the line), then `p` (paste below). Press `dd` to remove the duplicate. `x` deletes a single character.
    8. Search and replace in the whole file: type `:%s/colour/color/g` and press Enter. `%` means every line, `s` substitute, `g` every match on the line. The status line reports `2 substitutions on 2 lines`.
    9. Save and quit: type `:wq` and press Enter.

What just happened: You used the core `vim` survival kit. The quit commands are worth memorizing: `:w` saves, `:q` quits (only if there are no unsaved changes), `:wq` or `:x` or `ZZ` saves and quits, and `:q!` quits **without** saving, throwing changes away. If you ever get stuck, press Esc twice and type `:q!`. Searching uses `/pattern` (forward), `?pattern` (backward), `n` (next), and `N` (previous). For a full interactive course, run `vimtutor` later; it takes about 30 minutes.

4. Check your edits.

```
cat vim-practice.txt
```

```
The color of the server rack is grey.
This line has a typo: the.
The color of the cables is blue.
Added in vim.
```

5. Now edit a file with `nano`.

```
nano nano-practice.txt
```

`nano` opens directly in a typing mode. The bottom two lines list shortcuts: `^` means Ctrl and `M-` means Alt. Do the following:

1. Type three lines, pressing Enter after each one: `alpha`, `beta`, `gamma`. The cursor is now on an empty fourth line.
2. Press Ctrl+O (write Out), confirm the file name with Enter. This saves.
3. Press Ctrl+W (Where is), type `beta`, press Enter. The cursor jumps to it.
4. Press Ctrl+K to cut the whole `beta` line. Press Alt+/ to jump to the last (empty) line, then press Ctrl+U to paste the line there.
5. Press Ctrl+\ (replace), type `gamma`, press Enter, type `delta`, press Enter, then press `A` to replace all.
6. Try undo and redo: press Alt+U (the replacement is undone), then Alt+E (it is redone).
7. Press Ctrl+X to exit. If there are unsaved changes, `nano` asks “Save modified buffer?”; press `Y` and then Enter.

```
cat nano-practice.txt
```

```
alpha
delta
beta
```

What just happened: `nano` is a *modeless* editor: you type text directly and use Ctrl and Alt combinations for commands. Ctrl+O saves, Ctrl+X exits, Ctrl+W searches, Ctrl+K and Ctrl+U cut and paste lines, and Ctrl+G shows help. Start it with `nano -l file` to show line numbers, or use Ctrl+_ (or Alt+G) to jump to a line number. If your result differs, you probably pasted on a different line; that is fine, as long as you saw cut, paste, and replace work.

TIP

Ubuntu asks which editor to use for tools like `crontab -e` and `visudo` the first time you run them, or uses the `EDITOR` / `VISUAL` variables. `export EDITOR=vim` in `~/.profile` sets your choice; `sudo update-alternatives --config editor` changes the system default. On Rocky minimal, `vi` is `vim-minimal`; install `vim-enhanced` for the full `vim`, and `nano` with `sudo dnf install -y nano`.

Checkpoint: `grep -c color vim-practice.txt` prints `2` and `grep -c colour vim-practice.txt` prints `0`. You can name the `vim` keys to save and quit (`:wq`) and to quit without saving (`:q!`), and the `nano` keys to save (Ctrl+O) and exit (Ctrl+X).

Challenge

CHALLENGE

1. In one pipeline, print the three most requested paths in `access.log` with their counts, formatted with `printf` as `path ... count` in aligned columns.
2. Write a here doc that creates `~/labs/1.5/report.sh`, a script that prints the date, `uname -r`, the number of 404 and 500 responses in `access.log`, and the average bytes per request (use `awk` for the total and `bc` for the division). Run it and save its output with `tee report.txt`, sending any errors to `/dev/null`.
3. Add a temporary alias `top404` that prints the IP address with the most 404 responses. Explain why the alias would disappear if you defined it in `~/.profile` but then opened a non-login shell with `bash` on a system where `.profile` did not source `.bashrc`.

Clean up

1. Restore your original startup files and remove the backups.

```
mv ~/.bashrc.lab15.bak ~/.bashrc
mv ~/.profile.lab15.bak ~/.profile
rm -f ~/.bash_profile
```

   (If you had your own `~/.bash_profile` before the lab and backed it up, restore it instead of deleting it.)

2. Remove the lab files, the tools directory, and the root-owned test file.

```
rm -rf ~/labtools ~/labs/1.5
sudo rm -f /opt/lab15.txt
```

3. If you installed `x11-apps` for the optional X forwarding test, remove it.

```
sudo apt remove -y x11-apps
```

4. Log out and log back in so your shell starts with the original prompt, `PATH`, and aliases. Verify:

```
echo "$PATH" | grep -c labtools
type lab
```

```
0
bash: type: lab: not found
```

Exam corner

- Absolute paths start with `/` (or `~`, which expands to `$HOME`). Relative paths start from the current directory; `.` is the current directory, `..` the parent, and `cd -` returns to the previous directory (`$OLDPWD`).
- `HOME`, `USER`, `SHELL`, `PATH`, `PS1`, and `DISPLAY` are the key variables. `export` makes a variable visible to child processes; `env` / `printenv` list environment variables.
- `PATH` is searched left to right; the current directory is not in it, so run local scripts as `./script`.
- Login shells read `/etc/profile` then the first of `~/.bash_profile`, `~/.bash_login`, `~/.profile`. Non-login interactive shells read `~/.bashrc`. `source file` or `. file` applies changes to the current shell.
- Streams: stdin = 0, stdout = 1, stderr = 2. `>` overwrites, `>>` appends, `2>` redirects errors, `2>&1` joins stderr to stdout (order matters), `&>` sends both, `<` feeds a file to stdin, `<<` is a here doc, `<<<` is a here string, `/dev/null` discards.
- `|` passes only stdout. `tee` writes to a file and stdout at the same time; `| sudo tee file` writes a root-owned file.
- `uniq` only removes *adjacent* duplicates, so `sort | uniq -c | sort -rn` is the classic "count and rank" pipeline.
- `cut -d, -f2` for delimited fields; `awk '{print $2}'` for whitespace-separated columns; `sed 's/a/b/g'` to substitute; `sed -i` edits in place; `tr` changes or deletes single characters.
- `xargs` turns stdin lines into command arguments; `bc` does decimal math (`scale=`); `printf` formats output without an automatic newline.
- `!!` repeats the last command (`sudo !!`), `!n` runs history entry `n`, `!string` runs the latest command starting with `string`, `!$` is the last argument.
- `vim`: `i` insert, Esc normal mode, `:w` save, `:q!` quit without saving, `:wq` save and quit, `dd` delete line, `yy` / `p` copy/paste, `u` undo, `/text` search, `:%s/old/new/g` replace. `nano`: Ctrl+O save, Ctrl+X exit, Ctrl+W search.

Review questions

1. You run `./backup.sh > backup.log`, but error messages still appear on the screen. Which command captures both output and errors in `backup.log`?
   A. `./backup.sh 2>&1 > backup.log`
   B. `./backup.sh > backup.log 2>&1`
   C. `./backup.sh | backup.log`
   D. `./backup.sh >> backup.log 2>/dev/null`

2. A user adds `export PATH="$PATH:/opt/tools/bin"` to `~/.bash_profile` on an Ubuntu system. After the next SSH login, the PATH change works, but their aliases from `~/.bashrc` no longer load in that login. Why?

3. Which pipeline lists the unique values in column 2 of `data.csv` with how many times each appears, most frequent first?
   A. `cut -d, -f2 data.csv | uniq -c | sort -rn`
   B. `cut -d, -f2 data.csv | sort | uniq -c | sort -rn`
   C. `sort data.csv | cut -f2 | wc -l`
   D. `awk '{print $2}' data.csv | uniq`

4. You typed `systemctl restart nginx` and got "Access denied". Which is the quickest way to run the same command with root privileges?

5. Which command sends the string `scale=3; 22/7` to `bc` as standard input without using `echo` or a pipe?
   A. `bc < "scale=3; 22/7"`
   B. `bc << "scale=3; 22/7"`
   C. `bc <<< "scale=3; 22/7"`
   D. `bc > "scale=3; 22/7"`

6. In `vim`, you made changes you do not want to keep. Which command exits without saving?
   A. `:wq`
   B. `ZZ`
   C. `:q!`
   D. `:x`

Lab 1.6 Back Up and Restore a Linux Server

Objective: 1.6 Given a scenario, perform backup and restore operations for a Linux server.
Domain: 1.0 System Management (23% of the exam)
Time: about 120 minutes
VM: ubuntu-lab (Task 8 also uses rocky-lab)
Snapshot first: Yes. Task 9 writes raw data to the extra disk `/dev/sdb` with `dd`.

A backup you have never restored is only a hope. In this lab you back up `/etc` and a sample data directory with `tar` and `cpio`, take full and incremental backups, restore them to a scratch location and prove they match the original, compare the main compression tools, read compressed log files without unpacking them, copy data locally and to rocky-lab with `rsync`, and image a disk with `dd` and `ddrescue`. These are the exact tools the exam names, and the same tools you will use when a server breaks for real.

What you'll practice

- Archiving: `cpio`, `tar` (create, extract, list, `z` / `j` / `J` compression, `--exclude`, preserving permissions, incremental backups)
- Compression: 7-Zip (`7z`), `bzip2`, `gzip`, `unzip` (and `zip`), `xz`
- Other: `dd`, `ddrescue`, `rsync`, `zcat`, `zgrep`, `zless`
- Restore testing and the 3-2-1 backup rule

How it works

Archiving versus compression. These are two different jobs, and Linux keeps them separate:

- An **archiver** bundles many files and directories into *one* file, recording names, permissions, owners, and timestamps. `tar` (tape archive) and `cpio` (copy in/out) are archivers. They do not make data smaller.
- A **compressor** makes *one* stream of data smaller. `gzip`, `bzip2`, and `xz` are compressors. They work on single files, not directories.

So the classic Linux backup file `etc.tar.gz` is a tar archive that was then run through gzip, like putting papers in a box (`tar`) and then vacuum-sealing the box (`gzip`). `tar` can call the compressor for you with one letter: `z` for gzip, `j` for bzip2, `J` for xz. Tools from the Windows world, `zip` and 7-Zip (`7z`), do both jobs at once.

Tool	Extension	Speed	Size	Notes
`gzip`	`.gz`	fastest	largest	Universal default; used for rotated logs
`bzip2`	`.bz2`	medium	smaller	Older; mostly replaced by xz
`xz`	`.xz`	slow	very small	Used for kernel and package archives
7-Zip	`.7z`	slow	very small	LZMA2, like xz; package `7zip` on Ubuntu 24.04
`zip`	`.zip`	fast	like gzip	Archive + compress; best for sharing with Windows users

Copying and syncing. `rsync` copies files and, on later runs, sends only what changed. That makes it ideal for keeping a mirror of a directory on another disk or another server over SSH. It is not an archive format; the copy is just normal files.

Block-level imaging. `tar`, `cpio`, and `rsync` work with *files*. `dd` works with *raw blocks*: it copies bytes from one place to another without caring what they mean. You use it to copy a whole disk or partition, back up the boot sector, or create a blank disk image file. `ddrescue` does the same job for *failing* disks: it copies the good areas first, keeps a *mapfile* (a log of which areas were copied and which failed), and can resume or retry the bad spots later.

```
FILE level                          BLOCK level
tar / cpio / rsync                  dd / ddrescue
+---------+                         +-----------------------------+
| /etc    | -- names, perms -->     | sector 0 | sector 1 | ...   |
| /home   |    owners, data         |  (copies every byte, used or |
+---------+                         |   empty, without reading the |
small, selective, portable          |   file system)               |
                                    +-----------------------------+
                                    exact clone, includes boot code
```

Types of backup. A *full* backup copies everything. An *incremental* backup copies only what changed since the last backup of any kind; it is small and fast, but a restore needs the full backup plus *every* incremental in order. A *differential* backup copies everything changed since the last full backup; a restore needs only the full plus the latest differential. GNU `tar` makes incrementals with `--listed-incremental`, using a *snapshot file* that remembers what it saw last time.

The 3-2-1 rule. A widely used guideline for backups that survive real disasters:

- **3** copies of your data (the original plus two backups),
- on **2** different types of storage (for example the server disk and a NAS or tape),
- with **1** copy off-site (another building, another region, or the cloud).

Many teams add "1 copy offline or immutable" (ransomware cannot encrypt what it cannot reach) and "0 errors after a restore test". A backup is only proven when you have restored it and checked the result, which is what you will do in this lab every time.

Safety with dd. `dd` does exactly what you tell it, instantly, with no "Are you sure?". Swapping `if=` (input file) and `of=` (output file) overwrites the disk you meant to save. People joke that `dd` stands for "disk destroyer". Always run `lsblk` first, read the command twice, and take a VM snapshot.

Tasks

Task 1: Prepare the tools and a sample data directory

1. Install every tool the lab uses. Some are already installed; `apt` will say so.

```
sudo apt update
sudo apt install -y tar cpio gzip bzip2 xz-utils 7zip zip unzip rsync gddrescue
```

> **What just happened:** You installed the archivers (`tar` , `cpio`), the compressors (`gzip` , `bzip2` , `xz-utils`), 7-Zip, `zip` / `unzip` , `rsync` , and GNU ddrescue. Two package names are worth remembering. On Ubuntu 24.04, the `7zip` package provides the `7z` command (the older `p7zip-full` package still exists, but only as a transitional package that pulls in `7zip`). The `ddrescue` command comes from the package `gddrescue` ; the similarly named package `ddrescue` is not available on Ubuntu 24.04. On Rocky, both `p7zip` and `ddrescue` come from the EPEL repository.

2. Create the lab directory with a sample data set that has different permissions, a sub-directory to exclude, and a backups folder.

```
mkdir -p ~/labs/1.6 && cd ~/labs/1.6
mkdir -p data/{docs,scripts,cache} backups restore
echo "Quarterly report: all systems nominal." > data/docs/report.txt
seq 1 5000 > data/docs/numbers.txt
echo "db_password=Pa55w0rd" > data/docs/secret.txt
chmod 600 data/docs/secret.txt
printf '#!/bin/bash\necho "nightly job ran"\n' > data/scripts/nightly.sh
chmod 750 data/scripts/nightly.sh
dd if=/dev/urandom of=data/cache/blob.tmp bs=1K count=512 status=none
ls -lR data
```

```
data:
total 12
drwxrwxr-x 2 student student 4096 Oct 10 10:00 cache
drwxrwxr-x 2 student student 4096 Oct 10 10:00 docs
drwxrwxr-x 2 student student 4096 Oct 10 10:00 scripts

data/cache:
total 512
-rw-rw-r-- 1 student student 524288 Oct 10 10:00 blob.tmp

data/docs:
total 32
-rw-rw-r-- 1 student student 23893 Oct 10 10:00 numbers.txt
-rw-rw-r-- 1 student student    39 Oct 10 10:00 report.txt
-rw------- 1 student student    21 Oct 10 10:00 secret.txt

data/scripts:
total 4
-rwxr-x--- 1 student student 35 Oct 10 10:00 nightly.sh
```

What just happened: `mkdir -p data/{docs,scripts,cache}` uses brace expansion to create three subdirectories at once. `seq 1 5000` prints the numbers 1 to 5000, giving you a compressible text file. `chmod 600` makes `secret.txt` readable only by you, and `chmod 750` makes the script executable: you will check that backups preserve these permissions. The `dd` line reads 512 KB from `/dev/urandom` (random bytes) into a "cache" file; random data cannot be compressed, and you will exclude this directory from backups the way you would exclude a real cache.

Checkpoint: `which tar cpio gzip bzip2 xz 7z zip unzip rsync ddrescue` prints ten paths, and `ls -l data/docs/secret.txt` shows `-rw-------`.

Task 2: Create full backups with tar

1. Back up all of `/etc` into a gzip-compressed archive. You need `sudo` because some files in `/etc` (such as `/etc/shadow`) are readable only by root.

```
sudo tar -czpf backups/etc-full.tar.gz /etc
ls -lh backups/etc-full.tar.gz
```

```
tar: Removing leading `/' from member names
-rw-r--r-- 1 root root 1.3M Oct 10 10:02 backups/etc-full.tar.gz
```

What just happened: `tar` options are single letters that can be grouped: `-c` create a new archive, `-z` compress it with gzip, `-p` preserve permissions, and `-f` names the archive file, so `f` must come last, right before the file name. The message about the leading `/` is normal and protective: tar stores `etc/hostname` instead of `/etc/hostname`, so a later extract goes into the *current* directory instead of overwriting the live `/etc`. Your size will differ.

2. This archive contains password hashes and private keys from `/etc`. Protect it like the original.

```
sudo chmod 600 backups/etc-full.tar.gz
```

> **CAUTION**
> Backups are a favorite target for attackers because they hold everything in one file. Restrict permissions, and encrypt backups that leave the server (for example with `gpg --symmetric` or an encrypting backup tool).

3. List the contents without extracting.

```
sudo tar -tzf backups/etc-full.tar.gz | head -n 5
sudo tar -tzvf backups/etc-full.tar.gz etc/shadow etc/hostname
```

```
etc/
etc/fstab
etc/crypttab
etc/hostname
etc/ssh/
-rw-r----- root/shadow    1024 2026-10-10 09:12 etc/shadow
-rw-r--r-- root/root        11 2026-10-10 08:40 etc/hostname
```

> **What just happened:** `-t` lists the table of contents. Adding `-v` (verbose) shows permissions, owner/group, size, and date, so you can confirm tar recorded `/etc/shadow` with mode `640` and group `shadow`. Naming specific members after the archive limits the listing to those paths. Listing an archive is a quick health check: a damaged archive fails with an error.

4. Back up the data directory, leaving out the cache, and try all three compressors.

```
tar -czvf backups/data.tar.gz --exclude='data/cache' data
tar -cjf backups/data.tar.bz2 --exclude='data/cache' data
tar -cJf backups/data.tar.xz --exclude='data/cache' data
ls -l backups/data.tar.*
```

```
data/
data/docs/
data/docs/numbers.txt
data/docs/report.txt
data/docs/secret.txt
data/scripts/
data/scripts/nightly.sh
-rw-rw-r-- 1 student student  6374 Oct 10 10:03 backups/data.tar.bz2
-rw-rw-r-- 1 student student 11609 Oct 10 10:03 backups/data.tar.gz
-rw-rw-r-- 1 student student  2228 Oct 10 10:03 backups/data.tar.xz
```

What just happened: `-v` printed each file as it was added. `--exclude='data/cache'` skipped the cache directory; patterns can also use wildcards, such as `--exclude='*.tmp'`. The only difference between the three commands is the compression letter: `z` (gzip, `.tar.gz` or `.tgz`), `j` (bzip2, `.tar.bz2`), and `J` (xz, `.tar.xz`). When extracting, modern GNU tar detects the compression automatically, so `tar -xf file.tar.xz` works too. Here xz produced by far the smallest file, because `numbers.txt` is very repetitive text, which LZMA handles especially well. The exact byte counts on your system will be slightly different.

TIP

Put the date in backup names so they sort and never overwrite each other: `tar -czf backups/etc-$(date +%F).tar.gz /etc`. `$(date +%F)` is *command substitution*: the shell runs `date +%F` and inserts its output, such as `2026-10-10`.

Checkpoint: `ls backups` shows `etc-full.tar.gz` plus three `data.tar` files, and `tar -tf backups/data.tar.xz | grep -c cache` prints `0`.

Task 3: Restore to a scratch location and prove it matches

Never test a restore by extracting over the live system. Extract into an empty directory, compare, and only then copy what you need back.

1. Extract the `/etc` backup into `restore/`.

```
sudo tar -xzpf backups/etc-full.tar.gz -C restore
ls restore
sudo ls -l restore/etc/shadow /etc/shadow
```

```
etc
-rw-r----- 1 root shadow 1024 Oct 10 09:12 /etc/shadow
-rw-r----- 1 root shadow 1024 Oct 10 09:12 restore/etc/shadow
```

What just happened: `-x` extracts, and `-C restore` changes into the `restore` directory first, so the files land in `restore/etc/...`. Because you ran tar as root, it restored the original owners (`root:shadow`), permissions (`-p`), and timestamps. Run as a normal user, tar would make you the owner of every file and apply your umask unless you add `-p`; only root can restore other users' ownership.

2. Compare the restored copy with the original.

```
sudo diff -r --no-dereference /etc restore/etc && echo "RESTORE MATCHES"
```

```
RESTORE MATCHES
```

> **What just happened:** `diff -r` compares two directory trees recursively and prints any file that differs or exists in only one place. `--no-dereference` compares symbolic links as links instead of following them (some links in `/etc` point outside it, such as `/etc/resolv.conf`). No output means the trees are identical, so `&&` printed the message. If something in `/etc` changed after the backup (for example a package update), `diff` lists it, which is also useful: it shows exactly what changed.

3. Restore a single file, the most common real-world request ("I deleted my config, can you get it back?").

```
mkdir -p restore-one
sudo tar -xzf backups/etc-full.tar.gz -C restore-one etc/hostname
cat restore-one/etc/hostname
```

```
ubuntu-lab
```

> **What just happened:** After the archive name you can list member paths to extract only those. The path must match what `tar -t` shows (`etc/hostname`, without a leading `/`). Use `--wildcards 'etc/ssh/*'` to extract a pattern.

4. Restore the data backup as a normal user and check the permissions survived.

```
mkdir -p restore-data
tar -xJf backups/data.tar.xz -C restore-data
ls -l restore-data/data/docs/secret.txt restore-data/data/scripts/nightly.sh
diff -r --exclude=cache data restore-data/data && echo "DATA MATCHES"
```

```
-rw------- 1 student student 21 Oct 10 10:00 restore-data/data/docs/secret.txt
-rwxr-x--- 1 student student 35 Oct 10 10:00 restore-data/data/scripts/nightly.sh
DATA MATCHES
```

> **What just happened:** You restored the xz archive as `student` without `sudo`, and the modes `600` and `750` survived, because the files already belonged to you. `--exclude=cache` told `diff` to ignore the cache directory, which you deliberately left out of the backup. This is a complete restore test: extract to a scratch location, compare with the original, and check permissions. Doing it regularly is what turns a backup into a *proven* backup.

> **Checkpoint:** Both `diff` commands printed their "MATCHES" message, and `restore-data/data/docs/secret.txt` still has mode `-rw-------`.

Task 4: Take incremental backups with tar

1. Take a level-0 (full) backup that also records a snapshot file.

```
tar -czvf backups/data-level0.tar.gz --exclude='data/cache' \
    --listed-incremental=backups/data.snar data
```

```
tar: data: Directory is new
tar: data/docs: Directory is new
tar: data/scripts: Directory is new
data/
data/docs/
data/scripts/
data/docs/numbers.txt
data/docs/report.txt
data/docs/secret.txt
data/scripts/nightly.sh
```

What just happened: `--listed-incremental=backups/data.snar` tells GNU tar to keep a *snapshot file* (`.snar`). Because the file did not exist, tar made a full backup and wrote down every directory and file it saw, with timestamps. "Directory is new" just means "not in the snapshot yet". The backslash `\` at the end of the first line continues the command on the next line.

2. Simulate a day of changes: add a file, delete a file, and modify a file.

```
echo "new customer list" > data/docs/new.txt
rm data/docs/report.txt
echo 5001 >> data/docs/numbers.txt
```

3. Take a level-1 (incremental) backup using the same snapshot file.

```
tar -czvf backups/data-level1.tar.gz --exclude='data/cache' \
    --listed-incremental=backups/data.snar data
ls -l backups/data-level*.tar.gz
```

```
data/
data/docs/
data/scripts/
data/docs/new.txt
data/docs/numbers.txt
-rw-rw-r-- 1 student student 11781 Oct 10 10:06 backups/data-level0.tar.gz
-rw-rw-r-- 1 student student 11540 Oct 10 10:07 backups/data-level1.tar.gz
```

What just happened: tar compared the files with the snapshot and stored only the new and changed files (`new.txt` and `numbers.txt`), plus the directory entries. It also recorded that `report.txt` is gone. The snapshot file was updated, so the next run would be level 2, containing only changes since now. Here the two archives are similar in size only because `numbers.txt` is most of the data; on a real server the incremental is usually far smaller. To make more level-1 backups against the same full backup (a differential style), copy the level-0 `.snar` file before each run and use the copy.

4. Restore the chain in order into an empty directory.

```
mkdir -p restore-incr
tar -xzvf backups/data-level0.tar.gz --listed-incremental=/dev/null -C restore-incr
ls restore-incr/data/docs
tar -xzvf backups/data-level1.tar.gz --listed-incremental=/dev/null -C restore-incr
ls restore-incr/data/docs
diff -r --exclude=cache data restore-incr/data && echo "INCREMENTAL RESTORE MATCHES"
```

```
...
numbers.txt  report.txt  secret.txt
data/
data/docs/
tar: Deleting 'data/docs/report.txt'
data/scripts/
data/docs/new.txt
data/docs/numbers.txt
new.txt  numbers.txt  secret.txt
INCREMENTAL RESTORE MATCHES
```

What just happened: You restored the full backup first, then the incremental on top of it. `--listed-incremental=/dev/null` tells tar "this is an incremental archive, apply its directory information" without needing the snapshot file. Notice `Deleting 'data/docs/report.txt'`: tar removed the file that had been deleted between the backups, so the result matches the live directory exactly. Restoring incrementals out of order, or skipping one, gives the wrong result.

Checkpoint: `ls restore-incr/data/docs` shows `new.txt numbers.txt secret.txt` (no `report.txt`), and the final `diff` printed its success message.

Task 5: Archive with cpio

`cpio` is older than tar and works differently: it does not search directories itself. It reads a list of file names on standard input, usually from `find`. You meet it in the real world inside initramfs images and RPM packages.

1. Create a cpio archive of the data directory.

```
find data | cpio -ov -H newc > backups/data.cpio
```

```
data
data/cache
data/cache/blob.tmp
data/scripts
data/scripts/nightly.sh
data/docs
data/docs/new.txt
data/docs/secret.txt
data/docs/numbers.txt
1074 blocks
```

What just happened: `find data` prints every path under `data`, one per line, and the pipe feeds that list to `cpio`. `-o` is *copy-out* mode (create an archive), `-v` lists each file, and `-H newc` chooses the modern "new ASCII" format (the one initramfs uses), which handles large files and inode numbers well. cpio writes the archive to standard output, so you redirected it into a file with `>`. The block count is the archive size in 512-byte blocks. To leave out the cache you would filter the list, for example `find data -path data/cache -prune -o -print`.

2. List the archive and extract it into an empty directory.

```
cpio -itv < backups/data.cpio
mkdir -p restore-cpio && cd restore-cpio
cpio -idmv < ../backups/data.cpio
cd ..
diff -r data restore-cpio/data && echo "CPIO MATCHES"
```

```
drwxrwxr-x   5 student  student         0 Oct 10 10:00 data
...
-rw-------   1 student  student        21 Oct 10 10:00 data/docs/secret.txt
...
CPIO MATCHES
```

What just happened: `-i` is *copy-in* mode (extract or list), and `-t` with it means "only list the table of contents". cpio reads the archive from standard input, hence `<`. For the extract, `-d` creates directories as needed and `-m` keeps the original modification times. Unlike tar, cpio has no `-C` option, so you `cd` into the target directory first.

3. Use pass-through mode to copy files that match a pattern into another directory tree.

```
mkdir -p copy-scripts
find data -name '*.sh' | cpio -pdmv copy-scripts
ls -l copy-scripts/data/scripts
```

```
copy-scripts/data/scripts/nightly.sh
1 block
-rwxr-x--- 1 student student 35 Oct 10 10:00 nightly.sh
```

What just happened: `-p` (pass-through) copies the listed files straight into a directory, keeping their relative paths and attributes, without creating an archive. Combined with `find`'s power (by name, age, owner, size), this is a precise way to copy a selection of files.

TIP

You can look inside Ubuntu's initramfs, which is built from cpio archives, with `lsinitramfs /boot/initrd.img-$(uname -r) | head`.

Checkpoint: `cpio -it < backups/data.cpio | wc -l` prints `9`, and `restore-cpio/data` matches `data`.

Task 6: Compare gzip, bzip2, xz, 7-Zip, and zip

1. Build a realistic test file: a tar archive of Python's standard library (thousands of text files, present on every Ubuntu 24.04 system).

```
cd ~/labs/1.6
mkdir -p compress && cd compress
tar -cf sample.tar --exclude=__pycache__ /usr/lib/python3.12 2>/dev/null
ls -lh sample.tar
```

```
-rw-rw-r-- 1 student student 41M Oct 10 10:10 sample.tar
```

Your size may be somewhat different; that does not matter.

2. Compress it with each tool and time each one. The `-k` option keeps the original file so the next tool can use it.

```
time gzip -k sample.tar
time bzip2 -k sample.tar
time xz -k sample.tar
time 7z a sample.tar.7z sample.tar
time zip sample.zip sample.tar
ls -l sample.*
```

```
real    0m1.770s
...
real    0m3.948s
...
real    0m20.745s
...
real    0m10.467s
...
  adding: sample.tar (deflated 74%)
real    0m1.908s
...
-rw-rw-r-- 1 student student 42854400 Oct 10 10:10 sample.tar
-rw-rw-r-- 1 student student  6888671 Oct 10 10:12 sample.tar.7z
-rw-rw-r-- 1 student student  9716209 Oct 10 10:11 sample.tar.bz2
-rw-rw-r-- 1 student student 11172117 Oct 10 10:11 sample.tar.gz
-rw-rw-r-- 1 student student  7563380 Oct 10 10:11 sample.tar.xz
-rw-rw-r-- 1 student student 11172258 Oct 10 10:12 sample.zip
```

What just happened: `time` runs a command and reports how long it took; `real` is wall-clock time. Fill in your own numbers, but the pattern is typical: gzip and zip are fastest with the biggest output (they use the same "deflate" algorithm, which is why their sizes are almost identical), bzip2 is in the middle, and xz and 7-Zip (both based on LZMA) produce the smallest files but take the longest. 7-Zip was faster than xz here because it uses several CPU threads by default; `xz -T0` also uses all cores. All of them are *lossless*: decompressing gives back the exact original bytes. Each tool accepts a level from `-1` (fastest) to `-9` (smallest); the default is `-6` for gzip and xz and `-9` for bzip2.

3. Inspect the compressed files without decompressing.

```
gzip -l sample.tar.gz
xz -l sample.tar.xz
unzip -l sample.zip
7z l sample.tar.7z | tail -n 3
```

```
         compressed        uncompressed  ratio uncompressed_name
           11172117            42854400  73.9% sample.tar
Strms  Blocks   Compressed Uncompressed  Ratio  Check   Filename
    1       1   7386.1 KiB     40.9 MiB  0.176  CRC64   sample.tar.xz
Archive:  sample.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
 42854400  2026-10-10 10:10   sample.tar
---------                     -------
 42854400                     1 file
------------------- ----- ------------ ------------  ------------------------
2026-10-10 10:10:38           42854400      6888541  1 files
```

What just happened: `gzip -l` shows the space saved (73.9% smaller). `xz -l` shows the ratio as a fraction (0.176 means the file is 17.6% of the original). `unzip -l` and `7z l` list archive contents. Listing is a quick way to see what an archive holds before you extract it.

4. Decompress and verify the files are identical to the original.

```
mkdir -p out
gunzip -c sample.tar.gz > out/from-gz.tar
bzip2 -dc sample.tar.bz2 > out/from-bz2.tar
xz -dc sample.tar.xz > out/from-xz.tar
7z x sample.tar.7z -oout/7z > /dev/null
unzip -q sample.zip -d out/zip
sha256sum sample.tar out/from-*.tar out/7z/sample.tar out/zip/sample.tar
```

```
5f1c...e9a2  sample.tar
5f1c...e9a2  out/from-gz.tar
5f1c...e9a2  out/from-bz2.tar
5f1c...e9a2  out/from-xz.tar
5f1c...e9a2  out/7z/sample.tar
5f1c...e9a2  out/zip/sample.tar
```

What just happened: `-c` sends decompressed data to standard output instead of replacing the `.gz` file, so the compressed copies stay in place; `-d` means decompress (`gunzip` = `gzip -d`, `bunzip2` = `bzip2 -d`, `unxz` = `xz -d`). `7z x` extracts with full paths, and `-o` sets the output directory (note: no space after `-o`). `unzip -d` sets the destination directory, and `-q` keeps it quiet. `sha256sum` prints a cryptographic checksum; identical checksums prove the files are byte-for-byte identical. Recording checksums of backups and checking them after a restore is a good habit.

5. Use `zip` and `7z` to archive a whole directory (they archive and compress in one step), and test the archives.

```
cd ~/labs/1.6
zip -r backups/data.zip data -x 'data/cache/*'
unzip -l backups/data.zip
7z a backups/data.7z data -xr'!cache'
7z t backups/data.7z
```

```
  adding: data/ (stored 0%)
  adding: data/scripts/ (stored 0%)
  adding: data/scripts/nightly.sh (stored 0%)
  adding: data/docs/ (stored 0%)
  adding: data/docs/new.txt (stored 0%)
  adding: data/docs/secret.txt (stored 0%)
  adding: data/docs/numbers.txt (deflated 54%)
...
Everything is Ok
...
Size:       23972
Compressed: 2189
```

What just happened: `zip -r` recurses into directories, and `-x` excludes matching paths. `7z a` adds files to an archive, `-xr'!cache'` recursively excludes anything named `cache`, and `7z t` tests the archive's integrity. Unlike tar, zip and 7z do not store Linux owners and groups, so use tar for system backups and zip or 7z for sharing files.

Checkpoint: All six `sha256sum` lines show the same checksum, and you can rank the tools from fastest to smallest output on your VM.

Task 7: Read compressed logs with zcat, zgrep, and zless

`logrotate` compresses old logs with gzip to save space, so on a real server yesterday's evidence is usually in a `.gz` file. The `z` tools let you read those files without unpacking them.

1. Look for rotated, compressed logs on your system.

   ```
   ls -l /var/log/*.gz /var/log/*/*.gz 2>/dev/null | head
   ```

   A VM that has been running for a while shows files such as `/var/log/syslog.2.gz` or `/var/log/apt/history.log.1.gz`. A brand-new VM may show nothing yet, which is why you will also build your own.

2. Create a practice set of rotated logs, named the way `logrotate` names them.

```
mkdir -p ~/labs/1.6/logs && cd ~/labs/1.6/logs
printf '%s\n' "10:01 INFO  service started" "10:05 ERROR disk /dev/sdb not found" \
    "10:09 INFO  backup completed" > app.log
printf '%s\n' "09:01 INFO  service started" "09:30 WARN  disk 85% full" > app.log.1
printf '%s\n' "08:00 INFO  service started" "08:12 ERROR backup failed: disk full" \
    "08:15 ERROR backup failed: disk full" > app.log.2
printf '%s\n' "07:00 INFO  service started" "07:44 ERROR login failed for root" > app.log.3
gzip app.log.2 app.log.3
ls -l
```

```
-rw-rw-r-- 1 student student 93 Oct 10 10:20 app.log
-rw-rw-r-- 1 student student 54 Oct 10 10:20 app.log.1
-rw-rw-r-- 1 student student 99 Oct 10 10:20 app.log.2.gz
-rw-rw-r-- 1 student student 92 Oct 10 10:20 app.log.3.gz
```

What just happened: `printf '%s\n'` prints each argument on its own line, a tidy way to write several lines at once. `gzip` replaced each file with a compressed `.gz` version (it deletes the original unless you add `-k`). This matches the default rotation on Ubuntu: the current log, `.1` uncompressed (because of the `delaycompress` setting), and older ones compressed.

3. Read, search, and page through the compressed logs.

```
cat app.log.2.gz | head -c 40; echo
zcat app.log.2.gz
zgrep ERROR app.log*
zgrep -c ERROR app.log.*.gz
zless app.log.3.gz
```

```
...binary garbage...
08:00 INFO  service started
08:12 ERROR backup failed: disk full
08:15 ERROR backup failed: disk full
app.log:10:05 ERROR disk /dev/sdb not found
app.log.2.gz:08:12 ERROR backup failed: disk full
app.log.2.gz:08:15 ERROR backup failed: disk full
app.log.3.gz:07:44 ERROR login failed for root
app.log.2.gz:2
app.log.3.gz:1
```

Press `q` to leave `zless`.

What just happened: `cat` on a `.gz` file prints compressed bytes, which is unreadable. `zcat` decompresses to standard output (like `gunzip -c`) and leaves the file untouched. `zgrep` takes the same options as `grep` (`-i`, `-c`, `-v`, `-E`) and works on both compressed and plain files, so `app.log*` searched all four at once. `zless` opens a compressed file in `less`, with the same keys (`/` to search, `q` to quit). The same family exists for other formats: `bzcat` / `bzgrep` / `bzless` for `.bz2` and `xzcat` / `xzgrep` / `xzless` for `.xz`.

4. If step 1 found real rotated logs, try the tools on them. For example:

```
sudo zgrep -h "install " /var/log/apt/history.log.*.gz | head -n 5
sudo zcat /var/log/syslog.2.gz | tail -n 5
```

`-h` hides the file name prefix. Adjust the file names to match what `ls` showed.

Checkpoint: `zgrep -c "disk full" app.log.2.gz` prints `2`, and `ls` still shows `app.log.2.gz` (zcat and zgrep never decompress the file on disk).

Task 8: Synchronize data with rsync, locally and to rocky-lab

1. Make a local mirror and learn the trailing-slash rule.

```
cd ~/labs/1.6
rsync -av data mirror1/
rsync -av data/ mirror2/
ls mirror1 mirror2
```

```
sending incremental file list
created directory mirror1
data/
data/cache/
...
mirror1:
data

mirror2:
cache  docs  scripts
```

What just happened: `-a` (archive) is shorthand for `-rlptgoD`: recurse, copy symlinks as symlinks, and keep permissions, times, group, owner (owner only when run as root), and device files. `-v` lists what is copied. Now the important rule. Without a trailing slash on the *source* (`data`), rsync copies the directory itself, so you get `mirror1/data/...`. With a trailing slash (`data/`), rsync copies the directory's *contents*, so you get `mirror2/docs/...`. Think of `data/` as "everything inside data". A slash on the destination makes no difference. Getting this wrong is the most common rsync mistake, and with `--delete` it can be expensive.

2. Run the same sync again, then change a few files and preview the next run.

```
rsync -av data/ mirror2/
rm data/docs/new.txt
echo "draft" > data/docs/plan.txt
rsync -av --delete --dry-run data/ mirror2/
```

```
sending incremental file list

sent 261 bytes  received 15 bytes  552.00 bytes/sec
total size is 548,246  speedup is 1,986.40
sending incremental file list
deleting docs/new.txt
docs/
docs/plan.txt

sent 287 bytes  received 44 bytes  662.00 bytes/sec
total size is 548,242  speedup is 1,656.32 (DRY RUN)
```

What just happened: The second identical run copied nothing: rsync compares size and modification time and skips files that have not changed, which is why it is so fast for repeated backups. `--delete` removes files from the destination that no longer exist in the source, making an exact mirror. `--dry-run` (`-n`) shows what *would* happen, marked `(DRY RUN)`, without changing anything. Always dry-run a `--delete` command first.

3. Apply the changes for real and verify.

```
rsync -av --delete data/ mirror2/
diff -r data mirror2 && echo "MIRROR MATCHES"
```

CAUTION

`--delete` mirrors *deletions* too. If someone deletes a file by mistake (or ransomware encrypts everything), the next sync faithfully copies that damage. A mirror is not a backup on its own; keep versioned copies as well (for example dated tar archives, or `rsync --backup --backup-dir=...`).

4. Copy the data to rocky-lab over SSH. First make sure `rsync` is installed there, because rsync must run on **both** ends.

```
# on rocky-lab
sudo dnf install -y rsync
```

Then, back on ubuntu-lab:

```
rsync -avz data/ student@rocky-lab:backup-1.6/
```

```
The authenticity of host 'rocky-lab (192.168.100.20)' can't be established.
ED25519 key fingerprint is SHA256:...
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
student@rocky-lab's password:
sending incremental file list
created directory backup-1.6
./
cache/
cache/blob.tmp
...
```

What just happened: A destination in the form `user@host:path` makes rsync start itself on the remote host over SSH and send data through the encrypted connection. Answer `yes` to the host-key question the first time, and enter `student`'s password on rocky-lab (or nothing, if you set up keys in Lab 3.3). A relative remote path like `backup-1.6/` is relative to the remote user's home directory. `-z` compresses data in transit, which helps on slow links. If SSH listens on another port, add `-e "ssh -p 2222"`. If you see `rsync: command not found` from the remote side, rsync is missing there.

5. Restore from rocky-lab (pull), and check the result.

```
rsync -av student@rocky-lab:backup-1.6/ restore-rsync/
diff -r data restore-rsync && echo "REMOTE COPY MATCHES"
```

What just happened: Swapping source and destination pulls files back. This is your off-site copy (in 3-2-1 terms) being tested with a real restore.

Checkpoint: `ls mirror1` shows `data`, `ls mirror2` shows the contents, and `diff -r data restore-rsync` prints nothing. On rocky-lab, `ls ~/backup-1.6` shows `cache docs scripts`.

Task 9: Image disks with dd and ddrescue

CAUTION

Take a snapshot of ubuntu-lab now. This task writes directly to the extra disk. Run `lsblk` and make sure the disk you use is an **empty 5 GB lab disk** with no mount points. This book uses `/dev/sdb`; if your extra disks are named differently (`/dev/vdb`, `/dev/nvme0n2`), substitute your name in **every** command. Writing to the wrong device destroys its data instantly.

1. Identify the disk, then create a small partition with a file system and a file on it.

```
lsblk
sudo parted -s /dev/sdb mklabel msdos mkpart primary ext4 1MiB 201MiB
sudo mkfs.ext4 -q -L labdata /dev/sdb1
sudo mkdir -p /mnt/lab16
sudo mount /dev/sdb1 /mnt/lab16
echo "important data" | sudo tee /mnt/lab16/important.txt
sudo umount /mnt/lab16
lsblk /dev/sdb
```

```
important data
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sdb      8:16   0    5G  0 disk
└─sdb1   8:17   0  200M  0 part
```

What just happened: `parted -s` (script mode, no questions) wrote an MBR (`msdos`) partition table and one 200 MiB partition starting at 1 MiB. `mkfs.ext4 -q -L labdata` formatted it as ext4 with the label `labdata`. You mounted it, wrote a test file, and unmounted it. Unmounting matters: imaging a mounted, changing file system can give an inconsistent copy. (Lab 1.3 covers partitioning in depth.)

2. Back up the MBR, the first 512-byte sector of the disk.

```
cd ~/labs/1.6
sudo dd if=/dev/sdb of=backups/sdb-mbr.bin bs=512 count=1
file backups/sdb-mbr.bin
od -A d -t x1 -j 510 -N 2 backups/sdb-mbr.bin
```

```
1+0 records in
1+0 records out
512 bytes copied, 0.0003 s, 1.6 MB/s
backups/sdb-mbr.bin: DOS/MBR boot sector; partition 1 : ID=0x83, ... startsector 2048,
409600 sectors
0000510 55 aa
0000512
```

What just happened: `dd` copies data in blocks. `if=` is the input file (here the whole disk), `of=` is the output file, `bs=512` sets the block size to 512 bytes, and `count=1` copies just one block. The MBR holds 446 bytes of boot code, a 64-byte partition table (four entries), and the 2-byte signature `55 aa`. `file` recognized it, and `od` (octal dump) showed the two bytes at offset 510 in hexadecimal. `dd` reports records in and out when it finishes. GPT disks keep their partition table in the first 34 sectors and a backup copy at the end of the disk, so for GPT you would back up more (or use `sgdisk --backup`); for any disk, `sudo sfdisk -d /dev/sdb > table.txt` saves the partition layout as text.

3. Image the partition to a file, both raw and compressed, and record a checksum.

```
sudo dd if=/dev/sdb1 of=backups/sdb1.img bs=4M status=progress
sudo dd if=/dev/sdb1 bs=4M status=none | gzip > backups/sdb1.img.gz
ls -lh backups/sdb1.img*
sudo sha256sum /dev/sdb1 backups/sdb1.img
```

```
50+0 records in
50+0 records out
209715200 bytes (210 MB, 200 MiB) copied, 0.61 s, 344 MB/s
-rw-r--r-- 1 root    root    200M Oct 10 10:40 backups/sdb1.img
-rw-rw-r-- 1 student student 212K Oct 10 10:40 backups/sdb1.img.gz
3b7e...c1d0  /dev/sdb1
3b7e...c1d0  backups/sdb1.img
```

What just happened: The first `dd` copied every byte of the partition into `sdb1.img`. `bs=4M` reads 4 MiB at a time, which is much faster than the default of 512 bytes, and `status=progress` shows live progress. The second `dd` has no `of=`, so it wrote to standard output; the pipe sent the bytes through `gzip`. Because most of the partition is empty (zeros), the compressed image is tiny. The matching checksums prove the image is exact. The raw image is owned by root because `sudo dd` created it; the `.gz` file is yours because your shell created it through the redirection.

4. Create a blank disk image file and use it like a disk.

```
dd if=/dev/zero of=backups/blank.img bs=1M count=64 status=progress
mkfs.ext4 -q backups/blank.img
sudo mount -o loop backups/blank.img /mnt/lab16
df -h /mnt/lab16
sudo umount /mnt/lab16
```

```
64+0 records in
64+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 0.05 s, 1.3 GB/s
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop8       56M   14K   51M   1% /mnt/lab16
```

What just happened: `/dev/zero` produces endless zero bytes, so this `dd` created a 64 MiB file full of zeros. You formatted the *file* with ext4 and mounted it with `-o loop`, which makes the kernel present the file as a block device (a *loop device* such as `/dev/loop8`; the number varies, because snap packages already use the first few). Virtual machine disks, ISO images, and swap files are created this same way. (`fallocate -l 64M file` or `truncate -s 64M file` create such files faster.)

5. Disaster 1: destroy the partition table, then restore it from your MBR backup.

```
sudo wipefs -a /dev/sdb
lsblk /dev/sdb
```

```
/dev/sdb: 2 bytes were erased at offset 0x000001fe (dos): 55 aa
/dev/sdb: calling ioctl to re-read partition table: Success
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sdb    8:16   0   5G  0 disk
```

The partition is gone. Restore the MBR and ask the kernel to re-read it:

```
sudo dd if=backups/sdb-mbr.bin of=/dev/sdb bs=512 count=1
sudo partprobe /dev/sdb
lsblk /dev/sdb
sudo mount /dev/sdb1 /mnt/lab16 && cat /mnt/lab16/important.txt
sudo umount /mnt/lab16
```

```
1+0 records in
1+0 records out
512 bytes copied, 0.0004 s, 1.3 MB/s
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sdb      8:16   0    5G  0 disk
└─sdb1   8:17   0  200M  0 part
important data
```

What just happened: `wipefs -a` erased the MBR signature, so the kernel no longer saw a partition table. Note the direction of the restore: now the *image* is `if=` and the *disk* is `of=`. `partprobe` tells the kernel to re-read the partition table, and `sdb1` came back with its data intact, because only the table was damaged, not the file system.

6. Disaster 2: destroy the file system, then restore the partition image.

```
sudo wipefs -a /dev/sdb1
sudo mount /dev/sdb1 /mnt/lab16
```

```
/dev/sdb1: 2 bytes were erased at offset 0x00000438 (ext4): 53 ef
mount: /mnt/lab16: wrong fs type, bad option, bad superblock on /dev/sdb1, missing
codepage or helper program, or other error.
```

Restore from the compressed image, then mount and check:

```
zcat backups/sdb1.img.gz | sudo dd of=/dev/sdb1 bs=4M status=progress
sudo mount /dev/sdb1 /mnt/lab16 && cat /mnt/lab16/important.txt
sudo umount /mnt/lab16
```

```
0+3200 records in
0+3200 records out
209715200 bytes (210 MB, 200 MiB) copied, 0.9 s, 233 MB/s
important data
```

What just happened: `wipefs` erased the ext4 "magic number", so `mount` could not recognize the file system. `zcat` decompressed the image to standard output, and `sudo dd of=/dev/sdb1` wrote that stream onto the partition. `0+3200 records` means dd received only partial blocks from the pipe (a pipe delivers data in small pieces), which is normal; your numbers may differ. The file system and your file are back. You could restore from the raw image the same way with `sudo dd if=backups/sdb1.img of=/dev/sdb1 bs=4M`.

7. Image the partition with `ddrescue`, the tool you would use on a failing disk.

```
sudo ddrescue /dev/sdb1 backups/sdb1-rescue.img backups/sdb1.map
cat backups/sdb1.map
```

```
GNU ddrescue 1.27
Press Ctrl-C to interrupt
     ipos:  209649 kB, non-trimmed:        0 B,  current rate:    209 MB/s
     opos:  209649 kB, non-scraped:        0 B,  average rate:    209 MB/s
non-tried:        0 B,  bad-sector:        0 B,    error rate:       0 B/s
  rescued:  209715 kB,   bad areas:        0,        run time:          0s
pct rescued:  100.00%, read errors:        0,  remaining time:         n/a
                              time since last successful read:         n/a
Finished
# Mapfile. Created by GNU ddrescue version 1.27
# Command line: ddrescue /dev/sdb1 backups/sdb1-rescue.img backups/sdb1.map
...
#      pos        size  status
0x00000000  0x0C800000  +
```

What just happened: The arguments are input, output, and *mapfile*. ddrescue copied everything and wrote the mapfile, which records each area of the disk and its status: `+` means rescued, `-` bad sector, `?` not tried yet, `*` and `/` partly processed. On a dying disk you would see read errors and bad areas here. Unlike `dd`, which stops at the first read error (or, with `conv=noerror,sync`, pads bad blocks with zeros and moves on blindly), ddrescue skips bad areas quickly, rescues the healthy data first, then goes back to retry. Typical rescue plan: `ddrescue -n` (fast first pass, no scraping), then `ddrescue -r3` (retry bad areas three times) with the **same** mapfile. Always write the rescue image to a different, healthy disk.

8. Run the same command again to see the mapfile at work, and confirm the image is correct.

```
sudo ddrescue /dev/sdb1 backups/sdb1-rescue.img backups/sdb1.map
sudo cmp /dev/sdb1 backups/sdb1-rescue.img && echo "IMAGE IDENTICAL"
```

```
...
Finished
IMAGE IDENTICAL
```

What just happened: Because the mapfile says everything is already rescued, ddrescue finished immediately without re-reading the disk. After an interruption (Ctrl+C, a crash, a disk that needs to cool down), the same command resumes exactly where it stopped. `cmp` compares the partition and the image byte by byte and prints nothing when they match. (You compare with the partition itself, not with `sdb1.img`, because every mount since step 3 updated a few bytes in the file system's superblock, such as the last mount time.) ddrescue refuses to overwrite a *device* unless you add `-f` (force), a safety net against swapping input and output.

Checkpoint: `lsblk /dev/sdb` shows `sdb1`, `sudo mount /dev/sdb1 /mnt/lab16 && cat /mnt/lab16/important.txt` prints `important data` (unmount afterwards with `sudo umount /mnt/lab16`), and `cmp` reported the rescue image identical to the partition.

Challenge

CHALLENGE

1. Write a one-line command that creates a dated, xz-compressed tar backup of `/etc/ssh` and `/etc/netplan` in `~/labs/1.6/backups`, then a second command that proves the backup can be restored by extracting it to a new scratch directory and comparing it with `diff -r`.
2. Using `find`, `cpio`, and `gzip` in one pipeline, archive only the files under `/etc` modified in the last 7 days into `recent-etc.cpio.gz`, then list the archive contents with `zcat` and `cpio -it`.
3. Design a 3-2-1 backup plan for ubuntu-lab using only tools from this lab: which tool makes each copy, where each copy lives, how often it runs, and how you would test a restore.

Clean up

1. Make sure nothing is mounted, then wipe the lab disk so later labs start with a blank disk.

```
sudo umount /mnt/lab16 2>/dev/null
sudo wipefs -a /dev/sdb1
sudo wipefs -a /dev/sdb
sudo rmdir /mnt/lab16
lsblk /dev/sdb
```

   `lsblk` should show `sdb` with no partitions. (Substitute your disk name if it is not `/dev/sdb`.)

2. Delete the lab directory. Several files belong to root (created with `sudo`), so use `sudo`.

```
sudo rm -rf ~/labs/1.6
```

3. Remove the copy on rocky-lab.

```
# on rocky-lab
rm -rf ~/backup-1.6
```

4. The installed packages (`7zip`, `gddrescue`, `rsync`, `zip`, and so on) are harmless and later labs may use them, so you can keep them.

Exam corner

- `tar` modes: `-c` create, `-x` extract, `-t` list; `-f file` names the archive; `-v` verbose; `-C dir` extracts into `dir`; `-p` preserves permissions; `--exclude=PATTERN` skips files.
- `tar` compression: `-z` gzip (`.tar.gz` / `.tgz`), `-j` bzip2 (`.tar.bz2`), `-J` xz (`.tar.xz`). GNU tar strips the leading `/` so restores go into the current directory.
- Incremental tar: `--listed-incremental=file.snar`; restore the full archive first, then each incremental in order (with `--listed-incremental=/dev/null`).

- `cpio` reads file names from stdin: `find dir | cpio -ov > a.cpio` (copy-out), `cpio -idv < a.cpio` (copy-in, extract), `cpio -it < a.cpio` (list), `cpio -pdm dest` (pass-through copy). initramfs images are cpio archives.
- Compression speed versus size: `gzip` (fastest, largest) < `bzip2` < `xz` (slowest, smallest). Decompress with `gunzip` / `gzip -d`, `bunzip2` / `bzip2 -d`, `unxz` / `xz -d`; `-k` keeps the original, `-c` writes to stdout, `-1` to `-9` set the level.
- 7-Zip: `7z a` add, `7z l` list, `7z x` extract (`-oDIR`), `7z t` test; Ubuntu 24.04 package `7zip`. `zip -r` and `unzip` (`-l` list, `-d DIR` destination) for Windows-friendly archives.
- `zcat`, `zgrep`, `zless` read gzip files without decompressing them on disk; `bzcat` / `xzcat` are the bzip2/xz equivalents.
- `rsync -av src/ dest/`: `-a` keeps permissions, times, links, ownership; a trailing slash on the source copies its *contents*; `--delete` mirrors deletions; `--dry-run` / `-n` previews; `-z` compresses; `user@host:path` uses SSH.
- `dd if=INPUT of=OUTPUT bs=SIZE count=N status=progress`: block-level copy. MBR = first 512 bytes (`bs=512 count=1`). Swapping `if` and `of` destroys data.
- `ddrescue INPUT OUTPUT MAPFILE` images failing disks, skips bad areas first, and resumes using the mapfile; package `gddrescue` on Ubuntu.
- 3-2-1 rule: 3 copies, 2 media types, 1 off-site. A backup is not proven until a restore has been tested.

Review questions

1. Which command creates a bzip2-compressed archive of `/home` named `home.tar.bz2`?
   A. `tar -czf home.tar.bz2 /home`
   B. `tar -cjf home.tar.bz2 /home`
   C. `tar -cJf home.tar.bz2 /home`
   D. `tar -xjf home.tar.bz2 /home`

2. You run `rsync -a /srv/web /backup/` and find your files in `/backup/web/`, but you wanted them directly in `/backup/`. What should you change?

3. An administrator needs to search last week's compressed log `/var/log/auth.log.2.gz` for "Failed password" without decompressing it. Which command should be used?
   A. `grep "Failed password" /var/log/auth.log.2.gz`
   B. `zcat -l /var/log/auth.log.2.gz`
   C. `zgrep "Failed password" /var/log/auth.log.2.gz`
   D. `gunzip /var/log/auth.log.2.gz`

4. Which command backs up only the master boot record of `/dev/sda` to a file?
   A. `dd if=mbr.bin of=/dev/sda bs=512 count=1`
   B. `dd if=/dev/sda of=mbr.bin bs=512 count=1`
   C. `dd if=/dev/sda of=mbr.bin bs=1M`
   D. `cpio -o /dev/sda > mbr.bin`

5. A disk is failing with read errors. Why is `ddrescue` with a mapfile a better choice than plain `dd` for imaging it?

6. Which tool combination creates a cpio archive of the `/data` directory?
   A. `cpio -i /data > data.cpio`
   B. `tar -c /data | cpio -o > data.cpio`
   C. `find /data | cpio -o > data.cpio`
   D. `cpio -t /data > data.cpio`

Lab 1.7 Run Virtual Machines with QEMU, KVM, and libvirt

Objective: 1.7 Summarize virtualization on Linux systems.
Domain: 1.0 System Management (23% of the exam)
Time: about 120 minutes (longer without hardware acceleration)
VM: ubuntu-lab
Snapshot first: Yes. This lab installs the virtualization stack and creates virtual networks.

ubuntu-lab is itself a virtual machine, and in this lab it becomes a virtualization *host* too: you will run tiny CirrOS virtual machines inside it. Along the way you will inspect, convert, and resize disk images, build a baseline template with linked clones, drive VMs through their states with `virsh`, change their CPU, RAM, storage, and network, take and revert snapshots, clone a VM, practise the steps of a migration, and build a private virtual network. Almost every Linux server today runs on a hypervisor or hosts VMs, so these are core skills, and the exam expects you to know the vocabulary.

What you'll practice

- Hypervisors: QEMU, KVM
- VMs: paravirtualized drivers, VirtIO; disk image operations (convert, resize, image properties); VM states; nested virtualization
- VM operations: resources (storage, RAM, CPU, network); baseline image templates; cloning; migrations; snapshots
- Bare metal vs. virtual machines
- Network types: bridged, NAT, host-only/isolated, routed, open
- Tools: libvirt, `virsh`, `virt-manager`

How it works

Bare metal versus virtual machines. A *bare metal* server runs one operating system directly on the hardware. A *virtual machine* (VM) is a complete computer made of software: virtual CPUs, RAM, disks, and network cards. A program called a **hypervisor** shares one physical machine between many VMs and keeps them isolated from each other. A *type 1* hypervisor runs directly on the hardware (VMware ESXi, Microsoft Hyper-V, Xen, and Linux with KVM). A *type 2* hypervisor runs as an application on a normal desktop OS (VirtualBox, VMware Workstation). The physical machine is the *host*; each VM is a *guest*.

The Linux virtualization stack. Linux splits the job into layers, and the exam expects you to know each name:

```
virt-manager (GUI)   virsh (CLI)   virt-install / virt-clone
        \                |                /
         +---------- libvirt API ---------+      management layer
                          |                      (libvirtd daemon, XML configs)
              +-------------------------+
              | QEMU process (one/VM)   |         emulates the machine:
              |  VirtIO disk, VirtIO    |         disks, NICs, BIOS, display
              |  net, serial console    |
              +-------------------------+
                          |  /dev/kvm
              +-------------------------+
              | KVM kernel module       |         runs guest code directly on
              | (kvm_intel / kvm_amd)   |         the CPU with Intel VT-x / AMD-V
              +-------------------------+
                     Linux kernel  ->  hardware
```

- **KVM** (Kernel-based Virtual Machine) is a Linux kernel module. It uses the CPU's virtualization extensions (Intel VT-x, flag `vmx`; AMD-V, flag `svm`) so guest code runs on the real CPU at near-native speed.
- **QEMU** is a user-space program that emulates a whole computer: firmware, disk controllers, network cards, display. With KVM it is fast. Without KVM it can still run guests using **TCG** (Tiny Code Generator), which translates every guest instruction in software. TCG always works but is roughly 5 to 20 times slower.
- **libvirt** is a management layer (the `libvirtd` daemon plus an API) that stores each VM's configuration as XML and talks to QEMU for you. **virsh** is its command-line tool, **virt-install** creates VMs, **virt-clone** copies them, and **virt-manager** is the graphical tool.

Nested virtualization. Running a hypervisor inside a VM is *nested virtualization*. For KVM to work inside ubuntu-lab, the outer hypervisor (VirtualBox, VMware, Hyper-V, or KVM on your PC) must pass the CPU virtualization extensions through to ubuntu-lab. If it does not, this lab falls back to TCG automatically and everything still works, only slower.

Paravirtualized drivers and VirtIO. QEMU can pretend to be real, well-known hardware, such as an Intel e1000 network card or an IDE disk. That is *full emulation*: any OS has drivers for it, but every disk or network operation is translated, which is slow. With *paravirtualization*, the guest knows it is virtual and uses special drivers designed for a hypervisor. On KVM those drivers are **VirtIO**: `virtio-blk` / `virtio-scsi` for disks (they appear as `/dev/vda`), `virtio-net` for networking, `virtio-balloon` for returning memory, and `virtio-rng` for randomness. Linux has VirtIO drivers built in; Windows guests need the `virtio-win` driver ISO.

Disk images. A VM's disk is usually a file. A **raw** image is a byte-for-byte disk. **qcow2** (QEMU copy-on-write version 2) is QEMU's native format: it grows as data is written (thin provisioning), supports compression, internal snapshots, and *backing files*. A backing file is the key to templates: you keep one read-only **baseline image** and create small qcow2 *overlay* files on top of it. Each overlay stores only the blocks that VM changes. That is a *linked clone*. A *full clone* is an independent copy of the whole disk.

```
cirros-base.qcow2  (baseline template, never booted directly)
       ^                ^
       |  backing file  |
cirros1.qcow2      (other overlays...)
(only cirros1's changes)
```

VM states. libvirt reports a state for every VM: *running*, *idle* (running but waiting), *paused* (suspended in RAM, CPU stopped), *in shutdown*, *shut off*, *crashed*, and *pmsuspended* (the guest suspended itself to RAM). A *snapshot* records a VM's disk (and optionally memory) at a moment so you can return to it. *Migration* moves a VM to another host: *cold/offline* migration moves a stopped VM, *live* migration moves a running VM with only a brief pause.

Virtual network types.

Type	Guests can reach outside?	Outside can reach guests?	Typical use
NAT	Yes, through the host's IP	No (unless you forward ports)	Default; laptops and labs
Bridged	Yes, directly	Yes, guests are on the LAN with their own IPs	Servers that need real LAN addresses
Host-only / isolated	No	Only the host and other guests	Test networks, security labs
Routed	Yes, routed by the host without NAT	Yes, if the LAN router has a route back	Data centers, public subnets
Open	Whatever you configure	Whatever you configure	libvirt adds no firewall rules at all

Tasks

Task 1: Check for hardware virtualization and nested support

1. Look for the CPU virtualization flags.

```
grep -cE 'vmx|svm' /proc/cpuinfo
lscpu | grep -i virtualization
```

```
2
Virtualization:                       VT-x
```

> **What just happened:** `/proc/cpuinfo` lists every CPU core with its feature flags. `grep -c` counts the lines that contain `vmx` (Intel VT-x) or `svm` (AMD-V); `-E` allows the `|` ("or"). A number greater than 0 (usually your vCPU count) means the extensions are visible inside ubuntu-lab. `0` means your outer hypervisor is not passing them through. `lscpu` shows the same information in a friendlier form; with no extensions it prints nothing, or shows a `Hypervisor vendor:` line only.

2. Install `cpu-checker` and ask `kvm-ok` for a verdict.

```
sudo apt update
sudo apt install -y cpu-checker
sudo kvm-ok
```

With nested virtualization working:

```
INFO: /dev/kvm exists
KVM acceleration can be used
```

Without it:

```
INFO: Your CPU does not support KVM extensions
KVM acceleration can NOT be used
```

What just happened: `kvm-ok` (an Ubuntu tool from the `cpu-checker` package) checks the CPU flags and whether `/dev/kvm`, the device QEMU uses to talk to the KVM module, exists. Either result is fine for this lab. If KVM is available you will create VMs with `--virt-type kvm`; if not, with `--virt-type qemu` (TCG software emulation).

3. If you want KVM acceleration and `kvm-ok` said no, shut down ubuntu-lab and enable nested virtualization in the program that runs it, then boot and re-run `sudo kvm-ok`:
   - **VirtualBox:** VM Settings > System > Processor > tick **Enable Nested VT-x/AMD-V**. If it is greyed out, run `VBoxManage modifyvm "ubuntu-lab" --nested-hw-virt on` on your PC while the VM is off.
   - **VMware Workstation / Fusion:** VM Settings > Processors > tick **Virtualize Intel VT-x/EPT or AMD-V/RVI**. (ESXi: "Expose hardware assisted virtualization to the guest OS".)
   - **Hyper-V:** in an elevated PowerShell on the host, with the VM off: `Set-VMProcessor -VMName ubuntu-lab -ExposeVirtualizationExtensions $true`.
   - **KVM/libvirt or Proxmox on Linux:** give the VM the CPU model `host-passthrough` (Proxmox: CPU type `host`), and check `cat /sys/module/kvm_intel/parameters/nested` (or `kvm_amd`) on the physical host shows `Y` or `1`.

> **TIP**
> On Windows PCs with Hyper-V, WSL2, or "Core isolation / Memory integrity" turned on, VirtualBox and VMware run on top of Hyper-V and often cannot offer nested VT-x. You do not need to change your PC: this lab works without KVM, just more slowly.

4. Save the right virtualization type in a shell variable so later commands work either way.

```
if [ -e /dev/kvm ]; then VTYPE=kvm; else VTYPE=qemu; fi
echo "This lab will use --virt-type $VTYPE"
```

What just happened: `[ -e /dev/kvm ]` tests whether the KVM device file exists. The `if` sets `VTYPE` to `kvm` or `qemu`. Shell variables vanish when you log out, so if you log out later in the lab, run this line again.

Checkpoint: You know whether your ubuntu-lab can use KVM (`sudo kvm-ok`), and `echo $VTYPE` prints `kvm` or `qemu`.

Task 2: Install QEMU, KVM, and libvirt

1. Install the virtualization stack.

```
sudo apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system \
    libvirt-clients virtinst
```

> **What just happened:** `qemu-system-x86` is the QEMU emulator for x86 guests (it also provides the virtual package name `qemu-kvm`, which is what the package is called on Rocky/RHEL). `qemu-utils` provides `qemu-img`, the disk image tool. `libvirt-daemon-system` installs and starts the libvirt daemon and creates the default NAT network. `libvirt-clients` provides `virsh`. `virtinst` provides `virt-install` and `virt-clone`. The KVM kernel modules are already part of Ubuntu's kernel.

2. Check that the daemon is running and the KVM modules are loaded.

```
systemctl status libvirtd --no-pager | head -n 3
lsmod | grep kvm
```

```
● libvirtd.service - libvirt legacy monolithic daemon
     Loaded: loaded (/usr/lib/systemd/system/libvirtd.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-10 11:02:13 UTC; 1min ago
kvm_intel             487424  0
kvm                  1409024  1 kvm_intel
```

> **What just happened:** `libvirtd` is running. On Ubuntu 24.04 it is *socket-activated* and started with `--timeout 120`, so if you check again a few minutes later you may see `inactive (dead)` with a `TriggeredBy: libvirtd.socket` line. That is normal: the daemon exits when idle and systemd starts it again the moment `virsh` connects. `lsmod` lists loaded kernel modules: `kvm` is the core module and `kvm_intel` (or `kvm_amd`) is the CPU-specific part. If ubuntu-lab has no virtualization extensions, the modules may be missing; that is expected with TCG.

3. Let `student` manage system VMs, and point `virsh` at the system instance.

```
sudo usermod -aG libvirt student
echo 'export LIBVIRT_DEFAULT_URI=qemu:///system  # lab 1.7' >> ~/.bashrc
```

Log out and log back in (group changes only apply to new logins), then:

```
if [ -e /dev/kvm ]; then VTYPE=kvm; else VTYPE=qemu; fi
id | grep -o libvirt
echo "$LIBVIRT_DEFAULT_URI"
virsh list --all
```

```
libvirt
qemu:///system
 Id   Name   State
--------------------
```

What just happened: Members of the `libvirt` group may control the system-wide libvirt daemon without `sudo`. libvirt has two *connection URIs*: `qemu:///system` (VMs managed by the system daemon, with access to system networks such as the default NAT network) and `qemu:///session` (per-user VMs with limited networking). A normal user's `virsh` uses `qemu:///session` unless told otherwise, so you set the `LIBVIRT_DEFAULT_URI` environment variable in `~/.bashrc`. (`virsh -c qemu:///system ...` does the same for one command.) The empty table means you have no VMs yet.

4. Run libvirt's own host check.

```
virt-host-validate qemu
```

```
  QEMU: Checking for hardware virtualization                 : PASS
  QEMU: Checking if device '/dev/kvm' exists                 : PASS
  QEMU: Checking if device '/dev/kvm' is accessible          : PASS
  QEMU: Checking if device '/dev/vhost-net' exists           : PASS
  ...
```

Without nested virtualization the first lines show `FAIL` or `WARN`; with TCG you can ignore them. Lines about IOMMU or secure guest support are also fine to ignore in this lab.

What just happened: `virt-host-validate` (from `libvirt-clients`) runs a list of checks a virtualization host should pass: CPU extensions, the `/dev/kvm` device and its permissions, `/dev/vhost-net` (a kernel accelerator for VirtIO networking), cgroup controllers used to limit VM resources, and IOMMU support (needed only for passing physical PCI devices such as GPUs to VMs). It is a handy first step when VMs refuse to start on a new host.

Checkpoint: `virsh uri` prints `qemu:///system`, and `virsh list --all` runs without `sudo` and without errors.

Task 3: Download a cloud image and inspect, convert, and resize it

CirrOS is a tiny Linux distribution (about 20 MB) built for testing clouds. It boots in a few seconds with KVM, needs very little RAM, and comes as a ready-made disk image, so you do not need to run an installer.

1. Download the image into the lab directory.

```
mkdir -p ~/labs/1.7 && cd ~/labs/1.7
wget https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img
du -h cirros-0.6.2-x86_64-disk.img
```

```
21M     cirros-0.6.2-x86_64-disk.img
```

What just happened: `wget` downloaded the file over HTTPS into the current directory. `du -h` shows how much disk space it uses in human-readable units. The whole operating system fits in about 20 MB.

2. Look at the image properties with `qemu-img info`.

```
qemu-img info cirros-0.6.2-x86_64-disk.img
```

```
image: cirros-0.6.2-x86_64-disk.img
file format: qcow2
virtual size: 112 MiB (117440512 bytes)
disk size: 20.4 MiB
cluster_size: 65536
Format specific information:
    compat: 1.1
    compression type: zlib
    lazy refcounts: false
    refcount bits: 16
    corrupt: false
    extended l2: false
...
```

What just happened: Even though the file ends in `.img`, `qemu-img` read its header and found it is **qcow2**; never trust the extension. The **virtual size** (112 MiB) is how big the disk looks to the guest. The **disk size** (about 20 MiB) is how much space the file really uses on the host, because qcow2 only stores blocks that contain data. `cluster_size` is qcow2's allocation unit, `compat: 1.1` is the qcow2 version, and `corrupt: false` is a quick health indicator. `qemu-img check FILE` performs a full consistency check. Your exact numbers may differ slightly.

3. Convert the image from qcow2 to raw, and compare the two.

```
qemu-img convert -p -f qcow2 -O raw cirros-0.6.2-x86_64-disk.img cirros.raw
qemu-img info cirros.raw
ls -lhs cirros-0.6.2-x86_64-disk.img cirros.raw
```

```
    (100.00/100%)
image: cirros.raw
file format: raw
virtual size: 112 MiB (117440512 bytes)
disk size: 41.2 MiB
...
 21M -rw-rw-r-- 1 student student  21M Oct 10 11:08 cirros-0.6.2-x86_64-disk.img
 42M -rw-r--r-- 1 student student 112M Oct 10 11:10 cirros.raw
```

What just happened: `qemu-img convert` copies an image from one format to another: `-f` is the input format, `-O` (capital O) the output format, and `-p` shows progress. The raw file's *apparent* size (112M in the `ls -l` column) equals the virtual size, because raw is a plain byte-for-byte disk. `ls -s` (the first column) shows the blocks really used: `qemu-img` wrote a *sparse* file, skipping the empty areas, so it uses less than 112 MiB, but more than the qcow2 file, which is also compressed. Raw images are simple and slightly faster; qcow2 images add snapshots, backing files, and compression. Other formats `qemu-img` can convert to include `vmdk` (VMware), `vdi` (VirtualBox), and `vhdx` (Hyper-V), which is how you move VMs between hypervisors.

4. Convert back to qcow2 with compression, then resize a copy.

```
qemu-img convert -c -f raw -O qcow2 cirros.raw cirros-compressed.qcow2
cp cirros-compressed.qcow2 cirros-big.qcow2
qemu-img resize cirros-big.qcow2 1G
qemu-img info cirros-big.qcow2 | head -n 4
```

```
Image resized.
image: cirros-big.qcow2
file format: qcow2
virtual size: 1 GiB (1073741824 bytes)
disk size: 20.6 MiB
```

What just happened: `-c` compresses the qcow2 output. `qemu-img resize` changed the *virtual* size to 1 GiB, but the file stayed small because the new space is empty. Growing the virtual disk is only step one: the guest must also grow its partition and file system (with `growpart` and `resize2fs` or `xfs_growfs`; CirrOS and most cloud images do this automatically at boot). Shrinking needs `--shrink` and destroys data unless the guest's partitions were shrunk first, so always back up before shrinking. Never resize an image that a running VM is using.

Checkpoint: `qemu-img info cirros.raw | grep format` prints `file format: raw`, and `qemu-img info cirros-big.qcow2 | grep virtual` shows `1 GiB`.

Task 4: Build a baseline template and create a VM with VirtIO devices

1. Install the baseline image into libvirt's image directory, sized to 1 GiB.

```
sudo cp ~/labs/1.7/cirros-big.qcow2 /var/lib/libvirt/images/cirros-base.qcow2
sudo ls -l /var/lib/libvirt/images
```

What just happened: `/var/lib/libvirt/images` is libvirt's standard location for disk images. Using it avoids permission problems: QEMU runs as the user `libvirt-qemu`, which cannot read files inside your home directory (on Ubuntu 24.04, home directories are mode `750`), and the AppArmor profile for VMs expects images here. `cirros-base.qcow2` is now your **baseline image template**. You will never boot it directly; you build VMs on top of it. In production, a template is a fully patched, configured "golden image", cleaned of machine-specific data (SSH host keys, machine ID, logs) with a tool such as `virt-sysprep` from the `libguestfs-tools` package.

2. Create a linked clone: a small overlay disk that uses the template as its backing file.

```
sudo qemu-img create -f qcow2 -F qcow2 \
    -b /var/lib/libvirt/images/cirros-base.qcow2 \
    /var/lib/libvirt/images/cirros1.qcow2
sudo qemu-img info --backing-chain /var/lib/libvirt/images/cirros1.qcow2 | \
    grep -E '^(image|backing file|disk size):'
```

```
Formatting '/var/lib/libvirt/images/cirros1.qcow2', fmt=qcow2 cluster_size=65536 ...
image: /var/lib/libvirt/images/cirros1.qcow2
disk size: 196 KiB
backing file: /var/lib/libvirt/images/cirros-base.qcow2
image: /var/lib/libvirt/images/cirros-base.qcow2
disk size: 20.6 MiB
```

What just happened: `qemu-img create -f qcow2` makes a new qcow2 file. `-b` names the backing file, and `-F` states its format (QEMU 8 refuses to guess it, for security). No size was given, so the overlay inherits the template's 1 GiB. The `grep` pattern keeps only lines that *start* with `image:`, `backing file:`, or `disk size:` (`^` means start of line), which skips the indented details `qemu-img` also prints. The overlay is only 196 KiB: every read of an unchanged block comes from the template, and every write goes into the overlay. Ten VMs built this way cost ten tiny files instead of ten full copies. The catch: the template must never change or move, or every overlay built on it breaks. Use an absolute path for `-b`.

3. Create the VM with `virt-install`, using VirtIO for both disk and network.

```
virt-install \
    --name cirros1 \
    --virt-type "$VTYPE" \
    --memory 256 \
    --vcpus 1 \
    --disk path=/var/lib/libvirt/images/cirros1.qcow2,format=qcow2,bus=virtio \
    --network network=default,model=virtio \
    --osinfo cirros0.5.2 \
    --import \
    --graphics none \
    --noautoconsole
```

```
Starting install...
Creating domain...
Domain creation completed.
```

What just happened: `virt-install` wrote a libvirt XML definition and started the VM (libvirt calls a VM a *domain*). `--virt-type` chooses KVM or TCG. `--memory 256` gives 256 MiB of RAM and `--vcpus 1` one virtual CPU. `--disk ...,bus=virtio` attaches your overlay as a paravirtualized VirtIO disk, and `--network network=default,model=virtio` connects a VirtIO network card to libvirt's default NAT network. `--osinfo cirros0.5.2` tells libvirt which OS to tune defaults for (the newest CirrOS entry in Ubuntu 24.04's OS database; `--osinfo detect=on,require=off` would let it guess instead). `--import` means "the disk already has an OS, skip the installer". `--graphics none` gives the VM a serial console instead of a graphical display, and `--noautoconsole` returns your prompt instead of attaching to it.

4. Connect to the VM's console and log in.

```
virsh console cirros1
```

Press Enter if the screen is quiet. With KVM the login prompt appears within seconds; with TCG, boot can take 1 to 3 minutes (CirrOS waits for a cloud metadata service that does not exist, then continues). Log in as `cirros` with password `gocubsgo` (CirrOS prints these on the console).

```
Connected to domain 'cirros1'
Escape character is ^] (Ctrl + ])

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$
```

5. Inside the guest, confirm the VirtIO devices and the network.

```
ls /sys/bus/virtio/drivers
cat /proc/partitions
ip -4 addr show eth0
ping -c 2 192.168.122.1
```

```
...  virtio_blk  ...  virtio_net  ...
major minor  #blocks  name
 252        0    1048576 vda
 252        1    1035247 vda1
 ...
    inet 192.168.122.57/24 brd 192.168.122.255 scope global eth0
2 packets transmitted, 2 packets received, 0% packet loss
```

What just happened: You ran these commands *inside the guest*. The `virtio_blk` and `virtio_net` drivers are registered on the VirtIO bus (the exact list of drivers varies), and the disk appears as `vda` (`vd` = VirtIO disk; an emulated SATA disk would be `sda`). The guest grew its root partition to fill the 1 GiB disk at boot. It received a `192.168.122.x` address by DHCP from libvirt's NAT network and can reach the host side, `192.168.122.1`. Your address will differ.

6. Leave the console with **Ctrl+]** (hold Ctrl and press the closing square bracket). On some non-US keyboards use Ctrl+5. The VM keeps running.

7. Back on ubuntu-lab, look at the VM from the host side.

```
virsh list --all
virsh domifaddr cirros1
virsh dumpxml cirros1 | grep -E "bus='virtio'|model type="
```

```
 Id   Name      State
------------------------
 1    cirros1   running
 Name       MAC address          Protocol     Address
-------------------------------------------------------------------------------
 vnet0      52:54:00:3a:91:c2    ipv4         192.168.122.57/24
      <target dev='vda' bus='virtio'/>
      <model type='virtio'/>
```

What just happened: `virsh list --all` shows running *and* stopped VMs (without `--all`, only running ones). `domifaddr` shows the guest's IP address from the DHCP lease. `dumpxml` prints the full XML definition that libvirt stores in `/etc/libvirt/qemu/cirros1.xml`; the lines you filtered confirm VirtIO for disk and NIC. Edit a definition safely with `virsh edit cirros1`, which validates the XML when you save.

TIP

If `virt-install` fails with "Network not found: no network with matching name 'default'" or the network is inactive, run `virsh net-start default` and `virsh net-autostart default`. If starting it fails with "Network is already in use by interface", ubuntu-lab's own network uses `192.168.122.0/24` (common when your PC also runs libvirt). Run `virsh net-edit default`, change every `192.168.122` to `192.168.123`, save, and start the network again.

Checkpoint: `virsh list --all` shows `cirros1` as `running`, and from ubuntu-lab `ping -c 2` to the address from `virsh domifaddr cirros1` succeeds.

Task 5: Move a VM through its states

1. Check the current state and details.

```
virsh domstate cirros1
virsh dominfo cirros1
```

```
running

Id:             1
Name:           cirros1
UUID:           6f1d3c2a-...
OS Type:        hvm
State:          running
CPU(s):         1
CPU time:       14.2s
Max memory:     262144 KiB
Used memory:    262144 KiB
Persistent:     yes
Autostart:      disable
Managed save:   no
Security model: apparmor
...
```

What just happened: `domstate` prints just the state. `dominfo` adds the ID (a number given only while running), the UUID (permanent unique ID), `hvm` (hardware virtual machine, a full VM rather than a container), vCPU count, memory in KiB (262144 KiB = 256 MiB), whether the VM is *persistent* (has a saved definition, so it survives shutdown), and the AppArmor security model that confines the QEMU process.

2. Pause and resume the VM.

```
virsh suspend cirros1
virsh domstate cirros1
virsh resume cirros1
virsh domstate cirros1
```

```
Domain 'cirros1' suspended

paused

Domain 'cirros1' resumed

running
```

What just happened: `suspend` freezes the VM's virtual CPUs; its memory stays in the host's RAM, and it stops responding (try pinging it while paused). `resume` continues exactly where it stopped. This is useful for a moment of consistency (for example before copying a disk) or to free CPU time temporarily.

3. Shut it down gracefully, then start it again.

```
virsh shutdown cirros1
virsh list --all
```

Wait a few seconds and run `virsh list --all` again until the state is `shut off`.

```
Domain 'cirros1' is being shutdown

 Id   Name      State
--------------------------
 -    cirros1   shut off
```

```
virsh start cirros1
```

What just happened: `shutdown` sends an ACPI power-button event, and the guest OS shuts itself down cleanly, like pressing the power button briefly on a PC. While it does that, the state may show `in shutdown`. A stopped VM has no ID (`-`). `start` boots it again. `virsh reboot` performs a clean restart. If a guest ignores the ACPI request, use the next step.

4. Force it off.

```
virsh destroy cirros1
virsh domstate cirros1
```

```
Domain 'cirros1' destroyed

shut off
```

What just happened: Despite the scary name, `destroy` does *not* delete anything. It is a hard power-off, like pulling the plug: immediate, but the guest may lose unsaved data. Deleting a VM definition is `virsh undefine`. Other states you may see are `crashed` (the guest OS crashed), `pmsuspended` (the guest suspended itself to RAM), and saved state: `virsh managedsave` writes the RAM to disk and stops the VM, and the next `virsh start` resumes from that point.

Checkpoint: You have seen cirros1 in the `running`, `paused`, and `shut off` states, and you can explain the difference between `virsh shutdown`, `virsh destroy`, and `virsh undefine`.

Task 6: Change CPU, RAM, storage, and network resources

1. With cirros1 shut off, raise its RAM to 384 MiB and its vCPUs to 2. These changes go into the saved configuration (`--config`) and apply at the next boot.

```
virsh setmaxmem cirros1 384M --config
virsh setmem cirros1 384M --config
virsh setvcpus cirros1 2 --maximum --config
virsh setvcpus cirros1 2 --config
virsh dominfo cirros1 | grep -E 'CPU\(s\)|memory'
```

```
CPU(s):         2
Max memory:     393216 KiB
Used memory:    393216 KiB
```

What just happened: Memory has two values: *max memory* (the ceiling, fixed while the VM runs) and *current memory*. You raised the ceiling with `setmaxmem`, then the current amount with `setmem`. vCPUs work the same way: `--maximum` sets the ceiling, then the plain command sets the active count. `--config` changes the persistent definition, `--live` changes a running VM (possible for memory below the maximum, via the VirtIO balloon driver, and for CPUs if the guest supports hot-plug), and you can give both. Avoid giving a VM more vCPUs than the host has CPUs.

2. Start the VM and add a second VirtIO disk while it is running (hot-plug).

```
virsh start cirros1
sudo qemu-img create -f qcow2 /var/lib/libvirt/images/cirros1-data.qcow2 1G
virsh attach-disk cirros1 /var/lib/libvirt/images/cirros1-data.qcow2 vdb \
    --driver qemu --subdriver qcow2 --targetbus virtio --persistent
virsh domblklist cirros1
```

```
Domain 'cirros1' started

Formatting '/var/lib/libvirt/images/cirros1-data.qcow2', fmt=qcow2 ... size=1073741824 ...
Disk attached successfully

 Target   Source
------------------------------------------------------
 vda      /var/lib/libvirt/images/cirros1.qcow2
 vdb      /var/lib/libvirt/images/cirros1-data.qcow2
```

What just happened: You created an empty 1 GiB thin-provisioned disk and attached it as `vdb`. `--driver qemu --subdriver qcow2` tells libvirt the file format, `--targetbus virtio` makes it a VirtIO disk, and `--persistent` applies the change to both the running VM and its saved definition. `domblklist` lists the VM's disks.

3. Check the new resources inside the guest. Connect with `virsh console cirros1` (press Enter, log in as `cirros` / `gocubsgo` if needed), then:

```
grep -c ^processor /proc/cpuinfo
grep MemTotal /proc/meminfo
cat /proc/partitions | grep vd
```

```
2
MemTotal:         347764 kB
252        0    1048576 vda
252        1    1035247 vda1
...
252       16    1048576 vdb
```

Leave the console with Ctrl+]. (`MemTotal` is a little less than 384 MiB because the guest kernel reserves some memory for itself.)

4. Look at the VM's network resources from the host, and detach the extra disk.

```
virsh domiflist cirros1
virsh net-dhcp-leases default
virsh detach-disk cirros1 vdb --persistent
virsh domblklist cirros1
```

```
 Interface   Type      Source    Model    MAC
-------------------------------------------------------------
 vnet1       network   default   virtio   52:54:00:3a:91:c2
...
Disk detached successfully
```

> **What just happened:** `domiflist` lists the VM's network interfaces: `vnet1` is the host-side end of the virtual cable, connected to the `default` network, with a VirtIO model and a MAC address starting with `52:54:00` (the prefix QEMU uses). `net-dhcp-leases` shows the addresses libvirt's DHCP server handed out. You can add a NIC the same way you added a disk, with `virsh attach-interface` (you will in Task 8). Detaching `vdb` removed it from the VM, but the image file is still there.

5. Look at libvirt's storage pools (where it keeps images).

```
virsh pool-list --all
virsh vol-list default
```

If `pool-list` shows no `default` pool, that is fine; it is created the first time a tool needs it. A *storage pool* is a place libvirt manages volumes in: a directory, an LVM volume group, an iSCSI target, or NFS share.

> **Checkpoint:** `virsh dominfo cirros1` shows 2 CPUs and 393216 KiB, and `virsh domblklist cirros1` lists only `vda` again.

Task 7: Clone a VM and use snapshots

1. Shut down cirros1, then make a full clone of it with `virt-clone`.

```
virsh shutdown cirros1
```

Wait until `virsh domstate cirros1` prints `shut off` (use `virsh destroy cirros1` if it takes more than a minute), then:

```
virt-clone --original cirros1 --name cirros2 --auto-clone
virsh list --all
virsh domblklist cirros2
virsh domiflist cirros1 | grep -o '52:54[0-9a-f:]*'
virsh domiflist cirros2 | grep -o '52:54[0-9a-f:]*'
```

```
Allocating 'cirros2.qcow2'                                  | 1.0 GB  00:00:01

Clone 'cirros2' created successfully.
 Id   Name      State
--------------------------
 -    cirros1   shut off
 -    cirros2   shut off
 ...
 vda      /var/lib/libvirt/images/cirros2.qcow2
52:54:00:3a:91:c2
52:54:00:8e:17:4b
```

What just happened: `virt-clone` copied the disk of the stopped VM to a new file and created a new definition with a new name, a new UUID, and a new MAC address, so both VMs can run on the same network without conflict. `--auto-clone` chose the new disk file name for you. The source must be shut off (or paused) so its disk is not changing during the copy. This is a *full clone* made by libvirt; the overlay technique in Task 4 made a *linked clone*. Note that a clone still contains identical guest-level identity (hostname, SSH host keys, machine ID); in production you would clean those with `virt-sysprep` or cloud-init.

2. Take a snapshot of cirros1 before making a change.

```
virsh snapshot-create-as cirros1 before-change \
    --description "Clean state before testing"
virsh snapshot-list cirros1
```

```
Domain snapshot before-change created
 Name            Creation Time               State
------------------------------------------------------
 before-change   2026-10-10 11:40:12 +0000   shutoff
```

What just happened: `snapshot-create-as` created a snapshot with the name and description you gave. Because the disk is qcow2, libvirt stored an *internal* snapshot inside the disk file itself. The state `shutoff` records that the VM was off; a snapshot of a *running* VM also saves its RAM, so reverting returns it to the running moment. Snapshots are not backups: they live in the same file on the same storage, and long snapshot chains slow the VM down. Use them for short-term "undo" points before upgrades or risky changes.

3. Make a change inside the VM. Start it, connect, log in, and create a file:

```
virsh start cirros1
virsh console cirros1
```

Inside the guest:

```
echo "made after the snapshot" > ~/after.txt
ls ~
sync
sudo poweroff
```

The console shows the guest shutting down and then disconnects (or press Ctrl+]).

4. Revert to the snapshot and check that the change is gone.

```
virsh domstate cirros1
virsh snapshot-revert cirros1 before-change
virsh start cirros1
virsh console cirros1
```

Inside the guest, after logging in:

```
ls ~
```

```
$ ls ~
$
```

The home directory is empty again: `after.txt` disappeared. Leave with Ctrl+].

> **What just happened:** `snapshot-revert` threw away every change written since the snapshot and put the disk back to its snapshot state. This is the same idea as the snapshots you take of ubuntu-lab before risky labs. Related commands: `virsh snapshot-info cirros1 before-change` shows details, `virsh snapshot-delete cirros1 before-change` removes a snapshot (merging nothing back), and `virsh snapshot-current cirros1` shows the one you are currently based on.

5. See the snapshot from the image's point of view.

```
virsh destroy cirros1
sudo qemu-img snapshot -l /var/lib/libvirt/images/cirros1.qcow2
```

```
Domain 'cirros1' destroyed

Snapshot list:
ID        TAG               VM SIZE                DATE     VM CLOCK     ICOUNT
1         before-change         0 B 2026-10-10 11:40:12 00:00:00.000          0
```

`VM SIZE 0 B` confirms it is a disk-only snapshot (no RAM saved, because the VM was off).

> **Checkpoint:** `virsh list --all` shows `cirros1` and `cirros2`, both shut off, and `virsh snapshot-list cirros1` shows `before-change`.

Task 8: Explore virtual network types and build an isolated network

1. List libvirt's networks and read the default NAT network's definition.

```
virsh net-list --all
virsh net-dumpxml default
```

```
 Name      State    Autostart   Persistent
----------------------------------------------
 default   active   yes         yes

<network>
  <name>default</name>
  <uuid>...</uuid>
  <forward mode='nat'>
    <nat>
      <port start='1024' end='65535'/>
    </nat>
  </forward>
  <bridge name='virbr0' stp='on' delay='0'/>
  <mac address='52:54:00:...'/>
  <ip address='192.168.122.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='192.168.122.2' end='192.168.122.254'/>
    </dhcp>
  </ip>
</network>
```

What just happened: libvirt networks are defined in XML. `<forward mode='nat'>` makes this a **NAT** network: guest traffic leaving the host is rewritten to use the host's own IP address, like a home router, so guests can reach the internet but outside machines cannot start connections to them. `<bridge name='virbr0'>` is a virtual switch on the host that the guests' `vnet` interfaces plug into. The `<ip>` element gives the host the address `192.168.122.1` on that switch, and `<dhcp>` runs a DHCP server (dnsmasq) for guests.

2. See the host side of the NAT network.

```
ip -br addr show virbr0
sudo nft list ruleset | grep -i -m 3 masquerade
```

```
virbr0           UP             192.168.122.1/24
...masquerade...
```

What just happened: `virbr0` is the bridge with the host's address on the NAT network. libvirt also added firewall rules that *masquerade* guest traffic, which is the NAT. libvirt 10.0 on Ubuntu 24.04 writes these rules with `iptables`, which on Ubuntu is the nftables-based `iptables-nft`, so they also appear in the nftables rule set. If `grep` shows nothing with `nft`, try `sudo iptables -t nat -S | grep -i masq`.

3. Define an **isolated** network, which has no `<forward>` element: guests can talk to each other and to the host, but not beyond. Create the XML with a here doc (the shell copies the lines up to `EOF` into the file):

```
cat > ~/labs/1.7/isolated.xml <<'EOF'
<network>
  <name>isolated-lab</name>
  <bridge name='virbr10'/>
  <ip address='10.10.10.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='10.10.10.100' end='10.10.10.200'/>
    </dhcp>
  </ip>
</network>
EOF
virsh net-define ~/labs/1.7/isolated.xml
virsh net-start isolated-lab
virsh net-list --all
```

```
Network isolated-lab defined from /home/student/labs/1.7/isolated.xml

Network isolated-lab started

 Name           State    Autostart   Persistent
----------------------------------------------------
 default        active   yes         yes
 isolated-lab   active   no          yes
```

What just happened: `net-define` saved a persistent network definition, and `net-start` created the bridge `virbr10` and started DHCP on it. `Autostart: no` means it will not start at boot unless you run `virsh net-autostart isolated-lab`. Because there is no `<forward>` element, libvirt does not route or NAT this network anywhere. In VirtualBox and VMware this kind of network is called **host-only** (if the host has an address on it, as here) or *internal* (if it does not).

4. Give cirros2 a second network card on the isolated network, start it, and request an address.

```
virsh attach-interface cirros2 network isolated-lab --model virtio --config
virsh start cirros2
virsh domiflist cirros2
virsh console cirros2
```

Inside the guest (log in as `cirros` / `gocubsgo`):

```
sudo ip link set eth1 up
sudo udhcpc -i eth1
ip -4 addr show eth1
ping -c 2 10.10.10.1
```

```
udhcpc: started, v1...
udhcpc: broadcasting discover
udhcpc: lease of 10.10.10.142 obtained from 10.10.10.1, lease time 3600
    inet 10.10.10.142/24 scope global eth1
2 packets transmitted, 2 packets received, 0% packet loss
```

Leave the console with Ctrl+].

> **What just happened:** `attach-interface ... --config` added a VirtIO NIC to cirros2's saved definition, so it appeared at boot as `eth1`. CirrOS only configures `eth0` automatically, so you brought `eth1` up and ran `udhcpc`, BusyBox's DHCP client, to get an address from libvirt's DHCP server on `virbr10`. The guest can reach the host at `10.10.10.1`, but traffic on this network goes nowhere else. Back on ubuntu-lab, `virsh net-dhcp-leases isolated-lab` shows the lease.

5. Learn the other network types by writing (not applying) their configurations.

> TIP
>
> **Explore/simulate:** A **routed** network looks like NAT but with `<forward mode='route'/>`: the host forwards guest traffic without rewriting addresses, so your LAN router needs a route to the guest subnet (for example "10.20.0.0/24 via 192.168.100.10"). An **open** network uses `<forward mode='open'/>`: libvirt creates the bridge and routes, but adds **no** firewall rules at all, leaving filtering entirely to you. A **bridged** setup connects guests to a Linux bridge that also contains the host's physical NIC, so guests get addresses from your real LAN's DHCP server and are reachable like physical machines. On Ubuntu you create that bridge with netplan, then attach VMs with `virt-install --network bridge=br0,model=virtio`. Do **not** apply the netplan file below over SSH in this lab: moving the IP address to a bridge can cut off your connection.

A routed network definition, which you can save as `~/labs/1.7/routed.xml` for reference:

```
<network>
  <name>routed-lab</name>
  <forward mode='route'/>
  <bridge name='virbr20'/>
  <ip address='10.20.0.1' netmask='255.255.255.0'/>
</network>
```

A bridged host configuration in netplan (for example `/etc/netplan/60-bridge.yaml`, interface name from `ip -br link`):

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
  bridges:
    br0:
      interfaces: [enp0s3]
      dhcp4: true
```

> **What just happened:** You now have examples of every network type the exam lists: NAT (the `default` network), isolated/host-only (`isolated-lab`), routed (`forward mode='route'`), open (`forward mode='open'`), and bridged (a netplan bridge `br0` containing the physical NIC). In the bridged YAML, the physical interface loses its own address and the bridge gets the IP instead, which is exactly why applying it remotely is risky.

Checkpoint: `virsh net-list --all` shows `default` and `isolated-lab` active, and `virsh domiflist cirros2` lists two interfaces, one on each network.

Task 9: Practise migration and meet virt-manager

1. Perform the steps of a *cold migration* on one host: export the VM definition, remove it, and re-create it from the XML, exactly as you would on a second host after copying the disk.

```
virsh destroy cirros2
virsh dumpxml cirros2 > ~/labs/1.7/cirros2.xml
virsh undefine cirros2
virsh list --all
virsh define ~/labs/1.7/cirros2.xml
virsh list --all
```

```
Domain 'cirros2' destroyed

Domain 'cirros2' has been undefined

 Id   Name      State
--------------------------
 -    cirros1   shut off

Domain 'cirros2' defined from /home/student/labs/1.7/cirros2.xml

 Id   Name      State
--------------------------
 -    cirros1   shut off
 -    cirros2   shut off
```

What just happened: `dumpxml` saved the complete VM definition. `undefine` deleted the definition from libvirt but left the disk file alone (add `--remove-all-storage` to delete disks too). `define` registered the VM again from the XML. A real cold migration is these steps plus copying the disk image to the same path on the new host, for example with `rsync` or `scp`, while the VM is off.

2. Learn the live migration syntax.

TIP

Explore/simulate: Live migration needs two KVM hosts, so you cannot finish it in this lab. The command, run on the source host, would be: `virsh migrate --live --persistent --undefinesource --verbose cirros1 qemu+ssh://host2/system`. `--live` keeps the guest running while its memory is copied over the network, then pauses it for a fraction of a second to switch hosts. `qemu+ssh://host2/system` is a remote libvirt URI tunnelled over SSH. Requirements: both hosts reach the same storage (NFS, iSCSI, Ceph) or you add `--copy-storage-all` to copy the disk too; compatible CPUs (use a common CPU model); the same networks and bridges on both sides; and libvirt on both. `--offline --persistent` moves only the definition of a stopped VM. To see the failure mode, try `virsh -c qemu+ssh://student@rocky-lab/system list` and read the error, because rocky-lab does not run libvirt.

3. Understand virt-manager.

> TIP
>
> **Explore/simulate:** `virt-manager` is the graphical front end to libvirt: it lists VMs with their states, shows a VM's screen, and offers wizards for creating VMs, snapshots, networks, and storage pools. It needs a graphical desktop, which ubuntu-lab does not have. Two ways to use it with ubuntu-lab: (1) on a Linux desktop, install `virt-manager` and choose File > Add Connection > QEMU/KVM, tick "Connect to remote host over SSH", and enter `student@ubuntu-lab`; or (2) install it on ubuntu-lab (`sudo apt install -y virt-manager`, which pulls in many GUI libraries) and run it through X11 forwarding with `ssh -X student@ubuntu-lab virt-manager`. Everything it does is also possible with `virsh`, which is what you have used here and what the exam focuses on.

4. Compare bare metal and virtual machines.

	Bare metal	Virtual machine
Performance	Full hardware speed, no overhead	Near-native with KVM and VirtIO; slower with emulation
Density	One OS per server	Many VMs per host; better hardware use
Provisioning	Hours to days (rack, cable, install)	Seconds to minutes from a template
Snapshots and cloning	Not available (only disk imaging)	Built in
Mobility	Tied to the box	Live and cold migration between hosts
Isolation	Physical: strongest	Hypervisor-enforced; a hypervisor bug could affect all guests
Hardware access	Direct (GPUs, special cards)	Emulated or passed through (PCI passthrough)
Licensing and cost	Pay per physical server	Shared hardware, but hypervisor and per-VM licensing apply
Typical use	Databases needing every bit of I/O, HPC, hypervisor hosts	General servers, labs, test/dev, cloud

> **What just happened:** This table summarizes the trade-off the exam asks about. Bare metal wins on raw performance, direct hardware access, and the strongest isolation; VMs win on flexibility: templates, cloning, snapshots, migration, and packing many workloads onto one machine. Note that the "bare metal" in *bare-metal hypervisor* means the hypervisor itself runs directly on the hardware.

Checkpoint: You can describe the difference between cold and live migration, name what live migration needs (shared or copied storage, compatible CPUs, matching networks), and explain what virt-manager is used for.

Challenge

CHALLENGE

1. Create a second linked clone `cirros3.qcow2` from `cirros-base.qcow2` and a VM for it with `virt-install`, attached **only** to `isolated-lab` (use `--network network=isolated-lab,model=virtio`). Prove from inside the guest that it gets a `10.10.10.x` address but cannot ping `192.168.122.1` or the internet.
2. Take a snapshot of a *running* VM, write a file inside it, revert, and note how the VM's state after the revert differs from the shut-off snapshot you used in Task 7. Compare the `VM SIZE` column of `qemu-img snapshot -l` for both snapshots.
3. Convert `cirros-base.qcow2` to VirtualBox (`vdi`) and VMware (`vmdk`) formats with `qemu-img convert`, and compare the three files with `qemu-img info`.

Clean up

1. Stop and delete the VMs, including their snapshot metadata.

```
virsh destroy cirros1 2>/dev/null
virsh destroy cirros2 2>/dev/null
virsh undefine cirros1 --snapshots-metadata
virsh undefine cirros2
virsh list --all
```

(If you did the challenge, also run `virsh destroy cirros3` and `virsh undefine cirros3`.)

2. Delete the disk images.

```
sudo rm -f /var/lib/libvirt/images/cirros*.qcow2
sudo ls /var/lib/libvirt/images
```

3. Remove the isolated network.

```
virsh net-destroy isolated-lab
virsh net-undefine isolated-lab
virsh net-list --all
```

4. Remove the lab directory and the line you added to `~/.bashrc`.

```
rm -rf ~/labs/1.7
sed -i '/# lab 1.7$/d' ~/.bashrc
```

5. Remove the virtualization stack so later labs see a normal server (no `virbr0` bridge or extra firewall rules), and take `student` out of the `libvirt` group. If you plan to keep experimenting with VMs, you may skip this step. The first command stops the default NAT network, which removes `virbr0`, its DHCP server, and libvirt's firewall rules; removing the packages alone would leave them in place until the next reboot.

```
virsh net-destroy default
sudo apt purge -y libvirt-daemon-system virtinst qemu-system-x86 cpu-checker
sudo apt autoremove -y
sudo gpasswd -d student libvirt 2>/dev/null
ip -br link | grep virbr
```

`apt purge` removes the packages together with their configuration files, `apt autoremove` removes dependencies that nothing needs any more, and `gpasswd -d student libvirt` deletes `student` from the `libvirt` group. The last command should print nothing. Log out and back in so the group change and `~/.bashrc` change take effect.

Exam corner

- **KVM** is the Linux kernel module that uses Intel VT-x (`vmx`) or AMD-V (`svm`) for hardware-accelerated VMs; **QEMU** emulates the machine and devices and can run without KVM using slow software emulation (TCG). Check support with `grep -E 'vmx|svm' /proc/cpuinfo` or `kvm-ok`.
- **libvirt** is the management layer (daemon `libvirtd`, XML definitions in `/etc/libvirt/qemu/`); **virsh** is its CLI, **virt-install** creates VMs, **virt-clone** clones them, **virt-manager** is the GUI.
- **Nested virtualization** means running a hypervisor inside a VM; the outer hypervisor must expose VT-x/AMD-V to the guest.
- **Paravirtualized drivers** (VirtIO: `virtio-blk`, `virtio-net`, `virtio-balloon`) are faster than emulated hardware; VirtIO disks appear as `/dev/vda`.
- `qemu-img info` shows image properties (format, virtual size, disk size, backing file); `qemu-img convert -f qcow2 -O raw` changes format; `qemu-img resize img +10G` grows the virtual disk (then grow the partition and file system in the guest).
- qcow2 supports thin provisioning, compression, internal snapshots, and backing files; a baseline template plus qcow2 overlays (`qemu-img create -b base -F qcow2`) gives linked clones; `virt-clone` makes full clones with a new UUID and MAC.
- VM states: running, idle, paused (`virsh suspend` / `resume`), in shutdown, shut off, crashed, pmsuspended. `virsh shutdown` is graceful, `virsh destroy` is a forced power-off, `virsh undefine` deletes the definition.
- Resources: `virsh setmem` / `setmaxmem`, `virsh setvcpus`, `virsh attach-disk` / `detach-disk`, `virsh attach-interface`; `--config` (persistent), `--live` (running VM), `--persistent` (both, for attach/detach).
- Snapshots: `virsh snapshot-create-as`, `snapshot-list`, `snapshot-revert`, `snapshot-delete`. Snapshots are not backups.
- Migration: cold/offline (stopped VM: copy disk, `dumpxml` / `define`) versus live (`virsh migrate --live VM qemu+ssh://host2/system`, needs shared or copied storage and compatible CPUs).
- Networks: **NAT** (libvirt `default`, `virbr0`, 192.168.122.0/24), **bridged** (guests on the physical LAN), **host-only/isolated** (no `<forward>`), **routed** (`forward mode='route'`, no NAT), **open** (`forward mode='open'`, no firewall rules added).
- Bare metal: best performance and isolation, direct hardware. VMs: density, templates, snapshots, cloning, and migration.

Review questions

1. A technician runs `grep -cE 'vmx|svm' /proc/cpuinfo` inside a VM and gets `0`. What does this mean for running KVM guests inside that VM, and how can it be fixed?

2. Which command shows the format, virtual size, actual disk usage, and backing file of a VM disk image?
   A. `virsh dominfo disk.qcow2`
   B. `qemu-img info disk.qcow2`
   C. `qemu-img check disk.qcow2 --format`
   D. `lsblk disk.qcow2`

3. An administrator wants twenty test VMs that share one read-only baseline image and store only their own changes. Which approach does this?
   A. `virt-clone` twenty times
   B. `qemu-img convert` the image twenty times
   C. Create qcow2 overlay images with `qemu-img create -b base.qcow2 -F qcow2`
   D. Take twenty snapshots of one VM

4. What is the difference between `virsh shutdown web01` and `virsh destroy web01`?

5. A VM must be reachable from other computers on the office LAN at its own IP address, as if it were a physical server. Which virtual network type should be used?
   A. NAT
   B. Isolated
   C. Bridged
   D. Host-only

6. Why do VirtIO devices give better disk and network performance than emulated devices such as an IDE disk or e1000 NIC?

PART 2 · DOMAIN 2.0

Part 2: Services and User Management

20% of the exam · 6 labs

2.1	Given a scenario, manage files and directories on a Linux system.
2.2	Given a scenario, perform local account management in a Linux environment.
2.3	Given a scenario, manage processes and jobs in a Linux environment.
2.4	Given a scenario, configure and manage software in a Linux environment.
2.5	Given a scenario, manage Linux using systemd.
2.6	Given a scenario, manage applications in a container on a Linux server.

Lab 2.1 Manage Files and Directories

Objective: 2.1 Given a scenario, manage files and directories on a Linux system.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 90 minutes
VM: ubuntu-lab
Snapshot first: Not required. Everything happens inside `~/labs/2.1` and `/dev/shm`, and the clean-up removes it.

In this lab you build a small project directory tree and then work through every file utility on the exam the way an administrator really uses them: finding files, comparing config versions, checking timestamps, and tracking down which process is holding a file open. You will also see exactly how hard links and symbolic links differ, and take a guided tour of the device files in `/dev`. These commands are the everyday tools of any Linux job, and the exam expects you to pick the right one for a scenario.

What you'll practice

- Utilities: `cd`, `cp`, `diff`, `file`, `find`, `ln`, `locate`, `ls`, `lsof`, `mkdir`, `mv`, `pwd`, `rm`, `rmdir`, `sdiff`, `stat`, `touch`
- Links: symbolic link, hard link
- Device types in `/dev`: block devices, character devices, special character devices

How it works

Everything is a file. Linux presents almost everything as a file in one big tree that starts at `/` (the *root* directory). Regular files hold data. Directories are files that hold a list of names. Links, devices, sockets, and pipes are also files with special types. The first character of `ls -l` output tells you the type:

Character	Type
`-`	regular file
`d`	directory
`l`	symbolic link
`b`	block device
`c`	character device
`p`	named pipe (FIFO)
`s`	socket

Paths. An *absolute path* starts at `/`, like `/etc/hosts`. A *relative path* starts from your *current working directory* (the directory your shell is "in"), like `project/docs`. Two special names exist in every directory: `.` means "this directory" and `..` means "the parent directory". The `~` character is shell shorthand for your home directory (`/home/student`).

Inodes: the real identity of a file. A file on disk has two parts. The *inode* (index node) is a small record that stores everything about the file except its name: owner, permissions, size, timestamps, and where the data blocks live. Each inode has a number that is unique inside its filesystem. The *directory entry* is just a name that points to an inode number. Think of a library: the inode is the catalog card with all the facts about a book, and a directory entry is a label on a shelf that says "card number 1311234".

```
Directory entries (names)         Inode 1311234                Data blocks
------------------------          ---------------------        -----------
notes.txt       ---------+------> owner, perms, size,  ------>  "hello..."
notes-hard.txt  ---------+        timestamps, link count = 2
  (hard link: same inode)

notes-soft.txt  -> inode 1311240 (type l) contains the TEXT "notes.txt"
  (symbolic link: its own inode that stores a path)
```

This explains the two kinds of links:

- A **hard link** is a second directory entry that points to the *same inode*. Both names are equally "real". The inode keeps a *link count*. The data is only freed when the count drops to zero and no process has the file open. Hard links cannot cross filesystems (inode numbers are only unique inside one filesystem) and you cannot hard-link a directory.
- A **symbolic link** (symlink, soft link) is a tiny separate file whose content is a path to another file. It has its own inode. It can point anywhere, even to another filesystem or to a directory. If the target is deleted, the symlink becomes *dangling* (broken) because the path it stores no longer leads anywhere.

Three timestamps. Every inode records:

- **atime** (access): last time the content was read. Ubuntu mounts with `relatime`, so atime is only updated occasionally to save disk writes.
- **mtime** (modify): last time the *content* changed.
- **ctime** (change): last time the *inode* changed (content, permissions, owner, link count, or name). You cannot set ctime by hand.

Many filesystems (ext4, XFS) also record a **birth** (creation) time, which `stat` shows when available.

Finding files: `find` versus `locate`. `find` walks the directory tree live, every time, and can test almost anything (name, size, age, owner, permissions) and then run a command on each match. It is always accurate but can be slow on a big tree. `locate` searches a prebuilt database of file names, so it is nearly instant, but it only knows what existed the last time the database was built with `updatedb`. On Ubuntu 24.04 `locate` is provided by the `plocate` package.

Open files and `lsof`. When a program opens a file, the kernel gives it a *file descriptor* (a small number). `lsof` ("list open files") shows which processes have which files open. This answers classic admin questions: "Why can't I unmount this disk?" and "I deleted a huge log file, so why is the disk still full?" (Answer: a process still has the deleted

file open, so the inode and data are not freed yet.)

Device files in `/dev`. Hardware and kernel features appear as special files in `/dev`. They contain no data on disk; reading or writing them talks to a kernel driver. Each device file carries two numbers instead of a size: the **major number** identifies the driver, and the **minor number** identifies which device that driver handles.

- **Block devices** (`b`) move data in fixed-size blocks and allow random access, so you can jump to any position. Disks and partitions are block devices: `/dev/sda`, `/dev/sda1`, `/dev/nvme0n1`.
- **Character devices** (`c`) move data as a stream of bytes, one after another. Terminals and serial ports are character devices: `/dev/tty`, `/dev/ttyS0`.
- **Special character devices** are character devices that are not real hardware at all. The kernel creates them to provide useful behavior: `/dev/null` (throws away everything written to it, returns nothing when read), `/dev/zero` (returns endless zero bytes), `/dev/random` and `/dev/urandom` (return random bytes), and `/dev/full` (always reports "disk full" when written).

Tasks

Task 1: Navigate and build a project tree

You are setting up a working area for a small web application. You will use `pwd`, `cd`, `mkdir`, `ls`, and `touch`.

1. Create the lab directory and move into it.

```
mkdir -p ~/labs/2.1 && cd ~/labs/2.1
pwd
```

```
/home/student/labs/2.1
```

> **What just happened:** `mkdir` makes a directory. The `-p` (parents) option creates any missing parent directories along the way (`~/labs` and then `~/labs/2.1`) and does not complain if they already exist. `&&` runs the second command only if the first succeeded. `cd` (change directory) moves your shell into the new directory, and `pwd` (print working directory) shows the absolute path of where you are now.

2. Practice moving around. Run each line and watch the output of `pwd`.

```
cd /etc
pwd
cd ..
pwd
cd -
cd
pwd
cd ~/labs/2.1
```

What just happened: `cd /etc` uses an absolute path. `cd ..` moves up one level to `/`. `cd -` jumps back to the previous directory (`/etc`) and prints its name. `cd` with no argument takes you home (`/home/student`). The last line returns you to the lab directory using `~` for home.

3. Build the whole project tree in one command using *brace expansion*.

```
mkdir -p webapp/{src,docs,config,logs,backup}
ls
ls webapp
```

```
webapp
backup  config  docs  logs  src
```

What just happened: The shell expands `webapp/{src,docs,config,logs,backup}` into five separate arguments (`webapp/src` , `webapp/docs` , and so on) before `mkdir` runs. `mkdir -p` then creates `webapp` and the five subdirectories. Brace expansion is a shell feature, so it works with any command.

4. Create some empty files with `touch`, then add content to a few.

```
touch webapp/src/{app.py,utils.py,models.py}
touch webapp/docs/README.md webapp/logs/app.log
echo "print('hello')" > webapp/src/app.py
echo "Project notes" > webapp/docs/notes.txt
```

What just happened: `touch` creates a file if it does not exist (and updates its timestamps if it does). `echo "text" > file` writes text into a file, replacing anything already there. You will learn more about `touch` and timestamps in Task 4.

5. List the tree in several useful ways.

```
ls -l webapp
ls -la webapp/src
ls -lh webapp/docs
ls -R webapp
ls -ld webapp
ls -lt webapp/src
```

```
total 20
drwxrwxr-x 2 student student 4096 Oct 10 10:02 backup
drwxrwxr-x 2 student student 4096 Oct 10 10:02 config
drwxrwxr-x 2 student student 4096 Oct 10 10:03 docs
drwxrwxr-x 2 student student 4096 Oct 10 10:03 logs
drwxrwxr-x 2 student student 4096 Oct 10 10:03 src
```

What just happened: `ls -l` gives the *long listing*: type and permissions (`drwxrwxr-x`), link count, owner, group, size, modification time, and name. `-a` shows *all* files, including hidden ones whose names start with a dot (you will see `.` and `..`). `-h` prints sizes in human-readable units (K, M, G). `-R` lists recursively into every subdirectory. `-d` lists the directory itself instead of its contents. `-t` sorts by modification time, newest first.

Checkpoint: `ls -R ~/labs/2.1/webapp` shows five subdirectories, three `.py` files in `src` , `README.md` and `notes.txt` in `docs` , and `app.log` in `logs` .

Task 2: Copy, move, rename, and remove

Now you reorganize the project with `cp` , `mv` , `rm` , and `rmdir` .

1. Create a config file and make a backup copy of it.

```
cd ~/labs/2.1/webapp
printf 'port=8080\nworkers=4\ndebug=false\nlog_level=info\n' > config/app.conf
cp config/app.conf backup/app.conf.bak
cp -p config/app.conf backup/app.conf.preserved
ls -l config backup
```

What just happened: `printf` writes text and turns each `\n` into a new line, so `app.conf` gets four lines. `cp SOURCE DEST` copies a file. A plain copy gets a new modification time (now). `cp -p` *preserves* the mode, ownership, and timestamps of the original, which matters for backups because you want to know when the original last changed.

2. Copy a whole directory. `cp` refuses to copy a directory unless you ask for recursion.

```
cp src backup/
cp -r src backup/src-copy
cp -a docs backup/docs-archive
ls backup
```

```
cp: -r not specified; omitting directory 'src'
app.conf.bak  app.conf.preserved  docs-archive  src-copy
```

What just happened: The first `cp` failed with a clear message. `-r` (recursive) copies a directory and everything inside it. `-a` (archive) is the same as `-dR --preserve=all` : recursive, keeps symlinks as symlinks, and preserves permissions, ownership, and timestamps. Use `-a` when you want an exact copy.

3. Use `mv` to rename a file and to move files into another directory.

```
mv docs/notes.txt docs/project-notes.txt
mv src/models.py src/utils.py backup/src-copy/
ls docs src
```

What just happened: `mv` both renames and moves. With two names in the same directory it renames. With a directory as the last argument it moves all the earlier arguments into that directory. Inside one filesystem `mv` is instant because it only rewrites directory entries; the inode and data do not move. The original `utils.py` already existed in `backup/src-copy` from the `cp -r`, so `mv` silently replaced it. Add `-i` (interactive) to be asked before overwriting.

4. Try the safe, interactive versions. Answer `n` to each prompt.

```
cp -i config/app.conf backup/app.conf.bak
mv -i backup/app.conf.preserved backup/app.conf.bak
```

```
cp: overwrite 'backup/app.conf.bak'? n
mv: overwrite 'backup/app.conf.bak'? n
```

5. Remove files and directories. Start with an empty directory, then a non-empty one.

```
mkdir -p tmp/a/b/c
rmdir tmp/a/b/c
rmdir -p tmp/a/b
rmdir backup
```

```
rmdir: failed to remove 'backup': Directory not empty
```

What just happened: `rmdir` only removes *empty* directories, which makes it a safe choice: it can never delete files by accident. `rmdir -p tmp/a/b` removes `b`, then `a`, then `tmp`, as long as each becomes empty. It refused to remove `backup` because it still has contents.

6. Use `rm` for files and non-empty directories.

```
rm -i backup/app.conf.preserved
rm -r backup/docs-archive
ls backup
```

What just happened: `rm` removes files (it deletes directory entries; the data is freed when the link count reaches zero). `-i` asks for confirmation (answer `y`). `-r` removes a directory and everything under it. You will often see `rm -rf`, where `-f` (force) never prompts and ignores missing files. There is no recycle bin on the command line, so double-check paths before using `-rf`.

CAUTION

`rm -rf` with a typo can destroy a system. A classic mistake is a stray space: `rm -rf / home/olduser` deletes from `/`. Modern GNU `rm` refuses to operate on `/` itself unless you add `--no-preserve-root`, but it will happily delete any other directory you name. Run `ls` on the exact path first.

Checkpoint: `ls ~/labs/2.1/webapp/backup` shows `app.conf.bak` and `src-copy`. `ls ~/labs/2.1/webapp/src` shows only `app.py`, and `~/labs/2.1/webapp/tmp` no longer exists.

Task 3: Identify files with `file` and inspect them with `stat`

File names in Linux do not have to have extensions, and an extension can lie. The `file` command looks inside a file to see what it really is.

1. Run `file` on several kinds of objects.

```
cd ~/labs/2.1
cp /usr/bin/ls ./mystery
gzip -k webapp/config/app.conf
file mystery webapp/config/app.conf webapp/config/app.conf.gz
file /bin /etc /dev/sda /dev/null webapp/src/app.py
```

```
mystery:                    ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
                            dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, ...
webapp/config/app.conf:     ASCII text
webapp/config/app.conf.gz:  gzip compressed data, was "app.conf", ...
/bin:                       symbolic link to usr/bin
/etc:                       directory
/dev/sda:                   block special (8/0)
/dev/null:                  character special (1/3)
webapp/src/app.py:          ASCII text
```

What just happened: `file` reads the first bytes of each file and compares them to a database of known patterns called *magic numbers*. The file `mystery` has no extension, but `file` recognizes it as an ELF executable (the Linux program format). `gzip -k` compressed a copy while keeping (`-k`) the original. On Ubuntu 24.04, `/bin` is a symbolic link to `usr/bin`. For devices, `file` shows the type and the major/minor numbers. If your disk is `vda` or `nvme0n1`, `/dev/sda` will say "cannot open"; that is fine. `app.py` is reported as plain text because it has no `#!` line; with `#!/usr/bin/env python3` as its first line, `file` would call it a Python script.

2. Look at the full inode details of a file with `stat`.

```
stat webapp/config/app.conf
```

```
  File: webapp/config/app.conf
  Size: 47          Blocks: 8          IO Block: 4096   regular file
Device: 252,0   Inode: 1311290     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/ student)   Gid: ( 1000/ student)
Access: 2026-10-10 10:05:12.418263510 +0000
Modify: 2026-10-10 10:05:12.418263510 +0000
Change: 2026-10-10 10:05:12.418263510 +0000
 Birth: 2026-10-10 10:05:12.418263510 +0000
```

> **What just happened:** `stat` prints the inode contents. **Size** is in bytes; **Blocks** is the number of 512-byte units allocated. **Device** is the filesystem the file lives on (major,minor). **Inode** is the inode number and **Links** is the hard link count. **Access** shows permissions in octal (`0664`) and symbolic form, plus the owner UID and group GID. Then the three timestamps (atime, mtime, ctime) and the birth time. Your inode number and device will differ.

3. `stat` can also print just the fields you want, which is handy in scripts.

```
stat -c '%n  inode=%i  links=%h  size=%s  mode=%a  owner=%U' webapp/config/app.conf
stat -f /
```

> **What just happened:** `-c` (format) prints a custom line: `%n` name, `%i` inode, `%h` hard links, `%s` size, `%a` octal permissions, `%U` owner name. `stat -f` shows information about the *filesystem* instead of a file: type (for example `ext2/ext3` for ext4), block size, and free blocks and inodes.

Checkpoint: `file ~/labs/2.1/mystery` reports an ELF executable, and `stat -c %h ~/labs/2.1/webapp/config/app.conf` prints `1`.

Task 4: Control timestamps with `touch`

Backup and clean-up scripts often decide what to do based on file age. To test them you need files with old timestamps, and `touch` can create those.

1. Create a set of log files and give them different ages.

```
cd ~/labs/2.1/webapp/logs
touch -d "2024-01-15 09:30" old-jan.log
touch -d "10 days ago" ten-days.log
touch -t 202510010800 oct-first.log
touch today.log
ls -l --time-style=long-iso
```

```
total 0
-rw-rw-r-- 1 student student 0 2026-10-10 10:03 app.log
-rw-rw-r-- 1 student student 0 2024-01-15 09:30 old-jan.log
-rw-rw-r-- 1 student student 0 2025-10-01 08:00 oct-first.log
-rw-rw-r-- 1 student student 0 2026-09-30 10:10 ten-days.log
-rw-rw-r-- 1 student student 0 2026-10-10 10:10 today.log
```

> **What just happened:** `touch -d STRING` sets the atime and mtime to a date described in almost plain English (`"10 days ago"`, `"yesterday"`, `"2024-01-15 09:30"`). `touch -t` uses the compact format `[[CC]YY]MMDDhhmm[.ss]`, so `202510010800` means 2025-10-01 08:00. `--time-style=long-iso` makes `ls` show full dates. Your dates will be relative to today.

2. Change only one timestamp, and observe that ctime cannot be faked.

```
touch -a -d "2020-06-01" today.log
stat today.log | grep -E 'Access: [0-9]|Modify|Change'
```

```
Access: 2020-06-01 00:00:00.000000000 +0000
Modify: 2026-10-10 10:10:41.120093114 +0000
Change: 2026-10-10 10:11:02.551876020 +0000
```

What just happened: `touch -a` changes only the access time; `touch -m` changes only the modification time. Changing a timestamp is itself a change to the inode, so **ctime** jumped to *now*. There is no option to set ctime, which is why investigators trust it more than mtime.

Checkpoint: `ls -lt ~/labs/2.1/webapp/logs` lists `old-jan.log` last (oldest modification time).

Task 5: Find files with `find` and `locate`

1. Find by name. `find` takes a starting directory, then tests, then an optional action.

```
cd ~/labs/2.1
find . -name "*.log"
find webapp -iname "readme*"
find webapp -type d
find webapp -type f -name "*.py"
```

What just happened: `find .` searches from the current directory down. `-name "*.log"` matches names with a shell-style pattern; always quote the pattern so your shell does not expand `*` first. `-iname` is case-insensitive. `-type d` matches directories and `-type f` regular files (`-type l` matches symlinks). Tests placed one after another are joined with AND.

2. Find by time. Use the log files you aged in Task 4.

```
find webapp/logs -mtime +7
find webapp/logs -mtime -1
find webapp -mmin -30 -type f
```

What just happened: `-mtime` counts in 24-hour days. `+7` means "modified more than 7 days ago", `-1` means "modified less than 1 day ago". `-mmin` works the same way in minutes. `-atime` and `-ctime` test the other two timestamps. This is how log clean-up jobs pick old files.

3. Find by size. Make a large file first.

```
dd if=/dev/zero of=webapp/backup/big.img bs=1M count=50 status=none
find . -size +10M
find . -type f -size 0
find . -type f -empty
```

What just happened: `dd` copied 50 blocks of 1 MiB from `/dev/zero`, making a 50 MiB file (more on `/dev/zero` in Task 8). `-size +10M` finds files bigger than 10 MiB; suffixes are `k`, `M`, and `G`. `-size 0` and `-empty` both find empty files (`-empty` also matches empty directories, but `-type f` limits it to files here).

4. Find by permissions and owner. Search system directories as well; send errors to `/dev/null`.

```
chmod 777 webapp/config/app.conf
find webapp -perm 777
find webapp -perm -o=w
find /usr/bin -perm -4000 2>/dev/null
find /etc -user student 2>/dev/null
find /var/log -group adm -name "*.log" 2>/dev/null | head
```

What just happened: `-perm 777` matches *exactly* mode 777. `-perm -o=w` (with a leading dash) matches files that have *at least* the "others can write" bit, which is the usual way to hunt for world-writable files. `-perm -4000` finds *setuid* programs (like `passwd`), a common security audit. `-user` and `-group` match the owner. `2>/dev/null` hides "Permission denied" errors from directories you cannot read. The `/etc -user student` search probably prints nothing, which is the expected result.

5. Act on the results with `-exec`, `-delete`, and `-ok`.

```
find webapp -name "*.py" -exec ls -l {} \;
find webapp -name "*.py" -exec wc -l {} +
find webapp/logs -name "*.log" -mtime +7 -ok rm {} \;
find webapp -perm 777 -exec chmod 644 {} \;
find webapp/logs -mtime +7
```

What just happened: `-exec COMMAND {} \;` runs the command once per match, replacing `{}` with the file name; the `\;` (escaped semicolon) marks the end of the command. Ending with `+` instead passes many file names to a single command, which is faster. `-ok` is like `-exec` but asks `y/n` for each file (answer `y` to delete the old logs). The last line confirms that no logs older than 7 days remain. `-delete` also exists and removes every match without asking, so test the `find` without it first.

6. Install `plocate` and compare `locate` with `find`.

```
sudo apt update
sudo apt install -y plocate
touch webapp/config/fresh.conf
locate fresh.conf
sudo updatedb
locate fresh.conf
locate app.conf
locate -c "*.conf"
locate -i readme.md | head -5
```

```
/home/student/labs/2.1/webapp/config/fresh.conf
/home/student/labs/2.1/webapp/backup/app.conf.bak
/home/student/labs/2.1/webapp/config/app.conf
/home/student/labs/2.1/webapp/config/app.conf.gz
```

What just happened: On Ubuntu 24.04 the `locate` command comes from the `plocate` package, which also provides `updatedb`. When the package is first installed it builds the database once, so it already knows the files you made earlier. The new file `fresh.conf` was created *after* that, so the first `locate fresh.conf` printed nothing. `sudo updatedb` rebuilds the database (stored in `/var/lib/plocate/plocate.db`). After that, `locate` finds the files instantly. `-c` counts matches instead of listing them, and `-i` ignores case. A systemd timer (`plocate-updatedb.timer`) rebuilds the database daily. `locate` only shows you files you have permission to see.

7. Prove that `locate` can be out of date.

```
rm webapp/config/app.conf.gz
locate app.conf.gz
find ~ -name app.conf.gz
```

What just happened: `locate` still lists the deleted file because the database is stale, while `find` correctly finds nothing. Rule of thumb: use `locate` for a fast "where is that file?" search, and `find` when you need accurate, current results or tests beyond the name.

TIP

On Rocky Linux 9, `locate` comes from the `mlocate` package (`sudo dnf install -y mlocate`), and you run `sudo updatedb` the same way.

Checkpoint: `find ~/labs/2.1 -size +10M` prints `big.img`, `find ~/labs/2.1/webapp -perm 777` prints nothing, and `locate -c app.conf.bak` prints `1`.

Task 6: Compare files with `diff` and `sdiff`

A colleague edited the application config, and you need to see exactly what changed.

1. Create a second version of the config file.

```
cd ~/labs/2.1/webapp/config
printf 'port=8080\nworkers=4\ndebug=false\nlog_level=info\n' > app.conf
printf 'port=8443\nworkers=4\nlog_level=debug\ntls=on\n' > app.conf.new
```

2. Compare them with plain `diff`.

```
diff app.conf app.conf.new
```

```
1c1
< port=8080
---
> port=8443
3,4c3,4
< debug=false
< log_level=info
---
> log_level=debug
> tls=on
```

What just happened: `diff` prints instructions for turning the first file into the second. `1c1` means "line 1 of the first file must be **c**hanged into line 1 of the second". `3,4c3,4` means lines 3 to 4 change into lines 3 to 4. Other letters are `a` (add) and `d` (delete). Lines starting with `<` come from the first file, lines with `>` from the second. If the files are identical, `diff` prints nothing and exits with status 0.

3. Use the *unified* format, which is what patches and `git` use.

```
diff -u app.conf app.conf.new
diff -q app.conf app.conf.new
diff -r ../src ../backup/src-copy
```

```
--- app.conf    2026-10-10 10:20:01.000000000 +0000
+++ app.conf.new    2026-10-10 10:20:01.000000000 +0000
@@ -1,4 +1,4 @@
-port=8080
+port=8443
 workers=4
-debug=false
-log_level=info
+log_level=debug
+tls=on
```

What just happened: `-u` shows changes with a few lines of context: `-` lines are removed, `+` lines are added, and lines starting with a space are unchanged. The `@@ -1,4 +1,4 @@` header gives the line ranges. `-q` (quiet) only reports *whether* files differ. `-r` compares two directory trees recursively and reports files that exist in only one of them ("Only in ...").

4. View the two versions side by side with `sdiff`.

```
sdiff -w 70 app.conf app.conf.new
sdiff -s -w 70 app.conf app.conf.new
```

```
port=8080                       | port=8443
workers=4                         workers=4
debug=false                     | log_level=debug
log_level=info                  | tls=on
```

What just happened: `sdiff` prints the first file on the left and the second on the right. The middle column marks each line: `|` means the lines differ, `<` means the line is only in the left file, `>` only in the right file, and a blank means identical. `-w 70` sets the total width in characters. `-s` suppresses identical lines so you only see differences. (`sdiff -o merged.conf` can even merge interactively.)

Checkpoint: `diff -q app.conf app.conf.new` prints `Files app.conf and app.conf.new differ`, and `diff app.conf app.conf; echo $?` prints `0`.

Task 7: Hard links and symbolic links

1. Create a file and one link of each type, then compare inode numbers.

```
mkdir -p ~/labs/2.1/links && cd ~/labs/2.1/links
echo "original content" > notes.txt
ln notes.txt notes-hard.txt
ln -s notes.txt notes-soft.txt
ls -li
```

```
total 8
1311301 -rw-rw-r-- 2 student student 17 Oct 10 10:25 notes-hard.txt
1311302 lrwxrwxrwx 1 student student  9 Oct 10 10:25 notes-soft.txt -> notes.txt
1311301 -rw-rw-r-- 2 student student 17 Oct 10 10:25 notes.txt
```

What just happened: `ln TARGET LINKNAME` creates a hard link; `ln -s TARGET LINKNAME` creates a symbolic link. `ls -i` adds the inode number in the first column. `notes.txt` and `notes-hard.txt` share inode `1311301`, and the link count (third column) is `2` for both: they are two names for the same file. `notes-soft.txt` has its own inode, type `l`, permissions `rwxrwxrwx` (symlink permissions are never used; the target's permissions apply), and a size of 9, which is the length of the text `notes.txt` it stores.

2. Confirm with `stat`, and edit through each name.

```
stat -c '%n inode=%i links=%h type=%F' notes.txt notes-hard.txt notes-soft.txt
echo "added via hard link" >> notes-hard.txt
echo "added via soft link" >> notes-soft.txt
cat notes.txt
readlink notes-soft.txt
```

```
notes.txt inode=1311301 links=2 type=regular file
notes-hard.txt inode=1311301 links=2 type=regular file
notes-soft.txt inode=1311302 links=1 type=symbolic link
original content
added via hard link
added via soft link
```

What just happened: Writing through either link changes the same data, so `cat notes.txt` shows both new lines. Writing through a symlink follows it to the target. `>>` appends instead of overwriting. `readlink` prints the path a symlink stores.

3. Delete the original and see what happens to each link.

```
rm notes.txt
ls -li
cat notes-hard.txt
cat notes-soft.txt
```

```
1311301 -rw-rw-r-- 1 student student 57 Oct 10 10:26 notes-hard.txt
1311302 lrwxrwxrwx 1 student student  9 Oct 10 10:25 notes-soft.txt -> notes.txt
original content
added via hard link
added via soft link
cat: notes-soft.txt: No such file or directory
```

What just happened: Removing `notes.txt` only removed one name. The inode's link count dropped from 2 to 1, so the data survives and `notes-hard.txt` still works. The symlink still contains the text `notes.txt`, but nothing has that name any more, so it is now *dangling* (`ls` shows it in red on a color terminal). Recreate the target and the symlink works again:

```
echo "brand new file" > notes.txt
cat notes-soft.txt
find . -xtype l
rm notes.txt
find . -xtype l
```

What just happened: The symlink now points at the *new* `notes.txt`, a completely different inode. `find -xtype l` lists broken symlinks, a useful clean-up check (the first run prints nothing, the second prints `./notes-soft.txt`).

4. Try links to a directory and across filesystems. `/dev/shm` is a separate in-memory filesystem (tmpfs).

```
mkdir projdir
ln projdir projdir-hard
ln -s ~/labs/2.1/webapp/config confdir
ls -l confdir/
ln notes-hard.txt /dev/shm/cross-hard.txt
ln -s ~/labs/2.1/links/notes-hard.txt /dev/shm/cross-soft.txt
cat /dev/shm/cross-soft.txt
df -h . /dev/shm | awk '{print $1, $6}'
```

```
ln: projdir: hard link not allowed for directory
ln: failed to create hard link '/dev/shm/cross-hard.txt' => 'notes-hard.txt':
Invalid cross-device link
```

What just happened: You cannot hard-link a directory (it could create loops in the tree). You can symlink a directory: `confdir` acts like a shortcut to the config directory. The trailing `/` in `ls -l confdir/` makes `ls` follow the link and list the contents. A hard link to `/dev/shm` fails with "Invalid cross-device link" because inode numbers only make sense inside one filesystem. A symlink across filesystems works fine because it just stores a path. `df` shows the two paths are on different filesystems.

5. Notice relative versus absolute symlinks, and how `pwd` reports a path reached through a symlink.

```
cd confdir
pwd
pwd -P
cd ~/labs/2.1/links
```

```
/home/student/labs/2.1/links/confdir
/home/student/labs/2.1/webapp/config
```

What just happened: `pwd` shows the *logical* path, including the symlink you walked through. `pwd -P` shows the *physical* path with all symlinks resolved. A symlink created with a relative target (like `notes.txt`) is resolved relative to the directory the link lives in, so it breaks if you move the link somewhere else. An absolute target (like `confdir`'s) keeps working wherever the link is moved.

ON THE EXAM

On the exam: a hard link shares the inode and survives deletion of the original; it cannot span filesystems or point to a directory. A symbolic link has its own inode, can span filesystems and point to directories, and breaks when its target is removed. `ls -li` and `stat` show inode numbers and link counts.

Checkpoint: `cat ~/labs/2.1/links/notes-hard.txt` still shows three lines, `find ~/labs/2.1/links -xtype l` prints `notes-soft.txt`, and `cat /dev/shm/cross-soft.txt` prints the same content.

Task 8: Find open files with `lsof`

1. Install `lsof` (it is usually already present) and look at the files your own shell has open.

```
sudo apt install -y lsof
lsof -p $$
```

```
COMMAND  PID    USER   FD   TYPE DEVICE SIZE/OFF    NODE NAME
bash    1532 student  cwd    DIR  252,0     4096 1311300 /home/student/labs/2.1/links
bash    1532 student  rtd    DIR  252,0     4096       2 /
bash    1532 student  txt    REG  252,0  1446024  786563 /usr/bin/bash
bash    1532 student    0u   CHR  136,0      0t0       3 /dev/pts/0
bash    1532 student    1u   CHR  136,0      0t0       3 /dev/pts/0
bash    1532 student    2u   CHR  136,0      0t0       3 /dev/pts/0
...
```

What just happened: `$$` is a shell variable holding the PID (process ID) of your current shell, and `-p` limits `lsof` to that process. Columns: **COMMAND** and **PID** identify the process. **FD** is the file descriptor: `cwd` is the current working directory, `rtd` the root directory, `txt` the program file, and numbers like `0u`, `1u`, `2u` are standard input, output, and error (`u` means open for read and write). **TYPE** shows `DIR`, `REG` (regular file), or `CHR` (character device; your terminal `/dev/pts/0`).

2. Scenario: "I can't remove or unmount this directory because something is using it." Start a background process that sits inside a directory, then find it.

```
(cd ~/labs/2.1/webapp/logs && sleep 600) &
lsof +D ~/labs/2.1/webapp
```

```
COMMAND  PID    USER  FD   TYPE DEVICE SIZE/OFF    NODE NAME
sleep   1688 student cwd    DIR  252,0     4096 1311279 /home/student/labs/2.1/webapp/logs
```

What just happened: The parentheses run a *subshell* that changes into `logs` and then runs `sleep 600` in the background (`&`). `lsof +D DIR` searches the whole directory tree for anything open, including processes whose current directory is inside it. This is exactly how you find what is keeping a filesystem busy before `umount`. `lsof DIRNAME` without `+D` checks only that one directory.

3. Scenario: "I deleted a huge log file but the disk space did not come back." Create a 200 MB log file and hold it open with `tail -f`.

```
cd ~/labs/2.1
dd if=/dev/zero of=huge.log bs=1M count=200 status=none
tail -f huge.log > /dev/null &
df -h /
rm huge.log
df -h /
```

What just happened: `tail -f` keeps the file open, waiting for new lines. After `rm`, the name is gone, but the second `df -h /` shows the *same* used space as before, because the inode still has an open file descriptor. The kernel frees the data only when the link count is 0 **and** no process has it open.

4. Find the deleted-but-open file and release the space.

```
lsof +L1
sudo lsof -u student | grep deleted
kill %2
df -h /
lsof +L1
```

```
COMMAND  PID    USER  FD  TYPE DEVICE  SIZE/OFF NLINK    NODE NAME
tail    1701 student  3r   REG  252,0 209715200     0 1311320
/home/student/labs/2.1/huge.log (deleted)
```

What just happened: `lsof +L1` lists open files with a link count less than 1, which means files that were deleted but are still open. The **NLINK** column is `0` and the name ends with `(deleted)`. `-u student` shows all files opened by a user. `kill %2` stops background job number 2 (the `tail`; job control is covered in Lab 2.3). After that, `df` shows the 200 MB freed. On a real server the usual fix is to restart the service that holds the log.

5. Two more lookups you will use constantly.

```
sudo lsof -i :22
lsof /dev/null | head -5
```

What just happened: `-i :22` lists network sockets using port 22 (the SSH server and your SSH session). `lsof FILE` lists every process that has a particular file open.

Checkpoint: `lsof +L1` prints nothing (the deleted file was released), and `jobs` shows only the `sleep 600` job still running.

Task 9: Tour the device files in `/dev`

1. List block and character devices side by side.

```
lsblk
ls -l /dev/sd* /dev/vd* /dev/nvme* 2>/dev/null
ls -l /dev/null /dev/zero /dev/random /dev/urandom /dev/full /dev/tty
```

```
brw-rw---- 1 root disk 8,  0 Oct 10 09:01 /dev/sda
brw-rw---- 1 root disk 8,  1 Oct 10 09:01 /dev/sda1
brw-rw---- 1 root disk 8, 16 Oct 10 09:01 /dev/sdb
brw-rw---- 1 root disk 8, 32 Oct 10 09:01 /dev/sdc
crw-rw-rw- 1 root root 1, 7 Oct 10 09:01 /dev/full
crw-rw-rw- 1 root root 1, 3 Oct 10 09:01 /dev/null
crw-rw-rw- 1 root root 1, 8 Oct 10 09:01 /dev/random
crw-rw-rw- 1 root tty  5, 0 Oct 10 09:01 /dev/tty
crw-rw-rw- 1 root root 1, 9 Oct 10 09:01 /dev/urandom
crw-rw-rw- 1 root root 1, 5 Oct 10 09:01 /dev/zero
```

What just happened: `lsblk` lists your disks so you know their names (they may be `vda` or `nvme0n1` instead of `sda`). In `ls -l`, disks start with `b` (**block device**) and are owned by group `disk`. The others start with `c` (**character device**). Where the file size normally appears you see two numbers: **major, minor**. All the SCSI/SATA disks share major `8` (the `sd` driver); the minor number picks the disk and partition (each disk gets 16 minors: `sda`=0, `sdb`=16). Major `1` is the kernel's "memory" driver, which provides the special character devices: minor `3` is null, `5` zero, `7` full, `8` random, `9` urandom.

2. See which drivers own which major numbers, and count device types.

```
grep -A8 'Character devices' /proc/devices
grep -A8 'Block devices' /proc/devices
find /dev -maxdepth 1 -type b | wc -l
find /dev -maxdepth 1 -type c | wc -l
```

What just happened: `/proc/devices` lists registered drivers and their major numbers in two sections (for example `1 mem`, `5 /dev/tty`, `136 pts`, `8 sd`). `find -type b` and `-type c` match block and character device files. Most of `/dev` is character devices.

3. Use `/dev/null`, the "black hole".

```
echo "nobody will see this" > /dev/null
cat /dev/null | wc -c
ls /root 2>/dev/null; echo "exit status: $?"
cp /dev/null ~/labs/2.1/webapp/logs/app.log
```

What just happened: Anything written to `/dev/null` is discarded, and reading it returns end-of-file immediately (0 bytes). Its main use is hiding output you do not want, like the error from `ls /root` (you still get the exit status). Copying `/dev/null` over a file empties it, a quick way to truncate a log.

4. Use `/dev/zero` and `/dev/full`.

```
head -c 16 /dev/zero | od -An -tx1
dd if=/dev/zero of=~/labs/2.1/zeros.bin bs=1M count=5 status=none
ls -lh ~/labs/2.1/zeros.bin
echo test > /dev/full
```

```
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
-rw-rw-r-- 1 student student 5.0M Oct 10 10:40 /home/student/labs/2.1/zeros.bin
-bash: echo: write error: No space left on device
```

What just happened: `/dev/zero` returns as many zero bytes as you read. `head -c 16` takes 16 bytes and `od -An -tx1` dumps them as hex. With `dd` it creates files of a known size or wipes a disk. `/dev/full` always fails writes with "No space left on device", which developers use to test how programs handle a full disk.

5. Use `/dev/urandom` and `/dev/random`.

```
head -c 16 /dev/urandom | od -An -tx1
head -c 12 /dev/random | base64
tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 20; echo
```

What just happened: Both devices return random bytes from the kernel's cryptographic random generator. The last line keeps only letters and digits from the random stream and takes 20 of them, a quick random password. Historically `/dev/random` *blocked* (paused) when the kernel estimated it was low on randomness, while `/dev/urandom` never blocks. Since kernel 5.6 `/dev/random` only blocks very early at boot, before the generator is ready; on a running system both behave the same.

6. Use `/dev/tty`, which always means "the terminal of the current process".

```
tty
echo "hello terminal" > /dev/tty
echo "hidden" > /dev/null; echo "shown anyway" 2>/dev/null > /dev/tty
```

```
/dev/pts/0
hello terminal
shown anyway
```

What just happened: `tty` prints the device file of your terminal. Over SSH this is a *pseudo-terminal* like `/dev/pts/0` (character device, major 136). `/dev/tty` is a special alias that always points to whatever terminal the current process is attached to. Scripts write to `/dev/tty` (or read passwords from it) when they must reach the user even if normal output is redirected to a file.

TIP

Explore/simulate: Physical serial ports appear as `/dev/ttyS0`, `/dev/ttyS1` (character devices, major 4) and USB serial adapters as `/dev/ttyUSB0`. Run `ls -l /dev/ttyS*` to see them even though no cable is attached. Virtual consoles (the screens you get on the VM console with `Ctrl+Alt+F1` … `F6`) are `/dev/tty1` to `/dev/tty6`.

Checkpoint: `ls -l /dev/null` starts with `c` and shows `1, 3`. `ls -l` on your first disk starts with `b`. `stat -c '%F' /dev/zero` prints `character special file`.

Challenge

CHALLENGE

1. Using one `find` command, list every file under `/etc` that is larger than 100 KB and was modified in the last 30 days, and show each with `ls -lh`.
2. Create a file in `~/labs/2.1`, make three hard links to it in different subdirectories, then use `find ~/labs/2.1 -samefile FILE` (or `-inum`) to list all four names. What link count does `stat` report?
3. A colleague says "the disk is full but `du` doesn't show any big files." Reproduce the situation with `dd` and `less`, explain why `df` and `du` disagree, and fix it without rebooting.

Clean up

1. Stop the background `sleep` (and anything else left from this lab) and confirm no jobs remain.

```
jobs
kill %1 2>/dev/null
pkill -u student -x sleep
pkill -u student -x tail
jobs
```

2. Remove the lab files, including the symlink in `/dev/shm`.

```
cd ~
rm -f /dev/shm/cross-soft.txt /dev/shm/cross-hard.txt
rm -rf ~/labs/2.1
```

3. Rebuild the `locate` database so it forgets the deleted files.

```
sudo updatedb
locate app.conf.bak
```

The last command should print nothing. You can leave `plocate` and `lsof` installed; later labs may use them.

Exam corner

- `ls -l` first character: `-` file, `d` directory, `l` symlink, `b` block device, `c` character device, `p` pipe, `s` socket. `ls -i` shows inode numbers.
- `cp -r` copies directories; `cp -a` copies recursively and preserves everything; `cp -p` preserves mode, ownership, and timestamps. `mv` renames or moves. `rmdir` only removes empty directories; `rm -r` removes non-empty ones.
- `mkdir -p` creates parent directories as needed. `pwd -P` resolves symlinks. `cd -` returns to the previous directory.

- `touch` creates empty files or updates timestamps; `touch -d "date"` or `-t YYYYMMDDhhmm` sets a specific time. ctime cannot be set manually.
- `stat` shows the inode: size, blocks, inode number, link count, permissions, UID/GID, and atime/mtime/ctime/birth.
- `file` identifies content by magic numbers, not by extension.
- `find PATH TESTS ACTION`: `-name`, `-iname`, `-type`, `-size +10M`, `-mtime +7`, `-mmin -30`, `-perm -4000`, `-user`, `-group`, `-exec cmd {} \;`, `-exec cmd {} +`, `-ok`, `-delete`.
- `locate` searches a database built by `updatedb` (package `plocate` on Ubuntu 24.04, `mlocate` on Rocky 9). It is fast but can be stale.
- `diff` shows changes (`<` first file, `>` second, `c` / `a` / `d`); `diff -u` is unified format; `sdiff` shows side by side with `|`, `<`, `>`.
- Hard link: same inode, link count increases, survives deletion of the original, no directories, no cross-filesystem. Symbolic link (`ln -s`): own inode, stores a path, can cross filesystems and point to directories, breaks if the target is removed.
- `lsof +D dir` finds what is using a directory; `lsof +L1` finds deleted-but-open files; `lsof -i :port`; `lsof -u user`; `lsof -p PID`.
- Block devices (disks, partitions) use random block access; character devices (terminals, serial ports) are byte streams. Special character devices: `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom`, `/dev/full`, `/dev/tty`. Device files show major (driver) and minor (instance) numbers.

Review questions

1. An administrator deletes `/data/report.csv`, which had a hard link at `/archive/report.csv` on the same filesystem. What happens to `/archive/report.csv`?
   A. It becomes a broken link.
   B. It is deleted as well.
   C. It still works and its link count drops by one.
   D. It is converted to a symbolic link.

2. Which command finds all files under `/var` larger than 500 MB?
   A. `locate -size +500M /var`
   B. `find /var -size +500M`
   C. `find /var -mtime +500`
   D. `ls -lhR /var --size 500M`

3. `df` shows `/var` is 95% full, but you removed a 10 GB log file an hour ago. Which command best identifies the cause?
   A. `lsof +L1`
   B. `stat /var`
   C. `sdiff /var /tmp`
   D. `updatedb`

4. You run `ls -l /dev/sdb` and the line begins with `b` and shows `8, 16`. What do the `b` and the numbers mean?

5. A user runs `locate nginx.conf` immediately after installing nginx, and nothing is found. What should they do?

6. Which command shows two versions of a file side by side, marking differing lines with `|`?
   A. `diff -u`
   B. `cmp`
   C. `sdiff`
   D. `stat`

Lab 2.2 Manage Local Accounts

Objective: 2.2 Given a scenario, perform local account management in a Linux environment.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 100 minutes
VM: ubuntu-lab (Task 9 also uses rocky-lab)
Snapshot first: Recommended. This lab edits `/etc/skel`, `/etc/profile.d`, and the account databases. The clean-up undoes everything, but a snapshot is a cheap safety net.

A small team is joining your company: two developers, an operations engineer, and a service that needs its own account. In this lab you onboard them, look inside the files where Linux keeps account data, lock and expire accounts, enforce password aging, see who is logged in, and finally offboard everyone cleanly. Account management is one of the most common real admin tasks, and the exam tests both the commands and the file formats behind them.

What you'll practice

- Add: `adduser`, `groupadd`, `useradd`
- Delete: `deluser`, `groupdel`, `userdel`
- Modify: `chsh`, `groupmod`, `passwd`, `usermod`
- Lock: `chage`, `passwd`, `usermod`
- Expiration: configuration files (`/etc/login.defs`, `/etc/default/useradd`), `chage`
- List: `getent passwd`, `groups`, `id`, `last`, `lastlog`, `w`, `who`, `whoami`
- Templates: `/etc/profile`, `/etc/skel`
- Account files: `/etc/group`, `/etc/passwd`, `/etc/shadow`
- Attributes: UID, GID, EUID, EGID
- User vs system vs service accounts: UID range

How it works

Users are numbers. The kernel does not know names like `alice`. It knows a **UID** (user ID), a number. Every process runs with a UID and every file is owned by a UID. Groups work the same way with a **GID** (group ID). Names exist only for humans; the system translates them using the account databases.

The account files. Local accounts live in three plain-text files, one line per entry, fields separated by colons:

- **`/etc/passwd`** (readable by everyone) holds the basic account info: name, a placeholder `x` for the password, UID, primary GID, a comment (the *GECOS* field, usually the full name), home directory, and login shell. Despite its name, it no longer contains passwords.

- `/etc/shadow` (readable only by root and the `shadow` group) holds the password *hash* and the password aging data. A *hash* is a one-way scramble of the password: the system can check a typed password by hashing it the same way, but cannot turn the hash back into the password.
- `/etc/group` holds groups: name, `x`, GID, and a comma-separated list of supplementary members. (`/etc/gshadow` holds group passwords, rarely used.)

```
/etc/passwd   alice:x:1001:1001:Alice Ng,Dev:/home/alice:/bin/bash
                |   |   |    |       |          |          |
              name  |  UID  GID   comment     home       shell
                    +-- "password is in /etc/shadow"

/etc/shadow   alice:$y$j9T$...:20736:0:99999:7:::
                |        |       |   |   |   | ||
              name     hash  lastchg min max warn inactive expire

/etc/group    developers:x:3001:alice,bob
                  |      |  |      |
                name     | GID  supplementary members
```

Primary and supplementary groups. Each user has exactly one *primary group* (the GID in `/etc/passwd`); new files get this group. A user can also belong to many *supplementary groups*, listed in `/etc/group`. Ubuntu and Rocky both create a *user private group* for each new user: a group with the same name as the user, used as the primary group.

Real versus effective IDs. A running process carries a **real UID** (who started it) and an **effective UID (EUID)** (whose permissions it uses right now). Usually they are the same. They differ when you run `sudo` (EUID becomes 0, root) or a *setuid* program such as `passwd`: the file is owned by root and has the setuid bit, so it runs with EUID 0 even when you start it. That is how an ordinary user can update `/etc/shadow`, but only through `passwd`'s rules. Groups have the same pair: real GID and **effective GID (EGID)**, which decides the group ownership of new files.

Kinds of accounts and UID ranges. The ranges come from `/etc/login.defs`:

Account type	UID range (Ubuntu 24.04 and Rocky 9)	Examples
root	0	`root`
System accounts	1 to 999	`daemon`, `bin`, `systemd-network`, `sshd`
Regular (human) users	1000 to 60000	`student`, `alice`
Special	65534	`nobody` (owns nothing, used for least privilege)

A **system account** is any account in the low range, created by the OS or packages. A **service account** is a system account created so one service (a web server, a database, your own app) runs under its own identity instead of root. Service accounts usually have no password, no usable shell (`/usr/sbin/nologin`), and a home directory under `/var/lib` or none at all.

Templates and defaults. When `useradd -m` creates a home directory, it copies everything in **`/etc/skel`** (skeleton) into it, so every new user starts with the same `.bashrc`, `.profile`, and any files you add. **`/etc/profile`** (plus the scripts in `/etc/profile.d/`) runs for *every* user at each login shell, so it is the place for system-wide environment settings and login messages. Defaults for new accounts come from **`/etc/login.defs`** (UID ranges, password aging defaults, hash method, whether to create home directories) and **`/etc/default/useradd`** (default shell, skeleton directory, account inactivity and expiration defaults).

Ubuntu's two tools. `useradd`, `usermod`, `userdel`, `groupadd`, `groupmod`, and `groupdel` are the low-level tools found on every distribution. They do exactly what the options say and nothing more. Ubuntu (like Debian) also ships **`adduser`** and **`deluser`**, friendlier Perl scripts that ask questions, create the home directory, set a password, and follow the policy in `/etc/adduser.conf`. On Rocky, `adduser` is simply a link to `useradd`.

Tasks

Task 1: Explore the account files and UID ranges

1. Make a lab directory and look at your own identity.

```
mkdir -p ~/labs/2.2 && cd ~/labs/2.2
whoami
id
groups
```

```
student
uid=1000(student) gid=1000(student)
groups=1000(student),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),101(lxd)
student adm cdrom sudo dip plugdev lxd
```

> **What just happened:** `whoami` prints the user name of your *effective* UID. `id` prints your UID, primary GID, and all groups, with numbers and names. `groups` prints only the group names. Membership in `sudo` is what lets `student` run admin commands on Ubuntu. Your list of groups may be slightly different.

2. Look at the first lines of `/etc/passwd` and at your own entry.

```
head -5 /etc/passwd
grep '^student:' /etc/passwd
getent passwd student
getent passwd 0
```

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
student:x:1000:1000:student:/home/student:/bin/bash
student:x:1000:1000:student:/home/student:/bin/bash
root:x:0:0:root:/root:/bin/bash
```

What just happened: Each line has seven colon-separated fields: name, password placeholder (`x`), UID, GID, comment, home, and shell. `getent passwd NAME` ("get entries") asks the system's name service for the account instead of reading the file directly. The result is the same here, but `getent` also finds accounts from LDAP or Active Directory if the system is joined to a directory, so it is the correct way to look up users in scripts. It accepts a UID too (`getent passwd 0`).

3. Separate system accounts from human users with `awk`.

```
awk -F: '$3 < 1000 {print $3, $1, $7}' /etc/passwd | head -15
awk -F: '$3 >= 1000 && $3 < 60000 {print $3, $1, $7}' /etc/passwd
grep -E '^(UID_MIN|UID_MAX|SYS_UID_MIN|SYS_UID_MAX|GID_MIN|GID_MAX)' /etc/login.defs
```

```
UID_MIN          1000
UID_MAX         60000
GID_MIN          1000
GID_MAX         60000
```

What just happened: `awk -F:` splits each line on colons, so `$1` is the name, `$3` the UID, and `$7` the shell. The first command lists accounts below 1000: system accounts, nearly all with the shell `/usr/sbin/nologin`, which refuses interactive logins. The second lists regular users (probably only `student`). `/etc/login.defs` defines the regular-user range. On Ubuntu the `SYS_UID_MIN` / `SYS_UID_MAX` lines are commented out, so the built-in defaults apply (system accounts get UIDs below 1000, allocated from 999 downward).

4. Look at groups and at the protected shadow file.

```
grep -E '^(sudo|adm|student):' /etc/group
getent group sudo
ls -l /etc/passwd /etc/shadow /etc/group /etc/gshadow
cat /etc/shadow
sudo grep '^student:' /etc/shadow
```

```
adm:x:4:syslog,student
sudo:x:27:student
student:x:1000:
-rw-r--r-- 1 root root   1854 Oct 10 09:00 /etc/group
-rw-r----- 1 root shadow  774 Oct 10 09:00 /etc/gshadow
-rw-r--r-- 1 root root   1965 Oct 10 09:00 /etc/passwd
-rw-r----- 1 root shadow 1143 Oct 10 09:00 /etc/shadow
cat: /etc/shadow: Permission denied
student:$y$j9T$Hk3x...long hash...:20735:0:99999:7:::
```

What just happened: In `/etc/group` the last field lists *supplementary* members. The `student` group has an empty member list because `student` is in it as a *primary* group (from `/etc/passwd`), which is never repeated here. `/etc/passwd` and `/etc/group` are world-readable (`rw-r--r--`) because many programs need to map numbers to names. `/etc/shadow` is `rw-r-----` owned by `root:shadow`, so `cat` as a normal user fails and you need `sudo`. You will dissect the shadow fields in Task 4.

Checkpoint: You can name the seven fields of `/etc/passwd` and say why `cat /etc/shadow` fails without `sudo`. `getent passwd student | cut -d: -f3` prints `1000`.

Task 2: Prepare the templates: `/etc/skel`, `/etc/profile`, and defaults

Before anyone is created, set up what every new user should receive.

1. Look at the skeleton directory.

```
ls -la /etc/skel
```

```
total 20
drwxr-xr-x   2 root root 4096 Apr 23  2024 .
drwxr-xr-x 108 root root 4096 Oct 10 09:00 ..
-rw-r--r--   1 root root  220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root root 3771 Mar 31  2024 .bashrc
-rw-r--r--   1 root root  807 Mar 31  2024 .profile
```

What just happened: These three hidden files are copied into every new home directory. `.bashrc` runs for interactive Bash shells, `.profile` for login shells, and `.bash_logout` when a login shell exits.

2. Add a welcome file and a `bin` directory to the skeleton. This step uses a *heredoc*.

```
cat <<'EOF' | sudo tee /etc/skel/WELCOME.txt
Welcome to the company Linux server!
- Read the acceptable use policy in /srv/policy (coming soon).
- Put your own scripts in ~/bin.
- Ask the ops team for help: ops@example.com
EOF
sudo mkdir -p /etc/skel/bin
ls -la /etc/skel
```

What just happened: A *heredoc* (`<<'EOF' ... EOF`) feeds the lines between the markers into a command as if you had typed them. Quoting `'EOF'` stops the shell from expanding `$` variables inside. Because you cannot redirect into a root-owned file with `sudo echo ... >` (the redirection runs as you, not root), the text is piped into `sudo tee FILE`, which writes it as root and also prints it to the screen. Ubuntu's default `.profile` adds `~/bin` to `PATH` if the directory exists, so new users get a working personal script directory.

3. Add a system-wide login message through `/etc/profile`. Look at how it loads extra scripts first.

```
grep -n -A6 'profile.d' /etc/profile
cat <<'EOF' | sudo tee /etc/profile.d/lab-motd.sh
# Lab 2.2: printed for every login shell
echo "[profile] Hello $USER, you are logged in to $(hostname) at $(date +%H:%M)."
EOF
```

```
27:if [ -d /etc/profile.d ]; then
28-  for i in /etc/profile.d/*.sh; do
29-    if [ -r $i ]; then
30-      . $i
31-    fi
32-  done
33-  unset i
```

What just happened: `/etc/profile` runs for every user's *login shell* (an SSH login, a console login, or `su -`). It loops over every readable `*.sh` file in `/etc/profile.d/` and *sources* it (the `.` command runs it inside the current shell). Dropping a small file into `/etc/profile.d/` is better than editing `/etc/profile` itself, because package updates may replace `/etc/profile`. Here the quotes around `'EOF'` keep `$USER` unexpanded in the file, so it is evaluated at each login. Your line numbers may differ.

4. Check the defaults that `useradd` will apply.

```
useradd -D
cat /etc/default/useradd | grep -v '^#' | grep -v '^$'
grep -E '^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE)' /etc/login.defs
grep -E '^(ENCRYPT_METHOD|USERGROUPS_ENAB|CREATE_HOME|HOME_MODE)' /etc/login.defs
```

```
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/sh
SKEL=/etc/skel
CREATE_MAIL_SPOOL=no
LOG_INIT=yes
SHELL=/bin/sh
PASS_MAX_DAYS   99999
PASS_MIN_DAYS   0
PASS_WARN_AGE   7
HOME_MODE   0750
USERGROUPS_ENAB yes
ENCRYPT_METHOD SHA512
```

What just happened: `useradd -D` prints the defaults stored in `/etc/default/useradd` (merged with built-in values). On Ubuntu that file sets only one value, `SHELL=/bin/sh`; everything else is commented out, so built-in defaults fill the gaps. Note two Ubuntu surprises: the default shell is `/bin/sh` (not Bash), and there is no `CREATE_HOME yes` line in `login.defs`, so **plain `useradd` does not create a home directory** unless you add `-m`. `INACTIVE=-1` and an empty `EXPIRE` mean accounts never become inactive or expire by default. From `login.defs`: passwords effectively never expire (`99999` days), can be changed any time (`0`), users are warned 7 days before expiry, each user gets a private group, and home directories are created with mode `0750`. `ENCRYPT_METHOD SHA512` may surprise you: on Ubuntu, `passwd`, `chpasswd`, and `adduser` set passwords through PAM (Pluggable Authentication Modules, covered in Lab 3.1), and the line `pam_unix.so obscure yescrypt` in `/etc/pam.d/common-password` makes them use **yescrypt**. The `login.defs` value is only used by the few tools that bypass PAM. These aging values apply only to accounts created *after* you change them. You can change a `useradd` default with, for example, `sudo useradd -D -s /bin/bash`, but leave it as is for this lab.

Checkpoint: `ls -a /etc/skel` shows `WELCOME.txt` and `bin`, and `cat /etc/profile.d/lab-motd.sh` shows the `echo` line with `$USER` (not your name).

Task 3: Onboard the team with `groupadd`, `useradd`, and `adduser`

1. Create the team groups. Give them fixed GIDs so they match on every server.

```
sudo groupadd -g 3001 developers
sudo groupadd -g 3002 ops
sudo groupadd -g 3003 projectx
getent group developers ops projectx
```

```
developers:x:3001:
ops:x:3002:
projectx:x:3003:
```

What just happened: `groupadd NAME` creates a group. `-g` chooses the GID; without it, the next free GID from `GID_MIN` (1000) is used. Fixed GIDs matter when files are shared between servers over NFS, because ownership is stored as a number. `groupadd -r` would create a *system* group with a GID below 1000.

2. Create Alice, a developer, with `useradd` and every important option.

```
sudo useradd -m -s /bin/bash -c "Alice Ng,Developer" -G developers,projectx alice
getent passwd alice
id alice
sudo ls -la /home/alice
```

```
alice:x:1001:1001:Alice Ng,Developer:/home/alice:/bin/bash
uid=1001(alice) gid=1001(alice) groups=1001(alice),3001(developers),3003(projectx)
total 28
drwxr-x--- 3 alice alice 4096 Oct 10 10:15 .
drwxr-xr-x 4 root  root  4096 Oct 10 10:15 ..
-rw-r--r-- 1 alice alice  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 alice alice 3771 Mar 31  2024 .bashrc
-rw-r--r-- 1 alice alice  807 Mar 31  2024 .profile
-rw-r--r-- 1 alice alice  178 Oct 10 10:12 WELCOME.txt
drwxr-xr-x 2 alice alice 4096 Oct 10 10:12 bin
```

What just happened: `useradd` options: `-m` creates the home directory and copies `/etc/skel` into it, `-s` sets the login shell, `-c` sets the comment (GECOS) field, and `-G` sets supplementary groups (comma-separated, no spaces). The user name always comes last. A private group `alice` (GID 1001, matching the UID) was created as the primary group. `WELCOME.txt` and `bin` came from the skeleton, now owned by `alice`. The home directory is `drwxr-x---` (`0750`), so other users cannot look inside. Other useful options: `-u` to pick the UID, `-g` to pick the primary group, `-d` to choose a different home path, and `-e` to set an expiration date.

3. Create Bob with a different style: no `-m` and no `-s`, to see the Ubuntu defaults bite.

```
sudo useradd -c "Bob Ruiz" bob
getent passwd bob
ls -ld /home/bob
sudo userdel bob
sudo useradd -m -s /bin/bash -c "Bob Ruiz,Developer" -G developers bob
ls -ld /home/bob
```

```
bob:x:1002:1002:Bob Ruiz:/home/bob:/bin/sh
ls: cannot access '/home/bob': No such file or directory
drwxr-x--- 3 bob bob 4096 Oct 10 10:18 /home/bob
```

What just happened: Without `-m`, Ubuntu's `useradd` records `/home/bob` as the home directory but never creates it, and without `-s` the shell is `/bin/sh`. Bob would log in to a bare shell with no home directory. You deleted that half-made account with `userdel` (more on it in Task 8) and recreated it correctly.

4. Set passwords. Use `passwd` interactively for Alice, and `chpasswd` for Bob. Use the password `Welcome2Lab!` for every lab user.

```
sudo passwd alice
echo 'bob:Welcome2Lab!' | sudo chpasswd
sudo passwd -S alice
sudo passwd -S bob
```

```
New password:
Retype new password:
passwd: password updated successfully
alice P 2026-10-10 0 99999 7 -1
bob P 2026-10-10 0 99999 7 -1
```

What just happened: `sudo passwd USER` sets another user's password without knowing the old one (the typed characters are not shown). `chpasswd` reads `user:password` lines from standard input, handy for scripts. `passwd -S` (status) shows: user name, state (`P` usable password, `L` locked, `NP` no password), date of last change, minimum days, maximum days, warning days, and inactivity days (`-1` means disabled). On some versions the date appears as `10/10/2026`. Until you set a password, a `useradd` account has `!` in the shadow file and shows `L`.

5. Create Carol, the ops engineer, with Ubuntu's interactive `adduser`, then add her to `ops`.

```
sudo adduser carol
```

```
info: Adding user `carol' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `carol' (1003) ...
info: Adding new user `carol' (1003) with group `carol (1003)' ...
info: Creating home directory `/home/carol' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for carol
Enter the new value, or press ENTER for the default
    Full Name []: Carol Diaz
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `carol' to supplemental / extra groups `users' ...
info: Adding user `carol' to group `users' ...
```

Then add her to the `ops` group, again with `adduser`:

```
sudo adduser carol ops
id carol
getent passwd carol
```

What just happened: `adduser` did in one interactive run what took several `useradd` options: it created a private group, the user with shell `/bin/bash`, the home directory with skeleton files, set the password, and filled in the GECOS field (Full Name, Room, Work Phone, Home Phone, Other, separated by commas). Its settings come from `/etc/adduser.conf`. With two arguments, `adduser USER GROUP` adds an existing user to an existing group. The exact messages vary a little between versions.

> **TIP**
> On the exam and in scripts, prefer `useradd`: it exists on every distribution and never asks questions. `adduser` is a Debian/Ubuntu convenience. On Rocky Linux, `adduser` is just a symbolic link to `useradd` and behaves exactly like it.

6. Log in as Alice with `su -` to confirm the templates worked. Type Alice's password when asked.

```
su - alice
```

```
Password:
[profile] Hello alice, you are logged in to ubuntu-lab at 10:22.
```

Inside Alice's shell, run:

```
pwd
cat WELCOME.txt
echo $SHELL
exit
```

> **What just happened:** `su - alice` (switch user) starts a *login shell* as Alice; the `-` makes it a full login, so `/etc/profile` (and therefore your `/etc/profile.d/lab-motd.sh`) runs and you start in her home directory. `exit` returns to `student`.

> **Checkpoint:** `id alice`, `id bob`, and `id carol` all work; Alice is in `developers` and `projectx`, Bob in `developers`, Carol in `ops` and `users`. Each has a home directory containing `WELCOME.txt`.

Task 4: Dissect `/etc/shadow` and password hashes

1. Display the shadow entries for the new users.

```
sudo grep -E '^(alice|bob|carol|student):' /etc/shadow
```

```
student:$y$j9T$Hk3x...:20735:0:99999:7:::
alice:$y$j9T$3QeV2b0i8lF5d1pB/9W.r.$Kx1v...:20736:0:99999:7:::
bob:$y$j9T$aZ0...:20736:0:99999:7:::
carol:$y$j9T$p9L...:20736:0:99999:7:::
```

2. Read each field. Using Alice's line:

#	Field	Alice's value	Meaning
1	Login name	`alice`	Matches `/etc/passwd`
2	Password hash	`$y$j9T$...`	`$y$` = yescrypt; `!` or `*` = no password login
3	Last change	`20736`	Days since 1970-01-01 when the password was last changed
4	Minimum age	`0`	Days before the user may change it again
5	Maximum age	`99999`	Days until it must be changed
6	Warning	`7`	Days of warning before it expires
7	Inactive	(empty)	Days after expiry before the account is disabled
8	Expire	(empty)	Days since 1970-01-01 when the *account* expires
9	Reserved	(empty)	Unused

Convert the day number into a date:

```
date -d "1970-01-01 +20736 days" +%F
echo $(( $(date +%s) / 86400 ))
```

What just happened: Shadow dates are stored as a count of days since the *epoch* (1970-01-01). `date -d` converts that number into a calendar date. The second line divides the current time in seconds (`date +%s`) by 86,400 seconds per day to get today's day number. Use your own value from the shadow file.

3. Understand the hash format.

```
 $y$  j9T  $  3QeV2b0i8lF5d1pB/9W.r.  $  Kx1v...(43 chars)
  |    |          |                       |
 ID  params       salt                   hash

 $1$ MD5 (obsolete)  $5$ SHA-256  $6$ SHA-512 (Rocky 9)  $y$ yescrypt (Ubuntu 24.04)
```

Try it yourself by generating a hash with the same password but a new random salt:

```
sudo apt install -y whois
mkpasswd -m yescrypt 'Welcome2Lab!'
mkpasswd -m sha-512 'Welcome2Lab!'
```

What just happened: The first part between `$` signs names the algorithm. Ubuntu 24.04 uses **yescrypt** (`$y$`), chosen by the `yescrypt` option of `pam_unix.so` in `/etc/pam.d/common-password` ; Rocky 9 uses SHA-512 (`$6$`). The *salt* is a random string mixed into the password before hashing, so two users with the same password (like Alice and Bob) still have completely different hashes. `mkpasswd` (in the `whois` package on Ubuntu) creates a hash; run it twice and the output changes each time because of the new salt. In the password field, a leading `!` means "locked", and `*` or `!` alone means "no password login possible".

Checkpoint: Alice and Bob have the same password but different hash strings in `/etc/shadow` , and both start with `$y$` .

Task 5: Modify accounts with `usermod` , `chsh` , `groupmod` , and `passwd`

1. Update Bob's comment and move him into ops as well. First make the classic mistake.

```
sudo usermod -c "Bob Ruiz,Developer,Room 12" bob
sudo usermod -G ops bob
id bob
```

```
uid=1002(bob) gid=1002(bob) groups=1002(bob),3002(ops)
```

What just happened: `usermod -c` replaced the comment. But `usermod -G ops` *replaced* Bob's whole supplementary group list with just `ops` , so he lost `developers` . This mistake can lock people out of shared directories, or remove an admin from `sudo` .

2. Fix it with `-a` (append).

```
sudo usermod -aG developers bob
id bob
groups bob
```

```
uid=1002(bob) gid=1002(bob) groups=1002(bob),3002(ops),3001(developers)
bob : bob ops developers
```

What just happened: `-aG GROUP` *appends* the group to the existing list. Always use `-a` with `-G` unless you really mean to replace the list. Group changes take effect at the user's next login; a user who is already logged in keeps the old list until they log out and back in (or run `newgrp`).

3. Change login shells with `chsh` and `usermod -s` .

```
cat /etc/shells
sudo chsh -s /bin/sh bob
getent passwd bob | cut -d: -f7
sudo usermod -s /bin/bash bob
getent passwd bob | cut -d: -f7
```

What just happened: `chsh` (change shell) edits field 7 of `/etc/passwd`. `/etc/shells` lists the valid login shells. A normal user can run `chsh` (with no `sudo`) to change *their own* shell after typing their password, but only to a shell listed in `/etc/shells`. Root can set any shell for any user with `chsh -s SHELL USER` or `usermod -s SHELL USER`.

4. Let Alice change her own password, and see the rules for normal users.

```
su - alice -c passwd
```

```
Password:
Changing password for alice.
Current password:
New password:
Retype new password:
passwd: password updated successfully
```

What just happened: `su - alice -c passwd` runs a single command as Alice. Type `Welcome2Lab!` for the `su` prompt and as the current password, then choose a new password (for example `Welcome3Lab!`) and type it twice. A normal user must give their current password and can only change their own. If you choose something too short, `passwd` refuses with a message like "You must choose a longer password". Remember the new password for later tasks.

5. Rename a group and change a GID with `groupmod`.

```
sudo groupmod -n project-x projectx
sudo groupmod -g 3004 project-x
getent group project-x
id alice
```

```
project-x:x:3004:alice
uid=1001(alice) gid=1001(alice) groups=1001(alice),3001(developers),3004(project-x)
```

What just happened: `groupmod -n NEW OLD` renames a group and `-g` changes its GID. Members follow automatically because `/etc/group` stores member names. Files on disk store the *GID number*, so any file that belonged to GID 3003 would now show a bare number in `ls -l`; fix those with `find / -gid 3003 -exec chgrp project-x {} +`. The same is true for `usermod -u` with UIDs (though `usermod -u` does fix files inside the home directory).

6. A few more `usermod` options you should recognize.

```
sudo usermod -l robert bob
getent passwd robert
sudo usermod -l bob robert
getent passwd bob
```

What just happened: `usermod -l NEW OLD` changes the login name (the UID, home path, and files stay the same). You renamed Bob to `robert` and back. `usermod -d NEWPATH -m` changes the home directory and moves its contents to the new place. Other options: `-g` changes the primary group, `-u` changes the UID, `-e` sets an expiry date, and `-L` / `-U` lock and unlock (next task).

Checkpoint: `id bob` shows `ops` and `developers`, `getent passwd bob` ends in `/bin/bash`, and `getent group project-x` shows GID `3004` with member `alice`.

Task 6: Lock, unlock, expire, and age passwords

Scenario: Bob is on leave, Carol is a contractor whose contract ends soon, and security wants passwords changed every 90 days.

1. Lock Bob's password with `passwd -l`, check, and test it.

```
sudo passwd -l bob
sudo passwd -S bob
sudo grep '^bob:' /etc/shadow | cut -c1-20
su - bob
```

```
passwd: password changed.
bob L 2026-10-10 0 99999 7 -1
bob:!$y$j9T$aZ0...
Password:
su: Authentication failure
```

What just happened: `passwd -l` (lock) puts a `!` in front of the hash. No typed password can ever match a hash that starts with `!`, so password logins fail even with the correct password (type `Welcome2Lab!` at the `su` prompt to see it). `passwd -S` now shows `L`. The original hash is kept, so unlocking restores the old password.

2. Unlock with `passwd -u`, then lock and unlock again with `usermod`.

```
sudo passwd -u bob
sudo usermod -L bob
sudo passwd -S bob
sudo usermod -U bob
sudo passwd -S bob
```

What just happened: `passwd -u` and `usermod -U` remove the `!`; `usermod -L` adds it. They are two interfaces to the same change. Bob shows `P` again at the end.

> **CAUTION**
> Locking the password does **not** stop every kind of login. A user with an SSH key in `~/.ssh/authorized_keys` can still log in, because key logins never check the password hash. To fully disable an account, also expire it (next step) or set its shell to `/usr/sbin/nologin`.

3. Expire Bob's account completely with `chage -E 0`.

```
sudo chage -E 0 bob
sudo chage -l bob | grep 'Account expires'
su - bob
```

```
Account expires                     : Jan 01, 1970
Password:
Your account has expired; please contact your system administrator.
su: Authentication failure
```

> **What just happened:** `chage -E` sets the account *expiration* date (field 8 of shadow). `0` means day 0, 1970-01-01, which is in the past, so the account is expired immediately. This blocks all logins that go through PAM account checks, including SSH key logins. `usermod -e 1970-01-02` (or any past date) does the same thing.

4. Remove the expiration, then give Carol a real end date 30 days from now.

```
sudo chage -E -1 bob
sudo chage -l bob | grep 'Account expires'
sudo chage -E "$(date -d '+30 days' +%F)" carol
sudo usermod -e "$(date -d '+30 days' +%F)" carol
sudo chage -l carol
```

```
Account expires                     : never
Last password change                    : Oct 10, 2026
Password expires                    : never
Password inactive                   : never
Account expires                     : Nov 09, 2026
Minimum number of days between password change      : 0
Maximum number of days between password change      : 99999
Number of days of warning before password expires   : 7
```

> **What just happened:** `-E -1` clears the expiration date ("never"). `date -d '+30 days' +%F` prints a date 30 days ahead in `YYYY-MM-DD` form, and the `$( )` *command substitution* inserts it into the command. Both `chage -E` and `usermod -e` set the same shadow field; you used both to see that they agree. `chage -l` (list) shows all aging information in readable form; any user can run `chage -l` on their own account.

5. Apply the 90-day password policy to Alice.

```
sudo chage -M 90 -m 1 -W 14 -I 7 alice
sudo chage -l alice
sudo grep '^alice:' /etc/shadow | cut -d: -f3-8
```

```
Last password change                                    : Oct 10, 2026
Password expires                                        : Jan 08, 2027
Password inactive                                       : Jan 15, 2027
Account expires                                         : never
Minimum number of days between password change          : 1
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 14
20736:1:90:14:7:
```

What just happened: `-M 90` (maximum) forces a change every 90 days; `-m 1` (minimum) stops a user from changing the password several times in a row to get back to an old one; `-W 14` (warning) starts warning 14 days before expiry; `-I 7` (inactive) disables the account if the password stays expired for 7 days. These are shadow fields 5, 4, 6, and 7. Run `sudo chage alice` with no options to be prompted for each value interactively.

6. Force Bob to choose a new password at his next login.

```
sudo chage -d 0 bob
sudo chage -l bob | head -2
su - bob
```

```
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password:
You are required to change your password immediately (administrator enforced).
Changing password for bob.
Current password:
```

What just happened: `chage -d 0` sets the "last change" date to day 0, which makes the password already expired. At the next login Bob must type his current password (`Welcome2Lab!`) and choose a new one (for example `Welcome4Lab!`) before he gets a shell. This is the standard way to hand a new user a temporary password. `passwd -e bob` does the same thing. After you set the new password you are logged in as Bob; type `exit` .

ON THE EXAM

On the exam: `passwd -l` / `usermod -L` lock the password (prefix `!`); `passwd -u` / `usermod -U` unlock it. `chage -E 0` or `usermod -e` with a past date expires the whole account. `chage -d 0` or `passwd -e` forces a password change at next login. `chage -l` lists aging; `-M` , `-m` , `-W` , `-I` set max, min, warn, inactive.

Checkpoint: `sudo chage -l alice` shows maximum `90` ; `sudo chage -l carol` shows an account expiry date about 30 days ahead; `sudo passwd -S bob` shows `P` with today's date.

Task 7: See who is logged in and which IDs a process uses

1. Open a **second** SSH session to `ubuntu-lab` (keep the first one open), so there is more than one login to see. In either session run:

```
whoami
who
who -b
w
```

```
student
student  pts/0        2026-10-10 09:58 (192.168.100.1)
student  pts/1        2026-10-10 10:41 (192.168.100.1)
         system boot  2026-10-10 09:01
 10:42:07 up  1:41,  2 users,  load average: 0.00, 0.01, 0.00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
student  pts/0    192.168.100.1    09:58    1.00s  0.21s  0.00s w
student  pts/1    192.168.100.1    10:41   40.00s  0.02s  0.02s -bash
```

> **What just happened:** `who` lists current logins: user, terminal (`pts/N` for SSH sessions), login time, and origin IP. `who -b` shows the last boot time. `w` adds a header (time, uptime, number of users, load averages) and, for each session, idle time, CPU used, and **WHAT** the user is running right now. Both read the current login records in `/var/run/utmp`. Note that `su` does not create a new login record, so `su - alice` sessions do not appear here.

2. Look at login history with `last` and `lastlog`.

```
last -n 5
last reboot | head -3
sudo lastb | head -5
lastlog
lastlog -u alice
```

```
student  pts/1        192.168.100.1    Sat Oct 10 10:41   still logged in
student  pts/0        192.168.100.1    Sat Oct 10 09:58   still logged in
reboot   system boot  6.8.0-45-generic Sat Oct 10 09:01   still running
...
Username         Port     From             Latest
root                                       **Never logged in**
daemon                                     **Never logged in**
...
student          pts/1    192.168.100.1    Sat Oct 10 10:41:12 +0000 2026
alice                                      **Never logged in**
```

What just happened: `last` reads `/var/log/wtmp`, the history of logins, logouts, and reboots, newest first; `-n 5` limits it to five lines, and `last reboot` shows only boots. `lastb` reads `/var/log/btmp`, the record of *failed* logins (it needs root and may be empty on a fresh VM). `lastlog` reads `/var/log/lastlog` and shows the most recent login of *every* account, so it is a quick way to spot accounts nobody uses. Alice shows "Never logged in" because `su` does not update these records; only real logins (SSH, console) do. On a new VM some of these lists may be short or empty, which is normal.

TIP

Newer distributions are replacing `lastlog` with `lastlog2` (from util-linux) and `wtmp` with `wtmpdb`. Ubuntu 24.04 and Rocky 9 still ship the classic `lastlog` and `last`. If `lastlog` is ever "command not found", try `lastlog2`.

3. Compare real and effective UID with `sudo`. Close the second session first (`exit`).

```
id -u
sudo id -u
whoami
sudo whoami
sudo bash -c 'echo "UID=$UID EUID=$EUID"'
echo "UID=$UID EUID=$EUID"
```

```
1000
0
student
root
UID=0 EUID=0
UID=1000 EUID=1000
```

What just happened: `sudo` runs the command as root, so its effective UID is `0` and `whoami` (which reports the *effective* user) prints `root`. Bash exposes the IDs as the variables `$UID` and `$EUID`. Scripts often test `[ "$EUID" -eq 0 ]` to check whether they are running as root.

4. Catch a setuid program running with EUID 0. Start `passwd` in the background; it stops as soon as it tries to read your password.

```
ls -l /usr/bin/passwd
passwd &
sleep 1
ps -o pid,ruser,euser,ruid,euid,rgid,egid,stat,cmd -C passwd
```

```
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
[1] 2410
[1]+  Stopped                 passwd
    PID RUSER    EUSER     RUID  EUID  RGID  EGID STAT CMD
   2410 student  root      1000     0  1000  1000 T    passwd
```

What just happened: The `s` in `-rwsr-xr-x` is the *setuid* bit: the program runs with the file owner's UID (root) as its EUID. `passwd &` starts it as a background job; because a background job is not allowed to read from the terminal, the kernel stops it (state `T`). `ps -C passwd` selects processes by command name, and `-o` chooses columns: the **real** user is `student` (UID 1000), but the **effective** user is `root` (EUID 0). That is how `passwd` can write `/etc/shadow`. The real and effective GIDs stay 1000 because `passwd` is not setgid.

Bring it back to the foreground and cancel it:

```
fg
```

When you see `Current password:`, press `Ctrl+C`.

5. See EGID in action. Some programs are *setgid* instead, and you can also switch your effective group with `sg`.

```
ls -l /usr/bin/chage /usr/bin/crontab
sudo usermod -aG developers student
sg developers -c 'id; touch ~/labs/2.2/egid-test.txt'
ls -l ~/labs/2.2/egid-test.txt
```

```
-rwxr-sr-x 1 root shadow  72184 May 30  2024 /usr/bin/chage
-rwxr-sr-x 1 root crontab 39664 Mar 31  2024 /usr/bin/crontab
uid=1000(student) gid=3001(developers) groups=3001(developers),4(adm),...,1000(student)
-rw-rw-r-- 1 student developers 0 Oct 10 10:50 /home/student/labs/2.2/egid-test.txt
```

What just happened: An `s` in the *group* execute position means *setgid*: `chage` runs with EGID `shadow`, so a normal user can read their own aging data from `/etc/shadow` with `chage -l`. `sg GROUP -c COMMAND` runs a command with a different group as its real and effective GID (you must be a member, which `usermod -aG` just arranged; `sg` reads `/etc/group` directly, so no re-login is needed). The new file got the group `developers` because new files take the creator's **EGID**. You remove `student` from `developers` in the clean-up.

Checkpoint: You saw `EUID 0` with real UID `1000` for `passwd`, and `ls -l ~/labs/2.2/egid-test.txt` shows the group `developers`. `jobs` shows no stopped `passwd`.

Task 8: Create a service account, then offboard users

1. Create a dedicated service account for an application called `reportd`.

```
sudo useradd -r -s /usr/sbin/nologin -d /var/lib/reportd -m -c "Report daemon" reportd
getent passwd reportd
sudo passwd -S reportd
ls -ld /var/lib/reportd
```

```
reportd:x:987:987:Report daemon:/var/lib/reportd:/usr/sbin/nologin
reportd L 2026-10-10 -1 -1 -1 -1
drwxr-x--- 2 reportd reportd 4096 Oct 10 10:55 /var/lib/reportd
```

What just happened: `-r` creates a **system account**: its UID and GID come from the system range (below 1000, allocated from the highest free number below 1000 downward, so your number will differ), and no password aging is applied (`-1` values). `-s /usr/sbin/nologin` gives it a shell that refuses logins. `-d` with `-m` creates a home under `/var/lib`, where services keep data. It has no password, so `passwd -S` shows `L`. This is a **service account**: it exists only so the `reportd` service runs with its own limited permissions instead of as root. (`adduser --system reportd` is the Ubuntu equivalent.)

2. Prove that nobody can log in as it, but an admin can still run commands as it.

```
sudo su - reportd
sudo -u reportd whoami
sudo -u reportd touch /var/lib/reportd/data.db
ls -l /var/lib/reportd
```

```
This account is currently not available.
reportd
-rw-r--r-- 1 reportd reportd 0 Oct 10 10:56 data.db
```

What just happened: `su -` tried to start the account's shell, `/usr/sbin/nologin`, which printed its message and exited. `sudo -u USER COMMAND` runs one command as another user without starting a login shell, so it works. In real life a systemd unit would contain `User=reportd`, and the service would own only its own files.

3. Look at a few service accounts the system already has.

```
getent passwd www-data systemd-network sshd nobody
```

What just happened: `www-data` (UID 33) is the web server account, `systemd-network` runs `systemd-networkd`, `sshd` is used for SSH privilege separation, and `nobody` (UID 65534) is the least-privileged account. All use `nologin` or similar shells.

4. Offboard Bob with `userdel`. First see what files he owns, then remove the account and home.

```
sudo find / -user bob -not -path '/proc/*' 2>/dev/null | head
sudo userdel -r bob
getent passwd bob
ls -ld /home/bob
getent group bob
```

```
/home/bob
/home/bob/.bashrc
...
userdel: bob mail spool (/var/mail/bob) not found
ls: cannot access '/home/bob': No such file or directory
```

What just happened: Finding a user's files *before* deleting them is good practice, because files outside the home directory are left behind, showing only a number as the owner. `userdel -r` removes the account from `/etc/passwd`, `/etc/shadow`, and `/etc/group` memberships, deletes the home directory and mail spool (the warning just says Bob had no mail), and removes his private group `bob`. Plain `userdel bob` (without `-r`) keeps the home directory, which is useful when a manager needs to review the files first.

5. Offboard Carol with Ubuntu's `deluser`.

```
sudo deluser carol ops
sudo deluser --remove-home carol
getent passwd carol
```

```
info: Removing user `carol' from group `ops' ...
info: Looking for files to backup/remove ...
info: Removing files ...
info: Removing crontab ...
info: Removing user `carol' ...
```

What just happened: With two arguments, `deluser USER GROUP` only removes the user from that group. `deluser --remove-home USER` deletes the account and home directory (`--remove-all-files` would also delete every file she owns anywhere, and `--backup` saves a tarball first). On Rocky there is no `deluser`; use `userdel -r`.

6. Try to delete groups, including one that is still someone's primary group.

```
sudo groupdel alice
sudo groupdel ops
getent group ops
```

```
groupdel: cannot remove the primary group of user 'alice'
```

What just happened: `groupdel` refuses to remove a group that is the primary group of an existing user, because that user would be left with an invalid GID. Remove or modify the user first. Deleting `ops` worked because nobody uses it as a primary group; `getent` now prints nothing.

Checkpoint: `getent passwd bob carol` prints nothing, `ls /home` shows only `student` and `alice`, and `getent passwd reportd` shows a UID below 1000 with `/usr/sbin/nologin`.

Task 9: Compare the defaults on rocky-lab

1. Log in to `rocky-lab` and inspect the defaults.

```
# on rocky-lab
useradd -D
grep -E '^(UID_MIN|UID_MAX|SYS_UID_MIN|SYS_UID_MAX)' /etc/login.defs
grep -E '^(CREATE_HOME|ENCRYPT_METHOD|PASS_MAX_DAYS)' /etc/login.defs
ls -l /usr/sbin/adduser
```

```
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
CREATE_MAIL_SPOOL=yes
UID_MIN                  1000
UID_MAX                 60000
SYS_UID_MIN               201
SYS_UID_MAX               999
PASS_MAX_DAYS   99999
CREATE_HOME yes
ENCRYPT_METHOD SHA512
lrwxrwxrwx. 1 root root 7 Nov  5  2024 /usr/sbin/adduser -> useradd
```

What just happened: On Rocky the default shell is `/bin/bash`, `CREATE_HOME yes` means `useradd` creates the home directory even without `-m`, and passwords are hashed with SHA-512 (`$6$`). System UIDs for `useradd -r` come from 201 to 999. `adduser` is only a symbolic link to `useradd`.

2. Create a test admin user and give it sudo rights through the `wheel` group.

```
# on rocky-lab
sudo useradd -c "Dave Test" dave
ls -ld /home/dave
echo 'dave:Welcome2Lab!' | sudo chpasswd
sudo grep '^dave:' /etc/shadow | cut -c1-8
sudo usermod -aG wheel dave
id dave
sudo grep -E '^%wheel' /etc/sudoers
```

```
drwx------. 3 dave dave 78 Oct 10 11:05 /home/dave
dave:$6$
uid=1001(dave) gid=1001(dave) groups=1001(dave),10(wheel)
%wheel  ALL=(ALL)   ALL
```

What just happened: The home directory exists even though you did not use `-m`, and Rocky's homes are `0700`. The hash starts with `$6$` (SHA-512). The `wheel` group (GID 10) is Rocky's admin group, allowed to use `sudo` by the `%wheel` line in `/etc/sudoers`; on Ubuntu the same role belongs to the `sudo` group. The dot at the end of the permissions means the file has an SELinux label.

3. Remove the test user from rocky-lab.

```
# on rocky-lab
sudo userdel -r dave
getent passwd dave
```

What just happened: `userdel -r` works the same on both distributions. Rocky creates a mail spool for each user (`CREATE_MAIL_SPOOL=yes`), so this time `userdel -r` silently removes `/var/mail/dave` as well instead of warning that none exists. `getent` prints nothing because the account is gone.

Checkpoint: You can state three differences between Ubuntu and Rocky account defaults (default shell, home created by default, hash algorithm, admin group). `getent passwd dave` on rocky-lab prints nothing.

Challenge

CHALLENGE

1. Write a one-line `awk` command that prints every account whose shell is *not* `nologin` or `false`. Which ones are they, and should each of them be able to log in?
2. Create a user `tempuser` that expires automatically in 7 days, must change their password at first login, and must change it every 60 days after that. Verify with `chage -l`, then delete the user.
3. Set the system default so new users get `/bin/bash` without `-s`, prove it works with a new user, then restore `SHELL=/bin/sh` and delete the user.

Clean up

1. Make sure no stopped `passwd` job remains.

   ```
   jobs
   ```

   If a job is listed, run `fg` and press `Ctrl+C`.

2. Remove the remaining lab users and service account, with their home directories.

   ```
   sudo userdel -r alice
   sudo userdel -r reportd
   ```

   `userdel` may warn that no mail spool was found; that is fine.

3. Remove `student` from `developers`, then delete the lab groups.

   ```
   sudo gpasswd -d student developers
   sudo groupdel developers
   sudo groupdel project-x
   getent group developers project-x ops
   ```

The last command should print nothing.

4. Remove the template changes.

```
sudo rm -f /etc/skel/WELCOME.txt /etc/profile.d/lab-motd.sh
sudo rmdir /etc/skel/bin
ls -a /etc/skel
```

5. Remove the lab directory and confirm only the original users remain.

```
rm -rf ~/labs/2.2
awk -F: '$3 >= 1000 && $3 < 60000 {print $1}' /etc/passwd
ls /home
```

Both lists should show only `student`. You can leave the `whois` package installed. On rocky-lab, `dave` was already removed in Task 9.

Exam corner

- `/etc/passwd`: `name:x:UID:GID:GECOS:home:shell` (world-readable). `/etc/shadow`: `name:hash:lastchg:min:max:warn:inactive:expire:reserved` (root/shadow only). `/etc/group`: `name:x:GID:members`.
- Hash prefixes: `$1$` MD5, `$5$` SHA-256, `$6$` SHA-512 (Rocky 9), `$y$` yescrypt (Ubuntu 24.04). A leading `!` means locked; `*` or `!` alone means no password.
- UID 0 = root; 1 to 999 = system and service accounts (`useradd -r`); 1000 and up = regular users (`UID_MIN` / `UID_MAX` in `/etc/login.defs`); 65534 = `nobody`.
- `useradd -m -s /bin/bash -c "comment" -G grp1,grp2 user`. Ubuntu's plain `useradd` creates no home and uses `/bin/sh`; Rocky creates the home and uses `/bin/bash`. `adduser` is interactive on Ubuntu and a link to `useradd` on Rocky.
- `usermod -aG group user` appends; `-G` alone replaces the supplementary list. `usermod -l` renames, `-s` shell, `-d -m` move home, `-L` / `-U` lock/unlock, `-e` expire date.
- `passwd user` sets a password; `-l` / `-u` lock/unlock; `-S` status; `-e` force change. `chage -l` lists; `-M` max, `-m` min, `-W` warn, `-I` inactive, `-E` account expiry (`0` = expired now, `-1` = never), `-d 0` = force change at next login.
- Defaults: `/etc/login.defs` (UID ranges, `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`, `ENCRYPT_METHOD`, `CREATE_HOME`) and `/etc/default/useradd` (shell, skel, `INACTIVE`, `EXPIRE`; view with `useradd -D`).
- `/etc/skel` is copied into new home directories only at creation. `/etc/profile` and `/etc/profile.d/*.sh` run for every login shell.
- `userdel -r` deletes the user, home, and mail spool; `deluser --remove-home` on Ubuntu. `groupdel` fails on a user's primary group. `groupmod -n` renames, `-g` changes GID.
- Listing: `whoami` (effective user), `id`, `groups`, `getent passwd`, `who` and `w` (current logins, from utmp), `last` (history, wtmp), `lastb` (failed logins), `lastlog` (last login per account).

- EUID/EGID decide permissions; they differ from the real UID/GID under `sudo` or setuid/setgid programs such as `passwd` (setuid root) and `chage` (setgid shadow).
- Service accounts: system UID, `nologin` shell, no password, home in `/var/lib` or none. `chsh` changes a shell; valid shells are in `/etc/shells`.

Review questions

1. A developer was added with `sudo usermod -G docker dev1` and now cannot use the `developers` shared folder. What went wrong, and what command should have been used?

2. Which field of `/etc/shadow` controls when the entire account stops working, regardless of the password?
   A. Field 3 (last change)
   B. Field 5 (maximum age)
   C. Field 7 (inactive)
   D. Field 8 (expire)

3. A new hire must set their own password the first time they log in. Which command enforces this?
   A. `chage -E 0 newhire`
   B. `chage -d 0 newhire`
   C. `passwd -l newhire`
   D. `usermod -L newhire`

4. On Ubuntu 24.04, `sudo useradd jdoe` succeeded, but `jdoe` has no home directory. Why, and how do you avoid it?

5. Which UID range would you expect for an account created with `useradd -r -s /usr/sbin/nologin metricsd`?
   A. 0
   B. 1 to 999
   C. 1000 to 60000
   D. 65534

6. While `passwd` is running for user `student`, `ps -o ruid,euid -C passwd` shows `1000 0`. Why?
   A. The user is a member of the `sudo` group.
   B. `passwd` is a setuid-root program, so its effective UID is root while its real UID is the caller's.
   C. `passwd` always runs through `sudo`.
   D. The kernel gives every process EUID 0 while it reads a password.

Lab 2.3 Manage Processes and Jobs

Objective: 2.3 Given a scenario, manage processes and jobs in a Linux environment.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 120 minutes
VM: ubuntu-lab
Snapshot first: Not required. The lab creates a temporary user, a limits file, and scheduled jobs, and the clean-up removes them all.

Every program you run becomes a process, and much of a Linux administrator's day is spent watching, prioritizing, pausing, and stopping them. In this lab you will start deliberately misbehaving processes, inspect them with every tool on the exam, create each process state on purpose (including a zombie), change priorities, set resource limits, drive job control from the shell, send signals, and schedule work with `cron`, `at`, and `anacron`.

What you'll practice

- Process verification: `/proc/<PID>`, `atop`, `htop`, `lsof`, `mpstat`, `pidstat`, `ps`, `pstree`, `strace`, `top`
- Process ID: PPID, PID
- States: running, blocked, sleeping, stopped, zombie
- Priority: `nice`, `renice`
- Process limits: `ulimit`, `/etc/security/limits.conf`
- Jobs: `&`, `bg`, `Ctrl+C`, `Ctrl+D`, `Ctrl+Z`, `exec`, `fg`, `jobs`, `kill`, `killall`, `nohup`, `pkill`
- Signals: 1 HUP, 9 KILL, 15 TERM
- Scheduling: `anacron`, `at`, `crontab`

How it works

Programs and processes. A *program* is a file on disk, like `/usr/bin/sleep`. A *process* is a running copy of a program: it has memory, open files, an owner, and a unique number called the **PID** (process ID). Run `sleep` three times at once and you get three processes with three PIDs.

Parents and children. New processes are created by existing ones. A process makes a copy of itself (*fork*), and the copy usually replaces its program with a new one (*exec*). The creator is the *parent*, and its PID is the child's **PPID** (parent process ID). When you type `ls` in Bash, Bash forks, the child execs `ls`, and Bash waits. Everything descends from PID 1, which is `systemd` on Ubuntu and Rocky, so the processes form a family tree.

```
systemd (PID 1)
  +-- sshd (listener)
  |     +-- sshd (your session)
  |           +-- bash (PID 1532, PPID = sshd)
  |                 +-- yes   (PID 1601, PPID 1532)
  |                 +-- sleep (PID 1602, PPID 1532)
  +-- cron
  +-- systemd-journald
```

When a child exits, it leaves behind a tiny record holding its exit status until the parent collects it with the `wait()` system call. A child that has exited but has not been collected yet is a **zombie**. If the parent dies first, the orphaned children are adopted by PID 1 (or a nearby "subreaper"), which collects them.

Process states. At any instant a process is in one of a few states, shown in the `STAT` column of `ps`:

Code	State	Meaning
`R`	Running	Running on a CPU, or ready and waiting for a turn
`S`	Sleeping	Waiting for something (a timer, keyboard input, network data); can be woken by a signal
`D`	Blocked (uninterruptible sleep)	Waiting inside the kernel, usually for disk or network storage I/O; ignores signals until the I/O finishes
`T`	Stopped	Paused by `Ctrl+Z` or a STOP signal; resumes with CONT
`Z`	Zombie	Finished, waiting for its parent to read the exit status
`I`	Idle	An idle kernel thread (you will see many)

Extra characters may follow: `<` high priority, `N` low priority (niced), `s` session leader, `l` multi-threaded, `+` in the foreground of a terminal.

Signals. A *signal* is a small numbered message the kernel delivers to a process. A process can *catch* most signals and run its own handler (for example, reload its config), *ignore* them, or let the default action happen (usually exit). The exam's key three:

Number	Name	Default action and typical use
1	`SIGHUP` (hang up)	Sent when a terminal closes. Many daemons treat it as "reload your configuration".
9	`SIGKILL`	Kills immediately. Cannot be caught, blocked, or ignored. No clean-up happens. Last resort.
15	`SIGTERM`	Polite request to exit. The default for `kill`. Lets a program save work and clean up.

Others you will meet: 2 `SIGINT` (`Ctrl+C`), 19 `SIGSTOP` (pause, cannot be caught), 20 `SIGTSTP` (`Ctrl+Z`, can be caught), 18 `SIGCONT` (resume).

Priority. The scheduler shares CPUs between runnable processes. You influence it with the *nice value*, from `-20` (highest priority, "not nice to others") to `19` (lowest, "very nice"). The default is `0`. Any user can make their own processes *nicer* (raise the number), but only root can lower it below where it is.

Jobs. Your shell keeps its own list of the commands you started from it, called *jobs*, numbered `%1`, `%2`, and so on. One job at a time runs in the *foreground* (it owns the keyboard); others can run in the *background* or sit stopped. Job control lets you move them around.

Limits. The kernel enforces per-process *resource limits* (rlimits), such as the maximum number of open files or processes per user. Each limit has a *soft* value (currently enforced, and the user may raise it) and a *hard* value (the ceiling; only root can raise it). You view and change them in a shell with `ulimit`, and set them for logins in `/etc/security/limits.conf`.

Scheduling. `cron` runs commands repeatedly on a schedule (every night at 2:00). `at` runs a command once at a future time. `anacron` runs daily, weekly, or monthly jobs even on machines that are not always on: if the machine was off when a job was due, `anacron` runs it soon after the next boot.

Tasks

Task 1: Start a workload and view it with `ps` and `pstree`

1. Make a lab directory and install the tools for this lab.

```
mkdir -p ~/labs/2.3 && cd ~/labs/2.3
sudo apt update
sudo apt install -y procps psmisc htop atop sysstat strace lsof at anacron python3
```

> **What just happened:** `procps` provides `ps`, `top`, `pgrep`, `pkill`, and `uptime`. `psmisc` provides `pstree` and `killall`. `sysstat` provides `mpstat` and `pidstat`. `at` and `anacron` are schedulers. Several of these are already installed on Ubuntu Server; `apt` just reports that.

2. Start two CPU hogs and a sleeper in the background.

```
yes > /dev/null &
yes > /dev/null &
sleep 3000 &
echo "My shell's PID is $$"
```

```
[1] 1601
[2] 1602
[3] 1603
My shell's PID is 1532
```

What just happened: `yes` prints `y` forever as fast as it can; sending its output to `/dev/null` turns it into a pure CPU load. The `&` at the end runs a command in the *background*, so you get your prompt back immediately. Bash prints the job number in brackets and the PID. `$$` holds the PID of your current shell. Your numbers will differ; note them.

3. List processes with the two classic `ps` styles.

```
ps
ps aux | head -5
ps aux | grep -E 'yes|sleep' | grep -v grep
ps -ef | grep -E 'yes|sleep' | grep -v grep
```

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.3  22312 13148 ?        Ss   09:01   0:02 /sbin/init
...
student     1601 99.6  0.0   5704  1920 pts/0    R    11:02   1:05 yes
student     1602 99.4  0.0   5704  1920 pts/0    R    11:02   1:05 yes
student     1603  0.0  0.0   5684  1920 pts/0    S    11:02   0:00 sleep 3000
UID          PID    PPID  C STIME TTY          TIME CMD
student     1601    1532 99 11:02 pts/0    00:01:05 yes
student     1602    1532 99 11:02 pts/0    00:01:05 yes
student     1603    1532  0 11:02 pts/0    00:00:00 sleep 3000
```

What just happened: Plain `ps` shows only processes in your current terminal. `ps aux` (BSD style, no dash) shows **a**ll users' processes, in **u**ser-oriented format, including those with no terminal (**x**). Columns: `%CPU` and `%MEM` usage, `VSZ` virtual memory and `RSS` resident (real) memory in KiB, `TTY` terminal (`?` means none, typical for daemons), `STAT` state, `START` start time, `TIME` total CPU time used. `ps -ef` (UNIX style) shows **e**very process in **f**ull format, including the **PPID** column: all three processes have PPID `1532`, your shell. `grep -v grep` removes the `grep` command itself from the results.

4. Choose exactly the columns you want and sort them.

```
ps -eo pid,ppid,user,stat,ni,%cpu,%mem,cmd --sort=-%cpu | head -6
ps -o pid,ppid,stat,ni,cmd -p $$
```

What just happened: `-e` selects every process and `-o` lists output columns by name. `ni` is the nice value. `--sort=-%cpu` sorts by CPU, highest first (a leading `-` means descending). This is the fastest way to answer "what is eating my CPU?" in a script. `-p` selects processes by PID.

5. Find PIDs by name and view the family tree.

```
pgrep yes
pgrep -a sleep
pidof yes
pstree -p $$
pstree -p | head -20
```

```
1601
1602
1603 sleep 3000
1602 1601
bash(1532)-+-pstree(1690)
           |-sleep(1603)
           |-yes(1601)
           `-yes(1602)
```

What just happened: `pgrep NAME` prints the PIDs of matching processes; `-a` also prints the full command line. `pidof` does the same for an exact program name. `pstree` draws the parent-child tree; `-p` adds PIDs, and giving a PID starts the tree there. The full `pstree -p` starts at `systemd(1)` and shows how your shell hangs below `sshd`. Identical children are collapsed like `3*[agetty]` without `-p`.

Checkpoint: `ps -o pid,ppid,stat,cmd -C yes` shows two `yes` processes in state `R` whose PPID equals the output of `echo $$`.

Task 2: Watch the system with `top`, `htop`, `atop`, `mpstat`, and `pidstat`

1. Run `top` and learn its keys.

```
top
```

```
top - 11:05:31 up  2:04,  1 user,  load average: 1.98, 1.20, 0.55
Tasks: 112 total,   3 running, 109 sleeping,   0 stopped,   0 zombie
%Cpu(s): 99.8 us,  0.2 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3916.0 total,   2950.2 free,    410.5 used,    800.1 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   3505.5 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1601 student   20   0    5704   1920   1792 R 100.0   0.0   3:12.40 yes
   1602 student   20   0    5704   1920   1792 R  99.7   0.0   3:12.38 yes
```

While `top` is running, try these keys one at a time:

Key	Action
`1`	Toggle one line per CPU
`P` / `M`	Sort by CPU / by memory
`u`	Show one user's processes (type `student`, Enter)
`c`	Toggle full command lines
`r`	Renice a process (asks for PID and value)
`k`	Kill a process (asks for PID and signal; press Esc to cancel)
`h`	Help
`q`	Quit

What just happened: The header shows uptime and **load average** (the average number of runnable or blocked processes over 1, 5, and 15 minutes; with 2 CPUs and 2 hogs it climbs toward 2). The `Tasks` line counts processes in each state, including zombies. In the CPU line, `us` is user programs, `sy` kernel, `ni` niced processes, `id` idle, and `wa` time waiting for I/O. In the table, `PR` is the kernel priority (20 plus the nice value), `NI` the nice value, `RES` real memory, and `S` the state. `top -b -n 1` prints one snapshot in *batch* mode, useful in scripts.

2. Run `htop`, a friendlier version with color bars and mouse support.

```
htop
```

Try `F5` (tree view), `F6` (choose sort column), `F3` (search for `sleep`), `F4` (filter), `F7` / `F8` (lower or raise the nice value of the selected process), `F9` (send a signal; press Esc to cancel), and `F10` or `q` to quit.

What just happened: `htop` shows per-CPU usage bars, memory and swap bars, and a scrollable process list. You can select a process with the arrow keys and act on it with the function keys. If your SSH client steals the F-keys, use the letters shown in `htop`'s help (`h`): for example `t` for tree, `/` to search, `k` to kill.

3. Run `atop`, which also records history.

```
atop 2
```

Press `m` (memory view), `d` (disk view), `g` (generic view), and `q` to quit.

What just happened: `atop 2` refreshes every 2 seconds. It shows system-wide CPU, memory, disk, and network resources at the top and highlights resources under pressure in color. Unlike `top`, the `atop` package installs a service that writes snapshots to `/var/log/atop/`, so you can replay what happened earlier with `atop -r` (for example, to investigate a slowdown that happened at 3 a.m.).

4. Use `mpstat` for per-CPU statistics.

```
mpstat
mpstat -P ALL 1 3
```

```
11:08:01     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %idle
11:08:02     all   99.50    0.00    0.50    0.00    0.00    0.00    0.00   0.00
11:08:02       0   99.00    0.00    1.00    0.00    0.00    0.00    0.00   0.00
11:08:02       1  100.00    0.00    0.00    0.00    0.00    0.00    0.00   0.00
```

What just happened: `mpstat` (multiprocessor statistics) with no arguments prints averages since boot. `-P ALL` shows each CPU separately, and `1 3` means "every 1 second, 3 times", followed by an average. This spots problems like one CPU pinned at 100% while the others idle (a single-threaded bottleneck), or high `%iowait` (slow storage).

5. Use `pidstat` for per-process statistics over time.

```
pidstat 1 3
pidstat -p $(pgrep -d, yes) 1 3
pidstat -r -p $(pgrep -d, sleep) 1 2
```

What just happened: `pidstat 1 3` reports CPU use of active processes every second, three times. `-p` limits it to given PIDs; `pgrep -d,` prints matching PIDs separated by commas, the format `pidstat` wants. `-r` reports memory (page faults, `RSS`) instead of CPU, and `-d` would report disk I/O. Unlike `top`, `pidstat` gives you timestamped lines you can save to a file and compare.

Checkpoint: `top -b -n 1 | head -12` lists the two `yes` processes near the top with about 100% CPU each, and `mpstat 1 1` shows `%idle` near 0.

Task 3: Look inside a process with `/proc`, `lsof`, and `strace`

1. Every process has a directory in `/proc` named after its PID. Explore your `sleep` process.

```
SPID=$(pgrep -n sleep)
echo $SPID
ls /proc/$SPID
grep -E '^(Name|State|Pid|PPid|Uid|Gid|VmRSS|Threads)' /proc/$SPID/status
```

```
1603
Name:   sleep
State:  S (sleeping)
Pid:    1603
PPid:   1532
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
VmRSS:      1920 kB
Threads:    1
```

What just happened: `pgrep -n` returns the newest matching PID, saved in the shell variable `SPID`. `/proc` is a virtual filesystem: nothing is stored on disk, and the kernel generates the contents when you read them. `status` is a readable summary. The four `Uid` numbers are the real, effective, saved, and filesystem UIDs. `ps`, `top`, and friends get all their information from here.

2. Read the command line, open files, environment, and working directory.

```
tr '\0' ' ' < /proc/$SPID/cmdline; echo
ls -l /proc/$SPID/fd
ls -l /proc/$SPID/cwd /proc/$SPID/exe
tr '\0' '\n' < /proc/$SPID/environ | head -5
cat /proc/1/environ
```

```
sleep 3000
total 0
lrwx------ 1 student student 64 Oct 10 11:12 0 -> /dev/pts/0
lrwx------ 1 student student 64 Oct 10 11:12 1 -> /dev/pts/0
lrwx------ 1 student student 64 Oct 10 11:12 2 -> /dev/pts/0
lrwxrwxrwx 1 student student 0 Oct 10 11:12 /proc/1603/cwd -> /home/student/labs/2.3
lrwxrwxrwx 1 student student 0 Oct 10 11:12 /proc/1603/exe -> /usr/bin/sleep
SHELL=/bin/bash
PWD=/home/student/labs/2.3
LOGNAME=student
...
cat: /proc/1/environ: Permission denied
```

What just happened: `cmdline` holds the arguments separated by NUL (zero) bytes, so `tr '\0' ' '` turns them into spaces. `fd/` contains one symbolic link per open file descriptor: 0, 1, and 2 point at your terminal. `cwd` is the current directory and `exe` the program file. `environ` holds the environment variables the process started with. You can read these only for your own processes; PID 1 belongs to root, so it is denied.

3. Compare with `lsof`, which reads the same data and formats it.

```
lsof -p $SPID
```

What just happened: `lsof -p PID` lists everything the process has open: its working directory (`cwd`), program (`txt`), shared libraries (`mem`), and file descriptors. It is the same information as `/proc/PID/fd` and `/proc/PID/maps`, in one table.

4. Trace system calls with `strace`. A *system call* is a request from a program to the kernel (open a file, read, write, sleep).

```
strace -e trace=openat,read,write cat /etc/hostname
strace -c ls / > /dev/null
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0"..., 832) = 832
...
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
read(3, "ubuntu-lab\n", 131072)         = 11
write(1, "ubuntu-lab\n", 11ubuntu-lab
)            = 11
+++ exited with 0 +++
```

What just happened: `strace` runs a command and prints each system call it makes, with arguments and return values, to standard error. `-e trace=openat,read,write` limits the output to those calls. You can see `cat` load its libraries, open `/etc/hostname` (getting file descriptor 3), read 11 bytes, and write them to descriptor 1 (your screen). `-c` instead prints a summary table counting each call. When a program fails with an unhelpful error, `strace` often shows the exact file it could not open (`= -1 ENOENT (No such file or directory)`).

5. Attach to a process that is already running.

```
strace -p $SPID
sudo strace -p $SPID
```

Press `Ctrl+C` to detach after a few seconds.

```
strace: attach: ptrace(PTRACE_SEIZE, 1603): Operation not permitted
strace: Process 1603 attached
restart_syscall(<... resuming interrupted clock_nanosleep ...>
```

What just happened: `-p PID` attaches to a running process. Ubuntu enables a security setting (`kernel.yama.ptrace_scope = 1`) that lets normal users trace only their own *child* processes, so the first attempt fails and `sudo` is required. The `sleep` process is sitting inside a single sleeping system call, which explains its `S` state. `Ctrl+C` detaches `strace` without killing the process.

Checkpoint: `cat /proc/$SPID/status | grep PPid` shows your shell's PID, and you have seen `openat(... "/etc/hostname" ...)` in `strace` output.

Task 4: Create every process state on purpose

1. **Running (`R`) and sleeping (`S`):** you already have both.

```
ps -o pid,stat,cmd -C yes,sleep
```

```
    PID STAT CMD
   1601 R    yes
   1602 R    yes
   1603 S    sleep 3000
```

2. **Stopped (`T`):** pause one `yes` with a signal, and watch CPU use drop.

```
kill -STOP 1601
ps -o pid,stat,%cpu,cmd -p 1601
jobs
kill -CONT 1601
ps -o pid,stat,cmd -p 1601
```

```
    PID STAT %CPU CMD
   1601 T    82.1 yes
[1]+  Stopped                 yes > /dev/null
[2]   Running                 yes > /dev/null &
[3]-  Running                 sleep 3000 &
    PID STAT CMD
   1601 R    yes
```

> **What just happened:** Replace `1601` with your first `yes` PID. `kill -STOP` sends `SIGSTOP`, which pauses a process instantly and cannot be ignored. Its state becomes `T` and it uses no more CPU (the `%CPU` column is an average over its lifetime, so it falls slowly; `top` shows 0 immediately). `jobs` shows it as `Stopped`. `kill -CONT` resumes it. `Ctrl+Z` does the same thing to a foreground job, using `SIGTSTP`, as you will see in Task 7.

3. **Blocked (`D`):** a process waiting on disk I/O inside the kernel. It is usually too brief to see, but heavy direct reads from a disk often show it. Confirm your extra disk name with `lsblk` and substitute it (this only *reads* the disk, which is safe).

```
lsblk -d -o NAME,SIZE,TYPE
sudo -v
sudo dd if=/dev/sdb of=/dev/null bs=1M count=4000 iflag=direct status=none &
for i in 1 2 3 4 5; do ps -o pid,stat,cmd -C dd | tail -1; sleep 0.5; done
```

```
   1720 D    dd if=/dev/sdb of=/dev/null bs=1M count=4000 iflag=direct status=none
   1720 D    dd if=/dev/sdb of=/dev/null bs=1M count=4000 iflag=direct status=none
   1720 R    dd if=/dev/sdb of=/dev/null bs=1M count=4000 iflag=direct status=none
```

> **What just happened:** `sudo -v` refreshes your cached `sudo` password first, so the background `sudo` does not get stuck waiting for a password it cannot read. `dd` reads the disk with `iflag=direct`, bypassing the cache, so it spends most of its time waiting for the (virtual) disk. While the kernel waits for the I/O, the process is in **uninterruptible sleep** (`D`, "blocked"): even `kill -9` waits until the I/O returns. A handful of `D` processes is normal; many `D` processes that never go away point to failing disks or a hung NFS server, and they raise the load average even though the CPU is idle. On a fast virtual disk you may see only `R` or `S`, and the `dd` may finish in seconds. That is fine; the point is what `D` means.

4. **Zombie (`Z`):** write a tiny Python program whose parent never collects its child. Create `~/labs/2.3/zombie.py` with this content (the heredoc writes it for you):

```
cat > ~/labs/2.3/zombie.py <<'EOF'
import os, time

pid = os.fork()                 # make a copy of this process
if pid == 0:                    # in the child
    print(f"child  {os.getpid()}: exiting now")
    os._exit(0)                 # child ends immediately
print(f"parent {os.getpid()}: sleeping, never calling wait()")
time.sleep(600)                 # parent ignores its dead child
EOF
python3 ~/labs/2.3/zombie.py &
sleep 1
ps -o pid,ppid,stat,cmd --ppid $(pgrep -f zombie.py)
```

```
parent 1750: sleeping, never calling wait()
child  1751: exiting now
    PID    PPID STAT CMD
   1751    1750 Z    [python3] <defunct>
```

What just happened: A *heredoc* (`<<'EOF' ... EOF`) feeds the lines up to `EOF` into `cat`, which writes them to the file; the quotes around `'EOF'` stop the shell from changing anything inside. `os.fork()` creates a child. The child exits at once, but the parent never calls `os.wait()`, so the kernel keeps the child's exit status: a **zombie** (`Z`, shown as `<defunct>`). `--ppid` lists children of a given PID, and `pgrep -f` matches against the full command line. A zombie uses no CPU or memory, only a process-table slot and its PID.

5. Try to kill the zombie, then fix it the right way.

```
kill -9 1751
ps -o pid,stat,cmd -p 1751
kill $(pgrep -f zombie.py)
sleep 1
ps -o pid,stat,cmd -p 1751
```

What just happened: `kill -9` on a zombie does nothing; it is already dead. The fix is to deal with the **parent**: here you sent the parent `SIGTERM`. When the parent died, the zombie was adopted by PID 1 (`systemd`), which immediately collected it, so the last `ps` prints only the header. In production, zombies point to a bug in the parent program; restart or fix the parent. `top`'s `Tasks:` line and `ps aux | awk '$8 ~ /Z/'` are quick ways to find zombies.

Checkpoint: You saw `R`, `S`, `T`, and `Z` in the `STAT` column (and possibly `D`). `ps aux | awk '$8 ~ /Z/'` now prints nothing, and `pgrep dd` prints nothing once the `dd` finishes.

Task 5: Change priority with `nice` and `renice`

1. Start a low-priority hog with `nice` and compare.

```
nice -n 15 yes > /dev/null &
ps -o pid,ni,stat,%cpu,cmd -C yes
```

```
    PID  NI STAT %CPU CMD
   1601   0 R    93.0 yes
   1602   0 R    96.4 yes
   1780  15 RN   12.1 yes
```

What just happened: `nice -n 15 COMMAND` starts a command with nice value 15 (plain `nice COMMAND` uses 10). With three CPU hogs and two CPUs, the scheduler gives the niced one far less CPU time. The `N` in `RN` marks a low-priority process. Watch for a few seconds in `top` to see the difference grow; `%CPU` in `ps` is a lifetime average.

2. Change priorities of running processes with `renice`.

```
renice -n 19 -p 1780
renice -n 5 -p 1602
renice -n 0 -p 1602
```

```
1780 (process ID) old priority 15, new priority 19
1602 (process ID) old priority 0, new priority 5
renice: failed to set priority for 1602 (process ID): Permission denied
```

What just happened: `renice -n VALUE -p PID` sets a new nice value (use your PIDs). As a normal user you can make your processes nicer (higher numbers), but you cannot make them less nice again, so going back from 5 to 0 is denied. This prevents users from raising their own priority above others.

3. Use root to raise priority, and renice everything a user owns.

```
sudo renice -n -5 -p 1602
sudo renice -n 10 -u student
ps -o pid,ni,cmd -u student | head
sudo renice -n 0 -u student
```

What just happened: Root may set any value from `-20` to `19`, including negative ones. `-u USER` renices every process of that user (and `-g` a process group). You then reset all of `student`'s processes to 0. `sudo nice -n -10 COMMAND` would start a command with high priority.

4. Stop the niced hog.

```
kill 1780
```

What just happened: Plain `kill PID` sent `SIGTERM` to the niced `yes` (use your PID). Nice values only matter when processes compete for CPU: on an idle system a nice-19 process still gets a whole CPU, because nobody else wants it.

Checkpoint: `ps -o pid,ni,cmd -C yes` shows the two original `yes` processes with `NI` 0.

Task 6: Process limits with `ulimit` and `/etc/security/limits.conf`

1. View your current limits.

```
ulimit -a
ulimit -n
ulimit -Sn
ulimit -Hn
ulimit -u
```

```
real-time non-blocking time  (microseconds, -R) unlimited
core file size              (blocks, -c) 0
...
file size                   (blocks, -f) unlimited
...
open files                          (-n) 1024
...
max user processes                  (-u) 15319
...
1024
1024
524288
15319
```

What just happened: `ulimit` is a shell built-in. `-a` shows all limits. `-n` is the maximum number of open files per process, `-u` the maximum number of processes for your user, `-f` the largest file the shell's processes may create, and `-c` the core dump size. `-S` shows the **soft** limit (the one enforced right now) and `-H` the **hard** limit (the ceiling). Exact numbers depend on memory and system defaults.

2. Hit a limit on purpose inside a subshell, so your main shell is not affected.

```
(ulimit -n 20; python3 -c "f = [open('/etc/hostname') for _ in range(50)]")
(ulimit -f 1000; dd if=/dev/zero of=big.bin bs=1M count=10)
ls -lh big.bin
```

```
Traceback (most recent call last):
  ...
OSError: [Errno 24] Too many open files: '/etc/hostname'
File size limit exceeded
-rw-rw-r-- 1 student student 1000K Oct 10 11:30 big.bin
```

What just happened: The parentheses run a *subshell*, so the lowered limit disappears when it ends. In the first line Python tried to open 50 files but the limit was 20, so the kernel returned error 24, "Too many open files", a very common error on busy web and database servers. In the second, `-f 1000` limits file size to 1000 blocks of 1 KiB; when `dd` tried to write past that, the kernel sent it `SIGXFSZ` and stopped it at 1000 KiB. (Some systems add "(core dumped)" to the message; that depends on the core dump settings.)

3. See the soft/hard rule in action.

```
(ulimit -Sn 2048; ulimit -Sn)
(ulimit -Hn 4096; ulimit -Hn; ulimit -Hn 8192)
```

```
2048
4096
bash: ulimit: open files: cannot modify limit: Operation not permitted
```

What just happened: A user may raise the soft limit up to the hard limit, and may *lower* the hard limit, but can never raise the hard limit again. Only root can raise a hard limit. That is why permanent limits are set by root in a configuration file that applies at login.

4. Look at the permanent limits file and the PAM module that applies it.

```
grep -v '^#' /etc/security/limits.conf | grep -v '^$'
tail -n 14 /etc/security/limits.conf
grep -l '^session.*pam_limits' /etc/pam.d/*
```

```
#<domain>      <type>  <item>         <value>
#

#*               soft    core            0
#root            hard    core            100000
#*               hard    rss             10000
#@student        hard    nproc           20
#@faculty        soft    nproc           20
#@faculty        hard    nproc           50
#ftp             hard    nproc           0
#ftp             -       chroot          /ftp
#@student        -       maxlogins       4

# End of file
/etc/pam.d/cron
/etc/pam.d/login
/etc/pam.d/runuser
/etc/pam.d/sshd
/etc/pam.d/su
/etc/pam.d/sudo
/etc/pam.d/sudo-i
```

What just happened: The file is all comments by default, so the first command prints nothing; `tail` shows the commented examples at the end of the file. Each rule has four fields: **domain** (a user name, `@group`, or `*` for everyone), **type** (`soft`, `hard`, or `-` for both), **item** (`nofile`, `nproc`, `core`, `fsize`, `priority`, `maxlogins`, and more), and **value**. The rules are applied by the PAM module `pam_limits` when a session starts, so they only affect **new logins** through the listed services, never processes that are already running. Files in `/etc/security/limits.d/*.conf` are read too and use the same format.

5. Create a test user and give it custom limits. Use a drop-in file so clean-up is easy.

```
sudo useradd -m -s /bin/bash limituser
echo 'limituser:Welcome2Lab!' | sudo chpasswd
cat <<'EOF' | sudo tee /etc/security/limits.d/90-lab23.conf
# Lab 2.3 test limits
limituser   soft   nofile   2048
limituser   hard   nofile   4096
limituser   hard   nproc    50
EOF
```

What just happened: `useradd -m` created a user with a home directory, and `chpasswd` set its password. The file sets the soft open-files limit to 2048, the hard open-files limit to 4096, and a hard maximum of 50 processes, which is a classic defense against a *fork bomb* (a program that creates processes until the system falls over). `sudo tee` writes the file as root.

6. Test the limits with a **new login**. SSH to the VM itself as `limituser` (type `yes` if asked about the host key, then the password `Welcome2Lab!`).

```
ssh limituser@localhost 'ulimit -Sn; ulimit -Hn; ulimit -u'
sudo -iu limituser bash -c 'ulimit -Sn; ulimit -u'
ulimit -Sn
```

```
2048
4096
50
2048
50
1024
```

What just happened: The SSH login went through PAM, `pam_limits` read your file, and the new session got the new limits. `sudo -i` also starts a PAM session that loads `pam_limits` on Ubuntu, so it shows the same values. Your own shell still shows `1024` because limits are applied only when a session starts. If SSH password logins are disabled on your VM, the `sudo -iu` line is enough.

TIP

Services started by systemd do **not** read `limits.conf`, because they do not log in through PAM. For a service, set limits in its unit file, for example `LimitNOFILE=65535` in the `[Service]` section (see Lab 2.4).

Checkpoint: A new `limituser` session reports soft `nofile` 2048 and `nproc` 50, while your existing shell still reports 1024 for `ulimit -Sn`.

Task 7: Drive job control: `&`, `jobs`, `fg`, `bg`, `Ctrl+C`, `Ctrl+Z`, `Ctrl+D`, `nohup`, and `exec`

1. Clean the slate, then start fresh jobs.

```
kill %1 %2 %3 2>/dev/null
jobs
sleep 1000 &
sleep 2000 &
jobs -l
```

```
[1]-  1801 Running                 sleep 1000 &
[2]+  1802 Running                 sleep 2000 &
```

What just happened: `kill %1 %2 %3` sends `SIGTERM` to jobs 1 to 3 (the two `yes` and the old `sleep`); `%N` means "job N of this shell". `jobs` lists the shell's jobs; `-l` adds PIDs. The `+` marks the *current* job (the default for `fg` and `bg`), and `-` the previous one.

2. Bring a job to the foreground, stop it with `Ctrl+Z`, and resume it in the background.

```
fg %1
```

The terminal now waits on `sleep 1000`. Press `Ctrl+Z`:

```
sleep 1000
^Z
[1]+  Stopped                 sleep 1000
```

```
jobs
bg %1
jobs
```

What just happened: `fg %1` moved job 1 into the foreground, so it owns your keyboard. `Ctrl+Z` sends `SIGTSTP`, which stops the foreground job (state `T`) and returns your prompt. `bg %1` resumes the stopped job *in the background*, as if you had started it with `&`. This rescue move is very handy when you start a long command and forget the `&`.

3. Interrupt a foreground command with `Ctrl+C`.

```
sleep 500
```

Press `Ctrl+C`. The prompt returns at once, and `jobs` does not list it.

> **What just happened:** `Ctrl+C` sends `SIGINT` (signal 2) to the foreground job, which by default ends it. Compare: `Ctrl+C` ends a job, `Ctrl+Z` pauses it.

4. Signal end-of-input with `Ctrl+D`.

```
cat > notes.txt
```

Type `first line`, press Enter, type `second line`, press Enter, then press `Ctrl+D`.

```
cat notes.txt
bash
echo "inside a nested shell, PID $$"
```

Now press `Ctrl+D` on the empty prompt; you are back in your original shell.

> **What just happened:** `Ctrl+D` is not a signal. It tells the terminal "end of input" (EOF). `cat` was reading from the keyboard; at EOF it closed the file and exited. Bash also reads commands from the keyboard, so `Ctrl+D` on an empty line ends the shell, just like typing `exit`. That is also how you finish typing commands for `at` in Task 9.

5. Keep a job alive after the terminal goes away with `nohup`. You will simulate a dropped connection by sending `SIGHUP` to a nested shell.

```
bash
sleep 1111 &
nohup sleep 2222 &
kill -HUP $$
```

```
[1] 1830
[2] 1831
nohup: ignoring input and appending output to 'nohup.out'
Hangup
```

Back in your original shell, check which one survived:

```
pgrep -a sleep
```

```
1801 sleep 1000
1802 sleep 2000
1831 sleep 2222
```

> **What just happened:** When a terminal or SSH connection drops, the shell receives `SIGHUP`. An interactive Bash then resends `SIGHUP` to all its jobs and exits. `kill -HUP $$` simulated that for the nested shell. `sleep 1111` received `SIGHUP` and died (default action). `nohup` started `sleep 2222` with `SIGHUP` ignored, so it survived and was adopted by `systemd`. `nohup` also redirects output to `nohup.out` because the terminal will be gone. (Typing `exit` normally does not send HUP to background jobs, but closing the window or losing the network does.) Alternatives for long tasks: `tmux` or `screen`, or `systemd-run`.

6. Replace a shell with another program using `exec`.

```
bash
echo "nested shell PID: $$"
exec ps -o pid,ppid,cmd -p $$
```

```
nested shell PID: 1850
    PID    PPID CMD
   1850    1532 ps -o pid,ppid,cmd -p 1850
```

```
echo "back in shell $$"
```

What just happened: Normally Bash forks a child to run a command. `exec COMMAND` skips the fork: the shell process *becomes* the new program, keeping the same PID (`1850`). The output proves it: PID 1850 is now running `ps` , not `bash` . When `ps` finished, there was no shell left to return to, so you dropped back to your original shell (`1532`). Wrapper scripts end with `exec program` so the real program takes over the PID, and `exec` is also used for redirections like `exec > logfile` .

Checkpoint: `jobs` lists `sleep 1000` and `sleep 2000` as Running, `pgrep -a sleep` also shows `sleep 2222` (not a job of this shell any more), and `cat ~/labs/2.3/notes.txt` shows your two lines.

Task 8: Send signals with `kill` , `killall` , and `pkill`

1. List the signals.

```
kill -l
kill -l 15
kill -l HUP
```

```
 1) SIGHUP   2) SIGINT   3) SIGQUIT  4) SIGILL   5) SIGTRAP
 6) SIGABRT  7) SIGBUS   8) SIGFPE   9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
...
TERM
1
```

What just happened: `kill -l` lists signal numbers and names. Given a number it prints the name, and given a name it prints the number. You can refer to a signal as `-15` , `-TERM` , or `-SIGTERM` ; all mean the same thing.

2. Write a script that catches signals the way a daemon does.

```
cat > ~/labs/2.3/sigdemo.sh <<'EOF'
#!/bin/bash
trap 'echo "$(date +%T) HUP received: reloading configuration"' HUP
trap 'echo "$(date +%T) TERM received: saving work and exiting"; exit 0' TERM
echo "$(date +%T) started with PID $$"
while true; do sleep 1; done
EOF
chmod +x ~/labs/2.3/sigdemo.sh
~/labs/2.3/sigdemo.sh > ~/labs/2.3/sig.log 2>&1 &
```

What just happened: `trap 'COMMANDS' SIGNAL` tells Bash to run the commands when that signal arrives instead of taking the default action. HUP prints a "reloading" message and keeps going, like a real daemon rereading its config file. TERM prints a message and exits cleanly. `chmod +x` makes the script executable, and the output goes to `sig.log`. Bash runs a trap after the current command (`sleep 1`) finishes, so responses take up to a second.

3. Send HUP, then TERM, and read the log.

```
pgrep -f sigdemo.sh
kill -HUP $(pgrep -f sigdemo.sh)
kill -1 $(pgrep -f sigdemo.sh)
sleep 2
kill $(pgrep -f sigdemo.sh)
sleep 2
cat ~/labs/2.3/sig.log
```

```
11:41:02 started with PID 1880
11:41:10 HUP received: reloading configuration
11:41:11 HUP received: reloading configuration
11:41:13 TERM received: saving work and exiting
```

What just happened: `kill -HUP` and `kill -1` are the same signal; the script survived both. Plain `kill PID` sends signal 15 (TERM), which the script caught to clean up and exit. Real daemons react the same way: `kill -HUP` on many daemons (or `systemctl reload`) rereads the configuration without a restart.

4. Now use `KILL`, which cannot be caught.

```
~/labs/2.3/sigdemo.sh > ~/labs/2.3/sig2.log 2>&1 &
sleep 1
kill -9 $(pgrep -f sigdemo.sh)
sleep 1
cat ~/labs/2.3/sig2.log
```

```
[3]+  Killed                  ~/labs/2.3/sigdemo.sh > ~/labs/2.3/sig2.log 2>&1
11:42:30 started with PID 1902
```

What just happened: `SIGKILL` (9) is handled by the kernel, not the process, so the trap never ran and no "saving work" line appeared. The process had no chance to save data, delete temporary files, or release locks. Correct order on the job: try `kill PID` (TERM) first, wait a few seconds, and use `kill -9` only if it does not exit. Its child `sleep 1` was orphaned and ends on its own within a second.

5. Kill by name with `killall` and `pkill`.

```
for i in 1 2 3; do sleep 900 & done
killall -v sleep
pgrep -a sleep
```

```
Killed sleep(1801) with signal 15
Killed sleep(1802) with signal 15
Killed sleep(1831) with signal 15
...
```

What just happened: `killall NAME` sends `SIGTERM` to every process with exactly that name that you are allowed to signal; `-v` reports each one. Here it also removed `sleep 2222` (the `nohup` one). Use `killall -s HUP NAME` or `killall -9 NAME` for other signals, and `-i` to confirm each one.

CAUTION

On Linux `killall` kills by name, but on some UNIX systems (such as Solaris) `killall` kills **every** process. Never type it out of habit on an unfamiliar system.

6. Use `pkill` with patterns and users. Start some processes as `limituser` first.

```
sudo -u limituser bash -c 'sleep 700 & sleep 800 & yes > /dev/null &'
pgrep -u limituser -a
pkill -u limituser
sudo pkill -u limituser yes
sudo pkill -u limituser -f '^sleep 700'
pgrep -u limituser -a
sudo pkill -9 -u limituser
pgrep -u limituser -a
```

```
1950 sleep 700
1951 sleep 800
1952 yes
pkill: killing pid 1950 failed: Operation not permitted
...
1951 sleep 800
```

What just happened: `pgrep` and `pkill` share the same matching options. `-u USER` matches by owner, a name pattern matches the process name, and `-f` matches a regular expression against the whole command line (so `'^sleep 700'` hit only `sleep 700`). The `^` anchors the match to the start of the command line, so only a command line that *begins* with `sleep 700` matches. Anchoring `-f` patterns is a good habit: an unanchored pattern also matches any other command that merely *contains* the text (for example, a `sudo pkill -f 'sleep 700'` you run without `-u` could match its own `sudo` process). As `student`, you cannot signal another user's processes, so the first `pkill` failed; with `sudo` it works. `sudo pkill -9 -u limituser` is the classic way to clear out every process of a user, for example before deleting the account. Always preview with `pgrep` first, using the same options.

Checkpoint: `pgrep -a sleep` and `pgrep -u limituser` print nothing, and `sig.log` contains two HUP lines and one TERM line.

Task 9: Schedule jobs with `crontab`, `at`, and `anacron`

1. Create a personal cron job with `crontab -e`.

```
crontab -l
crontab -e
```

```
no crontab for student
no crontab for student - using an empty one

Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny
  4. /bin/ed

Choose 1-4 [1]: 1
```

Choose `nano`. At the bottom of the file add these two lines, then save (`Ctrl+O`, Enter) and exit (`Ctrl+X`):

```
*/2 * * * * date >> /home/student/labs/2.3/cron.log
30 2 * * 1-5 /usr/bin/du -sh /home/student > /home/student/labs/2.3/du.log 2>&1
```

```
crontab: installing new crontab
```

What just happened: `crontab -l` lists your cron table (none yet). `crontab -e` opens it in an editor and installs it, checking the syntax, when you save; the file is stored in `/var/spool/cron/crontabs/student`, but you should never edit that file directly. Each line has five time fields and then a command:

```
*/2   *    *    *    *    command
 |    |    |    |    |
 |    |    |    |    +-- day of week  0-7 (0 and 7 = Sunday, or sun-sat)
 |    |    |    +------- month        1-12 (or jan-dec)
 |    |    +------------ day of month 1-31
 |    +----------------- hour         0-23
 +---------------------- minute       0-59
```

`*` means "every", `*/2` "every 2nd", `1-5` a range (Monday to Friday), and `1,15` a list. So the first line runs every 2 minutes, and the second at 02:30 on weekdays. Cron jobs run with a minimal environment and `PATH`, so use full paths and redirect output to files. Shortcuts like `@reboot`, `@daily`, and `@hourly` also work in place of the five fields.

2. Wait two or three minutes, then check that the job ran.

```
crontab -l | tail -2
cat ~/labs/2.3/cron.log
journalctl -u cron --since "10 min ago" | tail -3
```

```
Sat Oct 10 11:50:01 UTC 2026
Sat Oct 10 11:52:01 UTC 2026
... CRON[2101]: (student) CMD (date >> /home/student/labs/2.3/cron.log)
```

What just happened: The `cron` daemon checks every minute for jobs that are due. The journal shows each command it started for each user. If a cron job "does not work", look here first, then check paths and permissions.

3. Look at system-wide cron locations and add a job in `/etc/cron.d`.

```
cat /etc/crontab
ls /etc/cron.d /etc/cron.daily /etc/cron.hourly
echo '*/5 * * * * root /usr/bin/uptime >> /var/log/lab23-uptime.log' | \
  sudo tee /etc/cron.d/lab23
```

```
# /etc/crontab: system-wide crontab
...
SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
...
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.daily; }
47 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.weekly; }
52 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report
/etc/cron.monthly; }
```

What just happened: `/etc/crontab` and files in `/etc/cron.d/` use the same five fields **plus a user name** before the command, because they are system files and must say who runs each job. User crontabs (from `crontab -e`) have no user field. Scripts dropped into `/etc/cron.hourly`, `cron.daily`, `cron.weekly`, and `cron.monthly` are run by `run-parts`. Note the `test -x /usr/sbin/anacron ||` part: when `anacron` is installed (you installed it), cron leaves the daily, weekly, and monthly directories to `anacron`.

4. Run a one-time job with `at`, and manage the queue with `atq` and `atrm`.

```
systemctl is-active atd
at now + 2 minutes
```

At the `at>` prompt type the line below, press Enter, then press `Ctrl+D`:

```
echo "at job ran at $(date)" > /home/student/labs/2.3/at.log
```

```
active
warning: commands will be executed using /bin/sh
at> echo "at job ran at $(date)" > /home/student/labs/2.3/at.log
at> <EOT>
job 1 at Sat Oct 10 11:56:00 2026
```

Queue two more jobs non-interactively, then manage the queue:

```
echo 'touch /home/student/labs/2.3/teatime' | at teatime tomorrow
echo 'date > /home/student/labs/2.3/later.log' | at 23:30
atq
at -c 2 | tail -3
atrm 2
atq
```

```
1   Sat Oct 10 11:56:00 2026 a student
2   Sun Oct 11 16:00:00 2026 a student
3   Sat Oct 10 23:30:00 2026 a student
```

What just happened: `at TIME` reads commands from standard input until `Ctrl+D` (shown as `<EOT>`) and runs them once at that time, using `/bin/sh`. The `atd` daemon executes them. Time formats include `now + 2 minutes`, `23:30`, `noon`, `teatime` (16:00), `tomorrow`, and `10:00 2026-12-25`. `atq` lists pending jobs (job number, time, queue letter, owner); `at -c N` prints the full script of a job, including the saved environment; `atrm N` deletes a job. After two minutes, `cat ~/labs/2.3/at.log` shows that job 1 ran and `atq` no longer lists it. Access is controlled by `/etc/at.allow` and `/etc/at.deny` (and `/etc/cron.allow` and `/etc/cron.deny` for cron).

5. Explore `anacron` and add a test job.

```
cat /etc/anacrontab
ls -l /var/spool/anacron/
```

```
# /etc/anacrontab: configuration file for anacron

# See anacron(8) and anacrontab(5) for details.

SHELL=/bin/sh
HOME=/root
LOGNAME=root

# These replace cron's entries
1   5   cron.daily  run-parts --report /etc/cron.daily
7   10  cron.weekly run-parts --report /etc/cron.weekly
@monthly    15  cron.monthly    run-parts --report /etc/cron.monthly
```

Add a lab job and run it immediately:

```
echo '1  2  lab23.daily  echo "anacron ran $(date)" >> /var/log/lab23-anacron.log' | \
  sudo tee -a /etc/anacrontab
sudo anacron -T && echo "syntax OK"
sudo anacron -f -n -d lab23.daily
cat /var/log/lab23-anacron.log
sudo cat /var/spool/anacron/lab23.daily
```

```
syntax OK
Anacron 2.3 started on 2026-10-10
Will run job `lab23.daily'
Jobs will be executed sequentially
Job `lab23.daily' started
Job `lab23.daily' terminated
Normal exit (1 job run)
anacron ran Sat Oct 10 12:01:14 UTC 2026
20261010
```

What just happened: Each `/etc/anacrontab` line has four fields: **period** in days (or `@monthly`), **delay** in minutes to wait before starting (so jobs do not all start at boot together), a unique **job identifier**, and the **command**. `anacron` records the last run date of each job in `/var/spool/anacron/<identifier>`; if a job's period has passed since that date (for example, because the laptop was off at 06:25), it runs the job. `-T` tests the file's syntax, `-f` forces jobs regardless of their timestamps, `-n` skips the delay, and `-d` stays in the foreground and prints messages. Unlike cron, anacron's smallest unit is one day, it only runs as root, and it does not care about the exact time of day.

6. Glance at the modern alternative, systemd timers.

```
systemctl list-timers --all | head -8
```

What just happened: Many scheduled tasks on Ubuntu 24.04 and Rocky 9 (log rotation, `apt` updates, `plocate` database updates, `fstrim`) are systemd *timers* rather than cron jobs. The list shows when each timer runs next and last. Lab 2.4 covers timers in detail.

TIP

On Rocky Linux 9, cron is provided by `cronie` (service `crond`, not `cron`), and anacron (`cronie-anacron`) is installed and runs the daily, weekly, and monthly jobs by default through `/etc/cron.hourly/0anacron`. `at` is installed with `sudo dnf install -y at` and enabled with `sudo systemctl enable --now atd`. User crontabs live in `/var/spool/cron/<user>`.

Checkpoint: `cat ~/labs/2.3/cron.log` shows a line every 2 minutes, `cat ~/labs/2.3/at.log` shows the `at` job ran, `atq` lists only job 3, and `/var/log/lab23-anacron.log` contains one line.

Challenge

CHALLENGE

1. Write a cron entry (do not install it permanently) that runs a script at 04:15 on the first day of every quarter (January, April, July, October). Test the syntax by installing it, listing it, and removing it.
2. Start a `yes` process, then from a second SSH session find it, lower its priority to 19, stop it, check its state in `/proc/<PID>/status`, resume it, and finally terminate it, using only its PID.
3. Make a script that ignores `SIGTERM` (`trap '' TERM`). Show that `kill` does not stop it but `kill -9` does. Then explain why `kill -9` should still be your last resort.

Clean up

1. Stop any lab processes still running.

```
jobs
kill %% 2>/dev/null
pkill -u student -x yes
pkill -u student -x sleep
pkill -f zombie.py
pkill -f sigdemo.sh
sudo pkill -u limituser
```

2. Remove your crontab, the `/etc/cron.d` file, and pending `at` jobs.

```
crontab -r
crontab -l
sudo rm -f /etc/cron.d/lab23
for j in $(atq | awk '{print $1}'); do atrm "$j"; done
atq
```

`crontab -l` should say `no crontab for student`, and `atq` should print nothing.

> **CAUTION**
> `crontab -r` deletes your **entire** crontab without asking. On a real system, back it up first with `crontab -l > crontab.bak`.

3. Remove the anacron test job and its files.

```
sudo sed -i '/lab23.daily/d' /etc/anacrontab
sudo rm -f /var/spool/anacron/lab23.daily
sudo rm -f /var/log/lab23-anacron.log /var/log/lab23-uptime.log
tail -3 /etc/anacrontab
```

4. Remove the limits file and the test user.

```
sudo rm -f /etc/security/limits.d/90-lab23.conf
sudo userdel -r limituser
```

`userdel` may warn that no mail spool exists; that is fine.

5. Remove the lab directory.

```
cd ~
rm -rf ~/labs/2.3
```

You can leave the installed packages (`htop`, `atop`, `sysstat`, `strace`, `at`, `anacron`). If you prefer to remove `anacron` so cron runs the daily jobs directly again, run `sudo apt remove -y anacron`.

Exam corner

- `ps aux` (BSD style) and `ps -ef` (UNIX style, shows PPID); `ps -eo pid,ppid,stat,ni,cmd --sort=-%cpu` for custom columns. `pstree -p` shows the parent-child tree. `pgrep -a` finds PIDs by name.

- `top` keys: `P` CPU, `M` memory, `k` kill, `r` renice, `1` per CPU, `q` quit. `htop` adds tree view (`F5`), search, and F-key actions. `atop` records history in `/var/log/atop`. `mpstat -P ALL` (per CPU) and `pidstat` (per process) come from `sysstat`.
- `/proc/<PID>/` holds `status`, `cmdline`, `environ`, `fd/`, `cwd`, and `exe`. `lsof -p PID` lists a process's open files. `strace` shows system calls; `strace -p PID` attaches (needs root on Ubuntu).
- States: `R` running, `S` sleeping (interruptible), `D` blocked (uninterruptible I/O wait), `T` stopped, `Z` zombie. Zombies cannot be killed; kill or fix the parent so PID 1 reaps them.
- Nice values run from -20 (highest priority) to 19 (lowest); default 0. `nice -n 10 cmd` starts, `renice -n 5 -p PID` changes, `renice -u user` targets a user. Only root can lower a nice value.
- `ulimit -a` shows limits; `-n` open files, `-u` processes, `-f` file size; `-S` soft, `-H` hard. Users may raise soft up to hard; only root raises hard. Permanent limits: `/etc/security/limits.conf` and `limits.d/` (`domain type item value`), applied by `pam_limits` at new logins.
- Job control: `cmd &` background; `jobs`; `fg %N`; `bg %N`; `Ctrl+Z` stop (SIGTSTP); `Ctrl+C` interrupt (SIGINT); `Ctrl+D` end of input (EOF, not a signal). `nohup cmd &` ignores SIGHUP and writes to `nohup.out`. `exec cmd` replaces the shell, keeping its PID.
- Signals: 1 `SIGHUP` (hang up, often "reload config"), 9 `SIGKILL` (cannot be caught), 15 `SIGTERM` (default for `kill`, graceful). `kill -l` lists them. `killall name` (exact name) and `pkill pattern` (`-u user`, `-f` full command line) signal by name.
- Cron fields: minute, hour, day of month, month, day of week. `crontab -e`, `-l`, `-r`. `/etc/crontab` and `/etc/cron.d/*` include a user field. `at TIME` runs once (`atq`, `atrm`, `at -c`). `anacron` runs daily/weekly/monthly jobs missed while the system was off (`/etc/anacrontab`: period, delay, identifier, command).

Review questions

1. A process shows `Z` in the `STAT` column and `kill -9` has no effect on it. What should the administrator do?
   A. Reboot immediately.
   B. Send `SIGKILL` again with `killall -9`.
   C. Identify its parent with `ps -o ppid` and restart or kill the parent.
   D. Run `renice -n 19` on it.

2. Which crontab entry runs `/usr/local/bin/backup.sh` at 23:45 every Sunday?
   A. `45 23 * * 0 /usr/local/bin/backup.sh`
   B. `23 45 * * 0 /usr/local/bin/backup.sh`
   C. `45 23 0 * * /usr/local/bin/backup.sh`
   D. `* 23 * * sun /usr/local/bin/backup.sh`

3. A user started a long compile in the foreground and wants it to keep running in the background so they can use the shell. Which key and command sequence does this?

4. Which signal does `kill 4321` send, and why is it preferred over `kill -9 4321`?

5. A web application logs "Too many open files." Which two things would you check or change?

6. A laptop is often powered off at night, so its daily maintenance jobs scheduled at 06:25 never run. Which tool solves this?
   A. `at`
   B. `anacron`
   C. `nohup`
   D. `renice`

Lab 2.4 Configure and Manage Software

Objective: 2.4 Given a scenario, configure and manage software in a Linux environment.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 180 minutes (it is fine to split it over two or three sessions)
VM: ubuntu-lab (most tasks) and rocky-lab (Task 4)
Snapshot first: Yes. This lab adds repositories, installs many packages, and starts network services.

Installing, updating, and removing software is the job a Linux administrator does most often. In this lab you will use `apt` and `dpkg` on Ubuntu, `dnf` and `rpm` on Rocky, add a signed third-party repository, build a program from source, try the language package managers `pip`, `cargo`, and `npm`, and run sandboxed apps. Then you will set up small but real DNS, NTP, DHCP, web (Apache and Nginx), SMTP, and IMAP4 services, because the exam expects you to recognize their basic configuration.

What you'll practice

- Installation, update, and removal of software from a **repository**, from **source**, and from package files, including **package dependencies and conflicts**
- **Package managers**: `apt`, `dpkg`, `dnf`, `rpm`
- **Language-specific** package managers: **pip**, **cargo**, **npm**
- **Repository management**: **enabling/disabling** repositories, adding a **third-party** repository, checking **GPG signatures**
- **Package and repository exclusions** (`apt-mark hold`, `exclude=` in `dnf.conf`, `--exclude`)
- **Update alternatives** (`update-alternatives`)
- **Software configuration** (files in `/etc`, drop-in directories)
- **Sandboxed applications** (snap, Flatpak, AppImage)
- **Basic configs of common services**: **DNS** (BIND), **NTP/PTP** (chrony, linuxptp), **DHCP** (Kea), **HTTP** (**Apache httpd**, **Nginx**), **SMTP** (Postfix), **IMAP4** (Dovecot)

How it works

Packages. A *package* is an archive that contains a program's files plus *metadata*: its name, version, a description, a list of the other packages it needs, and scripts to run during install and removal. Debian and Ubuntu use `.deb` packages. Red Hat, Rocky, Fedora, and SUSE use `.rpm` packages.

Two layers of tools. Each family has a low-level tool and a high-level tool:

```
                Debian / Ubuntu          RHEL / Rocky
High level      apt  (apt-get)           dnf  (yum)
  - talks to repositories, downloads, solves dependencies
                    |                        |
Low level       dpkg                     rpm
  - installs ONE local package file, keeps the database
                    |                        |
Database        /var/lib/dpkg/           /var/lib/rpm/
```

The low-level tool (`dpkg` , `rpm`) installs a single file you hand it and records what it installed. It does **not** download anything and does **not** fetch missing pieces. The high-level tool (`apt` , `dnf`) is the *package manager* you use day to day: it reads repository indexes, works out everything that must be installed, downloads it, and then calls the low-level tool.

Repositories. A *repository* (repo) is a web server or folder holding packages plus an index file that lists them. Your system has a list of repos it trusts. `apt update` (or `dnf makecache`) downloads fresh copies of the indexes so the package manager knows what versions exist. A *third-party* repository is one not run by your distribution, for example a vendor such as nginx.org or Docker.

Dependencies and conflicts. A *dependency* is another package that must be present for a package to work, such as a shared library. A *conflict* is a package that cannot be installed at the same time, often because both provide the same service or file. For example, only one NTP daemon or one mail server should own a port. Package managers read these rules from the metadata and either solve them for you or refuse with an error.

GPG signatures. Anyone could put a fake package on a mirror. To prevent that, the repository owner signs the index (apt) or each package (rpm) with a private **GPG** key. Your system holds the matching public key. Before installing, the package manager checks the signature with the public key. If the check fails, it refuses. That is why adding a third-party repo always has two steps: install its public key, then add the repo line.

Source code. Sometimes software is not packaged at all. Then you download the *source code* (usually a `.tar.gz` "tarball") and compile it yourself. The classic three steps are `./configure` (check the system and write a Makefile), `make` (compile), and `sudo make install` (copy files, usually under `/usr/local`). The package manager does not track these files, so you are responsible for updates and removal.

Language package managers. Programming languages have their own package managers: `pip` for Python, `npm` for JavaScript (Node.js), and `cargo` for Rust. They pull libraries from their own online registries (PyPI, the npm registry, crates.io). They can install packages *locally* (inside one project folder or virtual environment) or *globally* (for the whole system). Local is safer because it cannot break programs that the OS itself depends on.

Sandboxed applications. A *sandbox* is a restricted environment that limits what a program can see and change. Snap (Canonical), Flatpak (common on desktops), and AppImage (a single portable file) bundle an app with its libraries. Snap and Flatpak also confine the app, so it cannot read your whole home directory or write anywhere it likes.

Configuration. Almost all software is configured with text files under `/etc` . Modern packages use *drop-in directories* such as `/etc/nginx/conf.d/` or `/etc/chrony/conf.d/` . You add your own small file there instead of editing the big main file. Package updates then replace the main file safely while your file stays untouched.

Services you will meet. DNS turns names into IP addresses (BIND's daemon is `named`). NTP keeps clocks in sync (chrony); PTP is a more precise time protocol for local networks (linuxptp). DHCP hands out IP addresses automatically (Kea). HTTP serves web pages (Apache, called `apache2` on Ubuntu and `httpd` on Rocky, and Nginx). SMTP moves email between servers (Postfix). IMAP4 lets mail clients read mailboxes stored on a server (Dovecot).

Tasks

Task 1: Use apt to search, install, inspect, and remove packages

1. Make the lab folder and refresh the package indexes.

```
mkdir -p ~/labs/2.4 && cd ~/labs/2.4
sudo apt update
```

```
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
...
Reading package lists... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

> **What just happened:** `apt update` downloads the latest package *indexes* from every enabled repository. It installs nothing. `Hit` means the index had not changed; `Get` means a new one was downloaded. `noble` is the code name of Ubuntu 24.04, and `noble-updates` and `noble-security` are separate repos for fixes. Always run `apt update` before installing, or apt may try to fetch versions that no longer exist on the mirror.

2. See what could be upgraded, then search for a package.

```
apt list --upgradable
apt search --names-only '^tree$'
```

> **What just happened:** `apt list --upgradable` shows installed packages that have a newer version available. `apt search` searches names and descriptions; `--names-only` limits it to names, and `'^tree$'` is a regular expression meaning "exactly tree". The single quotes stop the shell from changing the special characters.

3. Look at a package's details before you install it.

```
apt show tree
apt-cache policy tree
```

```
Package: tree
Version: 2.1.1-2ubuntu3.24.04.2
Priority: optional
Section: universe/utils
...
Depends: libc6 (>= 2.38)
...
tree:
  Installed: (none)
  Candidate: 2.1.1-2ubuntu3.24.04.2
  Version table:
     2.1.1-2ubuntu3.24.04.2 500
        500 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages
     2.1.1-2ubuntu3 500
        500 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages
```

What just happened: `apt show` prints the metadata, including `Depends:` (the dependencies). `apt-cache policy` shows the installed version, the *candidate* (the version apt would install), and which repository offers each version. Here the original release version (`noble`) and a newer bug-fix version (`noble-updates`) both exist, and apt picks the newer one. The number `500` is the *priority*; when several repos offer a package, apt picks the highest priority, then the highest version. Your version numbers may differ.

4. Install the package and use it.

```
sudo apt install -y tree
tree -L 1 /etc/apt
```

What just happened: `apt install` resolved dependencies, downloaded the `.deb` , and called `dpkg` to install it. `-y` answers "yes" to the confirmation question. `tree -L 1` shows one level of the `/etc/apt` folder, where apt keeps its configuration.

5. Look at dependencies in both directions.

```
apt-cache depends tree
apt-cache rdepends --installed libc6 | head
```

What just happened: `apt-cache depends` lists what a package *needs*. `apt-cache rdepends` ("reverse depends") lists what needs *it*; `--installed` limits the list to installed packages. Almost everything depends on `libc6` , the C library, which is why you must never remove it. (`apt depends` and `apt rdepends` work too.)

6. Remove, purge, and clean up.

```
sudo apt remove -y tree
dpkg -l tree
sudo apt purge -y tree
sudo apt autoremove -y
```

```
||/ Name           Version               Architecture Description
+++-==============-=====================-============-=========================
rc  tree           2.1.1-2ubuntu3.24.04.2 amd64        displays an indented directory
```

What just happened: `apt remove` deletes the program but keeps its configuration files. `dpkg -l` shows the state as two letters: `rc` means "removed, config files remain" (`ii` means fully installed). `apt purge` removes the configuration files too. `apt autoremove` removes packages that were installed only as dependencies and are no longer needed by anything.

You do not need to run a full upgrade in this lab, but you must know the upgrade commands.

Upgrades in brief: `sudo apt upgrade` installs newer versions of installed packages but never removes a package. `sudo apt full-upgrade` (the old `apt-get dist-upgrade`) may also remove packages if that is needed to solve new dependencies. `sudo apt install --only-upgrade <pkg>` upgrades just one package. On a real server, read the list apt prints before you type `y` .

Checkpoint: `dpkg -l tree` should now print `dpkg-query: no packages found matching tree` , because the package was purged.

Task 2: Package files, dependency failures, conflicts, and holds

1. Download a `.deb` file without installing it. `fortune-mod` prints random quotes and needs a library that a server does not have.

```
cd ~/labs/2.4
apt download fortune-mod
ls *.deb
dpkg -I fortune-mod_*.deb | grep -E 'Package|Version|Depends'
```

What just happened: `apt download` fetches a package file into the current folder. `dpkg -I` (capital i, "info") reads the metadata inside the file. The `Depends:` line lists packages that must be installed first. The `*` is a shell wildcard so you do not need to type the long version number.

2. Install the file with the low-level tool and watch it fail.

```
sudo dpkg -i fortune-mod_*.deb
```

```
Selecting previously unselected package fortune-mod.
Preparing to unpack fortune-mod_1%3a1.99.1-7.3build1_amd64.deb ...
Unpacking fortune-mod (1:1.99.1-7.3build1) ...
dpkg: dependency problems prevent configuration of fortune-mod:
 fortune-mod depends on librecode0 (>= 3.6); however:
  Package librecode0 is not installed.

dpkg: error processing package fortune-mod (--install):
 dependency problems - leaving unconfigured
Errors were encountered while processing:
 fortune-mod
```

What just happened: `dpkg -i` (install) unpacked the files but could not *configure* the package, because `dpkg` never downloads dependencies. The package is now in a "broken" half-installed state. Until you fix it, apt will refuse most other installs and tell you to run `apt --fix-broken install`.

3. Let apt fix the broken dependency, then run the program.

```
sudo apt -f install -y
dpkg -l fortune-mod
fortune
apt-cache show fortune-mod | grep -E '^(Depends|Recommends)'
sudo apt install -y fortunes-min
fortune
```

```
No fortunes found
Depends: libc6 (>= 2.34), librecode0 (>= 3.6)
Recommends: fortunes-min | fortune-cookie-db
```

What just happened: `apt -f install` (long form `apt --fix-broken install`) finds the missing dependencies (here `librecode0`), downloads them, and finishes configuring `fortune-mod`. `dpkg -l` now shows `ii`. On Ubuntu the program lives in `/usr/games`, which is already in your `PATH`. The first `fortune` printed `No fortunes found`: the quote files are in a separate package that `fortune-mod` only *recommends*. A *Depends* must be installed; a *Recommends* is installed by `apt install` by default but is skipped by `dpkg -i` and by the fix-broken step. The `|` in `fortunes-min | fortune-cookie-db` means "any one of these". After installing `fortunes-min`, `fortune` prints a quote.

TIP

The easy way to install a local `.deb` is `sudo apt install ./file.deb`. The `./` tells apt it is a file, and apt resolves dependencies automatically. You may see a note that the download was "performed unsandboxed as root" because apt's `_apt` user cannot read your home folder; that is harmless.

4. Ask `dpkg` questions about installed packages.

```
dpkg -L fortune-mod | head
dpkg -S /usr/games/fortune
```

What just happened: `dpkg -L` lists every file a package installed. `dpkg -S` does the reverse: it searches for which package owns a file. These two are the Debian equivalents of `rpm -ql` and `rpm -qf` that you will use on Rocky.

5. See a real package conflict. Ubuntu 24.04 uses the small `systemd-timesyncd` time client by default, and `chrony` is a full NTP daemon. Only one time daemon may run, so the packages conflict. Simulate the install with `-s`.

```
apt-cache show chrony | grep -E '^(Conflicts|Provides|Replaces)'
apt install -s chrony | grep -E 'REMOVED|NEW|^Remv|^Inst'
```

```
Replaces: time-daemon
Provides: time-daemon
Conflicts: time-daemon
...
The following packages will be REMOVED:
The following NEW packages will be installed:
Remv systemd-timesyncd [255.4-1ubuntu8]
Inst chrony (4.5-1ubuntu4.2 Ubuntu:24.04/noble-updates [amd64])
```

What just happened: `-s` (simulate) makes apt print its plan without changing anything, so you do not need `sudo`. Both packages *provide* the virtual package `time-daemon`, and each *conflicts* with any other `time-daemon`, so apt plans to remove `systemd-timesyncd`. If chrony is already installed on your VM, apt instead says it is already the newest version; the `Conflicts:` line still shows the rule. (`apt-cache show` prints one block per available version, so the three lines may repeat.) Whenever apt says "will be REMOVED", stop and read the list.

6. Exclude a package from upgrades with a hold.

```
sudo apt-mark hold fortune-mod
apt-mark showhold
sudo apt-mark unhold fortune-mod
```

What just happened: `apt-mark hold` freezes a package at its current version: `apt upgrade` skips it and lists it as "kept back". This is how you *exclude* a package from updates on Debian/Ubuntu (for example, to keep a database at a tested version). `showhold` lists held packages, and `unhold` releases the hold.

Checkpoint: `fortune` prints a quote, `dpkg -l fortune-mod` begins with `ii`, and `apt-mark showhold` prints nothing.

Task 3: Manage Ubuntu repositories and add a signed third-party repo

1. Look at where Ubuntu 24.04 keeps its repository list.

```
ls /etc/apt/sources.list /etc/apt/sources.list.d/
cat /etc/apt/sources.list.d/ubuntu.sources
```

```
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg

Types: deb
URIs: http://security.ubuntu.com/ubuntu/
Suites: noble-security
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

What just happened: Ubuntu 24.04 uses the newer *deb822* format (named after the email-header style of RFC 822). Each block, separated by a blank line, is one source. `Types: deb` means binary packages (`deb-src` would mean source packages). `URIs` is the server, `Suites` are the release names, and `Components` are sections: `main` (supported free software), `restricted` (supported non-free drivers), `universe` (community-maintained), and `multiverse` (non-free). `Signed-By` names the public key file that must have signed this repo. The old one-line format in `/etc/apt/sources.list` still works, but on 24.04 that file usually only holds a comment.

2. Install the tools needed to add a third-party repository and create the folder for its key.

```
sudo apt install -y curl gnupg ca-certificates
sudo install -m 0755 -d /etc/apt/keyrings
```

What just happened: `curl` downloads the key, `gnupg` converts it, and `ca-certificates` lets curl trust HTTPS sites. `install -m 0755 -d` creates the directory `/etc/apt/keyrings` with permissions `0755` if it does not exist. That folder is the recommended place for keys you add by hand.

3. Download nginx.org's public signing key and store it in binary form.

```
curl -fsSL https://nginx.org/keys/nginx_signing.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/nginx-archive-keyring.gpg
gpg --show-keys /etc/apt/keyrings/nginx-archive-keyring.gpg
```

```
pub   rsa2048 2011-08-19 [SC] [expires: 2027-05-24]
      573BFD6B3D8FBC641079A6ABABF5BD827BD9BF62
uid                      nginx signing key <signing-key@nginx.com>
...
```

What just happened: `curl -fsSL` downloads quietly (`-s`), follows redirects (`-L`), shows errors (`-S`), and fails on HTTP errors (`-f`). The key arrives as text ("ASCII armored"). `gpg --dearmor` converts it to the binary format apt reads, and `-o` writes it to the file. `gpg --show-keys` prints the key's fingerprint without importing it anywhere. On a real system, compare the fingerprint with the one published on the vendor's website before trusting it. You may see more than one key; vendors rotate keys over time.

4. Add the repository in deb822 format. This uses a *heredoc*: everything between `<<'EOF'` and the line `EOF` is sent as input to `sudo tee`, which writes it to the file. The quotes around `EOF` stop the shell from changing anything inside.

```
cat <<'EOF' | sudo tee /etc/apt/sources.list.d/nginx.sources
Types: deb
URIs: http://nginx.org/packages/ubuntu
Suites: noble
Components: nginx
Signed-By: /etc/apt/keyrings/nginx-archive-keyring.gpg
EOF
sudo apt update
```

> **What just happened:** You created a new source file. `apt update` now also downloads `nginx.org`'s index and checks its signature with only the key named in `Signed-By`. Limiting a key to one repo with `Signed-By` is important: an old, deprecated method (`apt-key add`) trusted every key for every repo. The URL uses plain `http`, which is fine because the GPG signature, not the transport, protects the packages from tampering.

5. Compare the versions offered by the two repositories.

```
apt-cache policy nginx
```

```
nginx:
  Installed: (none)
  Candidate: 1.28.0-1~noble
  Version table:
     1.28.0-1~noble 500
        500 http://nginx.org/packages/ubuntu noble/nginx amd64 Packages
     ...
     1.24.0-2ubuntu7 500
        500 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages
```

> **What just happened:** Both repos have priority 500, so apt chooses the higher version, which comes from nginx.org. This is the power and the risk of third-party repos: they can replace distribution packages. You could raise or lower a repo's priority with a *pin* file in `/etc/apt/preferences.d/`. You will not install from nginx.org in this lab; later tasks use Ubuntu's own `nginx`.

6. Disable the third-party repository without deleting it, and confirm.

```
echo 'Enabled: no' | sudo tee -a /etc/apt/sources.list.d/nginx.sources
sudo apt update
apt-cache policy nginx | head -4
```

What just happened: In deb822 format, adding `Enabled: no` to a block disables that source. `tee -a` *appends* to the file instead of overwriting it. After `apt update`, the candidate version comes from Ubuntu again. To re-enable it, delete that line (or change it to `yes`). With the old one-line format, you disable a repo by putting `#` at the start of its line.

TIP

`add-apt-repository ppa:<user>/<name>` adds a Launchpad PPA (Personal Package Archive), which is another kind of third-party repo. It downloads the key and writes the source file for you. `add-apt-repository --remove` undoes it.

Checkpoint: `apt-cache policy nginx` shows a candidate version from `archive.ubuntu.com`, and `grep Enabled /etc/apt/sources.list.d/nginx.sources` prints `Enabled: no`.

Task 4: Use dnf and rpm on Rocky Linux

Run every command in this task on rocky-lab.

1. Install a package, read its information, and find which package provides a file.

```
# on rocky-lab
sudo dnf install -y tree
dnf info tree
dnf provides /usr/bin/dig
```

```
Name         : tree
Version      : 1.8.0
Release      : 10.el9
Architecture : x86_64
Size         : 56 k
Source       : tree-1.8.0-10.el9.src.rpm
Repository   : @System
From repo    : baseos
...
bind-utils-32:9.16.23-24.el9_5.x86_64 : Utilities for querying DNS name servers
Repo        : appstream
Matched from:
Filename    : /usr/bin/dig
```

What just happened: `dnf install` works like `apt install`: it resolves dependencies and downloads from repos. There is no separate "update the index" step, because dnf refreshes its cached metadata automatically when it is old (you can force it with `sudo dnf makecache`). `dnf info` shows metadata; `@System` means installed. `dnf provides` searches all repos for a file path, so you can find which package to install when a command is missing. `yum` still exists on Rocky 9 as a link to `dnf`.

2. Query the RPM database directly.

```
# on rocky-lab
rpm -qi tree | head -5
rpm -ql tree
rpm -qf /usr/bin/tree
rpm -qa | grep -c .
```

What just happened: `rpm -q` means query. `-qi` shows info, `-ql` lists the files in a package, `-qf` finds which package owns a file, and `-qa` lists all installed packages (`grep -c .` counts the lines). These read the local database only, so they work offline.

3. Use the history to undo a transaction.

```
# on rocky-lab
sudo dnf history
```

```
ID     | Command line             | Date and time    | Action(s)      | Altered
-------------------------------------------------------------------------------
     4 | install -y tree          | 2026-10-10 14:02 | Install        |    1
     3 | ...
```

Replace `4` below with the ID of your `install -y tree` line.

```
# on rocky-lab
sudo dnf history info 4
sudo dnf history undo -y 4
rpm -q tree
```

What just happened: dnf records every transaction. `history info` shows exactly what changed, and `history undo` reverses it (here, it removes `tree`). This is a quick way to back out a bad install. `rpm -q tree` now prints `package tree is not installed`. The normal way to remove a package is `sudo dnf remove tree`.

4. List repositories, then disable and enable one.

```
# on rocky-lab
dnf repolist
dnf repolist all
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --set-disabled extras
dnf repolist
sudo dnf config-manager --set-enabled extras
```

```
repo id                         repo name
appstream                       Rocky Linux 9 - AppStream
baseos                          Rocky Linux 9 - BaseOS
extras                          Rocky Linux 9 - Extras
```

What just happened: `dnf repolist` shows enabled repos; `repolist all` also shows disabled ones such as `crb`. Repos are defined in `.repo` files in `/etc/yum.repos.d/`. `config-manager` (from `dnf-plugins-core`) edits those files for you: `--set-disabled` writes `enabled=0` and `--set-enabled` writes `enabled=1`. For a single command only, use `--enablerepo=<id>` or `--disablerepo=<id>`.

5. Add a popular third-party repo: EPEL (Extra Packages for Enterprise Linux). The EPEL documentation tells you to enable the `crb` repo first because some EPEL packages need it.

```
# on rocky-lab
sudo dnf config-manager --set-enabled crb
sudo dnf install -y epel-release
dnf repolist
sudo dnf install -y htop
```

```
Importing GPG key 0x3228467C:
 Userid     : "Fedora (epel9) <epel@fedoraproject.org>"
 Fingerprint: FF8A D134 4597 106E CE81 3B91 8A38 72BF 3228 467C
 From       : /etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-9
Key imported successfully
```

What just happened: `epel-release` is a tiny package from Rocky's `extras` repo that drops `/etc/yum.repos.d/epel.repo` and the EPEL public key into place. The first time you install from EPEL, dnf imports that key (`-y` accepted it for you). `htop` is not in Rocky's own repos; it comes from EPEL.

6. Inspect GPG checking and verify a package file yourself.

```
# on rocky-lab
grep -E '^\[|gpgcheck|gpgkey' /etc/yum.repos.d/rocky.repo | head -6
rpm -q gpg-pubkey --qf '%{NAME}-%{VERSION}-%{RELEASE}  %{SUMMARY}\n'
cd /tmp && dnf download tree
rpm -K tree-*.rpm
```

```
[baseos]
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
...
gpg-pubkey-350d275d-6279464b  Rocky Enterprise Software Foundation - Release key 2022 ...
gpg-pubkey-3228467c-613798eb  Fedora (epel9) <epel@fedoraproject.org> public key
tree-1.8.0-10.el9.x86_64.rpm: digests signatures OK
```

What just happened: `gpgcheck=1` tells dnf to verify every package's signature against the key in `gpgkey=`. Imported keys appear in the RPM database as fake packages named `gpg-pubkey`. `dnf download` saves a package file without installing it. `rpm -K` (check signature) verifies the file: `digests signatures OK` means it is intact and signed by a trusted key. To trust a new key by hand, you would run `sudo rpm --import /path/to/KEY`. Never set `gpgcheck=0` to "fix" a signature error.

7. Exclude a package with `dnf.conf`, and on the command line.

```
# on rocky-lab
echo 'exclude=tree' | sudo tee -a /etc/dnf/dnf.conf
sudo dnf install -y tree
sudo dnf install -y tree --disableexcludes=all
sudo dnf remove -y tree --disableexcludes=all
sudo dnf upgrade --exclude='kernel*' --assumeno
```

```
All matches were filtered out by exclude filtering for argument: tree
Error: Unable to find a match: tree
```

What just happened: `exclude=` in the `[main]` section of `/etc/dnf/dnf.conf` hides matching packages from dnf completely, as if they were not in any repo. (The line you appended lands in `[main]` because it is the only section.) `--disableexcludes=all` ignores the exclusions for one command; you need it for `dnf remove` too, because excluded packages are hidden even when installed. `--exclude='kernel*'` (short `-x`) excludes packages for just one command; the quotes stop the shell from expanding `*`. `--assumeno` shows the plan and answers "no", so nothing is upgraded. You can also put `exclude=` inside one repo's section in a `.repo` file to exclude a package from just that repo.

Checkpoint: On rocky-lab, `dnf repolist` lists `epel`, `rpm -q htop` shows a version, and `tail -1 /etc/dnf/dnf.conf` prints `exclude=tree`. (You will revert these in Clean up.)

Task 5: Build and install a program from source

1. Install the compiler toolchain. `build-essential` is a meta-package that pulls in `gcc`, `g++`, `make`, and the C library headers.

```
sudo apt install -y build-essential wget
gcc --version | head -1
```

2. Download and unpack the source of GNU Hello, a tiny program the GNU project publishes as an example.

```
cd ~/labs/2.4
wget https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
tar -xzf hello-2.12.1.tar.gz
cd hello-2.12.1
ls
```

What just happened: `tar -xzf` extracts (`x`) a gzip-compressed (`z`) file (`f`). Look for `configure`, `README`, and `INSTALL`. Always read `README` and `INSTALL` first in real projects; they list required libraries (the "build dependencies").

3. Configure, compile, and install.

```
./configure --prefix=/usr/local
make
sudo make install
```

```
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
...
config.status: creating Makefile
...
 /usr/bin/mkdir -p '/usr/local/bin'
  /usr/bin/install -c hello '/usr/local/bin'
```

What just happened: `./configure` is a script that checks your system for a compiler and libraries and writes a `Makefile`. If a library is missing, it stops with an error naming it; you then install the matching `-dev` package (Ubuntu) or `-devel` package (Rocky) and run it again. `--prefix=/usr/local` (the default) means files go under `/usr/local`, which is reserved for software the package manager does not manage. `make` compiles according to the `Makefile`. `sudo make install` copies the results into place; only this step needs root.

4. Run it and confirm the package manager knows nothing about it.

```
hash -r
which hello
hello
dpkg -S /usr/local/bin/hello
```

```
/usr/local/bin/hello
Hello, world!
dpkg-query: no path found matching pattern /usr/local/bin/hello
```

What just happened: `hash -r` clears the shell's memory of command locations so it finds the new program. `dpkg -S` cannot find an owner, which proves source installs are invisible to `apt`. They are not updated by `apt upgrade` and cannot be removed with `apt remove`. Keep the source folder so you can run `sudo make uninstall` later.

Checkpoint: `hello --version | head -1` prints `hello (GNU Hello) 2.12.1`.

Task 6: Language-specific package managers: pip, cargo, and npm

1. **pip.** Install Python's tools and try a system-wide install.

```
sudo apt install -y python3-pip python3-venv
pip3 install cowsay
```

```
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.
...
```

What just happened: Ubuntu 24.04 marks the system Python as "externally managed" (rule PEP 668). Ubuntu's own tools depend on that Python, so pip refuses to change it and risk a *conflict* with apt-installed Python packages. The safe answer is a *virtual environment* (venv): a private folder with its own copy of `pip` and its own packages.

2. Create a venv, activate it, and install the package there.

```
cd ~/labs/2.4
python3 -m venv venv
source venv/bin/activate
pip install cowsay
cowsay -t "Hello from pip"
pip list
pip freeze > requirements.txt
cat requirements.txt
deactivate
```

What just happened: `python3 -m venv venv` creates the folder `venv`. `source venv/bin/activate` changes your `PATH` so `python` and `pip` come from that folder; your prompt shows `(venv)`. `pip install` downloads from PyPI into `venv/lib/...` only. `pip freeze` writes exact versions so another machine can rebuild the same set with `pip install -r requirements.txt`. `deactivate` returns to the normal shell. Deleting the folder removes everything.

3. **cargo.** Install Rust's compiler and package manager from Ubuntu, and create a project.

```
sudo apt install -y cargo
cd ~/labs/2.4
cargo new luckynum
cd luckynum
```

What just happened: The `cargo` package pulls in `rustc` (the compiler) as a dependency. `cargo new` creates a project folder with `Cargo.toml` (the project description and dependency list) and `src/main.rs`.

4. Add a dependency (a "crate") and some code. Open `Cargo.toml` with `nano Cargo.toml` and add this line under `[dependencies]`:

```
fastrand = "2"
```

Then replace `src/main.rs` with this program:

```
cat <<'EOF' > src/main.rs
fn main() {
    println!("Your lucky number is {}", fastrand::u8(1..=100));
}
EOF
cargo run
```

```
    Updating crates.io index
 Downloaded fastrand v2.3.0
   Compiling fastrand v2.3.0
   Compiling luckynum v0.1.0 (/home/student/labs/2.4/luckynum)
    Finished dev [unoptimized + debuginfo] target(s) in 3.10s
     Running `target/debug/luckynum`
Your lucky number is 42
```

What just happened: `cargo run` read `Cargo.toml`, downloaded the `fastrand` crate from crates.io, compiled it and your program, and ran the result. Dependencies are *local* to the project: they are recorded in `Cargo.lock` and built inside `target/`. `cargo install <crate>` instead builds a command-line program and puts it in `~/.cargo/bin` for your user. Ubuntu's Rust is an older, stable version; developers often use `rustup` to get the newest one.

5. **npm.** Install Node.js and npm, then install a package locally in a project.

```
sudo apt install -y nodejs npm
mkdir -p ~/labs/2.4/nodeapp && cd ~/labs/2.4/nodeapp
npm init -y
npm install cowsay
ls
npx cowsay "Hello from npm"
```

What just happened: The `npm` package from Ubuntu pulls in many small `node-*` dependencies, so the install takes a minute. `npm init -y` creates `package.json`, the project's description. `npm install cowsay` (local) downloads the package into `./node_modules` and records it in `package.json` and `package-lock.json`. `npx` runs a command from the local `node_modules/.bin` folder.

6. Compare with a *global* npm install.

```
sudo npm install -g cowsay
which cowsay
npm ls -g --depth=0
sudo npm uninstall -g cowsay
```

What just happened: `-g` installs for the whole system, into `/usr/local/lib/node_modules` with the command linked in `/usr/local/bin`, so it needs `sudo`. Global installs are convenient for tools but can clash between projects that need different versions. Rule of thumb for all three tools: install project libraries *locally* (venv, `Cargo.toml`, `node_modules`) and use global installs only for command-line tools.

Checkpoint: `ls ~/labs/2.4` shows `venv`, `luckynum`, `nodeapp`, and `requirements.txt`, and `ls ~/labs/2.4/nodeapp/node_modules | grep cowsay` prints `cowsay`.

Task 7: Update alternatives, software configuration, and sandboxed apps

1. Several programs can do the same job, such as text editors. Debian's *alternatives system* uses symbolic links to pick which one a generic name like `editor` runs.

```
sudo apt install -y vim nano
ls -l /usr/bin/editor /etc/alternatives/editor
update-alternatives --display editor
```

```
lrwxrwxrwx 1 root root 24 Oct 10 14:30 /usr/bin/editor -> /etc/alternatives/editor
lrwxrwxrwx 1 root root  9 Oct 10 14:30 /etc/alternatives/editor -> /bin/nano
editor - auto mode
  link best version is /bin/nano
  link currently points to /bin/nano
  link editor is /usr/bin/editor
...
/bin/ed - priority -100
/bin/nano - priority 40
/usr/bin/vim.basic - priority 30
/usr/bin/vim.tiny - priority 15
```

What just happened: `/usr/bin/editor` links to `/etc/alternatives/editor`, which links to the real program. Each candidate has a *priority*. In *auto mode* the highest priority wins (nano, 40). Programs like `crontab -e` or `visudo` run `editor` when you have not set `$EDITOR`.

2. Change the choice, check it, and go back to auto mode.

```
sudo update-alternatives --config editor
```

```
There are 4 choices for the alternative editor (providing /usr/bin/editor).

  Selection    Path                Priority   Status
------------------------------------------------------------
* 0            /bin/nano            40        auto mode
  1            /bin/ed             -100       manual mode
  2            /bin/nano            40        manual mode
  3            /usr/bin/vim.basic   30        manual mode
  4            /usr/bin/vim.tiny    15        manual mode

Press <enter> to keep the current choice[*], or type selection number:
```

Type the number for `/usr/bin/vim.basic` and press Enter. Then:

```
readlink -f /usr/bin/editor
sudo update-alternatives --auto editor
readlink -f /usr/bin/editor
```

What just happened: `--config` lets you choose interactively and switches to *manual mode*, so package updates will not change your choice. `readlink -f` follows the whole link chain and prints the final file. `--auto` returns to automatic mode (highest priority). In scripts use `--set editor /usr/bin/vim.basic`. Packages register choices with `--install`. On Rocky the same tool is called `alternatives` (for example, `alternatives --config python3`).

3. Look at how software configuration is organized under `/etc`.

```
ls /etc/apt/apt.conf.d/
ls -d /etc/*.d | head -20
cat /etc/default/useradd 2>/dev/null | head -5
```

What just happened: Directories ending in `.d` are *drop-in* folders: every file inside is read, usually in alphabetical order (hence names like `50unattended-upgrades` and `99local`). `/etc/default/` holds simple settings that service start scripts read. When a package update ships a new version of a config file you changed, `dpkg` asks whether to keep yours or take the new one (on Rocky, `rpm` saves the new one as `.rpmnew`). Using drop-in files avoids that question. `sudo dpkg-reconfigure <package>` re-runs a package's setup questions.

4. Run a sandboxed snap application. Ubuntu Server has `snapd` installed.

```
snap list
sudo snap install hello-world
hello-world
hello-world.evil
snap info hello-world | head -8
```

```
hello-world 6.4 from Canonical installed
Hello World!
Hello Evil World!
This example demonstrates the app confinement
You should see a permission denied error next
/snap/hello-world/29/bin/evil: 9: cannot create /var/tmp/myevil.txt: Permission denied
```

What just happened: `snap install` downloaded a compressed, read-only image from the Snap Store and mounted it under `/snap/hello-world/<revision>`. Snaps update themselves automatically in the background. `hello-world.evil` tries to write a file outside its sandbox, and *strict confinement* (enforced with AppArmor and seccomp) blocks it. `snap connections hello-world` lists what the snap is allowed to access through *interfaces*.

5. Learn the other two sandboxed/portable formats (no install needed).

What just happened: You are reading, not running. **Flatpak** is the common sandboxed format on desktop Linux. You add a remote such as Flathub (`flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo`) and run `flatpak install flathub <app-id>` ; apps share large "runtimes" and run in a sandbox built with bubblewrap. **AppImage** is a single executable file containing the app and its libraries: you download it, `chmod +x App.AppImage` , and run it. It needs no install and no root, but it is not sandboxed by default and does not update itself. On Ubuntu 24.04 older AppImages need the `libfuse2t64` package.

Checkpoint: `readlink -f /usr/bin/editor` points to the highest-priority editor again, and `snap list hello-world` shows the snap.

Task 8: Configure NTP, PTP, DNS, and DHCP services

1. **NTP with chrony.** Install chrony (apt removes the default `systemd-timesyncd` , as you saw in Task 2; the clean-up puts it back) and check its status.

```
sudo apt install -y chrony
systemctl status chrony --no-pager | head -5
chronyc sources -v
```

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
^- prod-ntp-3.ntp1.ps5.canonical.com  2   6    17    12  +1105us[+1105us] +/-   52ms
^* alphyn.canonical.com       2   6    17    13   -201us[ -274us] +/-   19ms
^+ ntp-ubuntu.example.net     2   6    17    13   +612us[ +539us] +/-   33ms
```

What just happened: `chronyc` is the control program for the `chronyd` daemon. In `sources` , `^` means an NTP server, `*` marks the source currently used to set the clock, `+` marks good backup sources, and `-` marks sources not being combined. *Stratum* is the distance from a reference clock (stratum 1 is attached to an atomic or GPS clock). *Reach* is an octal record of the last 8 polls (`377` means all succeeded).

2. Look at the configuration and add your own drop-in file that lets lab machines use this server.

```
grep -Ev '^\s*(#|$)' /etc/chrony/chrony.conf
echo 'allow 192.168.100.0/24' | sudo tee /etc/chrony/conf.d/lab.conf
sudo systemctl restart chrony
chronyc tracking
```

What just happened: `grep -Ev` hides comment and blank lines. Ubuntu's file has `pool` lines (a pool name returns several servers) and `confdir /etc/chrony/conf.d` , which reads your drop-in file. By default chrony is only a client; the `allow` line makes it also answer NTP requests (UDP port 123) from your lab subnet. Use your own subnet if it differs. `chronyc tracking` shows the clock's offset from true time ("System time") and its current stratum. On Rocky, the file is `/etc/chrony.conf` and the service is `chronyd` .

3. Optional test from rocky-lab: ask ubuntu-lab for the time without changing the clock.

```
# on rocky-lab
sudo chronyd -Q 'server ubuntu-lab iburst'
```

```
2026-10-10T14:40:12Z chronyd version 4.5 starting (...)
2026-10-10T14:40:12Z Disabled control of system clock
2026-10-10T14:40:17Z System clock wrong by -0.000312 seconds (ignored)
2026-10-10T14:40:17Z chronyd exiting
```

What just happened: `chronyd -Q` runs once, queries the given server, prints the difference, and exits without setting the clock. If it times out, check that ubuntu-lab's chrony shows a `*` source (an unsynchronized server will not serve time) and that no firewall blocks UDP 123.

4. **PTP.** The Precision Time Protocol syncs clocks to under a microsecond on a local network using hardware timestamps in the network card. Virtual NICs usually do not have that hardware.

TIP

Explore/simulate: Install the tools and check what your NIC supports (replace `enp0s3` with your interface from `ip -br link`):

```
sudo apt install -y linuxptp ethtool
ethtool -T enp0s3
sudo timeout 10 ptp4l -i enp0s3 -S -m
```

`ethtool -T` lists timestamping capabilities; a VM usually shows only `software-*` modes and `PTP Hardware Clock: none`. `ptp4l` is the PTP daemon: `-S` uses software timestamps, `-m` prints messages to the screen, and `timeout 10` stops it after 10 seconds. With no other PTP clock on the network it announces itself and ends up as the grandmaster (`assuming the grand master role`). On real hardware, `phc2sys` then copies the NIC's clock to the system clock. Config lives in `/etc/linuxptp/ptp4l.conf`.

5. **DNS with BIND.** Install the BIND server and look at its main file.

```
sudo apt install -y bind9 bind9-dnsutils
cat /etc/bind/named.conf
```

What just happened: The package is `bind9`; the daemon is `named` (name daemon). `named.conf` just includes three files: `named.conf.options` (global settings), `named.conf.local` (your zones), and `named.conf.default-zones`. Ubuntu's default setup is already a *caching resolver* for local clients. `systemd-resolved` keeps its own stub on `127.0.0.53`, so it does not clash with BIND on `127.0.0.1`. On Rocky the package is `bind`, the main file is `/etc/named.conf`, and zone files go in `/var/named/`.

6. Declare an authoritative zone for the made-up domain `lab.test` (`.test` is reserved for testing). Open the zones file with `sudo nano /etc/bind/named.conf.local` and add:

```
zone "lab.test" {
    type primary;
    file "/etc/bind/db.lab.test";
};
```

7. Create the zone file with the records. Replace the IPs with yours.

```
cat <<'EOF' | sudo tee /etc/bind/db.lab.test
$TTL 300
@       IN  SOA ns1.lab.test. admin.lab.test. (
                2026101001 ; serial
                3600       ; refresh
                900        ; retry
                604800     ; expire
                300 )      ; negative-cache TTL
        IN  NS    ns1.lab.test.
ns1     IN  A     192.168.100.10
web     IN  A     192.168.100.10
rocky   IN  A     192.168.100.20
www     IN  CNAME web
EOF
```

What just happened: A *zone file* lists a domain's records. `$TTL 300` means resolvers may cache answers for 300 seconds. `@` stands for the zone name itself. The `SOA` (Start of Authority) record names the primary server and the admin email (`admin.lab.test.` means `admin@lab.test`), plus timers; increase the *serial* every time you edit the file. `NS` names the zone's name server, `A` maps a name to an IPv4 address, and `CNAME` makes one name an alias for another. The trailing dot in `ns1.lab.test.` means "fully qualified"; without it BIND appends the zone name.

8. Check the syntax, restart, and query your server directly.

```
sudo named-checkconf
named-checkzone lab.test /etc/bind/db.lab.test
sudo systemctl restart named
dig @localhost www.lab.test +noall +answer
dig @localhost -x 8.8.8.8 +short
```

```
zone lab.test/IN: loaded serial 2026101001
OK
www.lab.test.        300   IN   CNAME   web.lab.test.
web.lab.test.        300   IN   A       192.168.100.10
dns.google.
```

What just happened: `named-checkconf` prints nothing when the config is valid. `named-checkzone` loads the zone and prints `OK` . Always run both before a restart, because a syntax error stops `named` from starting. `dig @localhost` sends the question straight to BIND instead of the system resolver. `+noall +answer` shows only the answer section. The second query (a reverse lookup) proves BIND also works as a recursive caching resolver for internet names.

9. **DHCP with Kea.** ISC's old `isc-dhcp-server` (dhcpd) reached end of life in 2022; its replacement is **Kea**. To avoid handing out addresses on your real lab network, you will run Kea on a *dummy* interface, a virtual NIC that is not connected to anything.

```
sudo apt install -y kea-dhcp4-server
sudo ip link add lab-dhcp0 type dummy
sudo ip addr add 10.99.0.1/24 dev lab-dhcp0
sudo ip link set lab-dhcp0 up
ip -br addr show lab-dhcp0
```

> **What just happened:** `ip link add ... type dummy` creates a fake interface named `lab-dhcp0`. You gave it the address `10.99.0.1/24` and brought it up. Nothing outside the VM can see it, so a DHCP server bound only to it is harmless. This interface disappears at reboot.

10. Back up the default Kea config and write a small one. Kea uses JSON: curly braces hold named settings, square brackets hold lists, and every comma matters.

```
sudo cp /etc/kea/kea-dhcp4.conf /etc/kea/kea-dhcp4.conf.orig
cat <<'EOF' | sudo tee /etc/kea/kea-dhcp4.conf
{
  "Dhcp4": {
    "interfaces-config": { "interfaces": [ "lab-dhcp0" ] },
    "lease-database": { "type": "memfile", "persist": true },
    "valid-lifetime": 3600,
    "subnet4": [
      {
        "id": 1,
        "subnet": "10.99.0.0/24",
        "pools": [ { "pool": "10.99.0.100 - 10.99.0.200" } ],
        "option-data": [
          { "name": "routers", "data": "10.99.0.1" },
          { "name": "domain-name-servers", "data": "10.99.0.1" },
          { "name": "domain-name", "data": "lab.test" }
        ]
      }
    ]
  }
}
EOF
sudo kea-dhcp4 -t /etc/kea/kea-dhcp4.conf
```

> **What just happened:** `interfaces` limits Kea to `lab-dhcp0`. `memfile` stores leases in a CSV file under `/var/lib/kea/`. `valid-lifetime` is the lease time in seconds. The `subnet4` entry defines the network, the *pool* of addresses to hand out, and *options* sent to clients: default gateway (`routers`), DNS servers, and domain name. `kea-dhcp4 -t` tests the file and exits; it reports the error and line if anything is wrong. Because `tee` overwrites an existing file, the file keeps its original owner and permissions, so the `_kea` service user can still read it.

11. Restart Kea and confirm it listens on UDP port 67 only on the dummy interface.

```
sudo systemctl restart kea-dhcp4-server
systemctl is-active kea-dhcp4-server
sudo ss -ulpn | grep ':67'
```

```
active
UNCONN 0  0   10.99.0.1:67   0.0.0.0:*   users:(("kea-dhcp4",pid=4321,fd=13))
```

What just happened: `ss -ulpn` lists UDP (`-u`) listening (`-l`) sockets with process names (`-p`) and numeric ports (`-n`). DHCP servers listen on UDP 67 and clients use UDP 68. For reference, the old `dhcpd` used `/etc/dhcp/dhcpd.conf` with blocks like `subnet 10.99.0.0 netmask 255.255.255.0 { range 10.99.0.100 10.99.0.200; option routers 10.99.0.1; }`.

Checkpoint: `chronyc tracking` shows a stratum number, `dig @localhost web.lab.test +short` prints your ubuntu-lab IP, and `systemctl is-active kea-dhcp4-server` prints `active`.

Task 9: Configure Nginx, Apache, Postfix (SMTP), and Dovecot (IMAP4)

Both web servers want port 80 by default, and a package's install script starts its service right away. So you will install Nginx first, move it to port 8080, and then install Apache.

1. Install Nginx and create a simple site on port 8080.

```
sudo apt install -y nginx
sudo mkdir -p /var/www/nginx-lab
echo '<h1>Hello from Nginx</h1>' | sudo tee /var/www/nginx-lab/index.html
cat <<'EOF' | sudo tee /etc/nginx/sites-available/lab
server {
    listen 8080;
    server_name ubuntu-lab;
    root /var/www/nginx-lab;
    index index.html;
}
EOF
sudo ln -s /etc/nginx/sites-available/lab /etc/nginx/sites-enabled/lab
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
curl http://localhost:8080/
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
<h1>Hello from Nginx</h1>
```

What just happened: On Ubuntu, Nginx reads every file linked into `/etc/nginx/sites-enabled/`; `sites-available/` is just storage. A `server` block is Nginx's *virtual host*: `listen` sets the port, `server_name` the host name it answers to, and `root` the folder of web files. Removing the `default` link (the file itself stays in `sites-available`) frees port 80. `nginx -t` tests the config. `reload` applies it without dropping current connections. Nginx packages from nginx.org and on Rocky use `/etc/nginx/conf.d/*.conf` instead of `sites-enabled`.

2. Install Apache and add a name-based virtual host on port 80.

```
sudo apt install -y apache2
sudo mkdir -p /var/www/apache-lab
echo '<h1>Hello from Apache site1</h1>' | sudo tee /var/www/apache-lab/index.html
cat <<'EOF' | sudo tee /etc/apache2/sites-available/site1.conf
<VirtualHost *:80>
    ServerName site1.lab.test
    DocumentRoot /var/www/apache-lab
    ErrorLog ${APACHE_LOG_DIR}/site1-error.log
    CustomLog ${APACHE_LOG_DIR}/site1-access.log combined
</VirtualHost>
EOF
sudo a2ensite site1
sudo apachectl configtest
sudo systemctl reload apache2
```

What just happened: A `<VirtualHost>` block is Apache's virtual host. `ServerName` is the host name it answers to, so many sites can share one IP and port (*name-based* virtual hosting). `DocumentRoot` is the folder of web files. `a2ensite` creates the link in `sites-enabled` (`a2dissite` removes it); `a2enmod` / `a2dismod` do the same for modules. `apachectl configtest` prints `Syntax OK` when the file is valid.

3. Test both web servers and see which process owns which port.

```
curl -H 'Host: site1.lab.test' http://localhost/
curl -s http://localhost/ | grep -o '<title>.*</title>'
sudo ss -tlnp | grep -E ':(80|8080) '
```

```
<h1>Hello from Apache site1</h1>
<title>Apache2 Ubuntu Default Page: It works</title>
LISTEN 0  511   0.0.0.0:8080   0.0.0.0:*  users:(("nginx",pid=5120,fd=6),...)
LISTEN 0  511         *:80           *:*  users:(("apache2",pid=6011,fd=4),...)
```

What just happened: `-H 'Host: site1.lab.test'` sends the host name a browser would send, so Apache picks your virtual host. Without it, Apache uses the first (default) site. `ss -tlnp` shows TCP listeners: Nginx owns 8080 and Apache owns 80. Two daemons can never listen on the same address and port.

TIP

On Rocky the Apache package and service are both called **httpd**: `sudo dnf install -y httpd`, `sudo systemctl enable --now httpd`. The main file is `/etc/httpd/conf/httpd.conf`, extra files go in `/etc/httpd/conf.d/*.conf`, there is no `a2ensite`, and firewalld blocks it until you run `sudo firewall-cmd --permanent --add-service=http` and `sudo firewall-cmd --reload`.

4. **SMTP with Postfix.** Postfix normally asks setup questions during install. Pre-answer them with `debconf-set-selections` so that it only delivers mail locally.

```
echo 'postfix postfix/main_mailer_type select Local only' | sudo debconf-set-selections
echo 'postfix postfix/mailname string ubuntu-lab' | sudo debconf-set-selections
sudo apt install -y postfix mailutils
postconf -n | grep -E 'inet_interfaces|mydestination|default_transport'
```

```
default_transport = error
inet_interfaces = loopback-only
mydestination = ubuntu-lab, $myhostname, localhost.localdomain, localhost
```

What just happened: `debconf-set-selections` stores answers before the package asks. *Local only* configures Postfix so it listens only on loopback (`inet_interfaces`), accepts mail for the names in `mydestination` (the exact list depends on your host name), and refuses to send anywhere else (`default_transport = error`). That is safe for a lab. `postconf -n` shows settings that differ from the defaults in `/etc/postfix/main.cf`. `mailutils` provides the `mail` command. Postfix is an MTA (Mail Transfer Agent); it speaks SMTP on TCP port 25.

5. Send two messages to yourself, one with `mail` and one with the classic `sendmail` interface, then read the mailbox file.

```
echo 'This is the body.' | mail -s 'Test 1 from mail' student
printf 'Subject: Test 2 from sendmail\n\nSecond body.\n' | /usr/sbin/sendmail student
sleep 2
mailq
grep '^Subject:' /var/mail/student
```

```
Mail queue is empty
Subject: Test 1 from mail
Subject: Test 2 from sendmail
```

What just happened: `mail -s` sets the subject and reads the body from standard input. `/usr/sbin/sendmail` is a command every MTA provides for compatibility; it reads a full message (headers, a blank line, then the body). `mailq` shows mail waiting to be delivered; empty means it was delivered. Postfix stored the messages in *mbox* format in `/var/mail/student` (one file per user). Delivery logs are in `journalctl -u postfix@-` or `/var/log/mail.log`.

6. **IMAP4 with Dovecot.** Install the IMAP server and check its mail location and ports.

```
sudo apt install -y dovecot-imapd
sudo doveconf mail_location
sudo ss -tlnp | grep dovecot
```

```
mail_location = mbox:~/mail:INBOX=/var/mail/%u
LISTEN 0 100  0.0.0.0:143  0.0.0.0:*  users:(("dovecot",pid=7001,fd=36))
LISTEN 0 100  0.0.0.0:993  0.0.0.0:*  users:(("dovecot",pid=7001,fd=38))
```

What just happened: Dovecot is an IMAP (and POP3) server: it lets mail clients read mail the MTA delivered. `mail_location` says the INBOX is `/var/mail/<user>` (`%u` is the user name), which is where Postfix put your messages. IMAP4 uses TCP port 143 (upgradable to TLS with STARTTLS) and IMAPS uses 993 (TLS from the start). Config files are in `/etc/dovecot/conf.d/`. Ubuntu creates a self-signed certificate for testing.

7. List your mailbox and read a message with `curl`, which speaks IMAP. Type your `student` password when asked.

```
curl -u student imap://localhost/
curl -u student 'imap://localhost/INBOX;UID=1'
```

```
* LIST (\HasNoChildren) "/" INBOX
...
Subject: Test 1 from mail
To: student@ubuntu-lab
...
This is the body.
```

What just happened: `curl -u student` logs in as `student`. The first command lists folders; the second fetches message UID 1 from INBOX. Dovecot normally refuses plain-text passwords on unencrypted connections, but it treats connections from the same machine (localhost) as secure, so this works locally.

8. Talk raw IMAP over TLS on port 993, the way a mail client does. Type each line that starts with `a` and press Enter; replace `YOURPASSWORD`.

```
openssl s_client -connect localhost:993 -quiet
```

```
a1 LOGIN student YOURPASSWORD
a2 LIST "" "*"
a3 SELECT INBOX
a4 FETCH 1:* (BODY[HEADER.FIELDS (SUBJECT)])
a5 LOGOUT
```

What just happened: `openssl s_client` opens a TLS connection and then lets you type. You will see certificate warnings because the certificate is self-signed; that is expected in a lab. Each IMAP command starts with a *tag* (`a1`, `a2` …) so the server's reply can say `a1 OK`. `SELECT` opens a folder and reports how many messages exist; `FETCH 1:*` gets the subject of every message.

Checkpoint: `curl -s http://localhost:8080/` prints the Nginx page, `curl -s -H 'Host: site1.lab.test' http://localhost/` prints the Apache site1 page, and `curl -s -u student:YOURPASSWORD 'imap://localhost/INBOX' -X 'STATUS INBOX (MESSAGES)'` reports `MESSAGES 2`.

Challenge

CHALLENGE

1. Re-enable the nginx.org repo, create `/etc/apt/preferences.d/99nginx` that pins `origin nginx.org` to priority 100, run `apt update`, and use `apt-cache policy nginx` to prove that Ubuntu's version is now the candidate. Then disable the repo and delete the pin file.
2. On rocky-lab, install `httpd`, serve a page that says "Hello from httpd", open the firewall, and fetch it from ubuntu-lab with `curl http://rocky-lab/`. Then remove it and close the firewall again.
3. Add an `A` record `mail.lab.test` to your BIND zone, increase the serial, reload with `sudo rndc reload`, and confirm with `dig @localhost mail.lab.test +short`.

Clean up

1. Stop and remove the network services on ubuntu-lab, and delete the dummy interface and lab web files.

```
sudo systemctl disable --now apache2 nginx named kea-dhcp4-server postfix dovecot
sudo apt purge -y apache2 nginx nginx-common bind9 kea-dhcp4-server \
  postfix mailutils dovecot-imapd dovecot-core linuxptp
sudo ip link delete lab-dhcp0
sudo rm -rf /var/www/nginx-lab /var/www/apache-lab /etc/bind /etc/kea
sudo rm -f /var/mail/student
rm -rf ~/mail
```

   `~/mail` is the folder Dovecot created in your home directory for its index files.

2. Put back Ubuntu's default time client. Removing chrony and installing `systemd-timesyncd` returns the VM to its original state (Lab 3.3 uses `systemd-timesyncd`).

```
sudo apt purge -y chrony
sudo apt install -y systemd-timesyncd
timedatectl
```

   `timedatectl` should show `NTP service: active` and, after a minute, `System clock synchronized: yes`.

3. Remove the language tools, test packages, snap, and the nginx.org repo and key.

```
sudo apt purge -y fortune-mod fortunes-min nodejs npm cargo rustc
sudo snap remove hello-world
sudo rm -f /etc/apt/sources.list.d/nginx.sources \
  /etc/apt/keyrings/nginx-archive-keyring.gpg
sudo apt autoremove -y
sudo apt update
```

4. Uninstall GNU Hello and delete the lab folder. Also remove any leftover global npm files.

```
cd ~/labs/2.4/hello-2.12.1 && sudo make uninstall
cd ~ && rm -rf ~/labs/2.4 ~/.npm ~/.cargo
sudo rm -rf /usr/local/lib/node_modules
sudo update-alternatives --auto editor
```

5. On rocky-lab, undo the dnf changes: remove the exclusion line, the EPEL packages, and disable `crb`.

```
# on rocky-lab
sudo sed -i '/^exclude=tree$/d' /etc/dnf/dnf.conf
sudo dnf remove -y htop epel-release
sudo dnf config-manager --set-disabled crb
sudo rm -f /tmp/tree-*.rpm
dnf repolist
```

The `sed -i` command edits the file in place and deletes (`d`) the line that is exactly `exclude=tree`. `dnf repolist` should list only `appstream`, `baseos`, and `extras`. The EPEL GPG key stays imported; you can remove it with `sudo rpm -e gpg-pubkey-3228467c` if you want.

Exam corner

- Low-level tools install one local file and do not resolve dependencies: `dpkg -i`, `rpm -ivh`. High-level package managers resolve dependencies from repositories: `apt`, `dnf` (`yum`), `zypper` (SUSE).
- Fix a broken `dpkg -i` with `apt -f install` (`--fix-broken`). `apt install ./file.deb` and `dnf install ./file.rpm` resolve dependencies for local files.
- `apt remove` keeps config files (`rc` state); `apt purge` deletes them; `apt autoremove` removes unneeded dependencies. `apt update` refreshes indexes; `apt upgrade` installs newer versions.
- Query commands: `dpkg -l`, `dpkg -L` (files in package), `dpkg -S` (owner of file); `rpm -qa`, `rpm -qi`, `rpm -ql`, `rpm -qf`; `dnf provides` finds a package by file name even if it is not installed.
- Ubuntu 24.04 repos are in `/etc/apt/sources.list.d/ubuntu.sources` (deb822); disable a block with `Enabled: no`. RHEL repos are `.repo` files in `/etc/yum.repos.d/` with `enabled=` and `gpgcheck=`; toggle with `dnf config-manager --set-enabled/--set-disabled`.
- Third-party repos need a GPG public key: on Ubuntu a keyring in `/etc/apt/keyrings/` referenced by `signed-by`; on RHEL `rpm --import` or `gpgkey=`. `rpm -K file.rpm` checks a package signature. EPEL is enabled with `dnf install epel-release`.

- Exclusions: `apt-mark hold`; `exclude=` in `/etc/dnf/dnf.conf` or a `.repo` section; `dnf --exclude=pkg` for one command; `--disableexcludes` overrides.
- `dnf history` and `dnf history undo <ID>` roll back a transaction.
- Building from source: `./configure`, `make`, `sudo make install` (into `/usr/local`); needs `build-essential` (Ubuntu) or "Development Tools" (`dnf groupinstall`) on RHEL. Not tracked by the package manager.
- `pip` (PyPI, use a venv; PEP 668 blocks system installs on new distros), `npm` (`node_modules` local, `-g` global), `cargo` (crates.io, `Cargo.toml`).
- `update-alternatives --config|--set|--auto|--display` manages `/etc/alternatives` symlinks; RHEL calls it `alternatives`.
- Sandboxed apps: snap (`snap install`, strict confinement), Flatpak (`flatpak install flathub ...`), AppImage (single executable, `chmod +x`).
- Services: BIND `named` (`named-checkconf`, `named-checkzone`, `dig @server`), chrony (`chronyc sources`, `allow`), PTP (`ptp4l`, `phc2sys`), DHCP (Kea `kea-dhcp4 -t`, old `dhcpd.conf`), Apache (`apache2` / `httpd`, `apachectl configtest`), Nginx (`nginx -t`), Postfix (SMTP port 25, `postconf`, `mailq`), Dovecot (IMAP 143, IMAPS 993).

Review questions

1. You ran `sudo dpkg -i app.deb` and it failed with "dependency problems - leaving unconfigured". Which command fixes it most directly?
   A. `sudo dpkg --purge app`
   B. `sudo apt -f install`
   C. `sudo apt update`
   D. `sudo dpkg-reconfigure app`

2. Which file and line disable the main Ubuntu repository block on Ubuntu 24.04?

3. On Rocky Linux, which command shows which package would provide `/usr/bin/dig`, even if it is not installed?
   A. `rpm -qf /usr/bin/dig`
   B. `dnf provides /usr/bin/dig`
   C. `rpm -ql bind-utils`
   D. `dnf list installed dig`

4. A team wants to keep kernel packages from being upgraded by `dnf upgrade` permanently. What do they add, and where?

5. Why does `pip3 install requests` fail on Ubuntu 24.04 outside a virtual environment, and what is the recommended fix?

6. Which command checks the syntax of an Nginx configuration before reloading it?
   A. `nginx -s check`
   B. `apachectl configtest`
   C. `nginx -t`
   D. `systemctl verify nginx`

Lab 2.5 Manage Linux Using systemd

Objective: 2.5 Given a scenario, manage Linux using systemd.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 120 minutes
VM: ubuntu-lab
Snapshot first: Yes. This lab changes the default target, the hostname, the time zone, and kernel settings (you will change them all back).

systemd is the first program the Linux kernel starts, and it starts, stops, watches, and logs almost everything else. In this lab you will write your own service, drive it through every state the exam lists, add a timer and a mount unit, work with targets, and use the systemd tools that manage the hostname, time, DNS resolution, kernel parameters, and boot speed. These are daily tasks on any modern Linux server, Ubuntu and Rocky alike.

What you'll practice

- **Units**: **services**, **timers**, **mounts**, **targets**
- **Utilities**: `hostnamectl`, `resolvectl`, `sysctl`, `systemctl`, `systemd-analyze`, `systemd-analyze blame` (the exam calls it "systemd-blame"), `systemd-resolved`, `timedatectl`
- **Unit states and commands**: `daemon-reload`, `disable`, `edit`, `enable`, `mask`, `reload`, `restart`, `start`, `status`, `stop`, `unmask`
- Reading a service's logs with `journalctl -u`

How it works

systemd and PID 1. When the kernel finishes booting, it starts one program as process ID 1. On Ubuntu and Rocky that program is systemd. Its job is to bring the system to a working state and keep it there: start services in the right order, restart them if they crash, mount filesystems, run scheduled jobs, and shut everything down cleanly.

Units. systemd manages *units*. A unit is anything systemd can start or stop, described by a small text file in INI style (sections in `[brackets]`, `Key=value` lines). The file's extension tells you the unit type:

Type	Extension	What it manages
Service	`.service`	A program or daemon (for example `ssh.service`)
Timer	`.timer`	Starts another unit on a schedule (a modern cron)
Mount	`.mount`	A mounted filesystem
Target	`.target`	A group of units, used as a synchronization point (like old runlevels)
Socket, path, device, swap, slice, scope	…	Other types you will meet later

Where unit files live. systemd reads three folders. When the same file name appears in more than one, the higher one wins:

```
/etc/systemd/system/       <- YOUR files and overrides (highest priority)
/run/systemd/system/       <- runtime units, gone after reboot
/usr/lib/systemd/system/   <- files installed by packages (do not edit)
```

Never edit files under `/usr/lib/systemd/system/`; a package update would overwrite your change. Instead, create a *drop-in* override with `systemctl edit`, or copy the file to `/etc/systemd/system/`.

Start versus enable. These two words confuse beginners, so learn them now:

- `start` / `stop` act **now**. They do not affect the next boot.
- `enable` / `disable` act **at boot**. `enable` creates a symlink from a target's `.wants/` folder to the unit, so the unit starts when that target is reached. It does not start the unit now (unless you add `--now`).

Mask. `mask` is a stronger "disable". It links the unit name to `/dev/null`, so nothing can start it, not even another unit that depends on it, until you `unmask` it.

Reload versus restart. `restart` stops the process and starts a new one: connections drop and the process ID changes. `reload` asks the running process to re-read its configuration without stopping (usually by sending it the `SIGHUP` signal), so connections stay up. Not every service supports reload. `systemctl daemon-reload` is different again: it tells **systemd itself** to re-read unit files after you edit them. It does not restart any service.

Targets. A target is a named group of units. `multi-user.target` means "a full text-mode system with networking", `graphical.target` adds a graphical login, and `rescue.target` and `emergency.target` are repair modes. The *default target* is the one systemd boots into. They replace the old SysV *runlevels* (3 = multi-user, 5 = graphical, 1 = rescue).

Dependencies and ordering. In a unit file, `Wants=` and `Requires=` say which other units must also be started (weak and strong). `After=` and `Before=` only set the **order**. In `[Install]`, `WantedBy=multi-user.target` is what `enable` uses to decide where to place the symlink.

The systemd tool family. systemd also includes small helpers that replace the old way of editing files by hand: `hostnamectl` (hostname), `timedatectl` (clock, time zone, NTP), `resolvectl` (DNS, through the `systemd-resolved` service), `journalctl` (logs), and `systemd-analyze` (boot performance). `sysctl` is not part of

systemd, but systemd applies your `sysctl` settings at boot through `systemd-sysctl.service`.

Tasks

Task 1: Explore units with systemctl

1. Create the lab folder and list running services.

```
mkdir -p ~/labs/2.5 && cd ~/labs/2.5
systemctl list-units --type=service --state=running
```

```
  UNIT                     LOAD   ACTIVE SUB     DESCRIPTION
  cron.service             loaded active running Regular background program processing
daemon
  ssh.service              loaded active running OpenBSD Secure Shell server
  systemd-journald.service loaded active running Journal Service
  systemd-resolved.service loaded active running Network Name Resolution
...
```

> **What just happened:** `systemctl` is the main command to control systemd. `list-units` shows units systemd has loaded into memory. `LOAD` says whether the unit file was read correctly, `ACTIVE` is the high-level state (`active`, `inactive`, `failed`), and `SUB` is the detailed state (`running`, `exited`, `dead`). `--type` and `--state` filter the list. Press `q` if the output opens in a pager.

2. List unit *files* and their boot setting.

```
systemctl list-unit-files --type=service | head -15
systemctl --failed
```

> **What just happened:** `list-unit-files` shows every installed unit file and its `STATE`: `enabled` (starts at boot), `disabled`, `static` (cannot be enabled itself; it is only started by other units), `masked`, or `alias`. `systemctl --failed` lists units in the `failed` state. On a healthy system it shows `0 loaded units listed`.

3. Inspect one service in detail.

```
systemctl status ssh --no-pager
systemctl cat ssh
systemctl is-active ssh
systemctl is-enabled ssh
```

```
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: active (running) since Sat 2026-10-10 09:12:44 UTC; 3h ago
TriggeredBy: ● ssh.socket
   Main PID: 912 (sshd)
      Tasks: 1 (limit: 4613)
     Memory: 6.1M
...
```

What just happened: `status` shows where the unit file is, whether it is enabled, whether it is running, its main process ID (PID), and the last few log lines. `--no-pager` prints everything at once. `systemctl cat` prints the unit file plus any drop-ins. `is-active` and `is-enabled` print a single word, which is handy in scripts. On Ubuntu 24.04, `ssh.service` may show `disabled` because it is started on demand by `ssh.socket` (`TriggeredBy:`); that is normal. On Rocky the service is called `sshd`.

Reading the status symbols: in `systemctl` output, ● (green) means active, ○ means inactive, and a red `×` means failed. `systemctl status` exit codes are useful in scripts too: `0` means running and `3` means not running.

Checkpoint: `systemctl is-active systemd-journald` prints `active`.

Task 2: Write your own service and use start, stop, enable, disable, and status

1. Create a system user for the service. Running services as a dedicated, unprivileged user limits the damage if the program is ever compromised.

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin labsvc
id labsvc
```

What just happened: `--system` creates a *system account* with a low UID and no password aging. `--no-create-home` skips the home folder, and the `nologin` shell means nobody can log in as this user.

2. Write the program the service will run. This uses a *heredoc*: everything between `<<'EOF'` and the line `EOF` is sent to `sudo tee`, which writes it to the file. The quotes around `EOF` stop the shell from expanding `$` signs inside, so the script is written exactly as shown.

```
cat <<'EOF' | sudo tee /usr/local/bin/labheartbeat.sh
#!/bin/bash
# Lab 2.5 demo: print a heartbeat line forever
INTERVAL="${INTERVAL:-5}"
count=0
while true; do
    count=$((count + 1))
    echo "heartbeat #$count from $(whoami), every ${INTERVAL}s"
    sleep "$INTERVAL"
done
EOF
sudo chmod 755 /usr/local/bin/labheartbeat.sh
```

What just happened: The script loops forever, printing one line every `INTERVAL` seconds (5 by default; `${INTERVAL:-5}` means "use `$INTERVAL` if set, otherwise 5"). Anything a service prints to standard output goes into the systemd journal, so you do not need a log file. `chmod 755` makes it executable.

3. Write the unit file. Open it with `sudo nano /etc/systemd/system/labheartbeat.service` and type the following, or use the heredoc below.

```
cat <<'EOF' | sudo tee /etc/systemd/system/labheartbeat.service
[Unit]
Description=Lab 2.5 heartbeat demo service
After=network.target

[Service]
Type=simple
User=labsvc
ExecStart=/usr/local/bin/labheartbeat.sh
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
```

What just happened: `[Unit]` holds a description and ordering: `After=network.target` means start after basic networking is up. `[Service]` says how to run it. `Type=simple` means the program in `ExecStart` *is* the service and stays in the foreground (other types: `forking` for old daemons that background themselves, `oneshot` for a command that runs and exits, `notify` for programs that tell systemd when they are ready). `User=` runs it as `labsvc` instead of root. `ExecStart` must be an absolute path. `Restart=on-failure` restarts it if it crashes or exits with an error, after `RestartSec=3` seconds. `[Install]` is used only by `enable`: `WantedBy=multi-user.target` means "start me when the normal multi-user system starts".

4. Tell systemd about the new file, check it, and look at its state before starting.

```
sudo systemctl daemon-reload
systemd-analyze verify /etc/systemd/system/labheartbeat.service
systemctl status labheartbeat --no-pager
```

```
○ labheartbeat.service - Lab 2.5 heartbeat demo service
     Loaded: loaded (/etc/systemd/system/labheartbeat.service; disabled; preset: enabled)
     Active: inactive (dead)
```

What just happened: `daemon-reload` makes systemd re-read all unit files from disk; run it every time you create or change one by hand. `systemd-analyze verify` checks a unit file for mistakes and prints nothing if it is fine. The empty circle `○` and `inactive (dead)` mean "not running"; `disabled` means it will not start at boot.

5. Start the service and observe it.

```
sudo systemctl start labheartbeat
systemctl is-active labheartbeat
systemctl is-enabled labheartbeat
systemctl status labheartbeat --no-pager
```

```
active
disabled
● labheartbeat.service - Lab 2.5 heartbeat demo service
     Loaded: loaded (/etc/systemd/system/labheartbeat.service; disabled; preset: enabled)
     Active: active (running) since Sat 2026-10-10 12:20:01 UTC; 6s ago
   Main PID: 3141 (labheartbeat.sh)
      Tasks: 2 (limit: 4613)
     CGroup: /system.slice/labheartbeat.service
             ├─3141 /bin/bash /usr/local/bin/labheartbeat.sh
             └─3150 sleep 5

Oct 10 12:20:01 ubuntu-lab labheartbeat.sh[3141]: heartbeat #1 from labsvc, every 5s
Oct 10 12:20:06 ubuntu-lab labheartbeat.sh[3141]: heartbeat #2 from labsvc, every 5s
```

What just happened: It is now `active` but still `disabled`: started now, not at boot. The filled dot `●` means running. The `CGroup` tree shows every process belonging to the service. systemd puts each service in its own *control group*, so it can track and kill all its processes, even children. The log lines prove it runs as `labsvc`.

6. Enable it for boot and see what `enable` really does.

```
sudo systemctl enable labheartbeat
ls -l /etc/systemd/system/multi-user.target.wants/ | grep labheartbeat
systemctl is-enabled labheartbeat
```

```
Created symlink /etc/systemd/system/multi-user.target.wants/labheartbeat.service →
/etc/systemd/system/labheartbeat.service.
lrwxrwxrwx 1 root root 40 Oct 10 12:21 labheartbeat.service ->
/etc/systemd/system/labheartbeat.service
enabled
```

What just happened: `enable` read `WantedBy=multi-user.target` and created a symbolic link in `multi-user.target.wants/`. At boot, when systemd reaches `multi-user.target`, it starts everything linked there. That is all "enabled" means.

7. Stop it and disable it, then use the shortcut that does both steps at once.

```
sudo systemctl stop labheartbeat
sudo systemctl disable labheartbeat
systemctl is-active labheartbeat; systemctl is-enabled labheartbeat
sudo systemctl enable --now labheartbeat
systemctl is-active labheartbeat; systemctl is-enabled labheartbeat
```

What just happened: `stop` ended the process now; `disable` removed the symlink. `enable --now` enables **and** starts in one command (`disable --now` disables and stops). The `;` runs two commands on one line.

Checkpoint: `systemctl is-active labheartbeat` prints `active` and `systemctl is-enabled labheartbeat` prints `enabled`.

Task 3: Restart on failure, read the logs, and daemon-reload after edits

1. Follow the service's log live. Open a **second terminal** (or SSH session) and run:

```
journalctl -u labheartbeat -f
```

What just happened: `journalctl` reads the systemd journal. `-u labheartbeat` shows only this unit, and `-f` (follow) keeps printing new lines, like `tail -f`. Leave it running and return to the first terminal. Other useful options: `-n 20` (last 20 lines), `--since "10 min ago"`, `-b` (this boot only), `-p err` (errors and worse).

2. Simulate a crash by killing the main process with `SIGKILL`, then watch systemd bring it back.

```
systemctl show -p MainPID --value labheartbeat
sudo kill -9 $(systemctl show -p MainPID --value labheartbeat)
sleep 5
systemctl show -p MainPID --value labheartbeat
systemctl show -p NRestarts --value labheartbeat
```

In the journal terminal you see:

```
labheartbeat.service: Main process exited, code=killed, status=9/KILL
labheartbeat.service: Failed with result 'signal'.
labheartbeat.service: Scheduled restart job, restart counter is at 1.
Started labheartbeat.service - Lab 2.5 heartbeat demo service.
labheartbeat.sh[3420]: heartbeat #1 from labsvc, every 5s
```

What just happened: `systemctl show -p MainPID --value` prints just the main PID. `$( ... )` inserts that number into the `kill` command. `kill -9` sends `SIGKILL`, which cannot be caught, so it looks like a crash. Because of `Restart=on-failure`, systemd waited `RestartSec=3` seconds and started a new process with a new PID. `NRestarts` counts automatic restarts. A normal `systemctl stop` is not a failure, so it never triggers a restart.

3. Edit the unit file directly and see why `daemon-reload` matters. Change the description with `sed`:

```
sudo sed -i 's/^Description=.*/Description=Lab 2.5 heartbeat (edited)/' \
  /etc/systemd/system/labheartbeat.service
systemctl status labheartbeat --no-pager | head -3
```

```
Warning: The unit file, source configuration file or drop-ins of labheartbeat.service
changed on disk. Run 'systemctl daemon-reload' to reload units.
● labheartbeat.service - Lab 2.5 heartbeat demo service
```

What just happened: `sed -i` edited the file in place, replacing the whole `Description=` line. systemd still has the **old** version in memory and warns you. Until you run `daemon-reload`, `start` and `restart` would use the old settings.

4. Reload systemd's view of unit files and confirm the change.

```
sudo systemctl daemon-reload
systemctl status labheartbeat --no-pager | head -3
systemctl is-active labheartbeat
```

What just happened: The new description appears and the warning is gone. Note that the service kept running with the same PID: `daemon-reload` reloads *systemd's configuration*, not the service. If you change `ExecStart=` or `User=`, you must also run `sudo systemctl restart labheartbeat` for the running process to pick it up.

5. Press `Ctrl+C` in the second terminal to stop following the journal.

Checkpoint: `systemctl show -p NRestarts --value labheartbeat` prints `1` (or more), and `journalctl -u labheartbeat -n 5 --no-pager` shows recent heartbeats.

Task 4: Override settings with systemctl edit

1. Create a drop-in override that changes the heartbeat interval.

```
sudo systemctl edit labheartbeat
```

An editor opens (nano on Ubuntu by default) showing the current unit file as comments. Type these two lines in the empty area between the two comment markers near the top, then save and exit (in nano: `Ctrl+O`, `Enter`, `Ctrl+X`):

```
[Service]
Environment=INTERVAL=2
```

What just happened: `systemctl edit` created `/etc/systemd/system/labheartbeat.service.d/override.conf` containing only your lines, and ran `daemon-reload` for you. A *drop-in* adds to or overrides individual settings of the main unit without touching it. This is the correct way to customize units installed by packages. You must repeat the section header (`[Service]`). `Environment=` sets an *environment variable* for the process. If you save an empty file, nothing is created.

2. Look at the merged result and apply it.

```
systemctl cat labheartbeat
sudo systemctl restart labheartbeat
sleep 5
journalctl -u labheartbeat -n 3 --no-pager
```

```
# /etc/systemd/system/labheartbeat.service
[Unit]
Description=Lab 2.5 heartbeat (edited)
...
# /etc/systemd/system/labheartbeat.service.d/override.conf
[Service]
Environment=INTERVAL=2

Oct 10 12:31:02 ubuntu-lab labheartbeat.sh[3511]: heartbeat #2 from labsvc, every 2s
Oct 10 12:31:04 ubuntu-lab labheartbeat.sh[3511]: heartbeat #3 from labsvc, every 2s
```

What just happened: `systemctl cat` shows the main file followed by each drop-in, with the file path as a comment. `restart` started a new process, which saw `INTERVAL=2`. To reset some settings, a drop-in can also *clear* a list setting first, for example `ExecStart=` on its own line, followed by a new `ExecStart=...`.

3. Know the other forms of `edit`.

```
systemd-delta --type=extended
```

What just happened: `systemd-delta` lists units that are overridden or extended by drop-ins, a quick audit of local changes. Other forms: `sudo systemctl edit --full <unit>` copies the whole unit to `/etc/systemd/system/` for editing; `sudo systemctl revert <unit>` deletes your drop-ins and overrides and returns to the vendor version.

Checkpoint: `cat /etc/systemd/system/labheartbeat.service.d/override.conf` shows the `Environment=INTERVAL=2` line, and new journal lines say `every 2s`.

Task 5: Reload versus restart, and mask versus unmask (with Nginx)

1. Install Nginx, a web server that supports reload, and look at how its unit defines reload.

```
sudo apt install -y nginx
systemctl cat nginx | grep -E '^Exec'
```

```
ExecStartPre=/usr/sbin/nginx -t -q -g 'daemon on; master_process on;'
ExecStart=/usr/sbin/nginx -g 'daemon on; master_process on;'
ExecReload=/usr/sbin/nginx -g 'daemon on; master_process on;' -s reload
ExecStop=-/sbin/start-stop-daemon --quiet --stop --retry QUIT/5 --pidfile /run/nginx.pid
```

What just happened: `ExecReload=` is the command systemd runs on `systemctl reload`. A unit without `ExecReload=` cannot be reloaded, and `reload` fails with "Job type reload is not applicable". `ExecStartPre=` runs a config test before every start.

2. Reload, and watch which processes change.

```
pgrep -a nginx
sudo systemctl reload nginx
pgrep -a nginx
```

```
4210 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
4211 nginx: worker process
4212 nginx: worker process
4210 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
4290 nginx: worker process
4291 nginx: worker process
```

What just happened: `pgrep -a` lists matching processes with their command lines. After `reload`, the **master** process kept its PID (4210); it re-read the configuration and started new worker processes, letting the old ones finish their current requests. No connection was dropped.

3. Now restart and compare.

```
sudo systemctl restart nginx
pgrep -a nginx
sudo systemctl reload-or-restart nginx
```

> **What just happened:** After `restart`, even the master has a new PID: everything was stopped and started again, which drops open connections. Use `reload` after configuration changes when the service supports it, and `restart` after upgrades or changes that need a fresh process. `reload-or-restart` reloads if possible and otherwise restarts.

4. Mask the service so nothing can start it.

```
sudo systemctl stop nginx
sudo systemctl mask nginx
ls -l /etc/systemd/system/nginx.service
sudo systemctl start nginx
systemctl is-enabled nginx
```

```
Created symlink /etc/systemd/system/nginx.service → /dev/null.
lrwxrwxrwx 1 root root 9 Oct 10 12:40 /etc/systemd/system/nginx.service -> /dev/null
Failed to start nginx.service: Unit nginx.service is masked.
masked
```

> **What just happened:** `mask` created a link named `nginx.service` in `/etc/systemd/system/`, which has priority over the real file in `/usr/lib/systemd/system/`, and pointed it at `/dev/null` (an empty file). systemd now sees an empty unit and refuses to start it, manually or as a dependency. Admins mask services that must never run, such as a conflicting firewall service. Because mask needs that name in `/etc/systemd/system/`, you cannot mask a unit whose real file lives there, like your `labheartbeat.service`; systemd would refuse because the file already exists.

5. Unmask and start it again.

```
sudo systemctl unmask nginx
sudo systemctl start nginx
systemctl is-active nginx
```

Checkpoint: `systemctl is-enabled nginx` prints `enabled` (not `masked`) and `curl -s localhost | grep -o 'Welcome to nginx'` prints `Welcome to nginx`.

Task 6: Schedule a job with a timer unit

A timer needs two units with the same base name: a `.service` that does the work and a `.timer` that says when.

1. Write a small report script and a `oneshot` service for it.

```
cat <<'EOF' | sudo tee /usr/local/bin/labreport.sh
#!/bin/bash
echo "$(date '+%F %T') load: $(cut -d' ' -f1-3 /proc/loadavg)" >> /var/log/labreport.log
EOF
sudo chmod 755 /usr/local/bin/labreport.sh
cat <<'EOF' | sudo tee /etc/systemd/system/labreport.service
[Unit]
Description=Lab 2.5 write a load report

[Service]
Type=oneshot
ExecStart=/usr/local/bin/labreport.sh
EOF
```

What just happened: The script appends the date and load averages to `/var/log/labreport.log`. `Type=oneshot` means the command runs, finishes, and the service goes back to inactive; systemd waits for it to finish. There is no `[Install]` section because the timer, not the boot process, will start this service.

2. Check the schedule syntax before you use it.

```
systemd-analyze calendar 'minutely'
systemd-analyze calendar 'Mon..Fri *-*-* 02:30:00'
```

```
  Original form: minutely
Normalized form: *-*-* *:*:00
    Next elapse: Sat 2026-10-10 12:46:00 UTC
       From now: 31s left
```

What just happened: `OnCalendar=` uses the format `DayOfWeek Year-Month-Day Hour:Minute:Second`, where `*` means "any". `minutely`, `hourly`, `daily`, and `weekly` are shortcuts. `systemd-analyze calendar` shows the normalized form and the next time it would fire, so you can test expressions safely. The second example means 02:30 on weekdays.

3. Write the timer unit.

```
cat <<'EOF' | sudo tee /etc/systemd/system/labreport.timer
[Unit]
Description=Run labreport every minute

[Timer]
OnCalendar=minutely
AccuracySec=1s
Persistent=true

[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now labreport.timer
```

What just happened: By default a timer starts the service with the same name (`labreport.service`); `Unit=` could name another one. `OnCalendar=` is a wall-clock schedule. `AccuracySec=1s` makes it fire close to the exact second (the default allows up to one minute of delay to save power). `Persistent=true` means that if the machine was off at the scheduled time, the job runs once soon after boot, like `anacron` . Other timer types are relative: `OnBootSec=5min` (5 minutes after boot) and `OnUnitActiveSec=1h` (one hour after it last ran). You enable the **timer**, not the service.

4. List timers and wait for a run.

```
systemctl list-timers --all | grep -E 'NEXT|labreport'
sleep 65
cat /var/log/labreport.log
journalctl -u labreport.service -n 4 --no-pager
```

```
NEXT                        LEFT LAST                        PASSED UNIT
ACTIVATES
Sat 2026-10-10 12:47:00 UTC  38s Sat 2026-10-10 12:46:00 UTC 21s ago labreport.timer
labreport.service
2026-10-10 12:46:00 load: 0.02 0.05 0.01
2026-10-10 12:47:00 load: 0.01 0.04 0.01
```

What just happened: `list-timers` shows when each timer fires next (`NEXT` , `LEFT`), when it last fired (`LAST` , `PASSED`), and which unit it starts. The log grows by one line per minute. Compared with cron, timers log every run in the journal, can catch up missed runs, and can depend on other units. You can also test the job at any time with `sudo systemctl start labreport.service` .

Checkpoint: `systemctl is-active labreport.timer` prints `active` and `/var/log/labreport.log` has at least two lines.

Task 7: Mount a filesystem with a mount unit

1. Find the required unit name for the mount point `/mnt/lab` .

```
systemd-escape -p --suffix=mount /mnt/lab
```

```
mnt-lab.mount
```

What just happened: A mount unit's file name **must** match its mount point: remove the leading `/` , turn every other `/` into `-` , and add `.mount` . So `/mnt/lab` becomes `mnt-lab.mount` and `/var/lib/data` would be `var-lib-data.mount` . `systemd-escape -p` (path mode) does the conversion, including escaping special characters. If the name does not match `Where=` , systemd refuses to load the unit.

2. Write the mount unit for a 64 MB `tmpfs` (a filesystem that lives in RAM).

```
cat <<'EOF' | sudo tee /etc/systemd/system/mnt-lab.mount
[Unit]
Description=Lab 2.5 tmpfs at /mnt/lab

[Mount]
What=tmpfs
Where=/mnt/lab
Type=tmpfs
Options=size=64M,mode=1777

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl start mnt-lab.mount
```

What just happened: The `[Mount]` section holds the same four facts as an `/etc/fstab` line: `What=` (device; for tmpfs any name works), `Where=` (mount point, created automatically if missing), `Type=` (filesystem type), and `Options=` (mount options; `size=64M` caps its size and `mode=1777` makes it writable by everyone with the sticky bit, like `/tmp`). Starting the unit performs the mount.

3. Check the mount and stop it.

```
findmnt /mnt/lab
systemctl status mnt-lab.mount --no-pager | head -4
sudo systemctl stop mnt-lab.mount
findmnt /mnt/lab || echo "not mounted"
```

```
TARGET   SOURCE FSTYPE OPTIONS
/mnt/lab tmpfs  tmpfs  rw,relatime,size=65536k,inode64
```

What just happened: `findmnt` shows the mounted filesystem with your options (`mode=1777` is missing only because it is already the default for `tmpfs`, and the kernel lists only non-default modes). `stop` unmounts it, just like `umount /mnt/lab`. To mount it at every boot, you would run `sudo systemctl enable mnt-lab.mount`.

4. See how your `/etc/fstab` lines become mount units.

```
systemctl list-units --type=mount --no-pager | head
ls /run/systemd/generator/
systemctl cat -- -.mount
```

```
# /run/systemd/generator/-.mount
# Automatically generated by systemd-fstab-generator

[Unit]
Documentation=man:fstab(5) man:systemd-fstab-generator(8)
SourcePath=/etc/fstab
...
[Mount]
What=/dev/disk/by-uuid/0b3f...
Where=/
Type=ext4
```

What just happened: At boot and on every `daemon-reload`, `systemd-fstab-generator` reads `/etc/fstab` and writes a mount unit for each line into `/run/systemd/generator/`. The root filesystem's unit is named `-.mount`. The `--` tells `systemctl` that `-.mount` is a name, not an option. That is why you must run `sudo systemctl daemon-reload` after editing `/etc/fstab`. For most mounts fstab is simpler; mount units are useful when you need systemd dependencies (for example, "start this service only after this mount is ready", with `RequiresMountsFor=`). A matching `.automount` unit mounts on first access.

Checkpoint: `systemctl is-active mnt-lab.mount` prints `inactive` after the stop, and `sudo systemctl start mnt-lab.mount && findmnt /mnt/lab` mounts it again. Stop it once more afterwards.

Task 8: Work with targets and analyze the boot

1. Look at the default target and the active targets.

```
systemctl get-default
systemctl list-units --type=target --no-pager
systemctl list-dependencies multi-user.target --no-pager | head -15
```

```
graphical.target
  UNIT                   LOAD   ACTIVE SUB    DESCRIPTION
  basic.target           loaded active active Basic System
  graphical.target       loaded active active Graphical Interface
  multi-user.target      loaded active active Multi-User System
  network-online.target  loaded active active Network is Online
  ...
```

What just happened: `get-default` shows the target systemd boots into; it is the symlink `/etc/systemd/system/default.target`. Ubuntu Server often reports `graphical.target` even without a desktop installed; with no display manager it behaves the same as `multi-user.target`. Rocky minimal reports `multi-user.target`. `list-dependencies` shows the tree of units a target pulls in.

2. Change the default target, check it, and change it back. Save the original first.

```
ORIG_TARGET=$(systemctl get-default)
echo "$ORIG_TARGET"
sudo systemctl set-default multi-user.target
ls -l /etc/systemd/system/default.target
sudo systemctl set-default "$ORIG_TARGET"
systemctl get-default
```

What just happened: `ORIG_TARGET=$(...)` stores the current default in a shell variable so you can restore it. `set-default` simply re-points the `default.target` symlink. It changes the **next** boot only. Old runlevel names still exist as aliases: `runlevel3.target` is `multi-user.target` and `runlevel5.target` is `graphical.target`.

3. Understand `isolate` (read, do not run it over SSH).

CAUTION

`sudo systemctl isolate rescue.target` switches **right now** to that target and stops every unit it does not need, including networking and SSH. Over SSH you would lock yourself out. Only use it at the VM console. To return, run `sudo systemctl isolate multi-user.target` (or `default`) at the console.

What just happened: `isolate` is like the old `init 1` or `telinit 3`: it changes the current state instead of the boot default. Only targets with `AllowIsolate=yes` can be isolated. To boot once into a target, add `systemd.unit=rescue.target` to the kernel command line in the GRUB menu (press `e` at the menu).

4. Measure the boot with `systemd-analyze`.

```
systemd-analyze
systemd-analyze blame --no-pager | head -10
systemd-analyze critical-chain
```

```
Startup finished in 2.511s (kernel) + 14.208s (userspace) = 16.720s
graphical.target reached after 14.170s in userspace.
      6.021s systemd-networkd-wait-online.service
      2.954s cloud-init-local.service
      1.402s snapd.seeded.service
       880ms dev-sda2.device
...
graphical.target @14.170s
└─multi-user.target @14.169s
  └─ssh.service @13.950s +210ms
    └─network.target @13.900s
```

What just happened: `systemd-analyze` prints total boot time split into kernel and userspace. `blame` (what the exam calls "systemd-blame") lists units sorted by how long they took to start. A slow unit is not always a problem, because many start in parallel. `critical-chain` shows the chain of units that actually delayed reaching the target: `@` is when a unit became active and `+` is how long it took to start. Start troubleshooting slow boots with this command.

Checkpoint: `systemctl get-default` prints the same value as `echo "$ORIG_TARGET"`.

Task 9: hostnamectl, timedatectl, resolvectl, and sysctl

1. **hostnamectl.** View and temporarily change the hostname.

```
hostnamectl
sudo hostnamectl set-hostname lab-temp
hostname; cat /etc/hostname
sudo hostnamectl set-hostname ubuntu-lab
hostnamectl --static
```

What just happened: `hostnamectl` shows the hostname plus OS, kernel, and virtualization details. `set-hostname` changes it immediately **and** writes `/etc/hostname`, so it survives reboot. While the name was `lab-temp`, `sudo` may warn `unable to resolve host lab-temp`, because `/etc/hosts` has no line for it; that shows why `/etc/hosts` must match the hostname. You set it back right away. On cloud images, cloud-init may reset the hostname at boot unless `preserve_hostname: true` is set in `/etc/cloud/cloud.cfg`.

2. **timedatectl.** View the clock settings, then change and restore the time zone.

```
timedatectl
ORIG_TZ=$(timedatectl show -p Timezone --value)
timedatectl list-timezones | grep -i new_york
sudo timedatectl set-timezone America/New_York
date
sudo timedatectl set-timezone "$ORIG_TZ"
timedatectl | grep 'Time zone'
```

```
               Local time: Sat 2026-10-10 12:58:20 UTC
           Universal time: Sat 2026-10-10 12:58:20 UTC
                 RTC time: Sat 2026-10-10 12:58:20
                Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

What just happened: `timedatectl` shows local time, UTC, the hardware clock (RTC), the time zone, and whether NTP sync is working. `show -p Timezone --value` prints just the zone name so you can save it. `set-timezone` changes the link `/etc/localtime`. `sudo timedatectl set-ntp true` turns automatic sync on, and `set-time "2026-10-10 13:00:00"` sets the clock by hand, but only when NTP is off. Servers usually keep UTC to make logs from different places easy to compare.

3. **systemd-resolved and resolvectl.** See how name resolution works on Ubuntu.

```
ls -l /etc/resolv.conf
grep -v '^#' /etc/resolv.conf
resolvectl status
```

```
lrwxrwxrwx 1 root root 39 Apr 23  2024 /etc/resolv.conf -> ../run/systemd/resolve/stub-
resolv.conf
nameserver 127.0.0.53
options edns0 trust-ad
search .
Global
       Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub

Link 2 (enp0s3)
    Current Scopes: DNS
         Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
Current DNS Server: 192.168.100.1
       DNS Servers: 192.168.100.1
```

What just happened: `systemd-resolved` is a local DNS service. Programs read `/etc/resolv.conf`, which is a symlink to a *stub* file that lists only `127.0.0.53`, an address where `systemd-resolved` listens. It then forwards questions to the real DNS servers, which it learns per interface from netplan/systemd-networkd or DHCP, and caches the answers. `resolvectl status` shows the real servers for each link. That is why `/etc/resolv.conf` alone does not tell you which DNS server is used on Ubuntu.

4. Query a name and look at the cache.

```
resolvectl query ubuntu.com
resolvectl statistics
sudo resolvectl flush-caches
```

```
ubuntu.com: 185.125.190.20                    -- link: enp0s3
            185.125.190.21                    -- link: enp0s3

-- Information acquired via protocol DNS in 31.8ms.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: network
```

What just happened: `resolvectl query` resolves a name through `systemd-resolved` and says which link and protocol answered. Run it twice and the second answer says `Data from: cache`. `statistics` shows cache hits and misses, and `flush-caches` empties the cache (useful after a DNS record changes). Global settings live in `/etc/systemd/resolved.conf` and drop-ins in `/etc/systemd/resolved.conf.d/`; restart `systemd-resolved` after changing them.

5. **sysctl.** Read kernel parameters at runtime.

```
sysctl -a 2>/dev/null | wc -l
sysctl vm.swappiness
cat /proc/sys/vm/swappiness
```

What just happened: `sysctl` reads and writes kernel parameters. `-a` lists all of them (more than a thousand; `2>/dev/null` hides permission warnings). Each parameter is also a file under `/proc/sys/`, with dots turned into slashes. `vm.swappiness` (default 60) controls how eagerly the kernel swaps memory to disk.

6. Change a parameter for now, then make it persistent.

```
sudo sysctl -w vm.swappiness=30
sysctl vm.swappiness
echo 'vm.swappiness = 30' | sudo tee /etc/sysctl.d/99-lab.conf
sudo sysctl --system | grep -E '99-lab|swappiness'
```

```
vm.swappiness = 30
vm.swappiness = 30
vm.swappiness = 30
* Applying /etc/sysctl.d/99-lab.conf ...
vm.swappiness = 30
```

What just happened: The first three lines come from `sysctl -w` (it echoes the new value), `sysctl vm.swappiness`, and `tee` (which echoes what it writes). `sysctl -w` changes the value in the running kernel immediately, but it is lost at reboot. To make it permanent, put it in a file in `/etc/sysctl.d/` ending in `.conf` (files are applied in name order, so `99-` wins over lower numbers). `/etc/sysctl.conf` also works. `sysctl --system` applies all files now (it first lists every file it reads, then prints every value it sets, so `grep` picks out the two lines you care about), the same way `systemd-sysctl.service` does at boot. `sysctl -p <file>` applies one file.

Checkpoint: `hostnamectl --static` prints `ubuntu-lab`, `timedatectl | grep 'Time zone'` shows your original zone, and `sysctl vm.swappiness` prints `30` (you will undo it in Clean up).

Challenge

CHALLENGE

1. Make `labheartbeat` stop automatically after 10 minutes of running: add `RuntimeMaxSec=600` with `systemctl edit`, and find in `man systemd.service` what result it reports when the limit is hit.
2. Replace the timer's `OnCalendar=` with `OnBootSec=2min` and `OnUnitActiveSec=30s` in a drop-in, and use `systemctl list-timers` to confirm the new schedule.
3. Make `labheartbeat.service` start only when `/mnt/lab` is mounted by adding `RequiresMountsFor=/mnt/lab`, then prove it: stop the mount, start the service, and see what systemd does with the mount.

Clean up

1. Stop, disable, and delete the lab service, timer, mount, and drop-in.

```
sudo systemctl disable --now labheartbeat.service labreport.timer
sudo systemctl stop mnt-lab.mount
sudo rm -rf /etc/systemd/system/labheartbeat.service \
  /etc/systemd/system/labheartbeat.service.d \
  /etc/systemd/system/labreport.service /etc/systemd/system/labreport.timer \
  /etc/systemd/system/mnt-lab.mount
sudo systemctl daemon-reload
sudo systemctl reset-failed
```

   `reset-failed` clears any leftover `failed` states from the list.

2. Remove the scripts, log file, mount point, and service user.

```
sudo rm -f /usr/local/bin/labheartbeat.sh /usr/local/bin/labreport.sh \
  /var/log/labreport.log
sudo rmdir /mnt/lab 2>/dev/null
sudo userdel labsvc
```

3. Remove Nginx.

```
sudo apt purge -y nginx nginx-common
sudo apt autoremove -y
```

4. Restore the kernel parameter.

```
sudo rm -f /etc/sysctl.d/99-lab.conf
sudo sysctl -w vm.swappiness=60
```

5. Confirm that the hostname, time zone, and default target are back to their original values, and delete the lab folder.

```
hostnamectl --static
timedatectl | grep 'Time zone'
systemctl get-default
rm -rf ~/labs/2.5
```

Exam corner

- `start` / `stop` / `restart` act now; `enable` / `disable` act at boot (symlinks in `<target>.wants/`); `enable --now` does both.
- `reload` re-reads the service's config without stopping it (needs `ExecReload=`); `restart` stops and starts (new PID); `daemon-reload` makes **systemd** re-read unit files after you edit them.
- `mask` links the unit to `/dev/null` so nothing can start it; `unmask` removes the link.
- `systemctl edit <unit>` creates a drop-in `/etc/systemd/system/<unit>.d/override.conf`; `--full` edits a full copy; `systemctl cat` shows the merged result; `systemctl revert` undoes it.
- Unit file priority: `/etc/systemd/system/` beats `/run/systemd/system/` beats `/usr/lib/systemd/system/`.
- Service essentials: `Type=simple|forking|oneshot|notify`, `ExecStart=`, `User=`, `Restart=on-failure`, `WantedBy=multi-user.target`.
- Timers: a `.timer` starts the `.service` of the same name; `OnCalendar=` (wall clock), `OnBootSec=` / `OnUnitActiveSec=` (relative), `Persistent=true`; list with `systemctl list-timers`.
- Mount unit names must match the path: `/mnt/lab` is `mnt-lab.mount` (`systemd-escape -p --suffix=mount`); fstab lines become mount units via `systemd-fstab-generator`.
- Targets: `systemctl get-default`, `set-default`, `isolate`; `multi-user.target` = runlevel 3, `graphical.target` = 5, `rescue.target` = 1.
- `systemd-analyze` (boot time), `systemd-analyze blame` (slowest units), `systemd-analyze critical-chain`, `systemd-analyze verify`, `systemd-analyze calendar`.
- `hostnamectl set-hostname`, `timedatectl set-timezone` / `set-ntp`, `resolvectl status` / `query` / `flush-caches`; `/etc/resolv.conf` points to the `systemd-resolved` stub at `127.0.0.53`.
- `sysctl -w key=value` is runtime only; persist in `/etc/sysctl.d/*.conf` and apply with `sysctl --system` or `sysctl -p`.

Review questions

1. You edited `/etc/systemd/system/app.service` by hand. Which command must you run before `systemctl restart app` uses the new settings?
   A. `systemctl reload app`
   B. `systemctl daemon-reload`
   C. `systemctl reenable app`
   D. `systemd-analyze verify app`

2. What is the difference between `systemctl disable nginx` and `systemctl mask nginx`?

3. What must the unit file be named to mount a filesystem at `/srv/backup`?
   A. `backup.mount`
   B. `srv-backup.mount`
   C. `srv/backup.mount`
   D. `-srv-backup.mount`

4. Which command shows the units that took the longest to start during boot?

5. An admin ran `sudo sysctl -w net.ipv4.ip_forward=1`, but forwarding is off again after a reboot. How do they make it permanent?

6. On Ubuntu, `/etc/resolv.conf` lists only `nameserver 127.0.0.53`. Which command shows the DNS servers actually being used?
   A. `cat /etc/hosts`
   B. `hostnamectl`
   C. `resolvectl status`
   D. `timedatectl`

Lab 2.6 Manage Applications in Containers

Objective: 2.6 Given a scenario, manage applications in a container on a Linux server.
Domain: 2.0 Services and User Management (20% of the exam)
Time: about 150 minutes
VM: ubuntu-lab (most tasks) and rocky-lab (Task 8)
Snapshot first: Yes. This lab installs container engines, adds you to the `docker` group, and creates virtual networks on your NIC.

Containers package an application with everything it needs and run it isolated from the rest of the server. In this lab you will install Docker and Podman, see the runtimes underneath them (containerd and runC), pull and build images, run and inspect containers, keep data in volumes, connect containers with different network types, and learn why privileged and rootful containers are risky. On rocky-lab you will meet SELinux and fix a blocked bind mount with `:Z`.

What you'll practice

- **Runtimes**: **runC**, **Podman**, **containerd**, **Docker**
- **Image operations**: pull, build (**Dockerfile**: `ENTRYPOINT`, `CMD`, `USER`, `FROM`), pruning, tags, layers
- **Container operations**: logs, map volumes, start/stop, inspect, delete (`rm`), run, exec, pruning, tags, environment variables
- **Volumes**: create, map, prune, **SELinux context** (`:Z`), **overlay**
- **Networks**: `network create`, **port mapping**, prune; types **macvlan**, **ipvlan**, **host**, **bridge**, **overlay**, **none**
- **Privileged vs. unprivileged** containers (and rootless vs. rootful)

How it works

Container versus virtual machine. A virtual machine runs a whole separate kernel on emulated hardware. A *container* is just a normal Linux process (or group of processes) that the kernel isolates with two features:

- **Namespaces** give the process its own view of the system: its own process list (PID namespace), network interfaces (network namespace), filesystem tree (mount namespace), hostname, and users.
- **Control groups (cgroups)** limit how much CPU, memory, and I/O it can use.

All containers share the host's kernel. That makes them start in milliseconds and use little memory, but it also means a kernel bug or a badly configured container can affect the host.

Images and layers. An *image* is a read-only template: a filesystem plus settings such as which command to run. It is built in *layers*. Each instruction in a `Dockerfile` that changes files adds a new layer on top of the previous ones. Layers are shared: ten images based on `alpine` store the alpine layer only once. An image is named

`registry/repository:tag`, for example `docker.io/library/nginx:alpine`. The *tag* (after the colon) is a version label; if you leave it out, it means `latest`.

Overlay filesystem. When you run a container, the engine stacks the image's read-only layers and adds one thin writable layer on top, using the kernel's **OverlayFS**. Older Docker versions call their storage driver `overlay2`; Docker 29 and later (the version in Ubuntu 24.04's updates) keep images in containerd's `overlayfs` *snapshotter* instead, which uses the same kernel feature:

```
merged/   <- what the container sees as "/"
  |
upperdir  <- writable layer (this container only; deleted with the container)
---------
lowerdir  <- image layer 3  (read-only, shared)
lowerdir  <- image layer 2  (read-only, shared)
lowerdir  <- image layer 1  (read-only, shared)
```

If the container changes a file from a lower layer, OverlayFS first copies it up into the upper layer ("copy-on-write"). When you delete the container, its upper layer is deleted too. That is why data that must survive goes into a **volume**: a folder managed by the engine (named volume) or a host folder (bind mount), mounted into the container and stored outside the overlay.

The runtime stack. "Docker" is really several programs:

```
docker (CLI) --> dockerd --> containerd --> containerd-shim --> runc --> your process
                 (API)       (image and     (keeps container    (creates namespaces
                              container      running if          and cgroups, then
                              lifecycle)     dockerd restarts)   starts the process)

podman (CLI, no daemon) ------------------> conmon ---------> crun or runc --> process
```

runC is the low-level *OCI runtime*: it reads a container spec and asks the kernel to create the namespaces and cgroups. **containerd** is a daemon that manages images and container lifecycles and calls runC; Kubernetes also uses it directly. **Docker** adds a friendly CLI, image building, networking, and volumes on top. **Podman** is a Red Hat-led, Docker-compatible tool with **no central daemon**. It can run *rootless*, as a normal user, which is the default on Rocky. The OCI (Open Container Initiative) standard means images built by one tool run in the others.

Networking. Each container usually gets its own network namespace. Docker's network *drivers* decide how it connects:

Driver	What it does
`bridge`	Default. A virtual switch (`docker0`) on the host with private IPs (172.17.0.0/16); reach containers from outside with **port mapping** (`-p 8080:80`).
`host`	No isolation: the container uses the host's network stack directly.
`none`	Only a loopback interface. No network at all.
`macvlan`	Each container gets its own MAC address and an IP on the physical LAN.
`ipvlan`	Like macvlan, but containers share the host NIC's MAC address.
`overlay`	Spans several hosts (needs Docker Swarm or another orchestrator).

Privileged versus unprivileged. By default a container is *unprivileged*: even if it runs as root inside, the kernel drops most root powers (Linux *capabilities*), blocks dangerous system calls (seccomp), and applies an AppArmor or SELinux profile. `--privileged` removes nearly all of those protections and gives access to host devices. Separately, *rootful* means the engine runs as root on the host (Docker by default), while *rootless* means it runs as your normal user (Podman by default), so root inside the container maps to an unprivileged user outside.

Tasks

Task 1: Install Docker and Podman, and look at the runtime stack

1. Install Docker (Ubuntu's package is `docker.io`) and Podman.

```
mkdir -p ~/labs/2.6 && cd ~/labs/2.6
sudo apt update
sudo apt install -y docker.io podman
sudo apt install -y docker-buildx
systemctl is-active docker containerd
```

> **What just happened:** `docker.io` pulls in `containerd` and `runc` as dependencies and starts the `docker` and `containerd` services. `podman` has no service to start. `docker-buildx` (from the `universe` component) adds the modern image builder (BuildKit) that `docker build` uses. On Rocky, Docker is not in the repos; Podman is the supported engine (`sudo dnf install -y podman`).

2. Allow your user to run `docker` without `sudo`, and understand the risk.

```
sudo usermod -aG docker student
newgrp docker
docker run --rm hello-world
```

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

What just happened: `usermod -aG docker student` adds (`-a`, append) `student` to the `docker` group (`-G`). Group changes apply to new logins, so `newgrp docker` starts a new shell that already has the group (or log out and back in). `docker run --rm hello-world` downloaded a tiny test image, ran it, and removed the container afterwards (`--rm`).

TIP

Rootless Podman needs *user namespaces*. Ubuntu 24.04 restricts them for unprivileged programs (`sysctl kernel.apparmor_restrict_unprivileged_userns` prints `1`) unless an AppArmor profile allows it. The `apparmor` package ships small profiles for `podman`, `crun`, `runc`, and `slirp4netns` in `/etc/apparmor.d/` that do exactly that, so rootless Podman works out of the box. If you ever see errors such as `cannot clone: Permission denied` or `newuidmap` failures, check that those profiles exist (`ls /etc/apparmor.d/podman`) and that `student` has a line in `/etc/subuid` (Task 7).

CAUTION

Membership in the `docker` group is equal to root access. Anyone in it can run `docker run -v /:/host -it alpine chroot /host` and get a root shell on the whole host filesystem. Only add trusted administrators to this group, or use rootless Podman instead.

3. Look at the layers underneath Docker.

```
docker info | grep -iE \
  'version|storage driver|cgroup|runtime'
runc --version
containerd --version
sudo ctr namespaces ls
```

```
 Version:    29.1.3
...
 Server Version: 29.1.3
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Cgroup Driver: systemd
 Cgroup Version: 2
 Runtimes: io.containerd.runc.v2 runc
 Default Runtime: runc
 containerd version: 2.2.1
 runc version: 1.3.4-0ubuntu1~24.04.1
...
runc version 1.3.4-0ubuntu1~24.04.1
spec: 1.2.1
...
containerd github.com/containerd/containerd/v2 2.2.1
NAME LABELS
moby
```

What just happened: `docker info` shows that Docker stores images through containerd's `overlayfs` snapshotter (on older Docker versions this line says `overlay2`) and starts containers with `runc` through containerd. `ctr` is containerd's own low-level CLI. Docker keeps its objects in a containerd *namespace* called `moby` (Docker's open-source project name). Kubernetes would use a namespace called `k8s.io`. Your version numbers will differ.

4. Check Podman's runtime.

```
podman info --format '{{.Host.OCIRuntime.Name}} {{.Host.OCIRuntime.Version}}'
podman info --format '{{.Store.GraphDriverName}} {{.Store.GraphRoot}}'
```

What just happened: `--format` with `{{ }}` (a Go template) extracts single fields. Podman uses `crun` (a fast runC alternative written in C) or `runc`. Because you ran it as `student`, its storage is in your home folder (`~/.local/share/containers/storage`), not in `/var/lib`: this is rootless Podman.

Checkpoint: `docker ps` runs without `sudo` and without a permission error, and `id` lists the `docker` group.

Task 2: Pull images, tag them, and look at layers

1. Pull two images and list them.

```
docker pull alpine:3.20
docker pull nginx:alpine
docker image ls
```

```
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:3.20          9f2c4e7f2a93       12.2MB         3.71MB
hello-world:latest   d2c94e258dcb       20.3kB         3.96kB
nginx:alpine         1ff4bb4faebc       94.4MB         27.2MB
```

What just happened: `docker pull` downloads an image from a *registry*; the default is Docker Hub (`docker.io`). `alpine:3.20` is a tiny Linux distribution, tag `3.20`. `nginx:alpine` is the Nginx web server built on alpine. Each layer is downloaded separately and shown as `Pull complete`. In the list, `IMAGE` is `repository:tag`, `ID` is the start of a SHA-256 hash that identifies the image, `CONTENT SIZE` is the compressed download size, and `DISK USAGE` is the space it takes once unpacked. (Older Docker versions show separate `REPOSITORY`, `TAG`, `IMAGE ID`, `CREATED`, and `SIZE` columns instead; the information is the same.)

2. Add your own tags and see that a tag is just a name.

```
docker tag alpine:3.20 mylab/base:1.0
docker tag alpine:3.20 mylab/base:latest
docker image ls | grep -E 'alpine|mylab'
docker rmi mylab/base:latest
```

```
alpine:3.20          9f2c4e7f2a93       12.2MB         3.71MB
mylab/base:1.0       9f2c4e7f2a93       12.2MB         3.71MB
mylab/base:latest    9f2c4e7f2a93       12.2MB         3.71MB
Untagged: mylab/base:latest
```

What just happened: `docker tag` gives an existing image an additional name. All three names have the same `ID`, so no extra disk space was used. `docker rmi` (remove image) on one of several tags only removes that tag (`Untagged:`); the image data is deleted only when its last tag goes. To push to a private registry, you tag with the registry's address first, for example `registry.example.com:5000/lab/base:1.0`, then `docker push` it.

3. Look at the layers of the Nginx image.

```
docker history nginx:alpine
docker image inspect --format '{{len .RootFS.Layers}} layers' nginx:alpine
docker image inspect \
  --format '{{json .Config.Cmd}} {{json .Config.ExposedPorts}}' nginx:alpine
```

```
IMAGE          CREATED       CREATED BY                                      SIZE
COMMENT
1ff4bb4faebc   2 weeks ago   RUN /bin/sh -c set -x     && apkArch="$(cat …   52.4MB
buildkit.dockerfile.v0
<missing>      2 weeks ago   CMD ["nginx" "-g" "daemon off;"]                0B
buildkit.dockerfile.v0
<missing>      2 weeks ago   STOPSIGNAL SIGQUIT                              0B
buildkit.dockerfile.v0
<missing>      2 weeks ago   EXPOSE map[80/tcp:{}]                           0B
buildkit.dockerfile.v0
<missing>      2 weeks ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B
buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY docker-entrypoint.sh / # buildkit          8.19kB
buildkit.dockerfile.v0
...
<missing>      3 weeks ago   CMD ["/bin/sh"]                                 0B
buildkit.dockerfile.v0
<missing>      3 weeks ago   ADD alpine-minirootfs-3.24.2-x86_64.tar.gz /…   9.08MB
buildkit.dockerfile.v0
8 layers
["nginx","-g","daemon off;"] {"80/tcp":{}}
```

What just happened: `docker history` lists the Dockerfile instructions that built the image, newest first. Lines with a size (`RUN` , `ADD` , `COPY`) created filesystem layers; lines with `0B` only changed settings. The bottom line is the Alpine base layer. (`nginx:alpine` is built on the newest Alpine release, so it is not necessarily the same layer as your `alpine:3.20` ; images that share the exact same base layer store it only once.) `<missing>` just means those intermediate steps were built on another machine. `docker image inspect` prints the image's full JSON; `--format` picks fields such as the layer list and the default command.

Checkpoint: `docker image ls mylab/base` shows only the tag `1.0` .

Task 3: Run, inspect, and manage containers

1. Run Nginx in the background, with a port mapping and an environment variable.

```
docker run -d --name web -p 8080:80 -e LAB_COLOR=blue nginx:alpine
docker ps
curl -s http://localhost:8080 | grep -o '<title>.*</title>'
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS
NAMES
5c1e2d7a9b01   nginx:alpine   "/docker-entrypoint.…"   3 seconds ago   Up 2 seconds
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   web
<title>Welcome to nginx!</title>
```

What just happened: `docker run` creates and starts a container from an image. `-d` (detached) runs it in the background. `--name web` gives it a name to use instead of the ID. `-p 8080:80` is **port mapping**: connections to port 8080 on the host go to port 80 inside the container (format `host:container`). `-e LAB_COLOR=blue` sets an **environment variable** inside the container, the usual way to pass settings such as database passwords or modes. `docker ps` lists running containers.

2. Read the container's logs.

```
docker logs web
docker logs --tail 3 web
```

Now follow the log live with `docker logs -f web`, open a second terminal, run `curl -s localhost:8080 > /dev/null` a few times, watch the new lines appear, and press `Ctrl+C` in the first terminal.

What just happened: `docker logs` shows what the container's main process wrote to standard output and standard error. That is why container images send logs to stdout instead of files. `--tail 3` shows the last 3 lines, `-f` follows new output, and `--since 10m` shows the last 10 minutes. Each `curl` produces one access-log line.

3. Run commands inside the running container with `exec`.

```
docker exec web nginx -v
docker exec -it web sh
```

Inside the container's shell, run:

```
echo "$LAB_COLOR"
hostname
ps
cat /etc/os-release | head -2
exit
```

What just happened: `docker exec` starts an extra process in an existing container. `-i` keeps standard input open and `-t` gives you a terminal, together `-it` for an interactive shell. Alpine has `sh`, not `bash`. Inside, `ps` shows only the container's own processes, with Nginx as PID 1, because of the PID namespace. The hostname is the container ID. `exit` leaves the shell; the container keeps running because its main process is Nginx, not your shell.

4. Inspect the container's configuration and state.

```
docker inspect web | head -20
docker inspect --format '{{.State.Status}} pid={{.State.Pid}}' web
docker inspect \
  --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web
docker inspect --format '{{json .Config.Env}}' web
docker port web
```

```
running pid=5873
172.17.0.2
["LAB_COLOR=blue","PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin","NGINX
_VERSION=...",...]
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

What just happened: `docker inspect` prints everything Docker knows about the container as JSON. `--format` picks out fields: its state, its host PID (the container is just a process to the host, visible in `ps -p 5873`), its IP on each network it is attached to (here only the default bridge; `range` loops over the networks), and its environment variables (yours plus those set by `ENV` in the image). `docker port` shows its port mappings.

5. Stop, start, and delete the container.

```
docker stop web
docker ps -a --filter name=web
docker start web
docker restart web
docker rm web
docker rm -f web
docker ps -a --filter name=web
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS
PORTS   NAMES
5c1e2d7a9b01   nginx:alpine   "/docker-entrypoint.…"   6 minutes ago   Exited (0) 2 seconds
ago           web
Error response from daemon: cannot remove container "web": container is running: stop the
container before removing or force remove
```

What just happened: `docker stop` sends `SIGTERM`, waits 10 seconds, then sends `SIGKILL`. A stopped container still exists (`docker ps -a` shows all containers, including `Exited` ones), with its writable layer intact. `docker start` runs it again with the same settings. `docker rm` deletes a container but refuses while it is running; `rm -f` (force) stops and deletes it. Podman uses exactly the same commands (`podman ps`, `podman logs`, and so on).

Checkpoint: `docker ps -a` no longer lists a container named `web`.

Task 4: Build your own image with a Dockerfile

1. Create a small application: a shell script that greets someone.

```
mkdir -p ~/labs/2.6/greeter && cd ~/labs/2.6/greeter
cat <<'EOF' > greet.sh
#!/bin/sh
echo "Hello, $1! I am $(whoami) (uid $(id -u)) in container $(hostname)."
echo "GREETING_STYLE is: ${GREETING_STYLE:-unset}"
EOF
```

What just happened: This uses a *heredoc*: everything between `<<'EOF'` and the line `EOF` is written to `greet.sh`. The quotes around `EOF` stop your shell from expanding `$1` and `$(whoami)` now; they must run later, inside the container. The script prints its first argument (`$1`), the user it runs as, and an environment variable.

2. Write the `Dockerfile`. Open it with `nano Dockerfile` and type:

```
FROM alpine:3.20
LABEL maintainer="student@ubuntu-lab"
ENV GREETING_STYLE=friendly
RUN adduser -D -u 10001 appuser
COPY greet.sh /usr/local/bin/greet.sh
RUN chmod 755 /usr/local/bin/greet.sh
USER appuser
ENTRYPOINT ["/usr/local/bin/greet.sh"]
CMD ["World"]
```

What just happened: A `Dockerfile` is a recipe, read top to bottom. `FROM` names the base image every image starts from. `LABEL` adds metadata. `ENV` sets a default environment variable. `RUN` executes a command at build time (here, creating a user with `adduser -D`, no password). `COPY` copies a file from the *build context* (this folder) into the image. `USER appuser` makes everything after it, including the running container, use that non-root user, a basic security practice. `ENTRYPOINT` is the program that always runs. `CMD` supplies default *arguments* to it, which users can replace. The JSON form `["..."]` runs the program directly without a shell.

3. Build the image and give it a tag.

```
docker build -t greeter:1.0 .
docker image ls greeter
docker history greeter:1.0 | head -8
```

```
[+] Building 3.1s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => [1/4] FROM docker.io/library/alpine:3.20
 => [2/4] RUN adduser -D -u 10001 appuser
 => [3/4] COPY greet.sh /usr/local/bin/greet.sh
 => [4/4] RUN chmod 755 /usr/local/bin/greet.sh
 => => naming to docker.io/library/greeter:1.0
```

What just happened: `docker build` reads the `Dockerfile` in the folder given as the last argument (`.` means "this folder", the build context). `-t greeter:1.0` tags the result. Each `RUN` and `COPY` step became a layer on top of alpine's layer. `docker history` shows your new layers above the alpine base.

4. Run it and see how `ENTRYPOINT` and `CMD` work together.

```
docker run --rm greeter:1.0
docker run --rm greeter:1.0 'Linux+ student'
docker run --rm -e GREETING_STYLE=formal greeter:1.0 Admin
docker run --rm --entrypoint /bin/sh greeter:1.0 -c 'id; ls -l /usr/local/bin'
```

```
Hello, World! I am appuser (uid 10001) in container 8d1f2c0b4e77.
GREETING_STYLE is: friendly
Hello, Linux+ student! I am appuser (uid 10001) in container 61a9b7c3d2e0.
GREETING_STYLE is: friendly
Hello, Admin! I am appuser (uid 10001) in container 0c4e1a2b3f55.
GREETING_STYLE is: formal
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
total 4
-rwxr-xr-x    1 root     root           135 Oct 10 13:20 greet.sh
```

What just happened: With no arguments, Docker ran `ENTRYPOINT` + `CMD`: `greet.sh World`. Anything you type after the image name **replaces** `CMD`, so the second run greeted `Linux+ student`. `-e` overrode the `ENV` default. To replace the `ENTRYPOINT` itself, you need `--entrypoint`; here you ran a shell instead. Rule of thumb: put the fixed program in `ENTRYPOINT` and default options in `CMD`. If an image has only `CMD` (like Nginx's `nginx -g 'daemon off;'`), the arguments you type replace the whole command.

5. Change the app, rebuild, and watch the layer cache and a *dangling* image.

```
sed -i 's/Hello/Greetings/' greet.sh
docker build -t greeter:1.1 -t greeter:latest .
docker run --rm greeter
docker tag greeter:1.1 greeter:1.0
docker image ls --filter dangling=true
```

```
 => CACHED [2/4] RUN adduser -D -u 10001 appuser
 => [3/4] COPY greet.sh /usr/local/bin/greet.sh
...
Greetings, World! I am appuser (uid 10001) in container 2b8c7d1e9f03.
IMAGE        ID             DISK USAGE   CONTENT SIZE   EXTRA
<untagged>   a7d3e5f1c2b4       12.2MB         3.64MB
```

What just happened: The `RUN adduser` step was `CACHED` because nothing above it changed; only the steps from the changed `COPY` onward were rebuilt. Order Dockerfile steps from least to most often changed to make builds fast. You can give several tags with several `-t` options. `docker run --rm greeter` used the tag `latest` by default. When you moved the tag `1.0` to the new image, the old image lost its last tag and became *dangling* (shown as `<untagged>`, or `<none>:<none>` on older Docker versions). Dangling images are what `docker image prune` cleans up in Task 9.

Checkpoint: `docker run --rm greeter:1.0` prints a line starting with `Greetings,` and `docker image ls greeter` shows tags `1.0`, `1.1`, and `latest` with the same image ID.

Task 5: Keep data with volumes and bind mounts, and see the overlay filesystem

1. Create a named volume and prove that data in it survives containers.

```
docker volume create labdata
docker run --rm -v labdata:/data alpine:3.20 \
  sh -c 'echo "saved at $(date)" > /data/note.txt'
docker run --rm -v labdata:/data alpine:3.20 cat /data/note.txt
docker volume ls
docker volume inspect labdata
```

```
saved at Sat Oct 10 13:31:07 UTC 2026
[
    {
        "Driver": "local",
        "Mountpoint": "/var/lib/docker/volumes/labdata/_data",
        "Name": "labdata",
        ...
    }
]
```

What just happened: `docker volume create` made a *named volume*, a folder Docker manages under `/var/lib/docker/volumes/`. `-v labdata:/data` **maps** that volume to `/data` inside the container (format `source:destination`). The first container wrote a file and was removed (`--rm`); the second, brand-new container still read it. Data written anywhere else in a container lives in its overlay upper layer and is lost when the container is deleted.

2. Use a *bind mount*: map a host folder into a container.

```
mkdir -p ~/labs/2.6/site
echo '<h1>Version 1</h1>' > ~/labs/2.6/site/index.html
docker run -d --name site -p 8081:80 \
  -v ~/labs/2.6/site:/usr/share/nginx/html:ro nginx:alpine
curl -s localhost:8081
echo '<h1>Version 2</h1>' > ~/labs/2.6/site/index.html
curl -s localhost:8081
docker exec site touch /usr/share/nginx/html/test
```

```
<h1>Version 1</h1>
<h1>Version 2</h1>
touch: /usr/share/nginx/html/test: Read-only file system
```

What just happened: When the source starts with `/` (or `~`, which the shell expands to `/home/student`), `-v` makes a *bind mount* of that host folder. Changes on the host appear in the container instantly, which is handy for configuration and development. `:ro` makes it read-only inside the container. The longer, clearer syntax is `--mount type=bind,source=/path,target=/path,readonly`. Use named volumes for application data (Docker manages them) and bind mounts when the host must own the files.

3. See the overlay filesystem of a running container.

```
docker info --format '{{.Driver}}'
docker exec site sh -c 'echo hi > /tmp/inside.txt'
docker diff site
PID=$(docker inspect --format '{{.State.Pid}}' site)
sudo head -1 /proc/$PID/mountinfo | tr ',' '\n' | grep -E 'overlay|dir='
UPPER=$(sudo grep -m1 -o 'upperdir=[^,]*' /proc/$PID/mountinfo | cut -d= -f2)
echo "$UPPER"
sudo ls "$UPPER/tmp"
```

```
overlayfs
C /run
A /run/nginx.pid
C /tmp
A /tmp/inside.txt
C /var
...
relatime - overlay overlay rw
lowerdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/34/fs:...
upperdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/35/fs
workdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/35/work
/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/35/fs
inside.txt
```

What just happened: `docker info --format '{{.Driver}}'` prints the storage driver (`overlayfs` here; `overlay2` on older Docker). `docker diff` lists what changed in the container's writable layer: `A` added, `C` changed, `D` deleted. Next you used the container's host PID to read `/proc/<PID>/mountinfo` , the list of mounts *as the container sees them*. Its first line is the container's `/` , a real OverlayFS mount: `lowerdir=` is the colon-separated list of read-only image layers, `upperdir=` is this container's writable layer, and `workdir=` is scratch space OverlayFS needs. `tr ',' '\n'` splits the long option list onto separate lines so you can read it. `grep -o` then cut out just the `upperdir` path, and `ls` proved that your new file physically lives there on the host. Delete the container and that folder goes too. (With the older `overlay2` driver the paths are under `/var/lib/docker/overlay2/` instead, but the idea is identical.)

TIP

Podman supports an *overlay volume* with `-v /host/dir:/dir:O` : the container sees the host folder but its writes go to a temporary upper layer and are thrown away when it stops, so the host folder is never changed.

Checkpoint: `docker run --rm -v labdata:/data alpine:3.20 cat /data/note.txt` still prints the saved line, and `curl -s localhost:8081` prints `Version 2` .

Task 6: Container networks

1. List the default networks and look at the default bridge.

```
docker network ls
ip -br addr show docker0
docker network inspect bridge --format '{{range .IPAM.Config}}{{.Subnet}}{{end}}'
```

```
NETWORK ID     NAME      DRIVER    SCOPE
3b1f0a9c2d11   bridge    bridge    local
8e2d4c6a0f13   host      host      local
c7a9e1b3d5f2   none      null      local
docker0          UP             172.17.0.1/16
172.17.0.0/16
```

What just happened: Docker creates three networks at install: `bridge` (the default, using the host's `docker0` virtual switch at `172.17.0.1`), `host`, and `none`. Containers on the bridge get private `172.17.x.x` addresses and reach the internet through NAT on the host.

2. Show that the default bridge has no name resolution, but a user-defined bridge does.

```
docker run --rm alpine:3.20 ping -c1 -W2 site
docker network create labnet
docker run -d --name web2 --network labnet nginx:alpine
docker run --rm --network labnet alpine:3.20 wget -qO- http://web2 \
  | grep -o '<title>.*</title>'
docker network inspect labnet \
  --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{end}}'
```

```
ping: bad address 'site'
<title>Welcome to nginx!</title>
web2 172.18.0.2/16
```

What just happened: On the default `bridge`, containers can only reach each other by IP. `docker network create labnet` made a *user-defined bridge* (driver `bridge` is the default, with its own subnet such as `172.18.0.0/16`). On user-defined networks, Docker runs an internal DNS server, so containers find each other **by name**. This is how an app container finds its database container. Containers on different networks are isolated from each other.

3. Practice port mapping options.

```
docker run -d --name local-only -p 127.0.0.1:8082:80 nginx:alpine
docker run -d --name random-port -P nginx:alpine
docker port local-only
docker port random-port
sudo ss -tlnp | grep -E '808[0-9]|docker-proxy' | head
```

What just happened: `-p 127.0.0.1:8082:80` publishes the port only on the host's loopback address, so other machines cannot reach it. `-P` (capital) publishes every port the image `EXPOSE`s on a random high host port. `ss` shows the host side listening, often through the `docker-proxy` helper. Docker also writes NAT rules into the host firewall; be aware that published ports can bypass `ufw` rules.

4. Use the `host` and `none` network types.

```
docker run -d --name hostweb --network host nginx:alpine
curl -s http://localhost:80 | grep -o '<title>.*</title>'
docker run --rm --network host alpine:3.20 ip -br addr | head -3
docker run --rm --network none alpine:3.20 ip -br addr
docker rm -f hostweb
```

```
<title>Welcome to nginx!</title>
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.10/24
docker0          UP             172.17.0.1/16
lo               UNKNOWN        127.0.0.1/8 ::1/128
```

What just happened: With `--network host`, the container shares the host's network namespace: Nginx listened directly on host port 80 with no `-p` (port mapping is ignored in this mode), and `ip addr` shows the host's real interfaces. It is the fastest option but gives no network isolation, and two containers cannot both use port 80. With `--network none`, the container has only loopback, ideal for batch jobs that must not touch the network.

5. Create **macvlan** and **ipvlan** networks on your VM's NIC. First find the NIC name and your LAN details.

```
NIC=$(ip route show default | awk '{print $5; exit}')
echo "$NIC"
ip route show default
```

Replace the subnet and gateway below with your lab network's values. The small `--ip-range` keeps Docker away from addresses your DHCP server or other VMs use.

```
docker network create -d macvlan \
  --subnet=192.168.100.0/24 --gateway=192.168.100.1 \
  --ip-range=192.168.100.240/29 -o parent="$NIC" labmacvlan
docker network inspect labmacvlan --format '{{.Driver}} parent={{.Options.parent}}'
docker run --rm --network labmacvlan alpine:3.20 ip -br addr show eth0
docker network rm labmacvlan
```

```
macvlan parent=enp0s3
eth0@if2         UP             192.168.100.240/24
```

What just happened: `-d macvlan` creates a network where each container gets its **own MAC address** and an IP directly on your LAN, as if it were a separate physical machine plugged into the switch. `-o parent=` names the host NIC it rides on. This suits legacy apps that must be reachable on the LAN without port mapping. Two catches: the host itself cannot talk to its own macvlan containers (by kernel design), and the switch or hypervisor must accept several MAC addresses on one port.

TIP

Explore/simulate: Most hypervisors drop frames from unknown MAC addresses, so a macvlan container may get an IP but fail to `ping` the gateway. To make it work, enable *promiscuous mode* (VirtualBox: "Allow All"; VMware: promiscuous mode and forged transmits; Hyper-V: MAC address spoofing) on the VM's network adapter. If you cannot change that, creating and inspecting the network is enough for the exam.

6. Create an ipvlan network on the same NIC and test it.

```
docker network create -d ipvlan \
  --subnet=192.168.100.0/24 --gateway=192.168.100.1 \
  --ip-range=192.168.100.240/29 -o parent="$NIC" -o ipvlan_mode=l2 labipvlan
docker run --rm --network labipvlan alpine:3.20 \
  sh -c 'ip link show eth0; ping -c2 192.168.100.1'
docker network rm labipvlan
```

What just happened: `-d ipvlan` also places containers directly on the LAN, but they all share the parent NIC's MAC address, so it usually works even where macvlan is blocked and does not need promiscuous mode. `ipvlan_mode=l2` works like a switch on the same subnet; `l3` mode routes between subnets instead. You created the networks one at a time because Docker will not let two networks use the same subnet.

7. See why `overlay` needs an orchestrator.

```
docker network create -d overlay laboverlay
```

```
Error response from daemon: This node is not a swarm manager. Use "docker swarm init" or
"docker swarm join" to connect this node to swarm and try again.
```

What just happened: An `overlay` network connects containers running on **different hosts** by tunneling their traffic (VXLAN) over the normal network. Docker needs Swarm mode to share the network's state between hosts, so it refuses on a standalone host. Kubernetes provides similar overlay networks through CNI plugins. You will meet Swarm in objective 4.1.

Checkpoint: `docker network ls` shows `bridge`, `host`, `none`, and `labnet`, and no macvlan or ipvlan network.

Task 7: Privileged vs. unprivileged, rootful vs. rootless

1. Try two root-only actions in a normal (unprivileged) container.

```
docker run --rm alpine:3.20 sh -c 'id; ip link add lab0 type dummy'
docker run --rm alpine:3.20 mount -t tmpfs none /mnt
```

```
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),...
ip: RTNETLINK answers: Operation not permitted
mount: permission denied (are you root?)
```

What just happened: The process is `root` (UID 0) inside the container, yet the kernel refuses to create a network interface or mount a filesystem. Docker drops most Linux *capabilities* (such as `CAP_NET_ADMIN` and `CAP_SYS_ADMIN`) and applies seccomp and AppArmor profiles. "Root in a container" is a limited root by default.

2. Run the same commands in a privileged container.

```
docker run --rm --privileged alpine:3.20 \
  sh -c 'ip link add lab0 type dummy && ip -br link show lab0'
docker run --rm --privileged alpine:3.20 sh -c 'mount -t tmpfs none /mnt && df -h /mnt'
docker run --rm --privileged alpine:3.20 sh -c 'ls /dev | grep -E "^(sd|vd|nvme)"'
```

```
lab0             DOWN           ...
Filesystem                Size      Used Available Use% Mounted on
none                      1.9G         0      1.9G   0% /mnt
sda
sda1
sda2
sdb
...
```

What just happened: `--privileged` gives the container all capabilities, disables the seccomp and AppArmor protections, and exposes all host devices. The last command shows it can see your real disks; it could format them. (The `lab0` interface was created in the container's own network namespace and vanished with it.) Avoid `--privileged`. If a container truly needs one power, add only that capability, for example `--cap-add NET_ADMIN`:

```
docker run --rm --cap-add NET_ADMIN alpine:3.20 \
  sh -c 'ip link add lab0 type dummy && echo created'
```

3. Compare a rootful Docker container with a rootless Podman container, as seen from the host.

```
docker run -d --name rootful alpine:3.20 sleep 600
podman run -d --name rootless docker.io/library/alpine:3.20 sleep 600
podman exec rootless id -u
ps -o user,pid,cmd -C sleep
```

```
0
USER         PID CMD
root        8812 sleep 600
student     8901 sleep 600
```

What just happened: Both containers run `sleep` as root **inside** (`id -u` prints `0`). On the host, the Docker one really is `root` , because `dockerd` runs as root (*rootful*). The Podman one is just `student` : rootless Podman uses a *user namespace* that maps container UID 0 to your own UID. If someone escaped from that container, they would only have your user's rights. Use the full name `docker.io/library/alpine` with Podman so it does not have to guess which registry you mean.

4. See the user namespace mapping behind rootless Podman.

```
grep student /etc/subuid
podman unshare cat /proc/self/uid_map
docker rm -f rootful
podman rm -f rootless
```

```
student:100000:65536
0       1000          1
1     100000      65536
```

What just happened: `/etc/subuid` gives `student` a private range of 65,536 extra UIDs starting at 100000. `podman unshare` runs a command inside Podman's user namespace. Each `uid_map` line reads "container UID, host UID, count": container UID 0 is host UID 1000 (`student`), and container UIDs 1 to 65536 are host UIDs 100000 and up, which belong to nobody on the host. Docker can also run rootless (`dockerd-rootless-setuptool.sh`), but it is not the default.

Checkpoint: You can explain why `ps` showed `root` for one `sleep` process and `student` for the other.

Task 8: SELinux contexts and bind mounts with Podman on Rocky

Run every command in this task on rocky-lab, as `student` (rootless).

1. Install Podman and confirm SELinux is enforcing.

```
# on rocky-lab
sudo dnf install -y podman
getenforce
mkdir -p ~/labs/2.6/web
echo '<h1>Hello from Rocky</h1>' > ~/labs/2.6/web/index.html
ls -Z ~/labs/2.6/web/index.html
```

```
Enforcing
unconfined_u:object_r:user_home_t:s0 /home/student/labs/2.6/web/index.html
```

What just happened: `getenforce` shows SELinux mode. `ls -Z` shows a file's SELinux *context* (`user:role:type:level`). The important part is the *type*, `user_home_t` : a file in a home folder. SELinux decides access by type, on top of normal permissions.

2. Bind-mount the folder into a container without a label option.

```
# on rocky-lab
podman run --rm -v ~/labs/2.6/web:/data:ro \
  docker.io/library/alpine:3.20 cat /data/index.html
sudo ausearch -m avc -ts recent | grep -o 'denied.*tclass=file' | tail -1
```

```
cat: can't open '/data/index.html': Permission denied
denied  { read } for  pid=4512 comm="cat" name="index.html" ...
scontext=system_u:system_r:container_t:s0:c214,c870
tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file
```

What just happened: The Unix permissions allow reading, but SELinux blocked it. Container processes run with the type `container_t`, and policy only lets them use files labeled `container_file_t`. The audit log entry (`ausearch -m avc` searches for *Access Vector Cache* denials) shows the source context (`container_t`) and target context (`user_home_t`). The `c214,c870` part is an MCS category pair, unique per container, which keeps containers away from each other's files too.

3. Fix it with `:Z`, and check the new label.

```
# on rocky-lab
podman run --rm -v ~/labs/2.6/web:/data:ro,Z \
  docker.io/library/alpine:3.20 cat /data/index.html
ls -Z ~/labs/2.6/web/index.html
```

```
<h1>Hello from Rocky</h1>
system_u:object_r:container_file_t:s0:c157,c623 /home/student/labs/2.6/web/index.html
```

What just happened: `:Z` (capital) tells Podman to relabel the host folder with `container_file_t` and **this container's private** MCS categories, so only this one container can use it. `:z` (lowercase) applies `container_file_t` without categories, so **several** containers can share the folder. Options are combined with commas, as in `:ro,Z`. Docker on SELinux systems supports the same `:z` and `:Z`.

CAUTION

Never use `:Z` or `:z` on system folders such as `/home`, `/etc`, or `/usr`. Relabeling them would stop the host's own services from reading their files. Only relabel folders made for the container. To relabel by hand you can use `chcon -t container_file_t <dir>`, or make it permanent with `semanage fcontext` and `restorecon`.

4. Run a rootless web container on Rocky with port mapping.

```
# on rocky-lab
podman run -d --name rockyweb -p 8080:80 \
  -v ~/labs/2.6/web:/usr/share/nginx/html:ro,Z docker.io/library/nginx:alpine
curl -s localhost:8080
podman ps
```

What just happened: Rootless containers cannot bind host ports below 1024 by default, so you used 8080. To reach it from ubuntu-lab you would also need `sudo firewall-cmd --add-port=8080/tcp` (firewalld is active on Rocky). Podman can run containers as systemd services through *Quadlet* files (`~/.config/containers/systemd/*.container`), the modern replacement for `podman generate systemd`.

Checkpoint: On rocky-lab, `curl -s localhost:8080` prints `<h1>Hello from Rocky</h1>`, and `ls -Z ~/labs/2.6/web` shows `container_file_t`.

Task 9: Prune unused containers, images, volumes, and networks

Unused objects pile up quickly and fill `/var/lib/docker`. Each object type has a `prune` command. Run these on ubuntu-lab.

1. See how much space Docker uses.

```
docker system df
```

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          7         2         155.2MB   48.5MB (31%)
Containers      4         4         20.5kB    0B (0%)
Local Volumes   1         0         48B       48B (100%)
Build Cache     9         0         12.3MB    20.48kB
```

2. Create a few stopped containers, then prune containers.

```
docker run --name junk1 alpine:3.20 true
docker run --name junk2 alpine:3.20 true
docker ps -a --filter status=exited
docker container prune -f
```

What just happened: `true` exits immediately, leaving two stopped containers. `docker container prune` deletes **all stopped** containers. `-f` skips the "Are you sure?" question. Running containers are never pruned.

3. Prune images.

```
docker image prune -f
docker image ls
```

What just happened: `docker image prune` removes only *dangling* images (the `<untagged>` image from Task 4). `docker image prune -a` would remove **every** image not used by at least one container, including `alpine:3.20` and `greeter`, so be careful with `-a`.

4. Prune volumes and networks.

```
docker volume prune -f
docker volume ls
docker volume prune -a -f
docker volume ls
docker rm -f web2
docker network prune -f
docker network ls
```

What just happened: On current Docker versions, `docker volume prune` removes only unused *anonymous* volumes (the unnamed ones created by `VOLUME` in images), so `labdata` survived. Adding `-a` (`--all`) also removes unused **named** volumes, which permanently deletes their data. `docker network prune` removes user-defined networks with no containers (here `labnet`, after you removed `web2`); the built-in `bridge`, `host`, and `none` are never removed.

Finally, know the all-in-one command. Do not run it yet; you will run it in Clean up.

The all-in-one prune: `docker system prune` removes stopped containers, unused networks, dangling images, and build cache in one go. `docker system prune -a --volumes` also removes all unused images and anonymous volumes. Podman has the same commands (`podman system prune`, `podman image prune`, and so on).

Checkpoint: `docker volume ls` is empty, and `docker network ls` shows only `bridge`, `host`, and `none`.

Challenge

CHALLENGE

1. Write a `Dockerfile` based on `python:3.12-slim` that serves the current folder with `python -m http.server 8000`, runs as a non-root `USER`, and uses `ENTRYPOINT ["python", "-m", "http.server"]` with `CMD ["8000"]`. Build it as `pyweb:1.0`, run it with `-p 9000:8000`, and then override `CMD` so it listens on port 9001 inside the container instead.
2. Build a two-container setup by hand: create a user-defined network and a named volume, start `docker.io/library/redis:alpine` on that network with the volume mapped to `/data`, and connect to it by name from a second container with `redis-cli -h <name> ping`.
3. On rocky-lab, find out with `podman inspect --format '{{.ProcessLabel}}' rockyweb` which SELinux label and MCS categories the container runs with, and compare them with the categories on the files you relabeled with `:Z`.

Clean up

1. On ubuntu-lab, remove all containers, images, volumes, networks, and build cache, for both Docker and Podman.

```
docker rm -f $(docker ps -aq) 2>/dev/null
docker system prune -a --volumes -f
docker volume prune -a -f
podman rm -af
podman rmi -af
docker system df
```

`docker ps -aq` prints only the IDs (`-q`) of all containers (`-a`), and `$( )` passes them to `docker rm -f`. The `2>/dev/null` hides the error if there are none.

2. Remove `student` from the `docker` group (it is equal to root access) and delete the lab folder.

```
sudo gpasswd -d student docker
rm -rf ~/labs/2.6
```

The group change takes effect at your next login. If you want to remove the engines completely, run `sudo apt purge -y docker.io docker-buildx podman containerd runc && sudo apt autoremove -y`; later labs install them again when needed.

3. On rocky-lab, remove the container, image, and lab folder.

```
# on rocky-lab
podman rm -f rockyweb
podman rmi -af
rm -rf ~/labs/2.6
```

Exam corner

- Runtime stack: Docker CLI -> `dockerd` -> **containerd** -> **runC** (OCI runtime). **Podman** is daemonless, Docker-compatible, and rootless by default (uses `crun` / `runc`).
- Image commands: `pull`, `build -t name:tag .`, `tag`, `push`, `rmi`, `image ls`, `history` (layers), `image inspect`, `image prune` (dangling; `-a` for all unused).
- Dockerfile: `FROM` (base), `RUN` (build-time command, new layer), `COPY` / `ADD`, `ENV`, `USER` (non-root), `EXPOSE`, `ENTRYPOINT` (fixed program), `CMD` (default arguments; replaced by arguments after the image name). Override the entrypoint with `--entrypoint`.
- Container commands: `run -d --name -p host:container -e VAR=value -v src:dst`, `ps -a`, `logs -f`, `exec -it name sh`, `inspect --format`, `stop`, `start`, `restart`, `rm` (`-f` to force), `container prune`.
- Volumes: `volume create`, `-v name:/path` (named) vs `-v /host/path:/path` (bind), `:ro`; data outside volumes lives in the container's overlay upper layer and is lost with `rm`. `volume prune` removes anonymous volumes; `-a` includes named ones.
- Storage: OverlayFS, through Docker's `overlay2` driver or (Docker 29+) containerd's `overlayfs` snapshotter; OverlayFS `lowerdir` (read-only image layers), `upperdir` (container writes), `merged` (the container's view).
- SELinux: bind mounts need `container_file_t`; `:z` = shared label, `:Z` = private label for one container; check with `ls -Z`, denials in `ausearch -m avc`.

- Networks: `network create` (user-defined bridge has DNS by container name), `bridge` (default, NAT, needs `-p`), `host` (shares host stack), `none` (loopback only), `macvlan` (own MAC on LAN, needs `-o parent=`, promiscuous mode), `ipvlan` (shares parent MAC), `overlay` (multi-host, needs Swarm). `network prune`.
- `-p 8080:80` maps host port 8080 to container port 80; `-P` maps all exposed ports to random ports; `-p 127.0.0.1:8080:80` binds only to loopback.
- Unprivileged containers lack most capabilities; `--privileged` grants all capabilities and host devices (avoid; use `--cap-add`). Rootless containers map container root to an unprivileged host UID (`/etc/subuid`).
- The `docker` group is root-equivalent.

Review questions

1. A Dockerfile ends with `ENTRYPOINT ["ping"]` and `CMD ["-c3", "localhost"]`. What runs when a user types `docker run img -c1 8.8.8.8`?
   A. `ping -c3 localhost`
   B. `-c1 8.8.8.8`
   C. `ping -c1 8.8.8.8`
   D. `ping -c3 localhost -c1 8.8.8.8`

2. On a Rocky Linux host, `podman run -v /srv/app:/app myimg` fails with "Permission denied" although the file permissions are correct. What is the most likely fix?

3. Which Docker network type lets containers resolve each other by container name on a single host?
   A. The default `bridge` network
   B. A user-defined bridge network created with `docker network create`
   C. `none`
   D. `host`

4. Which command removes stopped containers, unused networks, dangling images, and build cache in one step?

5. Why is adding a user to the `docker` group considered a security risk?

6. What is the role of runC in the Docker stack?
   A. It stores images in a registry
   B. It builds images from Dockerfiles
   C. It is the low-level OCI runtime that creates the namespaces and cgroups and starts the container process
   D. It provides DNS for user-defined networks

PART 3 · DOMAIN 3.0

Part 3: Security

18% of the exam · 6 labs

3.1	Summarize authorization, authentication, and accounting methods.
3.2	Given a scenario, configure and implement firewalls on a Linux system.
3.3	Given a scenario, apply operating system (OS) hardening techniques on a Linux system.
3.4	Explain account hardening techniques and best practices.
3.5	Explain cryptographic concepts and technologies in a Linux environment.
3.6	Explain the importance of compliance and audit procedures.

Lab 3.1 Authentication, Authorization, and Accounting

Objective: 3.1 Summarize authorization, authentication, and accounting methods.
Domain: 3.0 Security (18% of the exam)
Time: about 120 minutes
VM: ubuntu-lab (main), rocky-lab (Samba client only)
Snapshot first: Yes. This lab edits PAM, polkit, and logging configuration.

Every Linux security question boils down to three things: who are you, what may you do, and what did you do. In this lab you will touch every piece of that puzzle: PAM and directory services (LDAP, Kerberos, SSSD) for proving identity, polkit for deciding permissions, and journald, rsyslog, logrotate, and auditd for keeping records. On the exam, objective 3.1 is a "summarize" objective, so you must recognize each tool and its job; on the job, you will configure all of them.

What you'll practice

- Authentication through **PAM** (Pluggable Authentication Modules): reading `/etc/pam.d/` files, control flags, adding and removing a module
- Authorization with **Polkit**: `pkaction`, `pkcheck`, and a JavaScript rule in `/etc/polkit-1/rules.d/`
- Directory and network identity: **LDAP** (`slapd`, `ldapadd`, `ldapsearch`), **Kerberos** (`krb5.conf`, `kinit`), **SSSD/Winbind realm** (`realm discover`, `sssd.conf`)
- File sharing and Windows-style identity with **Samba** (`smb.conf`, `smbpasswd`, `smbclient`)
- Logging: **journalctl** filters, persistent journal, **rsyslog** rules, **logrotate**, a tour of **/var/log**
- System audit: **auditd**, `auditctl`, rules in `/etc/audit/rules.d/` compiled into **audit.rules**, `ausearch`, `aureport`

How it works

Security people use the term **AAA**:

- **Authentication** answers "Who are you?" You prove an identity with something you know (password), have (key, token), or are (fingerprint).
- **Authorization** answers "What are you allowed to do?" File permissions, `sudo` rules, and polkit rules are all authorization.
- **Accounting** answers "What did you do?" Logs and audit records let you reconstruct events later.

Think of a hotel. The front desk checks your ID (authentication), your key card only opens your room and the gym (authorization), and the card system records every door you opened (accounting).

PAM is the front desk for almost every Linux program that logs people in: `login`, `sshd`, `su`, `sudo`, `passwd`. Instead of each program knowing how to check passwords, each program asks PAM, and PAM runs a stack of small modules (`pam_unix.so`, `pam_access.so`, `pam_sss.so`, …) listed in a file under `/etc/pam.d/` named after the program. Each line has a **type** (`auth` = check identity, `account` = is this account allowed now, `password` = change a password, `session` = set things up at login) and a **control flag** that says how much that module's answer matters.

Polkit (formerly PolicyKit) is an authorization service for system daemons. When an unprivileged program asks `systemd` to restart a service, `systemd` asks polkit "is this user allowed to do action `org.freedesktop.systemd1.manage-units`?" Polkit checks its rules and answers yes, no, or "ask for an admin password". `sudo` grants whole commands; polkit grants specific actions inside daemons.

Directory services keep user accounts on a central server instead of in every machine's `/etc/passwd`:

- **LDAP** (Lightweight Directory Access Protocol, TCP 389, or 636 for LDAPS) is a database of entries arranged in a tree, like `uid=jdoe,ou=people,dc=lab,dc=local`. OpenLDAP's server is `slapd`.
- **Kerberos** (TCP/UDP 88) is a ticket-based authentication protocol. You prove your password once to the KDC (Key Distribution Center) with `kinit`, get a ticket, and then show tickets instead of passwords. Realms are written in capitals: `LAB.LOCAL`.
- **Active Directory** (Microsoft) is LDAP plus Kerberos plus DNS.
- **SSSD** (System Security Services Daemon) is the Linux client that talks to LDAP, Kerberos, AD, or FreeIPA and caches the answers, so `getent passwd jdoe` works even though jdoe is not in `/etc/passwd`. **Winbind** (part of Samba) does a similar job for AD. **realmd** (`realm` command) is a helper that discovers a domain and configures SSSD or Winbind for you.
- **Samba** implements the Windows SMB/CIFS file-sharing protocol (TCP 445) on Linux, and can even act as an AD domain controller.

Logging has two main systems on modern distributions:



`systemd-journald` collects everything (kernel, services, `syslog()` calls) in an indexed binary journal that you query with `journalctl`. `rsyslog` receives a copy and writes classic text files using **facility.priority** rules (for example `auth.*` or `local5.info`). `logrotate` stops text logs from filling the disk.

auditd is different from ordinary logging. The kernel itself generates audit records when watched files are touched or watched system calls are made, and `auditd` writes them to `/var/log/audit/audit.log`. Programs cannot avoid or fake these records, which is why auditors love them. Rules are loaded with `auditctl`, and persistent rules live in `/etc/audit/rules.d/*.rules`, which `augenrules` merges into `/etc/audit/audit.rules`.

Tasks

Task 1: Read PAM stacks and understand control flags

1. Create your working directory and look at the PAM configuration directory.

```
mkdir -p ~/labs/3.1 && cd ~/labs/3.1
ls /etc/pam.d/
```

```
chfn             common-password                login     runuser    sshd
chpasswd         common-session                 newusers  runuser-l  su
chsh             common-session-noninteractive  other     sudo       sudo-i
common-account   common-auth                    passwd    cron       su-l
```

What just happened: each file is a PAM "service" named after the program that uses it. `sshd` is read by the SSH server, `login` by console logins, `sudo` by sudo. The `common-*` files are Ubuntu's shared building blocks that other files include. (Rocky uses `system-auth` and `password-auth` for the same job.) The `other` file is the fallback for programs with no file of their own.

2. Look at the shared authentication stack, hiding comments and blank lines.

```
grep -Ev '^#|^$' /etc/pam.d/common-auth
```

```
auth    [success=1 default=ignore]  pam_unix.so nullok
auth    requisite           pam_deny.so
auth    required            pam_permit.so
auth    optional            pam_cap.so
```

What just happened: `grep -Ev` prints lines that do **not** match (`-v`) the extended regex (`-E`) "starts with `#` " or "is empty". Read the stack top to bottom:

- `pam_unix.so` checks the password against `/etc/shadow` . The bracket control `[success=1 default=ignore]` means "if the password is right, **skip the next 1 line**; otherwise ignore this result and continue".
- So on success you jump over `pam_deny.so` (which always fails) and land on `pam_permit.so` (which always succeeds). On failure you hit `pam_deny.so` , and because it is `requisite` , PAM stops immediately with failure.
- `pam_cap.so` is `optional` : its result only matters if it is the only module.

3. Learn the four classic control flags. Read this table carefully; it is a favorite exam topic.

Flag	If the module succeeds	If the module fails
`required`	keep going	remember the failure, **keep going**, fail at the end
`requisite`	keep going	**stop now** and fail
`sufficient`	**stop now** with success (if nothing required failed earlier)	ignore, keep going
`optional`	only matters if it is the only module	only matters if it is the only module

Two more keywords appear in the control column: `include` (pull in another file's lines of the same type) and `substack` (like include, but a `requisite` failure only ends the substack). Ubuntu also uses `@include common-auth`, which includes the whole file.

4. Now look at the SSH and console login stacks.

```
grep -Ev '^#|^$' /etc/pam.d/sshd
grep -n pam_access /etc/pam.d/sshd
grep -Ev '^#|^$' /etc/pam.d/login | head -15
```

```
@include common-auth
account    required     pam_nologin.so
@include common-account
session [success=ok ignore=ignore module_unknown=ignore default=bad] pam_selinux.so close
session    required     pam_loginuid.so
session    optional     pam_keyinit.so force revoke
@include common-session
session    optional     pam_motd.so  motd=/run/motd.dynamic
session    optional     pam_motd.so noupdate
session    optional     pam_mail.so standard noenv
session    required     pam_limits.so
session    required     pam_env.so
session    required     pam_env.so envfile=/etc/default/locale
...
12:# account  required     pam_access.so
```

What just happened: `sshd` reuses `common-auth` for passwords, then adds account checks such as `pam_nologin.so` (refuse non-root logins while `/etc/nologin` exists) and session modules such as `pam_motd.so` (print the "message of the day" banner) and `pam_limits.so` (apply `/etc/security/limits.conf`). The second command shows that line 12 contains a **commented-out** `pam_access.so` line. You will enable it next. The `login` file is similar but adds console-only modules like `pam_faildelay.so`.

Checkpoint: you can explain why a wrong password in `common-auth` ends at `pam_deny.so`, and you know the difference between `required` and `requisite`.

Task 2: Make a harmless PAM change, test it, and revert it

You will use `pam_access.so` to block SSH logins for one test user only. Your own `student` account is not affected, but PAM mistakes can lock you out, so follow the safety steps.

> **CAUTION**
> Take a snapshot now if you have not. Keep a **second terminal** logged in as `student` (or keep the VM console open) for the whole task. If logins break, you fix the file from the session that is still open: `sudo cp /etc/pam.d/sshd.bak /etc/pam.d/sshd`.

1. Create a test user and back up the file you will change.

```
sudo useradd -m -s /bin/bash labpam
sudo passwd labpam
sudo cp /etc/pam.d/sshd /etc/pam.d/sshd.bak
```

> **What just happened:** `useradd -m` creates the user and a home directory; `-s /bin/bash` sets the login shell. `passwd labpam` asks you to type a new password twice (pick something simple like `Lab3.1pass`). The `cp` makes a backup you can restore in one command.

2. Confirm that `labpam` can log in over SSH **before** you change anything.

```
ssh labpam@localhost whoami
```

```
labpam@localhost's password:
labpam
```

Type `yes` if asked to trust the host key. If you see `Permission denied (publickey)`, password logins are turned off in your SSH server; check `sudo sshd -T | grep passwordauthentication`, which should say `yes` for this test.

3. Enable `pam_access.so`. Open the file with `sudo nano /etc/pam.d/sshd`, find the line `# account required pam_access.so`, and delete the leading `#` so it reads:

```
account  required     pam_access.so
```

Save with Ctrl+O, Enter, and exit with Ctrl+X. Verify:

```
grep -n pam_access /etc/pam.d/sshd
```

```
12:account  required     pam_access.so
```

4. Add a rule to `/etc/security/access.conf`. The format is `permission:users:origins`, where `+` allows and `-` denies.

```
echo '-:labpam:ALL' | sudo tee -a /etc/security/access.conf
tail -1 /etc/security/access.conf
```

What just happened: `tee -a` appends (`-a`) the text to the file; you need `sudo tee` because a plain `>>` redirection would run as your normal user and fail. The rule means "deny (`-`) user `labpam` from ALL origins". Every other line in the file is a comment, so all other users are still allowed.

5. Test. PAM files are read fresh at every login, so no restart is needed.

```
ssh labpam@localhost whoami
sudo journalctl -u ssh -n 5 --no-pager | grep -i access
```

```
labpam@localhost's password:
Connection closed by 127.0.0.1 port 22
... sshd[2412]: pam_access(sshd:account): access denied for user `labpam'
    from `127.0.0.1'
```

What just happened: the password was correct, so the `auth` stack succeeded (authentication). Then the `account` stack ran `pam_access.so`, which said no (authorization), and SSH closed the connection. The journal entry is the accounting record. Confirm `ssh student@localhost whoami` still works.

6. Revert the change so later labs are not affected.

```
sudo cp /etc/pam.d/sshd.bak /etc/pam.d/sshd
sudo sed -i '/^-:labpam:ALL$/d' /etc/security/access.conf
ssh labpam@localhost whoami
```

What just happened: `sed -i '/pattern/d'` deletes, in place (`-i`), every line matching the pattern. The last command should print `labpam` again.

Checkpoint: `grep -c '^account.*pam_access' /etc/pam.d/sshd` prints `0`, and `ssh labpam@localhost whoami` prints `labpam`.

Task 3: Authorization with polkit

1. Install the polkit tools and list a few actions.

```
sudo apt install -y polkitd pkexec
pkaction | grep systemd1
pkaction --action-id org.freedesktop.systemd1.manage-units --verbose
```

```
org.freedesktop.systemd1.manage-units
org.freedesktop.systemd1.manage-unit-files
org.freedesktop.systemd1.reload-daemon
...
org.freedesktop.systemd1.manage-units:
  description:       Manage system services or other units
  message:           Authentication is required to manage system services or ...
  vendor:            The systemd Project
  implicit any:      auth_admin
  implicit inactive: auth_admin
  implicit active:   auth_admin_keep
```

What just happened: `pkaction` lists every action ID that daemons have registered (from XML files in `/usr/share/polkit-1/actions/`). The verbose view shows the **defaults**: `auth_admin` means "ask for an administrator's password", `auth_admin_keep` means "ask, then remember for a few minutes". "Active" means a user at the local console; "any" includes SSH sessions. Other possible values are `yes`, `no`, and `auth_self` (ask for the user's own password).

2. See who counts as an "administrator". On Ubuntu, a vendor rule makes the `sudo` group the admin group.

```
ls /usr/share/polkit-1/rules.d/ /etc/polkit-1/rules.d/
sudo cat /usr/share/polkit-1/rules.d/49-ubuntu-admin.rules
```

```
polkit.addAdminRule(function(action, subject) {
    return ["unix-group:sudo", "unix-group:admin"];
});
```

What just happened: `ls` lists both rules directories (the `/etc` one is empty until you add a rule). `addAdminRule` tells polkit which accounts count as "an administrator" when an action needs `auth_admin`: here, members of the `sudo` group (and the old `admin` group from very early Ubuntu releases). The file `50-default.rules` next to it says the same thing for `sudo` only.

TIP

Polkit version 0.106 and later (Ubuntu 23.04+, Debian 12+, all RHEL 8/9) uses **JavaScript `.rules` files** in `/etc/polkit-1/rules.d/` (local admin) and `/usr/share/polkit-1/rules.d/` (packages). Older Debian/Ubuntu releases used INI-style `.pkla` files under `/etc/polkit-1/localauthority/`. You may still see `.pkla` in old documentation.

3. Create a demo service and a non-admin operator account.

```
cat <<'EOF' | sudo tee /etc/systemd/system/labdemo.service
[Unit]
Description=Lab 3.1 polkit demo service

[Service]
ExecStart=/usr/bin/sleep infinity
EOF
sudo systemctl daemon-reload
sudo systemctl start labdemo
sudo groupadd labops
sudo useradd -m -s /bin/bash -G labops opsuser
sudo passwd opsuser
```

What just happened: `cat <<'EOF' ... EOF` is a **heredoc**: everything between the two `EOF` markers is fed to the command as input. Piping it to `sudo tee FILE` writes the file as root. The quotes around `'EOF'` stop the shell from expanding `$` variables inside. The unit simply runs `sleep infinity`, a process that does nothing forever. `-G labops` puts `opsuser` in the supplementary group `labops`.

4. Switch to `opsuser` and try to restart the service without sudo.

```
su - opsuser
systemctl restart labdemo
```

```
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to restart 'labdemo.service'.
Authenticating as: student
Password:
```

What just happened: `su -` starts a full login shell as another user (you typed opsuser's password). `systemctl` sent the request to systemd over D-Bus; systemd asked polkit; polkit answered `auth_admin`, so a text agent (`pkttyagent`) asks for the password of an admin (`student`, a member of `sudo`). Press **Ctrl+C** to cancel instead of typing it.

5. Ask polkit directly with `pkcheck`, still as `opsuser`.

```
pkcheck --action-id org.freedesktop.systemd1.manage-units --process $$; echo "exit=$?"
exit
```

```
Authorization requires authentication and -u wasn't passed.
exit=2
```

What just happened: `pkcheck` asks "is the process `$$` (the PID of your current shell) authorized for this action?" Exit code `0` = authorized, `1` = not authorized, `2` = would need a password (`-u`, short for `--allow-user-interaction`, would let it prompt). `exit` returns you to `student`. When systemd itself asks polkit, it also passes **details** such as which unit and which verb (`restart`); `pkcheck` has a `--detail key value` option for that, but polkit only accepts details from trusted callers (root or the daemon that owns the action), so as `opsuser` you test without them.

6. Write a rule that lets members of `labops` start, stop, and restart **only** `labdemo.service`.

```
cat <<'EOF' | sudo tee /etc/polkit-1/rules.d/50-labops.rules
// Let the labops group manage labdemo.service without a password
polkit.addRule(function(action, subject) {
    if (action.id == "org.freedesktop.systemd1.manage-units" &&
        action.lookup("unit") == "labdemo.service" &&
        subject.isInGroup("labops")) {
        var verb = action.lookup("verb");
        if (verb == "start" || verb == "stop" || verb == "restart") {
            return polkit.Result.YES;
        }
    }
});
EOF
sudo journalctl -u polkit -n 3 --no-pager
```

What just happened: polkit calls every `addRule` function, in file-name order, for each request. The function receives the `action` (with details from `action.lookup()`) and the `subject` (the user asking). Returning `polkit.Result.YES` grants the action; returning nothing means "no opinion, keep checking". Polkit watches the directory and reloads automatically; the journal shows a line like `Reloading rules` and `Finished loading, compiling and executing 3 rules`.

7. Test again as `opsuser`.

```
su - opsuser
systemctl restart labdemo && echo restarted
systemctl restart cron
exit
```

```
restarted
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to restart 'cron.service'.
```

Press Ctrl+C at the second prompt. The rule only covers `labdemo.service`.

Checkpoint: as `opsuser`, `systemctl restart labdemo` works with no prompt, but restarting any other unit asks for an admin password.

Task 4: Directory services: LDAP, SSSD, Kerberos, and realm

In a company you would join an existing directory. Here you will build a tiny local LDAP server, point SSSD at it, and explore the Kerberos and realm tools.

1. Pre-answer the OpenLDAP installer questions with **debconf**, then install the server and client tools.

```
cat <<'EOF' | sudo debconf-set-selections
slapd slapd/no_configuration boolean false
slapd slapd/domain string lab.local
slapd shared/organization string Lab Org
slapd slapd/password1 password LdapLab123
slapd slapd/password2 password LdapLab123
EOF
sudo DEBIAN_FRONTEND=noninteractive apt install -y slapd ldap-utils
```

> **What just happened:** Debian packages ask setup questions through **debconf**. `debconf-set-selections` stores answers in advance, and `DEBIAN_FRONTEND=noninteractive` tells apt not to ask. The domain `lab.local` becomes the directory's **base DN** (Distinguished Name) `dc=lab,dc=local` ("dc" = domain component), and the admin account is `cn=admin,dc=lab,dc=local` with password `LdapLab123`.

2. Query the empty directory anonymously.

```
ldapsearch -x -H ldap://localhost -b dc=lab,dc=local
```

```
# lab.local
dn: dc=lab,dc=local
objectClass: top
objectClass: dcObject
objectClass: organization
o: Lab Org
dc: lab
# search result
search: 2
result: 0 Success
```

> **What just happened:** `-x` uses simple authentication (here anonymous, since no bind DN was given) instead of SASL. `-H` is the server URI and `-b` the search base. Every entry has a `dn` and attributes; `objectClass` attributes decide which other attributes are allowed. `result: 0 Success` means the search worked.

3. Add an organizational unit and a user from an **LDIF** (LDAP Data Interchange Format) file.

```
cat <<'EOF' > ~/labs/3.1/users.ldif
dn: ou=people,dc=lab,dc=local
objectClass: organizationalUnit
ou: people

dn: uid=jdoe,ou=people,dc=lab,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
uid: jdoe
cn: Jane Doe
sn: Doe
mail: jdoe@lab.local
uidNumber: 10001
gidNumber: 10001
homeDirectory: /home/jdoe
loginShell: /bin/bash
EOF
ldapadd -x -D cn=admin,dc=lab,dc=local -W -f ~/labs/3.1/users.ldif
```

```
Enter LDAP Password:
adding new entry "ou=people,dc=lab,dc=local"

adding new entry "uid=jdoe,ou=people,dc=lab,dc=local"
```

What just happened: a blank line separates entries in LDIF. `posixAccount` adds the Unix attributes (`uidNumber`, `homeDirectory`, `loginShell`) that make the entry usable as a Linux login. `ldapadd` binds (`-D`) as the admin DN, `-W` prompts for its password (`LdapLab123`), and `-f` reads the file.

4. Search with a filter and ask for only two attributes.

```
ldapsearch -x -H ldap://localhost -b dc=lab,dc=local '(uid=jdoe)' cn mail
```

```
dn: uid=jdoe,ou=people,dc=lab,dc=local
cn: Jane Doe
mail: jdoe@lab.local
```

5. Install SSSD, the realm tool, and the Kerberos client. The Kerberos package also asks debconf questions, so install non-interactively.

```
sudo DEBIAN_FRONTEND=noninteractive apt install -y sssd sssd-tools realmd krb5-user
grep -E '^(passwd|group|shadow):' /etc/nsswitch.conf
```

```
passwd:         files systemd sss
group:          files systemd sss
shadow:         files systemd sss
```

What just happened: the `libnss-sss` package (pulled in by `sssd`) added `sss` to `/etc/nsswitch.conf`, the file that tells the C library where to look up users and groups. Lookups now check local files first, then systemd, then SSSD. The `libpam-sss` package also added `pam_sss.so` to the `common-*` PAM files, so PAM can authenticate directory users.

6. Point SSSD at your LDAP server. SSSD refuses to start unless its config file is owned by root with mode `600`.

```
cat <<'EOF' | sudo tee /etc/sssd/sssd.conf
[sssd]
services = nss, pam
domains = lab.local

[domain/lab.local]
id_provider = ldap
auth_provider = ldap
ldap_uri = ldap://localhost
ldap_search_base = dc=lab,dc=local
cache_credentials = false
EOF
sudo chmod 600 /etc/sssd/sssd.conf
sudo sssctl config-check
sudo systemctl restart sssd
getent passwd jdoe
id jdoe
```

```
Issues identified by validators: 0
jdoe:*:10001:10001:Jane Doe:/home/jdoe:/bin/bash
uid=10001(jdoe) gid=10001 groups=10001
```

What just happened: `id_provider = ldap` says "get user and group identities from LDAP"; `auth_provider` says where passwords are checked. `getent passwd jdoe` looked through NSS, found nothing in `/etc/passwd`, asked SSSD, and SSSD asked `slapd`. jdoe now exists on this machine as far as every program is concerned, without ever being in `/etc/passwd`. The group shows as a bare number because no group 10001 exists in LDAP. In production you must use TLS (`ldaps://` or StartTLS); SSSD will not send passwords over plain LDAP.

7. Explore the Kerberos client configuration and try to get a ticket.

```
grep -A3 '^\[libdefaults\]' /etc/krb5.conf
kinit jdoe@LAB.LOCAL
klist
```

```
[libdefaults]
    default_realm =
...
kinit: Cannot find KDC for realm "LAB.LOCAL" while getting initial credentials
klist: No credentials cache found (filename: /tmp/krb5cc_1000)
```

TIP

Explore/simulate: there is no Kerberos KDC in this lab, so `kinit` fails at the "find the KDC" step, which proves the client is working and looking. On a real network, `/etc/krb5.conf` would contain something like the block below, `kinit` would ask for a password and store a ticket, and `klist` would list it with an expiry time. `kdestroy` deletes your tickets.

The Kerberos client file `/etc/krb5.conf` for a real realm would look like this:

```
[libdefaults]
    default_realm = LAB.LOCAL
    dns_lookup_kdc = true

[realms]
    LAB.LOCAL = {
        kdc = dc1.lab.local
        admin_server = dc1.lab.local
    }

[domain_realm]
    .lab.local = LAB.LOCAL
```

8. Use `realm` to discover a domain.

```
realm discover lab.local
realm list
```

```
realm: No such realm found
```

What just happened: `realm discover` looks up DNS SRV records such as `_ldap._tcp.lab.local` and `_kerberos._udp.lab.local` to find an Active Directory or FreeIPA domain. None exists here, so it reports nothing, and `realm list` (joined domains) is empty. Against a real AD, `realm discover` prints the domain type and required packages, and `sudo realm join -U Administrator corp.example.com` joins the machine, writes `sssd.conf` and `krb5.conf`, and sets up PAM for you. `realm join --client-software=winbind` would configure **Winbind** instead of SSSD; Winbind is Samba's daemon (`winbindd`, checked with `wbinfo -u`) that maps AD users to Linux users.

Checkpoint: `ldapsearch -x -H ldap://localhost -b dc=lab,dc=local '(uid=jdoe)' dn` finds the entry, and `getent passwd jdoe` returns a line even though `grep jdoe /etc/passwd` returns nothing.

Task 5: Share a folder with Samba

1. Install Samba on ubuntu-lab, create a share directory, and a Samba-only user.

```
sudo apt install -y samba
sudo useradd -M -s /usr/sbin/nologin smbuser
sudo mkdir -p /srv/labshare
sudo chown smbuser: /srv/labshare
sudo smbpasswd -a smbuser
```

What just happened: `-M` skips the home directory and `nologin` blocks shell logins; this account exists only so Samba can map the share user to a Linux UID. Samba keeps its **own** password database (Windows-compatible hashes), so `smbpasswd -a` adds the user there and asks for an SMB password (use `Smb3.1pass`). `chown smbuser:` sets owner and that user's primary group.

2. Back up the config and add a share definition at the end of `/etc/samba/smb.conf`.

```
sudo cp /etc/samba/smb.conf /etc/samba/smb.conf.bak
cat <<'EOF' | sudo tee -a /etc/samba/smb.conf

[labshare]
   comment = Lab 3.1 share
   path = /srv/labshare
   browseable = yes
   read only = no
   valid users = smbuser
EOF
testparm -s 2>/dev/null | tail -7
sudo systemctl restart smbd
```

```
[labshare]
    comment = Lab 3.1 share
    path = /srv/labshare
    read only = No
    valid users = smbuser
```

What just happened: each `[name]` section is a share. `valid users` is Samba's own authorization list. `testparm` checks the syntax and prints the effective config (`-s` skips the "press Enter" pause). Always run it after editing `smb.conf`. `smbd` serves files on TCP 445.

3. Connect from rocky-lab with the command-line client.

```
# on rocky-lab
sudo dnf install -y samba-client
smbclient -L //ubuntu-lab -U smbuser
```

```
Password for [WORKGROUP\smbuser]:
    Sharename       Type      Comment
    ---------       ----      -------
    print$          Disk      Printer Drivers
    labshare        Disk      Lab 3.1 share
    IPC$            IPC       IPC Service (ubuntu-lab server (Samba, Ubuntu))
```

4. Open the share, upload a file, and list it.

```
# on rocky-lab
echo "hello from rocky" > /tmp/fromrocky.txt
smbclient //ubuntu-lab/labshare -U smbuser -c 'put /tmp/fromrocky.txt fromrocky.txt; ls'
```

```
putting file /tmp/fromrocky.txt as \fromrocky.txt (1.2 kb/s) (average 1.2 kb/s)
  .                                   D        0  Sat Oct 10 10:12:01 2026
  ..                                  D        0  Sat Oct 10 10:02:44 2026
  fromrocky.txt                       A       17  Sat Oct 10 10:12:01 2026
```

What just happened: `-c` runs smbclient commands non-interactively (without it you get an `smb: \>` prompt where `put`, `get`, `ls`, and `exit` work like FTP). Back on ubuntu-lab, `ls -l /srv/labshare` shows the file owned by `smbuser`: Samba authenticated the SMB user and then wrote the file as the mapped Linux user.

Checkpoint: on ubuntu-lab, `ls -l /srv/labshare/fromrocky.txt` shows a file owned by `smbuser`, and `sudo pdbedit -L` lists `smbuser`.

Task 6: Query the journal and tour /var/log

1. Check whether the journal is persistent (survives reboots).

```
ls -d /var/log/journal
grep -i '^#\?storage' /etc/systemd/journald.conf
journalctl --list-boots | tail -3
journalctl --disk-usage
```

```
/var/log/journal
#Storage=auto
-1 5d2c... Fri 2026-10-09 18:01:12 UTC  Fri 2026-10-09 22:40:05 UTC
 0 9a7f... Sat 2026-10-10 09:00:03 UTC  Sat 2026-10-10 10:15:44 UTC
Archived and active journals take up 56.0M in the file system.
```

What just happened: with `Storage=auto` (the default), journald writes to `/var/log/journal` if that directory exists (persistent) and to `/run/log/journal` in RAM if not (lost at reboot). Ubuntu creates the directory, so the journal is persistent and you see more than one boot. To force persistence anywhere, set `Storage=persistent` in a drop-in such as `/etc/systemd/journald.conf.d/persist.conf` and run `sudo systemctl restart systemd-journald`. `SystemMaxUse=` limits disk usage.

2. Practice the filters you must know for the exam.

```
journalctl -u ssh --no-pager -n 5
journalctl -p err -b --no-pager | tail -5
journalctl --since "30 minutes ago" --no-pager | tail -5
journalctl -k --no-pager | head -3
journalctl _COMM=sudo --no-pager -n 3
```

What just happened:

- `-u ssh` shows one systemd unit; `-n 5` the last 5 lines; `--no-pager` prints instead of opening `less`.
- `-p err` shows priority `err` and more severe (`crit`, `alert`, `emerg`). Priorities from 0 to 7: emerg, alert, crit, err, warning, notice, info, debug.
- `-b` limits to the current boot; `-b -1` is the previous boot.
- `--since` / `--until` accept times like `"2026-10-10 09:00"`, `yesterday`, or `"30 minutes ago"`.
- `-k` shows only kernel messages (like `dmesg`).
- `_COMM=sudo` filters on a journal field; `journalctl -o verbose -n 1` shows all fields of an entry.

3. Follow the journal live. Run this, then in a second terminal run `logger "test from terminal 2"`. Press Ctrl+C to stop.

```
journalctl -f
```

What just happened: `-f` (follow) works like `tail -f`. `logger` sends a message through the syslog API, so it appears in the journal and in rsyslog's files.

4. Tour `/var/log`.

```
ls -l /var/log | head -25
sudo tail -3 /var/log/auth.log
```

File or directory	What it holds (Ubuntu)	Rocky equivalent
`/var/log/syslog`	most system messages	`/var/log/messages`
`/var/log/auth.log`	logins, sudo, sshd, PAM	`/var/log/secure`
`/var/log/kern.log`	kernel messages	in `messages`
`/var/log/dpkg.log`, `apt/`	package history	`/var/log/dnf.log`
`/var/log/journal/`	binary systemd journal	same (if persistent)
`/var/log/audit/audit.log`	auditd records	same
`/var/log/samba/`	Samba logs	same
`/var/log/wtmp`, `btmp`	logins, failed logins (read with `last`, `lastb`)	same

What just happened: you looked at the same events through two windows. The journal is structured and indexed, so `journalctl` can filter by unit, priority, boot, or time instantly. The text files in `/var/log` come from rsyslog and are easy to `grep`, `tail`, and ship to other servers. Many shops keep both: the journal for quick troubleshooting, rsyslog for long-term text archives and central log servers.

Checkpoint: you can show only error-level messages from this boot with `journalctl -p err -b`, and you can name the Ubuntu and Rocky files that hold authentication logs.

Task 7: Route a custom facility with rsyslog and rotate it with logrotate

1. Check that rsyslog is running and see its default rules.

```
systemctl is-active rsyslog
grep -Ev '^#|^$' /etc/rsyslog.d/50-default.conf
```

```
active
auth,authpriv.*         /var/log/auth.log
*.*;auth,authpriv.none      -/var/log/syslog
kern.*              -/var/log/kern.log
mail.*              -/var/log/mail.log
mail.err            /var/log/mail.err
*.emerg             :omusrmsg:*
```

What just happened: each rule is `selector  action`. The selector is `facility.priority`: `auth,authpriv.*` means facilities auth and authpriv at any priority; `authpriv.none` excludes them. A `-` before a file name means "do not sync after every write" (faster). Facilities include `kern`, `user`, `mail`, `daemon`, `auth`, `authpriv`, `cron`, and `local0` to `local7`, which are free for your own applications.

2. Create a rule that sends facility `local5` to its own file and stops it from also going to `syslog`.

```
cat <<'EOF' | sudo tee /etc/rsyslog.d/30-lab.conf
# Lab 3.1: local5 messages go to their own file only
local5.*    /var/log/lab-local5.log
& stop
EOF
sudo rsyslogd -N1
sudo systemctl restart rsyslog
```

```
rsyslogd: version 8.2312.0, config validation run (level 1), master config
/etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
```

What just happened: files in `/etc/rsyslog.d/` are read in name order, so `30-lab.conf` runs before `50-default.conf`. `&` means "for the same messages as the previous line", and `stop` discards them so no later rule sees them. `rsyslogd -N1` checks the config without starting the daemon. Rsyslog on Ubuntu runs as user `syslog`, which can create files in `/var/log` because that directory is group-writable by `syslog`.

3. Send test messages with `logger` and check where they went.

```
logger -p local5.info -t labapp "Lab 3.1 info message"
logger -p local5.warning -t labapp "Lab 3.1 warning message"
cat /var/log/lab-local5.log
grep -c labapp /var/log/syslog
journalctl -t labapp --no-pager
```

```
2026-10-10T10:20:11.123456+00:00 ubuntu-lab labapp: Lab 3.1 info message
2026-10-10T10:20:12.654321+00:00 ubuntu-lab labapp: Lab 3.1 warning message
0
Oct 10 10:20:11 ubuntu-lab labapp[3301]: Lab 3.1 info message
Oct 10 10:20:12 ubuntu-lab labapp[3301]: Lab 3.1 warning message
```

What just happened: `-p facility.priority` sets the facility and priority; `-t` sets the tag (program name). The messages landed in the new file and not in `syslog` (count 0), but the journal still has them because journald receives everything before rsyslog does.

4. Create a logrotate policy for the new file.

```
cat <<'EOF' | sudo tee /etc/logrotate.d/lab-local5
/var/log/lab-local5.log {
    su root adm
    daily
    rotate 4
    compress
    delaycompress
    missingok
    notifempty
    create 0640 syslog adm
    postrotate
        /usr/lib/rsyslog/rsyslog-rotate
    endscript
}
EOF
```

What just happened: each directive in the block:

- `daily` rotate once a day (`weekly` , `monthly` , or `size 10M` are alternatives); `rotate 4` keep four old copies.
- `compress` gzip old logs; `delaycompress` wait one cycle so the newest old log (`.1`) stays plain text.
- `missingok` no error if the file is missing; `notifempty` skip rotation if it is empty.
- `create 0640 syslog adm` make a fresh empty file with these permissions and owner.
- `postrotate ... endscript` commands run after rotating; Ubuntu's `rsyslog-rotate` script tells rsyslog to reopen its files, otherwise it would keep writing into the renamed file.
- `su root adm` rotate as this user and group. It is needed because `/var/log` is group-writable, which logrotate considers insecure without it. (Ubuntu's main `/etc/logrotate.conf` sets this globally, but you are about to run this file on its own.)

5. Dry-run, then force a rotation.

```
sudo logrotate -d /etc/logrotate.d/lab-local5 2>&1 | tail -6
sudo logrotate -f /etc/logrotate.d/lab-local5
logger -p local5.info -t labapp "after rotation"
ls -l /var/log/lab-local5.log*
```

```
considering log /var/log/lab-local5.log
  Now: 2026-10-10 10:22
  Last rotated at 2026-10-10 10:00
  log does not need rotating (log has already been rotated)
...
-rw-r----- 1 syslog adm  64 Oct 10 10:23 /var/log/lab-local5.log
-rw-r----- 1 syslog adm 140 Oct 10 10:20 /var/log/lab-local5.log.1
```

What just happened: `-d` (debug) shows what logrotate **would** do and changes nothing. `-f` (force) rotates even if it is not due. The old content is now `lab-local5.log.1` (not compressed yet because of `delaycompress`), and the new message went to a fresh file, proving the postrotate script worked. On Ubuntu 24.04 logrotate runs daily from `logrotate.timer` (`systemctl list-timers logrotate*`), and it remembers last rotation times in `/var/lib/logrotate/status`.

Checkpoint: `ls /var/log/lab-local5.log*` shows the current file and `.1`, and the newest `logger` message is in the current file.

Task 8: Audit file changes with auditd

1. Install and check the audit daemon.

```
sudo apt install -y auditd
systemctl is-active auditd
sudo auditctl -s | head -4
sudo auditctl -l
```

```
active
enabled 1
failure 1
pid 4120
rate_limit 0
No rules
```

What just happened: `auditctl -s` shows kernel audit status (`enabled 1` = auditing on). `auditctl -l` lists loaded rules; there are none yet. On Rocky, auditd is installed and running by default.

2. Add a runtime watch on `/etc/passwd`.

```
sudo auditctl -w /etc/passwd -p wa -k passwd_changes
sudo auditctl -l
```

```
-w /etc/passwd -p wa -k passwd_changes
```

What just happened: `-w` watches a file or directory. `-p wa` sets which access types to record: `r` read, `w` write, `x` execute, `a` attribute change (permissions, owner). `-k` attaches a **key**, a label you search on later. This rule lives only in kernel memory and disappears at reboot.

3. Trigger the rule and search for it.

```
sudo useradd audittest
sudo ausearch -k passwd_changes -i | tail -8
```

```
type=PROCTITLE msg=audit(10/10/2026 10:30:02.114:812) : proctitle=useradd audittest
type=PATH msg=audit(10/10/2026 10:30:02.114:812) : item=1 name=/etc/passwd+ ...
type=SYSCALL msg=audit(10/10/2026 10:30:02.114:812) : arch=x86_64 syscall=rename
  success=yes ... auid=student uid=root ... comm=useradd exe=/usr/sbin/useradd
  key=passwd_changes
```

What just happened: `ausearch -k` finds records with that key; `-i` (interpret) turns numbers into names. Look at `auid=student uid=root`: the **audit UID** (login UID) is set once at login and survives `sudo`, so the record shows the real person (`student`) behind the root action. That is why audit logs are strong accounting evidence. The raw records are in `/var/log/audit/audit.log`.

4. Make a rule persistent. Rules files in `/etc/audit/rules.d/` are merged by `augenrules` into `/etc/audit/audit.rules`, which is loaded at boot.

```
cat <<'EOF' | sudo tee /etc/audit/rules.d/50-lab.rules
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k shadow_changes
EOF
sudo augenrules --load
sudo auditctl -l
sudo cat /etc/audit/audit.rules
```

```
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k shadow_changes
## This file is automatically generated from /etc/audit/rules.d
-D
-b 8192
--backlog_wait_time 60000
-f 1
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k shadow_changes
```

What just happened: `augenrules --load` concatenated all `*.rules` files in name order into `audit.rules` and loaded them. `-D` deletes old rules first, `-b` sets the kernel buffer size, `-f 1` means "print a message on failure" (`2` would panic the system). Never edit `audit.rules` directly; it is overwritten. Syscall rules are also possible, for example `-a always,exit -F arch=b64 -S unlink -F auid>=1000 -k deletes`.

5. Trigger the shadow rule and run reports.

```
echo 'audittest:Temp123!x' | sudo chpasswd
sudo ausearch -k shadow_changes -i --start recent | grep -m1 SYSCALL
sudo aureport --summary | head -12
sudo aureport -k
```

What just happened: `chpasswd` set a password, which writes `/etc/shadow`. `--start recent` limits the search to the last 10 minutes. `aureport` builds summaries from the audit log: `--summary` gives totals, `-k` lists events by key, `-au` lists authentication attempts, `-l` logins, `-f` file events.

Checkpoint: `sudo auditctl -l` shows both watches, and `sudo aureport -k` lists events for `passwd_changes` and `shadow_changes`.

Challenge

CHALLENGE

1. Write an rsyslog rule that sends all `authpriv` messages of priority `warning` or higher to `/var/log/lab-authwarn.log`, test it with `logger -p authpriv.warning`, and add a logrotate policy that keeps 7 compressed copies rotated weekly. Remove both when done.
2. Add an audit rule that records every execution of `/usr/bin/passwd` with key `passwd_exec`, run `passwd` as `student` (cancel with Ctrl+C), and find the event with `ausearch`. Remove the rule afterward.
3. Add `ExecReload=/usr/bin/true` to the `[Service]` section of `labdemo.service` (then `sudo systemctl daemon-reload`), change the polkit rule so `labops` members may also reload the unit (`verb == "reload"`), and prove that `systemctl reload labdemo` works as `opsuser` with no password prompt.

Clean up

1. On rocky-lab, remove the test file and Samba client (optional).

```
# on rocky-lab
rm -f /tmp/fromrocky.txt
sudo dnf remove -y samba-client
```

2. Remove the audit rules and auditd.

```
sudo rm -f /etc/audit/rules.d/50-lab.rules
sudo augenrules --load
sudo auditctl -D
sudo apt purge -y auditd
```

3. Remove the rsyslog rule, the log files, and the logrotate policy.

```
sudo rm -f /etc/rsyslog.d/30-lab.conf /etc/logrotate.d/lab-local5
sudo rm -f /var/log/lab-local5.log*
sudo systemctl restart rsyslog
```

4. Remove Samba, the share, and its user.

```
sudo cp /etc/samba/smb.conf.bak /etc/samba/smb.conf
sudo smbpasswd -x smbuser
sudo apt purge -y samba
sudo rm -rf /srv/labshare /etc/samba/smb.conf.bak
sudo userdel smbuser
```

5. Remove SSSD, Kerberos, realmd, and the LDAP server.

```
sudo systemctl stop sssd
sudo rm -f /etc/sssd/sssd.conf
sudo apt purge -y sssd sssd-tools sssd-common libnss-sss libpam-sss realmd \
  krb5-user krb5-config slapd ldap-utils
sudo apt autoremove -y
sudo rm -rf /var/lib/ldap /var/lib/sss /var/backups/slapd-* /var/backups/*.ldapdb
echo PURGE | sudo debconf-communicate slapd
grep -E '^(passwd|group):' /etc/nsswitch.conf
```

   The last command should show no `sss` entry. If it does, edit `/etc/nsswitch.conf` with `sudo nano` and remove the word `sss` from those lines.

6. Remove the polkit rule, demo service, and test users.

```
sudo rm -f /etc/polkit-1/rules.d/50-labops.rules
sudo systemctl stop labdemo
sudo rm -f /etc/systemd/system/labdemo.service
sudo systemctl daemon-reload
sudo userdel -r opsuser
sudo userdel -r labpam
sudo userdel audittest
sudo groupdel labops
sudo rm -f /etc/pam.d/sshd.bak
rm -rf ~/labs/3.1
```

Exam corner

- **AAA:** authentication = prove identity, authorization = decide what is allowed, accounting = record what happened.
- PAM files live in `/etc/pam.d/<service>`; types are `auth`, `account`, `password`, `session`.

- Control flags: `required` (fail at end), `requisite` (fail now), `sufficient` (succeed now if nothing failed), `optional`; plus `include` and `substack`.
- `pam_access.so` reads `/etc/security/access.conf`; `pam_limits.so` reads `/etc/security/limits.conf`; `pam_nologin.so` checks `/etc/nologin`.
- Polkit actions are listed by `pkaction`, tested with `pkcheck`, and run as root with `pkexec`. Local JavaScript rules go in `/etc/polkit-1/rules.d/*.rules`.
- LDAP: TCP 389, LDAPS 636; tools `ldapsearch`, `ldapadd`, `ldapmodify`; entries identified by DN such as `uid=jdoe,ou=people,dc=lab,dc=local`.
- Kerberos: port 88, KDC issues tickets; `kinit` gets a ticket, `klist` lists, `kdestroy` removes; config in `/etc/krb5.conf`; realm names are uppercase.
- SSSD (`/etc/sssd/sssd.conf`, mode 600) and Winbind connect Linux to LDAP/AD; `realm discover` and `realm join` set them up; `nsswitch.conf` needs `sss` (or `winbind`).
- Samba: config `/etc/samba/smb.conf`, check with `testparm`, users added with `smbpasswd -a`, client `smbclient`, port 445.
- `journalctl` filters: `-u unit`, `-p priority`, `-b`, `--since`, `-f`, `-k`; persistent storage needs `/var/log/journal` or `Storage=persistent`.
- rsyslog selectors are `facility.priority`; config in `/etc/rsyslog.conf` and `/etc/rsyslog.d/`; test messages with `logger`.
- logrotate: `/etc/logrotate.conf` and `/etc/logrotate.d/`; `-d` dry run, `-f` force. auditd: `auditctl -w ... -p ... -k ...`, persistent rules in `/etc/audit/rules.d/` compiled by `augenrules` into `audit.rules`; search with `ausearch -k`, report with `aureport`.

Review questions

1. A PAM module with the `requisite` flag fails. What happens next?
   A. PAM continues with the next module and fails at the end
   B. PAM stops immediately and returns failure
   C. PAM ignores the failure
   D. PAM returns success if a later module succeeds

2. Which command shows only error-level and more severe journal messages from the current boot?

3. You want an audit watch on `/etc/sudoers` to survive reboots. Where should you put the rule, and which command loads it?

4. Which file must contain `sss` on the `passwd:` line for SSSD users to appear in `getent passwd`?
   A. `/etc/sssd/sssd.conf`
   B. `/etc/pam.d/common-auth`
   C. `/etc/nsswitch.conf`
   D. `/etc/krb5.conf`

5. Which tool would you use to let a non-root group restart one specific systemd service without `sudo`?
   A. PAM
   B. Polkit

C. SSSD

D. logrotate

6. What does `logrotate -d /etc/logrotate.d/myapp` do?

Lab 3.2 Configure and Implement Firewalls

Objective: 3.2 Given a scenario, configure and implement firewalls on a Linux system.
Domain: 3.0 Security (18% of the exam)
Time: about 120 minutes
VM: ubuntu-lab and rocky-lab (both)
Snapshot first: Yes, on both VMs. A firewall mistake can cut off your SSH session.

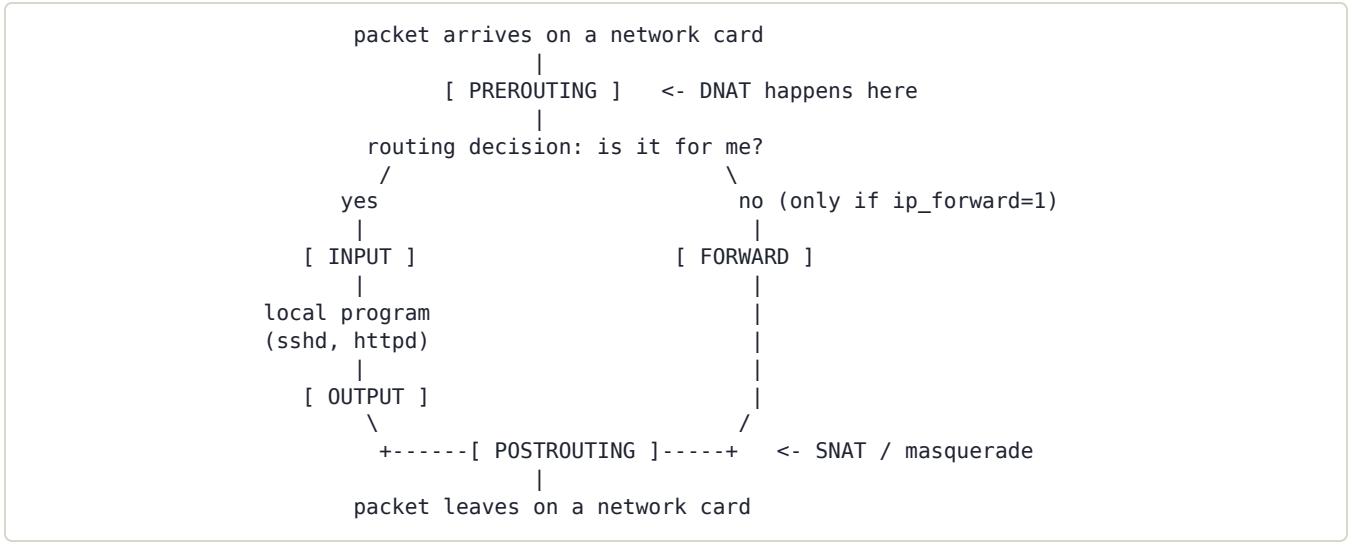
A firewall decides which network packets may enter, leave, or pass through a machine. In this lab you will drive the same Linux kernel firewall (Netfilter) through four different tools: firewalld on rocky-lab, ufw on ubuntu-lab, and raw nftables and iptables. Then you will turn ubuntu-lab into a small router that forwards a port to rocky-lab using NAT. These are daily tasks for any Linux administrator, and the exam expects you to pick the right command for a scenario.

What you'll practice

- **firewalld**: `firewall-cmd`, **runtime** vs **permanent** configuration, **rich rules**, **zones**, **ports** vs **services**
- **ufw**: allowing by **port** vs by **service** or application profile, numbered rules, deleting rules
- **nftables**: tables, chains, rules, counters, `nft list ruleset`
- **iptables** (the `iptables-nft` variant) and **ipset** block lists
- The **Netfilter** kernel module and its hooks
- Address translation: **NAT**, **PAT**, **DNAT**, **SNAT** (and masquerade)
- **Stateful** vs **stateless** filtering, connection tracking
- **IP forwarding** with `net.ipv4.ip_forward`

How it works

Every Linux firewall tool is just a front end. The real work happens in **Netfilter**, a framework inside the kernel that lets rules inspect packets at five fixed points called **hooks**:



- **INPUT** sees packets addressed to this machine. Most server firewall rules live here.
- **OUTPUT** sees packets this machine creates.
- **FORWARD** sees packets passing through (when the machine is a router).
- **PREROUTING** sees every incoming packet before the routing decision, so it is where **DNAT** (destination NAT, "port forwarding") changes where a packet goes.
- **POSTROUTING** sees every outgoing packet last, so it is where **SNAT** (source NAT) changes where a packet appears to come from.

Two rule languages talk to Netfilter:

- **iptables** is the classic tool (with `ip6tables`, `arptables`, `ebtables`). Rules live in tables (`filter`, `nat`, `mangle`, `raw`) and chains (`INPUT`, `FORWARD`, ...).
- **nftables** (`nft` command) is the modern replacement: one tool for IPv4, IPv6, and more, with your own named tables and chains. On Ubuntu 24.04 and Rocky 9, the `iptables` command is actually `iptables-nft`, which translates iptables syntax into nftables rules.

Higher-level managers write those rules for you:

- **firewalld** (default on Rocky/RHEL) groups rules into **zones** such as `public`, `internal`, and `trusted`. Each network interface or source address belongs to a zone, and each zone has its own allowed services and ports. Changes are either **runtime** (active now, lost at reload/reboot) or **permanent** (saved to disk, active after reload).
- **ufw** ("uncomplicated firewall", default on Ubuntu but **inactive** after install) uses simple commands like `ufw allow 22/tcp`.

Ports vs services: a port is a number and protocol (`80/tcp`). A service is a named bundle of one or more ports (`http` = 80/tcp, `samba` = 137-138/udp + 139,445/tcp). Services are easier to read and audit.

Stateful vs stateless: a **stateless** filter judges every packet alone: "is this packet to port 22?". A **stateful** filter uses **connection tracking** (`nf_conntrack`): it remembers connections, so you can write "allow anything that belongs to a connection I already accepted" (`ct state established,related`). Without state, you must open holes for reply traffic by hand, which is clumsy and less safe.

NAT (Network Address Translation) rewrites addresses:

- **SNAT** changes the source address (outbound). **Masquerade** is SNAT that automatically uses the outgoing interface's current address.
- **PAT** (Port Address Translation, also called NAPT or "NAT overload") lets many hosts share one public IP by also rewriting source ports. Home routers do this; Linux masquerade does too.
- **DNAT** changes the destination address/port (inbound), for example "send traffic for router:8080 to web server:80".

A Linux box only routes packets between hosts if **IP forwarding** is on: `net.ipv4.ip_forward = 1`.

Tasks

Task 1: Look at Netfilter on both VMs

1. On ubuntu-lab, install the tools used in this lab and check what is loaded.

```
mkdir -p ~/labs/3.2 && cd ~/labs/3.2
sudo apt install -y nftables iptables ipset conntrack ufw
sudo iptables -V
lsmod | grep -E '^(nf_tables|nf_conntrack|x_tables|nft_)' | head
sysctl net.ipv4.ip_forward
```

```
iptables v1.8.10 (nf_tables)
nf_tables             376832  0
nf_conntrack          196608  0
x_tables               65536  1 nf_tables
net.ipv4.ip_forward = 0
```

> **What just happened:** `iptables -V` shows `(nf_tables)`, proving that the iptables command is the nftables-backed version. `lsmod` lists loaded kernel modules: `nf_tables` is the nftables engine, `nf_conntrack` is connection tracking, `x_tables` supports old-style matches. Netfilter modules load on demand, so your list may be shorter. `ip_forward = 0` means ubuntu-lab will not route packets between hosts.

2. Show the whole ruleset and connection tracking table.

```
sudo nft list ruleset
sudo conntrack -L 2>/dev/null | head -3
```

```
tcp      6 431997 ESTABLISHED src=192.168.100.1 dst=192.168.100.10 sport=50522
  dport=22 src=192.168.100.10 dst=192.168.100.1 sport=22 dport=50522 [ASSURED] mark=0
```

What just happened: `nft list ruleset` prints every table in every family. With ufw inactive it is usually empty or holds only empty iptables-nft tables. `conntrack -L` shows connections the kernel is tracking. Each line has the original direction (your SSH client to port 22) and the reply direction; this table is what makes stateful filtering possible.

3. On rocky-lab, see that firewalld is active and has written its own nftables table.

```
# on rocky-lab
sudo firewall-cmd --state
sudo nft list tables
```

```
running
table inet firewalld
```

Checkpoint: you can explain which hook a packet to a local web server passes (PREROUTING, INPUT) and which hooks a routed packet passes (PREROUTING, FORWARD, POSTROUTING).

Task 2: firewalld zones, services, and ports

1. On rocky-lab, inspect the zones.

```
# on rocky-lab
sudo firewall-cmd --get-zones
sudo firewall-cmd --get-default-zone
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
```

```
block dmz drop external home internal nm-shared public trusted work
public
public
  interfaces: enp1s0
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp1s0
  sources:
  services: cockpit dhcpv6-client ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

What just happened: zones are trust levels. `drop` silently drops everything, `block` rejects everything, `public` allows only chosen services, `trusted` allows everything. The **default zone** gets any interface not assigned elsewhere. **Active zones** are those with an interface or source bound to them. `--list-all` shows what the zone allows: here only the `cockpit`, `dhcpv6-client`, and `ssh` services. Anything else is rejected. Your interface name (`enp1s0`, `ens18`, `eth0`) will differ.

2. Look at how a service is defined.

```
# on rocky-lab
sudo firewall-cmd --info-service=http
cat /usr/lib/firewalld/services/http.xml
```

```
http
  ports: 80/tcp
  protocols:
  ...
<?xml version="1.0" encoding="utf-8"?>
<service>
  <short>WWW (HTTP)</short>
  <description>HTTP is the protocol used to serve Web pages. ...</description>
  <port protocol="tcp" port="80"/>
</service>
```

What just happened: a firewalld service is an XML file listing ports. Packaged services live in `/usr/lib/firewalld/services/`; your own go in `/etc/firewalld/services/`. `firewall-cmd --get-services` lists all names.

3. Start a small test web server on rocky-lab. Open a **second SSH session** to rocky-lab and leave this running there.

```
# on rocky-lab (second session)
mkdir -p ~/labs/3.2/www
echo "Hello from rocky-lab" > ~/labs/3.2/www/index.html
python3 -m http.server 8000 --directory ~/labs/3.2/www
```

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

4. From ubuntu-lab, try to reach it.

```
curl -m 5 http://rocky-lab:8000/
```

```
curl: (7) Failed to connect to rocky-lab port 8000 after 3 ms: Couldn't connect
to server
```

What just happened: `-m 5` gives up after 5 seconds. The connection failed **immediately**: firewalld's `public` zone **rejects** unknown traffic (it sends back an ICMP "prohibited" message), so curl knows at once. A **drop** would make curl wait until the timeout instead.

5. Open the port (runtime only) and test again.

```
# on rocky-lab
sudo firewall-cmd --add-port=8000/tcp
sudo firewall-cmd --list-ports
```

```
curl -m 5 http://rocky-lab:8000/
```

```
success
8000/tcp
Hello from rocky-lab
```

What just happened: `--add-port=8000/tcp` opened the port in the default zone, effective immediately. The second command block runs on ubuntu-lab and now gets the page. The python server window on rocky-lab logs the request from `192.168.100.10`.

6. Use a source-based zone: put ubuntu-lab's IP into the `internal` zone.

```
# on rocky-lab
sudo firewall-cmd --zone=internal --add-source=192.168.100.10/32
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --zone=internal --list-services
```

```
success
internal
  sources: 192.168.100.10/32
public
  interfaces: enp1s0
cockpit dhcpv6-client mdns samba-client ssh
```

What just happened: firewalld picks a zone for each packet by **source address first**, then by interface. Packets from ubuntu-lab are now judged by the `internal` zone, not `public`. Port 8000 is open only in `public`, so from ubuntu-lab `curl -m 5 http://rocky-lab:8000/` fails again. This is how you give a trusted admin subnet more access than the internet. You do not need to undo it by hand: it is a runtime change, so the reload at the start of the next task removes it (the manual way is `sudo firewall-cmd --zone=internal --remove-source=192.168.100.10/32`).

Checkpoint: `sudo firewall-cmd --get-active-zones` on rocky-lab shows `internal` with your ubuntu-lab source, and you can explain why curl from ubuntu-lab now fails.

Task 3: Runtime vs permanent in firewalld

1. Reload firewalld and watch your runtime changes disappear.

```
# on rocky-lab
sudo firewall-cmd --reload
sudo firewall-cmd --list-ports
sudo firewall-cmd --get-active-zones
```

```
success

public
  interfaces: enp1s0
```

> **What just happened:** `--reload` throws away the **runtime** configuration and rebuilds it from the **permanent** configuration on disk (`/etc/firewalld/zones/*.xml`). Port 8000 and the `internal` source were runtime only, so they are gone. A reboot or `systemctl restart firewalld` has the same effect. From ubuntu-lab, curl fails again.

2. Add the port permanently and notice it is **not** active yet.

```
# on rocky-lab
sudo firewall-cmd --permanent --add-port=8000/tcp
sudo firewall-cmd --list-ports
sudo firewall-cmd --permanent --list-ports
sudo firewall-cmd --reload
sudo firewall-cmd --list-ports
```

```
success

8000/tcp
success
8000/tcp
```

> **What just happened:** `--permanent` writes only to disk; the running firewall is unchanged until a reload. That is why the first `--list-ports` was empty and the `--permanent --list-ports` showed the port. The usual pattern is either "run the command twice (with and without `--permanent`)" or "make runtime changes, test them, then save with `sudo firewall-cmd --runtime-to-permanent` ".

3. Allow a service by name instead of a port number, and look at the file firewalld saved.

```
# on rocky-lab
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
sudo cat /etc/firewalld/zones/public.xml
```

```
cockpit dhcpv6-client http ssh
<?xml version="1.0" encoding="utf-8"?>
<zone>
  <short>Public</short>
  ...
  <service name="ssh"/>
  <service name="dhcpv6-client"/>
  <service name="cockpit"/>
  <service name="http"/>
  <port port="8000" protocol="tcp"/>
  <forward/>
</zone>
```

4. Remove both permanent changes now; you will use rich rules next.

```
# on rocky-lab
sudo firewall-cmd --permanent --remove-service=http
sudo firewall-cmd --permanent --remove-port=8000/tcp
sudo firewall-cmd --reload
sudo firewall-cmd --list-all | grep -E 'services|ports'
```

> **TIP**
> A safe trick when testing remotely: `sudo firewall-cmd --add-port=8000/tcp --timeout=5m` opens a runtime rule that removes itself after five minutes.

Checkpoint: `sudo firewall-cmd --permanent --list-ports` is empty and `--list-services` shows only `cockpit dhcpv6-client ssh` again.

Task 4: firewalld rich rules

Rich rules express conditions that plain ports and services cannot: “only from this address”, “log and accept”, “reject this one host”.

1. Allow port 8000 only from ubuntu-lab. Replace the IP with yours.

```
# on rocky-lab
sudo firewall-cmd --add-rich-rule='rule family="ipv4"'\
' source address="192.168.100.10"'\
' port port="8000" protocol="tcp" accept'
sudo firewall-cmd --list-rich-rules
```

```
success
rule family="ipv4" source address="192.168.100.10" port port="8000" protocol="tcp"
  accept
```

The rule is one long string. To keep lines short, it is split into three single-quoted pieces; a `\` at the end of a line continues the command, and quoted pieces written right next to each other join into one argument. You can also type it as one line.

What just happened: the rich rule language reads like a sentence: `rule` + `family` + optional `source` / `destination` + one element (`service` , `port` , `protocol` , ...) + optional `log` + an action (`accept` , `reject` , `drop`). Keep the whole rule inside single quotes so the shell does not split it. From ubuntu-lab, `curl -m 5 http://rocky-lab:8000/` works; any other host would still be rejected.

2. Now open the port for everyone, but reject ubuntu-lab, and log the rejections.

```
# on rocky-lab
sudo firewall-cmd --remove-rich-rule='rule family="ipv4"'\
' source address="192.168.100.10"'\
' port port="8000" protocol="tcp" accept'
sudo firewall-cmd --add-port=8000/tcp
sudo firewall-cmd --add-rich-rule='rule family="ipv4"'\
' source address="192.168.100.10"'\
' port port="8000" protocol="tcp"'\
' log prefix="LAB32-REJECT " level="info" reject'
```

Test from ubuntu-lab:

```
curl -m 5 http://rocky-lab:8000/
```

```
curl: (7) Failed to connect to rocky-lab port 8000 after 2 ms: Couldn't connect
to server
```

Then read the log on rocky-lab:

```
# on rocky-lab
sudo journalctl -k --since "5 minutes ago" | grep LAB32
```

```
... kernel: LAB32-REJECT IN=enp1s0 OUT= MAC=... SRC=192.168.100.10 DST=192.168.100.20
  ... PROTO=TCP SPT=40216 DPT=8000 ... SYN
```

What just happened: firewalld evaluates rich rules that **deny or reject before** rules that allow, so the reject for one host wins even though the port is open to all. The `log` element writes a kernel message with your prefix, showing source, destination, ports, and TCP flags.

3. Reset to the permanent state.

```
# on rocky-lab
sudo firewall-cmd --reload
sudo firewall-cmd --list-rich-rules
```

Checkpoint: after the reload, `--list-rich-rules` prints nothing and `--list-ports` is empty. Stop the python server with Ctrl+C for now; you will start one again in Task 8.

Task 5: ufw on ubuntu-lab

> **CAUTION**
> If you enable ufw without allowing SSH first, the default "deny incoming" policy will block new SSH connections. Always allow OpenSSH **before** `ufw enable`, and keep the VM console available.

1. Check the status and the application profiles.

```
sudo ufw status
sudo ufw app list
sudo ufw app info OpenSSH
```

```
Status: inactive
Available applications:
  OpenSSH
Profile: OpenSSH
Title: Secure shell server, an rshd replacement
Description: OpenSSH is a free implementation of the Secure Shell protocol.

Port:
  22/tcp
```

> **What just happened:** ufw is installed but **inactive** by default on Ubuntu Server. Application profiles are small files in `/etc/ufw/applications.d/` that packages install (here by `openssh-server`), mapping a name to ports, similar to firewalld services.

2. Allow SSH, then enable the firewall.

```
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
```

```
Rules updated
Rules updated (v6)
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
OpenSSH                    ALLOW IN    Anywhere
OpenSSH (v6)               ALLOW IN    Anywhere (v6)
```

What just happened: the defaults deny all incoming traffic, allow all outgoing, and deny routed (forwarded) traffic. Your SSH session survives because it matches the OpenSSH rule (and existing connections are tracked as established). `enable` also makes ufw start at boot.

3. Start a test server on ubuntu-lab in a second session, and test from rocky-lab.

```
# ubuntu-lab, second session
mkdir -p ~/labs/3.2/www && echo "Hello from ubuntu-lab" > ~/labs/3.2/www/index.html
python3 -m http.server 8000 --directory ~/labs/3.2/www
```

```
# on rocky-lab
curl -m 5 http://ubuntu-lab:8000/
```

```
curl: (28) Connection timed out after 5001 milliseconds
```

What just happened: unlike firewalld's reject, ufw's default policy **drops** packets silently, so curl waits for the full timeout. Dropping gives attackers less information; rejecting gives legitimate users a fast error.

4. Allow by port, by service name, and from one address only.

```
sudo ufw allow 8000/tcp
sudo ufw allow http
sudo ufw allow from 192.168.100.20 to any port 9000 proto tcp
sudo ufw status numbered
```

```
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] OpenSSH                    ALLOW IN    Anywhere
[ 2] 8000/tcp                   ALLOW IN    Anywhere
[ 3] 80/tcp                     ALLOW IN    Anywhere
[ 4] 9000/tcp                   ALLOW IN    192.168.100.20
[ 5] OpenSSH (v6)               ALLOW IN    Anywhere (v6)
[ 6] 8000/tcp (v6)              ALLOW IN    Anywhere (v6)
[ 7] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

What just happened: `8000/tcp` is a port rule. `http` is a **service name** that ufw looks up in `/etc/services` (port 80/tcp). `OpenSSH` is an **application profile**. The `from ... to any port ... proto tcp` form limits a rule to one source. ufw checks rules top to bottom and the first match wins; `sudo ufw insert 1 deny from 192.168.100.50` would put a rule at the top. The curl from rocky-lab now works.

5. Delete rules by number and by specification.

```
sudo ufw delete 4
sudo ufw delete allow http
sudo ufw status numbered
```

What just happened: `delete 4` asks for confirmation and removes rule number 4. Numbers shift after each delete, so always re-run `status numbered` before deleting by number again. `delete allow http` removes the rule matching that exact specification (both IPv4 and IPv6 versions).

6. See that ufw is just another front end, then turn it off and reset it.

```
sudo nft list ruleset | grep -c ufw
sudo ufw disable
sudo ufw --force reset
sudo ufw status
```

```
208
Firewall stopped and disabled on system startup
Backing up 'user.rules' to '/etc/ufw/user.rules.20261010_103512'
...
Status: inactive
```

What just happened: ufw writes iptables rules, which `iptables-nft` stores as nftables rules, so `nft` can see them (the count of lines mentioning ufw). `--force reset` deletes all ufw rules without asking and backs up the old rule files. Disabling ufw before the next tasks keeps it from interfering.

Checkpoint: `sudo ufw status` says `Status: inactive`. Leave the ubuntu-lab python server on port 8000 running for the next task.

Task 6: nftables by hand, and stateful vs stateless

1. Create your own table, chain, and a rule with a counter.

```
sudo nft add table inet lab
sudo nft add chain inet lab input \
  '{ type filter hook input priority 0; policy accept; }'
sudo nft add rule inet lab input tcp dport 8000 counter drop
sudo nft list table inet lab
```

```
table inet lab {
    chain input {
        type filter hook input priority filter; policy accept;
        tcp dport 8000 counter packets 0 bytes 0 drop
    }
}
```

What just happened:

- A **table** is a container. Family `inet` covers both IPv4 and IPv6 (`ip` , `ip6` , `arp` , `bridge` , `netdev` also exist).
- A **base chain** attaches to a Netfilter hook: `type filter hook input priority 0` means "filter packets at the INPUT hook". Lower priority numbers run earlier. `policy accept` is what happens to packets no rule decides on.
- The **rule** matches TCP destination port 8000, counts matches (`counter`), and drops them.
- The braces are quoted so the shell does not interpret them.

2. Test from rocky-lab, then look at the counter.

```
# on rocky-lab
curl -m 5 http://ubuntu-lab:8000/
```

```
sudo nft list chain inet lab input
```

```
curl: (28) Connection timed out after 5001 milliseconds
        tcp dport 8000 counter packets 3 bytes 180 drop
```

What just happened: curl timed out because the packets were dropped silently. The counter shows how many packets matched (the original SYN plus retries). Counters are the fastest way to prove a rule is actually being hit.

3. Change the rule to **reject**, using its handle.

```
sudo nft -a list chain inet lab input
sudo nft delete rule inet lab input handle 2
sudo nft add rule inet lab input tcp dport 8000 counter reject with tcp reset
```

```
        tcp dport 8000 counter packets 3 bytes 180 drop # handle 2
```

What just happened: `-a` shows each rule's **handle** (an ID number); use the number you see in your output. Rules are deleted by handle. `reject with tcp reset` answers with a TCP RST packet, so from rocky-lab `curl -m 5 http://ubuntu-lab:8000/` now fails at once with `Connection refused` .

4. Delete the table and build a **stateful** firewall from a file. Loading a file with `nft -f` is atomic: all rules apply at once or none do, so you are never half-protected.

CAUTION

This ruleset has `policy drop` . It allows SSH, but if you mistype it you could lose access. As a safety net, the first command below schedules automatic deletion of the table in 15 minutes. If you get locked out, wait, or use the VM console to run `sudo nft delete table inet lab` .

```
sudo nft delete table inet lab
sudo systemd-run --on-active=15m /usr/sbin/nft delete table inet lab
cat <<'EOF' > ~/labs/3.2/stateful.nft
table inet lab {
    chain input {
        type filter hook input priority 0; policy drop;
        iif "lo" accept
        ct state established,related counter accept
        ct state invalid drop
        tcp dport 22 accept
        meta l4proto ipv6-icmp accept
        counter comment "dropped by policy"
    }
}
EOF
sudo nft -c -f ~/labs/3.2/stateful.nft && sudo nft -f ~/labs/3.2/stateful.nft
sudo nft list table inet lab
```

What just happened:

- `systemd-run --on-active=15m` creates a one-time timer that runs the command in 15 minutes: a "dead man's switch" for remote firewall work.
- The heredoc (`cat <<'EOF' > file`) writes everything up to `EOF` into the file.
- `nft -c -f` checks the file without applying it; `&&` only loads it if the check passed.
- Rules: allow loopback, allow packets of **established or related** connections (replies to things you started, and things like FTP data or ICMP errors linked to them), drop invalid packets, allow new SSH connections, allow IPv6 control messages (needed for IPv6 neighbor discovery), and count everything else before the drop policy kills it.

5. Prove stateful filtering works: outbound traffic still gets its replies.

```
ping -c 2 rocky-lab
sudo nft list chain inet lab input | grep established
```

```
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
        ct state established,related counter packets 45 bytes 3880 accept
```

What just happened: there is no rule allowing incoming ICMP echo replies, yet the ping worked. Connection tracking saw your outgoing echo request, so the incoming reply was part of an "established" flow and matched the `ct state` rule. Note that rocky-lab cannot ping **you** (new incoming connections are dropped), which is exactly what you want from a host firewall.

6. Now make it **stateless**: remove the conntrack rule and ping again.

```
sudo nft -a list chain inet lab input | grep established
sudo nft delete rule inet lab input handle 3
ping -c 2 -W 2 rocky-lab
```

```
        ct state established,related counter packets 51 bytes 4400 accept # handle 3
2 packets transmitted, 0 received, 100% packet loss, time 1025ms
```

What just happened: use the handle number from your own output. Without state, the reply packets are judged alone, and no rule allows them, so they are dropped. Your SSH session survives because each of its incoming packets matches `tcp dport 22` on its own merits. A stateless firewall must open reply traffic explicitly, like the next command. For ping that is fine, but for TCP you would need rules like "allow anything **from** source port 80", which an attacker can forge. That is why real firewalls are stateful.

```
sudo nft add rule inet lab input icmp type echo-reply accept
ping -c 2 rocky-lab
```

7. Remove the table (and cancel the safety timer if it has not fired).

```
sudo nft delete table inet lab
systemctl list-timers --all | grep run-
```

If a `run-...timer` is still listed, stop it with `sudo systemctl stop <name>.timer` (use the name shown).

Checkpoint: `sudo nft list tables` no longer shows `table inet lab`, and you can explain why ping replies were blocked once the `ct state` rule was removed.

Task 7: iptables and ipset

1. Add a rule with iptables and find it in nftables.

```
sudo iptables -A INPUT -p tcp --dport 8000 -j REJECT
sudo iptables -L INPUT -n -v --line-numbers
sudo nft list chain ip filter INPUT
```

```
Chain INPUT (policy ACCEPT 120 packets, 9010 bytes)
num   pkts bytes target  prot opt in  out  source     destination
1        0     0 REJECT  tcp  --  *   *    0.0.0.0/0  0.0.0.0/0   tcp dpt:8000
                                                      reject-with icmp-port-unreachable
# Warning: table ip filter is managed by iptables-nft, do not touch!
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy accept;
        tcp dport 8000 counter packets 0 bytes 0 reject
    }
}
```

What just happened: `-A INPUT` appends a rule to the INPUT chain of the default `filter` table. `-p tcp --dport 8000` matches TCP port 8000; `-j REJECT` is the target (action). For listing, `-L` lists, `-n` shows numbers instead of names (faster, no DNS), `-v` adds counters and interfaces, `--line-numbers` numbers the rules. The same rule appears in `nft` because `iptables-nft` translated it. From rocky-lab, curl to port 8000 now gets `Connection refused`.

TIP

Other iptables actions to know: `-I INPUT 1` inserts at position 1, `-D INPUT 1` deletes rule 1, `-F` flushes (deletes all rules in) a chain, `-P INPUT DROP` sets the chain policy, and `-t nat` selects the NAT table. A stateful rule in iptables looks like `iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`. Rules are lost at reboot unless saved with `iptables-save` and restored with `iptables-restore` (the `iptables-persistent` package does this on Ubuntu, but it conflicts with ufw).

2. Delete the rule.

```
sudo iptables -D INPUT 1
sudo iptables -L INPUT -n --line-numbers
```

3. Create an **ipset** block list and reference it from one iptables rule.

```
sudo ipset create blocklist hash:ip
sudo ipset add blocklist 192.168.100.20
sudo ipset list blocklist
sudo iptables -I INPUT -m set --match-set blocklist src -j DROP
```

```
Name: blocklist
Type: hash:ip
Revision: 6
Header: family inet hashsize 1024 maxelem 65536 bucketsize 12 initval 0x4a1c2b3d
Size in memory: 216
References: 0
Number of entries: 1
Members:
192.168.100.20
```

What just happened: an **ipset** is a kernel-side list of addresses (or networks with `hash:net`, or address+port with `hash:ip,port`) that can be looked up very quickly. One iptables rule (`-m set --match-set blocklist src`) checks whether the packet's **source** is in the set. Blocking 10,000 bad IPs with one rule and one set is far faster than 10,000 separate rules. Replace the IP with rocky-lab's real address. Do not add the address you are SSHing from.

4. Test, then change the list without touching iptables.

```
# on rocky-lab
ping -c 2 -W 2 ubuntu-lab
```

```
sudo ipset del blocklist 192.168.100.20
```

```
# on rocky-lab
ping -c 2 ubuntu-lab
```

What just happened: the first ping from rocky-lab gets 100% loss. After removing the address from the set on ubuntu-lab, the second ping works, although the iptables rule never changed. Tools like fail2ban use this trick. nftables has built-in sets that do the same job (`set blocklist { type ipv4_addr; }`).

5. Remove the rule, then the set (a set in use cannot be destroyed).

```
sudo iptables -D INPUT -m set --match-set blocklist src -j DROP
sudo ipset destroy blocklist
sudo ipset list
```

Checkpoint: `sudo iptables -L INPUT -n` shows no rules and `sudo ipset list` prints nothing. Stop the ubuntu-lab python server with Ctrl+C.

Task 8: NAT and IP forwarding: forward ubuntu-lab:8080 to rocky-lab:80

You will make ubuntu-lab act as a router: connections to `ubuntu-lab:8080` get forwarded (DNAT) to a web server on `rocky-lab:80`, and ubuntu-lab rewrites the source (masquerade, a form of SNAT/PAT) so replies come back through it.

```
client (rocky-lab)           ubuntu-lab (router)              server (rocky-lab:80)
src=rocky dst=ubuntu:8080 --> PREROUTING:  DNAT dst->rocky:80
                              FORWARD:     allowed?
                              POSTROUTING: SNAT src->ubuntu --> sees src=ubuntu,
                                                                replies to ubuntu
reply from ubuntu:8080   <--  conntrack reverses both    <--
```

With only two VMs, rocky-lab plays both the client and the web server. That is fine: the packets really do travel to ubuntu-lab and back.

What just happened: read the diagram left to right. Each arrow is a real packet on the virtual network. The router changes the destination on the way in (DNAT), changes the source on the way out (SNAT), and remembers both changes in the connection tracking table so it can undo them on the reply. The client never learns that the web server is really rocky-lab:80.

1. On rocky-lab, start a web server on port 80 and open the http service (runtime).

```
# on rocky-lab
sudo firewall-cmd --add-service=http
sudo python3 -m http.server 80 --directory /home/student/labs/3.2/www
```

Leave it running in that session.

2. On ubuntu-lab, confirm forwarding is off and the FORWARD policy is not blocking.

```
sysctl net.ipv4.ip_forward
sudo iptables -L FORWARD -n | head -1
```

```
net.ipv4.ip_forward = 0
Chain FORWARD (policy ACCEPT)
```

If the policy says `DROP`, ufw is probably still enabled; run `sudo ufw disable`.

3. Write the NAT ruleset. Replace `192.168.100.20` with rocky-lab's IP.

```
cat <<'EOF' > ~/labs/3.2/nat.nft
define ROCKY = 192.168.100.20

table ip labnat {
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;
        tcp dport 8080 counter dnat to $ROCKY:80
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        ip daddr $ROCKY tcp dport 80 ct state new counter accept
    }
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        ip daddr $ROCKY tcp dport 80 counter masquerade
    }
}
EOF
sudo nft -c -f ~/labs/3.2/nat.nft && sudo nft -f ~/labs/3.2/nat.nft
```

What just happened:

- `define` creates a variable used as `$ROCKY`.
- Chains of `type nat` do address translation. `priority dstnat` (-100) and `srcnat` (100) are the standard positions.
- **prerouting:** packets arriving for TCP 8080 get their destination rewritten to rocky-lab port 80. This is **DNAT** (a port forward).
- **forward:** a stateful forwarding filter. It drops everything except new connections to the web server and their replies.
- **postrouting:** forwarded packets to the web server get their source rewritten to ubuntu-lab's own address (**masquerade**, a dynamic **SNAT**). Because the kernel may also change the source port to keep connections unique, this is also **PAT**. Without it, rocky-lab would answer itself directly and the connection would break.

4. Test **before** enabling forwarding.

```
# on rocky-lab (first session)
curl -m 5 http://ubuntu-lab:8080/
```

```
sudo nft list chain ip labnat prerouting
```

```
curl: (28) Connection timed out after 5002 milliseconds
        tcp dport 8080 counter packets 3 bytes 180 dnat to 192.168.100.20:80
```

What just happened: the DNAT counter went up, so the packet was translated. But after translation the packet was not addressed to ubuntu-lab any more, and with `ip_forward = 0` the kernel refuses to route it, so it was discarded.

5. Turn on IP forwarding at runtime and test again.

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
# on rocky-lab (first session)
curl -m 5 http://ubuntu-lab:8080/
```

```
net.ipv4.ip_forward = 1
Hello from rocky-lab
```

Look at the python server window on rocky-lab:

```
192.168.100.10 - - [10/Oct/2026 10:55:02] "GET / HTTP/1.1" 200 -
```

What just happened: `sysctl -w` changes a kernel parameter immediately (the same as writing `1` to `/proc/sys/net/ipv4/ip_forward`). Now the request reached rocky-lab:80 through ubuntu-lab. The web server log shows the client as `192.168.100.10` (ubuntu-lab), not rocky-lab: that is the masquerade/SNAT at work.

6. Look at the translation in the connection tracking table.

```
# run within ~2 minutes of a curl
sudo conntrack -L -p tcp --orig-port-dst 8080 2>/dev/null
```

```
tcp  6 117 TIME_WAIT src=192.168.100.20 dst=192.168.100.10 sport=45872 dport=8080
  src=192.168.100.20 dst=192.168.100.10 sport=80 dport=45872 [ASSURED] mark=0 use=1
```

What just happened: the first half is the original packet (rocky to ubuntu:8080). The second half is the expected **reply**: from rocky port 80 back to ubuntu. The difference between the two halves is exactly the NAT that was applied: destination changed to rocky:80, and replies go to ubuntu (masquerade). Conntrack uses this entry to reverse the translation on reply packets.

7. Make forwarding persistent (how a real router is configured), then check it.

```
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-lab-forward.conf
sudo sysctl --system | grep -A1 99-lab
```

```
* Applying /etc/sysctl.d/99-lab-forward.conf ...
net.ipv4.ip_forward = 1
```

What just happened: files in `/etc/sysctl.d/*.conf` are applied at boot (and by `sysctl --system` now). This is the recommended way; editing `/etc/sysctl.conf` also works. Your nftables rules, however, are still runtime only: to keep them you would put them in `/etc/nftables.conf` and `sudo systemctl enable nftables`.

For comparison, the same NAT setup in iptables syntax (do not run it, just read it):

```
iptables -t nat -A PREROUTING -p tcp --dport 8080 \
  -j DNAT --to-destination 192.168.100.20:80
iptables -A FORWARD -p tcp -d 192.168.100.20 --dport 80 -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -t nat -A POSTROUTING -d 192.168.100.20 -p tcp --dport 80 -j MASQUERADE
# fixed-address SNAT instead of masquerade:
iptables -t nat -A POSTROUTING -o eth0 -j SNAT --to-source 203.0.113.5
```

TIP

On rocky-lab, firewalld can do the same: `firewall-cmd --add-forward-port=port=8080:proto=tcp:toport=80:toaddr=192.168.100.30` and `firewall-cmd --add-masquerade`. The `external` zone has masquerade on by default.

Checkpoint: from rocky-lab, `curl http://ubuntu-lab:8080/` returns `Hello from rocky-lab`, and the rocky-lab server log shows the request coming from ubuntu-lab's IP.

Challenge

CHALLENGE

1. On rocky-lab, create a permanent custom firewalld service called `labapp` for TCP 8000-8001 (put an XML file in `/etc/firewalld/services/` or use `firewall-cmd --permanent --new-service`), allow it only in the `internal` zone with ubuntu-lab as a source, and prove it works. Remove everything afterward.
2. Rewrite the Task 6 stateful firewall using an nftables named set of allowed TCP ports (`set allowed { type inet_service; elements = { 22, 8000 } }`) and a single rule `tcp dport @allowed accept`. Add port 9000 to the set at runtime with `nft add element`.
3. Using iptables syntax only, recreate the Task 8 port forward on ubuntu-lab and confirm the rules appear in `nft list ruleset`.

Clean up

1. On rocky-lab, stop both python servers (Ctrl+C) and reset firewalld to its permanent configuration.

```
# on rocky-lab
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
rm -rf ~/labs/3.2
```

   The services line should read `cockpit dhcpv6-client ssh`, with no ports or rich rules.

2. On ubuntu-lab, remove the NAT table, forwarding setting, and any leftovers.

```
sudo nft delete table ip labnat
sudo nft delete table inet lab 2>/dev/null
sudo rm -f /etc/sysctl.d/99-lab-forward.conf
sudo sysctl -w net.ipv4.ip_forward=0
sudo ipset destroy blocklist 2>/dev/null
sudo iptables -L INPUT -n --line-numbers
```

   The INPUT chain should list no lab rules; delete any you find with `sudo iptables -D INPUT <num>`.

3. Make sure ufw is off and reset, and remove the working directory.

```
sudo ufw disable
sudo ufw --force reset
sudo rm -f /etc/ufw/*.rules.[0-9]*
rm -rf ~/labs/3.2
sysctl net.ipv4.ip_forward
```

   The `rm -f` line deletes the timestamped backup copies that `ufw reset` made (names like `user.rules.20261010_103512`). The last command should print `net.ipv4.ip_forward = 0`.

Exam corner

- Netfilter hooks/chains: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING. DNAT in PREROUTING, SNAT/masquerade in POSTROUTING.
- `firewall-cmd` changes are **runtime** unless you add `--permanent`; permanent changes need `--reload`. `--runtime-to-permanent` saves the current runtime.
- firewalld: `--get-active-zones`, `--list-all`, `--add-service=`, `--add-port=80/tcp`, `--add-source=`, `--zone=`, `--add-rich-rule='rule ...'`, `--add-masquerade`, `--add-forward-port=`.
- A service is a named set of ports (`/usr/lib/firewalld/services/*.xml`); ufw service names come from `/etc/services` and app profiles from `/etc/ufw/applications.d/`.
- ufw: allow SSH **before** `ufw enable`; `ufw status numbered`, `ufw delete <n>`, `ufw allow from <ip> to any port <p> proto tcp`, `ufw default deny incoming`.
- nftables: `nft list ruleset`, `nft add table|chain|rule`, `nft -a` shows handles, `nft delete rule ... handle N`, `nft -f file` loads atomically; config in `/etc/nftables.conf`.

- iptables: `-A` append, `-I` insert, `-D` delete, `-L -n -v` list, `-F` flush, `-P` policy, `-t nat`, `-j ACCEPT|DROP|REJECT|DNAT|SNAT|MASQUERADE`; save with `iptables-save`.
- ipset: `ipset create NAME hash:ip`, `ipset add`, referenced by `-m set --match-set NAME src`.
- **Stateful** = connection tracking (`ct state established,related`, `-m conntrack --ctstate`); **stateless** = each packet judged alone.
- **NAT** types: SNAT (source, outbound), masquerade (dynamic SNAT), PAT/NAPT (many hosts share one IP via ports), DNAT (destination, port forwarding).
- Routing needs `net.ipv4.ip_forward = 1`: `sysctl -w` for now, `/etc/sysctl.d/*.conf` for boot.
- DROP = silent (client times out); REJECT = immediate error (client sees refused/unreachable).

Review questions

1. You run `firewall-cmd --add-port=8443/tcp`, test it, and it works. After a reboot the port is closed. Why, and what single command would have saved the working configuration?

2. Which Netfilter hook is used for DNAT (port forwarding)?
   A. INPUT
   B. OUTPUT
   C. PREROUTING
   D. POSTROUTING

3. A Linux router has correct NAT rules but forwarded traffic never leaves it. `sysctl net.ipv4.ip_forward` shows `0`. What command fixes it immediately, and where do you make it permanent?

4. Which ufw command must you run before `ufw enable` on a remote server?
   A. `ufw default allow incoming`
   B. `ufw allow OpenSSH`
   C. `ufw reset`
   D. `ufw logging on`

5. What is the main difference between a stateful and a stateless firewall rule?

6. Which statement about masquerade is true?
   A. It is a form of DNAT
   B. It is SNAT that uses the outgoing interface's current IP address
   C. It only works with IPv6
   D. It requires ipset

Lab 3.3 Operating System Hardening

Objective: 3.3 Given a scenario, apply operating system (OS) hardening techniques on a Linux system.
Domain: 3.0 Security (18% of the exam)
Time: about 180 minutes
VM: ubuntu-lab and rocky-lab (both)
Snapshot first: Yes, on both VMs. This lab edits `sudoers`, `sshd`, PAM, SELinux, and kernel module settings. A mistake can lock you out.

Hardening means closing the doors you are not using and putting good locks on the ones you need. This is the biggest objective in the Security domain, so this lab is long. You will tighten privilege escalation with `sudo`, protect files with attributes and permissions, add fine-grained ACLs, configure SELinux on rocky-lab, lock down SSH, run `fail2ban`, remove insecure services, block unused filesystems, and inspect Secure Boot. Every change here is something a real auditor or benchmark (such as the CIS Benchmarks) will ask you to prove.

What you'll practice

- Privilege escalation: `sudo`, `/etc/sudoers`, `/etc/sudoers.d`, `visudo`, `NOPASSWD`, `NOEXEC`, `sudo -i`, `su -`, the `wheel` and `sudo` groups
- File attributes: `chattr`, `lsattr` (immutable, append only)
- Permissions: `chown`, `chgrp`, `chmod` (octal and symbolic), special permissions (setuid, setgid, sticky bit), `umask`
- Access control: ACLs (`setfacl`, `getfacl`, default ACLs, mask)
- SELinux: states (enforcing, permissive, disabled), `getenforce`, `setenforce`, `ls -Z`, `restorecon`, `chcon`, `semanage`, `getsebool`, `setsebool`, `audit2allow`, `sealert`
- Secure remote access: SSHD (`PermitRootLogin`, `X11Forwarding`, `AllowUsers`, `AllowGroups`, key vs password auth), `ssh-agent`, `ssh-add`, SSH tunneling, SFTP chroot, `fail2ban`
- Avoiding insecure services (Telnet, FTP, TFTP), disabling unused filesystems, removing unnecessary SUID bits, Secure Boot (UEFI)

How it works

Hardening works on layers. If one layer fails, the next still protects you.

Privilege escalation. Nobody should log in as `root` for daily work. Instead, a trusted user runs a command with `sudo`, which checks `/etc/sudoers` (and drop-in files in `/etc/sudoers.d/`) and logs what happened. You edit sudoers only with `visudo`, which checks the syntax before saving so a typo cannot lock out every administrator. On Ubuntu the admin group is `sudo`; on Rocky it is `wheel`. Important options: `NOPASSWD` lets a rule run without a password (convenient but risky), and `NOEXEC` stops a command from spawning a shell. `sudo -i` starts a root login shell through sudo (logged, uses your password); `su -` switches to another user with a full login shell (needs that user's password).

File attributes are set on the filesystem itself, below permissions. `chattr +i` makes a file **immutable**: even root cannot change or delete it until the bit is removed. `chattr +a` makes it **append only**: you can add to it but not rewrite it, which is perfect for log files.

Permissions. Every file has an owner, a group, and three permission sets (user, group, other), each with read (4), write (2), execute (1). Octal mode `750` means owner `rwx` (7), group `r-x` (5), other none (0). Three **special** bits sit above them:

- **setuid** (4000) on an executable: it runs as the file's owner, not you. `passwd` is setuid root so you can change your own password in `/etc/shadow`.
- **setgid** (2000) on a directory: new files inside inherit the directory's group, great for shared folders.
- **sticky bit** (1000) on a directory: only a file's owner may delete it, which is why anyone can write in `/tmp` but cannot delete others' files.

`umask` is the mask subtracted from default permissions when a file is created.

ACLs (Access Control Lists) go beyond owner/group/other: `setfacl` grants a specific user or group its own permissions on a file. A **default ACL** on a directory is inherited by new files inside it. The **mask** is a ceiling on what named users and groups may get.

SELinux (on RHEL/Rocky; Ubuntu uses AppArmor) adds **mandatory access control**: every process and file has a security **context** (label), and a policy decides which process types may touch which file types, regardless of ordinary permissions. Its three states are **enforcing** (block and log), **permissive** (allow but log), and **disabled**. Most SELinux problems are a file with the wrong label; `restorecon` fixes the label back to policy.

Secure remote access. SSH keys are far stronger than passwords. You harden the SSH server by turning off password login and root login, limiting who may connect, and disabling features you do not use (like X11 forwarding). `fail2ban` watches logs and temporarily blocks IPs that fail to log in repeatedly.

Reducing attack surface. Remove cleartext services (Telnet, FTP, TFTP), block kernel modules for filesystems you never use, strip setuid bits that are not needed, and verify the UEFI **Secure Boot** chain that stops unsigned code from running at boot.

Tasks

Task 1: Controlled sudo with a drop-in file

> **CAUTION**
> Take a snapshot now. Keep a second terminal logged in as `student` the whole time you edit sudoers. If sudo breaks, fix it from a root shell you still have, or from the VM console: `pkexec visudo` or boot to recovery.

1. See which group grants admin rights and how, then set up a working directory.

```
mkdir -p ~/labs/3.3 && cd ~/labs/3.3
groups student
sudo grep -vE '^#|^$' /etc/sudoers | head
```

```
student : student sudo
Defaults    env_reset
Defaults    mail_badpass
Defaults    secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
%sudo   ALL=(ALL:ALL) ALL
root    ALL=(ALL:ALL) ALL
@includedir /etc/sudoers.d
```

What just happened: `student` is in the `sudo` group. The line `%sudo ALL=(ALL:ALL) ALL` reads "members of group `sudo` (`%` means group) may run, on ALL hosts, as (ALL users:ALL groups), ALL commands". `@includedir /etc/sudoers.d` pulls in every file from that directory, which is where you add your own rules without touching the main file. On Rocky the equivalent line is `%wheel ALL=(ALL) ALL`.

2. Create a limited helpdesk user. `passwd` asks you to type a password for it twice (for example `Help3.3desk`).

```
sudo useradd -m -s /bin/bash helpdesk
sudo passwd helpdesk
```

3. Write a sudoers drop-in with `visudo -f` so it is syntax-checked before saving. This lets `helpdesk` restart one service and read logs, nothing else.

```
sudo visudo -f /etc/sudoers.d/helpdesk
```

In the editor, type these lines, then save:

```
# Command alias: the exact commands helpdesk may run
Cmnd_Alias HELPDESK = /usr/bin/systemctl restart systemd-timesyncd, \
                      /usr/bin/journalctl -u systemd-timesyncd*
helpdesk ALL=(root) HELPDESK
```

What just happened: `visudo -f FILE` edits that specific file and refuses to save if the syntax is wrong (it would say "What now?" and let you fix it). `systemd-timesyncd` is the small time-sync (NTP) service Ubuntu 24.04 runs by default, a harmless service to restart. A `Cmnd_Alias` names a list of allowed commands. The rule `helpdesk ALL=(root) HELPDESK` lets `helpdesk` run exactly those commands as root. Listing full paths matters: if you wrote just `systemctl`, the user could restart any service. The backslash continues a long line.

4. Test as the helpdesk user.

```
su - helpdesk
sudo -l
sudo systemctl restart systemd-timesyncd && echo "restarted ok"
sudo systemctl restart ssh
```

```
User helpdesk may run the following commands on ubuntu-lab:
    (root) /usr/bin/systemctl restart systemd-timesyncd,
        /usr/bin/journalctl -u systemd-timesyncd*
restarted ok
Sorry, user helpdesk is not allowed to execute '/usr/bin/systemctl restart ssh'
as root on ubuntu-lab.
```

What just happened: `sudo` asked for **helpdesk's** password (the first time). `sudo -l` lists what the current user is allowed to run. The allowed `systemctl restart systemd-timesyncd` worked; restarting `ssh` was refused because it is not in the alias. This is **least privilege**: give exactly what the job needs and nothing more. Stay logged in as `helpdesk` for the next step.

5. See the `NOPASSWD` and `NOEXEC` options and why they matter. Type `exit` to leave the helpdesk shell, then, back as `student`, write the new rules to a scratch file, check it, and only then install it:

```
exit
cat <<'EOF' > ~/labs/3.3/helpdesk-extra
# NOPASSWD: no password needed (convenient, but a stolen session = instant root
# for these commands). Use only for safe, specific commands.
helpdesk ALL=(root) NOPASSWD: /usr/bin/systemctl restart systemd-timesyncd
# NOEXEC: block any attempt by the command to launch another program (a shell).
helpdesk ALL=(root) NOEXEC: /usr/bin/less /var/log/syslog
EOF
sudo visudo -cf ~/labs/3.3/helpdesk-extra
sudo install -o root -g root -m 0440 ~/labs/3.3/helpdesk-extra \
  /etc/sudoers.d/helpdesk-extra
sudo visudo -c
```

```
/home/student/labs/3.3/helpdesk-extra: parsed OK
/etc/sudoers: parsed OK
/etc/sudoers.d/README: parsed OK
/etc/sudoers.d/helpdesk: parsed OK
/etc/sudoers.d/helpdesk-extra: parsed OK
```

What just happened: `cat <<'EOF' > FILE ... EOF` is a **heredoc**: every line up to the closing `EOF` is written into the file (the quotes around `'EOF'` stop the shell from changing anything inside). You write to a scratch file first because a broken file in `/etc/sudoers.d/` can break `sudo` for everyone. `visudo -cf FILE` checks (`-c`) just that file (`-f`). `install -o root -g root -m 0440` copies it into place owned by root with read-only mode `0440`, the mode sudoers files should have. `visudo -c` then checks every sudoers file together. The first rule adds `NOPASSWD` so the timesyncd restart needs no password. The second uses `NOEXEC: less` (and `vi`, `more`, `man`) can normally launch a shell from inside with `!sh`, which would hand the user a **root shell**, defeating the whole limit. `NOEXEC` uses a library preload to stop the command from executing any other program, so that escape is blocked.

6. Demonstrate the NOEXEC protection.

```
su - helpdesk
sudo -k
sudo systemctl restart systemd-timesyncd
sudo less /var/log/syslog
```

Inside `less`, type `!sh` and press Enter. No shell starts: `less` just prints a short message such as `!done (press RETURN)` and stays where it was.

Press Enter, then `q` to quit `less`, then `exit` to leave the helpdesk shell.

> **What just happened:** `sudo -k` forgets the cached password, so you can see that the timesyncd restart did not ask for one (`NOPASSWD`). The `less` command asks for helpdesk's password because its rule has no `NOPASSWD`. Inside `less`, the shell-escape `!sh` was blocked by `NOEXEC`: `less` was not allowed to start another program. Without `NOEXEC`, you would have landed in a `#` root prompt. This is a classic hardening win and a common exam point.

7. Compare `sudo -i` and `su -`.

```
sudo -i whoami
sudo -i pwd
```

```
root
/root
```

> **What just happened:** `sudo -i` starts a root **login** shell (reads root's profile, lands in `/root`) but authenticates with **your** password and is logged by sudo. `su -` (which you have been using) switches users with a full login shell but needs the **target** user's password. `sudo -i` is preferred on servers because root's password can stay unset and every action is attributed to you in the logs. `sudo su -` and `sudo -s` are other variants (`-s` keeps your environment).

Checkpoint: `sudo -l -U helpdesk` (list the rules for user helpdesk) shows only the timesyncd, journalctl, and `less /var/log/syslog` commands, and the `!sh` escape inside `sudo less` did not give a shell.

Task 2: File attributes with chattr and lsattr

1. Protect a config-style file by making it immutable.

```
echo "important = yes" > ~/labs/3.3/critical.conf
sudo chattr +i ~/labs/3.3/critical.conf
lsattr ~/labs/3.3/critical.conf
```

```
----i---------e------- /home/student/labs/3.3/critical.conf
```

> **What just happened:** `chattr +i` sets the **immutable** attribute (the `i` in the `lsattr` output). The `e` (extent format) is normal on ext4. Attributes need root, so `chattr` runs under `sudo`.

2. Try to change or delete it, even as root.

```
echo "hacked" | sudo tee -a ~/labs/3.3/critical.conf
sudo rm -f ~/labs/3.3/critical.conf
sudo mv ~/labs/3.3/critical.conf /tmp/
```

```
tee: /home/student/labs/3.3/critical.conf: Operation not permitted
rm: cannot remove '.../critical.conf': Operation not permitted
mv: cannot move '.../critical.conf': Operation not permitted
```

What just happened: even root cannot write, delete, rename, or link an immutable file. Admins use this on files that must never change, such as `/etc/resolv.conf` on some setups. The downside: you must remember the bit is there, or updates will mysteriously fail.

3. Remove the immutable bit and set **append only** instead, which suits logs.

```
sudo chattr -i ~/labs/3.3/critical.conf
touch ~/labs/3.3/applog.txt
sudo chattr +a ~/labs/3.3/applog.txt
echo "line 1 - allowed" | sudo tee -a ~/labs/3.3/applog.txt
echo "overwrite attempt" | sudo tee ~/labs/3.3/applog.txt
lsattr ~/labs/3.3/applog.txt
```

```
line 1 - allowed
tee: /home/student/labs/3.3/applog.txt: Operation not permitted
-----a--------e------- /home/student/labs/3.3/applog.txt
```

What just happened: `touch` creates the empty file, and `+a` makes it **append only** (the `a` flag). `tee -a` (append) worked; plain `tee` (which truncates the file first) failed. An attacker who breaks in cannot erase an append-only audit log to hide their tracks. Remove the bit afterward with `sudo chattr -a`.

Checkpoint: `lsattr ~/labs/3.3/applog.txt` shows the `a` flag, and appending works while overwriting fails.

Task 3: Permissions workshop: ownership, special bits, and umask

1. Build a shared project directory for a team, using a group with setgid and the sticky bit.

```
sudo groupadd project
sudo useradd -m -s /bin/bash -G project alice
sudo useradd -m -s /bin/bash -G project bob
sudo mkdir -p /srv/project
sudo chgrp project /srv/project
sudo chmod 2770 /srv/project
ls -ld /srv/project
```

```
drwxrws--- 2 root project 4096 Oct 10 11:10 /srv/project
```

What just happened: `chgrp project` set the directory's group. `chmod 2770` means setgid (`2`) + `rwx` for owner + `rwx` for group + nothing for other. The `s` in `drwxrws---` is the **setgid** bit on the group's execute position. Now any file created inside inherits group `project`, so teammates can read each other's work without fiddling with groups each time.

2. Prove group inheritance works.

```
sudo -u alice bash -c 'umask 0007; echo hi > /srv/project/alice.txt'
sudo -u bob   bash -c 'umask 0007; echo yo > /srv/project/bob.txt'
ls -l /srv/project
```

```
-rw-rw---- 1 alice project 3 Oct 10 11:12 alice.txt
-rw-rw---- 1 bob   project 3 Oct 10 11:12 bob.txt
```

What just happened: both files got group `project` automatically because of the setgid directory, and group members can read and write them (`rw-` for group) thanks to the `0007` umask. Alice and bob can now collaborate.

3. Add the sticky bit so teammates cannot delete each other's files.

```
sudo chmod +t /srv/project
ls -ld /srv/project
sudo -u bob rm -f /srv/project/alice.txt
```

```
drwxrws--T 2 root project 4096 Oct 10 11:12 /srv/project
rm: cannot remove '/srv/project/alice.txt': Operation not permitted
```

What just happened: the trailing `T` is the **sticky bit** (capital `T` because "other" has no execute). With it, only a file's owner (or root) may delete or rename files in the directory, even though the whole group can write there. This is exactly how `/tmp` (mode `1777`) works. `+t` is the symbolic way; `chmod 3770` sets setgid + sticky in octal.

4. Understand setuid using `passwd`, then practice symbolic vs octal.

```
ls -l /usr/bin/passwd
ls -l /usr/bin/sudo
chmod u=rwx,g=rx,o=r ~/labs/3.3/critical.conf
ls -l ~/labs/3.3/critical.conf
chmod 640 ~/labs/3.3/critical.conf
ls -l ~/labs/3.3/critical.conf
```

```
-rwsr-xr-x 1 root root 68208 ... /usr/bin/passwd
-rwsr-xr-x 1 root root 277936 ... /usr/bin/sudo
-rwxr-xr-- 1 student student 16 ... critical.conf
-rw-r----- 1 student student 16 ... critical.conf
```

> **What just happened:** the `s` in `-rwsr-xr-x` on `passwd` is **setuid root**: when you run `passwd`, it runs as root so it can edit `/etc/shadow`, even though you are not root. `sudo` is setuid for the same reason. The symbolic form `u=rwx,g=rx,o=r` sets each class explicitly; `chmod 640` is the octal equivalent of `rw-r-----`. Both forms are testable on the exam.

5. See how `umask` decides default permissions. The math: files start from `666`, directories from `777`, and the umask bits are removed.

```
umask
touch ~/labs/3.3/defaultfile
mkdir ~/labs/3.3/defaultdir
ls -ld ~/labs/3.3/defaultfile ~/labs/3.3/defaultdir
( umask 077; touch ~/labs/3.3/privatefile; ls -l ~/labs/3.3/privatefile )
```

```
0002
-rw-rw-r-- 1 student student 0 ... defaultfile
drwxrwxr-x 2 student student ... defaultdir
-rw------- 1 student student 0 ... privatefile
```

> **What just happened:** Ubuntu gives normal users umask `0002`, so a new file is `666 - 002 = 664` (`rw-rw-r--`) and a new directory is `777 - 002 = 775`. (The system default in `/etc/login.defs` is `022`, but because `USERGROUPS_ENAB yes` is set and `student` has its own private group, `pam_umask` relaxes it to `002`. Root and Rocky users typically get `022`, which gives `644` files and `755` directories.) In the parentheses (a subshell, so the change is temporary) umask `077` removes all group and other permissions, giving `600` files and `700` directories. Set a user's umask in `~/.bashrc` or system-wide in `/etc/profile` or `/etc/login.defs` (`UMASK`).

> **Checkpoint:** `ls -ld /srv/project` shows `drwxrws--T`, new files there get group `project`, and bob cannot delete alice's file.

Task 4: Fine-grained permissions with ACLs

1. Grant one extra user access to a file without changing its group.

```
echo "team report" | sudo tee /srv/project/report.txt
sudo chown alice:project /srv/project/report.txt
sudo chmod 640 /srv/project/report.txt
sudo getfacl /srv/project/report.txt
```

```
# file: srv/project/report.txt
# owner: alice
# group: project
user::rw-
group::r--
other::---
```

What just happened: `student` is not in group `project`, so you cannot even enter `/srv/project` (mode `3770`); that is why every command here uses `sudo` (`sudo tee` writes the file as root). `getfacl` shows the ACL. With no extra entries it just mirrors the normal permissions. Right now only alice (owner) and the `project` group can read it.

2. Give user `bob` read-write and a non-member user read-only, with `setfacl`.

```
sudo useradd -m contractor
sudo setfacl -m u:bob:rw,u:contractor:r /srv/project/report.txt
sudo setfacl -m u:contractor:x /srv/project
sudo getfacl /srv/project/report.txt
sudo ls -l /srv/project/report.txt
sudo -u contractor cat /srv/project/report.txt
```

```
# file: srv/project/report.txt
# owner: alice
# group: project
user::rw-
user:bob:rw-
user:contractor:r--
group::r--
mask::rw-
other::---
-rw-rw----+ 1 alice project ... report.txt
team report
```

What just happened: `setfacl -m` modifies (adds) ACL entries: `u:bob:rw` and `u:contractor:r`. The `+` after the permissions in `ls -l` signals an ACL is present. A new line, `mask::rw-`, appeared: the **mask** is the maximum any named user or group may have. If the mask were `r--`, bob would be read-only no matter what his entry says. `ls -l` shows the mask in the group position, which is why you check real permissions with `getfacl`. The second `setfacl` gives contractor only `x` (search) on the directory itself: without it, contractor could not pass through `/srv/project` to reach the file at all. The last command proves contractor can now read the report.

3. Set a **default ACL** on a directory so new files inherit it.

```
sudo mkdir /srv/project/shared
sudo chmod 2770 /srv/project/shared
sudo setfacl -m u:contractor:rwx /srv/project/shared
sudo setfacl -d -m u:contractor:rwX /srv/project/shared
sudo getfacl /srv/project/shared | grep default
sudo -u alice bash -c 'echo new > /srv/project/shared/newfile.txt'
sudo getfacl /srv/project/shared/newfile.txt | grep -E 'contractor|mask'
```

```
default:user::rwx
default:user:contractor:rwx
default:group::rwx
default:mask::rwx
default:other::---
user:contractor:rwx #effective:rw-
mask::rw-
```

What just happened: `chmod 2770` lets group `project` write in the new directory (`mkdir` under `sudo` made it `755`). The first `setfacl` (without `-d`) lets contractor into the directory itself. `-d` sets a **default** ACL on the directory; new files and subdirectories automatically get these entries. The `X` (capital) means "execute only if it is a directory or already executable"; the target here is a directory, so it became `x`. The new file copied the entry `user:contractor:rwx`, but its mask is `rw-` because programs create data files without execute permission, so getfacl marks the real (effective) permission as `rw-`: the mask at work again. Check it with `sudo -u contractor cat /srv/project/shared/newfile.txt`. Remove an ACL with `setfacl -x u:bob FILE`, or wipe all ACLs with `setfacl -b FILE`.

Checkpoint: `getfacl /srv/project/report.txt` lists `user:bob:rw-` and `user:contractor:r--`, and files created in `/srv/project/shared` give contractor access automatically.

Task 5: SELinux on rocky-lab

SELinux is RHEL's mandatory access control. Ubuntu uses AppArmor, so this task is on rocky-lab.

1. Check the state and config.

```
# on rocky-lab
getenforce
sestatus | head -5
cat /etc/selinux/config | grep -vE '^#|^$'
```

```
Enforcing
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
SELINUX=enforcing
SELINUXTYPE=targeted
```

What just happened: `getenforce` prints the current mode. The three states are **enforcing** (deny and log violations), **permissive** (allow but log, great for troubleshooting), and **disabled** (off entirely, not recommended). `/etc/selinux/config` sets the mode at boot via `SELINUX=`. Changing it to `disabled` and rebooting fully turns SELinux off; switching between enforcing and permissive does not need a reboot.

2. Switch modes at runtime (no reboot) and back.

```
# on rocky-lab
sudo setenforce 0
getenforce
sudo setenforce 1
getenforce
```

```
Permissive
Enforcing
```

What just happened: `setenforce 0` = permissive, `setenforce 1` = enforcing. This is temporary and ignored at reboot. Use permissive to see whether SELinux is the cause of a problem without turning it off permanently: if the problem continues in permissive mode, SELinux is not to blame.

3. Look at security contexts.

```
# on rocky-lab
ls -Z /etc/passwd
ps -Zp 1
id -Z
```

```
system_u:object_r:passwd_file_t:s0 /etc/passwd
system_u:system_r:init_t:s0 ... /usr/lib/systemd/systemd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

What just happened: `-Z` shows the SELinux context, written `user:role:type:level`. The **type** is what matters most in the targeted policy: files have types like `passwd_file_t`, processes run in **domains** like `init_t`. The policy says, for example, that only certain domains may write `passwd_file_t`. Your login shell is `unconfined_t`, meaning the policy does not restrict it.

4. Install a web server and trigger a classic "wrong context" denial.

```
# on rocky-lab
sudo dnf install -y httpd setroubleshoot-server policycoreutils-python-utils
sudo systemctl enable --now httpd
sudo firewall-cmd --add-service=http
echo "Default web root" | sudo tee /var/www/html/index.html
echo "from home dir" > ~/mypage.html
sudo mv ~/mypage.html /var/www/html/mypage.html
curl -s http://localhost/index.html
curl -s http://localhost/mypage.html | grep -i title
```

```
Default web root
<title>403 Forbidden</title>
```

What just happened: `dnf` installed Apache (`httpd`), the `sealert` tool (`setroubleshoot-server`), and `semanage` (`policycoreutils-python-utils`). `enable --now` starts Apache and makes it start at boot, and `--add-service=http` opens port 80 in the firewall (runtime only). The page from the default root works, but `mypage.html` gives **403 Forbidden**. The file came from your home directory, so it kept the context `user_home_t`. Apache (domain `httpd_t`) is only allowed to read `httpd_sys_content_t`, so SELinux blocked it, even though the Unix permissions are fine. This is the single most common SELinux surprise.

5. Diagnose with the audit log and `sealert`, then fix with `restorecon`.

```
# on rocky-lab
ls -Z /var/www/html/
sudo ausearch -m avc -ts recent 2>/dev/null | tail -4
sudo sealert -a /var/log/audit/audit.log 2>/dev/null | grep -A3 'mypage' | head
sudo restorecon -v /var/www/html/mypage.html
curl -s http://localhost/mypage.html
```

```
unconfined_u:object_r:user_home_t:s0 mypage.html
unconfined_u:object_r:httpd_sys_content_t:s0 index.html
type=AVC ... avc:  denied  { read } ... comm="httpd"
  name="mypage.html" ... tcontext=...:user_home_t ...
Changing SELinux context ...user_home_t:s0->...httpd_sys_content_t:s0
from home dir
```

What just happened: `ls -Z` shows the bad file is `user_home_t` while its neighbor is `httpd_sys_content_t`. An **AVC** (Access Vector Cache) denial in the audit log records exactly what was blocked. `sealert` turns raw AVCs into plain-English advice (it is part of `setroubleshoot-server`). `restorecon` resets the file's context to what the policy says that path should be, and the page now loads. `restorecon -Rv /path` fixes a whole tree.

6. Serve content from a non-standard directory using `semanage fcontext` (the permanent way) and `chcon` (the temporary way).

```
# on rocky-lab
sudo mkdir -p /webdata
echo "custom location" | sudo tee /webdata/index.html
sudo chcon -t httpd_sys_content_t /webdata/index.html
ls -Z /webdata/index.html
sudo semanage fcontext -a -t httpd_sys_content_t '/webdata(/.*)?'
sudo restorecon -Rv /webdata
```

What just happened: `chcon -t` changes a context right now, but it is **temporary**: a filesystem relabel or `restorecon` would revert it, because `chcon` does not update the policy's rules. `semanage fcontext -a` **adds a rule** that says "anything matching `/webdata(/.*)?` should be `httpd_sys_content_t`", and `restorecon` then applies it. The `semanage` way survives relabels and is what you use in production. (You would also point Apache at `/webdata` with a config change to actually serve it.)

7. Let Apache listen on a non-standard port with `semanage port`.

```
# on rocky-lab
sudo semanage port -l | grep http_port_t
sudo semanage port -a -t http_port_t -p tcp 8088
sudo semanage port -l | grep http_port_t
```

```
http_port_t                    tcp      80, 81, 443, 488, 8008, 8009, 8443, 9000
http_port_t                    tcp      8088, 80, 81, 443, ...
```

What just happened: SELinux also labels **ports**. Apache's domain may only bind ports labeled `http_port_t`. If you set `Listen 8088` in Apache without this step, httpd would fail to start with a permission error. `semanage port -a -t http_port_t -p tcp 8088` adds 8088 to the allowed list.

8. Flip an SELinux boolean to allow optional behavior.

```
# on rocky-lab
getsebool httpd_can_network_connect
sudo setsebool -P httpd_can_network_connect on
getsebool httpd_can_network_connect
```

```
httpd_can_network_connect --> off
httpd_can_network_connect --> on
```

What just happened: **booleans** are on/off switches that turn whole chunks of policy on or off. By default Apache may not make outbound network connections (a safety measure against a compromised web app); turning this boolean on allows it, which you need if your site talks to a database or API on another host. `-P` makes it **persistent** across reboots; without `-P` it resets at boot. `getsebool -a` lists them all.

9. See what `audit2allow` does with a denial. This builds a custom policy module from AVCs. Read the output but do not load the module in this lab.

```
# on rocky-lab
sudo ausearch -m avc -ts recent 2>/dev/null | audit2allow -m labpolicy | head
```

```
module labpolicy 1.0;
require {
    type httpd_t;
    type user_home_t;
    class file read;
}
allow httpd_t user_home_t:file read;
```

What just happened: `audit2allow` reads denials and writes the policy rules that **would** allow them. This is a last resort: use it only when a program legitimately needs access that no boolean or correct label provides. The right fix for our earlier 403 was `restorecon`, not an `allow httpd_t user_home_t` rule, which would let Apache read every user's home directory. `audit2allow -M name` would compile a loadable module; `semodule -i name.pp` installs it.

Checkpoint: on rocky-lab, `curl http://localhost/mypage.html` returns the page after `restorecon`, and `getsebool httpd_can_network_connect` shows `on`.

Task 6: Secure SSH with keys and a hardened config

1. On ubuntu-lab, generate an ed25519 key pair and copy the public key to rocky-lab.

```
ssh-keygen -t ed25519 -C "student@ubuntu-lab lab3.3" -f ~/.ssh/id_ed25519 -N ""
ssh-copy-id -i ~/.ssh/id_ed25519.pub student@rocky-lab
ssh student@rocky-lab hostname
```

```
Your identification has been saved in /home/student/.ssh/id_ed25519
Number of key(s) added: 1
rocky-lab
```

What just happened: `ssh-keygen -t ed25519` makes a modern, small, strong key pair. `-f` sets the file name, `-N ""` makes an empty passphrase (for the lab; in real life use a passphrase). `ssh-copy-id` appends the **public** key to `~/.ssh/authorized_keys` on rocky-lab, so from now on you log in with the key instead of a password. The private key never leaves ubuntu-lab.

2. Use `ssh-agent` and `ssh-add` so a passphrase is typed only once per session. (The effect is clearer with a passphrase, but the commands are the same.)

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
ssh-add -l
ssh student@rocky-lab uptime
```

```
Agent pid 5123
Identity added: /home/student/.ssh/id_ed25519 (student@ubuntu-lab lab3.3)
256 SHA256:abc... student@ubuntu-lab lab3.3 (ED25519)
```

What just happened: `ssh-agent` is a background process that holds your unlocked private keys in memory. `eval "$(ssh-agent -s)"` starts it and sets the environment variables that let `ssh` find it. `ssh-add` loads a key into the agent (asking for the passphrase once); `ssh-add -l` lists loaded keys. With an agent, you authenticate many times without retyping the passphrase, and agent forwarding (`ssh -A`) can carry it to further hops.

3. Harden the SSH **server** on rocky-lab with a drop-in file.

> **CAUTION**
> This changes how you log in to rocky-lab. Keep your current rocky-lab SSH session open, and confirm key login works (step 1) before you turn off passwords. If you get locked out, fix the file from the VM console. Snapshot first.

```
# on rocky-lab
sudo groupadd sshusers
sudo usermod -aG sshusers student
ls /etc/ssh/sshd_config.d/
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
X11Forwarding no
AllowGroups sshusers
EOF
sudo sshd -t && echo "config OK"
sudo sshd -T | grep -E \
  '^(permitrootlogin|passwordauthentication|x11forwarding|allowgroups) '
```

```
50-redhat.conf
...
config OK
permitrootlogin no
passwordauthentication no
x11forwarding no
allowgroups sshusers
```

What just happened: `usermod -aG` **a**ppends `student` to the new group (without `-a` it would replace student's other groups). `sudo tee FILE <<'EOF'` writes the lines up to `EOF` (a **heredoc**) into the file as root. Files in `/etc/ssh/sshd_config.d/` are read at the very top of `/etc/ssh/sshd_config`, in name order, and for most settings **the first value sshd reads wins**. That is why the file name starts with `10-`: it is read before Rocky's `50-redhat.conf` (and a cloud image's `50-cloud-init.conf`). If `ls` also shows a file such as `01-permitrootlogin.conf` (the installer creates it when you allow root SSH login), it sorts before yours and wins, so change its value to `no` too. `sshd -T` prints the settings sshd will really use, so you can prove your values won. `KbdInteractiveAuthentication no` turns off the other password-style ("keyboard-interactive") login method too. The settings: **`PermitRootLogin no`** stops direct root login (attackers must first get a user account); **`PasswordAuthentication no`** forces key login, defeating password guessing; **`X11Forwarding no`** disables forwarding graphical apps (an attack surface you rarely need on a server); **`AllowGroups sshusers`** lets only members of that group connect (`AllowUsers student@192.168.100.0/24` would limit by user and source). `sshd -t` tests the config; never restart until it says OK.

4. Apply and test from a **new** terminal while the old one stays open.

```
# on rocky-lab
sudo systemctl reload sshd
```

```
ssh student@rocky-lab "echo key login works"
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no \
    student@rocky-lab true
```

```
key login works
student@rocky-lab: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

What just happened: `reload` re-reads the config without dropping existing connections (safer than `restart`). Key login still works; forcing password-only authentication is now refused, proving passwords are off. The list in brackets is the login methods the server still offers: `publickey` plus Rocky's Kerberos (`gssapi`) methods, and no `password`. Only after you confirm this should you trust the lockdown.

5. Create an SSH **tunnel** (local port forwarding). Apache on rocky-lab serves port 80; you will reach it from ubuntu-lab as `localhost:8080`.

```
# on rocky-lab, make sure httpd is running and listening on 80 (from Task 5)
ssh -f -N -L 8080:localhost:80 student@rocky-lab
curl -s http://localhost:8080/ | head -1
```

```
Default web root
```

What just happened: `-L 8080:localhost:80` means "listen on **my** port 8080 and forward through the SSH connection to `localhost:80` **as seen from rocky-lab**". So ubuntu-lab's 8080 reaches rocky-lab's web server, encrypted inside SSH, even if port 80 is firewalled off between the hosts. `-f` sends SSH to the background, `-N` runs no remote command. This is how admins safely reach internal services. Stop it with `pkill -f "8080:localhost:80"`. (`-R` does the reverse direction; `-D` makes a SOCKS proxy.)

Checkpoint: key login to rocky-lab works, password login is refused, and `curl http://localhost:8080/` on ubuntu-lab shows rocky-lab's default page through the tunnel.

Task 7: An SFTP-only chroot group

Give file-transfer users SFTP access locked into their own directory, with no shell and no view of the rest of the system.

1. On rocky-lab, create the group, user, and a correctly owned chroot directory.

```
# on rocky-lab
sudo groupadd sftponly
sudo useradd -M -g sftponly -s /sbin/nologin -d /home/sftproot/upload sftpuser
sudo passwd sftpuser
sudo mkdir -p /home/sftproot/upload
sudo chown root:root /home/sftproot
sudo chmod 755 /home/sftproot
sudo chown sftpuser:sftponly /home/sftproot/upload
```

What just happened: `-M` tells `useradd` not to create the home directory (you build it by hand next), `-g sftponly` sets the primary group, `-d` records the home path, and `-s /sbin/nologin` means no interactive shell. The **chroot** directory (`/home/sftproot`) and every directory above it **must be owned by root and not writable by others**, or sshd refuses the chroot for security. The writable `upload` subdirectory inside is where the user actually puts files.

2. Add an SSH server rule that chroots this group to the internal SFTP server.

```
# on rocky-lab
sudo usermod -aG sshusers sftpuser
sudo tee /etc/ssh/sshd_config.d/60-sftp.conf <<'EOF'
Match Group sftponly
    ChrootDirectory /home/sftproot
    ForceCommand internal-sftp -d /upload
    PasswordAuthentication yes
    AllowTcpForwarding no
    X11Forwarding no
EOF
sudo sshd -t && sudo systemctl reload sshd
```

What just happened: a `Match` block applies settings only to connections that match (here, group `sftponly`). `ChrootDirectory` jails them so `/` looks like `/home/sftproot` . `ForceCommand internal-sftp` runs SSH's built-in SFTP server instead of a shell, so these users can only transfer files; `-d /upload` makes the session start in the writable `upload` folder (paths are now relative to the jail). `PasswordAuthentication yes` re-enables passwords **for this group only**, because Task 6 turned them off for everyone and sftpuser has no key. We added sftpuser to `sshusers` because the Task 6 `AllowGroups` rule would otherwise block them.

3. Test from ubuntu-lab: SFTP works and is jailed, but shell login is refused.

```
sftp sftpuser@rocky-lab
```

At the `sftp>` prompt type `pwd` , then `ls` , then `put ~/.ssh/id_ed25519.pub test.pub` , then `exit` .

```
sftp> pwd
Remote working directory: /upload
sftp> put ~/.ssh/id_ed25519.pub test.pub
Uploading ... to /upload/test.pub
```

```
ssh sftpuser@rocky-lab
```

```
This service allows sftp connections only.
Connection to rocky-lab closed.
```

What just happened: inside SFTP, the working directory shows as `/upload`, not the real path, proving the chroot jail works. A normal `ssh` login is refused because `internal-sftp` replaces the shell. This is the standard way to hand someone a drop-box without giving them the server.

Checkpoint: `sftp sftpuser@rocky-lab` connects and shows `/upload`, while `ssh sftpuser@rocky-lab` is closed with the SFTP-only message.

Task 8: Block brute-force logins with fail2ban

1. Install fail2ban on ubuntu-lab and enable a jail for SSH.

```
sudo apt install -y fail2ban
sudo tee /etc/fail2ban/jail.d/sshd.local <<'EOF'
[sshd]
enabled = true
backend = systemd
maxretry = 3
findtime = 10m
bantime = 10m
EOF
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   `- Banned IP list:
```

What just happened: fail2ban reads logs through a **filter** and, when a pattern (like a failed SSH login) appears `maxretry` times within `findtime`, runs an **action** to ban the source for `bantime`. On Ubuntu the package's `/etc/fail2ban/jail.d/defaults-debian.conf` already enables the `sshd` jail with `backend = systemd` (read the journal) and `banaction = nftables` (ban with an nftables reject rule); your file adds stricter numbers. Settings go in `*.local` files so package updates do not overwrite them. The `+` in "Journal matches" means "or": it reads journal lines from the `sshd.service` unit or from any process named `sshd`, which catches Ubuntu's `ssh.service` too.

2. Make sure password login is allowed on **ubuntu-lab** for this test (it is by default), then trigger a ban from rocky-lab with wrong passwords. First create a test user you can safely fail against:

> **CAUTION**
> The ban blocks **all** SSH from rocky-lab's IP to ubuntu-lab, including a session that is already open. Do this step from the rocky-lab console (or an SSH session from your host computer to rocky-lab), and do not rely on an SSH session to ubuntu-lab that comes **from** rocky-lab. If you do get cut off, wait 10 minutes (`bantime`) or run the unban command in step 4 from the ubuntu-lab console.

```
sudo useradd -m baituser
sudo passwd baituser
```

```
# on rocky-lab: type a WRONG password each time, four times
for i in 1 2 3 4; do \
  ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no \
      -o NumberOfPasswordPrompts=1 -o StrictHostKeyChecking=no \
      baituser@ubuntu-lab true; done
```

> **What just happened:** each attempt uses the wrong password (type anything incorrect, or press Enter). The options force a password attempt rather than using any key, allow only one password prompt per connection, and skip the "trust this host key?" question. After three failures within the window, fail2ban should ban rocky-lab's IP. This is a defensive test of your own jail from a host you control; never run password-guessing against systems you do not own.

3. Confirm the ban on ubuntu-lab.

```
sudo fail2ban-client status sshd
sudo fail2ban-client get sshd banip
```

```
`- Actions
   |- Currently banned: 1
   `- Banned IP list:   192.168.100.20
192.168.100.20
```

While banned, a new SSH attempt from rocky-lab to ubuntu-lab fails at once with `Connection refused` (the ban is an nftables reject rule; see it with `sudo nft list ruleset | grep -A3 f2b`).

4. Unban so rocky-lab can reach you again.

```
sudo fail2ban-client set sshd unbanip 192.168.100.20
sudo fail2ban-client status sshd
```

> **What just happened:** `unbanip` removes the ban immediately instead of waiting for `bantime` . `fail2ban-client status` lists active jails, and `status <jail>` shows its counts. In production you would raise `bantime` (or use `bantime.increment = true` for repeat offenders) and may add a `recidive` jail.

Checkpoint: after the failed logins, `sudo fail2ban-client status sshd` showed rocky-lab banned, and `unbanip` cleared it.

Task 9: Remove insecure services and reduce attack surface

1. Confirm no cleartext services are listening, and check for their packages.

```
ss -tulpn | grep -E ':23 |:21 |:69 ' || echo "no telnet/ftp/tftp listening"
dpkg -l | grep -E 'telnetd|vsftpd|tftpd|inetutils-telnetd' || echo "none installed"
```

```
no telnet/ftp/tftp listening
none installed
```

What just happened: `ss -tulpn` lists listening TCP (`t`) and UDP (`u`) sockets with ports (`n`) and the owning program (`p`). Ports 23 (Telnet), 21 (FTP), and 69 (TFTP) send credentials and data in **cleartext**, so a hardened host must not run them. SSH (22) replaces Telnet, and SFTP/SCP replace FTP. If any were installed you would remove them: `sudo apt purge telnetd vsftpd tftpd-hpa`.

TIP

Explore/simulate: to see why Telnet is unsafe without exposing a real service, you can watch traffic on the loopback. In one terminal run `sudo tcpdump -i lo -A port 2323` and in another `nc -l 2323` plus a client `nc localhost 2323`; whatever you type appears in `tcpdump` as plain readable text. The same capture on an SSH session shows only encrypted bytes. This is the core reason to avoid Telnet/FTP/TFTP.

2. Disable unused file system kernel modules so they cannot be loaded (a CIS Benchmark item).

```
sudo tee /etc/modprobe.d/lab-hardening.conf <<'EOF'
install cramfs /bin/false
install freevxfs /bin/false
install udf /bin/false
blacklist cramfs
blacklist freevxfs
EOF
sudo modprobe -n -v cramfs
sudo modprobe cramfs; echo "exit=$?"
lsmod | grep cramfs || echo "cramfs not loaded"
```

```
install /bin/false
modprobe: ERROR: could not insert 'cramfs': ...
exit=1
cramfs not loaded
```

What just happened: `install cramfs /bin/false` tells modprobe to run `/bin/false` (which does nothing and returns failure) instead of loading the module, so an attacker cannot mount an exotic filesystem to smuggle in a setuid binary. `blacklist` stops automatic loading. `modprobe -n -v` shows what **would** happen (a dry run); the real `modprobe` then fails with exit code 1. These are filesystems a normal server never needs. (On an already-loaded module you would also `rmmod` it.)

3. Find setuid binaries and remove an unnecessary one from a **copy** (never strip the real `passwd` or `sudo`).

```
sudo find /usr/bin /usr/sbin -perm -4000 -type f 2>/dev/null | head
cp /bin/ping ~/labs/3.3/myping
sudo chown root:root ~/labs/3.3/myping
sudo chmod 4755 ~/labs/3.3/myping
ls -l ~/labs/3.3/myping
sudo chmod u-s ~/labs/3.3/myping
ls -l ~/labs/3.3/myping
```

```
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/mount
...
-rwsr-xr-x 1 root root ... myping
-rwxr-xr-x 1 root root ... myping
```

What just happened: `find -perm -4000` finds files with the setuid bit set (`-perm -4000` means "at least these bits"). Every setuid-root program is a potential path to full control if it has a bug, so you audit the list and remove the bit from anything that does not need it. `chmod 4755` added setuid to the copy (the `s` appears); `chmod u-s` removed it. In the real world you compare this list against a baseline and investigate anything new. `-perm -2000` finds setgid files.

4. Explore Secure Boot. VMs are often BIOS, so this is an explore/simulate step.

```
mokutil --sb-state 2>/dev/null || echo "mokutil not present or not UEFI"
[ -d /sys/firmware/efi ] && echo "booted in UEFI mode" \
  || echo "booted in BIOS/legacy mode"
```

```
mokutil not present or not UEFI
booted in BIOS/legacy mode
```

TIP

Explore/simulate: Secure Boot is a UEFI feature that only lets **signed** boot code run, so malware cannot replace your bootloader or kernel (a "bootkit"). The chain is: UEFI firmware verifies **shim** (signed by Microsoft), shim verifies **GRUB**, GRUB verifies the **signed kernel**, and the kernel only loads signed modules. The presence of `/sys/firmware/efi` means the machine booted via UEFI (needed for Secure Boot); your lab VM is likely BIOS, so `mokutil --sb-state` fails with `EFI variables are not supported on this system` (hidden above by `2>/dev/null`, so the fallback `echo` ran). The `[ -d ... ] && A || B` line prints A if the directory exists and B otherwise. On real hardware that command prints `SecureBoot enabled` or `disabled`, and `mokutil --list-enrolled` shows the trusted keys. You enroll your own keys with `mokutil --import` when signing custom modules.

Checkpoint: `sudo modprobe cramfs` fails, no Telnet/FTP/TFTP ports are listening, and you can list setuid files with `find / -perm -4000`.

Challenge

CHALLENGE

1. Write a sudoers drop-in that lets group `project` run `/usr/bin/apt update` and `/usr/bin/apt upgrade -y` with `NOPASSWD`, verify with `sudo -l` as alice, and explain one risk of granting `apt` this way.
2. On rocky-lab, change Apache to listen on port 8088 (edit `/etc/httpd/conf/httpd.conf`), add the SELinux port label you created in Task 5, open the firewall, and prove it serves a page on 8088. Clean up with `semanage port -d`.
3. Combine ACLs and setgid: create `/srv/audit` that group `project` can write to, where files are automatically readable by user `contractor` (default ACL) but nobody can delete another person's files (sticky bit). Prove all three behaviors.

Clean up

1. On ubuntu-lab, stop fail2ban and remove its jail, and the bait user.

```
sudo systemctl stop fail2ban
sudo rm -f /etc/fail2ban/jail.d/sshd.local
sudo apt purge -y fail2ban
sudo userdel -r baituser
```

2. Stop any SSH tunnel and clear the agent.

```
pkill -f "8080:localhost:80" 2>/dev/null
ssh-add -D 2>/dev/null
kill "$SSH_AGENT_PID" 2>/dev/null
```

3. Remove the module blacklist, the shared directory, ACL users, and attribute files.

```
sudo rm -f /etc/modprobe.d/lab-hardening.conf
sudo chattr -a ~/labs/3.3/applog.txt 2>/dev/null
sudo rm -rf /srv/project
sudo userdel -r alice; sudo userdel -r bob; sudo userdel -r contractor
sudo groupdel project 2>/dev/null
```

4. Remove the sudoers drop-ins and the helpdesk user.

```
sudo rm -f /etc/sudoers.d/helpdesk /etc/sudoers.d/helpdesk-extra
sudo visudo -c
sudo userdel -r helpdesk
rm -rf ~/labs/3.3
```

5. On rocky-lab, undo the SSH hardening, SFTP jail, SELinux additions, and reset the test key login (optional: keep key login if you like).

```
# on rocky-lab
sudo rm -f /etc/ssh/sshd_config.d/10-hardening.conf /etc/ssh/sshd_config.d/60-sftp.conf
sudo sshd -t && sudo systemctl reload sshd
sudo userdel -r sftpuser; sudo rm -rf /home/sftproot
sudo groupdel sftponly; sudo groupdel sshusers
sudo semanage port -d -t http_port_t -p tcp 8088 2>/dev/null
sudo semanage fcontext -d '/webdata(/.*)?' 2>/dev/null
sudo setsebool -P httpd_can_network_connect off
sudo rm -rf /webdata /var/www/html/mypage.html
sudo firewall-cmd --remove-service=http
sudo systemctl disable --now httpd
rm -rf ~/labs/3.3
```

6. (Optional) To remove the key you copied, edit `~/.ssh/authorized_keys` on rocky-lab and delete the `lab3.3` line, or restore your pre-lab snapshot.

Exam corner

- Edit sudoers only with `visudo`; drop-ins go in `/etc/sudoers.d/`; `visudo -c` checks syntax, `sudo -l` lists a user's rights.
- `NOPASSWD` runs without a password (convenience vs risk); `NOEXEC` blocks shell escapes from commands like `less` / `vi`. Admin group: `sudo` on Ubuntu, `wheel` on Rocky.
- `sudo -i` = root login shell via your password (logged); `su -` = switch user with a login shell via the target's password.
- `chattr +i` immutable (even root cannot change/delete), `chattr +a` append only; view with `lsattr`.
- Permissions: octal `4000` setuid, `2000` setgid, `1000` sticky; `chmod 2770`, `chmod +t`, symbolic `u=rwx,g=rx,o=`. setuid on files, setgid+sticky mainly on directories.
- `umask`: files `666 - umask`, directories `777 - umask`; `022` gives 644/755.
- ACLs: `setfacl -m u:name:rw file`, `-d` default ACL on a directory, `-x` remove, `-b` wipe; `getfacl` to view; the **mask** caps named entries; `+` in `ls -l` means an ACL exists.

- SELinux states: enforcing, permissive, disabled (`getenforce` , `setenforce 0/1` , `/etc/selinux/config`). Context = `user:role:type:level` ; view with `ls -Z` , `ps -Z` .
- Fix SELinux labels with `restorecon` (applies policy) or `chcon` (temporary); add path rules with `semanage fcontext -a` then `restorecon` ; ports with `semanage port -a` ; switches with `setsebool -P` ; diagnose with `ausearch -m avc` , `sealert` , and `audit2allow` .
- SSH hardening (`sshd_config` / `sshd_config.d/`): `PermitRootLogin no` , `PasswordAuthentication no` , `X11Forwarding no` , `AllowUsers` / `AllowGroups` ; always `sshd -t` before reload. Keys: `ssh-keygen` , `ssh-copy-id` , `ssh-agent` + `ssh-add` . Tunnels: `-L` local, `-R` remote, `-D` SOCKS.
- SFTP chroot: `Match Group` , `ChrootDirectory` (owned by root, not writable), `ForceCommand internal-sftp` .
- `fail2ban` : jails in `jail.d/*.local` , `fail2ban-client status <jail>` , `set <jail> unbanip <ip>` .
- Avoid cleartext Telnet (23), FTP (21), TFTP (69); check with `ss -tulpn` . Block filesystems in `/etc/modprobe.d/` with `install <fs> /bin/false` + `blacklist` . Remove unneeded SUID with `chmod u-s` (find with `find / -perm -4000`). Secure Boot: UEFI-only, checked with `mokutil --sb-state` .

Review questions

1. You must edit `/etc/sudoers` . Which command should you use and why?
2. A helpdesk user is allowed `sudo vi /etc/hosts` . Why is this dangerous, and which sudo option mitigates shell escapes?
   A. `NOPASSWD`
   B. `NOEXEC`
   C. `SETENV`
   D. `!authenticate`
3. A file copied into `/var/www/html` returns 403 from Apache although permissions are `644` . SELinux is enforcing. What is the most likely cause and the one-command fix?
4. Which `chmod` makes a shared directory where new files inherit the group and users cannot delete each other's files?
   A. `chmod 2770`
   B. `chmod 1770`
   C. `chmod 3770`
   D. `chmod 4770`
5. Which SSH server settings stop password guessing and direct root login? Name both directives.
6. What does `chattr +a` do, and why is it useful for log files?

Lab 3.4 Harden User Accounts

Objective: 3.4 Explain account hardening techniques and best practices.
Domain: 3.0 Security (18% of the exam)
Time: about 120 minutes
VM: ubuntu-lab (most tasks) and rocky-lab (account lockout)
Snapshot first: Yes. This lab edits PAM and SSH settings. A mistake in PAM can stop everyone from logging in.

Most real break-ins do not start with clever exploits. They start with a weak password, a reused password, or a forgotten account that still has a login shell. In this lab you will make passwords stronger, stop password reuse, set password expiration, lock accounts after failed guesses, add a second factor to SSH, check a password against a breach list, give accounts restricted shells, and stop using root directly. These are exactly the account hardening techniques the exam asks you to explain.

What you'll practice

- Passwords: Complexity, Length, Expiration, Reuse, History (`pam_pwquality` , `pwquality.conf` , `pam_pwhistory` , `chage` , `/etc/login.defs`)
- Account lockout with `pam_faillock` and `faillock` , and how `pam_tally2` (the old tool) did the same job
- MFA: time-based one-time passwords (TOTP) for SSH with `libpam-google-authenticator` , tested with `oathtool`
- Checking existing breach lists: the Have I Been Pwned (HIBP) range API and k-anonymity
- Restricted shells: `/sbin/nologin` (on Ubuntu, `/usr/sbin/nologin`) and `/bin/rbash`
- Avoid running as root: lock root, block root SSH logins, use `sudo` , and audit `sudo` use in the logs

How it works

PAM is the gatekeeper. PAM stands for *Pluggable Authentication Modules*. Programs like `login` , `sshd` , `su` , `sudo` , and `passwd` do not check passwords themselves. They ask PAM. PAM reads a small text file for that program in `/etc/pam.d/` and runs a list of modules, one after another, like a line of security guards. Each guard can say "pass", "fail", or "not my business".

Each line in a PAM file has this shape:

```
type     control      module            options
password requisite    pam_pwquality.so  retry=3
```

- **type** is the job: `auth` (prove who you are), `account` (are you allowed in right now, for example is the account expired), `password` (changing a password), `session` (set things up after login).

- **control** says what a result means. `required` means "must pass, but keep running the other modules". `requisite` means "must pass, and stop right now if it fails". `sufficient` means "if this passes, that is enough". Ubuntu also uses the bracket form `[success=1 default=ignore]`, which means "on success, skip the next 1 line".
- **module** is the shared library that does the work, for example `pam_unix.so` (checks `/etc/shadow`).

On Ubuntu, the main services include four shared files: `common-auth`, `common-account`, `common-password`, and `common-session`. Ubuntu builds these files with a tool named **`pam-auth-update`** from small "profile" files in `/usr/share/pam-configs/`. If you edit `common-*` files by hand, a later package install may run `pam-auth-update` and either refuse to touch your file or replace it. So on Ubuntu the safe way is to add a profile and let `pam-auth-update` write the file. Rocky Linux has a similar tool named **`authselect`**, which builds `/etc/pam.d/system-auth` and `password-auth` from profiles and "features".

```
user runs passwd
      |
      v
/etc/pam.d/passwd  --@include-->  /etc/pam.d/common-password
                                          |
   password requisite pam_pwquality.so    |  1. complex enough? long enough?
   password requisite pam_pwhistory.so    |  2. used in the last 5 changes?
   password [...]     pam_unix.so         |  3. hash it, save it in /etc/shadow
                                          v
                                  new password accepted
```

Password policy has five parts on the exam:

- **Complexity**: the mix of character classes (lowercase, uppercase, digits, symbols). `pam_pwquality` enforces it.
- **Length**: the minimum number of characters. Length matters more than complexity. A 16-character passphrase is much harder to guess than an 8-character jumble.
- **Expiration**: how long a password lives before it must change (`chage`, `/etc/login.defs`). Modern guidance (NIST SP 800-63B) says forced regular changes cause weak, predictable passwords, so many organizations now expire passwords only after a suspected compromise. The exam still expects you to know how to set expiration.
- **Reuse** and **History**: the system remembers old password hashes (in `/etc/security/opasswd`) and refuses a new password that matches one of the last N. `pam_pwhistory` does this.

Account lockout stops password guessing. After a few failed logins the account is locked for a while. The current module is `pam_faillock`. The older module, `pam_tally2`, was removed from Linux-PAM in version 1.5.0 (2020), so you will not find it on Ubuntu 24.04 or Rocky 9, but it still appears on the exam and in old documentation.

MFA (multi-factor authentication) means proving identity with two or more different *kinds* of factor: something you know (password), something you have (phone, hardware key), something you are (fingerprint). A TOTP app on a phone is "something you have". It computes a six-digit code from a shared secret and the current time, so the code changes every 30 seconds.

Breach lists are databases of passwords leaked in past data breaches. Attackers try these first. Good practice (and NIST guidance) is to reject any password that appears in such a list.

Restricted shells limit what an account can do. A service account that runs a program but never logs in should have the shell `/sbin/nologin`. A user who needs only a few commands can get `rbash`, a restricted Bash.

Avoid running as root. Root can do anything, so a typo or a malicious script running as root can destroy the system. Log in as a normal user and use `sudo` for single commands. `sudo` also writes a log line every time, so you know who did what.

Tasks

Task 1: Prepare a safe workspace

You are about to change how passwords and logins work. Before that, make sure you cannot lock yourself out.

> **CAUTION**
> Take a snapshot now on **both** VMs. Then keep one terminal with a root shell open for the whole lab. If a PAM change goes wrong, you can fix it from that root shell, or revert to the snapshot.

1. On ubuntu-lab, create the lab directory and open a second terminal (or a second SSH session). In that second terminal, start a root shell and leave it alone:

```
sudo -i
```

> **What just happened:** `sudo -i` starts an interactive login shell as root. Your prompt ends in `#` instead of `$`. Because this shell is already authenticated, it keeps working even if you break PAM in the other terminal. You will use it only for emergencies.

2. In your main terminal, create the working directory and two test users. You will never test password rules on `student` itself.

```
mkdir -p ~/labs/3.4 && cd ~/labs/3.4
sudo useradd -m -s /bin/bash pwtest
sudo passwd pwtest
```

When asked, set the password to `Orbit-Cactus-71!` (you will type it again later).

> **What just happened:** `useradd -m` creates the home directory and `-s /bin/bash` sets the login shell. `passwd pwtest` sets the password. Root can set any password for another user, so this works even before you add rules.

3. Back up the files you will change, so clean up is easy:

```
sudo cp -a /etc/pam.d /root/pam.d.backup-3.4
sudo cp -a /etc/login.defs /root/login.defs.backup-3.4
```

> **What just happened:** `cp -a` copies with all permissions and timestamps kept (“archive” mode). The whole `/etc/pam.d` folder now has a copy in root’s home folder.

Why all this preparation: PAM changes take effect the moment you save the file. There is no "apply" step and no undo button. The open root shell, the backups, and the snapshot give you three separate ways back. Testing on `pwtest` instead of `student` means a broken rule can only hurt an account you do not need.

Checkpoint: `id pwtest` shows the new user, `sudo ls /root/pam.d.backup-3.4` lists the PAM files, and your second terminal shows a `#` root prompt.

Task 2: Enforce password length and complexity with pam_pwquality

1. Install the module. On Ubuntu it is in the `libpam-pwquality` package.

```
sudo apt update
sudo apt install -y libpam-pwquality
grep pwquality /etc/pam.d/common-password
```

```
password    requisite           pam_pwquality.so retry=3
```

What just happened: The package ships a profile in `/usr/share/pam-configs/pwquality`. During install it ran `pam-auth-update`, which added the `pam_pwquality.so` line to `common-password`. `requisite` means "if the new password is weak, stop right away". `retry=3` gives the user three tries before `passwd` gives up.

2. Look at the profile that produced that line, so you understand how Ubuntu's system works:

```
cat /usr/share/pam-configs/pwquality
```

```
Name: Pwquality password strength checking
Default: yes
Priority: 1024
Conflicts: cracklib
Password-Type: Primary
Password:
    requisite           pam_pwquality.so retry=3
Password-Initial:
    requisite           pam_pwquality.so retry=3
```

What just happened: A profile has a name, a priority (higher numbers go earlier in the stack), and the lines to insert for each PAM type. `pam_unix` has priority 256, so `pam_pwquality` (1024) runs first and can reject a weak password before it is saved.

3. Set the rules in `/etc/security/pwquality.conf`. Back it up, then open it:

```
sudo cp /etc/security/pwquality.conf /etc/security/pwquality.conf.orig
sudo nano /etc/security/pwquality.conf
```

Every line in the file is a comment. Add these lines at the bottom, save (`Ctrl+O`, `Enter`) and exit (`Ctrl+X`):

```
minlen = 12
minclass = 3
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = 0
maxrepeat = 3
dictcheck = 1
usercheck = 1
```

What just happened: You set the policy:

- `minlen = 12` sets the minimum **length**. (pwquality can give "credit" for character classes, so read the next point.)
- `minclass = 3` means the password must use at least 3 of the 4 classes: digits, uppercase, lowercase, other (symbols). This is **complexity**.
- The `*credit` options work in two ways. A **negative** number means "require at least this many": `dcredit = -1` requires one digit, `ucredit = -1` one uppercase letter, `lcredit = -1` one lowercase letter. A **positive** number means "each such character counts as extra length", which lets a short password pass `minlen`. `ocredit = 0` means symbols are neither required nor given bonus length. Using negative credits keeps `minlen` an honest length.
- `maxrepeat = 3` rejects more than 3 identical characters in a row (`aaaa`).
- `dictcheck = 1` rejects passwords based on dictionary words (it uses the cracklib word list).
- `usercheck = 1` rejects passwords that contain the user name.

Changes take effect immediately. PAM reads this file every time someone changes a password.

4. Test the rules as the test user. A normal user must type the old password first, then a new one. Switch to `pwtest`:

```
su - pwtest
passwd
```

Enter the current password `Orbit-Cactus-71!`. Then try these new passwords one at a time and read each message:

Try this	Why it fails
`Sh0rt!`	too short
`alllowercaseletters`	no digit (and too few classes)
`Basketball12`	dictionary word
`Pwtest-Cactus-99`	contains the user name

```
Changing password for pwtest.
Current password:
New password:
BAD PASSWORD: The password is shorter than 12 characters
New password:
BAD PASSWORD: The password contains less than 1 digits
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
passwd: Have exhausted maximum number of retries for service
passwd: password unchanged
```

What just happened: Each `BAD PASSWORD:` line is `pam_pwquality` explaining the **first** rule that failed (the "at least one digit" rule from `dcredit = -1` is checked before `minclass`). After three failures (`retry=3`) `passwd` stops. Run `passwd` again to try the fourth example. The exact wording of the messages can differ a little between versions, but each one names the rule.

5. Still as `pwtest`, change the password to a good one, then return to `student`:

```
passwd
exit
```

Current password: `Orbit-Cactus-71!`. New password (twice): `Velvet-Harbor-38#`.

```
passwd: password updated successfully
```

TIP

When **root** runs `passwd someuser`, pwquality only prints a warning and lets root set a weak password anyway. Add `enforce_for_root` to `pwquality.conf` if you want the rules to apply to root too. This is why you always test password rules as a normal user.

Checkpoint: `sudo grep -v '^#' /etc/security/pwquality.conf | grep .` shows your nine settings, and `pwtest` was able to change to `Velvet-Harbor-38#` but not to the weak examples.

Task 3: Block password reuse with pam_pwhistory

`pam_pwquality` stops weak passwords, but a user could still switch back and forth between two strong passwords forever. `pam_pwhistory` keeps a **history** of old password hashes in `/etc/security/opasswd` and blocks **reuse**.

1. Ubuntu has no ready-made profile for `pam_pwhistory`, so you will write one. This uses a *heredoc*: everything between `<<'EOF'` and the line `EOF` is sent as input to `sudo tee`, which writes it to the file. The quotes around `'EOF'` stop the shell from changing anything inside. Note the line under `Password:` must start with a tab or spaces.

```
cat <<'EOF' | sudo tee /usr/share/pam-configs/pwhistory
Name: Remember the last 5 passwords (Lab 3.4)
Default: yes
Priority: 1000
Password-Type: Primary
Password:
    requisite pam_pwhistory.so remember=5 use_authtok
EOF
```

What just happened: You created a profile. Priority 1000 places it after `pam_pwquality` (1024) and before `pam_unix` (256). The options:

- `remember=5` keeps the last 5 passwords per user and rejects any of them.
- `use_authtok` means "do not ask for the new password again; use the one the earlier module already collected". Without it the user would be asked twice.

2. Apply the profile. `pam-auth-update` normally shows a menu; `--package` makes it apply enabled profiles without asking.

```
sudo pam-auth-update --package
grep -v '^#' /etc/pam.d/common-password | grep .
```

```
password    requisite           pam_pwquality.so retry=3
password    requisite pam_pwhistory.so remember=5 use_authtok
password    [success=1 default=ignore]  pam_unix.so obscure use_authtok try_first_pass
yescrypt
password    requisite           pam_deny.so
password    required            pam_permit.so
```

What just happened: The order is exactly what you want: quality check, then history check, then `pam_unix` saves the new hash (with the `yescrypt` algorithm) in `/etc/shadow`. `pam-auth-update` also adjusted the `[success=1 ...]` jump for you, which is the main reason to let the tool write the file. If your output shows `pam_pwhistory` after `pam_unix`, stop and use `sudo pam-auth-update` (menu mode) to check which profiles are enabled.

3. Test it. As `pwtest`, change the password twice, then try to go back to an old one:

```
su - pwtest
passwd
```

- Change `Velvet-Harbor-38#` to `Quiet-Lantern-52$` (works).
- Run `passwd` again and try to change `Quiet-Lantern-52$` back to `Velvet-Harbor-38#`:

```
Current password:
New password:
Password has been already used. Choose another.
passwd: Authentication token manipulation error
passwd: password unchanged
```

Type `exit` to return to `student`.

> **What just happened:** When `pwtest` changed away from `Velvet-Harbor-38#`, `pam_pwhistory` stored that old password's hash in `/etc/security/opasswd`. The second attempt matched a stored hash, so the module refused. Note: only *previous* passwords are stored; that is why the very first change in Task 2 had nothing to compare with.

4. Look at the history file (as root, it is readable only by root):

```
sudo cat /etc/security/opasswd
```

```
pwtest:1001:1:$y$j9T$...
```

> **What just happened:** Each line holds the user name, UID, the number of stored entries, and the old hashes (separated by commas once there is more than one). Only one is stored so far: `Velvet-Harbor-38#`, the password `pwtest` changed away from after the module was enabled. The passwords themselves are never stored, only hashes (`$y$` means yescrypt).

> **ON THE EXAM**
> On the exam, password history can also be done with `pam_unix.so remember=N` (older systems). Both store hashes in `/etc/security/opasswd`. On Rocky/RHEL the history settings can go in `/etc/security/pwhistory.conf`, and the feature is turned on with `authselect enable-feature with-pwhistory`.

> **Checkpoint:** `grep pwhistory /etc/pam.d/common-password` shows the line, and reusing `Velvet-Harbor-38#` was rejected.

Task 4: Set password expiration with chage and login.defs

1. See the current aging settings for `pwtest`:

```
sudo chage -l pwtest
```

```
Last password change                    : Oct 10, 2026
Password expires                      : never
Password inactive                     : never
Account expires                       : never
Minimum number of days between password change      : 0
Maximum number of days between password change      : 99999
Number of days of warning before password expires   : 7
```

> **What just happened:** `chage` ("change age") reads the aging fields from `/etc/shadow`. A maximum of 99999 days means the password effectively never expires.

2. Set an **expiration** policy for this user:

```
sudo chage -M 90 -m 1 -W 14 -I 7 -E 2027-12-31 pwtest
sudo chage -l pwtest
```

```
Last password change                    : Oct 10, 2026
Password expires                        : Jan 08, 2027
Password inactive                       : Jan 15, 2027
Account expires                         : Dec 31, 2027
Minimum number of days between password change      : 1
Maximum number of days between password change      : 90
Number of days of warning before password expires   : 14
```

> **What just happened:**
>
> - `-M 90` maximum password age: it must change every 90 days.
> - `-m 1` minimum age: the user must wait 1 day between changes. Without this, a user could change the password 5 times in a row to flush the history and get the old one back.
> - `-W 14` warn the user 14 days before expiry.
> - `-I 7` inactive days: after the password expires the user has 7 days to log in and change it; after that the account is locked.
> - `-E 2027-12-31` the whole *account* expires on that date (good for contractors).

3. Force a password change at the next login. This is what you do after creating an account with a temporary password:

```
sudo chage -d 0 pwtest
sudo chage -l pwtest | head -2
```

```
Last password change                    : password must be changed
Password expires                        : password must be changed
```

> **What just happened:** `-d 0` sets the "last change" date to day 0 (1 January 1970), so the password is already expired. `sudo passwd -e pwtest` does the same thing. At the next login the user is forced to choose a new password, and that new password must still pass pwquality and pwhistory.

4. Defaults for **new** accounts come from `/etc/login.defs`. Look at them:

```
grep -E '^PASS_(MAX|MIN|WARN)' /etc/login.defs
```

```
PASS_MAX_DAYS   99999
PASS_MIN_DAYS   0
PASS_WARN_AGE   7
```

Change the defaults with `sed` (stream editor), which edits the file in place (`-i`):

```
sudo sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t90/' /etc/login.defs
sudo sed -i 's/^PASS_MIN_DAYS.*/PASS_MIN_DAYS\t1/' /etc/login.defs
sudo useradd -m agetest
sudo chage -l agetest | tail -3
```

```
Minimum number of days between password change      : 1
Maximum number of days between password change      : 90
Number of days of warning before password expires   : 7
```

What just happened: `s/old/new/` replaces text; `^PASS_MAX_DAYS.*` matches the whole line that starts with `PASS_MAX_DAYS`; `\t` writes a tab. The new user `agetest` picked up the new defaults. Existing users like `pwtest` and `student` are **not** changed by `login.defs`; you must use `chage` on them.

Checkpoint: `sudo chage -l pwtest` says "password must be changed", and `sudo chage -l agetest` shows a maximum of 90 days.

Task 5: Lock accounts after failed logins with pam_faillock (rocky-lab)

On Ubuntu 24.04 `pam_faillock` is installed but not turned on, and enabling it means hand-editing `common-auth` with careful jump counts. On Rocky Linux it is one safe command with `authselect`, so you will do this task on rocky-lab.

1. On rocky-lab, open a second terminal with `sudo -i` and leave it open, just like in Task 1. Then check which authselect profile is active:

```
# on rocky-lab
sudo authselect current
```

```
Profile ID: sssd
Enabled features:
- with-silent-lastlog
- with-mdns4
```

What just happened: `authselect` manages the PAM files on Rocky. Your profile may be `sssd`, `minimal`, or `local`; all support faillock. If it says "No existing configuration detected", run `sudo authselect select sssd --force` first (you took a snapshot).

2. Create a test user and enable the faillock feature:

```
# on rocky-lab
sudo useradd -m locktest
echo 'locktest:Rocky-Lock-2026!' | sudo chpasswd
sudo authselect enable-feature with-faillock
grep faillock /etc/pam.d/system-auth
```

```
auth        required                                     pam_faillock.so preauth silent
auth        required                                     pam_faillock.so authfail
account     required                                     pam_faillock.so
```

What just happened: `chpasswd` reads `user:password` lines from standard input, which is handy in scripts. `enable-feature with-faillock` rewrote `system-auth` and `password-auth` (used by `sshd`) to add three faillock lines. `preauth` checks "is this account locked?" before asking for the password, `authfail` records a failure after a wrong password, and the `account` line blocks a locked user even if the password is right.

3. Set the lockout policy in `/etc/security/faillock.conf`. Back it up and append two settings:

```
# on rocky-lab
sudo cp /etc/security/faillock.conf /etc/security/faillock.conf.orig
printf 'deny = 3\nunlock_time = 120\n' | sudo tee -a /etc/security/faillock.conf
```

What just happened: `deny = 3` locks the account after 3 failures in a row; `unlock_time = 120` unlocks it automatically after 120 seconds. `unlock_time = 0` would mean "locked until an admin unlocks it". By default root is never locked (`even_deny_root` turns that on), which keeps you from locking out the only admin.

4. Trigger the lockout. SSH to `locktest` on the same machine and type a **wrong** password three times:

```
# on rocky-lab
ssh locktest@localhost
```

Type `yes` if asked about the host key. After three wrong passwords, try once more with the **correct** password `Rocky-Lock-2026!`. It is still refused.

5. Look at the failure records and reset them:

```
# on rocky-lab
sudo faillock --user locktest
```

```
locktest:
When                Type  Source                                           Valid
2026-10-10 14:02:11 RHOST ::1                                                 V
2026-10-10 14:02:15 RHOST ::1                                                 V
2026-10-10 14:02:19 RHOST ::1                                                 V
```

```
# on rocky-lab
sudo faillock --user locktest --reset
ssh locktest@localhost
```

What just happened: `faillock --user` lists recorded failures (stored in `/var/run/faillock/`). `V` means the record still counts. `--reset` clears them, so the correct password now works. Type `exit` to leave the `locktest` session. Failed attempts also show in `sudo journalctl -u sshd`.

ON THE EXAM

pam_tally2 is the deprecated predecessor of pam_faillock. You may see this syntax on the exam:

- PAM line: `auth required pam_tally2.so deny=3 unlock_time=600 onerr=fail`
- Show failures: `pam_tally2 --user bob`
- Reset: `pam_tally2 --user bob --reset`

The modern equivalents are `pam_faillock.so` with `/etc/security/faillock.conf`, `faillock --user bob`, and `faillock --user bob --reset`. Older still: `pam_tally` and `faillog`.

TIP

On Ubuntu, faillock is enabled by adding `pam_faillock.so preauth` before `pam_unix` and `pam_faillock.so authfail` after it in `/etc/pam.d/common-auth`, plus `account required pam_faillock.so` in `common-account`, adjusting the `[success=N]` jump numbers. It works, but one wrong number locks everyone out, which is why this lab uses Rocky.

Checkpoint: `sudo faillock --user locktest` showed three failures before the reset and none after it, and `locktest` could log in again.

Task 6: Add MFA (TOTP) to SSH

You will require a password **and** a six-digit TOTP code for one test user. Instead of a phone, you will use `oathtool`, which calculates the same codes from the secret. (An authenticator app works too if you want to try it.)

CAUTION

This changes `sshd` and its PAM file. Keep your root shell open. Do not close your current SSH session until you have tested a **new** login. Apply MFA only to the test user, as shown.

1. Install the PAM module and `oathtool`, and create the MFA test user:

```
sudo apt install -y libpam-google-authenticator oathtool
sudo useradd -m -s /bin/bash mfauser
echo 'mfauser:Mfa-Lab-Pass-42!' | sudo chpasswd
```

2. Become `mfauser` and create the TOTP secret:

```
sudo -iu mfauser
google-authenticator -t -d -f -r 3 -R 30 -w 3 -Q NONE
```

```
Your new secret key is: JBSWY3DPEHPK3PXPLABS7Q2RMA
Enter code from app (-1 to skip): -1
Code confirmation skipped
Your emergency scratch codes are:
  40812236
  ...
```

Type `-1` to skip the confirmation. Write down the secret key.

What just happened: The tool wrote `~/.google_authenticator` (mode 0400, readable only by `mfauser`). Options: `-t` time-based codes (TOTP); `-d` disallow reusing the same code; `-f` write the file without asking; `-r 3 -R 30` allow at most 3 logins every 30 seconds; `-w 3` accept the code before and after the current one to allow small clock differences; `-Q NONE` do not draw a QR code. Normally you would scan the QR code with a phone app. The scratch codes are one-time backup codes if the phone is lost.

3. Still as `mfauser`, generate the current code from the secret, then go back to `student`:

```
oathtool --totp -b "$(head -1 ~/.google_authenticator)"
exit
```

```
492039
```

What just happened: The first line of `~/.google_authenticator` is the Base32 secret. `oathtool --totp -b` decodes the Base32 secret (`-b`) and computes the TOTP code for the current 30-second window. Your phone app does exactly the same calculation. That is why both clocks must be correct; on Ubuntu 24.04 `systemd-timesyncd` keeps the VM clock in sync (check with `timedatectl`).

4. Tell PAM to ask for the code during SSH logins. Open `/etc/pam.d/sshd`:

```
sudo nano /etc/pam.d/sshd
```

Find the line `@include common-auth` near the top and add this line directly **below** it:

```
auth required pam_google_authenticator.so nullok
```

What just happened: After the normal password check (`common-auth`), PAM now runs the Google Authenticator module. `nullok` means "users without a `~/.google_authenticator` file can skip this step", so `student` and other users are not affected. In production you remove `nullok` once every user is enrolled.

5. Tell `sshd` to use keyboard-interactive login (the method that can ask several questions) and require it for `mfauser`. Create a drop-in file, and add a `Match` block at the very end of the main config:

```
echo 'KbdInteractiveAuthentication yes' | sudo tee /etc/ssh/sshd_config.d/10-mfa.conf
printf '\nMatch User mfauser\n    AuthenticationMethods keyboard-interactive\n' \
  | sudo tee -a /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload ssh
```

What just happened: Ubuntu's `sshd_config` sets `KbdInteractiveAuthentication no`. The `Include /etc/ssh/sshd_config.d/*.conf` line sits at the top of that file, and `sshd` keeps the **first** value it reads, so your drop-in wins. The `Match User mfauser` block must be at the end of the file because every line after a `Match` belongs to it. `AuthenticationMethods keyboard-interactive` makes `mfauser` go through the full PAM conversation (password, then code); a plain password or key alone is not enough. `sshd -t` tests the config for errors; `reload` applies it only if the test passed.

6. Confirm the settings, then test a **new** login:

```
sudo sshd -T -C user=mfauser,host=localhost,addr=127.0.0.1 | grep -i authenticationmethods
ssh mfauser@localhost
```

```
authenticationmethods keyboard-interactive
(mfauser@localhost) Password:
(mfauser@localhost) Verification code:
```

Type the password `Mfa-Lab-Pass-42!`. For the code, run `oathtool --totp -b "$(sudo head -1 /home/mfauser/.google_authenticator)"` in another terminal and type the result. You are logged in. Type `exit`. Then try again with a wrong code, and see the login refused.

What just happened: `sshd -T` prints the effective configuration; `-C` tells it which connection to simulate, so you can test `Match` blocks safely. The login now needs two factors: something you know (password) and something you have (the device holding the secret). Note that `-d` stops you reusing the same code in the same 30 seconds, so wait for a fresh code before a second login.

TIP

A stronger production setup is `AuthenticationMethods publickey,keyboard-interactive`: an SSH key first, then the TOTP code (with the password line removed from the PAM stack). Hardware security keys (FIDO2, `ssh-keygen -t ed25519-sk`) are another strong MFA option.

Checkpoint: `ssh mfauser@localhost` asks for both a password and a verification code, and `ssh student@localhost` still works with just a password (or key).

Task 7: Check a password against a breach list

Have I Been Pwned (HIBP) keeps a list of over 900 million passwords seen in real breaches. You can check a password against it **without sending the password**, thanks to a trick called *k-anonymity*.

> **CAUTION**
> Only use obviously bad test passwords like `password123` in this task. Never type a real password into a command line, because it is saved in your shell history.

1. Compute the SHA-1 hash of the test password:

```
echo -n 'password123' | sha1sum
```

```
cbfdac6008f9cab4083784cbd1874f76618d2a97  -
```

> **What just happened:** `echo -n` prints the text without a newline (a newline would change the hash). `sha1sum` computes the SHA-1 hash, a 40-character hex fingerprint. The same input always gives the same hash.

2. Send only the first 5 characters (`CBFDA`) to the range API and search the answer locally for the remaining 35 characters:

```
curl -s https://api.pwnedpasswords.com/range/CBFDA | grep -i
'C6008F9CAB4083784CBD1874F76618D2A97'
```

```
C6008F9CAB4083784CBD1874F76618D2A97:251682
```

> **What just happened:** The server returned every hash suffix that starts with `CBFDA` (several hundred lines) and how often each one was seen. `grep -i` (case-insensitive) found yours locally. The number after the colon is how many times this password appeared in breaches; yours will be different and probably larger. The server never learned your full hash, so it cannot know which password you checked. That is k-anonymity: your request hides among many possible answers.

3. Wrap it in a small reusable script that reads the password without showing it:

```
cat <<'EOF' > pwcheck.sh
#!/bin/bash
read -rsp "Password to check: " pw; echo
hash=$(printf '%s' "$pw" | sha1sum | awk '{print toupper($1)}')
prefix=${hash:0:5}; suffix=${hash:5}
count=$(curl -s "https://api.pwnedpasswords.com/range/$prefix" \
  | grep "^$suffix:" | cut -d: -f2 | tr -d '\r')
echo "Seen in breaches: ${count:-0} times"
EOF
chmod +x pwcheck.sh
./pwcheck.sh
```

Type `password123` (nothing appears as you type), then try a random string like `Xq7-unlikely-Plinth-9204`.

What just happened: `read -s` hides input, so the password never lands in shell history. `${hash:0:5}` takes the first 5 characters and `${hash:5}` the rest. `tr -d '\r'` removes the Windows-style line endings the API returns. A result of 0 means "not in a known breach", not "strong", so you still need length and complexity rules.

TIP

Some organizations plug this check directly into password changes, for example with a PAM module or a password manager feature. `pam_pwquality`'s `dictcheck` is a much smaller local version of the same idea.

Checkpoint: `./pwcheck.sh` reports a large number for `password123` and 0 for your random string.

Task 8: Use restricted shells (nologin and rbash)

1. First, look at where `nologin` lives on Ubuntu:

```
ls -ld /sbin /bin
ls -l /usr/sbin/nologin
grep -E 'nologin|rbash' /etc/shells
```

```
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
-rwxr-xr-x 1 root root 14640 Apr  5  2024 /usr/sbin/nologin
/bin/rbash
/usr/bin/rbash
```

What just happened: Ubuntu uses "merged /usr": `/sbin` is a symbolic link to `/usr/sbin` and `/bin` to `/usr/bin`. So `/sbin/nologin` (the path the exam uses, and the normal path on Rocky) and `/usr/sbin/nologin` are the same file. `/etc/shells` lists valid login shells; `nologin` is deliberately not in it.

2. Create a service account with no login shell:

```
sudo useradd -r -M -s /usr/sbin/nologin svcapp
getent passwd svcapp
sudo su - svcapp
```

```
svcapp:x:998:998::/home/svcapp:/usr/sbin/nologin
This account is currently not available.
```

What just happened: `-r` makes a system account (UID below 1000), `-M` skips the home folder, and `-s /usr/sbin/nologin` sets the shell. When anyone tries to log in, `nologin` prints a message and exits with a failure code. A program can still run *as* `svcapp` (for example from a systemd unit with `User=svcapp`), but no one gets an interactive shell. This is the standard for service accounts. `/bin/false` does the same job silently.

3. Customize the refusal message. This works on **rocky-lab**: its `nologin` comes from the `util-linux` package and prints the contents of `/etc/nologin.txt` if that file exists. (Ubuntu's `nologin` comes from the `login` package and always prints the fixed message you just saw.)

```
# on rocky-lab
sudo useradd -r -M -s /sbin/nologin svcapp
echo 'Service accounts cannot log in. Contact the admin team.' \
  | sudo tee /etc/nologin.txt
sudo su - svcapp
```

```
Service accounts cannot log in. Contact the admin team.
Service accounts cannot log in. Contact the admin team.
```

What just happened: you created the same kind of service account on rocky-lab, where the path is `/sbin/nologin`. The first line of output is `tee` echoing what it wrote; the second is `nologin` refusing the login with your custom text.

CAUTION

Do not confuse `/etc/nologin.txt` with `/etc/nologin`. If the file `/etc/nologin` exists, **all non-root users** are blocked from logging in (used during maintenance).

4. Now create a user with a restricted Bash shell, `rbash`, who can run only `ls`, `date`, and `whoami`:

```
sudo useradd -m -s /bin/rbash rbuser
echo 'rbuser:Rbash-Lab-Pass-7!' | sudo chpasswd
sudo mkdir /home/rbuser/bin
sudo ln -s /usr/bin/ls /usr/bin/date /usr/bin/whoami /home/rbuser/bin/
echo 'export PATH=$HOME/bin' | sudo tee /home/rbuser/.bash_profile
sudo chown root:root /home/rbuser/.bash_profile /home/rbuser/bin
sudo chmod 644 /home/rbuser/.bash_profile
```

What just happened: `rbash` is Bash in restricted mode (`/bin/rbash` is a link to `bash`; Bash restricts itself when started under that name). The startup file `.bash_profile` runs *before* the restrictions start, so it can set `PATH` to a folder with only three symbolic links. The file and folder are owned by root, so `rbuser` cannot change them.

5. Log in as `rbuser` and see what is blocked:

```
ssh rbuser@localhost
```

Inside that session, try each line:

```
whoami
ls /home
cd /tmp
/usr/bin/id
export PATH=/usr/bin
echo hello > test.txt
vim
```

```
rbuser
agetest  mfauser  pwtest  rbuser  student
-rbash: cd: restricted
-rbash: /usr/bin/id: restricted: cannot specify `/' in command names
-rbash: PATH: readonly variable
-rbash: test.txt: restricted: cannot redirect output
-rbash: vim: command not found
```

Type `exit`.

What just happened: In restricted mode Bash refuses to: change directory, run a command whose name contains `/`, change `PATH` (and `SHELL`, `ENV`, `BASH_ENV`), or redirect output with `>`. Since `PATH` contains only `~/bin`, the user can run only the three linked commands.

CAUTION

`rbash` is a speed bump, not a wall. If you link any program that can start another program (`vi`, `less`, `python3`, `awk`, `find`), the user can escape to a full shell. Also, because `rbuser` owns the home folder, file transfer with `sftp` / `scp` could still change files. For real isolation use containers, chroot jails, or SSH `ForceCommand`.

Checkpoint: on rocky-lab, `sudo su - svcapp` prints your custom message, and on ubuntu-lab `rbuser` can run `whoami` but gets "restricted" errors for `cd` and `/usr/bin/id`.

Task 9: Avoid running as root

1. Check the root account's password status:

```
sudo passwd -S root
```

```
root L 2024-04-22 0 99999 7 -1
```

What just happened: `passwd -S` shows status: user, state, date of the last password change, then the minimum, maximum, warning, and inactive days. `L` means the root password is **locked** (Ubuntu does this at install). No one can log in as root with a password; admins use `sudo` instead. `P` would mean a usable password and `NP` no password at all (dangerous). On Rocky, if root has a password you can lock it with `sudo passwd -l root` after confirming `student` can use `sudo`.

2. Block root logins over SSH completely:

```
sudo sshd -T | grep -i permitrootlogin
echo 'PermitRootLogin no' | sudo tee /etc/ssh/sshd_config.d/10-noroot.conf
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep -i permitrootlogin
```

```
permitrootlogin without-password
PermitRootLogin no
permitrootlogin no
```

What just happened: The default `prohibit-password` (shown as `without-password`) allows root to log in with an SSH key. `no` blocks root completely, so attackers cannot even try. Admins log in as themselves, which means every action is tied to a real person.

3. Use `sudo` for single commands instead of a root shell, and see what you are allowed to do:

```
sudo -l
getent group sudo
```

```
User student may run the following commands on ubuntu-lab:
    (ALL : ALL) ALL
sudo:x:27:student
```

What just happened: `sudo -l` lists your sudo rights. Members of the `sudo` group (on Rocky: `wheel`) may run any command as any user. Good practice: give people only the commands they need with files in `/etc/sudoers.d/` (always edited with `visudo`), and prefer `sudo command` over long-lived `sudo -i` shells like the emergency one you opened in Task 1.

4. Audit sudo usage. Every `sudo` call is logged:

```
sudo grep 'sudo:' /var/log/auth.log | grep COMMAND | tail -5
sudo journalctl _COMM=sudo --since today | grep COMMAND | tail -5
```

```
2026-10-10T14:31:02 ubuntu-lab sudo:  student : TTY=pts/0 ; PWD=/home/student/labs/3.4 ;
USER=root ; COMMAND=/usr/bin/passwd -S root
```

What just happened: Each line records who ran `sudo` (`student`), from which terminal, in which folder, as which user (`USER=root`), and the exact command. Failed attempts show as `authentication failure` or `user NOT in sudoers`. On Rocky the same lines go to `/var/log/secure` and the journal. A shared root password would give you none of this accountability.

Checkpoint: `sudo sshd -T | grep permitrootlogin` prints `no`, and `journalctl _COMM=sudo` shows your own recent commands.

Challenge

CHALLENGE

1. On rocky-lab, enable password history with `authselect enable-feature with-pwhistory`, set `remember = 5` in `/etc/security/pwhistory.conf`, and prove it works with a test user. Then turn the feature off again.
2. Create a user `reportuser` whose only allowed action over SSH is to run `date`. Use `ForceCommand` in a `Match User` block instead of `rbash`, and explain why it is harder to escape.
3. Write a one-line command that lists every account in `/etc/passwd` with a real login shell (not `nologin` or `false`) and a UID of 1000 or higher. Which ones should not have a shell?

Clean up

1. On ubuntu-lab, remove the SSH changes and reload:

```
sudo rm -f /etc/ssh/sshd_config.d/10-mfa.conf /etc/ssh/sshd_config.d/10-noroot.conf
sudo sed -i '/^Match User mfauser/,$d' /etc/ssh/sshd_config
sudo sed -i '/pam_google_authenticator.so/d' /etc/pam.d/sshd
sudo sshd -t && sudo systemctl reload ssh
```

   The first `sed` deletes from the `Match User mfauser` line to the end of the file (`$`); the second deletes the PAM line.

2. Remove the pwhistory profile and restore the password settings:

```
sudo pam-auth-update --remove pwhistory
sudo rm -f /usr/share/pam-configs/pwhistory /etc/security/opasswd
sudo cp /etc/security/pwquality.conf.orig /etc/security/pwquality.conf
sudo cp /root/login.defs.backup-3.4 /etc/login.defs
sudo apt purge -y libpam-pwquality
grep -v '^#' /etc/pam.d/common-password | grep .
```

   `common-password` should be back to the three lines it had before the lab (`pam_unix.so`, `pam_deny.so`, `pam_permit.so`): purging `libpam-pwquality` runs `pam-auth-update` again to take its line out.

3. Delete the lab users and files:

```
sudo userdel -r pwtest
sudo userdel -r agetest
sudo userdel -r mfauser
sudo userdel -r rbuser
sudo userdel svcapp
sudo rm -f /etc/security/pwquality.conf.orig
sudo rm -rf /root/pam.d.backup-3.4 /root/login.defs.backup-3.4 ~/labs/3.4
sudo apt purge -y libpam-google-authenticator oathtool
```

4. On rocky-lab, turn off faillock and remove the test user:

```
# on rocky-lab
sudo authselect disable-feature with-faillock
sudo cp /etc/security/faillock.conf.orig /etc/security/faillock.conf
sudo rm -f /etc/security/faillock.conf.orig
sudo userdel -r locktest
sudo userdel svcapp
sudo rm -f /etc/nologin.txt
```

5. Close the emergency root shells on both VMs (`exit`). Test that `ssh student@localhost` still works on both. (Or simply revert both VMs to the snapshot you took.)

Exam corner

- `pam_pwquality` (config `/etc/security/pwquality.conf`) enforces **length** (`minlen`) and **complexity** (`minclass`, `dcredit`, `ucredit`, `lcredit`, `ocredit`). Negative credit = required count; positive credit = bonus length.
- `pam_pwhistory remember=N` (or `pam_unix remember=N`) blocks password **reuse**; the **history** of hashes is in `/etc/security/opasswd`.
- `chage -M` max days, `-m` min days, `-W` warn days, `-I` inactive days, `-E` account expiry date, `-d 0` force change at next login, `-l` list. `passwd -e` also forces a change.
- `/etc/login.defs` (`PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`) sets defaults for **new** accounts only.
- Account lockout: `pam_faillock` with `/etc/security/faillock.conf` (`deny`, `unlock_time`); view/reset with `faillock --user NAME [--reset]`. **pam_tally2** (`pam_tally2 --user NAME --reset`) is the removed predecessor.
- Ubuntu manages PAM with `pam-auth-update`; Rocky/RHEL with `authselect` (`enable-feature with-faillock`).
- **MFA** = two or more different factor types. TOTP via `pam_google_authenticator.so` plus `KbdInteractiveAuthentication yes` and `AuthenticationMethods` in `sshd_config`.
- **Breach** checking: HIBP range API uses k-anonymity: only the first 5 hex characters of the SHA-1 hash leave your machine.
- **Restricted shell**: `/sbin/nologin` (or `/bin/false`) for service accounts; `/bin/rbash` blocks `cd`, `/` in commands, changing `PATH`, and output redirection.
- Avoid running as **root**: locked root password, `PermitRootLogin no`, least-privilege `sudo` rules, and `sudo` logs in `/var/log/auth.log` (Ubuntu) or `/var/log/secure` (RHEL).

Review questions

1. Which `pwquality.conf` setting requires at least two digits in every new password? A. `dcredit = 2` B. `dcredit = -2` C. `minclass = 2` D. `mindigit = 2`
2. A user keeps changing their password five times in a row to get back to their favorite password. Which two settings together stop this?

3. Which command clears the failed-login counter for user `bob` on a modern system? A. `pam_tally2 --user bob --reset` B. `faillog -r bob` C. `faillock --user bob --reset` D. `passwd -u bob`
4. What shell should a service account that must never log in interactively have?
5. When you check a password with the HIBP range API, what is actually sent to the server? A. The password B. The full SHA-1 hash C. The first 5 hex characters of the SHA-1 hash D. Nothing; the list is downloaded
6. Which `sshd_config` setting makes a user complete both a public key login and a TOTP prompt?

Lab 3.5 Use Cryptography on Linux

Objective: 3.5 Explain cryptographic concepts and technologies in a Linux environment.
Domain: 3.0 Security (18% of the exam)
Time: about 150 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes. This lab encrypts an extra disk, changes SSH algorithms, and changes the system crypto policy on rocky-lab.

Cryptography protects data in two situations: when it sits on a disk (**data at rest**) and when it travels over a network (**data in transit**). In this lab you will hash and sign files, encrypt files with GPG, encrypt a whole disk with LUKS2, build your own small certificate authority, serve a website over TLS, build a WireGuard VPN between your two VMs, and remove weak algorithms. By the end, terms like "Argon2", "trusted root", and "TLS 1.3" will be things you have touched, not just words to memorize.

What you'll practice

- Data at rest: File encryption (GPG); Filesystem encryption (LUKS2, Argon2)
- Data in transit: OpenSSL, WireGuard, LibreSSL, TLS protocol versions
- Hashing: SHA-256, HMAC
- Removal of weak algorithms (SSH algorithm lists, Rocky crypto policies, old TLS versions)
- Certificate management: Trusted root certificates (no-cost, commercial); Avoiding self-signed certificates

How it works

Three tools in the crypto toolbox.

1. **Hashing** turns any amount of data into a short, fixed-length fingerprint. SHA-256 always produces 256 bits (64 hex characters). Change one byte of input and the hash changes completely. You cannot turn a hash back into the data. Hashes prove **integrity** ("this file has not changed"). Old hashes like MD5 and SHA-1 are broken: researchers can create two different files with the same hash, so they are considered weak algorithms.

2. **Symmetric encryption** uses one secret key to lock and unlock, like a house key. It is fast, so it is used for bulk data. AES (Advanced Encryption Standard) and ChaCha20 are the common modern ciphers.

3. **Asymmetric (public-key) encryption** uses a key *pair*. The **public key** can be shared with anyone; the **private key** stays secret. Data encrypted with the public key can only be decrypted with the private key. The private key can also create a **digital signature** that anyone with the public key can check. RSA, ECDSA, and Ed25519 are common examples. Asymmetric crypto is slow, so real systems use it to agree on a symmetric key, then switch to symmetric encryption.

HMAC (Hash-based Message Authentication Code) mixes a secret key into a hash. A plain hash proves a file did not change *by accident*, but an attacker who changes the file can just compute a new hash. With HMAC, only someone who knows the key can compute the correct value, so it proves integrity **and** that the sender knew the key.

Data at rest.

- **GPG** (GNU Privacy Guard) encrypts and signs individual files. It can use a passphrase (symmetric) or public keys (asymmetric).
- **LUKS2** (Linux Unified Key Setup, version 2) encrypts a whole block device. The data is encrypted with one random *master key*. That master key is stored in the LUKS header, locked in up to 32 *key slots*. Each slot is protected by a passphrase or key file. To turn your passphrase into a key, LUKS2 uses **Argon2id**, a *key derivation function* designed to need lots of memory and time, which makes guessing passphrases with GPUs very expensive. (LUKS1 used PBKDF2, which needs almost no memory.)

```
passphrase "blue..."  --Argon2id-->  slot key  --unlocks-->  master key
passphrase #2         --Argon2id-->  slot key  --unlocks-->  (same master key)
                                                                  |
                                     AES-XTS encrypts every sector on /dev/sdb
```

Because every slot unlocks the same master key, you can add or remove passphrases without re-encrypting the disk. If you destroy all key slots, the master key is gone and the data is unreadable forever. That is **cryptographic erase**.

Data in transit.

- **TLS** (Transport Layer Security) protects HTTPS, secure email, and much more. SSL 2.0, SSL 3.0, TLS 1.0, and TLS 1.1 are obsolete and must be disabled (TLS 1.0/1.1 were formally deprecated by RFC 8996 in 2021). Use **TLS 1.2** and **TLS 1.3**. TLS 1.3 is faster (one round trip) and removed all old, weak cipher options.
- **OpenSSL** is the TLS library and command-line toolkit on almost every Linux system. **LibreSSL** is a fork of OpenSSL created by the OpenBSD project in 2014, after the Heartbleed bug, to clean up and simplify the code. It is the default on OpenBSD and is used by macOS, but Ubuntu and Rocky ship OpenSSL. Its `openssl` command works almost the same way.
- **WireGuard** is a modern VPN built into the Linux kernel since version 5.6. It uses a small fixed set of modern algorithms (Curve25519, ChaCha20-Poly1305, BLAKE2s) and a configuration much simpler than IPsec or OpenVPN. Each peer has a key pair, a bit like SSH keys.

Certificates and trust. A TLS certificate binds a public key to a name (like `www.example.com`) and is signed by a **certificate authority (CA)**. Your system has a list of **trusted root certificates** (on Ubuntu in `/etc/ssl/certs`, from the `ca-certificates` package). A browser trusts a site's certificate if it can follow the chain of signatures up to one of those roots.

```
Trusted root CA (in /etc/ssl/certs)      e.g. ISRG Root X1, DigiCert Global Root
       | signs
Intermediate CA                          e.g. Let's Encrypt R11
       | signs
Server (leaf) certificate                CN/SAN = www.example.com
```

- **No-cost** CAs: Let's Encrypt issues free certificates automatically using the **ACME** protocol (the `certbot` tool). The certificates are trusted everywhere and live 90 days, so renewal is automated.

- **Commercial** CAs (DigiCert, Sectigo, GlobalSign, and others) charge money. They offer organization-validated (OV) and extended-validation (EV) certificates, longer support, warranties, and help with special cases. The encryption is exactly the same strength.
- A **self-signed** certificate is signed by its own key. No one vouches for it, so clients show errors, and users learn to click "accept anyway", which makes man-in-the-middle attacks easy. Avoid self-signed certificates in production. For internal systems, run a private CA and install its root on your machines, as you will do in this lab.

Tasks

Task 1: Prove integrity with SHA-256 and HMAC

1. Create the working directory and two sample files:

```
mkdir -p ~/labs/3.5 && cd ~/labs/3.5
echo "Quarterly report: revenue up 4%" > report.txt
echo "Server inventory: web01 web02 db01" > inventory.txt
```

2. Compute SHA-256 hashes and save them to a checksum file:

```
sha256sum report.txt inventory.txt | tee SHA256SUMS
```

```
e4a693b44e226f2b9024b5427e19af895bedeceb02af483580bb1b15ccb9d826  report.txt
71b30de0ef10362245837072ad5b724c2961dfc0a7a97e3f8706065b57a24aa9  inventory.txt
```

> **What just happened:** `sha256sum` prints the hash and the file name for each file. `tee` shows the output and also writes it to `SHA256SUMS`. Because you used the exact same text, your hashes should match these exactly. That is the point of a hash: same input, same output, on any computer.

3. Verify the files, then tamper with one and verify again:

```
sha256sum -c SHA256SUMS
echo "Quarterly report: revenue up 40%" > report.txt
sha256sum -c SHA256SUMS
```

```
report.txt: OK
inventory.txt: OK
report.txt: FAILED
inventory.txt: OK
sha256sum: WARNING: 1 computed checksum did NOT match
```

> **What just happened:** `-c` (check) reads the checksum file, recomputes each hash, and compares. A single added character changed the hash, so the check failed. Linux distributions publish `SHA256SUMS` files next to their ISO images for exactly this reason.

4. Now the weakness: an attacker who can change `report.txt` can also regenerate `SHA256SUMS`. An HMAC fixes this, because it needs a secret key. Put the original text back and compute an HMAC:

```
echo "Quarterly report: revenue up 4%" > report.txt
openssl dgst -sha256 -hmac "lab-shared-key" report.txt
```

```
HMAC-SHA2-256(report.txt)= abfca3db36d0e38b4e4127178cf2623c5e6b7e5edbf6a6121ae7f481c34b49e2
```

What just happened: `openssl dgst` computes digests (hashes). `-sha256` picks the algorithm and `-hmac "lab-shared-key"` mixes in the secret key. Without the key, nobody can produce this value for a modified file. Run it again with a different key and the result is completely different. HMAC is used inside TLS, SSH (the "MAC" algorithms like `hmac-sha2-256`), JWT tokens, and API request signing.

5. Compare with the weak hashes, just to see them:

```
md5sum report.txt
sha1sum report.txt
```

What just happened: MD5 (128 bits) and SHA-1 (160 bits) still run, but collisions have been demonstrated for both (SHA-1 publicly in 2017). Use them only for non-security jobs like spotting accidental corruption. For security, use SHA-256 or stronger (SHA-384, SHA-512, SHA-3).

Checkpoint: `sha256sum -c SHA256SUMS` shows `OK` for both files again (you restored the original text), and `openssl dgst -sha256 -hmac "lab-shared-key" report.txt` prints the same HMAC as above.

Task 2: Encrypt and sign files with GPG

You will use both VMs: ubuntu-lab encrypts a file so that only rocky-lab can read it, and signs it so rocky-lab can prove who sent it.

1. On ubuntu-lab, install GnuPG and tell it which terminal to use for passphrase prompts (needed over SSH):

```
sudo apt install -y gnupg
export GPG_TTY=$(tty)
cd ~/labs/3.5
echo "Door code for the server room: 4417" > notes.txt
```

What just happened: GnuPG asks for passphrases through a helper program called *pinentry*. On a text-only server it draws a box in your terminal, and `GPG_TTY` tells it which terminal that is. Without it you may see "Inappropriate ioctl for device". Run the `export` line again in every new terminal you use for GPG.

2. **Symmetric** encryption with a passphrase:

```
gpg -c --cipher-algo AES256 notes.txt
ls notes.txt*
file notes.txt.gpg
```

Type a passphrase twice (for example `Lab-Gpg-Sym-2026`).

```
notes.txt  notes.txt.gpg
notes.txt.gpg: PGP symmetric key encrypted data - AES with 256-bit key salted & iterated -
SHA512 .
```

What just happened: `-c` means "symmetric" (conventional) encryption with a passphrase only, no keys. `--cipher-algo AES256` chooses AES with a 256-bit key. GPG wrote `notes.txt.gpg`; the original is not deleted. `file` identifies the result as encrypted data. Anyone with the passphrase can decrypt it, so this is good for backups or sending a file to someone you can give a passphrase to by another channel.

3. Decrypt it. First clear the passphrase cache so GPG really asks:

```
gpgconf --kill gpg-agent
gpg -o notes-decrypted.txt -d notes.txt.gpg
cat notes-decrypted.txt
```

What just happened: `gpg-agent` remembers passphrases for a few minutes; `gpgconf --kill gpg-agent` stops it, emptying the cache. `-d` decrypts and `-o` names the output file. You see the original text.

4. Create a key pair on ubuntu-lab. Use any passphrase you will remember, for example `Ubuntu-Gpg-Key-2026`:

```
gpg --quick-generate-key "Student Ubuntu <student@ubuntu-lab.example>" default default 1y
gpg --list-keys
```

```
pub   ed25519 2026-10-10 [SC] [expires: 2027-10-10]
      3F1A9C0E7B2D4E5F6A7B8C9D0E1F2A3B4C5D6E7F
uid           [ultimate] Student Ubuntu <student@ubuntu-lab.example>
sub   cv25519 2026-10-10 [E] [expires: 2027-10-10]
```

What just happened: `--quick-generate-key` takes a user ID (name and email), the algorithm (`default`), the usage (`default`), and an expiry (`1y`). GnuPG made a primary key for signing (`[S]`) and certifying other keys (`[C]`), plus a subkey for encryption (`[E]`). The long hex string is the **fingerprint**, the unique ID of the key. `[ultimate]` means you trust it completely, because it is yours. Some builds use RSA 3072 instead of Ed25519; either is fine.

5. On **rocky-lab**, do the same and export the public key:

```
# on rocky-lab
sudo dnf install -y gnupg2
export GPG_TTY=$(tty)
mkdir -p ~/labs/3.5 && cd ~/labs/3.5
gpg --quick-generate-key "Student Rocky <student@rocky-lab.example>" default default 1y
gpg --armor --export student@rocky-lab.example > rocky-pub.asc
head -1 rocky-pub.asc
scp rocky-pub.asc student@ubuntu-lab:labs/3.5/
```

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

What just happened: `--export` writes the public key; `--armor` makes it plain text (ASCII "armor") instead of binary, so it can be emailed or pasted. Only the **public** key left the machine. `scp` copied it to ubuntu-lab over SSH.

6. On ubuntu-lab, import Rocky's public key, export your own, and send it to rocky-lab:

```
gpg --import rocky-pub.asc
gpg --fingerprint student@rocky-lab.example
gpg --armor --export student@ubuntu-lab.example > ubuntu-pub.asc
scp ubuntu-pub.asc student@rocky-lab:labs/3.5/
```

What just happened: `--import` adds the key to your keyring. Before you trust an imported key, compare its fingerprint with the owner by a separate channel (in person, by phone). Here, compare it with `gpg --fingerprint` on rocky-lab. This check is what stops an attacker from slipping you a fake key.

7. Mark the key as verified by signing it locally, then encrypt a file **to** rocky-lab and sign a copy:

```
gpg --quick-lsign-key "$(gpg --with-colons --fingerprint student@rocky-lab.example \
  | awk -F: '/^fpr/{print $10; exit}')"
echo "Migration starts Friday at 22:00" > plans.txt
gpg --armor -o plans.enc.asc -e -r student@rocky-lab.example plans.txt
gpg --armor -o plans.txt.sig --detach-sign plans.txt
scp plans.enc.asc plans.txt.sig student@rocky-lab:labs/3.5/
```

What just happened:

- `--quick-lsign-key FINGERPRINT` signs Rocky's key with your key as "locally verified". The `awk` part pulls the fingerprint out of GPG's machine-readable (`--with-colons`) output. Without this, GPG warns "There is no assurance this key belongs to the named user".
- `-e -r student@rocky-lab.example` **encrypts** to that recipient's public key. Only Rocky's private key can decrypt it. Even you cannot decrypt `plans.enc.asc` now.
- `--detach-sign` creates a **signature** in a separate file, made with your private key (GPG asks for your key passphrase).

8. On rocky-lab, decrypt, import Ubuntu's key, and verify the signature:

```
# on rocky-lab
cd ~/labs/3.5
gpg -o plans.txt -d plans.enc.asc
cat plans.txt
gpg --import ubuntu-pub.asc
gpg --verify plans.txt.sig plans.txt
```

```
gpg: Signature made Sat 10 Oct 2026 03:12:44 PM UTC
gpg:                using EDDSA key 3F1A9C0E7B2D4E5F6A7B8C9D0E1F2A3B4C5D6E7F
gpg: Good signature from "Student Ubuntu <student@ubuntu-lab.example>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
```

What just happened: Rocky's private key decrypted the file (you typed Rocky's key passphrase). `--verify` checked the signature with Ubuntu's public key: "Good signature" means the file is exactly what Ubuntu signed. The warning only means rocky-lab has not yet verified the fingerprint of Ubuntu's key, like you did in step 7.

9. Tamper with the file and verify again:

```
# on rocky-lab
echo "Migration starts Saturday" > plans.txt
gpg --verify plans.txt.sig plans.txt
```

```
gpg: BAD signature from "Student Ubuntu <student@ubuntu-lab.example>" [unknown]
```

Checkpoint: rocky-lab decrypted `plans.enc.asc`, `gpg --verify` said "Good signature" on the original file, and "BAD signature" after you changed it.

Task 3: Encrypt a disk with LUKS2

CAUTION

This erases an extra disk. Take a snapshot of ubuntu-lab now. Run `lsblk` and make sure `/dev/sdb` is an empty 5 GB disk with no partitions or mount points. If your disk has a different name (`vdb`, `nvme0n2`), use that name in every command.

1. Confirm the disk and install the tools:

```
lsblk
sudo apt install -y cryptsetup
```

2. Format the disk as LUKS2:

```
sudo cryptsetup luksFormat --type luks2 /dev/sdb
```

```
WARNING!
========
This will overwrite data on /dev/sdb irrevocably.

Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for /dev/sdb:
Verify passphrase:
```

Type `YES` in capitals, then a passphrase such as `Luks-Lab-Pass-1`.

What just happened: `luksFormat` wrote a LUKS header to the start of the disk, generated a random master key, and locked it in key slot 0 with your passphrase. `--type luks2` asks for version 2 explicitly (it is already the default on Ubuntu 24.04 and Rocky 9). It took a couple of seconds because Argon2id deliberately works hard on each passphrase.

3. Look at the header:

```
sudo cryptsetup luksDump /dev/sdb
```

```
LUKS header information
Version:        2
...
Data segments:
  0: crypt
    offset: 16777216 [bytes]
    length: (whole device)
    cipher: aes-xts-plain64
    sector: 512 [bytes]

Keyslots:
  0: luks2
    Key:        512 bits
    Priority:   normal
    Cipher:     aes-xts-plain64
    Cipher key: 512 bits
    PBKDF:      argon2id
    Time cost:  4
    Memory:     1048576
    Threads:    2
...
```

What just happened: `Version: 2` confirms LUKS2. The data is encrypted with `aes-xts-plain64` (AES in XTS mode, the standard mode for disk encryption) with a 512-bit key (XTS splits it into two 256-bit keys). Key slot 0 uses `PBKDF: argon2id` with `Memory: 1048576` KiB, meaning every passphrase guess needs about 1 GiB of RAM. That is what makes brute-force attacks so slow and expensive. Your time and memory numbers may differ; `cryptsetup` benchmarks your machine.

4. Open (unlock) the disk, create a filesystem, and mount it:

```
sudo cryptsetup open /dev/sdb labcrypt
ls -l /dev/mapper/labcrypt
sudo mkfs.ext4 -q /dev/mapper/labcrypt
sudo mkdir -p /mnt/secure
sudo mount /dev/mapper/labcrypt /mnt/secure
echo "TOPSECRET payroll data" | sudo tee /mnt/secure/payroll.txt
lsblk /dev/sdb
```

```
NAME         MAJ:MIN RM SIZE RO TYPE  MOUNTPOINTS
sdb            8:16   0   5G  0 disk
└─labcrypt   252:0    0   5G  0 crypt /mnt/secure
```

What just happened: `cryptsetup open` asked for the passphrase and created a *mapped* device, `/dev/mapper/labcrypt`. Everything written to it is encrypted before it reaches `/dev/sdb`, and decrypted when read back. You put a normal ext4 filesystem **on top of** the encrypted layer. `lsblk` shows the `crypt` layer between the disk and the mount point.

5. Prove the raw disk holds only ciphertext:

```
sync
sudo grep -a -c TOPSECRET /dev/sdb
sudo blkid /dev/sdb
```

```
0
/dev/sdb: UUID="5b1e2c3d-..." TYPE="crypto_LUKS"
```

What just happened: `sync` flushes data to disk. `grep -a` treats the binary disk as text and `-c` counts matching lines. The word appears 0 times on the raw disk, because only encrypted bytes are stored there. If someone steals the disk or a backup image of it, they get nothing without a passphrase. `blkid` shows the type `crypto_LUKS` and the LUKS UUID.

6. Add a second passphrase (for example, a recovery passphrase kept in a safe) and test it:

```
sudo cryptsetup luksAddKey /dev/sdb
sudo cryptsetup luksDump /dev/sdb | grep -E '^ +[0-9]+: luks2'
sudo cryptsetup open --test-passphrase /dev/sdb && echo "passphrase OK"
```

```
  0: luks2
  1: luks2
passphrase OK
```

What just happened: `luksAddKey` first asks for an **existing** passphrase (to unlock the master key), then for the new one (try `Luks-Recovery-2`), and stores the master key in slot 1. For the test, type the new passphrase: `--test-passphrase` checks it without creating a mapping. To remove a slot later: `sudo cryptsetup luksKillSlot /dev/sdb 1`. Back up the header with `cryptsetup luksHeaderBackup`; if the header is damaged, all slots are lost.

7. Lock the disk again:

```
sudo umount /mnt/secure
sudo cryptsetup close labcrypt
ls /dev/mapper/
```

What just happened: Unmount first, then `close` removes the mapping and wipes the master key from memory. Now the disk is just ciphertext until someone types a passphrase.

TIP

To unlock a LUKS disk automatically at boot you add a line to `/etc/crypttab` and a normal line to `/etc/fstab`. In `/etc/crypttab` the fields are: mapped name, device, key file (`none` = ask for the passphrase at boot), options:

```
labcrypt  UUID=5b1e2c3d-...  none  luks
```

and in `/etc/fstab`: `/dev/mapper/labcrypt  /mnt/secure  ext4  defaults,nofail  0  2`. You do not need to add these for this lab. Ubuntu's installer uses the same mechanism (with LVM on top) for full-disk encryption.

Checkpoint: `sudo cryptsetup luksDump /dev/sdb` shows `Version: 2`, `PBKDF: argon2id`, and two key slots, and `/dev/mapper/labcrypt` no longer exists.

Task 4: Build a small CA and see why self-signed certificates fail

You will first serve a **self-signed** certificate and watch clients reject it. Then you will create your own CA, sign a server certificate with it, and add the CA to the trust store.

1. Make a folder for keys and certificates, and create a self-signed certificate:

```
mkdir -p ~/labs/3.5/pki && cd ~/labs/3.5/pki
openssl req -x509 -newkey rsa:2048 -nodes -keyout selfsigned.key \
  -out selfsigned.crt -days 30 -subj "/CN=localhost"
openssl x509 -in selfsigned.crt -noout -subject -issuer
```

```
subject=CN = localhost
issuer=CN = localhost
```

> **What just happened:** `openssl req` normally creates a certificate *request*; `-x509` makes it create a finished certificate instead. `-newkey rsa:2048` generates a new RSA key, `-nodes` ("no DES") leaves the private key unencrypted so a server can read it, and `-subj` sets the name without questions. The subject and issuer are the same: the certificate vouches for itself. That is what self-signed means.

2. Serve it with OpenSSL's test server. Open a **second terminal**, and run:

```
cd ~/labs/3.5/pki
openssl s_server -accept 8443 -cert selfsigned.crt -key selfsigned.key -www
```

> **What just happened:** `openssl s_server` is a tiny TLS server for testing. `-accept 8443` listens on port 8443, and `-www` answers web requests with a status page. Leave it running.

3. In your first terminal, connect as a client:

```
curl https://localhost:8443/
openssl s_client -connect localhost:8443 </dev/null 2>/dev/null | grep -E 'Verify return'
```

```
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html
...
Verify return code: 18 (self-signed certificate)
```

> **What just happened:** `curl` refused to connect because nothing it trusts signed this certificate. `openssl s_client` is the client-side test tool; `-connect host:port` opens a TLS connection, and `</dev/null` closes it right away. Error 18 means self-signed. You *could* use `curl -k` (ignore errors), but then curl would also accept an attacker's certificate. This is why self-signed leaf certificates are dangerous: they train people to bypass the warning.

Stop the server with `Ctrl+C` in the second terminal.

4. Create your own CA: a private key and a self-signed **root** certificate that is allowed to sign other certificates:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out ca.key
openssl req -x509 -new -key ca.key -sha256 -days 3650 \
  -subj "/O=Lab/CN=Lab Root CA" -out ca.crt
openssl x509 -in ca.crt -noout -text | grep -A1 'Basic Constraints'
```

```
            X509v3 Basic Constraints: critical
                CA:TRUE
```

What just happened: `genpkey` generated a 4096-bit RSA private key for the CA. `req -x509` made a 10-year (`-days 3650`) root certificate signed with SHA-256. `CA:TRUE` means this certificate may sign other certificates. A root CA is always self-signed; that is fine, because it is not used directly by a server. It is trusted only because you choose to install it, the same way Ubuntu installs the public roots for you. Protect `ca.key`: anyone who has it can create certificates your machines will trust.

5. Create a server key and a certificate signing request (CSR), then sign it with the CA:

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out server.key
openssl req -new -key server.key -subj "/CN=ubuntu-lab" -out server.csr
cat <<'EOF' > server.ext
basicConstraints = CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = DNS:ubuntu-lab, DNS:localhost, IP:127.0.0.1
EOF
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -days 397 -sha256 -extfile server.ext -out server.crt
openssl verify -CAfile ca.crt server.crt
```

```
Certificate request self-signature ok
subject=CN = ubuntu-lab
server.crt: OK
```

What just happened:

- The server got its own key (an elliptic-curve P-256 key, smaller and faster than RSA).
- The **CSR** contains the server's public key and name. In real life you send the CSR (never the key) to a CA.
- `server.ext` lists *extensions*. The most important is `subjectAltName` (SAN): modern clients check the name you connect to against the SAN list, not the CN. `extendedKeyUsage = serverAuth` limits the certificate to TLS servers. (The heredoc `<<'EOF'` writes the lines up to `EOF` into the file.)
- `openssl x509 -req` signed the CSR with the CA key. `-CAcreateserial` creates `ca.srl` to give each certificate a unique serial number. 397 days is the maximum lifetime public CAs may issue today.
- `openssl verify` confirms the chain: `server.crt` was signed by `ca.crt`.

6. Serve the CA-signed certificate (second terminal) and test from the first terminal:

```
openssl s_server -accept 8443 -cert server.crt -key server.key -www
```

```
curl -s https://localhost:8443/ | head -3
curl -s --cacert ca.crt https://localhost:8443/ | head -3
```

```
(first command prints nothing; curl exit code 60)
<HTML><BODY BGCOLOR="#ffffff">
<pre>
```

What just happened: The first `curl` still fails (run it without `-s` to see "unable to get local issuer certificate"), because your system does not know "Lab Root CA" yet. With `--cacert ca.crt` you told curl to trust that CA for this one command, and the connection succeeded with no warnings and no `-k`.

7. Install the CA into Ubuntu's trust store so every program trusts it:

```
sudo cp ca.crt /usr/local/share/ca-certificates/lab-root-ca.crt
sudo update-ca-certificates
ls -l /etc/ssl/certs/lab-root-ca.pem
curl -s https://localhost:8443/ | head -2
```

```
Updating certificates in /etc/ssl/certs...
1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
lrwxrwxrwx 1 root root 46 Oct 10 15:40 /etc/ssl/certs/lab-root-ca.pem ->
/usr/local/share/ca-certificates/lab-root-ca.crt
<HTML><BODY BGCOLOR="#ffffff">
<pre>
```

What just happened: Local CA certificates go in `/usr/local/share/ca-certificates/` and **must** end in `.crt`. `update-ca-certificates` links them into `/etc/ssl/certs/` and rebuilds the combined bundle `/etc/ssl/certs/ca-certificates.crt`, which curl, apt, Python, and most tools read. Now plain `curl` trusts your server. This is how companies trust their internal CA. Stop `s_server` with `Ctrl+C`.

8. Explore the public trusted roots that came with Ubuntu:

```
ls /usr/share/ca-certificates/mozilla/ | wc -l
ls /usr/share/ca-certificates/mozilla/ | grep -E 'ISRG|DigiCert_Global|Sectigo' | head
openssl x509 -in /usr/share/ca-certificates/mozilla/ISRG_Root_X1.crt -noout \
  -subject -enddate
```

```
146
DigiCert_Global_Root_CA.crt
DigiCert_Global_Root_G2.crt
ISRG_Root_X1.crt
ISRG_Root_X2.crt
Sectigo_Public_Server_Authentication_Root_E46.crt
subject=C = US, O = Internet Security Research Group, CN = ISRG Root X1
notAfter=Jun  4 11:04:38 2035 GMT
```

What just happened: The `ca-certificates` package ships about 140 to 150 root certificates selected by the Mozilla CA program. ISRG Root X1 is the root of **Let's Encrypt**, the no-cost CA. DigiCert and Sectigo are **commercial** CAs. Both kinds sit side by side in the same store and are trusted equally. On Rocky, the trust store lives in `/etc/pki/ca-trust/`; you add a CA to `/etc/pki/ca-trust/source/anchors/` and run `sudo update-ca-trust`.

Checkpoint: `openssl verify server.crt` (with no `-CAfile`) now prints `server.crt: OK`, because the system trust store contains your CA.

Task 5: Serve HTTPS with nginx and control TLS versions

1. Install nginx and nmap, and copy the server certificate and key into place:

```
sudo apt install -y nginx nmap
sudo mkdir -p /etc/nginx/lab-tls
sudo cp ~/labs/3.5/pki/server.crt ~/labs/3.5/pki/server.key /etc/nginx/lab-tls/
sudo chmod 600 /etc/nginx/lab-tls/server.key
grep -n ssl_protocols /etc/nginx/nginx.conf
```

```
33: ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3; # Dropping SSLv3, ref: POODLE
```

What just happened: The private key is readable only by root (`chmod 600`); nginx reads it as root at start-up before dropping privileges. The global `nginx.conf` may still list `TLSv1` and `TLSv1.1`, which are old and weak. You will override that in your site.

2. Create the HTTPS site in `/etc/nginx/sites-available/lab-tls`:

```
cat <<'EOF' | sudo tee /etc/nginx/sites-available/lab-tls
server {
    listen 443 ssl;
    server_name ubuntu-lab localhost;

    ssl_certificate     /etc/nginx/lab-tls/server.crt;
    ssl_certificate_key /etc/nginx/lab-tls/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         ECDHE+AESGCM:ECDHE+CHACHA20;

    root  /var/www/html;
    index index.html index.nginx-debian.html;
}
EOF
sudo ln -s /etc/nginx/sites-available/lab-tls /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

What just happened: `listen 443 ssl` turns on TLS on the HTTPS port. `ssl_certificate` and `ssl_certificate_key` point to the files. `ssl_protocols TLSv1.2 TLSv1.3` allows only the two modern versions, overriding the global setting. `ssl_ciphers` limits TLS 1.2 to cipher suites that use ECDHE key exchange with AES-GCM or ChaCha20 (nginx's default list also allows older CBC-mode suites; TLS 1.3 suites are always modern and are not affected by this line). `root` and `index` serve Ubuntu's default welcome page, `index.nginx-debian.html`. On Ubuntu, a site is enabled by linking it into `sites-enabled`. `nginx -t` tests the config; reload applies it only if the test passed.

3. Test with curl and see the negotiated protocol:

```
curl -sv https://ubuntu-lab/ -o /dev/null 2>&1 | grep -E 'SSL connection|issuer|verify'
curl -s --tls-max 1.2 -v https://ubuntu-lab/ -o /dev/null 2>&1 | grep 'SSL connection'
```

```
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519 / id-ecPublicKey
*  issuer: O=Lab; CN=Lab Root CA
*  SSL certificate verify ok.
* SSL connection using TLSv1.2 / ECDHE-ECDSA-AES256-GCM-SHA384 / X25519 / id-ecPublicKey
```

What just happened: `-v` (verbose) prints the TLS handshake details to standard error; `2>&1` sends them into `grep`. By default client and server agreed on TLS 1.3. `--tls-max 1.2` forced TLS 1.2, which also works. The certificate verified against your trusted Lab Root CA.

4. Test each TLS version with `openssl s_client`:

```
openssl s_client -connect ubuntu-lab:443 -tls1_3 </dev/null 2>/dev/null | grep '^New,'
openssl s_client -connect ubuntu-lab:443 -tls1_2 </dev/null 2>/dev/null | grep '^New,'
openssl s_client -connect ubuntu-lab:443 -tls1_1 </dev/null
```

```
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
New, TLSv1.2, Cipher is ECDHE-ECDSA-AES256-GCM-SHA384
40C7E2A5F27F0000:error:0A0000BF:SSL routines:tls_setup_handshake:no protocols available:...
```

What just happened: `-tls1_3` and `-tls1_2` force one version; both succeed. The TLS 1.1 attempt failed *in the client* with "no protocols available": Ubuntu builds OpenSSL so that its default security level 2 refuses TLS 1.0 and 1.1. Now force the client to allow them, so you can see that the **server** also refuses:

```
openssl s_client -connect ubuntu-lab:443 -tls1_1 -cipher 'DEFAULT:@SECLEVEL=0'
</dev/null
```

This time the error mentions `tlsv1 alert protocol version` (SSL alert number 70): the server said "I do not speak that version". Defense in depth: the library defaults and your `ssl_protocols` line both block old TLS.

5. Scan the server like an auditor would:

```
nmap --script ssl-enum-ciphers -p 443 ubuntu-lab
```

```
PORT    STATE SERVICE
443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|     compressors:
|       NULL
|     cipher preference: server
|   TLSv1.3:
|     ciphers:
|       TLS_AKE_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_AKE_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|       TLS_AKE_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|     cipher preference: server
|_  least strength: A
```

What just happened: The `ssl-enum-ciphers` script tries every protocol version and cipher and grades each one (A is best, F is worst). Only TLS 1.2 and 1.3 are listed, and, thanks to your `ssl_ciphers` line, every cipher uses ECDHE (forward secrecy: a stolen server key cannot decrypt old recorded traffic) and an AEAD cipher (AES-GCM or ChaCha20-Poly1305). Auditors use this exact scan to find servers still offering TLS 1.0, RC4, or 3DES.

6. Check which TLS library you have (LibreSSL versus OpenSSL):

```
openssl version
ldd "$(command -v curl)" | grep -E 'libssl|libcrypto'
apt-cache search --names-only libressl
```

```
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
    libssl.so.3 => /lib/x86_64-linux-gnu/libssl.so.3 (0x...)
    libcrypto.so.3 => /lib/x86_64-linux-gnu/libcrypto.so.3 (0x...)
```

What just happened: `openssl version` names the library. On a system using LibreSSL it would print something like `LibreSSL 3.9.2`. `ldd` lists the shared libraries a program uses; curl links to OpenSSL 3's `libssl.so.3`. The empty search result shows Ubuntu does not package LibreSSL. Other TLS libraries you may meet: GnuTLS, NSS (Firefox), and BoringSSL (Google).

TIP

Explore/simulate: A free Let's Encrypt certificate needs a public DNS name that points to your server, which a lab VM does not have. Install the tool and look at how renewal is automated:

```
sudo apt install -y certbot python3-certbot-nginx
certbot --version
systemctl list-timers certbot.timer
```

On a real public server with a DNS name you would run `sudo certbot --nginx -d www.example.com`. Certbot proves you control the name through the ACME protocol (an HTTP-01 file on port 80, or a DNS-01 TXT record), installs the certificate into nginx, and `certbot.timer` renews it automatically before the 90 days are up. With a commercial CA, you instead create a key and CSR with `openssl req -new` (as in Task 4), paste the CSR into the CA's website, prove domain or organization ownership, and download the signed certificate plus the intermediate chain. Never send the private key to anyone.

Checkpoint: `curl https://ubuntu-lab/` works with no `-k`, and the nmap scan lists only TLSv1.2 and TLSv1.3.

Task 6: Build a WireGuard tunnel between the VMs

You will create a private network `10.200.0.0/24` inside an encrypted tunnel: ubuntu-lab will be `10.200.0.1` and rocky-lab `10.200.0.2`.

```
ubuntu-lab  wg0 10.200.0.1  ===== encrypted UDP 51820 =====  wg0 10.200.0.2  rocky-lab
  192.168.100.10                 (over the lab network)          192.168.100.20
```

1. Install the tools and generate a key pair on **each** VM:

```
sudo apt install -y wireguard-tools
sudo mkdir -p /etc/wireguard
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/wg.key | wg pubkey >
/etc/wireguard/wg.pub'
sudo cat /etc/wireguard/wg.pub
```

```
# on rocky-lab
sudo dnf install -y wireguard-tools
sudo modprobe wireguard && lsmod | grep wireguard
sudo mkdir -p /etc/wireguard
sudo sh -c 'umask 077; wg genkey | tee /etc/wireguard/wg.key | wg pubkey >
/etc/wireguard/wg.pub'
sudo cat /etc/wireguard/wg.pub
```

What just happened: `wg genkey` creates a random private key; `wg pubkey` derives the public key from it. `umask 077` makes the new files readable only by root. `sh -c '...'` runs the whole pipeline as root, so the redirection `>` also happens as root. On Rocky, `modprobe` loads the WireGuard kernel module, which is part of the Rocky 9 kernel. Write down both **public** keys; each side needs the other's.

2. On ubuntu-lab, create `/etc/wireguard/wg0.conf`. Show your private key with `sudo cat /etc/wireguard/wg.key`, then open the file with `sudo nano /etc/wireguard/wg0.conf` and enter:

```
[Interface]
Address = 10.200.0.1/24
ListenPort = 51820
PrivateKey = <ubuntu-lab private key from /etc/wireguard/wg.key>

[Peer]
PublicKey = <rocky-lab public key>
Endpoint = rocky-lab:51820
AllowedIPs = 10.200.0.2/32
```

> **What just happened:** `[Interface]` describes this side: its tunnel address, its UDP port, and its private key. `[Peer]` describes the other side: its public key, where to reach it (`Endpoint`), and `AllowedIPs`, which means "packets for these addresses go to this peer, and packets from this peer may only use these source addresses". This is WireGuard's "cryptokey routing".

3. On rocky-lab, create the mirror image in `/etc/wireguard/wg0.conf` (`sudo vi` or `sudo nano` after `sudo dnf install -y nano`):

```
[Interface]
Address = 10.200.0.2/24
ListenPort = 51820
PrivateKey = <rocky-lab private key from /etc/wireguard/wg.key>

[Peer]
PublicKey = <ubuntu-lab public key>
Endpoint = ubuntu-lab:51820
AllowedIPs = 10.200.0.1/32
```

4. Open the WireGuard port in rocky-lab's firewall (runtime only, so it disappears at reboot), and bring up the tunnel on both VMs:

```
# on rocky-lab
sudo chmod 600 /etc/wireguard/wg0.conf
sudo firewall-cmd --add-port=51820/udp
sudo wg-quick up wg0
```

```
sudo chmod 600 /etc/wireguard/wg0.conf
sudo wg-quick up wg0
```

```
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.200.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
```

What just happened: `firewall-cmd --add-port=51820/udp` allows WireGuard packets into rocky-lab (Ubuntu's `ufw` is inactive by default, so no rule is needed there). `wg-quick up wg0` reads `wg0.conf` and prints each step: create a `wireguard` interface, load the keys and peer, add the address, and bring the link up. The MTU is 1420, smaller than 1500, to leave room for the encryption headers.

5. Test the tunnel and look at its status:

```
ping -c 3 10.200.0.2
sudo wg show
```

```
64 bytes from 10.200.0.2: icmp_seq=1 ttl=64 time=1.12 ms
...
interface: wg0
  public key: kX2u...=
  private key: (hidden)
  listening port: 51820

peer: 9fQe...=
  endpoint: 192.168.100.20:51820
  allowed ips: 10.200.0.2/32
  latest handshake: 4 seconds ago
  transfer: 1.02 KiB received, 1.14 KiB sent
```

What just happened: The ping went through the tunnel. `wg show` confirms a recent **handshake** (the two sides proved their keys and agreed on session keys) and counts bytes transferred. If "latest handshake" is missing, check the public keys, the endpoint name, and the Rocky firewall.

6. See the encryption on the wire. In a second terminal on ubuntu-lab start a capture, then ping again from the first terminal:

```
sudo tcpdump -ni any -c 4 udp port 51820
```

```
ping -c 2 10.200.0.2
```

```
15:58:01.123 enp0s3 Out IP 192.168.100.10.51820 > 192.168.100.20.51820: UDP, length 128
15:58:01.124 enp0s3 In  IP 192.168.100.20.51820 > 192.168.100.10.51820: UDP, length 128
```

What just happened: `tcpdump -ni any` captures on all interfaces without name lookups; `-c 4` stops after 4 packets. On the real network, an observer sees only encrypted UDP packets between the two VM addresses; the ICMP ping inside is invisible. Run `sudo tcpdump -ni wg0 -c 4 icmp` instead and you see the plain pings, because `wg0` is the inside of the tunnel.

Checkpoint: `ping -c 3 10.200.0.2` succeeds from ubuntu-lab, and `sudo wg show` on rocky-lab shows a recent handshake with ubuntu-lab.

Task 7: Remove weak algorithms

1. On ubuntu-lab, list the algorithms SSH supports and the ones the server actually offers:

```
ssh -Q cipher
ssh -Q mac
sudo sshd -T | grep -E '^(ciphers|macs|kexalgorithms) '
```

```
ciphers chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-
gcm@openssh.com,aes256-gcm@openssh.com
macs umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-
sha2-512-etm@openssh.com,hmac-sha1-etm@openssh.com,umac-64@openssh.com,umac-
128@openssh.com,hmac-sha2-256,hmac-sha2-512,hmac-sha1
kexalgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,...
```

What just happened: `ssh -Q` lists everything the SSH program *can* use, including old algorithms like `3des-cbc` kept for compatibility. `sshd -T` prints the server's *effective* settings. The server still offers MACs based on SHA-1 (`hmac-sha1`) and 64-bit tags (`umac-64`). They are not broken in practice inside SSH, but security baselines (CIS, STIG) require removing them.

2. First prove the server accepts `hmac-sha1`. `BatchMode=yes` stops at the password prompt, so you only test the negotiation:

```
ssh -o Ciphers=aes128-ctr -o MACs=hmac-sha1 -o BatchMode=yes \
  student@localhost true
```

```
student@localhost: Permission denied (publickey,password).
```

What just happened: `-o Ciphers=aes128-ctr` picks a classic cipher, because AEAD ciphers such as `chacha20-poly1305` (the default) and AES-GCM have authentication built in and SSH then ignores the MAC setting entirely. `-o MACs=hmac-sha1` offers only that MAC. "Permission denied" sounds bad, but it means the algorithm negotiation **succeeded** and SSH reached the login step (which BatchMode refuses to do interactively). If you set up SSH keys in Lab 3.3, the command simply logs in, runs `true`, and prints nothing; that also means it was accepted.

3. Remove the weak MACs with a drop-in file and test again:

```
cat <<'EOF' | sudo tee /etc/ssh/sshd_config.d/40-crypto.conf
MACs -hmac-sha1,hmac-sha1-etm@openssh.com,umac-64@openssh.com,umac-64-etm@openssh.com
EOF
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep '^macs '
ssh -o Ciphers=aes128-ctr -o MACs=hmac-sha1 -o BatchMode=yes \
  student@localhost true
```

```
macs umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-
etm@openssh.com,umac-128@openssh.com,hmac-sha2-256,hmac-sha2-512
Unable to negotiate with 127.0.0.1 port 22: no matching MAC found. Their offer: umac-128-
etm@openssh.com,hmac-sha2-256-etm@openssh.com,...
```

What just happened: A list that starts with `-` removes those entries from the built-in default list (a `+` would add, `^` would move to the front). Removing is safer than writing a full list, because you keep future good defaults. The drop-in is read before the main file and `sshd` uses the first value it finds, so it wins. Now a client that insists on `hmac-sha1` cannot connect. Old SSH ciphers like `3des-cbc` and the SHA-1 `ssh-rsa` signature are already disabled by default in OpenSSH 9.x.

4. On rocky-lab, weak algorithms are managed system-wide by **crypto policies**. See the current policy and what it means for SSH:

```
# on rocky-lab
update-crypto-policies --show
grep -E '^(Ciphers|MACs)' /etc/crypto-policies/back-ends/opensshserver.config
```

```
DEFAULT
Ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com,aes256-ctr,aes128-
gcm@openssh.com,aes128-ctr
MACs hmac-sha2-256-etm@openssh.com,hmac-sha1-etm@openssh.com,umac-128-etm@openssh.com,hmac-
sha2-512-etm@openssh.com,hmac-sha2-256,hmac-sha1,umac-128@openssh.com,hmac-sha2-512
```

What just happened: One setting controls OpenSSL, GnuTLS, NSS, OpenSSH, Kerberos, Java, and more. `update-crypto-policies` writes a file for each library in `/etc/crypto-policies/back-ends/`. The policies are: `LEGACY` (allows old algorithms for compatibility), `DEFAULT` (sensible today; SHA-1 signatures and TLS 1.0/1.1 already off), `FUTURE` (stricter: RSA keys at least 3072 bits, no SHA-1 at all, 128-bit AES ciphers removed), and `FIPS` (only FIPS 140 approved algorithms).

5. Switch to `FUTURE`, look at the change, then go back:

```
# on rocky-lab
sudo update-crypto-policies --set FUTURE
grep -E '^(Ciphers|MACs)' /etc/crypto-policies/back-ends/opensshserver.config
sudo systemctl restart sshd
```

```
Setting system policy to FUTURE
Note: System-wide crypto policies are applied on application start-up.
It is recommended to restart the system for the change of policies
to fully take place.
Ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com,aes256-ctr
MACs hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,hmac-sha2-256,hmac-sha2-512
```

What just happened: Under `FUTURE` the AES-128 ciphers and all SHA-1 MACs disappeared. Programs read the policy when they start, so you restart `sshd` (or reboot) to apply it. `FUTURE` can break connections to older systems, which is why `DEFAULT` is the normal choice. You can also add a small exception on top of a policy, for example `DEFAULT:SHA1`.

From ubuntu-lab, confirm SSH to rocky-lab still works (`ssh student@rocky-lab hostname`), then restore the default:

```
# on rocky-lab
sudo update-crypto-policies --set DEFAULT
sudo systemctl restart sshd
update-crypto-policies --show
```

ON THE EXAM

On the exam, "removal of weak algorithms" means: disable SSLv2/SSLv3/TLS 1.0/TLS 1.1, MD5 and SHA-1 for signatures, DES/3DES/RC4 ciphers, and short keys (RSA under 2048 bits). Tools: `ssh -Q`, `sshd -T`, `openssl s_client -tls1_x`, `nmap --script ssl-enum-ciphers`, and `update-crypto-policies` on RHEL-family systems.

Checkpoint: On ubuntu-lab, `sudo sshd -T | grep '^macs '` no longer lists `hmac-sha1`. On rocky-lab, `update-crypto-policies --show` prints `DEFAULT`.

Putting it together: You protected data at rest twice (GPG for single files, LUKS2 for a whole disk) and data in transit twice (TLS for one application, WireGuard for all traffic between two hosts). All of them rely on the same three building blocks: hashes for integrity, symmetric ciphers for speed, and public keys for identity and key exchange.

Challenge

CHALLENGE

1. Make rocky-lab trust your Lab Root CA: copy `ca.crt` to `/etc/pki/ca-trust/source/anchors/`, run `update-ca-trust`, and prove that `curl https://ubuntu-lab/` from rocky-lab works without `-k`. Then remove it again.
2. Add a key file as a third LUKS key slot (`dd if=/dev/urandom of=lab.key bs=64 count=1`, then `cryptsetup luksAddKey /dev/sdb lab.key`) and open the disk with `--key-file`. Explain why key files must be stored somewhere other than the encrypted disk.
3. Change the nginx site to allow only TLS 1.3 and prove it with `curl --tls-max 1.2` and nmap.

Clean up

1. Stop WireGuard on both VMs and remove the configs:

```
sudo wg-quick down wg0
sudo rm -rf /etc/wireguard/wg0.conf /etc/wireguard/wg.key /etc/wireguard/wg.pub
```

```
# on rocky-lab
sudo wg-quick down wg0
sudo rm -rf /etc/wireguard/wg0.conf /etc/wireguard/wg.key /etc/wireguard/wg.pub
sudo firewall-cmd --reload
```

(`firewall-cmd --reload` drops runtime-only rules like the 51820/udp opening.)

2. Remove the nginx site, nginx, certbot, and the lab CA from the trust store:

```
sudo rm -f /etc/nginx/sites-enabled/lab-tls /etc/nginx/sites-available/lab-tls
sudo rm -rf /etc/nginx/lab-tls
sudo apt purge -y nginx nginx-common certbot python3-certbot-nginx
sudo apt autoremove -y
sudo rm -f /usr/local/share/ca-certificates/lab-root-ca.crt
sudo update-ca-certificates --fresh
```

3. Remove the SSH drop-in:

```
sudo rm -f /etc/ssh/sshd_config.d/40-crypto.conf
sudo sshd -t && sudo systemctl reload ssh
```

4. Cryptographically erase and wipe the LUKS disk (make sure it is closed first):

```
sudo cryptsetup luksErase /dev/sdb
sudo wipefs -a /dev/sdb
sudo rmdir /mnt/secure
```

`luksErase` destroys every key slot (type `YES`), so the master key, and with it the data, is gone forever. `wipefs -a` then removes the LUKS signature.

5. Delete the GPG keys and lab files on both VMs. The first command looks up your own key's fingerprint for you:

```
fpr=$(gpg --list-secret-keys --with-colons | awk -F: '/^fpr/{print $10; exit}')
echo "$fpr"
gpg --batch --yes --delete-secret-and-public-key "$fpr"
gpg --batch --yes --delete-key student@rocky-lab.example
rm -rf ~/labs/3.5
```

```
# on rocky-lab
fpr=$(gpg --list-secret-keys --with-colons | awk -F: '/^fpr/{print $10; exit}')
echo "$fpr"
gpg --batch --yes --delete-secret-and-public-key "$fpr"
gpg --batch --yes --delete-key student@ubuntu-lab.example
rm -rf ~/labs/3.5
```

(The first line saves the first fingerprint, which belongs to your primary key, in the shell variable `fpr`; `echo` shows it. Deleting a secret key in batch mode requires the full fingerprint. If you have other GPG keys of your own, check that the printed fingerprint is the lab key before running the delete.)

Exam corner

- **Data at rest**: GPG for files (`gpg -c` symmetric; `gpg -e -r` to a recipient; `--detach-sign` and `--verify`), LUKS2 for block devices (`cryptsetup luksFormat --type luks2`, `open`, `close`, `luksAddKey`, `luksDump`, `luksErase`), and `/etc/crypttab` for unlocking at boot.
- LUKS2 uses **Argon2id** (memory-hard) as its default PBKDF; LUKS1 used PBKDF2. Up to 32 key slots unlock one master key.
- **Data in transit**: TLS (via OpenSSL or LibreSSL), SSH, and VPNs such as WireGuard (UDP 51820, `wg`, `wg-quick`, `[Interface]` / `[Peer]`, `AllowedIPs`).
- Allowed **TLS** versions: 1.2 and 1.3. SSLv2, SSLv3, TLS 1.0 and 1.1 are deprecated. Test with `openssl s_client -connect host:443 -tls1_2` and `nmap --script ssl-enum-ciphers`.
- **LibreSSL** is OpenBSD's 2014 fork of OpenSSL; `openssl version` tells you which you have.
- **SHA-256**: `sha256sum file`, `sha256sum -c SHA256SUMS`. **HMAC** adds a secret key: `openssl dgst -sha256 -hmac KEY file`.
- **Weak algorithms** to remove: MD5, SHA-1, DES/3DES, RC4, short RSA keys. Use `sshd_config` (`Ciphers`, `MACs`, `KexAlgorithms`) and `update-crypto-policies --set DEFAULT|FUTURE|LEGACY|FIPS` on RHEL.
- **Trusted root** stores: Ubuntu `/usr/local/share/ca-certificates/` + `update-ca-certificates` (bundle in `/etc/ssl/certs/`); RHEL `/etc/pki/ca-trust/source/anchors/` + `update-ca-trust`.
- **No-cost** CA: Let's Encrypt via ACME/`certbot`, 90-day automated certificates. **Commercial** CA: paid, OV/EV validation, support and warranties.
- Avoid **self-signed** leaf certificates; they cannot be verified and teach users to ignore warnings. Use a public CA or a private internal CA whose root you distribute.

Review questions

1. Which key derivation function does LUKS2 use by default to protect key slots? A. PBKDF2 B. bcrypt C. Argon2id D. scrypt
2. What is the difference between `sha256sum report.txt` and `openssl dgst -sha256 -hmac secret report.txt`?
3. You copied `corp-ca.crt` into `/usr/local/share/ca-certificates/` on Ubuntu, but curl still does not trust the internal site. What command did you forget?
4. Which nginx directive removes TLS 1.0 and TLS 1.1? A. `ssl_ciphers HIGH` B. `ssl_protocols TLSv1.2 TLSv1.3` C. `ssl_prefer_server_ciphers on` D. `ssl_version 1.2`
5. On Rocky Linux, which command makes the whole system reject SHA-1 and RSA keys shorter than 3072 bits?
6. In a WireGuard config, what does `AllowedIPs = 10.200.0.2/32` in a `[Peer]` section do?

Lab 3.6 Compliance, Auditing, and Detection

Objective: 3.6 Explain the importance of compliance and audit procedures.
Domain: 3.0 Security (18% of the exam)
Time: about 180 minutes (several scans run for a few minutes each)
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes. This lab destroys data on an extra disk, plants fake "attacker" artifacts, and changes login banners.

Hardening a system once is not enough. You must keep checking it: scan for malware, hunt for signs of an intruder, find known vulnerabilities, compare the system with a security standard, detect changed files, and prove that the software you install is genuine. When hardware leaves your hands, you must destroy the data on it. This lab walks through each of these audit and compliance jobs with the real tools, so you can explain on the exam what each one is for and when you would use it.

What you'll practice

- Detection and response: Anti-malware (ClamAV with the EICAR test file), Indicators of compromise (IoC)
- Vulnerability scanning: CVEs, CVSS, Backporting patches, Service misconfigurations; Tools: Port scanners (`nmap`), Protocol analyzer (`tcpdump`, `tshark`)
- Standards and audit: OpenSCAP, CIS Benchmarks
- File integrity: AIDE, rkhunter, Signed package verification, Installed file verification (`debsums`, `dpkg --verify`, `rpm -V`)
- Secure data destruction: `shred`, `badblocks -w`, `dd if=/dev/urandom`, Cryptographic destruction
- Software supply chain (the xz backdoor, verifying downloads, pinning, SBOMs)
- Security banners: `/etc/issue`, `/etc/issue.net`, `/etc/motd`

How it works

Why compliance and audit matter. Many organizations must follow rules: PCI DSS for card payments, HIPAA for health data, government baselines like DISA STIGs. Auditors ask you to *prove* that systems are configured correctly and watched. Even without a law, regular audits catch mistakes and intruders early. Think of it like a building: locks (hardening) are not enough without cameras (detection), inspections (audits), and a way to shred old documents (data destruction).

Detection and response.

- **Anti-malware** software compares files against signatures of known malware. On Linux the free standard is **ClamAV**. Linux servers are often scanned because they store or pass on files (mail servers, file shares) that Windows clients will open.

- **Indicators of compromise (IoCs)** are clues that an attacker has been on a system: unknown listening ports, new user accounts, strange cron jobs, unexpected SUID programs, logins from odd places at odd times, outbound connections to unknown addresses, processes using 100% CPU (crypto miners), and changed system binaries.

Vulnerability scanning.

- A **CVE** (Common Vulnerabilities and Exposures) is a unique ID for one publicly known vulnerability, for example `CVE-2024-6387` (the "regreSSHion" bug in OpenSSH).
- **CVSS** (Common Vulnerability Scoring System) rates severity from 0.0 to 10.0. Bands: None 0.0, Low 0.1 to 3.9, Medium 4.0 to 6.9, High 7.0 to 8.9, Critical 9.0 to 10.0. The score comes from a *vector* such as `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`: attack vector Network, attack complexity High, no privileges or user interaction needed, high impact on confidentiality, integrity, and availability. That vector scores 8.1 (High).
- **Backporting** means the distribution takes only the security fix from a newer upstream version and applies it to the older version it ships. The version number stays "old", but the bug is fixed. This confuses scanners that only compare version numbers, so you must check the distribution's changelog or security notices.
- **Service misconfigurations** are mistakes in settings rather than bugs in code: a database listening on all interfaces with no password, directory listing turned on, debug servers left running, root SSH logins allowed.
- A **port scanner** (`nmap`) finds which ports are open and which software versions answer. A **protocol analyzer** (`tcpdump`, Wireshark, `tshark`) captures and decodes network traffic, so you can see whether passwords travel in cleartext. Full vulnerability scanners such as OpenVAS/Greenbone, Nessus, or Trivy (for containers) combine these ideas with a CVE database.

Standards and audit. The **CIS Benchmarks** from the Center for Internet Security are free, detailed hardening guides for each OS ("Ensure /tmp is a separate partition", "Ensure SSH root login is disabled"). Level 1 is a practical baseline; Level 2 is stricter. **OpenSCAP** is a scanner that reads machine-readable security content (SCAP, the Security Content Automation Protocol) and checks a system against a profile such as CIS Level 1, then produces an HTML report.

File integrity. An attacker who gets root often replaces programs like `ls` or `ps` with versions that hide their files (a *rootkit*). File integrity tools catch this:

```
Day 0 (known good)            Later
+------------------+         +------------------+
| file -> SHA-256  |  compare| file -> SHA-256  |
| /etc/hosts  a1b2 | ------> | /etc/hosts  9f3c |  <-- CHANGED
| /bin/ls     77de |         | /bin/ls     77de |
+------------------+         +------------------+
  baseline database            current system
```

- **AIDE** (Advanced Intrusion Detection Environment) builds a database of file hashes and attributes, then reports differences.
- **rkhunter** (Rootkit Hunter) looks for known rootkit files, suspicious settings, and changed system binaries.
- The **package manager** already knows the correct hash of every file it installed. `debsums` and `dpkg --verify` (Debian/Ubuntu) and `rpm -V` (RHEL/Rocky) compare installed files with those hashes. **Signed packages** prove that packages came from the distribution: `apt` checks a GPG-signed repository index, and `dnf` / `rpm` check a GPG signature on every package.

Secure data destruction. Deleting a file only removes its name; the data stays on the disk. Overwriting tools (`shred`, `dd`, `badblocks -w`) write over the data. On SSDs and copy-on-write filesystems overwriting is not reliable, because the drive may write new data to a different physical place. That is why **cryptographic destruction** (also called crypto-erase) matters: if data was always encrypted, destroying the key makes it unreadable instantly.

Software supply chain. You run code written by thousands of people. An attacker who sneaks malicious code into a library, a build server, or a download site can reach every user downstream. Defenses: install only from trusted, signed sources, verify checksums and signatures of downloads, pin versions, and keep a software bill of materials (SBOM).

Security banners are legal and informational messages: `/etc/issue` is shown before a local console login, `/etc/issue.net` before a network login (when SSH is configured to show it), and `/etc/motd` (message of the day) after a successful login.

Tasks

Task 1: Scan for malware with ClamAV

1. Create the working directory and install ClamAV:

```
mkdir -p ~/labs/3.6 && cd ~/labs/3.6
sudo apt update
sudo apt install -y clamav clamav-freshclam
```

> **What just happened:** `clamav` provides the scanner `clamscan`. `clamav-freshclam` provides `freshclam`, the signature updater, plus a service that runs it automatically. Right after installation the service starts downloading the signature databases (a few hundred MB).

2. Make sure the signature databases have downloaded:

```
systemctl status clamav-freshclam --no-pager | head -5
ls -lh /var/lib/clamav/
```

```
-rw-r--r-- 1 clamav clamav 276K Oct 10 16:02 bytecode.cvd
-rw-r--r-- 1 clamav clamav  60M Oct 10 16:02 daily.cvd
-rw-r--r-- 1 clamav clamav 163M Oct 10 16:03 main.cvd
```

> **What just happened:** The `.cvd` (or `.cld`) files are the signature databases: `main` (the big base set), `daily` (new signatures, updated several times a day), and `bytecode` (detection programs). If the folder is still empty after a few minutes, run the updater by hand. It cannot run while the service holds its lock, so stop the service first: `sudo systemctl stop clamav-freshclam && sudo freshclam && sudo systemctl start clamav-freshclam`.

3. Create the EICAR test file. EICAR is a harmless 68-byte string that every anti-malware product agrees to detect as if it were a virus, so you can test scanners safely:

```
printf '%s' 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' >
eicar.com
echo "just a normal file" > normal.txt
wc -c eicar.com
```

```
68 eicar.com
```

What just happened: `printf '%s' '...'` prints the string exactly, with no newline, and the single quotes stop the shell from interpreting `!`, `$`, and `\`. The file is not malware; it is a test pattern.

4. Scan the folder:

```
clamscan -r -i ~/labs/3.6
```

```
/home/student/labs/3.6/eicar.com: Win.Test.EICAR_HDB-1 FOUND

----------- SCAN SUMMARY -----------
Known viruses: 8712345
Engine version: 1.5.4
Scanned directories: 1
Scanned files: 2
Infected files: 1
Data scanned: 0.00 MB
Time: 18.512 sec (0 m 18 s)
```

What just happened: `-r` scans folders recursively and `-i` prints only infected files. ClamAV loaded millions of signatures (that is why it takes a few seconds and about 1 GB of RAM) and matched the EICAR signature. In a real incident you would quarantine the file rather than delete it, so it can be analyzed.

5. Quarantine infected files automatically:

```
mkdir -p ~/labs/3.6/quarantine
clamscan -r -i --move="$HOME/labs/3.6/quarantine" ~/labs/3.6
ls ~/labs/3.6/quarantine
```

What just happened: `--move=DIR` moves each infected file into the quarantine folder. Other options: `--remove` (delete) and `--log=FILE`. For continuous protection, `clamav-daemon` provides `clamd` (a scanner that stays loaded in memory) and `clamonacc` for on-access scanning. Schedule `clamscan` with cron or a systemd timer for regular scans.

Checkpoint: `eicar.com` is now in `~/labs/3.6/quarantine`, and the scan summary showed `Infected files: 1`.

Task 2: Plant and hunt indicators of compromise

You will play both roles: first the “attacker” plants four harmless IoCs, then the “defender” finds them with a standard checklist.

> **CAUTION**
> Everything planted here is harmless (`/usr/bin/true` does nothing, and the listener only accepts connections). Still, remove every item at the end of the task, as shown.

1. **Attacker:** plant the IoCs.

```
sudo useradd -m -s /bin/bash sysupdate
echo '*/5 * * * * root /usr/bin/true # system updater' | sudo tee /etc/cron.d/sys-updater
sudo cp /usr/bin/true /usr/local/bin/sys-helper
sudo chmod 4755 /usr/local/bin/sys-helper
nohup nc -lk 4444 >/dev/null 2>&1 &
```

> **What just happened:** You created a backdoor-looking user, a cron job that runs as root every 5 minutes, a program with the **SUID** bit (mode `4755` : it runs as its owner, root, no matter who starts it), and a process listening on TCP port 4444 (`nc -l` listen, `-k` keep listening). Real attackers use the same tricks for persistence and remote access. `nohup ... &` keeps the listener running in the background.

2. **Defender:** look for unexpected listening ports.

```
sudo ss -tulpn
```

```
Netid State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp   UNCONN 0      0      127.0.0.54:53       0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=16))
tcp   LISTEN 0      1      0.0.0.0:4444        0.0.0.0:*     users:(("nc",pid=4120,fd=3))
tcp   LISTEN 0      4096   *:22                *:*           users:(("sshd",pid=845,fd=3),
("systemd",pid=1,fd=86))
```

> **What just happened:** `ss` shows sockets: `-t` TCP, `-u` UDP, `-l` listening only, `-p` the owning process, `-n` numbers instead of names. You know why port 22 (SSH) and 53 on 127.0.0.54 (the local DNS resolver) are open; port 22 lists both `sshd` and `systemd` because Ubuntu 24.04 starts SSH through a systemd socket unit (`ssh.socket`). Port 4444 owned by `nc` is not part of any service you installed. That is an IoC. Follow up with `ls -l /proc/4120/exe` to see which program it is (use the PID from your output).

3. Look for new accounts and the log line that created them:

```
awk -F: '$3 >= 1000 && $3 < 65534 {print $1, $3, $7}' /etc/passwd
sudo grep 'new user' /var/log/auth.log | tail -3
```

```
student 1000 /bin/bash
sysupdate 1001 /bin/bash
2026-10-10T16:20:11.402 ubuntu-lab useradd[4101]: new user: name=sysupdate, UID=1001,
GID=1001, home=/home/sysupdate, shell=/bin/bash, from=/dev/pts/0
```

What just happened: `awk -F:` splits `/etc/passwd` on colons; `$3` is the UID, so this lists normal (human) accounts. `sysupdate` is not one of your users. The auth log shows exactly when it was created and from which terminal. Also check for a second UID 0 account: `awk -F: '$3 == 0' /etc/passwd` should print only `root`.

4. Look for persistence in scheduled jobs:

```
ls -la /etc/cron.d/
sudo find /etc/cron* /var/spool/cron -type f -mtime -1
systemctl list-timers --all --no-pager | head
```

What just happened: `find -mtime -1` lists files changed in the last day. `/etc/cron.d/sys-updater` stands out: a new root job with a reassuring name. Attackers also hide in user crontabs (`/var/spool/cron/crontabs/`), systemd timers and services, and shell startup files like `~/.bashrc`.

5. Look for SUID programs that no package installed:

```
for f in $(sudo find / -xdev -perm -4000 -type f 2>/dev/null); do
  dpkg -S "$f" >/dev/null 2>&1 || dpkg -S "${f#/usr}" >/dev/null 2>&1 \
    || echo "NOT FROM A PACKAGE: $f"
done
```

```
NOT FROM A PACKAGE: /usr/local/bin/sys-helper
```

What just happened: `find / -xdev -perm -4000 -type f` lists every regular file with the SUID bit on the root filesystem (`-xdev` stays on one filesystem). For each one, `dpkg -S` asks "which package owns this file?". The second `dpkg -S "${f#/usr}"` retries without the leading `/usr`, because some packages still register files under `/bin`. Legitimate SUID programs (`sudo`, `passwd`, `su`, `mount`) belong to packages. Your planted file does not. On Rocky use `rpm -qf FILE` instead of `dpkg -S`.

6. Check logins and connections:

```
last -n 5
sudo lastb -n 5
sudo ss -tnp state established
ps aux --sort=-%cpu | head -5
```

What just happened: `last` reads `/var/log/wtmp` (successful logins), and `lastb` reads `/var/log/btmp` (failed logins; many failures from one address means password guessing). `ss ... state established` lists active connections; an unknown outbound connection could be malware calling home. `ps aux --sort=-%cpu` shows the busiest processes, which would expose a crypto miner.

7. **Respond:** document what you found, then remove the IoCs.

```
pkill -f 'nc -lk 4444'
sudo rm -f /etc/cron.d/sys-updater /usr/local/bin/sys-helper
sudo userdel -r sysupdate
sudo ss -tlpn | grep 4444 || echo "port 4444 closed"
```

What just happened: `pkill -f` kills processes whose full command line matches. In a real incident you would first preserve evidence (copy logs, take a VM snapshot or memory image), then contain the system (isolate it from the network), and only then clean up. Because an attacker with root can hide things you did not find, the safest recovery is often to rebuild from a known-good image.

Checkpoint: You found all four IoCs (port 4444, user `sysupdate`, `/etc/cron.d/sys-updater`, and the SUID `sys-helper`), and the final command prints `port 4444 closed`.

Task 3: Look up CVEs, CVSS scores, and backported fixes

1. On ubuntu-lab, check a specific CVE with the Ubuntu Pro client (installed by default; no subscription is needed for packages in `main`):

```
pro fix CVE-2024-6387
```

```
CVE-2024-6387: OpenSSH vulnerability
 - https://ubuntu.com/security/CVE-2024-6387

1 affected source package is installed: openssh
(1/1) openssh:
A fix is available in Ubuntu standard updates.
The update is already installed.

CVE-2024-6387 is resolved.
```

What just happened: `pro fix` looked up the CVE in Ubuntu's security database, found which installed package it affects, and checked whether the fixed version is installed. If it were not, `pro fix` would offer to install it (use `--dry-run` to only look). `pro security-status` summarizes the support status of all installed packages.

2. See the version and the backported fix:

```
dpkg -l openssh-server | tail -1
ssh -V
apt changelog openssh-server 2>/dev/null | grep -m3 'CVE-2024-6387'
```

```
ii  openssh-server 1:9.6p1-3ubuntu13.5 amd64        secure shell (SSH) server, for secure
access from remote machines
OpenSSH_9.6p1 Ubuntu-3ubuntu13.5, OpenSSL 3.0.13 30 Jan 2024
    - debian/patches/CVE-2024-6387.patch: don't log in sshsigdie() as
    - CVE-2024-6387
```

What just happened: Upstream OpenSSH fixed regreSSHion in version 9.8p1. Ubuntu still ships **9.6p1**, yet the bug is fixed: Ubuntu **backported** the patch and raised only the Ubuntu part of the version (`-3ubuntu13.x`). Read the version string as: `1:` epoch, `9.6p1` upstream version, `3ubuntu13.5` Debian/Ubuntu revision. A naive scanner that says "OpenSSH older than 9.8 is vulnerable" would raise a false positive here. `apt changelog` downloads the package changelog, where every security fix lists its CVE.

3. Get the CVSS score from the US National Vulnerability Database (NVD) API:

```
curl -s "https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2024-6387" \
  | grep -oE '"(baseScore|baseSeverity|vectorString)":"?[^,"]*' | head -3
```

```
"vectorString":"CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H
"baseScore":8.1
"baseSeverity":"HIGH
```

What just happened: The API returns JSON about the CVE; `grep -o` pulls out three fields. The vector explains the score: reachable over the **N**etwork, but attack **C**omplexity **H**igh (the attacker must win a timing race), no privileges or user interaction needed, and full impact on confidentiality, integrity, and availability. Score 8.1 is **High**. The NVD API is rate-limited; if you get nothing, wait a minute or look the CVE up at `nvd.nist.gov` in a browser.

4. On rocky-lab, list pending security updates and check the same CVE:

```
# on rocky-lab
sudo dnf updateinfo list --security
rpm -q openssh
rpm -q --changelog openssh | grep -m2 'CVE-2024-6387'
```

```
openssh-8.7p1-43.el9.x86_64
- Possible remote code execution due to a race condition (CVE-2024-6387)
```

What just happened: `dnf updateinfo list --security` shows security advisories (named `RLSA-...` on Rocky) for packages that have updates waiting; empty output means you are up to date. Rocky 9 ships OpenSSH **8.7p1** from 2021, yet the changelog shows the 2024 fix was backported. Enterprise distributions do this for the whole 10-year life of a release, so the software stays stable but secure.

Checkpoint: You can state the CVSS score and severity of CVE-2024-6387 (8.1, High) and show from the changelogs that both VMs have the fix even though neither runs OpenSSH 9.8.

Task 4: Find misconfigurations with a port scanner and a protocol analyzer

1. On ubuntu-lab, install nmap and scan rocky-lab:

```
sudo apt install -y nmap
nmap -sV rocky-lab
```

```
Nmap scan report for rocky-lab (192.168.100.20)
Not shown: 998 filtered tcp ports (host-prohibited)
PORT     STATE  SERVICE VERSION
22/tcp   open   ssh     OpenSSH 8.7 (protocol 2.0)
9090/tcp closed zeus-admin
```

What just happened: `nmap` probed the 1000 most common TCP ports. `-sV` connects to open ports and identifies the software and version. "filtered" means the firewall blocked the probes (good; firewalld's `public` zone rejects them with an ICMP "host prohibited" message, which nmap reports as the reason); "closed" means the firewall allowed it but nothing is listening (9090 is the Cockpit port that Rocky's firewall allows by default). Only SSH is open. Scan only systems you own or have written permission to scan.

2. Now create a typical misconfiguration on rocky-lab: a developer starts a quick web server in a folder that contains a backup of a config file with a password, and opens the firewall for it.

```
# on rocky-lab
mkdir -p ~/labs/3.6/share && cd ~/labs/3.6/share
echo "db_password=Winter2026" > config.bak
sudo firewall-cmd --add-port=8080/tcp
python3 -m http.server 8080
```

What just happened: `python3 -m http.server 8080` serves the current folder over plain HTTP, with directory listing, to anyone who can reach port 8080. The firewall rule is runtime-only. Leave the server running in this terminal.

3. On ubuntu-lab, scan again and look at what is exposed:

```
nmap -sV -p 22,8080 rocky-lab
curl -s http://rocky-lab:8080/ | grep href
```

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.7 (protocol 2.0)
8080/tcp open  http    SimpleHTTPServer 0.6 (Python 3.9.18)
<li><a href="config.bak">config.bak</a></li>
```

> **What just happened:** The scan found a new service and identified it as Python's built-in test server, which should never face a network. `curl` showed the directory listing including `config.bak`. This is a service **misconfiguration**: no bug in Python, just an unsafe way of using it. Vulnerability scanners flag exactly this kind of finding.

4. Capture the traffic with a **protocol analyzer**. Open a second terminal on ubuntu-lab and start `tcpdump`:

```
sudo tcpdump -i any -w /tmp/http.pcap host rocky-lab and tcp port 8080
```

In the first terminal, fetch the file using a (fake) user name and password:

```
curl -s -u admin:SuperSecret http://rocky-lab:8080/config.bak
```

Then press `Ctrl+C` in the `tcpdump` terminal.

> **What just happened:** `tcpdump -i any` captures on all interfaces, `-w /tmp/http.pcap` writes raw packets to a file (the standard pcap format that Wireshark opens), and the filter `host rocky-lab and tcp port 8080` keeps only this conversation. The file goes to `/tmp` because `tcpdump` drops root privileges to a `tcpdump` user before writing. `curl -u` sends HTTP Basic authentication.

5. Read the capture:

```
tcpdump -A -r /tmp/http.pcap | grep -E 'Authorization|db_password'
echo 'YWRtaW46U3VwZXJTZWNyZXQ=' | base64 -d; echo
```

```
reading from file /tmp/http.pcap, link-type LINUX_SLL2 (Linux cooked v2)
Authorization: Basic YWRtaW46U3VwZXJTZWNyZXQ=
db_password=Winter2026
admin:SuperSecret
```

> **What just happened:** `-r` reads the file and `-A` prints packet contents as ASCII text. The password inside the file **and** the login credentials crossed the network in cleartext. "Basic" authentication is only Base64 encoding, which `base64 -d` reverses instantly; it is not encryption. Anyone on the path could read both. The fix is TLS (Lab 3.5) and never exposing such files at all.

6. Decode the same capture with `tshark`, the command-line version of Wireshark:

```
sudo DEBIAN_FRONTEND=noninteractive apt install -y tshark
tshark -r /tmp/http.pcap -Y http.request \
  -T fields -e ip.src -e http.request.uri -e http.authorization
```

```
192.168.100.10  /config.bak Basic YWRtaW46U3VwZXJTZWNyZXQ=
```

What just happened: `DEBIAN_FRONTEND=noninteractive` skips the installer question about letting normal users capture packets (the answer defaults to "no"). `tshark -r` reads the file, `-Y http.request` is a *display filter* that keeps only HTTP requests, and `-T fields -e ...` prints chosen fields. Unlike `tcpdump`, Wireshark/tshark understands hundreds of protocols and names every field. Run `tshark -r /tmp/http.pcap -Y http.request -V | grep -i credentials` to see tshark decode the password for you.

7. Fix the misconfiguration: on rocky-lab press `Ctrl+C` to stop the Python server, then remove the firewall opening:

```
# on rocky-lab
sudo firewall-cmd --reload
sudo firewall-cmd --list-ports
```

What just happened: `--reload` throws away runtime-only changes and reloads the saved configuration, so port 8080 is closed again. An empty `--list-ports` output confirms it.

Checkpoint: `nmap -sV -p 8080 rocky-lab` now shows `filtered`, and you saw the cleartext credentials in both `tcpdump -A` and `tshark` output.

Task 5: Audit rocky-lab against the CIS Benchmark with OpenSCAP

1. Install the scanner and the security content:

```
# on rocky-lab
sudo dnf install -y openscap-scanner scap-security-guide
ls /usr/share/xml/scap/ssg/content/
```

```
ssg-rl9-ds.xml
```

What just happened: `openscap-scanner` provides the `oscap` command. `scap-security-guide` (SSG, from the ComplianceAsCode project) provides the content: a *data stream* file (`-ds.xml`) that bundles the rules, checks, and profiles for this OS. Use the file name your system shows (it may be `ssg-rhel9-ds.xml`) in the next step.

2. List the profiles and find the CIS ones:

```
# on rocky-lab
DS=/usr/share/xml/scap/ssg/content/ssg-rl9-ds.xml
oscap info --profiles "$DS" | grep -i cis
```

```
xccdf_org.ssgproject.content_profile_cis:CIS Red Hat Enterprise Linux 9 Benchmark for Level
2 - Server
xccdf_org.ssgproject.content_profile_cis_server_l1:CIS Red Hat Enterprise Linux 9 Benchmark
for Level 1 - Server
xccdf_org.ssgproject.content_profile_cis_workstation_l1:CIS Red Hat Enterprise Linux 9
Benchmark for Level 1 - Workstation
xccdf_org.ssgproject.content_profile_cis_workstation_l2:CIS Red Hat Enterprise Linux 9
Benchmark for Level 2 - Workstation
```

What just happened: A *profile* is a selection of rules for one standard. Each has a long ID (before the colon) and a title. Other profiles in the same file include `pci-dss`, `stig`, `hipaa`, and `anssi`. You will use `cis_server_l1`, the CIS Level 1 baseline for servers. Rocky is rebuilt from RHEL sources, so it uses the RHEL 9 benchmark.

3. Run the evaluation. It reads the system only; nothing is changed. It takes a few minutes:

```
# on rocky-lab
mkdir -p ~/labs/3.6 && cd ~/labs/3.6
sudo oscap xccdf eval \
  --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --results results.xml --report report.html "$DS" | tee scan.txt
```

```
Title   Ensure /tmp Located On Separate Partition
Rule    xccdf_org.ssgproject.content_rule_partition_for_tmp
Ident   CCE-83869-8
Result  fail

Title   Disable SSH Root Login
Rule    xccdf_org.ssgproject.content_rule_sshd_disable_root_login
Ident   CCE-90799-8
Result  fail
...
```

What just happened: `oscap xccdf eval` evaluated every rule in the profile. For each rule it printed the title, rule ID, an identifier (CCE number), and the result: `pass`, `fail`, `notapplicable`, or `notchecked`. `--results` saves machine-readable results (XML) for other tools, `--report` writes a human-friendly HTML report, and `tee` saves the screen output to `scan.txt`. The command exits with code 2 when any rule fails; that is normal.

4. Summarize the results:

```
# on rocky-lab
awk '$1=="Result" {count[$2]++} END {for (r in count) print r, count[r]}' scan.txt
grep -B3 -E '^Result\s+fail' scan.txt | grep '^Title' | head -10
```

```
pass 142
fail 61
notapplicable 21
notchecked 3
```

What just happened: `awk` counts each kind of result. `grep -B3` prints the 3 lines *before* each failing result, and the second `grep` keeps only the titles, giving you a to-do list. A fresh minimal install typically fails dozens of CIS rules (separate partitions, audit rules, password policy, banner text). Your numbers will differ. If almost everything is `notapplicable`, the content does not recognize your OS; check that you used the data stream for your distribution.

5. Look at the HTML report and the fix script OpenSCAP can generate:

```
# on rocky-lab
sudo oscap xccdf generate fix --fix-type bash \
  --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --output cis-fix.sh "$DS"
grep -c 'BEGIN fix' cis-fix.sh
ls -lh report.html
```

What just happened: `generate fix` writes a Bash script containing the remediation for every rule in the profile (it can also produce Ansible playbooks with `--fix-type ansible`). Do **not** run it on this lab VM; CIS fixes can lock accounts, change SSH, and alter partitions. Review fix scripts and test them on a copy first. To read `report.html`, copy it to your host computer (for example `scp student@rocky-lab:labs/3.6/report.html .` from your host's terminal) and open it in a browser. Each failed rule explains the risk and the fix.

TIP

Ubuntu and CIS: Canonical's CIS tool for Ubuntu is `usg` (Ubuntu Security Guide). It needs an Ubuntu Pro subscription (free for personal use on up to 5 machines): `sudo pro attach <token>`, `sudo pro enable usg`, `sudo apt install usg`, then `sudo usg audit cis_level1_server`. Under the hood it also uses OpenSCAP. You can also download the CIS Benchmark PDFs for free from the CIS website and follow them by hand.

Checkpoint: `scan.txt`, `results.xml`, and `report.html` exist in `~/labs/3.6` on rocky-lab, and you can name three failed rules.

Task 6: Detect changed files with AIDE and rkhunter

1. On ubuntu-lab, install AIDE without extra recommended packages (which would pull in a mail server):

```
sudo apt install -y --no-install-recommends aide
ls /etc/aide/
```

```
aide.conf  aide.conf.d
```

What just happened: On Ubuntu the AIDE configuration is `/etc/aide/aide.conf` plus many small rule files in `/etc/aide/aide.conf.d/`. The rules say which folders to watch and which attributes to record (hashes, permissions, owner, size, timestamps). `--no-install-recommends` skips optional packages such as a mail transfer agent, which AIDE would use to email daily reports.

2. Build the baseline database. This scans most of the filesystem and takes several minutes:

```
sudo aideinit -y -f
ls -lh /var/lib/aide/
```

```
Running aide --init...
Start timestamp: 2026-10-10 16:45:02 +0000 (AIDE 0.18.6)
AIDE initialized database at /var/lib/aide/aide.db.new
...
-rw------- 1 root root 21M Oct 10 16:51 aide.db
-rw------- 1 root root 21M Oct 10 16:51 aide.db.new
```

What just happened: `aideinit` is Ubuntu's wrapper around `aide --config /etc/aide/aide.conf --init`. It writes the new database `aide.db.new` and then copies it to `aide.db`, the file AIDE compares against. `-y` answers yes to overwriting an existing database and `-f` forces it. In production, copy `aide.db` to read-only or offline storage, because an attacker with root could otherwise update it to hide changes.

3. Simulate an attacker: change a watched file and add a new one:

```
sudo cp /etc/hosts /etc/hosts.lab-backup
echo "203.0.113.66 updates.example.com" | sudo tee -a /etc/hosts
echo "evil=1" | sudo tee /etc/lab-aide-test.conf
```

What just happened: Changing `/etc/hosts` is a classic attacker trick: it silently redirects a name (here an update server) to an attacker's address. `203.0.113.0/24` is a documentation-only address range, so nothing real is affected.

4. Run the check:

```
sudo aide -c /etc/aide/aide.conf --check
echo "exit code: $?"
```

```
Start timestamp: 2026-10-10 16:58:40 +0000 (AIDE 0.18.6)
AIDE found differences between database and filesystem!!

Summary:
  Total number of entries:  104321
  Added entries:        2
  Removed entries:      0
  Changed entries:      1

---------------------------------------------------
Added entries:
---------------------------------------------------

f++++++++++++++++: /etc/hosts.lab-backup
f++++++++++++++++: /etc/lab-aide-test.conf

---------------------------------------------------
Changed entries:
---------------------------------------------------

f   ...    .C... : /etc/hosts
...
exit code: 5
```

What just happened: `-c` (same as `--config`) points to the Ubuntu config file, and `--check` compares the filesystem with `aide.db`. AIDE lists added files (`+` in every column), and changed files with a letter per changed attribute (`C` means the checksum changed; `s` size, `m` modification time, `p` permissions, `u` owner). Further down it shows old and new hash values. The exit code is a bitmask: 1 = added, 2 = removed, 4 = changed, so 1 + 4 = 5. Scripts and monitoring tools use this code. Your list may include other files that legitimately changed (logs, apt lists).

5. After an approved change you accept it into the baseline with `--update`. Here you will instead undo the change and confirm:

```
sudo mv /etc/hosts.lab-backup /etc/hosts
sudo rm /etc/lab-aide-test.conf
systemctl list-timers --all --no-pager | grep -i aide
```

What just happened: The files are back to their original state. To accept real changes you would run `sudo aide -c /etc/aide/aide.conf --update` and then `sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db`. Ubuntu runs a daily check automatically through `dailyaidecheck.timer` (older releases used `/etc/cron.daily/aide`). On Rocky, AIDE's config is `/etc/aide.conf` and you run `aide --init` then `aide --check` directly.

6. Now run **rkhunter**:

```
sudo apt install -y --no-install-recommends rkhunter
sudo rkhunter --update
sudo rkhunter --propupd
sudo rkhunter --check --sk --rwo
```

```
[ Rootkit Hunter version 1.4.6 ]
File updated: searched for 180 files, found 146
Warning: The file properties have changed: ...
```

What just happened:

- `--update` downloads new rootkit signature data. On Ubuntu it may print an error about `WEB_CMD`, because Ubuntu disables rkhunter's own downloads (`WEB_CMD="/bin/false"` in `/etc/rkhunter.conf`) and ships updates through apt instead. That error is expected.
- `--propupd` records the current properties (hashes, sizes, permissions) of important system binaries as the known-good baseline. Run it right after installation and after every system update.
- `--check` runs all tests: known rootkit files, changed binaries, hidden files, suspicious startup files, SSH settings, and more. `--sk` skips the "press Enter" pauses, and `--rwo` reports warnings only.

Warnings are not proof of compromise. Each one needs a look; common false positives are scripts that replaced binaries in newer packages and hidden files that packages create. The full details are in `/var/log/rkhunter.log`.

Checkpoint: `sudo aide -c /etc/aide/aide.conf --check` listed `/etc/hosts` as changed (before you restored it), and `sudo grep -c Warning /var/log/rkhunter.log` returns a number you can explain.

Task 7: Verify installed files, signed packages, and the supply chain

1. On ubuntu-lab, install `debsums`, then tamper with a harmless file from the `nano` package:

```
sudo apt install -y debsums nano
sudo debsums -s nano && echo "nano: all files OK"
echo "# tampered" | sudo tee -a /usr/share/nano/sh.nanorc
sudo debsums -s nano
dpkg --verify nano
```

```
nano: all files OK
debsums: changed file /usr/share/nano/sh.nanorc (from nano package)
??5?????? /usr/share/nano/sh.nanorc
```

What just happened: Every `.deb` package ships an MD5 list of its files in `/var/lib/dpkg/info/PACKAGE.md5sums`. `debsums` compares installed files with that list; `-s` (silent) prints only problems. `dpkg --verify` does the same and prints an `rpm -V`-style line where `5` means the checksum differs and `?` means "not checked". Run `sudo debsums -s` with no package name to check the whole system (it takes a few minutes); add `-a` to include configuration files, which admins are expected to change.

2. Repair the file by reinstalling the package:

```
sudo apt install --reinstall -y nano
sudo debsums -s nano && echo "nano: all files OK"
```

3. Do the same on rocky-lab with `rpm -V`:

```
# on rocky-lab
sudo dnf install -y nano
echo "# tampered" | sudo tee -a /usr/share/nano/sh.nanorc
rpm -V nano
sudo rpm -Va 2>/dev/null | head -5
sudo dnf reinstall -y nano
rpm -V nano && echo "nano: all files OK"
```

```
S.5....T.    /usr/share/nano/sh.nanorc
S.5....T.  c /etc/sysconfig/...
...
nano: all files OK
```

What just happened: `rpm -V PACKAGE` verifies one package; `rpm -Va` verifies all installed packages (it takes a while). Each letter is a failed test: `S` size, `M` mode (permissions), `5` digest (checksum), `D` device, `L` symlink, `U` user, `G` group, `T` modification time, `P` capabilities. A dot means the test passed. `c` marks a configuration file, where changes are often legitimate. A `5` on a binary in `/usr/bin` with no update to explain it is a serious warning sign. Note: an attacker with root can also change the RPM database, so the most trustworthy check boots from clean media or compares with AIDE data stored offline.

4. On rocky-lab, check package **signatures**:

```
# on rocky-lab
grep -m1 gpgcheck /etc/yum.repos.d/rocky.repo
rpm -q gpg-pubkey --qf '%{NAME}-%{VERSION}  %{SUMMARY}\n'
rpm -qi nano | grep Signature
sudo dnf install -y dnf-plugins-core
cd ~/labs/3.6 && dnf download nano
rpm -K nano-*.rpm
```

```
gpgcheck=1
gpg-pubkey-350d275d  Rocky Enterprise Software Foundation - Release key 2022
<releng@rockylinux.org> public key
Signature   : RSA/SHA256, Mon 13 May 2024 10:21:16 PM UTC, Key ID 702d426d350d275d
nano-5.6.1-6.el9.x86_64.rpm: digests signatures OK
```

What just happened: `gpgcheck=1` makes `dnf` refuse any package whose signature does not verify. The imported public keys appear as fake `gpg-pubkey` packages. Every installed package records which key signed it. `dnf download` (part of `dnf-plugins-core`, usually already installed) saves a package file without installing it. `rpm -K` (also `rpm --checksig`) checks a package file before you install it: "digests signatures OK" means the file is intact and was signed by a trusted key. A package from an unknown source would say `NOKEY` or `NOT OK`.

5. On ubuntu-lab, see how **apt** verifies signed packages. Apt signs the repository index, not each `.deb`:

```
grep -E '^(URIs|Suites|Signed-By)' /etc/apt/sources.list.d/ubuntu.sources | head -3
ls /var/lib/apt/lists/ | grep InRelease | head -3
cd ~/labs/3.6 && apt-get download nano
sha256sum nano_*.deb
apt-cache show nano | grep -m1 SHA256
```

```
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
archive.ubuntu.com_ubuntu_dists_noble-backports_InRelease
archive.ubuntu.com_ubuntu_dists_noble-updates_InRelease
archive.ubuntu.com_ubuntu_dists_noble_InRelease
7d2f...e41a  nano_7.2-2ubuntu0.1_amd64.deb
SHA256: 7d2f...e41a
```

What just happened: This is apt's chain of trust:

```
ubuntu-archive-keyring.gpg --verifies--> InRelease (GPG-signed, was Release + Release.gpg)
InRelease  --lists SHA256 of-->  Packages index
Packages   --lists SHA256 of-->  every .deb file
```

`Signed-By` names the only key allowed to sign this repository. During `apt update`, apt checks the signature on `InRelease`; during install, it checks each `.deb` against the hash in the signed index. You just did the last step by hand: the hash of the downloaded `.deb` matches the index. If any link fails, apt refuses with an error like "The following signatures couldn't be verified". Avoid adding random third-party repositories or running `curl ... | sudo bash` installers; each one is a new link in your supply chain.

6. **Supply chain case study: the xz backdoor (CVE-2024-3094, CVSS 10.0).** In March 2024, Microsoft engineer Andres Freund noticed SSH logins on Debian unstable (sid) using 500 ms more CPU than expected. He found that versions 5.6.0 and 5.6.1 of `xz` / `liblzma` contained a backdoor, hidden in test files by a contributor who had spent about two years earning the trust of the project. Because some distributions link `sshd` to `libsystemd`, which loads `liblzma`, the backdoor could let the attacker take over `sshd`. It was caught before reaching stable releases. Check what your VMs run:

```
xz --version
dpkg -l xz-utils | tail -1
```

```
# on rocky-lab
rpm -q xz
```

```
xz (XZ Utils) 5.4.5
liblzma 5.4.5
ii  xz-utils  5.6.1+really5.4.5-1ubuntu0.3  amd64  XZ-format compression utilities
xz-5.2.5-8.el9_0.x86_64
```

What just happened: Ubuntu 24.04 was about to ship 5.6.1, so Ubuntu rebuilt the safe 5.4.5 code under the version name `5.6.1+really5.4.5`, which is how Debian-family distributions say "the version number is higher, but the content is really the older release". Rocky 9 uses 5.2.5 and was never affected. Lessons: trust is a supply chain attack surface, reproducible builds and code review matter, and odd performance changes deserve investigation.

7. Verify a download the way you should for every ISO or tool you install. Fetch Ubuntu's checksum list and its signature, import Ubuntu's CD signing keys, and verify:

```
mkdir -p ~/labs/3.6/iso && cd ~/labs/3.6/iso
curl -sO https://releases.ubuntu.com/24.04/SHA256SUMS
curl -sO https://releases.ubuntu.com/24.04/SHA256SUMS.gpg
gpg --keyid-format long --keyserver hkps://keyserver.ubuntu.com \
  --recv-keys 0x46181433FBB75451 0xD94AA3F0EFE21092
gpg --keyid-format long --verify SHA256SUMS.gpg SHA256SUMS
cat SHA256SUMS
```

```
gpg: Signature made Thu 07 Aug 2025 05:12:33 PM UTC
gpg:                using RSA key 843938DF228D22F7B3742BC0D94AA3F0EFE21092
gpg: Good signature from "Ubuntu CD Image Automatic Signing Key (2012) <cdimage@ubuntu.com>"
[unknown]
gpg: WARNING: This key is not certified with a trusted signature!
Primary key fingerprint: 8439 38DF 228D 22F7 B374  2BC0 D94A A3F0 EFE2 1092
c3514bf0...d3d8274b *ubuntu-24.04.3-live-server-amd64.iso
```

What just happened: Two separate checks protect a download. The **GPG signature** proves the checksum list really came from Ubuntu (compare the fingerprint with the one published on ubuntu.com). The **SHA-256 checksum** then proves the ISO is intact. If you had the ISO in this folder you would finish with `sha256sum -c SHA256SUMS --ignore-missing`, which checks only the files present. An attacker who hacks a download mirror can replace the ISO and even the `SHA256SUMS` file, but cannot create a valid signature without Ubuntu's private key.

8. Two more supply chain controls: **pinning** and an **SBOM**:

```
sudo apt-mark hold nano
apt-mark showhold
sudo apt-mark unhold nano
dpkg-query -W -f='${Package},${Version}\n' | head -5
dpkg-query -W | wc -l
```

What just happened: `apt-mark hold` pins a package at its current version so upgrades skip it (Rocky: the `dnf versionlock` plugin). Pinning gives you control over *when* a new version arrives, so it can be tested first; you must still apply security fixes. `dpkg-query -W` lists every installed package and version: a basic inventory. A real **SBOM** (software bill of materials) is a standard machine-readable list (SPDX or CycloneDX format, created by tools like `syft`) of every component and dependency in a product, so that when the next xz-style CVE appears you can answer "are we affected?" in minutes.

Checkpoint: `debsums -s nano` and `rpm -V nano` are silent again after the reinstall, `rpm -K` printed "signatures OK", and `gpg --verify` printed "Good signature" for `SHA256SUMS`.

Task 8: Destroy data securely

CAUTION

This task destroys all data on an extra disk. Take a snapshot now. Run `lsblk` and confirm `/dev/sdc` is an empty 5 GB lab disk with no mount points. If your second extra disk has another name, use it everywhere. Writing to the wrong device (like `/dev/sda`) destroys your VM.

1. Start with a file. `shred` overwrites a file's contents before deleting it:

```
cd ~/labs/3.6
echo "customer card numbers" > secret.txt
shred -v -n 1 -z -u secret.txt
```

```
shred: secret.txt: pass 1/2 (random)...
shred: secret.txt: pass 2/2 (000000)...
shred: secret.txt: removing
shred: secret.txt: renamed to 0000000000
...
shred: secret.txt: removed
```

What just happened: `-n 1` overwrites once with random data, `-z` adds a final pass of zeros to hide that shredding happened, `-u` deletes the file afterwards (after renaming it several times to hide the name), and `-v` shows progress. One random pass is enough for modern hard disks. **Caveat:** on journaling or copy-on-write filesystems (ext4 with data journaling, Btrfs, ZFS), on SSDs, and when backups or snapshots exist, old copies may survive elsewhere. For files, shred is best effort; for whole devices it is reliable on hard disks.

2. Write a recognizable "secret" to the raw disk so you can prove each method destroys it:

```
lsblk /dev/sdc
echo "TOP-SECRET-LAB-DATA" | sudo dd of=/dev/sdc conv=fsync status=none
sudo head -c 1M /dev/sdc | grep -a -c TOP-SECRET
```

```
1
```

> **What just happened:** `dd of=/dev/sdc` writes its input straight to the start of the disk; `conv=fsync` makes sure it reaches the disk. `head -c 1M` reads the first megabyte and `grep -a -c` counts lines containing the secret: 1, so it is there. You will repeat this "plant and check" pattern for each method.

3. **Method 1: `shred` on a whole device.**

```
sudo shred -v -n 1 /dev/sdc
sudo head -c 1M /dev/sdc | grep -a -c TOP-SECRET
```

```
shred: /dev/sdc: pass 1/1 (random)...
shred: /dev/sdc: pass 1/1 (random)...1.1GiB/5.0GiB 22%
...
shred: /dev/sdc: pass 1/1 (random)...5.0GiB/5.0GiB 100%
0
```

> **What just happened:** `shred` overwrote all 5 GiB with random data (about a minute on a virtual disk). The count is now 0. On a hard disk this is a proper "clear" of the whole device.

4. **Method 2: `badblocks -w`.** Plant the secret again, then run a destructive write test over the first 100 MiB:

```
echo "TOP-SECRET-LAB-DATA" | sudo dd of=/dev/sdc conv=fsync status=none
sudo badblocks -wsv -b 4096 /dev/sdc 25599 0
sudo hexdump -C -n 64 /dev/sdc
```

```
Checking for bad blocks in read-write mode
From block 0 to 25599
Testing with pattern 0xaa: done
Reading and comparing: done
Testing with pattern 0x55: done
Reading and comparing: done
Testing with pattern 0xff: done
Reading and comparing: done
Testing with pattern 0x00: done
Reading and comparing: done
Pass completed, 0 bad blocks found. (0/0/0 errors)
00000000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000040
```

> **What just happened:** `badblocks` is really a disk-testing tool. `-w` is the **destructive** write mode: it writes four patterns (`0xaa` , `0x55` , `0xff` , `0x00`) to every block and reads each back to find bad sectors, destroying all data as a side effect. `-s` shows progress, `-v` is verbose, and `-b 4096` uses 4 KiB blocks. The two numbers are the *last* block and the *first* block to test: blocks 0 to 25599 are 100 MiB, which keeps the lab fast; without them it tests the whole disk. `hexdump -C` shows the last pattern, all zeros, where the secret used to be. Never use `-w` on a disk with data you want; `-n` is the non-destructive read-write mode.

5. **Method 3: `dd if=/dev/urandom`.**

```
echo "TOP-SECRET-LAB-DATA" | sudo dd of=/dev/sdc conv=fsync status=none
sudo dd if=/dev/urandom of=/dev/sdc bs=1M count=100 status=progress
sudo hexdump -C -n 64 /dev/sdc
```

```
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.4 s, 262 MB/s
00000000  9c 4e 1a f7 30 d2 88 5b  e1 07 c4 39 ae 52 6d 13  |.N..0..[...9.Rm.|
...
```

What just happened: `if=` is the input file (`/dev/urandom` produces endless random bytes) and `of=` the output (the disk). `bs=1M` writes in 1 MiB blocks and `count=100` stops after 100 blocks (100 MiB); leave out `count` to wipe the whole disk (dd then ends with a harmless "No space left on device"). `status=progress` shows live progress. `dd if=/dev/zero` is faster and is just as good on modern hard disks; random data hides which parts were ever used.

6. **Method 4: cryptographic destruction.** Encrypt the disk, store data, then destroy only the keys:

```
sudo cryptsetup luksFormat --type luks2 /dev/sdc
sudo cryptsetup open /dev/sdc wipetest
sudo mkfs.ext4 -q /dev/mapper/wipetest
sudo mkdir -p /mnt/wipetest && sudo mount /dev/mapper/wipetest /mnt/wipetest
echo "TOP-SECRET-LAB-DATA" | sudo tee /mnt/wipetest/secret.txt
sudo umount /mnt/wipetest && sudo cryptsetup close wipetest
sudo cryptsetup luksErase /dev/sdc
sudo cryptsetup luksDump /dev/sdc | sed -n '/^Keyslots:/,/^Tokens:/p'
```

Type `YES` and a passphrase (for example `Wipe-Lab-1`) for `luksFormat` , the same passphrase for `open` , and `YES` again for `luksErase` .

```
Keyslots:
Tokens:
```

What just happened: All data on the disk was encrypted with a random master key. `luksErase` overwrote every key slot, so no passphrase can ever recover that master key, and the 5 GB of ciphertext is now permanently meaningless. It took less than a second, which is why crypto-erase is the preferred method for SSDs, cloud volumes, and large arrays: you do not have to reach every physical copy of the data. Try `sudo cryptsetup open /dev/sdc wipetest` ; no passphrase works. Many SSDs and NVMe drives do the same internally ("self-encrypting drives"; `nvme format --ses=2` or the ATA secure erase command).

TIP

Standards such as NIST SP 800-88 describe three levels: **clear** (overwrite with normal commands, like the methods above), **purge** (stronger, such as crypto-erase or the drive's built-in sanitize command), and **destroy** (shred, crush, or incinerate the physical media). Always record what was destroyed, how, by whom, and when (a certificate of destruction).

Checkpoint: `sudo cryptsetup luksDump /dev/sdc` shows an empty `Keyslots:` section, and you saw the secret count drop to 0 or zeros after each method.

Task 9: Set security banners

1. Look at the current banners on ubuntu-lab and back them up:

```
cat /etc/issue
cat /etc/issue.net
ls -l /etc/motd
ls /etc/update-motd.d/
sudo cp /etc/issue /etc/issue.orig
sudo cp /etc/issue.net /etc/issue.net.orig
```

```
Ubuntu 24.04.3 LTS \n \l

Ubuntu 24.04.3 LTS
ls: cannot access '/etc/motd': No such file or directory
00-header  10-help-text  50-landscape-sysinfo  50-motd-news  90-updates-available
91-contract-ua-esm-status  91-release-upgrade  92-unattended-upgrades  95-hwe-eol
98-fsck-at-reboot  98-reboot-required
```

What just happened: `/etc/issue` is printed by `agetty` on the local console before the login prompt; `\n` is replaced with the host name and `\l` with the terminal name. `/etc/issue.net` is meant for network logins, but its escape codes are not expanded. Both announce the exact OS version, which helps attackers choose exploits. Ubuntu 24.04 usually has no static `/etc/motd` ; instead the scripts in `/etc/update-motd.d/` build a dynamic message at each login.

2. Write a legal warning to `/etc/issue` (local console) and preview it:

```
cat <<'EOF' | sudo tee /etc/issue
WARNING: Authorized users only. All activity on this system is monitored
and logged. Unauthorized access is prohibited and may be prosecuted.
Host: \n   Terminal: \l

EOF
sudo agetty --show-issue
```

```
WARNING: Authorized users only. All activity on this system is monitored
and logged. Unauthorized access is prohibited and may be prosecuted.
Host: ubuntu-lab   Terminal: pts/0
```

What just happened: `agetty --show-issue` prints the issue file the way the console login would, with escapes expanded, and exits. To see it for real, log out on the VM's console window: the new text appears above `ubuntu-lab login:` . A clear warning helps an organization take legal action and removes the excuse "I did not know it was private". It deliberately does not reveal the OS version.

3. Show a banner before SSH logins with `/etc/issue.net`:

```
cat <<'EOF' | sudo tee /etc/issue.net
WARNING: Authorized users only. All activity on this system is monitored
and logged. Unauthorized access is prohibited and may be prosecuted.
EOF
echo 'Banner /etc/issue.net' | sudo tee /etc/ssh/sshd_config.d/20-banner.conf
sudo sshd -t && sudo systemctl reload ssh
ssh student@localhost
```

```
WARNING: Authorized users only. All activity on this system is monitored
and logged. Unauthorized access is prohibited and may be prosecuted.
student@localhost's password:
```

What just happened: `sshd` does not show `/etc/issue.net` unless the `Banner` setting points to it (the default is `none`). The banner appears **before** authentication, so even someone who never gets in sees the warning. After logging in, stay in the session for the next step.

4. You are now logged in, so you also saw the message of the day. Customize it: disable one dynamic script, add your own, and add a static `/etc/motd`:

```
exit
sudo chmod -x /etc/update-motd.d/10-help-text
cat <<'EOF' | sudo tee /etc/update-motd.d/99-lab-notice
#!/bin/sh
echo ""
echo "Lab notice: $(hostname) has been $(uptime -p)."
EOF
sudo chmod 755 /etc/update-motd.d/99-lab-notice
echo "Maintenance window: Sundays 02:00-04:00 UTC. Report problems to the admin team." \
  | sudo tee /etc/motd
ssh student@localhost
```

```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-85-generic x86_64)
...
Lab notice: ubuntu-lab has been up 3 hours, 12 minutes.
Maintenance window: Sundays 02:00-04:00 UTC. Report problems to the admin team.
```

What just happened: At login, the PAM module `pam_motd` (see `/etc/pam.d/sshd`) runs every **executable** script in `/etc/update-motd.d/` in name order, saves the output to `/run/motd.dynamic`, and shows it, followed by the static `/etc/motd`. Removing the execute bit (`chmod -x`) disables a script without deleting it, so package updates do not bring it back. Preview the dynamic part any time with `sudo run-parts /etc/update-motd.d/`. On Rocky there are no dynamic scripts: edit `/etc/motd` or drop files into `/etc/motd.d/`. Type `exit` to close the session.

Checkpoint: `ssh student@localhost` shows the warning before the password prompt and your lab notice plus the maintenance line after login, and `sudo agetty --show-issue` shows the new console warning.

Why all of this belongs together: Compliance is a cycle, not a one-time task. You set a baseline (CIS, OpenSCAP, banners), record known-good state (AIDE, package hashes, signatures), keep checking for drift and attackers (ClamAV, rkhunter, IoC hunting, vulnerability scans), fix what you find (patches, configuration), and safely retire data and hardware at the end (destruction). Auditors want evidence of each step, which is why every tool here can write a report or log.

Challenge

CHALLENGE

1. Write a script `ioc-check.sh` that runs the IoC checks from Task 2 (listening ports, UID 0 accounts, recently changed cron files, SUID files not owned by a package) and prints a short report. Plant one IoC and confirm the script finds it.
2. On rocky-lab, pick one failed rule from the OpenSCAP report (for example the SSH root login rule), fix it by hand, and re-run only that rule with `oscap xccdf eval --profile ... --rule <rule-id>` to show it now passes. Then undo your change.
3. Install AIDE on rocky-lab, initialize it (`aide --init`, then move `/var/lib/aide/aide.db.new.gz` to `aide.db.gz`), change a file in `/etc`, and find it with `aide --check`.

Clean up

1. On ubuntu-lab, restore the banners and motd:

```
sudo mv /etc/issue.orig /etc/issue
sudo mv /etc/issue.net.orig /etc/issue.net
sudo rm -f /etc/ssh/sshd_config.d/20-banner.conf /etc/update-motd.d/99-lab-notice
sudo rm -f /etc/motd
sudo chmod +x /etc/update-motd.d/10-help-text
sudo sshd -t && sudo systemctl reload ssh
```

   (Remove `/etc/motd` only if it did not exist before the lab, which is normal on Ubuntu 24.04.)

2. Wipe the lab disk signature and remove the mount point:

```
sudo wipefs -a /dev/sdc
sudo rmdir /mnt/wipetest
lsblk /dev/sdc
```

3. Remove the tools installed for this lab (optional, but keeps the VM lean) and the capture file:

```
sudo apt purge -y clamav clamav-freshclam aide aide-common rkhunter tshark debsums
sudo apt autoremove -y
sudo rm -rf /var/lib/aide /var/lib/clamav
sudo rm -f /tmp/http.pcap
```

4. Remove the Ubuntu signing keys you imported into your GPG keyring, and the lab folder:

```
gpg --batch --yes --delete-keys 843938DF228D22F7B3742BC0D94AA3F0EFE21092
gpg --batch --yes --delete-keys 46181433FBB75451 2>/dev/null
rm -rf ~/labs/3.6
```

5. On rocky-lab, remove the lab files and make sure the firewall has no leftover rules:

```
# on rocky-lab
rm -rf ~/labs/3.6
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
sudo dnf remove -y openscap-scanner scap-security-guide
```

Exam corner

- **Anti-malware** on Linux: ClamAV (`freshclam` updates signatures, `clamscan -r -i` scans, `--move` quarantines). Test safely with the EICAR string.
- **Indicators of compromise**: unknown listening ports (`ss -tulpn`), new or UID 0 accounts, new cron jobs/timers, SUID files not owned by a package (`find / -perm -4000`), odd logins (`last` , `lastb`), unknown outbound connections, high CPU processes, changed binaries.
- **CVE** = unique ID for a public vulnerability; **CVSS** = 0.0 to 10.0 severity score (Low under 4, Medium 4 to 6.9, High 7 to 8.9, Critical 9 to 10) built from a vector.
- **Backporting**: distributions apply security fixes to older versions; check changelogs (`apt changelog` , `rpm -q --changelog`), `pro fix CVE-...` , or `dnf updateinfo` , not just upstream version numbers.
- **Port scanner**: `nmap -sV` finds open ports and versions; `nmap --script` adds checks. **Protocol analyzer**: `tcpdump -w` / `-r` / `-A` , Wireshark, `tshark` . Service **misconfiguration** = unsafe settings, not a code bug.
- **OpenSCAP**: `oscap xccdf eval --profile <id> --results r.xml --report r.html <datastream>` with content from `scap-security-guide` . **CIS** Benchmarks: Level 1 (baseline) and Level 2 (strict); Ubuntu's tool is `usg` (Ubuntu Pro).
- **AIDE**: `aide --init` (Ubuntu `aideinit`), `aide --check` , `aide --update` ; config `/etc/aide/aide.conf` (Ubuntu) or `/etc/aide.conf` (RHEL). **rkhunter**: `--propupd` baseline, `--check --sk` .
- Installed file verification: `debsums -s` , `dpkg --verify` (Debian/Ubuntu), `rpm -V pkg` / `rpm -Va` (codes S, M, 5, D, L, U, G, T). **Signed package** checks: `rpm -K` , `gpgcheck=1` , apt `InRelease` + `Signed-By` .
- Destruction: `shred -v -n 1 -z -u file` , `badblocks -w` (destructive write test), `dd if=/dev/urandom of=/dev/sdX bs=1M` , and **cryptographic** erase (`cryptsetup luksErase`), best for SSDs.
- **Supply chain**: xz backdoor (CVE-2024-3094); verify downloads with checksums **and** GPG signatures; pin versions; keep an SBOM (SPDX, CycloneDX).

- Banners: `/etc/issue` (local, before login), `/etc/issue.net` (network, needs `Banner /etc/issue.net` in `sshd_config`), `/etc/motd` (after login; Ubuntu adds dynamic `/etc/update-motd.d/`).

Review questions

1. A scanner reports that a Rocky 9 server is vulnerable because it runs OpenSSH 8.7, but the CVE was fixed upstream in 9.8. What is the most likely explanation, and how do you confirm it?
2. Which command verifies all installed RPM packages against the RPM database? A. `rpm -K` B. `rpm -qa` C. `rpm -Va` D. `rpm --import`
3. You need to erase a company laptop's NVMe SSD quickly and reliably. The disk was encrypted with LUKS. Which method is best? A. `shred -n 3` on every file B. `dd if=/dev/zero` once C. Destroy the LUKS key slots (crypto-erase) D. `badblocks -n`
4. Which file is shown to SSH users before they authenticate, if `sshd` is configured for it? A. `/etc/motd` B. `/etc/issue` C. `/etc/issue.net` D. `/etc/nologin`
5. Which tool would you use to check a Rocky Linux server against the CIS Level 1 Server benchmark and produce an HTML report?
6. During an investigation you find `/usr/local/bin/sys-helper` with permissions `-rwsr-xr-x` owned by root, and `dpkg -S` says no package owns it. Why is this an indicator of compromise?

PART 4 · DOMAIN 4.0

Part 4: Automation, Orchestration, and Scripting

17% of the exam · 5 labs

4.1	Summarize the use cases and techniques of automation and orchestration in a Linux environment.
4.2	Given a scenario, perform automated tasks using shell scripting.
4.3	Summarize Python basics used for Linux system administration.
4.4	Given a scenario, implement version control using Git.
4.5	Summarize best practices and responsible uses of artificial intelligence (AI).

Lab 4.1 Automation and Orchestration

Objective: 4.1 Summarize the use cases and techniques of automation and orchestration in a Linux environment.
Domain: 4.0 Automation, Orchestration, and Scripting (17% of the exam)
Time: about 4 hours (do it in sittings: Part A is Tasks 1-4, Part B is Tasks 5-7, Part C is Tasks 8-9)
VM: ubuntu-lab and rocky-lab
Snapshot first: Yes, on both VMs. This lab installs many tools, a web server, Docker, and a Kubernetes cluster.

Modern Linux admins rarely configure servers one by one. They describe the servers they want in text files, store those files in Git, and let tools build and check the servers for them. In this lab you will try the main tools the exam names: Ansible, Puppet, OpenTofu, Kickstart, cloud-init, a small CI/CD pipeline, Docker Compose, Docker Swarm, and Kubernetes. The goal is not to master each one but to touch each idea with your own hands so the exam terms make sense.

What you'll practice

- Infrastructure as code (IaC) with **Ansible**: playbooks, inventory, modules, ad hoc commands, collections, facts, agentless design
- IaC with **Puppet**: classes, certificates, modules, facts, agent and agentless (`puppet apply`) modes
- IaC with **OpenTofu**: provider, resource, state, API
- Unattended deployment: **Kickstart** and **cloud-init**
- CI/CD: version control, shift left testing, GitOps, pipelines, DevSecOps
- Orchestration with **Kubernetes**: ConfigMaps, Secrets, Pods, Deployments, Volumes, Services, variables
- **Docker Swarm**: service, nodes, tasks, networks, scale
- **Docker/Podman Compose**: compose file, up/down, logs

How it works

Infrastructure as code (IaC) means you describe your servers, networks, and services in text files instead of clicking or typing commands by hand. Think of a recipe card versus cooking from memory. With a recipe, anyone can make the same dish again, you can improve the recipe over time, and you can see exactly what changed between versions. IaC files live in **version control** (Git), so every change has an author, a date, and a reason.

Most IaC tools are **declarative**: you describe the end state ("nginx is installed and running"), not the steps. The tool compares the current state with the wanted state and only fixes the difference. Running the same code twice changes nothing the second time. This property is called **idempotence**, and it is what makes automation safe to repeat.

The tools in this lab fill different roles:

```
Build the machine          Configure the OS          Run the apps
------------------         -----------------         ------------------
OpenTofu (cloud APIs)      Ansible (push, SSH)       Docker Compose (1 host)
Kickstart (RHEL install)   Puppet (pull, agent)      Docker Swarm (cluster)
cloud-init (1st boot)                                Kubernetes (cluster)

           ^ all of it stored in Git and tested by a CI/CD pipeline ^
```

Ansible is **agentless**. Nothing extra runs on the managed servers. The control node (your ubuntu-lab) connects over SSH, copies small Python programs called **modules** to the target, runs them, and deletes them. An **inventory** file lists the hosts and groups. A **playbook** is a YAML file of tasks. An **ad hoc** command runs one module from the command line without a playbook. Before running tasks, Ansible gathers **facts** (OS name, IP addresses, memory) about each host. **Collections** are bundles of extra modules you download with `ansible-galaxy`.

Puppet traditionally uses an **agent**. A Puppet agent service runs on every managed node and, every 30 minutes, asks the Puppet server for its configuration (a pull model). Agents and the server trust each other with TLS **certificates**: the agent sends a certificate signing request, and an admin signs it on the server. Puppet code is written in **manifests** (`.pp` files) that define **classes** (named, reusable blocks of configuration), grouped into **modules**. Puppet uses a tool called `facter` to collect **facts**. You can also run Puppet without a server (agentless) with `puppet apply`, which is what you will do.

OpenTofu is the open-source fork of Terraform. It builds infrastructure such as cloud VMs, DNS records, or containers. A **provider** is a plugin that knows how to talk to one platform's **API** (application programming interface: the set of web or socket calls a platform accepts). A **resource** is one thing the provider manages. OpenTofu remembers what it built in a **state** file, so next time it can compare the code, the state, and the real world.

Kickstart (RHEL family) and **cloud-init** (almost every cloud image, and Ubuntu autoinstall) give **unattended deployment**: a text file answers every installer or first-boot question so no human has to click.

CI/CD stands for continuous integration and continuous delivery/deployment. Every push to Git starts a **pipeline**: a series of automatic stages such as lint, test, security scan, and deploy. **Shift left testing** means moving tests earlier (to the left on a timeline), for example running a linter before you even commit. **DevSecOps** adds security checks (secret scanning, vulnerability scanning) into the same pipeline instead of at the end. **GitOps** goes one step further: the Git repository is the single source of truth, and an agent such as Argo CD or Flux keeps the live cluster matching what is in Git.

Orchestration means running and coordinating many containers across one or more hosts: starting them, restarting them when they crash, scaling them, and connecting them. **Docker Compose** runs a multi-container app on one host from a **compose file**. **Docker Swarm** clusters Docker hosts (**nodes**) and runs **services** made of **tasks** (one task = one container). **Kubernetes** is the industry standard: it runs **Pods** (one or more containers), managed by **Deployments**, reached through **Services**, configured with **ConfigMaps** and **Secrets**, and given storage through **volumes**.

Tasks

Task 1: Set up Ansible and run ad hoc commands

1. Make a working directory for the whole lab and install Ansible on ubuntu-lab. On Ubuntu 24.04 the `ansible` package includes `ansible-core` plus a large set of community collections.

```
mkdir -p ~/labs/4.1/ansible && cd ~/labs/4.1/ansible
sudo apt update
sudo apt install -y ansible
ansible --version
```

```
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/student/.ansible/plugins/modules', ...]
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/student/.ansible/collections:/usr/share/...
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, ...) [GCC 13.2.0] (/usr/bin/python3)
```

> **What just happened:** `ansible --version` shows the core version (2.16 on Ubuntu 24.04), which Python it uses, and where it looks for collections. `config file = None` means no `ansible.cfg` was found yet. You will create one in this directory.

2. Ansible connects to managed hosts with SSH, so set up key login to rocky-lab. If you have not done Lab 3.3, create a key and copy it now. Press Enter to accept the defaults if `ssh-keygen` asks questions.

```
test -f ~/.ssh/id_ed25519 || ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ssh-copy-id student@rocky-lab
ssh student@rocky-lab hostname
```

> **What just happened:** `test -f` checks whether the key file already exists; `||` runs `ssh-keygen` only if it does not. `-N ""` sets an empty passphrase (fine for a lab, not for production). `ssh-copy-id` appends your public key to `~/.ssh/authorized_keys` on rocky-lab. The last command must print `rocky-lab` without asking for a password. That is all Ansible needs on the target: SSH and Python. This is what **agentless** means.

3. Create the **inventory**, the list of hosts Ansible manages. You will use a heredoc to write files. A heredoc (`<<'EOF' ... EOF`) feeds every line up to `EOF` into the command. The quotes around `'EOF'` stop the shell from expanding `$` inside the text.

```
cat > inventory.ini <<'EOF'
[web_ubuntu]
ubuntu-lab ansible_connection=local

[web_rocky]
rocky-lab ansible_user=student

[webservers:children]
web_ubuntu
web_rocky
EOF
```

What just happened: The file `inventory.ini` uses INI format. `[web_ubuntu]` and `[web_rocky]` are groups. `ansible_connection=local` tells Ansible to run tasks on ubuntu-lab directly instead of SSHing to itself. `ansible_user=student` sets the SSH login for rocky-lab. `[webservers:children]` makes a parent group that contains both groups, so you can target both VMs with one name. Every host is also in the built-in group `all`.

4. Create `ansible.cfg` in the same directory so you do not have to type `-i inventory.ini` every time, then show the inventory as a tree.

```
cat > ansible.cfg <<'EOF'
[defaults]
inventory = inventory.ini
EOF
ansible-inventory --graph
```

```
@all:
  |--@ungrouped:
  |--@webservers:
  |  |--@web_ubuntu:
  |  |  |--ubuntu-lab
  |  |--@web_rocky:
  |  |  |--rocky-lab
```

What just happened: Ansible reads `ansible.cfg` from the current directory first, then `~/.ansible.cfg`, then `/etc/ansible/ansible.cfg`. `ansible-inventory --graph` proves the groups were parsed the way you meant.

5. Run your first **ad hoc** command. An ad hoc command runs a single module against a host pattern: `ansible <pattern> -m <module> -a '<arguments>'`.

```
ansible all -m ansible.builtin.ping
```

```
ubuntu-lab | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
rocky-lab | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

What just happened: The `ping` **module** is not ICMP ping. It logs in, finds Python, runs a tiny module, and returns `pong`. `ansible.builtin.ping` is the module's full name (FQCN, fully qualified collection name): collection `ansible.builtin`, module `ping`. `"changed": false` means nothing on the host was modified.

6. Gather **facts**. The `setup` module collects hundreds of facts; the `filter` argument narrows them.

```
ansible all -m ansible.builtin.setup -a 'filter=ansible_distribution*'
ansible rocky-lab -m ansible.builtin.setup -a 'filter=ansible_memtotal_mb'
```

```
ubuntu-lab | SUCCESS => {
    "ansible_facts": {
        "ansible_distribution": "Ubuntu",
        "ansible_distribution_major_version": "24",
        "ansible_distribution_release": "noble",
        "ansible_distribution_version": "24.04",
        ...
rocky-lab | SUCCESS => {
    "ansible_facts": {
        "ansible_distribution": "Rocky",
        "ansible_distribution_major_version": "9",
        ...
```

What just happened: Facts are variables Ansible discovers about each host. Playbooks use them to make decisions, for example "install `httpd` if the OS family is RedHat, `nginx` if it is Debian".

7. Run a shell command on all hosts, then install a package on rocky-lab only. Installing needs root, so add `-b` (become, which uses `sudo`) and `-K` (ask for the sudo password).

```
ansible all -m ansible.builtin.command -a 'uptime'
ansible web_rocky -b -K -m ansible.builtin.dnf -a 'name=tree state=present'
ansible web_rocky -b -K -m ansible.builtin.dnf -a 'name=tree state=present'
```

```
BECOME password:
rocky-lab | CHANGED => {
    "changed": true,
    "msg": "",
    "rc": 0,
    "results": [
        "Installed: tree-1.8.0-10.el9.x86_64"
    ]
}
BECOME password:
rocky-lab | SUCCESS => {
    "changed": false,
    "msg": "Nothing to do",
    ...
```

What just happened: The `command` module runs a program and returns its output (`CHANGED` is printed because Ansible cannot know whether an arbitrary command changed anything). The `dnf` module is declarative: `state=present` means "make sure it is installed". The first run installs it (`CHANGED`); the second finds it already there (`SUCCESS` , `changed: false`). That is idempotence in action.

TIP

`-K` sends one password to every host. Keep the `student` password the same on both VMs for this lab. In production you would use SSH keys plus passwordless sudo for an automation account, or store secrets with `ansible-vault` .

Checkpoint: `ansible all -m ansible.builtin.ping` returns `pong` from both hosts, and `ssh student@rocky-lab rpm -q tree` prints a `tree-...el9` version.

Task 2: Write a playbook with variables, handlers, and a collection

1. A real project separates data from logic. Create **group variables** so each OS group gets the right package name. Ansible automatically loads `group_vars/<groupname>.yml` for hosts in that group.

```
mkdir -p group_vars templates
cat > group_vars/web_ubuntu.yml <<'EOF'
web_pkg: nginx
web_service: nginx
web_conf: /etc/nginx/conf.d/security.conf
web_conf_content: |
  server_tokens off;
EOF
cat > group_vars/web_rocky.yml <<'EOF'
web_pkg: httpd
web_service: httpd
web_conf: /etc/httpd/conf.d/security.conf
web_conf_content: |
  ServerTokens Prod
  ServerSignature Off
EOF
```

What just happened: Each file defines **variables** (`web_pkg` , `web_service` , and so on). The `|` in YAML starts a multi-line string. Both configs hide the web server's version number from HTTP headers, a common hardening step.

2. Create a Jinja2 **template** for the home page. Text inside `{{ }}` is replaced with variable or fact values when the file is copied. The file is `templates/index.html.j2` :

```
cat > templates/index.html.j2 <<'EOF'
<h1>{{ site_title }}</h1>
<p>Host {{ ansible_facts['hostname'] }} runs {{ ansible_facts['distribution'] }}
{{ ansible_facts['distribution_version'] }}</p>
EOF
```

3. Now write the **playbook** `webservers.yml` . Read every line; the explanation follows.

```
cat > webservers.yml <<'EOF'
---
- name: Configure the lab web servers
  hosts: webservers
  become: true
  vars:
    site_title: "Deployed by Ansible"

  tasks:
    - name: Install the web server package
      ansible.builtin.package:
        name: "{{ web_pkg }}"
        state: present

    - name: Hide the server version in HTTP headers
      ansible.builtin.copy:
        dest: "{{ web_conf }}"
        content: "{{ web_conf_content }}"
        owner: root
        group: root
        mode: "0644"
      notify: Restart web server

    - name: Publish the home page
      ansible.builtin.template:
        src: index.html.j2
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"

    - name: Start the web server now and at boot
      ansible.builtin.service:
        name: "{{ web_service }}"
        state: started
        enabled: true

    - name: Open HTTP in firewalld (Rocky only)
      ansible.posix.firewalld:
        service: http
        permanent: true
        immediate: true
        state: enabled
      when: ansible_facts['os_family'] == 'RedHat'

  handlers:
    - name: Restart web server
      ansible.builtin.service:
        name: "{{ web_service }}"
        state: restarted
EOF
```

What just happened: A playbook is a list of **plays**. This play targets the `webservers` group, uses `become: true` (sudo) for every task, and defines a play variable `site_title`. Each task calls one module:

- `package` picks apt or dnf automatically, so one task works on both OSes.
- `copy` with `content:` writes a file. `notify:` means "if this task changed something, run the handler called *Restart web server* at the end of the play".
- `template` renders the Jinja2 file with variables and facts.
- `service` makes sure the service is running and enabled at boot.
- `firewalld` comes from the `ansible.posix` collection. The `when:` condition uses a fact so the task only runs on RedHat-family hosts (Ubuntu's firewall, `ufw`, is inactive by default).

Handlers run once, only when notified, after all tasks. That avoids restarting a service when nothing changed.

4. Check the syntax, then run the playbook.

```
ansible-playbook webservers.yml --syntax-check
ansible-playbook webservers.yml -K
```

```
PLAY [Configure the lab web servers] ***************************************

TASK [Gathering Facts] *****************************************************
ok: [ubuntu-lab]
ok: [rocky-lab]

TASK [Install the web server package] **************************************
changed: [ubuntu-lab]
changed: [rocky-lab]
...
TASK [Open HTTP in firewalld (Rocky only)] *********************************
skipping: [ubuntu-lab]
changed: [rocky-lab]

RUNNING HANDLER [Restart web server] ***************************************
changed: [ubuntu-lab]
changed: [rocky-lab]

PLAY RECAP *****************************************************************
rocky-lab   : ok=7  changed=6  unreachable=0  failed=0  skipped=0 ...
ubuntu-lab  : ok=6  changed=4  unreachable=0  failed=0  skipped=1 ...
```

What just happened: Ansible ran each task on both hosts in parallel. Read the `PLAY RECAP` first: `failed=0` and `unreachable=0` mean success. `skipping` shows the `when:` condition worked.

5. Test both web servers, then prove idempotence by running the playbook again.

```
curl -s http://ubuntu-lab
curl -s http://rocky-lab
curl -sI http://rocky-lab | grep -i '^server'
ansible-playbook webservers.yml -K
```

```
<h1>Deployed by Ansible</h1>
<p>Host ubuntu-lab runs Ubuntu
24.04</p>
<h1>Deployed by Ansible</h1>
<p>Host rocky-lab runs Rocky
9.4</p>
Server: Apache
...
PLAY RECAP *********************************************************************
rocky-lab   : ok=6  changed=0  unreachable=0  failed=0  skipped=0 ...
ubuntu-lab  : ok=5  changed=0  unreachable=0  failed=0  skipped=1 ...
```

What just happened: The `Server:` header says only `Apache`, with no version, because of your `ServerTokens Prod` setting. On the second run every task reports `ok` and `changed=0`, and the handler did not run. The servers already matched the description, so Ansible did nothing.

6. Preview a change without making it. `--check` is a dry run and `--diff` shows the lines that would change. `-e` overrides a variable from the command line (extra vars beat play vars).

```
ansible-playbook webservers.yml -K --check --diff -e site_title="Version 2"
```

```
TASK [Publish the home page] ***************************************************
--- before: /var/www/html/index.html
+++ after: /home/student/.ansible/tmp/.../index.html.j2
@@ -1,3 +1,3 @@
-<h1>Deployed by Ansible</h1>
+<h1>Version 2</h1>
changed: [ubuntu-lab]
```

What just happened: Ansible reported what *would* change but did not touch the file. Run `curl -s http://ubuntu-lab` to confirm the page still says "Deployed by Ansible". Check mode is a great habit before changing production.

7. Use a **collection**. Collections bundle modules, plugins, and roles under a namespace, such as `community.general`. See which version Ubuntu's package already ships, then install a newer one into your home directory with `ansible-galaxy`.

```
ansible-galaxy collection list community.general
ansible-galaxy collection install 'community.general:>=9.0.0,<10.0.0'
ansible-galaxy collection list community.general
```

```
# /usr/lib/python3/dist-packages/ansible_collections
Collection        Version
----------------- -------
community.general 8.3.0
...
community.general:9.5.2 was installed successfully

# /home/student/.ansible/collections/ansible_collections
Collection        Version
----------------- -------
community.general 9.5.2

# /usr/lib/python3/dist-packages/ansible_collections
Collection        Version
----------------- -------
community.general 8.3.0
```

What just happened: `ansible-galaxy` downloaded the collection from Ansible Galaxy (galaxy.ansible.com). The version range keeps you on the 9.x series, which supports the ansible-core 2.16 that Ubuntu 24.04 ships. Your home copy is listed first, so it wins. Your exact version number may differ.

TIP

If you run `ansible-galaxy collection install community.general` with no version, it prints "Nothing to do" because a copy is already installed. In real projects, list collections and versions in a `requirements.yml` file and run `ansible-galaxy collection install -r requirements.yml`, so every admin gets the same versions.

8. Use a module from the collection: `community.general.archive` makes a compressed backup of the web root on every web server.

```
cat > backup.yml <<'EOF'
---
- name: Back up the web root with a collection module
  hosts: webservers
  become: true
  tasks:
    - name: Create a compressed archive of /var/www/html
      community.general.archive:
        path: /var/www/html
        dest: /tmp/webroot-backup.tar.gz
        format: gz
        mode: "0600"
EOF
ansible-playbook backup.yml -K
ansible webservers -b -K -m ansible.builtin.command \
  -a 'tar -tzf /tmp/webroot-backup.tar.gz'
```

What just happened: The module name `community.general.archive` tells Ansible exactly which collection provides the module. The ad hoc `tar -tzf` lists the archive contents on each host to prove the backup exists.

Checkpoint: `curl -s http://rocky-lab` and `curl -s http://ubuntu-lab` show your page, a second playbook run shows `changed=0` for both hosts, and `ansible-galaxy collection list community.general` shows a 9.x copy under your home directory.

Task 3: Puppet facts, classes, modules, and certificates

1. Install Puppet from the Ubuntu archive. Ubuntu 24.04 ships Puppet 8 and Facter 4. The package enables the Puppet agent service, which would keep trying to reach a server called `puppet` that does not exist, so stop and disable it.

```
sudo apt install -y puppet
puppet --version
systemctl status puppet --no-pager
sudo systemctl disable --now puppet
```

```
8.4.0
...
```

What just happened: `puppet.service` is the **agent**. In agent mode it wakes up every 30 minutes, sends its facts to the Puppet server, receives a compiled catalog (the list of resources it should have), and applies it. You disabled it because this lab uses agentless `puppet apply`.

2. Look at the **facts** that Puppet's `facter` tool collects. Facts are named with dots for nested values.

```
facter os.name os.release.full
facter networking.hostname networking.ip
facter memory.system.total
```

```
os.name => Ubuntu
os.release.full => 24.04
networking.hostname => ubuntu-lab
networking.ip => 192.168.100.10
memory.system.total => 3.82 GiB
```

What just happened: Facts in Puppet play the same role as Ansible facts: they describe the node so code can adapt to it. In manifests you read them as `$facts['os']['name']`.

3. Build a Puppet **module** containing a **class**. A module is a directory with a fixed layout; the main class lives in `manifests/init.pp` and has the same name as the module.

```
mkdir -p ~/labs/4.1/puppet/modules/labbase/manifests
cd ~/labs/4.1/puppet
cat > modules/labbase/manifests/init.pp <<'EOF'
# Class labbase: baseline settings for every lab server
class labbase (
  String $banner_text = 'Managed by Puppet',
) {
  package { 'figlet':
    ensure => installed,
  }

  file { '/etc/puppet-demo.txt':
    ensure  => file,
    owner   => 'root',
    group   => 'root',
    mode    => '0644',
    content => "${banner_text} on ${facts['networking']['hostname']} \
(${facts['os']['name']} ${facts['os']['release']['full']})\n",
    require => Package['figlet'],
  }
}
EOF
cat > site.pp <<'EOF'
class { 'labbase':
  banner_text => 'Configured by the labbase class',
}
EOF
puppet parser validate site.pp modules/labbase/manifests/init.pp
```

What just happened: `class labbase (...)` defines a class with one parameter and a default value. Inside are two **resources**: a `package` and a `file`. Every resource has a type, a title, and attributes. `require => Package['figlet']` sets the order. The `\` at the end of a line inside the double-quoted string joins it with the next line. `site.pp` declares (uses) the class and overrides the parameter. `puppet parser validate` checks syntax and prints nothing if all is well.

4. Preview with `--noop` (no operation, Puppet's dry run), then apply for real, then apply again.

```
sudo puppet apply --modulepath=modules site.pp --noop
sudo puppet apply --modulepath=modules site.pp
cat /etc/puppet-demo.txt
sudo puppet apply --modulepath=modules site.pp
```

```
Notice: Compiled catalog for ubuntu-lab in environment production in 0.21 seconds
Notice: /Stage[main]/Labbase/Package[figlet]/ensure: current_value 'purged', should
be 'present' (noop)
...
Notice: /Stage[main]/Labbase/Package[figlet]/ensure: created
Notice: /Stage[main]/Labbase/File[/etc/puppet-demo.txt]/ensure: defined content as
'{sha256}...'
Notice: Applied catalog in 4.10 seconds
Configured by the labbase class on ubuntu-lab (Ubuntu 24.04)
Notice: Compiled catalog for ubuntu-lab in environment production in 0.20 seconds
Notice: Applied catalog in 0.95 seconds
```

What just happened: `puppet apply` compiled the catalog locally and enforced it, with no server and no certificates. This is agentless Puppet. The third run printed no resource changes because the system already matched (idempotence again). `--modulepath=modules` tells Puppet where to find your module.

5. **Explore/simulate** the agent and certificate workflow. You do not have a Puppet server, but you can see the settings the agent would use.

```
sudo puppet config print server certname ssldir runinterval
```

```
certname = ubuntu-lab
runinterval = 1800
server = puppet
ssldir = /var/lib/puppet/ssl
```

TIP

Explore/simulate: In agent mode the flow is: (1) the agent creates a private key and a certificate signing request named after `certname`; (2) on the server an admin runs `puppetserver ca list` and `puppetserver ca sign --certname ubuntu-lab`; (3) the agent downloads its signed certificate into `ssldir`; (4) every `runinterval` seconds (1800 = 30 minutes) it pulls and applies its catalog, which you can trigger by hand with `sudo puppet agent --test`. Certificates give mutual TLS trust: the node knows it is talking to the real server, and the server knows which node is asking.

Checkpoint: `cat /etc/puppet-demo.txt` shows your banner with the host name and OS, and `systemctl is-enabled puppet` prints `disabled`.

Task 4: Unattended deployment with Kickstart and cloud-init

Kickstart answers every question of the RHEL installer (Anaconda). cloud-init configures a machine the first time it boots, most often in a cloud.

1. On rocky-lab, look at the Kickstart file that Anaconda saved when you installed Rocky. It records every choice you made in the installer.

```
# on rocky-lab
sudo cat /root/anaconda-ks.cfg
```

```
# Generated by Anaconda 34.25.4.9
# Generated by pykickstart v3.32
#version=RHEL9
# Use graphical install
graphical
...
%packages
@^minimal-environment
%end
...
```

What just happened: Anaconda always writes `/root/anaconda-ks.cfg`. It is the easiest starting point for your own Kickstart file: install one server by hand, then reuse its file to install hundreds the same way.

2. Install the Kickstart tools and write your own file. First create a hashed password so no plain-text password sits in the file.

```
# on rocky-lab
sudo dnf install -y pykickstart
mkdir -p ~/labs/4.1 && cd ~/labs/4.1
cat > ks.cfg <<'EOF'
#version=RHEL9
# Installation mode and source
text
url --url="https://dl.rockylinux.org/pub/rocky/9/BaseOS/x86_64/os/"
repo --name="AppStream" --
baseurl="https://dl.rockylinux.org/pub/rocky/9/AppStream/x86_64/os/"

# Localization
lang en_US.UTF-8
keyboard --xlayouts='us'
timezone America/New_York --utc

# Network and host name
network --bootproto=dhcp --device=link --activate --hostname=web01.lab.local

# Accounts
rootpw --lock
user --name=student --groups=wheel --iscrypted --password=HASH_HERE

# Disk layout (ERASES the target disk)
ignoredisk --only-use=sda
zerombr
clearpart --all --initlabel --drives=sda
autopart --type=lvm
bootloader --boot-drive=sda

# Security and finish
selinux --enforcing
firewall --enabled --service=ssh
firstboot --disable
reboot

%packages
@^minimal-environment
vim-enhanced
chrony
%end

%post --log=/root/ks-post.log
echo "Provisioned by Kickstart on $(date)" > /etc/motd
%end
EOF
HASH=$(openssl passwd -6 'Lab-Passw0rd!')
sed -i "s|HASH_HERE|$HASH|" ks.cfg
grep '^user' ks.cfg
```

What just happened: The file is a list of Kickstart commands. `text` runs a text-mode install; `url` and `repo` point to the package source; `rootpw --lock` disables root login; `user ... --iscrypted` creates `student` with a SHA-512 hash made by `openssl passwd -6`. `zerombr`, `clearpart`, and `autopart --type=lvm` wipe the disk and create an automatic LVM layout. `%packages ... %end` lists package groups (`@^` marks an environment group) and packages. `%post ... %end` is a shell script that runs inside the new system after installation. The `sed` command swapped the placeholder for the real hash (the `|` delimiter is used because the hash contains `/`).

3. Validate the file with `ksvalidator`, then break it on purpose to see an error.

```
# on rocky-lab
ksvalidator -v RHEL9 ks.cfg && echo "ks.cfg is valid"
sed 's/^autopart/autopartt/' ks.cfg > broken-ks.cfg
ksvalidator -v RHEL9 broken-ks.cfg
```

```
Checking kickstart file ks.cfg

ks.cfg is valid
Checking kickstart file broken-ks.cfg

The following problem occurred on line 23 of the kickstart file:

Unknown command: autopartt
```

What just happened: `ksvalidator` (from `pykickstart`) parses the file using the syntax for the version given by `-v RHEL9`. It catches typos and removed commands before you waste an install. It cannot check that URLs work or that disk `sda` exists.

TIP

Explore/simulate: To use the file, you publish it on a web server (or a USB stick or NFS share) and add a boot option at the installer's GRUB menu, for example `inst.ks=http://192.168.100.20/ks.cfg`. In a PXE setup the same option goes in the PXE menu, so new machines install themselves with no keyboard at all. Ubuntu's equivalent is **autoinstall**, which uses a cloud-init style YAML file.

4. Back on ubuntu-lab, check cloud-init. Ubuntu Server ships cloud-init and runs it on first boot.

```
cloud-init --version
cloud-init status --long
sudo tail -n 5 /var/log/cloud-init.log
ls /etc/cloud/cloud.cfg.d/
```

```
/usr/bin/cloud-init 26.1-0ubuntu1~24.04.1
status: done
extended_status: done
boot_status_code: enabled-by-generator
last_update: Thu, 01 Jan 1970 00:00:12 +0000
detail: DataSourceNone
errors: []
...
90_dpkg.cfg  99-installer.cfg  README
```

What just happened: `status: done` means cloud-init finished its boot stages. `detail` names the **datasource**, where cloud-init found its configuration (a cloud metadata service, a NoCloud seed disk, or `DataSourceNone` on an ISO install like yours). If you see `status: disabled`, the installer turned cloud-init off; the rest of this task still works. `/var/log/cloud-init.log` is the detailed log and `/var/log/cloud-init-output.log` holds the output of your commands.

5. Write a **user-data** file. It must start with the exact line `#cloud-config`. Create it in `~/labs/4.1/cloud`:

```
mkdir -p ~/labs/4.1/cloud && cd ~/labs/4.1/cloud
cat > user-data <<'EOF'
#cloud-config
hostname: web01
timezone: America/New_York
package_update: true
packages:
  - nginx
  - git
users:
  - default
  - name: deploy
    groups: sudo
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    ssh_authorized_keys:
      - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyOnly student@ubuntu-lab
write_files:
  - path: /var/www/html/index.html
    permissions: "0644"
    content: |
      <h1>Built by cloud-init</h1>
runcmd:
  - systemctl enable --now nginx
final_message: "cloud-init finished after $UPTIME seconds"
EOF
printf 'instance-id: web01-001\nlocal-hostname: web01\n' > meta-data
cloud-init schema --config-file user-data
```

```
Valid schema user-data
```

What just happened: Each top-level key is a cloud-init module: `packages` installs software, `users` creates accounts (`default` keeps the image's normal user), `write_files` drops files, and `runcmd` runs commands once at the end of first boot. `meta-data` holds the instance ID; cloud-init runs per-instance modules again only when the ID changes. `cloud-init schema` checks the YAML against the official schema.

6. Break the file to see the validator catch a typo, then build a **NoCloud** seed image, the format you attach to a VM as a CD to feed it this configuration.

```
sed 's/^packages:/pakages:/' user-data > bad-user-data
cloud-init schema --config-file bad-user-data
sudo apt install -y cloud-image-utils
cloud-localds seed.iso user-data meta-data
blkid seed.iso
```

```
Invalid user-data bad-user-data
Error: Cloud config schema errors: pakages: Additional properties are not allowed
('pakages' was unexpected)
...
seed.iso: BLOCK_SIZE="2048" UUID="..." LABEL="cidata" TYPE="iso9660"
```

TIP

Explore/simulate: If you booted an Ubuntu cloud image (from cloud-images.ubuntu.com) with `seed.iso` attached, cloud-init would find the disk labeled `cidata` (the NoCloud datasource), set the host name, create `deploy` , install nginx, and write the page with no human input. In AWS, Azure, or GCP you paste the same `#cloud-config` text into the "user data" field.

Checkpoint: `ksvalidator -v RHEL9 ks.cfg` on rocky-lab and `cloud-init schema --config-file user-data` on ubuntu-lab both report a valid file, and `blkid seed.iso` shows `LABEL="cidata"` .

Task 5: Build a CI/CD pipeline with shift left testing

You will build a tiny local pipeline and a Git hook that runs it before every commit, then read the GitHub Actions version.

1. Create a Git repository with one script that has a classic bug: unquoted variables. Git config here is set for this repository only (Lab 4.4 covers Git in depth).

```
sudo apt install -y git shellcheck
mkdir -p ~/labs/4.1/pipeline/scripts && cd ~/labs/4.1/pipeline
git init -b main
git config user.name "Student"
git config user.email "student@lab.local"
cat > scripts/cleanup-tmp.sh <<'EOF'
#!/bin/bash
# List files older than DAYS days in DIR (a cleanup report)
days=$1
target=$2
find $target -type f -mtime +$days -print
EOF
chmod +x scripts/cleanup-tmp.sh
```

2. Write the pipeline script `ci.sh`. Each stage must pass before the next one runs.

```
cat > ci.sh <<'EOF'
#!/usr/bin/env bash
# ci.sh - a tiny local CI pipeline: lint -> syntax -> secrets -> test
set -euo pipefail

echo "== Stage 1: lint (shellcheck) =="
shellcheck scripts/*.sh

echo "== Stage 2: syntax check (bash -n) =="
for f in scripts/*.sh; do
    bash -n "$f"
done

echo "== Stage 3: secret scan =="
if grep -rnE '(PASSWORD|SECRET|TOKEN)=[^$ ]+' scripts/; then
    echo "FAIL: possible hard-coded secret" >&2
    exit 1
fi

echo "== Stage 4: test =="
tmpdir=$(mktemp -d)
touch -d '10 days ago' "$tmpdir/old.log"
touch "$tmpdir/new.log"
result=$(scripts/cleanup-tmp.sh 7 "$tmpdir")
rm -rf "$tmpdir"
if [[ "$result" != *old.log ]] || [[ "$result" == *new.log* ]]; then
    echo "FAIL: expected only old.log, got: $result" >&2
    exit 1
fi

echo "All stages passed"
EOF
chmod +x ci.sh
```

> **What just happened:** `set -euo pipefail` makes the script stop at the first failing command, so a failed stage fails the whole pipeline. Stage 1 lints with ShellCheck. Stage 2 asks Bash to parse each script without running it. Stage 3 is a **DevSecOps** check: a simple secret scanner that looks for things like `TOKEN=abc123`. Stage 4 is a real test: it makes one old file and one new file and checks that the script reports only the old one.

3. Install the pipeline as a Git **pre-commit hook**. Git runs `.git/hooks/pre-commit` before every commit and cancels the commit if the hook fails. This is **shift left testing**: problems are caught on your laptop, minutes after you write them, instead of in production.

```
printf '#!/usr/bin/env bash\nexec ./ci.sh\n' > .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
git add .
git commit -m "Add cleanup report script"
```

```
== Stage 1: lint (shellcheck) ==

In scripts/cleanup-tmp.sh line 5:
find $target -type f -mtime +$days -print
     ^-----^ SC2086 (info): Double quote to prevent globbing and word splitting.
                       ^---^ SC2086 (info): Double quote to prevent globbing and ...
```

> **What just happened:** The commit was refused. Run `git log` and you will see "does not have any commits yet". ShellCheck warned that an unquoted `$target` containing a space would split into two arguments.

4. Fix the script, commit again, then try to sneak in a secret.

```
cat > scripts/cleanup-tmp.sh <<'EOF'
#!/bin/bash
# List files older than DAYS days in DIR (a cleanup report)
days="$1"
target="$2"
find "$target" -type f -mtime +"$days" -print
EOF
git add .
git commit -m "Add cleanup report script"
echo 'export API_TOKEN=abc123' >> scripts/cleanup-tmp.sh
git commit -am "Add token"
git checkout -- scripts/cleanup-tmp.sh
git log --oneline
```

```
== Stage 1: lint (shellcheck) ==
== Stage 2: syntax check (bash -n) ==
== Stage 3: secret scan ==
== Stage 4: test ==
All stages passed
[main (root-commit) 5e49a92] Add cleanup report script
...
== Stage 3: secret scan ==
scripts/cleanup-tmp.sh:6:export API_TOKEN=abc123
FAIL: possible hard-coded secret
5e49a92 Add cleanup report script
```

What just happened: The fixed script passed all four stages and was committed. The token commit was blocked by the security stage, and `git checkout --` threw the bad change away.

5. Hooks live only on your machine, so teams run the same pipeline on a server for every push. Create the GitHub Actions version as `.github/workflows/ci.yml`:

```
mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-24.04
    steps:
      - name: Check out the code
        uses: actions/checkout@v4
      - name: Install ShellCheck
        run: sudo apt-get update && sudo apt-get install -y shellcheck
      - name: Run the pipeline
        run: ./ci.sh
EOF
git add .github
git commit -m "Add CI workflow"
```

What just happened: `on:` lists the triggers (every push to `main` and every pull request). A **job** runs on a fresh runner VM (`ubuntu-24.04`), and its **steps** check out the code, install tools, and run your pipeline. GitLab CI (`.gitlab-ci.yml`) and Jenkins (`Jenkinsfile`) express the same idea. A failing job blocks the pull request from merging. A CD stage would add a final step that deploys, for example by running `ansible-playbook` against production.

> TIP
> **GitOps** extends this: you would keep the Kubernetes YAML you write in Task 9 in a Git repository, and a controller such as Argo CD or Flux would watch that repository and apply every merged change to the cluster. Nobody runs `kubectl apply` by hand; to change production you open a pull request.

Checkpoint: `git log --oneline` shows two commits, and running `./ci.sh` prints `All stages passed`.

Task 6: Run a multi-container app with Docker Compose

1. Install Docker and the Compose v2 plugin from the Ubuntu archive, and add yourself to the `docker` group so you (and OpenTofu, in Task 7) can use Docker without `sudo`.

```
sudo apt install -y docker.io docker-compose-v2
sudo usermod -aG docker "$USER"
exit
```

Log back in (SSH again or log in at the console) so your new group membership takes effect, then check:

```
id -nG
docker version --format '{{.Server.Version}}'
docker compose version
```

```
student adm cdrom sudo dip plugdev lxd docker
29.1.3
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1
```

> CAUTION
> Members of the `docker` group can get root on the host (they can start a container that mounts `/`). Only give it to trusted admins. You remove it in Clean up.

2. Write a **compose file**. Compose looks for `compose.yaml` in the current directory.

```
mkdir -p ~/labs/4.1/compose/html && cd ~/labs/4.1/compose
echo '<h1>Hello from Compose</h1>' > html/index.html
cat > compose.yaml <<'EOF'
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    depends_on:
      - cache
    restart: unless-stopped

  cache:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - cache-data:/data
    restart: unless-stopped

volumes:
  cache-data:
EOF
docker compose config --quiet && echo "compose.yaml is valid"
```

What just happened: `services:` defines two containers. `web` publishes host port 8080 to container port 80 and bind-mounts `./html` read-only (`:ro`). `depends_on` starts `cache` first. `cache` stores data in a named **volume**, `cache-data` , which survives container restarts. Compose also creates a private network for the project, and each service can reach the others by service name (`cache`). `docker compose config` validates and merges the file.

3. Bring the app **up** in the background (`-d` , detached), list its containers, and test both services.

```
docker compose up -d
docker compose ps
curl -s http://localhost:8080
docker compose exec cache redis-cli INCR visits
docker compose exec cache redis-cli INCR visits
```

```
 [+] Running 4/4
 Network compose_default      Created
 Volume "compose_cache-data"  Created
 Container compose-cache-1    Started
 Container compose-web-1      Started
NAME              IMAGE            COMMAND                  SERVICE   ...  STATUS
compose-cache-1   redis:7-alpine   "docker-entrypoint.s…"   cache     ...  Up 5 seconds
compose-web-1     nginx:alpine     "/docker-entrypoint.…"   web       ...  Up 4 seconds
<h1>Hello from Compose</h1>
(integer) 1
(integer) 2
```

What just happened: Compose pulled both images, created the network and volume, and started the containers. Names follow `<project>-<service>-<number>` ; the project name defaults to the directory name. `docker compose exec` runs a command inside a running service container.

4. Read the **logs**, then take the app **down**.

```
docker compose logs --tail 5
docker compose logs web
docker compose down
docker volume ls
docker compose down -v
docker volume ls
```

What just happened: `docker compose logs` merges the output of all services (add `-f` to follow live, like `tail -f`). `down` stops and removes the containers and network but keeps named volumes, so your Redis data would still be there next time. `down -v` also deletes the volumes.

TIP

Podman reads the same file. On Rocky, `sudo dnf install -y podman` and then `podman compose up -d` (Podman 4.7+ calls an external provider such as `podman-compose` or `docker-compose`), or install `podman-compose` from EPEL. On Ubuntu the package is `podman-compose` .

Checkpoint: After `docker compose down -v` , `docker compose ps` lists nothing and `docker volume ls` no longer shows `compose_cache-data` .

Task 7: Build infrastructure with OpenTofu

1. Install OpenTofu with its official installer, using the Debian package method. The script adds the OpenTofu apt repository and installs the `tofu` package.

```
cd ~/labs/4.1
curl --proto '=https' --tlsv1.2 -fsSL https://get.opentofu.org/install-opentofu.sh \
  -o install-opentofu.sh
chmod +x install-opentofu.sh
./install-opentofu.sh --install-method deb
rm -f install-opentofu.sh
tofu version
```

```
OpenTofu v1.10.6
on linux_amd64
```

What just happened: `curl --proto '=https' --tlsv1.2` refuses anything but modern HTTPS. Downloading the script to a file first (instead of piping straight into `sh`) lets you read it before running it, a good habit with any install script. The script uses `sudo` to add the repository and its signing key.

2. Write a configuration with two **providers**: `local` (manages files) and `kreuzwerker/docker` (talks to the Docker Engine **API** through `/var/run/docker.sock`). OpenTofu files use HCL and end in `.tf`. Create `~/labs/4.1/tofu/main.tf`:

```
mkdir -p ~/labs/4.1/tofu && cd ~/labs/4.1/tofu
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {
  host = "unix:///var/run/docker.sock"
}

variable "web_port" {
  type    = number
  default = 8081
}

resource "local_file" "notes" {
  filename = "${path.module}/deployed.txt"
  content  = "nginx is published on port ${var.web_port}\n"
}

resource "docker_image" "nginx" {
  name         = "nginx:alpine"
  keep_locally = true
}

resource "docker_container" "web" {
  name  = "tofu-web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = var.web_port
  }
}

output "url" {
  value = "http://localhost:${var.web_port}"
}
EOF
```

What just happened: The `terraform {}` block (OpenTofu keeps this name for compatibility) lists required providers and version limits; `~> 3.0` means 3.x but not 4.0. Each `resource "TYPE" "NAME"` block describes one object. `docker_image.nginx.image_id` refers to an attribute of another resource, which also tells OpenTofu to create the image before the container. `variable` and `output` are inputs and printed results.

3. Initialize, plan, and apply.

```
tofu init
tofu fmt
tofu validate
tofu plan
tofu apply
```

When `apply` asks `Do you want to perform these actions?`, type `yes`.

TIP

If `plan` or `apply` fails with `permission denied` on `/var/run/docker.sock`, you did not log out and back in after joining the `docker` group (Task 6). If it fails with `client version 1.41 is too old`, the Docker provider release is older than your Docker Engine supports (Docker 29 needs API 1.44 or newer); run `tofu init -upgrade` to fetch the newest 3.x release and try again.

```
Initializing provider plugins...
- Installing hashicorp/local v2.5.3...
- Installing kreuzwerker/docker v3.6.2...
OpenTofu has been successfully initialized!
...
Plan: 3 to add, 0 to change, 0 to destroy.
...
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

url = "http://localhost:8081"
```

What just happened: `init` downloaded the provider plugins from the OpenTofu registry (`registry.opentofu.org`, which serves the same `hashicorp/...` and `kreuzwerker/...` addresses as Terraform's registry) into `.terraform/` and wrote `.terraform.lock.hcl` to pin their versions. Your version numbers will probably be newer than the sample. `fmt` tidies the formatting, `validate` checks the syntax, and `plan` shows what would change without changing anything. `apply` made the API calls: the Docker provider asked the Docker daemon to pull the image and start the container.

4. Check the result and inspect the **state**.

```
curl -s http://localhost:8081 | grep -i '<title>'
cat deployed.txt
tofu state list
tofu state show docker_container.web | head -n 15
ls -l terraform.tfstate
```

```
<title>Welcome to nginx!</title>
nginx is published on port 8081
docker_container.web
docker_image.nginx
local_file.notes
...
```

What just happened: `terraform.tfstate` is a JSON file mapping each resource in your code to the real object's ID. Teams keep state in a shared, locked backend (an S3 bucket, for example), never in Git, because it can contain secrets.

5. Simulate drift (someone changes things by hand) and let OpenTofu repair it.

```
docker rm -f tofu-web
tofu plan
tofu apply -auto-approve
docker ps --filter name=tofu-web
```

```
  # docker_container.web will be created
...
Plan: 1 to add, 0 to change, 0 to destroy.
```

What just happened: `plan` refreshed the state by asking the Docker API what really exists, noticed the container was gone, and planned to recreate only that resource. `-auto-approve` skips the `yes` prompt (handy in pipelines, risky by hand).

6. Destroy everything this configuration created.

```
tofu destroy -auto-approve
docker ps -a --filter name=tofu-web
```

Checkpoint: After `tofu destroy`, `tofu state list` prints nothing, `deployed.txt` is gone, and no `tofu-web` container exists.

Task 8: Orchestrate a service with Docker Swarm

1. Turn ubuntu-lab into a one-node swarm. Use your ubuntu-lab IP address.

```
docker swarm init --advertise-addr 192.168.100.10
docker node ls
```

```
Swarm initialized: current node (k3x9...) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-... 192.168.100.10:2377
...
ID          HOSTNAME     STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
k3x9... *   ubuntu-lab   Ready     Active         Leader           29.1.3
```

What just happened: A swarm is a cluster of Docker hosts called **nodes**. **Managers** keep the cluster state (using the Raft consensus algorithm) and schedule work; **workers** just run containers. The `join` command printed above would add another Docker host as a worker. `docker node ls` works only on a manager.

2. Create a **service** with three replicas. A service is the desired state ("3 copies of nginx on port 8080"); each copy is a **task**, and each task runs one container.

```
docker service create --name web --replicas 3 \
  --publish published=8080,target=80 nginx:alpine
docker service ls
docker service ps web
curl -s http://localhost:8080 | grep -i '<title>'
```

```
ID          NAME   MODE         REPLICAS   IMAGE          PORTS
pq1...      web    replicated   3/3        nginx:alpine   *:8080->80/tcp
ID          NAME    IMAGE          NODE         DESIRED STATE   CURRENT STATE
a1b...      web.1   nginx:alpine   ubuntu-lab   Running         Running 10 seconds ago
c2d...      web.2   nginx:alpine   ubuntu-lab   Running         Running 10 seconds ago
e3f...      web.3   nginx:alpine   ubuntu-lab   Running         Running 10 seconds ago
<title>Welcome to nginx!</title>
```

What just happened: `docker service ps` lists the tasks. Port 8080 is published on the swarm's **routing mesh**: any node accepts the connection and forwards it to a healthy task, even one on another node.

3. **Scale** the service and watch Swarm heal itself.

```
docker service scale web=5
docker service ps web --filter desired-state=running
docker rm -f $(docker ps -q --filter name=web.1)
sleep 5
docker service ps web
```

What just happened: `scale` changed the desired replica count; Swarm started two more tasks. When you killed a container by hand, Swarm saw that only 4 of 5 were running and started a replacement task. The old task shows as `Shutdown` / `Failed` in the history. This self-healing is the heart of orchestration. `$(...)` puts the container ID found by `docker ps -q` into the command.

4. Create an **overlay network** and attach a second service. Overlay networks span all nodes in the swarm, so services on different hosts can talk privately.

```
docker network create --driver overlay labnet
docker service create --name cache --network labnet redis:7-alpine
docker service update --network-add labnet web
docker network ls --filter driver=overlay
```

```
NETWORK ID     NAME      DRIVER    SCOPE
x1y...         ingress   overlay   swarm
z2w...         labnet    overlay   swarm
```

> **What just happened:** `ingress` is the built-in overlay that powers the routing mesh. `labnet` is yours. `service update --network-add` performs a rolling update: Swarm replaces web's tasks one at a time so the service stays available.

5. Remove the services, the network, and the swarm.

```
docker service rm web cache
docker network rm labnet
docker swarm leave --force
docker info --format '{{.Swarm.LocalNodeState}}'
```

```
web
cache
labnet
Node left the swarm.
inactive
```

> **Checkpoint:** `docker info --format '{{.Swarm.LocalNodeState}}'` prints `inactive` and `docker ps` shows no `web` or `cache` containers.

Task 9: Orchestrate with Kubernetes (k3s)

k3s is a small, certified Kubernetes distribution that installs as a single binary. It needs about 2 GB of RAM, which your 4 GB ubuntu-lab has.

1. Install k3s. The options make the kubeconfig file readable by your user (so `kubectl` works without `sudo`) and skip the Traefik ingress controller, which you do not need.

```
curl -sfL https://get.k3s.io | sh -s - --write-kubeconfig-mode 644 --disable traefik
kubectl get nodes
```

```
[INFO]  Finding release for channel stable
[INFO]  Using v1.33.5+k3s1 as release
...
[INFO]  systemd: Starting k3s
NAME         STATUS   ROLES                  AGE   VERSION
ubuntu-lab   Ready    control-plane,master   30s   v1.33.5+k3s1
```

What just happened: The script installed `/usr/local/bin/k3s`, created `kubectl` as a link to it, and started the `k3s` systemd service. `sh -s -` passes the following options to the script. If `kubectl get nodes` says `NotReady`, wait 30 seconds and try again. Your version will be newer or older than the sample.

2. Create a **Deployment**, which keeps a set number of identical **Pods** running. A Pod is the smallest unit Kubernetes runs: one or more containers that share a network address.

```
kubectl create deployment web --image=nginx:alpine --replicas=2
kubectl get deployments
kubectl get pods -o wide
kubectl scale deployment web --replicas=3
kubectl get pods
```

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    2/2     2            2           20s
NAME                   READY   STATUS    RESTARTS   AGE   IP          NODE ...
web-5d8f7c9b6d-4kq2x   1/1     Running   0          20s   10.42.0.9   ubuntu-lab
web-5d8f7c9b6d-xm7rt   1/1     Running   0          20s   10.42.0.10  ubuntu-lab
...
```

What just happened: The Deployment created a ReplicaSet, which created the Pods. Their names add random suffixes. Each Pod gets its own IP on the cluster network. Pods come and go, so you never rely on their IPs. That is what Services are for.

3. Expose the Deployment with a **Service** of type NodePort, which opens a port (30000-32767) on every node.

```
kubectl expose deployment web --type=NodePort --port=80
kubectl get service web
NODEPORT=$(kubectl get service web -o jsonpath='{.spec.ports[0].nodePort}')
curl -s "http://localhost:$NODEPORT" | grep -i '<title>'
```

```
NAME   TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
web    NodePort   10.43.85.120   <none>        80:31742/TCP   3s
<title>Welcome to nginx!</title>
```

What just happened: The Service has a stable virtual IP (`CLUSTER-IP`) and DNS name (`web`) inside the cluster, and load-balances across all Pods with the label `app=web`. `80:31742` means Service port 80 is also reachable on node port 31742. `-o jsonpath` pulls one field out of the API object.

4. Create a **ConfigMap** (plain settings) and a **Secret** (sensitive values), and look at how the Secret is stored.

```
kubectl create configmap app-config \
  --from-literal=APP_COLOR=blue --from-literal=APP_MODE=lab
kubectl create secret generic app-secret --from-literal=DB_PASSWORD='S3cret-Lab'
kubectl get configmap app-config -o yaml | grep -A2 '^data'
kubectl get secret app-secret -o jsonpath='{.data.DB_PASSWORD}'; echo
kubectl get secret app-secret -o jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
```

```
data:
  APP_COLOR: blue
  APP_MODE: lab
UzNjcmV0LUxhYg==
S3cret-Lab
```

> **CAUTION**
>
> Secret values are only **base64-encoded**, not encrypted. Anyone allowed to read Secrets can decode them. Protect them with RBAC (role-based access control), encryption at rest, or an external secret manager, and never commit real Secrets to Git.

5. Write a manifest that uses everything: a PersistentVolumeClaim for storage, and a Pod that receives the ConfigMap and Secret as environment **variables** and the ConfigMap as files in a **volume**. Create `~/labs/4.1/k8s/app.yaml`:

```
mkdir -p ~/labs/4.1/k8s && cd ~/labs/4.1/k8s
cat > app.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path
  resources:
    requests:
      storage: 100Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: app
  labels:
    app: demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c"]
      args:
        - echo "started at $(date) in $APP_MODE mode" >> /data/log.txt;
          echo "color=$APP_COLOR"; sleep 3600
      envFrom:
        - configMapRef:
            name: app-config
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: DB_PASSWORD
      volumeMounts:
        - name: config
          mountPath: /etc/app
          readOnly: true
        - name: data
          mountPath: /data
  volumes:
    - name: config
      configMap:
        name: app-config
    - name: data
      persistentVolumeClaim:
        claimName: app-data
EOF
kubectl apply -f app.yaml
kubectl wait --for=condition=Ready pod/app --timeout=120s
kubectl get pvc,pod
```

```
persistentvolumeclaim/app-data created
pod/app created
pod/app condition met
NAME                              STATUS   VOLUME        CAPACITY   ACCESS MODES ...
persistentvolumeclaim/app-data    Bound    pvc-6c1e...   100Mi      RWO          ...
NAME      READY   STATUS    RESTARTS   AGE
pod/app   1/1     Running   0          12s
```

What just happened: `---` separates two objects in one file. The PVC asks for 100 MiB from k3s's built-in `local-path` storage class, which creates a directory on the node and binds it only once a Pod uses the claim. `kubectl wait` pauses until the Pod is `Ready` (the first run downloads the `busybox` image, so it can take a little while). In the Pod, `envFrom` turns every ConfigMap key into an environment variable, `env.valueFrom.secretKeyRef` injects one Secret key, and `volumes` mounts the ConfigMap as files (one file per key) and the PVC at `/data`. `kubectl apply -f` is declarative: you can apply the same file again and again, which is exactly what GitOps tools do.

6. Inspect the Pod with the everyday troubleshooting commands.

```
kubectl logs app
kubectl exec app -- env | grep -E 'APP_|DB_'
kubectl exec app -- ls /etc/app
kubectl exec app -- cat /etc/app/APP_COLOR; echo
kubectl exec app -- cat /data/log.txt
kubectl describe pod app | tail -n 8
```

```
color=blue
APP_COLOR=blue
APP_MODE=lab
DB_PASSWORD=S3cret-Lab
APP_COLOR
APP_MODE
blue
started at Sat Oct 10 14:02:11 UTC 2026 in lab mode
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  60s   default-scheduler  Successfully assigned default/app ...
  Normal  Pulled     58s   kubelet            Container image "busybox:1.36" ...
  Normal  Started    58s   kubelet            Started container app
```

What just happened: `kubectl logs` shows the container's standard output. `kubectl exec POD -- COMMAND` runs a command inside it (add `-it` and `sh` for an interactive shell). `kubectl describe` shows settings and, at the bottom, **Events**, the first place to look when a Pod is stuck in `Pending`, `ImagePullBackOff`, or `CrashLoopBackOff`.

7. Prove the volume is persistent: delete the Pod, recreate it, and read the log file again.

```
kubectl delete pod app
kubectl apply -f app.yaml
kubectl wait --for=condition=Ready pod/app --timeout=60s
kubectl exec app -- cat /data/log.txt
```

```
started at Sat Oct 10 14:02:11 UTC 2026 in lab mode
started at Sat Oct 10 14:05:40 UTC 2026 in lab mode
```

What just happened: `kubectl delete pod` takes about 30 seconds here: `sleep` ignores the polite stop signal (SIGTERM), so Kubernetes waits for the grace period to end and then kills the container. The container's own filesystem was thrown away with the old Pod, but `/data` lives on the PersistentVolume, so the first line survived and the new Pod added a second line.

What just happened in this task: You used the same declarative idea as Ansible, Puppet, and OpenTofu, but for running applications. You told Kubernetes the desired state (3 web Pods, one Service, one app Pod with its settings and storage), and the control plane keeps working to make reality match. Swarm does the same with simpler objects; Kubernetes adds richer building blocks such as ConfigMaps, Secrets, and PersistentVolumeClaims.

Checkpoint: `kubectl get deployment web` shows `3/3`, `kubectl get svc web` shows a NodePort, and `/data/log.txt` inside the `app` Pod has two lines.

Challenge

CHALLENGE

1. Convert the `webservers.yml` playbook into an Ansible **role** (`ansible-galaxy role init webserver`) and call it from a short playbook. Run it twice to confirm it is still idempotent.
2. Write a Kubernetes Deployment manifest for `web` (instead of `kubectl create deployment`) that sets `replicas: 4`, reads `APP_COLOR` from the ConfigMap, and add it to the `pipeline` Git repo with a CI stage that runs `kubectl apply --dry-run=client -f`.
3. Add a variable to the OpenTofu configuration for the container name, pass it with `tofu apply -var 'container_name=lab-web'`, and explain what `tofu plan` reports when you change it.

Clean up

1. Remove Kubernetes. The uninstall script stops k3s, deletes every Pod, Service, and volume, and removes the binary and `kubectl` link.

```
sudo /usr/local/bin/k3s-uninstall.sh
```

2. Remove leftover Docker images and leave the `docker` group. Docker itself can stay installed.

```
docker swarm leave --force 2>/dev/null || true
docker system prune -a --volumes -f
sudo gpasswd -d "$USER" docker
```

The group change takes effect at your next login. If you want to remove Docker completely, run `sudo apt purge -y docker.io docker-compose-v2 && sudo apt autoremove -y`; later labs install it again when needed.

3. Remove OpenTofu and its apt repository files.

```
sudo apt remove -y tofu
sudo rm -f /etc/apt/sources.list.d/opentofu.list /etc/apt/keyrings/opentofu*.gpg
sudo apt update
```

4. Remove the Ansible web servers. First on rocky-lab:

```
# on rocky-lab
sudo firewall-cmd --permanent --remove-service=http
sudo firewall-cmd --reload
sudo dnf remove -y httpd tree pykickstart
sudo rm -f /etc/httpd/conf.d/security.conf /var/www/html/index.html
sudo rm -f /tmp/webroot-backup.tar.gz
rm -rf ~/labs/4.1
```

Then on ubuntu-lab:

```
sudo apt purge -y nginx nginx-common
sudo rm -f /etc/nginx/conf.d/security.conf /var/www/html/index.html
sudo rm -f /tmp/webroot-backup.tar.gz
rm -rf ~/.ansible/collections/ansible_collections/community/general
```

5. Remove Puppet's demo file, the demo package, Puppet itself, and the cloud image tools. (On Ubuntu 24.04, `puppet` is a small package that pulls in the real one, `puppet-agent`, so remove both.)

```
sudo rm -f /etc/puppet-demo.txt
sudo apt remove -y figlet puppet puppet-agent cloud-image-utils
sudo apt autoremove -y
```

6. Delete the lab directory.

```
rm -rf ~/labs/4.1
```

Exam corner

- **IaC** = infrastructure described in version-controlled text files; declarative tools are **idempotent** (running twice changes nothing the second time).
- **Ansible**: agentless, push over SSH, needs only Python on targets; **inventory** lists hosts and groups; **playbooks** are YAML; **modules** do the work; **ad hoc** = `ansible <hosts> -m <module> -a <args>`; **facts** come from the `setup` module; **collections** install with `ansible-galaxy collection install`.

- Ansible handlers run once at the end of a play, only when notified. `--check` is a dry run, `--diff` shows changes.
- **Puppet**: agent pulls its catalog from the server every 30 minutes; trust uses TLS **certificates** signed with `puppetserver ca sign`; code is in manifests (`.pp`) with **classes** inside **modules**; facts come from `facter`; `puppet apply` runs without a server.
- **OpenTofu** (Terraform fork): **providers** call platform **APIs**; **resources** are managed objects; **state** (`terraform.tfstate`) maps code to real objects. Workflow: `init`, `plan`, `apply`, `destroy`.
- **Kickstart**: RHEL unattended install file; sample at `/root/anaconda-ks.cfg`; boot option `inst.ks=`; validate with `ksvalidator`.
- **cloud-init**: first-boot config; user-data starts with `#cloud-config`; logs in `/var/log/cloud-init.log`; check with `cloud-init status`; NoCloud seed volume label `cidata`.
- **CI/CD pipeline** stages: build, lint, test, security scan, deploy. **Shift left** = test earlier. **DevSecOps** = security inside the pipeline. **GitOps** = Git is the source of truth and an agent syncs the cluster to it.
- **Kubernetes**: Pod (smallest unit), Deployment (keeps N Pods, rolling updates), Service (stable IP/DNS, ClusterIP/NodePort/LoadBalancer), ConfigMap (settings), Secret (base64, not encrypted), volumes/PVCs (storage), env variables from ConfigMaps/Secrets.
- **Docker Swarm**: `docker swarm init`, nodes are managers or workers, a service has tasks (one container each), overlay networks span nodes, `docker service scale web=5`.
- **Compose**: `compose.yaml`, `docker compose up -d`, `ps`, `logs -f`, `down` (add `-v` to delete volumes); `podman compose` / `podman-compose` read the same file.

Review questions

1. Which statement best describes why Ansible is called agentless?
   A. It uses a pull model over HTTPS
   B. It needs only SSH and Python on the managed host
   C. It runs only on the local machine
   D. It uses certificates instead of SSH keys

2. In Puppet, what must happen before a new agent can receive its catalog from the server?

3. Which OpenTofu file records the mapping between your configuration and the real objects that were created?
   A. `main.tf`
   B. `.terraform.lock.hcl`
   C. `terraform.tfstate`
   D. `variables.tf`

4. A RHEL installer should read its answers from a web server. Which boot option do you add?

5. In Docker Swarm, what is the relationship between a service and a task?

6. A developer adds a ShellCheck step that runs in a Git pre-commit hook instead of only on the build server. Which practice does this demonstrate?
   A. GitOps

B. Shift left testing
C. Blue-green deployment
D. Agentless orchestration

Lab 4.2 Automate Tasks with Shell Scripts

Objective: 4.2 Given a scenario, perform automated tasks using shell scripting.
Domain: 4.0 Automation, Orchestration, and Scripting (17% of the exam)
Time: about 2.5 hours
VM: ubuntu-lab
Snapshot first: Recommended. The lab creates and then deletes three test users and a cron job.

A shell script is a text file full of the same commands you type at the prompt, plus logic: variables, tests, loops, and functions. In this lab you learn each building block with small experiments, then put them together into a real admin tool that reads a CSV file of new employees, validates every row, creates the accounts, and reports on them. You finish by checking the script with ShellCheck and scheduling it with cron, which is exactly how scripts are used on the job.

What you'll practice

- The shebang line `#!` and running scripts
- Variables: environmental, arguments, assignments (`alias`, `export`, `local`, `set`, `unalias`, `unset`), return codes `$?`
- Expansion: parameter expansion `${var}`, command substitution `$(foo)` and `` `foo` ``, subshell `(foo)`
- `IFS` (input field separator) and `OFS` (awk's output field separator)
- Test: `!`, `-d`, `-f`, `-n`, `-z`
- Comparisons: numerical `-eq -ge -gt -le -lt -ne`; string `>`, `<`, `==`, `=`, `=~`, `!=`
- Regular expressions: `[[ $foo =~ regex ]]`
- Conditionals: `if`, `case`
- Loops: `for`, `while`, `until`
- Functions

How it works

When you run a script, the kernel reads the first line. If it starts with `#!` (called the **shebang**), the rest of the line names the interpreter to run the file with, such as `#!/bin/bash` or `#!/usr/bin/env bash` (which finds `bash` in your `PATH`). Without a shebang, your current shell guesses, which can go wrong. The file also needs execute permission (`chmod +x`) to be run as `./script.sh`; otherwise you can still run it with `bash script.sh`.

Bash reads each line, then performs **expansions** before running it. Think of expansion as filling in blanks on a form before handing it in:

```
you type:    echo "Hi $USER, kernel $(uname -r), file ${f%.txt}.bak"
                   |              |                 |
expansion:   variable / parameter   command          parameter expansion
                   |              substitution      (trim .txt)
                   v              v                 v
bash runs:   echo "Hi student, kernel 6.8.0-45-generic, file notes.bak"
```

- **Parameter expansion** (`$var`, `${var}`, and forms like `${var:-default}`) inserts and transforms variable values.
- **Command substitution** (`$(command)` or the older `` `command` ``) runs a command and inserts its output.
- **Arithmetic expansion** (`$(( 2 + 3 ))`) does integer math.
- **Word splitting** cuts unquoted results into separate words using the characters in **IFS** (by default: space, tab, newline). This is why you should almost always put variables in double quotes: `"$var"`.

Every command finishes with an **exit status** (also called a **return code**), a number from 0 to 255. **0 means success**; anything else means some kind of failure. The special variable `$?` holds the status of the last command. `if`, `while`, `until`, `&&`, and `||` all make decisions based on exit status, not on output. Even `[[ ... ]]` is just a command that returns 0 (true) or 1 (false).

Variables come in two kinds. A **shell variable** exists only in the current shell. An **environment variable** has been exported, so child processes (programs and scripts you start) get a copy. A child can never change its parent's variables. That is also why a **subshell**, a child copy of the shell started with `( ... )`, cannot change variables or the current directory of the shell that started it. Inside functions, `local` keeps a variable private to that function.

Scripts get input from **arguments**: `$1`, `$2`, … are the positional parameters, `$#` is how many there are, `$@` is all of them as separate words, and `$0` is the script's own name.

Finally, scripts make decisions with tests. `[ ... ]` (the same as the `test` command) is the old POSIX form. `[[ ... ]]` is Bash's improved version: it does not word-split variables, supports `&&` and `||` inside, pattern matching with `==`, and **regular expressions** with `=~`. Use `[[ ]]` in Bash scripts. Numbers are compared with `-eq`, `-lt`, and friends (or inside `(( ))`), while `==`, `<`, and `>` inside `[[ ]]` compare **strings** by sort order. Mixing these up is one of the most common scripting bugs.

Tasks

Task 1: Your first script, variables, and return codes

1. Create the lab directory and install ShellCheck, a linter that finds bugs in shell scripts.

```
mkdir -p ~/labs/4.2 && cd ~/labs/4.2
sudo apt install -y shellcheck
bash --version | head -n 1
```

```
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
```

2. Write a script that uses a shebang, arguments, and command substitution. You will create files with heredocs in this lab: `cat > file <<'EOF'` sends every following line into `file` until a line containing only `EOF`. The quotes around `'EOF'` stop the shell from expanding `$` while writing the file, so the script keeps its `$` signs. (You can also open the file in `nano hello.sh` and type the lines.)

```
cat > hello.sh <<'EOF'
#!/bin/bash
# hello.sh - my first script
echo "Hello, $USER. Today is $(date +%A)."
echo "This script is called: $0"
echo "You gave me $# arguments: $*"
echo "The first argument is: ${1:-<none>}"
exit 0
EOF
chmod +x hello.sh
./hello.sh one "two words" three
./hello.sh
```

```
Hello, student. Today is Saturday.
This script is called: ./hello.sh
You gave me 3 arguments: one two words three
The first argument is: one
Hello, student. Today is Saturday.
This script is called: ./hello.sh
You gave me 0 arguments:
The first argument is: <none>
```

What just happened: `#!/bin/bash` tells the kernel to run the file with Bash. Lines starting with `#` are comments. `$USER` is an environment variable set at login. `$(date +%A)` is command substitution. `$0` is the script name, `$#` the number of arguments, and `$*` all arguments joined into one string. Quotes made `"two words"` a single argument, so `$#` is 3. `${1:-<none>}` is parameter expansion: use `$1`, or `<none>` if it is empty or unset. `exit 0` ends the script with a success return code.

3. Look at return codes. Run each line and read the number `$?` prints.

```
ls /etc/hostname; echo "exit status: $?"
ls /no/such/file; echo "exit status: $?"
grep -q '^root:' /etc/passwd; echo "exit status: $?"
bash -c 'exit 42'; echo "exit status: $?"
nosuchcommand; echo "exit status: $?"
```

```
/etc/hostname
exit status: 0
ls: cannot access '/no/such/file': No such file or directory
exit status: 2
exit status: 0
exit status: 42
nosuchcommand: command not found
exit status: 127
```

What just happened: `$?` always holds the return code of the most recent command, so read it immediately (even `echo` resets it). 0 is success. Each program documents its own failure codes (see the EXIT STATUS section of its man page); `127` is Bash's code for "command not found" and `126` for "found but not executable". `grep -q` prints nothing and only returns 0 (match) or 1 (no match), which is perfect for scripts.

4. Practice **assignments** and the difference between shell and **environment** variables.

```
greeting="Hello"
echo "$greeting"
greeting = "Hello"
color=blue
bash -c 'echo "child sees: [$color]"'
export color
bash -c 'echo "child sees: [$color]"'
env | grep '^color='
set | grep '^color='
unset color
echo "after unset: [$color]"
```

```
Hello
greeting: command not found
child sees: []
child sees: [blue]
color=blue
color=blue
after unset: []
```

What just happened: An assignment is `name=value` with **no spaces** around `=`. With spaces, Bash thinks `greeting` is a command. `bash -c '...'` starts a child shell: before `export`, the child does not see `color`; after `export`, it does. `env` (or `printenv`) lists only environment variables; `set` with no arguments lists all shell variables *and* functions. `unset` deletes a variable. You can also export in one step: `export EDITOR=nano`.

TIP

`set` has a second job: it turns shell options on and off. `set -x` traces commands, `set -e` exits on errors, and `set -- a b c` replaces the positional parameters `$1 $2 $3`. You will use all three later.

5. Create and remove an **alias**, then see how aliases behave inside scripts.

```
alias today='date +%F'
today
type today
unalias today
today
cat > alias-demo.sh <<'EOF'
#!/bin/bash
# alias-demo.sh - aliases are off in scripts unless you turn them on
alias today='date +%F'
today
shopt -s expand_aliases
alias today='date +%F'
today
EOF
bash alias-demo.sh
```

```
2026-10-10
today is aliased to `date +%F'
today: command not found
alias-demo.sh: line 4: today: command not found
2026-10-10
```

What just happened: An alias is a text shortcut that Bash substitutes for the first word of a command. `type` shows what a name really is. `unalias` removes it. Non-interactive shells (scripts) ignore aliases unless you run `shopt -s expand_aliases`, and an alias only works on lines read after it is defined. Aliases you put in `~/.bashrc` are for your interactive typing. In scripts, use a **function** instead; it is clearer and always works.

Checkpoint: `./hello.sh a b; echo $?` prints `You gave me 2 arguments: a b` and then `0`, and `bash -c 'echo $HOME'` prints your home directory (because `HOME` is an environment variable).

Task 2: Expansions, subshells, IFS, and OFS

1. Try the most useful forms of **parameter expansion**. Type each line and compare with the output.

```
file="/var/log/nginx/access.log.1"
echo "${file}"
echo "${#file}"
echo "${file##*/}"
echo "${file%/*}"
echo "${file%.*}"
echo "${file%%.*}"
echo "${file/log/LOG}"
echo "${file//log/LOG}"
echo "${file:5:3}"
```

```
/var/log/nginx/access.log.1
27
access.log.1
/var/log/nginx
/var/log/nginx/access.log
/var/log/nginx/access
/var/LOG/nginx/access.log.1
/var/LOG/nginx/access.LOG.1
log
```

What just happened: Braces `${...}` mark exactly where the variable name ends and allow transformations:

Form	Meaning
`${#var}`	length in characters
`${var#pat}` / `${var##pat}`	remove shortest / longest match of `pat` from the **front**
`${var%pat}` / `${var%%pat}`	remove shortest / longest match from the **end**
`${var/a/b}` / `${var//a/b}`	replace first / every `a` with `b`
`${var:offset:length}`	substring (offset starts at 0)

So `${file##*/}` works like `basename` and `${file%/*}` like `dirname`, without starting a new process. A memory trick: `#` is left of `$` on a US keyboard (front), `%` is right of it (end).

2. Handle empty or missing values with default-value expansions.

```
unset name
echo "${name:-guest}"
echo "[$name]"
echo "${name:=guest}"
echo "[$name]"
echo "${name^^}"
( echo "${missing:?is not set}" ); echo "status: $?"
```

```
guest
[]
guest
[guest]
GUEST
bash: missing: is not set
status: 1
```

What just happened: `${name:-guest}` *uses* a default but leaves `name` unset. `${name:=guest}` uses the default *and assigns it.* `${name^^}` converts to upper case (`${name,,}` for lower). `${missing:?message}` prints an error and, in a script, exits immediately; it is a quick way to require an argument, as in `path="${1:?Usage: script PATH}"`. It ran inside `( )` here so it would not affect your login shell.

3. Use both forms of **command substitution** and arithmetic.

```
kernel=$(uname -r)
logged_in=`who | wc -l`
echo "Kernel $kernel, $logged_in users logged in"
echo "Config dir: $(basename "$(dirname /etc/ssh/sshd_config)")"
echo "Next year is $(( $(date +%Y) + 1 ))"
```

```
Kernel 6.8.0-45-generic, 1 users logged in
Config dir: ssh
Next year is 2027
```

What just happened: `$(...)` and backticks both run a command and insert its output (trailing newlines removed). Prefer `$(...)`: it nests cleanly, as the `basename "$(dirname ...)"` line shows, and is easier to read. Backticks still appear in old scripts and on the exam, so recognize them. `$(( ... ))` does integer arithmetic.

4. See how a **subshell** isolates changes, compared with a `{ }` group that runs in the current shell.

```
cd /tmp
count=1
( cd /etc; count=99; echo "inside ( ): PWD=$PWD count=$count" )
echo "after ( ):  PWD=$PWD count=$count"
{ cd /etc; count=99; echo "inside { }: PWD=$PWD count=$count"; }
echo "after { }:  PWD=$PWD count=$count"
echo "hello" | read -r word
echo "word=[$word]"
cd ~/labs/4.2
```

```
inside ( ): PWD=/etc count=99
after ( ):  PWD=/tmp count=1
inside { }: PWD=/etc count=99
after { }:  PWD=/etc count=99
word=[]
```

What just happened: `( ... )` ran the commands in a child copy of the shell. Its `cd` and assignment vanished when it exited. `{ ...; }` only groups commands in the *current* shell (it needs spaces inside the braces and a `;` before `}`), so its changes stuck. The `read` example is a classic trap: each part of a pipeline runs in its own subshell, so `word` was set in a child that immediately exited. Use `read -r word <<< "hello"` or `while ... done < file` instead.

5. Control word splitting with **IFS** (input field separator).

```
printf '%q\n' "$IFS"
line="jdoe,John Doe,sales,/bin/bash"
IFS=, read -r user full dept shell <<< "$line"
echo "user=$user | full=$full | dept=$dept | shell=$shell"
set -- one two three
IFS=,
echo "$*"
unset IFS
echo "$*"
```

```
$' \t\n'
user=jdoe | full=John Doe | dept=sales | shell=/bin/bash
one,two,three
one two three
```

What just happened: The default `IFS` is space, tab, and newline (`printf %q` shows invisible characters). Writing `IFS=, read ...` sets IFS **only for that one command**, so `read` split the line at commas and `John Doe` stayed together. `<<<` is a here-string: it feeds a string to a command's standard input. `"$*"` joins the positional parameters with the first character of IFS. `unset IFS` restores the default behavior. Changing IFS globally is risky, so prefer the one-command form.

6. Control output with **OFS**, awk's output field separator. awk is often used inside shell scripts to reformat data.

```
awk -F: -v OFS=' | ' '$3 >= 1000 && $3 < 65534 { print $1, $6, $7 }' /etc/passwd
echo "a b c" | awk -v OFS=, '{ $1 = $1; print }'
```

```
student | /home/student | /bin/bash
a,b,c
```

What just happened: `-F:` sets awk's *input* field separator to `:`. `-v OFS=' | '` sets the *output* field separator, which awk places between items separated by commas in `print`. The condition picks regular users (UID 1000 to 65533). In the second example, `$1 = $1` forces awk to rebuild the line using OFS, turning spaces into commas.

Checkpoint: Without looking back, predict and then check the output of `f=/home/student/report.final.csv; echo "${f##*/} ${f%.*} ${#f}"`. You should get `report.final.csv /home/student/report.final 30`.

Task 3: Tests and comparisons

1. Use the **test** operators for files and strings. `test EXPR`, `[ EXPR ]`, and `[[ EXPR ]]` all return 0 when the expression is true.

```
test -d /etc && echo "/etc is a directory"
[ -f /etc/hostname ] && echo "/etc/hostname is a regular file"
[[ -f /etc/nope ]] || echo "/etc/nope does not exist"
[[ ! -d /etc/hostname ]] && echo "/etc/hostname is NOT a directory"
empty=""
word="hi"
[[ -z $empty ]] && echo "empty has zero length"
[[ -n $word ]] && echo "word is not empty"
```

```
/etc is a directory
/etc/hostname is a regular file
/etc/nope does not exist
/etc/hostname is NOT a directory
empty has zero length
word is not empty
```

What just happened: `-d` is true for a directory, `-f` for a regular file, `-z` for a zero-length string, and `-n` for a non-empty string. `!` reverses (negates) any test. `cmd1 && cmd2` runs `cmd2` only if `cmd1` succeeded; `cmd1 || cmd2` runs `cmd2` only if `cmd1` failed. Other common file tests: `-e` exists, `-r` readable, `-w` writable, `-x` executable, `-s` not empty, `-L` symbolic link.

CAUTION

With single brackets, always quote variables: `[ -n "$var" ]`. If `var` is empty and unquoted, the test becomes `[ -n ]`, which is true. `[[ ]]` does not have this problem, but quoting is still a good habit.

2. Compare **numbers** with the numeric operators.

```
a=7
b=10
[[ $a -eq 7 ]] && echo "a -eq 7"
[[ $a -ne $b ]] && echo "a -ne b"
[[ $a -lt $b ]] && echo "a -lt b"
[[ $a -le 7 ]] && echo "a -le 7"
[[ $b -gt $a ]] && echo "b -gt a"
[[ $b -ge 10 ]] && echo "b -ge 10"
(( a <= b )) && echo "(( a <= b )) is true"
```

What just happened: All six lines print. `-eq` equal, `-ne` not equal, `-lt` less than, `-le` less or equal, `-gt` greater than, `-ge` greater or equal. Inside `(( ))`, Bash does arithmetic and you may use the familiar `<`, `<=`, `>`, `>=`, `==`, `!=` symbols on numbers, without `$` in front of variable names.

3. Compare **strings**. Here `==`, `!=`, `<`, and `>` mean text equality and alphabetical (sort) order.

```
[[ "apple" == "apple" ]] && echo "== equal"
[ "apple" = "apple" ] && echo "= equal (POSIX)"
[[ "apple" != "pear" ]] && echo "!= different"
[[ "apple" < "banana" ]] && echo "apple < banana"
[[ "pear" > "banana" ]] && echo "pear > banana"
[[ "report.txt" == *.txt ]] && echo "pattern match"
[[ 10 < 9 ]] && echo "as strings, 10 sorts before 9"
[[ 10 -lt 9 ]] || echo "as numbers, 10 is not less than 9"
```

What just happened: `=` is the POSIX string-equality operator (works in `[ ]` and `[[ ]]`); `==` is the Bash form. In `[[ ]]`, an unquoted right side of `==` is a glob pattern, so `*.txt` matched. `<` and `>` compare sort order, character by character, which is why the *string* `"10"` comes before `"9"` (because `1` sorts before `9`). For numbers, always use `-lt` and friends or `(( ))`.

4. Find out what happens with `<=` and `>=` on strings, and learn the alternatives.

```
bash -c '[[ "a" <= "b" ]] && echo yes'
[ "a" < "b" ]
x="apple"
y="banana"
! [[ $x > $y ]] && echo "$x sorts before or equal to $y"
[[ $x < $y || $x == $y ]] && echo "same test, written out"
printf '%s\n' "$x" "$y" | sort -C && echo "sort -C agrees: already in order"
[ "$x" \< "$y" ] && echo "single brackets need an escaped \<"
```

```
bash: -c: line 1: syntax error in conditional expression
bash: -c: line 1: syntax error near `"b"'
bash: -c: line 1: `[[ "a" <= "b" ]] && echo yes'
bash: b: No such file or directory
apple sorts before or equal to banana
same test, written out
sort -C agrees: already in order
single brackets need an escaped \<
```

What just happened: `<=` and `>=` are **not valid string operators** in `[[ ]]`; Bash reports a syntax error. (The test ran in a child shell with `bash -c` because a syntax error in a script stops the whole script.) They only work as arithmetic operators inside `(( ))`. To test "x sorts before or equal to y", negate the opposite test (`! [[ $x > $y ]]`) or combine `<` with `==`. `sort -C` returns 0 if its input is already sorted. In single brackets, a bare `<` is a **redirection**: `[ "a" < "b" ]` tried to read from a file named `b`. Escape it as `\<` there.

5. Match a **regular expression** with `=~`. A regular expression (regex) describes a text pattern. Bash uses POSIX extended regex, like `grep -E`.

```
ip="192.168.100.10"
re='^([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$'
if [[ $ip =~ $re ]]; then
    echo "IPv4-like; first octet ${BASH_REMATCH[1]}, last ${BASH_REMATCH[4]}"
fi
[[ $ip =~ "^192" ]] || echo "quoted pattern is a literal string, not a regex"
[[ $ip =~ ^192\. ]] && echo "unquoted regex matches"
```

```
IPv4-like; first octet 192, last 10
quoted pattern is a literal string, not a regex
unquoted regex matches
```

What just happened: `^` anchors the start and `$` the end, `[0-9]{1,3}` means one to three digits, `\.` is a literal dot, and `( )` creates capture groups. After a successful match, Bash stores the whole match in `${BASH_REMATCH[0]}` and each group in `${BASH_REMATCH[1]}` and up. **Do not quote the regex**: quoting makes it a plain string. Storing the regex in a variable (`re='...'`) avoids quoting problems with special characters. Note this pattern still accepts `999.1.1.1` ; checking each octet is 255 or less needs extra code.

Checkpoint: `[[ 9 -lt 10 ]] && echo num; [[ 9 < 10 ]] || echo str` prints both `num` and `str` . If you understand why, you have the most important comparison rule.

Task 4: Make decisions with if and case

1. Write a script that uses `if` , `elif` , and `else` with file tests, and requires an argument.

```
cat > checkfile.sh <<'EOF'
#!/bin/bash
# checkfile.sh - describe the path given as the first argument
path="${1:?Usage: checkfile.sh PATH}"

if [[ -L $path ]]; then
    echo "$path is a symbolic link to $(readlink "$path")"
elif [[ -d $path ]]; then
    echo "$path is a directory with $(find "$path" -mindepth 1 -maxdepth 1 | wc -l) entries"
elif [[ -f $path ]]; then
    echo "$path is a regular file of $(stat -c %s "$path") bytes"
else
    echo "$path does not exist" >&2
    exit 1
fi
EOF
chmod +x checkfile.sh
for p in /etc /etc/hostname /bin /nope; do ./checkfile.sh "$p"; echo "  -> status $?"; done
./checkfile.sh
```

```
/etc is a directory with 205 entries
  -> status 0
/etc/hostname is a regular file of 11 bytes
  -> status 0
/bin is a symbolic link to usr/bin
  -> status 0
/nope does not exist
  -> status 1
./checkfile.sh: line 3: 1: Usage: checkfile.sh PATH
```

What just happened: `if` runs a command (here a `[[ ]]` test) and branches on its return code. Bash checks each `elif` in order and runs the first true branch; `else` catches the rest; `fi` ("if" backwards) closes the block. `-L` is tested first because `-d` and `-f` follow symbolic links (on Ubuntu `/bin` is a link to `/usr/bin`). `>&2` sends the error message to standard error, and `exit 1` returns a failure code to the caller.

2. Write a menu with `case`. `case` compares one value against several patterns, which is cleaner than a long `if/elif` chain.

```
cat > menu.sh <<'EOF'
#!/bin/bash
# menu.sh - a tiny admin menu
while true; do
    echo "1) Disk usage  2) Memory  3) Who is logged in  q) Quit"
    read -rp "Choose: " choice
    case "$choice" in
        1) df -h / ;;
        2) free -h ;;
        3) who ;;
        [4-9]) echo "Option $choice is not used yet" ;;
        q|Q|quit) echo "Bye"; break ;;
        *) echo "Unknown choice: '$choice'" ;;
    esac
done
EOF
chmod +x menu.sh
./menu.sh
```

Try `1`, `2`, `7`, `hello`, and finally `q`.

What just happened: `read -rp "Choose: " choice` shows a prompt and stores what you type (`-r` keeps backslashes literal). Each `case` branch is `pattern) commands ;;`. Patterns are globs: `[4-9]` matches one digit from 4 to 9, `q|Q|quit` matches any of three words, and `*` matches anything (the default branch, always last). `esac` closes the block. `break` leaves the `while true` loop.

Checkpoint: `printf '7\nq\n' | ./menu.sh` prints `Option 7 is not used yet` and then `Bye`.

Task 5: Repeat work with for, while, and until loops

1. Create a sample CSV file of new employees. You will use it for the rest of the lab. Notice the problems hidden in it.

```
cat > users.csv <<'EOF'
username,full_name,department,shell
jdoe,John Doe,sales,/bin/bash
asmith,Alice Smith,it,/bin/bash
Bad User,Bad Entry,it,/bin/bash
bwayne,Bruce Wayne,ops,/bin/zsh
mlee,Mary Lee,it,
EOF
```

> **What just happened:** The file `users.csv` has a header line and five rows. `Bad User` is not a valid Linux user name, `/bin/zsh` is not installed on Ubuntu Server by default, and `mlee` has no shell (the field is empty).

2. Use `for` loops: over a list of files, over a range, and C-style.

```
for f in /etc/*.conf; do
    [[ -f $f ]] || continue
    printf '%-28s %6s bytes\n' "$f" "$(stat -c %s "$f")"
done | head -n 4
for n in {1..3}; do echo "brace pass $n"; done
for ((i = 1; i <= 3; i++)); do echo "C-style pass $i"; done
```

```
/etc/adduser.conf              3444 bytes
/etc/ca-certificates.conf      5287 bytes
/etc/debconf.conf              2969 bytes
/etc/deluser.conf              1706 bytes
brace pass 1
...
C-style pass 3
```

> **What just happened:** `for VAR in LIST; do ...; done` runs the body once per item. The glob `/etc/*.conf` expands to a list of file names. `continue` skips to the next item (here, anything that is not a regular file); `break` would leave the loop. The whole loop's output was piped into `head`. `{1..3}` is brace expansion, and `for ((...))` is the C-style counting form.

3. Use `while read` to process the CSV file line by line, with IFS set to a comma.

```
while IFS=, read -r name full dept shell; do
    echo "$name ($full) works in $dept and uses ${shell:-the default shell}"
done < <(tail -n +2 users.csv)
```

```
jdoe (John Doe) works in sales and uses /bin/bash
asmith (Alice Smith) works in it and uses /bin/bash
Bad User (Bad Entry) works in it and uses /bin/bash
bwayne (Bruce Wayne) works in ops and uses /bin/zsh
mlee (Mary Lee) works in it and uses the default shell
```

What just happened: `while` repeats as long as its command (`read`) succeeds; `read` fails at end of file, which ends the loop. `IFS=,` splits each line at commas into the four variables. `tail -n +2` prints from line 2 on (skipping the header). `< <(command)` is process substitution: it feeds the command's output into the loop *without* a pipe, so the loop runs in the current shell and any variables it sets survive afterwards.

4. Use `until` to wait for something to happen. `until` is the opposite of `while` : it repeats as long as its command **fails**.

```
rm -f ready.flag
( sleep 3; touch ready.flag ) &
tries=0
until [[ -f ready.flag ]]; do
    tries=$((tries + 1))
    echo "Waiting for ready.flag (check $tries)..."
    sleep 1
done
echo "Found ready.flag after $tries checks"
```

```
Waiting for ready.flag (check 1)...
Waiting for ready.flag (check 2)...
Waiting for ready.flag (check 3)...
Found ready.flag after 3 checks
```

What just happened: The subshell `( sleep 3; touch ready.flag ) &` ran in the background (`&`) and created the file after 3 seconds, like a service finishing its startup. The `until` loop checked once per second until the test became true. Real scripts use this pattern to wait for a port to open, a host to answer `ping` , or a lock file to disappear, usually with a maximum number of tries so they cannot loop forever.

Checkpoint: Write a one-line loop that prints only the user names from `users.csv` (skip the header). One answer: `tail -n +2 users.csv | while IFS=, read -r u _; do echo "$u"; done` .

Task 6: Functions, local variables, and return codes

1. Write a script with several functions. Read it carefully before running it.

```
cat > functions.sh <<'EOF'
#!/bin/bash
# functions.sh - arguments, local variables, and return codes
counter=0

is_even() {
    local n="$1"
    (( n % 2 == 0 ))
}

bump() {
    local step="${1:-1}"
    counter=$((counter + step))
}

scope_demo() {
    local counter=100
    echo "inside scope_demo: counter=$counter"
}

retry() {
    local max="$1"
    shift
    local attempt=1
    until "$@"; do
        if (( attempt >= max )); then
            echo "Giving up after $attempt attempts: $*" >&2
            return 1
        fi
        echo "Attempt $attempt failed; retrying..." >&2
        attempt=$((attempt + 1))
        sleep 1
    done
}

for n in 3 8; do
    if is_even "$n"; then
        echo "$n is even"
    else
        echo "$n is odd"
    fi
done
is_even 7
echo "is_even 7 returned $?"

bump
bump 5
echo "global counter=$counter"
scope_demo
echo "after scope_demo: counter=$counter"

retry 3 test -e /etc/hostname && echo "found /etc/hostname"
retry 3 test -e /no/such/file
echo "retry returned $?"
```

```
EOF
chmod +x functions.sh
./functions.sh
```

```
3 is odd
8 is even
is_even 7 returned 1
global counter=6
inside scope_demo: counter=100
after scope_demo: counter=6
found /etc/hostname
Attempt 1 failed; retrying...
Attempt 2 failed; retrying...
Giving up after 3 attempts: test -e /no/such/file
retry returned 1
```

What just happened: A **function** is a named group of commands, defined with `name() { ...; }` before it is used. Inside a function, `$1`, `$2`, `$@` are the *function's* arguments, not the script's. Key points:

- `is_even` returns the status of its last command. `(( n % 2 == 0 ))` returns 0 (true) when the remainder is zero, so the function can be used directly in `if`.
- `bump` changes the **global** `counter` because it does not declare it `local`.
- `scope_demo` declares `local counter=100`, which hides the global variable only inside the function. The global stayed 6.
- `retry` takes a maximum and a command. `shift` drops the first argument, so `"$@"` becomes the command to run. It uses `until` to retry and `return 1` to report failure.

`return N` sets a function's return code (0 to 255). To send data back, `echo` it and capture it with `result=$(myfunc)`.

2. Turn on strict mode and see what each option does. Many professional scripts start with `set -euo pipefail`.

```
bash -c 'set -e; false; echo "not printed"'; echo "status: $?"
bash -c 'set -u; echo "value: $undefined_var"'; echo "status: $?"
bash -c 'false | true'; echo "status: $?"
bash -c 'set -o pipefail; false | true'; echo "status: $?"
```

```
status: 1
bash: line 1: undefined_var: unbound variable
status: 127
status: 0
status: 1
```

What just happened: `set -e` exits the script as soon as a command fails (except in tests such as `if`, `while`, `&&`, `||`). `set -u` treats use of an unset variable as an error, which catches typos like `$USRE`. Normally a pipeline's status is that of its *last* command, hiding earlier failures; `set -o pipefail` makes the pipeline fail if *any* part fails. Together they make a script stop instead of continuing in a broken state.

> **CAUTION**
> With `set -e`, the arithmetic command `((count++))` fails when `count` is 0, because the expression's old value 0 counts as "false". Write `count=$((count + 1))` instead, as this lab does.

Checkpoint: Add `echo "script args: $*"` near the top of `functions.sh` and run `./functions.sh x y`. The script sees `x y`, but inside the functions `$1` is still each function's own argument.

Task 7: Build the user-provisioning tool

Now combine everything into a tool an admin could really use. It reads `users.csv`, validates each row with a regular expression and a file test, and has four actions selected with `case`: `check`, `add`, `report`, and `remove`.

1. Create `~/labs/4.2/provision-users.sh`:

```
cat > provision-users.sh <<'EOF'
#!/usr/bin/env bash
# provision-users.sh - check, add, report, or remove lab users listed in a CSV file
# Usage: provision-users.sh {check|add|report|remove} [csv-file]

set -euo pipefail

readonly GROUP="labusers"
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_DIR
readonly LOG="${LOG_FILE:-$SCRIPT_DIR/provision.log}"
readonly NAME_RE='^[a-z_][a-z0-9_-]{0,31}$'
INVALID=0                                   # global counter of bad CSV rows

log() {
    local msg="$1"
    printf '%s %s\n' "$(date '+%F %T')" "$msg" | tee -a "$LOG"
}

die() {
    echo "ERROR: $*" >&2
    exit 1
}

need_root() {
    if [[ $EUID -ne 0 ]]; then
        die "the '$1' action must be run with sudo"
    fi
}

valid_user() {
    local name="$1" shell="$2"
    if [[ ! $name =~ $NAME_RE ]]; then
        log "SKIP '$name': invalid username"
        return 1
    fi
    if [[ ! -x $shell ]]; then
        log "SKIP '$name': shell $shell is not installed"
        return 2
    fi
    return 0
}

user_exists() {
    id "$1" &>/dev/null
}

add_user() {
    local name="$1" full="$2" shell="$3"
    if user_exists "$name"; then
        log "OK   '$name' already exists"
        return 0
    fi
    useradd -m -c "$full" -s "$shell" -G "$GROUP" "$name"
    log "ADD  '$name' ($full, $shell)"
```

```
}

remove_user() {
    local name="$1"
    if user_exists "$name"; then
        userdel -r "$name" 2>/dev/null
        log "DEL  '$name'"
    fi
}

process_csv() {
    local action="$1" file="$2"
    local name full dept shell
    local line=0
    while IFS=, read -r name full dept shell || [[ -n $name ]]; do
        line=$((line + 1))
        if [[ $line -eq 1 || -z $name ]]; then
            continue                      # skip the header and blank lines
        fi
        shell="${shell%$'\r'}"            # strip a Windows line ending
        shell="${shell:-/bin/bash}"       # default shell when the field is empty
        if ! valid_user "$name" "$shell"; then
            INVALID=$((INVALID + 1))
            continue
        fi
        case "$action" in
            check)  log "VALID '$name' ($dept)" ;;
            add)    add_user "$name" "$full" "$shell" ;;
            remove) remove_user "$name" ;;
        esac
    done < "$file"
    log "Processed $((line - 1)) rows from ${file##*/}, $INVALID invalid"
}

report() {
    local members user
    members=$(getent group "$GROUP" | cut -d: -f4)
    if [[ -z $members ]]; then
        echo "No members in group $GROUP"
        return 0
    fi
    echo "Members of $GROUP on $(hostname) at $(date '+%F %R'):"
    for user in ${members//,/ }; do
        getent passwd "$user"
    done | awk -F: -v OFS=' | ' '{ print $1, $3, $5, $7 }'
}

main() {
    local action="${1:-}"
    local csv="${2:-users.csv}"

    case "$action" in
        check)
            [[ -f $csv ]] || die "CSV file '$csv' not found"
            process_csv check "$csv"
```

```
            ;;
        add)
            need_root add
            [[ -f $csv ]] || die "CSV file '$csv' not found"
            getent group "$GROUP" >/dev/null || groupadd "$GROUP"
            process_csv add "$csv"
            ;;
        remove)
            need_root remove
            [[ -f $csv ]] || die "CSV file '$csv' not found"
            process_csv remove "$csv"
            if getent group "$GROUP" >/dev/null; then
                groupdel "$GROUP"
            fi
            ;;
        report)
            report
            ;;
        *)
            echo "Usage: ${0##*/} {check|add|report|remove} [csv-file]" >&2
            exit 2
            ;;
    esac

    if [[ $INVALID -gt 0 ]]; then
        exit 3                          # some rows were skipped
    fi
}

main "$@"
EOF
chmod +x provision-users.sh
```

What just happened: Walk through the script from top to bottom:

- **Settings.** `set -euo pipefail` turns on strict mode. `readonly` makes constants that cannot be changed later. `SCRIPT_DIR` uses nested command substitution and `${BASH_SOURCE[0]}` (the script's path) to find the folder the script lives in, so the log always lands next to the script. `${LOG_FILE:-...}` lets a caller override the log location with an **environment variable**.
- **Regex.** `NAME_RE` allows a lower-case letter or underscore first, then up to 31 lower-case letters, digits, underscores, or dashes, which matches the safe Linux user name rules.
- **Small functions** with `local` variables each do one job. `valid_user` returns **different return codes** (1 for a bad name, 2 for a missing shell) so a caller could tell the failures apart. `user_exists` simply returns the status of `id`.
- `process_csv` reads the file with `while IFS=, read -r`. The `|| [[ -n $name ]]` part also processes a last line that has no trailing newline. Parameter expansion strips a Windows carriage return (`${shell%$'\r'}`) and supplies a default shell (`${shell:-/bin/bash}`). `if ! valid_user ...` uses `!` to negate a function's return code.
- `report` turns the comma-separated member list into spaces with `${members//,/ }`, loops over it with `for`, and formats the output with awk's `OFS`.
- `main` uses `case` to choose the action from the first **argument** and exits with 2 for bad usage or 3 when rows were skipped, so a caller (or cron) can check `$?`. The last line, `main "$@"`, passes all script arguments to `main` with quoting intact.

2. Run it with no action, with a missing file, and in safe `check` mode. Check the return code each time.

```
./provision-users.sh; echo "status: $?"
./provision-users.sh check nofile.csv; echo "status: $?"
./provision-users.sh check users.csv; echo "status: $?"
```

```
Usage: provision-users.sh {check|add|report|remove} [csv-file]
status: 2
ERROR: CSV file 'nofile.csv' not found
status: 1
2026-10-10 10:03:00 VALID 'jdoe' (sales)
2026-10-10 10:03:00 VALID 'asmith' (it)
2026-10-10 10:03:00 SKIP 'Bad User': invalid username
2026-10-10 10:03:00 SKIP 'bwayne': shell /bin/zsh is not installed
2026-10-10 10:03:00 VALID 'mlee' (it)
2026-10-10 10:03:00 Processed 5 rows from users.csv, 2 invalid
status: 3
```

3. Try `add` without and with `sudo`, then run it again to show that it is safe to repeat.

```
./provision-users.sh add users.csv; echo "status: $?"
sudo ./provision-users.sh add users.csv; echo "status: $?"
sudo ./provision-users.sh add users.csv | grep -E 'OK|ADD'
./provision-users.sh report
```

```
ERROR: the 'add' action must be run with sudo
status: 1
2026-10-10 10:05:12 ADD  'jdoe' (John Doe, /bin/bash)
2026-10-10 10:05:12 ADD  'asmith' (Alice Smith, /bin/bash)
2026-10-10 10:05:12 SKIP 'Bad User': invalid username
2026-10-10 10:05:12 SKIP 'bwayne': shell /bin/zsh is not installed
2026-10-10 10:05:12 ADD  'mlee' (Mary Lee, /bin/bash)
2026-10-10 10:05:12 Processed 5 rows from users.csv, 2 invalid
status: 3
2026-10-10 10:05:20 OK   'jdoe' already exists
2026-10-10 10:05:20 OK   'asmith' already exists
2026-10-10 10:05:20 OK   'mlee' already exists
Members of labusers on ubuntu-lab at 2026-10-10 10:05:
jdoe | 1001 | John Doe | /bin/bash
asmith | 1002 | Alice Smith | /bin/bash
mlee | 1003 | Mary Lee | /bin/bash
```

What just happened: `need_root` tested `$EUID` (the effective user ID, 0 for root) with `-ne` and refused to run without root. With `sudo`, the script created the group and three valid users. The second run reported `OK ... already exists`: like Ansible, a good script checks the current state before changing it, so it is safe to run again. Your UIDs may differ.

4. Debug the script with tracing. `bash -x` prints each command after expansion, prefixed with `+`, before running it.

```
bash -x ./provision-users.sh check users.csv 2>&1 | head -n 30
```

```
+ set -euo pipefail
+ readonly GROUP=labusers
+ GROUP=labusers
+++ dirname ./provision-users.sh
++ cd .
++ pwd
+ SCRIPT_DIR=/home/student/labs/4.2
...
+ main check users.csv
+ local action=check
+ local csv=users.csv
+ case "$action" in
+ [[ -f users.csv ]]
+ process_csv check users.csv
...
+ [[ ! jdoe =~ ^[a-z_][a-z0-9_-]{0,31}$ ]]
```

What just happened: Trace output goes to standard error, so `2>&1` merges it into the pipe. More `+` signs mean deeper nesting (a command substitution inside a command substitution). You can see the exact values every variable expanded to, which is the fastest way to find a bug. To trace only part of a script, put `set -x` before the suspicious lines and `set +x` after them.

What just happened in this task: Every building block from Tasks 1 to 6 now has a job in one tool: arguments pick the action, `case` routes it, functions with `local` variables keep the code readable, `while read` with `IFS` parses the CSV, a regular expression and file tests validate input, parameter expansion cleans values, and return codes tell the caller what happened. This is the shape of most real admin scripts.

Checkpoint: `getent group labusers` lists `jdoe,asmith,mlee`, and `cat provision.log` shows every action with a timestamp.

Task 8: Lint with ShellCheck and schedule with cron

1. Run ShellCheck on all your scripts. It finds quoting bugs, unused variables, and portability problems.

```
shellcheck provision-users.sh functions.sh checkfile.sh menu.sh hello.sh
echo "status: $?"
```

```
status: 0
```

2. Now see what ShellCheck catches in a buggy script.

```
cat > buggy.sh <<'EOF'
#!/bin/bash
dir=$1
count=`ls $dir | wc -l`
if [ $count > 10 ]; then
    echo "Too many files in $dir"
fi
EOF
shellcheck buggy.sh
```

```
In buggy.sh line 3:
count=`ls $dir | wc -l`
      ^---------------^ SC2006 (style): Use $(...) notation instead of legacy
backticks `...`.
         ^--^ SC2086 (info): Double quote to prevent globbing and word splitting.
...
In buggy.sh line 4:
if [ $count > 10 ]; then
            ^-- SC2071 (error): > is for string comparisons. Use -gt instead.
...
```

What just happened: ShellCheck found the exact mistakes this lab warned about: backticks instead of `$()`, unquoted variables, and `>` used for numbers. In `[ ]`, that `>` is even worse: it is a redirection, so running the script would silently create a file named `10`! The fix is `if [[ $count -gt 10 ]]`. Each warning has a code (such as SC2086) you can look up at shellcheck.net.

3. Fix `buggy.sh` so ShellCheck reports nothing, then run it.

```
cat > buggy.sh <<'EOF'
#!/bin/bash
dir="${1:?Usage: buggy.sh DIR}"
count=$(find "$dir" -mindepth 1 -maxdepth 1 | wc -l)
if [[ $count -gt 10 ]]; then
    echo "Too many files in $dir ($count)"
fi
EOF
chmod +x buggy.sh
shellcheck buggy.sh && ./buggy.sh /etc
```

4. Schedule the report with **cron**, the time-based job scheduler. Add a line to your personal crontab that runs the report every two minutes (you will remove it soon; a real job might run daily).

```
(crontab -l 2>/dev/null; echo "*/2 * * * * $HOME/labs/4.2/provision-users.sh report \
>> $HOME/labs/4.2/cron.log 2>&1") | crontab -
crontab -l
```

```
*/2 * * * * /home/student/labs/4.2/provision-users.sh report >> /home/student/labs/...
```

What just happened: The subshell printed your existing crontab (if any) plus one new line, and `crontab -` installed the result. The five fields are minute, hour, day of month, month, and day of week; `*/2` in the minute field means "every 2 minutes". `>> ... 2>&1` appends both output and errors to a log, because cron has no terminal. `$HOME` was expanded now, so the crontab holds a full path. Cron runs jobs with a minimal environment (a short `PATH`, no aliases, no `.bashrc`), which is why scripts should use full paths or set `PATH` themselves.

5. Wait two or three minutes, then check that cron ran the job.

```
cat ~/labs/4.2/cron.log
grep CRON /var/log/syslog | tail -n 3
```

```
Members of labusers on ubuntu-lab at 2026-10-10 10:12:
jdoe | 1001 | John Doe | /bin/bash
asmith | 1002 | Alice Smith | /bin/bash
mlee | 1003 | Mary Lee | /bin/bash
```

TIP

If `cron.log` stays empty, check `systemctl status cron` and look for errors with `journalctl -u cron --since "10 min ago"`. A common cause is a script that works in your terminal but depends on a variable or alias that cron does not have.

Checkpoint: `cron.log` contains at least one report, and `shellcheck *.sh` in `~/labs/4.2` prints nothing.

Challenge

CHALLENGE

1. Add a `--dry-run` option to `provision-users.sh` that shows what `add` and `remove` would do without changing anything. Hint: parse options with a `while [[ $1 == --* ]]` loop and `shift` before calling `main`.
2. Write `disk-alert.sh` that loops over the output of `df -P --output=pcent,target`, uses parameter expansion to strip the `%`, compares with `-ge` against a threshold given as the first argument (default 80), and exits with status 1 if any file system is over the limit. Lint it with ShellCheck.
3. Improve the IPv4 regex check from Task 3 so that it also rejects octets above 255, using a `for` loop over `BASH_REMATCH`.

Clean up

1. Remove the cron job (this keeps any other cron jobs you have).

```
crontab -l | grep -v 'provision-users.sh' | crontab -
crontab -l
```

2. Remove the test users and the `labusers` group with the script itself, then confirm they are gone.

```
cd ~/labs/4.2
sudo ./provision-users.sh remove users.csv
getent group labusers || echo "group removed"
id jdoe || echo "jdoe removed"
```

3. Delete the lab directory.

```
cd ~
rm -rf ~/labs/4.2
```

Exam corner

- The first line `#!/bin/bash` (shebang) selects the interpreter; make scripts executable with `chmod +x`.
- `$?` holds the last return code; 0 means success. `$#` is the argument count, `$1` … the arguments, `$@` all arguments, `$0` the script name.
- `export VAR` makes a variable an **environment variable** visible to child processes; `local` limits a variable to a function; `unset` deletes it; `set` lists variables or sets options (`-e`, `-u`, `-x`, `-o pipefail`); `alias` / `unalias` manage command shortcuts (not expanded in scripts by default).
- Parameter expansion: `${var:-default}`, `${var:=default}`, `${#var}`, `${var#pat}`, `${var%pat}`, `${var/old/new}`, `${var:offset:len}`.
- Command substitution: `$(cmd)` (preferred) and `` `cmd` `` (legacy). Subshell: `( cmds )` cannot change the parent's variables or directory.

- `IFS` controls how Bash splits words (`IFS=, read -r a b c`); awk's `OFS` sets the separator between printed fields.
- Numbers: `-eq -ne -lt -le -gt -ge` (or `(( ))`). Strings: `=`, `==`, `!=`, `<`, `>` (sort order). `<=` and `>=` are not valid string operators in `[[ ]]`.
- File and string tests: `-d` directory, `-f` regular file, `-e` exists, `-z` empty string, `-n` non-empty string, `!` negates.
- Regex: `[[ $var =~ ^regex$ ]]`, unquoted pattern, captures in `BASH_REMATCH`.
- `case` patterns end with `)`, branches end with `;;`, and the block ends with `esac`. `if` ends with `fi`. Loops end with `done`.
- `while` loops while a command succeeds; `until` loops while it fails; `for` loops over a list.

Review questions

1. A script contains `if [[ $count > 9 ]]` and `count=10`. The branch does not run. Why, and what is the fix?
2. Which command makes the variable `APP_ENV` available to programs started from the current shell?
   A. `local APP_ENV=prod`
   B. `set APP_ENV=prod`
   C. `export APP_ENV=prod`
   D. `alias APP_ENV=prod`
3. Given `f=backup.tar.gz`, what does `echo "${f%%.*}"` print?
   A. `backup.tar`
   B. `backup`
   C. `gz`
   D. `tar.gz`
4. Which loop keeps running as long as its test command returns a non-zero exit status?
5. What does `[[ -z $name ]]` test, and what is the opposite test?
6. A script runs `cp file /backup/; echo $?` and prints `1`. What does this tell you?

Lab 4.3 Python Basics for System Administration

Objective: 4.3 Summarize Python basics used for Linux system administration.
Domain: 4.0 Automation, Orchestration, and Scripting (17% of the exam)
Time: about 2 hours
VM: ubuntu-lab
Snapshot first: Not required. Everything happens in your home directory, plus two small apt packages.

Python is the second language of Linux administration after Bash. Ansible, cloud-init, firewalld, and many vendor tools are written in it, and it is the natural choice when a script outgrows Bash: when you need real data structures, JSON, web APIs, or clean error handling. In this lab you explore Python's data types, write admin scripts with built-in modules, install third-party packages safely in a virtual environment, build your own package, and check your code against the PEP 8 style guide.

What you'll practice

- Fundamentals: indentation, current versions (Python 3)
- Data types: Boolean, Dictionary, Floating point, Integer, List, String
- Built-in modules (`os`, `sys`, `shutil`, `subprocess`, `pathlib`, `json`, `argparse`, `datetime`, `socket`, `platform`)
- Virtual environment (`venv`)
- Installing dependencies (`pip install`, `requirements.txt`)
- Extensible using modules and packages
- PEP 8 (checked with `flake8` and `pycodestyle`)

How it works

Python is an **interpreted** language: you write a `.py` text file, and the `python3` program reads and runs it line by line. There is no compile step. Python 2 reached end of life in January 2020; every current distribution ships **Python 3** (Ubuntu 24.04 has Python 3.12, Rocky 9 has 3.9 with newer versions available). The command is `python3`. On many systems plain `python` does not exist at all, or points to Python 3 only if a compatibility package is installed.

Python has two features that surprise people coming from Bash:

1. **Indentation is syntax.** Bash ends blocks with words like `fi` and `done`. Python uses a colon `:` to start a block and the indentation of the following lines to show what belongs to it. PEP 8 says to use 4 spaces per level. Mixing levels causes an `IndentationError`.
2. **Values have types.** Bash treats almost everything as a string. Python knows whether a value is an integer, a floating-point number, a string, a Boolean, a list, or a dictionary, and each type has its own operations.

Type	Python name	Example	Typical admin use
Integer	`int`	`42`	port numbers, counts, UIDs
Floating point	`float`	`0.75`	percentages, load averages
String	`str`	`"ubuntu-lab"`	host names, paths, log lines
Boolean	`bool`	`True` / `False`	is the service running?
List	`list`	`["ssh", "cron"]`	an ordered group of items
Dictionary	`dict`	`{"ip": "10.0.0.5"}`	key-value data, parsed JSON

Python is **extensible using modules and packages**. A **module** is simply a `.py` file whose functions you can `import`. A **package** is a directory of modules with an `__init__.py` file. Python's **standard library** comes with hundreds of **built-in modules** ("batteries included"): `os` and `sys` talk to the operating system and interpreter, `subprocess` runs commands, `pathlib` handles files and paths, `json` reads and writes JSON, `argparse` parses command-line options, and so on. Anything else (for example `requests` for web APIs or `psutil` for system metrics) comes from the **Python Package Index (PyPI)** and is installed with `pip`.

Installing packages with `pip` into the system Python can break tools the OS depends on, because apt and pip would fight over the same files. Since Ubuntu 23.04 and Debian 12, the system Python is marked **externally managed** (PEP 668), and `pip install` refuses to touch it. The solution is a **virtual environment** (venv): a private folder with its own copy of the `python` and `pip` commands and its own `site-packages` directory.

```
System Python (apt-managed)          ~/labs/4.3/venv  (yours)
/usr/bin/python3                     venv/bin/python  -> links to /usr/bin/python3.12
/usr/lib/python3/dist-packages       venv/lib/python3.12/site-packages
  python3-apt, python3-yaml ...        requests, psutil, flake8 ...
         ^                                    ^
    apt install python3-xyz              pip install xyz  (after: source venv/bin/activate)
```

Each project gets its own venv, so two projects can use different versions of the same library. You record a project's **dependencies** in `requirements.txt` so anyone can rebuild the same venv with one command.

Finally, **PEP 8** is the official style guide for Python code (PEP means Python Enhancement Proposal). It covers 4-space indentation, line length (79 characters), naming (`snake_case` for functions and variables, `CapWords` for classes, `UPPER_CASE` for constants), spaces around operators, and blank lines between functions. Tools like `pycodestyle` and `flake8` check it automatically.

Tasks

Task 1: Meet Python 3 and its data types

1. Check which Python you have and where it lives.

```
mkdir -p ~/labs/4.3 && cd ~/labs/4.3
python3 --version
which python3
ls -l /usr/bin/python3
python --version
```

```
Python 3.12.3
/usr/bin/python3
lrwxrwxrwx 1 root root 10 Aug  8  2024 /usr/bin/python3 -> python3.12
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
```

What just happened: `python3` is a symbolic link to the real interpreter, `python3.12`. Ubuntu does not provide a plain `python` command unless you install `python-is-python3`. Always write `python3` in commands and in shebang lines (`#!/usr/bin/env python3`) so it is clear you mean Python 3.

2. Start the interactive interpreter, called the **REPL** (read-eval-print loop). The prompt changes to `>>>`. Type the lines below one at a time (do not type the `>>>`); the REPL prints the value of each expression.

```
python3
```

```
>>> count = 42
>>> type(count)
<class 'int'>
>>> count + 1, 7 / 2, 7 // 2, 7 % 2, 2 ** 10
(43, 3.5, 3, 1, 1024)
>>> ratio = 0.75
>>> type(ratio)
<class 'float'>
>>> 0.1 + 0.2
0.30000000000000004
>>> round(0.1 + 0.2, 2)
0.3
```

What just happened: `count = 42` creates a variable holding an **integer**. There is no `$` and no need to declare a type; Python works it out. `/` always gives a **floating point** result (`3.5`), `//` is integer division, `%` is the remainder, and `**` is a power. Floats are stored in binary, so `0.1 + 0.2` is not exactly `0.3`; use `round()` when displaying them.

3. Continue in the same REPL with **strings** and **Booleans**.

```
>>> name = "ubuntu-lab"
>>> type(name)
<class 'str'>
>>> name.upper()
'UBUNTU-LAB'
>>> name.split("-")
['ubuntu', 'lab']
>>> len(name), name[0:6]
(10, 'ubuntu')
>>> f"Host: {name}"
'Host: ubuntu-lab'
>>> is_up = True
>>> type(is_up)
<class 'bool'>
>>> is_up and False, not is_up, 5 > 3
(False, False, True)
>>> bool(""), bool("x"), bool(0)
(False, True, False)
```

What just happened: A **string** is text in single or double quotes. Strings have methods (functions attached to them) such as `.upper()` and `.split()`; `.split("-")` returns a list. `name[0:6]` is a slice (characters 0 to 5). An **f-string** (`f"..."`) inserts values inside `{}`, like `"$var"` in Bash. A **Boolean** is `True` or `False` (capitalized). Comparisons return Booleans. Empty values (`""`, `0`, `[]`, `{}`) count as false.

4. Now **lists** and **dictionaries**, the two structures you will use most.

```
>>> services = ["ssh", "cron", "chrony"]
>>> services.append("nginx")
>>> services
['ssh', 'cron', 'chrony', 'nginx']
>>> services[0], services[-1], len(services)
('ssh', 'nginx', 4)
>>> "cron" in services
True
>>> server = {"name": "ubuntu-lab", "ip": "192.168.100.10", "cpus": 2}
>>> server["ip"]
'192.168.100.10'
>>> server["ram_gb"] = 4
>>> server.get("os", "unknown")
'unknown'
>>> for key, value in server.items():
...     print(key, "=", value)
...
name = ubuntu-lab
ip = 192.168.100.10
cpus = 2
ram_gb = 4
```

What just happened: A **list** is an ordered, changeable collection in square brackets. Index 0 is the first item and `-1` the last. `in` tests membership. A **dictionary** maps keys to values in curly braces, just like a JSON object. Look up a value with `server["ip"]`; `.get()` returns a default instead of raising an error when the key is missing. For the `for` loop, the REPL showed `...` because the line ended with `:`. You must type 4 spaces before `print`, then press Enter on an empty line to finish the block.

5. See that Python does not convert types silently, then leave the REPL.

```
>>> int("42") + 1
43
>>> str(42) + "!"
'42!'
>>> "42" + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> exit()
```

What just happened: `int()`, `float()`, `str()`, and `bool()` convert between types. Adding a string to an integer is an error, which is a good thing: Bash would happily produce nonsense. Read a **traceback** from the bottom up: the last line names the error type and message. `exit()` (or Ctrl+D) leaves the REPL.

What just happened in this task: You met all six data types the exam lists. A useful way to remember them is by what you would store: counts in an **integer**, percentages in a **float**, names in a **string**, yes/no answers in a **Boolean**, groups of items in a **list**, and labeled data (like JSON) in a **dictionary**.

Checkpoint: `python3 -c 'print(type([]), type({}), type(1.0), type(True))'` prints `<class 'list'> <class 'dict'> <class 'float'> <class 'bool'>`.

Task 2: Indentation and the shape of a Python script

1. Create a script with a missing indent and run it. The file is `~/labs/4.3/indent_bad.py`:

```
cat > indent_bad.py <<'EOF'
services = ["ssh", "cron"]
for svc in services:
print(svc)
EOF
python3 indent_bad.py
```

```
  File "/home/student/labs/4.3/indent_bad.py", line 3
    print(svc)
    ^
IndentationError: expected an indented block after 'for' statement on line 2
```

> **What just happened:** You used a heredoc (`cat > file <<'EOF' ... EOF`), which writes every line up to `EOF` into the file; the quotes around `'EOF'` stop the shell from changing anything inside. Python then refused to run the file: a line ending in `:` must be followed by an indented block. Python checks the whole file's syntax before running any of it.

2. Write a correct script that shows `if` / `elif` / `else`, a `for` loop, a function, and exit codes. The file is `~/labs/4.3/services.py`:

```
cat > services.py <<'EOF'
#!/usr/bin/env python3
"""services.py - report whether some systemd services are active."""

import subprocess
import sys

SERVICES = ["ssh", "cron", "systemd-journald", "nosuch"]


def is_active(name):
    """Return True if systemctl reports the service as active."""
    result = subprocess.run(["systemctl", "is-active", "--quiet", name])
    return result.returncode == 0


def main():
    """Check every service and return the number that are down."""
    down = 0
    for name in SERVICES:
        if is_active(name):
            print(f"{name:<17} active")
        elif name.startswith("no"):
            print(f"{name:<17} (test name that does not exist)")
            down += 1
        else:
            print(f"{name:<17} NOT active")
            down += 1
    return down


if __name__ == "__main__":
    sys.exit(main())
EOF
chmod +x services.py
./services.py
echo "exit status: $?"
```

```
ssh               active
cron              active
systemd-journald  active
nosuch            (test name that does not exist)
exit status: 1
```

What just happened: Read the script by its indentation. `def is_active(name):` defines a **function**; its body is the indented lines below it. The text in triple quotes right after `def` is a **docstring** that documents the function. `subprocess.run([...])` runs a command, passing each argument as a separate list item (no shell, so no quoting problems), and `.returncode` holds its exit status. `main()` returns a count, and `sys.exit(main())` turns that count into the script's exit status, so Bash and cron can test it with `$?`. The block `if __name__ == "__main__":` runs only when the file is executed directly, not when another script imports it.

TIP

`ssh` might show `NOT active` if your VM starts SSH by socket activation (`ssh.socket`) and no one has connected yet. Since you are probably logged in over SSH, it should be active.

Checkpoint: Change `"nosuch"` to `"cron"` in `services.py`, run it again, and confirm `echo $?` prints `0`.

Task 3: Build a disk report with built-in modules

1. Built-in modules are part of every Python install. See where one lives and read its built-in help.

```
python3 -c 'import shutil, sys; print(shutil.__file__); print(sys.version)'
python3 -c 'import shutil; help(shutil.disk_usage)' | cat
```

```
/usr/lib/python3.12/shutil.py
3.12.3 (main, ...) [GCC 13.2.0]
Help on function disk_usage in module shutil:

disk_usage(path)
    Return disk usage statistics about the given path.

    Returned value is a named tuple with attributes 'total', 'used' and
    'free', which are the amount of total, used and free space, in bytes.
```

What just happened: `python3 -c '...'` runs a short program from the command line. The standard library is installed under `/usr/lib/python3.12/`. `help()` shows the documentation for any module or function; piping into `cat` prints it instead of opening a pager.

2. Write a real admin tool, `diskreport.py`, that uses six built-in modules. Read the comments and docstrings as you type or paste it.

```
cat > diskreport.py <<'EOF'
#!/usr/bin/env python3
"""diskreport.py - report disk usage for one or more mount points."""

import argparse
import datetime
import json
import shutil
import socket
import sys

GIB = 1024 ** 3


def usage_for(path):
    """Return a dictionary describing disk usage for path."""
    total, used, free = shutil.disk_usage(path)
    return {
        "path": path,
        "total_gib": round(total / GIB, 1),
        "used_gib": round(used / GIB, 1),
        "free_gib": round(free / GIB, 1),
        # same formula as df: used / (used + available to normal users)
        "percent_used": round(used / (used + free) * 100, 1),
    }


def parse_args():
    """Define and read the command-line options."""
    parser = argparse.ArgumentParser(description="Report disk usage.")
    parser.add_argument("paths", nargs="*", default=["/"],
                        help="mount points to check (default: /)")
    parser.add_argument("-t", "--threshold", type=float, default=80.0,
                        help="warn at or above this percent (default: 80)")
    parser.add_argument("--json", action="store_true",
                        help="print JSON instead of a table")
    return parser.parse_args()


def main():
    """Build the report and return an exit code."""
    args = parse_args()
    results = []
    for path in args.paths:
        try:
            results.append(usage_for(path))
        except FileNotFoundError:
            print(f"warning: {path} does not exist", file=sys.stderr)

    over = [r for r in results if r["percent_used"] >= args.threshold]

    if args.json:
        report = {
            "host": socket.gethostname(),
            "time": datetime.datetime.now().isoformat(timespec="seconds"),
```

```
            "threshold": args.threshold,
            "filesystems": results,
        }
        print(json.dumps(report, indent=2))
    else:
        print(f"{'PATH':<10}{'SIZE':>8}{'USED':>8}{'FREE':>8}{'USE%':>7}")
        for r in results:
            flag = "  <-- over threshold" if r in over else ""
            print(f"{r['path']:<10}{r['total_gib']:>8}{r['used_gib']:>8}"
                  f"{r['free_gib']:>8}{r['percent_used']:>6}%{flag}")

    return 1 if over else 0


if __name__ == "__main__":
    sys.exit(main())
EOF
chmod +x diskreport.py
```

What just happened: Here is what each built-in module does in this script:

- `shutil.disk_usage()` returns total, used, and free bytes. The line `total, used, free = ...` unpacks three values at once.
- `argparse` turns command-line arguments into `args.paths`, `args.threshold`, and `args.json`, and generates `--help` for free. `type=float` converts the option's text to a float.
- `json.dumps()` converts a dictionary into JSON text.
- `datetime` gives the current time; `socket.gethostname()` the host name.
- `sys.stderr` and `sys.exit()` send warnings to standard error and set the exit status.

The function returns a **dictionary** per file system, and `results` is a **list** of those dictionaries. `over = [r for r in results if ...]` is a **list comprehension**: a one-line loop that builds a new list of the items that pass the test. `try` / `except FileNotFoundError` catches a bad path instead of crashing.

3. Run it in several ways and compare with `df`.

```
./diskreport.py --help
./diskreport.py
df -h /
./diskreport.py / /boot /nope --threshold 20; echo "exit status: $?"
./diskreport.py --json
```

```
usage: diskreport.py [-h] [-t THRESHOLD] [--json] [paths ...]

Report disk usage.
...
PATH          SIZE    USED    FREE   USE%
/             23.4     6.1    16.1  27.5%
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   24G  6.2G   17G  28% /
warning: /nope does not exist
PATH          SIZE    USED    FREE   USE%
/             23.4     6.1    16.1  27.5%  <-- over threshold
/boot          1.7     0.2     1.4  11.6%
exit status: 1
{
  "host": "ubuntu-lab",
  "time": "2026-10-10T10:15:02",
  "threshold": 80.0,
  "filesystems": [
    {
      "path": "/",
      ...
```

What just happened: The percentage matches `df` because both divide used space by used plus available space (ext4 reserves about 5% for root, so `SIZE` is a bit more than `USED + FREE`). The exit status was 1 because `/` was over the 20% threshold. JSON output is easy for other programs to read: try `./diskreport.py --json | python3 -m json.tool --sort-keys`, or with `jq` if it is installed. Your sizes will differ.

Checkpoint: `./diskreport.py --json | python3 -c 'import json,sys; d=json.load(sys.stdin); print(d["filesystems"][0]["percent_used"])'` prints the same percentage as the table.

Task 4: Install dependencies in a virtual environment

1. Install `pip` and the `venv` module. On Ubuntu these are separate packages.

```
sudo apt install -y python3-pip python3-venv
pip3 --version
```

```
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
```

2. Try to install a package system-wide with pip, the way many old tutorials show.

```
pip3 install requests
```

```
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.
    ...
hint: See PEP 668 for the detailed specification.
```

What just happened: Ubuntu 24.04 ships the file `/usr/lib/python3.12/EXTERNALLY-MANAGED`, and pip refuses to install into the system Python when that file exists (PEP 668). This protects OS tools such as `apt`'s Python helpers, cloud-init, and netplan from being broken by a pip upgrade. You have three good options: install the Debian package (`sudo apt install python3-requests`), use a virtual environment (best for your own scripts), or use `pipx` for command-line applications. The `--break-system-packages` option exists, but its name tells you what it does.

CAUTION

Never run `sudo pip install` on a modern distribution. It mixes pip-installed files into directories managed by apt and can break system tools in ways that are hard to repair.

3. Create a **virtual environment** and activate it.

```
cd ~/labs/4.3
python3 -m venv venv
ls venv venv/bin
source venv/bin/activate
which python pip
python --version
```

```
bin  include  lib  lib64  pyvenv.cfg
Activate.ps1  activate  activate.csh  activate.fish  pip  pip3  pip3.12  python
python3  python3.12
(venv) student@ubuntu-lab:~/labs/4.3$
/home/student/labs/4.3/venv/bin/python
/home/student/labs/4.3/venv/bin/pip
Python 3.12.3
```

What just happened: `python3 -m venv venv` runs the built-in `venv` module and creates a directory called `venv` (any name works). `source venv/bin/activate` adds `venv/bin` to the front of your `PATH` and changes the prompt to `(venv)`, so `python` and `pip` now mean the venv's copies. Inside a venv, plain `python` exists and is Python 3. Nothing outside the folder is changed; deleting the folder deletes the environment. Do not commit the `venv` folder to Git.

4. Now **pip install** the packages. This time pip installs them inside the venv.

```
pip install requests psutil
pip list
pip show psutil | head -n 3
```

```
Collecting requests
...
Successfully installed certifi-2025.8.3 charset-normalizer-3.4.3 idna-3.10 psutil-7.0.0
requests-2.32.5 urllib3-2.5.0
Package            Version
------------------ ---------
certifi            2025.8.3
charset-normalizer 3.4.3
idna               3.10
pip                24.0
psutil             7.0.0
requests           2.32.5
urllib3            2.5.0
Name: psutil
Version: 7.0.0
Summary: Cross-platform lib for process and system monitoring in Python.
```

What just happened: pip downloaded the packages from PyPI, along with the **dependencies** that `requests` needs (`certifi` , `idna` , `urllib3` , `charset-normalizer`). Everything went into `venv/lib/python3.12/site-packages/` . Your version numbers will be newer.

5. Record the dependencies in `requirements.txt` , then prove you can rebuild the environment from it.

```
pip freeze > requirements.txt
cat requirements.txt
deactivate
python3 -m venv /tmp/venv-test
/tmp/venv-test/bin/pip install -r requirements.txt
/tmp/venv-test/bin/pip list | grep -E 'psutil|requests'
rm -rf /tmp/venv-test
```

```
certifi==2025.8.3
charset-normalizer==3.4.3
idna==3.10
psutil==7.0.0
requests==2.32.5
urllib3==2.5.0
...
psutil             7.0.0
requests           2.32.5
```

What just happened: `pip freeze` prints every installed package with its exact version (`==`), and you saved it to a file. `deactivate` left the venv (the prompt lost `(venv)`). A fresh venv then installed identical versions with `pip install -r requirements.txt`. Notice you can use a venv's `pip` or `python` by full path without activating it, which is how cron jobs and systemd services should call venv scripts.

Checkpoint: With the venv deactivated, `python3 -c 'import psutil'` fails (or uses an apt copy if one is installed), while `~/labs/4.3/venv/bin/python -c 'import psutil; print(psutil.__version__)'` prints a version.

Task 5: Use third-party packages in a script

1. Write `check_versions.py`, which uses `psutil` for system metrics and `requests` to call the PyPI web API.

```
cd ~/labs/4.3
cat > check_versions.py <<'EOF'
#!/usr/bin/env python3
"""check_versions.py - show system load and compare packages with PyPI."""

import sys
from importlib.metadata import version

import psutil
import requests

PACKAGES = ["requests", "psutil"]


def latest_on_pypi(name):
    """Ask the PyPI JSON API for the newest version of a package."""
    url = f"https://pypi.org/pypi/{name}/json"
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    return response.json()["info"]["version"]


def main():
    """Print memory, CPU, and package version information."""
    mem = psutil.virtual_memory()
    print(f"CPU cores: {psutil.cpu_count()}, CPU busy: "
          f"{psutil.cpu_percent(interval=1)}%")
    print(f"Memory used: {mem.percent}% of {mem.total // 1024 ** 2} MiB")
    for name in PACKAGES:
        installed = version(name)
        try:
            latest = latest_on_pypi(name)
        except requests.RequestException as err:
            print(f"{name}: cannot reach PyPI ({err})", file=sys.stderr)
            continue
        status = "up to date" if installed == latest else f"{latest} available"
        print(f"{name:<9} installed {installed:<8} {status}")


if __name__ == "__main__":
    main()
EOF
```

What just happened: Imports are grouped as PEP 8 recommends: standard library first (`sys` , `importlib.metadata`), then a blank line, then third-party packages. `from importlib.metadata import version` imports one function from a module. `requests.get()` makes an HTTPS request; `timeout=10` stops it hanging forever, and `raise_for_status()` turns an HTTP error (like 404) into an exception. `.json()` parses the reply into a **dictionary**, so `["info"]["version"]` digs out one value. `except requests.RequestException` handles network failures cleanly.

2. Run it with the system Python, then with the venv's Python.

```
python3 check_versions.py
~/labs/4.3/venv/bin/python check_versions.py
```

```
Traceback (most recent call last):
  File "/home/student/labs/4.3/check_versions.py", line 7, in <module>
    import psutil
ModuleNotFoundError: No module named 'psutil'
CPU cores: 2, CPU busy: 3.1%
Memory used: 18.4% of 3916 MiB
requests  installed 2.32.5   up to date
psutil    installed 7.0.0    up to date
```

What just happened: The system Python cannot see packages installed in the venv, so the import failed. The venv's interpreter found both packages. To make the script always use the venv, you could change its first line to `#!/home/student/labs/4.3/venv/bin/python`.

Checkpoint: `~/labs/4.3/venv/bin/python check_versions.py` prints CPU, memory, and one line per package.

Task 6: Create your own module and package

1. Turn reusable code into a package called `labtools`. A package is a directory with an `__init__.py` file; every `.py` file inside is a module.

```
cd ~/labs/4.3
mkdir -p labtools
cat > labtools/__init__.py <<'EOF'
"""labtools - small helpers for Linux system administration."""

__version__ = "0.1.0"
EOF
cat > labtools/disk.py <<'EOF'
"""Disk usage helpers."""

import shutil

GIB = 1024 ** 3


def usage_for(path):
    """Return a dictionary describing disk usage for path."""
    total, used, free = shutil.disk_usage(path)
    return {
        "path": path,
        "total_gib": round(total / GIB, 1),
        "free_gib": round(free / GIB, 1),
        "percent_used": round(used / (used + free) * 100, 1),
    }
EOF
```

2. Add a module that uses `os`, `platform`, `subprocess`, and `pathlib`.

```
cat > labtools/system.py <<'EOF'
"""Facts about the running Linux system."""

import os
import platform
import subprocess
from pathlib import Path


def os_name():
    """Return PRETTY_NAME from /etc/os-release."""
    for line in Path("/etc/os-release").read_text().splitlines():
        if line.startswith("PRETTY_NAME="):
            return line.split("=", 1)[1].strip('"')
    return "unknown"


def uptime():
    """Return the output of 'uptime -p' as a string."""
    result = subprocess.run(["uptime", "-p"], capture_output=True,
                            text=True, check=True)
    return result.stdout.strip()


def summary():
    """Return a dictionary of basic system facts."""
    return {
        "os": os_name(),
        "kernel": platform.release(),
        "user": os.environ.get("USER", "unknown"),
        "uid": os.getuid(),
        "uptime": uptime(),
        "load_1min": round(os.getloadavg()[0], 2),
    }
EOF
```

What just happened: `pathlib.Path(...).read_text()` reads a whole file in one call, and `.splitlines()` makes a list of lines. `line.split("=", 1)[1]` takes everything after the first `=`, and `.strip('"')` removes the quotes. `subprocess.run(..., capture_output=True, text=True, check=True)` captures the command's output as a string and raises an error if it fails. `os.environ` is a dictionary of **environment variables**, the same ones Bash exports. `os.getuid()` returns your UID.

3. Add a `__main__.py` so the package can run with `python3 -m labtools`, and a separate script that imports it.

```
cat > labtools/__main__.py <<'EOF'
"""Run with: python3 -m labtools"""

from labtools import __version__
from labtools.disk import usage_for
from labtools.system import summary

print(f"labtools {__version__}")
for key, value in summary().items():
    print(f"  {key:<10} {value}")
root = usage_for("/")
print(f"  {'root fs':<10} {root['percent_used']}% used, "
      f"{root['free_gib']} GiB free")
EOF
cat > health.py <<'EOF'
#!/usr/bin/env python3
"""health.py - print a one-line health summary using the labtools package."""

import labtools.system
from labtools.disk import usage_for

facts = labtools.system.summary()
root = usage_for("/")
status = "OK" if root["percent_used"] < 80 else "WARN"
print(f"[{status}] {facts['os']} | load {facts['load_1min']:.2f} | "
      f"/ {root['percent_used']}% used")
EOF
python3 -m labtools
python3 health.py
```

```
labtools 0.1.0
  os         Ubuntu 24.04.3 LTS
  kernel     6.8.0-85-generic
  user       student
  uid        1000
  uptime     up 2 hours, 5 minutes
  load_1min  0.08
  root fs    27.5% used, 16.1 GiB free
[OK] Ubuntu 24.04.3 LTS | load 0.08 | / 27.5% used
```

What just happened: There are two import styles. `import labtools.system` imports a module, and you call `labtools.system.summary()` with its full name. `from labtools.disk import usage_for` imports one name, so you call `usage_for()` directly. Python found `labtools` because it searches the script's own directory first. `python3 -m labtools` runs the package's `__main__.py`.

4. See where Python looks for modules, and what happens from another directory.

```
python3 -c 'import sys; print("\n".join(sys.path))'
cd /tmp && python3 -c 'import labtools'; cd ~/labs/4.3
```

```
/usr/lib/python312.zip
/usr/lib/python3.12
/usr/lib/python3.12/lib-dynload
/usr/local/lib/python3.12/dist-packages
/usr/lib/python3/dist-packages
...
ModuleNotFoundError: No module named 'labtools'
```

What just happened: `sys.path` is the list of directories Python searches, in order. The empty first entry means "the current directory" (for `-c`) or the script's directory. From `/tmp`, `labtools` is not on the path. To share a package properly you would add a `pyproject.toml` and install it into a venv with `pip install .`, the same way `requests` was installed.

Checkpoint: `ls labtools` shows `__init__.py`, `__main__.py`, `disk.py`, `system.py` (and a `__pycache__` directory of compiled byte code), and `python3 -m labtools` prints your system summary.

Task 7: Check your code against PEP 8

1. Install the style checkers into the venv. They are development tools, so keep them in a separate requirements file from what the scripts need at run time.

```
cd ~/labs/4.3
source venv/bin/activate
pip install flake8 pycodestyle
pip freeze | grep -iE 'flake8|pycodestyle|pyflakes' > requirements-dev.txt
cat requirements-dev.txt
```

What just happened: `pycodestyle` checks PEP 8 style. `flake8` combines `pycodestyle` with `pyflakes`, which finds real errors such as unused imports and undefined names. `requirements-dev.txt` now lists them so another admin can run the same checks.

2. Write a deliberately messy file that works only by luck, then check it.

```
cat > messy.py <<'EOF'
import os,sys
import json
def CheckDisk( path ):
    total,used,free=shutil.disk_usage(path)
    if used/total>0.8 :
        return True
    else:
      return False
x = CheckDisk ('/')
print( x )
EOF
flake8 messy.py
```

```
messy.py:1:1: F401 'os' imported but unused
messy.py:1:1: F401 'sys' imported but unused
messy.py:1:10: E401 multiple imports on one line
messy.py:1:10: E231 missing whitespace after ','
messy.py:2:1: F401 'json' imported but unused
messy.py:3:1: E302 expected 2 blank lines, found 0
messy.py:3:15: E201 whitespace after '('
messy.py:3:20: E202 whitespace before ')'
messy.py:4:10: E231 missing whitespace after ','
messy.py:4:15: E231 missing whitespace after ','
messy.py:4:20: E225 missing whitespace around operator
messy.py:4:21: F821 undefined name 'shutil'
messy.py:5:18: E225 missing whitespace around operator
messy.py:5:22: E203 whitespace before ':'
messy.py:8:7: E111 indentation is not a multiple of 4
messy.py:9:1: E305 expected 2 blank lines after class or function definition, found 0
messy.py:9:14: E211 whitespace before '('
messy.py:10:7: E201 whitespace after '('
messy.py:10:9: E202 whitespace before ')'
```

What just happened: Each line is `file:line:column: CODE message`. Codes starting with `E` or `W` are PEP 8 style issues from pycodestyle; codes starting with `F` come from pyflakes. Look at `F821 undefined name 'shutil'`: that is a real bug. The program would crash with `NameError` when `CheckDisk` runs. The function name `CheckDisk` also breaks PEP 8 naming (functions should be `snake_case`), which flake8 does not check by default. Running `python3 messy.py` would show the crash.

3. Rewrite it in clean PEP 8 style, and check it again.

```
cat > messy.py <<'EOF'
"""Check whether a file system is nearly full."""
import shutil


def check_disk(path, limit=0.8):
    """Return True if path is more than limit (0 to 1) full."""
    total, used, _free = shutil.disk_usage(path)
    return used / total > limit


if __name__ == "__main__":
    print(check_disk("/"))
EOF
flake8 messy.py && echo "flake8: no problems"
python3 messy.py
```

```
flake8: no problems
False
```

What just happened: The fixes: one import per line and only what is used; two blank lines around top-level functions; `snake_case` names; spaces after commas and around operators but not inside parentheses; 4-space indentation; and a docstring. `if ...: return True else: return False` became `return used / total > limit`, because a comparison already produces a Boolean. `_free` starts with an underscore to show it is intentionally unused.

4. Check all of your lab code. First delete `indent_bad.py`, which contains a deliberate syntax error. Fix anything reported (the other files from this lab should already be clean), then leave the venv.

```
rm -f indent_bad.py
flake8 --exclude venv .
pycodestyle --exclude=venv .
echo "exit status: $?"
deactivate
```

```
exit status: 0
```

TIP

Many teams also use an auto-formatter such as `black` or `ruff format`, which rewrites code into a consistent PEP 8 style for you, and run `flake8` or `ruff check` in their CI pipeline (see Lab 4.1) so badly styled code cannot be merged.

Checkpoint: `~/labs/4.3/venv/bin/flake8 --exclude venv ~/labs/4.3` prints nothing.

Challenge

CHALLENGE

1. Add a `--json` option to `health.py` with `argparse`, and make it exit with status 1 when the root file system is at least 80% used.
2. Write `topmem.py` that uses `psutil.process_iter(["pid", "name", "memory_percent"])` to print the five processes using the most memory, sorted with `sorted(..., key=..., reverse=True)`. Run it from the venv and check it with `flake8`.
3. Write a `pyproject.toml` for `labtools`, install it into the venv with `pip install -e .`, and show that `python -m labtools` now works from `/tmp` while the venv is active.

Clean up

1. Make sure no venv is active (the prompt must not start with `(venv)`). If it is, run `deactivate`.
2. Delete the lab directory, including the venv and everything installed in it.

```
cd ~
rm -rf ~/labs/4.3
```

3. The `python3-pip` and `python3-venv` packages are harmless and useful in later labs, so you may keep them. To remove them:

```
sudo apt remove -y python3-pip python3-venv
sudo apt autoremove -y
```

Exam corner

- Use **Python 3** (`python3`); Python 2 is end-of-life. Ubuntu 24.04 ships Python 3.12.
- **Indentation** defines code blocks; a line ending in `:` starts a block; PEP 8 says 4 spaces.
- Data types: **integer** (`int`), **floating point** (`float`), **string** (`str`), **Boolean** (`True` / `False`), **list** (`[ ]`, ordered), **dictionary** (`{key: value}`).
- **Built-in modules** (standard library) need no install: `os`, `sys`, `subprocess`, `shutil`, `pathlib`, `json`, `argparse`, `datetime`, `re`, `logging`, `socket`.
- A **module** is a `.py` file; a **package** is a directory of modules with `__init__.py`. Use them with `import x` or `from x import y`.
- **Virtual environment**: `python3 -m venv venv`, `source venv/bin/activate`, `deactivate`. Ubuntu needs `python3-venv`.
- **pip install** packages from PyPI inside a venv; `pip freeze > requirements.txt` records **dependencies**; `pip install -r requirements.txt` restores them.
- `error: externally-managed-environment` (PEP 668) means pip will not modify the system Python: use a venv, `apt install python3-xyz`, or `pipx`.
- **PEP 8** is Python's style guide: 4-space indents, 79-character lines, `snake_case` functions and variables, `CapWords` classes, `UPPER_CASE` constants, imports at the top. Check it with `pycodestyle` or `flake8`.
- `if __name__ == "__main__":` runs code only when the file is executed directly; `sys.exit(n)` sets the exit status.

Review questions

1. An admin runs `pip install requests` on Ubuntu 24.04 and gets `externally-managed-environment`. What is the recommended fix for a personal script?
   A. Run `sudo pip install requests`
   B. Create a virtual environment and install the package inside it
   C. Delete `/usr/lib/python3.12/EXTERNALLY-MANAGED`
   D. Downgrade to Python 2

2. Which Python data type stores key-value pairs such as `{"host": "web01", "port": 22}`?

3. What causes `IndentationError: expected an indented block`?

4. Which command saves the exact versions of all packages in the active venv so they can be reinstalled elsewhere?

5. What is the difference between a Python module and a Python package?

6. Which tool would you run to check a script against PEP 8?
   A. `pip check`
   B. `python3 -m venv`
   C. `pycodestyle` (or `flake8`)
   D. `pydoc`

Lab 4.4 Version Control with Git

Objective: 4.4 Given a scenario, implement version control using Git.
Domain: 4.0 Automation, Orchestration, and Scripting (17% of the exam)
Time: about 2.5 hours
VM: ubuntu-lab and rocky-lab
Snapshot first: Not required. All work happens in your home directories.

Git records every change to a set of files: who changed what, when, and why. Admins use it for scripts, configuration files, Ansible playbooks, and Kubernetes manifests, so they can review changes, undo mistakes, and work together without overwriting each other. In this lab you put a folder of admin scripts under Git, work with branches, merge (including a real conflict), squash, rebase, stash, reset, and tag. Then you host a shared repository on rocky-lab over SSH and simulate two admins pushing and pulling, exactly as you would with GitHub or GitLab.

What you'll practice

- `git config`, `git init`, `.gitignore`
- `git add`, `git commit`, `git diff`, `git log`
- `git branch`, `git checkout` (and `git switch`)
- `git merge` (fast-forward, three-way, conflicts, and `--squash`)
- `git rebase`
- `git stash`
- `git reset` (`--soft`, `--mixed`, `--hard`) and recovery with `git reflog`
- `git tag` (lightweight and annotated)
- `git clone`, `git push`, `git fetch`, `git pull` with a remote repository

How it works

A Git **repository** (repo) is a project folder plus a hidden `.git` directory that stores the complete history. Git is **distributed**: every clone has the full history, so you can commit, branch, and view logs with no network at all. You only need the network to exchange commits with another copy (a **remote**).

Every file you change moves through three areas:

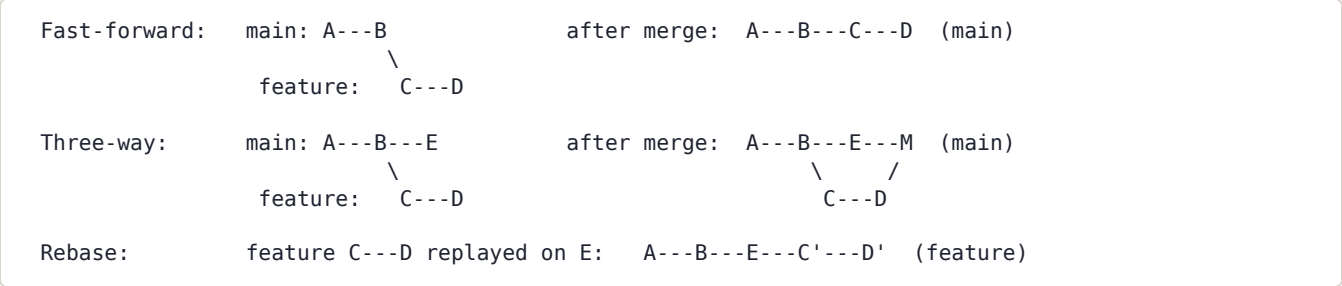
```
 working directory  --git add-->  staging area (index)  --git commit-->  repository (.git)
  (files you edit)                (what the next commit                 (permanent history
                                    will contain)                         of snapshots)
      ^                                                                        |
      +-------------------------- git checkout / git switch ----------------+
```

- The **working directory** is the normal files you edit.

- The **staging area** (also called the **index**) is a draft of the next commit. `git add` copies changes into it, so you can choose exactly what goes into each commit.
- A **commit** is a snapshot of the staged files, with an author, date, message, and a pointer to its parent commit(s). Each commit is named by a 40-character SHA-1 hash, which you can shorten to the first 7 characters.

A **branch** is just a movable label that points to a commit. `main` is the usual default branch. When you commit, the current branch's label moves forward. `HEAD` is a pointer to the branch (and commit) you have checked out right now. Because branches are so cheap, admins create one for every change: `feature/new-backup`, `fix/ssh-timeout`, and so on.

Merging brings the commits of one branch into another. If the target branch has not moved since you branched, Git simply slides the label forward: a **fast-forward** merge. If both branches have new commits, Git makes a new **merge commit** with two parents: a **three-way merge** (it compares both tips with their common ancestor). If both branches changed the same lines, Git cannot decide and stops with a **merge conflict** for you to fix. **Rebasing** is the alternative: it replays your branch's commits on top of the other branch, producing a straight-line history instead of a merge commit. A **squash merge** combines all of a branch's commits into one new commit.



A **remote** is another copy of the repository, usually on a server. GitHub, GitLab, and Bitbucket host remotes with web interfaces, but any Linux server with SSH and Git can host a **bare** repository (one with no working directory, used only for pushing and pulling). `git push` sends your commits to the remote, `git fetch` downloads new commits without changing your files, and `git pull` is `fetch` followed by `merge` (or `rebase`). `git clone` copies a whole remote repository to a new folder.

Tasks

Task 1: Install and configure Git

1. Install Git on both VMs and check the version.

```
sudo apt install -y git
git --version
```

```
# on rocky-lab
sudo dnf install -y git
git --version
```

```
git version 2.43.0
```

2. Tell Git who you are. Every commit records this name and email address. Also set a few sensible defaults.

```
git config --global user.name "Student Admin"
git config --global user.email "student@ubuntu-lab.local"
git config --global init.defaultBranch main
git config --global core.editor nano
git config --global pull.rebase false
git config --list --show-origin
```

```
file:/home/student/.gitconfig   user.name=Student Admin
file:/home/student/.gitconfig   user.email=student@ubuntu-lab.local
file:/home/student/.gitconfig   init.defaultbranch=main
file:/home/student/.gitconfig   core.editor=nano
file:/home/student/.gitconfig   pull.rebase=false
```

What just happened: `git config` reads and writes settings at three levels: `--system` (`/etc/gitconfig`, every user), `--global` (`~/.gitconfig`, you), and `--local` (`.git/config`, one repository; the default when you are inside a repo). The more specific level wins. `init.defaultBranch main` names the first branch `main` instead of `master`. `core.editor` picks the editor Git opens for messages. `pull.rebase false` tells `git pull` to merge when histories have diverged (you will see why in Task 7). `--show-origin` shows which file each setting came from.

Checkpoint: `git config user.name` prints `Student Admin`.

Task 2: Create a repository, ignore files, and commit

1. Create a project folder with some admin files, then turn it into a repository with `git init`.

```
mkdir -p ~/labs/4.4/scripts && cd ~/labs/4.4/scripts
git init
git status
```

```
Initialized empty Git repository in /home/student/labs/4.4/scripts/.git/
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

What just happened: `git init` created the hidden `.git` directory that holds the history and settings. Delete `.git` and the folder becomes a normal folder again. `git status` is the command you will run most: it tells you the branch and the state of every file.

2. Create a script, a config file, and some files that should never be committed: a log, a scratch folder, and a secrets file. You will use heredocs: `cat > file <<'EOF'` writes all lines up to `EOF` into `file`, and the quotes around `'EOF'` keep `$` signs literal.

```
cat > backup.sh <<'EOF'
#!/bin/bash
# backup.sh - archive /etc to /var/backups
tar -czf "/var/backups/etc-$(date +%F).tar.gz" /etc
EOF
cat > sshd-hardening.conf <<'EOF'
PermitRootLogin no
PasswordAuthentication yes
EOF
touch debug.log
mkdir -p tmp && touch tmp/scratch.txt
echo 'DB_PASSWORD=Secret123' > secrets.env
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        backup.sh
        debug.log
        secrets.env
        sshd-hardening.conf
        tmp/
```

What just happened: **Untracked** files are files Git sees but is not managing. Git will never record them until you `git add` them. The danger is that a careless `git add .` would add the log and, much worse, the password file.

3. Create a `.gitignore` file listing patterns Git should ignore. The file is `~/labs/4.4/scripts/.gitignore`:

```
cat > .gitignore <<'EOF'
# Logs and scratch files
*.log
tmp/

# Never commit secrets
*.env
EOF
git status --short
git check-ignore -v debug.log secrets.env tmp/scratch.txt
```

```
?? .gitignore
?? backup.sh
?? sshd-hardening.conf
.gitignore:2:*.log      debug.log
.gitignore:6:*.env      secrets.env
.gitignore:3:tmp/       tmp/scratch.txt
```

What just happened: Each line of `.gitignore` is a pattern: `*.log` matches any file ending in `.log`, `tmp/` matches a directory, and `#` starts a comment. The ignored files disappeared from `git status`. `--short` gives a compact view where `??` means untracked. `git check-ignore -v` explains which line ignores a file, which is handy when something is ignored unexpectedly. `.gitignore` itself should be committed so everyone shares the same rules.

CAUTION

`.gitignore` only affects untracked files. If a secret was already committed, adding it to `.gitignore` does not remove it from history. Treat any secret that reached a shared repository as leaked and change it.

4. Stage files with **git add**, then make the first commit with **git commit**.

```
git add .gitignore backup.sh sshd-hardening.conf
git status --short
git commit -m "Add backup script and SSH hardening config"
git log
```

```
A  .gitignore
A  backup.sh
A  sshd-hardening.conf
[main (root-commit) 4491226] Add backup script and SSH hardening config
 3 files changed, 11 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 backup.sh
 create mode 100644 sshd-hardening.conf
commit 4491226a7b9854d4a2b8e44c0cfb8665ec99afe3 (HEAD -> main)
Author: Student Admin <student@ubuntu-lab.local>
Date:   Sat Oct 10 10:13:58 2026 +0000

    Add backup script and SSH hardening config
```

What just happened: `A` in the status means "added to the staging area". `git commit -m "message"` saved a snapshot of the staged files. Without `-m`, Git opens your editor for the message. A good message says *what* and *why* in the imperative mood ("Add …", "Fix …"). `git log` shows the history, newest first. `(HEAD -> main)` means you are on `main`, and `main` points to this commit. Your hash will differ.

5. Change a file and use **git diff** to see exactly what changed, before and after staging.

```
sed -i 's/PasswordAuthentication yes/PasswordAuthentication no/' sshd-hardening.conf
git diff
git add sshd-hardening.conf
git diff
git diff --staged
git commit -m "Disable SSH password authentication"
git log --oneline
```

```
diff --git a/sshd-hardening.conf b/sshd-hardening.conf
index aa9d833..d3b3049 100644
--- a/sshd-hardening.conf
+++ b/sshd-hardening.conf
@@ -1,2 +1,2 @@
 PermitRootLogin no
-PasswordAuthentication yes
+PasswordAuthentication no
...
[main fc7035c] Disable SSH password authentication
 1 file changed, 1 insertion(+), 1 deletion(-)
fc7035c (HEAD -> main) Disable SSH password authentication
4491226 Add backup script and SSH hardening config
```

What just happened: Plain `git diff` compares the working directory with the staging area. After `git add`, it printed nothing, because the change had moved into staging; `git diff --staged` (same as `--cached`) compares staging with the last commit. In the diff, `-` lines were removed and `+` lines added; `@@ -1,2 +1,2 @@` says which lines are shown. `git log --oneline` shows one short line per commit. Try `git show HEAD` to see the latest commit and its diff, and `git diff HEAD~1 HEAD` to compare the last two commits (`HEAD~1` means "one commit before HEAD").

What just happened in this task: You used the basic Git loop that you will repeat thousands of times: change files, check with `git status` and `git diff`, stage with `git add`, and record with `git commit`. The staging area lets you build each commit deliberately, and `.gitignore` keeps logs and secrets out of history from the start.

Checkpoint: `git log --oneline` shows two commits, and `git status` says `nothing to commit, working tree clean` (the ignored files do not count).

Task 3: Branches and merges

1. List branches with **git branch**, create a new one with **git checkout -b**, and commit on it.

```
git branch
git checkout -b feature/retention
cat >> backup.sh <<'EOF'
# keep only the 7 newest archives
ls -1t /var/backups/etc-*.tar.gz | tail -n +8 | xargs -r rm -f
EOF
git commit -am "Keep only 7 backup archives"
git branch -v
```

```
* main
Switched to a new branch 'feature/retention'
[feature/retention cb4156a] Keep only 7 backup archives
 1 file changed, 2 insertions(+)
* feature/retention cb4156a Keep only 7 backup archives
  main              fc7035c Disable SSH password authentication
```

What just happened: `git checkout -b NAME` creates a branch at the current commit and switches to it in one step (the same as `git branch NAME` followed by `git checkout NAME`). The `*` marks the current branch. `cat >>` appended lines to the file. `git commit -a` stages every change to already-tracked files before committing, which saves a `git add` (it never adds untracked files). `git branch -v` shows the commit each branch points to.

2. Switch back to `main` and **merge** the branch. Because `main` has not moved, this is a fast-forward.

```
git switch main
tail -n 2 backup.sh
git merge feature/retention
tail -n 2 backup.sh
git log --oneline --graph
git branch -d feature/retention
```

```
Switched to branch 'main'
tar -czf "/var/backups/etc-$(date +%F).tar.gz" /etc
Updating fc7035c..cb4156a
Fast-forward
 backup.sh | 2 ++
 1 file changed, 2 insertions(+)
# keep only the 7 newest archives
ls -1t /var/backups/etc-*.tar.gz | tail -n +8 | xargs -r rm -f
* cb4156a (HEAD -> main) Keep only 7 backup archives
* fc7035c Disable SSH password authentication
* 4491226 Add backup script and SSH hardening config
Deleted branch feature/retention (was cb4156a).
```

What just happened: `git switch` (Git 2.23 and later) is a newer, clearer command just for changing branches; `git checkout main` does the same thing and is still widely used, so know both. When you switched to `main`, Git changed the files in your working directory to match: the new lines vanished, then came back after the merge. `Fast-forward` means Git just moved the `main` label forward; no merge commit was needed. `git branch -d` deletes a branch that has been merged (the commits stay in `main`).

3. Create a **three-way merge**: commit on a new branch *and* on `main`, then merge.

```
git switch -c feature/motd
echo 'Authorized use only. Activity is logged.' > motd.txt
git add motd.txt
git commit -m "Add login banner"
git switch main
echo 'ClientAliveInterval 300' >> sshd-hardening.conf
git commit -am "Add SSH idle timeout"
git merge feature/motd --no-edit
git log --oneline --graph
git branch -d feature/motd
```

```
Merge made by the 'ort' strategy.
 motd.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 motd.txt
*   7046d2e (HEAD -> main) Merge branch 'feature/motd'
|\
| * 72e85af Add login banner
* | 1ad82ab Add SSH idle timeout
|/
* cb4156a Keep only 7 backup archives
...
```

What just happened: `git switch -c` creates and switches, like `checkout -b`. Both branches had new commits, so Git combined them with a merge commit that has two parents; `--graph` draws the two lines of history joining. `ort` is Git's default merge strategy. `--no-edit` accepted the default message (`Merge branch 'feature/motd'`) instead of opening the editor. The branches changed different files, so there was no conflict.

4. Now cause a **merge conflict**: change the same line differently on two branches.

```
git switch -c fix/auth-tries
echo 'MaxAuthTries 3' >> sshd-hardening.conf
git commit -am "Limit SSH auth tries to 3"
git switch main
echo 'MaxAuthTries 5' >> sshd-hardening.conf
git commit -am "Limit SSH auth tries to 5"
git merge fix/auth-tries
git status
cat sshd-hardening.conf
```

```
Auto-merging sshd-hardening.conf
CONFLICT (content): Merge conflict in sshd-hardening.conf
Automatic merge failed; fix conflicts and then commit the result.
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   sshd-hardening.conf
PermitRootLogin no
PasswordAuthentication no
ClientAliveInterval 300
<<<<<<< HEAD
MaxAuthTries 5
=======
MaxAuthTries 3
>>>>>>> fix/auth-tries
```

What just happened: Both branches added a different line at the same place, so Git stopped and wrote **conflict markers** into the file. Between `<<<<<<< HEAD` and `=======` is your current branch's version (`main`); between `=======` and `>>>>>>> fix/auth-tries` is the incoming branch's version. The merge is paused; `git merge --abort` would cancel it and put everything back.

5. Resolve the conflict. You decide the team policy is 3 tries. Edit the file so it contains the final content you want with **no markers left**: open it with `nano sshd-hardening.conf` and make it look like this, or overwrite it with the command below.

```
printf 'PermitRootLogin no\nPasswordAuthentication no\nClientAliveInterval 300\nMaxAuthTries
3\n' \
  > sshd-hardening.conf
git add sshd-hardening.conf
git commit --no-edit
git log --oneline --graph -4
git branch -d fix/auth-tries
```

```
[main 8ee72ad] Merge branch 'fix/auth-tries'
*   8ee72ad (HEAD -> main) Merge branch 'fix/auth-tries'
|\
| * a80af20 Limit SSH auth tries to 3
* | 51cf885 Limit SSH auth tries to 5
|/
...
```

What just happened: The resolution steps are always: (1) edit each conflicted file into its final form and remove all markers, (2) `git add` each file to mark it resolved, (3) `git commit` to finish the merge. Before committing, `grep -n '<<<<<<<\|>>>>>>>' *` is a quick check that no markers were left behind; a config file with markers in it would break the service.

Checkpoint: `cat sshd-hardening.conf` ends with `MaxAuthTries 3` , `git status` is clean, and `git branch` lists only `main` .

Task 4: Squash merges and rebasing

1. Make a feature branch with several messy work-in-progress commits.

```
git switch -c feature/cleanup
printf '#!/bin/bash\n# cleanup.sh - remove old files from /tmp\n' > cleanup.sh
git add cleanup.sh
git commit -m "WIP: start cleanup script"
echo 'find /tmp -type f -mtime +7 -print' >> cleanup.sh
git commit -am "WIP: list old files"
sed -i 's/-print/-delete/' cleanup.sh
git commit -am "fix typo"
git log --oneline main..feature/cleanup
```

```
7ea46b6 (HEAD -> feature/cleanup) fix typo
c62bf41 WIP: list old files
da9718e WIP: start cleanup script
```

What just happened: `main..feature/cleanup` means "commits on `feature/cleanup` that are not on `main`". Three small commits like these are normal while working, but they clutter the shared history.

2. Merge with **squash** so `main` gets one clean commit.

```
git switch main
git merge --squash feature/cleanup
git status --short
git commit -m "Add /tmp cleanup script"
git log --oneline -3
git branch -d feature/cleanup
git branch -D feature/cleanup
```

```
Updating 8ee72ad..7ea46b6
Fast-forward
Squash commit -- not updating HEAD
 cleanup.sh | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 cleanup.sh
A  cleanup.sh
[main 563159c] Add /tmp cleanup script
 1 file changed, 3 insertions(+)
 create mode 100644 cleanup.sh
563159c (HEAD -> main) Add /tmp cleanup script
8ee72ad Merge branch 'fix/auth-tries'
51cf885 Limit SSH auth tries to 5
error: the branch 'feature/cleanup' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/cleanup'
Deleted branch feature/cleanup (was 7ea46b6).
```

What just happened: `git merge --squash` applied the combined changes of all three commits to your working directory and staging area but did **not** commit (`not updating HEAD`). You then wrote one good commit message. Because the original three commits never became part of `main`, Git thinks the branch is unmerged and `-d` refuses to delete it; `-D` forces the delete. GitHub and GitLab offer the same choice as a "Squash and merge" button on pull requests.

3. Set up a situation for **rebase**: a feature branch, and a new commit on `main` made after the branch was created.

```
git switch -c feature/report
printf '#!/bin/bash\n# report.sh - show disk and memory usage\ndf -h /\nfree -h\n' >
report.sh
git add report.sh
git commit -m "Add system report script"
git switch main
echo 'X11Forwarding no' >> sshd-hardening.conf
git commit -am "Disable X11 forwarding"
git log --oneline --graph --all -4
```

```
* 1c9891c (feature/report) Add system report script
| * f1a8daf (HEAD -> main) Disable X11 forwarding
|/
* 563159c Add /tmp cleanup script
...
```

4. Rebase the feature branch onto `main`, then merge it (now a fast-forward).

```
git switch feature/report
git rebase main
git log --oneline --graph --all -4
git switch main
git merge feature/report
git branch -d feature/report
```

```
Successfully rebased and updated refs/heads/feature/report.
* 6112214 (HEAD -> feature/report) Add system report script
* f1a8daf (main) Disable X11 forwarding
* 563159c Add /tmp cleanup script
...
Updating f1a8daf..6112214
Fast-forward
```

What just happened: `git rebase main` took the commits unique to `feature/report`, set them aside, moved the branch to the tip of `main`, and replayed them one by one. The commit got a **new hash** (`1c9891c` became `6112214`) because its parent changed. The history is now a straight line and the merge was a fast-forward. If a replayed commit conflicts, Git pauses: fix the file, `git add` it, and run `git rebase --continue` (or `git rebase --abort` to cancel).

CAUTION

Golden rule: **never rebase commits that you have already pushed and others may have pulled.** Rebasing rewrites history (new hashes), so other people's copies no longer match. Rebase only your own local, unshared work.

Checkpoint: `git log --oneline --graph -3` shows a straight line with `Add system report script` on top of `Disable X11 forwarding`.

Task 5: Stash, reset, and recover with reflog

1. Use **git stash** to put unfinished work aside when an urgent change comes in.

```
echo 'uptime' >> report.sh
git status --short
git stash push -m "half-done uptime line"
git status --short
git stash list
echo 'LoginGraceTime 30' >> sshd-hardening.conf
git commit -am "Shorten SSH login grace time"
git stash pop
git status --short
git commit -am "Show uptime in report"
```

```
 M report.sh
Saved working directory and index state On main: half-done uptime line
stash@{0}: On main: half-done uptime line
[main c599ce2] Shorten SSH login grace time
 1 file changed, 1 insertion(+)
On branch main
Changes not staged for commit:
...
        modified:   report.sh
Dropped refs/stash@{0} (00dd6ad...)
 M report.sh
[main 1317d07] Show uptime in report
```

What just happened: `git stash` (or `git stash push`) saved your uncommitted changes on a stack and reset the working directory to the last commit, so the urgent fix went into its own clean commit. `git stash pop` reapplied the saved changes and removed them from the stack. `git stash apply` would reapply but keep the stash; `git stash drop` deletes one without applying. `M` means modified but not staged.

2. Learn the three kinds of **git reset**. First make a commit to play with, then undo it with `--soft`.

```
echo 'echo "Report done"' >> report.sh
git commit -am "Print a final message"
git log --oneline -2
git reset --soft HEAD~1
git status --short
git log --oneline -1
```

```
[main d9e417e] Print a final message
d9e417e (HEAD -> main) Print a final message
1317d07 Show uptime in report
M  report.sh
1317d07 (HEAD -> main) Show uptime in report
```

What just happened: `git reset` moves the current branch to another commit. `HEAD~1` is the commit before `HEAD`. With `--soft`, only the branch moves: the change is still staged (`M` in the first column), ready to be recommitted, for example with a better message.

3. Commit again, then undo with `--mixed` (the default) and with `--hard`.

```
git commit -m "Print a final message"
git reset HEAD~1
git status --short
git commit -am "Print a final message"
git reset --hard HEAD~1
git status --short
tail -n 1 report.sh
```

```
Unstaged changes after reset:
M       report.sh
 M report.sh
HEAD is now at 1317d07 Show uptime in report
uptime
```

What just happened: The three modes differ in how much they reset:

Mode	Moves branch	Resets staging area	Resets working files
`--soft`	yes	no	no
`--mixed` (default)	yes	yes	no
`--hard`	yes	yes	yes

With `--mixed`, the change stayed in your file but was unstaged (`M` in the second column). With `--hard`, the change was thrown away completely; the file no longer has the `echo "Report done"` line. `git reset FILE` (no mode) is also how you unstage a single file, and `git restore --staged FILE` is the newer way to do the same.

CAUTION

`git reset --hard` discards uncommitted work with no confirmation and no undo. Committed work can usually be recovered (next step); changes that were never committed or stashed are gone for good.

4. Recover the "lost" commit using **reflog**, Git's diary of where `HEAD` has been.

```
git reflog | head -n 4
git reset --hard HEAD@{1}
tail -n 1 report.sh
```

```
1317d07 HEAD@{0}: reset: moving to HEAD~1
d9e417e HEAD@{1}: commit: Print a final message
1317d07 HEAD@{2}: reset: moving to HEAD~1
d9e417e HEAD@{3}: commit: Print a final message
HEAD is now at d9e417e Print a final message
echo "Report done"
```

What just happened: Every time `HEAD` moves (commit, reset, checkout, merge, rebase), Git writes a reflog entry. `HEAD@{1}` means "where HEAD was one move ago". Resetting back to it restored the commit. The reflog is local (it is never pushed) and entries expire after about 90 days, so it is a safety net, not a backup.

Checkpoint: `git log --oneline -1` shows `Print a final message` and `git status` is clean.

Task 6: Tag releases

1. Create a lightweight tag and an annotated tag with **git tag**.

```
git tag v1.0 HEAD~1
git tag -a v1.1 -m "Release 1.1: SSH hardening and reports"
git tag
git tag -n
git cat-file -t v1.0
git cat-file -t v1.1
git show v1.1 | head -n 8
```

```
v1.0
v1.1
v1.0            Show uptime in report
v1.1            Release 1.1: SSH hardening and reports
commit
tag
tag v1.1
Tagger: Student Admin <student@ubuntu-lab.local>
Date:   Sat Oct 10 10:14:34 2026 +0000

Release 1.1: SSH hardening and reports

commit d9e417e...
Author: Student Admin <student@ubuntu-lab.local>
```

What just happened: A tag is a permanent name for one commit, usually a release version. Unlike a branch, it never moves. A **lightweight** tag (`git tag v1.0 COMMIT`) is just a name pointing straight at a commit (`cat-file -t` says `commit`). An **annotated** tag (`-a` with `-m`) is a full Git object with its own tagger, date, and message (`cat-file -t` says `tag`); use annotated tags for releases. With no commit given, the tag goes on `HEAD` .

2. Use a tag to look at old files without changing your branch.

```
git diff v1.0 v1.1 --stat
git show v1.0:report.sh
```

What just happened: Tags work anywhere a commit hash works. `git diff v1.0 v1.1` compares two releases, and `git show TAG:FILE` prints a file as it was at that tag. To get an old version of a file back into your working directory, use `git checkout v1.0 -- report.sh` (or `git restore --source v1.0 report.sh`).

Checkpoint: `git tag -n` lists `v1.0` and `v1.1` with their messages.

Task 7: Share work through a remote on rocky-lab

You will make rocky-lab your "Git server" and simulate two admins: **you** in `~/labs/4.4/scripts` and **Bob** in a second clone, `~/labs/4.4/bob`.

1. SSH key login to rocky-lab is required. If you have not done Lab 3.3, set it up now (press Enter at any prompts).

```
test -f ~/.ssh/id_ed25519 || ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ssh-copy-id student@rocky-lab
```

2. Create an empty **bare** repository on rocky-lab, run from ubuntu-lab over SSH.

```
ssh student@rocky-lab 'git init --bare -b main ~/repos/scripts.git'
ssh student@rocky-lab 'ls ~/repos/scripts.git'
```

```
Initialized empty Git repository in /home/student/repos/scripts.git/
HEAD  branches  config  description  hooks  info  objects  refs
```

What just happened: `--bare` creates a repository with no working directory: just the contents that would normally be inside `.git`. Shared repositories are bare so nobody edits files directly on the server. By convention their names end in `.git`. `-b main` makes `main` the default branch to match yours. GitHub and GitLab create exactly this kind of repository when you click "New repository".

3. Connect your repository to the remote and **git push** your history and tags.

```
cd ~/labs/4.4/scripts
git remote add origin student@rocky-lab:repos/scripts.git
git remote -v
git push -u origin main
git push origin --tags
```

```
origin  student@rocky-lab:repos/scripts.git (fetch)
origin  student@rocky-lab:repos/scripts.git (push)
Enumerating objects: 45, done.
...
To rocky-lab:repos/scripts.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
To rocky-lab:repos/scripts.git
 * [new tag]         v1.0 -> v1.0
 * [new tag]         v1.1 -> v1.1
```

What just happened: `git remote add origin URL` saved the remote under the name `origin` (the conventional name for the main remote). The URL `user@host:path` is SSH syntax; the path is relative to the user's home directory. `git push -u origin main` uploaded `main`, and `-u` set it as the **upstream** so later you can type just `git push` or `git pull`. Tags are not pushed by default, so `--tags` sent them. With GitHub the URL would look like `git@github.com:yourname/scripts.git`.

4. Become Bob: **git clone** the repository into a second folder and give that clone Bob's identity.

```
git clone student@rocky-lab:repos/scripts.git ~/labs/4.4/bob
cd ~/labs/4.4/bob
git config user.name "Bob Admin"
git config user.email "bob@ubuntu-lab.local"
git log --oneline -3
git branch -a
git tag
```

```
Cloning into '/home/student/labs/4.4/bob'...
...
d9e417e (HEAD -> main, tag: v1.1, origin/main, origin/HEAD) Print a final message
1317d07 (tag: v1.0) Show uptime in report
c599ce2 Shorten SSH login grace time
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
v1.0
v1.1
```

What just happened: `git clone` copied the full history, created the `origin` remote automatically, and checked out `main`. `git config` without `--global` wrote to this clone's `.git/config`, so commits here are signed "Bob Admin". `remotes/origin/main` is a **remote-tracking branch**: your local record of where `main` was on the remote the last time you talked to it.

5. Bob commits and pushes. Then you **git fetch** to see his work before merging it, and **git pull** to bring it in.

```
echo 'PermitEmptyPasswords no' >> sshd-hardening.conf
git commit -am "Forbid empty SSH passwords"
git push
cd ~/labs/4.4/scripts
git fetch
git status | head -n 3
git log --oneline main..origin/main
git pull
```

```
To rocky-lab:repos/scripts.git
   d9e417e..b7fe844  main -> main
From rocky-lab:repos/scripts
   d9e417e..b7fe844  main       -> origin/main
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)
b7fe844 Forbid empty SSH passwords
Updating d9e417e..b7fe844
Fast-forward
 sshd-hardening.conf | 1 +
 1 file changed, 1 insertion(+)
```

What just happened: `git fetch` downloaded Bob's commit and moved `origin/main`, but did not touch your files or your `main`. That lets you review incoming work first (`git log main..origin/main`, or `git diff main origin/main`). `git pull` then merged it, which was a fast-forward because you had no new commits of your own.

6. Now both admins work at the same time and the histories **diverge**. Bob pushes first; your push is rejected.

```
cd ~/labs/4.4/bob
git pull
echo 'df -i /' >> report.sh
git commit -am "Show inode usage in report"
git push
cd ~/labs/4.4/scripts
echo 'Banner /etc/issue.net' >> sshd-hardening.conf
git commit -am "Show a pre-login banner"
git push
```

```
To rocky-lab:repos/scripts.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'rocky-lab:repos/scripts.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

What just happened: Git only accepts a push that is a fast-forward of the remote branch, so it can never silently throw away someone else's commits. Bob's commit is on the server but not in your history, so you must integrate it first. (Never "fix" this with `git push --force` on a shared branch; that would delete Bob's work from the server.)

7. Pull to integrate Bob's work, check the result, and push again.

```
git pull --no-edit
git log --oneline --graph -4
git push
```

```
From rocky-lab:repos/scripts
   b7fe844..fb91df2  main       -> origin/main
Merge made by the 'ort' strategy.
 report.sh | 1 +
 1 file changed, 1 insertion(+)
*   eee9bf1 (HEAD -> main) Merge branch 'main' of rocky-lab:repos/scripts
|\
| * fb91df2 (origin/main) Show inode usage in report
* | 9bc3f67 Show a pre-login banner
|/
* b7fe844 Forbid empty SSH passwords
To rocky-lab:repos/scripts.git
   fb91df2..eee9bf1  main -> main
```

What just happened: Because you set `pull.rebase false` in Task 1, `git pull` fetched and then made a merge commit joining the two lines of work. Without that setting, Git 2.27 and later stop with `fatal: Need to specify how to reconcile divergent branches` and ask you to choose. The alternative, `git pull --rebase`, would replay your commit on top of Bob's for a straight-line history; that is safe here because your commit had not been pushed yet. If both of you had edited the same line, the pull would stop with a conflict, which you would resolve exactly as in Task 3.

8. Finish as Bob: pull everything, including your merge.

```
cd ~/labs/4.4/bob
git pull
git log --oneline -1
tail -n 1 sshd-hardening.conf
```

```
Updating fb91df2..eee9bf1
Fast-forward
 sshd-hardening.conf | 1 +
 1 file changed, 1 insertion(+)
eee9bf1 (HEAD -> main, origin/main, origin/HEAD) Merge branch 'main' of rocky-lab:...
Banner /etc/issue.net
```

TIP

On GitHub or GitLab the workflow adds one step: instead of pushing straight to `main`, you push a branch (`git push -u origin feature/x`) and open a **pull request** (GitLab calls it a **merge request**). Teammates review the diff, the CI pipeline runs tests (see Lab 4.1), and the platform merges it with a merge commit, a squash, or a rebase. The Git commands underneath are the ones you just used.

Checkpoint: In both `~/labs/4.4/scripts` and `~/labs/4.4/bob`, `git log --oneline -1` shows the same merge commit hash, and `git status` says `Your branch is up to date with 'origin/main'`.

Challenge

CHALLENGE

1. As Bob, change `MaxAuthTries 3` to `MaxAuthTries 4` and push. As yourself, without pulling, change the same line to `MaxAuthTries 2` and commit. Run `git pull --rebase`, resolve the conflict, finish with `git rebase --continue`, and push. Draw the resulting history with `git log --oneline --graph`.
2. Create a branch `hotfix/backup-path`, commit a change, push the branch to `origin` with `git push -u origin hotfix/backup-path`, and from Bob's clone use `git fetch` and `git switch hotfix/backup-path` to review it before merging it into `main`.
3. Use `git stash` with two different stashes, list them, and apply the older one (`git stash apply stash@{1}`) first.

Clean up

1. Delete the bare repository on rocky-lab.

```
ssh student@rocky-lab 'rm -rf ~/repos/scripts.git && rmdir ~/repos 2>/dev/null; ls ~'
```

2. Delete both local repositories.

```
cd ~
rm -rf ~/labs/4.4
```

3. Your global Git identity is useful in later labs, so keep it. To remove the other settings this lab added:

```
git config --global --unset core.editor
git config --global --unset pull.rebase
git config --global --list
```

Exam corner

- `git config --global user.name/user.email` sets your identity; levels are `--system`, `--global`, `--local`.
- `git init` creates a repo; `git init --bare` creates a server-side repo with no working directory; `git clone URL` copies a remote repo.
- Workflow: edit, `git add` (stage), `git commit -m` (snapshot). `git status` shows state; `git log` (`--oneline --graph --all`) shows history.
- `git diff` = working vs staged; `git diff --staged` = staged vs last commit; `git diff A B` = between commits.
- `.gitignore` lists patterns (`*.log`, `tmp/`) for untracked files Git should ignore; it does not remove files already committed.
- `git branch` lists/creates branches; `git checkout -b NAME` or `git switch -c NAME` creates and switches; `git branch -d` deletes a merged branch (`-D` forces).
- `git merge BRANCH`: fast-forward when possible, otherwise a merge commit. Conflicts show `<<<<<<<`, `=======`, `>>>>>>>`; fix, `git add`, `git commit`. `git merge --squash` combines a branch into one staged change you then commit.
- `git rebase main` replays your commits on top of `main` (linear history, new hashes). Never rebase published commits.
- `git stash` saves uncommitted work; `git stash pop` restores it; `git stash list` shows the stack.
- `git reset --soft` (keep staged), `--mixed` (default, keep in working dir), `--hard` (discard everything). `git reflog` finds lost commits.
- `git tag v1.0` (lightweight) vs `git tag -a v1.0 -m "msg"` (annotated); push tags with `git push --tags`.
- `git fetch` downloads without merging; `git pull` = fetch + merge (or `--rebase`); `git push` uploads and is rejected if it is not a fast-forward.

Review questions

1. You staged a file by mistake but want to keep your edits in the working directory. Which command unstages it?
   A. `git reset --hard`
   B. `git reset FILE` (or `git restore --staged FILE`)
   C. `git stash drop`
   D. `git rm FILE`

2. What is the difference between `git fetch` and `git pull`?

3. A teammate's feature branch has 12 small commits. You want them to appear on `main` as a single commit. Which command do you use before committing?

4. Which file tells Git never to track `*.log` files?

5. You ran `git reset --hard HEAD~2` and lost two commits you needed. Which command helps you find them?
   A. `git log --all`
   B. `git stash list`
   C. `git reflog`
   D. `git tag -n`

6. What is the difference between a lightweight tag and an annotated tag?

Lab 4.5 Use AI Responsibly as a Linux Admin

Objective: 4.5 Summarize best practices and responsible uses of artificial intelligence (AI).
Domain: 4.0 Automation, Orchestration, and Scripting (17% of the exam)
Time: about 2 hours
VM: ubuntu-lab
Snapshot first: Not required. You never run the unsafe code, and everything else stays in `~/labs/4.5`.

AI assistants built on large language models can write a Bash script, a regex, or an Ansible playbook in seconds. They can also confidently produce code that deletes the wrong directory, a regex that accepts garbage, or a playbook that leaks a password. In this lab you play the role of the senior admin who reviews AI output before it goes anywhere near production. You do not need an AI account: the book gives you realistic AI-generated artifacts, and you verify them with real Linux tools. You then practice writing better prompts and protecting company data before anything is sent to a model.

What you'll practice

- Use cases: generation of code, regular expressions, infrastructure as code, documenting code and creating documentation, recommendations for compliance, security review, code optimization, code linting
- Best practices: avoid copy/paste without review and QA, verify output
- Data governance: security of data (LLM training, human review), local models (private vs public)
- Adhere to corporate policy
- Prompt engineering

How it works

A **large language model (LLM)** is a program trained on huge amounts of text, including public code, documentation, and forum posts. Given a prompt, it predicts the most likely next words, one after another. That makes it excellent at producing text that *looks* right: familiar command patterns, plausible YAML, well-structured explanations. It does not run your code, check your system, or know your environment. When it does not know something, it can **hallucinate**: invent an option, a module name, or a package that does not exist, with the same confident tone as a correct answer.

Think of an AI assistant as a fast, well-read junior colleague who has never seen your servers. Their drafts save you time, but you would never let them push to production without review. The rule for every use case is the same:

```
prompt  -->  AI draft  -->  REVIEW  -->  VERIFY  -->  TEST (lab)  -->  commit/PR  -->  production
                            (read       (lint, -n,    (safe data,      (human code
                            every       --check,      snapshot)         review, CI)
                            line)       test cases)
```

Where AI helps an admin:

Use case	Example request	How you verify
Generation of code	"Write a Bash script that rotates logs"	`bash -n`, ShellCheck, run in a sandbox
Regular expressions	"Regex for an IPv4 address"	`grep -E` against good and bad test inputs
Infrastructure as code	"Ansible playbook to install nginx"	`ansible-lint`, `--syntax-check`, `--check --diff`
Document code / create documentation	"Add comments and a usage section"	Read it: does it match what the code really does?
Recommendations for compliance	"Which sshd settings does CIS recommend?"	Compare with the official benchmark and `sshd -T`
Security review	"Find security problems in this script"	Treat findings as leads; confirm each one yourself
Code optimization	"Make this loop faster"	Time both versions, confirm identical output
Code linting	"Explain this ShellCheck warning"	Re-run the linter after the fix

The second big concern is **data governance**: rules about what data may go where. Anything you paste into a public AI service leaves your control. Depending on the provider and the plan, prompts may be stored, used for **training** future models, or read by the provider's staff during **human review** for safety and quality. A password, customer record, or internal host name pasted into a chat is effectively disclosed. Options range from **public** consumer services, to business plans with contractual promises not to train on your data, to **private** or **local models** that run entirely on your own hardware (for example with Ollama), where data never leaves your network but answers are usually less capable.

Finally, your employer's **corporate policy** decides which tools are approved, what data classes may be shared, whether AI-generated code must be labeled, and who is accountable. The accountable person is always the human who commits the code, never the AI.

Tasks

Task 1: Prepare your review toolkit

1. Create the lab directory and install the verification tools you will use.

```
mkdir -p ~/labs/4.5 && cd ~/labs/4.5
sudo apt update
sudo apt install -y shellcheck ansible ansible-lint python3
shellcheck --version | head -n 2
ansible-lint --version
python3 --version
```

```
ShellCheck - shell script analysis tool
version: 0.9.0
ansible-lint 6.17.2 using ansible-core:2.16.3 ...
Python 3.12.3
```

What just happened: You installed a **linter** for each language you will review. Code linting means automatically checking code for bugs, risky patterns, and style problems without running it. ShellCheck lints shell scripts, `ansible-lint` lints Ansible playbooks, and Python itself can turn warnings into errors. Linters are the first, cheapest layer of verification; they never replace reading the code.

Checkpoint: `which shellcheck ansible-lint python3` prints three paths.

Task 2: Review AI-generated Bash code

An admin asked an AI assistant: "Write a script that deletes logs older than N days and empties our app's temp directory." This is what came back.

1. Save the AI's script exactly as given. You will use a heredoc: `cat > file <<'EOF'` writes every line up to `EOF` into `file`, and the quotes around `'EOF'` keep `$` signs from being expanded.

```
cat > cleanup_old_logs.sh <<'EOF'
#!/bin/bash
# cleanup_old_logs.sh - generated by an AI assistant
# Deletes application logs older than N days, then empties the temp directory.
LOG_DIR=$1
DAYS=$2
TMP_DIR=$(cat /etc/myapp/tmpdir 2>/dev/null)

for f in $(find $LOG_DIR -name *.log -mtime +$DAYS); do
    rm $f
done

rm -rf $TMP_DIR/*
echo "Cleanup complete"
EOF
```

CAUTION

Do not run this script. It is saved only so you can analyze it. On this VM, `/etc/myapp/tmpdir` does not exist, and you are about to see what that means for the `rm -rf` line.

2. Read it yourself first. Then run a syntax check. Does passing it mean the script is safe?

```
bash -n cleanup_old_logs.sh && echo "syntax OK"
```

```
syntax OK
```

> **What just happened:** `bash -n` parses the script without running it. It only proves the script is valid Bash grammar. A script can be perfectly valid and still destroy a server. Syntax checks are necessary but nowhere near sufficient.

3. Lint it with ShellCheck.

```
shellcheck cleanup_old_logs.sh
```

```
In cleanup_old_logs.sh line 8:
for f in $(find $LOG_DIR -name *.log -mtime +$DAYS); do
         ^-- SC2044 (warning): For loops over find output are fragile. Use find -exec
or a while read loop.
                ^------^ SC2086 (info): Double quote to prevent globbing and word ...
                                ^---^ SC2061 (warning): Quote the parameter to -name so
the shell won't interpret it.
...
In cleanup_old_logs.sh line 12:
rm -rf $TMP_DIR/*
       ^--------^ SC2115 (warning): Use "${var:?}" to ensure this never expands to /* .
...
```

> **What just happened:** ShellCheck found real bugs:
>
> - **SC2115** is the dangerous one. If `TMP_DIR` is empty (the config file is missing, unreadable, or empty), `rm -rf $TMP_DIR/*` becomes `rm -rf /*`, and as root that deletes the whole system.
> - **SC2061**: `-name *.log` is unquoted, so if the current directory contains `a.log`, the shell turns it into `-name a.log` and `find` only looks for that one name.
> - **SC2044 / SC2086**: `for f in $(find ...)` splits on spaces, so a file named `old app.log` becomes two words, `old` and `app.log`. `rm` would fail on both, or delete the wrong files.

4. Prove the danger safely: replace `rm -rf` with `echo` to see what the line would really run when the variable is empty.

```
TMP_DIR=""
echo rm -rf $TMP_DIR/* | cut -c1-70
unset TMP_DIR
```

```
rm -rf /bin /bin.usr-is-merged /boot /cdrom /dev /etc /home /lib /lib64
```

> **What just happened:** The empty variable left just `/*`, and the shell expanded that into every top-level directory. Putting `echo` in front of a destructive command is a simple, powerful way to **verify output** before trusting it.

5. Write the reviewed version. It refuses empty or dangerous arguments, quotes everything, avoids looping over `find` output, and has a dry-run mode.

```
cat > cleanup_fixed.sh <<'EOF'
#!/bin/bash
# cleanup_fixed.sh - the AI script after human review
# Usage: cleanup_fixed.sh LOG_DIR DAYS TMP_DIR [--dry-run]
set -euo pipefail

LOG_DIR="${1:?Usage: cleanup_fixed.sh LOG_DIR DAYS TMP_DIR [--dry-run]}"
DAYS="${2:?missing DAYS}"
TMP_DIR="${3:?missing TMP_DIR}"
MODE="${4:-}"

if [[ ! $DAYS =~ ^[0-9]+$ ]]; then
    echo "DAYS must be a whole number" >&2
    exit 2
fi
for d in "$LOG_DIR" "$TMP_DIR"; do
    if [[ ! -d $d || $(realpath "$d") == / ]]; then
        echo "Refusing to use '$d': not a directory, or it is /" >&2
        exit 2
    fi
done

if [[ $MODE == --dry-run ]]; then
    find "$LOG_DIR" -type f -name '*.log' -mtime +"$DAYS" -print
    find "$TMP_DIR" -mindepth 1 -maxdepth 1 -print
else
    find "$LOG_DIR" -type f -name '*.log' -mtime +"$DAYS" -delete
    find "$TMP_DIR" -mindepth 1 -maxdepth 1 -exec rm -rf -- {} +
fi
echo "Cleanup complete"
EOF
chmod +x cleanup_fixed.sh
shellcheck cleanup_fixed.sh && echo "ShellCheck: clean"
```

What just happened: `${3:?...}` aborts if an argument is missing or empty, so an empty `TMP_DIR` can never reach the delete. The regex test makes sure `DAYS` is a number. `realpath` resolves tricks like `/tmp/..` to check the real path is not `/`. `find -delete` and `find -exec rm -rf -- {} +` handle any file name safely; `--` stops a file named `-rf` from being read as an option. This is also a **code optimization**: `find -delete` removes files in one process instead of starting `rm` once per file.

6. Test it, but only on throwaway data. Build a sandbox with tricky file names, then try bad arguments, a dry run, and a real run.

```
mkdir -p sandbox/logs sandbox/tmp/sub
touch -d '30 days ago' "sandbox/logs/old app.log" sandbox/logs/old2.log
touch sandbox/logs/new.log sandbox/tmp/a sandbox/tmp/-rf
./cleanup_fixed.sh sandbox/logs 7 ""; echo "status: $?"
./cleanup_fixed.sh sandbox/logs 7 /; echo "status: $?"
./cleanup_fixed.sh sandbox/logs abc sandbox/tmp; echo "status: $?"
./cleanup_fixed.sh sandbox/logs 7 sandbox/tmp --dry-run
./cleanup_fixed.sh sandbox/logs 7 sandbox/tmp
ls -A sandbox/logs sandbox/tmp
```

```
./cleanup_fixed.sh: line 8: 3: missing TMP_DIR
status: 1
Refusing to use '/': not a directory, or it is /
status: 2
DAYS must be a whole number
status: 2
sandbox/logs/old app.log
sandbox/logs/old2.log
sandbox/tmp/-rf
sandbox/tmp/sub
sandbox/tmp/a
Cleanup complete
Cleanup complete
sandbox/logs:
new.log

sandbox/tmp:
```

What just happened: This is QA (quality assurance): testing the edge cases the AI never considered. The dangerous inputs were rejected, the dry run listed exactly the right files (including the one with a space and the one named `-rf`), and the real run deleted only old logs and the temp contents. Only now would the script be ready for a pull request and a human code review.

Checkpoint: `shellcheck cleanup_fixed.sh` prints nothing, and `ls sandbox/logs` shows only `new.log`.

Task 3: Verify an AI-generated regular expression

The admin asked: "Give me a regex that matches valid IPv4 addresses, for `grep -E`." The AI offered two answers on different tries:

- `^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$`
- `^([0-9]{1,3}\.){3}[0-9]{1,3}$`

1. Build a test file that contains both valid and invalid addresses. Good test data is the heart of verifying a regex.

```
printf '%s\n' 192.168.1.10 10.0.0.255 0.0.0.0 255.255.255.255 \
  256.1.1.1 999.999.999.999 192.168.1 1.2.3.4.5 a.b.c.d 192.168.001.010 > ips.txt
cat ips.txt
```

What just happened: The first four lines are valid IPv4 addresses (including the edge values 0 and 255). The rest are invalid: octets above 255, too few or too many parts, letters, and leading zeros (which some tools treat as octal, so many standards reject them).

2. Test the first AI regex.

```
grep -E '^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$' ips.txt; echo "status: $?"
```

```
status: 1
```

What just happened: It matched **nothing**, not even the valid addresses. `\d` ("a digit") is Perl-style syntax. POSIX extended regex, used by `grep -E`, `sed -E`, and Bash's `=~`, does not support it (GNU grep treats it as a literal `d`, and some versions print a "stray " warning). It would work with `grep -P` (Perl mode), but the AI did not mention that. Regex dialects are a classic source of AI mistakes.

3. Test the second AI regex.

```
grep -E '^([0-9]{1,3}\.){3}[0-9]{1,3}$' ips.txt
```

```
192.168.1.10
10.0.0.255
0.0.0.0
255.255.255.255
256.1.1.1
999.999.999.999
192.168.001.010
```

What just happened: It looks right on the "happy path" but accepts `256.1.1.1` and `999.999.999.999`, because "one to three digits" is not the same as "0 to 255". If this regex guarded a firewall script, invalid input would get through.

4. Write and test a correct pattern. Build it from a pattern for one octet so it stays readable.

```
OCTET='(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])'
echo "Should match:"
grep -E "^($OCTET\.){3}$OCTET$" ips.txt
echo "Should NOT match:"
grep -vE "^($OCTET\.){3}$OCTET$" ips.txt
```

```
Should match:
192.168.1.10
10.0.0.255
0.0.0.0
255.255.255.255
Should NOT match:
256.1.1.1
999.999.999.999
192.168.1
1.2.3.4.5
a.b.c.d
192.168.001.010
```

What just happened: The octet pattern lists the allowed ranges as alternatives: `25[0-5]` (250 to 255), `2[0-4][0-9]` (200 to 249), `1[0-9]{2}` (100 to 199), and `[1-9]?[0-9]` (0 to 99, no leading zero). `grep -v` prints lines that do *not* match, so you can check both directions. Keep `ips.txt` as a permanent test file: if anyone (human or AI) changes the regex later, rerun both commands.

Checkpoint: The final pattern matches exactly 4 lines: `grep -cE "^($OCTET\.){3}$OCTET$" ips.txt` prints `4`.

Task 4: Security review of AI-generated infrastructure as code

The admin asked: "Write an Ansible playbook that installs our monitoring agent and writes its config with our API token."

1. Save the AI's playbook.

```
cat > deploy_app.yml <<'EOF'
---
# deploy_app.yml - generated by an AI assistant
- name: Deploy the monitoring agent
  hosts: localhost
  become: true
  vars:
    api_token: "sk-live-4f9a8b7c6d5e"
  tasks:
    - name: Install the agent
      ansible.builtin.shell: curl -k http://downloads.example.com/agent.sh | bash

    - name: Write the agent config
      ansible.builtin.copy:
        dest: /etc/agent.conf
        content: "token={{ api_token }}\n"
        mode: "0777"
EOF
```

2. Run the automated checks.

```
ansible-playbook deploy_app.yml --syntax-check
ansible-lint deploy_app.yml
```

```
playbook: deploy_app.yml
WARNING  Listing 3 violation(s) that are fatal
command-instead-of-module: curl used in place of get_url or uri module
deploy_app.yml:9 Task/Handler: Install the agent

no-changed-when: Commands should not change things if nothing needs doing.
deploy_app.yml:9 Task/Handler: Install the agent

risky-shell-pipe: Shells that use pipes should set the pipefail option.
deploy_app.yml:9 Task/Handler: Install the agent
...
Failed: 3 failure(s), 0 warning(s) on 1 files.
```

What just happened: The syntax is valid, and `ansible-lint` found three problems, all on the shell task. Now look at what the linter did **not** report. This is why a human security review is still required.

3. Do the human security review. Read the playbook line by line and compare with this checklist. Write down every problem you find before reading the explanation.

```
Security review checklist for scripts and IaC
[ ] Secrets hard-coded in the file (passwords, tokens, keys)?
[ ] Downloads over plain HTTP, or TLS verification disabled (-k, validate_certs: no)?
[ ] Remote code piped straight into a shell (curl ... | bash)?
[ ] File permissions wider than needed (0777, 0666, world-readable secrets)?
[ ] Secrets printed in logs or command output?
[ ] Commands that are not idempotent, or that run as root without need?
[ ] Untrusted input used in a shell command (injection)?
```

What just happened: The review finds five serious issues the linter missed or understated:

1. **Hard-coded secret:** `api_token` sits in plain text in a file that will be committed to Git and shared.
2. **No transport security:** `http://` plus `curl -k` (do not verify certificates) lets anyone on the network path swap the installer.
3. **`curl | bash` as root:** whatever the server returns is executed immediately, with no checksum.
4. **Mode `0777`:** every user on the server can read the token and even rewrite the config.
5. **Secret in output:** without `no_log`, Ansible may print the token in its output and logs.

4. Write the reviewed version.

```
cat > deploy_app_fixed.yml <<'EOF'
---
# deploy_app_fixed.yml - the AI playbook after human review
- name: Deploy the monitoring agent
  hosts: localhost
  become: true
  vars:
    agent_url: https://downloads.example.com/agent.sh
    agent_sha256: "0000000000000000000000000000000000000000000000000000000000000000"
    api_token: "{{ lookup('ansible.builtin.env', 'AGENT_API_TOKEN') }}"
  tasks:
    - name: Download the installer over verified HTTPS
      ansible.builtin.get_url:
        url: "{{ agent_url }}"
        dest: /usr/local/sbin/agent-install.sh
        checksum: "sha256:{{ agent_sha256 }}"
        owner: root
        group: root
        mode: "0750"

    - name: Run the installer only if the agent is not installed yet
      ansible.builtin.command: /usr/local/sbin/agent-install.sh
      args:
        creates: /opt/agent/bin/agent

    - name: Write the agent config readable by root only
      ansible.builtin.copy:
        dest: /etc/agent.conf
        content: "token={{ api_token }}\n"
        owner: root
        group: root
        mode: "0600"
      no_log: true
EOF
ansible-playbook deploy_app_fixed.yml --syntax-check
ansible-lint deploy_app_fixed.yml
```

```
playbook: deploy_app_fixed.yml
Passed: 0 failure(s), 0 warning(s) on 1 files. Last profile that met the validation
criteria was 'production'.
```

What just happened: The token now comes from an environment variable at run time, so it never lands in Git (in production you would use `ansible-vault` or a secrets manager). `get_url` downloads over HTTPS with certificate checks on by default, and the `checksum` makes Ansible reject a tampered file (the zeros are a placeholder for the vendor's published SHA-256). `creates:` makes the installer idempotent. The config is `0600` and `no_log: true` hides the task's data from output. Do not run this playbook: `downloads.example.com` is a placeholder.

Checkpoint: `ansible-lint deploy_app_fixed.yml` reports `Passed`, and you can name the five problems in the original without looking.

Task 5: Verify AI-generated Python, compliance advice, and documentation

1. The admin asked for "a Python script that logs a syslog warning when a disk is over 80% full". Save the answer.

```
cat > disk_alert.py <<'EOF'
#!/usr/bin/env python3
"""disk_alert.py - generated by an AI assistant."""
import datetime
import os
import shutil
import sys


def check(path):
    total, used, free = shutil.disk_usage(path)
    pct = used / total * 100
    if pct > 80:
        os.system("logger -t disk_alert 'Disk " + path + " is "
                  + str(int(pct)) + "% full'")
    return pct


path = sys.argv[1] if len(sys.argv) > 1 else "/"
print(datetime.datetime.utcnow().isoformat(), round(check(path), 1))
EOF
python3 disk_alert.py
python3 -W error disk_alert.py; echo "status: $?"
```

```
/home/student/labs/4.5/disk_alert.py:19: DeprecationWarning: datetime.datetime.utcnow()
is deprecated and scheduled for removal in a future version. Use timezone-aware objects
to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  print(datetime.datetime.utcnow().isoformat(), round(check(path), 1))
2026-10-10T10:19:19.127546 26.5
Traceback (most recent call last):
  File "/home/student/labs/4.5/disk_alert.py", line 19, in <module>
...
DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal
in a future version. ...
status: 1
```

What just happened: The script worked, but Python 3.12 warned that `datetime.utcnow()` is **deprecated** (scheduled for removal). AI models learn from years of older code, so they often suggest outdated calls. `python3 -W error` turns every warning into an error, a strict way to catch these in testing or CI. Inside an imported module this warning is hidden by default, so without `-W` it could slip by unnoticed until the function is removed.

2. Review the rest of the code. The `os.system()` line builds a shell command by gluing strings together, including `path`, which comes from the command line. A path containing a single quote and a semicolon, such as `/mnt/x';reboot;'`, would end the quoted string and run `reboot` as a separate command. This is **command injection**. Also, `used / total` gives a different percentage from `df`. Write the reviewed version:

```
cat > disk_alert_fixed.py <<'EOF'
#!/usr/bin/env python3
"""disk_alert_fixed.py - the AI script after human review."""
import datetime
import shutil
import subprocess
import sys


def check(path, limit=80.0):
    """Return percent used for path; log a syslog warning above limit."""
    _total, used, free = shutil.disk_usage(path)
    pct = used / (used + free) * 100
    if pct > limit:
        message = f"Disk {path} is {pct:.0f}% full"
        subprocess.run(["logger", "-t", "disk_alert", message], check=True)
    return pct


if __name__ == "__main__":
    target = sys.argv[1] if len(sys.argv) > 1 else "/"
    now = datetime.datetime.now(datetime.timezone.utc)
    print(now.isoformat(timespec="seconds"), round(check(target), 1))
EOF
python3 -W error disk_alert_fixed.py; echo "status: $?"
```

```
2026-10-10T10:19:19+00:00 27.5
status: 0
```

What just happened: `subprocess.run([...])` passes each argument separately with no shell, so quotes and semicolons in `path` are just characters. A timezone-aware `datetime.now(datetime.timezone.utc)` replaces the deprecated call. The percentage now matches `df -h /`. The function also gained a docstring and a configurable limit.

3. Check a **compliance recommendation**. Suppose an AI says: "To comply with the CIS Ubuntu benchmark, set `PermitRootLogin no`, `PasswordAuthentication no`, `MaxAuthTries 4`, and `X11Forwarding no` in sshd." Do not edit anything yet. First see what your server actually enforces now.

```
sudo sshd -T | grep -Ei
'^(permitrootlogin|passwordauthentication|maxauthtries|x11forwarding) '
```

```
permitrootlogin without-password
passwordauthentication yes
maxauthtries 6
x11forwarding yes
```

What just happened: `sshd -T` prints the *effective* configuration after all files in `/etc/ssh/sshd_config.d/` are merged, which is the truth you compare against. The AI's list is a reasonable starting point, but compliance is defined by the official benchmark document for your exact OS version (CIS, DISA STIG, or your company's baseline), not by a chat answer. Verify each item against that source, test the change on a lab VM, and keep evidence (such as this `sshd -T` output) for auditors. Turning off password logins before your SSH keys work could lock you out.

4. Check AI-written **documentation**. An assistant asked to "document `cleanup_fixed.sh`" produced this usage text. Compare every claim with the script.

```
cleanup_fixed.sh - remove old logs and temp files
Usage: cleanup_fixed.sh LOG_DIR DAYS TMP_DIR [--dry-run]
  LOG_DIR    directory searched recursively for *.log and *.gz files
  DAYS       delete logs modified more than DAYS days ago
  TMP_DIR    directory whose contents are deleted (the directory itself is kept)
  --dry-run  list what would be deleted without deleting anything
Exit codes: 0 success, 1 missing argument, 2 invalid argument
```

What just happened: Almost everything is right, which makes the one error easy to miss: the script deletes only `*.log` files, not `*.gz`. Wrong documentation is worse than none, because people trust it. Correct the line before committing. AI is genuinely useful for drafting documentation, comments, and README files, but the author still verifies every statement.

Checkpoint: `python3 -W error disk_alert_fixed.py` exits with status 0, and you have found the `*.gz` error in the documentation.

Task 6: Prompt engineering

Prompt engineering is the skill of writing requests that get useful, accurate, safe answers. Vague prompts get generic code that ignores your environment. A good prompt states the role, the context, the task, the constraints, the output format, and examples.

1. Save a reusable prompt template. The file is `~/labs/4.5/prompt-template.txt`:

```
cat > prompt-template.txt <<'EOF'
ROLE:        You are a senior Linux administrator who writes safe, idempotent code.
CONTEXT:     <OS and version, tools available, what already exists>
TASK:        <one clear job, with inputs and expected result>
CONSTRAINTS: <shell or language version, no root unless needed, must pass a linter,
              no hard-coded secrets, handle errors and empty input, dry-run option>
OUTPUT:      <format: one file in a code block, then a short list of assumptions
              and a test plan>
EXAMPLES:    <sample input and the exact output you expect>
DO NOT:      <include real host names, IPs, passwords, or customer data>
EOF
cat prompt-template.txt
```

2. Compare a vague prompt with an engineered one for the same job.

```
VAGUE:
write a script to clean up logs

ENGINEERED:
ROLE: You are a senior Linux administrator who writes safe, idempotent Bash.
CONTEXT: Ubuntu Server 24.04, Bash 5.2, GNU findutils. Logs are in a directory
passed as an argument. The script runs from cron as an unprivileged user.
TASK: Delete *.log files older than N days (N is the second argument).
CONSTRAINTS: Must pass ShellCheck with no warnings. Quote all variables. Refuse to
run if an argument is empty, not a directory, or resolves to /. No loops over
find output. Add a --dry-run option. Exit 2 on invalid arguments.
OUTPUT: One script in a code block, then a list of assumptions, then three test
commands I can run on a throwaway directory.
EXAMPLES: "cleanup.sh /var/log/myapp 14 --dry-run" lists matching files only.
```

What just happened: The engineered prompt removes guesswork. It names the OS and versions (so the model avoids outdated or non-GNU syntax), states safety rules that would have prevented every bug in Task 2, requires a linter pass, and asks for assumptions and tests, which makes the answer easier to verify. Even with a perfect prompt, you still review and test the result.

3. Practice: rewrite each vague prompt below into a file called `my-prompts.txt` using the template. Write at least the CONTEXT, CONSTRAINTS, and OUTPUT lines for each.

```
1. "make a regex for emails"
2. "ansible playbook for nginx"
3. "why is my server slow"
```

```
nano my-prompts.txt
```

> **TIP**
> For number 3, good context includes the output of `uptime`, `free -h`, `df -h`, and `top -bn1 | head -n 15`, **after** you check it for host names, user names, and IP addresses you are not allowed to share. Useful extra techniques: ask the model to "list your assumptions" and "say if you are not sure", ask for the answer in steps, and paste the exact error message instead of describing it.

4. Optional: if your organization approves an AI assistant, paste one vague and one engineered prompt into it and compare the answers. Run any code it returns through the same checks you used in Tasks 2 to 5 before you consider it.

Checkpoint: `my-prompts.txt` contains three rewritten prompts, each stating the OS version, at least two constraints, and an output format.

Task 7: Data governance, local models, and corporate policy

1. Create a realistic config file that an admin might want to paste into an AI chat while troubleshooting. The values are fake, but treat them as real.

```
cat > app.conf <<'EOF'
# myapp configuration
server_name = web01.corp.example.com
listen_ip = 10.20.30.40
db_host = db01.corp.example.com
db_user = appuser
db_password = Winter2026!
api_key = sk-live-51HxQ2eZvKYlo2C0aBcDeFgH
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
admin_email = jane.doe@example.com
log_level = debug
# old key kept for reference: AKIAJ7QZEXAMPLE2ABCD
EOF
```

2. Write a redaction script that masks secrets and identifying data before anything is shared.

```
cat > redact.sh <<'EOF'
#!/bin/bash
# redact.sh - mask secrets and identifying data before sharing a file
# Usage: redact.sh FILE > FILE.redacted
set -euo pipefail
file="${1:?Usage: redact.sh FILE}"

# "name = value" lines whose name contains pass, secret, token, or key
secret_line='^([[:space:]]*[[:alnum:]_]*(pass|secret|token|key)[[:alnum:]_]*'
secret_line+='[[:space:]]*[=:][[:space:]]*).+'

sed -E \
    -e "s/${secret_line}/\1<REDACTED>/I" \
    -e 's/AKIA[0-9A-Z]{16}/<AWS-KEY-ID>/g' \
    -e 's/\b([0-9]{1,3}\.){3}[0-9]{1,3}\b/<IP>/g' \
    -e 's/[[:alnum:]._%+-]+@[[:alnum:].-]+\.[[:alpha:]]{2,}/<EMAIL>/g' \
    -e 's/\b[[:alnum:]-]+\.corp\.example\.com\b/<HOST>/g' \
    "$file"
EOF
chmod +x redact.sh
shellcheck redact.sh && ./redact.sh app.conf | tee app.conf.redacted
```

```
# myapp configuration
server_name = <HOST>
listen_ip = <IP>
db_host = <HOST>
db_user = appuser
db_password = <REDACTED>
api_key = <REDACTED>
aws_access_key_id = <REDACTED>
aws_secret_access_key = <REDACTED>
admin_email = <EMAIL>
log_level = debug
# old key kept for reference: <AWS-KEY-ID>
```

What just happened: `sed -E` applied five substitution rules in order. Rule 1 keeps the setting name and replaces the value of any setting whose name contains `pass`, `secret`, `token`, or `key`; `\1` puts back the captured name and `=`, and the `I` flag makes the match case-insensitive. The other rules catch values wherever they appear: AWS access key IDs (which always start with `AKIA`), IPv4 addresses, email addresses, and internal host names. The comment line shows why several layers matter: rule 1 did not touch it, but the AWS rule did. The setting *names* stay, so the AI can still understand the structure.

3. Verify the redaction instead of trusting it. Search the output for every sensitive value you know is in the original.

```
grep -nE 'AKIA|sk-live|Winter2026|wJalr|@example\.com|10\.20\.|corp\.example' \
  app.conf.redacted || echo "No known secrets found"
```

```
No known secrets found
```

What just happened: `grep` exits with status 1 when nothing matches, so `||` printed the success message. Pattern-based redaction is never perfect: a password in a setting called `credential` or a host name in another domain would slip through. Always read the redacted file yourself before sharing it, and remember that if a real secret is ever pasted into a public AI service, you must rotate (change) it.

4. Decide where data may go. Work through this checklist for three scenarios: (a) the redacted `app.conf`, (b) the original `app.conf`, and (c) a customer database export. Record your decisions in `decisions.txt`.

```
AI data-governance decision checklist
1. What is the data classification? (public / internal / confidential / regulated)
2. Does it contain secrets, personal data, customer data, or regulated data
   (PCI, HIPAA, GDPR)? If yes: do not send to any external model.
3. Is the AI tool on the approved list in corporate policy?
4. Public consumer service: may the provider store prompts, use them for LLM
   training, or have staff read them in human review? Assume yes unless a
   contract says otherwise.
5. Business/enterprise plan: is there a signed agreement (no training on our data,
   retention limits, data location)?
6. Private or local model (runs in our network): is it approved and patched?
   Data stays in-house, but output still needs review.
7. Can the task be done with redacted or synthetic data instead?
8. Who reviews and is accountable for the output before it is used?
```

```
nano decisions.txt
```

What just happened: A reasonable set of answers: (a) the redacted file is internal data with secrets removed, so it may go to an *approved* tool, preferably a business plan that does not train on your data. (b) The original contains live credentials, so it must never go to any external model; at most a private or local model, and even then only if policy allows. (c) A customer export is regulated personal data: no AI tool without explicit approval from data owners and compliance, and in most companies simply no.

TIP

Explore/simulate: A **local model** runs an open-weight LLM on your own hardware, so prompts never leave the machine. One popular tool is Ollama: `curl -fsSL https://ollama.com/install.sh | sh` installs it, and `ollama run llama3.2:1b` downloads a small model of about 1.3 GB and opens a chat. Even small models want several GB of RAM and run slowly without a GPU, so this is optional and best done on a bigger machine than your lab VM. The trade-off is privacy and control versus the capability and convenience of large public models.

5. Finish with a **corporate policy** checklist. Save it as `ai-policy-checklist.txt` and compare it with your employer's or school's real AI policy if one exists.

```
Corporate AI use policy - admin checklist
[ ] Use only AI tools on the approved list, with company accounts (not personal ones).
[ ] Never paste secrets, credentials, customer data, or regulated data into prompts.
[ ] Redact host names, IPs, user names, and internal URLs unless the tool is approved
    for internal data.
[ ] Do not copy/paste AI output into production without review and QA.
[ ] Verify all AI code: lint, syntax-check, dry run, test in a lab or staging.
[ ] AI-generated changes go through the normal change process: Git, pull request,
    human code review, CI pipeline, change ticket.
[ ] Check licenses: AI output may resemble copyrighted or copyleft code.
[ ] Label AI-assisted work if the policy requires it.
[ ] The person who commits the change is accountable for it, not the AI.
[ ] Report any accidental data exposure to security immediately and rotate secrets.
```

Checkpoint: `ls ~/labs/4.5` shows `app.conf.redacted`, `decisions.txt`, `my-prompts.txt`, and `ai-policy-checklist.txt`, and the redacted file contains no real secret values.

Challenge

CHALLENGE

1. Ask an approved AI assistant (or imagine its answer) for a Bash one-liner that finds the 10 largest files under `/var`. Verify it on your VM: check the options in the man pages, compare its output with `sudo du -ah /var 2>/dev/null | sort -rh | head -n 10`, and note any differences.
2. Extend `redact.sh` with a rule for private SSH keys (everything between `-----BEGIN` and `-----END` lines; hint: a `sed` address range like `/BEGIN/,/END/c\<PRIVATE KEY>`) and a test file that proves it works.
3. Add the regex test from Task 3 to a small `test_regex.sh` script that exits non-zero if any valid address is rejected or any invalid address is accepted, so it could run in a CI pipeline.

Clean up

1. Nothing in this lab changed system settings. Delete the lab directory, including the sandbox and all AI artifacts.

```
cd ~
rm -rf ~/labs/4.5
```

2. The review tools (`shellcheck`, `ansible`, `ansible-lint`) are useful in other labs. To remove them anyway:

```
sudo apt remove -y ansible-lint
sudo apt autoremove -y
```

Exam corner

- AI use cases for admins: **generation of code**, **regular expressions**, **infrastructure as code**, **documenting code and creating documentation**, **recommendations for compliance**, **security review**, **code optimization**, and **code linting**.
- **Never copy/paste AI output without review and QA.** LLMs can hallucinate options, modules, and packages, and they often suggest deprecated or insecure patterns.
- **Verify output** with tools: `bash -n` and ShellCheck for scripts, `grep -E` with good and bad test cases for regex, `ansible-lint`, `--syntax-check`, and `--check --diff` for IaC, `python3 -W error` and linters for Python.
- A passing linter does not mean code is secure; hard-coded secrets, `curl | bash`, disabled TLS checks, and `0777` permissions need **human review**.
- Test AI code in a lab or sandbox first, then send it through Git, pull requests, peer review, and CI like any other change.
- **Data governance:** data sent to a public AI service may be stored, used for **LLM training**, or read during **human review** by the provider. Never share secrets, personal data, or customer data.
- **Public** models: most capable, data leaves your control. **Private** or **local models** (for example via Ollama): data stays in-house, less capable, you maintain them.
- Redact secrets, IPs, host names, and emails before sharing; verify the redaction; rotate any secret that leaked.
- **Adhere to corporate policy:** approved tools only, allowed data classes, labeling rules, and human accountability.
- **Prompt engineering:** give role, context (OS and versions), a clear task, constraints, output format, and examples; ask for assumptions and tests.

Review questions

1. An AI assistant gives you a Bash script that passes `bash -n`. What does that prove?
   A. The script is safe to run as root
   B. The script only has valid Bash syntax
   C. The script is idempotent
   D. The script has no security issues

2. Name two things that can happen to data pasted into a public AI chat service, according to typical provider terms.

3. A regex from an AI assistant uses `\d` and you test it with `grep -E`. It matches nothing. Why?

4. Which option best protects confidential configuration data when you want AI help?
   A. Paste it into any free public chatbot quickly, then delete the chat
   B. Redact secrets and identifiers, verify the redaction, and use only an approved tool (or a local model) as policy allows
   C. Base64-encode the file before pasting it
   D. Ask the AI not to remember the data

5. List four elements of a well-engineered prompt.

6. ansible-lint reports no problems for an AI-generated playbook. Is it ready for production? Explain.

PART 5 · DOMAIN 5.0

Part 5: Troubleshooting

22% of the exam · 5 labs

5.1	Summarize monitoring concepts and configurations in a Linux system.
5.2	Given a scenario, analyze and troubleshoot hardware, storage, and Linux OS issues.
5.3	Given a scenario, analyze and troubleshoot networking issues on a Linux system.
5.4	Given a scenario, analyze and troubleshoot security issues on a Linux system.
5.5	Given a scenario, analyze and troubleshoot performance issues.

Lab 5.1 Monitoring Concepts and Configurations

Objective: 5.1 Summarize monitoring concepts and configurations in a Linux system.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 100 minutes
VM: ubuntu-lab and rocky-lab
Snapshot first: Recommended. This lab installs several services and changes logging on both VMs.

You cannot fix what you cannot see. In this lab you build a small but real monitoring setup: a service level objective for a web server with a health check that measures it, threshold checks that raise alerts and send a webhook, an agent (node_exporter) compared with agentless polling, SNMP polling and traps between your two VMs, and central log aggregation with rsyslog. Every piece is a concept the exam names, and every piece is something you will meet at work.

What you'll practice

- Service monitoring: service level agreement (SLA), service level indicator (SLI), service level objective (SLO), error budgets
- Data acquisition: SNMP (`snmpwalk`, `snmpget`, community strings, SNMPv3 users, traps with `snmptrap` / `snmptrapd`, MIBs and OIDs)
- Data acquisition: agent versus agentless monitoring (`prometheus-node-exporter` versus SSH polling)
- Data acquisition: webhooks, health checks, and log aggregation (rsyslog forwarding over TCP 514)
- Configurations: thresholds, alerts, events, notifications, and logging (`logger`, `journalctl`, `wall`)

How it works

Monitoring means collecting data about systems over time and reacting when something looks wrong. It has three layers: *what you promise*, *what you measure*, and *how you react*.

What you promise: SLA, SLO, SLI. Picture a pizza shop.

- The **SLA (service level agreement)** is the contract with the customer: "Delivery in 45 minutes or the pizza is free." It is a business document with consequences (refunds, credits, penalties).
- The **SLO (service level objective)** is the internal target the shop aims for, usually stricter than the SLA so there is a safety margin: "We aim to deliver 99% of pizzas in 30 minutes."
- The **SLI (service level indicator)** is the actual measurement: "Last month, 98.7% of pizzas arrived within 30 minutes."

For a web server, a typical SLI is *availability* (the percentage of health checks that succeeded) or *latency* (the percentage of requests answered in under 300 ms). The SLO might be "99.9% available per month." The gap between 100% and the SLO is the **error budget**: 0.1% of a 30-day month is 43.2 minutes of allowed downtime. When the

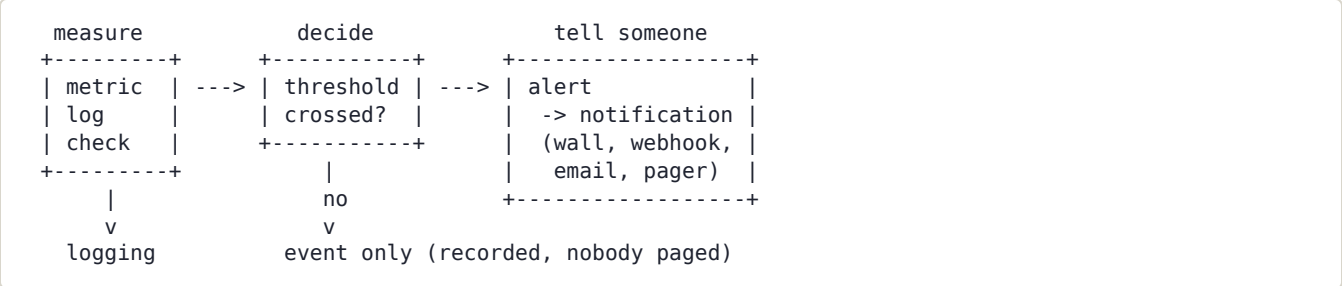
budget is used up, the team stops shipping risky changes.

What you measure: data acquisition. There are several ways to get data out of a system:

- A **health check** is a small, repeated test that asks "are you working?" For a web server, that means requesting a page and checking for HTTP status `200`. Load balancers use health checks to remove broken servers from rotation.
- An **agent** is software you install on the monitored host that collects data and exposes or sends it. Prometheus `node_exporter`, the Zabbix agent, and the Datadog agent are examples. Agents give rich, detailed data but you must install, update, and secure them on every host.
- **Agentless** monitoring uses something the host already has, usually SSH (run `uptime` remotely) or SNMP. Nothing new to install, but the data is limited to what those protocols expose, and the monitoring server needs credentials.
- **SNMP (Simple Network Management Protocol)** is the classic protocol for polling network devices and servers. A manager *polls* an SNMP agent on UDP port 161 and asks for values. Each value has an **OID (object identifier)**, a dotted number such as `1.3.6.1.2.1.1.5.0` (the host name). A **MIB (Management Information Base)** is a text file that gives names and descriptions to those numbers, so `1.3.6.1.2.1.1.5.0` can be shown as `SNMPv2-MIB::sysName.0`. A **trap** is the opposite direction: the agent sends an unrequested message to the manager on UDP port 162 when something happens (link down, disk full). SNMPv1 and v2c use a plain-text password called a **community string**; SNMPv3 adds real users with authentication and encryption.
- A **webhook** is an HTTP POST, usually with a JSON body, that one system sends to a URL on another system when an event happens. Chat tools, ticketing systems, and paging services all accept webhooks.
- **Log aggregation** means shipping logs from many hosts to one central place, so you can search them together and keep them safe if a host dies or is compromised. On Linux the simplest form is rsyslog forwarding to a central rsyslog server.

How you react: configurations. These words are related but not the same:

- **Logging** is recording what happened (to the journal or a log file).
- An **event** is any notable thing that happened ("nginx restarted", "check failed").
- A **threshold** is a limit that turns a measurement into a problem ("disk usage above 80%").
- An **alert** is the decision that a threshold was crossed and a human should care.
- A **notification** is how the alert reaches a human or system: email, `wall` message, chat webhook, pager.



A common mistake is alerting on everything. When people get dozens of meaningless alerts a day they start ignoring all of them, including the important one. This is called **alert fatigue**. Good alerts are actionable (someone can do something), tied to user impact (the SLO), and tuned so they rarely fire falsely.

Tasks

Task 1: Define an SLO and build a health check

You will run nginx on ubuntu-lab, define an SLO for it, and measure the SLI with a health check that a systemd timer runs every 15 seconds.

1. Create the lab directory and install nginx and curl.

```
mkdir -p ~/labs/5.1 && cd ~/labs/5.1
sudo apt update
sudo apt install -y nginx curl
curl -s -o /dev/null -w '%{http_code} %{time_total}\n' http://localhost/
```

```
200 0.001412
```

> **What just happened:** nginx installs and starts automatically on Ubuntu. The `curl` options: `-s` is silent (no progress bar), `-o /dev/null` throws away the page body, and `-w` (write-out) prints chosen variables after the transfer. `%{http_code}` is the HTTP status code and `%{time_total}` is the total time in seconds. This one line is a complete health check: the service is up (`200`) and fast (1.4 ms).

2. Write down your service levels. Create a short file so you can refer to it. A **heredoc** (`cat <<'EOF' > file` … `EOF`) sends all the lines between the markers into the file; the quotes around `EOF` stop the shell from changing `$` signs inside.

```
cat <<'EOF' > ~/labs/5.1/service-levels.txt
Service: lab web site (nginx on ubuntu-lab)
SLA (promise to customers): 99.5% monthly availability, else service credit
SLO (internal target):      99.9% of health checks return HTTP 200
SLI (what we measure):      successful checks / total checks, every 15 s
Latency SLI:                checks answered in under 0.300 s
EOF
cat ~/labs/5.1/service-levels.txt
```

> **What just happened:** You wrote the three levels in the order a team would. The SLO (99.9%) is stricter than the SLA (99.5%) so you get warned before you break the contract. The SLI is defined precisely enough that a script can calculate it.

3. Create the health check script. Open it with `sudo nano /usr/local/bin/web-healthcheck` or use the heredoc below. `tee` writes its input to the file; `sudo tee` lets you write to a root-owned path.

```
cat <<'EOF' | sudo tee /usr/local/bin/web-healthcheck > /dev/null
#!/bin/bash
# web-healthcheck: probe the web service once and record the result
URL="http://localhost/"
LOG=/var/log/web-health.log
result=$(curl -s -o /dev/null -w '%{http_code} %{time_total}' \
         --max-time 2 "$URL")
code=${result%% *}
secs=${result##* }
if [ "$code" = "200" ]; then state=UP; else state=DOWN; fi
echo "$(date -Is) $state $code $secs" >> "$LOG"
if [ "$state" = "DOWN" ]; then
    logger -t healthcheck -p daemon.err "web check FAILED code=$code url=$URL"
fi
EOF
sudo chmod 755 /usr/local/bin/web-healthcheck
sudo /usr/local/bin/web-healthcheck
cat /var/log/web-health.log
```

```
2026-10-10T10:02:11+00:00 UP 200 0.001377
```

What just happened: The script runs one check and appends one line: timestamp, state, status code, response time. `--max-time 2` makes curl give up after 2 seconds, so a hung server counts as a failure instead of hanging the check. `${result%% *}` keeps the text before the first space (the code) and `${result##* }` keeps the text after the last space (the time). If nginx is down, curl prints code `000`. On failure, `logger` writes an **event** into the system log with tag `healthcheck` and priority `daemon.err` (facility `daemon`, severity `err`).

4. Run the check every 15 seconds with a systemd service and timer. The service says *what* to run; the timer says *when*.

```
cat <<'EOF' | sudo tee /etc/systemd/system/web-healthcheck.service
[Unit]
Description=Health check for the lab web site

[Service]
Type=oneshot
ExecStart=/usr/local/bin/web-healthcheck
EOF
cat <<'EOF' | sudo tee /etc/systemd/system/web-healthcheck.timer
[Unit]
Description=Run the web health check every 15 seconds

[Timer]
OnBootSec=30s
OnUnitActiveSec=15s
AccuracySec=1s

[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now web-healthcheck.timer
systemctl list-timers web-healthcheck.timer
```

```
NEXT                         LEFT LAST                         PASSED UNIT
Sat 2026-10-10 10:03:01 UTC  9s  Sat 2026-10-10 10:02:46 UTC 5s ago web-healthcheck.timer
```

What just happened: `Type=oneshot` means the service runs once and exits. In the timer, `OnUnitActiveSec=15s` starts the service 15 seconds after it last started, `OnBootSec=30s` starts the first run 30 seconds after boot, and `AccuracySec=1s` stops systemd from batching the run up to a minute late (the default accuracy). `enable --now` starts the timer immediately and at every boot.

5. Create some downtime so the SLI has something to measure. Wait about a minute, stop nginx for about one minute, then start it again.

```
sleep 60; sudo systemctl stop nginx; sleep 60; sudo systemctl start nginx
tail -n 8 /var/log/web-health.log
journalctl -t healthcheck --since "10 minutes ago"
```

```
2026-10-10T10:04:16+00:00 UP 200 0.001290
2026-10-10T10:04:31+00:00 DOWN 000 0.000214
2026-10-10T10:04:46+00:00 DOWN 000 0.000198
2026-10-10T10:05:01+00:00 DOWN 000 0.000202
2026-10-10T10:05:16+00:00 DOWN 000 0.000187
2026-10-10T10:05:31+00:00 UP 200 0.001544
Oct 10 10:04:31 ubuntu-lab healthcheck[2210]: web check FAILED code=000
url=http://localhost/
```

6. Calculate the availability SLI and the latency SLI with `awk`.

```
awk '{ total++ } $2 == "UP" { up++ }
     END { printf "checks=%d up=%d availability=%.2f%%\n", total, up, 100*up/total }' \
     /var/log/web-health.log
awk '{ total++ } $2 == "UP" && $4 < 0.300 { fast++ }
     END { printf "latency SLI=%.2f%% under 300 ms\n", 100*fast/total }' \
     /var/log/web-health.log
```

```
checks=14 up=10 availability=71.43%
latency SLI=71.43% under 300 ms
```

What just happened: `awk` reads the log line by line. Field `$2` is the state and `$4` is the response time. It counts all lines, counts the good ones, and prints the percentage at the `END`. Your numbers will differ. Over a short window, one minute of downtime destroys the SLO; over a month, that same minute would use only about 2% of a 99.9% error budget (1 of 43.2 minutes). This is why SLOs are always stated over a time window.

Checkpoint: `systemctl list-timers web-healthcheck.timer` shows the timer, `/var/log/web-health.log` grows by one line every 15 seconds, it contains some `DOWN` lines, and `journalctl -t healthcheck` shows the failure events.

Task 2: Thresholds, events, alerts, and notifications

Now build a threshold check for disk usage and load average. It records every run as an event, and only raises an alert (with a notification) when a threshold is crossed.

1. Look at the two raw numbers you will check.

```
df --output=pcent / | tail -n 1
cat /proc/loadavg
nproc
```

```
 23%
0.08 0.05 0.01 1/187 2380
2
```

What just happened: `df --output=pcent /` prints only the "use percent" column for the root filesystem; `tail -n 1` drops the header. `/proc/loadavg` holds the 1, 5, and 15-minute load averages (the average number of processes running or waiting). A load equal to the number of CPUs (`nproc`, here 2) means the CPUs are fully busy, so `nproc` is a sensible default load threshold.

2. Create the threshold script.

```
cat <<'EOF' | sudo tee /usr/local/bin/threshold-check > /dev/null
#!/bin/bash
# threshold-check: compare disk and load against limits and alert
DISK_LIMIT=${DISK_LIMIT:-80}          # percent
LOAD_LIMIT=${LOAD_LIMIT:-$(nproc)}    # 1-minute load average
WEBHOOK=${WEBHOOK:-}                  # optional URL for notifications

disk=$(df --output=pcent / | tail -n 1 | tr -dc '0-9')
load=$(cut -d ' ' -f 1 /proc/loadavg)

alert() {
    logger -t threshold -p user.warning "ALERT $1"
    wall "ALERT on $(hostname): $1"
    if [ -n "$WEBHOOK" ]; then
        curl -s -X POST -H 'Content-Type: application/json' \
            -d "{\"host\":\"$(hostname)\",\"severity\":\"warning\",\"message\":\"$1\"}" \
            "$WEBHOOK" > /dev/null
    fi
}

if [ "$disk" -ge "$DISK_LIMIT" ]; then
    alert "disk / at ${disk}% (limit ${DISK_LIMIT}%)"
fi
if awk -v l="$load" -v m="$LOAD_LIMIT" 'BEGIN { exit !(l > m) }'; then
    alert "load ${load} above ${LOAD_LIMIT}"
fi
logger -t threshold -p user.info "event: check done disk=${disk}% load=${load}"
EOF
sudo chmod 755 /usr/local/bin/threshold-check
```

What just happened: `${DISK_LIMIT:-80}` means "use the variable if it is set, otherwise 80", so you can change limits without editing the script. `tr -dc '0-9'` deletes every character that is not a digit, turning `23%` into `23`. Bash cannot compare decimals like `0.08`, so `awk` does the load comparison and sets the exit status. Every run logs an informational **event**; only a crossed threshold calls `alert`, which logs a warning and sends **notifications** with `wall` (a message to every logged-in terminal) and, if configured, a webhook.

3. Run it with normal limits, then with a deliberately tiny disk limit to force an alert.

```
threshold-check
DISK_LIMIT=1 threshold-check
journalctl -t threshold --since "5 minutes ago"
```

```
Broadcast message from student@ubuntu-lab (pts/0) (Sat Oct 10 10:12:40 2026):

ALERT on ubuntu-lab: disk / at 23% (limit 1%)

Oct 10 10:12:31 ubuntu-lab threshold[2401]: event: check done disk=23% load=0.02
Oct 10 10:12:40 ubuntu-lab threshold[2412]: ALERT disk / at 23% (limit 1%)
Oct 10 10:12:40 ubuntu-lab threshold[2415]: event: check done disk=23% load=0.02
```

4. Filter the journal by severity. Severity (priority) levels go from `0` (emerg) to `7` (debug); `-p warning` shows warning and anything more severe.

```
journalctl -t threshold -p warning --since "5 minutes ago"
```

> **What just happened:** Only the alert line appears, not the routine events. Monitoring systems work the same way: everything is logged, but only high-severity items reach people. If you set `DISK_LIMIT=1` permanently, you would get a `wall` message every run for a condition nobody needs to fix. That is how alert fatigue starts. A good threshold is high enough to be meaningful and low enough to leave time to act (for example, warn at 80%, page at 95%).

> **Checkpoint:** `journalctl -t threshold -p warning` shows exactly one `ALERT` line from the forced test, and you saw the `Broadcast message` from `wall` in your terminal.

Task 3: Send alerts with a webhook

Real alerts usually go to a chat or paging system through a webhook. You will run a tiny webhook receiver on ubuntu-lab and point the threshold script at it. (Note: `python3 -m http.server` only answers GET requests, so it cannot receive a webhook. You need a listener that handles POST.)

1. Create a 15-line Python listener.

```
cat <<'EOF' > ~/labs/5.1/webhook-listener.py
#!/usr/bin/env python3
# Minimal webhook receiver: prints the JSON body of every POST
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", 0))
        body = json.loads(self.rfile.read(length))
        print("WEBHOOK RECEIVED:", body, flush=True)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok\n")

HTTPServer(("127.0.0.1", 8099), Hook).serve_forever()
EOF
python3 ~/labs/5.1/webhook-listener.py
```

> **What just happened:** `BaseHTTPRequestHandler` calls `do_POST` for each POST request. The handler reads exactly `Content-Length` bytes, parses the JSON, prints it, and answers `200 ok`. It listens only on `127.0.0.1` port 8099, so nothing outside the VM can reach it. Leave it running in this terminal.

2. Open a **second** SSH session to ubuntu-lab and send a test webhook by hand.

```
curl -s -X POST -H 'Content-Type: application/json' \
    -d '{"host":"ubuntu-lab","severity":"info","message":"test"}' \
    http://127.0.0.1:8099/alert
```

In the first terminal you see:

```
WEBHOOK RECEIVED: {'host': 'ubuntu-lab', 'severity': 'info', 'message': 'test'}
127.0.0.1 - - [10/Oct/2026 10:20:05] "POST /alert HTTP/1.1" 200 -
```

What just happened: `-X POST` sets the HTTP method, `-H` adds a header telling the receiver the body is JSON, and `-d` supplies the body. This is exactly what Slack, Microsoft Teams, PagerDuty, or a ticketing system would receive.

3. Trigger a real threshold alert that notifies through the webhook.

```
DISK_LIMIT=1 WEBHOOK=http://127.0.0.1:8099/alert threshold-check
```

```
WEBHOOK RECEIVED: {'host': 'ubuntu-lab', 'severity': 'warning', 'message': 'disk / at 23%
(limit 1%)'}
```

4. Stop the listener in the first terminal with `Ctrl+C`.

Checkpoint: The listener printed a `WEBHOOK RECEIVED` line with `severity: warning` that came from `threshold-check`.

Task 4: Agent versus agentless monitoring

1. Install the Prometheus node exporter, an **agent** that exposes hundreds of metrics over HTTP.

```
sudo apt install -y prometheus-node-exporter
systemctl is-active prometheus-node-exporter
sudo ss -tlnp | grep 9100
curl -s http://localhost:9100/metrics | grep -E '^node_load1
|^node_memory_MemAvailable_bytes'
```

```
active
LISTEN 0  4096   *:9100   *:*   users:(("prometheus-node",pid=3437,fd=3))
node_load1 0.04
node_memory_MemAvailable_bytes 3.412783104e+09
```

What just happened: The package installs and starts the `prometheus-node-exporter` service on TCP port 9100. `ss -tlnp` lists listening TCP sockets (`-t` TCP, `-l` listening, `-n` numbers, `-p` process; the process column needs `sudo`). The `/metrics` page is plain text, one metric per line; a Prometheus server would *scrape* (fetch) it every 15 to 60 seconds and store the history. `grep -E` picks out the 1-minute load and the available memory in bytes.

2. Count how much data the agent gives you.

```
curl -s http://localhost:9100/metrics | grep -vc '^#'
```

```
1247
```

3. Now do **agentless** monitoring from rocky-lab: nothing extra is installed on ubuntu-lab, you just use SSH, which is already there. Enter the student password when asked.

```
# on rocky-lab
ssh student@ubuntu-lab 'uptime; df -h / | tail -n 1; free -m | grep Mem'
```

```
 10:31:02 up  2:11,  2 users,  load average: 0.03, 0.04, 0.01
/dev/mapper/ubuntu--vg-ubuntu--lv   24G  5.4G   17G  25% /
Mem:            3915     560    2780       1     810    3354
```

What just happened: Tools like Ansible and many monitoring systems work this way: they log in over SSH, run commands, and parse the output. Compare the two styles:

	Agent (node_exporter)	Agentless (SSH, SNMP)
Install on host	Yes, on every host	No, uses existing services
Data detail	Very rich (1,000+ metrics)	Whatever commands or MIBs expose
Credentials	Usually none to read metrics	Monitoring server needs SSH keys or SNMP credentials
Overhead	Small always-running process	A login or poll each interval
Good for	Servers you control	Network gear, appliances, locked-down hosts

Checkpoint: `curl -s localhost:9100/metrics | grep node_load1` on ubuntu-lab and the SSH one-liner on rocky-lab both show the load average of ubuntu-lab.

Task 5: Poll a host with SNMP and learn MIBs

You will make ubuntu-lab an SNMP agent and poll it from rocky-lab with SNMPv2c and SNMPv3.

1. Install the SNMP agent (`snmpd`), the client tools (`snmp`), and the MIB downloader on ubuntu-lab. `snmp-mibs-downloader` comes from the `multiverse` component, which Ubuntu 24.04 enables by default.

```
sudo apt install -y snmpd snmp snmp-mibs-downloader
sudo cp /etc/snmp/snmpd.conf /etc/snmp/snmpd.conf.orig
```

2. Replace the agent configuration. The default only listens on localhost and only shows the `system` group. Substitute your lab subnet if it is not `192.168.100.0/24` .

   The file is `/etc/snmp/snmpd.conf` :

```
cat <<'EOF' | sudo tee /etc/snmp/snmpd.conf
# Listen on all IPv4 addresses, UDP port 161
agentaddress udp:161
sysLocation  Lab rack 1
sysContact   student <student@ubuntu-lab>
# A view is the part of the OID tree a client may read
view labview included .1.3.6.1.2.1.1
view labview included .1.3.6.1.2.1.25
view labview included .1.3.6.1.4.1.2021
# SNMPv2c: read-only community "labcomm" from the lab subnet only
rocommunity labcomm 192.168.100.0/24 -V labview
rocommunity labcomm 127.0.0.1 -V labview
# SNMPv3: read-only user that must authenticate and encrypt
rouser labv3 priv -V labview
EOF
```

What just happened: `agentaddress udp:161` makes snmpd listen on every interface. A `view` limits which branches of the OID tree are visible: `.1.3.6.1.2.1.1` is the `system` group (name, uptime, location), `.1.3.6.1.2.1.25` is HOST-RESOURCES (storage, processes), and `.1.3.6.1.4.1.2021` is the UCD-SNMP group (load averages, memory). `rocommunity` creates a read-only community string that only works from the listed source addresses. `rouser labv3 priv` allows the SNMPv3 user `labv3` read-only access, but only with authentication **and** privacy (encryption).

3. Create the SNMPv3 user. snmpd reads `createUser` lines from its persistent file at start-up, replaces them with hashed keys, and removes the clear-text passwords.

```
sudo systemctl stop snmpd
echo 'createUser labv3 SHA "AuthPass123" AES "PrivPass123"' | \
    sudo tee -a /var/lib/snmp/snmpd.conf
sudo systemctl start snmpd
sudo grep -c createUser /var/lib/snmp/snmpd.conf
ss -ulnp | grep :161
```

```
0
UNCONN 0  0   0.0.0.0:161   0.0.0.0:*
```

What just happened: `SHA` is the authentication hash and `AES` is the encryption cipher. The count of `createUser` lines is now `0` because snmpd converted it into a `usmUser` line with localized keys. `ss -ulnp` shows snmpd listening on UDP 161 on all addresses. If `ufw` is active on your VM, also run `sudo ufw allow from 192.168.100.0/24 to any port 161 proto udp`.

TIP

The `snmpd` package also ships a helper that does the same job: with snmpd stopped, `sudo net-snmp-create-v3-user -ro -a SHA -A 'AuthPass123' -x AES -X 'PrivPass123' labv3` appends the `createUser` line to `/var/lib/snmp/snmpd.conf` and a `rouser labv3` line to `/etc/snmp/snmpd.conf`. You already wrote a `rouser` line with a view, so the manual method is used here.

4. Turn on MIB names for the Ubuntu client. Debian and Ubuntu disable MIB loading by default for licensing reasons, so you only see numbers until you comment out the `mibs :` line.

```
snmpget -v2c -c labcomm localhost 1.3.6.1.2.1.1.5.0
sudo sed -i 's/^mibs :/#mibs :/' /etc/snmp/snmp.conf
snmpget -v2c -c labcomm localhost 1.3.6.1.2.1.1.5.0
ls /usr/share/snmp/mibs/ietf | grep -E '^(SNMPv2-MIB|HOST-RESOURCES-MIB|IF-MIB)$'
```

```
iso.3.6.1.2.1.1.5.0 = STRING: "ubuntu-lab"
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
HOST-RESOURCES-MIB
IF-MIB
SNMPv2-MIB
```

What just happened: The first answer is just a number (`iso.3.6.1.2.1.1.5.0`). After enabling the MIBs downloaded by `snmp-mibs-downloader` , the same OID is translated to `SNMPv2-MIB::sysName.0` . The standard IETF MIBs live in `/usr/share/snmp/mibs/ietf` and `iana` (links into `/var/lib/mibs` , filled by the downloader); the Net-SNMP ones (`UCD-SNMP-MIB.txt` and others) sit directly in `/usr/share/snmp/mibs` . The MIB changed only how the *client* displays the answer; the agent sent the same data. `snmptranslate -Td SNMPv2-MIB::sysName.0` would print the MIB description of that object.

5. Poll ubuntu-lab from rocky-lab. Install the client tools there; Rocky ships the standard MIBs with them and loads them by default.

```
# on rocky-lab
sudo dnf install -y net-snmp-utils
snmpwalk -v2c -c labcomm ubuntu-lab system
snmpget -v2c -c labcomm ubuntu-lab SNMPv2-MIB::sysUpTime.0 UCD-SNMP-MIB::laLoad.1
```

```
SNMPv2-MIB::sysDescr.0 = STRING: Linux ubuntu-lab 6.8.0-45-generic #45-Ubuntu SMP ...
SNMPv2-MIB::sysObjectID.0 = OID: NET-SNMP-MIB::netSnmpAgentOIDs.10
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (791265) 2:11:52.65
SNMPv2-MIB::sysContact.0 = STRING: student <student@ubuntu-lab>
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
SNMPv2-MIB::sysLocation.0 = STRING: Lab rack 1
...
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (791402) 2:11:54.02
UCD-SNMP-MIB::laLoad.1 = STRING: 0.02
```

What just happened: `-v2c` selects SNMP version 2c and `-c labcomm` gives the community string. `snmpwalk` walks every OID under a starting point (`system` is a MIB name for `.1.3.6.1.2.1.1`). `snmpget` fetches exact OIDs; the `.0` suffix means "the single instance" of a scalar value and `.1` in `laLoad.1` is the first row of the load table (the 1-minute load). Try `snmpwalk -v2c -c wrong ubuntu-lab system` : it times out, because a wrong community gets no reply at all.

6. Poll with SNMPv3 and compare what a sniffer would see.

```
# on rocky-lab
snmpget -v3 -l authPriv -u labv3 -a SHA -A 'AuthPass123' \
    -x AES -X 'PrivPass123' ubuntu-lab SNMPv2-MIB::sysName.0
```

```
SNMPv2-MIB::sysName.0 = STRING: ubuntu-lab
```

What just happened: `-l authPriv` is the security level (authentication and privacy), `-u` the user, `-a` / `-A` the auth protocol and passphrase, `-x` / `-X` the privacy protocol and passphrase. With v2c, the community string and every value cross the network in clear text; with v3 authPriv, nobody can read or forge them. Use SNMPv3 in production.

Checkpoint: From rocky-lab, `snmpwalk -v2c -c labcomm ubuntu-lab system` shows `sysLocation.0 = STRING: Lab rack 1`, and the SNMPv3 `snmpget` returns `sysName.0`.

Task 6: Send and receive SNMP traps

Polling asks "how are you?" every few minutes. A trap lets the device shout "something happened!" immediately.

1. Install and configure the trap receiver on ubuntu-lab. The file is `/etc/snmp/snmptrapd.conf`. Ubuntu starts `snmptrapd` with `-LOw` (log only messages of priority *warning* and above), but received traps are logged at priority *info*, so out of the box traps arrive and are silently not logged. A small systemd **drop-in** (an extra file that overrides part of a unit) changes the option to `-LOi`. The empty `ExecStart=` line clears the packaged command before the new one is set.

```
sudo apt install -y snmptrapd
echo 'authCommunity log,execute,net labtrap' | sudo tee /etc/snmp/snmptrapd.conf
sudo mkdir -p /etc/systemd/system/snmptrapd.service.d
cat <<'EOF' | sudo tee /etc/systemd/system/snmptrapd.service.d/lab-logging.conf
[Service]
ExecStart=
ExecStart=/usr/sbin/snmptrapd -LOi -f udp:162 udp6:162
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now snmptrapd.socket
sudo systemctl restart snmptrapd.service
sudo ss -ulnp | grep :162
```

```
UNCONN 0  0   0.0.0.0:162   0.0.0.0:*   users:(("snmptrapd",pid=3105,fd=3),
("systemd",pid=1,fd=60))
```

What just happened: On Ubuntu, `snmptrapd.socket` owns UDP port 162 (**socket activation**: systemd opens the port and hands it to the daemon), which is why `ss` shows both `systemd` and `snmptrapd`. `authCommunity log,execute,net labtrap` says: accept traps that use community `labtrap`, and allow them to be logged, passed to scripts (`execute`), and forwarded (`net`). Traps with any other community are dropped without a log line. `-LOi` sends log messages of priority `info` and above to standard output, which systemd stores in the journal.

2. Watch the receiver's log in one terminal on ubuntu-lab.

```
sudo journalctl -u snmptrapd -f
```

3. Send a trap from rocky-lab. This uses an example notification defined in `NET-SNMP-EXAMPLES-MIB` with one variable attached.

```
# on rocky-lab
snmptrap -v2c -c labtrap ubuntu-lab '' \
    NET-SNMP-EXAMPLES-MIB::netSnmpExampleHeartbeatNotification \
    netSnmpExampleHeartbeatRate i 123456
```

On ubuntu-lab you see something like:

```
snmptrapd[3105]: 2026-10-10 10:48:12 rocky-lab [UDP: [192.168.100.20]:46125->
[192.168.100.10]:162]:
snmptrapd[3105]: DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (1234567) 3:25:45.67
  SNMPv2-MIB::snmpTrapOID.0 = OID: NET-SNMP-EXAMPLES-
MIB::netSnmpExampleHeartbeatNotification
  NET-SNMP-EXAMPLES-MIB::netSnmpExampleHeartbeatRate = INTEGER: 123456
```

(snmptrapd prints all the variables on one long line separated by tabs; they are wrapped here to fit the page.)

What just happened: The arguments are: version, community, destination, then the uptime value (`''` means "use the current uptime"), the trap OID (what kind of event this is), and then variable bindings written as *OID type value* (`i` means integer, `s` would mean string). Real devices send traps like `IF-MIB::linkDown` or vendor traps for fan failures. Stop the journal view with `Ctrl+C`.

TIP

Traps use UDP and are fire-and-forget: if the packet is lost, the event is lost. SNMPv2c and v3 also support **informs**, which are traps the receiver must acknowledge (`snmpinform`). This is one reason monitoring systems combine traps with regular polling.

Checkpoint: `sudo journalctl -u snmptrapd --since "10 minutes ago"` on ubuntu-lab shows the trap from rocky-lab with `netSnmpExampleHeartbeatRate = INTEGER: 123456`.

Task 7: Aggregate logs from rocky-lab on ubuntu-lab

You will make ubuntu-lab a central log server and forward all rocky-lab logs to it over TCP port 514.

1. On ubuntu-lab, create a directory for remote logs owned by the `syslog` user (Ubuntu's rsyslog drops root privileges to that user), then add a receiver configuration in `/etc/rsyslog.d/10-remote-in.conf`.

```
sudo mkdir -p /var/log/remote
sudo chown syslog:adm /var/log/remote
cat <<'EOF' | sudo tee /etc/rsyslog.d/10-remote-in.conf
# Receive syslog over TCP 514 and write one file per sending host
template(name="PerHost" type="string" string="/var/log/remote/%HOSTNAME%.log")
ruleset(name="remote") {
    action(type="omfile" dynaFile="PerHost")
}
module(load="imtcp")
input(type="imtcp" port="514" ruleset="remote")
EOF
sudo rsyslogd -N1
sudo systemctl restart rsyslog
ss -tlnp | grep :514
```

```
rsyslogd: version 8.2312.0, config validation run (level 1), master config /etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
LISTEN 0  25   0.0.0.0:514   0.0.0.0:*
```

What just happened: `imtcp` is the TCP input module. The `input` binds it to port 514 and sends everything it receives into the ruleset `remote`, so remote messages are kept apart from local ones (they do not end up in `/var/log/syslog`). The `template` builds a file name from the sender's host name, and `dynaFile` uses that template, so each host gets its own file. `rsyslogd -N1` checks the syntax without starting anything; always run it before a restart. If `ufw` is active, run `sudo ufw allow from 192.168.100.0/24 to any port 514 proto tcp`.

2. On rocky-lab, forward everything to ubuntu-lab. The file is `/etc/rsyslog.d/90-forward.conf`.

```
# on rocky-lab
sudo dnf install -y rsyslog
cat <<'EOF' | sudo tee /etc/rsyslog.d/90-forward.conf
# Forward all messages to the central log server over TCP
*.* action(type="omfwd" target="ubuntu-lab" port="514" protocol="tcp"
           queue.type="linkedList" queue.size="10000"
           action.resumeRetryCount="-1")
EOF
sudo rsyslogd -N1
sudo systemctl restart rsyslog
logger -t labtest "hello from rocky-lab to the central log server"
```

What just happened: `*.*` selects every facility and every severity. `omfwd` is the forwarding module; `protocol="tcp"` is more reliable than UDP because lost packets are re-sent. The `queue` settings give the action its own memory queue, and `action.resumeRetryCount="-1"` means retry forever, so messages are held, not lost, while the server is unreachable. (In legacy syntax the same rule is `*.* @@ubuntu-lab:514`, where `@@` means TCP and a single `@` means UDP.)

3. On ubuntu-lab, check that the logs arrived.

```
ls -l /var/log/remote/
sudo tail -n 3 /var/log/remote/rocky-lab.log
```

```
-rw-r----- 1 syslog adm 1873 Oct 10 11:02 rocky-lab.log
2026-10-10T11:02:15.112345+00:00 rocky-lab systemd[1]: Started System Logging Service.
2026-10-10T11:02:16.004211+00:00 rocky-lab labtest[4012]: hello from rocky-lab to the
central log server
```

TIP

If nothing arrives: on rocky-lab run `journalctl -u rsyslog -n 20` (connection errors) and `sudo ausearch -m avc -ts recent` (SELinux denials). On ubuntu-lab check `journalctl -u rsyslog -n 20`, that port 514 is listening, and that `/var/log/remote` is owned by `syslog`. Test the network path with `nc -zv ubuntu-lab 514` from rocky-lab (`sudo dnf install -y nmap-ncat`). Ubuntu 24.04 also confines rsyslogd with an AppArmor profile; if `sudo journalctl -k | grep -i 'apparmor="DENIED".*rsyslogd'` shows denials for network sockets, add the rules in a local snippet that the profile includes, then restart (the rsyslog unit reloads its profile at start-up): `printf 'network inet stream,\nnetwork inet6 stream,\n' | sudo tee /etc/apparmor.d/rsyslog.d/lab-imtcp` and `sudo systemctl restart rsyslog`. Delete that file in Clean up if you created it.

4. Prove why central logs matter: search both hosts' events from one place.

```
sudo grep -h -i "fail\|error" /var/log/remote/*.log | tail -n 5
```

What just happened: With log aggregation, one search covers every host that forwards to the server. It also protects evidence: an attacker who gains root on rocky-lab can delete its local logs, but the copies already sent to ubuntu-lab remain. Larger shops use the same idea with tools such as the Elastic stack, Graylog, Loki, or Splunk, which add indexing, dashboards, and alerting on top of the collected logs.

Checkpoint: `/var/log/remote/rocky-lab.log` exists on ubuntu-lab and contains your `hello from rocky-lab` line.

Challenge

CHALLENGE

1. Create `threshold-check.service` and `threshold-check.timer` units that run the threshold script every 5 minutes, and make the webhook URL part of the service with `Environment=WEBHOOK=...`.
2. Change the health check so it also checks that the page body contains the word `nginx` (a content check), not just the status code.
3. Configure rocky-lab as a second SNMP agent (`sudo dnf install -y net-snmp`), open UDP 161 in firewalld, and poll it from ubuntu-lab with SNMPv3.

Clean up

1. On ubuntu-lab, stop and remove the health check timer, scripts, and logs.

```
sudo systemctl disable --now web-healthcheck.timer
sudo rm -f /etc/systemd/system/web-healthcheck.service \
    /etc/systemd/system/web-healthcheck.timer
sudo systemctl daemon-reload
sudo rm -f /usr/local/bin/web-healthcheck /usr/local/bin/threshold-check \
    /var/log/web-health.log
```

2. On rocky-lab, stop forwarding logs.

```
# on rocky-lab
sudo rm -f /etc/rsyslog.d/90-forward.conf
sudo systemctl restart rsyslog
```

3. On ubuntu-lab, remove the log receiver and the remote logs.

```
sudo rm -f /etc/rsyslog.d/10-remote-in.conf /etc/apparmor.d/rsyslog.d/lab-imtcp
sudo systemctl restart rsyslog
sudo rm -rf /var/log/remote
```

4. Remove the lab services and restore the SNMP client configuration on ubuntu-lab.

```
sudo apt purge -y snmpd snmptrapd prometheus-node-exporter nginx nginx-common
sudo apt autoremove -y
sudo rm -rf /etc/systemd/system/snmptrapd.service.d
sudo systemctl daemon-reload
sudo sed -i 's/^#mibs :/mibs :/' /etc/snmp/snmp.conf
sudo rm -rf /var/lib/snmp/snmpd.conf /etc/snmp/snmpd.conf.orig
rm -rf ~/labs/5.1
```

5. If you opened any `ufw` ports, remove them with `sudo ufw status numbered` and `sudo ufw delete <number>`.

Exam corner

- **SLA** = contract with customers (with penalties); **SLO** = internal target (stricter than the SLA); **SLI** = the actual measurement (for example, % of successful requests).
- Error budget = 100% minus the SLO. 99.9% per 30 days allows about 43 minutes of downtime.
- SNMP agents listen on **UDP 161**; traps go to the manager on **UDP 162**. `snmpget` reads one OID, `snmpwalk` reads a subtree, `snmptrap` sends a trap, `snmptrapd` receives traps.
- A **MIB** maps numeric **OIDs** to names and descriptions. On Ubuntu, `snmp-mibs-downloader` and commenting out `mibs :` in `/etc/snmp/snmp.conf` enable names.
- SNMPv1/v2c use clear-text **community strings**; SNMPv3 adds users with authentication (`-a SHA`) and privacy/encryption (`-x AES`), security level `authPriv`.

- **Agent** monitoring installs software on each host (node_exporter on port 9100, Zabbix agent); **agentless** uses existing protocols (SSH, SNMP, WMI).
- A **webhook** is an HTTP POST (usually JSON) sent to a URL when an event occurs.
- A **health check** is a repeated probe (for example `curl` expecting HTTP 200) used by monitors and load balancers.
- **Log aggregation**: rsyslog `imtcp` / `imudp` on the server, `omfwd` (or `@@host:514` for TCP, `@host:514` for UDP) on clients.
- Threshold crossed leads to an alert, which is delivered as a notification. Too many non-actionable alerts cause **alert fatigue**.
- `logger -t tag -p facility.severity "msg"` writes events to the log; `journalctl -t tag -p warning` filters them.

Review questions

1. A company promises customers 99.5% uptime with service credits if it is missed, and internally aims for 99.9%. What is the 99.9% figure?
   A. SLA B. SLI C. SLO D. MIB

2. Which port and protocol does an SNMP manager use to receive traps?

3. You run `snmpget` on Ubuntu and see `iso.3.6.1.2.1.1.5.0` instead of `SNMPv2-MIB::sysName.0`. What is missing?
   A. The SNMP agent B. MIB files loaded by the client C. An SNMPv3 user D. A trap receiver

4. Which rsyslog client line forwards all messages to `logs.example.com` over TCP?
   A. `*.* @logs.example.com:514` B. `*.* @@logs.example.com:514` C. `*.* >logs.example.com` D. `*.* tcp://logs.example.com`

5. An administrator is paged 40 times a day for CPU spikes that go away on their own, and starts ignoring pages. What is this called, and what is one fix?

6. What is the main trade-off of agent-based monitoring compared with agentless monitoring?

Lab 5.2 Troubleshoot Hardware, Storage, and OS Issues

Objective: 5.2 Given a scenario, analyze and troubleshoot hardware, storage, and Linux OS issues.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 4 hours (do it over two or three sittings; each scenario stands alone)
VM: ubuntu-lab (rocky-lab only for one SSH test)
Snapshot first: Yes. Take a snapshot now. Several scenarios deliberately break booting, and you need console access to the VM (the hypervisor's window, not SSH) for those.

This is a break-and-fix lab. In each scenario you deliberately cause a real problem, look at it the way you would on a production server, find the root cause with the right tools, fix it, and prove the fix worked. Exam questions for this objective describe a symptom ("users report No space left on device but df shows 40% free") and ask for the cause or the next command, so the goal is to connect symptoms to causes quickly.

What you'll practice

- A repeatable troubleshooting method: identify symptoms, gather data, form a hypothesis, test, fix, verify, document
- OS filesystem full (including deleted-but-open files) and inode exhaustion
- Partition not writable (read-only mount, immutable attribute, permissions) and quota issues
- Filesystem will not mount and data corruption issues (`e2fsck` , backup superblocks, checksums)
- GRUB misconfiguration, filesystem will not mount at boot (`/etc/fstab` , emergency mode)
- Kernel panic and kernel corruption issues (`sysrq` , `kdump` , `dpkg -V` , reinstalling kernel packages)
- Missing or disabled drivers (`lsmod` , `modprobe` , `modinfo` , `/etc/modprobe.d`)
- Package dependency issues (`dpkg` , `apt --fix-broken install` , `rpm` / `dnf` equivalents)
- Segmentation fault (`dmesg` , `coredumpctl` , `gdb`)
- Killed processes (OOM killer) and memory leaks (`ps` , `pmap` , `smem`)
- PATH misconfiguration issues
- Systemd unit failures (`systemctl status` , `journalctl -xeu` , `daemon-reload`)
- Unresponsive process (process states, `strace` , signals)
- Device failure (`dmsetup` error target, `badblocks` , `mdadm` degraded array, `smartctl`)
- Server inaccessible and server not turning on (console, BMC/IPMI, boot logs)

How it works

The troubleshooting method. Experienced admins do not guess randomly. They follow a loop. CompTIA's own methodology is very close to this:

```
1. Identify the symptoms   -> What exactly fails? Exact error text? Since when?
        |                     What changed recently? Who/what is affected?
2. Gather data             -> logs (journalctl, dmesg), status (systemctl),
        |                     resources (df, free, ps), config files
3. Form a hypothesis       -> "I think the disk is out of inodes"
        |
4. Test the hypothesis     -> run the one command that proves or disproves it
        |  (wrong? go back to 2 or 3)
5. Fix                     -> smallest safe change; back up first
        |
6. Verify                  -> repeat the original failing action; it works now
        |
7. Document                -> what happened, cause, fix, how to prevent it
```

Three habits make this loop fast:

- **Read the exact error message.** "No space left on device", "Read-only file system", "Permission denied", "Operation not permitted", and "Disk quota exceeded" are five different problems. The kernel chooses the message carefully.
- **Ask "what changed?"** Most outages follow a change: an update, a config edit, a full disk after a log storm.
- **Change one thing at a time** and keep a copy of every file before you edit it (`sudo cp file file.bak`), so you can always undo.

Where the evidence lives.

Evidence	Command
Kernel messages (hardware, drivers, filesystems, OOM, segfaults)	`dmesg -T`, `journalctl -k`
Everything systemd logged this boot / last boot	`journalctl -b`, `journalctl -b -1`
One service	`systemctl status unit`, `journalctl -xeu unit`
Disk space and inodes	`df -h`, `df -i`, `du`, `lsof +L1`
Mounts and their options	`findmnt`, `mount`
Memory and processes	`free -h`, `ps`, `top`, `/proc/PID/status`
Packages	`dpkg -l`, `dpkg -V`, `apt`, `rpm -V`, `dnf`

Practice safely with loop devices. Most storage scenarios use *loop devices*: a regular file that the kernel presents as a disk (`/dev/loop0`). `mount -o loop file.img /mnt/x` sets one up automatically. You can corrupt, fill, and break these files as much as you like without touching your real disks; deleting the file removes the "disk".

Documenting. Keep a simple incident log while you work. For every scenario, write one line per method step. You will create the log file in Task 1. In real jobs this becomes the ticket notes or post-incident review.

Tasks

Task 1: Prepare the lab and the incident log

1. Take a snapshot of ubuntu-lab now if you have not already. Then create the lab directory and install the tools used across the scenarios.

```
mkdir -p ~/labs/5.2 && cd ~/labs/5.2
sudo apt update
sudo apt install -y lsof e2fsprogs quota strace gcc gdb smartmontools mdadm smem
```

> **What just happened:** `lsof` lists open files, `e2fsprogs` provides `mkfs.ext4`, `e2fsck`, `debugfs`, `tune2fs`, and `badblocks`, `quota` provides the quota tools, `strace` shows the system calls a process makes, `gcc` and `gdb` compile and debug a C program, `smartmontools` provides `smartctl`, `mdadm` manages software RAID, and `smem` reports memory use per process. Installing `mdadm` may rebuild the initramfs; that is normal.

2. Create the incident log. A **heredoc** sends the lines between `<<'EOF'` and `EOF` into a command; the quotes around `EOF` stop the shell from expanding `$` signs inside.

```
cat <<'EOF' > ~/labs/5.2/incident-log.txt
# Template - copy for every scenario
Scenario:
1 Symptom:
2 Data gathered:
3 Hypothesis:
4 Test result:
5 Fix:
6 Verification:
7 Prevention:
EOF
cat ~/labs/5.2/incident-log.txt
```

3. Learn the shape of every scenario below. Each one has the same parts, in bold, so you always know where you are: **Break it** (you create the problem), **Symptom** (what a user would see), **Investigate** (gather data and test a hypothesis), **Root cause**, **Fix**, **Verify**, and **Prevent**. Each scenario ends with an **Undo** step so the next scenario starts clean.

> **Why break things on purpose:** On the job, problems arrive without warning and under pressure. Creating each failure yourself, in a VM you can restore, lets you see the exact error text, the log lines, and the tool output once in a calm setting, so you recognize them instantly later. Scenarios that need the VM console (Tasks 6, 7, 8, and 17) are marked; do the others over SSH.

> **Checkpoint:** `ls ~/labs/5.2` shows `incident-log.txt`, and `which lsof strace gdb smartctl mdadm` prints five paths.

Task 2: Scenario 1: "No space left on device" (filesystem full and inode exhaustion)

The same error message has two very different causes. You will create both.

Part A: the filesystem is full

1. **Break it.** Create a small 64 MB filesystem, fill it with a log file, hide 20 MB in a deleted file that a process still holds open, then fill the rest.

```
cd ~/labs/5.2
truncate -s 64M small.img
mkfs.ext4 -q small.img
sudo mkdir -p /mnt/small
sudo mount -o loop small.img /mnt/small
sudo chown student: /mnt/small
fallocate -l 25M /mnt/small/app.log
fallocate -l 20M /mnt/small/huge.tmp
sleep infinity < /mnt/small/huge.tmp &
rm /mnt/small/huge.tmp
dd if=/dev/zero of=/mnt/small/filler bs=4k status=none
```

> **What just happened:** `truncate -s 64M` creates an empty 64 MB file, `mkfs.ext4 -q` formats it quietly, and `mount -o loop` attaches it to a free loop device and mounts it. `fallocate -l` reserves space for a file instantly. `sleep infinity < huge.tmp &` starts a background process whose standard input is `huge.tmp`, so the file stays open. Then `rm` deletes the *name*, but the space is not freed while a process holds the file open. Finally `dd` writes zeros in 4 KB blocks until the disk is full and stops with `No space left on device`, which is expected. (Small blocks make sure even the last few free blocks get used.)

2. **Symptom.** Try to save a file, as a user or application would.

```
echo "hello" > /mnt/small/test.txt
```

```
-bash: echo: write error: No space left on device
```

3. **Investigate.** Start with `df` (disk free) for the whole filesystem, then `du` (disk usage) for the files you can see.

```
df -h /mnt/small
du -sh /mnt/small/* 2>/dev/null
du -sh /mnt/small 2>/dev/null
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop3       56M   52M     0 100% /mnt/small
25M     /mnt/small/app.log
6.5M    /mnt/small/filler
16K     /mnt/small/lost+found
0       /mnt/small/test.txt
32M     /mnt/small
```

> **What just happened:** `df` says 52 MB used, but `du` can only find about 32 MB of files. (`test.txt` is the empty file your failed `echo` created; `2>/dev/null` hides the "Permission denied" message for `lost+found` , which only root can read.) When `df` and `du` disagree, the usual hypothesis is a **deleted file that is still open**. Also notice `Avail 0` while `Used` is less than `Size` : ext4 reserves 5% of blocks for root, so ordinary users run out first. Root can still write, which keeps the system alive.

4. **Test the hypothesis** with `lsof +L1` , which lists open files whose link count is less than 1 (deleted but still open).

```
lsof +L1 /mnt/small
```

```
COMMAND  PID    USER FD  TYPE DEVICE SIZE/OFF NLINK NODE NAME
sleep   4120 student 0r  REG    7,3 20971520     0   13 /mnt/small/huge.tmp (deleted)
```

5. **Root cause.** Two things: a large log and filler data used the space, and 20 MB is held by a deleted file that process `4120` (`sleep`) still has open. This is extremely common in real life: someone deletes a huge log file, but the service still has it open, so the space never comes back.

6. **Fix.** Remove the junk, empty the log without deleting it, and release the deleted file by stopping the process that holds it.

```
rm /mnt/small/filler
truncate -s 0 /mnt/small/app.log
kill %1
df -h /mnt/small
```

> **What just happened:** `truncate -s 0` empties a file in place. Use it (or `: > file`) on logs that a service is writing; `rm` would leave the space held. `kill %1` stops background job 1 (the `sleep`), which closes the deleted file and frees its 20 MB. On a real server you would restart the service instead (for example `sudo systemctl restart nginx`). If you cannot restart it, you can empty the deleted file through `/proc` : `: > /proc/PID/fd/FD` .

7. **Verify.** `echo "hello" > /mnt/small/test.txt && cat /mnt/small/test.txt` now succeeds and `df -h` shows space available.

8. **Prevent.** Rotate logs with `logrotate` (using `copytruncate` or a reload), alert on disk usage above a threshold (Lab 5.1), put `/var` and `/home` on separate filesystems so a log storm cannot fill `/` , and find big items quickly with `sudo du -xh / --max-depth=2 | sort -h | tail` .

Part B: out of inodes

9. **Break it.** An **inode** is the small record that stores a file's metadata (owner, permissions, where its data blocks are). Every file and directory uses exactly one, and ext4 creates a fixed number when you format. Create a filesystem with only about 2,000 inodes and fill them with tiny "session" files.

```
cd ~/labs/5.2
truncate -s 64M inodes.img
mkfs.ext4 -q -N 2048 inodes.img
sudo mkdir -p /mnt/inodes
sudo mount -o loop inodes.img /mnt/inodes
sudo chown student: /mnt/inodes
mkdir /mnt/inodes/sessions
for i in $(seq 1 3000); do : > /mnt/inodes/sessions/sess_$i; done 2>/dev/null
```

What just happened: `-N 2048` asks `mkfs.ext4` for about 2,048 inodes instead of the 16,384 it would normally create on a 64 MB filesystem. The `for` loop tries to create 3,000 empty files (`: > file` creates an empty file). It runs out partway through; `2>/dev/null` hides the error messages.

10. **Symptom.**

```
touch /mnt/inodes/report.txt
```

```
touch: cannot touch '/mnt/inodes/report.txt': No space left on device
```

11. **Investigate.** Check space, then inodes.

```
df -h /mnt/inodes
df -i /mnt/inodes
du --inodes -s /mnt/inodes/* 2>/dev/null | sort -n | tail -n 3
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop4       60M   88K   55M   1% /mnt/inodes
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop4       2048  2048     0  100% /mnt/inodes
1       /mnt/inodes/lost+found
2037    /mnt/inodes/sessions
```

What just happened: `df -h` shows 1% used, so space is not the problem. `df -i` shows inodes (`IUse%` 100%). `du --inodes -s` counts inodes per directory, so the `sessions` directory is the culprit. On a real server, look at mail queues, PHP session directories, cache directories, and `/tmp`.

12. **Root cause.** Inode exhaustion: millions of tiny files (here about 2,036) used every inode while most of the space is free.

13. **Fix.** Delete the unneeded files. `find ... -delete` handles any number of files; `rm sessions/*` would fail with "Argument list too long" on a real directory with millions of files.

```
find /mnt/inodes/sessions -name 'sess_*' -delete
df -i /mnt/inodes
touch /mnt/inodes/report.txt && ls /mnt/inodes
```

14. **Prevent.** Clean old files automatically (`systemd-tmpfiles` or a cron job with `find -mtime +7 -delete`), monitor `df -i` as well as `df -h`, and format filesystems for many small files with more inodes (`mkfs.ext4 -i 4096` gives one inode per 4 KB). XFS and Btrfs allocate inodes dynamically, so they rarely hit this.

15. **Undo.**

```
sudo umount /mnt/small /mnt/inodes
rm -f ~/labs/5.2/small.img ~/labs/5.2/inodes.img
```

Checkpoint: You can explain why `df -h` and `du` disagreed in Part A (deleted-but-open file found with `lsof +L1`) and why `df -h` showed 1% in Part B (inodes, found with `df -i`). `findmnt /mnt/small /mnt/inodes` prints nothing.

Task 3: Scenario 2: "I can't save my file" (partition not writable)

A partition can be not writable for several reasons, and each gives a different error. You will create three at once and use the messages to tell them apart.

1. **Break it.**

```
cd ~/labs/5.2
truncate -s 64M rw.img
mkfs.ext4 -q rw.img
sudo mkdir -p /mnt/rw /mnt/rw2
sudo mount -o loop rw.img /mnt/rw
sudo mkdir /mnt/rw/locked /mnt/rw/reports
sudo chown student: /mnt/rw/reports
echo "Q3 numbers" > /mnt/rw/reports/q3.txt
sudo chattr +i /mnt/rw/reports/q3.txt
truncate -s 64M ro.img && mkfs.ext4 -q ro.img
sudo mount -o loop ro.img /mnt/rw2
sudo mount -o remount,ro /mnt/rw2
```

What just happened: You created a filesystem with two directories (`locked` owned by root, `reports` owned by student), made `q3.txt` immutable with `chattr +i`, and mounted a second filesystem, then switched it to read-only with `remount,ro`, which is exactly what the kernel does on its own after detecting errors. (Mounting with `-o loop,ro` directly would also make the *loop device* read-only, and then even `remount,rw` would fail with "is write-protected".)

2. **Symptom.** Try three writes as student.

```
touch /mnt/rw/locked/new.txt
echo "more" >> /mnt/rw/reports/q3.txt
sudo touch /mnt/rw2/new.txt
```

```
touch: cannot touch '/mnt/rw/locked/new.txt': Permission denied
-bash: /mnt/rw/reports/q3.txt: Operation not permitted
touch: cannot touch '/mnt/rw2/new.txt': Read-only file system
```

What just happened: Three different messages for three different causes. "Permission denied" (`EACCES`) means the normal permission check failed. "Operation not permitted" (`EPERM`) on a file you own often means a file **attribute** blocks it. "Read-only file system" (`EROFS`) means the whole mount is read-only, so even root fails.

3. **Investigate cause 1: permissions.**

```
ls -ld /mnt/rw/locked
namei -l /mnt/rw/locked/new.txt
id
```

```
drwxr-xr-x 2 root root 4096 Oct 10 12:01 /mnt/rw/locked
f: /mnt/rw/locked/new.txt
drwxr-xr-x root root /
drwxr-xr-x root root mnt
drwxr-xr-x root root rw
drwxr-xr-x root root locked
                     new.txt - No such file or directory
```

What just happened: Creating a file needs **write** and **execute** permission on the *directory*. `locked` is owned by root with mode `755` , so others (student) have only `r-x` . `namei -l` shows permissions for every directory along a path, which quickly finds a blocking parent directory. `id` confirms which user and groups you are.

4. **Investigate cause 2: attributes.**

```
ls -l /mnt/rw/reports/q3.txt
lsattr /mnt/rw/reports/q3.txt
```

```
-rw-rw-r-- 1 student student 11 Oct 10 12:01 /mnt/rw/reports/q3.txt
----i---------e------- /mnt/rw/reports/q3.txt
```

What just happened: The normal permissions allow student to write, yet the write failed. `lsattr` shows extended filesystem attributes: `i` is **immutable** (nobody, not even root, can modify, rename, or delete the file) and `e` just means the file uses extents (normal on ext4). Another attribute you may meet is `a` , append-only, often used on logs.

5. **Investigate cause 3: read-only mount.**

```
findmnt -o TARGET,SOURCE,OPTIONS /mnt/rw2
sudo dmesg -T | grep -i -E "remount|read-only|EXT4-fs error" | tail -n 5
```

```
TARGET   SOURCE     OPTIONS
/mnt/rw2 /dev/loop6 ro,relatime
```

What just happened: `findmnt` shows the mount options; `ro` means read-only. Here you remounted it read-only on purpose, so `dmesg` shows nothing interesting. On a real server, a filesystem that *suddenly* becomes read-only was usually remounted by the kernel after it detected errors. Then `dmesg` shows lines like the ones below, and the right response is to find the disk problem, not just remount:

```
EXT4-fs error (device sdb1): ext4_lookup:1855: inode #2: comm ls: deleted inode
referenced: 12
EXT4-fs (sdb1): Remounting filesystem read-only
```

The behavior is set per filesystem with the `errors=` mount option (`continue`, `remount-ro`, or `panic`); `sudo tune2fs -l /dev/sdb1 | grep -i behavior` shows the default stored in the superblock.

6. **Root cause.** Cause 1: directory permissions. Cause 2: immutable attribute. Cause 3: read-only mount. (A fourth cause, a full filesystem, gives "No space left on device", which you saw in Scenario 1.)

7. **Fix** each one with the matching tool.

```
sudo chown student: /mnt/rw/locked
sudo chattr -i /mnt/rw/reports/q3.txt
sudo mount -o remount,rw /mnt/rw2
```

What just happened: `chown` gives student ownership of the directory (you could also use `chmod` or an ACL with `setfacl -m u:student:rwx`). `chattr -i` removes the immutable attribute. `mount -o remount,rw` changes a mounted filesystem to read-write without unmounting it. If the kernel had forced it read-only after errors, you would instead unmount it, run `e2fsck`, then mount it again.

8. **Verify.** Repeat the three writes from step 2; all succeed.

```
touch /mnt/rw/locked/new.txt
echo "more" >> /mnt/rw/reports/q3.txt
sudo touch /mnt/rw2/new.txt
```

9. **Prevent.** Document why a file is immutable when you set it (it confuses the next admin), check `findmnt` and `dmesg` first when writes fail for everyone, and use `namei -l` before changing permissions so you change the right directory.

10. **Undo.**

```
sudo umount /mnt/rw /mnt/rw2
rm -f ~/labs/5.2/rw.img ~/labs/5.2/ro.img
```

Checkpoint: Without looking back, match each message to its cause: "Permission denied", "Operation not permitted", "Read-only file system", "No space left on device".

Task 4: Scenario 3: "Disk quota exceeded"

A **disk quota** limits how much space (blocks) or how many files (inodes) a user or group may use on a filesystem. The **soft limit** can be exceeded for a **grace period** (7 days by default); the **hard limit** can never be exceeded.

1. **Break it.** Create a filesystem with user quotas turned on and give student a 5 MB soft and 6 MB hard limit.

```
cd ~/labs/5.2
truncate -s 128M quota.img
mkfs.ext4 -q quota.img
sudo mkdir -p /mnt/quota
sudo mount -o loop,usrquota quota.img /mnt/quota
sudo quotacheck -cum /mnt/quota
sudo quotaon -v /mnt/quota
sudo chmod 1777 /mnt/quota
sudo setquota -u student 5000 6000 0 0 /mnt/quota
```

```
/dev/loop3 [/mnt/quota]: user quotas turned on
```

What just happened: The `usrquota` mount option enables user quota accounting. `quotacheck -cum` scans the filesystem and **c**reates the quota file `aquota.user` for **u**sers, without trying to re**m**ount read-only first. `quotaon` starts enforcing limits. `chmod 1777` makes the directory writable by everyone (with the sticky bit, like `/tmp`). `setquota -u student 5000 6000 0 0` sets block soft 5000 KB, block hard 6000 KB, and no inode limits (`0` means unlimited). Quota block numbers are in 1 KB units.

TIP

If `quotaon` reports that the quota format is not supported by the kernel, install the extra kernel modules with `sudo apt install -y linux-modules-extra-$(uname -r)` and run `quotaon` again.

2. **Symptom.** Student tries to save a 10 MB file.

```
dd if=/dev/zero of=/mnt/quota/video.raw bs=1M count=10
```

```
dd: error writing '/mnt/quota/video.raw': Disk quota exceeded
6+0 records in
5+1 records out
6144000 bytes (6.1 MB, 5.9 MiB) copied, 0.0201 s, 306 MB/s
```

`5+1` means five full 1 MB blocks and one partial block were written before the limit (6000 KB = 6,144,000 bytes) stopped it. Your byte count may be slightly lower.

3. **Investigate.** The filesystem has plenty of space, so check quotas.

```
df -h /mnt/quota
quota -u student
sudo repquota /mnt/quota
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop3      115M  6.0M  100M   6% /mnt/quota
Disk quotas for user student (uid 1000):
     Filesystem  blocks   quota   limit   grace   files   quota   limit   grace
     /dev/loop3   6000*    5000    6000   6days       1       0       0
*** Report for user quotas on device /dev/loop3
Block grace time: 7days; Inode grace time: 7days
                        Block limits                File limits
User            used    soft    hard  grace    used  soft  hard  grace
----------------------------------------------------------------------
root      --      20       0       0              2     0     0
student   +-    6000    5000    6000  6days       1     0     0
```

What just happened: `quota -u student` shows one user's usage; the `*` after `6000` marks a limit that is exceeded, and `grace` shows time left before the soft limit is enforced like a hard limit. `repquota` reports every user; in the `+-` column, the first character is blocks and the second is files, and `+` means over the soft limit. student has hit the hard limit (6000), so writes fail even though the disk is 94% free.

4. **Root cause.** The user hit their hard block quota.
5. **Fix.** Either the user deletes files, or an admin raises the limit if it is justified. Raise it to 20 MB soft, 25 MB hard. (`sudo edquota -u student` does the same in an editor; `sudo edquota -t` changes grace periods.)

```
rm /mnt/quota/video.raw
sudo setquota -u student 20000 25000 0 0 /mnt/quota
```

6. **Verify.**

```
dd if=/dev/zero of=/mnt/quota/video.raw bs=1M count=10 status=none && quota -u student
```

7. **Prevent.** Tell users their limits, use `warnquota` to email users who pass their soft limit, and review `repquota -a` regularly. When quotas are enabled permanently, add `usrquota` / `grpquota` to the options in `/etc/fstab`.
8. **Undo.**

```
sudo quotaoff /mnt/quota
sudo umount /mnt/quota
rm -f ~/labs/5.2/quota.img
```

Checkpoint: You saw `Disk quota exceeded` with `df` showing lots of free space, and `sudo repquota` showed student with `+-` before your fix.

Task 5: Scenario 4: Filesystem will not mount, and data corruption

Data corruption comes in two flavors: damaged *metadata* (the filesystem's own structures, which `fsck` can repair) and damaged *file contents* (which `fsck` cannot see at all). You will create both.

1. **Prepare** a 256 MB ext4 filesystem with an important file, a checksum, and a backup copy.

```
cd ~/labs/5.2
truncate -s 256M data.img
mkfs.ext4 -q -b 4096 data.img
sudo mkdir -p /mnt/data
sudo mount -o loop data.img /mnt/data
seq 1 200000 | sudo tee /mnt/data/important.txt > /dev/null
(cd /mnt/data && sha256sum important.txt) > important.sha256
cp /mnt/data/important.txt important.bak
sudo umount /mnt/data
```

What just happened: `-b 4096` fixes the block size at 4 KB so the numbers below are predictable. `seq 1 200000` writes 200,000 numbered lines (about 1.2 MB). `sha256sum` records a fingerprint of the file; any change to even one byte changes the fingerprint. The parentheses run `cd` in a subshell, so your own shell stays in `~/labs/5.2`.

2. **Break it (metadata).** The **superblock** is the master record of an ext filesystem (size, block counts, features). It starts at byte 1024. Overwrite it with zeros.

```
dd if=/dev/zero of=data.img bs=1024 seek=1 count=1 conv=notrunc
```

What just happened: `bs=1024 seek=1 count=1` writes one 1 KB block of zeros starting at offset 1024. `conv=notrunc` tells `dd` not to shorten the image file. This imitates a bad write, a failing disk, or someone running a tool against the wrong device.

3. **Symptom.**

```
sudo mount -o loop data.img /mnt/data
```

```
mount: /mnt/data: wrong fs type, bad option, bad superblock on /dev/loop3,
       missing codepage or helper program, or other error.
       dmesg(1) may have more information after failed mount system call.
```

4. **Investigate.**

```
sudo dmesg | tail -n 2
sudo blkid data.img; echo "blkid exit code: $?"
dumpe2fs -h data.img
```

```
[ 9231.441021] EXT4-fs (loop3): VFS: Can't find ext4 filesystem
blkid exit code: 2
dumpe2fs 1.47.0 (5-Feb-2023)
dumpe2fs: Bad magic number in super-block while trying to open data.img
```

What just happened: The kernel cannot find an ext4 filesystem (you may also see lines from other filesystem types that `mount` tried), `blkid` cannot identify any filesystem type (exit code `2` means nothing found), and `dumpe2fs -h` (print the superblock header) reports a bad **magic number**, the signature bytes that identify an ext superblock. Hypothesis: the primary superblock is damaged. ext filesystems keep **backup superblocks**, so it is fixable.

5. **Test the hypothesis:** find where the backups are. `mke2fs -n` is a dry run: it prints what it *would* do (including backup locations) without writing anything. You must give it the same block size you used originally.

```
mke2fs -n -b 4096 data.img | grep -A1 -i backup
```

```
Superblock backups stored on blocks:
        32768
```

CAUTION

Never forget the `-n`. Without it, `mke2fs` creates a new, empty filesystem and destroys whatever was there.

6. **Fix the metadata.** Run `e2fsck` using the backup superblock.

```
e2fsck -f -y -b 32768 -B 4096 data.img
```

```
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
Free blocks count wrong for group #0 (28629, counted=28314).
Fix? yes
...
Block bitmap differences: Group 0 block bitmap does not match checksum.
FIXED.

data.img: ***** FILE SYSTEM WAS MODIFIED *****
data.img: 12/65536 files (0.0% non-contiguous), 8583/65536 blocks
```

What just happened: `-b 32768` reads the superblock from the backup, `-B 4096` gives the block size, `-f` forces a full check, and `-y` answers yes to every repair question. When it finishes, it writes a good primary superblock. The backup superblock is older than the primary, so its free-block and free-inode counts are out of date; `e2fsck` recounts them and fixes the bitmaps. Your exact messages may differ; what matters is that it finishes. (An exit code of `1` means "errors were corrected".) On a real disk you run `e2fsck` (or `fsck`) on the device, for example `sudo e2fsck -f /dev/sdb1`, and **only when it is unmounted**. XFS uses `xfs_repair` instead.

7. **Verify** the mount works and the file is intact.

```
sudo mount -o loop data.img /mnt/data
(cd /mnt/data && sha256sum -c ~/labs/5.2/important.sha256)
sudo umount /mnt/data
```

```
important.txt: OK
```

8. **Break it (file contents).** Now damage the file's data instead. `debugfs` can tell you which disk blocks hold the file.

```
BLK=$(debugfs -R "blocks /important.txt" data.img 2>/dev/null | awk '{print $1}')
echo "first data block of important.txt: $BLK"
dd if=/dev/urandom of=data.img bs=4096 seek="$BLK" count=1 conv=notrunc status=none
```

> **What just happened:** `debugfs -R "blocks /important.txt"` prints the block numbers that store the file; `awk '{print $1}'` keeps the first one. `dd` then overwrites that 4 KB block with random bytes. This is what a failing disk, bad RAM, or a buggy controller does silently.

9. **Symptom and investigation.**

```
e2fsck -f -n data.img | tail -n 1
sudo mount -o loop data.img /mnt/data
head -c 120 /mnt/data/important.txt | cat -v | head -n 3
(cd /mnt/data && sha256sum -c ~/labs/5.2/important.sha256)
```

```
data.img: 12/65536 files (0.0% non-contiguous), 8583/65536 blocks
M-^Z^Q^GM-2kM-^@M-?^]x...
important.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

> **What just happened:** `e2fsck -n` (check only, change nothing) reports the filesystem as healthy, because all metadata is fine. Yet the file starts with garbage (`cat -v` shows unprintable bytes safely) and the checksum fails. **`fsck` does not protect file contents.** Only checksums, backups, and checksumming filesystems (Btrfs, ZFS) detect this kind of corruption.

10. **Root cause.** One data block of the file was overwritten (silent data corruption).

11. **Fix.** Restore the file from backup.

```
sudo cp ~/labs/5.2/important.bak /mnt/data/important.txt
```

12. **Verify.** `(cd /mnt/data && sha256sum -c ~/labs/5.2/important.sha256)` prints `important.txt: OK`.

13. **Prevent.** Keep tested backups, store checksums for critical data, use ECC memory on servers, monitor disks with SMART (Task 16), and check `dmesg` for I/O errors. Never run `fsck` on a mounted filesystem.

14. **Undo.**

```
sudo umount /mnt/data
rm -f ~/labs/5.2/data.img ~/labs/5.2/important.bak ~/labs/5.2/important.sha256
```

Checkpoint: You repaired a "bad superblock" mount failure with `e2fsck -b 32768`, and you saw `fsck` report a clean filesystem while `sha256sum -c` caught corrupted file data.

Task 6: Scenario 5: The server boots to a BusyBox prompt (GRUB misconfiguration)

GRUB is the bootloader: it loads the kernel and initramfs and passes the kernel its command line (for example `root=` to say where the root filesystem is). On Ubuntu you edit `/etc/default/grub` and run `update-grub`, which regenerates `/boot/grub/grub.cfg`. You never edit `grub.cfg` directly; it is overwritten on every kernel update.

CAUTION

This scenario stops the VM from booting until you fix it. Use the VM console (hypervisor window), not SSH, and make sure you have a snapshot.

1. **Prepare: make the GRUB menu visible.** Ubuntu hides the menu on single-OS systems, which makes recovery hard. Showing it for 10 seconds is a correct and common server change.

```
sudo cp /etc/default/grub /etc/default/grub.lab-orig
sudo sed -i 's/^GRUB_TIMEOUT_STYLE=.*/GRUB_TIMEOUT_STYLE=menu/;
s/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=10/' \
    /etc/default/grub
grep -E '^GRUB_(TIMEOUT|TIMEOUT_STYLE|CMDLINE_LINUX_DEFAULT)=' /etc/default/grub
sudo update-grub
```

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=10
GRUB_CMDLINE_LINUX_DEFAULT=""
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-45-generic
Found initrd image: /boot/initrd.img-6.8.0-45-generic
done
```

What just happened: `sed -i` edits the file in place with two substitutions separated by `;`. `GRUB_TIMEOUT_STYLE=menu` always shows the menu, and `GRUB_TIMEOUT=10` waits 10 seconds before booting the default entry. `update-grub` (a wrapper for `grub-mkconfig -o /boot/grub/grub.cfg`) rebuilds the real configuration. On Rocky the equivalent is `sudo grub2-mkconfig -o /boot/grub2/grub.cfg`, or `grubby` for kernel arguments. Files in `/etc/default/grub.d/` are read after `/etc/default/grub` and win; if `grep -rs TIMEOUT /etc/default/grub.d/` prints anything (common on cloud images), the menu may stay hidden, so you would also need to change those values.

2. **Break it.** Add a bad `root=` kernel parameter. Because `/etc/default/grub` is a shell script, a line at the end can redefine a variable; `$GRUB_CMDLINE_LINUX_DEFAULT` keeps whatever was there before.

```
echo 'GRUB_CMDLINE_LINUX_DEFAULT="$GRUB_CMDLINE_LINUX_DEFAULT root=/dev/sdz9"' | \
    sudo tee -a /etc/default/grub
sudo update-grub
grep -m1 -o 'root=/dev/sdz9' /boot/grub/grub.cfg
sudo reboot
```

3. **Symptom.** Watch the console. The GRUB menu appears and boots normally, then after about 30 seconds:

```
Gave up waiting for root file system device.  Common problems:
 - Boot args (cat /proc/cmdline)
   - Check rootdelay= (did the system wait long enough?)
 - Missing modules (cat /proc/modules; ls /dev)
ALERT!  /dev/sdz9 does not exist.  Dropping to a shell!

BusyBox v1.36.1 (Ubuntu 1:1.36.1-6ubuntu3.1) built-in shell (ash)
Enter 'help' for a list of built-in commands.

(initramfs)
```

4. **Investigate** from the `(initramfs)` prompt. This tiny shell runs from the initramfs, before the real root filesystem is mounted.

```
cat /proc/cmdline
```

```
BOOT_IMAGE=/vmlinuz-6.8.0-45-generic root=/dev/mapper/ubuntu--vg-ubuntu--lv ro
root=/dev/sdz9
```

> **What just happened:** `/proc/cmdline` shows the exact kernel command line GRUB passed. There are two `root=` parameters and the last one wins. `/dev/sdz9` does not exist, so the initramfs cannot mount the root filesystem and drops to a rescue shell.

5. **Root cause.** A bad kernel parameter in `/etc/default/grub` was written into `grub.cfg` by `update-grub`.

6. **Fix for this boot: edit the entry in the GRUB menu.** At the `(initramfs)` prompt type `reboot -f` (or reset the VM from the hypervisor). When the GRUB menu appears, press an arrow key to stop the countdown, highlight **Ubuntu**, and press `e`. Find the line that starts with `linux` and delete `root=/dev/sdz9` from its end (use the arrow keys and Backspace). Press `Ctrl+X` or `F10` to boot.

> **What just happened:** Pressing `e` edits a menu entry **for this boot only**; nothing is saved. This is how you fix a bad kernel parameter, add `single` or `systemd.unit=rescue.target` to boot into rescue mode, or add `init=/bin/bash` to get a root shell without a password (useful when the root account is locked; remount `/` with `mount -o remount,rw /` before editing files, then run `exec /sbin/init` or reset). The **Advanced options for Ubuntu** submenu lists every installed kernel plus a **recovery mode** entry for each.

7. **Fix permanently.** Log in and remove the bad line, then regenerate the configuration.

```
sudo sed -i '/root=\/dev\/sdz9/d' /etc/default/grub
sudo update-grub
grep -c 'sdz9' /boot/grub/grub.cfg
```

```
0
```

8. **Verify.** `sudo reboot`; the VM boots normally without any editing, and `cat /proc/cmdline` shows a single `root=`.

9. **Prevent.** Keep a copy of `/etc/default/grub` before editing, check the generated file with `grep` before rebooting, keep the menu visible (or know that holding `Shift` on BIOS or pressing `Esc` on UEFI shows a hidden menu), and keep at least one older kernel installed.

> TIP
> If GRUB itself cannot find its files (for example after a disk change), you see a `grub rescue>` prompt instead. There you use `ls` to find partitions, `set prefix=(hd0,gpt2)/boot/grub` and `set root=(hd0,gpt2)`, then `insmod normal` and `normal`; afterwards boot the system and run `sudo grub-install /dev/sda` (Rocky: `grub2-install`) to fix it for good.

Leave the GRUB menu visible for the next two scenarios; the Clean up section restores the original file.

> **Checkpoint:** The VM boots normally, `grep -c sdz9 /boot/grub/grub.cfg` prints `0`, and you know how to edit a boot entry with `e` and boot it with `Ctrl+X`.

Task 7: Scenario 6: Boot stops in emergency mode (filesystem will not mount)

At boot, systemd mounts everything listed in `/etc/fstab`. If a required filesystem will not mount, `local-fs.target` fails and systemd drops into **emergency mode**: a minimal root shell for repairs.

> CAUTION
> This scenario stops normal booting. Use the VM console and have a snapshot.

1. **Prepare: give root a password.** Ubuntu locks the root account, and emergency mode then refuses to give you a shell ("Cannot open access to console, the root account is locked"). Many servers set a root password for exactly this reason. You will lock it again in Clean up.

```
sudo passwd root
```

> TIP
> If you skip this step, you can still recover: edit the boot entry with `e` (Task 6), add `init=/bin/bash` to the `linux` line, boot, run `mount -o remount,rw /`, fix `/etc/fstab`, run `sync`, and reset the VM.

2. **Break it.** Back up `fstab` and add a filesystem whose UUID does not exist, as happens when a disk is replaced or a UUID is mistyped.

```
sudo cp /etc/fstab /etc/fstab.lab-orig
echo 'UUID=11111111-2222-3333-4444-555555555555 /srv/data ext4 defaults 0 2' | \
    sudo tee -a /etc/fstab
sudo systemctl daemon-reload
```

3. **Investigate before rebooting** (the step people forget). Two commands catch fstab mistakes safely:

```
sudo findmnt --verify
sudo mount -a
```

```
/srv/data
   [E] unreachable on boot required target: No such file or directory
   [E] unreachable on boot required source: UUID=11111111-2222-3333-4444-555555555555
0 parse errors, 2 errors, 0 warnings
mount: /srv/data: can't find UUID=11111111-2222-3333-4444-555555555555.
```

> **What just happened:** `findmnt --verify` checks every fstab line (syntax, devices, mount points, filesystem types) without mounting anything. `mount -a` tries to mount everything in fstab that is not yet mounted, which is what boot will do. Both report the problem: the source device does not exist, and neither does the mount point (systemd would create it, but the device is the real issue). In real life you would stop here and fix it; for practice, reboot anyway.

4. **Symptom.** `sudo reboot` and watch the console. After about 90 seconds:

```
[ TIME ] Timed out waiting for device dev-disk-by\x2duuid-11111111...device
[DEPEND] Dependency failed for srv-data.mount - /srv/data.
[DEPEND] Dependency failed for local-fs.target - Local File Systems.
You are in emergency mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, "systemctl default" or "exit"
to boot into default mode.
Give root password for maintenance
(or press Control-D to continue):
```

5. **Investigate** in emergency mode. Type the root password, then:

```
systemctl --failed
journalctl -xb -p err | tail -n 20
```

> **What just happened:** `systemctl --failed` lists failed units (you will see `srv-data.mount`). systemd turns each fstab line into a `.mount` unit named after its path (`/srv/data` becomes `srv-data.mount`). `journalctl -xb -p err` shows errors from this boot with explanations (`-x`). The 90-second wait is systemd's default device timeout.

6. **Root cause.** A required fstab entry points to a device that does not exist.

7. **Fix** in emergency mode. The root filesystem may be read-only here, so remount it read-write first, then remove the bad line (or restore the backup).

```
mount -o remount,rw /
cp /etc/fstab.lab-orig /etc/fstab
systemctl daemon-reload
mount -a
systemctl default
```

What just happened: `mount -o remount,rw /` makes `/` writable. You restore the saved fstab, `daemon-reload` makes systemd regenerate the mount units from the fixed file, `mount -a` proves every entry now mounts, and `systemctl default` continues booting to the normal target (you could also `reboot`).

8. **Verify.** You reach the normal login prompt. Log in and run `systemctl --failed`; it shows `0 loaded units listed`.
9. **Prevent.** Always run `sudo findmnt --verify` and `sudo mount -a` after editing fstab. Use UUIDs from `blkid`, not `/dev/sdX` names. For disks that are not needed to boot (USB, extra data, network shares), add `nofail` (boot continues if missing) and optionally `x-systemd.device-timeout=10s`; network filesystems also get `_netdev`.

Checkpoint: You saw "You are in emergency mode", repaired `/etc/fstab` from the emergency shell, and `systemctl --failed` is now empty.

Task 8: Scenario 7: Kernel panic and kernel corruption

A **kernel panic** is a fatal kernel error: the kernel stops because it cannot safely continue. The console shows `Kernel panic - not syncing:` and a reason. Common causes: the root filesystem cannot be mounted (`VFS: Unable to mount root fs on unknown-block(0,0)`, usually a missing or broken initramfs), `Attempted to kill init!` (PID 1 died), faulty RAM or CPU, and buggy drivers.

Part A: simulate a kernel panic (explore/simulate)

TIP

Explore/simulate: You cannot make a real hardware fault, but the kernel's magic SysRq interface can deliberately crash it. This is completely safe on a VM with a snapshot, but it is an instant power-cut for the OS: save your work and close other sessions first.

1. Look at the panic-related settings, then make the kernel reboot automatically 10 seconds after a panic (the default `0` means hang forever, waiting for a human).

```
sysctl kernel.panic kernel.panic_on_oops kernel.sysrq
sudo sysctl -w kernel.panic=10
```

```
kernel.panic = 0
kernel.panic_on_oops = 0
kernel.sysrq = 176
kernel.panic = 10
```

What just happened: `kernel.panic` is the number of seconds before automatic reboot after a panic. `kernel.panic_on_oops=0` means a less serious kernel error (an **oops**) kills only the offending task instead of panicking. `kernel.sysrq` controls which SysRq key combinations work from the keyboard; root can always use `/proc/sysrq-trigger`. `sysctl -w` changes a value until the next reboot.

2. **Break it.** From the VM console:

> **CAUTION**
> This crashes the whole VM instantly, like pulling the power cord. Make sure you have a snapshot, run it from the VM console (an SSH session just freezes and drops), and close other sessions and editors first. Never run it on a real server.

```
sync
echo c | sudo tee /proc/sysrq-trigger
```

3. **Symptom.** The console fills with a trace and ends like this, then the VM reboots after 10 seconds:

```
sysrq: Trigger a crash
Kernel panic - not syncing: sysrq triggered crash
CPU: 1 PID: 2112 Comm: tee Not tainted 6.8.0-45-generic #45-Ubuntu
Call Trace:
 <TASK>
 dump_stack_lvl+0x48/0x70
 panic+0x33c/0x370
 sysrq_handle_crash+0x1a/0x20
...
Kernel Offset: 0x2a600000 from 0xffffffff81000000
Rebooting in 10 seconds..
```

4. **Investigate after the reboot.** The panic itself is rarely in the journal, because the kernel stopped before anything could be written to disk. What you *can* see is that the previous boot ended abruptly.

```
journalctl --list-boots | tail -n 3
journalctl -b -1 -n 5 --no-pager
last -x | head -n 5
```

What just happened: `journalctl --list-boots` lists boots (index `0` is the current one, `-1` the previous). `journalctl -b -1 -n 5` shows the last five lines of the previous boot: they stop suddenly, with no shutdown messages. `last -x` reads the login records; a session that ended with `crash` instead of a normal shutdown confirms an unclean stop. To keep the actual panic text, you need a crash dump.

5. **Learn crash dumps (kdump). kdump** reserves memory for a second, small "capture" kernel. When the main kernel panics, the capture kernel boots and saves the crashed kernel's memory to `/var/crash`, where it can be analyzed with the `crash` tool or sent to a vendor. On Ubuntu you install `linux-crashdump` (which brings

`kdump-tools`; check it with `kdump-config show`); on Rocky, `kexec-tools` and `kdump.service`. Both add a `crashkernel=` parameter to the kernel command line and need a reboot. You do not need to install it for this lab, but know the names.

Part B: kernel corruption

Kernel corruption means the kernel image (`/boot/vmlinuz-*`), the initramfs (`/boot/initrd.img-*`), or kernel modules (`/lib/modules/<version>/`) on disk are damaged or incomplete, often after a full `/boot`, an interrupted update, or a disk problem. You will corrupt one unimportant module.

6. **Break it.** Find the file for the `dummy` network module (a fake network interface driver), note which package owns it, keep a copy, and overwrite part of it.

```
sudo modprobe -r dummy 2>/dev/null
MOD=$(modinfo -n dummy); echo "$MOD"
PKG=$(dpkg -S "$MOD" | cut -d: -f1); echo "$PKG"
cp "$MOD" ~/labs/5.2/dummy-module.bak
sudo dd if=/dev/urandom of="$MOD" bs=1 count=64 seek=200 conv=notrunc status=none
```

```
/lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
linux-modules-6.8.0-45-generic
```

> **What just happened:** `modinfo -n` prints the file name of a module. `dpkg -S` searches the package database for the package that installed a file; `cut -d: -f1` keeps the package name. Ubuntu compresses modules with zstd (`.ko.zst`). `dd` overwrites 64 bytes in the middle of the file with random data. (If `modinfo` cannot find `dummy`, your kernel is missing the extra modules package; see Task 9.)

7. **Symptom.**

```
sudo modprobe dummy
```

```
modprobe: ERROR: could not insert 'dummy': Invalid argument
```

The exact text can differ (for example `Exec format error`); any error loading a module that used to load is the symptom. Also check `sudo dmesg | tail -n 3` for a decompression or module format error.

8. **Investigate.** Ask the package manager to verify the files of the kernel packages against their recorded checksums.

```
dpkg -V "$PKG"
dpkg -V "linux-image-$(uname -r)"
```

```
??5??????   /lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
```

> **What just happened:** `dpkg -V` (verify) prints only files that differ from the package. The `5` in the third column means the MD5 checksum does not match: the file content changed. No output for `linux-image` means the kernel image itself is fine. On Rocky, `rpm -V kernel-core` does the same (`S.5....T.` means size, checksum, and time differ).

9. **Root cause.** A kernel module file on disk is corrupted.
10. **Fix.** Reinstall the package that owns the file; this rewrites all its files from a fresh download.

```
sudo apt install --reinstall -y "$PKG"
dpkg -V "$PKG"
```

> TIP
> If apt answers that the package "cannot be downloaded" (your running kernel is older than anything left in the archive), put the saved copy back instead, `sudo cp ~/labs/5.2/dummy-module.bak "$MOD"`, and plan a kernel update. Keeping a copy before you touch a file is what makes this possible.

11. **Verify.**

```
sudo modprobe dummy && lsmod | grep dummy
sudo modprobe -r dummy
rm ~/labs/5.2/dummy-module.bak
```

12. **Know the other kernel corruption fixes:**
    - Broken initramfs: `sudo update-initramfs -u -k $(uname -r)` (Rocky: `sudo dracut -f`).
    - Broken or unbootable new kernel: at the GRUB menu choose **Advanced options for Ubuntu**, boot the previous kernel, then `sudo apt install --reinstall linux-image-<version>` or remove the bad one. List installed kernels with `ls /boot/vmlinuz-*` or `dpkg -l 'linux-image-*'`.
    - `/boot` full during an update: clean old kernels with `sudo apt autoremove --purge`, then `sudo dpkg --configure -a`.
13. **Prevent.** Keep at least two kernels installed, monitor free space on `/boot`, never interrupt package updates, and verify packages with `dpkg -V` / `rpm -Va` after disk problems.

> **Checkpoint:** `journalctl --list-boots` shows the crashed boot, `dpkg -V "$PKG"` prints nothing, and `sudo modprobe dummy` works again.

Task 9: Scenario 8: The network card does not work (missing or disabled driver)

A **driver** is kernel code, usually a loadable **kernel module**, that controls a device. A device without a driver is visible on the bus (`lspci`, `lsusb`) but does not work. Drivers can be missing (not installed), disabled (blacklisted), or failing (firmware missing).

1. **Explore** how drivers attach to devices on your VM.

```
lspci -k | grep -A3 -i -E "ethernet|network"
lsmod | head -n 5
```

```
00:03.0 Ethernet controller: Red Hat, Inc. Virtio network device
        Subsystem: Red Hat, Inc. Virtio network device
        Kernel driver in use: virtio-pci
Module                  Size  Used by
...
```

What just happened: `lspci -k` shows each PCI device and the **Kernel driver in use** plus the **Kernel modules** that could drive it. A device with no `Kernel driver in use` line has no working driver: that is the first clue for a missing driver. `lsmod` lists loaded modules. Your device names depend on the hypervisor (virtio, e1000, vmxnet3, hv_netvsc).

2. **Break it.** Disable the `dummy` driver the way an admin (or a hardening script) might.

```
cat <<'EOF' | sudo tee /etc/modprobe.d/lab-disable-dummy.conf
# Lab 5.2: disable the dummy driver
blacklist dummy
install dummy /bin/false
EOF
```

3. **Symptom.** Try to create a dummy network interface (Lab 5.3 uses these).

```
sudo ip link add lab0 type dummy
sudo modprobe dummy
```

```
Error: Unknown device type.
modprobe: ERROR: ../libkmod/libkmod-module.c:1088 command_do() Error running install command
'/bin/false' for module dummy: retcode 1
modprobe: ERROR: could not insert 'dummy': Invalid argument
```

4. **Investigate.**

```
lsmod | grep -w dummy; echo "loaded? exit=$?"
modinfo dummy | head -n 4
grep -rn dummy /etc/modprobe.d/ /lib/modprobe.d/ 2>/dev/null
```

```
loaded? exit=1
filename:       /lib/modules/6.8.0-45-generic/kernel/drivers/net/dummy.ko.zst
alias:          rtnl-link-dummy
description:    Dummy netdevice driver
license:        GPL
/etc/modprobe.d/lab-disable-dummy.conf:1:# Lab 5.2: disable the dummy driver
/etc/modprobe.d/lab-disable-dummy.conf:2:blacklist dummy
/etc/modprobe.d/lab-disable-dummy.conf:3:install dummy /bin/false
```

> **What just happened:** The module is not loaded, but `modinfo` proves it is installed, so it is **disabled**, not missing. The `grep` finds the configuration. `blacklist dummy` only stops *automatic* loading by alias (for example when `ip link` asks for `rtnl-link-dummy`); an explicit `modprobe dummy` would still work. `install dummy /bin/false` replaces loading with running `/bin/false`, which always fails, so the driver cannot load at all. If `modinfo` itself said `Module dummy not found`, the driver would be **missing** instead.

5. **Root cause.** A modprobe configuration file disables the driver.
6. **Fix.** Remove the file. If the driver is needed early at boot (storage, some network cards), also rebuild the initramfs, because it contains a copy of the modprobe configuration.

```
sudo rm /etc/modprobe.d/lab-disable-dummy.conf
sudo update-initramfs -u
```

7. **Verify.**

```
sudo ip link add lab0 type dummy && ip -br link show lab0
sudo ip link del lab0
sudo modprobe -r dummy
```

8. **Missing drivers in real life.** If `modinfo` says the module is not found: on Ubuntu, many drivers live in `linux-modules-extra-$(uname -r)`; vendor drivers often come as DKMS packages that rebuild for each kernel (`dkms status`). If the driver loads but the device still fails, look for firmware errors: `sudo dmesg | grep -i firmware` (for example `failed to load ... firmware`), fixed by installing `linux-firmware`. After installing, load it with `sudo modprobe <name>`.
9. **Prevent.** Comment every blacklist with the reason, keep `linux-modules-extra` installed on hardware servers, and check `lspci -k` after kernel updates.

> **Checkpoint:** `ls /etc/modprobe.d/lab-disable-dummy.conf` reports no such file, and creating a dummy interface works again.

Task 10: Scenario 9: apt refuses to install anything (package dependency issues)

A **dependency** is another package a package needs to work. `apt` and `dnf` resolve dependencies automatically; the low-level tools `dpkg` and `rpm` do not. Installing a downloaded `.deb` with `dpkg -i` is the classic way to break this.

1. **Break it.** Build a small local package that depends on `sl` (a harmless joke program), make sure `sl` is not installed, then install the package with `dpkg`.

```
cd ~/labs/5.2
sudo apt purge -y sl 2>/dev/null
mkdir -p labtool/DEBIAN labtool/usr/local/bin
cat <<'EOF' > labtool/DEBIAN/control
Package: labtool
Version: 1.0
Architecture: all
Maintainer: Student <student@ubuntu-lab>
Depends: sl
Description: Lab 5.2 demo tool that needs the sl package
EOF
printf '#!/bin/sh\nexec /usr/games/sl "$@"\n' > labtool/usr/local/bin/labtool
chmod 755 labtool/usr/local/bin/labtool
dpkg-deb --build --root-owner-group labtool labtool_1.0_all.deb
sudo dpkg -i labtool_1.0_all.deb
```

```
dpkg-deb: building package 'labtool' in 'labtool_1.0_all.deb'.
Selecting previously unselected package labtool.
(Reading database ... 112233 files and directories currently installed.)
Preparing to unpack labtool_1.0_all.deb ...
Unpacking labtool (1.0) ...
dpkg: dependency problems prevent configuration of labtool:
 labtool depends on sl; however:
  Package sl is not installed.

dpkg: error processing package labtool (--install):
 dependency problems - leaving unconfigured
Errors were encountered while processing:
 labtool
```

What just happened: A `.deb` is an archive of files plus a `DEBIAN/control` file with metadata such as `Depends:`. `dpkg-deb --build` packs the directory; `--root-owner-group` makes the files owned by root inside the package. `dpkg -i` unpacked the files but refused to **configure** the package because `sl` is missing, and `dpkg` never downloads dependencies itself.

2. **Symptom.** Now try to install anything else.

```
sudo apt install -y tree
```

```
You might want to run 'apt --fix-broken install' to correct these.
The following packages have unmet dependencies:
 labtool : Depends: sl but it is not going to be installed
E: Unmet dependencies. Try 'apt --fix-broken install' with no packages (or specify a
solution).
```

3. **Investigate.**

```
dpkg -l labtool | tail -n 1
sudo dpkg --audit
dpkg -I labtool_1.0_all.deb | grep Depends
apt-cache policy sl
```

```
iU  labtool        1.0          all          Lab 5.2 demo tool that needs the sl package
The following packages have been unpacked but not yet configured.
...
 labtool              Lab 5.2 demo tool that needs the sl package
 Depends: sl
sl:
  Installed: (none)
  Candidate: 5.02-1
```

What just happened: In `dpkg -l`, the first letter is the desired state (`i` = install) and the second is the actual state (`U` = unpacked, not configured); a healthy package shows `ii`. `dpkg --audit` lists half-installed packages. `dpkg -I` shows a package file's control data, and `apt-cache policy sl` shows that `sl` is available from the repositories (`Candidate: 5.02-1`), so apt can fix this.

4. **Root cause.** A package installed with `dpkg -i` has an unmet dependency, which blocks apt.

5. **Fix.**

```
sudo apt --fix-broken install -y
```

```
Correcting dependencies... Done
The following NEW packages will be installed:
  sl
...
Setting up sl (5.02-1) ...
Setting up labtool (1.0) ...
```

What just happened: `apt --fix-broken install` (short form `apt -f install`) works out what is needed to satisfy all dependencies: here it installs `sl` and then configures `labtool`. Always read its plan before confirming: if a dependency does not exist in any repository, its "fix" is to **remove** the broken package instead. A related command, `sudo dpkg --configure -a`, finishes configuring packages left half-done by an interrupted install (power loss, closed SSH session).

6. **Verify.** `dpkg -l labtool | tail -n 1` starts with `ii`, and `sudo apt install -y tree` now works.

7. **Prevent.** Install local packages with apt, which resolves dependencies: `sudo apt install ./labtool_1.0_all.deb`. Do not mix repositories from different releases, and do not interrupt updates.

> **TIP**
> On Rocky the same situation looks like `rpm -ivh pkg.rpm` failing with `error: Failed dependencies: sl is needed by labtool-1.0`. Use `sudo dnf install ./pkg.rpm` to pull dependencies from repositories, `rpm -qpR pkg.rpm` to list what a package file requires, `dnf repoquery --requires <name>` for repository packages, and `sudo dnf check` to find broken dependencies on the system.

8. **Undo.**

```
sudo apt purge -y labtool sl
rm -rf ~/labs/5.2/labtool ~/labs/5.2/labtool_1.0_all.deb
```

Checkpoint: You saw `iU` change to `ii` after `apt --fix-broken install`, and you can explain why `dpkg -i` caused the problem.

Task 11: Scenario 10: A program crashes with "Segmentation fault"

A **segmentation fault** (segfault, signal `SIGSEGV`, number 11) happens when a program touches memory it is not allowed to: a NULL pointer, freed memory, or past the end of a buffer. The kernel kills the program and can save a **core dump**, a snapshot of its memory, for debugging.

1. **Prepare core dump collection.** `systemd-coredump` stores core dumps and adds the `coredumpctl` command.

```
sudo apt install -y systemd-coredump
cat /proc/sys/kernel/core_pattern
```

```
|/usr/lib/systemd/systemd-coredump %P %u %g %s %t 9223372036854775808 %h %d
```

What just happened: `core_pattern` tells the kernel what to do with a crashing program's memory. A value starting with `|` means "pipe it to this program". On Ubuntu, `systemd-coredump` conflicts with apport's crash hook package, so if apport is installed, apt removes `apport-core-dump-handler` (you see it under "REMOVED"); Clean up puts it back. If `core_pattern` still shows `/usr/share/apport/apport`, run `sudo sysctl -p /usr/lib/sysctl.d/50-coredump.conf` and check again.

2. **Break it.** Write and compile a tiny C program with a bug.

```
cd ~/labs/5.2
cat <<'EOF' > crashme.c
#include <stdio.h>
int main(void) {
    int *p = NULL;                  /* pointer to address 0 */
    printf("writing through a NULL pointer...\n");
    *p = 42;                        /* invalid memory write */
    return 0;
}
EOF
gcc -g -o crashme crashme.c
```

> **What just happened:** `gcc -g` compiles with debugging symbols, so the debugger can show source line numbers. Line 5 writes to address 0, which is never mapped in a Linux process.

3. **Symptom.**

```
ulimit -c unlimited
./crashme
echo "exit code: $?"
```

```
writing through a NULL pointer...
Segmentation fault (core dumped)
exit code: 139
```

> **What just happened:** `ulimit -c unlimited` removes this shell's core file size limit (the default is `0`). systemd-coredump on Ubuntu 24.04 keeps dumps even with limit `0` (note the fixed large number its `core_pattern` passes instead of the real limit), but when `core_pattern` is a plain file name such as `core`, a limit of `0` means no core file at all, so setting it is a good habit. The exit code `139` is 128 + 11: a process killed by a signal exits with 128 plus the signal number.

4. **Investigate.** First the kernel log, then the saved core dump.

```
sudo dmesg | grep crashme | tail -n 1
coredumpctl list crashme
coredumpctl info crashme | grep -E "Signal|Executable|#0"
```

```
crashme[5512]: segfault at 0 ip 00005616c4a2a16d sp 00007ffd2b1c0e40 error 6 in
crashme[5616c4a2a000+1000] likely on CPU 0
TIME                         PID  UID  GID SIG     COREFILE EXE
Sat 2026-10-10 14:10:22 UTC 5512 1000 1000 SIGSEGV present  /home/student/labs/5.2/crashme
      Signal: 11 (SEGV)
  Executable: /home/student/labs/5.2/crashme
               #0  0x00005616c4a2a16d main (crashme + 0x116d)
```

> **What just happened:** The kernel line says `segfault at 0`: the bad address was 0, a NULL pointer. `ip` is the instruction pointer and `error 6` means a user-mode *write* to a page that is not present. `coredumpctl list` shows the stored dump (`COREFILE present`), and `coredumpctl info` shows the signal and a stack trace pointing at `main`.

5. **Test the hypothesis** in the debugger to find the exact line.

```
coredumpctl debug crashme
```

At the `(gdb)` prompt type `bt` (backtrace), then `quit`:

```
Program terminated with signal SIGSEGV, Segmentation fault.
#0  main () at crashme.c:5
5           *p = 42;                        /* invalid memory write */
(gdb) bt
#0  main () at crashme.c:5
```

6. **Root cause.** A NULL pointer dereference on line 5 of the program.
7. **Fix** the code so the pointer points at real memory, then rebuild.

```
sed -i 's|int \*p = NULL;.*|int value = 0; int *p = \&value;|' crashme.c
gcc -g -o crashme crashme.c
```

8. **Verify.** `./crashme; echo "exit code: $?"` prints the message and `exit code: 0`.
9. **Prevent / real-world advice.** You usually cannot fix the code of a packaged program. If a packaged program segfaults: update it (`apt list --upgradable`), check its files with `dpkg -V package` (a corrupted library also causes segfaults), and send the core dump to the developer. If *many unrelated* programs segfault randomly, suspect hardware (RAM): check `journalctl -k` for machine check errors and run a memory test.

Checkpoint: `coredumpctl list crashme` shows a SIGSEGV entry, and the fixed program exits with code `0`.

Task 12: Scenario 11: Processes are "Killed" (OOM killer and memory leaks)

When Linux runs out of memory, the **OOM (out-of-memory) killer** picks a process (the one with the highest `oom_score`, usually the biggest) and kills it with `SIGKILL` to save the system. The victim prints just `Killed` and exits with code 137 (128 + 9). A **memory leak** is a program that keeps allocating memory and never frees it; given time, it triggers the OOM killer.

1. **Prepare** a small program that leaks memory on purpose.

```
cat <<'EOF' > ~/labs/5.2/leak.py
#!/usr/bin/env python3
# Leaks STEP MB every DELAY seconds and never frees it.
import sys, time
step = int(sys.argv[1]) if len(sys.argv) > 1 else 5
delay = float(sys.argv[2]) if len(sys.argv) > 2 else 1
cache = []
while True:
    cache.append(b"x" * (step * 1024 * 1024))   # kept forever: the leak
    print(f"holding {len(cache) * step} MB", flush=True)
    time.sleep(delay)
EOF
```

2. **Break it (killed process).** Run the leak quickly inside a **cgroup** limited to 200 MB, so only this program is affected, never your whole VM.

```
sudo systemd-run --scope -p MemoryMax=200M -p MemorySwapMax=0 \
    python3 ~/labs/5.2/leak.py 20 0.2
echo "exit code: $?"
```

```
Running as unit: run-r3c1e1a2b9f6d4c4f8d8b0b8f0f3e2a1d.scope; invocation ID: ...
holding 20 MB
holding 40 MB
...
holding 180 MB
Killed
exit code: 137
```

What just happened: `systemd-run --scope` runs a command in a new transient cgroup. `-p MemoryMax=200M` caps its memory, and `-p MemorySwapMax=0` forbids swapping so the limit is hit quickly. When the program passed 200 MB, the kernel's OOM killer killed it inside that cgroup. Without the limit, it would eat all RAM and swap first, slowing the whole VM down before something (maybe something important) is killed.

3. **Investigate the kill.** The kernel always logs OOM kills.

```
journalctl -k --since "10 minutes ago" | grep -i -E "out of memory|oom-kill|killed process"
```

```
kernel: python3 invoked oom-killer: gfp_mask=0xcc0(GFP_KERNEL), order=0, oom_score_adj=0
kernel: oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=
(null),cpuset=/,mems_allowed=0,oom_memcg=/system.slice/run-r3c1....scope,...
kernel: Memory cgroup out of memory: Killed process 6120 (python3) total-vm:221464kB, anon-
rss:203520kB, file-rss:2304kB, shmem-rss:0kB, UID:0 pgtables:480kB oom_score_adj:0
```

What just happened: `constraint=CONSTRAINT_MEMCG` says a cgroup limit was hit (a system-wide shortage would say `CONSTRAINT_NONE` and `Out of memory: Killed process ...`). `anon-rss` is the program's private memory at the moment it died. For a service, `systemctl status` would also show `Failed with result 'oom-kill'`. You can see any process's current score in `/proc/PID/oom_score` and bias it with `/proc/PID/oom_score_adj` (-1000 = never kill, 1000 = kill first).

4. **Break it (memory leak).** Now run the leak slowly in the background, as a real leaking service would, 2 MB per second.

```
python3 ~/labs/5.2/leak.py 2 1 > /dev/null &
LEAKPID=$!
```

5. **Symptom.** Nothing fails yet. Free memory just keeps shrinking. Watch the process's memory over time.

```
for i in 1 2 3 4; do ps -o pid,rss,vsz,etime,cmd -p "$LEAKPID" | tail -n 1; sleep 5; done
```

```
 6301 13420 25100       00:04 python3 /home/student/labs/5.2/leak.py 2 1
 6301 23664 35344       00:09 python3 /home/student/labs/5.2/leak.py 2 1
 6301 33908 45588       00:14 python3 /home/student/labs/5.2/leak.py 2 1
 6301 44152 55832       00:19 python3 /home/student/labs/5.2/leak.py 2 1
```

What just happened: `ps -o` picks columns: `rss` (resident set size, the physical RAM in KB actually used), `vsz` (virtual size, everything mapped), and `etime` (elapsed time). RSS climbs by about 10 MB every 5 seconds and never levels off. A healthy program grows at start-up and then stays roughly flat; steady growth over hours or days is the signature of a leak.

6. **Investigate** with more memory tools.

```
grep -E "VmRSS|VmHWM" /proc/$LEAKPID/status
pmap -x $LEAKPID | tail -n 1
smem -k -P leak.py
free -h
```

What just happened: `/proc/PID/status` shows `VmRSS` (current RAM) and `VmHWM` (high-water mark, the peak). `pmap -x` lists every memory mapping; the last line totals them, and a growing `[ anon ]` (heap) region points to a leak. `smem` adds **PSS** (proportional set size), which splits shared memory fairly between processes, a better "real cost" number. `free -h` shows the system view; watch the `available` column fall.

7. **Root cause.** The program appends data to a list it never clears: a memory leak.

8. **Fix.** Short term: restart the leaking process (memory is returned instantly). Long term: fix the code, and protect the system with a memory limit and automatic restart so a leak cannot take the server down.

```
kill $LEAKPID
free -h
```

What just happened: All of the process's memory is freed when it exits. For a real service, add `MemoryMax=500M` and `Restart=on-failure` with `sudo systemctl edit <service>`; the service is then killed and restarted cleanly at the limit instead of exhausting the whole machine.

9. **Verify.** `pgrep -f leak.py` prints nothing and `free -h` shows `available` back where it started.

10. **Prevent.** Monitor memory trends per process (not just totals), set cgroup limits on services, and report leaks to the developer with the data you collected.

Checkpoint: `journalctl -k | grep -i "killed process"` shows the OOM kill of `python3`, and you watched RSS grow steadily with `ps -o rss`.

Task 13: Scenario 12: “command not found” (PATH misconfiguration)

The **PATH** variable is a colon-separated list of directories the shell searches, in order, when you type a command name without a `/`. A broken PATH makes ordinary commands “disappear”; a PATH in the wrong order can run the wrong program.

1. **Break it.** Simulate a careless edit of `~/.bashrc` (for example, someone tried to add a directory and overwrote PATH instead of appending to it).

```
cp ~/.bashrc ~/labs/5.2/bashrc.bak
cat <<'EOF' >> ~/.bashrc
# lab 5.2 break: PATH overwritten instead of appended
export PATH=/usr/local/sbin:/usr/locall/bin:/home/student/bin
EOF
source ~/.bashrc
```

2. **Symptom.**

```
ls
```

```
Command 'ls' is available in the following places
 * /bin/ls
 * /usr/bin/ls
The command could not be located because '/bin:/usr/bin' is not included in the PATH
environment variable.
ls: command not found
```

3. **Investigate.** Shell built-ins like `echo` and `type` still work because they are part of bash itself.

```
echo "$PATH"
type ls
type -P ls || echo "ls is not in any PATH directory"
/usr/bin/grep -n PATH ~/.bashrc ~/.profile /etc/environment
```

```
/usr/local/sbin:/usr/locall/bin:/home/student/bin
ls is aliased to `ls --color=auto'
ls is not in any PATH directory
/home/student/.bashrc:118:# lab 5.2 break: PATH overwritten instead of appended
/home/student/.bashrc:119:export PATH=/usr/local/sbin:/usr/locall/bin:/home/student/bin
/home/student/.profile:19:# set PATH so it includes user's private bin if it exists
/home/student/.profile:21:    PATH="$HOME/bin:$PATH"
...
/etc/environment:1:PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/g
ames:/usr/local/games:/snap/bin"
```

> **What just happened:** `echo $PATH` shows that `/usr/bin` and `/bin` are missing and that `/usr/locall/bin` is a typo. On Ubuntu, `ls` is an alias for `ls --color=auto`, but the alias still needs the real `ls` program; `type -P` forces a search of PATH only, and finds nothing. Running a program by its **absolute path** (`/usr/bin/grep`) always works, because the shell does not search PATH when the name contains `/`. `grep -n` shows where PATH is set: the system default in `/etc/environment`, additions in `~/.profile`, and the bad line in `~/.bashrc`.

4. **Root cause.** `~/.bashrc` replaces PATH with a short, misspelled list instead of appending to it.

5. **Fix.** Repair the current shell first, then the file.

```
export
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
cp ~/labs/5.2/bashrc.bak ~/.bashrc
```

> **What just happened:** The first line restores a working PATH for this shell only. Restoring the backup fixes all future shells. The correct way to *add* a directory is to keep the old value: `export PATH="$PATH:/opt/tool/bin"`.

6. **Verify.** Start a fresh interactive login shell, which reads `~/.profile` and `~/.bashrc` just like an SSH login: `bash -lic 'echo $PATH; type -P ls'` shows the full PATH and `/usr/bin/ls`. (Without `-i`, bash skips most of `~/.bashrc`, because the file returns early for non-interactive shells, so the test would miss the broken line.)

7. **The other PATH problem: wrong order.** Directories earlier in PATH win. Create a fake `top` in `~/bin` and put `~/bin` first:

```
mkdir -p ~/bin
printf '#!/bin/sh\necho "this is NOT the real top"\n' > ~/bin/top
chmod 755 ~/bin/top
export PATH="$HOME/bin:$PATH"
hash -r
top
type -a top
```

```
this is NOT the real top
top is /home/student/bin/top
top is /usr/bin/top
```

> **What just happened:** `type -a` lists every match in PATH order, so you can see which one runs. `hash -r` clears bash's memory of where it found commands earlier. This is also an attack technique: a writable directory early in PATH lets someone plant a fake command. That is why `sudo` ignores your PATH and uses `secure_path` from `/etc/sudoers`, and why you should never put `.` (the current directory) in PATH.

8. **Undo.**

```
rm -f ~/bin/top ~/labs/5.2/bashrc.bak
rmdir ~/bin 2>/dev/null
export
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/game
s:/snap/bin
hash -r
```

9. **Prevent.** Always append (`PATH="$PATH:/new"`), test edits with `bash -lic 'echo $PATH'` before logging out, and keep a second SSH session open while editing login files.

> **Checkpoint:** `type -a top` shows only `/usr/bin/top` , and `tail -n 2 ~/.bashrc` no longer contains the `lab 5.2 break` line.

Task 14: Scenario 13: A service will not start (systemd unit failure)

1. **Break it.** Create a small app and a unit file with a typo in `ExecStart` .

```
cat <<'EOF' | sudo tee /usr/local/bin/labapp > /dev/null
#!/bin/bash
echo "labapp started with options: $*"
exec sleep infinity
EOF
sudo chmod 755 /usr/local/bin/labapp
cat <<'EOF' | sudo tee /etc/systemd/system/labapp.service
[Unit]
Description=Lab 5.2 demo application

[Service]
ExecStart=/usr/local/bin/labap --port 8080
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now labapp
```

2. **Symptom.** The command returns without error, but the app is not running.

```
systemctl status labapp --no-pager
systemctl --failed
```

```
x labapp.service - Lab 5.2 demo application
     Loaded: loaded (/etc/systemd/system/labapp.service; enabled; preset: enabled)
     Active: failed (Result: exit-code) since Sat 2026-10-10 14:40:02 UTC; 3s ago
    Process: 7012 ExecStart=/usr/local/bin/labap --port 8080 (code=exited, status=203/EXEC)
   Main PID: 7012 (code=exited, status=203/EXEC)
Oct 10 14:40:02 ubuntu-lab systemd[1]: labapp.service: Scheduled restart job, restart
counter is at 5.
Oct 10 14:40:02 ubuntu-lab systemd[1]: labapp.service: Start request repeated too quickly.
Oct 10 14:40:02 ubuntu-lab systemd[1]: labapp.service: Failed with result 'exit-code'.
  UNIT           LOAD   ACTIVE SUB    DESCRIPTION
x labapp.service loaded failed failed Lab 5.2 demo application
```

What just happened: `Active: failed` and `status=203/EXEC` mean systemd could not execute the program at all. Because of `Restart=on-failure`, systemd retried 5 times within 10 seconds, then gave up ("Start request repeated too quickly", the **start limit**). Other common codes: `200/CHDIR` (bad `WorkingDirectory`), `217/USER` (the `User=` does not exist), and plain `status=1/FAILURE` (the program started and then exited with an error, so read its own messages).

3. **Investigate.**

```
journalctl -xeu labapp --no-pager | grep -i -E "exec|locate" | head -n 3
systemctl cat labapp
systemd-analyze verify /etc/systemd/system/labapp.service
```

```
labapp.service: Unable to locate executable '/usr/local/bin/labap': No such file or
directory
labapp.service: Failed at step EXEC spawning /usr/local/bin/labap: No such file or directory
...
labapp.service: Command /usr/local/bin/labap is not executable: No such file or directory
```

What just happened: `journalctl -u labapp` shows the unit's logs; `-x` adds explanations and `-e` jumps to the end. The message names the missing file. `systemctl cat` prints the unit file systemd is actually using (including any drop-in overrides), and `systemd-analyze verify` checks a unit file for errors without starting it, which is worth doing before every `enable --now`.

4. **Root cause.** A typo in `ExecStart` (`labap` instead of `labapp`).

5. **Fix.** Correct the file, reload systemd's copy of it, clear the failed state, and start it.

```
sudo sed -i 's|/usr/local/bin/labap |/usr/local/bin/labapp |'
/etc/systemd/system/labapp.service
sudo systemctl daemon-reload
sudo systemctl reset-failed labapp
sudo systemctl restart labapp
```

> **What just happened:** systemd keeps unit files in memory; after editing one on disk you **must** run `daemon-reload` or systemd keeps using the old version (`systemctl status` warns "unit file changed on disk"). `reset-failed` clears the failed state and the start-limit counter so the unit can start again.

6. **Verify.**

```
systemctl is-active labapp
journalctl -u labapp -n 1 --no-pager
```

```
active
Oct 10 14:45:10 ubuntu-lab labapp[7101]: labapp started with options: --port 8080
```

7. **Prevent.** Run `systemd-analyze verify` after writing unit files, use `sudo systemctl edit name` (it creates drop-in overrides and reloads automatically), and check `systemctl --failed` after every change and every boot.

8. **Undo.**

```
sudo systemctl disable --now labapp
sudo rm -f /etc/systemd/system/labapp.service /usr/local/bin/labapp
sudo systemctl daemon-reload
```

> **Checkpoint:** You saw `status=203/EXEC`, found the typo with `journalctl -xeu`, and the service reached `active` after `daemon-reload`.

Task 15: Scenario 14: A program stops responding (unresponsive process)

A process can look "hung" for different reasons, and the fix depends on its **state**, shown in the `STAT` column of `ps`: `R` running, `S` sleeping (waiting for something, interruptible), `D` uninterruptible sleep (waiting on disk or network I/O, cannot be killed until the I/O finishes), `T` stopped, `Z` zombie (finished, waiting for its parent to collect it).

1. **Break it.** Start a "ticker" that writes the time every second, a reader that waits on a named pipe, and then stop the ticker.

```
cd ~/labs/5.2
bash -c 'while true; do date >> ~/labs/5.2/ticker.log; sleep 1; done' &
TPID=$!
mkfifo ~/labs/5.2/jobqueue
cat ~/labs/5.2/jobqueue &
RPID=$!
sleep 3; kill -STOP $TPID
```

> **What just happened:** The ticker appends a line every second. `mkfifo` creates a **named pipe**: a reader blocks until a writer opens the other end, like an application waiting for a job that never arrives. `kill -STOP` sends `SIGSTOP`, which freezes a process; it cannot be caught or ignored (pressing `Ctrl+Z` in a terminal sends the similar `SIGTSTP`).

2. **Symptom.** Neither process does anything: `ticker.log` stopped growing, and the reader never prints.

```
tail -n 1 ticker.log; sleep 3; tail -n 1 ticker.log
```

3. **Investigate the states.**

```
ps -o pid,stat,wchan:20,etime,cmd -p $TPID,$RPID
grep State /proc/$TPID/status
```

```
    PID STAT WCHAN                    ELAPSED CMD
   7310 T    do_signal_stop             00:15 bash -c while true; do date >> ...
   7313 S    wait_for_partner           00:12 cat /home/student/labs/5.2/jobqueue
State:  T (stopped)
```

> **What just happened:** `STAT T` means the ticker is stopped. `WCHAN` (wait channel) is the kernel function a sleeping process is waiting in: the reader is in a FIFO function (`wait_for_partner` here; some kernels show `fifo_open`), waiting for someone to open the pipe for writing. `/proc/PID/status` shows the same state in words.

4. **Test the hypothesis with strace.** `strace -p` attaches to a running process and shows the system call it is stuck in. Ubuntu restricts attaching to other processes, so use `sudo`. Press `Ctrl+C` to detach.

```
sudo strace -p $RPID
```

```
strace: Process 7313 attached
openat(AT_FDCWD, "/home/student/labs/5.2/jobqueue", O_RDONLY
```

> **What just happened:** The unfinished `openat(... O_RDONLY` line shows the process is waiting to open the pipe for reading. On real servers the same technique shows a process stuck in `read()` on a network socket, `futex()` (waiting on a lock), or `connect()` to a server that does not answer, which tells you where to look next.

5. **Root cause.** The ticker was stopped by a signal; the reader is waiting for input that never comes.

6. **Fix.**

```
kill -CONT $TPID
echo "job 1" > ~/labs/5.2/jobqueue
```

```
job 1
```

> **What just happened:** `SIGCONT` resumes a stopped process. Writing to the pipe gives the reader its input, so `cat` prints `job 1` and exits. If a process is truly stuck and must go, send `SIGTERM` first (`kill PID`, lets it clean up), and only if that fails `SIGKILL` (`kill -9 PID`). A process in `D` state ignores even `SIGKILL` until its I/O completes; fix the storage or NFS server it is waiting for (Lab 5.5 shows this).

7. **Verify.** `tail -n 1 ticker.log` shows the current time again, and `ps -p $RPID` shows the reader is gone.
8. **Undo.**

```
kill $TPID
rm -f ~/labs/5.2/ticker.log ~/labs/5.2/jobqueue
```

9. **Prevent.** Use timeouts in scripts and applications (`timeout 30 command`), monitor that work is actually progressing (a health check, Lab 5.1), and do not leave jobs suspended with `Ctrl+Z` .

Checkpoint: You saw `STAT T` and `STAT S` with a FIFO wait channel, used `strace -p` to see the blocked system call, and resumed the stopped process with `kill -CONT` .

Task 16: Scenario 15: Disk errors (device failure)

Real disks fail by returning read errors, dropping off the bus, or slowly accumulating bad sectors. You will simulate each safely.

Part A: a disk with bad sectors (device-mapper error target)

1. **Break it. Device-mapper** builds virtual block devices from a table. Make a 100 MB device where a small range of sectors always returns an I/O error.

```
cd ~/labs/5.2
truncate -s 100M dev.img
LOOP=$(sudo losetup --find --show dev.img); echo "$LOOP"
SECTORS=$(sudo blockdev --getsz "$LOOP"); echo "$SECTORS sectors"
cat <<EOF | sudo dmsetup create flaky
0 100000 linear $LOOP 0
100000 8 error
100008 $((SECTORS - 100008)) linear $LOOP 100008
EOF
```

What just happened: `losetup --find --show` attaches the file to a free loop device and prints its name. `blockdev --getsz` prints the size in 512-byte sectors (204800). This heredoc has **no quotes** around `EOF` , so the shell fills in `$LOOP` and the arithmetic. Each table line is *start length target*: sectors 0 to 99999 map straight to the loop device, sectors 100000 to 100007 use the `error` target (every access fails), and the rest map normally. The new device is `/dev/mapper/flaky` .

2. **Symptom.**

```
sudo dd if=/dev/mapper/flaky of=/dev/null bs=4k status=none; echo "exit: $?"
```

```
dd: error reading '/dev/mapper/flaky': Input/output error
exit: 1
```

3. **Investigate.**

```
sudo dmesg -T | grep -i -E "i/o error|dm-" | tail -n 3
sudo badblocks -sv /dev/mapper/flaky
```

```
[Sat Oct 10 15:02:11 2026] Buffer I/O error on dev dm-1, logical block 12500, async page
read
Checking blocks 0 to 102399
Checking for bad blocks (read-only test): 50000
50001
50002
50003
done
Pass completed, 4 bad blocks found. (4/0/0 errors)
```

What just happened: The kernel logged a buffer I/O error on `dm-1`, your flaky device (`dm-0` is usually Ubuntu's LVM root volume; `ls -l /dev/mapper` shows which `dm-N` each name points to). `badblocks -sv` reads every block (read-only test by default) with **s**how-progress and **v**erbose output. It uses 1 KB blocks, so 8 bad 512-byte sectors starting at sector 100000 are 1 KB blocks 50000 to 50003. On a real disk you would also see the device name (`sdb`) and SCSI or ATA error lines.

4. **Root cause and fix.** Bad sectors on the "disk". A disk that has started returning read errors is failing: copy the data off (`ddrescue` copies around bad areas) and replace it. Remove the simulation:

```
sudo dmsetup remove flaky
sudo losetup -d "$LOOP"
rm -f dev.img
```

Part B: a failed disk in a RAID 1 mirror (mdadm)

5. **Break it.** Build a RAID 1 array from two loop devices, then mark one as failed.

```
cd ~/labs/5.2
truncate -s 200M r1.img r2.img r3.img
L1=$(sudo losetup --find --show r1.img)
L2=$(sudo losetup --find --show r2.img)
L3=$(sudo losetup --find --show r3.img)
yes | sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 "$L1" "$L2"
sudo mdadm --wait /dev/md0
sudo mdadm --manage /dev/md0 --fail "$L2"
```

What just happened: `mdadm --create` builds `/dev/md0` as a mirror (`--level=1`) of two devices; `yes |` answers its "Continue creating array?" question. `--wait` waits until the initial sync finishes. `--fail` marks a member as failed, exactly what the kernel does on its own when a member disk returns errors.

6. **Symptom and investigation.**

```
cat /proc/mdstat
sudo mdadm --detail /dev/md0 | grep -E "State :|Failed Devices|faulty"
journalctl -k -n 5 --no-pager | grep md
```

```
md0 : active raid1 loop9[1](F) loop8[0]
      203776 blocks super 1.2 [2/1] [U_]
        State : clean, degraded
   Failed Devices : 1
       1       7        9        -      faulty   /dev/loop9
kernel: md/raid1:md0: Disk failure on loop9, disabling device.
                      md/raid1:md0: Operation continuing on 1 devices.
```

What just happened: In `/proc/mdstat`, `(F)` marks the failed member, `[2/1]` means 2 devices expected and 1 working, and `[U_]` shows one up (`U`) and one missing (`_`). `mdadm --detail` says `degraded`: the data is still available, but one more failure would lose it. The `mdadm` monitor service can email these events if `MAILADDR` is set in `/etc/mdadm/mdadm.conf`.

7. **Fix.** Remove the failed member, add the replacement, and watch it rebuild.

```
sudo mdadm --manage /dev/md0 --remove "$L2"
sudo mdadm --manage /dev/md0 --add "$L3"
sudo mdadm --wait /dev/md0; cat /proc/mdstat
```

```
md0 : active raid1 loop10[2] loop8[0]
      203776 blocks super 1.2 [2/2] [UU]
```

8. **Undo.**

```
sudo mdadm --stop /dev/md0
sudo mdadm --zero-superblock "$L1" "$L2" "$L3"
sudo losetup -d "$L1" "$L2" "$L3"
rm -f ~/labs/5.2/r1.img ~/labs/5.2/r2.img ~/labs/5.2/r3.img
```

Part C: a disk disappears, and SMART health (explore/simulate)

TIP

Explore/simulate: Removing a disk from the SCSI layer imitates a disk dropping off the bus. It only works if your extra disks are SCSI/SATA (`sdX`); skip it for `vdX` or NVMe disks. SMART data is usually not available on virtual disks, so you read the tool's output and a realistic sample instead.

9. Confirm the transport type and pick an unused extra disk (here `/dev/sde`; substitute yours). It must have no partitions, mount, LVM, or RAID use.

```
lsblk -S -o NAME,HCTL,TRAN,MODEL
lsblk /dev/sde
findmnt -S /dev/sde
```

> **CAUTION**
> Only ever do this with one of the blank **extra** disks, never `sda` (your OS disk) and never a disk that `lsblk` shows with partitions or children. Removing the OS disk crashes the VM. Have your snapshot ready; if the disk does not come back after the rescan, a reboot (or reverting the snapshot) restores it.

10. Make it vanish, look at the evidence, and bring it back with a rescan.

```
echo 1 | sudo tee /sys/block/sde/device/delete
lsblk | grep sde; sudo dmesg | tail -n 2
for h in /sys/class/scsi_host/host*/scan; do echo '- - -' | sudo tee "$h" > /dev/null; done
lsblk | grep sd
```

> **What just happened:** Writing `1` to `device/delete` removes the disk from the kernel, as if it failed or was unplugged; `lsblk` no longer shows it and `dmesg` logs `Synchronizing SCSI cache`. Writing `- - -` (all channels, targets, LUNs) to each SCSI host's `scan` file asks the controller to look for disks again, so it comes back (possibly with a different name). Admins use the rescan after hot-adding disks.

11. Check SMART. On most VMs it reports no SMART support, which is itself useful to recognize. (Some hypervisors emulate a minimal SMART that always says `PASSED`.)

```
sudo smartctl -i -H /dev/sda
```

```
SMART support is: Unavailable - device lacks SMART capability.
```

On a physical failing disk, `sudo smartctl -a /dev/sda` shows lines like these. The attributes to watch are reallocated, pending, and uncorrectable sectors; any non-zero and rising value means "replace soon".

```
SMART overall-health self-assessment test result: FAILED!
ID# ATTRIBUTE_NAME          FLAG     VALUE WORST THRESH TYPE      RAW_VALUE
  5 Reallocated_Sector_Ct   0x0033   010   010   036    Pre-fail  1984
197 Current_Pending_Sector  0x0012   100   100   000    Old_age   312
198 Offline_Uncorrectable   0x0010   100   100   000    Old_age   87
```

12. **Prevent.** Use RAID or replication for important data, run the `smartd` service to alert on SMART changes, watch `dmesg` and `/proc/mdstat`, and keep spare disks.

> **Checkpoint:** `badblocks` found 4 bad blocks on the flaky device, `/proc/mdstat` showed `[U_]` and then `[UU]` after the rebuild, and `cat /proc/mdstat` now lists no `md0`.

Task 17: Scenario 16: The server is inaccessible or will not turn on

Part A: "I can't SSH to the server" (server inaccessible)

1. **Break it.** From the **ubuntu-lab console** (not SSH), stop the SSH listener. Ubuntu 24.04 starts `sshd` on demand through `ssh.socket`, so stop both units.

```
sudo systemctl stop ssh.socket ssh.service
```

2. **Symptom.** From rocky-lab:

```
# on rocky-lab
ssh student@ubuntu-lab
```

```
ssh: connect to host ubuntu-lab port 22: Connection refused
```

3. **Investigate from the client side first**, working up the layers.

```
# on rocky-lab
ping -c 2 ubuntu-lab
nc -zv -w 3 ubuntu-lab 22
```

```
64 bytes from ubuntu-lab (192.168.100.10): icmp_seq=1 ttl=64 time=0.41 ms
Ncat: Connection refused.
```

> **What just happened:** `ping` works, so the host is powered on and the network path is fine. `nc -zv` tests one TCP port (install with `sudo dnf install -y nmap-ncat` if needed). The difference between messages matters: **Connection refused** means the host answered but nothing is listening on that port (service down). **Connection timed out** means no answer at all (a firewall dropping packets, wrong IP, or the host is down). **No route to host** usually means a firewall reject or the host is unreachable on the local network.

4. **Investigate on the server** through the console (the out-of-band path you always need: hypervisor console, cloud serial console, or a physical server's BMC remote console).

```
ss -tlnp | grep ':22 '
systemctl status ssh.socket ssh.service --no-pager | grep -E "Loaded|Active"
```

```
     Loaded: loaded (/usr/lib/systemd/system/ssh.socket; enabled; preset: enabled)
     Active: inactive (dead) since Sat 2026-10-10 15:30:12 UTC; 1min ago
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: inactive (dead) since Sat 2026-10-10 15:30:12 UTC; 1min ago
```

5. **Root cause and fix.** Nothing is listening on port 22. Start it again and verify from rocky-lab.

```
sudo systemctl start ssh.socket
ss -tlnp | grep ':22 '
```

Then on rocky-lab, `ssh student@ubuntu-lab` works again.

6. **Other causes of an inaccessible server**, and the command that checks each: network interface down or wrong IP (`ip -br addr`), firewall (`sudo ufw status`, `sudo nft list ruleset`, Rocky `sudo firewall-cmd --list-all`), broken `sshd_config` (`sudo sshd -t` prints the syntax error), root filesystem full (`df -h /`; logins can fail), account locked or expired (Lab 5.4), and a system so overloaded it cannot respond (Lab 5.5).

Part B: the server will not turn on (explore/simulate)

> TIP
> **Explore/simulate:** A VM cannot lose power or fail POST, so you walk the checklist, run the tools that exist, and read their output or error.

7. Learn the checklist, from the outside in:
   - **Power:** cables, PDU and outlet, power supply LEDs, redundant PSUs.
   - **POST** (power-on self-test): beep codes or front-panel/diagnostic LEDs point to RAM, CPU, or board faults; reseat RAM if it fails.
   - **BMC/IPMI** (iDRAC, iLO, XClarity): the baseboard management controller has its own network port and works even when the server is off. `ipmitool chassis status` shows power state; `ipmitool sel elist` lists the hardware event log; `ipmitool chassis power on` powers it on remotely.
   - **Firmware/boot:** firmware screen reached but no OS means the boot order, disk, or bootloader (Tasks 6 and 7).
   - **VMs:** check the hypervisor (`virsh list --all`, `virsh start name`, `virsh console name` on KVM), host resources, and the VM's disk file.
8. Try the BMC tool on your VM and read the error.

```
sudo apt install -y ipmitool
sudo ipmitool chassis status
```

```
Could not open device at /dev/ipmi0 or /dev/ipmi/0 or /dev/ipmidev/0: No such file or
directory
```

> **What just happened:** There is no BMC in a VM, so there is no `/dev/ipmi0`. On a real server, the same command (or `ipmitool -I lanplus -H bmc-ip -U admin chassis status` over the network) would print `System Power : off` and the last power event. A typical event log entry for a dead power supply looks like `Power Supply #0x51 | Failure detected | Asserted`.

9. After a server comes back, find out how it went down: check the previous boot's last messages and the shutdown records.

```
journalctl -b -1 -n 10 --no-pager
last -x shutdown reboot | head -n 5
sudo dmidecode -s system-manufacturer; sudo dmidecode -s system-serial-number
```

> **What just happened:** A clean shutdown ends with systemd stopping services; a log that just stops suggests power loss, a hang, or a panic. `last -x` lists shutdown and reboot records. `dmidecode -s` reads the manufacturer and serial number from firmware tables, which you need when you open a hardware support case.

Checkpoint: You can explain the difference between “Connection refused” and “Connection timed out”, `ssh student@ubuntu-lab` works again from rocky-lab, and you can list the power, POST, BMC, and hypervisor checks for a server that will not turn on.

Challenge

CHALLENGE

1. Combine scenarios: on a loop filesystem with user quotas, fill your quota with a file that a background process keeps open, then delete it. Explain why `quota -u student` still shows the space used and fix it.
2. Write a service unit that runs `leak.py 5 1` with `MemoryMax=150M` and `Restart=on-failure`. Watch it get OOM-killed and restarted with `journalctl -fu`, and find `oom-kill` in `systemctl status`.
3. Write a one-page incident report from your `incident-log.txt` for one scenario, covering all seven method steps.

Clean up

Most scenarios have their own Undo step. These commands catch anything left behind; errors such as “not mounted” or “not found” are fine.

1. Unmount and detach any lab loop devices and images.

```
for m in /mnt/small /mnt/inodes /mnt/rw /mnt/rw2 /mnt/quota /mnt/data; do
    sudo umount "$m" 2>/dev/null
done
sudo mdadm --stop /dev/md0 2>/dev/null
sudo dmsetup remove flaky 2>/dev/null
losetup -a | grep labs/5.2 | cut -d: -f1 | xargs -r sudo losetup -d
sudo rmdir /mnt/small /mnt/inodes /mnt/rw /mnt/rw2 /mnt/quota /mnt/data 2>/dev/null
```

2. Restore GRUB to its original settings.

```
sudo cp /etc/default/grub.lab-orig /etc/default/grub
sudo rm /etc/default/grub.lab-orig
sudo update-grub
```

3. Make sure `fstab` is the original, then lock the root password again.

```
sudo cp /etc/fstab.lab-orig /etc/fstab && sudo rm /etc/fstab.lab-orig
sudo systemctl daemon-reload
sudo passwd -l root
```

4. Remove lab configuration, units, and packages built in the lab, and kill leftover processes.

```
sudo rm -f /etc/modprobe.d/lab-disable-dummy.conf
sudo systemctl disable --now labapp 2>/dev/null
sudo rm -f /etc/systemd/system/labapp.service /usr/local/bin/labapp
sudo systemctl daemon-reload
sudo apt purge -y labtool sl 2>/dev/null
pkill -f leak.py; pkill -f 'sleep infinity'
sudo systemctl start ssh.socket
```

5. Give the crash hook back to apport (if apport is installed, reinstalling `apport-core-dump-handler` also removes `systemd-coredump`, because the two conflict), remove `systemd-coredump` and the stored dumps, then the lab directory.

   ```
   dpkg -s apport > /dev/null 2>&1 && sudo apt install -y apport-core-dump-handler
   sudo apt purge -y systemd-coredump
   sudo rm -rf /var/lib/systemd/coredump
   cat /proc/sys/kernel/core_pattern
   rm -rf ~/labs/5.2
   ```

   With apport back, `core_pattern` starts with `|/usr/share/apport/apport`. (If apt says `systemd-coredump` is not installed, that is fine: the first command already removed it.)

6. Confirm the system is clean: `systemctl --failed` lists no units, `findmnt | grep /mnt/` prints nothing, and `grep -c sdz9 /boot/grub/grub.cfg` prints `0`. If anything is odd, revert to your snapshot.

Exam corner

- Method: identify symptoms, gather data, hypothesize, test, fix, verify, document. Ask "what changed?"
- "No space left on device" with free space in `df -h` means **inode exhaustion** (`df -i`). `df` larger than `du` means a **deleted-but-open file** (`lsof +L1`); restart the process or truncate instead of `rm`.
- Error messages: "Permission denied" = permissions/ACL; "Operation not permitted" = immutable attribute (`lsattr`, `chattr -i`); "Read-only file system" = `ro` mount or kernel remount after errors (`dmesg`); "Disk quota exceeded" = `quota`, `repquota`, `setquota`, `edquota`.
- "Bad superblock": `e2fsck -b 32768 /dev/sdX1` (find backups with `mke2fs -n` or `dumpe2fs`); never `fsck` a mounted filesystem; XFS uses `xfs_repair`. `fsck` cannot detect corrupted file contents.
- Bad `/etc/fstab` = emergency mode. Fix: `mount -o remount,rw /`, edit fstab, `systemctl daemon-reload`, `mount -a`. Prevent with `findmnt --verify`, `mount -a`, and `nofail`.
- GRUB: edit `/etc/default/grub`, then `update-grub` (Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (Rocky). Press `e` at the menu for one-time edits; `Ctrl+X` boots. `(initramfs)` prompt = root device not found; `grub rescue>` = GRUB cannot find its files.
- Kernel panic: read the console; `kernel.panic` sets auto-reboot seconds; kdump saves crash dumps to `/var/crash`; `journalctl -b -1` shows the previous boot. Kernel corruption: boot an older kernel from **Advanced options**, `dpkg -V` / `rpm -V`, reinstall the kernel package, `update-initramfs -u` / `dracut -f`.
- Drivers: `lspci -k`, `lsmod`, `modinfo`, `modprobe`; disabled via `blacklist` or `install <mod> /bin/false` in `/etc/modprobe.d/`; missing ones in `linux-modules-extra`, DKMS, or `linux-firmware`.

- Dependencies: `dpkg -i` does not resolve them (`iU` state); fix with `apt --fix-broken install` / `apt -f install`, `dpkg --configure -a`; Rocky: `dnf install ./file.rpm`, `rpm -qpR`.
- Segfault = signal 11, exit code 139; `dmesg` shows `segfault at`; `coredumpctl list/info/debug`. OOM kill = `SIGKILL` (9), exit code 137, `journalctl -k | grep -i "killed process"`; limit with `MemoryMax=`.
- Memory leak = RSS growing steadily (`ps -o rss`, `pmap -x`, `smem`, `/proc/PID/status`).
- Unit failure: `systemctl status`, `journalctl -xeu unit`, `status=203/EXEC` = bad `ExecStart` path; always `daemon-reload` after edits; `reset-failed` after start-limit hits.
- Process states: `R`, `S`, `D` (uninterruptible, unkillable), `T` (stopped, `kill -CONT`), `Z` (zombie). `strace -p PID` shows where it waits.

Review questions

1. Users get "No space left on device" when creating files, but `df -h` shows the filesystem 30% used. Which command most likely confirms the cause?
   A. `du -sh /*` B. `df -i` C. `lsattr` D. `quota -u`

2. After deleting a 10 GB log file, `df` still shows the space as used. What is the most likely reason, and which command proves it?

3. A server boots into emergency mode right after an administrator added a new disk to `/etc/fstab`. What two commands should the administrator have run before rebooting to catch the error?

4. A program exits with code 139 and the kernel log shows `segfault at 0`. What happened?
   A. It was killed by the OOM killer B. It dereferenced a NULL pointer C. It hit a disk quota D. Its unit file had a typo

5. `systemctl status app` shows `status=203/EXEC`. What should you check first?

6. A process shows state `D` in `ps` and does not die with `kill -9`. Why, and where should you look?

Lab 5.3 Troubleshoot Networking Issues

Objective: 5.3 Given a scenario, analyze and troubleshoot networking issues on a Linux system.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 3 hours
VM: ubuntu-lab (most scenarios) and rocky-lab (firewall and dual stack scenarios)
Snapshot first: Recommended. Most scenarios run inside isolated network namespaces and cannot cut off your SSH session, but the last two change real interfaces and firewall rules.

Network problems are the most common "it's broken" tickets, and they are easy to misdiagnose if you jump straight to the application. In this lab you build a private practice network inside ubuntu-lab using network namespaces, then break it in all the ways the exam lists: links that go down, MTU mismatches, wrong subnet masks, bad gateways, broken routing, DNS failures, IP conflicts, DHCP failures, bond failover, and MAC spoofing protection. You finish with a firewall problem and a dual stack (IPv4 and IPv6) problem between your two real VMs.

What you'll practice

- A bottom-up (OSI layer by layer) troubleshooting flow, starting with "cannot ping server"
- Link down and link negotiation issues (`ip link`, `ethtool`, NO-CARRIER)
- Interface misconfiguration: MTU mismatch, bonding, MAC spoofing, subnet, cannot ping server
- Routing issues: wrong or missing gateway, missing return routes, IP forwarding (`ip route`, `tracepath`, `traceroute`, `tcpdump`)
- Server unreachable: telling "refused", "timed out", and "no route to host" apart
- DNS issues (`resolvectl`, `dig`, `getent`, `/etc/hosts`, `/etc/resolv.conf`)
- IP conflicts (`arping -D`, `ip neigh`, `ip monitor neigh`)
- DHCP issues (DHCP server logs, client leases, APIPA 169.254.x.x addresses)
- Misconfigured firewalls (`firewall-cmd`, logging denied packets)
- Dual stack issues (IPv4 and IPv6): link-local addresses, address selection, `curl -4` / `-6`, `/etc/gai.conf`

How it works

Troubleshoot from the bottom up. Networking is built in layers. Each layer depends on the one below it, so if you check them in order you never waste time on DNS when the cable is unplugged.

Layer	Question	Commands
1 Physical / link	Is the link up? Speed and duplex right?	`ip -br link`, `ethtool IF`, `networkctl`
2 Data link	Right MAC? Can I resolve neighbors (ARP)? Same MTU? Bond healthy?	`ip neigh`, `arping`, `ip link` (mtu), `/proc/net/bonding/*`
3 Network	Right IP and mask? Route and gateway? Can packets get there and back?	`ip -br addr`, `ip route`, `ip route get`, `ping`, `tracepath`
4 Transport	Is the port open and listening? Is a firewall in the way?	`ss -tulnp`, `nc -zv`, `firewall-cmd`, `nft list ruleset`
7 Application	Does the name resolve? Does the service answer?	`dig`, `getent hosts`, `resolvectl`, `curl -v`

The "cannot ping server" ladder. When a host cannot reach a server, ping in this order and stop at the first failure. That failure tells you which layer is broken:

1. `ping 127.0.0.1` (the TCP/IP stack works)
2. `ping <my own IP>` (the interface has the address)
3. `ping <default gateway>` (the local network, cable, and ARP work)
4. `ping <remote server IP>` (routing works in both directions)
5. `ping <remote server name>` (DNS works)

Remember that some hosts and firewalls block ping on purpose, so a failed ping is a clue, not proof. Test the actual service port too (`nc -zv host port`).

Error messages are clues:

- `Network is unreachable`: your own host has no route to that network (often no default gateway).
- `Destination Host Unreachable`: ARP for the next hop failed; nobody answered on the local network.
- `Connection refused`: the target host answered but nothing listens on that port.
- `Connection timed out`: no answer at all (firewall silently dropping, host down, routing black hole).
- `No route to host` (for TCP): often a firewall *rejecting* the connection with an ICMP message.
- `Temporary failure in name resolution`: DNS failed.

Your practice network. A **network namespace** is a separate copy of the Linux network stack (its own interfaces, addresses, routes, and firewall), like a tiny virtual computer that shares the kernel. A **veth pair** is a virtual cable: two interfaces where whatever goes in one end comes out the other. A **bridge** is a virtual switch. You will build this inside ubuntu-lab:

```
 client ns                router ns                          server ns
+------------+        +--------------------------+        +-------------+
| veth-c     |--------| veth-rc                  |        |             |
| 10.10.1.10 |  LAN 1 |   \_ br0 (switch)        |  LAN 2 |  veth-s     |
+------------+        |      10.10.1.1/24        |--------|  10.10.2.20 |
                      |   veth-rs 10.10.2.1/24   |        +-------------+
(later scenarios plug |   ip_forward=1 (router)  |
 more hosts into br0) +--------------------------+
```

Your real `enp0s3` (or similar) interface is never touched by these scenarios, so you cannot lose your SSH session. Every command that changes a namespace needs root: `sudo ip -n client ...` runs an `ip` command inside the `client` namespace, and `sudo ip netns exec client <command>` runs any command there.

Tasks

Task 1: Take a baseline and build the practice network

1. Install the tools for this lab.

```
mkdir -p ~/labs/5.3 && cd ~/labs/5.3
sudo apt update
sudo apt install -y iproute2 iputils-ping iputils-arping iputils-tracepath traceroute \
    tcpdump bind9-dnsutils ipcalc ethtool netcat-openbsd dnsmasq-base busybox nftables
```

> **What just happened:** Besides the usual tools, `iputils-arping` sends ARP probes, `bind9-dnsutils` provides `dig`, `ipcalc` does subnet math, `dnsmasq-base` is a small DHCP and DNS server (only the program, so it does not start a service that would fight with systemd-resolved), `busybox` includes a tiny DHCP client (`udhcpc`), and `nftables` provides the `nft` firewall command (normally already installed).

2. Take a baseline of your real network. A **baseline** is a record of the healthy state, so later you can compare.

```
ip -br link
ip -br addr
ip route
resolvectl status | head -n 20
networkctl list
```

```
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp0s3           UP             08:00:27:3a:5c:11 <BROADCAST,MULTICAST,UP,LOWER_UP>
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.10/24 fe80::a00:27ff:fe3a:5c11/64
default via 192.168.100.1 dev enp0s3 proto static
192.168.100.0/24 dev enp0s3 proto kernel scope link src 192.168.100.10
...
IDX LINK   TYPE     OPERATIONAL SETUP
  1 lo     loopback carrier     unmanaged
  2 enp0s3 ether    routable    configured
```

What just happened: `-br` means brief output. `LOWER_UP` means the link has carrier (a working "cable"). Each interface has an IPv4 address and an IPv6 **link-local** address starting with `fe80::`. The default route (`default via ...`) is the gateway. `resolvectl status` shows which DNS servers systemd-resolved uses, and `networkctl list` shows that systemd-networkd manages `enp0s3` and that it is `routable`. Your interface name may be different (`ens18` , `eth0`); use yours throughout.

3. Write the script that builds the practice network. Open `~/labs/5.3/labnet-up.sh` with `nano` , or use the heredoc below (a heredoc sends every line up to `EOF` into the file; quoting `'EOF'` stops the shell from changing `$` signs).

```
cat <<'EOF' > ~/labs/5.3/labnet-up.sh
#!/bin/bash
# labnet-up.sh: build an isolated practice network with namespaces
set -e
for ns in client router server; do ip netns add "$ns"; done
# cables (veth pairs): client<->router and router<->server
ip link add veth-c type veth peer name veth-rc
ip link add veth-s type veth peer name veth-rs
ip link set veth-c netns client
ip link set veth-rc netns router
ip link set veth-rs netns router
ip link set veth-s netns server
# a switch (bridge) for LAN 1 inside the router namespace
ip -n router link add br0 type bridge
ip -n router link set br0 address 02:00:00:00:01:01
ip -n router link set veth-rc master br0
# addresses
ip -n client addr add 10.10.1.10/24 dev veth-c
ip -n router addr add 10.10.1.1/24 dev br0
ip -n router addr add 10.10.2.1/24 dev veth-rs
ip -n server addr add 10.10.2.20/24 dev veth-s
# bring everything up
for ns in client router server; do ip -n "$ns" link set lo up; done
ip -n client link set veth-c up
ip -n router link set br0 up
ip -n router link set veth-rc up
ip -n router link set veth-rs up
ip -n server link set veth-s up
# routing
ip -n client route add default via 10.10.1.1
ip -n server route add default via 10.10.2.1
ip netns exec router sysctl -qw net.ipv4.ip_forward=1
echo "lab network is up"
EOF
```

What just happened: `ip netns add` creates the three namespaces. `ip link add ... type veth peer name ...` creates a cable with two ends, and `ip link set ... netns` moves each end into a namespace. The bridge `br0` acts as a switch for LAN 1 so you can plug more hosts in later. It gets a fixed MAC address because a Linux bridge otherwise copies the lowest MAC of its ports, so its address (the client's gateway MAC) would change every time you plug in or remove a host, leaving the client with a stale ARP entry. `net.ipv4.ip_forward=1` lets the router namespace pass packets between its interfaces; it is set only inside that namespace. `set -e` stops the script at the first error.

4. Write the matching tear-down script, then build the network and test it.

```
cat <<'EOF' > ~/labs/5.3/labnet-down.sh
#!/bin/bash
# labnet-down.sh: remove every practice namespace (veth pairs vanish with them)
pkill -f '^dnsmasq --no-daemon' 2>/dev/null
for ns in client router server intruder laptop bondhost; do
    ip netns del "$ns" 2>/dev/null
done
rm -rf /etc/netns/client /tmp/labnet-leases
echo "lab network removed"
EOF
chmod 755 ~/labs/5.3/labnet-up.sh ~/labs/5.3/labnet-down.sh
sudo ~/labs/5.3/labnet-up.sh
sudo ip netns list
sudo ip netns exec client ping -c 2 10.10.2.20
sudo ip netns exec client tracepath -n 10.10.2.20
```

```
lab network is up
server (id: 2)
router (id: 1)
client (id: 0)
PING 10.10.2.20 (10.10.2.20) 56(84) bytes of data.
64 bytes from 10.10.2.20: icmp_seq=1 ttl=63 time=0.432 ms
64 bytes from 10.10.2.20: icmp_seq=2 ttl=63 time=0.038 ms
 1?: [LOCALHOST]                      pmtu 1500
 1:  10.10.1.1                                             0.104ms
 2:  10.10.2.20                                            0.015ms reached
     Resume: pmtu 1500 hops 2 back 2
```

What just happened: The client reached the server through the router. `ip netns list` shows each namespace (the `id` numbers are internal and may be missing). `ttl=63` (one less than the default 64) proves the packet crossed one router. `tracepath` lists each hop and the path MTU (`pmtu 1500`). If anything goes badly wrong later, run `sudo ~/labs/5.3/labnet-down.sh` and then `labnet-up.sh` to start fresh.

TIP

This lab uses `sudo` constantly. If a command that ends in `&` (background) seems to hang, sudo is probably waiting for your password in the background. Run `sudo -v` first to refresh your sudo credentials, then repeat the command.

Checkpoint: `sudo ip netns exec client ping -c 2 10.10.2.20` succeeds with `ttl=63`, and you saved your real baseline (`ip -br addr`, `ip route`) somewhere you can see it.

Task 2: Scenario 1: "The network is dead" (link down and link negotiation)

1. **Break it.** Unplug the client's "cable" at the switch end by shutting down the router-side port.

```
sudo ip -n router link set veth-rc down
```

2. **Symptom.** The client cannot reach even its gateway.

```
sudo ip netns exec client ping -c 2 -W 1 10.10.1.1
```

```
2 packets transmitted, 0 received, 100% packet loss, time 1012ms
```

3. **Investigate layer 1 on the client.**

```
sudo ip -n client -br link show veth-c
sudo ip netns exec client ethtool veth-c | grep -E "Speed|Duplex|Link detected"
sudo ip -n client route
```

```
veth-c@if5       DOWN           0a:00:0c:29:b1:ae <NO-CARRIER,BROADCAST,MULTICAST,UP>
        Speed: 10000Mb/s
        Duplex: Full
        Link detected: no
default via 10.10.1.1 dev veth-c linkdown
10.10.1.0/24 dev veth-c proto kernel scope link src 10.10.1.10 linkdown
```

What just happened: The flags contain `UP` (the admin turned the interface on) but also `NO-CARRIER`, and the state is `DOWN`. That combination means "configured on, but no signal from the other end": an unplugged cable, a dead switch port, or a shut-down port on the other side. `ethtool` agrees (`Link detected: no`), and the kernel marks routes through that interface `linkdown`. If the flags had **no** `UP`, the interface itself would be administratively down on this host (`ip link set dev X up` fixes that).

4. **Root cause and fix.** The other end of the link is down. Bring it up (on real hardware: reseat or replace the cable, enable the switch port, check the switch for errors).

```
sudo ip -n router link set veth-rc up
sudo ip netns exec client ping -c 2 10.10.1.1
```

5. **Real systems.** On ubuntu-lab, link changes are logged by systemd-networkd; on Rocky, by NetworkManager. Look at what a healthy link logged at boot:

```
journalctl -u systemd-networkd -b | grep -i -E "carrier|link" | tail -n 5
```

```
systemd-networkd[612]: enp0s3: Link UP
systemd-networkd[612]: enp0s3: Gained carrier
```

A flapping link shows repeated `Lost carrier` / `Gained carrier` pairs, which points to a bad cable, transceiver, or switch port. `ip monitor link` shows the changes live.

6. **Link negotiation (explore/simulate).** Ethernet devices *auto-negotiate* speed and duplex. If one side is forced (for example `100 Mb/s full duplex`) and the other auto-negotiates, the auto side often falls back to **half duplex**. The link is "up" but slow, with collisions and CRC errors. This is a **duplex mismatch**.

> TIP
> **Explore/simulate:** Virtual NICs do not negotiate, so you inspect the real NIC, try the change on a veth to see the error a virtual device gives, and read a realistic sample from physical hardware.

```
sudo ethtool $(ip route show default | awk '{print $5; exit}')
sudo ip netns exec client ethtool -s veth-c speed 100 duplex half autoneg off
```

```
Settings for enp0s3:
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Advertised auto-negotiation: Yes
        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: on
        Link detected: yes
netlink error: Operation not supported
```

What just happened: `$(ip route show default | awk '{print $5; exit}')` finds your main interface name (the fifth word of the default route). `ethtool IF` shows supported modes, speed, duplex, and whether auto-negotiation is on. An emulated e1000 NIC shows values like those above; a virtio NIC may show `Speed: Unknown!`. The veth refuses `ethtool -s` because it has no physical layer. On a physical server with a duplex mismatch you would see something like this:

```
Speed: 100Mb/s
Duplex: Half
Auto-negotiation: on
```

plus rising error counters in `ip -s link show eth0` (`errors`, `collsns`) and `ethtool -S eth0` (`rx_crc_errors`, `tx_late_collisions`). The fix is to set **both** ends the same way, normally auto-negotiation on both: `sudo ethtool -s eth0 autoneg on`. To make it permanent use a systemd `.link` file (`AutoNegotiation=`, `BitsPerSecond=`, `Duplex=`) on Ubuntu or `nmcli connection modify eth0 802-3-ethernet.auto-negotiate yes` on Rocky.

Checkpoint: You can explain the difference between `NO-CARRIER` with `UP` (other end or cable) and missing `UP` (admin down here), and you know the symptoms of a duplex mismatch (link up, slow, collisions, CRC errors).

Task 3: Scenario 2: "Small requests work, big transfers hang" (MTU mismatch)

The **MTU** (maximum transmission unit) is the largest packet an interface sends, normally 1500 bytes on Ethernet. Both ends of a link must agree. If one side sends 1500-byte packets and the other side (or a device in between) only handles 1400, small packets pass but big ones are dropped.

1. **Break it.** Lower the MTU on the router side of LAN 1, and turn off *segmentation offload* on the client so the virtual cable behaves like a physical one (offload lets virtual devices hand around oversized "super packets", which would hide the problem).

```
sudo ip -n router link set veth-rc mtu 1400
sudo ip netns exec client ethtool -K veth-c tso off gso off
```

2. **Symptom.** Ping works and a TCP connection opens, but a 2 MB upload never finishes. Start a listener on the server, then send a file from the client.

```
sudo -v
sudo ip netns exec server timeout 15 nc -l 8000 > /dev/null &
head -c 2000000 /dev/zero > ~/labs/5.3/big.bin
sudo ip netns exec client ping -c 1 10.10.2.20 | sed -n 2p
sudo ip netns exec client timeout 5 nc -N 10.10.2.20 8000 < ~/labs/5.3/big.bin
echo "exit code: $?"
```

```
64 bytes from 10.10.2.20: icmp_seq=1 ttl=63 time=0.061 ms
exit code: 124
```

> **What just happened:** `nc -l 8000` listens on TCP port 8000 and `timeout 15` ends it after 15 seconds. On the client, `nc -N` connects, sends the file, and closes the connection at the end. Exit code `124` means `timeout` had to kill it after 5 seconds: the transfer hung. Ping used small 84-byte packets and worked. "Connects, then hangs on big data" (an SSH session that freezes on `ls -l`, a web page that loads halfway) is the classic MTU symptom.

3. **Investigate.** Compare the MTU on both ends, then probe with pings of exact sizes. `-M do` sets the "don't fragment" flag, and `-s` sets the ICMP payload size. Payload + 8 bytes ICMP header + 20 bytes IP header = packet size, so `-s 1472` makes a 1500-byte packet.

```
sudo ip -n client link show veth-c | grep -o 'mtu [0-9]*'
sudo ip -n router link show veth-rc | grep -o 'mtu [0-9]*'
sudo ip netns exec client ping -c 1 -W 1 -M do -s 1472 10.10.1.1
sudo ip netns exec client ping -c 1 -W 1 -M do -s 1372 10.10.1.1
sudo ip -s -n client link show veth-c | grep -A1 TX
```

```
mtu 1500
mtu 1400
PING 10.10.1.1 (10.10.1.1) 1472(1500) bytes of data.
ping: sendmsg: No buffer space available
PING 10.10.1.1 (10.10.1.1) 1372(1400) bytes of data.
1380 bytes from 10.10.1.1: icmp_seq=1 ttl=64 time=0.043 ms
    TX:  bytes packets errors dropped carrier collsns
         10177      78      0       5       0       0
```

What just happened: The two ends disagree (1500 versus 1400). A 1400-byte packet passes, a 1500-byte packet does not, and the `dropped` counter on the client rises. On a veth pair the kernel refuses the oversized packet locally ("No buffer space available"); on a real network it usually just disappears (100% loss with no error). Either way, the largest size that works tells you the real path MTU. `tracepath` also reports the path MTU when routers send back "fragmentation needed" messages.

4. **Root cause.** MTU mismatch on LAN 1: the client sends 1500-byte frames the other side cannot accept.
5. **Fix.** Make both sides match, then restore offload.

```
sudo ip -n router link set veth-rc mtu 1500
sudo ip netns exec client ethtool -K veth-c tso on gso on
```

6. **Verify.** Wait until the first listener has exited (about 15 seconds after you started it), then repeat the `nc -l` and `nc -N` commands from step 2. The transfer now finishes in well under a second with `exit code: 0`.
7. **Prevent.** Set MTU deliberately and the same everywhere on a segment: netplan `mtu: 1500` (or `9000` for jumbo frames on a storage network, only if *every* switch and host supports it), `nmcli connection modify eth0 802-3-ethernet.mtu 9000` on Rocky. Remember that VPNs and tunnels add headers and need a smaller MTU, and never block all ICMP on firewalls: path MTU discovery depends on "fragmentation needed" messages.

Checkpoint: You found the working packet size with `ping -M do -s`, and the 2 MB transfer completes after the fix.

Task 4: Scenario 3: "I can ping the gateway but not the server" (subnet mask)

A **subnet mask** (or prefix length, like `/24`) tells a host which addresses are on its own local network (reach directly with ARP) and which are remote (send to the gateway). A wrong mask makes the host make the wrong choice.

1. **Break it.** Give the client a `/16` mask instead of `/24`.

```
sudo ip -n client addr flush dev veth-c
sudo ip -n client addr add 10.10.1.10/16 dev veth-c
sudo ip -n client route add default via 10.10.1.1
```

What just happened: `addr flush` removes all addresses from the interface (and with them the routes that used it). You add the address back with the wrong prefix and restore the default route.

2. **Symptom.**

```
sudo ip netns exec client ping -c 1 10.10.1.1 | sed -n 2p
sudo ip netns exec client ping -c 4 10.10.2.20
```

```
64 bytes from 10.10.1.1: icmp_seq=1 ttl=64 time=0.062 ms
PING 10.10.2.20 (10.10.2.20) 56(84) bytes of data.
From 10.10.1.10 icmp_seq=1 Destination Host Unreachable
From 10.10.1.10 icmp_seq=2 Destination Host Unreachable
From 10.10.1.10 icmp_seq=3 Destination Host Unreachable
```

3. **Investigate.**

```
sudo ip -n client -br addr show veth-c
sudo ip -n client route get 10.10.2.20
sudo ip -n client neigh
ipcalc 10.10.1.10/16 | grep -E "Network|HostMin|HostMax"
```

```
veth-c@if5       UP             10.10.1.10/16
10.10.2.20 dev veth-c src 10.10.1.10 uid 0
    cache
10.10.2.20 dev veth-c FAILED
10.10.1.1 dev veth-c lladdr 02:00:00:00:01:01 REACHABLE
Network:   10.10.0.0/16         00001010.00001010. 00000000.00000000
HostMin:   10.10.0.1            00001010.00001010. 00000000.00000001
HostMax:   10.10.255.254        00001010.00001010. 11111111.11111110
```

> **What just happened:** `ip route get` asks the kernel which route it would use for an address. The answer has **no** `via 10.10.1.1`: the client thinks `10.10.2.20` is on its own LAN, because `/16` covers `10.10.0.0` to `10.10.255.255` (as `ipcalc` shows). So it sends ARP requests for `10.10.2.20` directly; nobody on LAN 1 has that address, the neighbor entry stays `INCOMPLETE` while it is trying and becomes `FAILED` when it gives up, and ping reports `Destination Host Unreachable` from the client's own address.

4. **The opposite mistake.** A mask that is too *small* can put the gateway outside the subnet:

```
sudo ip -n client addr flush dev veth-c
sudo ip -n client addr add 10.10.1.10/29 dev veth-c
sudo ip -n client route add default via 10.10.1.1
```

```
Error: Nexthop has invalid gateway.
```

> **What just happened:** `/29` covers only `10.10.1.8` to `10.10.1.15`, so the gateway `10.10.1.1` is not on the local network and the kernel refuses the route. In netplan or NetworkManager the same mistake produces a host with no default route ("Network is unreachable" for everything remote).

5. **Root cause and fix.** Wrong prefix length. Put back the correct `/24`.

```
sudo ip -n client addr flush dev veth-c
sudo ip -n client addr add 10.10.1.10/24 dev veth-c
sudo ip -n client route add default via 10.10.1.1
```

6. **Verify.** `sudo ip netns exec client ping -c 2 10.10.2.20` succeeds and `sudo ip -n client route get 10.10.2.20` shows `via 10.10.1.1`.
7. **Prevent.** Copy the prefix from the network documentation, check new configs with `ip route get <remote>` (it must say `via <gateway>`), and on Ubuntu run `sudo netplan try`, which rolls back automatically after 120 seconds if you lose access.

Checkpoint: You can explain why `/16` caused `Destination Host Unreachable` and why `/29` caused `Nexthop has invalid gateway`.

Task 5: Scenario 4: "Local network works, nothing remote does" (gateway issues)

1. **Break it.** Point the client's default route at an address where no router exists.

```
sudo ip -n client route replace default via 10.10.1.254
```

2. **Symptom.**

```
sudo ip netns exec client ping -c 4 10.10.2.20
```

```
From 10.10.1.10 icmp_seq=1 Destination Host Unreachable
From 10.10.1.10 icmp_seq=2 Destination Host Unreachable
...
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3059ms
```

3. **Investigate.**

```
sudo ip -n client route show default
sudo ip -n client neigh
sudo ip netns exec client ping -c 1 -W 1 10.10.1.254
```

```
default via 10.10.1.254 dev veth-c
10.10.1.254 dev veth-c FAILED
10.10.1.1 dev veth-c lladdr 02:00:00:00:01:01 STALE
1 packets transmitted, 0 received, 100% packet loss, time 0ms
```

What just happened: The default route uses `10.10.1.254`, but ARP for that address never completes (`INCOMPLETE` while trying, then `FAILED`) and it does not answer ping. The real router (`10.10.1.1`) is reachable, which proves the link and subnet are fine. Hypothesis: wrong gateway address.

4. **Break it a second way: no default route at all.**

```
sudo ip -n client route del default
sudo ip netns exec client ping -c 1 10.10.2.20
```

```
ping: connect: Network is unreachable
```

What just happened: With no route for `10.10.2.20`, the kernel fails immediately with `Network is unreachable` without sending anything. Compare: a *wrong* gateway gives `Destination Host Unreachable` after a few seconds of ARP attempts; a *missing* gateway gives `Network is unreachable` instantly.

5. **Fix.**

```
sudo ip -n client route replace default via 10.10.1.1
sudo ip netns exec client ping -c 2 10.10.2.20
```

What just happened: `ip route replace` adds the route, or overwrites it if one exists, so it is safe either way. Changes made with `ip` are temporary. On a real Ubuntu server you fix the gateway permanently in netplan (`routes: - to: default via: 192.168.100.1`, then `sudo netplan apply`); on Rocky with `sudo nmcli connection modify eth0 ipv4.gateway 192.168.100.1` and `sudo nmcli connection up eth0`.

6. **Check your real gateway the same way.**

```
ip route show default
ping -c 2 $(ip route show default | awk '{print $3; exit}')
```

Checkpoint: The client can ping `10.10.2.20` again, and you can tell a wrong gateway (`Destination Host Unreachable`) from a missing one (`Network is unreachable`).

Task 6: Scenario 5: "The server is unreachable" (routing beyond the gateway)

When the local side is fine, the problem is further along the path, and you must remember that packets need a route **there and back**.

1. **Break it (part 1).** Turn off forwarding on the router.

```
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=0
```

2. **Symptom and investigation.** Walk the path hop by hop.

```
sudo ip netns exec client ping -c 1 -W 1 10.10.1.1 | sed -n 2p
sudo ip netns exec client ping -c 1 -W 1 10.10.2.1 | sed -n 2p
sudo ip netns exec client ping -c 2 -W 1 10.10.2.20 | tail -n 2
sudo ip netns exec client traceroute -n -w 1 -q 1 -m 4 10.10.2.20
```

```
64 bytes from 10.10.1.1: icmp_seq=1 ttl=64 time=0.104 ms
64 bytes from 10.10.2.1: icmp_seq=1 ttl=64 time=0.261 ms
2 packets transmitted, 0 received, 100% packet loss, time 1014ms
traceroute to 10.10.2.20 (10.10.2.20), 4 hops max, 60 byte packets
 1  *
 2  *
 3  *
 4  *
```

What just happened: The client can ping both router addresses (a host answers for all of its own addresses, even when not forwarding), but nothing beyond. `traceroute` sends probes with increasing TTL (time to live) and lists who answers at each hop; `-n` skips name lookups, `-w 1` waits one second, `-q 1` sends one probe per hop, and `-m 4` stops after 4 hops. Even hop 1 shows `*` because the router drops the packets instead of forwarding them. Check the router:

```
sudo ip netns exec router sysctl net.ipv4.ip_forward
```

```
net.ipv4.ip_forward = 0
```

3. **Fix part 1.** Turn forwarding back on. (On a real Linux router, make it permanent with a file such as `/etc/sysctl.d/90-router.conf` containing `net.ipv4.ip_forward = 1`, then `sudo sysctl --system`.)

```
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1
```

4. **Break it (part 2).** Remove the server's default route, so it does not know how to send replies back to LAN 1.

```
sudo ip -n server route del default
```

5. **Symptom and investigation.** The client still gets no replies. Watch the server's interface with `tcpdump` while the client pings.

```
sudo -v
sudo ip netns exec server timeout 4 tcpdump -lni veth-s icmp &
sleep 1; sudo ip netns exec client ping -c 2 -W 1 10.10.2.20 > /dev/null; wait
sudo ip netns exec server ip route get 10.10.1.10
```

```
10:13:39.494603 IP 10.10.1.10 > 10.10.2.20: ICMP echo request, id 11946, seq 1, length 64
10:13:40.515955 IP 10.10.1.10 > 10.10.2.20: ICMP echo request, id 11946, seq 2, length 64
RTNETLINK answers: Network is unreachable
```

What just happened: `tcpdump -lni veth-s icmp` captures ICMP packets on the server's interface (`-n` no name lookups, `-l` line-buffered output, `-i` interface). Requests **arrive**, but no replies are sent. `ip route get 10.10.1.10` on the server explains why: it has no route back. This one-way pattern ("requests in, no replies out") is the signature of a missing return route or asymmetric routing.

6. **Fix part 2.**

```
sudo ip -n server route add default via 10.10.2.1
sudo ip netns exec client tracepath -n 10.10.2.20
```

7. **Verify.** `tracepath` reaches `10.10.2.20` in 2 hops.

8. **Prevent.** Document routes on every host and router, persist them in netplan/NetworkManager (not just `ip route`), and use `tcpdump` on both ends to prove where packets stop instead of guessing.

Checkpoint: You saw requests arrive at the server without replies, and fixed both forwarding and the return route. `sudo ip netns exec client ping -c 1 10.10.2.20` succeeds.

Task 7: Scenario 6: "The website is down" but it is really DNS

DNS turns names into IP addresses. On Ubuntu, programs ask `/etc/resolv.conf`, which points to the local systemd-resolved stub at `127.0.0.53`; systemd-resolved forwards to the real DNS servers. Before DNS, the system checks `/etc/hosts`, because `/etc/nsswitch.conf` says `hosts: files dns`.

1. **Investigate the real VM first** (read-only).

```
ls -l /etc/resolv.conf
grep nameserver /etc/resolv.conf
resolvectl dns
resolvectl query ubuntu.com
grep ^hosts: /etc/nsswitch.conf
```

```
lrwxrwxrwx 1 root root 39 Apr 23 10:02 /etc/resolv.conf -> ../run/systemd/resolve/stub-
resolv.conf
nameserver 127.0.0.53
Global:
Link 2 (enp0s3): 192.168.100.1
ubuntu.com: 185.125.190.20                    -- link: enp0s3
hosts:          files dns
```

What just happened: `/etc/resolv.conf` is a symlink to the stub file, so it always says `127.0.0.53`; the *real* upstream servers are shown by `resolvectl dns` (here `192.168.100.1` on `enp0s3`, learned from DHCP or netplan). `resolvectl query` resolves through systemd-resolved and shows which link answered. On Rocky, `/etc/resolv.conf` lists the real servers directly (written by NetworkManager).

2. **Break it.** In the practice network, run a DNS server for the `lab` domain in the server namespace and point the client at it. `ip netns exec` uses files in `/etc/netns/<name>/` instead of the same files in `/etc`, so the client gets its own `resolv.conf`. Then misconfigure the client's DNS server address.

```
sudo -v
sudo ip netns exec server dnsmasq --no-daemon --no-resolv --no-hosts \
    --listen-address=10.10.2.20 --bind-interfaces \
    --address=/web.lab/10.10.2.20 --log-queries > ~/labs/5.3/dns.log 2>&1 &
sudo mkdir -p /etc/netns/client
echo "nameserver 10.10.2.20" | sudo tee /etc/netns/client/resolv.conf
sudo ip netns exec client getent hosts web.lab
echo "nameserver 10.10.2.99" | sudo tee /etc/netns/client/resolv.conf
```

```
nameserver 10.10.2.20
10.10.2.20      web.lab
nameserver 10.10.2.99
```

What just happened: `dnsmasq` answers DNS queries on `10.10.2.20`; `--address=/web.lab/10.10.2.20` creates a record, `--no-resolv` and `--no-hosts` stop it from reading the host's own files, and `--log-queries` logs every question. The first `getent` proves it works. Then you changed the client to a DNS server address that does not exist, a typical typo.

3. **Symptom.**

```
sudo ip netns exec client ping -c 1 web.lab
sudo ip netns exec client ping -c 1 10.10.2.20 | sed -n 2p
```

```
ping: web.lab: Temporary failure in name resolution
64 bytes from 10.10.2.20: icmp_seq=1 ttl=63 time=0.051 ms
```

4. **Investigate.** The name fails but the IP works: that alone points to DNS. Now test the configured server and a known-good server directly.

```
sudo ip netns exec client cat /etc/resolv.conf
sudo ip netns exec client dig web.lab | grep -E 'timed out|reached'
sudo ip netns exec client dig @10.10.2.20 +short web.lab
```

```
nameserver 10.10.2.99
;; communications error to 10.10.2.99#53: timed out
;; communications error to 10.10.2.99#53: timed out
;; communications error to 10.10.2.99#53: timed out
;; no servers could be reached
10.10.2.20
```

What just happened: `dig` sends a DNS query and shows the full answer. With no `@server`, it uses `/etc/resolv.conf`, tries three times, and gives up (after about 15 seconds; be patient). The `grep` keeps just the error lines. `dig @10.10.2.20` asks a specific server, and it answers, so the DNS service is fine and the client's configuration is wrong. On the real Ubuntu VM you would run `dig @192.168.100.1 ubuntu.com` (or another known server) the same way.

5. **Fix.**

```
echo "nameserver 10.10.2.20" | sudo tee /etc/netns/client/resolv.conf
sudo ip netns exec client ping -c 1 web.lab | head -n 1
```

6. **Break it a second way: a stale `/etc/hosts` entry.** Give the client its own hosts file with an old address for `web.lab`.

```
{ cat /etc/hosts; echo "10.10.2.99 web.lab"; } | sudo tee /etc/netns/client/hosts >
/dev/null
sudo ip netns exec client getent hosts web.lab
sudo ip netns exec client dig +short web.lab
sudo ip netns exec client grep -n web.lab /etc/hosts
```

```
10.10.2.99      web.lab
10.10.2.20
12:10.10.2.99 web.lab
```

> **What just happened:** `getent hosts` resolves the way applications do (files first, then DNS) and returns the wrong `10.10.2.99`. `dig` only talks DNS and returns the right `10.10.2.20`. `grep -n` shows the stale line at the end of the file (your line number will differ). **When `getent` and `dig` disagree, look in `/etc/hosts`.** This is a very common cause of "it works for me but not on that server".

7. **Fix and verify.**

```
sudo rm /etc/netns/client/hosts
sudo ip netns exec client getent hosts web.lab
tail -n 2 ~/labs/5.3/dns.log
sudo pkill -f '^dnsmasq --no-daemon'
```

> TIP
> The `^` in the `pkill -f` pattern matters. `-f` matches against the whole command line, and the `sudo pkill -f ...` process has the pattern in its own command line, so without `^` (start of line) pkill can kill its own `sudo` parent. With `^`, only a command line that *starts* with `dnsmasq --no-daemon` matches.

8. **Prevent.** Configure DNS servers in netplan (`nameservers: addresses: [...]`) or NetworkManager rather than editing `/etc/resolv.conf` by hand (it is regenerated), keep `/etc/hosts` small and documented, and after DNS changes flush caches with `resolvectl flush-caches`.

> **Checkpoint:** You can name the tool that bypasses `/etc/hosts` (`dig`) and the one that uses it (`getent hosts`), and `web.lab` resolves to `10.10.2.20` in the client namespace.

Task 8: Scenario 7: "The connection keeps dropping" (IP conflict)

Two hosts with the same IP address on one network fight over it. Other devices keep updating their ARP table to whichever host spoke last, so traffic goes to one, then the other: intermittent failures that are maddening to debug.

> **CAUTION**
> Never create an IP conflict on a real network: you can knock someone else's server offline. Here it happens only inside your private namespaces.

1. **Break it.** Plug a new host, `intruder`, into the LAN 1 switch and give it the client's address.

```
sudo ip netns add intruder
sudo ip link add veth-i type veth peer name veth-ri
sudo ip link set veth-i netns intruder
sudo ip link set veth-ri netns router
sudo ip -n router link set veth-ri master br0
sudo ip -n router link set veth-ri up
sudo ip -n intruder link set veth-i up
sudo ip -n intruder addr add 10.10.1.10/24 dev veth-i
sudo ip netns exec intruder ping -c 1 -W 1 10.10.1.1 > /dev/null
```

> **What just happened:** You built another cable, attached one end to the bridge (`master br0`), and configured `10.10.1.10` on the new host. Its ping to the router makes the router learn the intruder's MAC address for `10.10.1.10`.

2. **Symptom.** The client, which did nothing wrong, loses connectivity. (If the first try works, wait a few seconds and try again: failing *sometimes* is exactly how IP conflicts behave.)

```
sudo ip netns exec client ping -c 3 -W 1 10.10.2.20 | tail -n 2
```

```
3 packets transmitted, 0 received, 100% packet loss, time 2025ms
```

3. **Investigate.** Look at the router's ARP (neighbor) table and compare MAC addresses.

```
sudo ip -n router neigh show 10.10.1.10
sudo ip -n client -br link show veth-c
sudo ip -n intruder -br link show veth-i
```

```
10.10.1.10 dev br0 lladdr fa:cb:e3:f0:00:d2 REACHABLE
veth-c@if5       UP             0a:00:0c:29:b1:ae <BROADCAST,MULTICAST,UP,LOWER_UP>
veth-i@if13      UP             fa:cb:e3:f0:00:d2 <BROADCAST,MULTICAST,UP,LOWER_UP>
```

> **What just happened:** The router maps `10.10.1.10` to `fa:cb:...`, which is the intruder's MAC, not the client's. So the router sends the client's replies to the intruder. On a real network you would see the MAC change when you check repeatedly.

4. **Test the hypothesis** with duplicate address detection: `arping -D` asks "is anyone using this address?" without claiming it.

```
sudo ip netns exec client arping -D -I veth-c -c 2 10.10.1.10
echo "exit code: $?"
```

```
ARPING 10.10.1.10 from 0.0.0.0 veth-c
Unicast reply from 10.10.1.10 [FA:CB:E3:F0:00:D2]  0.594ms
Sent 1 probes (1 broadcast(s))
Received 1 response(s)
exit code: 1
```

What just happened: `-D` is duplicate address detection mode, `-I` the interface, `-c 2` the probe count. Another MAC answered for "our" address, so there is a conflict (exit code `1`; `0` would mean the address is free). Running this *before* assigning a static address prevents conflicts.

5. **Watch the flapping live.** Each host announcing itself (a *gratuitous ARP*) steals the address back.

```
sudo -v
sudo ip netns exec router ip monitor neigh &
sleep 1
sudo ip netns exec intruder arping -U -I veth-i -c 1 10.10.1.10 > /dev/null
sudo ip netns exec client arping -U -I veth-c -c 1 10.10.1.10 > /dev/null
sleep 1; sudo pkill -f '^ip monitor neigh'
```

```
10.10.1.10 dev br0 lladdr fa:cb:e3:f0:00:d2 STALE
10.10.1.10 dev br0 lladdr 0a:00:0c:29:b1:ae STALE
```

6. **Root cause and fix.** Two hosts share one IP. Remove the duplicate (on a real network: find the device by its MAC in the switch's MAC table, then readdress it).

```
sudo ip netns del intruder
sudo ip netns exec client arping -U -I veth-c -c 1 10.10.1.10 > /dev/null
sudo ip netns exec client ping -c 2 10.10.2.20 | tail -n 2
```

What just happened: Deleting the namespace deletes its veth pair too. The client's gratuitous ARP (`-U`, unsolicited) updates the router immediately instead of waiting for the old entry to expire.

7. **Prevent.** Use DHCP with reservations instead of hand-picked static addresses, keep an IP address management (IPAM) list, exclude static ranges from DHCP pools, and probe with `arping -D` before assigning an address. NetworkManager can check automatically (`ipv4.dad-timeout`).

Checkpoint: `arping -D` found the duplicate before the fix, and the client pings the server reliably after it.

Task 9: Scenario 8: "The new machine gets no IP address" (DHCP issues)

DHCP hands out addresses in four messages: the client broadcasts **Discover**, servers send an **Offer**, the client sends a **Request**, and the server confirms with an **ACK** (remember DORA). If no server answers, many clients give themselves an **APIPA** (automatic private IP addressing) link-local address in `169.254.0.0/16`, which works only with other machines on the same link. Seeing `169.254.x.x` almost always means "DHCP failed".

1. **Look at the real DHCP client first.** On ubuntu-lab, systemd-networkd is the DHCP client (if your VM uses DHCP).

```
networkctl status $(ip route show default | awk '{print $5; exit}') | grep -i -E
"address|gateway|dhcp"
journalctl -u systemd-networkd -b | grep -i dhcp | tail -n 3
```

```
         Address: 192.168.100.10 (DHCPv4 via 192.168.100.1)
         Gateway: 192.168.100.1
systemd-networkd[612]: enp0s3: DHCPv4 address 192.168.100.10/24, gateway 192.168.100.1
acquired from 192.168.100.1
```

> **What just happened:** `networkctl status IF` shows where each address came from (`DHCPv4 via` the DHCP server). The journal shows each lease. If your VM uses a static address, these lines are absent; that is fine. On Rocky, use `nmcli device show eth0` and `journalctl -u NetworkManager | grep -i dhcp`.

2. **Break it.** Plug a new `laptop` namespace into LAN 1 and run a DHCP server on the router with a pool from the **wrong subnet** (`10.10.9.x` instead of `10.10.1.x`), a common copy-paste mistake.

```
sudo ip netns add laptop
sudo ip link add veth-l type veth peer name veth-rl
sudo ip link set veth-l netns laptop
sudo ip link set veth-rl netns router
sudo ip -n router link set veth-rl master br0
sudo ip -n router link set veth-rl up
sudo ip -n laptop link set lo up
sudo ip -n laptop link set veth-l up
sudo -v
sudo ip netns exec router dnsmasq --no-daemon --port=0 --interface=br0 --bind-interfaces \
    --dhcp-range=10.10.9.100,10.10.9.150,255.255.255.0,1h \
    --dhcp-option=option:router,10.10.1.1 --dhcp-leasefile=/tmp/labnet-leases \
    > ~/labs/5.3/dhcp.log 2>&1 &
```

> **What just happened:** `--port=0` turns off dnsmasq's DNS function so it is only a DHCP server. `--interface=br0 --bind-interfaces` makes it listen only on LAN 1. `--dhcp-range` is the pool to hand out (start, end, netmask, lease time), `--dhcp-option=option:router` is the gateway given to clients, and `--dhcp-leasefile` is where it records leases.

3. **Write a tiny DHCP client hook.** BusyBox's `udhcpc` gets the lease, then runs a script to apply it, passing the details in variables.

```
cat <<'EOF' > ~/labs/5.3/dhcp-apply.sh
#!/bin/sh
# Called by udhcpc. $1 is the event; lease details arrive in variables.
[ "$1" = "bound" ] || exit 0
ip addr add "$ip/$mask" dev "$interface"
[ -n "$router" ] && ip route add default via "$router"
echo "configured $interface with $ip/$mask, gateway $router"
EOF
chmod 755 ~/labs/5.3/dhcp-apply.sh
```

4. **Symptom.**

```
sudo ip netns exec laptop busybox udhcpc -i veth-l -n -q -f -s ~/labs/5.3/dhcp-apply.sh
sudo ip -n laptop -br addr show veth-l
```

```
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: no lease, failing
veth-l@if19      UP
```

What just happened: `-i` is the interface, `-n` exits if no lease is obtained, `-q` exits after getting one, `-f` stays in the foreground, and `-s` names the hook script. The client broadcast Discover three times and got no Offer, so the interface has no address.

5. **Investigate the server side.**

```
cat ~/labs/5.3/dhcp.log
```

```
dnsmasq: started, version 2.90 DNS disabled
dnsmasq-dhcp: DHCP, IP range 10.10.9.100 -- 10.10.9.150, lease time 1h
dnsmasq-dhcp: DHCP, sockets bound exclusively to interface br0
dnsmasq-dhcp: no address range available for DHCP request via br0
dnsmasq-dhcp: no address range available for DHCP request via br0
```

What just happened: The server *did* hear the requests (so the network path and the client are fine), but its pool `10.10.9.x` does not belong to the subnet of the interface the request arrived on (`br0` is `10.10.1.1/24`). A DHCP server only hands out addresses that match the receiving network (or the relay's network). Other common DHCP failures: the server is down, the pool is exhausted (log says `no address available`), a firewall blocks UDP ports 67/68, or a different VLAN has no DHCP relay (`ip helper-address` on the router).

6. **Fix.** Restart the server with the correct pool.

```
sudo pkill -f '^dnsmasq --no-daemon'
sudo -v
sudo ip netns exec router dnsmasq --no-daemon --port=0 --interface=br0 --bind-interfaces \
    --dhcp-range=10.10.1.100,10.10.1.150,255.255.255.0,1h \
    --dhcp-option=option:router,10.10.1.1 --dhcp-leasefile=/tmp/labnet-leases \
    > ~/labs/5.3/dhcp.log 2>&1 &
```

7. **Verify.**

```
sudo ip netns exec laptop busybox udhcpc -i veth-l -n -q -f -s ~/labs/5.3/dhcp-apply.sh
sudo ip netns exec laptop ping -c 1 10.10.2.20 | sed -n 2p
grep -E "DHCP(DISCOVER|OFFER|REQUEST|ACK)" ~/labs/5.3/dhcp.log
cat /tmp/labnet-leases
```

```
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.10.1.138, server 10.10.1.1
udhcpc: lease of 10.10.1.138 obtained from 10.10.1.1, lease time 3600
configured veth-l with 10.10.1.138/24, gateway 10.10.1.1
64 bytes from 10.10.2.20: icmp_seq=1 ttl=63 time=0.304 ms
dnsmasq-dhcp: DHCPDISCOVER(br0) 42:a4:2b:22:cb:48
dnsmasq-dhcp: DHCPOFFER(br0) 10.10.1.138 42:a4:2b:22:cb:48
dnsmasq-dhcp: DHCPREQUEST(br0) 10.10.1.138 42:a4:2b:22:cb:48
dnsmasq-dhcp: DHCPACK(br0) 10.10.1.138 42:a4:2b:22:cb:48
1791630729 42:a4:2b:22:cb:48 10.10.1.138 * 01:42:a4:2b:22:cb:48
```

What just happened: You can see all four DORA steps in the log. The lease file records expiry time (in seconds since 1970), MAC, IP, and client ID. dnsmasq waits a few seconds before its first offer while it pings the address to make sure nobody already uses it, which is why the client may broadcast Discover twice.

8. **Prevent.** Monitor DHCP pool usage, keep static addresses outside the pool, and document relay agents. Stop the DHCP server now:

```
sudo pkill -f '^dnsmasq --no-daemon'
sudo ip netns del laptop
```

Checkpoint: You saw `no address range available` in the server log, and after the fix you saw DISCOVER, OFFER, REQUEST, ACK and a working lease in the laptop namespace.

Task 10: Scenario 9: “One cable failed and the server stayed up” (bonding)

Bonding (also called teaming or link aggregation) combines several network interfaces into one logical interface, `bond0`. Common modes: `active-backup` (mode 1: one link carries traffic, the other takes over if it fails; works with any switch), `802.3ad` (mode 4, LACP: all links carry traffic; the switch must be configured for LACP), and `balance-rr`, `balance-alb`, and others.

1. **Build it.** Create a `bondhost` namespace with two “NICs” both plugged into the LAN 1 switch, and bond them in active-backup mode.

```
sudo ip netns add bondhost
sudo ip link add eth0b type veth peer name veth-rb0
sudo ip link add eth1b type veth peer name veth-rb1
sudo ip link set eth0b netns bondhost
sudo ip link set eth1b netns bondhost
sudo ip link set veth-rb0 netns router
sudo ip link set veth-rb1 netns router
sudo ip -n router link set veth-rb0 master br0
sudo ip -n router link set veth-rb1 master br0
sudo ip -n router link set veth-rb0 up
sudo ip -n router link set veth-rb1 up
sudo ip -n bondhost link add bond0 type bond mode active-backup miimon 100
sudo ip -n bondhost link set eth0b master bond0
sudo ip -n bondhost link set eth1b master bond0
sudo ip -n bondhost link set lo up
sudo ip -n bondhost link set bond0 up
sudo ip -n bondhost addr add 10.10.1.30/24 dev bond0
sudo ip -n bondhost route add default via 10.10.1.1
```

What just happened: `type bond mode active-backup miimon 100` creates the bond and checks each member link's carrier every 100 ms (MII monitoring). `master bond0` enslaves each interface; the IP address goes on `bond0`, never on the members. If `ip link add ... type bond` says `Unknown device type`, the `bonding` module could not load: check `modinfo bonding` and `/etc/modprobe.d/` for a blacklist (see Lab 5.2, missing drivers).

2. **Check its health.** The kernel reports bond status in `/proc/net/bonding/`.

```
sudo ip netns exec bondhost cat /proc/net/bonding/bond0
sudo ip netns exec bondhost ping -c 2 10.10.2.20 | tail -n 2
```

```
Ethernet Channel Bonding Driver: v6.8.0-45-generic

Bonding Mode: fault-tolerance (active-backup)
Primary Slave: None
Currently Active Slave: eth0b
MII Status: up
MII Polling Interval (ms): 100

Slave Interface: eth0b
MII Status: up
Speed: 10000 Mbps
Duplex: full
Link Failure Count: 0

Slave Interface: eth1b
MII Status: up
Speed: 10000 Mbps
Duplex: full
Link Failure Count: 0
```

3. **Break it.** Fail the active link by shutting the switch port it is plugged into.

```
sudo ip -n router link set veth-rb0 down
sleep 1
sudo ip netns exec bondhost grep -E "Currently Active|Slave Interface|MII Status|Link
Failure" \
    /proc/net/bonding/bond0
sudo ip netns exec bondhost ping -c 2 10.10.2.20 | tail -n 2
```

```
Currently Active Slave: eth1b
MII Status: up
Slave Interface: eth0b
MII Status: down
Link Failure Count: 1
Slave Interface: eth1b
MII Status: up
Link Failure Count: 0
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
```

What just happened: The bond detected the lost carrier on `eth0b` within 100 ms, switched the active member to `eth1b`, and traffic kept flowing. The bond's overall `MII Status` is still `up`. This is why a degraded bond is dangerous: nothing visibly breaks, so monitor `/proc/net/bonding/*` or `Link Failure Count`, or the second failure takes the server down.

4. **Fix and verify.** Restore the failed link and confirm both members are up again.

```
sudo ip -n router link set veth-rb0 up
sleep 1
sudo ip netns exec bondhost grep -A1 "Slave Interface" /proc/net/bonding/bond0
```

5. **Common bonding misconfigurations.** In `802.3ad` mode with a switch that is *not* configured for LACP, `/proc/net/bonding/bond0` shows `Partner Mac Address: 00:00:00:00:00:00` and members in different aggregators, and only one link (or none) passes traffic: configure LACP on the switch ports or use `active-backup`. Members plugged into different, unconnected switches cause one-way traffic in load-balancing modes. A permanent Ubuntu bond is defined in netplan; save it as a file under `/etc/netplan/` and run `sudo netplan try`:

```
network:
  version: 2
  ethernets:
    enp0s8: {}
    enp0s9: {}
  bonds:
    bond0:
      interfaces: [enp0s8, enp0s9]
      addresses: [192.168.50.10/24]
      parameters:
        mode: active-backup
        mii-monitor-interval: 100
```

On Rocky: `nmcli connection add type bond ifname bond0 bond.options "mode=active-backup,miimon=100"` plus one `type ethernet ... master bond0` connection per member.

6. **Undo.** `sudo ip netns del bondhost`

> **Checkpoint:** You saw `Currently Active Slave` change from `eth0b` to `eth1b` with zero ping loss, and `Link Failure Count: 1` recorded the failure.

Task 11: Scenario 10: "I changed the MAC address and lost the network" (MAC spoofing)

Every NIC has a burned-in MAC address, but Linux can set any MAC with `ip link set dev IF address XX:...`. **MAC spoofing** is using a MAC address that is not the one assigned. Attackers use it to bypass MAC filters or impersonate devices, so hypervisors and switches often block it: VirtualBox and Hyper-V have a "MAC address spoofing" setting, VMware has "MAC address changes" and "Forged transmits" (set to Reject), libvirt uses network filters, and physical switches use *port security*. With protection on, frames with any other source MAC are dropped.

1. **Break it (part 1: the protection).** Simulate a hypervisor's anti-spoofing rule on the router's switch port: an nftables bridge rule that drops frames from `veth-rc` unless they carry the client's assigned MAC.

```
MAC=$(sudo ip -n client -br link show veth-c | awk '{print $3}'); echo "assigned MAC: $MAC"
sudo ip netns exec router nft -f - <<EOF
table bridge portsec {
    chain pre {
        type filter hook prerouting priority 0; policy accept;
        iifname "veth-rc" ether saddr != $MAC counter drop
    }
}
EOF
sudo ip netns exec client ping -c 1 10.10.2.20 | sed -n 2p
```

> **What just happened:** The `bridge` family of nftables filters Ethernet frames as they enter the switch (`hook prerouting`). The rule drops any frame arriving on port `veth-rc` whose source MAC is not the assigned one, and counts them. This heredoc has no quotes around `EOF` , so the shell fills in `$MAC` . The client still works because it uses its assigned MAC.

2. **Break it (part 2: the change).** Inside the "VM", an admin sets a different MAC (perhaps copied from an old server's config).

```
sudo ip -n client link set dev veth-c down
sudo ip -n client link set dev veth-c address 02:11:22:33:44:55
sudo ip -n client link set dev veth-c up
sudo ip -n client route add default via 10.10.1.1
```

> **What just happened:** Many NICs only accept a MAC change while down, so you take the interface down, set the address, and bring it up. Bringing it down removed the default route, so you add it back. `02:` at the start marks a *locally administered* address, the correct kind to invent.

3. **Symptom.**

```
sudo ip netns exec client ping -c 2 -W 1 10.10.2.20 | tail -n 2
```

```
2 packets transmitted, 0 received, 100% packet loss, time 1027ms
```

4. **Investigate.** Layer 1 is fine (link `UP`, `LOWER_UP`), so check layer 2: compare the current MAC with the assigned one, and look for drops at the "switch".

```
sudo ip -n client -br link show veth-c
echo "assigned: $MAC"
sudo ip netns exec router nft list table bridge portsec | grep counter
```

```
veth-c@if5       UP             02:11:22:33:44:55 <BROADCAST,MULTICAST,UP,LOWER_UP>
assigned: 0a:00:0c:29:b1:ae
iifname "veth-rc" ether saddr != 0a:00:0c:29:b1:ae counter packets 2 bytes 56 drop
```

> **What just happened:** The interface uses a MAC that differs from the one the "hypervisor" assigned, and the drop counter is rising (these first packets are ARP requests, 28 bytes each). On a real VM, compare `ip link` with the MAC shown in the hypervisor's VM settings; on physical NICs, `ip link` shows a `permaddr` field when the MAC was changed from the burned-in one.

5. **Root cause and fix.** The MAC was changed, and anti-spoofing protection drops the frames. Either restore the assigned MAC (usual fix) or, if the change is legitimate (for example a cluster virtual MAC), allow it in the hypervisor or switch.

```
sudo ip -n client link set dev veth-c down
sudo ip -n client link set dev veth-c address "$MAC"
sudo ip -n client link set dev veth-c up
sudo ip -n client route add default via 10.10.1.1
sudo ip netns exec client ping -c 1 10.10.2.20 | sed -n 2p
```

6. **Prevent.** Find where a MAC is set permanently: netplan `macaddress:` under an interface, NetworkManager `ethernet.cloned-mac-address` (`nmcli -f 802-3-ethernet connection show eth0`). Leave spoofing protection **on** for normal VMs and enable it only for workloads that need it (nested virtualization, some clusters and load balancers). Tools such as `arpwatch` alert when a known IP suddenly appears with a new MAC.

7. **Undo.** `sudo ip netns exec router nft delete table bridge portsec`

> **Checkpoint:** The drop counter showed the spoofed frames, and after restoring the assigned MAC the client reaches the server again.

Task 12: Scenario 11: "The web server works locally but not from other hosts" (misconfigured firewall)

Now use your two real VMs. Rocky runs firewalld, which blocks incoming connections except for allowed services.

1. **Break it.** Install and start Apache on rocky-lab, but do not open the firewall (a very common oversight).

```
# on rocky-lab
sudo dnf install -y httpd
echo "hello from rocky-lab" | sudo tee /var/www/html/index.html
sudo systemctl enable --now httpd
```

2. **Symptom.** From ubuntu-lab:

```
curl -v --max-time 5 http://rocky-lab/ 2>&1 | grep -E "Trying|connect|Failed"
nc -zv -w 3 rocky-lab 80
```

```
*   Trying 192.168.100.20:80...
* connect to 192.168.100.20 port 80 from 192.168.100.10 port 51514 failed: No route to host
curl: (7) Failed to connect to rocky-lab port 80 after 3 ms: Couldn't connect to server
nc: connect to rocky-lab (192.168.100.20) port 80 (tcp) failed: No route to host
```

> **What just happened:** `curl -v` shows each step of the connection. "No route to host" returned within milliseconds is typical of a firewall that **rejects** (sends back an ICMP "administratively prohibited" message), which is firewalld's default. A firewall that silently **drops** would give "Connection timed out" instead. Either way, ping can still work, because firewalld allows ICMP by default.

3. **Investigate on rocky-lab**, bottom up: is the service listening, does it work locally, and what does the firewall allow?

```
# on rocky-lab
sudo ss -tlnp | grep ':80 '
curl -s http://localhost/
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
```

```
LISTEN 0  511   *:80   *:*   users:(("httpd",pid=2211,fd=4),...)
hello from rocky-lab
public
  interfaces: ens18
public (active)
  target: default
  interfaces: ens18
  services: cockpit dhcpv6-client ssh
  ports:
  ...
```

> **What just happened:** httpd listens on all addresses (`*:80`) and answers locally, so the application is fine. The active zone `public` allows only `cockpit` , `dhcpv6-client` , and `ssh` ; `http` is missing. Hypothesis confirmed: the firewall blocks port 80.

4. **Prove it with firewall logging** (very useful when the rules are complicated).

```
# on rocky-lab
sudo firewall-cmd --set-log-denied=all
```

Run the `curl` from ubuntu-lab again, then on rocky-lab:

```
# on rocky-lab
sudo journalctl -k --since "2 minutes ago" | grep -i reject | tail -n 1
sudo firewall-cmd --set-log-denied=off
```

```
kernel: filter_IN_public_REJECT: IN=ens18 OUT= MAC=... SRC=192.168.100.10 DST=192.168.100.20
LEN=60 ... PROTO=TCP SPT=51514 DPT=80 WINDOW=64240 SYN URGP=0
```

What just happened: `--set-log-denied=all` makes firewalld log every rejected or dropped packet to the kernel log. The entry shows the zone (`public`), the source, and the destination port (`DPT=80`) of the blocked SYN. Turn logging off afterwards; it is noisy.

5. **Fix.** Allow the service permanently, then reload.

```
# on rocky-lab
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
```

What just happened: `--permanent` writes the rule to the saved configuration, and `--reload` applies the saved configuration to the running firewall. A rule added **without** `--permanent` works immediately but disappears at the next reload or reboot: a classic "it worked yesterday" firewall problem. (`--runtime-to-permanent` saves current runtime rules.)

6. **Verify.** On ubuntu-lab, `curl http://rocky-lab/` prints `hello from rocky-lab`.
7. **Ubuntu side.** ufw is inactive by default on Ubuntu. If it is active, the same investigation is `sudo ufw status verbose`, the fix is `sudo ufw allow 80/tcp`, and denied packets are logged as `[UFW BLOCK]` in `journalctl -k`. Under both tools, `sudo nft list ruleset` shows the real nftables rules.

Leave httpd running for the next scenario.

Checkpoint: `curl http://rocky-lab/` from ubuntu-lab returns the page, and `sudo firewall-cmd --list-services` on rocky-lab includes `http`.

Task 13: Scenario 12: "SSH to rocky-lab takes several seconds to connect" (dual stack issues)

Dual stack means a host runs IPv4 and IPv6 at the same time. Every IPv6 interface has a **link-local** address (`fe80::/10`), usable only on its own link. When a name has both an IPv4 (`A`) and IPv6 (`AAAA`) address, the system chooses an order (address selection rules from RFC 6724, tuned in `/etc/gai.conf`). IPv6 usually wins. If the IPv6 path is broken, programs try IPv6 first, wait, and then fall back to IPv4.

1. **Explore IPv6 between the VMs.** Find rocky-lab's link-local address and ping it from ubuntu-lab. A link-local address needs the outgoing interface after `%`.

```
# on rocky-lab
ip -6 addr show scope link
```

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 fe80::7c1e:52ff:fe4a:9b20/64 scope link noprefixroute
```

```
IF=$(ip route show default | awk '{print $5; exit}'); echo "$IF"
ping -6 -c 2 fe80::7c1e:52ff:fe4a:9b20%$IF
```

```
PING fe80::7c1e:52ff:fe4a:9b20%enp0s3(fe80::7c1e:52ff:fe4a:9b20%enp0s3) 56 data bytes
64 bytes from fe80::7c1e:52ff:fe4a:9b20%enp0s3: icmp_seq=1 ttl=64 time=0.402 ms
```

What just happened: Use *your* rocky-lab address from the first command. `%enp0s3` (the **zone index**) tells ping which link to use, because every interface has its own `fe80::` network. IPv6 on the link works.

2. **Break it.** Give ubuntu-lab a global IPv6 address and add an IPv6 address for `rocky-lab` to `/etc/hosts`, but do **not** configure it on rocky-lab. (This is like a DNS `AAAA` record that points to an address the server does not really have.) `2001:db8::/32` is reserved for documentation and labs.

```
sudo cp /etc/hosts /etc/hosts.lab-orig
sudo ip -6 addr add 2001:db8:100::10/64 dev "$IF"
echo "2001:db8:100::20 rocky-lab" | sudo tee -a /etc/hosts
```

3. **Symptom.**

```
time ssh -v -o BatchMode=yes student@rocky-lab true 2>&1 | grep -E "Connecting to|connect to
address|Connection established"
```

```
debug1: Connecting to rocky-lab [2001:db8:100::20] port 22.
debug1: connect to address 2001:db8:100::20 port 22: No route to host
debug1: Connecting to rocky-lab [192.168.100.20] port 22.
debug1: Connection established.

real    0m3.112s
```

What just happened: `ssh -v` prints debug messages, `-o BatchMode=yes` stops it from asking for a password (it will fail authentication, which does not matter here), and `true` is the remote command. SSH tried the IPv6 address first, waited about 3 seconds while IPv6 neighbor discovery failed, then fell back to IPv4. Users just see "SSH is slow". Some programs with short timeouts fail completely instead.

4. **Investigate.**

```
getent ahosts rocky-lab | grep STREAM
curl -4 -s --max-time 3 http://rocky-lab/; echo "IPv4 exit: $?"
curl -6 -s --max-time 6 http://rocky-lab/; echo "IPv6 exit: $?"
ip -6 neigh show 2001:db8:100::20
```

```
2001:db8:100::20 STREAM rocky-lab
192.168.100.20  STREAM
hello from rocky-lab
IPv4 exit: 0
IPv6 exit: 7
2001:db8:100::20 dev enp0s3 FAILED
```

What just happened: `getent ahosts` shows every address for a name **in the order programs will try them**: IPv6 first. `curl -4` and `curl -6` force one protocol, which isolates the broken one: IPv4 works, IPv6 fails (exit 7, cannot connect; the longer `--max-time` lets the roughly 3-second neighbor discovery give up first, otherwise you would see exit 28, timeout). `ip -6 neigh` shows IPv6 neighbor discovery (the IPv6 version of ARP) `FAILED` for that address: nobody on the link has it. (`curl` without `-4` / `-6` uses "Happy Eyeballs", trying both almost at once, so browsers and curl hide this problem while SSH shows it.)

5. **Root cause.** The name has an IPv6 address that the server does not actually answer on: broken dual stack.

6. **Fix option A: make dual stack really work.** Configure the address on rocky-lab. The commands below use `ens18`; replace it with rocky-lab's interface name from step 1.

   ```
   # on rocky-lab
   sudo ip -6 addr add 2001:db8:100::20/64 dev ens18
   ```

   Then on ubuntu-lab:

   ```
   curl -6 -s http://rocky-lab/
   time ssh -o BatchMode=yes student@rocky-lab true 2>/dev/null
   ```

   What just happened: Now IPv6 works end to end: `curl -6` gets the page and SSH connects instantly. firewalld rules apply to IPv4 and IPv6 alike, so `http` is already allowed. With plain `nftables` / `iptables`, remember IPv6 needs its own rules (`ip6tables`, or the `inet` family in nftables): a classic dual stack mistake is opening a port for IPv4 only.

7. **Fix option B: prefer IPv4** (a workaround when you cannot fix IPv6 right away). Remove the rocky-lab IPv6 address again, then tell the resolver library to sort IPv4 first. In `/etc/gai.conf`, this line is present but commented out:

   ```
   # on rocky-lab
   sudo ip -6 addr del 2001:db8:100::20/64 dev ens18
   ```

```
sudo cp /etc/gai.conf /etc/gai.conf.lab-orig
sudo sed -i 's/^#precedence ::ffff:0:0\/96  100/precedence ::ffff:0:0\/96  100/'
/etc/gai.conf
grep '^precedence' /etc/gai.conf
getent ahosts rocky-lab | grep STREAM
```

```
precedence ::ffff:0:0/96  100
192.168.100.20  STREAM rocky-lab
2001:db8:100::20 STREAM
```

What just happened: `::ffff:0:0/96` is how the address selection table represents all IPv4 addresses; giving it precedence `100` puts IPv4 ahead of IPv6. `getent ahosts` now lists IPv4 first, so SSH connects instantly. This hides the real problem, so treat it as temporary.

8. **Undo.**

```
sudo cp /etc/gai.conf.lab-orig /etc/gai.conf && sudo rm /etc/gai.conf.lab-orig
sudo cp /etc/hosts.lab-orig /etc/hosts && sudo rm /etc/hosts.lab-orig
sudo ip -6 addr del 2001:db8:100::10/64 dev "$IF"
```

9. **Prevent.** Only publish `AAAA` records for services that really listen on IPv6, test both protocols explicitly (`curl -4`, `curl -6`, `ping -4`, `ping -6`), and keep IPv4 and IPv6 firewall rules in sync.

Checkpoint: You saw SSH try IPv6 first and fall back after about 3 seconds, used `getent ahosts` to see the order, and isolated the broken protocol with `curl -4` and `curl -6`.

Challenge

CHALLENGE

1. Rebuild the practice network and, without looking at the scenarios, have a partner (or a script with `shuf`) apply one random break: wrong mask, wrong gateway, forwarding off, missing return route, or MTU mismatch. Diagnose it using only the bottom-up flow and write down which command first revealed the cause.
2. Add a second server namespace on LAN 2 that runs `python3 -m http.server 8080`, then add an nftables rule in the router namespace that drops TCP port 8080. Diagnose it from the client using `nc -zv`, `tcpdump` on both sides of the router, and `nft list ruleset`.
3. Write a netplan file (do not apply it) that gives ubuntu-lab a static IPv4 address, a static IPv6 address, a default gateway for each, and two DNS servers. Validate it with `sudo netplan generate`, then delete it.

Clean up

1. Remove every practice namespace and the files the lab created.

```
sudo ~/labs/5.3/labnet-down.sh
sudo ip netns list
```

2. Make sure the dual stack changes are gone on ubuntu-lab (errors such as “No such file” or “Cannot assign” are fine).

```
IF=$(ip route show default | awk '{print $5; exit}')
sudo ip -6 addr del 2001:db8:100::10/64 dev "$IF" 2>/dev/null
grep 2001:db8 /etc/hosts
grep '^precedence' /etc/gai.conf
```

   The last two commands should print nothing. If they do, restore the `.lab-orig` copies as in Task 13 step 8.

3. On rocky-lab, remove the IPv6 address (if still present), close the firewall, and remove Apache.

```
# on rocky-lab
sudo ip -6 addr del 2001:db8:100::20/64 dev ens18 2>/dev/null
sudo firewall-cmd --permanent --remove-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --set-log-denied=off
sudo systemctl disable --now httpd
sudo dnf remove -y httpd
sudo rm -rf /var/www/html/index.html
```

4. Remove the lab directory.

```
rm -rf ~/labs/5.3
```

Exam corner

- Troubleshoot bottom up: link (`ip link` , `ethtool`), MAC/ARP/MTU (`ip neigh` , `arping`), IP/routing (`ip addr` , `ip route` , `ping` , `traceroute` / `tracepath`), ports/firewall (`ss` , `nc` , `firewall-cmd`), then DNS/app (`dig` , `curl`).
- `NO-CARRIER` with `UP` = cable, switch port, or far end down (**link down**). Duplex mismatch (**link negotiation**) = link up but slow with collisions/CRC errors; fix with auto-negotiation on both sides (`ethtool -s IF autoneg on`).
- **MTU mismatch**: small packets work, large transfers hang. Test with `ping -M do -s 1472` (1472 + 28 = 1500).
- Wrong **subnet** mask: too wide makes remote hosts look local (`Destination Host Unreachable` , ARP `INCOMPLETE`); too narrow puts the gateway off-net. `ip route get <ip>` shows the decision.
- **Gateway**: wrong = `Destination Host Unreachable` ; missing = `Network is unreachable` . Fix with `ip route replace default via X` , persist in netplan/nmcli.
- **Routing**: packets need a route back. `tcpdump` showing requests with no replies means a missing return route; a Linux router needs `net.ipv4.ip_forward=1` .
- Server **unreachable**: “Connection refused” = nothing listening; “timed out” = dropped (firewall/routing/host down); TCP “No route to host” = often a firewall reject.

- **DNS**: "Temporary failure in name resolution". `dig` bypasses `/etc/hosts`; `getent hosts` follows `nsswitch.conf`. Ubuntu: `resolvectl status`, stub `127.0.0.53`.
- **IP conflict**: intermittent connectivity, ARP entry flapping between MACs; detect with `arping -D -I IF IP` (exit 1 = conflict).
- **DHCP**: DORA (Discover, Offer, Request, ACK), UDP 67/68; `169.254.x.x` (APIPA) means no DHCP server answered. Check server logs and `journalctl -u systemd-networkd` / `NetworkManager`.
- **Bonding**: status in `/proc/net/bonding/bond0`; `active-backup` = mode 1, `802.3ad` (LACP) = mode 4 and needs switch support.
- **MAC spoofing**: `ip link set dev IF address ...`; hypervisors/switches may block non-assigned MACs.
- **Dual stack**: test with `ping -4/-6`, `curl -4/-6`; `getent ahosts` shows the order; `precedence ::ffff:0:0/96 100` in `/etc/gai.conf` prefers IPv4; link-local `fe80::` needs `%interface`.
- **Firewall**: `firewall-cmd --list-all`, `--permanent` + `--reload`, `--set-log-denied=all`; Ubuntu `ufw status verbose`.

Review questions

1. A host can ping its gateway but not a server on another subnet. `ip route get <server>` returns a route with no `via`. What is the most likely cause?
   A. DNS failure B. Subnet mask too wide C. Link down D. MTU mismatch

2. Users can SSH to a server, but the session freezes whenever a command produces a lot of output. Which test best confirms the suspected cause?

3. `ip link` shows `<NO-CARRIER,BROADCAST,MULTICAST,UP>` for `eth0`. What does this mean and what should you check?

4. `getent hosts app01` returns `10.0.0.5` but `dig +short app01` returns `10.0.0.50`. Where should you look first?

5. A new workstation shows the address `169.254.33.7`. What failed?
   A. DNS B. DHCP C. The default gateway D. Bonding

6. From a client, `nc -zv web01 443` fails instantly with "No route to host", but `ping web01` works. What is the most likely cause?

Lab 5.4 Troubleshoot Security Issues

Objective: 5.4 Given a scenario, analyze and troubleshoot security issues on a Linux system.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 3.5 hours
VM: rocky-lab (SELinux, faillock, repositories) and ubuntu-lab (everything else)
Snapshot first: Yes. Take a snapshot of both VMs now. This lab changes SSH, PAM, firewall, and repository settings, and a mistake there can lock you out.

Security problems show up in two ways: something legitimate is blocked ("403 Forbidden", "Permission denied (publickey)", "certificate has expired") or something dangerous is allowed (a database open to the network, TLS 1.0 still enabled, months of missing patches). In this break-and-fix lab you create both kinds, find the evidence in the logs, and fix them the right way, not by turning security off.

What you'll practice

- SELinux issues: file **context** (`ls -Z`, `restorecon`), **booleans** (`getsebool`, `setsebool -P`), and **policy** (port labels with `semanage port`); reading AVC denials with `ausearch` and `sealert`
- File and directory permission issues: missing execute on directories, **ACL** mask problems (`getfacl`, `setfacl`), and **attributes** (`lsattr`, `chattr`)
- Account access: locked, expired, and `nologin` accounts (`passwd -S`, `chage -l`, `usermod`), and `faillock` lockouts
- Remote access issues: `AllowUsers`, wrong `~/.ssh` permissions, reading `ssh -v` and sshd logs
- Unpatched vulnerable systems: pending security updates, held packages, unattended-upgrades, `dnf updateinfo`, reboot-required checks
- Exposed or misconfigured services: a database listening on all interfaces, found with `ss` and `nmap`
- Certificate issues: untrusted, missing intermediate, hostname mismatch, and expired certificates (`openssl s_client`, `openssl x509`, `curl -v`)
- Misconfigured package repository: bad URLs and wrong signing keys (`NO_PUBKEY`) with apt and dnf
- Use of obsolete or insecure protocols and ciphers, and cipher negotiation issues (TLS and SSH, `nmap ssl-enum-ciphers`, `update-crypto-policies`)

How it works

The same method, with a security twist. You still identify symptoms, gather data, form a hypothesis, test, fix, verify, and document (Lab 5.2). Security adds one rule: **the fix must not weaken security**. "Disable SELinux", "chmod 777", "PermitRootLogin yes", and "skip certificate checks" make the symptom go away while creating a bigger problem. The right fix corrects the label, permission, key, or configuration that was actually wrong.

Where security evidence lives.

What	Ubuntu	Rocky
Logins, sudo, SSH	`/var/log/auth.log`, `journalctl -u ssh`	`/var/log/secure`, `journalctl -u sshd`
SELinux / AppArmor denials	`journalctl -k | grep apparmor`	`/var/log/audit/audit.log` (`ausearch -m avc`)
Web server errors	`/var/log/nginx/error.log`	`/var/log/httpd/error_log`
Package updates	`/var/log/apt/history.log`, `/var/log/unattended-upgrades/`	`dnf history`

SELinux in one page. SELinux (Security-Enhanced Linux, enforcing by default on Rocky) labels every process and file with a **context** like `system_u:system_r:httpd_t:s0`. The third part is the **type**. The **policy** is a large set of rules saying which process types may do what to which object types: for example `httpd_t` may read files of type `httpd_sys_content_t` and bind to ports of type `http_port_t`. Anything not allowed is denied and logged as an **AVC** (access vector cache) denial, *even for root and even if normal file permissions allow it*. **Booleans** are on/off switches built into the policy for common optional behavior (for example `httpd_can_network_connect`). So an SELinux problem is always one of three things:

```
denied!  ->  is the FILE/PORT labeled wrong?    -> restorecon / semanage fcontext
         ->  is an optional feature switched off? -> setsebool -P <boolean> on
         ->  is it truly not in the policy?       -> semanage port, or (last resort)
                                                     audit2allow -M local module
```

Ubuntu uses **AppArmor** instead, which confines programs by file path rather than labels. Its denials appear in the kernel log as `apparmor="DENIED"`; `sudo aa-status` lists confined programs.

Permissions are layered. Access must pass *every* layer: directory execute permission on every parent directory, the file's mode bits, any **ACL** (access control list) entries limited by the ACL **mask**, file **attributes** such as immutable (`i`) or append-only (`a`), and finally SELinux or AppArmor. Check them in that order.

TLS certificates in one paragraph. A server certificate binds a name (in the Subject Alternative Name, **SAN**) to a public key, is valid between two dates, and is signed by a CA (certificate authority), usually through an **intermediate** CA. A client trusts it only if it can build a chain from the server certificate, through intermediates *the server sends*, up to a root CA in its own trust store, and the name it connected to is in the SAN, and today's date is within the validity period.

Tasks

Task 1: Prepare both VMs

1. On rocky-lab, confirm SELinux is enforcing and install the SELinux troubleshooting tools and Apache.

```
# on rocky-lab
mkdir -p ~/labs/5.4 && cd ~/labs/5.4
getenforce
sudo dnf install -y httpd policycoreutils-python-utils setroubleshoot-server
echo "hello from rocky-lab" | sudo tee /var/www/html/index.html
sudo systemctl enable --now httpd
curl -s http://localhost/
```

```
Enforcing
...
hello from rocky-lab
```

What just happened: `getenforce` prints the SELinux mode: `Enforcing` (deny and log), `Permissive` (log only), or `Disabled`. `policycoreutils-python-utils` provides `semanage`, `audit2allow`, and `audit2why`; `setroubleshoot-server` provides `sealert`, which explains denials in plain English. Apache answers locally.

2. On ubuntu-lab, create the lab directory and two test users. `bob` has no password (you will act as him with `sudo -u bob`); `carol` gets a password so she can log in over SSH.

```
mkdir -p ~/labs/5.4 && cd ~/labs/5.4
sudo apt update
sudo apt install -y acl openssl nginx nmap faketime
sudo useradd -m -s /bin/bash bob
sudo useradd -m -s /bin/bash carol
echo 'carol:Lab54-Pass!' | sudo chpasswd
```

What just happened: `useradd -m` creates the home directory and `-s` sets the login shell. `chpasswd` reads `user:password` lines and sets passwords without prompting. (On a real system never put a password on a command line; it lands in your shell history. It is acceptable for a throwaway lab account.)

Checkpoint: `getenforce` on rocky-lab prints `Enforcing`, `curl -s http://localhost/` there prints the page, and `id bob carol` on ubuntu-lab shows both users.

Task 2: Scenario 1: "403 Forbidden" after moving a file (SELinux context)

1. **Break it.** On rocky-lab, create a page in your home directory and *move* it into the web root, as many admins do.

```
# on rocky-lab
echo "<h1>Quarterly report</h1>" > ~/report.html
sudo mv ~/report.html /var/www/html/
```

2. **Symptom.**

```
# on rocky-lab
curl -s -o /dev/null -w '%{http_code}\n' http://localhost/report.html
```

```
403
```

3. **Investigate the normal layers first:** file permissions and the web server's own log.

```
# on rocky-lab
ls -l /var/www/html/
sudo tail -n 1 /var/log/httpd/error_log
```

```
-rw-r--r--. 1 root    root    21 Oct 10 14:02 index.html
-rw-rw-r--. 1 student student 26 Oct 10 14:05 report.html
[core:error] [pid 2345:tid 2401] (13)Permission denied: [client ::1:54322] AH00132: file
permissions deny server access: /var/www/html/report.html
```

What just happened: The mode bits allow everyone to read (`r--` for others), yet Apache reports "Permission denied". The `.` after the permissions means the file has an SELinux context. When normal permissions look fine but access is denied, suspect SELinux.

4. **Investigate SELinux.** Compare the labels and look for AVC denials.

```
# on rocky-lab
ls -Z /var/www/html/
sudo ausearch -m avc -ts recent
```

```
system_u:object_r:httpd_sys_content_t:s0 index.html
unconfined_u:object_r:user_home_t:s0 report.html
----
time->Sat Oct 10 14:05:31 2026
type=AVC msg=audit(1791641131.512:412): avc:  denied  { read } for  pid=2345 comm="httpd"
name="report.html" dev="dm-0" ino=17112 scontext=system_u:system_r:httpd_t:s0
tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file permissive=0
```

What just happened: `ls -Z` shows contexts: `index.html` has the web content type `httpd_sys_content_t`, but `report.html` kept `user_home_t` from your home directory, because `mv` keeps a file's label (while `cp` creates a new file that gets the destination directory's label). `ausearch -m avc -ts recent` searches the audit log for AVC messages from the last 10 minutes. Read it as: process `httpd` with context `scontext` (`httpd_t`) was denied `read` on a `file` with context `tcontext` (`user_home_t`). `permissive=0` means it was really blocked.

5. **Test the hypothesis** with the plain-English explanation and the expected label.

```
# on rocky-lab
sudo sealert -a /var/log/audit/audit.log | grep -A6 "SELinux is preventing" | head -n 8
matchpathcon /var/www/html/report.html
```

```
SELinux is preventing /usr/sbin/httpd from read access on the file report.html.

*****  Plugin restorecon (99.5 confidence) suggests   ************************

If you want to fix the label.
/var/www/html/report.html default label should be httpd_sys_content_t.
Then you can run restorecon.
/var/www/html/report.html   system_u:object_r:httpd_sys_content_t:s0
```

6. **Root cause.** Wrong SELinux file context (`user_home_t` instead of `httpd_sys_content_t`).
7. **Fix.** Reset the label to what the policy says it should be.

```
# on rocky-lab
sudo restorecon -v /var/www/html/report.html
```

```
Relabeled /var/www/html/report.html from unconfined_u:object_r:user_home_t:s0 to
unconfined_u:object_r:httpd_sys_content_t:s0
```

> **What just happened:** `restorecon` sets files back to the default context defined by the policy's file-context rules (`-v` shows what changed, `-R` would recurse into directories). If you serve content from a non-standard directory such as `/srv/site` , first teach the policy the right label with `sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"` , then run `sudo restorecon -Rv /srv/site` . Using `chcon` also changes a label, but only until the next relabel.

8. **Verify.** The `curl` from step 2 now prints `200` .
9. **Prevent.** Use `cp` (or `mv` followed by `restorecon`) when placing web content, and never "fix" SELinux with `setenforce 0` . Switching to permissive mode is only acceptable for a few seconds as a *test* to confirm SELinux is involved.

> **Checkpoint:** `ls -Z /var/www/html/report.html` shows `httpd_sys_content_t` and the page returns `200` .

Task 3: Scenario 2: "503 Service Unavailable" from a reverse proxy (SELinux boolean)

1. **Break it.** Start a small back-end application on port 5000 and make Apache a reverse proxy for it.

```
# on rocky-lab
mkdir -p /tmp/backend && echo "backend app says hi" > /tmp/backend/index.html
python3 -m http.server 5000 --bind 127.0.0.1 --directory /tmp/backend > /dev/null 2>&1 &
curl -s http://127.0.0.1:5000/
cat <<'EOF' | sudo tee /etc/httpd/conf.d/lab-proxy.conf
# Forward /app/ to the back-end application
ProxyPass        "/app/" "http://127.0.0.1:5000/" retry=0
ProxyPassReverse "/app/" "http://127.0.0.1:5000/"
EOF
sudo systemctl reload httpd
```

What just happened: The back end answers directly on `127.0.0.1:5000`. `ProxyPass` makes Apache forward requests for `/app/` to it, and `retry=0` makes Apache retry a failed back end immediately instead of waiting 60 seconds (handy while troubleshooting). A **heredoc** (`cat <<'EOF' | sudo tee file`) writes all lines up to `EOF` into the file.

2. **Symptom.**

```
# on rocky-lab
curl -s -o /dev/null -w '%{http_code}\n' http://localhost/app/
sudo tail -n 2 /var/log/httpd/error_log
```

```
503
[proxy:error] [pid 2412:tid 2460] (13)Permission denied: AH00957: http: attempt to connect
to 127.0.0.1:5000 (127.0.0.1:5000) failed
[proxy_http:error] [pid 2412:tid 2460] [client ::1:41236] AH01114: HTTP: failed to make
connection to backend: 127.0.0.1
```

3. **Investigate.** The back end works when you call it directly, and Apache gets "Permission denied" when *connecting*. Look for the AVC.

```
# on rocky-lab
sudo ausearch -m avc -ts recent | grep name_connect | tail -n 1
sudo ausearch -m avc -ts recent | audit2why | tail -n 6
```

```
type=AVC msg=audit(1791641402.218:431): avc:  denied  { name_connect } for  pid=2412
comm="httpd" dest=5000 scontext=system_u:system_r:httpd_t:s0
tcontext=system_u:object_r:commplex_main_port_t:s0 tclass=tcp_socket permissive=0

        Was caused by:
        One of the following booleans was set incorrectly.
        Description:
        Allow httpd to can network connect

        Allow access by executing:
        # setsebool -P httpd_can_network_connect 1
```

What just happened: `name_connect` on a `tcp_socket` means httpd tried to open an outgoing TCP connection to port 5000 (whose type here is `commplex_main_port_t`), and the policy does not allow that by default, so that a hacked web server cannot easily attack other services. `audit2why` reads AVC messages and explains why each was denied; here it names the boolean that controls this behavior.

4. **Look at the boolean.**

```
# on rocky-lab
getsebool httpd_can_network_connect
sudo semanage boolean -l | grep -E "^httpd_can_network_connect "
getsebool -a | grep -c httpd
```

```
httpd_can_network_connect --> off
httpd_can_network_connect      (off  ,  off)  Allow httpd to can network connect
48
```

What just happened: `getsebool` shows the current value. `semanage boolean -l` shows the current and the default (saved) value plus a description. There are dozens of httpd booleans alone, such as `httpd_enable_homedirs` (serve `~/public_html`), `httpd_can_network_connect_db` (connect to databases), and `httpd_can_sendmail`.

5. **Root cause and fix.** The optional "web server may make network connections" feature is off. Turn it on **permanently** with `-P`.

```
# on rocky-lab
sudo setsebool -P httpd_can_network_connect on
getsebool httpd_can_network_connect
```

What just happened: `setsebool` changes a boolean immediately; `-P` also writes it to the policy on disk so it survives a reboot (it takes a few seconds). Without `-P` the fix silently disappears at the next boot, a classic "it broke again after maintenance" problem.

6. **Verify.** `curl -s http://localhost/app/` prints `backend app says hi`.

7. **Prevent.** Enable only the booleans you need and document why. Before writing custom policy, always check whether a boolean already covers the case (`audit2why` or `sealert` will say so).

Checkpoint: `getsebool httpd_can_network_connect` shows `on` and `/app/` returns the back end's page.

Task 4: Scenario 3: Apache will not start on a new port (SELinux policy)

1. **Break it.** Ask Apache to also listen on port 8090.

```
# on rocky-lab
echo "Listen 8090" | sudo tee /etc/httpd/conf.d/lab-port.conf
sudo systemctl restart httpd
```

```
Job for httpd.service failed because the control process exited with error code.
See "systemctl status httpd.service" and "journalctl -xeu httpd.service" for details.
```

2. **Symptom and investigation.**

```
# on rocky-lab
sudo journalctl -u httpd -n 5 --no-pager
sudo ausearch -m avc -ts recent | grep name_bind | tail -n 1
sudo semanage port -l | grep -w 8090; echo "exit: $?"
sudo semanage port -l | grep ^http_port_t
```

```
httpd[2601]: (13)Permission denied: AH00072: make_sock: could not bind to address [::]:8090
httpd[2601]: (13)Permission denied: AH00072: make_sock: could not bind to address
0.0.0.0:8090
httpd[2601]: no listening sockets available, shutting down
systemd[1]: httpd.service: Main process exited, code=exited, status=1/FAILURE
type=AVC msg=audit(1791641620.004:447): avc:  denied  { name_bind } for  pid=2601
comm="httpd" src=8090 scontext=system_u:system_r:httpd_t:s0
tcontext=system_u:object_r:unreserved_port_t:s0 tclass=tcp_socket permissive=0
exit: 1
http_port_t                    tcp      80, 81, 443, 488, 8008, 8009, 8443, 9000
```

What just happened: Apache, running as root, still got "Permission denied" when binding (`name_bind`) to port 8090. SELinux labels ports too: 8090 has the generic `unreserved_port_t` type, and the policy only lets `httpd_t` bind to ports labeled `http_port_t` , which lists 80, 443, 8443 and a few others.

3. **Root cause.** The policy has no rule allowing the web server on this port, because the port is not labeled as a web port.

4. **Fix.** Add the port to the `http_port_t` type, then start Apache.

```
# on rocky-lab
sudo semanage port -a -t http_port_t -p tcp 8090
sudo semanage port -l | grep ^http_port_t
sudo systemctl restart httpd
curl -s http://localhost:8090/
```

```
http_port_t                    tcp      8090, 80, 81, 443, 488, 8008, 8009, 8443, 9000
hello from rocky-lab
```

What just happened: `semanage port -a` (add) labels TCP port 8090 as `http_port_t` ; the change is permanent. If a port already has another type, `-a` fails with "already defined" and you use `-m` (modify) instead. For remote clients you would also open the port in the firewall (`sudo firewall-cmd --permanent --add-port=8090/tcp` , `sudo firewall-cmd --reload`): SELinux and the firewall are separate layers.

5. **Last resort: a custom policy module.** When no label or boolean fits, you can turn denials into a small local policy module. Look at what it *would* allow before installing anything:

```
# on rocky-lab
sudo ausearch -m avc -ts today | audit2allow | head -n 8
```

What just happened: `audit2allow` turns AVC denials into `allow` rules. `audit2allow -M mymodule` would create `mymodule.pp` , which `sudo semodule -i mymodule.pp` installs. Review the rules carefully: blindly allowing every denial (for example letting `httpd_t` read `user_home_t`) can open real holes when the right fix was a label or boolean. In this lab, the right fixes already exist, so do not install a module.

6. **Prevent.** When you change a service's port or directories, update SELinux in the same change (`semanage port`, `semanage fcontext`), and check `sudo ausearch -m avc -ts recent` whenever a confined service fails mysteriously.

> **Checkpoint:** `sudo semanage port -l | grep ^http_port_t` includes `8090`, and `curl http://localhost:8090/` returns the page.

Task 5: Scenario 4: "Permission denied" on shared files (permissions, ACLs, attributes)

1. **Break it.** On ubuntu-lab, create a shared finance directory where `bob` should be able to read and edit the budget, then make three "small" changes that each break something.

```
sudo mkdir -p /srv/finance
echo "Q4 budget: 100" | sudo tee /srv/finance/budget.txt > /dev/null
echo "2026-10-01 audit started" | sudo tee /srv/finance/audit.log > /dev/null
sudo setfacl -m u:bob:rw /srv/finance/budget.txt
sudo -u bob cat /srv/finance/budget.txt
sudo chmod o-x /srv/finance
sudo chmod 644 /srv/finance/budget.txt
sudo chattr +a /srv/finance/audit.log
```

> **What just happened:** `setfacl -m u:bob:rw` adds an ACL entry giving bob read and write on the budget, and the `cat` proves he can read it. Then: `chmod o-x` removes "execute" for others on the directory, `chmod 644` is a routine-looking permission reset on the budget, and `chattr +a` makes the audit log append-only.

2. **Symptom 1: bob cannot reach any file.**

```
sudo -u bob cat /srv/finance/budget.txt
sudo -u bob ls -l /srv/finance
```

```
cat: /srv/finance/budget.txt: Permission denied
ls: cannot access '/srv/finance/audit.log': Permission denied
ls: cannot access '/srv/finance/budget.txt': Permission denied
total 0
-????????? ? ? ? ?            ? audit.log
-????????? ? ? ? ?            ? budget.txt
```

3. **Investigate.**

```
namei -l /srv/finance/budget.txt
```

```
f: /srv/finance/budget.txt
drwxr-xr-x root root /
drwxr-xr-x root root srv
drwxr-xr-- root root finance
-rw-r--r-- root root budget.txt
```

> **What just happened:** On a **directory**, `r` lets you list names, `w` lets you create and delete entries, and `x` lets you *enter* it and reach anything inside. `finance` is `drwxr-xr--`: others (bob) have `r` but not `x`, so `ls` can read the names but not look at the files (hence the `?` marks), and `cat` cannot reach the file at all. `namei -l` shows each directory on the path so you can spot the one that blocks.

4. **Fix 1.** Give others execute on the directory again, then test.

```
sudo chmod o+x /srv/finance
sudo -u bob cat /srv/finance/budget.txt
```

5. **Symptom 2: bob can read but no longer edit the budget, even though his ACL says `rw`.**

```
sudo -u bob sh -c 'echo "Q4 budget: 120" >> /srv/finance/budget.txt'
ls -l /srv/finance/budget.txt
getfacl /srv/finance/budget.txt
```

```
sh: 1: cannot create /srv/finance/budget.txt: Permission denied
-rw-r--r--+ 1 root root 15 Oct 10 15:01 /srv/finance/budget.txt
# file: srv/finance/budget.txt
# owner: root
# group: root
user::rw-
user:bob:rw-                    #effective:r--
group::r--
mask::r--
other::r--
```

> **What just happened:** The `+` after the mode means the file has an ACL. `getfacl` shows bob's entry is `rw-` but **effective** `r--`. The **mask** is the maximum permission any named user, named group, or the owning group can get; it is `r--`. On a file with an ACL, the "group" bits shown by `ls -l` *are* the mask, so `chmod 644` (group `r--`) silently lowered the mask and took bob's write access away.

6. **Fix 2.** Raise the mask (or use `chmod g+w`, which does the same thing here).

```
sudo setfacl -m m::rw /srv/finance/budget.txt
getfacl /srv/finance/budget.txt | grep -E "bob|mask"
sudo -u bob sh -c 'echo "Q4 budget: 120" >> /srv/finance/budget.txt' && echo "bob can write"
```

7. **Symptom 3: log rotation fails, even for root.**

```
sudo sh -c 'echo "2026-10-02 new entry" >> /srv/finance/audit.log' && echo "append works"
sudo sh -c ': > /srv/finance/audit.log'
sudo rm /srv/finance/audit.log
lsattr /srv/finance/audit.log
```

```
append works
sh: 1: cannot create /srv/finance/audit.log: Operation not permitted
rm: cannot remove '/srv/finance/audit.log': Operation not permitted
-----a--------e------- /srv/finance/audit.log
```

What just happened: Appending works, but truncating or deleting fails with "Operation not permitted" even as root. `lsattr` shows the `a` (append-only) **attribute**: the file can only grow. It is a deliberate protection for audit logs, but it breaks log rotation. The immutable attribute `i` is even stricter (no changes at all). Only root can change these attributes, with `chattr`.

8. **Fix 3.** If rotation is legitimately needed, remove the attribute, rotate, and decide whether to set it again.

```
sudo chattr -a /srv/finance/audit.log
lsattr /srv/finance/audit.log
```

9. **Prevent.** Prefer ACLs plus `setfacl -m d:u:bob:rw` (a *default* ACL on the directory, inherited by new files) over repeated `chmod` on files with ACLs, remember that `chmod` on the group bits changes the mask, and document every `chattr` use.

Checkpoint: bob can read and append to `budget.txt`, `getfacl` shows `mask::rw-`, and `lsattr /srv/finance/audit.log` no longer shows `a`.

Task 6: Scenario 5: "carol can't log in" (account access)

Many different account settings produce "I can't log in". You will create four on purpose, using SSH to `localhost` on ubuntu-lab, and identify each from its evidence.

1. **Confirm carol works first.** Answer `yes` to the host key question the first time, then type carol's password (`Lab54-Pass!`).

```
ssh carol@localhost whoami
```

```
carol@localhost's password:
carol
```

2. **Break 1: locked password.**

```
sudo passwd -l carol
ssh carol@localhost whoami
```

```
passwd: password changed.
carol@localhost's password:
Permission denied, please try again.
```

Press `Ctrl+C`, then investigate:

```
sudo passwd -S carol
sudo grep carol /etc/shadow | cut -d: -f1,2 | cut -c1-20
sudo journalctl -u ssh -n 5 --no-pager | grep carol
```

```
carol L 2026-10-10 0 99999 7 -1
carol:!$y$j9T$xQ
sshd[3302]: Failed password for carol from 127.0.0.1 port 49826 ssh2
```

What just happened: `passwd -S` prints the status: `L` = locked (`P` = usable password, `NP` = no password), followed by the last change date and aging fields. Locking puts a `!` in front of the password hash in `/etc/shadow`, so no typed password can ever match. The log just says "Failed password", which looks like a typo; only `passwd -S` reveals the lock. Note: a locked *password* does not block SSH *key* logins. To block an account completely, also expire it (`chage -E 0`) or set a `nologin` shell.

Fix: `sudo passwd -u carol` and confirm with `sudo passwd -S carol` (`P`).

3. **Break 2: expired account.**

```
sudo chage -E 0 carol
ssh carol@localhost whoami
```

```
carol@localhost's password:
Your account has expired; please contact your system administrator.
Connection closed by 127.0.0.1 port 22
```

Investigate:

```
sudo chage -l carol
sudo journalctl -u ssh -n 5 --no-pager | grep -i -E "expired|denied"
```

```
Last password change                                    : Oct 10, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jan 01, 1970
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
sshd[3330]: pam_unix(sshd:account): account carol has expired (account expired)
sshd[3330]: fatal: Access denied for user carol by PAM account configuration [preauth]
```

What just happened: The password was correct (authentication passed), but the PAM **account** check failed. `chage -l` lists password aging and account expiry; `-E 0` set the account expiry to day 0 (January 1, 1970), so it is expired. Contractors' accounts are often set to expire this way. Do not confuse *account* expiry with *password* expiry: an expired password forces a password change at login instead.

Fix: `sudo chage -E -1 carol` (`-1` means never expires), then check `sudo chage -l carol | grep "Account expires"`.

4. **Break 3: no login shell.**

```
sudo usermod -s /usr/sbin/nologin carol
ssh carol@localhost
```

```
carol@localhost's password:
This account is currently not available.
Connection to localhost closed.
```

Investigate and fix:

```
getent passwd carol
sudo usermod -s /bin/bash carol
```

What just happened: The last field of the `passwd` entry is the login shell. `/usr/sbin/nologin` prints "This account is currently not available." and exits, which is correct for service accounts (`www-data`, `mysql`) and wrong for people. `/bin/false` does the same silently.

5. **Break 4: too many failed logins (faillock, on rocky-lab). pam_faillock** locks an account temporarily after repeated failed logins, a defense against password guessing. Rocky enables it through `authselect`.

```
# on rocky-lab
sudo authselect current
sudo authselect enable-feature with-faillock
grep -E "^#? ?(deny|unlock_time) =" /etc/security/faillock.conf
sudo useradd -m carol
echo 'carol:Lab54-Pass!' | sudo chpasswd
```

```
Profile ID: sssd
Enabled features: None
# deny = 3
# unlock_time = 600
```

What just happened: `authselect` manages the PAM files on RHEL-family systems; `enable-feature with-faillock` adds `pam_faillock` to them. The commented defaults in `/etc/security/faillock.conf` apply: lock after 3 failures (`deny`) for 600 seconds (`unlock_time`). If `authselect current` says no configuration is present, skip this step; your VM's PAM is managed differently.

From ubuntu-lab, log in as carol to rocky-lab and type a **wrong** password three times, then try once more with the **right** password:

```
ssh carol@rocky-lab
ssh carol@rocky-lab
```

```
carol@rocky-lab's password:
Permission denied, please try again.
carol@rocky-lab's password:
Permission denied, please try again.
carol@rocky-lab's password:
carol@rocky-lab: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
carol@rocky-lab's password:
Permission denied, please try again.
```

Investigate on rocky-lab:

```
# on rocky-lab
sudo faillock --user carol
sudo grep carol /var/log/secure | tail -n 3
```

```
carol:
When                Type  Source                                           Valid
2026-10-10 15:31:02 RHOST 192.168.100.10                                      V
2026-10-10 15:31:06 RHOST 192.168.100.10                                      V
2026-10-10 15:31:09 RHOST 192.168.100.10                                      V
sshd[4021]: pam_faillock(sshd:auth): Consecutive login failures for user carol account
temporarily locked
sshd[4021]: Failed password for carol from 192.168.100.10 port 50122 ssh2
```

What just happened: `faillock --user carol` lists the recorded failures and where they came from. With three valid (`V`) failures inside the interval, the account is locked, so even the correct password is rejected. The log line "account temporarily locked" is the giveaway; without it, this looks exactly like a wrong password. Many failures from one source address is also a sign of a brute-force attack (Lab 5.5 covers that).

Fix: `sudo faillock --user carol --reset` on rocky-lab, then `ssh carol@rocky-lab whoami` from ubuntu-lab works.

6. **Prevent.** When someone cannot log in, check in this order: `passwd -S` (locked?), `chage -l` (expired?), `getent passwd` (shell?), `faillock --user` (locked out?), and the auth log for the exact message. Document account expiry dates and lockout policies so the help desk knows them.

Checkpoint: carol can log in to ubuntu-lab (`ssh carol@localhost whoami`) and to rocky-lab after `faillock --reset` , and you can name the command that reveals each of the four causes.

Task 7: Scenario 6: "Permission denied (publickey)" (remote access issues)

1. **Set up key login for carol.** Create a key pair as student and install the public key in carol's account (enter carol's password once).

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/lab54_key
ssh-copy-id -i ~/.ssh/lab54_key.pub carol@localhost
ssh -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost whoami
```

```
carol
```

What just happened: `ssh-keygen -t ed25519 -N ""` makes a key pair with no passphrase (fine for a lab), `ssh-copy-id` appends the public key to `/home/carol/.ssh/authorized_keys`, and the test uses only that key (`IdentitiesOnly=yes`) and never prompts for a password (`BatchMode=yes`), so it proves key login works.

2. **Break 1: wrong permissions.** Someone "helpfully" makes carol's `authorized_keys` writable by everyone.

```
sudo chmod 666 /home/carol/.ssh/authorized_keys
ssh -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost whoami
```

```
carol@localhost: Permission denied (publickey,password).
```

3. **Investigate both sides.** The client's verbose output shows what was tried; the server's log shows why it was refused.

```
ssh -v -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost true 2>&1
| \
    grep -E "Offering|Authentications that can continue" | head -n 3
sudo journalctl -u ssh -n 10 --no-pager | grep -i -E "refused|bad ownership"
ls -la /home/carol/.ssh/
```

```
debug1: Authentications that can continue: publickey,password
debug1: Offering public key: /home/student/.ssh/lab54_key ED25519 SHA256:3bX... explicit
debug1: Authentications that can continue: publickey,password
sshd[3411]: Authentication refused: bad ownership or modes for file
/home/carol/.ssh/authorized_keys
drwx------ 2 carol carol 4096 Oct 10 15:40 .
-rw-rw-rw- 1 carol carol   99 Oct 10 15:40 authorized_keys
```

What just happened: The client offered the key and the server simply moved on ("Authentications that can continue" again). The server log gives the reason: sshd's `StrictModes yes` (the default) refuses to use an `authorized_keys` file, `.ssh` directory, or home directory that other users could write to, because anyone could add their own key. The required modes: home directory not group/world-writable, `~/.ssh` `700`, `authorized_keys` `600`, all owned by the user.

4. **Fix and verify.**

```
sudo chmod 600 /home/carol/.ssh/authorized_keys
ssh -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost whoami
```

5. **Break 2: AllowUsers.** A hardening change restricts SSH to the admin account. Keep your current session open while you do this.

> **CAUTION**
> A mistake in `AllowUsers` can lock you out of SSH. Keep this session open, include `student` in the list, and test with `sudo sshd -t` before restarting.

```
echo "AllowUsers student" | sudo tee /etc/ssh/sshd_config.d/50-lab-allowusers.conf
sudo sshd -t && sudo systemctl restart ssh
ssh -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost whoami
```

```
carol@localhost: Permission denied (publickey,password).
```

6. **Investigate.**

```
sudo journalctl -u ssh -n 10 --no-pager | grep -i "not allowed"
sudo sshd -T | grep -i -E "^allowusers|^allowgroups|^denyusers|^denygroups"
```

```
sshd[3502]: User carol from 127.0.0.1 not allowed because not listed in AllowUsers
allowusers student
```

> **What just happened:** The log names the exact rule. `sshd -T` prints the *effective* configuration after reading `sshd_config` and every file in `sshd_config.d/`, which is the quickest way to find which file set what. `AllowUsers`, `AllowGroups`, `DenyUsers`, and `DenyGroups` are checked before any password or key, so the user gets the same generic "Permission denied" either way.

7. **Fix.** Add carol (or better, use a group such as `AllowGroups sshusers` and add people to the group), test the syntax, and restart.

```
echo "AllowUsers student carol" | sudo tee /etc/ssh/sshd_config.d/50-lab-allowusers.conf
sudo sshd -t && sudo systemctl restart ssh
ssh -i ~/.ssh/lab54_key -o IdentitiesOnly=yes -o BatchMode=yes carol@localhost whoami
```

8. **Other remote access problems and their tell-tale messages:**

Message	Cause	Fix
`Connection refused` / `timed out`	sshd down, firewall, network (Lab 5.3)	`ss -tlnp`, firewall rules
`REMOTE HOST IDENTIFICATION HAS CHANGED!`	Server reinstalled or a man-in-the-middle	Verify the new key out of band, then `ssh-keygen -R host`
`Permission denied (publickey)` only	`PasswordAuthentication no` and no valid key	Install the key, check modes
Root cannot log in	`PermitRootLogin prohibit-password` or `no`	Log in as a user and use `sudo`
`no matching cipher` / `key exchange`	Algorithm mismatch (Task 12)	Update the old client or device

9. **Prevent.** Always run `sudo sshd -t` before restarting sshd, keep a second session open while changing SSH or PAM, manage access with groups, and check modes with `ls -la ~/.ssh` when keys stop working.

> **Checkpoint:** Key login for carol works again, and you saw both "bad ownership or modes" and "not listed in AllowUsers" in the sshd log.

Task 8: Scenario 7: "Are we patched?" (unpatched vulnerable systems)

An unpatched system is the most common way attackers get in. The symptoms are quiet: security updates waiting, automatic updates switched off, packages held back, or updates installed but the old vulnerable code still running because nothing was restarted.

1. **Check what is pending on ubuntu-lab.**

```
sudo apt update
apt list --upgradable 2>/dev/null | head -n 5
apt list --upgradable 2>/dev/null | grep -c -- -security
pro security-status | head -n 12
```

```
Listing...
libssl3t64/noble-updates,noble-security 3.0.13-0ubuntu3.6 amd64 [upgradable from: 3.0.13-
0ubuntu3.4]
openssl/noble-updates,noble-security 3.0.13-0ubuntu3.6 amd64 [upgradable from: 3.0.13-
0ubuntu3.4]
...
7
```

> **What just happened:** `apt list --upgradable` lists packages with a newer version available; the part after `/` shows which pockets offer it, and `-security` means it fixes a vulnerability. `pro security-status` (the Ubuntu Pro client, installed by default) summarizes how many installed packages get security coverage. Your counts will differ; a freshly updated VM may show `0`.

2. **Break 1: automatic updates switched off.** Ubuntu installs security updates automatically with `unattended-upgrades`. Simulate someone disabling it.

```
cat /etc/apt/apt.conf.d/20auto-upgrades
sudo cp /etc/apt/apt.conf.d/20auto-upgrades ~/labs/5.4/20auto-upgrades.bak
sudo sed -i 's/Unattended-Upgrade "1"/Unattended-Upgrade "0"/' /etc/apt/apt.conf.d/20auto-
upgrades
```

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

3. **Investigate** the way you would on an unfamiliar server.

```
apt-config dump | grep -E "Periodic::(Update-Package-Lists|Unattended-Upgrade)"
systemctl list-timers 'apt-daily*' --no-pager
sudo tail -n 3 /var/log/unattended-upgrades/unattended-upgrades.log
grep " upgrade " /var/log/dpkg.log | tail -n 3
```

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "0";
NEXT                         LEFT     LAST                         PASSED  UNIT
Sun 2026-10-11 06:21:44 UTC 14h left Sat 2026-10-10 06:40:01 UTC 9h ago  apt-daily-
upgrade.timer
...
```

> **What just happened:** `apt-config dump` shows the effective APT settings from every file in `/etc/apt/apt.conf.d/`. `Unattended-Upgrade "0"` means the daily job runs but installs nothing. The timers still fire, so a quick glance at `systemctl` looks healthy; the log in `/var/log/unattended-upgrades/` and `/var/log/dpkg.log` show when packages were last actually upgraded.

4. **Fix and test** with a dry run (shows what would be installed, changes nothing).

```
sudo cp ~/labs/5.4/20auto-upgrades.bak /etc/apt/apt.conf.d/20auto-upgrades
apt-config dump | grep "Periodic::Unattended-Upgrade"
sudo unattended-upgrade --dry-run -v 2>&1 | tail -n 5
```

5. **Break 2: a held package.** Holding stops a package from being upgraded, often done “temporarily” and forgotten.

```
sudo apt-mark hold openssh-server
apt-mark showhold
sudo apt-get -s upgrade | grep -A2 "kept back"
```

```
openssh-server set on hold.
openssh-server
The following packages have been kept back:
  openssh-server
```

> **What just happened:** `apt-mark showhold` lists held packages. `apt-get -s upgrade` is a simulation (`-s` changes nothing). If an update exists for the held package, it appears under "kept back"; if your VM is fully updated, nothing is listed, but the hold is still there waiting to block the next security fix. unattended-upgrades also skips held packages. On Rocky the equivalent is the `versionlock` plugin (`dnf versionlock list`) or `exclude=` lines in `/etc/dnf/dnf.conf`.

Fix: `sudo apt-mark unhold openssh-server`.

6. **Updated but still vulnerable: restarts.** A library update does not fix programs that already loaded the old library, and a kernel update needs a reboot.

```
ls /var/run/reboot-required* 2>/dev/null && cat /var/run/reboot-required.pkgs
uname -r; ls /boot/vmlinuz-*
sudo needrestart -r l
```

> **What just happened:** Ubuntu creates `/var/run/reboot-required` (and lists the reason in `.pkgs`) when an update needs a reboot. If the newest `vmlinuz` in `/boot` is newer than `uname -r`, you are running an old kernel. `needrestart -r l` (list mode) shows services still using outdated libraries and whether the kernel is outdated.

7. **Check rocky-lab the RHEL way.**

```
# on rocky-lab
sudo dnf install -y dnf-plugins-core
sudo dnf updateinfo summary
sudo dnf updateinfo list --security | head -n 5
sudo dnf needs-restarting -r; echo "exit: $?"
rpm -q --changelog openssl | grep -m 3 CVE
```

```
Updates Information Summary: available
    5 Security notice(s)
        2 Important Security notice(s)
        3 Moderate Security notice(s)
RLSA-2026:1234 Important/Sec.  openssl-1:3.2.2-6.el9_5.1.x86_64
...
Core libraries or services have been updated since boot-up:
  * kernel
Reboot is required to fully utilize these updates.
exit: 1
```

> **What just happened:** `dnf updateinfo` reads the security advisories (RLSA for Rocky, RHSA for Red Hat) published with updates. `sudo dnf upgrade --security` would install only security updates. `dnf needs-restarting -r` exits with `1` and explains when a reboot is needed (`0` means not needed). `rpm -q --changelog` shows which CVEs a package version fixed, which answers "are we patched against CVE-X?".

8. **Prevent.** Keep automatic security updates on (Ubuntu `unattended-upgrades`, Rocky `dnf-automatic`), review holds monthly, schedule reboots after kernel updates, and track patch status in monitoring.

Checkpoint: `apt-config dump | grep Unattended-Upgrade` shows `"1"`, `apt-mark showhold` prints nothing, and you know how to list security updates on both distributions.

Task 9: Scenario 8: "Why is our cache server on the network?" (exposed or misconfigured service)

An **exposed service** is a program that listens on a network interface where it should not, usually because someone changed a setting "just to make it work". Redis, a fast in-memory database often used as a cache, is a classic example: by default it listens only on the local machine, but one edited line makes it reachable from anywhere. In this task you play the administrator who audits **their own server**. Everything happens on ubuntu-lab; you never connect to the service from another machine.

1. **Install Redis and look at the safe defaults.**

```
sudo apt install -y redis-server
systemctl is-active redis-server
sudo grep -E "^(bind|protected-mode|port) " /etc/redis/redis.conf
```

```
active
bind 127.0.0.1 -::1
protected-mode yes
port 6379
```

What just happened: The package installs and starts the `redis-server` service. `bind 127.0.0.1 -::1` means "listen only on the IPv4 and IPv6 loopback addresses" (the `-` in front of `::1` means "skip it without an error if IPv6 is not available"). **Protected mode** is Redis's safety net: when it is `yes` and no password is set, Redis refuses clients that do not come from loopback. Port `6379` is the Redis default.

2. **Break it.** A developer wants an application on another host to use the cache, so they open Redis to every interface and switch the safety net off. First keep a copy of the original file.

```
sudo cp /etc/redis/redis.conf ~/labs/5.4/redis.conf.bak
sudo sed -i 's/^bind 127.0.0.1 -::1/bind 0.0.0.0/' /etc/redis/redis.conf
sudo sed -i 's/^protected-mode yes/protected-mode no/' /etc/redis/redis.conf
sudo systemctl restart redis-server
```

What just happened: `sed -i 's/old/new/'` edits the file in place, replacing the first match on each line. `bind 0.0.0.0` means "every IPv4 address this machine has", and `protected-mode no` removes the safety net. Redis also has no password by default, so now anyone who can reach port 6379 can read, change, or delete every key, and with some tricks even write files on the server. Real attackers scan the internet for exactly this.

3. **Symptom.** There is no error message. Exposed services are silent; you only find them by auditing. Start with the listening sockets.

```
sudo ss -tlnp
```

```
State  Recv-Q Send-Q Local Address:Port  Peer Address:Port Process
LISTEN 0      4096   127.0.0.53%lo:53         0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=15))
LISTEN 0      4096      127.0.0.54:53         0.0.0.0:*     users:(("systemd-
resolve",pid=612,fd=17))
LISTEN 0      511          0.0.0.0:80         0.0.0.0:*     users:(("nginx",pid=1820,fd=5))
LISTEN 0      511          0.0.0.0:6379       0.0.0.0:*     users:(("redis-
server",pid=5120,fd=6))
LISTEN 0      4096               *:22               *:*     users:(("sshd",pid=1105,fd=3),
("systemd",pid=1,fd=108))
LISTEN 0      511             [::]:80            [::]:*     users:(("nginx",pid=1820,fd=5))
```

What just happened: `ss` lists sockets: `-t` TCP only, `-l` listening sockets only, `-n` numbers instead of names, `-p` the process that owns each socket (needs `sudo`). Read the **Local Address** column: `127.0.0.1` or `127.0.0.53` means "this machine only"; `0.0.0.0` (IPv4), `[::]` (IPv6), or `*` (both) means "every interface", which is reachable from the network unless a firewall blocks it. `0.0.0.0:6379` owned by `redis-server` is the finding. SSH and nginx are meant to be reachable; a cache database is not.

4. **See what a scanner sees. nmap** is a port scanner. Scan your own server, first through loopback and then through its network address. Replace `192.168.100.10` with ubuntu-lab's real IP (`ip -4 addr` shows it).

```
sudo apt install -y nmap
sudo nmap -sT -p- localhost
sudo nmap -sT -p 22,80,6379 192.168.100.10
```

```
Nmap scan report for localhost (127.0.0.1)
Not shown: 65532 closed tcp ports (conn-refused)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
6379/tcp open  redis

Nmap scan report for ubuntu-lab (192.168.100.10)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
6379/tcp open  redis
```

What just happened: `-sT` is a TCP connect scan (it completes the normal TCP handshake for each port and then closes it), and `-p-` means all 65,535 ports instead of nmap's default top 1,000. The first scan shows everything listening on loopback. The second scan targets the server's LAN address: Redis answers there too, which proves it is bound to the network interface, not just loopback. Only scan systems you own or are authorized to test. Note that a scan of your own IP travels over the loopback interface inside the kernel, so it shows what is *bound*, not what a host firewall would let through from outside; you check the firewall separately with `ufw status`.

5. **Investigate the configuration and the firewall.**

```
sudo grep -E "^(bind|protected-mode|requirepass|port) " /etc/redis/redis.conf
sudo ufw status verbose
```

```
bind 0.0.0.0
protected-mode no
port 6379
Status: inactive
```

6. **Root cause.** Three layers failed together: Redis binds to all interfaces (`bind 0.0.0.0`), the protected-mode safety net is off, there is no password (`requirepass` is missing), and no host firewall filters incoming connections (ufw is inactive, the Ubuntu default). Any one of these layers would have limited the damage.

7. **Fix the service.** Restore loopback-only listening, turn protected mode back on, and require a password. Open the file with `sudo nano /etc/redis/redis.conf` and make sure these three lines read exactly like this (`requirepass` is commented out near the "SECURITY" section; add the line if it is not there):

   ```
   bind 127.0.0.1 -::1
   protected-mode yes
   requirepass S3cure-Lab54-Redis!
   ```

   Then restart and verify.

   ```
   sudo systemctl restart redis-server
   sudo ss -tlnp | grep 6379
   redis-cli ping
   REDISCLI_AUTH='S3cure-Lab54-Redis!' redis-cli ping
   ```

   ```
   LISTEN 0      511        127.0.0.1:6379       0.0.0.0:*     users:(("redis-
   server",pid=5301,fd=6))
   LISTEN 0      511            [::1]:6379          [::]:*     users:(("redis-
   server",pid=5301,fd=7))
   NOAUTH Authentication required.
   PONG
   ```

   What just happened: `ss` now shows Redis on `127.0.0.1` and `[::1]` only. `redis-cli ping` (run locally, on the server itself) is refused with `NOAUTH`, and the same command with the password in the `REDISCLI_AUTH` environment variable gets `PONG`, which proves the password is enforced. In production, prefer Redis **ACLs** (named users with limited commands, configured with `user` lines or an `aclfile`) over one shared password, and use a long random password. If an application on another host truly needs Redis, the safe design is to bind to one private interface, require authentication with TLS, and allow only that application server's IP in the firewall.

8. **Defense in depth: turn on the host firewall.** A host firewall would have blocked the exposure even with the bad Redis config. Allow SSH **first**, then set the default to deny incoming, then enable.

> **CAUTION**
> If you are connected to ubuntu-lab over SSH, run `sudo ufw allow OpenSSH` before `sudo ufw enable`, or you will cut off your own session. If that happens, use the VM console.

```
sudo ufw allow OpenSSH
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose
```

```
Rules updated
Rules updated (v6)
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp (OpenSSH)           ALLOW IN    Anywhere
22/tcp (OpenSSH (v6))      ALLOW IN    Anywhere (v6)
```

> **What just happened:** `ufw allow OpenSSH` uses an application profile (port 22/tcp). `default deny incoming` drops every new incoming connection that no rule allows, and `default allow outgoing` lets the server start connections (updates, DNS). `ufw enable` activates the rules now and at every boot. Only SSH is allowed in, so even if someone breaks the Redis config again, port 6379 (and port 80) are not reachable from other hosts. Loopback traffic is always allowed, so local tools keep working.

9. **Prevent.** Practice **least exposure**: services listen on loopback unless they must serve the network, every network service requires authentication, and a default-deny firewall sits in front of everything. Review listening ports regularly (`sudo ss -tlnp` and `sudo ss -ulnp` for UDP) and compare them with a documented list of expected ports; a new `0.0.0.0` line is a finding to investigate.

> **Checkpoint:** `sudo ss -tlnp | grep 6379` shows only `127.0.0.1` and `[::1]`, `redis-cli ping` returns `NOAUTH`, and `sudo ufw status` shows `Status: active` with only OpenSSH allowed.

Task 10: Scenario 9: "Your connection is not private" (certificate issues)

Certificate problems are some of the most common tickets an administrator sees. In this task you build a small private **PKI** (public key infrastructure: a root CA, an intermediate CA, and server certificates), serve the certificates with nginx on ubuntu-lab, and create and diagnose the four classic failures: untrusted, missing intermediate, hostname

mismatch, and expired.

```
Lab54 Root CA  (self-signed, in the client's trust store)
     |  signs
Lab54 Intermediate CA  (the server must SEND this one)
     |  signs
ubuntu-lab server certificate  (SAN: DNS:ubuntu-lab, valid from..to)
```

1. **Build the CA chain.** Install the tools (they are probably there from Task 1) and work in a dedicated directory.

```
sudo apt install -y openssl nginx faketime
mkdir -p ~/labs/5.4/pki && cd ~/labs/5.4/pki
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
    -keyout rootCA.key -out rootCA.crt -subj "/CN=Lab54 Root CA"
```

> **What just happened:** `openssl req` creates certificate requests; with `-x509` it creates a **self-signed certificate** instead (the issuer and subject are the same). `-newkey rsa:2048` makes a new 2048-bit RSA private key, `-nodes` ("no DES") leaves the key unencrypted so services can read it without a passphrase (protect it with file permissions instead), `-days 3650` makes it valid for ten years, and `-subj` sets the name without prompting. OpenSSL's default config marks it as a CA (`CA:TRUE`). This is your **root CA**.

2. **Create the intermediate CA.** Real CAs keep the root offline and sign server certificates with an intermediate. First write a small file of **extensions** (extra fields in the certificate) that make it a CA.

```
cat > ca-int.ext <<'EOF'
basicConstraints = critical, CA:TRUE, pathlen:0
keyUsage = critical, keyCertSign, cRLSign
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid
EOF
openssl req -new -newkey rsa:2048 -nodes \
    -keyout intCA.key -out intCA.csr -subj "/CN=Lab54 Intermediate CA"
openssl x509 -req -in intCA.csr -CA rootCA.crt -CAkey rootCA.key \
    -CAcreateserial -days 1825 -extfile ca-int.ext -out intCA.crt
```

```
Certificate request self-signature ok
subject=CN = Lab54 Intermediate CA
```

> **What just happened:** `openssl req -new` creates a **CSR** (certificate signing request: the public key plus the requested name). `openssl x509 -req` signs a CSR: `-CA` and `-CAkey` are the signer's certificate and key, `-CAcreateserial` creates a serial-number file (`rootCA.srl`) so each certificate gets a unique serial, and `-extfile` adds the extensions. `CA:TRUE, pathlen:0` means "may sign server certificates but not further CAs"; `keyCertSign` allows signing certificates.

3. **Create the server certificate**, signed by the intermediate, valid for the name `ubuntu-lab` only.

```
cat > server.ext <<'EOF'
basicConstraints = CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = DNS:ubuntu-lab
EOF
openssl req -new -newkey rsa:2048 -nodes \
    -keyout server.key -out server.csr -subj "/CN=ubuntu-lab"
openssl x509 -req -in server.csr -CA intCA.crt -CAkey intCA.key \
    -CAcreateserial -days 365 -extfile server.ext -out server.crt
openssl x509 -in server.crt -noout -subject -issuer -dates -ext subjectAltName
```

```
subject=CN = ubuntu-lab
issuer=CN = Lab54 Intermediate CA
notBefore=Oct 10 16:20:11 2026 GMT
notAfter=Oct 10 16:20:11 2027 GMT
X509v3 Subject Alternative Name:
    DNS:ubuntu-lab
```

What just happened: `openssl x509 -noout` reads a certificate without printing its encoded form; `-subject` and `-issuer` show who it is for and who signed it, `-dates` shows the validity window (`notBefore` and `notAfter`), and `-ext subjectAltName` shows the names it is valid for. Modern clients check **only the SAN** for the hostname; the CN (common name) in the subject is ignored. You will use this exact command in every scenario below.

4. **Set up an HTTPS site in nginx.** The site always serves `/etc/nginx/lab54/site.crt` and `site.key`, so each scenario only swaps those two files. Write the site file with a heredoc:

```
sudo mkdir -p /etc/nginx/lab54
cat <<'EOF' | sudo tee /etc/nginx/sites-available/lab54-tls
server {
    listen 443 ssl;
    server_name ubuntu-lab;
    ssl_certificate     /etc/nginx/lab54/site.crt;
    ssl_certificate_key /etc/nginx/lab54/site.key;
    root /var/www/html;
}
EOF
sudo ln -s /etc/nginx/sites-available/lab54-tls /etc/nginx/sites-enabled/
```

5. **Scenario (a): untrusted self-signed certificate.** Someone “quickly” made a self-signed certificate for the server.

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 \
    -keyout selfsigned.key -out selfsigned.crt -subj "/CN=ubuntu-lab" \
    -addext "subjectAltName=DNS:ubuntu-lab"
sudo cp selfsigned.crt /etc/nginx/lab54/site.crt
sudo cp selfsigned.key /etc/nginx/lab54/site.key
sudo nginx -t && sudo systemctl reload nginx
curl -sS https://ubuntu-lab/ -o /dev/null
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
curl: (60) SSL certificate problem: self-signed certificate
```

Investigate what the server sends:

```
openssl s_client -connect ubuntu-lab:443 -servername ubuntu-lab </dev/null 2>/dev/null | \
    grep -E "^depth|verify error|Verify return code"
openssl s_client -connect ubuntu-lab:443 -servername ubuntu-lab </dev/null 2>/dev/null | \
    openssl x509 -noout -subject -issuer
```

```
depth=0 CN = ubuntu-lab
verify error:num=18:self-signed certificate
Verify return code: 18 (self-signed certificate)
subject=CN = ubuntu-lab
issuer=CN = ubuntu-lab
```

What just happened: `openssl s_client` is a TLS test client. `-connect host:port` opens the connection, `-servername` sends the **SNI** (Server Name Indication: the hostname the client wants, so a server with many sites picks the right certificate), and `</dev/null` closes the connection right after the handshake. `depth=0` is the server certificate itself; error 18 means it is self-signed. Piping the output to `openssl x509` extracts the first certificate the server sent: subject and issuer are identical, so no CA vouches for it. Browsers show "Your connection is not private" for the same reason.

Root cause and fix. Nobody the client trusts signed the certificate. Serve a CA-issued certificate, and make clients trust your private root CA by adding it to the system trust store.

```
sudo cp server.crt /etc/nginx/lab54/site.crt
sudo cp server.key /etc/nginx/lab54/site.key
sudo nginx -t && sudo systemctl reload nginx
sudo cp rootCA.crt /usr/local/share/ca-certificates/lab54-root.crt
sudo update-ca-certificates
```

```
Updating certificates in /etc/ssl/certs...
1 added, 0 removed; done.
```

What just happened: On Ubuntu, `.crt` files placed in `/usr/local/share/ca-certificates/` are added to the system bundle `/etc/ssl/certs/ca-certificates.crt` by `update-ca-certificates`; curl, wget, apt, and most programs use that bundle. (Rocky: copy to `/etc/pki/ca-trust/source/anchors/` and run `sudo update-ca-trust`.) The other fix, `curl -k` or "ignore certificate errors", turns off the protection TLS exists to provide, so never make it permanent.

6. **Scenario (b): missing intermediate.** The server now has a properly issued certificate and the client trusts the root, yet:

```
curl -sS https://ubuntu-lab/ -o /dev/null
curl -sS --cacert rootCA.crt https://ubuntu-lab/ -o /dev/null
```

```
curl: (60) SSL certificate problem: unable to get local issuer certificate
curl: (60) SSL certificate problem: unable to get local issuer certificate
```

Investigate: look at the whole chain the server sends with `-showcerts`, and test the chain offline with `openssl verify`.

```
openssl s_client -connect ubuntu-lab:443 -servername ubuntu-lab -showcerts \
    -CAfile rootCA.crt </dev/null 2>/dev/null | grep -E "^ *[0-9] s:|i:|Verify return"
openssl verify -CAfile rootCA.crt server.crt
openssl verify -CAfile rootCA.crt -untrusted intCA.crt server.crt
```

```
 0 s:CN = ubuntu-lab
   i:CN = Lab54 Intermediate CA
Verify return code: 21 (unable to verify the first certificate)
CN = ubuntu-lab
error 20 at 0 depth lookup: unable to get local issuer certificate
error server.crt: verification failed
server.crt: OK
```

What just happened: `-showcerts` prints every certificate the server sends, numbered from `0`; `s:` is the subject and `i:` the issuer. The server sends only certificate `0`, issued by "Lab54 Intermediate CA", but the client only has the root, so it cannot connect the two: there is a missing link. `-CAfile` tells `s_client` which roots to trust. `openssl verify -CAfile root` fails the same way (error 20), and adding the intermediate with `-untrusted` (meaning "an intermediate you may use to build the chain, but not trust as a root") makes it `OK`. That proves the certificate is fine and only the **served chain** is incomplete. Browsers sometimes hide this problem because they cache intermediates from other sites, which is why "it works in my browser but not in curl or the app" is the classic symptom.

Root cause and fix. nginx must send the server certificate followed by the intermediate in one file (a **full chain**). Order matters: server first, then the intermediate. The root is not needed.

```
cat server.crt intCA.crt > fullchain.crt
sudo cp fullchain.crt /etc/nginx/lab54/site.crt
sudo nginx -t && sudo systemctl reload nginx
curl -sS https://ubuntu-lab/ -o /dev/null -w '%{http_code}\n'
```

```
200
```

(Apache uses the same idea: a full-chain file in `SSLCertificateFile`.) Running the `-showcerts` command again now lists certificates `0` and `1` and `Verify return code: 0 (ok)`.

7. **Scenario (c): hostname mismatch.** A user connects by a different name than the certificate covers.

```
curl -sS https://localhost/ -o /dev/null
openssl s_client -connect localhost:443 -servername localhost \
    -verify_hostname localhost </dev/null 2>/dev/null | grep "Verify return"
openssl x509 -in /etc/nginx/lab54/site.crt -noout -ext subjectAltName
```

```
curl: (60) SSL: no alternative certificate subject name matches target host name 'localhost'
Verify return code: 62 (hostname mismatch)
    DNS:ubuntu-lab
```

What just happened: The chain is valid, but the certificate's SAN lists only `ubuntu-lab`, and the client asked for `localhost`. `openssl s_client` checks the chain but **not the hostname** unless you add `-verify_hostname`, so without it a mismatch can look like `Verify return code: 0 (ok)`. `openssl x509 -in file` reads the first certificate in the full-chain file, which is the server's. The same error appears when users browse to a server by IP address or by a short name that is not in the SAN.

Root cause and fix. The certificate does not include every name clients use. Reissue it with all of them in the SAN (do not tell users to ignore the warning).

```
sed -i 's/^subjectAltName.*/subjectAltName = DNS:ubuntu-lab, DNS:localhost, IP:127.0.0.1/' \
    server.ext
openssl x509 -req -in server.csr -CA intCA.crt -CAkey intCA.key \
    -CAcreateserial -days 365 -extfile server.ext -out server.crt
cat server.crt intCA.crt > fullchain.crt
sudo cp fullchain.crt /etc/nginx/lab54/site.crt
sudo systemctl reload nginx
curl -sS https://localhost/ -o /dev/null -w '%{http_code}\n'
```

```
Certificate request self-signature ok
subject=CN = ubuntu-lab
200
```

8. **Scenario (d): expired certificate.** To create a certificate that is already expired, you need it to be valid in the past. On Ubuntu 24.04 (OpenSSL 3.0) `openssl x509` has no option for a start date, so use the `faketime` package, which makes a single command believe it is a different date.

```
faketime '2023-01-01' openssl x509 -req -in server.csr -CA intCA.crt -CAkey intCA.key \
    -CAcreateserial -days 30 -extfile server.ext -out expired.crt
cat expired.crt intCA.crt > expired-chain.crt
sudo cp expired-chain.crt /etc/nginx/lab54/site.crt
sudo systemctl reload nginx
curl -sS https://ubuntu-lab/ -o /dev/null
```

```
Certificate request self-signature ok
subject=CN = ubuntu-lab
curl: (60) SSL certificate problem: certificate has expired
```

What just happened: `faketime '2023-01-01' command` runs one command with a fake clock (it preloads a small library that changes the time functions programs call). OpenSSL therefore stamped the certificate as valid from January 1, 2023 for 30 days. The system clock is not changed. Expired certificates are the most common certificate outage in real life: someone forgot to renew.

Investigate the dates of what the server is actually serving:

```
openssl s_client -connect ubuntu-lab:443 -servername ubuntu-lab </dev/null 2>/dev/null | \
    openssl x509 -noout -subject -issuer -dates -ext subjectAltName
curl -v https://ubuntu-lab/ -o /dev/null 2>&1 | grep -E "expire|problem"
date -u
```

```
subject=CN = ubuntu-lab
issuer=CN = Lab54 Intermediate CA
notBefore=Jan  1 00:00:00 2023 GMT
notAfter=Jan 31 00:00:00 2023 GMT
X509v3 Subject Alternative Name:
    DNS:ubuntu-lab, DNS:localhost, IP Address:127.0.0.1
* TLSv1.3 (OUT), TLS alert, certificate expired (557):
* SSL certificate problem: certificate has expired
Sat Oct 10 16:41:02 UTC 2026
```

What just happened: `notAfter` is in the past compared with `date -u`, so the certificate is expired. `curl -v` (verbose) prints the TLS handshake steps and the certificate details when it succeeds, and the exact reason when it fails. Always check the client's clock too: a client whose clock is years off reports valid certificates as expired or "not yet valid". On Rocky, `timedatectl` and `chronyc tracking` show whether the clock is synchronized.

Root cause and fix. The certificate's validity period ended. Install a current certificate (here, the one from scenario (c)) and reload.

```
sudo cp fullchain.crt /etc/nginx/lab54/site.crt
sudo systemctl reload nginx
curl -sS https://ubuntu-lab/ -o /dev/null -w '%{http_code}\n'
openssl x509 -in fullchain.crt -noout -checkend 2592000 && echo "OK for 30+ days"
```

```
200
Certificate will not expire
OK for 30+ days
```

What just happened: `-checkend 2592000` asks "will this certificate still be valid in 2,592,000 seconds (30 days)?" and exits `0` if yes, `1` if no, which makes it perfect for a monitoring script or cron job.

9. **Prevent.** Automate renewal (for public sites, an ACME client such as `certbot` renews Let's Encrypt certificates automatically), monitor expiry with `openssl x509 -checkend`, always deploy the full chain, put every name and IP clients use in the SAN, and test with `openssl s_client -verify_hostname` and `curl` (not only a browser) after every certificate change.

Checkpoint: `curl -sS https://ubuntu-lab/ -o /dev/null -w '%{http_code}\n'` prints `200` without `-k`, and you can name the error text for each of the four scenarios: "self-signed certificate", "unable to get local issuer certificate", "no alternative certificate subject name matches", and "certificate has expired".

Task 11: Scenario 10: "apt update fails" (misconfigured package repository)

A **repository** is a server that holds packages plus signed index files. The package manager downloads the index, checks its digital signature against a trusted key, and only then believes the package list. Two things go wrong constantly: the URL is wrong, or the key is wrong. Both block updates, which quietly turns into an unpatched system (Task 8).

1. **Break 1 on ubuntu-lab: a typo in the URL.** Ubuntu 24.04 uses **deb822** source files: `/etc/apt/sources.list.d/*.sources`, with one `Field: value` per line. Create one with a typo (`ubuntuu`):

```
cat <<'EOF' | sudo tee /etc/apt/sources.list.d/lab54-typo.sources
Types: deb
URIs: http://archive.ubuntu.com/ubuntuu
Suites: noble
Components: main
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
EOF
```

   What just happened: `Types: deb` means binary packages, `URIs` is the repository's base URL, `Suites` is the release (`noble` is the codename for 24.04), `Components` lists sections, and `Signed-By` names the only keyring whose keys may sign this repository. Compare it with the real file, `/etc/apt/sources.list.d/ubuntu.sources`.

2. **Break 2: the wrong signing key.** Add Ubuntu's debug-symbol repository (`ddebs.ubuntu.com`, a real repository signed with a different key) but point `Signed-By` at the normal archive keyring.

```
cat <<'EOF' | sudo tee /etc/apt/sources.list.d/lab54-ddebs.sources
Types: deb
URIs: http://ddebs.ubuntu.com
Suites: noble
Components: main
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
EOF
```

3. **Symptom.**

```
sudo apt update
```

```
Ign:5 http://archive.ubuntu.com/ubuntuu noble InRelease
Err:6 http://ddebs.ubuntu.com noble InRelease
  The following signatures couldn't be verified because the public key is not available:
NO_PUBKEY C8CAB6595FDFF622
Err:7 http://archive.ubuntu.com/ubuntuu noble Release
  404  Not Found [IP: 91.189.91.83 80]
Reading package lists... Done
W: GPG error: http://ddebs.ubuntu.com noble InRelease: The following signatures couldn't be
verified because the public key is not available: NO_PUBKEY C8CAB6595FDFF622
E: The repository 'http://ddebs.ubuntu.com noble InRelease' is not signed.
E: The repository 'http://archive.ubuntu.com/ubuntuu noble Release' does not have a Release
file.
N: Updating from such a repository can't be done securely, and is therefore disabled by
default.
```

What just happened: Read every `Err:` and `E:` line; each names the repository URL. **"404 Not Found" plus "does not have a Release file"** means apt reached the server but the path does not exist: a typo, a wrong suite name (for example `jammy` content on a `noble` system, or a release the vendor never published), or a repository that moved. **"NO_PUBKEY C8CAB6595FDFF622" plus "is not signed"** means the index was signed by a key that is not in the keyring named by `Signed-By`. apt then refuses the repository instead of trusting unsigned data, which is exactly the right behavior.

4. **Investigate.** Find which files define the failing repositories, check what apt actually uses, and compare the key IDs.

```
grep -rl -e ubuntuu -e ddebs /etc/apt/sources.list /etc/apt/sources.list.d/
apt-cache policy | grep -E "ddebs|ubuntuu"; echo "exit: $?"
curl -sI http://archive.ubuntu.com/ubuntuu/dists/noble/Release | head -n 1
curl -sI http://archive.ubuntu.com/ubuntu/dists/noble/Release | head -n 1
gpg --show-keys /usr/share/keyrings/ubuntu-archive-keyring.gpg | grep -E "^ +[0-9A-F]{40}"
```

```
/etc/apt/sources.list.d/lab54-ddebs.sources
/etc/apt/sources.list.d/lab54-typo.sources
exit: 1
HTTP/1.1 404 Not Found
HTTP/1.1 200 OK
790BC7277767219C42C86F933B4FE6ACC0B21F32
843938DF228D22F7B3742BC0D94AA3F0EFE21092
F6ECB3762474EDA9D21B7022871920D1991BC93C
```

What just happened: `grep -rl` lists the files that contain the bad names. `apt-cache policy` shows every repository apt is actually using, with its priority; the two broken ones are absent (exit `1` from `grep`), so their packages are invisible. `curl -sI` fetches only the HTTP headers: the typo'd path returns `404` while the correct path returns `200`, which confirms the URL error. `gpg --show-keys` lists the fingerprints in a keyring without importing anything; the last 16 hex digits of a fingerprint are its key ID. `C8CAB6595FDFF622` is not among them, so the wrong keyring is named.

5. **Root cause and fix.** The first file has a wrong URL; the second names the wrong keyring. Fix the URL, and install the correct keyring package for the debug repository and point `Signed-By` at it.

```
sudo sed -i 's#/ubuntuu$#/ubuntu#' /etc/apt/sources.list.d/lab54-typo.sources
sudo apt install -y ubuntu-dbgsym-keyring
sudo sed -i 's#ubuntu-archive-keyring.gpg#ubuntu-dbgsym-keyring.gpg#' \
    /etc/apt/sources.list.d/lab54-ddebs.sources
sudo apt update 2>&1 | grep -E "^(Err|E:|W:)"
apt-cache policy | grep -A1 ddebs | head -n 2
```

```
W: Target Packages (main/binary-amd64/Packages) is configured multiple times in
/etc/apt/sources.list.d/lab54-typo.sources:1 and /etc/apt/sources.list.d/ubuntu.sources:1
W: Target Packages (main/binary-all/Packages) is configured multiple times in
/etc/apt/sources.list.d/lab54-typo.sources:1 and /etc/apt/sources.list.d/ubuntu.sources:1
...
 500 http://ddebs.ubuntu.com noble/main amd64 Packages
     release o=Ubuntu,a=noble,n=noble,l=Ubuntu,c=main,b=amd64
```

What just happened: `sed` with `#` as the separator avoids escaping the slashes in the URL. The `ubuntu-dbgsym-keyring` package puts the right public key in `/usr/share/keyrings/ubuntu-dbgsym-keyring.gpg`, so the `Err` and `E:` lines are gone and `apt-cache policy` now lists the debug repository with the default priority `500`. The new `W:` warnings reveal a third, very common misconfiguration: with the typo fixed, `lab54-typo.sources` now defines exactly the same repository as the stock `ubuntu.sources` ("configured multiple times"). The correct fix for that file is to delete it. For a third-party repository, the right way to fix a key is to download the vendor's key from their official HTTPS site, check its fingerprint against their documentation, store it in `/etc/apt/keyrings/`, and reference it with `Signed-By`.

6. **Remove the lab repositories** (the duplicate, and the debug repository you do not need) and confirm apt is clean.

```
sudo rm /etc/apt/sources.list.d/lab54-typo.sources /etc/apt/sources.list.d/lab54-
ddebs.sources
sudo apt purge -y ubuntu-dbgsym-keyring
sudo apt update 2>&1 | grep -E "^(Err|E:|W:)"; echo "exit: $?"
```

```
exit: 1
```

`grep` found no error or warning lines (exit `1`), so every repository is healthy again.

7. **Break 3 on rocky-lab: a wrong baseurl.** dnf repositories are `.repo` files in `/etc/yum.repos.d/`, in INI format. Create one with a typo (`AppStreem`):

```
# on rocky-lab
cat <<'EOF' | sudo tee /etc/yum.repos.d/lab54-typo.repo
[lab54-typo]
name=Lab 5.4 typo repo
baseurl=https://dl.rockylinux.org/pub/rocky/9/AppStreem/x86_64/os/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
EOF
sudo dnf makecache
```

```
Lab 5.4 typo repo                                       1.2 kB/s | 196  B     00:00
Errors during downloading metadata for repository 'lab54-typo':
  - Status code: 404 for
https://dl.rockylinux.org/pub/rocky/9/AppStreem/x86_64/os/repodata/repomd.xml (IP:
151.101.2.132)
Error: Failed to download metadata for repo 'lab54-typo': Cannot download repomd.xml: Cannot
download repodata/repomd.xml: All mirrors were tried
```

What just happened: The section name in `[brackets]` is the repo ID; `baseurl` is where the metadata lives, `enabled=1` turns it on, `gpgcheck=1` makes dnf verify every package signature, and `gpgkey` names the key to verify with. `dnf makecache` downloads the metadata for all enabled repositories. dnf first fetches `repodata/repomd.xml` (the index of the metadata) under the `baseurl`; a `404` there means the URL is wrong. Because one repository failed, the whole command fails, so this one typo also blocks `dnf upgrade`.

8. **Investigate and fix.**

```
# on rocky-lab
dnf repolist -v 2>/dev/null | grep -A8 "Repo-id *: lab54-typo"
grep -E "^baseurl" /etc/yum.repos.d/rocky.repo /etc/yum.repos.d/lab54-typo.repo
sudo sed -i 's#/AppStreem/#/AppStream/#' /etc/yum.repos.d/lab54-typo.repo
sudo dnf makecache
```

What just happened: `dnf repolist -v` (verbose) prints each repository's ID, name, base URL, and metadata status, so you can see the exact URL dnf uses. In the stock file `rocky.repo` the URLs are commented out (`#baseurl`) because Rocky uses a `mirrorlist`, which is why the `grep` only matches the lab file; reading the commented lines still shows the correct spelling. After the `sed` correction, `makecache` succeeds. As a temporary workaround while a vendor repository is down, `skip_if_unavailable=True` in the `.repo` file lets dnf continue without it; never use that to hide a typo.

9. **Break 4: a GPG check failure.** Add the EPEL repository (Extra Packages for Enterprise Linux, which provides `htop`) but with the **Rocky** key instead of EPEL's key.

```
# on rocky-lab
cat <<'EOF' | sudo tee /etc/yum.repos.d/lab54-epel.repo
[lab54-epel]
name=Lab 5.4 EPEL with the wrong key
baseurl=https://dl.fedoraproject.org/pub/epel/9/Everything/x86_64/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
EOF
sudo dnf install -y htop
```

```
warning: /var/cache/dnf/lab54-epel-.../packages/htop-3.3.0-4.el9.x86_64.rpm: Header V4
RSA/SHA256 Signature, key ID 3228467c: NOKEY
Lab 5.4 EPEL with the wrong key                          1.6 MB/s | 1.6 kB     00:00
GPG key at file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9 (0x350D275D) is already installed
The GPG keys listed for the "Lab 5.4 EPEL with the wrong key" repository are already
installed but they are not correct for this package.
Check that the correct key URLs are configured for this repository.. Failing package is:
htop-3.3.0-4.el9.x86_64
 GPG Keys are configured as: file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
Public key for htop-3.3.0-4.el9.x86_64.rpm is not installed. Failing package is: htop-3.3.0-
4.el9.x86_64
 GPG Keys are configured as: file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
The downloaded packages were saved in cache until the next successful transaction.
You can remove cached packages by executing 'dnf clean packages'.
Error: GPG check FAILED
```

What just happened: The package is signed with key ID `3228467c` (EPEL 9's key), shown as `NOKEY` because that key is not imported into the RPM database. The repository's `gpgkey` points at the Rocky key (ID `350D275D`), which cannot verify it, so dnf stops with "GPG check FAILED". Your version numbers may differ.

10. **Investigate and fix the key, never the check.**

```
# on rocky-lab
rpm -qa gpg-pubkey --qf '%{VERSION}\t%{SUMMARY}\n'
sudo sed -i 's#^gpgkey=.*#gpgkey=https://dl.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-9#'
\
    /etc/yum.repos.d/lab54-epel.repo
sudo dnf install -y htop
```

```
350d275d    Rocky Enterprise Software Foundation - Release key 2022 <releng@rockylinux.org>
public key
...
Importing GPG key 0x3228467C:
 Userid     : "Fedora (epel9) <epel@fedoraproject.org>"
 Fingerprint: FF8A D134 4597 106E CE81 3B91 8A38 72BF 3228 467C
 From       : https://dl.fedoraproject.org/pub/epel/RPM-GPG-KEY-EPEL-9
Key imported successfully
...
Complete!
```

What just happened: `rpm -qa gpg-pubkey` lists the public keys RPM trusts (each key is stored as a pseudo-package named `gpg-pubkey-<keyid>-<date>`); `--qf` chooses which fields to print. With the correct `gpgkey` URL, dnf shows the new key's fingerprint, imports it (with `-y` it accepts automatically; interactively you would compare the fingerprint with the one published on the EPEL website before typing `y`), and the signature check passes. In real life the normal way to add EPEL on Rocky is simply `sudo dnf install epel-release`, which ships the correct repo file and key.

CAUTION

Never "fix" a signature error with `gpgcheck=0`, `--nogpgcheck`, `trusted=yes` (apt), or `[allow-insecure=yes]`. Signatures are what stop a hacked mirror or a man-in-the-middle from feeding you malicious packages. A signature error means: find and install the right key, or do not use the repository.

11. **Prevent.** Copy repository definitions from the vendor's official documentation, keep one file per repository so mistakes are easy to find, store keys with `Signed-By` (apt) or `gpgkey` (dnf) per repository instead of trusting a key for everything, and watch for `apt update` / `dnf makecache` errors in monitoring or the output of your automatic update jobs.

Checkpoint: On ubuntu-lab, `sudo apt update` finishes without `E:` or `W:` lines and `ls /etc/apt/sources.list.d/` shows no `lab54` files. On rocky-lab, `sudo dnf makecache` succeeds and `rpm -q htop` shows the installed package.

Task 12: Scenario 11: Old protocols and "no matching cipher" (obsolete and insecure protocols and ciphers)

Obsolete protocols such as SSLv3, TLS 1.0, and TLS 1.1 and **insecure** ciphers such as RC4, 3DES, CBC-mode SSH ciphers, and SHA-1-based key exchange have known weaknesses. Leaving them enabled is a security finding (and fails compliance standards such as PCI DSS). Removing them can then cause **cipher negotiation** failures with old clients. You handle both sides here.

How negotiation works: the client sends its list of supported protocol versions and algorithms; the server picks the best one that appears in *both* lists. If there is no overlap, the handshake fails before any login. So "no matching cipher" is never fixed by guessing; you compare the two lists.

1. **Break it on ubuntu-lab: enable TLS 1.0 and 1.1.** An admin wants an ancient monitoring tool to connect, so they add old protocols to the Task 10 site. On Ubuntu 24.04 OpenSSL's default **security level** (2) already refuses TLS 1.0/1.1 even if nginx lists them, so the "make it work" change also lowers the security level. Open the site file with `sudo nano /etc/nginx/sites-available/lab54-tls` and add these two lines inside the `server { }` block, under `ssl_certificate_key`:

    ```
        ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3;
        ssl_ciphers DEFAULT:@SECLEVEL=0;
    ```

    Then test and reload:

    ```
    sudo nginx -t && sudo systemctl reload nginx
    ```

> **What just happened:** `ssl_protocols` lists the TLS versions nginx accepts. `ssl_ciphers` sets the OpenSSL cipher string; `DEFAULT` is OpenSSL's default list, and `@SECLEVEL=0` drops the security level to 0, which allows old protocols, SHA-1, and small keys. Note that Ubuntu's main `/etc/nginx/nginx.conf` itself contains `ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3;` in its `http` block; a server-level `ssl_protocols` overrides it. Check with `grep -rn ssl_protocols /etc/nginx/`.

2. **Detect it the way an auditor would.** nmap's `ssl-enum-ciphers` script performs a handshake for every protocol version and cipher it knows and lists what the server accepts.

```
nmap --script ssl-enum-ciphers -p 443 localhost
```

```
PORT    STATE SERVICE
443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.0:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (secp256r1) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (secp256r1) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
|   TLSv1.1:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (secp256r1) - A
|       ...
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (secp256r1) - A
|       ...
|   TLSv1.3:
|     ciphers:
|       TLS_AKE_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       ...
|_  least strength: A
```

> **What just happened:** nmap implements TLS itself in its script engine, so it can test versions your local OpenSSL refuses to use. The headings `TLSv1.0:` and `TLSv1.1:` are the finding: the server accepts obsolete protocol versions. The letter grades rate each cipher's key strength, not the protocol version, so an `A` does not mean "safe". Scan the server's IP from another host you are authorized to test for the external view; the loopback result here is the same because nothing filters it.

3. **Try it with OpenSSL, and understand the client-side refusal.**

```
openssl s_client -connect localhost:443 -tls1 </dev/null 2>&1 | grep -E "error|Protocol  :"
openssl s_client -connect localhost:443 -tls1 -cipher 'DEFAULT:@SECLEVEL=0' </dev/null 2>&1
| \
    grep -E "error|Protocol  :|Cipher    :"
```

```
40C7A1E2C17F0000:error:0A0000BF:SSL routines:tls_setup_handshake:no protocols
available:../ssl/statem/statem_lib.c:104:
    Protocol  : TLSv1
    Cipher    : ECDHE-RSA-AES256-SHA
```

What just happened: `-tls1` forces TLS 1.0. The first attempt fails *before anything is sent*: "no protocols available" comes from your **client's** OpenSSL, which at security level 2 will not speak TLS 1.0. Do not mistake that for proof that the server is safe. Lowering the client's level with `-cipher 'DEFAULT:@SECLEVEL=0'` lets the client try, and the server accepts: `Protocol : TLSv1`. That confirms the server-side weakness. (Use `-tls1_1`, `-tls1_2`, `-tls1_3` to test the other versions.)

4. **Root cause and fix.** The server allows obsolete protocols and a lowered security level. Allow only TLS 1.2 and 1.3 and remove the `ssl_ciphers` override so the secure defaults apply. Edit the site file again so that the two lines become one:

```
    ssl_protocols TLSv1.2 TLSv1.3;
```

```
sudo nginx -t && sudo systemctl reload nginx
nmap --script ssl-enum-ciphers -p 443 localhost | grep -E "TLSv1\.[0-9]:"
openssl s_client -connect localhost:443 -tls1 -cipher 'DEFAULT:@SECLEVEL=0' </dev/null 2>&1
| \
    grep -o "alert protocol version"
```

```
|   TLSv1.2:
|   TLSv1.3:
alert protocol version
```

What just happened: Only TLS 1.2 and 1.3 remain. Now the server itself refuses the TLS 1.0 attempt with a `protocol version` alert (TLS alert 70). This is how a negotiation failure looks from the client side when the server is strict; an old client would report a similar "unsupported protocol" or "handshake failure" error. The right fix for that old client is to upgrade or replace it, not to re-enable TLS 1.0.

5. **SSH cipher negotiation: break it from the client side.** Simulate an old SSH client (or a script written for an old device) that insists on algorithms modern OpenSSH no longer offers.

```
ssh -c aes128-cbc student@localhost
ssh -o KexAlgorithms=diffie-hellman-group1-sha1 student@rocky-lab
```

```
Unable to negotiate with 127.0.0.1 port 22: no matching cipher found. Their offer: chacha20-
poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-gcm@openssh.com,aes256-
gcm@openssh.com
Unable to negotiate with 192.168.100.20 port 22: no matching key exchange method found.
Their offer: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,...
```

What just happened: `-c` forces the encryption cipher and `-o KexAlgorithms=` forces the **key exchange** method (how the two sides agree on a session key). `aes128-cbc` is a CBC-mode cipher and `diffie-hellman-group1-sha1` uses a 1024-bit group with SHA-1; both are considered weak and are disabled by default on modern servers. The error appears before any password prompt and prints "Their offer", the server's list. The overlap with the client's list is empty, so negotiation fails.

6. **Investigate both sides.**

```
ssh -vv -c aes128-cbc student@localhost 2>&1 | grep -E "ciphers ctos|peer server KEXINIT"
ssh -Q cipher
ssh -Q kex | head -n 4
sudo sshd -T | grep -Ei '^(ciphers|kexalgorithms|macs)'
sudo journalctl -u ssh -n 20 --no-pager | grep -i "negotiate"
```

```
debug2: ciphers ctos: aes128-cbc
debug2: peer server KEXINIT proposal
debug2: ciphers ctos: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-
gcm@openssh.com,aes256-gcm@openssh.com
3des-cbc
aes128-cbc
aes192-cbc
aes256-cbc
aes128-ctr
...
diffie-hellman-group1-sha1
diffie-hellman-group14-sha1
diffie-hellman-group14-sha256
diffie-hellman-group16-sha512
ciphers chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-ctr,aes128-
gcm@openssh.com,aes256-gcm@openssh.com
macs umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,...
kexalgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-
sha256@libssh.org,...
sshd[5532]: Unable to negotiate with 127.0.0.1 port 51344: no matching cipher found. Their
offer: aes128-cbc [preauth]
```

What just happened: `ssh -vv` (very verbose) prints the client's proposal (`ciphers ctos` , client-to-server, contains only `aes128-cbc`) and then the server's proposal (`peer server KEXINIT proposal`). `ssh -Q cipher` and `ssh -Q kex` list every algorithm the client *program* supports, not what it offers by default; old algorithms are still compiled in so you can talk to legacy devices on purpose. `sshd -T` prints the server's effective `Ciphers` , `KexAlgorithms` , and `MACs` (message authentication codes, which protect integrity). The server log records the failed negotiation with the client's offer, so you can see which clients are outdated.

7. **Decide on the right fix.**

- **Right fix:** upgrade or reconfigure the client (a current OpenSSH, PuTTY, or network-device firmware supports modern algorithms). Do not weaken the server for everyone.

- **Temporary, documented exception:** if a legacy device cannot be upgraded yet, enable the old algorithm **only on the client, only for that host**, in `~/.ssh/config`, and record a ticket with an end date. For example (you do not need to add this):

  ```
  Host old-switch
      HostName 192.168.100.50
      KexAlgorithms +diffie-hellman-group14-sha1
      HostKeyAlgorithms +ssh-rsa
  ```

  The `+` adds an algorithm to the default list instead of replacing it. Never add old algorithms globally in `/etc/ssh/sshd_config` "just in case".

8. **Rocky's system-wide crypto policy.** RHEL-family systems set allowed protocols and algorithms for OpenSSL, OpenSSH, GnuTLS, NSS, and others in one place.

   ```
   # on rocky-lab
   update-crypto-policies --show
   ls /usr/share/crypto-policies/policies/
   grep -i "^ciphers" /etc/crypto-policies/back-ends/openssh.config
   sudo sshd -T | grep -Ei '^(ciphers|kexalgorithms)'
   ```

   ```
   DEFAULT
   DEFAULT.pol  EMPTY.pol  FIPS.pol  FUTURE.pol  LEGACY.pol  modules
   Ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com,aes256-ctr,aes128-
   gcm@openssh.com,aes128-ctr
   ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com,aes256-ctr,aes128-
   gcm@openssh.com,aes128-ctr
   kexalgorithms curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,...
   ```

   What just happened: `update-crypto-policies --show` prints the active policy. **DEFAULT** is secure for today; **FUTURE** is stricter; **FIPS** limits algorithms to FIPS 140 approved ones; **LEGACY** re-enables older algorithms (such as SHA-1 signatures, smaller keys, and CBC-mode SSH ciphers) for compatibility. The generated per-program settings live in `/etc/crypto-policies/back-ends/`, which is why the sshd cipher list matches the policy file. Running `sudo update-crypto-policies --set LEGACY` (followed by a reboot or service restarts) would weaken every service on the machine at once, so **do not run it**. If one old algorithm is truly required, a narrow **subpolicy** such as `sudo update-crypto-policies --set DEFAULT:SHA1` is less harmful, and it must be documented and removed later.

9. **Prevent.** Scan your TLS services regularly with `nmap --script ssl-enum-ciphers`, allow only TLS 1.2 and 1.3, keep SSH on the distribution defaults (or the crypto policy on Rocky), watch sshd logs for "Unable to negotiate" to find outdated clients, and replace legacy clients instead of weakening servers.

Checkpoint: `nmap --script ssl-enum-ciphers -p 443 localhost` lists only `TLSv1.2` and `TLSv1.3`, you can explain the "no matching cipher found" message using `ssh -vv` and `sshd -T`, and `update-crypto-policies --show` on rocky-lab still prints `DEFAULT`.

Challenge

CHALLENGE

1. Write a short audit script, `~/labs/5.4/audit.sh`, that prints every TCP and UDP socket listening on a non-loopback address (`ss -tulnH` plus `grep -v`), the ufw status, any certificate under `/etc/nginx/lab54/` that expires within 30 days (`openssl x509 -checkend`), and whether nginx accepts TLS 1.0 (`nmap --script ssl-enum-ciphers`). Run it before and after re-breaking one of the scenarios.
2. On rocky-lab, serve a page from `/srv/web` with Apache on port 8443 over HTTPS using a certificate from your lab CA (copy the files over with `scp`). Make it work with SELinux enforcing, the firewall open for that port only, the full chain served, and the root CA added to rocky-lab's trust store with `update-ca-trust`. Verify with `curl https://rocky-lab:8443/` from both VMs without `-k`.
3. Have a partner (or a script using `shuf`) secretly apply one break from Tasks 2–12. Diagnose it using only logs and read-only commands, write down the first command that revealed the cause, then fix it without weakening security.

Clean up

1. **ubuntu-lab: remove the Redis service and the firewall rules** from Task 9. `ufw --force reset` disables ufw and deletes all its rules, returning it to the Ubuntu default (inactive).

```
sudo apt purge -y redis-server redis-tools
sudo ufw --force reset
sudo ufw status
```

2. **ubuntu-lab: remove the TLS site, certificates, and the trusted lab root CA** from Tasks 10 and 12.

```
sudo rm -f /etc/nginx/sites-enabled/lab54-tls /etc/nginx/sites-available/lab54-tls
sudo rm -rf /etc/nginx/lab54
sudo rm -f /usr/local/share/ca-certificates/lab54-root.crt
sudo update-ca-certificates --fresh
sudo apt purge -y nginx nginx-common
```

   `update-ca-certificates --fresh` rebuilds `/etc/ssl/certs` from scratch, so the lab root disappears from the trust store. Purging nginx removes the service and its configuration files.

3. **ubuntu-lab: make sure no lab repository files remain** (Task 11).

```
sudo rm -f /etc/apt/sources.list.d/lab54-*.sources
sudo apt purge -y ubuntu-dbgsym-keyring
sudo apt update
```

4. **ubuntu-lab: undo the SSH, update, and file changes** from Tasks 5, 7, and 8.

```
sudo rm -f /etc/ssh/sshd_config.d/50-lab-allowusers.conf
sudo sshd -t && sudo systemctl restart ssh
sudo apt-mark unhold openssh-server
sudo cp ~/labs/5.4/20auto-upgrades.bak /etc/apt/apt.conf.d/20auto-upgrades
apt-config dump | grep "Periodic::Unattended-Upgrade"
sudo chattr -a /srv/finance/audit.log 2>/dev/null
sudo rm -rf /srv/finance
rm -f ~/.ssh/lab54_key ~/.ssh/lab54_key.pub
```

5. **ubuntu-lab: delete the lab users, the optional tools, and the lab directory.**

```
sudo userdel -r bob
sudo userdel -r carol
sudo apt purge -y faketime nmap
sudo apt autoremove -y
rm -rf ~/labs/5.4
```

   (`acl` and `openssl` are standard Ubuntu packages; leave them installed. `userdel -r` may print "mail spool not found", which is harmless.)

6. **rocky-lab: remove the lab repositories, htop, and the EPEL key** from Task 11. Find the EPEL key's exact package name first; it looks like `gpg-pubkey-3228467c-<date>`.

```
# on rocky-lab
sudo rm -f /etc/yum.repos.d/lab54-typo.repo /etc/yum.repos.d/lab54-epel.repo
sudo dnf remove -y htop
rpm -qa gpg-pubkey --qf '%{NAME}-%{VERSION}-%{RELEASE}\t%{SUMMARY}\n'
sudo rpm -e gpg-pubkey-3228467c
sudo dnf clean all
sudo dnf makecache
```

7. **rocky-lab: undo the SELinux, Apache, and account changes** from Tasks 1–6.

```
# on rocky-lab
pkill -f "http.server 5000"
sudo setsebool -P httpd_can_network_connect off
sudo semanage port -d -t http_port_t -p tcp 8090
sudo rm -f /etc/httpd/conf.d/lab-proxy.conf /etc/httpd/conf.d/lab-port.conf
sudo systemctl disable --now httpd
sudo rm -f /var/www/html/index.html /var/www/html/report.html
sudo dnf remove -y httpd
sudo faillock --user carol --reset
sudo userdel -r carol
sudo authselect disable-feature with-faillock
rm -rf /tmp/backend ~/labs/5.4
```

8. **rocky-lab: confirm the crypto policy was never changed** (Task 12). It must print `DEFAULT`; if it does not, run `sudo update-crypto-policies --set DEFAULT` and reboot.

```
# on rocky-lab
update-crypto-policies --show
getenforce
```

If you took snapshots at the start, you can also simply revert both VMs to them.

Exam corner

- Security fixes must not weaken security: fix the label, boolean, permission, key, or config; never "disable SELinux", `chmod 777`, `curl -k`, or `gpgcheck=0`.
- **SELinux**: wrong **context** = `ls -Z`, `restorecon -Rv` (persistent custom paths: `semanage fcontext -a`); optional feature off = **boolean** (`getsebool`, `setsebool -P`); port not allowed = **policy** (`semanage port -a -t http_port_t -p tcp 8090`). Evidence: `ausearch -m avc -ts recent`, `audit2why`, `sealert -a`. `mv` keeps labels, `cp` does not.
- **Permissions**: directories need `x` to be entered (`namei -l`); on a file with an **ACL**, `chmod` group bits change the **mask** (`getfacl` shows `#effective:`); **attributes** `i` (immutable) and `a` (append-only) block even root (`lsattr`, `chattr`).
- **Account access**: `passwd -S` (L = locked), `chage -l` / `chage -E` (account expiry), `getent passwd` (`nologin` shell), `faillock --user X --reset` (pam_faillock lockout).
- **Remote access**: `ssh -v` on the client plus `journalctl -u ssh` / `/var/log/secure` on the server; `StrictModes` needs `~/.ssh` `700` and `authorized_keys` `600`; `AllowUsers` / `AllowGroups`; test with `sshd -t`, view effective config with `sshd -T`.
- **Unpatched** systems: `apt list --upgradable`, `unattended-upgrades`, `apt-mark showhold`, `/var/run/reboot-required`, `needrestart`; Rocky `dnf updateinfo list --security`, `dnf upgrade --security`, `dnf needs-restarting -r`.
- **Exposed** service: `ss -tlnp` shows `0.0.0.0`, `[::]`, or `*` for all interfaces versus `127.0.0.1` for local only; confirm with `nmap -sT -p-`; fix by binding to loopback, requiring authentication, and a default-deny firewall (`ufw default deny incoming` after `ufw allow OpenSSH`).
- **Certificate** errors: "self-signed certificate" (untrusted; add the CA to the trust store with `update-ca-certificates` / `update-ca-trust`), "unable to get local issuer certificate" (missing intermediate; serve the full chain), "no alternative certificate subject name matches" (fix the SAN), "certificate has expired" (renew; check `notAfter` and the clock).
- Certificate tools: `openssl s_client -connect host:443 -servername host -showcerts`, `openssl x509 -noout -subject -issuer -dates -ext subjectAltName`, `openssl verify -CAfile root -untrusted intermediate cert`, `curl -v --cacert`, `openssl x509 -checkend`.
- **Repository** errors: apt "404 Not Found" / "does not have a Release file" = wrong URL or suite; "NO_PUBKEY" / "is not signed" = wrong or missing key in `Signed-By`. dnf: "Cannot download repomd.xml" = wrong `baseurl`; "GPG check FAILED" = wrong `gpgkey`. Check with `apt-cache policy`, `dnf repolist -v`, `dnf makecache`.
- **Obsolete/insecure** protocols: SSLv3, TLS 1.0, TLS 1.1, RC4, 3DES, CBC SSH ciphers, `diffie-hellman-group1-sha1`. Detect with `nmap --script ssl-enum-ciphers -p 443 host`; nginx `ssl_protocols TLSv1.2 TLSv1.3;`.

- **Cipher** negotiation: “no matching cipher found” / “no matching key exchange method found. Their offer: …” means the lists do not overlap; diagnose with `ssh -vv`, `ssh -Q cipher`, `sshd -T`; fix the old client. Rocky: `update-crypto-policies --show` (DEFAULT, LEGACY, FUTURE, FIPS).

Review questions

1. A web page copied with `mv` from a home directory into `/var/www/html` returns “403 Forbidden” on Rocky Linux, although the file mode is `644`. Which command fixes it the right way?
   A. `setenforce 0`
   B. `chmod 777 /var/www/html/page.html`
   C. `restorecon -v /var/www/html/page.html`
   D. `setsebool -P httpd_enable_homedirs on`

2. A security audit runs `ss -tlnp` on a server and sees `0.0.0.0:6379` owned by `redis-server`. What does this mean?
   A. Redis is listening only on the loopback interface
   B. Redis is listening on every IPv4 interface and may be reachable from the network
   C. Redis is not running
   D. Redis is blocked by the firewall

3. `curl https://app.example.lan/` fails with “SSL certificate problem: unable to get local issuer certificate”, but the site works in a browser. `openssl s_client -showcerts` shows only certificate `0`. What is the best fix?
   A. Add `-k` to every curl command
   B. Configure the server to send the server certificate followed by the intermediate certificate
   C. Add the server certificate’s SAN to `/etc/hosts`
   D. Set the client’s clock back

4. `sudo apt update` prints “The following signatures couldn’t be verified because the public key is not available: NO_PUBKEY 1234ABCD5678EF90” for a third-party repository. What should the administrator do?
   A. Add `trusted=yes` to the source entry
   B. Delete `/var/lib/apt/lists/` and retry
   C. Obtain the vendor’s correct key from an official source, verify its fingerprint, and reference it with `Signed-By`
   D. Change `http` to `https` in the URL

5. A user’s script fails with “Unable to negotiate with 10.0.0.5 port 22: no matching key exchange method found. Their offer: curve25519-sha256,…”. The script uses `-oKexAlgorithms=diffie-hellman-group1-sha1`. What is the best fix?
   A. Add `diffie-hellman-group1-sha1` to the server’s `KexAlgorithms`
   B. Remove the forced old algorithm from the script (or upgrade the client) so a modern key exchange is negotiated
   C. Set the crypto policy to LEGACY on the server
   D. Restart sshd

6. Which command lists the TLS protocol versions and ciphers a web server accepts?
   A. `nmap --script ssl-enum-ciphers -p 443 server`
   B. `ss -tlnp`

C. `openssl x509 -noout -dates`

D. `update-ca-certificates`

Lab 5.5 Troubleshoot Performance Issues

Objective: 5.5 Given a scenario, analyze and troubleshoot performance issues.
Domain: 5.0 Troubleshooting (22% of the exam)
Time: about 4 hours (do it over two or three sittings; each task after Task 1 stands alone)
VM: ubuntu-lab (rocky-lab for Tasks 7 and 10)
Snapshot first: Yes. Take a snapshot of both VMs now. This lab loads the CPU, memory, and disks on purpose, formats one extra disk, adds a service that slows down booting, and changes traffic control on a network interface.

"The server is slow" is the most common and the vaguest complaint an administrator hears. In this break-and-fix lab you first record what normal looks like (a baseline), then create each slowdown yourself with load generators, traffic control, and misconfigured services, measure it with the standard Linux tools, find the root cause, fix it, and prove the fix. Exam questions for this objective describe a symptom ("high load average but the CPU is mostly idle") and ask what it means or which command to run next, so the goal is to connect each symptom to the right tool quickly.

What you'll practice

- Building a performance baseline with `sysstat` (`sar`, `iostat`, `mpstat`, `pidstat`), `vmstat`, `free`, `top`, `iotop`, and pressure stall information, and spotting **exceeding baselines**
- **CPU bottleneck**, **high CPU usage**, and **high load average** (`uptime`, `top`, `mpstat -P ALL`, `pidstat`, `nice` / `renice`, cgroup `CPUQuota`)
- **High context switching** (`vmstat` `cs`, `pidstat -w`, `sar -w`)
- **Swapping** and **out of memory** conditions (`vmstat` `si` / `so`, `free`, `swapon --show`, `MemoryMax`, the OOM killer in `journalctl -k`)
- **High I/O wait time**, **high disk latency**, **blocked processes**, and **low throughput** from storage (`iostat -x`, `vmstat` `wa` and `b`, process state `D`, `fio`, `iotop`)
- **System unresponsiveness** and **sluggish terminal behavior** (pressure stall information, cgroups, `nice`)
- **Slow startup** (`systemd-analyze blame`, `critical-chain`)
- **High failed log-in attempts** (`journalctl -u ssh`, `/var/log/auth.log`, `lastb`, `fail2ban`)
- Network performance: **high latency**, **jitter**, **packet drops**, **random timeouts**, **random disconnects**, **low throughput**, **slow response times**, and **slow application response** (`ping`, `mtr`, `curl -w`, `iperf3`, `tc` with `netem` and `tbf`)
- **Slow remote storage response** over NFS (`nfsiostat`, `mountstats`, `nfsstat`)
- **Hardware errors** (`journalctl -k -p err`, `dmesg`, `rasdaemon` / `ras-mc-ctl`, `smartctl`)

How it works

Use the same method as Lab 5.2. Lab 5.2 introduced the troubleshooting loop: identify the symptoms, gather data, form a hypothesis, test it, fix, verify, document. Performance problems follow the same loop. The difference is that nothing is "broken" with an error message. Instead, something is *slower than normal*, so you need numbers, and you need to know what normal numbers look like. Every scenario below uses the same parts as Lab 5.2: **Break it**, **Symptom**, **Investigate**, **Root cause**, **Fix**, **Verify**, and **Prevent**.

The four resources. Almost every slowdown comes from one of four resources running out: **CPU**, **memory**, **storage I/O**, or the **network**. For each resource, ask three questions (this is called the **USE method**):

- **Utilization**: how busy is it? (CPU 100% busy, disk `%util` 100%)
- **Saturation**: is work waiting in a queue? (run queue longer than the number of CPUs, disk queue growing, memory pushed to swap)
- **Errors**: is anything failing? (dropped packets, I/O errors, hardware errors in the kernel log)

```
Resource   Utilization            Saturation (queue)            Errors
--------   -------------------    -------------------------     -----------------
CPU        top %us+%sy, mpstat    vmstat r > CPUs, load avg     (rare) MCE in dmesg
Memory     free "used"            vmstat si/so, swap used, OOM  OOM killer messages
Disk       iostat %util           iostat aqu-sz, await, vmstat b I/O error in dmesg
Network    sar -n DEV, iperf3     ping latency, tc backlog      ip -s link drops/errs
```

Load average. `uptime` prints three numbers, for example `load average: 3.80, 2.10, 0.95`. They are the average number of tasks that were *runnable* (running or waiting for a CPU) or in *uninterruptible sleep* (usually waiting for disk I/O) over the last 1, 5, and 15 minutes. Compare them with the number of CPUs (`nproc`). On a 2-CPU VM, a load of 2.0 means both CPUs are fully used with nobody waiting; 4.0 means that on average two tasks are waiting all the time. Because Linux also counts tasks waiting for disks, a high load average with an idle CPU points at storage, not CPU. The order of the three numbers tells you the trend: a 1-minute value much higher than the 15-minute value means the problem just started.

Baselines. A number is only "high" compared with something. A **baseline** is a record of normal values (CPU, memory, disk, network, load) captured when the system is healthy. When users complain, you compare current values with the baseline and look for the metric that changed. The `sysstat` package collects this automatically every few minutes, and `sar` reads the history back.

Controlling who gets resources. Two mechanisms decide how resources are shared:

- **nice values** (from `-20`, most favored, to `19`, least favored) tell the CPU scheduler how to share CPU time between processes. `nice -n 19 command` starts a command politely; `renice` changes a running process.
- **cgroups** (control groups) put processes in boxes with limits. systemd puts every service in its own cgroup, so you can say "this service may use at most half a CPU (`CPUQuota=50%`) and 400 MB of memory (`MemoryMax=400M`)". `systemd-run` starts any command in a new temporary cgroup with such limits. This is how you run a test or a batch job without hurting `sshd` and other important services.

Practice safely. You will create load with `stress-ng` (a load generator with a timeout) and `fio` (a storage benchmark), always with time limits, in cgroups where possible. Network problems are created inside network namespaces with veth pairs, the same pattern as Lab 5.3, so your real network interface and your SSH session are

never touched (except in Task 10, which delays only NFS traffic). You will never use a fork bomb or anything without an off switch.

Tasks

Task 1: Record a baseline

1. Create the lab directory and install the tools used throughout the lab.

```
mkdir -p ~/labs/5.5 && cd ~/labs/5.5
sudo apt update
sudo apt install -y sysstat stress-ng fio iotop-c iperf3 mtr-tiny
```

> **What just happened:** `sysstat` provides `sar`, `sadc` (the data collector), `iostat`, `mpstat`, and `pidstat`. `stress-ng` creates CPU, memory, and context-switch load. `fio` generates disk I/O and measures it. `iotop-c` is the maintained C version of `iotop` (on Ubuntu 24.04 it also installs the `iotop` command). `iperf3` measures network throughput, and `mtr-tiny` provides `mtr`, a combined `ping` and `traceroute`. If `iperf3` asks whether to start a daemon at boot, answer **No**.

2. Turn on sysstat data collection. On Ubuntu 24.04 the package installs with collection **disabled** (`ENABLED="false"` in `/etc/default/sysstat`, and the systemd units are disabled to match).

```
sudo sed -i 's/^ENABLED="false"/ENABLED="true"/' /etc/default/sysstat
sudo systemctl enable --now sysstat sysstat-collect.timer sysstat-summary.timer
systemctl list-timers 'sysstat*'
```

```
NEXT                         LEFT     LAST PASSED UNIT                    ACTIVATES
Sat 2026-10-10 10:10:00 UTC 4min 1s  -    -      sysstat-collect.timer  sysstat-
collect.service
Sun 2026-10-11 00:07:00 UTC 13h left -    -      sysstat-summary.timer  sysstat-
summary.service
```

> **What just happened:** `sed -i` edits the file in place and changes the setting to `true`, so a later `dpkg-reconfigure sysstat` or package upgrade keeps collection on. `systemctl enable --now` enables the units at boot and starts them now. `sysstat.service` writes a "restart" marker at boot, `sysstat-collect.timer` runs `/usr/lib/sysstat/sa1` every 10 minutes to append a sample to `/var/log/sysstat/saDD` (DD is the day of the month), and `sysstat-summary.timer` writes a daily report shortly after midnight. On Rocky Linux, `sudo dnf install -y sysstat` followed by `sudo systemctl enable --now sysstat` is enough, and the files live in `/var/log/sa/`.

3. Ten minutes between samples is fine for production but slow for a lab. Add a systemd **drop-in** (a small file that overrides part of a unit) to sample every 2 minutes. A **heredoc** sends the lines between `<<'EOF'` and `EOF` into a command; the quotes around `EOF` stop the shell from changing `$` signs.

```
sudo mkdir -p /etc/systemd/system/sysstat-collect.timer.d
cat <<'EOF' | sudo tee /etc/systemd/system/sysstat-collect.timer.d/lab.conf
[Timer]
OnCalendar=
OnCalendar=*:00/2
EOF
sudo systemctl daemon-reload
sudo systemctl restart sysstat-collect.timer
systemctl list-timers sysstat-collect.timer
```

What just happened: The empty `OnCalendar=` clears the original schedule, and `*:00/2` means "every hour, starting at minute 0, every 2 minutes". `daemon-reload` makes systemd reread unit files. The `NEXT` column now shows a time less than 2 minutes away. You will remove this drop-in in the clean-up.

4. Check the CPU count and the load average.

```
nproc
uptime
cat /proc/loadavg
```

```
2
 10:06:12 up 25 min,  1 user,  load average: 0.08, 0.04, 0.01
0.08 0.04 0.01 1/187 2214
```

What just happened: `nproc` prints the number of CPUs available (2 on ubuntu-lab). `uptime` shows the time, how long the system has been up, logged-in users, and the 1-, 5-, and 15-minute load averages. `/proc/loadavg` has the same three numbers plus `1/187` (1 task running now out of 187 total) and the last process ID used. A load near zero on 2 CPUs means the system is idle. Write your numbers down: this is your first baseline value.

5. Take the tour of the live tools. Start with `vmstat` (virtual memory statistics), which shows CPU, memory, swap, and I/O in one line per interval.

```
vmstat 2 5
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 0  0      0 2961204  48212 612044    0    0    52    31  118  205  1  1 98  0  0  0
 0  0      0 2961204  48212 612044    0    0     0     0   61  112  0  0 100 0  0  0
 0  0      0 2961204  48220 612044    0    0     0    10   58  109  0  0 100 0  0  0
 ...
```

What just happened: `vmstat 2 5` prints 5 lines, 2 seconds apart. **The first line is an average since boot; ignore it and read the others.** Columns: `r` = runnable tasks (running or waiting for CPU), `b` = **blocked processes** (uninterruptible sleep, usually waiting for I/O), `swpd` = swap in use (KB), `free` / `buff` / `cache` = memory (KB), `si` / `so` = swap in/out per second (any steady non-zero value means **swapping**), `bi` / `bo` = blocks read/written per second, `in` = interrupts, `cs` = **context switches** per second, and CPU percentages `us` (user programs), `sy` (kernel), `id` (idle), `wa` (**I/O wait**: idle while waiting for disk), `st` (steal: time the hypervisor gave your vCPU to another VM), and `gu` (time running guest VMs).

6. Look at memory and swap.

```
free -h
swapon --show
```

```
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       382Mi       2.8Gi       1.1Mi       791Mi       3.4Gi
Swap:          3.8Gi          0B       3.8Gi
NAME      TYPE SIZE USED PRIO
/swap.img file 3.8G   0B   -2
```

What just happened: `free -h` shows memory in human units. The important column is **available**: memory that programs can still get, including cache the kernel can drop. A low `free` with a high `available` is healthy, because Linux uses spare RAM as disk cache. `swapon --show` lists swap areas; the Ubuntu installer creates a swap file called `/swap.img`. If `swapon --show` prints nothing, you have no swap; Task 3 shows how to add a temporary swap file.

7. Look at each CPU, at each disk, and at each process.

```
mpstat -P ALL 2 1
iostat -dxz -y 2 1
pidstat 2 1
```

```
Average:     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice
%idle
Average:     all    0.50    0.00    0.25    0.00    0.00    0.00    0.00    0.00    0.00
99.25
Average:       0    0.50    0.00    0.50    0.00    0.00    0.00    0.00    0.00    0.00
99.00
Average:       1    0.50    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
99.50

Device     r/s  rkB/s ... r_await ...  w/s  wkB/s ... w_await ... aqu-sz  %util
sda       0.00   0.00 ...    0.00 ... 2.50  14.00 ...    0.40 ...   0.00   0.20
```

What just happened: `mpstat -P ALL 2 1` shows one 2-second sample for all CPUs together (`all`) and for each CPU, so you can see whether one CPU is maxed out while the other is idle. `iostat -d` shows devices only, `-x` extended columns, `-z` hides idle devices, and `-y` skips the since-boot report. The `iostat -x` line is very wide; the output above is trimmed to the columns you will use most: `r/s` and `w/s` (reads and writes per second), `r_await` and `w_await` (average milliseconds per read and write, including time in the queue: this is **disk latency**), `aqu-sz` (average queue length), and `%util` (percentage of time the device was busy). `pidstat 2 1` shows per-process CPU use; with no activity it may print only a header.

8. Look at the interactive views. Run `top` , press `1` to show each CPU, `M` to sort by memory, `P` to sort by CPU, and `q` to quit. Then enable delay accounting (needed for the `IO>` column of `iotop`) and run `iotop` in batch mode.

```
top
sudo sysctl -w kernel.task_delayacct=1
sudo iotop -o -b -n 2 -d 2
```

What just happened: `top` refreshes every 3 seconds; its header shows the load average, task counts by state, and the same `us sy ni id wa hi si st` CPU split as `vmstat` . `kernel.task_delayacct=1` turns on per-task delay accounting until the next reboot; without it `iotop` warns and cannot show how much time each process waits for I/O. `iotop -o` shows only processes doing I/O, `-b` is batch (non-interactive) mode, `-n 2` takes two samples, and `-d 2` waits 2 seconds between them. On an idle system it shows only the totals lines.

9. Check **pressure stall information** (PSI): the percentage of time tasks were stalled waiting for a resource.

```
grep . /proc/pressure/*
```

```
/proc/pressure/cpu:some avg10=0.00 avg60=0.00 avg300=0.00 total=1739022
/proc/pressure/cpu:full avg10=0.00 avg60=0.00 avg300=0.00 total=0
/proc/pressure/io:some avg10=0.00 avg60=0.00 avg300=0.00 total=2214930
/proc/pressure/io:full avg10=0.00 avg60=0.00 avg300=0.00 total=1874210
/proc/pressure/memory:some avg10=0.00 avg60=0.00 avg300=0.00 total=0
/proc/pressure/memory:full avg10=0.00 avg60=0.00 avg300=0.00 total=0
```

What just happened: `some avg10=12.5` would mean that in the last 10 seconds, at least one task was stalled waiting for that resource 12.5% of the time; `full` means *all* non-idle tasks were stalled at once. PSI is the most direct measure of “the system feels slow”, and zero on all lines is a healthy baseline.

10. Save a baseline you can compare against later. Create a small script that captures a snapshot, and a “probe” that measures how quickly the system can start 300 tiny programs. The probe is your stand-in for “how fast does the application respond”.

```
cat <<'EOF' > ~/labs/5.5/baseline.sh
#!/bin/bash
# baseline.sh: capture a performance snapshot of this host
echo "== $(hostname) $(date) =="
echo "-- CPUs: $(nproc)"; uptime
echo "-- memory"; free -m; swapon --show
echo "-- vmstat (5 samples, 2 s apart)"; vmstat 2 5
echo "-- mpstat"; mpstat -P ALL 2 1 | grep Average
echo "-- iostat"; iostat -dxz -y 2 1
echo "-- pressure"; grep . /proc/pressure/*
echo "-- network counters"; ip -s link
echo "-- top processes"; ps -eo pid,user,pcpu,pmem,stat,comm --sort=-pcpu | head -8
EOF
cat <<'EOF' > ~/labs/5.5/probe.sh
#!/bin/bash
# probe.sh: time 300 tiny program starts (a stand-in for "the app")
TIMEFORMAT='probe: %R seconds'
time (for i in $(seq 300); do /bin/true; done)
EOF
chmod 755 ~/labs/5.5/baseline.sh ~/labs/5.5/probe.sh
~/labs/5.5/baseline.sh > ~/labs/5.5/baseline.txt
~/labs/5.5/probe.sh | tee -a ~/labs/5.5/baseline.txt
```

```
probe: 0.42 seconds
```

What just happened: `baseline.sh` runs the commands from this task and `> baseline.txt` saves everything to a file. `probe.sh` uses the bash `time` keyword; `TIMEFORMAT='probe: %R seconds'` makes it print only the real (wall-clock) time. `tee -a` prints the result and appends it to the baseline file. Run the probe two or three times and note a typical value; yours depends on your host. In every scenario below you will run the probe again and compare.

Checkpoint: `ls ~/labs/5.5` shows `baseline.sh`, `baseline.txt`, and `probe.sh`; `systemctl is-active sysstat-collect.timer` prints `active`; and after a few minutes `sar -u` prints at least one sample line instead of "Cannot open /var/log/sysstat/saDD".

Task 2: CPU bottleneck, high CPU usage, high load average, and high context switching

Scenario A: CPU-bound jobs

1. **Break it.** Start four CPU-hungry workers on your 2-CPU VM. They stop by themselves after 5 minutes.

```
cd ~/labs/5.5
stress-ng --cpu 4 --timeout 300s > /dev/null 2>&1 &
```

What just happened: `stress-ng --cpu 4` starts four workers that do nothing but math, and `--timeout 300s` stops them after 300 seconds. `> /dev/null 2>&1` discards its messages and `&` runs it in the background so you keep your prompt. Four workers on two CPUs means two of them are always waiting for a CPU.

2. **Symptom.** Wait about a minute, then run the probe and check the load.

```
~/labs/5.5/probe.sh
uptime
```

```
probe: 1.31 seconds
 10:21:40 up 40 min,  1 user,  load average: 3.12, 1.20, 0.47
```

The "application" takes about three times longer than the baseline, and the 1-minute load average is well above the CPU count (2) and still climbing toward 4.

3. **Investigate.** Is the CPU really busy, and by whom?

```
top -b -n 1 -o %CPU | head -12
mpstat -P ALL 2 1 | grep Average
vmstat 2 3
```

```
top - 10:22:05 up 40 min,  1 user,  load average: 3.48, 1.38, 0.54
Tasks: 192 total,   5 running, 187 sleeping,   0 stopped,   0 zombie
%Cpu(s): 99.6 us,  0.4 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3916.1 total,   2874.4 free,    410.2 used,    807.6 buff/cache
MiB Swap:   3915.0 total,   3915.0 free,      0.0 used.   3505.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2301 student   20   0  310780   5892   2724 R  50.0   0.1   0:31.02 stress-ng-cpu
   2302 student   20   0  310780   5892   2724 R  50.0   0.1   0:30.95 stress-ng-cpu
   2303 student   20   0  310780   5892   2724 R  50.0   0.1   0:31.10 stress-ng-cpu
   2304 student   20   0  310780   5892   2724 R  50.0   0.1   0:30.87 stress-ng-cpu
Average:     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice
%idle
Average:     all   99.75    0.00    0.25    0.00    0.00    0.00    0.00    0.00    0.00
0.00
Average:       0   99.50    0.00    0.50    0.00    0.00    0.00    0.00    0.00    0.00
0.00
Average:       1  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
0.00
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 4  0      0 2943380  48900 778212    0    0     0     0  561  412 100  0  0  0  0  0
 5  0      0 2943380  48900 778212    0    0     0     6  548  398 100  0  0  0  0  0
```

What just happened: `top -b -n 1 -o %CPU` prints one snapshot sorted by CPU: four processes in state `R` (runnable) each get about half a CPU. `mpstat` shows **high CPU usage** on *every* CPU (`%idle 0`), mostly `%usr` (high `%sys` would point at the kernel, high `%steal` at an overloaded hypervisor). `vmstat` shows `r` of 4 to 5 on 2 CPUs: tasks are queuing. 100% utilization plus a run queue longer than the CPU count is a **CPU bottleneck**. `wa` is 0, so storage is not involved.

4. Identify the processes precisely with `pidstat`, which also shows how much each one *waits* for a CPU.

```
pidstat -u 2 1 | grep -E 'Average|stress'
```

```
Average:      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
Average:     1000      2301   49.75    0.00    0.00   49.75   49.75     -  stress-ng-cpu
Average:     1000      2302   50.25    0.00    0.00   49.25   50.25     -  stress-ng-cpu
```

What just happened: `%wait` is the percentage of time the task was ready to run but waiting for a CPU. Values near 50% mean each worker spends half its life in the queue. On a real server this column tells you that a process is slow not because it is inefficient but because it is starved of CPU.

5. **Root cause.** More CPU-bound work than CPUs: four always-runnable processes on two CPUs. In real life this is a runaway process, a batch job started at the wrong time, too many worker processes for the hardware, or a VM with too few vCPUs.

6. **Fix, option 1: lower the priority.** Make the workers "nice" so that everything else wins when it competes with them.

```
renice -n 19 -p $(pgrep -d ' ' stress-ng-cpu)
ps -o pid,ni,pcpu,comm -C stress-ng-cpu
~/labs/5.5/probe.sh
uptime
```

```
2301 (process ID) old priority 0, new priority 19
2302 (process ID) old priority 0, new priority 19
...
    PID  NI %CPU COMMAND
   2301  19 49.6 stress-ng-cpu
...
probe: 0.47 seconds
 10:24:02 up 42 min,  1 user,  load average: 4.02, 2.31, 0.98
```

What just happened: `pgrep -d ' ' stress-ng-cpu` prints the matching PIDs separated by spaces, and `renice -n 19 -p` sets their nice value to 19, the lowest priority. A normal user can only make their own processes *nicer* (higher numbers); lowering the value needs `sudo`. The probe is fast again because the scheduler now prefers it. Notice that the **load average stays high**: the workers are still runnable, they just yield. Load average measures demand, not user pain, which is why you always check it together with the other tools.

7. **Fix, option 2: cap the job with a cgroup.** Stop the workers and start the same job as a transient systemd service, then limit it to half of one CPU while it runs.

```
pkill stress-ng
sudo systemd-run --unit=cpuhog stress-ng --cpu 4 --timeout 300s
sleep 10; mpstat 2 1 | grep Average
sudo systemctl set-property --runtime cpuhog.service CPUQuota=50%
sleep 10; mpstat 2 1 | grep Average
systemctl status cpuhog --no-pager | head -12
```

```
Running as unit: cpuhog.service; invocation ID: 6c1e...
Average:      all   99.50    0.00    0.50    0.00    0.00    0.00    0.00    0.00    0.00
0.00
Average:      all   25.06    0.00    0.25    0.00    0.00    0.00    0.00    0.00    0.00
74.69
● cpuhog.service - /usr/bin/stress-ng --cpu 4 --timeout 300s
     Loaded: loaded (/run/systemd/transient/cpuhog.service; transient)
  Transient: yes
    Drop-In: /run/systemd/system.control/cpuhog.service.d
             └─50-CPUQuota.conf
     Active: active (running) since Sat 2026-10-10 10:25:11 UTC; 21s ago
      Tasks: 5 (limit: 4613)
        CPU: 21.904s
     CGroup: /system.slice/cpuhog.service
```

What just happened: `pkill stress-ng` sends `SIGTERM` to every process named `stress-ng...`. `systemd-run --unit=cpuhog` runs the command as a temporary (transient) service in its own cgroup, `system.slice/cpuhog.service`. `systemctl set-property --runtime cpuhog.service CPUQuota=50%` changes the limit live (`--runtime` means it is not saved after reboot): the whole cgroup may use 50% of one CPU, which is 25% of this 2-CPU system, as `mpstat` shows. You could also pass the limit at start with `systemd-run -p CPUQuota=50% ...`, or put `CPUQuota=` in a service's unit file permanently.

8. **Verify** and stop the job.

```
~/labs/5.5/probe.sh
sudo systemctl stop cpuhog
uptime
```

The probe is back near the baseline. After a few minutes all three load average numbers fall back below 1.

9. **Prevent.** Start batch jobs with `nice -n 19` (or as a systemd unit with `Nice=19` and `CPUQuota=`), size worker pools to the CPU count, and alert when the load average stays above the number of CPUs. Note that `top`'s `st` (steal) column being high means the *host* is overloaded; adding vCPUs inside the VM will not help.

Scenario B: high context switching

10. **Break it.** A **context switch** happens every time a CPU stops running one task and starts another. A few thousand per second is normal; hundreds of thousands means the CPUs spend their time switching instead of working. Start two workers that pass messages back and forth through pipes as fast as they can.

```
stress-ng --switch 2 --timeout 120s > /dev/null 2>&1 &
```

11. **Investigate.** Compare the `cs` column with your baseline, then find the process responsible.

```
vmstat 2 3
pidstat -w 2 1 | sort -k4 -n -r | head -5
sar -w 2 2
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 3  0      0 2940120  49020 778400    0    0     0     0 91340 352870  2 46 52  0  0  0
 2  0      0 2940120  49020 778400    0    0     0     4 93115 361204  1 47 52  0  0  0
Average:      1000      2420  44938.00   2028.00  stress-ng-switc
Average:      1000      2422  44030.00   2095.00  stress-ng-switc
Average:      1000      2421  44014.00   2262.00  stress-ng-switc
10:31:14       proc/s   cswch/s
10:31:16         0.50 355102.00
```

What just happened: `cs` jumped from about 100 in the baseline to over 350,000 per second, and `sy` (kernel time) is near 50% even though `us` is tiny: the kernel is busy switching. `pidstat -w` shows per-process `cswch/s` (voluntary switches: the task gave up the CPU because it was waiting, for example for a pipe or lock) and `nvcswch/s` (involuntary switches: the scheduler took the CPU away, a sign of CPU competition). `sort -k4 -n -r` sorts by the fourth column, the voluntary rate. `sar -w` reports `proc/s` (new processes per second) and the system-wide `cswch/s`. The `Command` column is cut at 15 characters, which is why it shows `stress-ng-switc`.

12. **Root cause** and **fix.** Processes that hand tiny pieces of work to each other millions of times (or hundreds of threads fighting over one lock) cause **high context switching**. Stop it, then confirm `cs` is back to baseline.

```
pkill stress-ng
vmstat 2 3
```

13. **Prevent.** Size thread pools to roughly the CPU count, batch small messages, and look for lock contention. A high `proc/s` in `sar -w` means something starts processes in a tight loop (for example a crashing service that systemd keeps restarting).

Checkpoint: `pgrep stress-ng` prints nothing, `systemctl is-active cpuhog` prints `inactive` (or "unknown"), and `vmstat 2 3` shows `r` near 0, `id` near 100, and `cs` back near your baseline.

Task 3: Swapping and out of memory

Swap is disk space used as overflow for RAM. Moving rarely used pages to swap is fine. **Swapping** as a *problem* means pages are moved out and back in constantly (also called thrashing), and because a disk is thousands of times slower than RAM, everything waits. When memory and swap run out completely, the kernel's **OOM killer** (out of memory killer) kills a process to free memory.

You will create both conditions inside a cgroup with a memory limit, so only the test process suffers and the rest of the VM keeps working.

1. Make sure swap exists. If `swapon --show` printed a line in Task 1, skip to step 2. If it printed nothing, create a temporary 2 GB swap file.

```
sudo fallocate -l 2G /swap-lab.img
sudo chmod 600 /swap-lab.img
sudo mkswap /swap-lab.img
sudo swapon /swap-lab.img
swapon --show
```

What just happened: `fallocate -l 2G` reserves a 2 GB file, `chmod 600` makes it readable only by root (required for swap), `mkswap` writes a swap signature, and `swapon` activates it. The clean-up removes it.

Part A: swapping

2. **Break it.** Run a memory worker that needs 800 MB in a cgroup limited to 400 MB of RAM. The kernel must keep half of the worker's memory in swap, and because the worker touches all of it constantly, pages go out and come back in all the time.

```
sudo systemd-run --unit=memhog -p MemoryMax=400M \
    stress-ng --vm 1 --vm-bytes 800M --vm-keep --timeout 180s
```

What just happened: `--vm 1` starts one worker that allocates memory, `--vm-bytes 800M` sets the amount, and `--vm-keep` keeps rewriting the same memory instead of freeing it. `-p MemoryMax=400M` is the hard RAM limit for the cgroup. Swap is still allowed for this cgroup, so instead of being killed the worker is forced to swap. The limit stands in for "a server with too little RAM for its workload".

3. **Symptom and investigation.** Watch the swap columns and the swap totals.

```
vmstat 2 5
free -h
swapon --show
sar -W 2 2
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1  1 412380 2501220  49532 781200 48210 49870 48230 49880 9120 14233 3 19 41 37 0  0
 0  2 415016 2500980  49532 781204 51364 52920 51380 52930 9644 15010 2 21 39 38 0  0
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       789Mi       2.4Gi       1.1Mi       763Mi       3.0Gi
Swap:          3.8Gi       403Mi       3.4Gi
NAME      TYPE SIZE   USED PRIO
/swap.img file 3.8G 402.7M   -2
10:40:12     pswpin/s pswpout/s
10:40:14     12610.50  12988.00
```

What just happened: `si` and `so` (KB swapped in and out per second) are tens of thousands every interval: active **swapping**. Each swap-in is a disk read, so `bi`, `bo`, `wa`, and `b` rise too. `free` and `swapon` show about 400 MB of swap in use, and `sar -W` shows the same activity in pages per second (with `-f`, historically). `available` RAM still looks fine because the pressure is inside the cgroup; on a whole server short of RAM, `available` would also be near zero.

4. Find which process is swapping. Sort by resident memory (RSS, the part in RAM), then ask the kernel how much of that process is in swap.

```
ps -eo pid,rss,vsz,comm --sort=-rss | head -4
systemctl status memhog --no-pager | grep -E 'Memory|CGroup'
```

```
    PID   RSS    VSZ COMM
   2561 393420 1131040 stress-ng-vm
   1322  45232  290812 python3
    Memory: 400.0M (max: 400.0M available: 0B peak: 400.0M swap: 405.1M swap peak: 405.4M)
    CGroup: /system.slice/memhog.service
```

Then look at the process itself (use the PID from your output):

```
grep -E 'VmRSS|VmSwap' /proc/2561/status
```

```
VmRSS:    393420 kB
VmSwap:   412880 kB
```

What just happened: `ps --sort=-rss` lists the biggest RAM users first. `systemctl status` shows the cgroup's memory use, its limit, and its swap use. `VmSwap` in `/proc/PID/status` is the amount of that process currently in swap. In `top` you can see the same thing by pressing `f` and enabling the `SWAP` field.

5. **Root cause.** The workload's working set (the memory it actively uses, 800 MB) is larger than the RAM it is allowed to use (400 MB), so the kernel keeps moving pages between RAM and swap.

6. **Fix.** Stop the job (on a real server: fix the leak, lower the job's memory need, raise the limit, or add RAM).

```
sudo systemctl stop memhog
vmstat 2 3
cat /proc/sys/vm/swappiness
```

What just happened: After the job stops, `si` and `so` return to 0. `swpd` may stay above zero for a while: pages already in swap stay there until something needs them, and that is harmless. `vm.swappiness` (default 60, range 0 to 200) tells the kernel how willing it is to swap out program memory instead of dropping file cache. Lowering it (for example `sudo sysctl vm.swappiness=10`) makes the kernel prefer to keep programs in RAM, but it cannot fix a workload that simply does not fit.

7. **Verify and prevent.** `vmstat` shows `si` / `so` of 0 and the probe is back to baseline. Prevent it by monitoring `si` / `so` (any sustained non-zero value is a warning), giving services realistic `MemoryMax=` / `MemoryHigh=` limits, and fixing memory leaks (see Lab 5.2 for `pmap` and `smem`).

Part B: out of memory

8. **Break it.** Create a small program that allocates memory in 50 MB steps until it reaches a target size, then run it with a target of 400 MB in a cgroup that allows only 300 MB and **no swap**.

   Save this as `~/labs/5.5/eatmem.py` (with `nano`, or with the heredoc below):

```
cat <<'EOF' > ~/labs/5.5/eatmem.py
import sys
import time

target_mb = int(sys.argv[1]) if len(sys.argv) > 1 else 400
chunks = []
while len(chunks) * 50 < target_mb:
    chunks.append(b"x" * (50 * 1024 * 1024))   # 50 MB, really written to RAM
    print(f"allocated {len(chunks) * 50} MB", flush=True)
    time.sleep(0.3)
print("finished: holding the memory for 5 seconds", flush=True)
time.sleep(5)
EOF
sudo systemd-run --scope -p MemoryMax=300M -p MemorySwapMax=0 \
    python3 ~/labs/5.5/eatmem.py 400
echo "exit code: $?"
```

```
Running as unit: run-r5c1d8f2b1e0a4c6f9c0b2d3e4f5a6b7c.scope; invocation ID: 1f0e...
allocated 50 MB
allocated 100 MB
allocated 150 MB
allocated 200 MB
allocated 250 MB
Killed
exit code: 137
```

   What just happened: `systemd-run --scope` runs the command in the foreground (attached to your terminal) inside a new cgroup called a scope. `MemoryMax=300M` limits RAM to 300 MB and `MemorySwapMax=0` forbids swap for this cgroup, so when the program tries to grow past the limit the kernel cannot reclaim anything and invokes the OOM killer *for this cgroup only*. `Killed` and exit code `137` (128 + signal 9, `SIGKILL`) are what a user or application log shows when a process is OOM-killed.

9. **Investigate.** The kernel log always records an OOM kill.

```
journalctl -k --since "10 min ago" | grep -i -E 'oom|killed process'
```

```
Oct 10 10:52:31 ubuntu-lab kernel: python3 invoked oom-killer: gfp_mask=0xcc0(GFP_KERNEL),
order=0, oom_score_adj=0
Oct 10 10:52:31 ubuntu-lab kernel: oom-kill:constraint=CONSTRAINT_MEMCG,nodemask=
(null),cpuset=/,mems_allowed=0,oom_memcg=/system.slice/run-
r5c1...scope,task_memcg=/system.slice/run-r5c1...scope,task=python3,pid=2688,uid=0
Oct 10 10:52:31 ubuntu-lab kernel: Memory cgroup out of memory: Killed process 2688
(python3) total-vm:322100kB, anon-rss:301312kB, file-rss:5888kB, shmem-rss:0kB, UID:0
pgtables:700kB oom_score_adj:0
```

What just happened: `journalctl -k` shows only kernel messages (the same as `dmesg`, but with readable timestamps and history). The key line is `Killed process 2688 (python3)`. `constraint=CONSTRAINT_MEMCG` and "Memory cgroup out of memory" mean a cgroup limit was hit; a whole-system shortage says `constraint=CONSTRAINT_NONE` and just "Out of memory: Killed process". `anon-rss` (about 300 MB) shows how much the victim was using. Users usually report this as "the application crashed for no reason", so always check the kernel log when a process disappears.

10. **Root cause.** The job legitimately needs about 400 MB, but its limit is 300 MB with no swap.

11. **Fix and verify.** Give the job a realistic limit and run it again.

```
sudo systemd-run --scope -p MemoryMax=600M -p MemorySwapMax=0 \
    python3 ~/labs/5.5/eatmem.py 400
echo "exit code: $?"
```

    The program reaches `allocated 400 MB`, prints `finished`, and exits with code `0`.

12. **Prevent.** Size memory limits from measured peaks (`systemctl status` shows `peak:`), alert on OOM messages in the log, and protect critical processes: each process has `/proc/PID/oom_score` (higher means more likely to be killed) and `/proc/PID/oom_score_adj` (from `-1000`, never kill, to `1000`); a service can set `OOMScoreAdjust=` in its unit file.

Checkpoint: `systemctl is-active memhog` is not `active`, `vmstat 2 3` shows `si` and `so` at 0, and `journalctl -k | grep -c 'Killed process'` shows at least 1.

Task 4: High I/O wait, high disk latency, blocked processes, and low throughput

CAUTION

This task formats one extra disk. Run `lsblk` and make sure the disk you use is an empty 5 GB extra disk, not your OS disk. The commands use `/dev/sdb`; substitute your real name (for example `/dev/vdb`).

1. Prepare a test filesystem on the extra disk.

```
lsblk
DISK=/dev/sdb
sudo mkfs.ext4 -F -q "$DISK"
sudo mkdir -p /mnt/perf
sudo mount "$DISK" /mnt/perf
sudo chown student: /mnt/perf
```

What just happened: `DISK=/dev/sdb` stores the device name in a shell variable so you type it only once (if you open a new terminal, set it again). `mkfs.ext4 -F -q` formats it without asking and without the usual progress text. The filesystem is mounted on `/mnt/perf` and owned by `student` so `fio` can run without `sudo`.

2. Measure normal throughput and latency first. **Throughput** is how much data moves per second (MB/s); **latency** is how long one request takes (ms).

```
cd ~/labs/5.5
dd if=/dev/zero of=/mnt/perf/victim bs=1M count=200 oflag=direct
```

```
200+0 records in
200+0 records out
209715200 bytes (210 MB, 200 MiB) copied, 0.61 s, 344 MB/s
```

What just happened: `dd` copies 200 blocks of 1 MB from `/dev/zero` (endless zeros) into a file. `oflag=direct` bypasses the page cache so you measure the disk, not RAM. Write down the MB/s value; on your host it will be different.

3. **Break it.** Start a "noisy neighbor": 16 processes that each write small random blocks and force every write to disk with `fsync`, for 5 minutes.

```
fio --name=noisy --directory=/mnt/perf --size=256M --rw=randwrite --bs=4k \
    --direct=1 --ioengine=psync --numjobs=16 --fsync=1 \
    --time_based --runtime=300 --group_reporting > noisy.log 2>&1 &
```

What just happened: `--rw=randwrite --bs=4k` writes 4 KB blocks at random offsets, the hardest pattern for any disk. `--direct=1` skips the page cache, `--ioengine=psync` uses ordinary blocking `pwrite()` calls, `--numjobs=16` runs 16 processes, and `--fsync=1` calls `fsync()` after every write, which also forces ext4 journal commits. `--time_based --runtime=300` runs for exactly 300 seconds and `--group_reporting` prints one summary for all jobs into `noisy.log`.

4. **Symptom.** Repeat the `dd` test, run the probe, and check the load.

```
dd if=/dev/zero of=/mnt/perf/victim bs=1M count=200 oflag=direct
~/labs/5.5/probe.sh
uptime
```

```
209715200 bytes (210 MB, 200 MiB) copied, 9.87 s, 21.2 MB/s
probe: 0.45 seconds
 11:05:44 up 1:24,  1 user,  load average: 15.62, 6.80, 2.71
```

Three symptoms at once: **low throughput** (21 MB/s instead of 344 MB/s), a load average far above the CPU count, and yet the CPU probe is still fast. That combination points away from the CPU.

5. **Investigate with `vmstat` and `top`.**

```
vmstat 2 3
top -b -n 1 | head -4
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
 1 15      0 2861020  50112 781550    0    0     0 26112 9881 30544  1  9  3 87  0  0
 0 16      0 2860912  50112 781552    0    0     0 25930 9706 30102  1  8  4 87  0  0
top - 11:06:01 up 1:24,  1 user,  load average: 15.81, 7.12, 2.86
Tasks: 208 total,   1 running, 207 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.8 us,  8.5 sy,  0.0 ni,  3.4 id, 87.3 wa,  0.0 hi,  0.0 si,  0.0 st
```

What just happened: `wa` near 90% is **high I/O wait time**: the CPUs have nothing to run because the tasks are waiting for the disk. `b` is 15 to 16: that many **blocked processes** in uninterruptible sleep. Those blocked tasks are exactly what pushes the load average to 15 on a machine whose CPUs are almost idle. `r` is near 0, so this is not a CPU problem.

6. List the blocked processes. State `D` means uninterruptible sleep; such a process cannot even be killed until its I/O completes.

```
ps -eo state,pid,user,wchan:20,comm | awk '$1 == "D"'
```

```
D    3104 student  jbd2_log_wait_commit fio
D    3105 student  jbd2_log_wait_commit fio
D    3106 student  wait_on_page_bit     fio
...
```

What just happened: `ps -eo` chooses the columns; `wchan:20` shows (in 20 characters) the kernel function where the process is sleeping. `jbd2_log_wait_commit` means it is waiting for the ext4 journal to be written. `awk '$1 == "D"'` keeps only lines whose first column is `D`. If your `dd` from step 4 were still running, it would be in this list too.

7. Find which disk is struggling and how badly.

```
iostat -dxz -y 2 2
```

```
Device     r/s  rkB/s ... r_await ...     w/s     wkB/s ... w_await ... aqu-sz  %util
sda       0.00   0.00 ...    0.00 ...    3.00     20.00 ...    1.17 ...   0.01   0.40
sdb       0.00   0.00 ...    0.00 ... 6420.50  25682.00 ...   24.73 ...  15.88 100.00
```

What just happened: Only `sdb` is busy. `%util 100.00` means it always had requests in flight (saturated; SSD and NVMe handle requests in parallel, so read `%util` together with `await`). `aqu-sz 15.88` means about 16 requests waiting, one per `fio` job. `w_await 24.73` is the average milliseconds per write including queueing: **high disk latency**. Roughly, a healthy SSD answers in well under 1 ms and a spinning disk in 5 to 15 ms. Your numbers depend on your host, but `aqu-sz` and `%util` tell the same story.

8. Find the process doing the I/O.

```
sudo iotop -o -b -P -n 1 -d 2 | head -8
pidstat -d 2 1 | grep -E 'Average|fio' | head -5
```

```
Total DISK READ:         0.00 B/s | Total DISK WRITE:        25.10 M/s
Current DISK READ:       0.00 B/s | Current DISK WRITE:      38.40 M/s
    PID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN      IO    COMMAND
   3104 be/4 student      0.00 B/s    1.57 M/s  0.00 %  97.12 % fio --name=noisy ...
   3105 be/4 student      0.00 B/s    1.56 M/s  0.00 %  96.88 % fio --name=noisy ...
Average:      UID       PID   kB_rd/s   kB_wr/s kB_ccwr/s iodelay  Command
Average:     1000      3104      0.00   1604.00      0.00     196  fio
```

What just happened: `iotop -P` shows processes instead of threads. `IO` is the percentage of time the process spent waiting on I/O (this needs the `kernel.task_delayacct=1` setting from Task 1). `pidstat -d` shows per-process kB read and written per second and `iodelay`, the clock ticks spent waiting for block I/O. Both tools point at `fio`.

9. **Root cause.** One workload issues thousands of small synchronous writes and saturates `sdb`. Every other request to that disk waits in the same queue, which shows up as high I/O wait, high disk latency, blocked processes, a high load average, and low throughput for everyone else.

10. **Fix.** Stop the noisy job, then run it again under a cgroup I/O limit so it can no longer starve others.

```
pkill -x fio
sudo systemd-run --unit=noisy -p "IOWriteIOPSMax=$DISK 300" \
    fio --name=noisy --directory=/mnt/perf --size=256M --rw=randwrite --bs=4k \
    --direct=1 --ioengine=psync --numjobs=16 --fsync=1 --time_based --runtime=300
sleep 5
dd if=/dev/zero of=/mnt/perf/victim bs=1M count=200 oflag=direct
iostat -dxz -y 2 1
```

> **What just happened:** `pkill -x fio` kills processes whose name is exactly `fio`. `IOWriteIOPSMax=/dev/sdb 300` limits the cgroup to 300 write operations per second on that device (`IOReadBandwidthMax=`, `IOWriteBandwidthMax=`, and `IOWeight=` are related settings). The victim `dd` now gets most of its original throughput back and `w_await` drops sharply. You may know `ionice -c3` (idle I/O class) as another tool: it only has an effect with the BFQ I/O scheduler, and `cat /sys/block/sdb/queue/scheduler` usually shows `[mq-deadline]` or `[none]` on VMs, so cgroup limits are the reliable choice.

11. **Verify** and undo the scenario.

```
sudo systemctl stop noisy
vmstat 2 3
sudo umount /mnt/perf
sudo wipefs -a "$DISK"
```

`vmstat` shows `b` at 0 and `wa` at 0 again. `wipefs -a` removes the filesystem signature so the disk is blank for later labs.

12. **Prevent.** Separate busy workloads onto different disks, put I/O limits or weights on batch jobs, avoid unnecessary `fsync` storms (for example, many tiny log writes), and watch `await` and `aqu-sz` against your baseline with `sar -d`.

> **Checkpoint:** `pgrep fio` prints nothing, `findmnt /mnt/perf` prints nothing, `lsblk -f "$DISK"` shows no filesystem, and `vmstat 2 3` shows `b` and `wa` at 0.

Task 5: System unresponsiveness and sluggish terminal behavior

Sluggish terminal behavior is what a user feels first: characters appear a moment after you type them, `ls` takes a second, tab completion hangs. Every keystroke over SSH needs `sshd` and your shell to get CPU time and memory; when other tasks hog both, the terminal lags. In the worst case the system becomes **unresponsive**: SSH logins time out and even the console is slow.

1. **Break it.** Start CPU and memory pressure together, in your own login session, with a time limit.

```
stress-ng --cpu 8 --vm 2 --vm-bytes 1G --vm-keep --timeout 120s > /dev/null 2>&1 &
```

> **What just happened:** Eight CPU workers on two CPUs create a long run queue, and two memory workers keep 2 GB of RAM constantly busy. The 120-second timeout is your safety net. Never test this with a fork bomb (a function that copies itself endlessly): it has no off switch and usually needs a hard reset.

2. **Symptom.** Type a few commands and notice the delay, then measure it.

```
~/labs/5.5/probe.sh
time ls -R /usr/share/doc > /dev/null
```

```
probe: 2.37 seconds
real    0m1.941s
```

Both are several times slower than normal, and key presses may echo late.

3. **Investigate.** Pressure stall information summarizes exactly this feeling.

```
grep . /proc/pressure/cpu /proc/pressure/memory
uptime
vmstat 2 3
```

```
/proc/pressure/cpu:some avg10=78.12 avg60=41.30 avg300=10.02 total=48211890
/proc/pressure/cpu:full avg10=0.00 avg60=0.00 avg300=0.00 total=0
/proc/pressure/memory:some avg10=3.41 avg60=1.10 avg300=0.25 total=1520120
/proc/pressure/memory:full avg10=1.90 avg60=0.61 avg300=0.14 total=842003
11:20:03 up 1:39,  1 user,  load average: 9.85, 4.10, 1.66
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
10  0      0  736120  50300 792040    0    0     0     0 1302 1987 93  7  0  0  0  0
```

What just happened: `cpu some avg10=78.12` means that during the last 10 seconds, some task was waiting for a CPU 78% of the time; your shell and `sshd` are among those tasks. Memory pressure is starting too, and `free` memory is low. The load average of about 10 and `r` = 10 on 2 CPUs confirm the run queue. If memory pressure grew further, swapping would begin and `wa` would rise; that combination is what turns "sluggish" into "unresponsive".

4. **Root cause.** A heavy job runs at the same priority, in the same cgroup, as your interactive session, so it competes equally for CPU and memory.

5. **Fix.** Stop it, then run the same job the right way: in its own cgroup with a low CPU weight, a CPU cap, a memory cap, and a nice value.

```
pkill stress-ng
sudo systemd-run --unit=batchjob --nice=19 -p CPUWeight=20 -p CPUQuota=100% \
    -p MemoryMax=2500M stress-ng --cpu 8 --vm 2 --vm-bytes 1G --vm-keep --timeout 120s
sleep 15
~/labs/5.5/probe.sh
grep some /proc/pressure/cpu
```

```
probe: 0.51 seconds
some avg10=6.20 avg60=20.15 avg300=9.88 total=55120330
```

What just happened: `--nice=19` starts the job with the lowest CPU priority. `CPUWeight=20` gives its cgroup a fifth of the default weight (100) when CPUs are contended. `CPUQuota=100%` caps it at one full CPU out of two, so one CPU always stays free for everything else, and `MemoryMax=2500M` stops it from taking all the RAM. The job still runs, but the probe is close to normal and pressure falls. `avg60` is still high because it includes the earlier minute.

6. See how systemd protects the important services. Each service lives in its own cgroup under `system.slice`; your login lives under `user.slice`.

```
systemctl status ssh --no-pager | grep -A2 CGroup
systemd-cgls --no-pager -u user.slice | head -6
sudo systemd-cgtop -b -n 1 --depth=2 | head -10
cat /proc/$(pgrep -o sshd)/oom_score_adj
```

> **What just happened:** `sshd` runs in `/system.slice/ssh.service` and your session in `user.slice/user-1000.slice/session-N.scope`. When CPUs are busy, slices share CPU by weight, so a job boxed in its own unit cannot take everything. `systemd-cgtop` is a `top` for cgroups. An `oom_score_adj` of `-1000` would mean the OOM killer never chooses that process. `TasksMax=` in a unit (or `ulimit -u` per user) caps the number of processes, the real defense against fork bombs.

7. **Verify** and stop the job.

```
sudo systemctl stop batchjob
~/labs/5.5/probe.sh
```

8. **Prevent.** Run heavy jobs as systemd units or with `systemd-run` and resource limits, use `nice`, leave headroom (do not size a server to 100% of its CPU and RAM), and when a terminal is sluggish, check `uptime`, `vmstat`, and `/proc/pressure/*` first: they tell you in seconds whether CPU, memory, or I/O is the bottleneck. If a system is too unresponsive to log in, use the console, or (if enabled) the magic SysRq key described in Lab 5.2.

> **Checkpoint:** `pgrep stress-ng` prints nothing, the probe is back near your baseline, and `grep some /proc/pressure/cpu` shows `avg10` near 0 after a minute.

Task 6: Slow startup

1. Record how long booting takes now.

```
systemd-analyze
```

```
Startup finished in 2.105s (kernel) + 6.311s (userspace) = 8.416s
graphical.target reached after 6.270s in userspace.
```

> **What just happened:** `systemd-analyze` reports the time spent in the kernel and in userspace (systemd starting services) during the last boot. Ubuntu Server's default target is `graphical.target` even without a desktop. Some hypervisors also show `(firmware)` and `(loader)` times. This is your boot baseline.

2. **Break it.** Create a service that waits 15 seconds before it starts, and enable it at boot. Open `/etc/systemd/system/slowlab.service` with `sudo nano`, or use the heredoc:

```
cat <<'EOF' | sudo tee /etc/systemd/system/slowlab.service
[Unit]
Description=Lab 5.5 deliberately slow service

[Service]
Type=oneshot
ExecStartPre=/bin/sleep 15
ExecStart=/bin/echo "slowlab finished starting"
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable slowlab.service
sudo reboot
```

What just happened: `Type=oneshot` means systemd considers the service started only when `ExecStart` has finished, and `ExecStartPre=/bin/sleep 15` runs first. Because `multi-user.target` *wants* this unit, systemd also orders the target after it, so the target (and `graphical.target` after it) cannot be reached until the 15 seconds are over. `RemainAfterExit=yes` keeps the unit "active" after the command exits. This imitates a real service that waits for a slow network, database, or storage device at boot.

3. **Symptom.** Log in again (wait a minute after the reboot; `systemd-analyze` says "Bootup is not yet finished" until the default target is reached) and check the boot time.

```
systemd-analyze
```

```
Startup finished in 2.098s (kernel) + 21.402s (userspace) = 23.500s
graphical.target reached after 21.367s in userspace.
```

Userspace startup went from about 6 seconds to over 21: **slow startup**.

4. **Investigate.** Find which unit took the time, and whether it was on the critical path.

```
systemd-analyze blame | head -5
systemd-analyze critical-chain
journalctl -b -u slowlab --no-pager
```

```
15.021s slowlab.service
 1.802s systemd-networkd-wait-online.service
  812ms cloud-init.service
  503ms snapd.seeded.service
  ...
graphical.target @21.367s
└─multi-user.target @21.366s
  └─slowlab.service @6.338s +15.021s
    └─basic.target @1.954s
      └─sockets.target @1.953s
...
Oct 10 11:40:02 ubuntu-lab systemd[1]: Starting slowlab.service - Lab 5.5 deliberately slow
service...
Oct 10 11:40:17 ubuntu-lab echo[702]: slowlab finished starting
Oct 10 11:40:17 ubuntu-lab systemd[1]: Finished slowlab.service - Lab 5.5 deliberately slow
service.
```

What just happened: `systemd-analyze blame` lists units by how long they took to start, slowest first. A unit can be slow without delaying boot if nothing waits for it, so always confirm with `systemd-analyze critical-chain`, which shows the chain of units the default target actually waited for: `@6.338s` is when `slowlab` started and `+15.021s` is how long it took. The journal shows 15 seconds between "Starting" and "Finished". `systemd-analyze plot > boot.svg` draws the whole boot as a timeline you can open in a browser.

5. **Root cause.** A service on the boot critical path waits 15 seconds (`ExecStartPre=/bin/sleep 15`) before it reports success.

6. **Fix.** Override the delay with a drop-in instead of editing the original file (the same technique you would use for a vendor unit in `/lib/systemd/system`).

```
sudo mkdir -p /etc/systemd/system/slowlab.service.d
cat <<'EOF' | sudo tee /etc/systemd/system/slowlab.service.d/fix.conf
[Service]
ExecStartPre=
EOF
sudo systemctl daemon-reload
systemctl cat slowlab --no-pager | tail -4
sudo reboot
```

What just happened: An empty `ExecStartPre=` clears the list of pre-start commands. `sudo systemctl edit slowlab` would open the same kind of drop-in in an editor, and `systemctl cat` shows the unit plus every drop-in. Other real fixes: use `Type=exec` or `Type=simple` when nothing needs to wait for a service, add `nofail` (and `_netdev` for network storage) to optional `/etc/fstab` entries, and mark interfaces that may be down `optional: true` in netplan so `systemd-networkd-wait-online` does not stall.

7. **Verify.** After the reboot:

```
systemd-analyze
systemd-analyze blame | head -3
```

Userspace time is back near the baseline and `slowlab.service` is now near the bottom of the list, taking only milliseconds.

8. **Prevent.** Check `systemd-analyze` after every change to services or `/etc/fstab`, keep the boot time in your baseline notes, and disable services you do not need (`systemctl disable --now name`).

> **Checkpoint:** `systemd-analyze critical-chain` no longer lists `slowlab.service` with a `+15s` delay, and `systemctl cat slowlab` shows the `fix.conf` drop-in.

Task 7: High failed log-in attempts

Many failed logins are a security problem, but they are also a performance problem: every SSH connection makes `sshd` start a new process and do cryptographic work, logs grow fast, and `sshd` starts refusing new connections when too many are unauthenticated at once (`MaxStartups`), so real users see slow or dropped logins.

> **CAUTION**
> In step 5 you enable `fail2ban`, which will block **all** SSH from rocky-lab to ubuntu-lab, including an open session. Run the rocky-lab commands from the rocky-lab console or from an SSH session from your host computer to rocky-lab. If you get cut off, wait 10 minutes or run the unban command from the ubuntu-lab console.

1. Make sure `fail2ban` is not running yet on ubuntu-lab (it may still be enabled from Lab 3.3), so you see the raw attempts.

```
sudo systemctl stop fail2ban 2>/dev/null
systemctl is-active fail2ban
```

2. **Break it.** From rocky-lab, try to log in five times as a user that does not exist on ubuntu-lab (`ghost`), typing any wrong password at each prompt. This is a handful of manual attempts against your own lab VM, not a password-guessing tool; never do this against systems you do not own.

```
# on rocky-lab: type any password at each prompt
for i in 1 2 3 4 5; do \
  ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no \
      -o NumberOfPasswordPrompts=1 -o StrictHostKeyChecking=no \
      ghost@ubuntu-lab true; done
```

```
ghost@ubuntu-lab's password:
Permission denied, please try again.
...
```

> **What just happened:** The options force password authentication, allow one password prompt per connection, and skip the host key question. sshd still asks for a password for a non-existent user (so attackers cannot easily tell which usernames exist) and then refuses.

3. **Symptom and investigation.** On ubuntu-lab, read the SSH logs. On Ubuntu 24.04 the service is `ssh.service`, and rsyslog also writes authentication messages to `/var/log/auth.log`.

```
sudo journalctl -u ssh --since "15 min ago" --no-pager | grep -E 'Invalid user|Failed
password'
sudo grep -c 'Failed password' /var/log/auth.log
```

```
Oct 10 12:01:07 ubuntu-lab sshd[2412]: Invalid user ghost from 192.168.100.20 port 49822
Oct 10 12:01:10 ubuntu-lab sshd[2412]: Failed password for invalid user ghost from
192.168.100.20 port 49822 ssh2
Oct 10 12:01:11 ubuntu-lab sshd[2415]: Invalid user ghost from 192.168.100.20 port 49830
Oct 10 12:01:14 ubuntu-lab sshd[2415]: Failed password for invalid user ghost from
192.168.100.20 port 49830 ssh2
...
5
```

What just happened: `journalctl -u ssh` shows messages from the SSH service; `grep -E` keeps lines matching either pattern. "Invalid user" means the account does not exist (a common sign of automated scanning, which tries names like `admin` and `test`); "Failed password for student" would mean a real account with a wrong password. `grep -c` counts matching lines in the classic log file. On Rocky Linux the same messages are in `/var/log/secure` and `journalctl -u sshd`.

4. Summarize who is failing and from where.

```
sudo lastb -n 10
sudo journalctl -u ssh --since today --no-pager | grep 'Failed password' \
    | grep -oE 'from [0-9.]+' | sort | uniq -c | sort -rn
sudo journalctl -u ssh --since today --no-pager | grep -i maxstartups
```

```
ghost    ssh:notty    192.168.100.20   Sat Oct 10 12:01 - 12:01  (00:00)
ghost    ssh:notty    192.168.100.20   Sat Oct 10 12:01 - 12:01  (00:00)
...
btmp begins Sat Oct 10 12:01:07 2026
      5 from 192.168.100.20
```

What just happened: `lastb` reads `/var/log/btmp`, the binary log of failed logins (it needs `sudo`); `-n 10` shows the last 10. The pipeline extracts the source address of every failure (`grep -oE` prints only the matching part), counts each address (`sort | uniq -c`), and sorts by count. Thousands of lines from a few addresses is a brute-force attack. The last command finds `beginning MaxStartups throttling` messages, which mean sshd started dropping new connections; here it prints nothing because five attempts are far too few.

5. **Root cause and fix.** The cause is repeated failed authentication from one source. The fix is to block sources automatically with `fail2ban`, exactly as in Lab 3.3 Task 8.

```
sudo apt install -y fail2ban
cat <<'EOF' | sudo tee /etc/fail2ban/jail.d/sshd.local
[sshd]
enabled = true
backend = systemd
maxretry = 3
findtime = 10m
bantime = 10m
EOF
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
```

What just happened: The jail watches the SSH journal entries and, after 3 failures within 10 minutes (`maxretry` , `findtime`), bans the source address for 10 minutes (`bantime`) with an nftables rule. Lab 3.3 explains each setting in detail.

6. **Verify.** Repeat the five attempts from rocky-lab (step 2). After three failures the next attempts fail at once with `Connection refused` . On ubuntu-lab:

```
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip 192.168.100.20
```

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     3
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     1
   `- Banned IP list:   192.168.100.20
1
```

The ban worked; `unbanip` removes it (prints `1`) so rocky-lab can connect again. Substitute rocky-lab's real IP.

7. **Prevent.** Use SSH keys and set `PasswordAuthentication no` , limit who may log in (`AllowUsers` / `AllowGroups`), lower `MaxAuthTries` , allow SSH only from management networks in the firewall, and alert when the failure count in `auth.log` exceeds your baseline. All of these are covered in Lab 3.3.

Checkpoint: `sudo lastb | grep -c ghost` shows your failed attempts, and `sudo fail2ban-client status sshd` showed rocky-lab's address banned before you unbanned it.

Task 8: High latency, jitter, slow response times, and slow application response

Network scenarios run in an isolated practice network inside ubuntu-lab, built with network namespaces and veth pairs exactly as in Lab 5.3 (see its "How it works" for the background). Your real interface and SSH session are never touched.

```
namespace: client        namespace: router              namespace: server
+------------+  veth   +-------------------------+  veth  +--------------+
| veth-c     |---------| veth-rc        veth-rs  |--------| veth-s       |
| 10.10.1.10 |         | 10.10.1.1    10.10.2.1  |        | 10.10.2.20   |
+------------+         +-------------------------+        | web, iperf3  |
                         tc qdisc goes on veth-rs         +--------------+
```

1. Write the build and tear-down scripts.

```
cat <<'EOF' > ~/labs/5.5/labnet-up.sh
#!/bin/bash
# labnet-up.sh: client <-> router <-> server practice network
set -e
for ns in client router server; do ip netns add "$ns"; done
ip link add veth-c type veth peer name veth-rc
ip link add veth-s type veth peer name veth-rs
ip link set veth-c netns client
ip link set veth-rc netns router
ip link set veth-rs netns router
ip link set veth-s netns server
ip -n client addr add 10.10.1.10/24 dev veth-c
ip -n router addr add 10.10.1.1/24 dev veth-rc
ip -n router addr add 10.10.2.1/24 dev veth-rs
ip -n server addr add 10.10.2.20/24 dev veth-s
for ns in client router server; do ip -n "$ns" link set lo up; done
ip -n client link set veth-c up
ip -n router link set veth-rc up
ip -n router link set veth-rs up
ip -n server link set veth-s up
ip -n client route add default via 10.10.1.1
ip -n server route add default via 10.10.2.1
ip netns exec router sysctl -qw net.ipv4.ip_forward=1
echo "lab network is up"
EOF
cat <<'EOF' > ~/labs/5.5/labnet-down.sh
#!/bin/bash
# labnet-down.sh: stop everything in the namespaces and delete them
for ns in client router server; do
    ip netns pids "$ns" 2>/dev/null | xargs -r kill
    ip netns del "$ns" 2>/dev/null
done
echo "lab network removed"
EOF
chmod 755 ~/labs/5.5/labnet-up.sh ~/labs/5.5/labnet-down.sh
```

What just happened: `labnet-up.sh` creates three namespaces, connects them with two veth pairs (virtual cables), assigns addresses, and turns on forwarding in the router namespace only. `labnet-down.sh` uses `ip netns pids` to find every process running inside each namespace (the test servers you start below), kills them with `xargs -r kill` (`-r` means do nothing if there are none), and deletes the namespaces; the veth pairs disappear with them.

2. Build the network, create a small web page, and start a web server in the server namespace.

```
sudo ~/labs/5.5/labnet-down.sh
sudo ~/labs/5.5/labnet-up.sh
mkdir -p ~/labs/5.5/www && echo "hello from the lab server" > ~/labs/5.5/www/index.html
sudo -v
sudo ip netns exec server python3 -m http.server 8080 \
    --directory ~/labs/5.5/www > /dev/null 2>&1 &
```

What just happened: Running `labnet-down.sh` first removes any leftover namespaces (from Lab 5.3 or an earlier attempt), so `labnet-up.sh` does not fail with "File exists". `sudo -v` refreshes your sudo credentials so the next background `sudo` does not wait invisibly for a password. `python3 -m http.server 8080` is a tiny web server that serves the files in `--directory` on port 8080.

3. Create a helper that shows where the time goes in one web request, then record the baseline.

```
cat <<'EOF' > ~/labs/5.5/webtime.sh
#!/bin/bash
# webtime.sh URL: show where the time goes in one HTTP request
curl -s -o /dev/null -m 10 \
  -w 'connect=%{time_connect}s first-byte=%{time_starttransfer}s total=%{time_total}s\n'
"$1"
EOF
chmod 755 ~/labs/5.5/webtime.sh
sudo ip netns exec client ping -c 10 -i 0.2 -q 10.10.2.20
sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8080/
```

```
--- 10.10.2.20 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1843ms
rtt min/avg/max/mdev = 0.031/0.048/0.092/0.017 ms
connect=0.000231s first-byte=0.002410s total=0.002475s
```

What just happened: `ping -c 10 -i 0.2 -q` sends 10 packets 0.2 seconds apart and prints only the summary. `rtt min/avg/max/mdev` is the round-trip time (to the server and back): minimum, average, maximum, and mean deviation. `mdev` is a good measure of **jitter**. In `webtime.sh`, `curl -s -o /dev/null` downloads quietly and throws the page away, `-m 10` gives up after 10 seconds, and `-w` prints timers: `time_connect` (TCP handshake done, about one round trip), `time_starttransfer` (first byte of the answer received: network plus server processing time), and `time_total`.

Scenario A: high latency

4. **Break it.** Add 150 ms of delay to every packet the router sends toward the server.

```
sudo tc -n router qdisc add dev veth-rs root netem delay 150ms
```

What just happened: `tc` (traffic control) manages **queueing disciplines** (qdiscs), the queues every interface uses for outgoing packets. `netem` (network emulator) is a qdisc that can add delay, jitter, loss, duplication, and reordering. `-n router` runs `tc` inside the router namespace, `dev veth-rs` is the interface toward the server, and `root` makes it the interface's main qdisc. On Ubuntu 24.04 the `sch_netem` module is part of the standard kernel modules package.

5. **Symptom.** Measure again.

```
sudo ip netns exec client ping -c 10 -i 0.2 -q 10.10.2.20
sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8080/
```

```
rtt min/avg/max/mdev = 150.081/150.213/150.402/0.098 ms
connect=0.150612s first-byte=0.301845s total=0.301933s
```

What just happened: Every round trip now costs about 150 ms: **high latency**. The web request needs one round trip for the TCP handshake and one for the request and answer, so it takes about 0.3 seconds instead of 0.002. Applications that make many sequential requests (a web page loading 50 files, a database client running 200 small queries) multiply this delay, which users report as **slow response times**.

6. **Investigate.** Find *where* on the path the delay is added.

```
sudo ip netns exec client ping -c 5 -i 0.2 -q 10.10.1.1
sudo ip netns exec client mtr -n -r -c 10 10.10.2.20
```

```
rtt min/avg/max/mdev = 0.029/0.041/0.060/0.011 ms
Start: 2026-10-10T12:30:41+0000
HOST: ubuntu-lab                  Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.10.1.1                  0.0%    10    0.1   0.1   0.1   0.1   0.0
  2.|-- 10.10.2.20                 0.0%    10  150.2 150.2 150.1 150.4   0.1
```

What just happened: The first hop (the router) answers in under a millisecond, but the server takes 150 ms, so the delay is between the router and the server. `mtr` combines `traceroute` and `ping`: `-n` skips DNS lookups, `-r` prints a report instead of the live screen, and `-c 10` sends 10 probes per hop. `Loss%` is lost probes per hop, `Avg` / `Best` / `Wrst` are latencies, and `StDev` is the jitter. On a real network the hop where latency jumps (and stays high for every later hop) is where to look.

7. Now look at the suspect interface, the router's link toward the server.

```
sudo tc -n router -s qdisc show dev veth-rs
```

```
qdisc netem 8001: root refcnt 2 limit 1000 delay 150ms
 Sent 4810 bytes 51 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
```

> **What just happened:** `tc -s qdisc show` lists the qdiscs with statistics. A `netem` qdisc with `delay 150ms` on a production interface would be a forgotten test setting. On real networks, latency usually comes from distance (a WAN or VPN link), a congested link with deep queues, a wrong route sending traffic the long way, or Wi-Fi/radio problems; the investigation method is the same.

8. **Root cause.** A 150 ms delay on the router-to-server link. **Fix and verify.**

```
sudo tc -n router qdisc del dev veth-rs root
sudo ip netns exec client ping -c 10 -i 0.2 -q 10.10.2.20
```

The average RTT is back below 1 ms.

Scenario B: jitter

9. **Break it.** Add a delay that varies randomly by up to 80 ms in each direction around 100 ms.

```
sudo tc -n router qdisc add dev veth-rs root netem delay 100ms 80ms
sudo ip netns exec client ping -c 20 -i 0.2 10.10.2.20 | tail -4
sudo ip netns exec client mtr -n -r -c 20 10.10.2.20
```

```
64 bytes from 10.10.2.20: icmp_seq=19 ttl=63 time=34.6 ms
64 bytes from 10.10.2.20: icmp_seq=20 ttl=63 time=171.2 ms
20 packets transmitted, 20 received, 0% packet loss, time 3811ms
rtt min/avg/max/mdev = 22.803/101.611/176.940/46.218 ms
HOST: ubuntu-lab                  Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.10.1.1                  0.0%    20    0.1   0.1   0.1   0.1   0.0
  2.|-- 10.10.2.20                 0.0%    20   58.3  99.8  21.0 177.5  47.1
```

> **What just happened:** The average is about 100 ms, but individual packets take anywhere from about 20 to 180 ms. That variation is **jitter**, visible as a large `mdev` in `ping` and a large `StDev` in `mtr`. File downloads hardly notice jitter, but voice calls, video, online games, and remote desktops become choppy because packets arrive irregularly (and, with heavy jitter, out of order). Real causes are congestion that comes and goes, overloaded routers or firewalls, Wi-Fi interference, and a busy VM host.

10. **Fix and verify.** Remove the qdisc and confirm that `mdev` is small again.

```
sudo tc -n router qdisc del dev veth-rs root
sudo ip netns exec client ping -c 20 -i 0.2 -q 10.10.2.20
```

Scenario C: slow application response (the network is fine)

11. **Break it.** Start a second web application on the server that takes 2 seconds to build each answer, like an application waiting on a slow database query. Save it as `~/labs/5.5/slowapp.py`:

```
cat <<'EOF' > ~/labs/5.5/slowapp.py
import http.server
import time

class SlowHandler(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(2)                # simulate a slow database query
        body = b"report ready\n"
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

http.server.ThreadingHTTPServer(("0.0.0.0", 8081), SlowHandler).serve_forever()
EOF
sudo -v
sudo ip netns exec server python3 ~/labs/5.5/slowapp.py > /dev/null 2>&1 &
```

12. **Symptom and investigation.** Compare the static page (port 8080) with the application (port 8081).

```
sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8080/
sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8081/
sudo ip netns exec client ping -c 5 -i 0.2 -q 10.10.2.20
pidstat -u -C python3 2 1
```

```
connect=0.000204s first-byte=0.002201s total=0.002260s
connect=0.000198s first-byte=2.004110s total=2.004180s
rtt min/avg/max/mdev = 0.030/0.045/0.071/0.014 ms
Average:      UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
Average:        0      4120    0.00    0.00    0.00    0.00    0.00     -  python3
```

What just happened: The connection to port 8081 is set up in 0.2 ms, so the network is fast, but the first byte arrives 2 seconds later. A fast `time_connect` with a slow `time_starttransfer` means the time is spent *inside the server application*: **slow application response**, not a network problem. `ping` confirms the network is healthy. `pidstat -C python3` shows the application is not even using CPU, so it is *waiting* for something (here a `sleep`; in real life a database, a lock, an external API, or DNS). The next step on a real system would be the application's own logs, `strace -p PID` to see what it waits for, or the database's slow-query log.

13. **Root cause:** the application spends 2 seconds per request waiting. **Fix:** remove the delay in the code and restart the application.

```
sudo ip netns pids server | xargs -r sudo kill
sed -i 's/time.sleep(2)/time.sleep(0)/' ~/labs/5.5/slowapp.py
sudo -v
sudo ip netns exec server python3 ~/labs/5.5/slowapp.py > /dev/null 2>&1 &
sudo ip netns exec server python3 -m http.server 8080 \
    --directory ~/labs/5.5/www > /dev/null 2>&1 &
sleep 1
sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8081/
```

> **What just happened:** `ip netns pids server` lists every process in the server namespace, and `xargs -r sudo kill` stops them (both web servers). `sed -i` changes the sleep to zero, and both servers are started again. `first-byte` is now a few milliseconds: **verified**.

14. **Prevent.** Measure response time from the user's side (`curl -w`, synthetic checks, application metrics) and keep a baseline for it, so you can tell at once whether the network part (`time_connect`) or the application part (`time_starttransfer` minus `time_connect`) changed.

> **Checkpoint:** `sudo tc -n router qdisc show dev veth-rs` shows no `netem` line, `ping` RTT is below 1 ms, and `webtime.sh` for port 8081 prints a `first-byte` value in milliseconds. Leave the practice network running for Task 9.

Task 9: Packet drops, random timeouts, random disconnects, and low throughput

Use the practice network from Task 8. If you rebooted or removed it, run `sudo ~/labs/5.5/labnet-up.sh` and start the web server on port 8080 again (Task 8, step 2).

Scenario A: packet drops

1. **Break it.** Make the router drop 15% of the packets it sends to the server.

```
sudo tc -n router qdisc add dev veth-rs root netem loss 15%
```

2. **Symptom and investigation.**

```
sudo ip netns exec client ping -c 100 -i 0.1 -q 10.10.2.20
sudo tc -n router -s qdisc show dev veth-rs
for i in $(seq 10); do \
  sudo ip netns exec client ~/labs/5.5/webtime.sh http://10.10.2.20:8080/; done
```

```
--- 10.10.2.20 ping statistics ---
100 packets transmitted, 86 received, 14% packet loss, time 9915ms
rtt min/avg/max/mdev = 0.027/0.046/0.101/0.013 ms
qdisc netem 8002: root refcnt 2 limit 1000 loss 15%
 Sent 9044 bytes 103 pkt (dropped 17, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
connect=0.000218s first-byte=0.002390s total=0.002461s
connect=1.032870s first-byte=1.035011s total=1.035090s
connect=0.000201s first-byte=0.212650s total=0.212731s
...
```

> **What just happened:** About 15% of pings get no reply, and the qdisc counts the **packet drops** (`dropped 17`). TCP hides losses by retransmitting, so web requests still succeed, but some take 0.2 seconds (a retransmitted segment) or over 1 second (a lost connection request). On real interfaces, check `ip -s link show dev IF` (`dropped`, `errors`, `missed`), `ethtool -S IF` (driver counters such as CRC errors), and `nstat -az TcpRetransSegs`; counts rising faster than in your baseline mean loss somewhere.

3. **Root cause:** a lossy link (here `netem loss 15%`; in real life a bad cable or transceiver, a duplex mismatch, a full queue on a congested link, or a firewall dropping traffic). **Fix and verify:**

```
sudo tc -n router qdisc del dev veth-rs root
sudo ip netns exec client ping -c 100 -i 0.1 -q 10.10.2.20
```

   Loss is back to `0%`.

Scenario B: random timeouts and random disconnects

4. Prepare a long-lived connection to watch: a small server in the server namespace that sends a timestamp to each client every second. Save it as `~/labs/5.5/tick.py`:

```
cat <<'EOF' > ~/labs/5.5/tick.py
import socket
import threading
import time

srv = socket.socket()
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("0.0.0.0", 9000))
srv.listen()

def talk(conn):
    try:
        while True:
            conn.sendall(time.strftime("%H:%M:%S tick\n").encode())
            time.sleep(1)
    except OSError:
        conn.close()

while True:
    conn, addr = srv.accept()
    threading.Thread(target=talk, args=(conn,), daemon=True).start()
EOF
sudo -v
sudo ip netns exec server python3 ~/labs/5.5/tick.py > /dev/null 2>&1 &
```

5. **Break it.** Create a "flaky link" script that, at random intervals, drops *every* packet for 5 seconds (a burst of loss, like a link that briefly fails), and run it for 3 minutes.

```
cat <<'EOF' > ~/labs/5.5/flap.sh
#!/bin/bash
# flap.sh: every 15-25 seconds, drop ALL packets for 5 seconds
tc -n router qdisc add dev veth-rs root netem delay 1ms
while true; do
    sleep $((15 + RANDOM % 10))
    tc -n router qdisc change dev veth-rs root netem delay 1ms loss 100%
    sleep 5
    tc -n router qdisc change dev veth-rs root netem delay 1ms
done
EOF
chmod 755 ~/labs/5.5/flap.sh
sudo -v
sudo timeout 180 ~/labs/5.5/flap.sh &
```

What just happened: The script adds a nearly harmless `netem` qdisc, then in a loop waits a random 15 to 24 seconds (`$RANDOM` is a random number; `% 10` keeps the remainder 0 to 9), switches to `loss 100%` for 5 seconds, and switches back. `qdisc change` modifies the existing qdisc in place. `timeout 180` stops the script after 3 minutes no matter what. A `netem` option for realistic bursty loss also exists (`netem loss gemodel 1% 10% 100% 0%` , the Gilbert-Elliott model), but its bursts are random in length, which makes a lab harder to follow.

6. **Symptom: random timeouts.** Make one web request per second with a 2-second timeout.

```
for i in $(seq 40); do
    if sudo ip netns exec client curl -s -o /dev/null -m 2 http://10.10.2.20:8080/; then
        echo "$(date +%T) ok"
    else
        echo "$(date +%T) TIMEOUT"
    fi
    sleep 1
done
```

```
12:51:02 ok
12:51:03 ok
...
12:51:21 TIMEOUT
12:51:23 TIMEOUT
12:51:25 ok
...
```

What just happened: `curl -m 2` gives up after 2 seconds and exits with an error, so the `if` prints `TIMEOUT` . Most requests work; a few fail in clusters at unpredictable times. That is the classic **random timeouts** report: “it works when I try it, but users see errors now and then”.

7. **Symptom: random disconnects.** Connect to the tick server with a client that gives up after 4 seconds of silence (like an application or SSH client with keepalive settings).

```
sudo ip netns exec client nc -w 4 10.10.2.20 9000; echo "$(date +%T) disconnected"
```

```
12:52:40 tick
12:52:41 tick
...
12:52:58 tick
12:53:03 disconnected
```

What just happened: `nc -w 4` (OpenBSD netcat) closes the connection when it has been idle for 4 seconds. The ticks arrive every second until the next loss burst; after 4 silent seconds the client drops the connection. This is a **random disconnect**: SSH sessions freezing and closing ("Broken pipe", "Connection reset", "client_loop: send disconnect"), database connections dropping, VPNs reconnecting. If `flap.sh` already stopped (3 minutes), run it again.

8. **Investigate.** Watch the pattern from the client with `ping -O`, which reports every missing reply.

```
sudo ip netns exec client ping -O -i 0.5 -c 80 10.10.2.20 | grep -E 'no answer|statistics' -A1
sudo tc -n router -s qdisc show dev veth-rs
```

```
no answer yet for icmp_seq=37
no answer yet for icmp_seq=38
...
no answer yet for icmp_seq=46
--- 10.10.2.20 ping statistics ---
80 packets transmitted, 70 received, 12% packet loss, time 39805ms
qdisc netem 8003: root refcnt 2 limit 1000 delay 1ms loss 100%
```

What just happened: `ping -O` prints "no answer yet" for each lost packet, so you see *bursts* of about 10 consecutive losses (5 seconds) separated by healthy periods. Bursts are the signature of something that briefly fails: a flapping link or bad cable, a Wi-Fi roam, an IP address conflict (Lab 5.3), or an overloaded firewall. On a real host also check `journalctl -k | grep -i 'link is'` and `ip -s link` for carrier changes at the same times. Disconnects *without* loss often come from firewall or NAT idle timeouts; TCP keepalives (`ServerAliveInterval` for SSH) solve those.

9. **Root cause:** periodic 5-second total loss on the router-to-server link. **Fix and verify:** stop the script and remove the qdisc, then rerun the `curl` loop from step 6; every line says `ok`.

```
sudo pkill -x flap.sh
sudo tc -n router qdisc del dev veth-rs root
```

What just happened: `pkill -x flap.sh` matches the process *name* exactly (a script's process name is its file name), so it does not accidentally match the `sudo` command line. If the 3-minute timeout already ended the script, `pkill` simply finds nothing.

Scenario C: low throughput

10. Measure baseline throughput with `iperf3`, which sends as much data as it can over TCP and reports the rate.

```
sudo ip netns exec server iperf3 -s -D
sudo ip netns exec client iperf3 -c 10.10.2.20 -t 5
```

```
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-5.00   sec  12.1 GBytes  20.8 Gbits/sec    0             sender
[  5]   0.00-5.00   sec  12.1 GBytes  20.8 Gbits/sec                  receiver
```

What just happened: `iperf3 -s -D` starts a server as a daemon (in the background) in the server namespace. `iperf3 -c 10.10.2.20 -t 5` connects to it and sends data for 5 seconds. veth pairs are limited only by CPU speed, so you see many gigabits per second. `Retr` is the number of TCP retransmissions, a sign of loss.

11. **Break it.** Limit the router's link to the server to 20 megabits per second with a **token bucket filter** (`tbf`), the same thing an ISP or a misconfigured QoS policy does.

```
sudo tc -n router qdisc add dev veth-rs root tbf rate 20mbit burst 32kb latency 50ms
sudo ip netns exec client iperf3 -c 10.10.2.20 -t 5
sudo ip netns exec client iperf3 -c 10.10.2.20 -t 5 -R
```

```
[  5]   0.00-5.00   sec  12.2 MBytes  20.5 Mbits/sec   89             sender
[  5]   0.00-5.00   sec  11.4 MBytes  19.1 Mbits/sec                  receiver
...
[  5]   0.00-5.00   sec  12.0 GBytes  20.6 Gbits/sec    0             sender
```

What just happened: `tbf rate 20mbit` lets data out at 20 Mbit/s on average, `burst 32kb` allows short bursts of 32 KB, and `latency 50ms` drops packets that would wait longer than 50 ms in the queue (that is where the `Retr` count comes from). Client-to-server throughput collapsed to about 20 Mbit/s: **low throughput**. `-R` (reverse) makes the server send to the client instead; that direction is still fast, because only the router's outgoing queue toward the server is limited. Testing both directions is a quick way to tell a one-way problem (shaping, a policer, a duplex mismatch) from a general one.

12. **Investigate.** Check the qdisc and the interface counters.

```
sudo tc -n router -s qdisc show dev veth-rs
sudo ip -n router -s link show dev veth-rs
```

```
qdisc tbf 8004: root refcnt 2 rate 20Mbit burst 32Kb lat 50ms
 Sent 25480120 bytes 16851 pkt (dropped 178, overlimits 21094 requeues 0)
 backlog 0b 0p requeues 0
```

What just happened: `overlimits` counts packets that had to wait because they exceeded the rate, and `dropped` counts packets thrown away. On physical servers, also check the negotiated speed (`ethtool IF` showing `Speed: 100Mb/s` instead of `1000Mb/s`), duplex mismatches, packet loss, and latency: a single TCP stream can keep only so much data in flight per round trip, so compare with `iperf3 -P 4` (4 parallel streams).

13. **Root cause:** a rate limit of 20 Mbit/s on the path. **Fix and verify.**

```
sudo tc -n router qdisc del dev veth-rs root
sudo ip netns exec client iperf3 -c 10.10.2.20 -t 5
```

    Throughput is back to gigabits per second.

14. **Prevent** (all three scenarios). Keep `ping`, `mtr`, and `iperf3` results in your baseline, monitor interface `dropped` / `errors` counters and TCP retransmissions, and document any intentional shaping so the next admin does not chase it.

15. Undo the practice network.

```
sudo ~/labs/5.5/labnet-down.sh
sudo ip netns list
```

    `ip netns list` prints nothing.

> **Checkpoint:** `sudo ip netns list` is empty, `pgrep -x flap.sh` and `pgrep iperf3` print nothing, and `ip link` shows no `veth` interfaces.

Task 10: Slow remote storage response (NFS)

Here you mount a real NFS share from rocky-lab and slow down only the NFS traffic, so you can compare a healthy and a slow storage path while SSH and `ping` keep working normally.

1. Export a directory from rocky-lab.

```
# on rocky-lab
sudo dnf install -y nfs-utils
sudo mkdir -p /srv/nfslab
sudo chmod 1777 /srv/nfslab
echo '/srv/nfslab ubuntu-lab(rw,sync)' | sudo tee /etc/exports.d/nfslab.exports
sudo systemctl enable --now nfs-server
sudo exportfs -rv
sudo firewall-cmd --permanent --add-service=nfs
sudo firewall-cmd --reload
```

> **What just happened:** `nfs-utils` provides the NFS server. `chmod 1777` lets any user write files (with the sticky bit, so users cannot delete each other's files). The export line shares `/srv/nfslab` with ubuntu-lab, read-write, with `sync` (the server confirms writes only after they are on disk). `exportfs -rv` re-reads all export files and lists them, and the firewall rule opens TCP port 2049 (NFSv4 needs only that port).

2. Mount it on ubuntu-lab and measure normal performance.

```
sudo apt install -y nfs-common
sudo mkdir -p /mnt/nfslab
sudo mount -t nfs4 rocky-lab:/srv/nfslab /mnt/nfslab
nfsstat -m
time dd if=/dev/zero of=/mnt/nfslab/test.bin bs=64k count=100 oflag=direct
```

```
/mnt/nfslab from rocky-lab:/srv/nfslab
 Flags:
rw,relatime,vers=4.2,rsize=262144,wsize=262144,namlen=255,hard,proto=tcp,timeo=600,retrans=2
,sec=sys,clientaddr=192.168.100.10,local_lock=none,addr=192.168.100.20
6553600 bytes (6.6 MB, 6.2 MiB) copied, 0.262 s, 25.0 MB/s
real    0m0.266s
```

What just happened: `nfs-common` provides the NFS client, `mount.nfs4`, `nfsstat`, `nfsiostat`, and `mountstats`. `nfsstat -m` shows the mount options actually in use: NFS version 4.2, `hard` (keep retrying forever if the server stops answering), `timeo=600` (60 seconds, in tenths of a second, before a retry), and `retrans=2`. The `dd` writes 100 blocks of 64 KB with `oflag=direct`, so each block is a separate synchronous NFS WRITE; the whole test takes about a quarter of a second.

3. **Break it.** On ubuntu-lab, add 100 ms of delay to packets going to rocky-lab's NFS port only. This uses a `prio` qdisc with an extra fourth band that normal traffic never uses, and a filter that sends only NFS packets into it.

CAUTION

This step changes the qdisc on your real network interface. The filter matches only traffic to rocky-lab's TCP port 2049, so SSH is not affected. If anything goes wrong, `sudo tc qdisc del dev "$IF" root` (from the console if needed) restores the default immediately.

```
IF=$(ip route show default | awk '{print $5; exit}')
ROCKY=$(getent ahostsv4 rocky-lab | awk '{print $1; exit}')
echo "interface=$IF rocky=$ROCKY"
sudo tc qdisc add dev "$IF" root handle 1: prio bands 4
sudo tc qdisc add dev "$IF" parent 1:4 handle 40: netem delay 100ms
sudo tc filter add dev "$IF" parent 1: protocol ip prio 1 u32 \
    match ip dst "$ROCKY"/32 match ip dport 2049 0xffff flowid 1:4
```

What just happened: The first two lines find your main interface (the fifth word of the default route) and rocky-lab's IPv4 address. `prio bands 4` creates a priority qdisc with four bands (sub-queues); its default map only ever puts traffic in bands 1 to 3, so band 4 (`1:4`) is empty unless a filter sends packets there. The `netem delay 100ms` qdisc is attached to band 4. The `u32` filter matches packets whose destination IP is rocky-lab *and* whose destination port is 2049 (`0xffff` means all 16 bits of the port must match) and sends them to band 4 (`flowid 1:4`).

4. **Symptom.**

```
time dd if=/dev/zero of=/mnt/nfslab/test.bin bs=64k count=100 oflag=direct
ping -c 3 -q rocky-lab
```

```
6553600 bytes (6.6 MB, 6.2 MiB) copied, 10.31 s, 636 kB/s
real    0m10.318s
rtt min/avg/max/mdev = 0.412/0.455/0.501/0.037 ms
```

The same write now takes 10 seconds instead of 0.27: **slow remote storage response**. Yet `ping` to the storage server is perfectly fast, which is exactly the confusing picture you often get in real life.

5. **Investigate.** Run `nfsiostat` while a write is in progress. It works like `iostat`, but per NFS mount.

```
dd if=/dev/zero of=/mnt/nfslab/test.bin bs=64k count=100 oflag=direct 2> /dev/null &
nfsiostat 4 2 /mnt/nfslab
```

```
rocky-lab:/srv/nfslab mounted on /mnt/nfslab:

           ops/s       rpc bklog
           9.750           0.000

read:              ops/s            kB/s           kB/op         retrans    avg RTT (ms)
avg exe (ms)  avg queue (ms)          errors
                   0.000           0.000           0.000        0 (0.0%)           0.000
0.000           0.000        0 (0.0%)
write:             ops/s            kB/s           kB/op         retrans    avg RTT (ms)
avg exe (ms)  avg queue (ms)          errors
                   9.750         624.000          64.000        0 (0.0%)         101.204
101.390          0.028        0 (0.0%)
```

What just happened: `nfsiostat 4 2` prints two reports 4 seconds apart; read the second (the first covers the time since mounting). **avg RTT** is the time from sending a request to getting the reply (network plus server). **avg exe** is the total time on the client, including queueing before sending. RTT is about 100 ms per 64 KB write and exe is almost the same, so the client is not queuing: the delay is in the network or the server. If exe were much larger than RTT, the client would be the bottleneck. `retrans` above 0 means requests were lost and re-sent.

6. Get more detail, then check the server side to rule it out.

```
mountstats /mnt/nfslab | grep -A3 '^WRITE:'
nfsstat -c | head -6
```

```
# on rocky-lab
uptime
vmstat 2 3
```

What just happened: `mountstats` shows per-operation averages (RTT and execute time for WRITE, READ, GETATTR, LOOKUP, and so on), and `nfsstat -c` shows client RPC totals including `retrans`. On rocky-lab the load is near zero and the CPUs and disk are idle, so the server is not slow. Fast server + fast `ping` + high NFS RTT = something on the path treats NFS differently (QoS, shaping, a deep-inspection firewall, a separate storage route). On ubuntu-lab, `tc qdisc show dev "$IF"` reveals the `netem` qdisc.

7. **Root cause.** NFS requests to rocky-lab are delayed 100 ms each on the client's outgoing interface. Every synchronous write waits a full round trip, so 100 writes take 10 seconds. **Fix:**

```
sudo tc qdisc del dev "$IF" root
tc qdisc show dev "$IF"
```

> **What just happened:** Deleting the root qdisc removes the `prio` qdisc, its `netem` child, and the filter in one step; the interface goes back to its default qdisc (`fq_codel` , or `mq` with `fq_codel` children on a multi-queue NIC).

8. **Verify.**

```
time dd if=/dev/zero of=/mnt/nfslab/test.bin bs=64k count=100 oflag=direct
nfsiostat 2 2 /mnt/nfslab
```

   The write is back to a fraction of a second, and avg RTT in the second report is below 1 ms (or 0 if there was no NFS traffic during the interval).

9. **Prevent.** Keep NFS RTT in your baseline, use a dedicated storage network where possible, mount network filesystems with `_netdev` (and `nofail` if they are optional) in `/etc/fstab` so a slow server does not cause a slow startup, and watch the kernel log for `nfs: server rocky-lab not responding, still trying` , which appears when a `hard` mount stops getting answers.

10. Undo the scenario.

```
rm -f /mnt/nfslab/test.bin
sudo umount /mnt/nfslab
```

```
# on rocky-lab
sudo rm -f /etc/exports.d/nfslab.exports
sudo exportfs -rv
sudo systemctl disable --now nfs-server
sudo firewall-cmd --permanent --remove-service=nfs
sudo firewall-cmd --reload
sudo rm -rf /srv/nfslab
```

> **Checkpoint:** `findmnt /mnt/nfslab` prints nothing, `tc qdisc show dev "$IF"` shows no `prio` or `netem` qdisc, and on rocky-lab `sudo exportfs` prints nothing.

Task 11: Hardware errors and exceeding baselines

Part A: hardware errors

Failing hardware often shows up first as a performance problem: a disk with bad sectors retries each read for seconds, a NIC with a bad cable drops packets, ECC memory corrects errors, and a CPU throttles when it overheats. The evidence is in the kernel log.

1. Search the kernel log for errors from this boot.

```
journalctl -k -p err -b --no-pager | tail -10
sudo dmesg -T --level=err,warn | tail -10
journalctl -k -b --no-pager | grep -i -E 'mce|machine check|edac|hardware error|i/o
error|thermal|throttl'
```

What just happened: `journalctl -k -p err -b` shows kernel messages of priority `err` and more severe, from the current boot. `dmesg -T --level=err,warn` shows the kernel ring buffer with readable times, errors and warnings only. The `grep` looks for the usual signs of **hardware errors**: `mce` and `machine check` (the CPU reported a hardware fault in itself, its cache, or memory), `EDAC` (error detection and correction: memory controller errors), `I/O error` (disk or controller problems), and thermal throttling. A healthy VM usually prints few or no lines here.

2. Install and query `rasdaemon`, the RAS (reliability, availability, serviceability) daemon that records hardware error events in a database.

```
sudo apt install -y rasdaemon smartmontools
sudo systemctl enable --now rasdaemon
sudo ras-mc-ctl --status
sudo ras-mc-ctl --summary
sudo ras-mc-ctl --errors
```

```
ras-mc-ctl: drivers not loaded.
No Memory errors.

No PCIe AER errors.

No MCE errors.
No Memory errors.
...
```

What just happened: `rasdaemon` listens for kernel hardware error events (memory, PCIe AER, MCE, disk) and stores them in `/var/lib/rasdaemon/`. `ras-mc-ctl --status` reports whether EDAC memory controller drivers are loaded; in a VM there is no real memory controller, so they are not. `--summary` counts errors by type and `--errors` lists each event. On a physical server, a growing count of corrected memory errors on one DIMM means "replace that DIMM soon", before it starts causing crashes. The older `mcelog` tool does a similar job on some distributions.

3. Check disk health with SMART.

```
lsblk -d -o NAME,MODEL,SIZE
sudo smartctl -i /dev/sda
sudo smartctl -H /dev/sda
```

```
Device Model:     VBOX HARDDISK
...
SMART support is: Unavailable - device lacks SMART capability.
```

What just happened: `smartctl -i` shows the disk identity and whether it supports SMART (Self-Monitoring, Analysis and Reporting Technology), and `-H` asks for the overall health verdict. Virtual disks usually do not support SMART (a `virtio` disk like `/dev/vda` may even report "Unable to detect device type"). On a physical disk, `sudo smartctl -a /dev/sda` shows attributes such as `Reallocated_Sector_Ct`, `Current_Pending_Sector`, and `UDMA_CRC_Error_Count`; any non-zero, growing value means the disk or its cable is failing. Lab 5.2 covers device failures in more depth.

4. **Explore/simulate** an I/O error and see what it looks like in the log and to a user.

TIP

Explore/simulate: A VM cannot grow real bad sectors, so you build a small fake disk with a device-mapper `error` target in the middle (the same technique Lab 5.2 uses), read through it, and look at the evidence. The loop file lives in your lab directory and is deleted afterwards.

```
cd ~/labs/5.5
truncate -s 20M hw.img
LOOP=$(sudo losetup --find --show hw.img)
printf '0 20480 linear %s 0\n20480 2048 error\n22528 18432 linear %s 22528\n' \
    "$LOOP" "$LOOP" | sudo dmsetup create badlab
sudo dd if=/dev/mapper/badlab of=/dev/null bs=1M
journalctl -k --since "2 min ago" --no-pager | grep -i error | tail -3
```

```
dd: error reading '/dev/mapper/badlab': Input/output error
10+0 records in
10+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 0.0213 s, 492 MB/s
Oct 10 13:40:12 ubuntu-lab kernel: Buffer I/O error on dev dm-0, logical block 2560, async
page read
```

What just happened: `losetup --find --show` attaches the 20 MB file as a loop device, and the `dmsetup` table (in 512-byte sectors) maps it normally except for 1 MB in the middle (sectors 20480 to 22527), which returns errors. `dd` stops at exactly 10 MiB with `Input/output error`, and the kernel logs a `Buffer I/O error`. A real failing disk also logs `ata` or `nvme` resets and retries, and each retry can take seconds, so a dying disk often looks like extreme disk latency before it fails outright.

5. Remove the fake disk.

```
sudo dmsetup remove badlab
sudo losetup -d "$LOOP"
rm -f ~/labs/5.5/hw.img
```

Part B: exceeding baselines

6. sysstat has been sampling every 2 minutes since Task 1, so it recorded every scenario. Look at today's CPU and load history. `LC_ALL=C` forces 24-hour times so the columns are always in the same place.

```
LC_ALL=C sar -u | head -15
LC_ALL=C sar -q | head -15
```

```
10:10:00        CPU     %user     %nice   %system   %iowait    %steal     %idle
10:12:00        all      0.41      0.00      0.30      0.02      0.00     99.27
10:24:00        all     72.13      0.00      0.51      0.00      0.00     27.36
11:06:00        all      0.92      0.00      8.81     86.40      0.00      3.87
...
10:10:00      runq-sz  plist-sz   ldavg-1   ldavg-5  ldavg-15   blocked
10:12:00            0       187      0.04      0.03      0.00         0
10:24:00            4       195      4.01      2.31      0.98         0
11:06:00            0       211     15.81      7.12      2.86        16
```

What just happened: With no file argument, `sar` reads today's file `/var/log/sysstat/saDD`. `-u` is CPU use, `-q` is the run queue size, the number of processes, the three load averages, and `blocked` (tasks waiting for I/O). You can see each scenario: the CPU test (high `%user`, `runq-sz` 4), and the disk test (high `%iowait`, `blocked` 16). Each line is an average over the 2-minute interval, so short spikes look smaller than they felt.

7. Look at the other resources and narrow the time window.

```
LC_ALL=C sar -r | tail -4
LC_ALL=C sar -W -s 10:30:00 -e 11:00:00
LC_ALL=C sar -b | tail -4
LC_ALL=C sar -d -p | grep -E 'DEV|sdb' | head -6
LC_ALL=C sar -n DEV | grep -E 'IFACE|enp' | tail -3
LC_ALL=C sar -w | tail -3
ls /var/log/sysstat/
LC_ALL=C sar -q -f /var/log/sysstat/sa$(date +%d) | tail -3
```

What just happened: `-r` is memory, `-W` swap paging, `-b` overall I/O, `-d -p` per-disk statistics with readable names (`await`, `aqu-sz`, `%util`), `-n DEV` per-interface traffic (`-n EDEV` for errors and drops), and `-w` process creations and context switches. `-s` / `-e` limit the time window, and `-f` reads another day's file (for example `sa09`); Ubuntu keeps 7 days (`HISTORY` in `/etc/sysstat/sysstat`). The practice network does not appear in `sar -n DEV` because sysstat sees only the main network namespace.

8. Find the moments when the system was **exceeding baselines**: here, when the 1-minute load was higher than the number of CPUs, and when the CPU was less than 20% idle.

```
LC_ALL=C sar -q | awk -v cpus="$(nproc)" '$4+0 > cpus'
LC_ALL=C sar -u | awk '$NF ~ /^[0-9.]+$/ && $NF+0 < 20'
```

```
10:24:00            4       195      4.01      2.31      0.98         0
11:06:00            0       211     15.81      7.12      2.86        16
11:20:00           10       214      9.85      4.10      1.66         0
10:24:00        all     72.13      0.00      0.51      0.00      0.00     27.36
11:06:00        all      0.92      0.00      8.81     86.40      0.00      3.87
11:20:00        all     93.05      0.00      6.70      0.00      0.00      0.25
```

What just happened: `awk` reads each line as columns. In `sar -q` output, column 4 is `ldavg-1`; `$4+0 > cpus` prints the lines where it exceeds the CPU count passed in with `-v cpus=...` (header lines have text in column 4, which counts as 0). In `sar -u`, `$NF` is the last column, `%idle`; the regular expression keeps only numeric values. These are the time stamps to investigate, and the 11:06 line tells you at once that it was an I/O problem (`%iowait` 86) rather than CPU.

9. Compare a fresh snapshot with the baseline file from Task 1.

```
~/labs/5.5/baseline.sh > ~/labs/5.5/now.txt
~/labs/5.5/probe.sh >> ~/labs/5.5/now.txt
diff ~/labs/5.5/baseline.txt ~/labs/5.5/now.txt | less
```

What just happened: `diff` shows the lines that changed between the two snapshots (press `q` to leave `less`). Small differences are normal; after the scenarios every value should be back in the baseline's range. One that is not (`si` / `so` above zero, a probe twice as slow) is the metric to investigate. **Prevent:** update the baseline after every planned change, keep sysstat running on every server, and turn the key numbers (load per CPU, `%iowait`, `await`, swap activity, packet drops, failed log-ins, NFS RTT) into monitoring alerts.

Checkpoint: `sudo dmsetup ls` does not list `badlab`, `sudo ras-mc-ctl --summary` runs, and `LC_ALL=C sar -q | awk -v cpus="$(nproc)" '$4+0 > cpus'` lists the time stamps of your overload scenarios.

Challenge

CHALLENGE

1. Ask a friend (or write a script while not looking at it later) to start one hidden problem on ubuntu-lab with a 5-minute timeout: a `stress-ng --cpu`, `--vm`, or `--switch` load, a `fio` writer, or a `netem` qdisc in the practice network. Using only `uptime`, `vmstat`, `iostat`, `pidstat`, `/proc/pressure/*`, and `tc -s qdisc`, identify which resource is affected and the responsible process or interface within two minutes.
2. Create a permanent service `/etc/systemd/system/nightly-report.service` that runs `stress-ng --cpu 2 --timeout 60s` with `Nice=19`, `CPUQuota=50%`, `MemoryMax=300M`, and `IOWeight=10`, start it, and prove with `systemd-cgtop` and `mpstat` that the limits apply. Remove it afterwards.
3. Use `sadf -d -- -q` (sysstat's export tool) to write today's load history as semicolon-separated values to a file, and identify the busiest 10-minute window of the day from it.

Clean up

1. Stop any load generators, transient units, and test services that may still be running.

```
pkill stress-ng
pkill -x fio
for u in cpuhog memhog noisy batchjob; do sudo systemctl stop "$u" 2>/dev/null; done
sudo systemctl reset-failed
```

2. Remove the practice network and any qdisc on your real interface.

```
sudo ~/labs/5.5/labnet-down.sh
IF=$(ip route show default | awk '{print $5; exit}')
sudo tc qdisc del dev "$IF" root 2>/dev/null
tc qdisc show dev "$IF"
```

   The last command should show only the default qdisc (`fq_codel` or `mq`), not `prio` or `netem`.

3. Remove the slow test unit and its drop-in.

```
sudo systemctl disable slowlab.service
sudo rm -rf /etc/systemd/system/slowlab.service /etc/systemd/system/slowlab.service.d
sudo systemctl daemon-reload
```

4. Undo the fail2ban changes (and unban rocky-lab if it is still banned). No test user was created (`ghost` never existed), but remove `baituser` if it is left over from Lab 3.3.

```
sudo fail2ban-client set sshd unbanip 192.168.100.20 2>/dev/null
sudo systemctl disable --now fail2ban
sudo rm -f /etc/fail2ban/jail.d/sshd.local
id baituser 2>/dev/null && sudo userdel -r baituser
```

5. Make sure the NFS mount and export are gone (repeat Task 10 step 10 if you skipped it).

```
sudo umount /mnt/nfslab 2>/dev/null
sudo rmdir /mnt/nfslab /mnt/perf 2>/dev/null
```

```
# on rocky-lab
sudo rm -f /etc/exports.d/nfslab.exports
sudo exportfs -ra
sudo systemctl disable --now nfs-server
sudo firewall-cmd --permanent --remove-service=nfs
sudo firewall-cmd --reload
sudo rm -rf /srv/nfslab
```

6. Wipe the extra disk, remove the fake bad disk, and remove the temporary swap file if you created one. Substitute your disk name.

```
sudo umount /mnt/perf 2>/dev/null
sudo wipefs -a /dev/sdb
sudo dmsetup remove badlab 2>/dev/null
sudo swapoff /swap-lab.img 2>/dev/null
sudo rm -f /swap-lab.img
```

7. Put sysstat back to its normal 10-minute schedule (keeping collection on is harmless and useful; to turn it off completely, also run `sudo systemctl disable --now sysstat sysstat-collect.timer sysstat-summary.timer`), stop `rasdaemon`, and turn delay accounting off.

```
sudo rm -rf /etc/systemd/system/sysstat-collect.timer.d
sudo systemctl daemon-reload
sudo systemctl restart sysstat-collect.timer
sudo systemctl disable --now rasdaemon
sudo sysctl -w kernel.task_delayacct=0
```

8. Remove the lab directory.

```
rm -rf ~/labs/5.5
```

Exam corner

- Work resource by resource (CPU, memory, disk, network): utilization, saturation, errors. "High" means higher than the **baseline**.
- **Load average** = runnable + uninterruptible (`D`) tasks over 1, 5, 15 minutes; compare with `nproc`. High load with busy CPUs = **CPU bottleneck**; high load with idle CPUs and high `wa` = storage.
- `vmstat`: `r` run queue, `b` **blocked processes**, `si` / `so` **swapping**, `cs` **context switches**, `wa` **I/O wait**, `st` steal. Ignore the first line.
- `iostat -x`: `r_await` / `w_await` = **disk latency** (ms), `aqu-sz` = queue, `%util` near 100 = saturated. A `D`-state process cannot be killed until its I/O completes.
- **Out of memory**: "Killed process" in `journalctl -k` / `dmesg`, exit code 137; control with `MemoryMax=` and `oom_score_adj` / `OOMScoreAdjust=`.
- `nice` / `renice` range -20 (favored) to 19; only root can lower the value. Cgroup limits: `CPUQuota=`, `CPUWeight=`, `MemoryMax=`, `IOWriteIOPSMax=` via unit files, `systemd-run`, or `systemctl set-property`.
- **Slow startup**: `systemd-analyze`, `systemd-analyze blame`, `systemd-analyze critical-chain`.
- **Failed log-in attempts**: `journalctl -u ssh` (Rocky `sshd`), `/var/log/auth.log` (Rocky `/var/log/secure`), `lastb`; fix with `fail2ban` and key-only SSH.
- Network: **latency** = RTT, **jitter** = `mdev` / `StDev`, **packet drops** = loss, `ip -s link`, `tc -s qdisc`; **low throughput** = `iperf3`. `netem` emulates delay/jitter/loss, `tbf` limits rate.
- `curl -w`: slow `time_connect` = network (**slow response times**); fast connect, slow `time_starttransfer` = **slow application response**.
- **Random timeouts/disconnects** come in bursts (flapping links, IP conflicts, firewall or NAT idle timeouts). **Slow remote storage**: `nfsiostat` (RTT vs exe), `mountstats`, `nfsstat`.
- **Hardware errors**: `journalctl -k -p err`, MCE/EDAC/I/O errors in `dmesg`, `ras-mc-ctl --summary`, `smartctl -a`. **Sluggish terminal behavior/unresponsive** system: `uptime`, `vmstat`, `/proc/pressure/*`.

Symptom	First tools to reach for
CPU bottleneck, high CPU usage, high load average	`uptime`, `top`, `mpstat -P ALL`, `pidstat -u`, `vmstat` (`r`)
High context switching	`vmstat` (`cs`), `pidstat -w`, `sar -w`
Swapping, out of memory	`free -h`, `swapon --show`, `vmstat` (`si`/`so`), `sar -W`, `journalctl -k | grep -i oom`
High I/O wait, high disk latency, blocked processes	`vmstat` (`wa`, `b`), `iostat -x`, `iotop`, `pidstat -d`, `ps` state `D`
Slow startup	`systemd-analyze blame`, `systemd-analyze critical-chain`
High failed log-in attempts	`journalctl -u ssh`, `/var/log/auth.log`, `lastb`, `fail2ban-client status`
High latency, jitter, packet drops	`ping`, `mtr`, `ip -s link`, `tc -s qdisc`, `nstat`
Low throughput	`iperf3`, `ethtool`, `tc -s qdisc`, `iostat` (for disk)
Slow response times, slow application response	`curl -w`, `ss`, application logs, `strace`
Random timeouts, random disconnects	`ping -O`, `journalctl -k`, `ip -s link`, firewall/NAT logs
Slow remote storage response	`nfsiostat`, `mountstats`, `nfsstat`
Hardware errors	`journalctl -k -p err`, `dmesg`, `ras-mc-ctl`, `smartctl`
System unresponsiveness, sluggish terminal behavior	`uptime`, `vmstat`, `/proc/pressure/*`, `systemd-cgtop`
Exceeding baselines	`sar -u/-q/-r/-b/-d/-n DEV -f /var/log/sysstat/saDD` (Rocky: `/var/log/sa/`)

Review questions

1. A 4-CPU server shows `load average: 12.4, 11.9, 10.2`, but `top` shows the CPUs 85% `wa` and only 5% `us`. What is the most likely cause?
   A. A CPU bottleneck caused by too many processes
   B. Processes blocked waiting for slow storage
   C. A memory leak causing the OOM killer to run
   D. High context switching

2. `vmstat 2` shows `si` and `so` values in the thousands on every line, and users say the application is slow. Which statement is correct?
   A. The system is swapping because the workload needs more RAM than is available
   B. The disk is full
   C. The CPU is overheating
   D. The network is dropping packets

3. An application process disappears without writing an error to its own log, and its parent reports exit code 137. Which command most quickly confirms the cause?
   A. `systemd-analyze blame`
   B. `journalctl -k | grep -i 'killed process'`
   C. `iostat -x 2`
   D. `lastb`

4. Users report that a web application is slow. `curl -w '%{time_connect} %{time_starttransfer}\n'` prints `0.001 3.204`. Where should you look next?
   A. The network path, because the connection is slow
   B. The DNS server
   C. The application or its back end (database, API), because the server takes 3 seconds to answer
   D. The client's network card

5. After adding a new service, a server takes 45 seconds longer to boot. Which command shows whether that service is on the chain of units the boot waited for?
   A. `systemd-analyze critical-chain`
   B. `systemctl list-timers`
   C. `sar -q`
   D. `journalctl -k -p err`

6. NFS clients complain that file saves are very slow. `ping` to the NFS server is fast, the server's CPU and disks are idle, and `nfsiostat` shows `avg RTT` of 95 ms with `avg exe` of 96 ms. What does this tell you?
   A. The client is queuing requests internally; increase the number of client threads
   B. The NFS server's disk is failing
   C. Requests take about 95 ms on the network path, and something on the path treats NFS traffic differently than ping
   D. The mount is read-only

REFERENCE

Appendices

Answer key · Exam corner review sheet · Coverage map · Lab VM troubleshooting · Acronyms

Appendix A: Answer Key

Answers to the review questions at the end of every lab.

Lab 1.1: Explore Basic Linux Concepts

1. **B.** Changes to `/etc/default/grub` only reach the bootloader after `update-grub` (or `grub-mkconfig`) regenerates `/boot/grub/grub.cfg`.
2. **C.** The initrd/initramfs is a temporary root filesystem in RAM that carries the drivers and tools (such as `lvm`) needed to find and mount `/`.
3. **PXE** (Preboot eXecution Environment); the bootloader is downloaded with **TFTP** after DHCP supplies the server address and file name.
4. **C.** `/proc` is a virtual filesystem whose contents are generated by the kernel on demand.
5. **B.** AArch64 is called `arm64` in Debian and Ubuntu package names; `amd64` is x86_64.
6. **B.** Copyleft requires anyone who distributes a modified version to provide the source under the same license.

Lab 1.2: Manage Devices, Kernel Modules, and the initrd

1. **B.** `lspci -k` lists each PCI device with the "Kernel driver in use" and the candidate kernel modules.
2. **B.** `depmod -a` rebuilds `modules.dep` and the alias files so `modprobe` can find the new module.
3. **C.** An `install` line replaces loading with a command; `/bin/false` always fails, so even explicit `modprobe` cannot load it. A blacklist only stops alias-based automatic loading.
4. **`sudo dracut -f`**, which rebuilds `/boot/initramfs-$(uname -r).img`; `-f` forces overwriting the existing image.
5. **C.** `ipmitool` talks to the BMC over IPMI, for example `ipmitool -I lanplus -H BMC_IP -U user -P pass chassis power cycle`.
6. **A.** `dmidecode` decodes the DMI/SMBIOS table, for example `dmidecode -s system-serial-number`.

Lab 1.3: Manage Storage

1. **B.** XFS is grown with `xfs_growfs`, which takes the mount point; `resize2fs` is for ext4.
2. **Inode usage with `df -i`.** The filesystem has run out of inodes (too many small files), so no new files can be created even though blocks are free.
3. **B.** `nofail` tells the boot process to continue if the device is missing; `noauto` would stop it from mounting at boot at all.
4. **C.** `pvmove` migrates extents from one PV to the others in the VG while LVs stay online; `vgreduce` then removes the empty PV.

5. **The RAID 1 array is degraded:** `sdc` has failed (`F`), only one of two members (`sdb`) is working, and the data is still available from it. Replace the disk with `mdadm --remove` and `--add` .

6. `noexec` prevents running programs from that filesystem; `nosuid` makes the kernel ignore setuid and setgid bits.

Lab 1.4: Manage Network Services and Configurations

1. **B.** `netplan try` applies the configuration and reverts it automatically unless you confirm within the timeout, protecting remote sessions.

2. **The name is defined in `/etc/hosts` , not in DNS.** `ping` follows `/etc/nsswitch.conf` (files first), while `dig` queries DNS servers directly and ignores `/etc/hosts` .

3. **B.** `nmcli connection modify` saves the setting in the connection profile, and NetworkManager writes it into `/etc/resolv.conf` ; manual edits to `resolv.conf` are overwritten.

4. **A.** `ss -tulpn` lists listening TCP and UDP sockets with numeric ports and the owning process.

5. **The host is reachable but nothing is listening on port 5432** (or the service is bound only to another address such as `127.0.0.1`); a firewall would usually cause a timeout or "No route to host" instead.

6. **B.** `iperf3` runs a client and server pair to measure network throughput.

Lab 1.5: Use Common Shell Operations

1. **B.** Redirections are processed left to right, so stdout must point to the file before `2>&1` copies stderr to the same place; option A sends errors to the screen.

2. Because `~/.bash_profile` now exists, a login Bash reads it instead of `~/.profile` , and on Ubuntu it is `~/.profile` that sources `~/.bashrc` ; the fix is to add `[ -f ~/.profile ] && . ~/.profile` (or source `~/.bashrc`) to `~/.bash_profile` .

3. **B.** `uniq -c` only counts adjacent duplicates, so the values must be sorted first, then `sort -rn` ranks by count.

4. `sudo !!` , which re-runs the previous command with `sudo` in front.

5. **C.** `<<<` is a here string that feeds one string to standard input; `<` expects a file name and `<<` expects a here doc delimiter.

6. **C.** `:q!` quits and discards changes; `:wq` , `ZZ` , and `:x` all save first.

Lab 1.6: Back Up and Restore a Linux Server

1. **B.** `-j` selects bzip2; `-z` is gzip, `-J` is xz, and `-x` extracts instead of creating.

2. Add a trailing slash to the source (`rsync -a /srv/web/ /backup/`) so rsync copies the directory's contents rather than the directory itself.

3. **C.** `zgrep` searches inside gzip-compressed files directly; plain `grep` sees only compressed bytes, and `gunzip` would decompress the file on disk.

4. **B.** The input (`if`) is the disk and the output (`of`) is the file, copying one 512-byte block; option A does the reverse and overwrites the MBR.
5. `ddrescue` copies the good areas first, skips and later retries bad areas, and records progress in the mapfile so it can resume and retry without re-reading what it already rescued; `dd` stops at errors or blindly pads them.
6. **C.** `cpio` in copy-out mode (`-o`) reads the list of file names from standard input, which `find` provides.

Lab 1.7: Run Virtual Machines with QEMU, KVM, and libvirt

1. The CPU virtualization extensions are not exposed to the VM, so KVM acceleration is unavailable (guests would run only with slow QEMU TCG emulation); enable nested virtualization in the outer hypervisor (for example VirtualBox "Enable Nested VT-x/AMD-V", VMware "Virtualize Intel VT-x/EPT", or Hyper-V `Set-VMProcessor -ExposeVirtualizationExtensions $true`).
2. **B.** `qemu-img info` reports image properties including file format, virtual size, disk size, and any backing file.
3. **C.** qcow2 overlays with a backing file are linked clones that store only each VM's changed blocks; `virt-clone` makes full copies.
4. `shutdown` asks the guest OS to shut down cleanly via an ACPI event, while `destroy` immediately powers the VM off like pulling the plug; neither deletes the VM.
5. **C.** A bridged network connects the VM directly to the physical LAN so it gets its own LAN address; NAT hides guests behind the host, and isolated/host-only networks do not reach the LAN.
6. VirtIO drivers are paravirtualized: the guest knows it is virtual and exchanges data with the hypervisor through efficient shared-memory queues instead of having QEMU emulate every register access of real hardware.

Lab 2.1: Manage Files and Directories

1. **C.** A hard link is another name for the same inode; deleting one name only lowers the link count, and the data remains while any name (or open descriptor) exists.
2. **B.** `find /var -size +500M` walks `/var` and matches files bigger than 500 MiB; `locate` only searches names.
3. **A.** `lsof +L1` lists deleted files that a process still holds open, which keeps their disk space allocated until the process closes them or restarts.
4. `b` means it is a block device; `8` is the major number (the `sd` disk driver) and `16` is the minor number (the second disk, `sdb`).
5. Run `sudo updatedb` to rebuild the locate database (or use `find`), because `locate` only knows files that existed when the database was last built.
6. **C.** `sdiff` prints the files in two columns and marks changed lines with `|` , left-only lines with `<` , and right-only lines with `>` .

Lab 2.2: Manage Local Accounts

1. `-G` without `-a` replaced all of the user's supplementary groups with only `docker`; the correct command is `sudo usermod -aG docker dev1` (then add `developers` back with `usermod -aG developers dev1`).
2. **D.** Field 8 is the account expiration date (set with `chage -E` or `usermod -e`); after it passes, all logins are refused.
3. **B.** `chage -d 0` sets the last-change date to day 0, so the password is already expired and must be changed at the next login.
4. Ubuntu's `login.defs` does not set `CREATE_HOME yes`, so `useradd` only creates a home with `-m`; use `useradd -m` (or `adduser`, which always creates the home).
5. **B.** `-r` creates a system account with a UID from the system range below 1000, which is typical for service accounts.
6. **B.** The setuid bit on `/usr/bin/passwd` makes it run with the owner's (root's) effective UID, while the real UID stays the caller's (1000).

Lab 2.3: Manage Processes and Jobs

1. **C.** A zombie has already exited and cannot be killed; its parent has not collected its exit status, so fix, restart, or kill the parent and PID 1 will reap the zombie.
2. **A.** The fields are minute (45), hour (23), day of month (`*`), month (`*`), and day of week (`0` = Sunday).
3. Press `Ctrl+Z` to stop the foreground job, then run `bg` (or `bg %N`) to resume it in the background.
4. It sends signal 15, `SIGTERM`, which the program can catch to save data and exit cleanly; `SIGKILL` (9) cannot be caught, so no clean-up happens.
5. Check the current limit with `ulimit -n` (or `/proc/<PID>/limits`) and count open files with `lsof -p PID`, then raise `nofile` in `/etc/security/limits.conf` (or `LimitNOFILE=` in the systemd unit for a service).
6. **B.** `anacron` runs daily, weekly, and monthly jobs that were missed while the system was off, shortly after it starts again.

Lab 2.4: Configure and Manage Software

1. **B.** `apt -f install` (`--fix-broken`) downloads the missing dependencies and finishes configuring the half-installed package.
2. In `/etc/apt/sources.list.d/ubuntu.sources`, add the line `Enabled: no` to that source's block (deb822 format).
3. **B.** `dnf provides` searches repository metadata; `rpm -qf` only knows about installed files.
4. Add `exclude=kernel*` to the `[main]` section of `/etc/dnf/dnf.conf` (use `--disableexcludes=all` when you do want to upgrade them).

5. The system Python is marked “externally managed” (PEP 668) to avoid conflicts with apt-installed packages; create a venv with `python3 -m venv`, activate it, and run `pip install` inside it (or install the `python3-requests` package with apt).
6. **C.** `nginx -t` tests the configuration; `apachectl configtest` is the Apache equivalent.

Lab 2.5: Manage Linux Using systemd

1. **B.** `daemon-reload` makes systemd re-read unit files from disk; without it systemd keeps using the old version in memory.
2. `disable` removes the boot-time symlinks, but the unit can still be started manually or as a dependency; `mask` links it to `/dev/null` so it cannot be started at all until `unmask`.
3. **B.** The name is the path without the leading slash, with `/` turned into `-`, plus `.mount`.
4. `systemd-analyze blame`.
5. Put `net.ipv4.ip_forward = 1` in a file such as `/etc/sysctl.d/99-forward.conf` (or `/etc/sysctl.conf`) and apply it with `sudo sysctl --system`.
6. **C.** `resolvectl status` shows the per-link DNS servers that `systemd-resolved` forwards to.

Lab 2.6: Manage Applications in Containers

1. **C.** Arguments after the image name replace `CMD`, while `ENTRYPOINT` stays, so the command is `ping -c1 8.8.8.8`.
2. SELinux is blocking access because the folder is not labeled `container_file_t`; add `:Z` (private) or `:z` (shared) to the volume option, for example `-v /srv/app:/app:Z`.
3. **B.** User-defined bridge networks include Docker’s embedded DNS, so containers find each other by name; the default bridge does not.
4. `docker system prune` (add `-a` for all unused images and `--volumes` for anonymous volumes).
5. The Docker daemon runs as root, so any `docker` group member can start a container that mounts the host’s root filesystem or runs privileged, which is the same as having root on the host.
6. **C.** runC is the OCI runtime that containerd calls to actually create and start containers.

Lab 3.1: Authentication, Authorization, and Accounting

1. **B.** `requisite` ends processing at once with failure; `required` would keep going and fail at the end.
2. `journalctl -p err -b` (priority `err` includes crit, alert, and emerg).
3. In a file such as `/etc/audit/rules.d/50-sudoers.rules`, loaded with `augenrules --load`, which regenerates `/etc/audit/audit.rules`.
4. **C.** NSS reads `/etc/nsswitch.conf` to decide which sources (files, sss, winbind, ...) to query.
5. **B.** A polkit rule in `/etc/polkit-1/rules.d/` can grant `org.freedesktop.systemd1.manage-units` for one unit to one group.

6. It runs in debug (dry-run) mode, showing what would be rotated without changing any files.

Lab 3.2: Configure and Implement Firewalls

1. The rule was **runtime** only; `firewall-cmd --runtime-to-permanent` (or repeating the command with `--permanent`) saves it to disk.
2. **C.** DNAT must change the destination before the routing decision, which happens after PREROUTING.
3. `sysctl -w net.ipv4.ip_forward=1`; make it permanent in a file under `/etc/sysctl.d/` (or `/etc/sysctl.conf`).
4. **B.** The default policy denies incoming traffic, so SSH must be allowed first or you lose remote access.
5. A stateful rule uses connection tracking to recognize packets belonging to existing connections (established/related); a stateless rule judges each packet only by its own headers.
6. **B.** Masquerade is dynamic SNAT; it rewrites the source to whatever address the outgoing interface has (and usually ports too, which is PAT).

Lab 3.3: Operating System Hardening

1. `visudo`. It locks the file against simultaneous edits and checks the syntax before saving, so a mistake cannot lock every administrator out of sudo.
2. **B.** `vi` can spawn a shell with `:!sh`, giving a root shell; `NOEXEC` prevents the command from executing any other program, blocking the escape.
3. The file has the wrong SELinux context (likely `user_home_t` instead of `httpd_sys_content_t`); fix it with `restorecon -v /var/www/html/<file>`.
4. **C.** `3770` sets both setgid (2000, group inheritance) and the sticky bit (1000, owner-only deletion). `2770` lacks the sticky bit; `1770` lacks setgid.
5. `PasswordAuthentication no` (forces key login, stopping password guessing) and `PermitRootLogin no` (blocks direct root login).
6. It makes a file **append only**: data can be added but the file cannot be overwritten, truncated, renamed, or deleted, so an intruder cannot erase or rewrite the log to hide activity.

Lab 3.4: Harden User Accounts

1. **B.** A negative credit is a required minimum count, so `dcredit = -2` requires two digits; a positive value would only give bonus length.
2. Password history (`pam_pwhistory remember=N`) plus a minimum password age (`chage -m 1` or `PASS_MIN_DAYS 1`), so the user cannot cycle through the history quickly.
3. **C.** `faillock --user bob --reset` works with `pam_faillock`; `pam_tally2` was removed from Linux-PAM 1.5.0.
4. `/sbin/nologin` (on Ubuntu `/usr/sbin/nologin`, the same file through merged `/usr`), or `/bin/false`.

5. **C.** Only the 5-character prefix of the hash is sent (k-anonymity); the matching is done locally.
6. `AuthenticationMethods publickey,keyboard-interactive` (with `KbdInteractiveAuthentication yes` and the TOTP module in the `sshd` PAM file).

Lab 3.5: Use Cryptography on Linux

1. **C.** LUKS2 defaults to Argon2id, a memory-hard function that makes passphrase guessing expensive; LUKS1 used PBKDF2.
2. `sha256sum` is a plain hash anyone can recompute after changing the file; the HMAC mixes in a secret key, so only someone with the key can produce a valid value, proving integrity and authenticity.
3. `sudo update-ca-certificates`, which links the new CA into `/etc/ssl/certs` and rebuilds the CA bundle.
4. **B.** `ssl_protocols TLSv1.2 TLSv1.3` allows only those two versions.
5. `sudo update-crypto-policies --set FUTURE` (then restart services or reboot).
6. It routes traffic for `10.200.0.2` to that peer and only accepts packets from that peer whose source address is `10.200.0.2` (cryptokey routing).

Lab 3.6: Compliance, Auditing, and Detection

1. The fix was **backported** into the distribution's 8.7p1 package; confirm with `rpm -q --changelog openssh | grep CVE-...` or `dnf updateinfo`, since version-only scanners give false positives.
2. **C.** `rpm -Va` verifies every installed package; `rpm -K` checks the signature of a package file.
3. **C.** Crypto-erase destroys the keys, making all data unreadable instantly; overwriting is unreliable on SSDs because of wear leveling.
4. **C.** `/etc/issue.net`, shown when `Banner /etc/issue.net` is set in `sshd_config`; `/etc/motd` appears only after login.
5. OpenSCAP: `oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis_server_l1 --report report.html` with the `scap-security-guide` data stream.
6. It is a SUID-root program (it runs as root for any user) that no package installed, so nobody can vouch for it; attackers plant such files as privilege escalation backdoors.

Lab 4.1: Automation and Orchestration

1. **B.** Ansible connects over SSH and runs temporary Python modules, so no permanent agent is installed on targets.
2. The agent's certificate signing request must be signed on the Puppet server (for example with `puppetserver ca sign --certname <node>`).
3. **C.** `terraform.tfstate` stores the state that maps each resource in code to its real-world ID.
4. `inst.ks=http://<server>/ks.cfg` (the Kickstart location).
5. A service is the desired state (image, replicas, ports); each replica is a task, and each task runs one container on a node.

6. **B.** Running tests and linters earlier in the development process is shift left testing.

Lab 4.2: Automate Tasks with Shell Scripts

1. `>` inside `[[ ]]` compares strings by sort order, and `"10"` sorts before `"9"`; use `[[ $count -gt 9 ]]` or `(( count > 9 ))`.
2. **C.** `export` marks the variable for the environment of child processes.
3. **B.** `%%.*` removes the longest suffix that starts with a dot, leaving `backup`.
4. The `until` loop.
5. It is true if `$name` is an empty (zero-length) string; `-n` tests for a non-empty string.
6. The `cp` command failed (a non-zero return code means failure); check its error message.

Lab 4.3: Python Basics for System Administration

1. **B.** A virtual environment gives the script its own `pip` and packages without touching the system Python managed by apt.
2. A dictionary (`dict`).
3. A line ending with `:` (such as `if`, `for`, or `def`) is not followed by an indented block.
4. `pip freeze > requirements.txt`.
5. A module is a single `.py` file; a package is a directory of modules containing an `__init__.py` file.
6. **C.** `pycodestyle` checks PEP 8 style; `flake8` adds error checks on top of it.

Lab 4.4: Version Control with Git

1. **B.** `git reset FILE` (mixed reset of one path) removes it from the staging area but leaves the working file unchanged.
2. `git fetch` only downloads new commits and updates remote-tracking branches; `git pull` fetches and then merges (or rebases) them into your current branch.
3. `git merge --squash BRANCH`, then `git commit`.
4. `.gitignore` (with a line such as `*.log`).
5. **C.** The reflog records every position of `HEAD`, including commits no longer on any branch.
6. A lightweight tag is only a name pointing to a commit; an annotated tag is a full object with tagger, date, and message (created with `git tag -a`).

Lab 4.5: Use AI Responsibly as a Linux Admin

1. **B.** `bash -n` only parses the script; it says nothing about safety, correctness, or security.
2. It may be stored by the provider, used to train future models, and/or read by provider staff during human review.
3. `\d` is Perl-style syntax that POSIX extended regex (`grep -E`) does not support; use `[0-9]` or `grep -P`.

4. **B.** Redaction plus an approved or local tool follows data governance and corporate policy; base64 is encoding, not protection.
5. Any four of: role, context (OS/versions/environment), clear task, constraints, output format, examples, and a request for assumptions or tests.
6. No. Linters miss many security problems (hard-coded secrets, insecure downloads, loose permissions); it still needs human security review, testing (`--check`, lab run), and the normal review process.

Lab 5.1: Monitoring Concepts and Configurations

1. **C.** The SLO is the internal target; the SLA is the 99.5% contract and the SLI is the measured value.
2. **UDP 162.** Agents are polled on UDP 161 and send traps to the manager on UDP 162.
3. **B.** The data arrived; the client just has no MIB to translate the OID into a name (on Ubuntu, install `snmp-mibs-downloader` and comment out `mibs :`).
4. **B.** `@@` means TCP; a single `@` means UDP.
5. **Alert fatigue.** Fix it by raising the threshold or requiring the condition to last several minutes, and by alerting only on user-impacting, actionable conditions.
6. Agents give much richer data but must be installed, updated, and secured on every host; agentless needs nothing installed but is limited to what SSH or SNMP expose and needs credentials on the monitoring server.

Lab 5.2: Troubleshoot Hardware, Storage, and OS Issues

1. **B.** Space is free, so the filesystem has probably run out of inodes; `df -i` shows `IUse%` at 100%.
2. A process still has the deleted file open, so the space is not freed; `lsof +L1` lists it, and restarting that process (or truncating the file through `/proc/PID/fd`) releases the space.
3. `sudo findmnt --verify` and `sudo mount -a`; both report bad UUIDs, devices, or mount points without rebooting.
4. **B.** Exit code 139 is 128 + signal 11 (SIGSEGV), and `segfault at 0` means it accessed address 0, a NULL pointer.
5. The `ExecStart=` path: 203/EXEC means systemd could not execute the program (missing file, typo, or not executable). Confirm with `journalctl -xeu app`.
6. `D` is uninterruptible sleep: the process is waiting inside the kernel for I/O and cannot handle signals until it completes. Check the storage or NFS server it is waiting on (`dmesg`, `iostat`, mount status).

Lab 5.3: Troubleshoot Networking Issues

1. **B.** No `via` means the host thinks the server is on its own subnet, which happens when the mask is too wide (for example `/16` instead of `/24`).
2. `ping -M do -s 1472 <server>` (and smaller sizes): if large packets with "don't fragment" fail while small ones pass, it is an MTU mismatch.

3. The interface is administratively up but has no carrier: the cable, switch port, or other end is down (layer 1). Check cabling, the switch port, and `ethtool eth0` (`Link detected: no`).
4. `/etc/hosts`: `getent` checks local files before DNS, while `dig` queries DNS only, so a stale hosts entry is the likely cause.
5. **B.** A `169.254.0.0/16` (APIPA) address means the client got no answer from a DHCP server.
6. A host firewall on web01 rejecting TCP 443 (for example firewalld without the `https` service); ping works because ICMP is allowed.

Lab 5.4: Troubleshoot Security Issues

1. **C.** `mv` kept the `user_home_t` label; `restorecon` resets the file to the policy's default type `httpd_sys_content_t` without weakening SELinux.
2. **B.** `0.0.0.0` means all IPv4 addresses, so Redis is reachable from other hosts unless a firewall blocks it; it should bind to `127.0.0.1`.
3. **B.** The served chain is missing the intermediate (browsers often cache it); serving a full chain lets every client build the path to the trusted root.
4. **C.** The index was signed by a key apt does not have; the fix is the correct, verified key, never disabling signature checks.
5. **B.** The server correctly refuses an obsolete key exchange; the client should use modern algorithms instead of weakening the server.
6. **A.** The `ssl-enum-ciphers` NSE script handshakes with each protocol version and cipher and lists what the server accepts.

Lab 5.5: Troubleshoot Performance Issues

1. **B.** Linux counts tasks in uninterruptible sleep (`D`, waiting for I/O) in the load average, so high load with high `wa` and idle CPU means blocked processes waiting for storage; confirm with `iostat -x` and `vmstat` (`b` column).
2. **A.** Continuous swap-in and swap-out (`si` / `so`) means pages are being moved between RAM and swap all the time: the working set does not fit in memory.
3. **B.** Exit code 137 is 128 + 9 (`SIGKILL`), the signature of the OOM killer, which always logs "Killed process" in the kernel log.
4. **C.** The TCP connection took 1 ms, so the network is fine; the 3 seconds until the first byte are spent inside the server application or the systems it waits on.
5. **A.** `systemd-analyze critical-chain` shows the units the default target actually waited for, with their start time and duration; `blame` alone does not show whether a slow unit delayed boot.
6. **C.** RTT (network plus server time) is high while exe is almost the same, so the client is not queuing; with an idle server and a fast `ping`, the delay is on the network path for NFS traffic only (for example QoS, a firewall, or traffic shaping).

Appendix B: Exam Corner Review Sheet

Every lab's key facts in one place. Read this section in the days before your exam.

Lab 1.1: Explore Basic Linux Concepts

- Boot order: firmware (BIOS/UEFI) then bootloader (GRUB 2) then kernel then initrd/initramfs then systemd (PID 1) then the default target.
- Edit `/etc/default/grub`, never `grub.cfg`. Rebuild with `update-grub` or `grub-mkconfig -o /boot/grub/grub.cfg` (Debian/Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (RHEL/Rocky). `grubby` changes kernel parameters on RHEL-family systems.
- `GRUB_CMDLINE_LINUX` applies to all entries; `GRUB_CMDLINE_LINUX_DEFAULT` to normal boots only. `GRUB_TIMEOUT` and `GRUB_DEFAULT` control the menu.
- The running kernel's parameters are in `/proc/cmdline`. Press `e` at the GRUB menu for a one-time change.
- The initrd/initramfs holds drivers needed to mount `/`. Ubuntu: `update-initramfs`, `lsinitramfs`. Rocky: `dracut`, `lsinitrd`.
- PXE = network boot: DHCP gives an address and the boot file name, TFTP delivers the bootloader, then the kernel and initrd.
- FHS: `/etc` config, `/var` variable data and logs, `/proc` virtual kernel and process info, `/dev` device files, `/boot` kernel and initrd, `/usr` installed software, `/tmp` temporary (sticky bit), `/home` user files, `/bin` and `/sbin` and `/lib` essential commands and libraries.
- x86_64 = AMD64 = `amd64` (Debian). AArch64 = ARM64 = `arm64` (Debian). RISC-V is an open instruction set. `uname -m` shows the architecture.
- dpkg-based: `.deb`, `dpkg`, `apt` (Debian, Ubuntu). RPM-based: `.rpm`, `rpm`, `dnf` / `yum` / `zypper` (RHEL, Rocky, Fedora, SUSE).
- Display manager = graphical login (GDM, SDDM, LightDM). Window manager = places and decorates windows. X Server (Xorg) and Wayland are display servers; Wayland is the modern default.
- Copyleft (GPL) requires derivative works to keep the same license. Permissive (MIT, BSD, Apache) does not. Proprietary = closed source, restricted redistribution. Free software = the four freedoms.

Lab 1.2: Manage Devices, Kernel Modules, and the initrd

- `lsmod` lists loaded modules (from `/proc/modules`). `modinfo` shows a module's file, license, dependencies, and parameters.
- `modprobe NAME` loads a module plus its dependencies and reads `/etc/modprobe.d/*.conf`; `modprobe -r NAME` unloads it.
- `insmod /full/path/file.ko` loads one file without dependencies; `rmmod NAME` unloads one module.
- `depmod -a` rebuilds `modules.dep` in `/lib/modules/$(uname -r)/`; run it after adding a module file by hand.
- In `/etc/modprobe.d/`: `options NAME key=value` sets parameters, `blacklist NAME` stops automatic (alias) loading, `install NAME /bin/false` stops all loading. `/etc/modules-load.d/` loads modules at boot.
- `lspci -k` shows which kernel driver each PCI device uses; `lsusb` lists USB devices; `lscpu` CPU; `lsmem` memory blocks; `lshw` everything (`-short`, `-class`).
- `dmidecode` reads the firmware DMI/SMBIOS table: manufacturer, model, serial number, memory slots. `dmesg` shows kernel (hardware detection) messages.
- `lm_sensors`: run `sensors-detect`, then `sensors` for temperatures, voltages, and fans. `ipmitool` talks to the BMC locally (`/dev/ipmi0`) or remotely (`-I lanplus -H host`).

- Ubuntu rebuilds the initrd with `update-initramfs -u` (`-k all` for all kernels). RHEL/Rocky uses `dracut -f`; `mkinitrd` is the legacy tool (now a dracut wrapper). `lsinitrd` and `lsinitramfs` list contents.
- Embedded systems: single-purpose devices, often ARM/RISC-V, BusyBox, device tree, flash storage. GPUs are used for AI/ML, HPC, video transcoding, and VDI; monitor them with `nvtop`.

Lab 1.3: Manage Storage

- LVM order: `pvcreate` then `vgcreate` then `lvcreate` then `mkfs` then `mount`. Removal is the reverse: `umount`, `lvremove`, `vgremove`, `pvremove`.
- Summaries: `pvs`, `vgs`, `lvs`. Details: `pvdisplay`, `vgdisplay`, `lvdisplay`. Discovery: `pvscan`, `vgscan`.
- Grow: `vgextend` adds a PV; `lvextend` / `lvresize -L +SIZE`; then `resize2fs` (ext4) or `xfs_growfs MOUNTPOINT` (XFS), or use `-r` to do both. XFS cannot shrink.
- `pvmove` migrates extents off a PV online; then `vgreduce` and `pvremove`. `pvresize` after the disk grows. Move a VG between hosts: `vgchange -an`, `vgexport`, then `vgimport`, `vgchange -ay`. `lvchange -an/-ay` deactivates/activates one LV.
- MBR: 4 primary partitions, 2 TiB limit. GPT: 128 partitions, backup table. `fdisk` and `gdisk` save with `w`; `parted` writes immediately. `growpart DISK N` grows a partition (cloud disks).
- `blkid` shows UUID and TYPE; use `UUID=` in `/etc/fstab`. Fields: device, mount point, type, options, dump, fsck pass. Test with `mount -a` and `findmnt --verify`.
- Options: `noexec`, `nosuid`, `nodev` (harden `/tmp` and removable media), `ro` / `rw`, `remount`, `noatime` / `nodiratime`, `nofail` (do not block boot), `_netdev` (network mounts).
- `/proc/mounts` is the kernel's live list; `/etc/mtab` links to it.
- `fsck` (ext family) and `xfs_repair` (XFS) only on unmounted filesystems. `df -h` space, `df -i` inodes, `du -sh` folder size, `fio` benchmarks I/O.
- "No space left on device" with free space = inode exhaustion; check `df -i`.
- RAID 0 stripe, 1 mirror, 5 parity (3+ disks), 6 double parity (4+), 10 striped mirrors. `mdadm --create`, `--detail`, `--fail`, `--remove`, `--add`; status in `/proc/mdstat` (`[UU]` healthy, `[U_]` degraded).
- NFS: `/etc/exports`, `exportfs -ra`, `showmount -e`, `mount -t nfs host:/path`; `root_squash` is the default. SMB: Samba `smb.conf`, `smbpasswd -a`, `mount -t cifs //host/share`. autofs: `/etc/auto.master` plus map files, mounts on access.

Lab 1.4: Manage Network Services and Configurations

- `ip address` (addresses), `ip link` (interfaces, `set dev X up/down`), `ip route` (routes, `ip route add NET via GW`). `ip` changes are not persistent.
- Netplan: YAML in `/etc/netplan/`, spaces not tabs. `netplan try` applies with automatic rollback (default 120 s), `netplan apply` applies immediately, `netplan status` shows state, `netplan generate` builds backend files. Renderer is `networkd` (server) or `NetworkManager` (desktop).
- NetworkManager: `nmcli general status`, `nmcli device status`, `nmcli connection show|add|modify|up|down|delete`, `nmcli device reapply`. Profiles are `.nmconnection` keyfiles in `/etc/NetworkManager/system-connections/` ("nmconnect"). `nmtui` is the text menu.
- Name lookup order comes from the `hosts:` line in `/etc/nsswitch.conf` (`files dns`); `/etc/hosts` is checked first. `/etc/resolv.conf` lists `nameserver` and `search`; on Ubuntu it points to the systemd-resolved stub `127.0.0.53` (`resolvectl status` shows real servers).
- `dig` and `nslookup` query DNS directly and ignore `/etc/hosts`; `getent hosts` follows nsswitch. `dig @SERVER NAME TYPE`, `dig +short`, `dig -x IP` for reverse.

- `ping` / `ping6` test reachability; `arp -n` / `ip neigh` show IP-to-MAC mappings; `traceroute`, `tracepath` (no root, shows PMTU), and `mtr` (continuous) show the path.
- `ss -tulpn` lists listening TCP/UDP ports with processes (replaces `netstat`). `nc -l PORT` listens, `nc -zv HOST PORT` tests a port. `nmap HOST` scans ports: open, closed, filtered.
- `curl -I URL` shows HTTP headers and status code; `-L` follows redirects, `-o` saves output.
- `iperf3 -s` (server) and `iperf3 -c HOST` (client) measure throughput, default TCP port 5201.
- `ethtool IFACE` shows speed, duplex, link; `-i` driver; `-S` statistics; `-s` changes settings.
- `tcpdump -i IFACE -n FILTER`, `-c N` count, `-w file.pcap` write, `-r file.pcap` read.
- `hostname` shows the name, `hostname -I` the addresses; `hostnamectl set-hostname` changes it permanently.

Lab 1.5: Use Common Shell Operations

- Absolute paths start with `/` (or `~`, which expands to `$HOME`). Relative paths start from the current directory; `.` is the current directory, `..` the parent, and `cd -` returns to the previous directory (`$OLDPWD`).
- `HOME`, `USER`, `SHELL`, `PATH`, `PS1`, and `DISPLAY` are the key variables. `export` makes a variable visible to child processes; `env` / `printenv` list environment variables.
- `PATH` is searched left to right; the current directory is not in it, so run local scripts as `./script`.
- Login shells read `/etc/profile` then the first of `~/.bash_profile`, `~/.bash_login`, `~/.profile`. Non-login interactive shells read `~/.bashrc`. `source file` or `. file` applies changes to the current shell.
- Streams: stdin = 0, stdout = 1, stderr = 2. `>` overwrites, `>>` appends, `2>` redirects errors, `2>&1` joins stderr to stdout (order matters), `&>` sends both, `<` feeds a file to stdin, `<<` is a here doc, `<<<` is a here string, `/dev/null` discards.
- `|` passes only stdout. `tee` writes to a file and stdout at the same time; `| sudo tee file` writes a root-owned file.
- `uniq` only removes *adjacent* duplicates, so `sort | uniq -c | sort -rn` is the classic "count and rank" pipeline.
- `cut -d, -f2` for delimited fields; `awk '{print $2}'` for whitespace-separated columns; `sed 's/a/b/g'` to substitute; `sed -i` edits in place; `tr` changes or deletes single characters.
- `xargs` turns stdin lines into command arguments; `bc` does decimal math (`scale=`); `printf` formats output without an automatic newline.
- `!!` repeats the last command (`sudo !!`), `!n` runs history entry `n`, `!string` runs the latest command starting with `string`, `!$` is the last argument.
- `vim`: `i` insert, Esc normal mode, `:w` save, `:q!` quit without saving, `:wq` save and quit, `dd` delete line, `yy` / `p` copy/paste, `u` undo, `/text` search, `:%s/old/new/g` replace. `nano`: Ctrl+O save, Ctrl+X exit, Ctrl+W search.

Lab 1.6: Back Up and Restore a Linux Server

- `tar` modes: `-c` create, `-x` extract, `-t` list; `-f file` names the archive; `-v` verbose; `-C dir` extracts into `dir`; `-p` preserves permissions; `--exclude=PATTERN` skips files.
- `tar` compression: `-z` gzip (`.tar.gz` / `.tgz`), `-j` bzip2 (`.tar.bz2`), `-J` xz (`.tar.xz`). GNU tar strips the leading `/` so restores go into the current directory.
- Incremental tar: `--listed-incremental=file.snar`; restore the full archive first, then each incremental in order (with `--listed-incremental=/dev/null`).
- `cpio` reads file names from stdin: `find dir | cpio -ov > a.cpio` (copy-out), `cpio -idv < a.cpio` (copy-in, extract), `cpio -it < a.cpio` (list), `cpio -pdm dest` (pass-through copy). initramfs images are cpio archives.
- Compression speed versus size: `gzip` (fastest, largest) < `bzip2` < `xz` (slowest, smallest). Decompress with `gunzip` / `gzip -d`, `bunzip2` / `bzip2 -d`, `unxz` / `xz -d`; `-k` keeps the original, `-c` writes to stdout, `-1` to `-9` set the level.

- 7-Zip: `7z a` add, `7z l` list, `7z x` extract (`-oDIR`), `7z t` test; Ubuntu 24.04 package `7zip`. `zip -r` and `unzip` (`-l` list, `-d DIR` destination) for Windows-friendly archives.
- `zcat`, `zgrep`, `zless` read gzip files without decompressing them on disk; `bzcat` / `xzcat` are the bzip2/xz equivalents.
- `rsync -av src/ dest/`: `-a` keeps permissions, times, links, ownership; a trailing slash on the source copies its *contents*; `--delete` mirrors deletions; `--dry-run` / `-n` previews; `-z` compresses; `user@host:path` uses SSH.
- `dd if=INPUT of=OUTPUT bs=SIZE count=N status=progress`: block-level copy. MBR = first 512 bytes (`bs=512 count=1`). Swapping `if` and `of` destroys data.
- `ddrescue INPUT OUTPUT MAPFILE` images failing disks, skips bad areas first, and resumes using the mapfile; package `gddrescue` on Ubuntu.
- 3-2-1 rule: 3 copies, 2 media types, 1 off-site. A backup is not proven until a restore has been tested.

Lab 1.7: Run Virtual Machines with QEMU, KVM, and libvirt

- **KVM** is the Linux kernel module that uses Intel VT-x (`vmx`) or AMD-V (`svm`) for hardware-accelerated VMs; **QEMU** emulates the machine and devices and can run without KVM using slow software emulation (TCG). Check support with `grep -E 'vmx|svm' /proc/cpuinfo` or `kvm-ok`.
- **libvirt** is the management layer (daemon `libvirtd`, XML definitions in `/etc/libvirt/qemu/`); **virsh** is its CLI, **virt-install** creates VMs, **virt-clone** clones them, **virt-manager** is the GUI.
- **Nested virtualization** means running a hypervisor inside a VM; the outer hypervisor must expose VT-x/AMD-V to the guest.
- **Paravirtualized drivers** (VirtIO: `virtio-blk`, `virtio-net`, `virtio-balloon`) are faster than emulated hardware; VirtIO disks appear as `/dev/vda`.
- `qemu-img info` shows image properties (format, virtual size, disk size, backing file); `qemu-img convert -f qcow2 -O raw` changes format; `qemu-img resize img +10G` grows the virtual disk (then grow the partition and file system in the guest).
- qcow2 supports thin provisioning, compression, internal snapshots, and backing files; a baseline template plus qcow2 overlays (`qemu-img create -b base -F qcow2`) gives linked clones; `virt-clone` makes full clones with a new UUID and MAC.
- VM states: running, idle, paused (`virsh suspend` / `resume`), in shutdown, shut off, crashed, pmsuspended. `virsh shutdown` is graceful, `virsh destroy` is a forced power-off, `virsh undefine` deletes the definition.
- Resources: `virsh setmem` / `setmaxmem`, `virsh setvcpus`, `virsh attach-disk` / `detach-disk`, `virsh attach-interface`; `--config` (persistent), `--live` (running VM), `--persistent` (both, for attach/detach).
- Snapshots: `virsh snapshot-create-as`, `snapshot-list`, `snapshot-revert`, `snapshot-delete`. Snapshots are not backups.
- Migration: cold/offline (stopped VM: copy disk, `dumpxml` / `define`) versus live (`virsh migrate --live VM qemu+ssh://host2/system`, needs shared or copied storage and compatible CPUs).
- Networks: **NAT** (libvirt `default`, `virbr0`, 192.168.122.0/24), **bridged** (guests on the physical LAN), **host-only/isolated** (no `<forward>`), **routed** (`forward mode='route'`, no NAT), **open** (`forward mode='open'`, no firewall rules added).
- Bare metal: best performance and isolation, direct hardware. VMs: density, templates, snapshots, cloning, and migration.

Lab 2.1: Manage Files and Directories

- `ls -l` first character: `-` file, `d` directory, `l` symlink, `b` block device, `c` character device, `p` pipe, `s` socket. `ls -i` shows inode numbers.
- `cp -r` copies directories; `cp -a` copies recursively and preserves everything; `cp -p` preserves mode, ownership, and timestamps. `mv` renames or moves. `rmdir` only removes empty directories; `rm -r` removes non-empty ones.
- `mkdir -p` creates parent directories as needed. `pwd -P` resolves symlinks. `cd -` returns to the previous directory.

- `touch` creates empty files or updates timestamps; `touch -d "date"` or `-t YYYYMMDDhhmm` sets a specific time. ctime cannot be set manually.
- `stat` shows the inode: size, blocks, inode number, link count, permissions, UID/GID, and atime/mtime/ctime/birth.
- `file` identifies content by magic numbers, not by extension.
- `find PATH TESTS ACTION`: `-name`, `-iname`, `-type`, `-size +10M`, `-mtime +7`, `-mmin -30`, `-perm -4000`, `-user`, `-group`, `-exec cmd {} \;`, `-exec cmd {} +`, `-ok`, `-delete`.
- `locate` searches a database built by `updatedb` (package `plocate` on Ubuntu 24.04, `mlocate` on Rocky 9). It is fast but can be stale.
- `diff` shows changes (`<` first file, `>` second, `c` / `a` / `d`); `diff -u` is unified format; `sdiff` shows side by side with `|`, `<`, `>`.
- Hard link: same inode, link count increases, survives deletion of the original, no directories, no cross-filesystem. Symbolic link (`ln -s`): own inode, stores a path, can cross filesystems and point to directories, breaks if the target is removed.
- `lsof +D dir` finds what is using a directory; `lsof +L1` finds deleted-but-open files; `lsof -i :port`; `lsof -u user`; `lsof -p PID`.
- Block devices (disks, partitions) use random block access; character devices (terminals, serial ports) are byte streams. Special character devices: `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/urandom`, `/dev/full`, `/dev/tty`. Device files show major (driver) and minor (instance) numbers.

Lab 2.2: Manage Local Accounts

- `/etc/passwd`: `name:x:UID:GID:GECOS:home:shell` (world-readable). `/etc/shadow`: `name:hash:lastchg:min:max:warn:inactive:expire:reserved` (root/shadow only). `/etc/group`: `name:x:GID:members`.
- Hash prefixes: `$1$` MD5, `$5$` SHA-256, `$6$` SHA-512 (Rocky 9), `$y$` yescrypt (Ubuntu 24.04). A leading `!` means locked; `*` or `!` alone means no password.
- UID 0 = root; 1 to 999 = system and service accounts (`useradd -r`); 1000 and up = regular users (`UID_MIN` / `UID_MAX` in `/etc/login.defs`); 65534 = `nobody`.
- `useradd -m -s /bin/bash -c "comment" -G grp1,grp2 user`. Ubuntu's plain `useradd` creates no home and uses `/bin/sh`; Rocky creates the home and uses `/bin/bash`. `adduser` is interactive on Ubuntu and a link to `useradd` on Rocky.
- `usermod -aG group user` appends; `-G` alone replaces the supplementary list. `usermod -l` renames, `-s` shell, `-d -m` move home, `-L` / `-U` lock/unlock, `-e` expire date.
- `passwd user` sets a password; `-l` / `-u` lock/unlock; `-S` status; `-e` force change. `chage -l` lists; `-M` max, `-m` min, `-W` warn, `-I` inactive, `-E` account expiry (`0` = expired now, `-1` = never), `-d 0` = force change at next login.
- Defaults: `/etc/login.defs` (UID ranges, `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`, `ENCRYPT_METHOD`, `CREATE_HOME`) and `/etc/default/useradd` (shell, skel, `INACTIVE`, `EXPIRE`; view with `useradd -D`).
- `/etc/skel` is copied into new home directories only at creation. `/etc/profile` and `/etc/profile.d/*.sh` run for every login shell.
- `userdel -r` deletes the user, home, and mail spool; `deluser --remove-home` on Ubuntu. `groupdel` fails on a user's primary group. `groupmod -n` renames, `-g` changes GID.
- Listing: `whoami` (effective user), `id`, `groups`, `getent passwd`, `who` and `w` (current logins, from utmp), `last` (history, wtmp), `lastb` (failed logins), `lastlog` (last login per account).
- EUID/EGID decide permissions; they differ from the real UID/GID under `sudo` or setuid/setgid programs such as `passwd` (setuid root) and `chage` (setgid shadow).

- Service accounts: system UID, `nologin` shell, no password, home in `/var/lib` or none. `chsh` changes a shell; valid shells are in `/etc/shells`.

Lab 2.3: Manage Processes and Jobs

- `ps aux` (BSD style) and `ps -ef` (UNIX style, shows PPID); `ps -eo pid,ppid,stat,ni,cmd --sort=-%cpu` for custom columns. `pstree -p` shows the parent-child tree. `pgrep -a` finds PIDs by name.
- `top` keys: `P` CPU, `M` memory, `k` kill, `r` renice, `1` per CPU, `q` quit. `htop` adds tree view (`F5`), search, and F-key actions. `atop` records history in `/var/log/atop`. `mpstat -P ALL` (per CPU) and `pidstat` (per process) come from `sysstat`.
- `/proc/<PID>/` holds `status`, `cmdline`, `environ`, `fd/`, `cwd`, and `exe`. `lsof -p PID` lists a process's open files. `strace` shows system calls; `strace -p PID` attaches (needs root on Ubuntu).
- States: `R` running, `S` sleeping (interruptible), `D` blocked (uninterruptible I/O wait), `T` stopped, `Z` zombie. Zombies cannot be killed; kill or fix the parent so PID 1 reaps them.
- Nice values run from -20 (highest priority) to 19 (lowest); default 0. `nice -n 10 cmd` starts, `renice -n 5 -p PID` changes, `renice -u user` targets a user. Only root can lower a nice value.
- `ulimit -a` shows limits; `-n` open files, `-u` processes, `-f` file size; `-S` soft, `-H` hard. Users may raise soft up to hard; only root raises hard. Permanent limits: `/etc/security/limits.conf` and `limits.d/` (`domain type item value`), applied by `pam_limits` at new logins.
- Job control: `cmd &` background; `jobs`; `fg %N`; `bg %N`; `Ctrl+Z` stop (SIGTSTP); `Ctrl+C` interrupt (SIGINT); `Ctrl+D` end of input (EOF, not a signal). `nohup cmd &` ignores SIGHUP and writes to `nohup.out`. `exec cmd` replaces the shell, keeping its PID.
- Signals: 1 `SIGHUP` (hang up, often "reload config"), 9 `SIGKILL` (cannot be caught), 15 `SIGTERM` (default for `kill`, graceful). `kill -l` lists them. `killall name` (exact name) and `pkill pattern` (`-u user`, `-f` full command line) signal by name.
- Cron fields: minute, hour, day of month, month, day of week. `crontab -e`, `-l`, `-r`. `/etc/crontab` and `/etc/cron.d/*` include a user field. `at TIME` runs once (`atq`, `atrm`, `at -c`). `anacron` runs daily/weekly/monthly jobs missed while the system was off (`/etc/anacrontab`: period, delay, identifier, command).

Lab 2.4: Configure and Manage Software

- Low-level tools install one local file and do not resolve dependencies: `dpkg -i`, `rpm -ivh`. High-level package managers resolve dependencies from repositories: `apt`, `dnf` (`yum`), `zypper` (SUSE).
- Fix a broken `dpkg -i` with `apt -f install` (`--fix-broken`). `apt install ./file.deb` and `dnf install ./file.rpm` resolve dependencies for local files.
- `apt remove` keeps config files (`rc` state); `apt purge` deletes them; `apt autoremove` removes unneeded dependencies. `apt update` refreshes indexes; `apt upgrade` installs newer versions.
- Query commands: `dpkg -l`, `dpkg -L` (files in package), `dpkg -S` (owner of file); `rpm -qa`, `rpm -qi`, `rpm -ql`, `rpm -qf`; `dnf provides` finds a package by file name even if it is not installed.
- Ubuntu 24.04 repos are in `/etc/apt/sources.list.d/ubuntu.sources` (deb822); disable a block with `Enabled: no`. RHEL repos are `.repo` files in `/etc/yum.repos.d/` with `enabled=` and `gpgcheck=`; toggle with `dnf config-manager --set-enabled/--set-disabled`.
- Third-party repos need a GPG public key: on Ubuntu a keyring in `/etc/apt/keyrings/` referenced by `signed-by`; on RHEL `rpm --import` or `gpgkey=`. `rpm -K file.rpm` checks a package signature. EPEL is enabled with `dnf install epel-release`.
- Exclusions: `apt-mark hold`; `exclude=` in `/etc/dnf/dnf.conf` or a `.repo` section; `dnf --exclude=pkg` for one command; `--disableexcludes` overrides.
- `dnf history` and `dnf history undo <ID>` roll back a transaction.

- Building from source: `./configure`, `make`, `sudo make install` (into `/usr/local`); needs `build-essential` (Ubuntu) or "Development Tools" (`dnf groupinstall`) on RHEL. Not tracked by the package manager.
- `pip` (PyPI, use a venv; PEP 668 blocks system installs on new distros), `npm` (`node_modules` local, `-g` global), `cargo` (crates.io, `Cargo.toml`).
- `update-alternatives --config|--set|--auto|--display` manages `/etc/alternatives` symlinks; RHEL calls it `alternatives`.
- Sandboxed apps: snap (`snap install`, strict confinement), Flatpak (`flatpak install flathub ...`), AppImage (single executable, `chmod +x`).
- Services: BIND `named` (`named-checkconf`, `named-checkzone`, `dig @server`), chrony (`chronyc sources`, `allow`), PTP (`ptp4l`, `phc2sys`), DHCP (Kea `kea-dhcp4 -t`, old `dhcpd.conf`), Apache (`apache2` / `httpd`, `apachectl configtest`), Nginx (`nginx -t`), Postfix (SMTP port 25, `postconf`, `mailq`), Dovecot (IMAP 143, IMAPS 993).

Lab 2.5: Manage Linux Using systemd

- `start` / `stop` / `restart` act now; `enable` / `disable` act at boot (symlinks in `<target>.wants/`); `enable --now` does both.
- `reload` re-reads the service's config without stopping it (needs `ExecReload=`); `restart` stops and starts (new PID); `daemon-reload` makes **systemd** re-read unit files after you edit them.
- `mask` links the unit to `/dev/null` so nothing can start it; `unmask` removes the link.
- `systemctl edit <unit>` creates a drop-in `/etc/systemd/system/<unit>.d/override.conf`; `--full` edits a full copy; `systemctl cat` shows the merged result; `systemctl revert` undoes it.
- Unit file priority: `/etc/systemd/system/` beats `/run/systemd/system/` beats `/usr/lib/systemd/system/`.
- Service essentials: `Type=simple|forking|oneshot|notify`, `ExecStart=`, `User=`, `Restart=on-failure`, `WantedBy=multi-user.target`.
- Timers: a `.timer` starts the `.service` of the same name; `OnCalendar=` (wall clock), `OnBootSec=` / `OnUnitActiveSec=` (relative), `Persistent=true`; list with `systemctl list-timers`.
- Mount unit names must match the path: `/mnt/lab` is `mnt-lab.mount` (`systemd-escape -p --suffix=mount`); fstab lines become mount units via `systemd-fstab-generator`.
- Targets: `systemctl get-default`, `set-default`, `isolate`; `multi-user.target` = runlevel 3, `graphical.target` = 5, `rescue.target` = 1.
- `systemd-analyze` (boot time), `systemd-analyze blame` (slowest units), `systemd-analyze critical-chain`, `systemd-analyze verify`, `systemd-analyze calendar`.
- `hostnamectl set-hostname`, `timedatectl set-timezone` / `set-ntp`, `resolvectl status` / `query` / `flush-caches`; `/etc/resolv.conf` points to the `systemd-resolved` stub at `127.0.0.53`.
- `sysctl -w key=value` is runtime only; persist in `/etc/sysctl.d/*.conf` and apply with `sysctl --system` or `sysctl -p`.

Lab 2.6: Manage Applications in Containers

- Runtime stack: Docker CLI -> `dockerd` -> **containerd** -> **runC** (OCI runtime). **Podman** is daemonless, Docker-compatible, and rootless by default (uses `crun` / `runc`).
- Image commands: `pull`, `build -t name:tag .`, `tag`, `push`, `rmi`, `image ls`, `history` (layers), `image inspect`, `image prune` (dangling; `-a` for all unused).

- Dockerfile: `FROM` (base), `RUN` (build-time command, new layer), `COPY` / `ADD`, `ENV`, `USER` (non-root), `EXPOSE`, `ENTRYPOINT` (fixed program), `CMD` (default arguments; replaced by arguments after the image name). Override the entrypoint with `--entrypoint`.
- Container commands: `run -d --name -p host:container -e VAR=value -v src:dst`, `ps -a`, `logs -f`, `exec -it name sh`, `inspect --format`, `stop`, `start`, `restart`, `rm` (`-f` to force), `container prune`.
- Volumes: `volume create`, `-v name:/path` (named) vs `-v /host/path:/path` (bind), `:ro`; data outside volumes lives in the container's overlay upper layer and is lost with `rm`. `volume prune` removes anonymous volumes; `-a` includes named ones.
- Storage: OverlayFS, through Docker's `overlay2` driver or (Docker 29+) containerd's `overlayfs` snapshotter; OverlayFS `lowerdir` (read-only image layers), `upperdir` (container writes), `merged` (the container's view).
- SELinux: bind mounts need `container_file_t`; `:z` = shared label, `:Z` = private label for one container; check with `ls -Z`, denials in `ausearch -m avc`.
- Networks: `network create` (user-defined bridge has DNS by container name), `bridge` (default, NAT, needs `-p`), `host` (shares host stack), `none` (loopback only), `macvlan` (own MAC on LAN, needs `-o parent=`, promiscuous mode), `ipvlan` (shares parent MAC), `overlay` (multi-host, needs Swarm). `network prune`.
- `-p 8080:80` maps host port 8080 to container port 80; `-P` maps all exposed ports to random ports; `-p 127.0.0.1:8080:80` binds only to loopback.
- Unprivileged containers lack most capabilities; `--privileged` grants all capabilities and host devices (avoid; use `--cap-add`). Rootless containers map container root to an unprivileged host UID (`/etc/subuid`).
- The `docker` group is root-equivalent.

Lab 3.1: Authentication, Authorization, and Accounting

- **AAA:** authentication = prove identity, authorization = decide what is allowed, accounting = record what happened.
- PAM files live in `/etc/pam.d/<service>`; types are `auth`, `account`, `password`, `session`.
- Control flags: `required` (fail at end), `requisite` (fail now), `sufficient` (succeed now if nothing failed), `optional`; plus `include` and `substack`.
- `pam_access.so` reads `/etc/security/access.conf`; `pam_limits.so` reads `/etc/security/limits.conf`; `pam_nologin.so` checks `/etc/nologin`.
- Polkit actions are listed by `pkaction`, tested with `pkcheck`, and run as root with `pkexec`. Local JavaScript rules go in `/etc/polkit-1/rules.d/*.rules`.
- LDAP: TCP 389, LDAPS 636; tools `ldapsearch`, `ldapadd`, `ldapmodify`; entries identified by DN such as `uid=jdoe,ou=people,dc=lab,dc=local`.
- Kerberos: port 88, KDC issues tickets; `kinit` gets a ticket, `klist` lists, `kdestroy` removes; config in `/etc/krb5.conf`; realm names are uppercase.
- SSSD (`/etc/sssd/sssd.conf`, mode 600) and Winbind connect Linux to LDAP/AD; `realm discover` and `realm join` set them up; `nsswitch.conf` needs `sss` (or `winbind`).
- Samba: config `/etc/samba/smb.conf`, check with `testparm`, users added with `smbpasswd -a`, client `smbclient`, port 445.
- `journalctl` filters: `-u unit`, `-p priority`, `-b`, `--since`, `-f`, `-k`; persistent storage needs `/var/log/journal` or `Storage=persistent`.
- rsyslog selectors are `facility.priority`; config in `/etc/rsyslog.conf` and `/etc/rsyslog.d/`; test messages with `logger`.

- logrotate: `/etc/logrotate.conf` and `/etc/logrotate.d/`; `-d` dry run, `-f` force. auditd: `auditctl -w ... -p ... -k ...`, persistent rules in `/etc/audit/rules.d/` compiled by `augenrules` into `audit.rules`; search with `ausearch -k`, report with `aureport`.

Lab 3.2: Configure and Implement Firewalls

- Netfilter hooks/chains: PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING. DNAT in PREROUTING, SNAT/masquerade in POSTROUTING.
- `firewall-cmd` changes are **runtime** unless you add `--permanent`; permanent changes need `--reload`. `--runtime-to-permanent` saves the current runtime.
- firewalld: `--get-active-zones`, `--list-all`, `--add-service=`, `--add-port=80/tcp`, `--add-source=`, `--zone=`, `--add-rich-rule='rule ...'`, `--add-masquerade`, `--add-forward-port=`.
- A service is a named set of ports (`/usr/lib/firewalld/services/*.xml`); ufw service names come from `/etc/services` and app profiles from `/etc/ufw/applications.d/`.
- ufw: allow SSH **before** `ufw enable`; `ufw status numbered`, `ufw delete <n>`, `ufw allow from <ip> to any port <p> proto tcp`, `ufw default deny incoming`.
- nftables: `nft list ruleset`, `nft add table|chain|rule`, `nft -a` shows handles, `nft delete rule ... handle N`, `nft -f file` loads atomically; config in `/etc/nftables.conf`.
- iptables: `-A` append, `-I` insert, `-D` delete, `-L -n -v` list, `-F` flush, `-P` policy, `-t nat`, `-j ACCEPT|DROP|REJECT|DNAT|SNAT|MASQUERADE`; save with `iptables-save`.
- ipset: `ipset create NAME hash:ip`, `ipset add`, referenced by `-m set --match-set NAME src`.
- **Stateful** = connection tracking (`ct state established,related`, `-m conntrack --ctstate`); **stateless** = each packet judged alone.
- **NAT** types: SNAT (source, outbound), masquerade (dynamic SNAT), PAT/NAPT (many hosts share one IP via ports), DNAT (destination, port forwarding).
- Routing needs `net.ipv4.ip_forward = 1`: `sysctl -w` for now, `/etc/sysctl.d/*.conf` for boot.
- DROP = silent (client times out); REJECT = immediate error (client sees refused/unreachable).

Lab 3.3: Operating System Hardening

- Edit sudoers only with `visudo`; drop-ins go in `/etc/sudoers.d/`; `visudo -c` checks syntax, `sudo -l` lists a user's rights.
- `NOPASSWD` runs without a password (convenience vs risk); `NOEXEC` blocks shell escapes from commands like `less` / `vi`. Admin group: `sudo` on Ubuntu, `wheel` on Rocky.
- `sudo -i` = root login shell via your password (logged); `su -` = switch user with a login shell via the target's password.
- `chattr +i` immutable (even root cannot change/delete), `chattr +a` append only; view with `lsattr`.
- Permissions: octal `4000` setuid, `2000` setgid, `1000` sticky; `chmod 2770`, `chmod +t`, symbolic `u=rwx,g=rx,o=`. setuid on files, setgid+sticky mainly on directories.
- `umask`: files `666 - umask`, directories `777 - umask`; `022` gives 644/755.
- ACLs: `setfacl -m u:name:rw file`, `-d` default ACL on a directory, `-x` remove, `-b` wipe; `getfacl` to view; the **mask** caps named entries; `+` in `ls -l` means an ACL exists.
- SELinux states: enforcing, permissive, disabled (`getenforce`, `setenforce 0/1`, `/etc/selinux/config`). Context = `user:role:type:level`; view with `ls -Z`, `ps -Z`.
- Fix SELinux labels with `restorecon` (applies policy) or `chcon` (temporary); add path rules with `semanage fcontext -a` then `restorecon`; ports with `semanage port -a`; switches with `setsebool -P`; diagnose with `ausearch -m avc`, `sealert`, and `audit2allow`.

- SSH hardening (`sshd_config` / `sshd_config.d/`): `PermitRootLogin no` , `PasswordAuthentication no` , `X11Forwarding no` , `AllowUsers` / `AllowGroups` ; always `sshd -t` before reload. Keys: `ssh-keygen` , `ssh-copy-id` , `ssh-agent` + `ssh-add` . Tunnels: `-L` local, `-R` remote, `-D` SOCKS.
- SFTP chroot: `Match Group` , `ChrootDirectory` (owned by root, not writable), `ForceCommand internal-sftp` .
- `fail2ban` : jails in `jail.d/*.local` , `fail2ban-client status <jail>` , `set <jail> unbanip <ip>` .
- Avoid cleartext Telnet (23), FTP (21), TFTP (69); check with `ss -tulpn` . Block filesystems in `/etc/modprobe.d/` with `install <fs> /bin/false` + `blacklist` . Remove unneeded SUID with `chmod u-s` (find with `find / -perm -4000`). Secure Boot: UEFI-only, checked with `mokutil --sb-state` .

Lab 3.4: Harden User Accounts

- `pam_pwquality` (config `/etc/security/pwquality.conf`) enforces **length** (`minlen`) and **complexity** (`minclass` , `dcredit` , `ucredit` , `lcredit` , `ocredit`). Negative credit = required count; positive credit = bonus length.
- `pam_pwhistory remember=N` (or `pam_unix remember=N`) blocks password **reuse**; the **history** of hashes is in `/etc/security/opasswd` .
- `chage -M` max days, `-m` min days, `-W` warn days, `-I` inactive days, `-E` account expiry date, `-d 0` force change at next login, `-l` list. `passwd -e` also forces a change.
- `/etc/login.defs` (`PASS_MAX_DAYS` , `PASS_MIN_DAYS` , `PASS_WARN_AGE`) sets defaults for **new** accounts only.
- Account lockout: `pam_faillock` with `/etc/security/faillock.conf` (`deny` , `unlock_time`); view/reset with `faillock --user NAME [--reset]` . **pam_tally2** (`pam_tally2 --user NAME --reset`) is the removed predecessor.
- Ubuntu manages PAM with `pam-auth-update` ; Rocky/RHEL with `authselect` (`enable-feature with-faillock`).
- **MFA** = two or more different factor types. TOTP via `pam_google_authenticator.so` plus `KbdInteractiveAuthentication yes` and `AuthenticationMethods` in `sshd_config` .
- **Breach** checking: HIBP range API uses k-anonymity: only the first 5 hex characters of the SHA-1 hash leave your machine.
- **Restricted shell**: `/sbin/nologin` (or `/bin/false`) for service accounts; `/bin/rbash` blocks `cd` , `/` in commands, changing `PATH` , and output redirection.
- Avoid running as **root**: locked root password, `PermitRootLogin no` , least-privilege `sudo` rules, and `sudo` logs in `/var/log/auth.log` (Ubuntu) or `/var/log/secure` (RHEL).

Lab 3.5: Use Cryptography on Linux

- **Data at rest**: GPG for files (`gpg -c` symmetric; `gpg -e -r` to a recipient; `--detach-sign` and `--verify`), LUKS2 for block devices (`cryptsetup luksFormat --type luks2` , `open` , `close` , `luksAddKey` , `luksDump` , `luksErase`), and `/etc/crypttab` for unlocking at boot.
- LUKS2 uses **Argon2id** (memory-hard) as its default PBKDF; LUKS1 used PBKDF2. Up to 32 key slots unlock one master key.
- **Data in transit**: TLS (via OpenSSL or LibreSSL), SSH, and VPNs such as WireGuard (UDP 51820, `wg` , `wg-quick` , `[Interface]` / `[Peer]` , `AllowedIPs`).
- Allowed **TLS** versions: 1.2 and 1.3. SSLv2, SSLv3, TLS 1.0 and 1.1 are deprecated. Test with `openssl s_client -connect host:443 -tls1_2` and `nmap --script ssl-enum-ciphers` .
- **LibreSSL** is OpenBSD's 2014 fork of OpenSSL; `openssl version` tells you which you have.
- **SHA-256**: `sha256sum file` , `sha256sum -c SHA256SUMS` . **HMAC** adds a secret key: `openssl dgst -sha256 -hmac KEY file` .
- **Weak algorithms** to remove: MD5, SHA-1, DES/3DES, RC4, short RSA keys. Use `sshd_config` (`Ciphers` , `MACs` , `KexAlgorithms`) and `update-crypto-policies --set DEFAULT|FUTURE|LEGACY|FIPS` on RHEL.

- **Trusted root** stores: Ubuntu `/usr/local/share/ca-certificates/` + `update-ca-certificates` (bundle in `/etc/ssl/certs/`); RHEL `/etc/pki/ca-trust/source/anchors/` + `update-ca-trust`.
- **No-cost** CA: Let's Encrypt via ACME/`certbot`, 90-day automated certificates. **Commercial** CA: paid, OV/EV validation, support and warranties.
- Avoid **self-signed** leaf certificates; they cannot be verified and teach users to ignore warnings. Use a public CA or a private internal CA whose root you distribute.

Lab 3.6: Compliance, Auditing, and Detection

- **Anti-malware** on Linux: ClamAV (`freshclam` updates signatures, `clamscan -r -i` scans, `--move` quarantines). Test safely with the EICAR string.
- **Indicators of compromise**: unknown listening ports (`ss -tulpn`), new or UID 0 accounts, new cron jobs/timers, SUID files not owned by a package (`find / -perm -4000`), odd logins (`last`, `lastb`), unknown outbound connections, high CPU processes, changed binaries.
- **CVE** = unique ID for a public vulnerability; **CVSS** = 0.0 to 10.0 severity score (Low under 4, Medium 4 to 6.9, High 7 to 8.9, Critical 9 to 10) built from a vector.
- **Backporting**: distributions apply security fixes to older versions; check changelogs (`apt changelog`, `rpm -q --changelog`), `pro fix CVE-...`, or `dnf updateinfo`, not just upstream version numbers.
- **Port scanner**: `nmap -sV` finds open ports and versions; `nmap --script` adds checks. **Protocol analyzer**: `tcpdump -w` / `-r` / `-A`, Wireshark, `tshark`. Service **misconfiguration** = unsafe settings, not a code bug.
- **OpenSCAP**: `oscap xccdf eval --profile <id> --results r.xml --report r.html <datastream>` with content from `scap-security-guide`. **CIS** Benchmarks: Level 1 (baseline) and Level 2 (strict); Ubuntu's tool is `usg` (Ubuntu Pro).
- **AIDE**: `aide --init` (Ubuntu `aideinit`), `aide --check`, `aide --update`; config `/etc/aide/aide.conf` (Ubuntu) or `/etc/aide.conf` (RHEL). **rkhunter**: `--propupd` baseline, `--check --sk`.
- Installed file verification: `debsums -s`, `dpkg --verify` (Debian/Ubuntu), `rpm -V pkg` / `rpm -Va` (codes S, M, 5, D, L, U, G, T). **Signed package** checks: `rpm -K`, `gpgcheck=1`, apt `InRelease` + `Signed-By`.
- Destruction: `shred -v -n 1 -z -u file`, `badblocks -w` (destructive write test), `dd if=/dev/urandom of=/dev/sdX bs=1M`, and **cryptographic** erase (`cryptsetup luksErase`), best for SSDs.
- **Supply chain**: xz backdoor (CVE-2024-3094); verify downloads with checksums **and** GPG signatures; pin versions; keep an SBOM (SPDX, CycloneDX).
- Banners: `/etc/issue` (local, before login), `/etc/issue.net` (network, needs `Banner /etc/issue.net` in `sshd_config`), `/etc/motd` (after login; Ubuntu adds dynamic `/etc/update-motd.d/`).

Lab 4.1: Automation and Orchestration

- **IaC** = infrastructure described in version-controlled text files; declarative tools are **idempotent** (running twice changes nothing the second time).
- **Ansible**: agentless, push over SSH, needs only Python on targets; **inventory** lists hosts and groups; **playbooks** are YAML; **modules** do the work; **ad hoc** = `ansible <hosts> -m <module> -a <args>`; **facts** come from the `setup` module; **collections** install with `ansible-galaxy collection install`.
- Ansible handlers run once at the end of a play, only when notified. `--check` is a dry run, `--diff` shows changes.
- **Puppet**: agent pulls its catalog from the server every 30 minutes; trust uses TLS **certificates** signed with `puppetserver ca sign`; code is in manifests (`.pp`) with **classes** inside **modules**; facts come from `facter`; `puppet apply` runs without a server.
- **OpenTofu** (Terraform fork): **providers** call platform **APIs**; **resources** are managed objects; **state** (`terraform.tfstate`) maps code to real objects. Workflow: `init`, `plan`, `apply`, `destroy`.

- **Kickstart**: RHEL unattended install file; sample at `/root/anaconda-ks.cfg`; boot option `inst.ks=`; validate with `ksvalidator`.
- **cloud-init**: first-boot config; user-data starts with `#cloud-config`; logs in `/var/log/cloud-init.log`; check with `cloud-init status`; NoCloud seed volume label `cidata`.
- **CI/CD pipeline** stages: build, lint, test, security scan, deploy. **Shift left** = test earlier. **DevSecOps** = security inside the pipeline. **GitOps** = Git is the source of truth and an agent syncs the cluster to it.
- **Kubernetes**: Pod (smallest unit), Deployment (keeps N Pods, rolling updates), Service (stable IP/DNS, ClusterIP/NodePort/LoadBalancer), ConfigMap (settings), Secret (base64, not encrypted), volumes/PVCs (storage), env variables from ConfigMaps/Secrets.
- **Docker Swarm**: `docker swarm init`, nodes are managers or workers, a service has tasks (one container each), overlay networks span nodes, `docker service scale web=5`.
- **Compose**: `compose.yaml`, `docker compose up -d`, `ps`, `logs -f`, `down` (add `-v` to delete volumes); `podman compose` / `podman-compose` read the same file.

Lab 4.2: Automate Tasks with Shell Scripts

- The first line `#!/bin/bash` (shebang) selects the interpreter; make scripts executable with `chmod +x`.
- `$?` holds the last return code; 0 means success. `$#` is the argument count, `$1` ... the arguments, `$@` all arguments, `$0` the script name.
- `export VAR` makes a variable an **environment variable** visible to child processes; `local` limits a variable to a function; `unset` deletes it; `set` lists variables or sets options (`-e`, `-u`, `-x`, `-o pipefail`); `alias` / `unalias` manage command shortcuts (not expanded in scripts by default).
- Parameter expansion: `${var:-default}`, `${var:=default}`, `${#var}`, `${var#pat}`, `${var%pat}`, `${var/old/new}`, `${var:offset:len}`.
- Command substitution: `$(cmd)` (preferred) and `` `cmd` `` (legacy). Subshell: `( cmds )` cannot change the parent's variables or directory.
- `IFS` controls how Bash splits words (`IFS=, read -r a b c`); awk's `OFS` sets the separator between printed fields.
- Numbers: `-eq -ne -lt -le -gt -ge` (or `(( ))`). Strings: `=`, `==`, `!=`, `<`, `>` (sort order). `<=` and `>=` are not valid string operators in `[[ ]]`.
- File and string tests: `-d` directory, `-f` regular file, `-e` exists, `-z` empty string, `-n` non-empty string, `!` negates.
- Regex: `[[ $var =~ ^regex$ ]]`, unquoted pattern, captures in `BASH_REMATCH`.
- `case` patterns end with `)`, branches end with `;;`, and the block ends with `esac`. `if` ends with `fi`. Loops end with `done`.
- `while` loops while a command succeeds; `until` loops while it fails; `for` loops over a list.

Lab 4.3: Python Basics for System Administration

- Use **Python 3** (`python3`); Python 2 is end-of-life. Ubuntu 24.04 ships Python 3.12.
- **Indentation** defines code blocks; a line ending in `:` starts a block; PEP 8 says 4 spaces.
- Data types: **integer** (`int`), **floating point** (`float`), **string** (`str`), **Boolean** (`True` / `False`), **list** (`[ ]`, ordered), **dictionary** (`{key: value}`).
- **Built-in modules** (standard library) need no install: `os`, `sys`, `subprocess`, `shutil`, `pathlib`, `json`, `argparse`, `datetime`, `re`, `logging`, `socket`.
- A **module** is a `.py` file; a **package** is a directory of modules with `__init__.py`. Use them with `import x` or `from x import y`.

- **Virtual environment**: `python3 -m venv venv`, `source venv/bin/activate`, `deactivate`. Ubuntu needs `python3-venv`.
- **pip install** packages from PyPI inside a venv; `pip freeze > requirements.txt` records **dependencies**; `pip install -r requirements.txt` restores them.
- `error: externally-managed-environment` (PEP 668) means pip will not modify the system Python: use a venv, `apt install python3-xyz`, or `pipx`.
- **PEP 8** is Python's style guide: 4-space indents, 79-character lines, `snake_case` functions and variables, `CapWords` classes, `UPPER_CASE` constants, imports at the top. Check it with `pycodestyle` or `flake8`.
- `if __name__ == "__main__":` runs code only when the file is executed directly; `sys.exit(n)` sets the exit status.

Lab 4.4: Version Control with Git

- `git config --global user.name/user.email` sets your identity; levels are `--system`, `--global`, `--local`.
- `git init` creates a repo; `git init --bare` creates a server-side repo with no working directory; `git clone URL` copies a remote repo.
- Workflow: edit, `git add` (stage), `git commit -m` (snapshot). `git status` shows state; `git log` (`--oneline --graph --all`) shows history.
- `git diff` = working vs staged; `git diff --staged` = staged vs last commit; `git diff A B` = between commits.
- `.gitignore` lists patterns (`*.log`, `tmp/`) for untracked files Git should ignore; it does not remove files already committed.
- `git branch` lists/creates branches; `git checkout -b NAME` or `git switch -c NAME` creates and switches; `git branch -d` deletes a merged branch (`-D` forces).
- `git merge BRANCH`: fast-forward when possible, otherwise a merge commit. Conflicts show `<<<<<<<`, `=======`, `>>>>>>>`; fix, `git add`, `git commit`. `git merge --squash` combines a branch into one staged change you then commit.
- `git rebase main` replays your commits on top of `main` (linear history, new hashes). Never rebase published commits.
- `git stash` saves uncommitted work; `git stash pop` restores it; `git stash list` shows the stack.
- `git reset --soft` (keep staged), `--mixed` (default, keep in working dir), `--hard` (discard everything). `git reflog` finds lost commits.
- `git tag v1.0` (lightweight) vs `git tag -a v1.0 -m "msg"` (annotated); push tags with `git push --tags`.
- `git fetch` downloads without merging; `git pull` = fetch + merge (or `--rebase`); `git push` uploads and is rejected if it is not a fast-forward.

Lab 4.5: Use AI Responsibly as a Linux Admin

- AI use cases for admins: **generation of code**, **regular expressions**, **infrastructure as code**, **documenting code and creating documentation**, **recommendations for compliance**, **security review**, **code optimization**, and **code linting**.
- **Never copy/paste AI output without review and QA.** LLMs can hallucinate options, modules, and packages, and they often suggest deprecated or insecure patterns.
- **Verify output** with tools: `bash -n` and ShellCheck for scripts, `grep -E` with good and bad test cases for regex, `ansible-lint`, `--syntax-check`, and `--check --diff` for IaC, `python3 -W error` and linters for Python.
- A passing linter does not mean code is secure; hard-coded secrets, `curl | bash`, disabled TLS checks, and `0777` permissions need **human review**.
- Test AI code in a lab or sandbox first, then send it through Git, pull requests, peer review, and CI like any other change.
- **Data governance:** data sent to a public AI service may be stored, used for **LLM training**, or read during **human review** by the provider. Never share secrets, personal data, or customer data.

- **Public** models: most capable, data leaves your control. **Private** or **local models** (for example via Ollama): data stays in-house, less capable, you maintain them.
- Redact secrets, IPs, host names, and emails before sharing; verify the redaction; rotate any secret that leaked.
- **Adhere to corporate policy:** approved tools only, allowed data classes, labeling rules, and human accountability.
- **Prompt engineering:** give role, context (OS and versions), a clear task, constraints, output format, and examples; ask for assumptions and tests.

Lab 5.1: Monitoring Concepts and Configurations

- **SLA** = contract with customers (with penalties); **SLO** = internal target (stricter than the SLA); **SLI** = the actual measurement (for example, % of successful requests).
- Error budget = 100% minus the SLO. 99.9% per 30 days allows about 43 minutes of downtime.
- SNMP agents listen on **UDP 161**; traps go to the manager on **UDP 162**. `snmpget` reads one OID, `snmpwalk` reads a subtree, `snmptrap` sends a trap, `snmptrapd` receives traps.
- A **MIB** maps numeric **OIDs** to names and descriptions. On Ubuntu, `snmp-mibs-downloader` and commenting out `mibs :` in `/etc/snmp/snmp.conf` enable names.
- SNMPv1/v2c use clear-text **community strings**; SNMPv3 adds users with authentication (`-a SHA`) and privacy/encryption (`-x AES`), security level `authPriv` .
- **Agent** monitoring installs software on each host (node_exporter on port 9100, Zabbix agent); **agentless** uses existing protocols (SSH, SNMP, WMI).
- A **webhook** is an HTTP POST (usually JSON) sent to a URL when an event occurs.
- A **health check** is a repeated probe (for example `curl` expecting HTTP 200) used by monitors and load balancers.
- **Log aggregation**: rsyslog `imtcp` / `imudp` on the server, `omfwd` (or `@@host:514` for TCP, `@host:514` for UDP) on clients.
- Threshold crossed leads to an alert, which is delivered as a notification. Too many non-actionable alerts cause **alert fatigue**.
- `logger -t tag -p facility.severity "msg"` writes events to the log; `journalctl -t tag -p warning` filters them.

Lab 5.2: Troubleshoot Hardware, Storage, and OS Issues

- Method: identify symptoms, gather data, hypothesize, test, fix, verify, document. Ask “what changed?”
- “No space left on device” with free space in `df -h` means **inode exhaustion** (`df -i`). `df` larger than `du` means a **deleted-but-open file** (`lsof +L1`); restart the process or truncate instead of `rm` .
- Error messages: “Permission denied” = permissions/ACL; “Operation not permitted” = immutable attribute (`lsattr` , `chattr -i`); “Read-only file system” = `ro` mount or kernel remount after errors (`dmesg`); “Disk quota exceeded” = `quota` , `repquota` , `setquota` , `edquota` .
- “Bad superblock”: `e2fsck -b 32768 /dev/sdX1` (find backups with `mke2fs -n` or `dumpe2fs`); never `fsck` a mounted filesystem; XFS uses `xfs_repair` . `fsck` cannot detect corrupted file contents.
- Bad `/etc/fstab` = emergency mode. Fix: `mount -o remount,rw /` , edit fstab, `systemctl daemon-reload` , `mount -a` . Prevent with `findmnt --verify` , `mount -a` , and `nofail` .
- GRUB: edit `/etc/default/grub` , then `update-grub` (Ubuntu) or `grub2-mkconfig -o /boot/grub2/grub.cfg` (Rocky). Press `e` at the menu for one-time edits; `Ctrl+X` boots. `(initramfs)` prompt = root device not found; `grub rescue>` = GRUB cannot find its files.
- Kernel panic: read the console; `kernel.panic` sets auto-reboot seconds; kdump saves crash dumps to `/var/crash` ; `journalctl -b -1` shows the previous boot. Kernel corruption: boot an older kernel from **Advanced options**, `dpkg -V` / `rpm -V` , reinstall the kernel package, `update-initramfs -u` / `dracut -f` .

- Drivers: `lspci -k`, `lsmod`, `modinfo`, `modprobe`; disabled via `blacklist` or `install <mod> /bin/false` in `/etc/modprobe.d/`; missing ones in `linux-modules-extra`, DKMS, or `linux-firmware`.
- Dependencies: `dpkg -i` does not resolve them (`iU` state); fix with `apt --fix-broken install` / `apt -f install`, `dpkg --configure -a`; Rocky: `dnf install ./file.rpm`, `rpm -qpR`.
- Segfault = signal 11, exit code 139; `dmesg` shows `segfault at`; `coredumpctl list/info/debug`. OOM kill = `SIGKILL` (9), exit code 137, `journalctl -k | grep -i "killed process"`; limit with `MemoryMax=`.
- Memory leak = RSS growing steadily (`ps -o rss`, `pmap -x`, `smem`, `/proc/PID/status`).
- Unit failure: `systemctl status`, `journalctl -xeu unit`, `status=203/EXEC` = bad `ExecStart` path; always `daemon-reload` after edits; `reset-failed` after start-limit hits.
- Process states: `R`, `S`, `D` (uninterruptible, unkillable), `T` (stopped, `kill -CONT`), `Z` (zombie). `strace -p PID` shows where it waits.

Lab 5.3: Troubleshoot Networking Issues

- Troubleshoot bottom up: link (`ip link`, `ethtool`), MAC/ARP/MTU (`ip neigh`, `arping`), IP/routing (`ip addr`, `ip route`, `ping`, `traceroute` / `tracepath`), ports/firewall (`ss`, `nc`, `firewall-cmd`), then DNS/app (`dig`, `curl`).
- `NO-CARRIER` with `UP` = cable, switch port, or far end down (**link down**). Duplex mismatch (**link negotiation**) = link up but slow with collisions/CRC errors; fix with auto-negotiation on both sides (`ethtool -s IF autoneg on`).
- **MTU mismatch**: small packets work, large transfers hang. Test with `ping -M do -s 1472` (1472 + 28 = 1500).
- Wrong **subnet** mask: too wide makes remote hosts look local (`Destination Host Unreachable`, ARP `INCOMPLETE`); too narrow puts the gateway off-net. `ip route get <ip>` shows the decision.
- **Gateway**: wrong = `Destination Host Unreachable`; missing = `Network is unreachable`. Fix with `ip route replace default via X`, persist in netplan/nmcli.
- **Routing**: packets need a route back. `tcpdump` showing requests with no replies means a missing return route; a Linux router needs `net.ipv4.ip_forward=1`.
- Server **unreachable**: "Connection refused" = nothing listening; "timed out" = dropped (firewall/routing/host down); TCP "No route to host" = often a firewall reject.
- **DNS**: "Temporary failure in name resolution". `dig` bypasses `/etc/hosts`; `getent hosts` follows `nsswitch.conf`. Ubuntu: `resolvectl status`, stub `127.0.0.53`.
- **IP conflict**: intermittent connectivity, ARP entry flapping between MACs; detect with `arping -D -I IF IP` (exit 1 = conflict).
- **DHCP**: DORA (Discover, Offer, Request, ACK), UDP 67/68; `169.254.x.x` (APIPA) means no DHCP server answered. Check server logs and `journalctl -u systemd-networkd` / `NetworkManager`.
- **Bonding**: status in `/proc/net/bonding/bond0`; `active-backup` = mode 1, `802.3ad` (LACP) = mode 4 and needs switch support.
- **MAC spoofing**: `ip link set dev IF address ...`; hypervisors/switches may block non-assigned MACs.
- **Dual stack**: test with `ping -4/-6`, `curl -4/-6`; `getent ahosts` shows the order; `precedence ::ffff:0:0/96 100` in `/etc/gai.conf` prefers IPv4; link-local `fe80::` needs `%interface`.
- **Firewall**: `firewall-cmd --list-all`, `--permanent` + `--reload`, `--set-log-denied=all`; Ubuntu `ufw status verbose`.

Lab 5.4: Troubleshoot Security Issues

- Security fixes must not weaken security: fix the label, boolean, permission, key, or config; never "disable SELinux", `chmod 777`, `curl -k`, or `gpgcheck=0`.

- **SELinux**: wrong **context** = `ls -Z`, `restorecon -Rv` (persistent custom paths: `semanage fcontext -a`); optional feature off = **boolean** (`getsebool`, `setsebool -P`); port not allowed = **policy** (`semanage port -a -t http_port_t -p tcp 8090`). Evidence: `ausearch -m avc -ts recent`, `audit2why`, `sealert -a`. `mv` keeps labels, `cp` does not.
- **Permissions**: directories need `x` to be entered (`namei -l`); on a file with an **ACL**, `chmod` group bits change the **mask** (`getfacl` shows `#effective:`); **attributes** `i` (immutable) and `a` (append-only) block even root (`lsattr`, `chattr`).
- **Account access**: `passwd -S` (L = locked), `chage -l` / `chage -E` (account expiry), `getent passwd` (`nologin` shell), `faillock --user X --reset` (pam_faillock lockout).
- **Remote access**: `ssh -v` on the client plus `journalctl -u ssh` / `/var/log/secure` on the server; `StrictModes` needs `~/.ssh` `700` and `authorized_keys` `600`; `AllowUsers` / `AllowGroups`; test with `sshd -t`, view effective config with `sshd -T`.
- **Unpatched** systems: `apt list --upgradable`, `unattended-upgrades`, `apt-mark showhold`, `/var/run/reboot-required`, `needrestart`; Rocky `dnf updateinfo list --security`, `dnf upgrade --security`, `dnf needs-restarting -r`.
- **Exposed** service: `ss -tlnp` shows `0.0.0.0`, `[::]`, or `*` for all interfaces versus `127.0.0.1` for local only; confirm with `nmap -sT -p-`; fix by binding to loopback, requiring authentication, and a default-deny firewall (`ufw default deny incoming` after `ufw allow OpenSSH`).
- **Certificate** errors: "self-signed certificate" (untrusted; add the CA to the trust store with `update-ca-certificates` / `update-ca-trust`), "unable to get local issuer certificate" (missing intermediate; serve the full chain), "no alternative certificate subject name matches" (fix the SAN), "certificate has expired" (renew; check `notAfter` and the clock).
- Certificate tools: `openssl s_client -connect host:443 -servername host -showcerts`, `openssl x509 -noout -subject -issuer -dates -ext subjectAltName`, `openssl verify -CAfile root -untrusted intermediate cert`, `curl -v --cacert`, `openssl x509 -checkend`.
- **Repository** errors: apt "404 Not Found" / "does not have a Release file" = wrong URL or suite; "NO_PUBKEY" / "is not signed" = wrong or missing key in `Signed-By`. dnf: "Cannot download repomd.xml" = wrong `baseurl`; "GPG check FAILED" = wrong `gpgkey`. Check with `apt-cache policy`, `dnf repolist -v`, `dnf makecache`.
- **Obsolete/insecure** protocols: SSLv3, TLS 1.0, TLS 1.1, RC4, 3DES, CBC SSH ciphers, `diffie-hellman-group1-sha1`. Detect with `nmap --script ssl-enum-ciphers -p 443 host`; nginx `ssl_protocols TLSv1.2 TLSv1.3;`.
- **Cipher** negotiation: "no matching cipher found" / "no matching key exchange method found. Their offer: ..." means the lists do not overlap; diagnose with `ssh -vv`, `ssh -Q cipher`, `sshd -T`; fix the old client. Rocky: `update-crypto-policies --show` (DEFAULT, LEGACY, FUTURE, FIPS).

Lab 5.5: Troubleshoot Performance Issues

- Work resource by resource (CPU, memory, disk, network): utilization, saturation, errors. "High" means higher than the **baseline**.
- **Load average** = runnable + uninterruptible (`D`) tasks over 1, 5, 15 minutes; compare with `nproc`. High load with busy CPUs = **CPU bottleneck**; high load with idle CPUs and high `wa` = storage.
- `vmstat`: `r` run queue, `b` **blocked processes**, `si` / `so` **swapping**, `cs` **context switches**, `wa` **I/O wait**, `st` steal. Ignore the first line.
- `iostat -x`: `r_await` / `w_await` = **disk latency** (ms), `aqu-sz` = queue, `%util` near 100 = saturated. A `D`-state process cannot be killed until its I/O completes.
- **Out of memory**: "Killed process" in `journalctl -k` / `dmesg`, exit code 137; control with `MemoryMax=` and `oom_score_adj` / `OOMScoreAdjust=`.
- `nice` / `renice` range -20 (favored) to 19; only root can lower the value. Cgroup limits: `CPUQuota=`, `CPUWeight=`, `MemoryMax=`, `IOWriteIOPSMax=` via unit files, `systemd-run`, or `systemctl set-property`.

- **Slow startup**: `systemd-analyze`, `systemd-analyze blame`, `systemd-analyze critical-chain`.
- **Failed log-in attempts**: `journalctl -u ssh` (Rocky `sshd`), `/var/log/auth.log` (Rocky `/var/log/secure`), `lastb`; fix with `fail2ban` and key-only SSH.
- Network: **latency** = RTT, **jitter** = `mdev` / `StDev`, **packet drops** = loss, `ip -s link`, `tc -s qdisc`; **low throughput** = `iperf3`. `netem` emulates delay/jitter/loss, `tbf` limits rate.
- `curl -w`: slow `time_connect` = network (**slow response times**); fast connect, slow `time_starttransfer` = **slow application response**.
- **Random timeouts/disconnects** come in bursts (flapping links, IP conflicts, firewall or NAT idle timeouts). **Slow remote storage**: `nfsiostat` (RTT vs exe), `mountstats`, `nfsstat`.
- **Hardware errors**: `journalctl -k -p err`, MCE/EDAC/I/O errors in `dmesg`, `ras-mc-ctl --summary`, `smartctl -a`. **Sluggish terminal behavior/unresponsive** system: `uptime`, `vmstat`, `/proc/pressure/*`.

Symptom	First tools to reach for
CPU bottleneck, high CPU usage, high load average	`uptime`, `top`, `mpstat -P ALL`, `pidstat -u`, `vmstat` (`r`)
High context switching	`vmstat` (`cs`), `pidstat -w`, `sar -w`
Swapping, out of memory	`free -h`, `swapon --show`, `vmstat` (`si`/`so`), `sar -W`, `journalctl -k | grep -i oom`
High I/O wait, high disk latency, blocked processes	`vmstat` (`wa`, `b`), `iostat -x`, `iotop`, `pidstat -d`, `ps` state `D`
Slow startup	`systemd-analyze blame`, `systemd-analyze critical-chain`
High failed log-in attempts	`journalctl -u ssh`, `/var/log/auth.log`, `lastb`, `fail2ban-client status`
High latency, jitter, packet drops	`ping`, `mtr`, `ip -s link`, `tc -s qdisc`, `nstat`
Low throughput	`iperf3`, `ethtool`, `tc -s qdisc`, `iostat` (for disk)
Slow response times, slow application response	`curl -w`, `ss`, application logs, `strace`
Random timeouts, random disconnects	`ping -O`, `journalctl -k`, `ip -s link`, firewall/NAT logs
Slow remote storage response	`nfsiostat`, `mountstats`, `nfsstat`
Hardware errors	`journalctl -k -p err`, `dmesg`, `ras-mc-ctl`, `smartctl`
System unresponsiveness, sluggish terminal behavior	`uptime`, `vmstat`, `/proc/pressure/*`, `systemd-cgtop`
Exceeding baselines	`sar -u/-q/-r/-b/-d/-n DEV -f /var/log/sysstat/saDD` (Rocky: `/var/log/sa/`)

Appendix C: Exam Objective Coverage Map

This map shows where each topic from the official XK0-006 objectives is practiced. Topics are listed under the part of the lab where they first appear.

1.1 Explain basic Linux concepts.

Practiced in Lab 1.1.

Where in the lab	Exam topics practiced
What you'll practice	`Filesystem Hierarchy Standard`
Task 1: Look at the boot files on ubuntu-lab	`configuration file`, `kernel`, `initrd`, `/boot`
Task 2: Read the bootloader configuration files and the kernel command line	`bootloader`, `parameter`, `/dev`, `/etc`, `/proc`
Task 3: Read the story of the last boot	`/var`
Task 5: Compare GRUB tools and look inside the initrd on both systems	`/bin`, `/sbin`, `x86`, `x86_64`
Task 6: Explore PXE network booting	`PXE`
Task 7: FHS scavenger hunt	`/home`, `/lib`, `/tmp`, `/usr`
Task 8: Identify the architecture and the distribution family	`AArch64`, `RISC-V`, `AMD64`, `RPM`, `dpkg`
Task 9: Explore the GUI stack and software licenses	`display manager`, `window manager`, `X Server`, `Wayland`, `open source`, `free software`, `proprietary`, `copyleft`
Exam corner	`boot process`

1.2 Summarize Linux device management concepts and tools.

Practiced in Lab 1.2.

Where in the lab	Exam topics practiced
Task 1: List and inspect kernel modules	`kernel module`, `lsmod`, `modinfo`, `modprobe`
Task 2: Load and unload modules with modprobe, insmod, and rmmod	`insmod`, `rmmod`
Task 3: Configure modules with /etc/modprobe.d and depmod	`depmod`
Task 4: Take a hardware inventory	`dmesg`, `dmidecode`, `lscpu`, `lshw`, `lsmem`, `lspci`, `lsusb`
Task 5: Try the server and GPU hardware tools	`ipmitool`, `lm_sensors`, `GPU`, `nvtop`
Task 7: Rebuild the initrd on Rocky with dracut (and meet mkinitrd)	`dracut`, `mkinitrd`
Task 8: Explore embedded-system traits	`embedded`

1.3 Given a scenario, manage storage in a Linux system.

Practiced in Lab 1.3.

Where in the lab	Exam topics practiced
What you'll practice	`Logical Volume Manager`
Task 1: Find your extra disks and install the tools	`gdisk`, `growpart`, `lsblk`, `parted`, `xfs`, `btrfs`, `fio`, `RAID`, `mdadm`, `autofs`, `mount`, `ro`, `NFS`, `SMB`, `Samba`
Task 2: Partition a disk with fdisk (MBR) and parted	`fdisk`, `ext4`
Task 3: Create a GPT partition with gdisk, then grow it with growpart	`blkid`, `df`, `mkfs`, `resize2fs`, `xfs_growfs`, `/etc/fstab`, `umount`, `rw`, `inode`
Task 4: Build LVM storage	`lvcreate`, `lvdisplay`, `lvs`, `vgcreate`, `vgdisplay`, `vgs`, `pvcreate`, `pvdisplay`, `pvs`
Task 5: Grow, move, export, and import LVM storage	`lvchange`, `lvresize`, `lvextend`, `vgchange`, `vgexport`, `vgextend`, `vgimport`, `vgscan`, `pvmove`, `pvremove`, `pvresize`, `pvscan`, `du`
Task 6: Work with btrfs, tmpfs, inodes, and the filesystem utilities	`tmpfs`, `fsck`, `xfs_repair`
Task 7: Use mount options, /etc/fstab, and the mount tables	`/etc/mtab`, `/proc/mounts`, `noatime`, `nodev`, `nodiratime`, `noexec`, `nofail`, `nosuid`, `remount`
Task 8: Build and break a RAID 1 mirror with mdadm	`/proc/mdstat`
Clean up	`lvremove`, `vgremove`

1.4 Given a scenario, manage network services and configurations on a Linux server.

Practiced in Lab 1.4.

Where in the lab	Exam topics practiced
Task 1: Inspect the network with hostname and ip	`NetworkManager`, `hostname`, `ip address`, `ip link`, `ip route`, `ss`
Task 2: Follow name resolution through hosts, nsswitch, and resolv.conf	`/etc/hosts`, `/etc/resolv.conf`, `/etc/nsswitch.conf`, `dig`, `ping`
Task 3: Configure a second IP address with Netplan	`netplan apply`, `netplan status`, `netplan try`, `/etc/netplan`, `nc`
Task 4: Manage connections with NetworkManager (nmcli and nmtui) on rocky-lab	`nmcli`, `nmconnect`
Task 5: Test reachability and the path with ping, arp, and traceroute tools	`arp`, `mtr`, `ping6`, `tracepath`, `traceroute`
Task 6: Query DNS with dig and nslookup, and test web services with curl	`curl`, `nslookup`
Task 7: Find listening ports and test them with ss, nc, and nmap	`nmap`
Task 8: Measure, inspect, and capture with iperf3, ethtool, and tcpdump	`ethtool`, `iperf3`, `tcpdump`

1.5 Given a scenario, manage a Linux system using common shell operations.

Practiced in Lab 1.5.

Where in the lab	Exam topics practiced
Task 1: Prepare the lab and build a practice dataset	`HOME`, `SHELL`, `USER`, `standard output`, `standard input`, `here doc`, `awk`, `bc`, `cat`, `cut`, `grep`, `less`, `more`, `sed`, `sort`, `uname`, `wc`, `vim`, `nano`
Task 2: Navigate with absolute and relative paths	`PATH`, `absolute`, `relative`, `tr`
Task 3: Explore environment variables	`DISPLAY`, `PS1`, `echo`, `head`
Task 4: Customize your shell with PS1, PATH, aliases, and startup files	`.bashrc`, `.bash_profile`, `.profile`, `alias`, `source`, `tail`
Task 5: Redirect standard input, output, and error	`standard error`, `tee`
Task 6: View and filter text with cat, less, more, head, tail, grep, cut, sort, uniq, and wc	`uniq`
Task 7: Transform text with awk, sed, tr, printf, bc, and xargs	`printf`, `xargs`
Task 8: Reuse commands with history, !, and !!	`history`

1.6 Given a scenario, perform backup and restore operations for a Linux server.

Practiced in Lab 1.6.

Where in the lab	Exam topics practiced
Task 1: Prepare the tools and a sample data directory	`cpio`, `tar`, `7-Zip`, `bzip2`, `gzip`, `unzip`, `xz`, `dd`, `ddrescue`, `rsync`
Task 7: Read compressed logs with zcat, zgrep, and zless	`zcat`, `zgrep`, `zless`

1.7 Summarize virtualization on Linux systems.

Practiced in Lab 1.7.

Where in the lab	Exam topics practiced
Task 1: Check for hardware virtualization and nested support	`QEMU`, `KVM`, `nested virtualization`, `RAM`, `CPU`, `libvirt`
Task 2: Install QEMU, KVM, and libvirt	`VirtIO`, `network`, `NAT`, `virsh`
Task 3: Download a cloud image and inspect, convert, and resize it	`convert`, `resize`, `image properties`, `snapshot`
Task 4: Build a baseline template and create a VM with VirtIO devices	`paravirtualized`, `baseline image`, `template`
Task 6: Change CPU, RAM, storage, and network resources	`storage`
Task 8: Explore virtual network types and build an isolated network	`bridged`, `host-only`, `isolated`, `routed`, `open`
Task 9: Practise migration and meet virt-manager	`cloning`, `migration`, `bare metal`, `virt-manager`
Exam corner	`VM states`

2.1 Given a scenario, manage files and directories on a Linux system.

Practiced in Lab 2.1.

Where in the lab	Exam topics practiced
Task 1: Navigate and build a project tree	`cd`, `file`, `ls`, `mkdir`, `pwd`, `rm`, `touch`
Task 2: Copy, move, rename, and remove	`cp`, `mv`, `rmdir`
Task 3: Identify files with `file` and inspect them with `stat`	`diff`, `locate`, `stat`, `symbolic link`, `hard link`
Task 5: Find files with `find` and `locate`	`find`
Task 6: Compare files with `diff` and `sdiff`	`sdiff`
Task 7: Hard links and symbolic links	`ln`
Task 8: Find open files with `lsof`	`lsof`, `character device`
Task 9: Tour the device files in `/dev`	`block device`, `special character device`

2.2 Given a scenario, perform local account management in a Linux environment.

Practiced in Lab 2.2.

Where in the lab	Exam topics practiced
Task 1: Explore the account files and UID ranges	`passwd`, `getent passwd`, `groups`, `id`, `last`, `who`, `whoami`, `/etc/group`, `/etc/passwd`, `/etc/shadow`, `UID`, `GID`, `system account`, `UID range`
Task 2: Prepare the templates: `/etc/skel`, `/etc/profile`, and defaults	`adduser`, `useradd`, `/etc/profile`, `/etc/skel`
Task 3: Onboard the team with `groupadd`, `useradd`, and `adduser`	`groupadd`, `userdel`, `lock`, `expiration`
Task 5: Modify accounts with `usermod`, `chsh`, `groupmod`, and `passwd`	`chsh`, `groupmod`, `usermod`
Task 6: Lock, unlock, expire, and age passwords	`chage`
Task 7: See who is logged in and which IDs a process uses	`lastlog`, `EUID`, `EGID`
Task 8: Create a service account, then offboard users	`deluser`, `groupdel`, `service account`

2.3 Given a scenario, manage processes and jobs in a Linux environment.

Practiced in Lab 2.3.

Where in the lab	Exam topics practiced
Task 1: Start a workload and view it with `ps` and `pstree`	`atop`, `htop`, `lsof`, `mpstat`, `pidstat`, `ps`, `pstree`, `strace`, `top`, `PPID`, `PID`, `nice`, `kill`, `killall`, `pkill`, `KILL`, `TERM`, `anacron`, `at`
Task 2: Watch the system with `top`, `htop`, `atop`, `mpstat`, and `pidstat`	`running`, `blocked`, `sleeping`, `stopped`, `zombie`, `renice`
Task 3: Look inside a process with `/proc`, `lsof`, and `strace`	`/proc/`, `Ctrl+C`, `exec`
Task 4: Create every process state on purpose	`Ctrl+Z`, `jobs`
Task 6: Process limits with `ulimit` and `/etc/security/limits.conf`	`process limits`, `ulimit`
Task 7: Drive job control: `&`, `jobs`, `fg`, `bg`, `Ctrl+C`, `Ctrl+Z`, `Ctrl+D`, `nohup`, and `exec`	`bg`, `Ctrl+D`, `fg`, `nohup`, `HUP`
Task 9: Schedule jobs with `crontab`, `at`, and `anacron`	`crontab`

2.4 Given a scenario, configure and manage software in a Linux environment.

Practiced in Lab 2.4.

Where in the lab	Exam topics practiced
Task 1: Use apt to search, install, inspect, and remove packages	`repository`, `dependenc`, `enabl`, `HTTP`
Task 2: Package files, dependency failures, conflicts, and holds	`conflict`, `exclu`, `sandbox`, `NTP`
Task 3: Manage Ubuntu repositories and add a signed third-party repo	`source`, `disabl`, `third-party`, `GPG`, `nginx`
Task 4: Use dnf and rpm on Rocky Linux	`DNS`
Task 5: Build and install a program from source	`package manager`
Task 6: Language-specific package managers: pip, cargo, and npm	`pip`, `cargo`, `npm`
Task 7: Update alternatives, software configuration, and sandboxed apps	`alternatives`
Task 8: Configure NTP, PTP, DNS, and DHCP services	`PTP`, `DHCP`
Task 9: Configure Nginx, Apache, Postfix (SMTP), and Dovecot (IMAP4)	`httpd`, `SMTP`, `IMAP`

2.5 Given a scenario, manage Linux using systemd.

Practiced in Lab 2.5.

Where in the lab	Exam topics practiced
Task 1: Explore units with systemctl	`service`, `systemctl`, `systemd-resolved`, `disable`, `enable`, `mask`, `start`, `status`
Task 2: Write your own service and use start, stop, enable, disable, and status	`target`, `systemd-analyze`, `daemon-reload`, `reload`, `restart`, `stop`
Task 3: Restart on failure, read the logs, and daemon-reload after edits	`edit`
Task 5: Reload versus restart, and mask versus unmask (with Nginx)	`unmask`
Task 6: Schedule a job with a timer unit	`timer`
Task 7: Mount a filesystem with a mount unit	`mount`
Task 8: Work with targets and analyze the boot	`blame`
Task 9: hostnamectl, timedatectl, resolvectl, and sysctl	`hostnamectl`, `resolvectl`, `sysctl`, `timedatectl`

2.6 Given a scenario, manage applications in a container on a Linux server.

Practiced in Lab 2.6.

Where in the lab	Exam topics practiced
Task 1: Install Docker and Podman, and look at the runtime stack	`runC`, `Podman`, `containerd`, `Docker`, `pull`, `build`, `USER`, `FROM`, `layer`, `start`, `rm`, `run`, `overlay`, `port`, `host`, `privileged`, `unprivileged`
Task 2: Pull images, tag them, and look at layers	`Dockerfile`, `ENTRYPOINT`, `CMD`, `tag`, `stop`, `inspect`
Task 3: Run, inspect, and manage containers	`logs`, `exec`, `environment variable`, `bridge`
Task 4: Build your own image with a Dockerfile	`prune`, `none`
Task 5: Keep data with volumes and bind mounts, and see the overlay filesystem	`volume`
Task 6: Container networks	`network create`, `macvlan`, `ipvlan`
Task 8: SELinux contexts and bind mounts with Podman on Rocky	`SELinux`, `:Z`

3.1 Summarize authorization, authentication, and accounting methods.

Practiced in Lab 3.1.

Where in the lab	Exam topics practiced
Task 1: Read PAM stacks and understand control flags	`PAM`
Task 2: Make a harmless PAM change, test it, and revert it	`journalctl`
Task 3: Authorization with polkit	`Polkit`
Task 4: Directory services: LDAP, SSSD, Kerberos, and realm	`SSSD`, `Winbind`, `LDAP`, `Kerberos`, `Samba`
Task 6: Query the journal and tour /var/log	`rsyslog`, `/var/log`, `auditd`
Task 7: Route a custom facility with rsyslog and rotate it with logrotate	`logrotate`
Task 8: Audit file changes with auditd	`audit.rules`

3.2 Given a scenario, configure and implement firewalls on a Linux system.

Practiced in Lab 3.2.

Where in the lab	Exam topics practiced
Task 1: Look at Netfilter on both VMs	`firewalld`, `firewall-cmd`, `port`, `ufw`, `nftables`, `iptables`, `ipset`, `netfilter`, `stateful`, `net.ipv4.ip_forward`
Task 2: firewalld zones, services, and ports	`runtime`, `rich rule`, `zone`, `service`
Task 3: Runtime vs permanent in firewalld	`permanent`, `PAT`
Task 4: firewalld rich rules	`NAT`
Task 6: nftables by hand, and stateful vs stateless	`stateless`
Task 8: NAT and IP forwarding: forward ubuntu-lab:8080 to rocky-lab:80	`DNAT`, `SNAT`, `IP forwarding`

3.3 Given a scenario, apply operating system (OS) hardening techniques on a Linux system.

Practiced in Lab 3.3.

Where in the lab	Exam topics practiced
Task 1: Controlled sudo with a drop-in file	`sudo`, `/etc/sudoers`, `NOEXEC`, `NOPASSWD`, `/etc/sudoers.d`, `visudo`, `sudo -i`, `wheel`, `su -`
Task 2: File attributes with chattr and lsattr	`chattr`, `lsattr`, `immutable`, `append only`
Task 3: Permissions workshop: ownership, special bits, and umask	`sudo group`, `chgrp`, `chmod`, `octal`, `symbolic`, `sticky bit`, `setuid`, `setgid`, `umask`
Task 4: Fine-grained permissions with ACLs	`chown`, `ACL`, `setfacl`, `getfacl`
Task 5: SELinux on rocky-lab	`SELinux`, `restorecon`, `semanage`, `chcon`, `ls -Z`, `getenforce`, `setenforce`, `getsebool`, `setsebool`, `audit2allow`, `sealert`, `enforcing`, `permissive`, `disabled`
Task 6: Secure SSH with keys and a hardened config	`sshd`, `key`, `tunnel`, `PermitRootLogin`, `X11Forwarding`, `AllowUsers`, `AllowGroups`, `ssh-agent`
Task 7: An SFTP-only chroot group	`SFTP`, `chroot`, `FTP`
Task 8: Block brute-force logins with fail2ban	`fail2ban`
Task 9: Remove insecure services and reduce attack surface	`Telnet`, `TFTP`, `unused file system`, `Secure Boot`, `UEFI`
Exam corner	`SUID`

3.4 Explain account hardening techniques and best practices.

Practiced in Lab 3.4.

Where in the lab	Exam topics practiced
Task 1: Prepare a safe workspace	`root`
Task 2: Enforce password length and complexity with pam_pwquality	`complexity`, `length`
Task 3: Block password reuse with pam_pwhistory	`reuse`, `history`
Task 4: Set password expiration with chage and login.defs	`expiration`
Task 5: Lock accounts after failed logins with pam_faillock (rocky-lab)	`pam_tally2`
Task 6: Add MFA (TOTP) to SSH	`MFA`
Task 7: Check a password against a breach list	`breach`
Task 8: Use restricted shells (nologin and rbash)	`restricted shell`, `/sbin/nologin`, `/bin/rbash`

3.5 Explain cryptographic concepts and technologies in a Linux environment.

Practiced in Lab 3.5.

Where in the lab	Exam topics practiced
Task 1: Prove integrity with SHA-256 and HMAC	`OpenSSL`, `TLS`, `SHA-256`, `HMAC`
Task 2: Encrypt and sign files with GPG	`GPG`
Task 3: Encrypt a disk with LUKS2	`LUKS2`, `Argon2`
Task 4: Build a small CA and see why self-signed certificates fail	`certificate`, `trusted root`, `no-cost`, `commercial`, `self-signed`
Task 5: Serve HTTPS with nginx and control TLS versions	`LibreSSL`
Task 6: Build a WireGuard tunnel between the VMs	`WireGuard`
Task 7: Remove weak algorithms	`data at rest`, `data in transit`, `weak algorithm`

3.6 Explain the importance of compliance and audit procedures.

Practiced in Lab 3.6.

Where in the lab	Exam topics practiced
Task 1: Scan for malware with ClamAV	`anti-malware`
Task 2: Plant and hunt indicators of compromise	`indicators of compromise`
Task 3: Look up CVEs, CVSS scores, and backported fixes	`CVE`, `CVSS`, `backport`
Task 4: Find misconfigurations with a port scanner and a protocol analyzer	`vulnerability scan`, `misconfiguration`, `port scanner`, `protocol analyzer`
Task 5: Audit rocky-lab against the CIS Benchmark with OpenSCAP	`OpenSCAP`, `CIS`
Task 6: Detect changed files with AIDE and rkhunter	`AIDE`, `rkhunter`
Task 7: Verify installed files, signed packages, and the supply chain	`signed package`, `rpm -V`, `supply chain`
Task 8: Destroy data securely	`shred`, `badblocks -w`, `dd if=/dev/urandom`, `cryptographic`
Task 9: Set security banners	`/etc/issue`, `/etc/issue.net`, `/etc/motd`

4.1 Summarize the use cases and techniques of automation and orchestration in a Linux environment.

Practiced in Lab 4.1.

Where in the lab	Exam topics practiced
What you'll practice	`infrastructure as code`, `version control`
Tasks	`task`
Task 1: Set up Ansible and run ad hoc commands	`Ansible`, `playbook`, `inventory`, `module`, `ad hoc`, `collection`, `fact`, `agentless`, `agent`, `state`, `Secret`, `variable`, `up`, `logs`
Task 2: Write a playbook with variables, handlers, and a collection	`service`, `down`
Task 3: Puppet facts, classes, modules, and certificates	`Puppet`, `class`, `certificate`, `resource`, `node`, `network`
Task 4: Unattended deployment with Kickstart and cloud-init	`Kickstart`, `cloud-init`, `Deployment`
Task 5: Build a CI/CD pipeline with shift left testing	`API`, `CI/CD`, `shift left`, `GitOps`, `pipeline`, `DevSecOps`, `Kubernetes`
Task 6: Run a multi-container app with Docker Compose	`OpenTofu`, `provider`, `Pod`, `volume`, `Compose`, `compose file`
Task 8: Orchestrate a service with Docker Swarm	`Docker Swarm`, `scale`
Task 9: Orchestrate with Kubernetes (k3s)	`ConfigMap`

4.2 Given a scenario, perform automated tasks using shell scripting.

Practiced in Lab 4.2.

Where in the lab	Exam topics practiced
Task 1: Your first script, variables, and return codes	`parameter expansion`, `command substitution`, `function`, `if`, `until`, `for`, `while`, `-d`, `-n`, `environment`, `argument`, `alias`, `export`, `set`, `unalias`, `unset`, `return code`
Task 2: Expansions, subshells, IFS, and OFS	`subshell`, `IFS`, `OFS`, `case`, `-ge`, `test`, `-f`
Task 3: Tests and comparisons	`-eq`, `-gt`, `-le`, `-lt`, `-ne`, `regular expression`, `-z`
Task 6: Functions, local variables, and return codes	`local`

4.3 Summarize Python basics used for Linux system administration.

Practiced in Lab 4.3.

Where in the lab	Exam topics practiced
Task 1: Meet Python 3 and its data types	`Python 3`, `boolean`, `dictionary`, `float`, `integer`, `list`, `string`, `module`
Task 2: Indentation and the shape of a Python script	`indentation`
Task 3: Build a disk report with built-in modules	`built-in module`
Task 4: Install dependencies in a virtual environment	`virtual environment`, `venv`, `pip install`, `dependenc`, `package`
Task 5: Use third-party packages in a script	`PEP 8`

4.4 Given a scenario, implement version control using Git.

Practiced in Lab 4.4.

Where in the lab	Exam topics practiced
Task 1: Install and configure Git	`git config`, `git pull`
Task 2: Create a repository, ignore files, and commit	`.gitignore`, `git add`, `git commit`, `git diff`, `git init`, `git log`
Task 3: Branches and merges	`git branch`, `git checkout`, `git merge`
Task 4: Squash merges and rebasing	`squash`, `git rebase`
Task 5: Stash, reset, and recover with reflog	`git reset`, `git stash`
Task 6: Tag releases	`git tag`
Task 7: Share work through a remote on rocky-lab	`git clone`, `git fetch`, `git push`

4.5 Summarize best practices and responsible uses of artificial intelligence (AI).

Practiced in Lab 4.5.

Where in the lab	Exam topics practiced
Task 1: Prepare your review toolkit	`code`, `lint`, `review`
Task 2: Review AI-generated Bash code	`generat`, `optimiz`, `verify`, `human review`
Task 3: Verify an AI-generated regular expression	`regular expression`
Task 4: Security review of AI-generated infrastructure as code	`infrastructure as code`, `security review`
Task 5: Verify AI-generated Python, compliance advice, and documentation	`document`, `compliance`
Task 6: Prompt engineering	`prompt engineering`
Task 7: Data governance, local models, and corporate policy	`data governance`, `LLM`, `training`, `local model`, `private`, `public`, `corporate policy`

5.1 Summarize monitoring concepts and configurations in a Linux system.

Practiced in Lab 5.1.

Where in the lab	Exam topics practiced
Task 1: Define an SLO and build a health check	`SLA`, `SLI`, `SLO`, `health check`, `event`
Task 2: Thresholds, events, alerts, and notifications	`webhook`, `threshold`, `alert`, `notification`
Task 4: Agent versus agentless monitoring	`SNMP`, `MIB`, `agent`, `agentless`
Task 6: Send and receive SNMP traps	`trap`, `logging`
Task 7: Aggregate logs from rocky-lab on ubuntu-lab	`log aggregation`

5.2 Given a scenario, analyze and troubleshoot hardware, storage, and Linux OS issues.

Practiced in Lab 5.2.

Where in the lab	Exam topics practiced
What you'll practice	`not turning on`
Task 1: Prepare the lab and the incident log	`PATH`, `quota`
Task 2: Scenario 1: "No space left on device" (filesystem full and inode exhaustion)	`filesystem full`, `inode exhaustion`
Task 3: Scenario 2: "I can't save my file" (partition not writable)	`not writable`
Task 5: Scenario 4: Filesystem will not mount, and data corruption	`data corruption`, `will not mount`
Task 6: Scenario 5: The server boots to a BusyBox prompt (GRUB misconfiguration)	`GRUB`
Task 8: Scenario 7: Kernel panic and kernel corruption	`kernel panic`, `kernel corruption`, `driver`
Task 10: Scenario 9: apt refuses to install anything (package dependency issues)	`package dependency`
Task 11: Scenario 10: A program crashes with "Segmentation fault"	`segmentation fault`
Task 12: Scenario 11: Processes are "Killed" (OOM killer and memory leaks)	`killed process`, `memory leak`
Task 14: Scenario 13: A service will not start (systemd unit failure)	`unit failure`
Task 15: Scenario 14: A program stops responding (unresponsive process)	`unresponsive`
Task 16: Scenario 15: Disk errors (device failure)	`device failure`
Task 17: Scenario 16: The server is inaccessible or will not turn on	`inaccessible`

5.3 Given a scenario, analyze and troubleshoot networking issues on a Linux system.

Practiced in Lab 5.3.

Where in the lab	Exam topics practiced
Task 1: Take a baseline and build the practice network	`firewall`, `DHCP`, `DNS`, `interface`, `MTU`, `subnet`, `ping`, `routing`, `gateway`, `IPv6`
Task 2: Scenario 1: "The network is dead" (link down and link negotiation)	`link down`, `link negotiation`
Task 4: Scenario 3: "I can ping the gateway but not the server" (subnet mask)	`unreachable`
Task 8: Scenario 7: "The connection keeps dropping" (IP conflict)	`IP conflict`
Task 10: Scenario 9: "One cable failed and the server stayed up" (bonding)	`bonding`
Task 11: Scenario 10: "I changed the MAC address and lost the network" (MAC spoofing)	`MAC spoofing`
Task 13: Scenario 12: "SSH to rocky-lab takes several seconds to connect" (dual stack issues)	`dual stack`

5.4 Given a scenario, analyze and troubleshoot security issues on a Linux system.

Practiced in Lab 5.4.

Where in the lab	Exam topics practiced
Task 1: Prepare both VMs	`SELinux`, `policy`, `ACL`
Task 2: Scenario 1: "403 Forbidden" after moving a file (SELinux context)	`context`, `permission`
Task 3: Scenario 2: "503 Service Unavailable" from a reverse proxy (SELinux boolean)	`boolean`
Task 5: Scenario 4: "Permission denied" on shared files (permissions, ACLs, attributes)	`attribute`
Task 6: Scenario 5: "carol can't log in" (account access)	`account access`
Task 7: Scenario 6: "Permission denied (publickey)" (remote access issues)	`remote access`, `cipher`
Task 8: Scenario 7: "Are we patched?" (unpatched vulnerable systems)	`unpatched`
Task 9: Scenario 8: "Why is our cache server on the network?" (exposed or misconfigured service)	`exposed`
Task 10: Scenario 9: "Your connection is not private" (certificate issues)	`certificate`
Task 11: Scenario 10: "apt update fails" (misconfigured package repository)	`repository`, `insecure`
Task 12: Scenario 11: Old protocols and "no matching cipher" (obsolete and insecure protocols and ciphers)	`obsolete`

5.5 Given a scenario, analyze and troubleshoot performance issues.

Practiced in Lab 5.5.

Where in the lab	Exam topics practiced
Task 1: Record a baseline	`swapping`, `load average`, `context switch`, `I/O wait`, `latency`, `disk latency`, `throughput`, `blocked process`, `baseline`
Task 2: CPU bottleneck, high CPU usage, high load average, and high context switching	`high CPU`, `timeout`, `CPU bottleneck`
Task 3: Swapping and out of memory	`out of memory`
Task 5: System unresponsiveness and sluggish terminal behavior	`unresponsive`, `sluggish terminal`
Task 6: Slow startup	`slow startup`
Task 7: High failed log-in attempts	`failed log-in`
Task 8: High latency, jitter, slow response times, and slow application response	`slow application`, `jitter`, `slow response`
Task 9: Packet drops, random timeouts, random disconnects, and low throughput	`packet drop`, `random disconnect`
Task 10: Slow remote storage response (NFS)	`remote storage`
Task 11: Hardware errors and exceeding baselines	`hardware error`

Appendix D: Lab VM Troubleshooting

Things go wrong in labs. That is part of learning. This appendix covers the problems readers hit most often and how to get unstuck. When in doubt, **restore your snapshot** and start the lab again. That is always faster than digging out of a mess you do not yet understand.

I cannot connect with SSH from my host

1. Log in at the VM console and check that the SSH server is running:

```
systemctl status ssh        # Ubuntu (the service is named ssh)
```

```
# on rocky-lab
systemctl status sshd       # Rocky (the service is named sshd)
```

2. Check the VM's address with `ip -br address`, and make sure you connect to that address.
3. **VirtualBox NAT Network:** the host reaches the VMs only through the port-forwarding rules from Lab 0, Task 8 (`ssh -p 2210 student@127.0.0.1`).
4. Check the firewall. On Ubuntu, `sudo ufw status` must show `OpenSSH ALLOW` if ufw is active. On Rocky, `sudo firewall-cmd --list-services` must include `ssh`.
5. If you changed `/etc/ssh/sshd_config` or a file in `/etc/ssh/sshd_config.d/`, test the configuration with `sudo sshd -t`. It prints the line with the error. Fix it, then restart the service.

I lost network access after changing the network configuration

Work at the VM console, not over SSH.

- **Ubuntu (Netplan):** list the files with `ls /etc/netplan/`. Move the file you changed out of the way and re-apply:

```
sudo mv /etc/netplan/60-static.yaml /root/
sudo netplan apply
```

  Always use `sudo netplan try` for future changes. It rolls back automatically if you do not confirm.

- **Rocky (NetworkManager):** show the connection with `nmcli connection show enp0s3`, fix the wrong value with `sudo nmcli connection modify`, then run `sudo nmcli connection up enp0s3`. To go back to DHCP: `sudo nmcli connection modify enp0s3 ipv4.method auto ipv4.addresses "" ipv4.gateway ""`.

Name resolution fails (ping by name does not work)

- `ping rocky-lab` fails but `ping 192.168.100.20` works: check `/etc/hosts` on the VM (`cat /etc/hosts`). The entry must contain the current address.

- `ping ubuntu.com` fails: on Ubuntu run `resolvectl status` and check that a DNS server is listed. On Rocky run `cat /etc/resolv.conf`. Re-check the `nameservers` (Netplan) or `ipv4.dns` (nmcli) settings from Lab 0.

apt says “Could not get lock /var/lib/dpkg/lock-frontend”

Another package process is running, usually the automatic `unattended-upgrades` job that runs shortly after boot. Wait a few minutes and try again. Check what holds the lock with:

```
ps aux | grep -E 'apt|dpkg' | grep -v grep
```

Never delete the lock files while a package process is running. That can corrupt the package database.

dpkg was interrupted

If an installation was cut off, run:

```
sudo dpkg --configure -a
sudo apt -f install
```

The first command finishes configuring half-installed packages. The second fixes broken dependencies.

The extra disks have different names

Your disks may be named `vdb`, `vdc`, and so on (VirtIO), or `nvme0n2`, `nvme0n3`, and so on (NVMe), instead of `sdb` to `sde`. Run `lsblk` and substitute your names in every command. Partitions on NVMe disks get a `p`, for example `nvme0n2p1` instead of `sdb1`.

A disk is busy and cannot be unmounted or wiped

```
sudo lsof +f -- /mnt/data      # which processes use the mount point?
sudo fuser -vm /mnt/data       # another view of the same thing
```

`cd` out of the directory in every terminal, stop the listed processes, then try `umount` again. Leftover LVM or RAID devices keep disks busy too: run `sudo vgchange -an`, `sudo mdadm --stop /dev/md0`, and then `sudo wipefs -a /dev/sdX`.

The VM boots into emergency mode

This almost always means a bad line in `/etc/fstab`, for example a disk that was removed or a mistyped UUID.

1. Enter the root password (Rocky) or press Enter for a maintenance shell.
2. Remount the root filesystem read-write and edit fstab:

```
mount -o remount,rw /
nano /etc/fstab
```

3. Put a `#` in front of the bad line, save, and run `systemctl daemon-reload` and `mount -a` to check. Then reboot.

Lab 5.2 practices exactly this scenario.

> **TIP**
> Ubuntu locks the root account, so its emergency mode may say "root account is locked" and refuse a shell. Reboot, open the GRUB menu (hold **Shift** on BIOS or tap **Esc** on UEFI), choose **Advanced options > recovery mode**, then **root - Drop to root shell prompt**.

I forgot my password or broke sudo

Restoring the snapshot is easiest. To fix it in place on Ubuntu:

1. Reboot and open the GRUB menu (hold **Shift** or tap **Esc** during boot).
2. Highlight the Ubuntu entry and press **e**.
3. On the line that starts with `linux`, add `init=/bin/bash` at the end, then press **Ctrl+X** to boot.
4. You get a root shell with the root filesystem read-only. Run:

```
mount -o remount,rw /
passwd student          # set a new password
visudo                  # or fix /etc/sudoers.d/ files
sync
reboot -f
```

On Rocky, add `rd.break` instead, then run `mount -o remount,rw /sysroot`, `chroot /sysroot`, `passwd student`, `touch /.autorelabel` (so SELinux relabels the files you changed), `exit`, and `exit`.

I locked myself out with the firewall

Use the VM console, which the firewall does not affect.

- Ubuntu: `sudo ufw disable`, or `sudo nft flush ruleset` for raw nftables rules.
- Rocky: `sudo firewall-cmd --reload` drops runtime-only changes. `sudo firewall-cmd --permanent --add-service=ssh` followed by `sudo firewall-cmd --reload` restores SSH access.

SELinux blocks something and I do not know why

On rocky-lab:

```
sudo ausearch -m AVC -ts recent      # recent denials
sudo sealert -a /var/log/audit/audit.log | less
```

`sealert` (package `setroubleshoot-server`) explains each denial and suggests a fix. Switching to permissive mode with `sudo setenforce 0` is fine for a quick test, but always switch back with `sudo setenforce 1` and fix the real cause. Labs 3.3 and 5.4 teach this.

The VM is very slow or runs out of memory

- Give the VM more RAM in the hypervisor (power it off first).
- Check memory with `free -h` and the biggest processes with `top`, sorted by memory (press **M**).
- Labs that install k3s (4.1) or run nested VMs (1.7) need the full 4 GB. Stop other lab services first (`sudo systemctl stop ...`).
- Running out of disk space? `df -h` shows usage. `sudo apt clean`, `sudo journalctl --vacuum-size=100M`, and `docker system prune -a` (if you did Lab 2.6) free space quickly.

Nested virtualization does not work (Lab 1.7)

If `grep -cE 'vmx|svm' /proc/cpuinfo` prints `0`, the labs use QEMU software emulation instead (`--virt-type qemu`). It is slower but every command works. On Windows hosts, VirtualBox may not offer nested virtualization while Hyper-V or "Virtual Machine Platform" is enabled. In that case, use emulation.

Time is wrong in the VM

Wrong time breaks TLS certificates, package downloads, and Kerberos. Check with `timedatectl`. `System clock synchronized: yes` is what you want. If it says `no`, check the time service: `systemctl status systemd-timesyncd` or `chronyd` on Ubuntu, or `chronyc tracking` on Rocky.

Appendix E: Acronym List

The acronyms from the official exam objectives. Know what each one stands for.

ACL	Access Control List
ACME	Automated Certificate Management Environment
AI	Artificial Intelligence
AIDE	Advanced Intrusion Detection Environment
API	Application Programming Interface
ARM	Advanced RISC (Reduced Instruction Set Computer) Machine
BIOS	Basic Input/Output System
CA	Certificate Authority
CI/CD	Continuous Integration/Continuous Deployment
CIFS	Common Internet File System
CIS	Center for Internet Security
CMS	Content Management System
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUPS	Common UNIX Printing System
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DHCP	Dynamic Host Configuration Protocol
DNAT	Destination Network Address Translation
DNS	Domain Name System
EGID	Effective Group Identifier
EPEL	Extra Packages for Enterprise Linux
EUID	Effective User Identifier
FEC	Forward Error Correction
FHS	Filesystem Hierarchy Standard
FQDN	Fully Qualified Domain Name
FTP	File Transfer Protocol
FUSE	Filesystem in Userspace
GDPR	General Data Protection Regulation
GID	Group Identifier
GNU	GNU's Not Unix
GPG2	GNU Privacy Guard 2
GPG	GNU Privacy Guard
GPT	GUID (Globally Unique Identifier) Partition Table
GPU	Graphics Processing Unit
GRUB	Grand Unified Bootloader
GUI	Graphical User Interface
GUID	Globally Unique Identifier
HMAC	Hashed Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPD	HyperText Transfer Protocol Daemon
IaC	Infrastructure as Code
ICMP	Internet Control Message Protocol
IFS/OFS	Internal Field Separator/Output Field Separator
IMAP4	Internet Message Access Protocol 4
initrd	Initial RAM Disk
I/O	Input/Output
IoC	Indicators of Compromise
IOPS	Input/Output Operations Per Second
IP	Internet Protocol
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KRB5	Kerberos 5
KVM	Kernel-based Virtual Machine
LAN	Local Area Network
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Model
LUKS	Linux Unified Key Setup
LUKS2	Linux Unified Key Setup 2
LVM	Logical Volume Manager
MAC	Media Access Control
MBR	Master Boot Record
MFA	Multifactor Authentication
MIB	Management Information Base
MTU	Maximum Transmission Unit
NAS	Network-Attached Storage
NAT	Network Address Translation

NFS	Network File System
NIC	Network Interface Card
NTP	Network Time Protocol
NVMe	Non-Volatile Memory Express
OOM	Out of Memory
OpenSCAP	Open Security Content Automation Protocol
OpenSSL	Open Secure Sockets Layer
OS	Operating System
PAM	Pluggable Authentication Modules
PAT	Port Address Translation
PEP	Python Enhancement Proposal
PHP	PHP: Hypertext Preprocessor
PID	Process Identification Number
PKI	Public Key Infrastructure
PPID	Parent Process Identification Number
PTP	Precision Time Protocol
PV	Physical Volume
PXE	Preboot Execution Environment
qcow	QEMU Copy On Write
QEMU	Quick Emulator
RAID	Redundant Array of Independent Disks
RAM	Random Access Memory
RISC	Reduced Instruction Set Computer
RPM	Red Hat Package Manager (RPM Package Manager)
SAN	Storage Area Network
SELinux	Security-Enhanced Linux
setGID	Set Group Identity
setUID	Set User Identity
SFTP	Secure File Transfer Protocol
SLA	Service-Level Agreement
SLES	SUSE Linux Enterprise Server
SLI	Service-Level Indicator
SLO	Service-Level Objective
SMB	Server Message Block
SMTP	Simple Mail Transfer Protocol
SNAT	Source Network Address Translation
SNMP	Simple Network Management Protocol
SR-IOV	Single Root Input/Output Virtualization
SSD	Solid-State Drive
SSH	Secure Shell
SSHD	Secure Shell Daemon
SSL	Secure Sockets Layer
SSO	Single Sign-On
SSSD	System Security Services Daemon
TCP	Transmission Control Protocol
TFTP	Trivial File Transfer Protocol
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
TTL	Time to Live
UDP	User Datagram Protocol
UEFI	Unified Extensible Firmware Interface
UFW	Uncomplicated Firewall
UID	User Identifier
USB	Universal Serial Bus
UUID	Universally Unique Identifier
Vim	Vi Improved
VM	Virtual Machine
WAN	Wide Area Network
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language